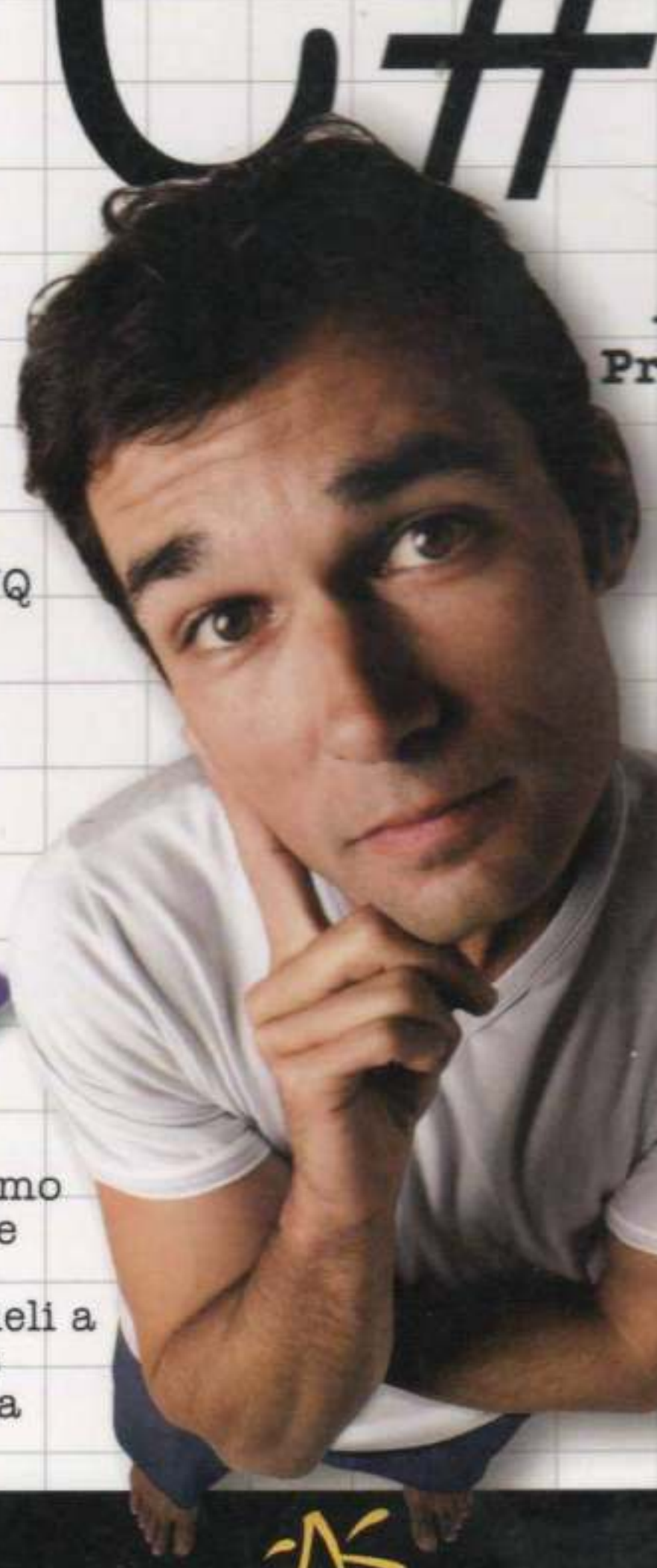
Um Guia Para Aprender Fácil

Inclui C# 3.0 e Visual Studio 2008

Use a Cabeça!

C#



Um Guia de Aprendizagem Para Programação no Mundo Real em C# e .NET

Mande nos seus dados usando LINQ

Descubra os Segredos da Abstração e da Herança

Crie um jogo arcade retrô clássico totalmente funcional

Aprenda como extensões de métodos ajudaram Sueli a contornar as regras na Vila Objeto

Veja como Jaime usou coleções genéricas para vencer a briga com seus dados

O'REILLY®

ALTA BOOKS®

Andrew Stellman
& Jennifer Greene

Conteúdo (Sumário)

Introdução	xxix
1 Seja produtivo com C#: Aplicativos visuais em 10 minutos ou menos	1
2 Tudo é somente código: Nos bastidores	39
3 Objetos orientem-se: Fazendo o código ter sentido	77
4 Tipos e referências: São 10 horas. Você sabe onde seus dados estão?	113
Laboratório C# 1: Um dia de corridas	147
5 Encapsulamento: Mantenha o que é particular... particular	155
6 Herança: Sua árvore genealógica de objeto	183
7 Interfaces e classes abstratas: Fazendo as classes manterem suas promessas	223
8 Enumerações e coleções: Guardando um monte de dados	269
Laboratório C# 2: A Missão	313
9 Lendo e salvando arquivos: Salve o conjunto de bytes, salve o mundo	331
10 Tratamento de exceções: Apagar incêndios é coisa do passado	377
11 Eventos e delegados: O que o seu código faz quando você não está olhando	411
12 Revisão e prévia: Conhecimento, poder e desenvolver coisas legais	439
13 Controles e gráficos: Deixe bonito	477
14 Capitão Maravilhoso: A Morte do objeto	527
15 LINQ: Controle seus dados	559
Laboratório C# 3: Invasores	583
Apendice i: Sobras	599

Conteúdo (a coisa real)

Introdução

Sua mente em C#. Você está sentado tentando aprender alguma coisa, mas sua mente fica insistindo em dizer-lhe que este aprendizado não é importante. Sua mente diz "melhor deixar espaço para coisas mais importantes, como, por exemplo, quais animais selvagens evitar e se um local para a prática de arco e flecha para nudistas é uma idéia ruim". Então como você engana sua mente para que ela pense que a sua vida realmente depende de aprender C#?

Pra quem é este livro?	xxx
Sabemos o que você está pensando	xxx
Metacognição: pensando sobre o pensar	xxxii
Aqui está o que VOCÊ pode fazer para forçar seu cérebro a ser submisso	xxxiii
Do que você precisa para ler este livro	xxxiv
Leia-me	xxxv
A equipe de revisão técnica	xxxvi
Agradecimentos	xxxvii

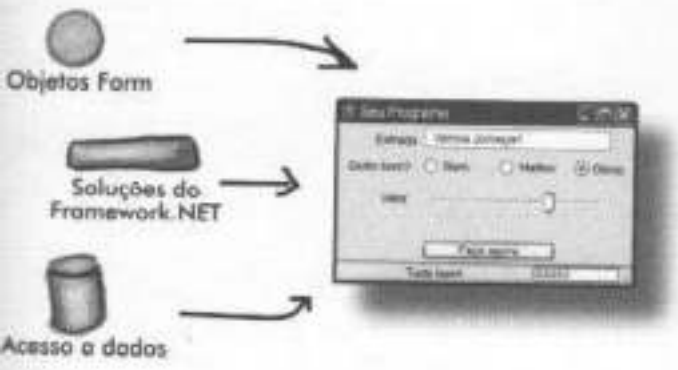
Seja produtivo com C#

1 Aplicativos Visuais em 10 minutos ou menos

Quer criar ótimos programas realmente rápido?

O C# é uma linguagem de programação poderosa e uma ferramenta valiosa na palma de sua mão. Com a IDE do Visual Studio, você nunca mais terá que gastar horas escrevendo código obscuro para fazer com que um botão funcione. Melhor ainda, você poderá se concentrar em realizar o seu trabalho, ao invés de lembrar qual parâmetro de qual método é o nome para um botão, e qual é o rótulo dele. Soa interessante? Vire a página, e vamos começar a programar.

Por que você deve aprender C#	2
O C# e o IDE do Visual Studio facilitam muitas coisas	3
Ajude o diretor geral a eliminar os papéis	4
Conheça as necessidades dos usuários antes de começar a fazer seu programa	4
Aqui está o que você vai desenvolver	6
O que você faz no Visual Studio...	8
O que o Visual Studio faz por você...	8
Desenvolva a interface de usuário	12
Visual Studio, nos bastidores	14
Complete o código gerado automaticamente	15
Você já pode executar seu aplicativo	15
Precisamos de uma base de dados para armazenar nossas informações	17
Criando a tabela para a Lista de Contatos	19
Os espaços em branco no cartão de contato são colunas na tabela People	20
Termine de construir a tabela	23
Faça um diagrama com seus dados para que seu aplicativo possa acessá-los	23
Insira seus dados dos cartões no banco de dados	26
Conecte seu formulário à sua base de dados usando uma fonte de dados	28
Adicione controles vinculados a dados em seu formulário	30
Bons programas são usados intuitivamente	32
Como transformar o SEU aplicativo num aplicativo para TODOS	33
Dê o aplicativo aos seus usuários	34
Você NÃO terminou: teste sua instalação	35
Você fez um aplicativo dirigido a dados completo	36



Tudo É Somente Código

2 Nos bastidores

Você é um programador, não apenas usuário de IDE

Você pode fazer muito usando a IDE. Mas existe um limite para o que a IDE pode fazer por você. Claro, existem muitas tarefas repetitivas que tem que ser feitas quando se desenvolve um aplicativo. E a IDE é ótima para fazer essas coisas. Mas trabalhar com a IDE é apenas o começo. Pode-se fazer um aplicativo realizar muito mais - e escrever código C# é a forma de conseguir isso. Uma vez que se tenha pego o jeito da coisa com relação a programar, não há nada que seus programas não possam fazer.

Enquanto você está fazendo isso... 40
...o IDE está fazendo isso 41
De onde vêm os programas 42
O IDE ajuda você a fazer seu código 44
Quando coisas são alteradas no IDE, o código é mudado também 46
Anatomia de um programa 48
Seu programa sabe por onde começar 50
Você pode mudar o ponto de entrada de seu programa 51
Duas classes podem estar no mesmo namespace 57
Seus programas usam variáveis para trabalhar com dados 57
O C# usa simbolos matemáticos familiares 59
Laços executam uma ação repetidamente 60
Use um trecho de código para escrever laços for simples 61
Comandos if/else tomam decisões 62
Estabeleça condições e veja se elas são verdadeiras 63

* *

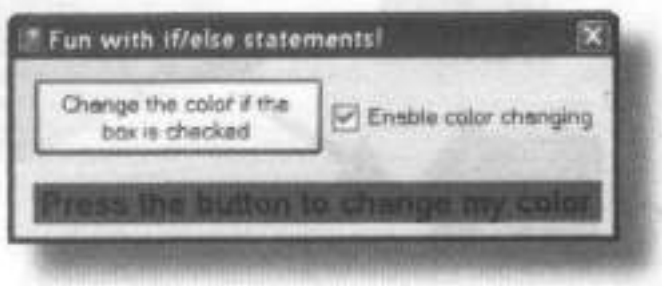
Crie este formulário ➘

*

Toda vez que você cria um novo programa, define um namespace para ele de forma q seu código fique separado das classes do Framework .NET.

Uma classe contém uma parte de seu programa (embora alguns programas bem pequenos possam ter apenas uma única classe).

Uma classe tem um ou mais métodos. Seus métodos sempre tem que existir dentro de classes. E eles são feitos de comandos - como aqueles que já vimos.



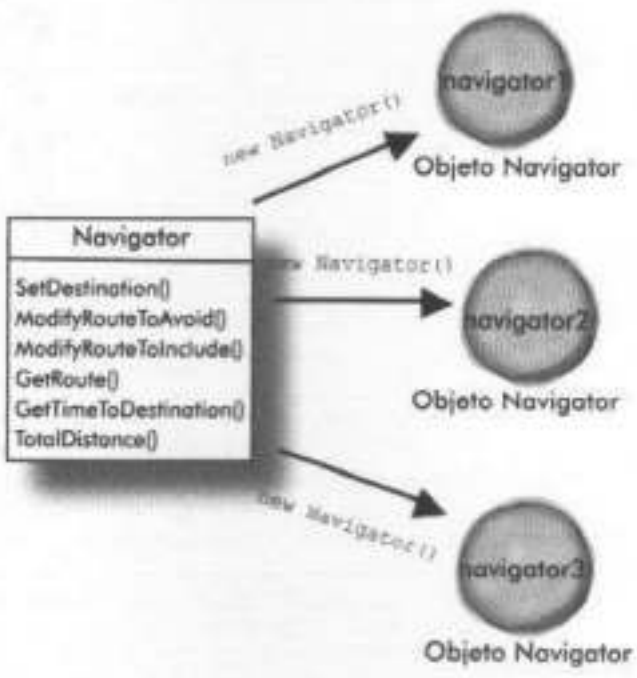
3 Objetos: se orientem!

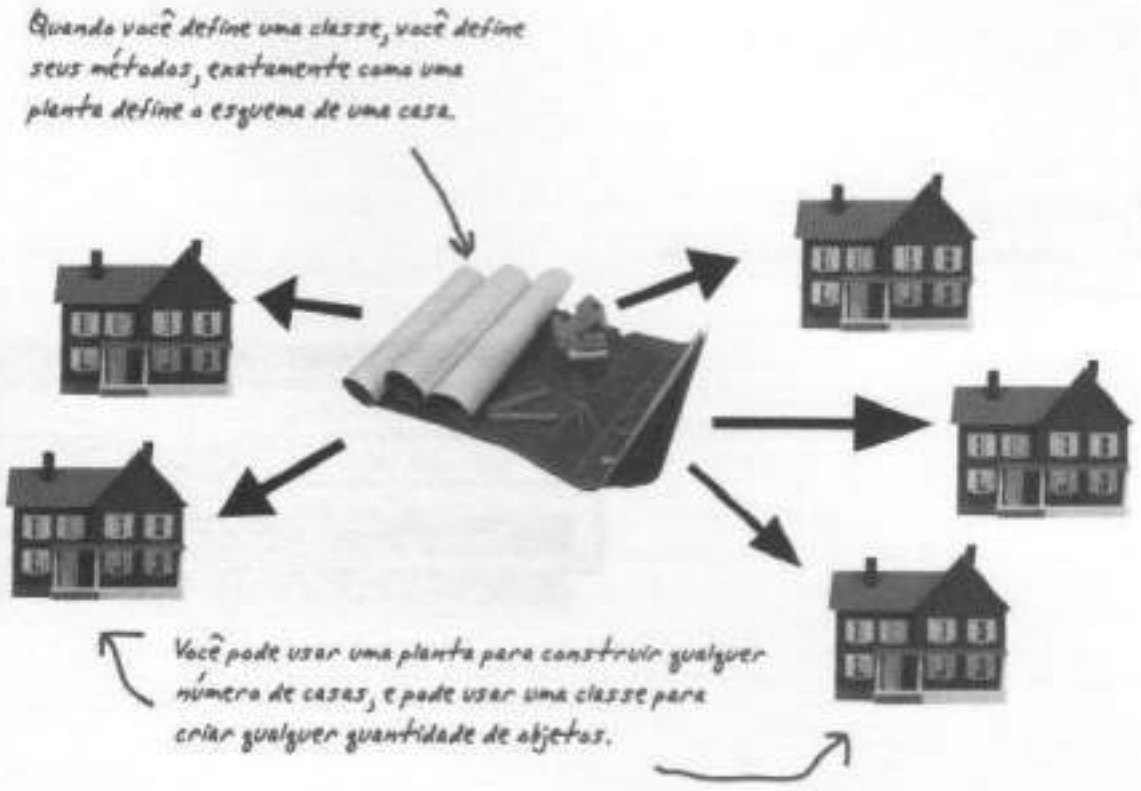
Fazendo o Código Ter Sentido

Cada programa que você escreve resolve um problema.

Quando você está desenvolvendo um programa, é sempre uma boa idéia começar pensando a respeito de qual problema ele deveria resolver. É por isso que objetos são tão úteis. Eles permitem que se estruture o código baseando-se no problema que se está resolvendo, de forma que você pode empregar seu tempo pensando sobre o problema que precisa ser enfrentando ao invés de ficar atolado na mecânica de escrever código. Quando se usa um objeto corretamente, acaba-se com código que é intuitivo de escrever, e fácil de ler e alterar.

Como Miguel pensa a respeito de seus problemas 78
Como o sistema de navegação do carro de Miguel pensa sobre este problema 79
A classe de navegação de Miguel tem métodos para criar e modificar rotas 80
Use o que você aprendeu para desenvolver um aplicativo simples 80
Miguel tem uma idéia 81
Miguel pode usar objetos para resolver o problema 82
Você pode usar uma classe para construir um objeto 83
Quando você cria um novo objeto a partir de uma classe, ele é chamado de uma instância dessa classe 83
Uma solução melhor… o que os objetos conseguiram para você! 84
Uma instância usa campos para manter registro sobre coisas 87
Vamos criar algumas instâncias! 88
Graças à memória 89
O que passa pela mente de seu programa 90
Você pode usar nomes de classes e métodos para deixar seu código mais intuitivo 91
Dê às suas classes uma estrutura natural 93
Diagramas de classe ajudam a organizar o código de forma que ele faça sentido 96
Desenvolva uma classe para trabalhar com alguns caras 99
Existe uma forma ainda mais fácil de incializar objetos 105
Algumas idéias para projetar classes intuitivas 106



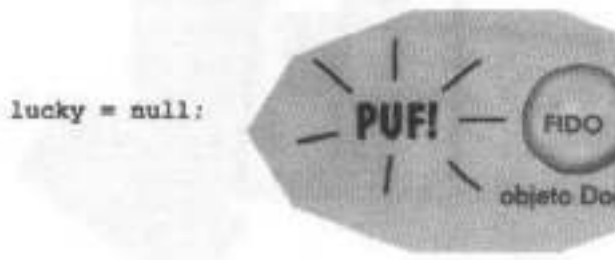


Tipos e Referências

4 São 10 horas. Você sabe onde seus dados estão?

Tipos de dados, bases de dados, Tenente-Comandante Data... é tudo importante. Sem dados, seus programas são inúteis. Informações oriundas dos usuários são necessárias, e são usadas para procurar ou criar informação nova para devolver a eles. Na verdade, quase tudo que você faz em programação envolve trabalho com dados de uma forma ou de outra. Neste capítulo, você aprenderá as vantagens e desvantagens dos tipos de dados C#, como trabalhar com dados em seu programa e até descobrir alguns segredos sujos sobre objetos (shhhhh...objetos são dados também).

O tipo da variável determina qual tipo de dado ela pode armazenar 114
Uma variável é um copo cheio de dados para a viagem 116
Cinco quilos de dados num saco de 2,5 quilos 116
Mesmo quando um número tem o tamanho certo, você não pode apenas atribui-lo a qualquer variável 117
Quando você converte um valor grande demais, o C# irá ajustá-lo automaticamente 118
O C# faz alguma coerção automaticamente 119
Quando você chama um método, as variáveis devem corresponder aos tipos dos parâmetros 121
Combinando = com um operador 125
Objetos são variáveis também 125
Refira-se aos seus objetos com variáveis de referência 126
Referências são como etiquetas para seu objeto 127
Se não há mais nenhuma referência, seu objeto é coletado como lixo 127
Múltiplas referências e seus efeitos colaterais 129
Duas referências significam DUAS formas de mudar um dado do objeto 133
Um caso especial: matrizes 133
Matrizes podem conter um monte de variáveis de referência também 134
Bem-vindo à lanchonete de sanduíches com desconto do João Podrão! 135
Objetos usam referências para falar uns com os outros 137
Onde nenhum objeto jamais esteve 138



Laboratório C# 1

Um dia de corridas

João, Beto e Alfredo gostam de apostar em corridas, mas estão cansados de perder dinheiro. Eles querem que você construa um simulador para eles para que eles possam determinar os vencedores antes de colocar dinheiro na coisa. E, se você fizer um bom trabalho, eles dividirão com você os lucros.

Especificações: Desenvolva um simulador de pista de corridas 148
O Produto Final 154

encapsulamento

5 Mantenha o que é particular... em particular

Você já quis ter um pouco mais de privacidade?

Às vezes seus objetos sentem-se da mesma forma. Assim como você não quer que ninguém que você não confie leia seu diário ou veja seus extratos bancários, um bom objeto não deixa outro objeto xeretar seus campos. Neste capítulo, você vai aprender sobre o poder do encapsulamento. Você vai tornar seus dados de objeto privados, e adicionar métodos para proteger a forma como esses dados são acessados.

Catarina é uma promotora de eventos 156
O que um estimador faz? 156
O teste da Catarina 162
Cada opção deve ser calculada individualmente 163
É fácil usar acidentalmente seus objetos de forma errada 164
Encapsulamento significa manter em particular alguns dados de uma classe 165
Use encapsulamento para controlar acesso aos métodos e campos de sua classe 165
Mas o campo NomeVerdadeiro fica REALMENTE protegido? 166
Campos e métodos privados só podem ser acessados de dentro da classe 167
Algumas idéias para encapsular classes 170
O encapsulamento mantém seus dados em boas condições 170
Propriedades facilitam o encapsulamento 171
Crie um aplicativo para testar a classe Farmer 171
Use propriedades automáticas para terminar a classe 172
E se quisermos alterar o multiplicador de ração? 173
Use um construtor para inicializar campos privados 174

agenteCIA

agenteKGB

agenteMI5

herança

6 Sua árvore genealógica de objeto

Às vezes você REALMENTE quer ser exatamente como seus pais.

Você já encontrou sem querer um objeto que quase faz exatamente o que você quer que o seu objeto faça? Já se pegou pensando se você pudesse apenas mudar algumas coisas, aquele objeto seria perfeito? Bom, essa é apenas uma razão que faz da herança um dos conceitos e técnicas mais poderoso na linguagem C#. Antes de avançar neste capítulo, você vai aprender como criar uma subclasse a partir de um objeto para herdar seu comportamento, mas mantendo a flexibilidade para alterar esse comportamento. Você vai evitar duplicar código, vai modelar o mundo real mais corretamente e vai ter código que é mais fácil de manter.

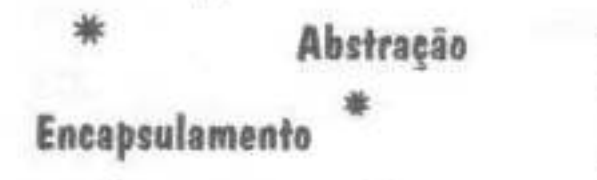
Catarina organiza aniversários também 184
Precisamos de uma classe BirthdayParty 184
Mais uma coisa...Você pode acrescentar uma taxa de R$100 para festas com mais de 12 pessoas? 190
Quando suas classes usam herança, você só precisa escrever seu código uma vez 190
Faça seu modelo de classe começando pelo geral e especificando cada vez mais 191
Como você criaria um simulador de zoológico? 192
Use herança para evitar duplicação de código em subclasses 192
Animais diferentes produzem sons diferentes 193
Pense em como agrupar os animais 194
Crie a hierarquia de classe 195
Toda subclasse estende sua classe base 195
Use dois-pontos para herdar de uma classe base 196
Sabemos que herança adiciona os campos, propriedades e métodos da classe base à subclasse... 199
Uma subclasse pode substituir métodos para alterar ou mudar completamente os métodos herdados 200
Em qualquer lugar onde se pode usar uma classe base, pode-se utilizar uma de suas subclasses 200
Uma subclasse pode acessar sua classe base usando a palavra-chave base 204
Quando uma classe base tem um construtor, sua subclasse precisa de um também 205
Agora você está pronto para terminar o trabalho para a Catarina! 206
Construa um sistema de gerenciamento de colmeia 210
Primeiro você construirá o sistema básico 211
Use herança para estender o sistema de gerenciamento de abelhas 217

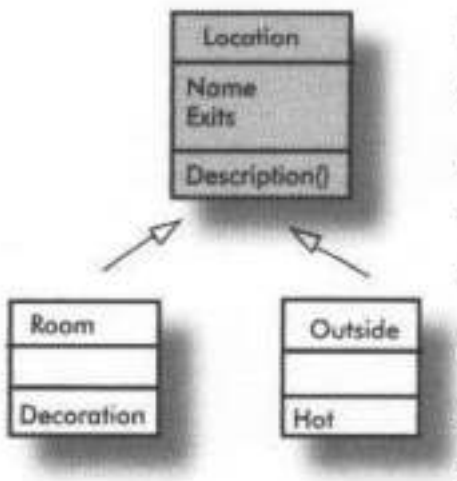
Interfaces e classes abstratas

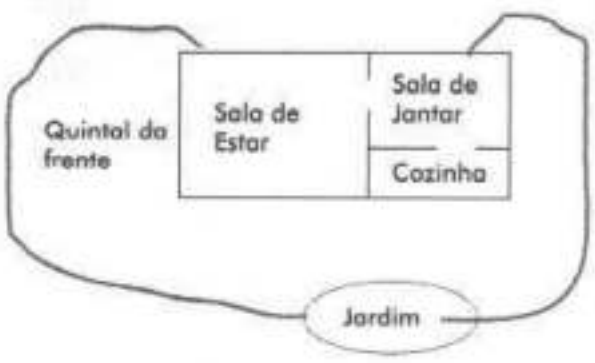
7 Fazendo as classes manterem suas promessas

Ações são mais importantes do que palavras.

Algumas vezes você precisa agrupar seus objetos baseando-se nas coisas que eles podem fazer ao invés de com relação a quais classes eles herdam características. É aí que entram as interfaces - elas permitem que você trabalhe com qualquer classe que possa fazer o trabalho. Mas com grande poder vem grande responsabilidade, e qualquer classe que implementar uma interface precisa se comprometer a preencher todos os requisitos... ou o compilador quebrará as pernas delas, entende?







Vamos voltar ao colmei-ço	224
Podemos usar herança para criar classes para tipos diferentes de abelhas	224
Uma interface diz para uma classe que ela precisa implementar certos métodos e propriedades	225
Use a palavra-chave interface para definir uma	226
Agora você pode criar uma instância de NectarStinger que realize ambas as tarefas	226
Classes que implementarem interfaces devem incluir TODOS os métodos	227
Pratique um pouco com interfaces	228
Você não pode criar instâncias de uma interface, mas pode criar referências	230
Referências de interfaces funcionam exatamente como referências de objetos	230
Você pode descobrir se uma classe implementa uma interface específica usando "is"	231
Interfaces podem herdar de outras	232
O RoboBee 4000 pode fazer o trabalho de uma abelha operária sem usar o precioso mel	233
"is" diz a você o que um objeto implementa	
"as" diz ao compilador como tratar um objeto	233
Uma cafeteira também é um eletrodoméstico	234
Supercoerção funciona tanto com objetos quanto com interfaces	234
Subcoerção deixa que você transforme seu eletrodoméstico de volta em cafeteira	236
Supercoerção e subcoerção funcionam com interfaces, também	237
Nesses lugares mais do que exatamente publico e privado	240
Os modificadores de acesso mudam o escopo	241
Algumas classes nunca deveriam ser instanciadas	243
Uma classe abstrata é como um cruzamento entre uma classe e uma interface	244
Algumas classes nunca devem ser intanciadas	245
Um método abstrato não tem corpo	246
Polimorfismo quer dizer que um objeto pode tomar muitas formas diferentes	253

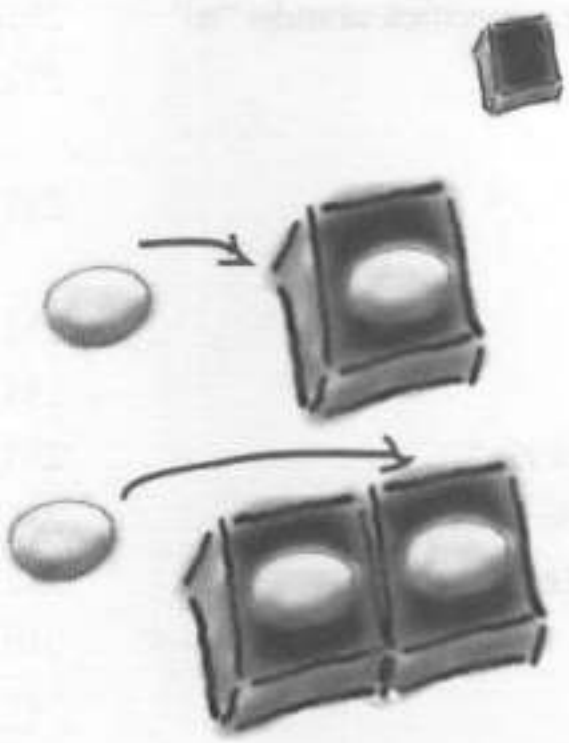
enumerações e coleções

8 Guardando muitos dados

Quando chove, pode ser garoa ou tempestade.

No mundo real, você não costuma lidar com seus dados em pequenas quantidades. Não, seus dados vão surgir em quantidade, aos montes, em pilhas. Você precisará de ferramentas muito poderosas para organizá-los todos, e é aí que as coleções entram. Elas permitem que você armazene, ordene e gerencie todos os dados que seu programa precisa para se concentrar. Dessa forma você pode pensar sobre escrever programas que trabalhem com seus dados, e deixar as coleções se preocuparem com tomar conta deles pra você.

Strings nem sempre servem para armazenar categorias de dados 270
Enums permitem enumerar um conjunto de valores válidos 270
Enums permitem representar números usando nomes 271
Nós poderíamos ter usado uma matriz para criar um baralho... 271
Matrizes são difíceis de trabalhar 275
Listas facilitam guardar coleções de... qualquer coisa 276
Listas são mais flexíveis que matrizes 276
Listas encolhem e aumentam dinamicamente 279
Objetos List podem armazenar qualquer tipo 280
Inicializadores de Coleções funcionam exatamente como os de objeto 284
Vamos criar uma lista de Patos 284
Listas são fáceis, mas ORDENAR pode ser complicado 285
Duas formas de ordenas seus patos 286
Use IComparer para dizer às suas listas como se ordenar 286
Crie uma instância de seu objeto de comparação 287
IComparer pode fazer comparações complexas 288
Use um dicionário para armazenar chaves e valores 290
Lista de Funcionalidades de um Dicionário 292
Chaves e valores também podem ter tipos diferentes 293
Você pode fazer seus próprios métodos sobrecarregados 298
E ainda MAIS outros tipos de coleções... 306
Uma fila é PEPS - Primeiro a Entrar, Primeiro a Sair 307
Uma pilha é UEPS - Último a Entrar, Primeiro a Sair 307

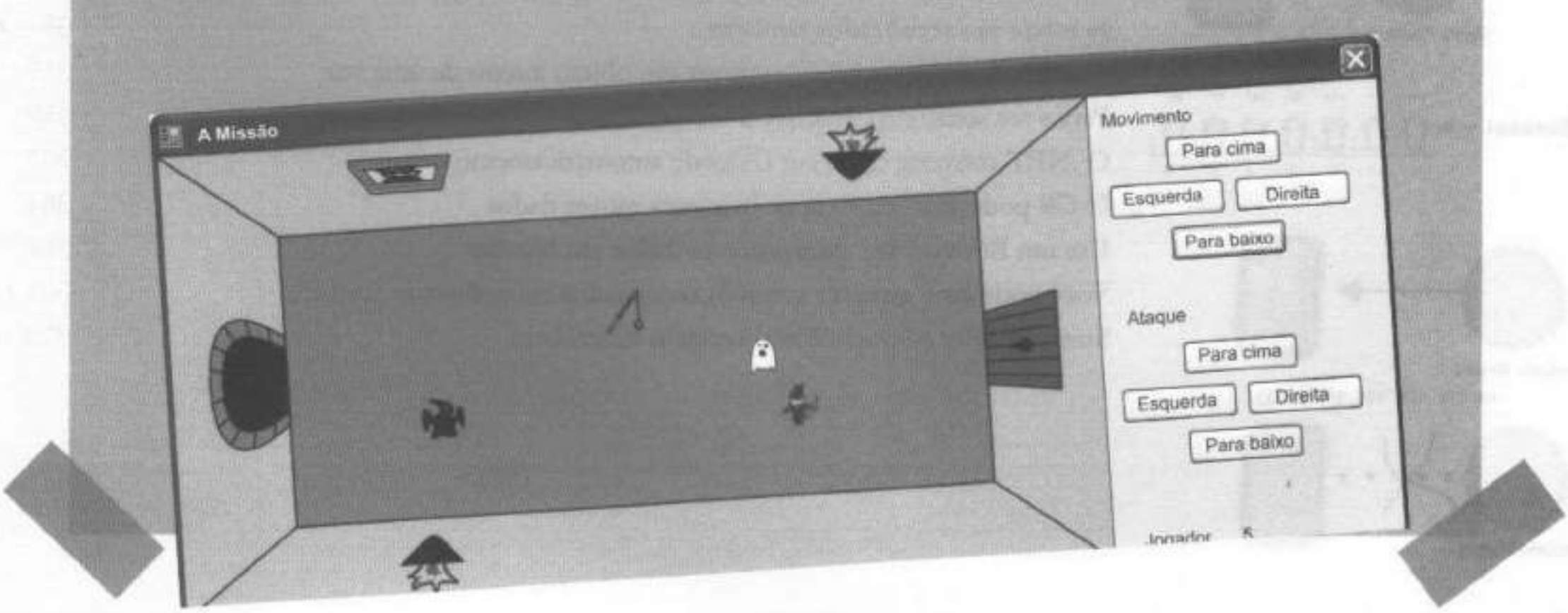


Laboratório C# 2

A Missão

Seu trabalho é criar um jogo de aventura onde um aventureiro importante está em uma missão para derrotar inimigos mortalmente perigosos nível após nível. Você vai criar um sistema de turnos, o que quer dizer que o jogador faz um movimento e então os inimigos fazem um. O jogador pode se mover ou atacar, e então cada inimigo tem a mesma chance. O jogo continua até que o jogador ou derrote todos os inimigos nos sete níveis ou morra.

A especificação: fazer um jogo de aventura 314
A diversão está apenas começando! 330



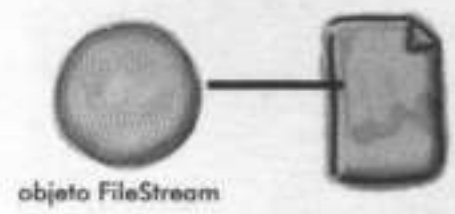
lendo e escrevendo arquivos

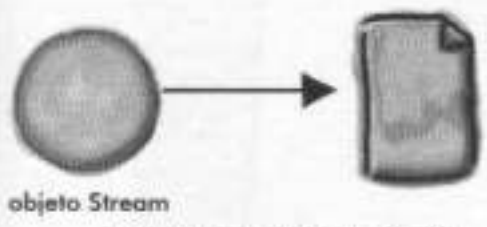
9 Salve a matriz de bytes, salve o mundo

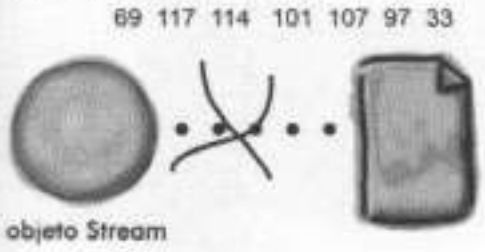
Às vezes vale a pena ser um pouco persistente.

Até agora, todos os seus programas têm funcionado por muito pouco tempo. Eles são ativados, executam por um curto intervalo e desligam. Mas isso nem sempre é o bastante, principalmente quando você está lidando com informação importante. Você precisa ser capaz de salvar seu trabalho. Neste capítulo, veremos como escrever dados em um arquivo, e então como ler essa informação de volta de um arquivo. Você aprenderá sobre as classes stream (fluxo, literalmente riacho) do .NET e também dará uma olhada nos mistérios do hexadecimal e do binário.

O C# usa streams para ler e escrever dados 332
Streams diferentes lêem e escrevem coisas diferentes 332
Um FileStream escreve bytes em um arquivo 333
Ler e escrever implica em dois objetos 337
Os dados podem passar por mais de uma stream 337
Use objetos nativos para exibir caixas de diálogo padrão 340
Caixas de diálogo também são objetos 342
Use as classes nativas File e Directory para trabalhar com arquivos e diretórios 342
IDisposable garante o descarte apropriado de seus objetos 347
Evite erros no sistema de arquivos com comandos using 348
Escrever arquivos geralmente envolve tomar muitas decisões 354
Use um comando switch para escolher a opção correta 354
Acrescente um construtor Deck() sobrecarregado que leia um baralho de um arquivo 356
O que acontece com um objeto quando ele é serializado? 357
Mas o que É exatamente um estado de objeto? O que precisa ser salvo? 357
Quando um objeto é serializado, todos os objetos ao qual se refere são serializados também... 358
Serialização permite ler ou escrever um objeto inteiro de uma vez 359
Para s ser serializada, marque-a com o atributo [Serializable] 359
O .NET converte texto em Unicode automaticamente 363
O C# pode usar matrizes de byte para mover dados 364
Use um BinaryWriter para escrever dados em binário 364
Você pode ler e escrever arquivos serializados manualmente também 367
StreamReader e StreamWriter servirão muito bem 370







Tratamento de Exceções

10 Apagar incêndios é coisa do passado

Programadores não são treinados para serem bombeiros.

Você trabalhou muito, enfrentou árduas leituras de manuais e alguns poucos livros empolgantes da série Use a Cabeça, e atingiu o ápice da sua profissão: programador senior. Mas você fica recebendo notas da sua empresa porque seu programa levanta erros e fecha, ou não se comporta como deveria. Nada tira mais você do caminho da programação do que ter que corrigir um erro estranho... mas com tratamento de exceções, você pode escrever código para lidar com problemas que aparecem. Ainda melhor, você pode até mesmo reagir a esses problemas, e manter as coisas funcionando.

Aldo precisa de desculpas para passear 378
Quando seu programa lança uma exceção, o .NET gera um objeto Exception 381
O código do Aldo fez algo inesperado 382
Todos os objetos de exceção herdam de Exception 383
A barra de ferramentas de depuração ajuda a rastrear e prevenir exceções 384
Use o depurador do IDE para desentocar o erro no gerenciador de desculpas 385
Opa, o código ainda está com problemas... 387
Trate de exceções com try e catch 389
O que fazer quando um método que você quer chamar apresenta riscos? 390
Use o depurador para seguir o fluxo do try/catch 391
Se você tem código que SEMPRE deve executar, use um bloco finally 393
Use o objeto Exception para obter informações sobre o problema 396
Use mais de um bloco catch para lidar com múltiplos tipos de exceção. 397
Uma classe lança uma exceção, outra a captura 397
As abelhas precisam de uma exceção para esgotamento de mel 398
Uma forma fácil de evitar muitos problemas: using
providencia um try e um finally pra você 401
Evitando exceções: implemente IDisposable para fazer sua própria limpeza 402
A pior captura: bloco de comentários 403
Tudo bem usar soluções temporárias (temporariamente) 404
Algumas idéias simples para tratamento de exceções 405
Aldo finalmente consegue suas férias... 410

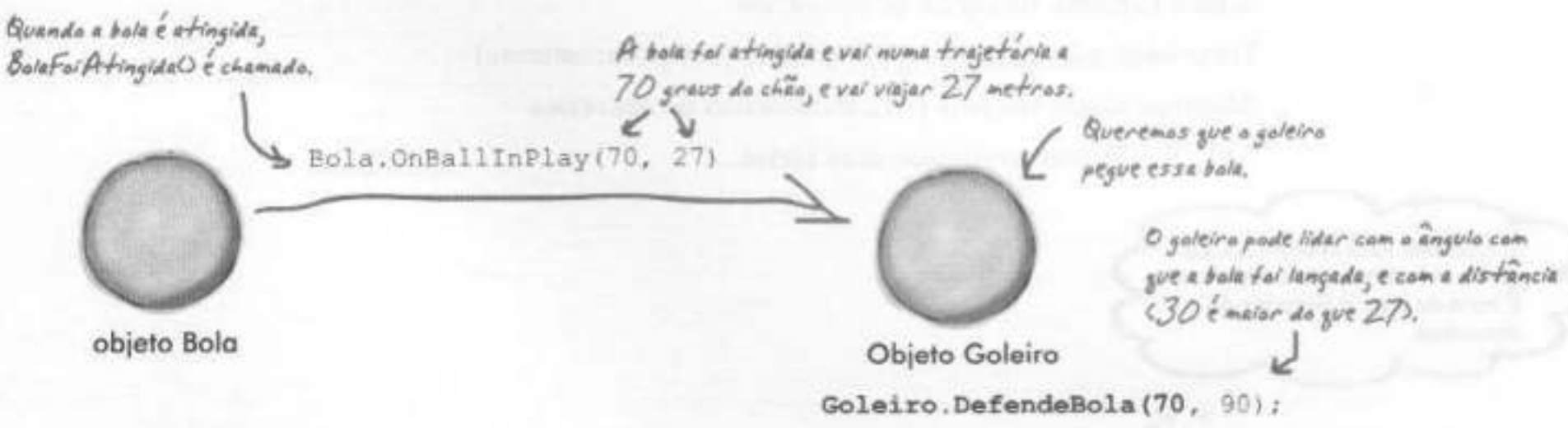
eventos e delegados

11 O que o seu código faz quando você não está olhando

Seus objetos estão começando a pensar por eles mesmos.

Você nem sempre pode controlar o que um objeto está fazendo. Algumas coisas... simplesmente acontecem. E quando acontecem, você vai querer que seus objetos sejam inteligentes o suficente para responder a qualquer coisa que apareça. É para isso que servem os eventos. Um objeto publica um evento, outro objeto se inscreve nele, e todos trabalham juntos para manter as coisas funcionando. O que é ótimo, até que você tenha um monte de objetos respondendo ao mesmo evento. Nesses casos os callbacks serão úteis.

Já desejou que seus objetos pensassem por si mesmos? 412
Mas como um objeto SABE como responder? 412
Quando um EVENTO ocorre... os objetos prestam atenção 412
Um objeto dispara seus eventos e os outros ficam atentos 413
Então outros objetos tratam o evento 414
Ligando os pontos 415
O IDE cria tratadores de eventos para você automaticamente 418
Os formulários que você esteve fazendo usaram eventos 424
Conectando remetentes com destinatários de eventos 426
Um delegado COLOCA-SE NO LUGAR de um método real 426
Delegados em ação 427
Qualquer objeto pode inscrever-se num evento público 430
Use um callback, ao invés de um evento, para associar exatamente um único objeto a um delegado 432
Callbacks usam delegados, mas NÃO eventos 432



revisão e prévia

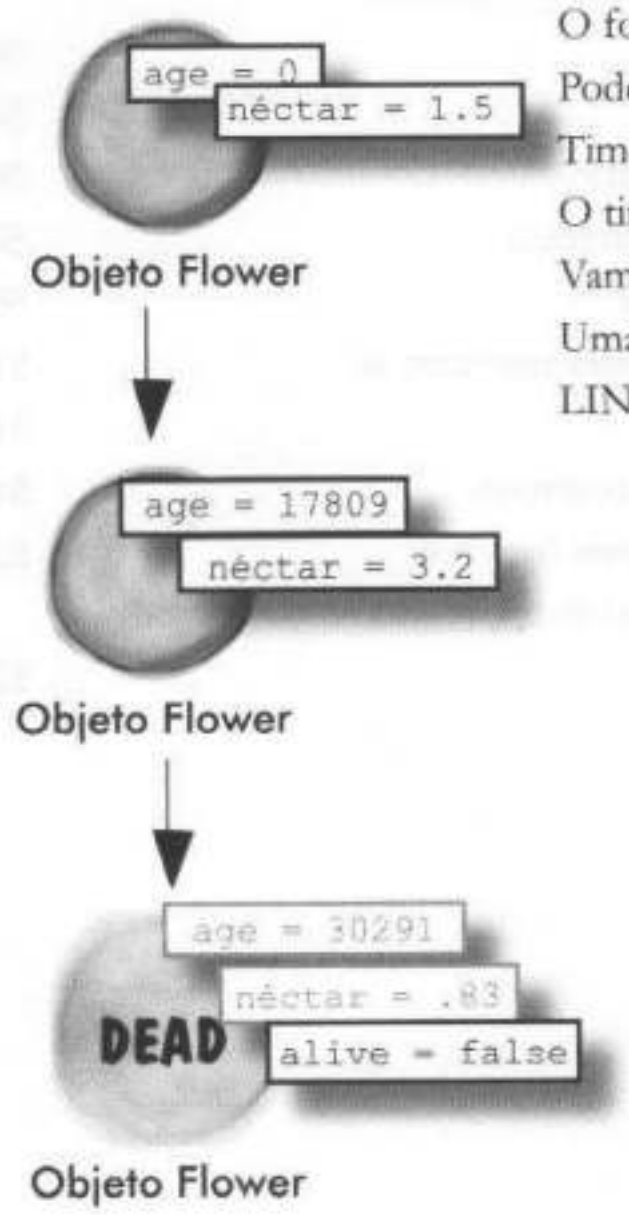
12 Conhecimento, poder, e desenvolver coisas legais

Aprender não serve de muito até que você CRIE algo.

Até que você tenha realmente escrito código funcional, é difícil ter certeza se você realmente sabe algunsdos conceitos mais complexos do C#. Nesse capítulo, aprenderemos sobre alguns novos conceitos e amarraremos algumas pontas soltas: falaremos sobre timers e como lidar com coleções usando LINQ, entre outras coisas. Nós também vamos desenvolver a fase 1 de um aplicativo realmente complexo, e nos certificar que você tenha um bom domínio sobre o que já aprendeu nos capítulos anteriores. Então ponha o cinto de segurança... é hora de desenvolver um software legal.

Você percorreu um longo caminho, baby 440
Nós ja fomos apicultores 441
A estrutura do simulador de colméia 442
Desenvolvendo o simulador de colméia 442
Vida e morte de uma flor 447
Agora precisamos de uma classe Bee 448
Preenchendo a classe Hive 455
O método Go() da colméia 456
Estamos prontos para o mundo 456
Estamos criando um sistema baseado em turnos 457
Dê comportamento para as abelhas 462
O formulário principal diz ao mundo para funcionar 463
Podemos usar o mundo para conseguir estatísticas 464
Timers disparam eventos repetidamente 465
O timer está usando um delegado nos bastidores 466
Vamos trabalhar com grupos de abelhas 471
Uma coleção coleciona... DADOS 471
LINQ facilita trabalhar com dados em coleções e bases de dados 473

Vida e morte de uma flor



controles e gráficos

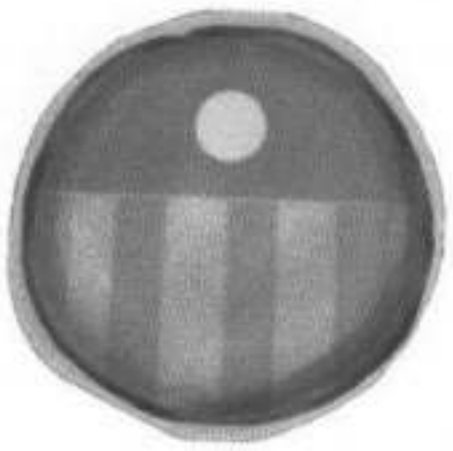
13 Deixe bonito

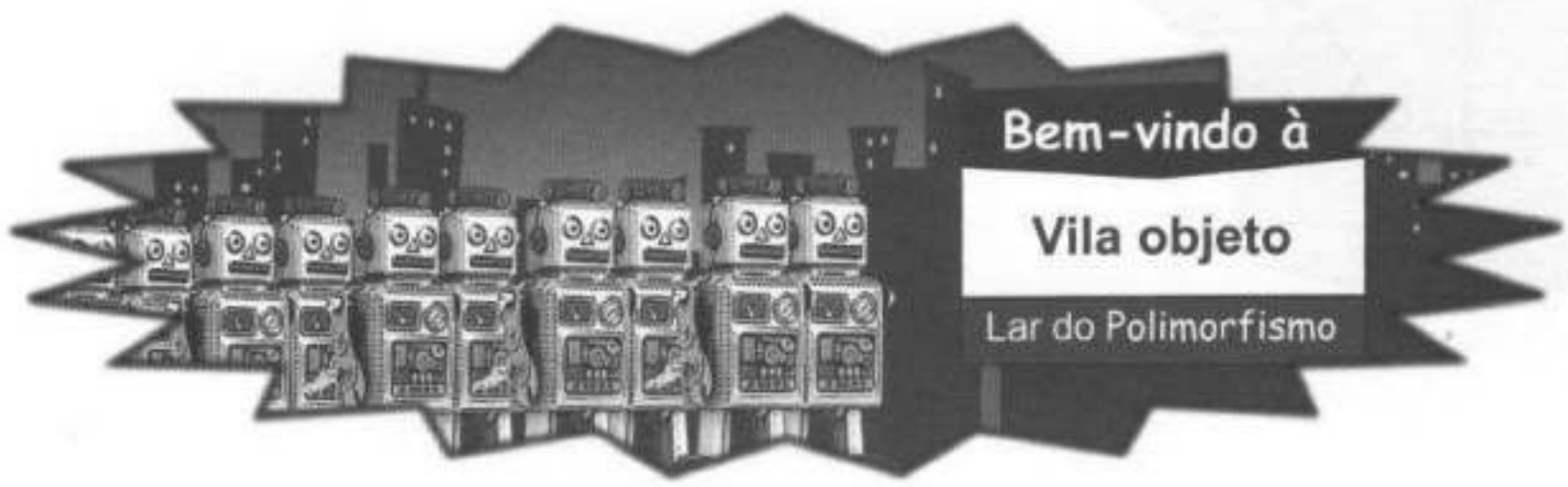
Algumas vezes você precisa fazer os gráficos você mesmo.
Passamos bastante tempo confiando que os controles lidassem com tudo o que fosse visual nos nossos aplicativos. Mas algumas vezes isso não é suficiente - por exemplo quando você tem que animar uma imagem. E uma vez que você comece a lidar com animação, você vai acabar criando seus próprios controles para seus programas .NET, talvez adicionando um buffer duplo, e mesmo desenhando diretamente nos formulários. Tudo começa com o objeto Graphics, os Bitmaps, e uma determinação de não aceitar os gráficos como eles são.

Você esteve usando controles até agora para interagir com seus programas 478
Controles de formulário são apenas objetos 478
Adicione um renderizador à sua estrutura 480
Controles são adequados como elementos visuais 482
Desenvolva seu primeiro controle animado 485
Seus controles precisam descartar os controles deles também! 488
Um UserControl é uma forma fácil de desenvolver um controle 489
Adicione os formulários de colméia e de campo ao projeto 492
Desenvolva o Renderizador 493
Vamos examinar de perto esses problemas de performance 498
Você redimensionou seus Bitmaps usando um objeto Graphics 500
Os seus recursos que são imagens ficam armazenados em objetos Bitmap 501
Use System.Drawing para ASSUMIR O CONTROLE dos gráficos você mesmo 502
Um tour de 30 segundos pelos gráficos da GDI+ 502
Use gráficos para desenhar uma figura num formulário 503
Gráficos podem resolver nosso problema de transparência 507
Use o evento Paint para fazer seus gráficos durarem 508
Examinando de perto como os formulários e controles repintam-se 511
Buffer duplo suaviza sua animação 513
Buffer duplo está implementado nos formulários e controles 514
Use um objeto Graphics e um tratador de eventos para imprimir 520
PrintDocument trabalha em conjunto com o diálogo de impressão e o de visualização do Windows 521

14 CAPITÃO MARAVILHOSO A MORTE DO OBJETO

Capitão Maravilhoso, o objeto mais maravilhoso da Vila Objeto, persegue seu arqui-inimigo… 528
Sua última chance de FAZER alguma coisa… o finalizador de seu objeto 534
Quando EXATAMENTE um finalizador executa? 534
Dispose() funciona com using, finalizadores com o coletor de lixo 535
Finalizadores não podem depender de estabilidade 537
Faça com que um objeto serialize a si mesmo em seu Dispose() 538
Enquanto isso, nas ruas da Vila Objeto… 541
Uma estrutura parece-se com um objeto... 542
...mas não fica na pilha dinâmica 542
Valores são copiados, referências são atribuídas 542
Estruturas são tipos de valor; objetos são tipos de referência 544
Pilha dinâmica versus estática: mais sobre a memória 546
Capitão Maravilhoso... mas nem tanto 550
Métodos de extensão adicionam novos comportamentos a classes EXISTENTES 551
Estendendo um tipo básico: string 554





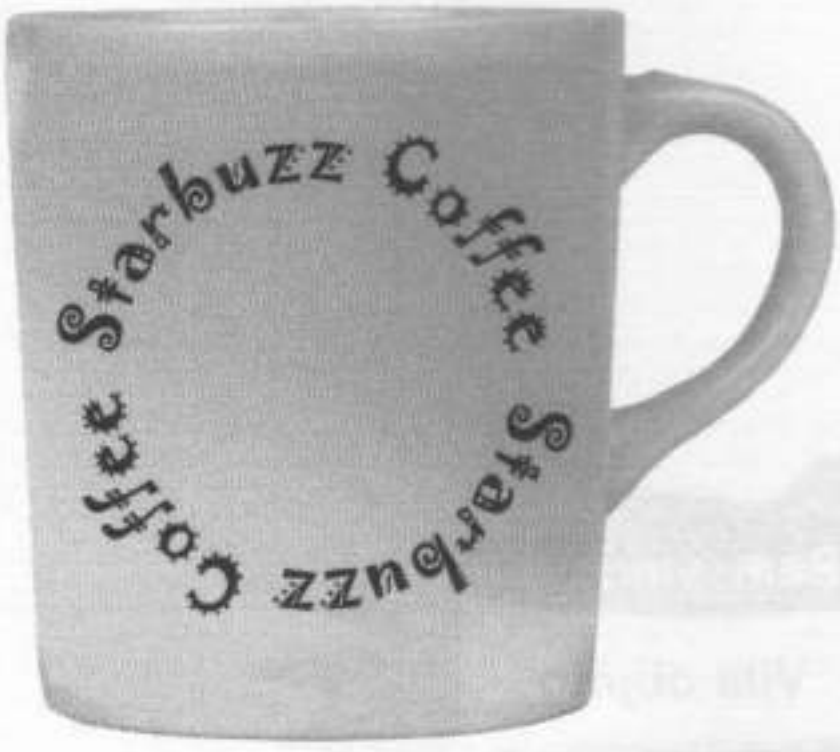
LINQ

15 Controle seus dados

É um mundo voltado para dados... é melhor você saber como viver nele.

Foram-se os tempos em que você podia programar por dias, mesmo semanas, sem ter que lidar com montes de dados. Nos dias de hoje, tudo tem relação com dados. De fato, você com frequência vai ter que lidar com dados oriundos de mais de uma fonte... e em mais de um formato. Bases de dados, XML, coleções de outros programas... é tudo parte do trabalho de um bom programador C#. E é aí que o LINQ entra. Ela não só permite que se consulte dados de uma forma simples e intuitiva, mas permite também fazer agrupamentos, e unir dados de muitas fontes.

Um projeto fácil... 560
...mas os dados estão espalhados pra todo lado 560
LINQ pode buscar dados em múltiplas fontes 561
Coleções do .NET já são preparadas para usar LINQ 561
LINQ facilita as consultas 562
A LINQ é simples, mas suas consultas não deve ser 563
A LINQ é versátil 566
A LINQ pode combinar seus resultados em grupos 571
Combine os valores de Jaime em grupos 572
Use join para combinar duas coleções numa única consulta 574
Jaime poupa um monte de grana 575
Conecte a LINQ a uma base de dados SQL. 577
Use uma consulta de junção para conectar o Starbuzz e a Vila Objeto 580



Laboratório C# 3

Invasores

Neste laboratório você vai fazer homenagem ao mais popular, reverenciado e copiado ícone da história dos videogames, um jogo que não precisa de maiores apresentações. É hora de você desenvolver Space Invaders.

O avô de todos os videogames 584

E ainda pode ser feito mais... 598

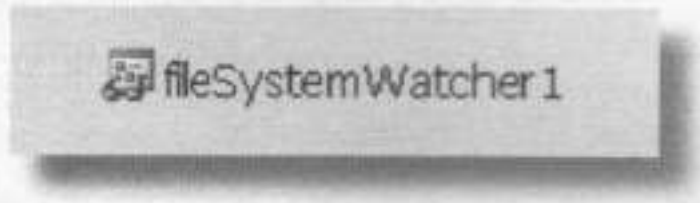
sobras

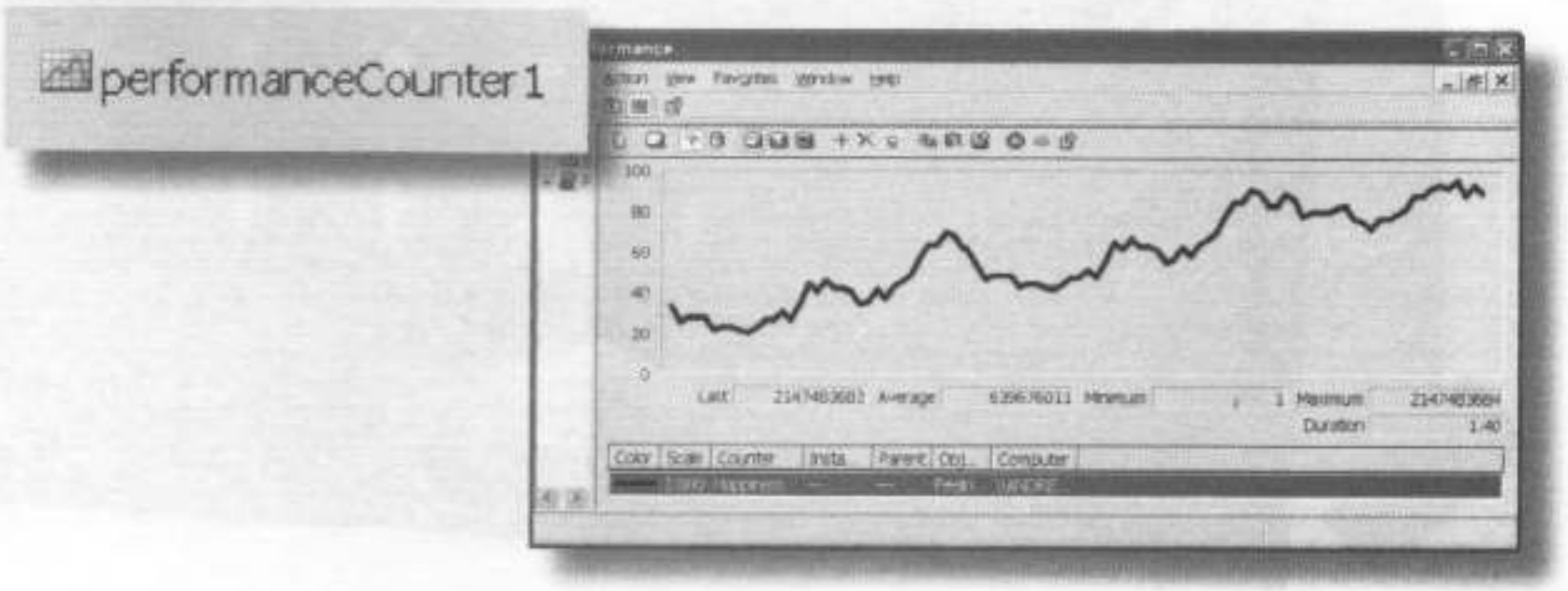
9 Os 5 principais tópicos que gostaríamos de ter incluido neste livro

A diversão está apenas começando!

Mostramos a você muitas ferramentas ótimas para desenvolver programas poderosos em C#. Mas não seria possível incluir cada uma das ferramentas, tecnologias e técnicas neste livro - simplesmente não existem páginas o suficiente. Tivemos que fazer algumas escolhas realmente difíceis a respeito do que incluir e o que deixar de fora. Aqui apresentamos alguns itens que não sobreviveram aos cortes. Mas mesmo que não possamos nos aprofundar neles, ainda acreditamos que eles são importantes e úteis, e queremos dar a você uma pequena introdução a respeito deles.

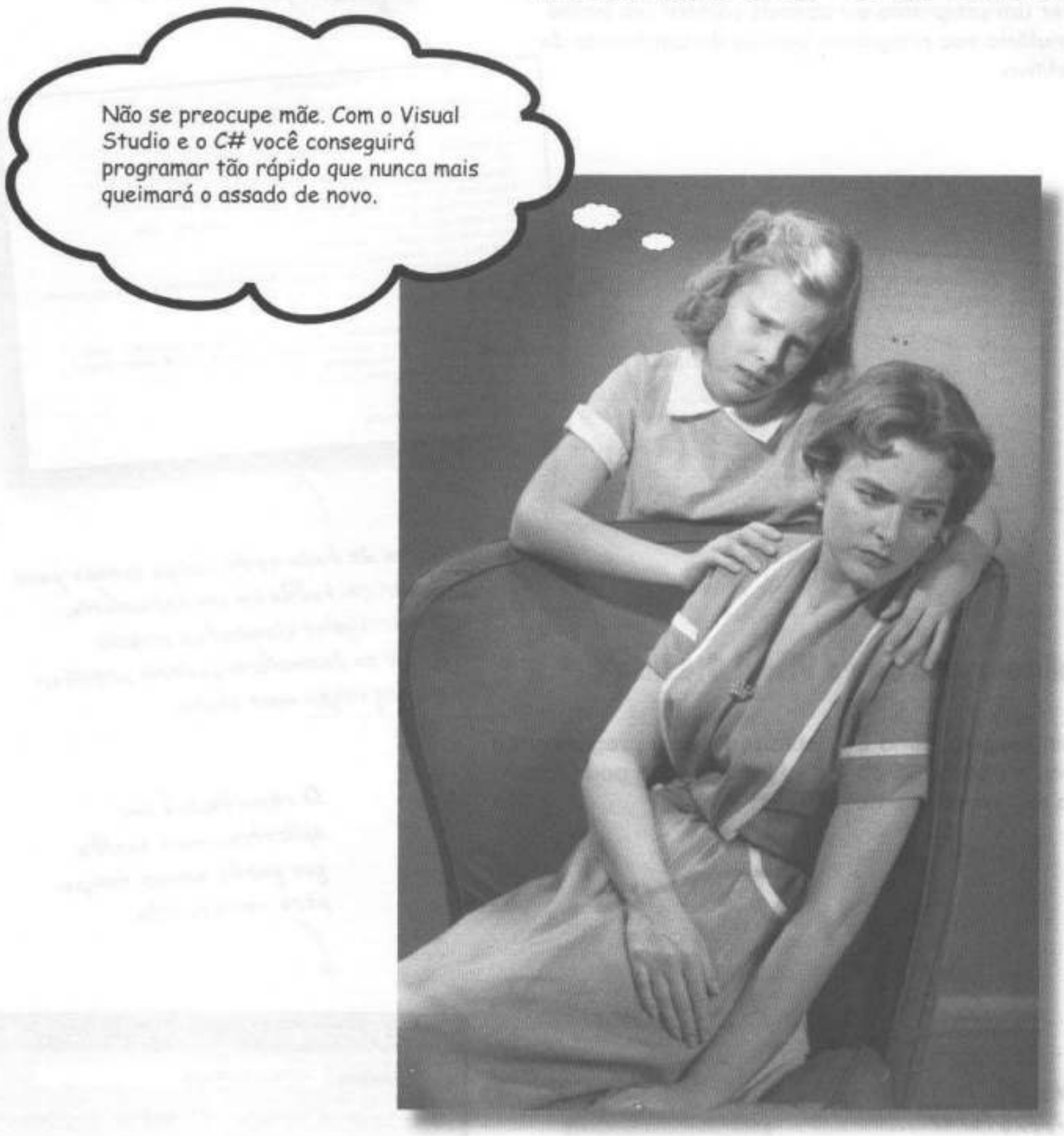
1 LINQ para XML 600
2 Refatoração 602
3 Alguns de nossos componentes favoritos na caixa de ferramentas 604
4 Aplicativos de Console 605
5 Framework de Apresentação do Windows 606
Você sabia que com o C# e o Framework .NET você pode... 608





1 Seja produtivo com C#

Aplicativos visuais em 10 minutos ou menos



Quer criar ótimos programas realmente rápidos?

O C# é uma linguagem de programação poderosa e uma ferramenta valiosa na palma de sua mão. **Com a IDE do Visual Studio você nunca mais gastará horas escrevendo código obscuro para um botão funcionar**. Melhor ainda, você poderá concentrar-se em realizar o seu trabalho, em vez de lembrar qual parâmetro de qual método é o nome para um botão e qual é o seu rótulo. Interessante? Vire a página e vamos começar a programar.

Por que você deve aprender C#

O C# e o IDE do Visual Studio facilitam o trabalho de escrever código e de desenvolve -lo rapidamente. Quando você estiver trabalhando com o C# o IDE será seu melhor amigo e companhia constante.

O IDE - ou Visual Studio Integrated Development Environment (Ambiente Integrado de Desenvolvimento) - é uma parte importante de trabalhar com C#. É um programa que ajuda a editar seu código, gerenciar seus arquivos e publicar seus projetos.

Aqui vemos o que o IDE automatiza para você...

Para escrever um programa ou apenas colocar um botão em um formulário seu programa precisa de um monte de código repetitivo.

```
using System;
using System.Collections.Generic;
using System.Windows.Forms;
namespace A_New_Program
{
    static class Program
    {
        /// <summary>
        /// The main entry point for the application.
        /// </summary>
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new Form1());
        }
    }
}
```

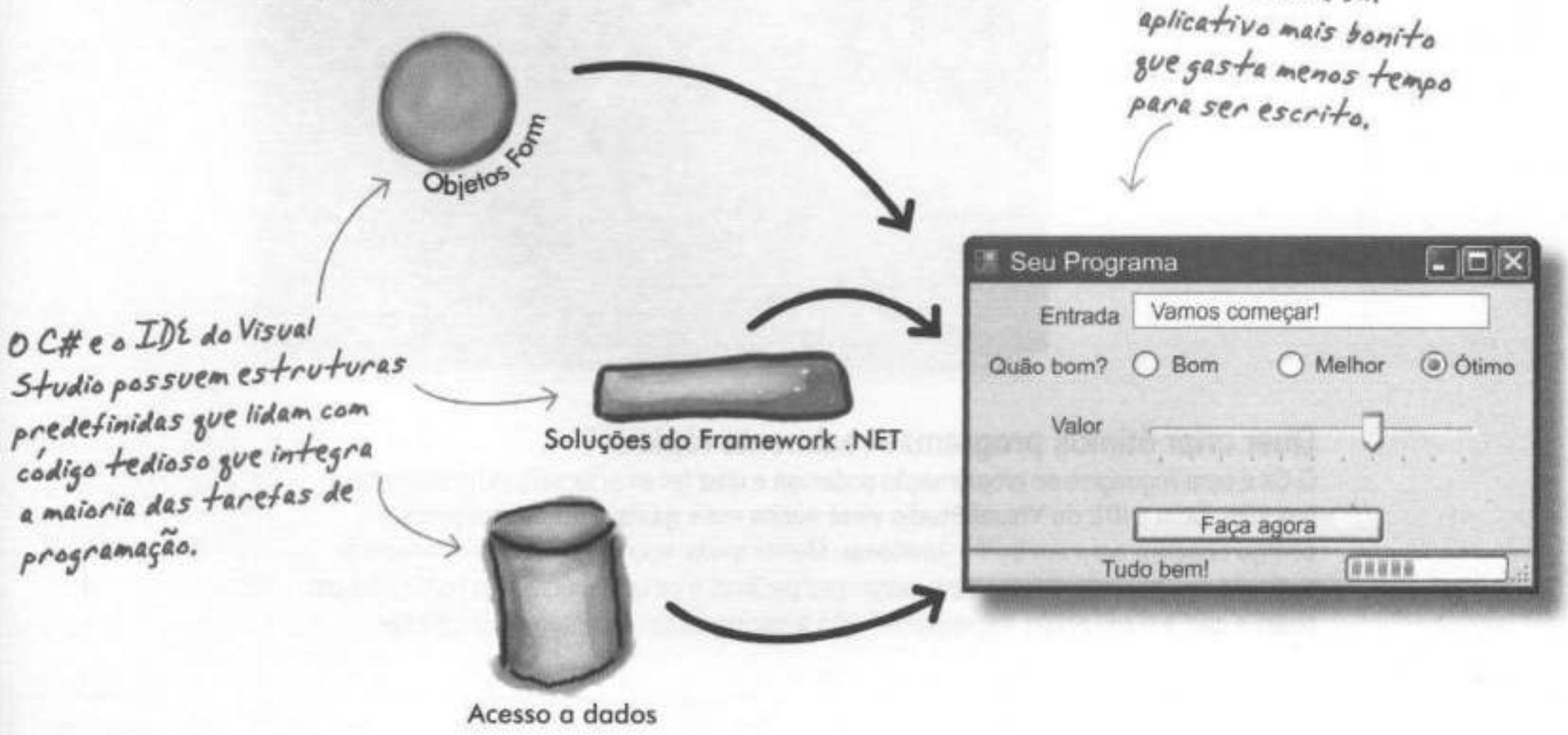
```
private void InitializeComponent()
{
    this.button1 = new System.Windows.Forms.Button();
    this.SuspendLayout();
    //
    // button1
    //
    this.button1.Location = new System.Drawing.Point(105, 56);
    this.button1.Name = "button1";
    this.button1.Size = new System.Drawing.Size(75, 23);
    this.button1.TabIndex = 0;
    this.button1.Text = "button1";
    this.button1.UseVisualStyleBackColor = true;
    this.button1.Click += new System.EventHandler(this.button1_Click);
    //
    // Form1
    //
    this.AutoScaleDimensions = new System.Drawing.SizeF(8F, 16F);
    this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
    this.ClientSize = new System.Drawing.Size(292, 267);
    this.Controls.Add(this.button1);
    this.Name = "Form1";
    this.Text = "Form1";
    this.ResumeLayout(false);
}
```

É preciso de todo este código apenas para desenhar um botão em um formulário. Adicionar alguns elementos visuais adicionais ao formulário poderia precisar até de dez vezes mais código.

O que você consegue com o Visual Studio e C#...

Com uma linguagem como C#, otimizada para programação em Windows, e com o IDE do Visual Studio, você pode focar-se no que o seu programa deve **fazer**:

O resultado é um aplicativo mais bonito que gasta menos tempo para ser escrito.



O C# e o IDE do Visual Studio facilitam muitas coisas

Quando você usa C# e o Visual Studio tem todas estas grandes características ao seu alcance, sem nenhum trabalho extra. Juntos, eles permitem que você:

1. **Faça um aplicativo RAPIDAMENTE.** Criar programas em C# leva segundos. A linguagem é poderosa e fácil de aprender; e o IDE do Visual Studio faz muito do trabalho automaticamente para você. Podem-se deixar tarefas de código simples para o IDE e concentrar-se no que o seu código deveria fazer.

2. **Faça uma interface de usuário com boa aparência.** O Form Designer no IDE do Visual Studio é uma das ferramentas de design mais fáceis de usar que existem por aí. Ela faz tanto que você descobrirá que criar interfaces de usuário lindíssimas é umas das partes mais satisfatórias de desenvolver um aplicativo em C#. Você pode fazer programas profissionais com todas as características sem ter de passar horas escrevendo uma interface de usuário gráfica totalmente do zero.

3. **Crie e interaja com bases de dados.** O IDE inclui uma interface simples para construir bases de dados e integra-se perfeitamente ao SQL Server Express, assim como muitos outros sistemas de base de dados populares.

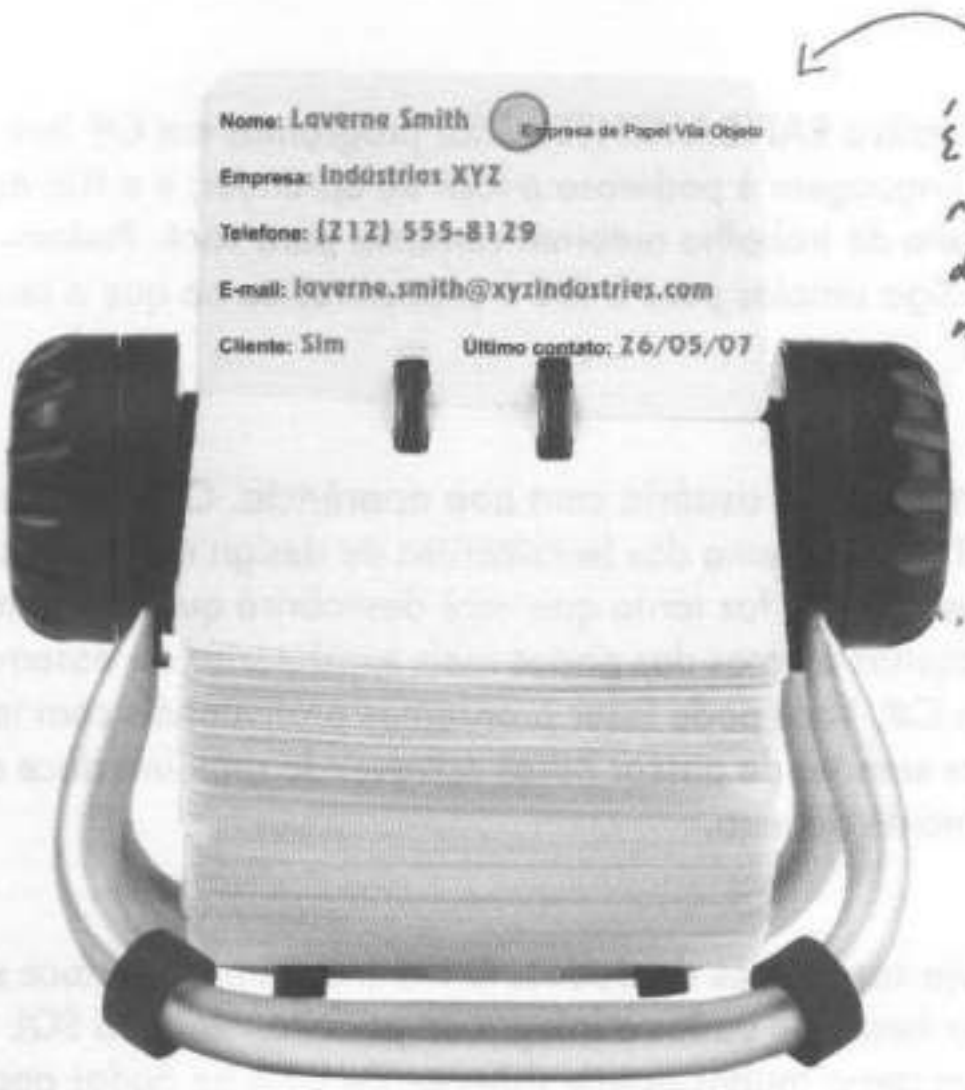
4. **Concentre-se em resolver seus problemas REAIS.** O IDE faz muito, mas você ainda está no controle do que é feito com o C#. Ele permite-lhe concentrar-se em seu programa, em seu trabalho (ou diversão!) e em seus clientes; mas cuida de todo o trabalho repetitivo, como:
   - ★ manter o registro todos os seus projetos;
   - ★ facilitar a edição do código de seu projeto;
   - ★ manter o registro sobre os gráficos, áudio, ícones e outros recursos de seu projeto;
   - ★ gerenciar e interagir com bases de dados.

Isto significa que todo o tempo gasto para esta programação de rotina pode ser seu e usado para **criar programas muito legais**.

Você verá exatamente o que queremos dizer a seguir.

Ajude o diretor geral a eliminar os papéis

A Empresa de Papel Vila Objeto contratou um novo diretor geral. Ele adora fazer caminhadas, café e a natureza... e ele decidiu ajudar a salvar as florestas; quer ser um executivo "sem papel", começando com seus contatos. Ele está a caminho de Aspen para esquiar no fim de semana e quer ter um novo programa de agenda pronto quando voltar. Caso contrário... bem... não será apenas o antigo diretor geral que estará em busca de um emprego.

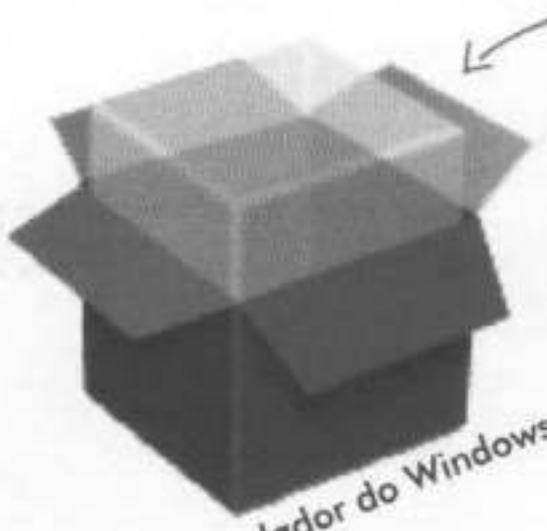


É melhor você encontrar rapidamente uma forma destes dados entrarem no notebook do diretor.

Conheça as necessidades dos usuários antes de começar a fazer seu programa

Antes que possamos começar a escrever o aplicativo de agenda – ou *qualquer* outro programa – precisamos de um minuto para pensar em **quem irá usá-lo** e **o que eles precisam** que seja feito.

① O diretor geral precisa conseguir executar seu programa de agenda no trabalho e também em seu notebook. Ele precisará de um instalador para ter certeza de que todos os arquivos corretos estejam em cada máquina.



O diretor quer poder executar seu programa no desktop e no notebook, então um instalador é uma necessidade.

② A equipe de vendas da Empresa de Papel Vila Objeto quer acessar sua agenda também. Eles podem usar seus dados para fazer listas de e-mail para obter mais ordens de compra de papel de seus clientes.

O diretor geral acha que uma base de dados seria a melhor forma para que todos na empresa pudessem ter acesso aos dados dele. Assim, ele pode manter apenas uma cópia de todos os seus contatos.

Já sabemos que o Visual C# facilita o trabalho com bases de dados. Ter os contatos em uma base de dados permite que o diretor e a equipe de vendas tenham acesso às informações, ainda que não exista apenas uma cópia dos dados.

Aqui está o que você vai desenvolver

Você precisará de um aplicativo com uma interface gráfica de usuário, objetos para comunicarem-se com uma base de dados, a própria base de dados e um instalador. Parece muito trabalhoso, mas você fará isto tudo nas próximas páginas.

Aqui está a estrutura do programa que criaremos:

Você criará um formulário Windows com vários controles visuais.

O aplicativo tem uma camada de dados separada que interage com a base de dados.

Objetos Visuais .NET

Objetos de Base de dados .NET

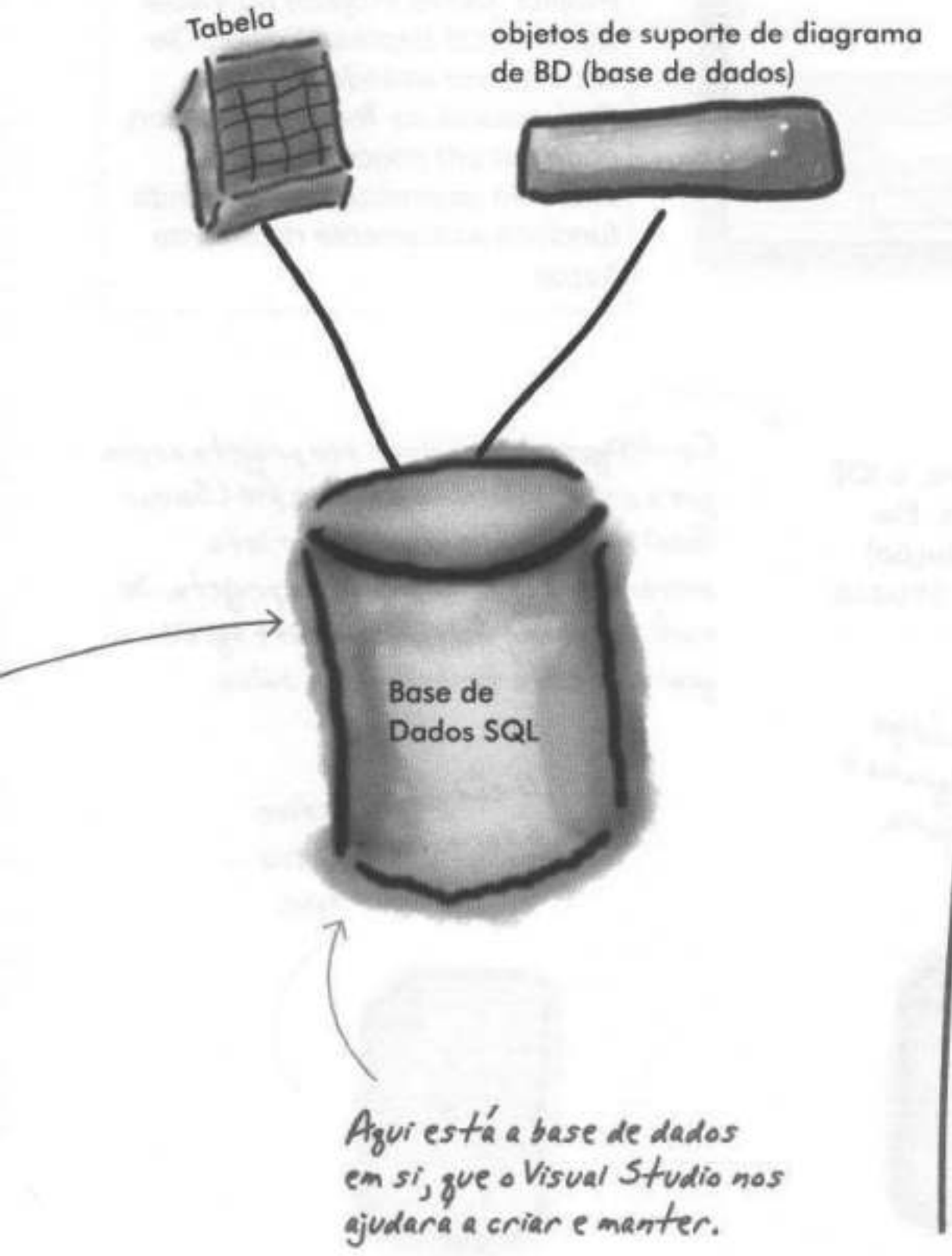
Cada um destes objetos representa um controle no formulário de agenda que criaremos.

Vamos precisar de objetos para comunicarem-se com nossas tabelas, um diagrama para fazer com que nosso aplicativo saiba qual é a estrutura da base de dados e outras coisas mais.

Os dados são armazenados em uma tabela na base de dados SQL Server Express.

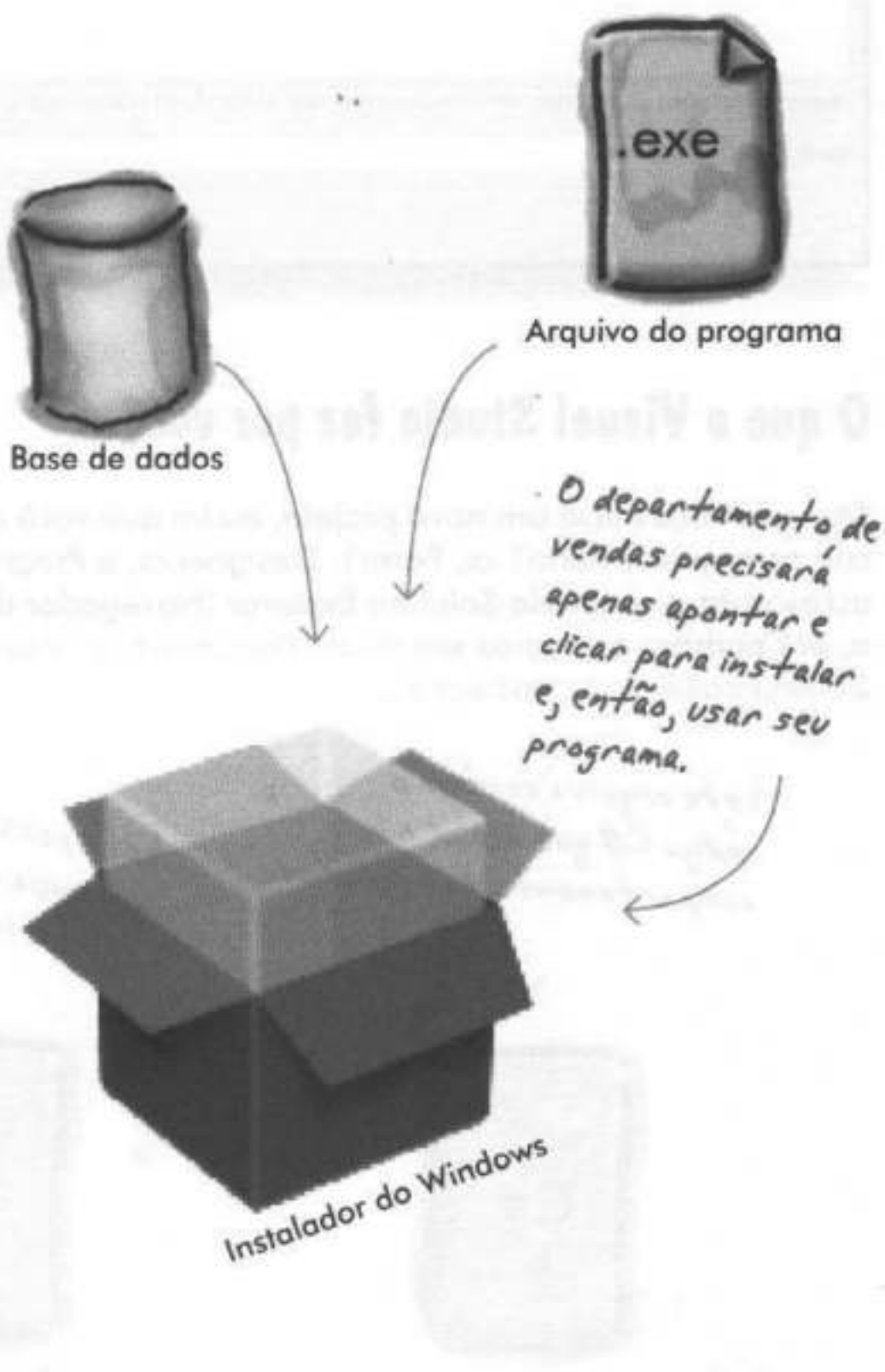
Uma vez que o programa tenha sido feito, ele será incluído num pacote do instalador do Windows.

Armazenamento de Dados



Aqui está a base de dados em si, que o Visual Studio nos ajudará a criar e manter.

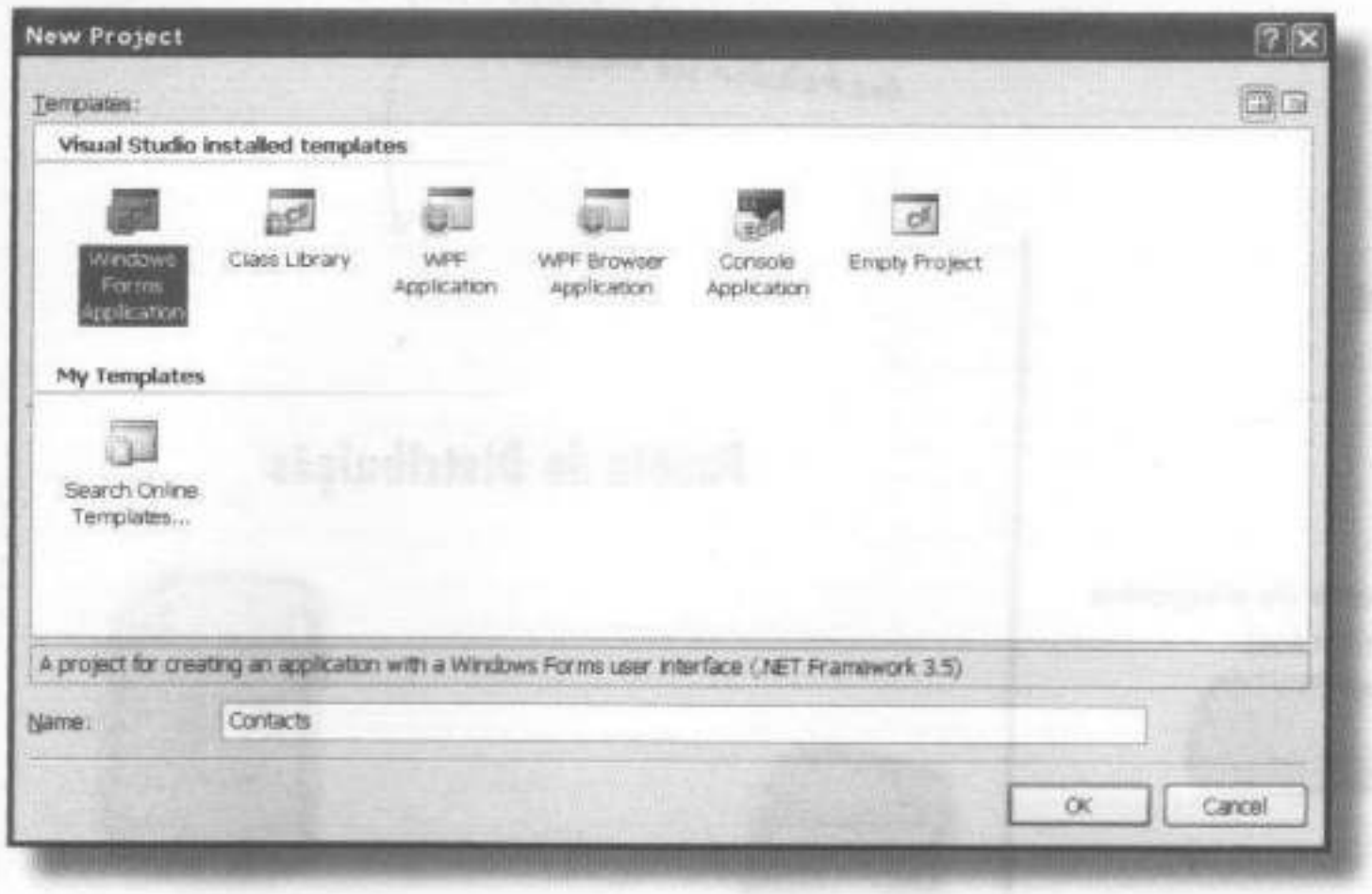
Pacote de Distribuição



O departamento de vendas precisará apenas apontar e clicar para instalar e, então, usar seu programa.

O que você faz no Visual Studio...

Vá em frente e instale o Visual Studio, se ainda não o fez. Pule a página inicial e selecione New Project (novo projeto) no menu **File** (Arquivo). Nomeie seu projeto como "Contatos" e clique em OK.



Veja bem!

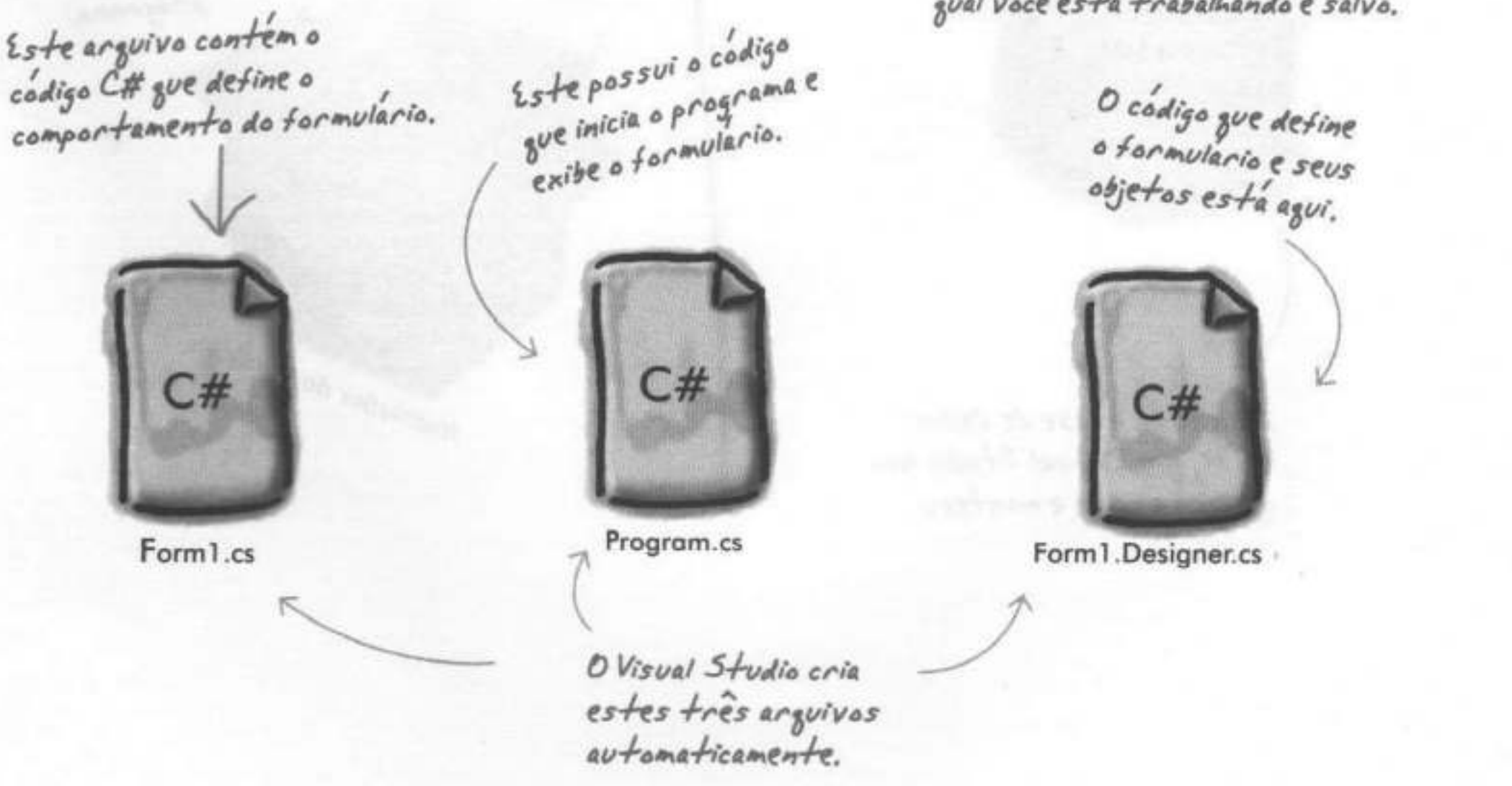
As coisas podem parecer um pouco diferentes em seu IDE.

Esta é a aparência da janela "New Project" (Novo Projeto) no Visual Studio 2008 Express Edition. Se você estiver usando a edição Professional ou Team Foundation, pode ser um pouco diferente. Mas não se preocupe, tudo ainda funciona exatamente da mesma forma.

O que o Visual Studio faz por você...

Quando você inicia um novo projeto, assim que você o salva, o IDE cria os arquivos Form1.cs, Form1. Designer.cs, e Program.cs. Ele acrescenta-os à janela Solution Explorer (Navegador de Solução) e, por padrão, coloca-os em `Meus Documentos\Visual Studio 2008\Projects\Contacts\`.

Certifique-se de salvar seu projeto assim que o criar, selecionando Save All (Salvar Tudo) no menu File (Arquivo) – isto salvará todos os arquivos do projeto. Se você selecionar Salvar, somente aquele no qual você está trabalhando é salvo.



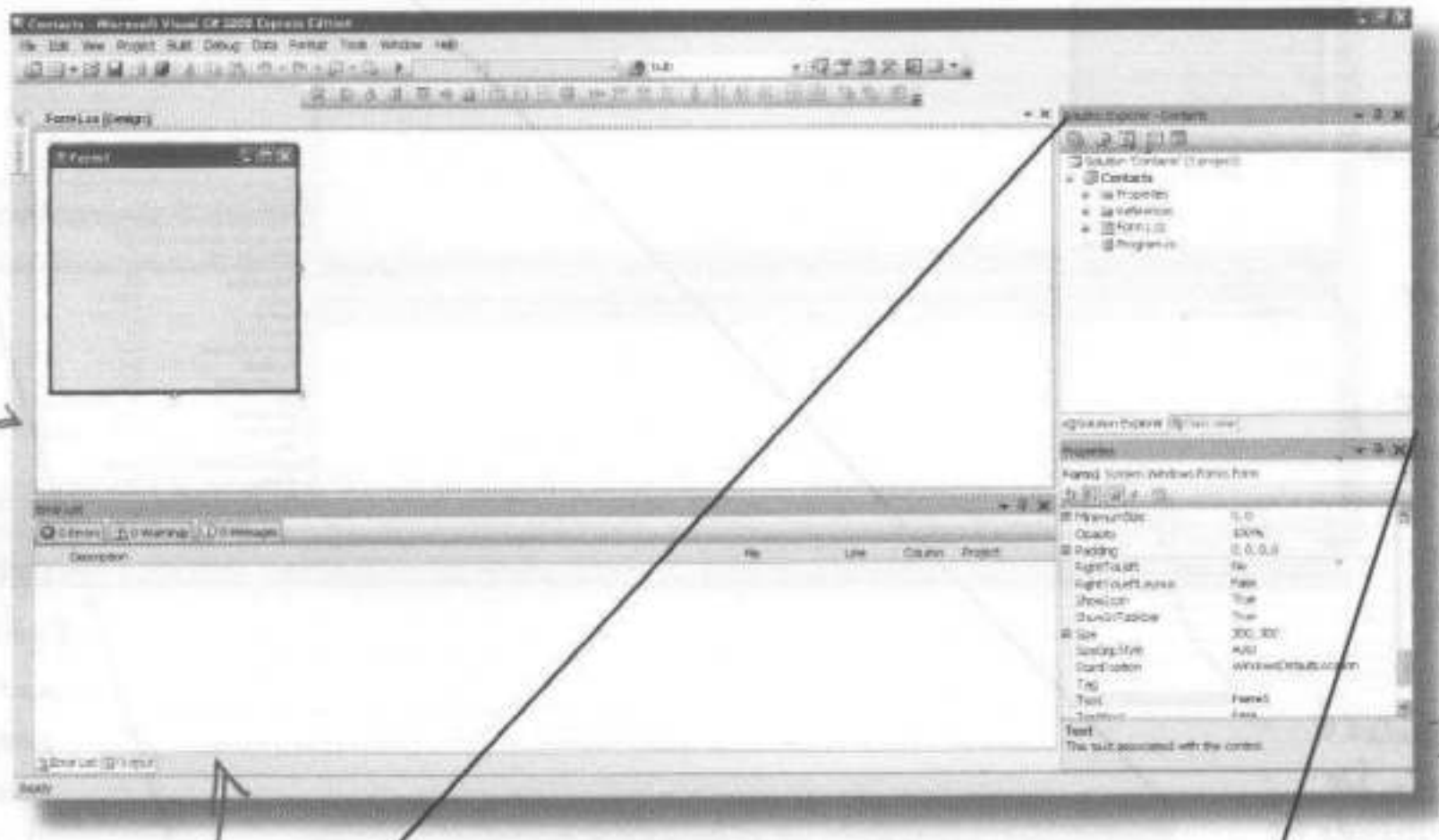
Aponte seu lápis

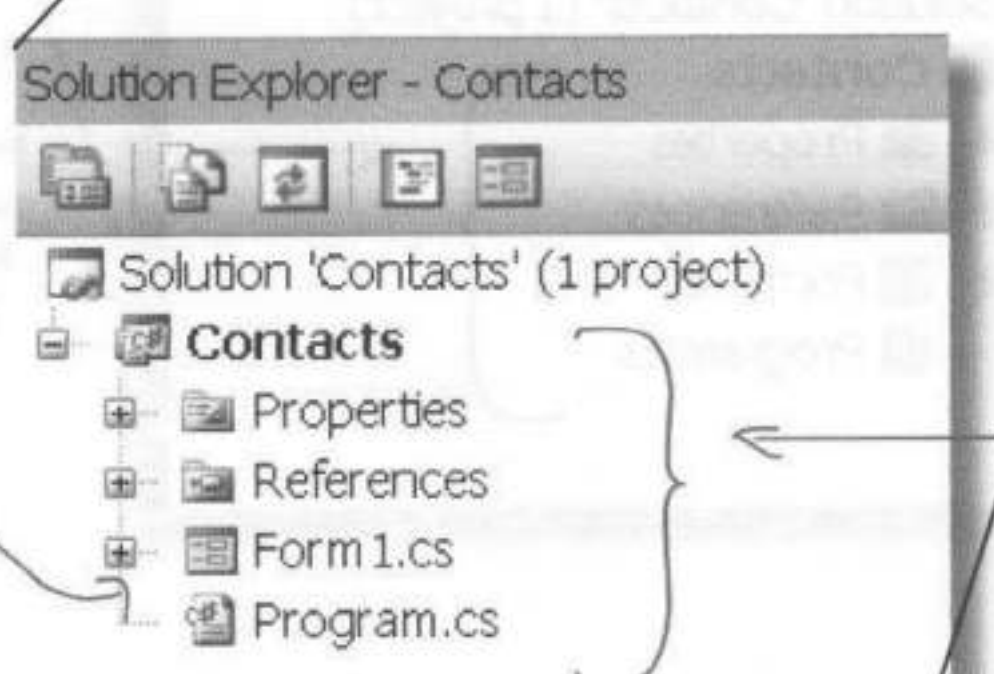
Abaixo vemos como, provavelmente, sua tela está agora. Você já deve ter percebido que a maioria destas janelas e arquivos se baseiam no que você já sabe. Preencha cada um dos espaços em branco com uma anotação, tentando descrever o que aquela parte do IDE faz. Adiantamos seu trabalho fazendo um para você.

Esta barra de ferramentas possui botões correspondentes ao que você está fazendo atualmente no IDE.

Se o seu IDE não parece exatamente com o desta figura, você pode selecionar Reset Window Layout (Reiniciar o Layout das Janelas) no menu Window (Janela).

Ampliamos esta janela abaixo para que você tenha mais espaço.





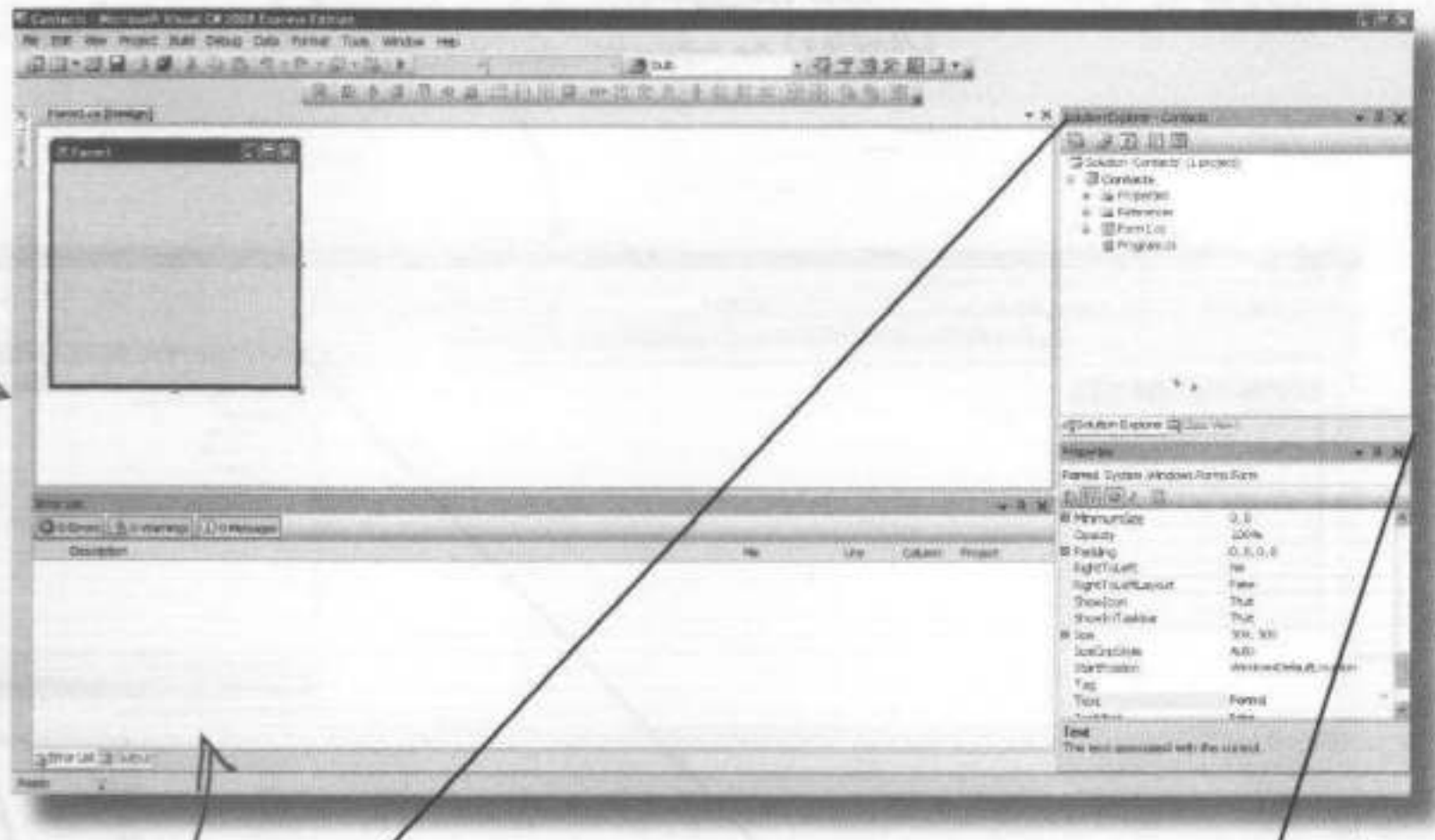
Você também pode configurar a essas janelas, selecionando Solucion Explorer, Propriedade de erro ou a partir da lista Exibir menu.

Aponte seu lápis
Solução

Preenchemos os campos com as anotações sobre as diferentes seções do IDE do Visual Studio C#. Você pode ter escrito algumas coisas diferentes, mas já deve ter percebido o básico sobre as finalidades de cada grupo de janelas do IDE.

Esta barra de ferramentas possui botões que correspondem ao que você está fazendo atualmente no IDE.

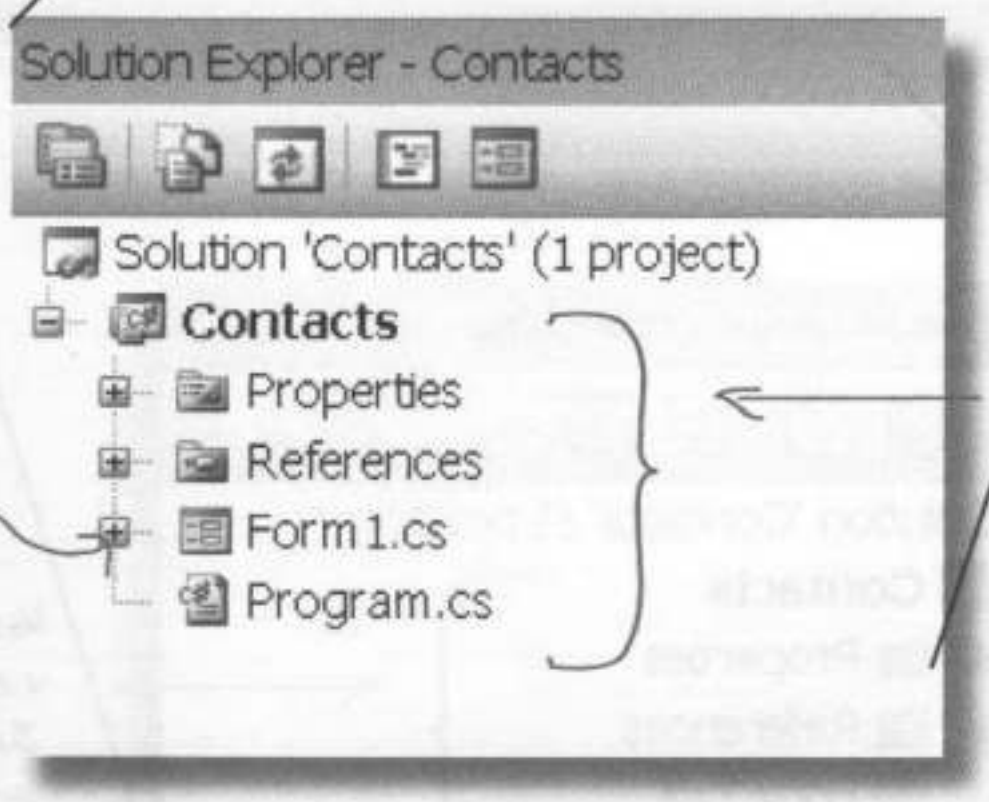
Ampliamos essa janela abaixo para que você tenha mais espaço.



Essa é a caixa de ferramentas. Ela possui vários controles visuais que você pode arrastar para seu formulário.

Este painel abaixo é para depuração. Ele mostra-lhe quando há erros em seu código.

Esta janela mostra todas as propriedades dos controles em seu formulário.



Aparecem no Navegador de Solução os arquivos Form1.cs e Program.cs que o IDE criou quando o novo projeto foi iniciado.

Você pode trocar de arquivos usando o Navegador de Solução do IDE.

não existem Perguntas Idiotas

P: Se o IDE escreve todo esse código para mim, aprender C# é apenas uma questão de aprender como usar o IDE?

R: Não. O IDE é ótimo em gerar automaticamente algum código para você, mas é só isso que ele pode fazer. Há algumas coisas em que ele é muito bom, como estabelecer bons pontos de partida e mudar propriedades de controles automaticamente em seus formulários. Mas a parte difícil da programação – descobrir e implementar o que seu programa precisa fazer – é algo que nenhum IDE pode fazer por você. Embora o IDE do Visual Studio seja um dos ambientes de desenvolvimento mais avançados, ele só pode ir até esse ponto. É você – não o IDE – que escreve o código de ação, ou o código que faz o trabalho.

P: Eu criei um novo projeto no Visual Studio, mas quando entrei na pasta "Projetos", em Meus Documentos, não o vi lá. O que acontece?

R: Em primeiro lugar, você deve estar usando o Visual Studio 2008 – no 2005 isso não acontecia. Quando você cria pela primeira vez um novo projeto no Visual Studio 2008, o IDE cria o projeto em sua pasta `Local Settings\Application Data\TemporaryProjects`. Quando você salva o projeto pela primeira vez, ele pede, através de uma janela, um novo nome de arquivo e salva-o na pasta `Meus Documentos\Visual Studio2008\Projects`. Se você tentar abrir um novo projeto ou fechar o temporário, uma janela se abrirá para perguntar se você quer salvar ou descartar o projeto temporário.

P: E se o IDE criar código que eu não queira em meu projeto?

R: Você pode modificá-lo. O IDE é feito para criar o código baseando-se na forma que o elemento que você arrastou ou adicionou é mais usado comumente. Mas, às vezes, não é exatamente isto que você quer. Tudo que o IDE faz por você – todas as linhas de código que ele cria, todos os arquivos que adiciona – pode ser alterado manualmente, editando-se os arquivos, ou através de uma interface simples de usar no IDE.

P: Tudo bem se eu baixei e instalei o Visual Studio Express? Ou eu preciso usar uma das versões do Visual Studio que não são de graça para fazer o descrito no livro?

R: Não há nada neste livro que você não possa fazer com a versão grátis do Visual Studio (que você pode baixar na página da Microsoft). As principais diferenças entre o Express e as outras edições (Professional e Team Foundation) não atrapalharão a forma de escrever em C# e de criar aplicativos totalmente funcionais e completos.

P: Posso mudar os nomes dos arquivos que o IDE cria?

R: Claro, você pode mudar qualquer aspecto de seu programa. Mas o IDE está programado para nomear seus arquivos de forma coerente. Quando você adiciona um arquivo ao seu projeto, o nome do arquivo que você escolhe afeta a forma na qual o código é gerado e o código criado incluirá esse nome. Em alguns casos, se você renomear o arquivo, ou terá de mudar outras partes em todo o código, ou terá de lidar com a diferença entre o nome do arquivo e do código dentro dele. Já que isto é um pouco incômodo, recomendamos que você não mude os nomes dos arquivos a não ser que tenha uma boa razão para isso.

P: Estou olhando para o IDE neste momento, mas minha tela não parece com a sua! Algumas janelas estão faltando e outras estão no lugar errado. O que acontece?

R: Se você clicar no comando "Reset Window Layout" (Reiniciar Layout de Janelas) no menu "Window" (janela), o IDE deve restaurar o layout padrão das janelas. Então sua tela ficará igualzinha as deste capítulo.

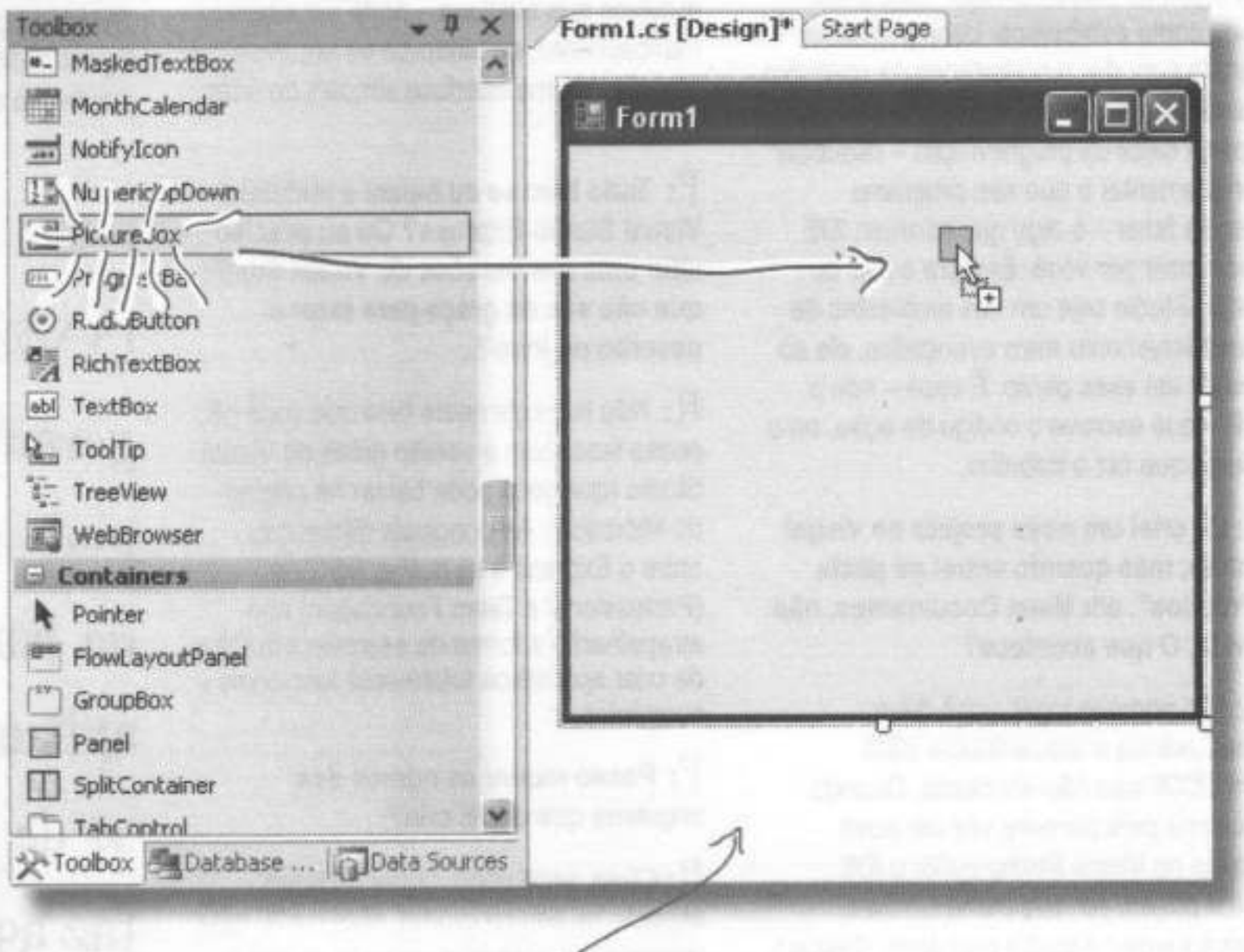
O Visual Studio gerará código que pode ser um ponto de partida para seus aplicativos. Certificar-se de que o aplicativo faz aquilo que deve fazer ainda cabe a você.

Desenvolva a interface de usuário

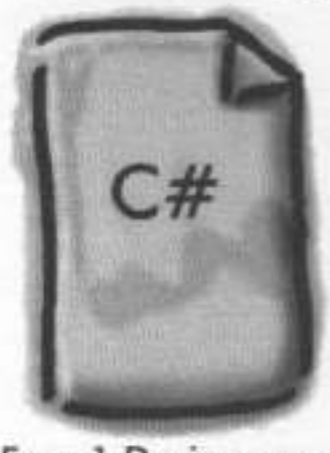
Adicionar controles e arrumar a interface de usuário é tão fácil quanto arrastar e soltar no IDE do Visual Studio. Vamos acrescentar um logo ao formulário:

1. **Utilize o controle PictureBox para acrescentar uma figura.** Clique no controle PictureBox (caixa de imagem) na Caixa de Ferramentas e arraste-o para o seu formulário. Nos bastidores, o IDE adicionou código em `Form1.Designer.cs` para um novo controle de imagens.

Se você não estiver vendo a caixa de ferramentas, tente colocar o mouse sobre a palavra "Toolbox" (caixa de ferramentas) que aparece no canto superior esquerdo do IDE. Se não estiver lá, selecione "Toolbox" do menu View (Visualizar) para fazê-la aparecer.



Toda vez que você alterar uma propriedade de controle no formulário, o código em Form1.Designer.cs também será mudado pelo IDE.

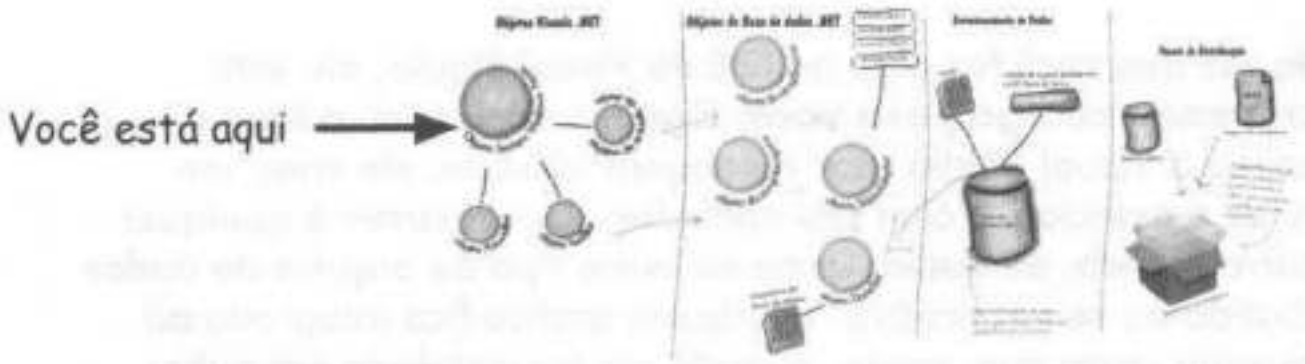


Form1.Designer.cs

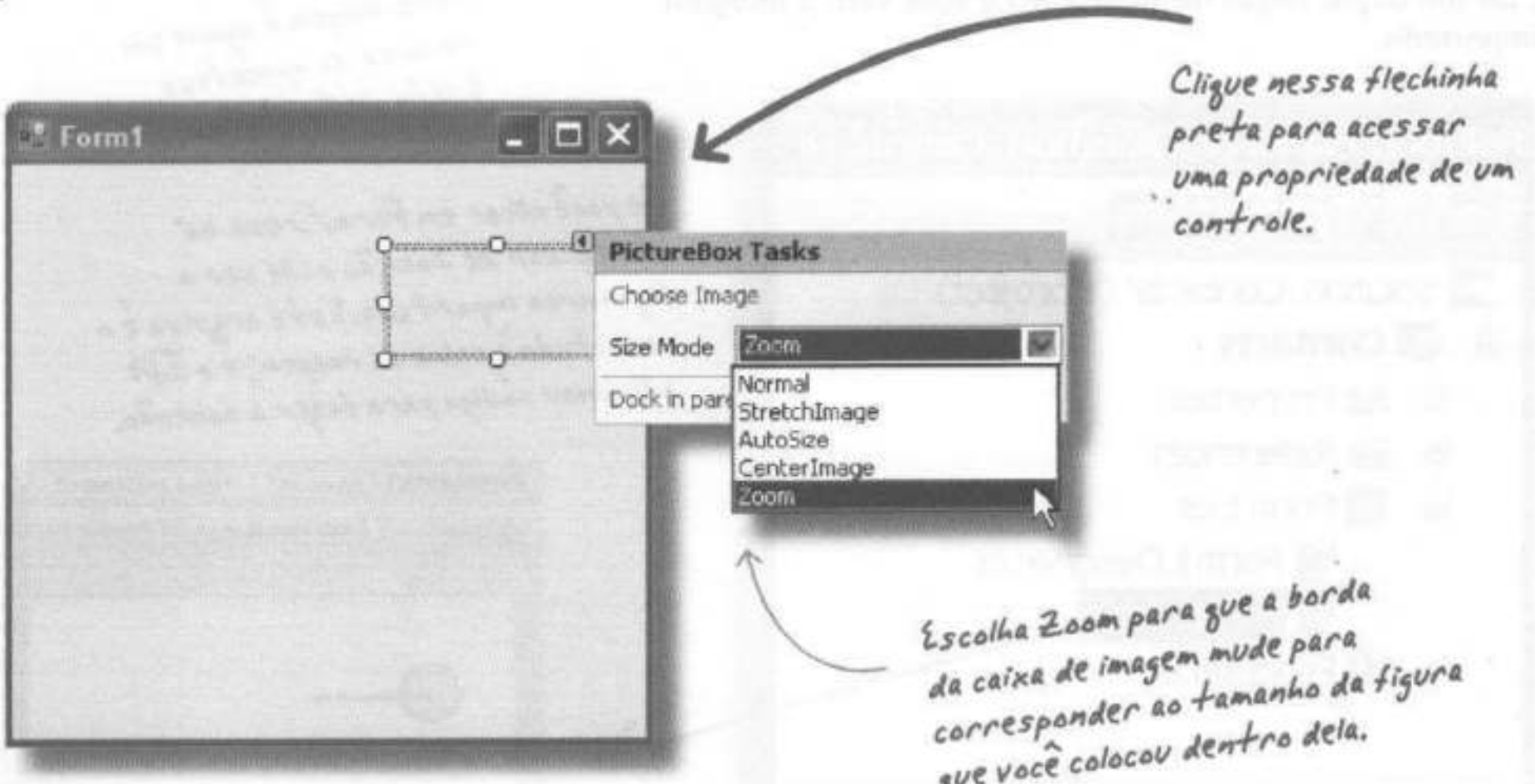
Relaxe

Tudo bem se você não for um profissional em design de interface de usuário.

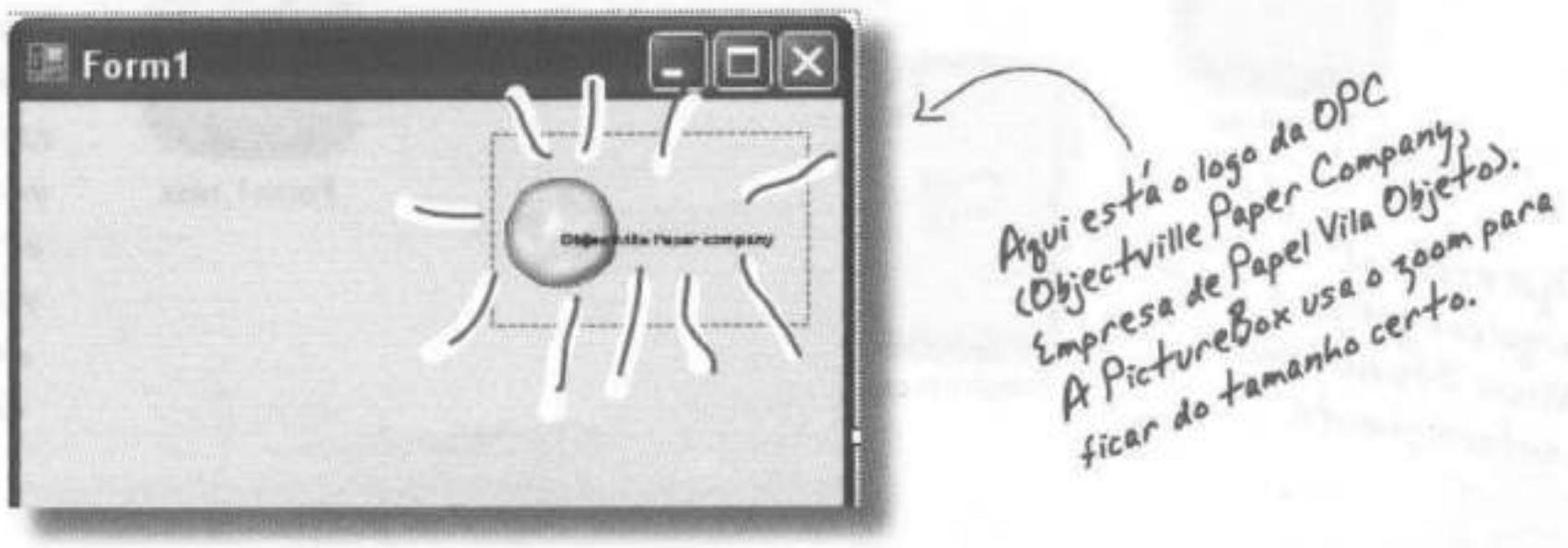
Falaremos muito mais sobre criar boas interfaces de usuário mais tarde. Por enquanto, vamos apenas colocar o logo e outros controles em seu formulário e preocupar-nos com o comportamento. Adicionaremos mais um pouco de estilo mais tarde.



2 **Coloque a caixa de imagem em modo Zoom.**
Todos os controles em seu formulário possuem propriedades ajustáveis. Clique na flechinha preta para um controle para acessá-las. Altere a propriedade Size (Tamanho) da PictureBox para "Zoom" para ver como isto funciona:



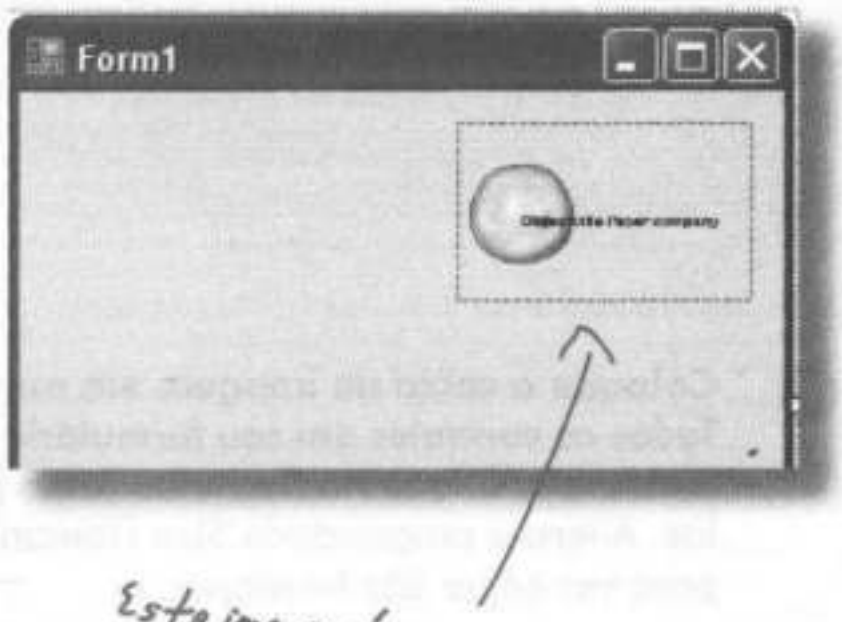
3 **Baixe o logo da Empresa de Papel Vila Objeto.**
Baixe o logo da Empresa de Papel Vila Objeto dos laboratórios do Use a Cabeça (www.altabooks.com.br) e salve-o em seu disco rígido. Então clique na seta de propriedades da PictureBox e selecione Escolher Imagem. Clique em Importar, encontre seu logo e está tudo pronto:



Visual Studio, nos bastidores

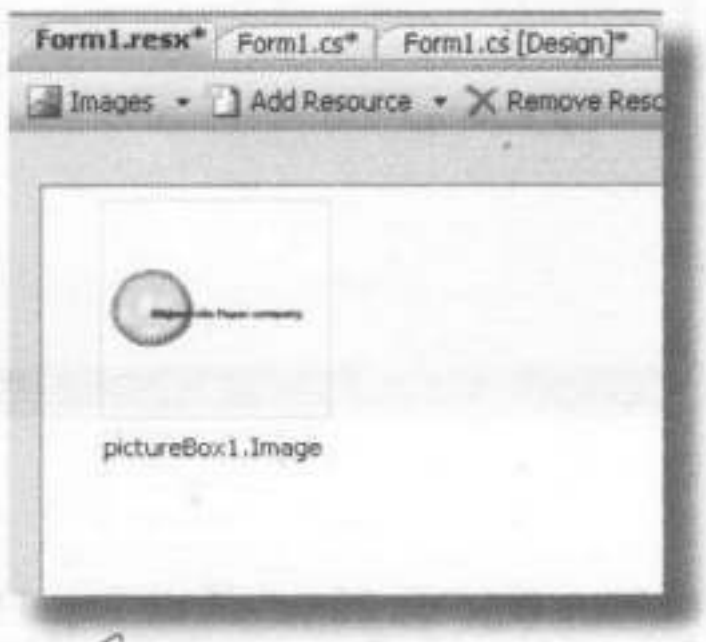
Toda vez que você faz algo no IDE do Visual Studio, ele está **escrevendo código para você**. Quando você criou o logo e mandou o Visual Studio usar a imagem baixada, ele criou um recurso e associou-o com seu aplicativo. Um **recurso** é qualquer arquivo gráfico, de áudio, ícone ou outro tipo de arquivo de dados embutido no seu aplicativo. O arquivo gráfico fica integrado ao programa, para que, então, quando ele for instalado em outro computador, o gráfico seja instalado junto com ele e a PictureBox possa usá-lo.

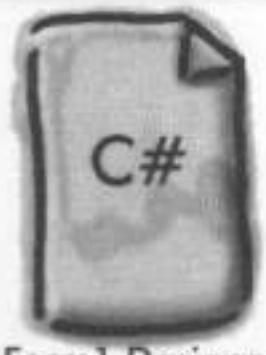
Quando você arrastou o controle PictureBox para o seu formulário, o IDE automaticamente criou um arquivo de recurso chamado Form1.resx para armazená-lo e mantê-lo em seu projeto. Dê um duplo clique neste arquivo e você verá a imagem recém-importada.



Esta imagem é agora um recurso do aplicativo Lista de Contatos.

Se você clicar em Form1.resx no Navegador de Solução pode ver a logomarca importada. Este arquivo é o conectado à caixa de imagem; e o IDE adicionou código para fazer a conexão.



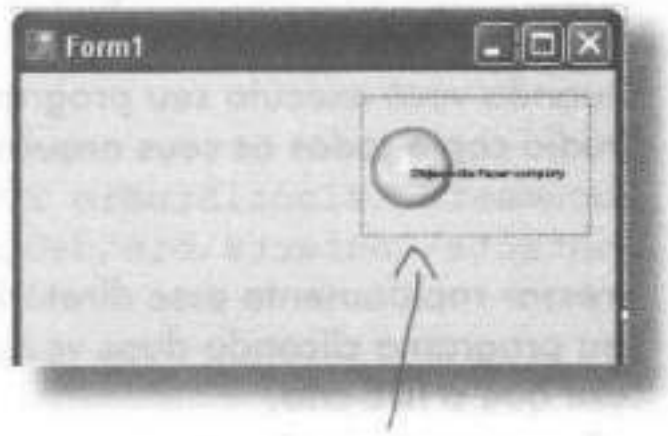


Aqui estão os arquivos que o Visual Studio criou anteriormente.

Quando você importou a imagem, o IDE criou este arquivo para você. Ele contém todos os recursos (gráficos, vídeo, áudio e outros dados armazenados) associados ao Form1.

Complete o código gerado automaticamente

O IDE cria muito código para você, mas você ainda quer ter acesso a ele e acrescentar-lhe coisas. Vamos usar o logo para mostrar uma caixa de mensagem sobre quando os usuários executarem o programa e clicar cobre o logotipo.



Certifique-se de que seu formulário aparece no IDE e clique duas vezes no controle de caixa de imagem. Você deve ver algum código semelhante ao seguinte aparecer:

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private void pictureBox1_Click(object sender, EventArgs e)
    {
        MessageBox.Show("Contact List 1.0.\nWritten by: Your Name", "About");
    }
}
```

Quando você clicou duas vezes no controle de caixa de imagem, o IDE criou este método. Ele será executado sempre que um usuário clicar no logo com o aplicativo em execução.

Este nome de método dá uma boa idéia sobre quando ele executa: quando alguém clica no controle PictureBox.

Quando você clicar duas vezes na caixa de imagem, ela abrirá este código com um cursor piscando bem aqui. Ignore qualquer janela que o IDE mostrar enquanto você digita. Ele está tentando ajudá-lo mas não precisamos disso agora.

Digite esta linha de código. Uma caixa de mensagem aparecerá com o texto que você digitou. A caixa terá o título About (Sobre).

Uma vez que você tenha digitado a linha de código, salve usando o ícone Save na barra de ferramentas do IDE ou selecionando Save no menu File. Adquira o hábito de clicar em Save All regularmente!

não existem Perguntas Idiotas

P: O que é um método?

R: Um método é apenas um bloco de código com um nome. Falaremos muito mais sobre métodos no capítulo 2.

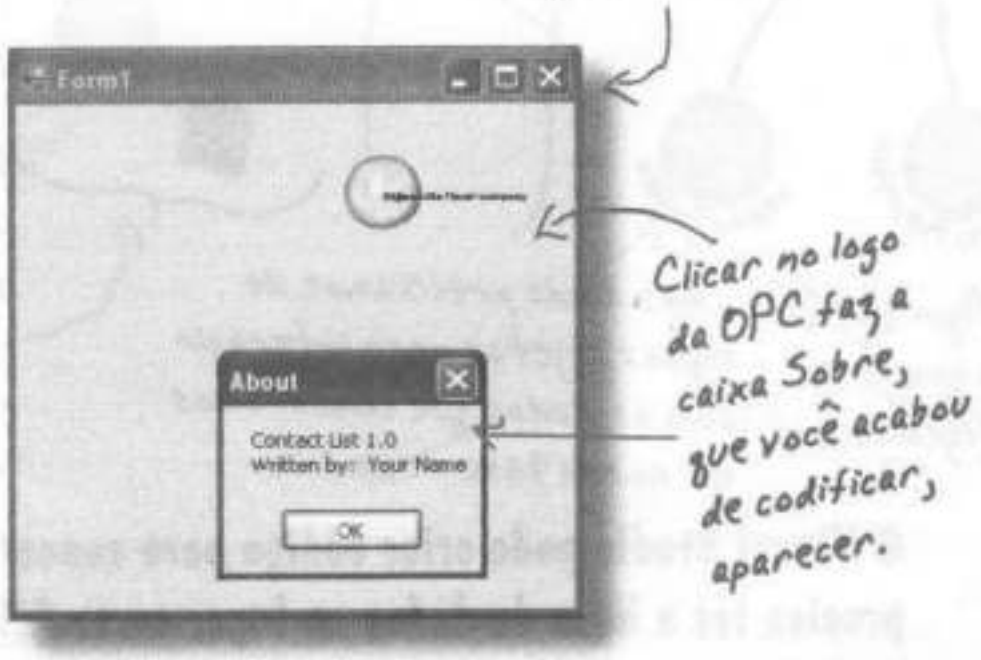
P: O que aquele \n faz?

R: Isto é uma quebra de linha. Ela fala ao C# para colocar "Contact List 1.0" na linha um, e então começar uma nova linha para "Written by:" (Escrito por).

Você já pode executar seu aplicativo

Pressione F5 no seu teclado ou clique no botão com a seta verde (▶) na barra de ferramentas para checar o que você fez até agora. (Isto se chama "Depurar", o que significa apenas executar seu programa usando o IDE). Você pode parar de depurar selecionando "Stop Debugging" (Parar a Depuração) no menu Debug (Depurar) ou clicando neste botão na barra de ferramentas: ■

Esses três botões funcionam – e você não teve que escrever nenhum código para eles.

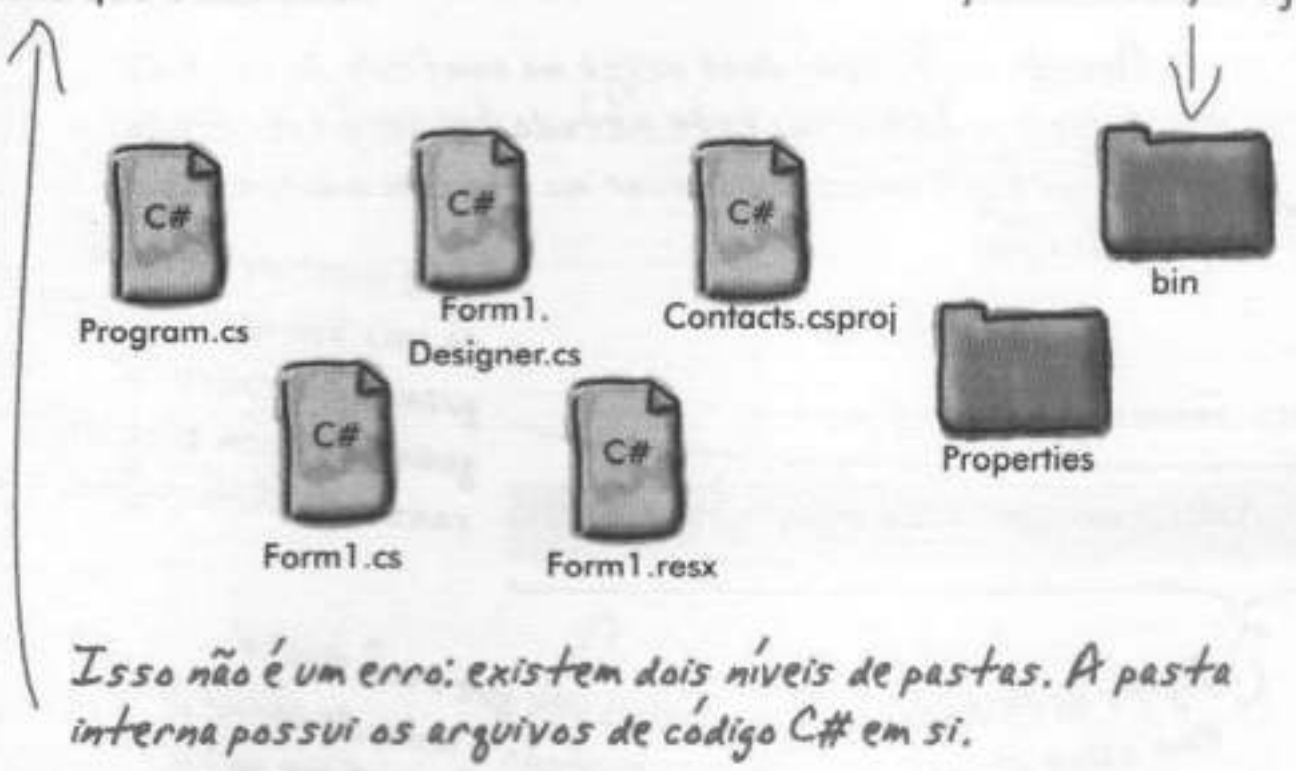


Clicar no logo da OPC faz a caixa Sobre, que você acabou de codificar, aparecer.

Onde estão meus arquivos?

Quando você executa seu programa, o Visual Studio copia todos os seus arquivos para `Meus Documentos\VisualStudio 2008\Projects\Contacts\Contacts\bin\debug`. Você pode acessar rapidamente esse diretório e executar seu programa clicando duas vezes no arquivo .exe que o IDE cria.

O C# transforma seu programa num arquivo que você pode executar, chamado de **executável**. Você o encontrará aqui, na pasta de depuração.



Isso não é um erro: existem dois níveis de pastas. A pasta interna possui os arquivos de código C# em si.

não existem
Perguntas Idiotas

P: No meu IDE, a seta verde está marcada como "Debug". Isso é ruim?

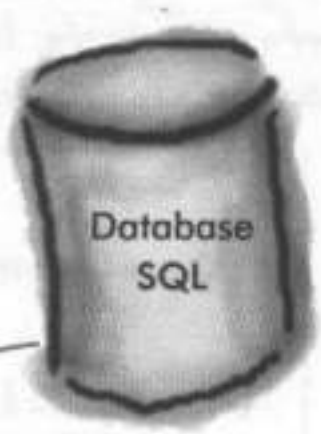
R: Não. Depurar, pelo menos para nossos propósitos agora, significa apenas executar seu aplicativo dentro do IDE. Falaremos muito mais sobre depuração adiante. Por enquanto, você pode pensar sobre isso como uma forma de executar seu programa.

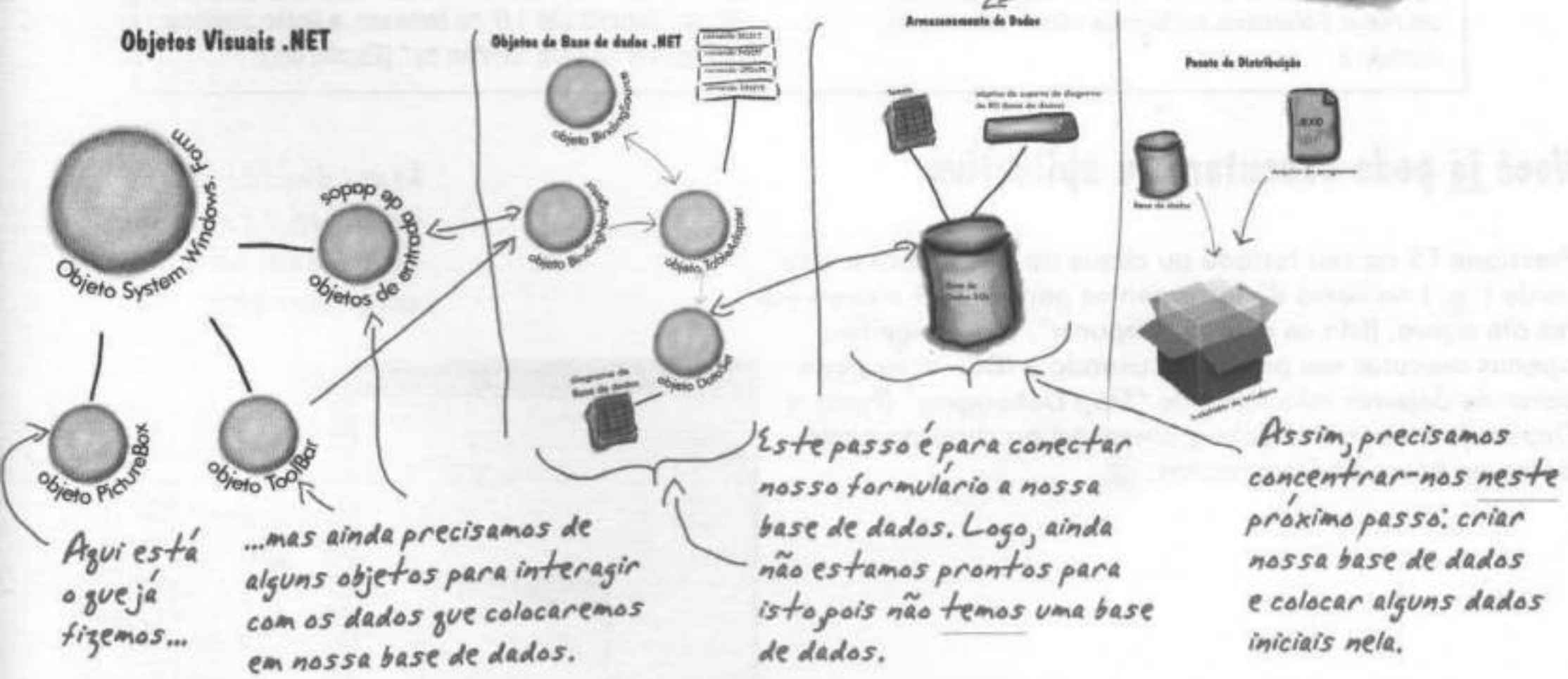
P: Eu não vejo o botão Stop Debugging em minha barra de ferramentas. O que acontece?

R: O botão Stop Debugging aparece apenas em uma barra de ferramentas especial que, por sua vez, aparece somente quando seu programa está em execução. Tente iniciar o aplicativo de novo e veja se ele aparece.

O que já fizemos até agora

Criamos um formulário e um objeto PictureBox que abre uma caixa de mensagem quando clicamos nela. A seguir, precisamos adicionar todos os outros campos do cartão, como o nome para contato e telefone.
Vamos armazenar essa informação em uma base de dados. O Visual Studio pode conectar campos diretamente a ela, ou seja, não precisamos fazer uma bagunça com um monte de código de acesso a dados (o que é bom). Mas para isto funcionar, precisamos criar nossa base para que os controles no formulário possam usar seus dados. Então vamos pular dos objetos visuais .NET direto para a seção de armazenamento de dados.





O Visual Studio pode criar código para conectar seu formulário a uma base de dados, mas você precisa ter a base de dados no lugar certo ANTES de gerar o código

Precisamos de uma base de dados para armazenar nossas informações

Antes de adicionarmos o restante dos campos no formulário, precisamos criar uma base de dados para associar-lhe. O IDE pode criar muito código para conectar nosso formulário com nossos dados, mas precisamos definir a própria base de dados primeiro.

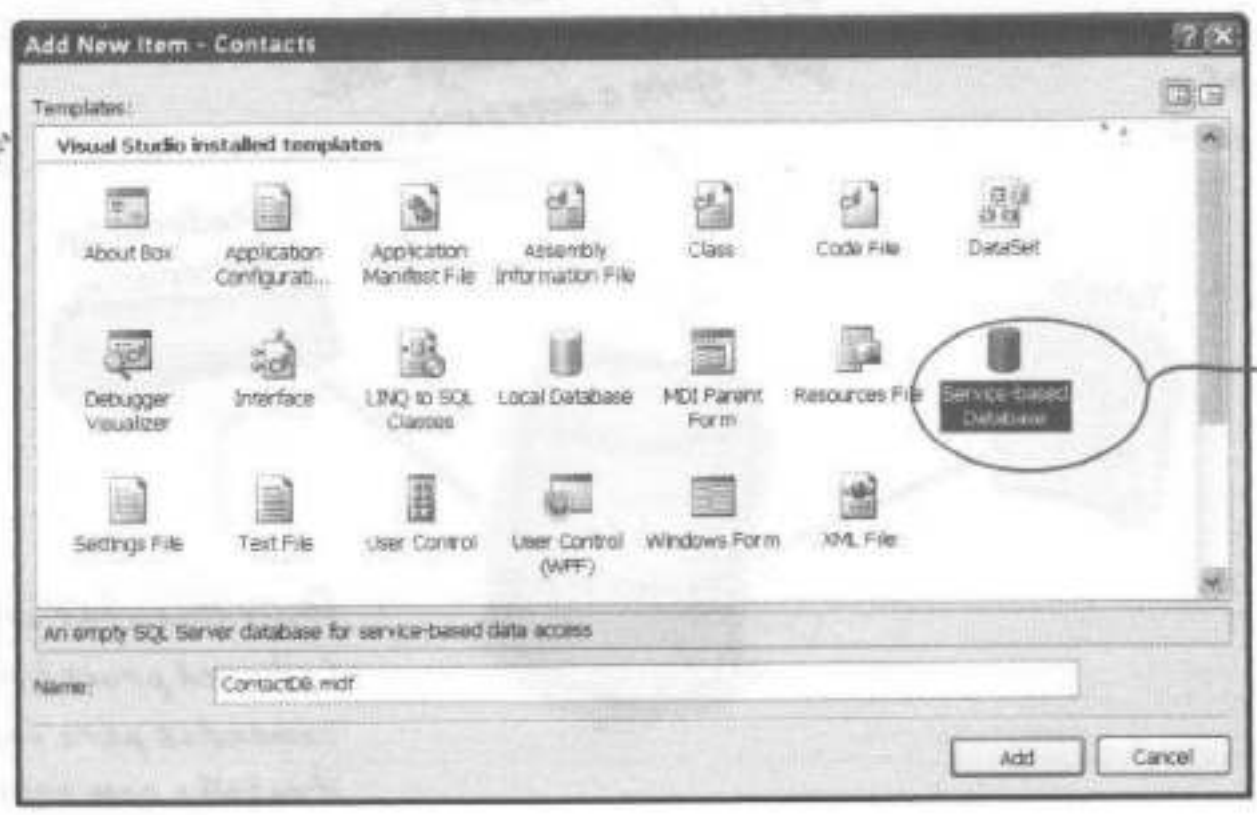
Certifique-se de que parou de depurar antes de continuar.

1. **Adicione uma nova base de dados SQL ao seu projeto.**
No Navegador de Solução, clique com o **botão direito no projeto Contacts** e escolha New Item (Novo Item). Escolha o ícone SQL Database (Base de Dados SQL) e nomeie-o ContactDB.mdf.

ContactDB.mdf

Este arquivo é nossa nova base de dados.

Escolha o ícone certo para a versão que você está usando. Escolha **SQL Database** se você está usando o Visual Studio Express 2005 e **Service-Based Database** (Base de Dados baseada em Serviços) se você está usando o 2008.



O ícone SQL Database funciona apenas se você tiver o SQL Server Express instalado. Volte para o README (Leia-me) se você não tem certeza sobre como fazer isto).

2. **Cancele o Assistente de Configuração de Fonte de Dados.**
Por enquanto, queremos pular a configuração de uma fonte de dados, então clique no botão Cancelar. Voltaremos a isto quando tivermos criado a estrutura de nossa base de dados.

3. **Examine sua base de dados no Navegador de Solução.**
Vá para o Navegador de Solução e você verá que ContactDB foi adicionado à lista de arquivos. Clique duas vezes em ContactDB.mdf e olhe no lado esquerdo de sua tela. A Toolbox mudou para Navegador de Base de Dados.

Veja bem!

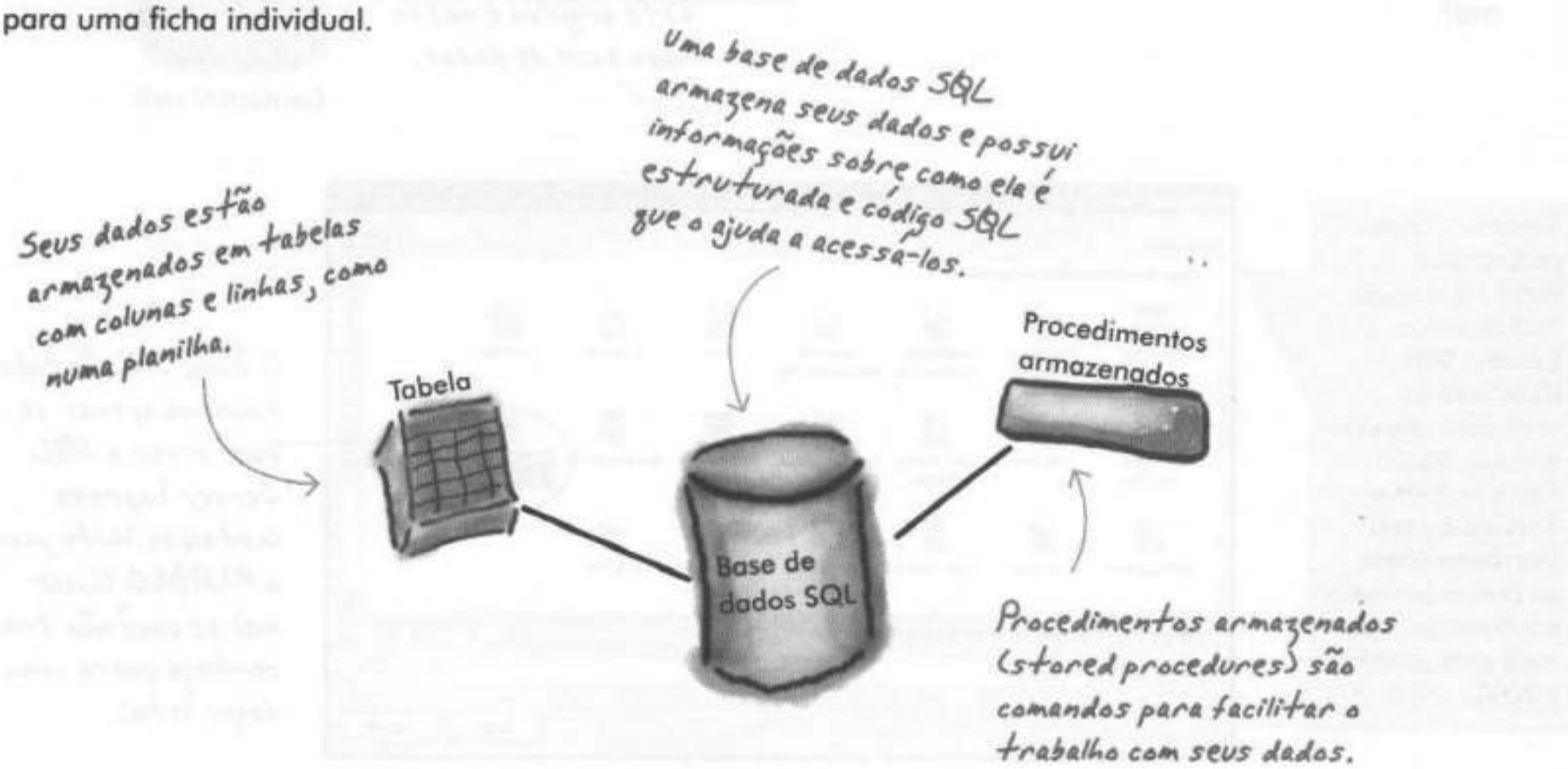
Se você não está usando a edição Express, verá "Server Explorer", em vez de "Database Explorer".

As edições Professional e Team Foundation do Visual Studio 2008 não possuem uma janela de Navegador de Solução chamada Database Explorer (explorador, ou navegador, de base de dados). Em vez disso, elas possuem uma janela Server Explorer (navegador de servidores), que faz tudo que a Database Explorer faz, mas também lhe permite explorar dados em sua rede.

O IDE criou uma base de dados

Quando você mandou o IDE adicionar uma nova base de dados SQL ao seu projeto, ele criou uma nova para você. Uma **base de dados** SQL é um sistema que armazena dados para você de uma forma organizada e inter-relacionada. O IDE tem todas as ferramentas de que você precisa para manter seus dados e bases.

Os dados na base de dados SQL ficam em tabelas. Por enquanto, você pode pensar em uma tabela como uma planilha. Ela organiza sua informação em colunas e linhas. As colunas são as categorias dos dados, como nome e telefone de contato e cada linha são os dados em si para uma ficha individual.



O SQL é uma linguagem em si mesmo

SQL significa **Linguagem Estruturada de Consulta** (Structured Query Language). É uma linguagem de programação para acessar dados em bases de dados. Possui sua própria sintaxe, palavras-chave e estrutura. O código SQL tem a forma de **comandos** e **consultas**, que acessam e recuperam os dados. Uma base de dados SQL pode possuir **procedimentos armazenados** (stored procedures), que são vários comandos e consultas SQL armazenados na base de dados e podem ser executados a qualquer momento. O IDE gerou comandos SQL e procedimentos armazenados automaticamente para você para permitir ao seu programa acessar os dados na base de dados.

ContactDB.mdf

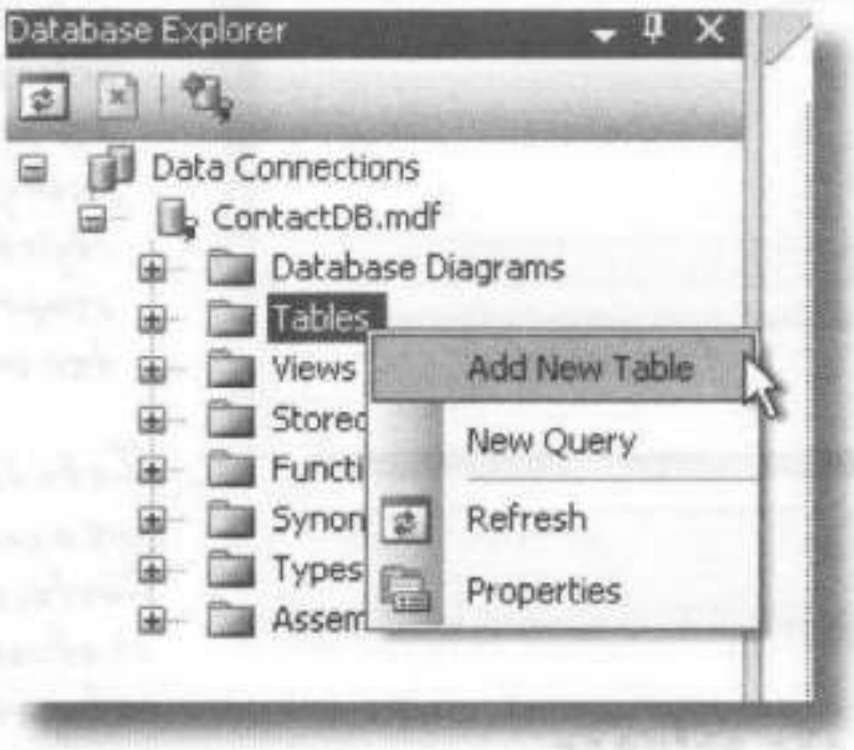
A base de dados SQL está neste arquivo. Agora vamos definir tabelas e dados para ele e tudo isto será armazenado aqui também.

[nota do marketing: podemos colocar uma referência sobre o Use a Cabeça SQL aqui?]

Criando a tabela para a Lista de Contatos

Temos uma base de dados e agora precisamos armazenar informações nela. Mas, na verdade, nossas informações tem de ir para uma tabela, estruturas usadas para guardar porções individuais dos dados. Para nosso aplicativo, vamos criar uma tabela chamada "People" (pessoas) para armazenar todas as informações para contato:

1. **Adicione uma tabela à base de dados ContactDB**
Clique com o botão direito em Tables (tabelas) no Navegador de Base de Dados e selecione Add New Table (Adicionar Nova Tabela). Será aberta uma janela onde você pode definir as colunas na tabela que acabou de criar.



Agora precisamos adicionar colunas à nossa tabela. Primeiro, vamos adicionar uma coluna chamada ContactID na tabela People, para que cada registro de Contato tenha sua própria identidade única.

2. **Adicione uma coluna ContactID à tabela People.**
Digite "ContactID" no campo Column Name (nome da coluna) e selecione Int na caixa de combinação Data Type. Certifique-se de desmarcar a caixa Allow Nulls (Permitir Nulos).

Finalmente, vamos fazer desta coluna a chave primária de nossa tabela. Selecione a coluna ContactID que acabou de criar e clique no botão Primary Key (Chave Primária). Isto informa à base de dados que cada linha terá um valor único de chave primária.

Este é o botão Primary Key. Uma chave primária ajuda sua base de dados a procurar dados rapidamente.

não existem Perguntas Idiotas

P: O que é mesmo uma coluna?

R: Uma coluna é um campo de uma tabela. Então numa tabela chamada Pessoas, você pode ter uma coluna para Primeiro Nome e outra para Sobrenome. Ela sempre terá um tipo de dado, também, como String (seqüência de caracteres), Date (data) ou Bool (booleano).

P: Por que precisamos desta coluna ContactID?

R: Ela ajuda a ter uma única identidade (ID) para cada registro na maioria das tabelas em bases de dados. Já que estamos armazenando informações de contato para pessoas individuais, decidimos criar uma coluna para isso e chamá-la de ContactID.

P: O que aquele Int em Data Type significa?

R: O tipo de dado (data type) informa à base de dados qual tipo de informação ela deve esperar de uma coluna. Int significa apenas um número inteiro. Então a coluna ContactID terá números inteiros nela.

P: Tem muitos detalhes aí. Eu deveria entender tudo isto?

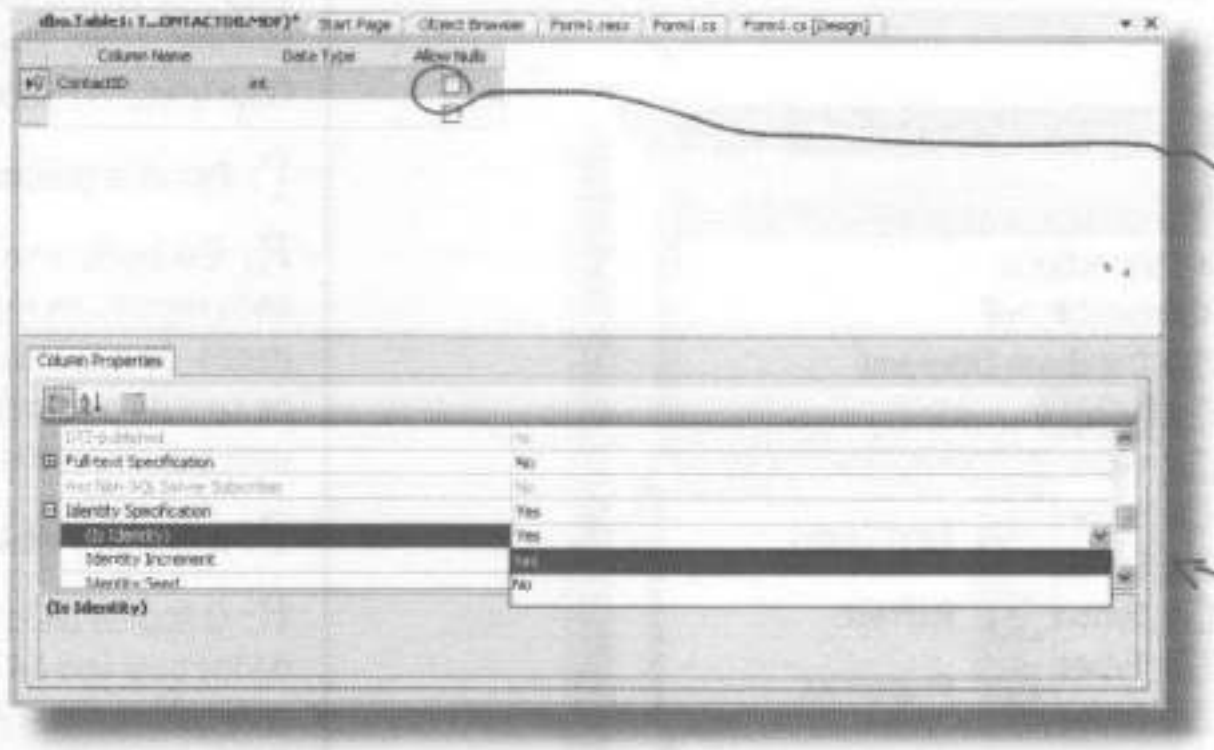
R: Não, tudo bem se você não entender tudo agora. Concentre-se nos passos básicos. Gastaremos muito mais tempo com base de dados nos últimos capítulos. Se você está curioso para saber mais agora, pode sempre ler o ***Use a Cabeça SQL*** junto com este livro.

Você está aqui

3 **Mande a base de dados gerar identidades automaticamente.**
Já que ContactID é um número que serve para a base de dados e não para nossos usuários, podemos mandar nossa base lidar com a criação e atribuição de identidades automaticamente. Desta forma, não temos que nos preocupar em escrever código para fazer isto.

Nas propriedades abaixo de sua tabela, desça a barra até Identity Specification (Especificação de Identidade), clique no botão + e selecione Yes (sim) perto da propriedade "Is Identity" (É Identidade).

Esta janela é o que você usa para definir sua tabela e os dados que ela armazenará.

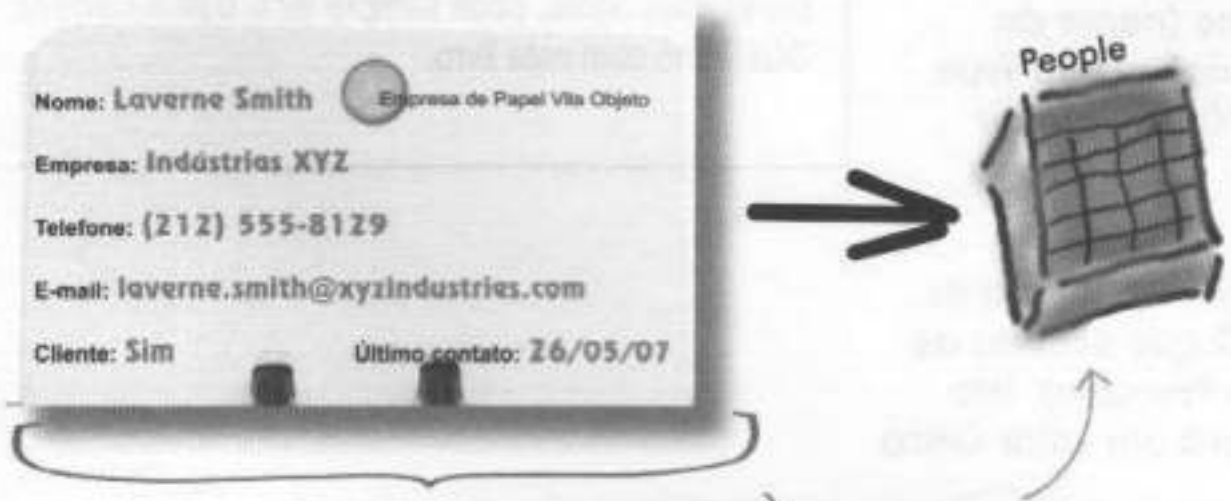


É importante que você deixe este controle desmarcado. Como a chave primária é a forma principal pela qual seu programa localizará registros, ela sempre precisa ter um valor.

Isto fará com que o campo ContactID se atualize automaticamente sempre que um novo registro for adicionado.

Os espaços em branco no cartão de contato são colunas na tabela People

Com uma chave primária para a tabela, é preciso definir todos os campos que serão mantidos na base de dados. Cada campo em nosso cartão de contato impresso deve tornar-se uma coluna na tabela People.



Para cada pessoa, queremos armazenar dados: seu nome, empresa, telefone, e-mail, se ela é um cliente da OPC e a data da sua última ligação.

Cada espaço em branco no cartão deve ser mapeado para uma coluna na tabela People.

PODER DO CÉREBRO

Que tipos de problemas poderiam resultar do armazenamento de várias linhas para a mesma pessoa?

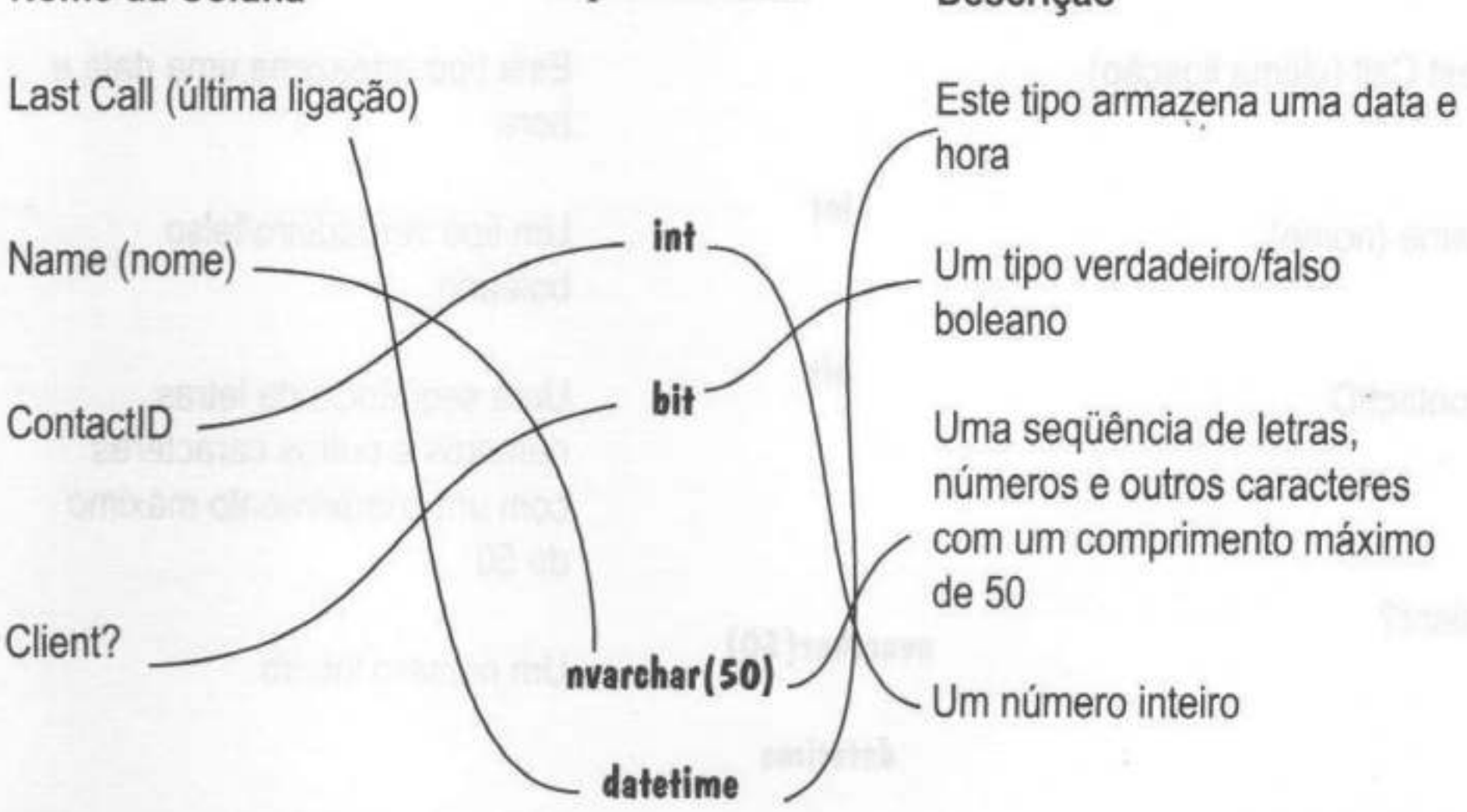
QUEM FAZ O QUE?

Agora que você criou uma tabela People e uma coluna de chave primária, precisa adicionar colunas para todos os campos de dados. Veja se consegue descobrir qual tipo de dado combina com cada uma das colunas em sua tabela e também associe o tipo de dado à sua descrição correta.

Nome da Coluna	Tipo de Dado	Descrição
Last Call (última ligação)		Este tipo armazena uma data e hora
Name (nome)	**int**	Um tipo verdadeiro/falso boleano
ContactID	**bit**	Uma seqüência de letras, números e outros caracteres com um comprimento máximo de 50
Client?	**nvarchar(50)**	Um número inteiro
	datetime	

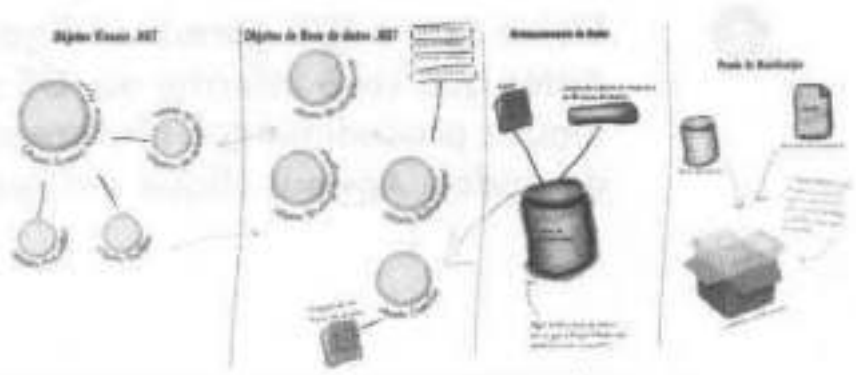
QUEM FAZ O QUE?

Agora que você criou uma tabela People e uma coluna de chave primária, precisa adicionar colunas para todos os campos de dados. Veja se consegue descobrir qual tipo de dado combina com cada uma das colunas em sua tabela e também associe o tipo de dado à sua descrição correta.

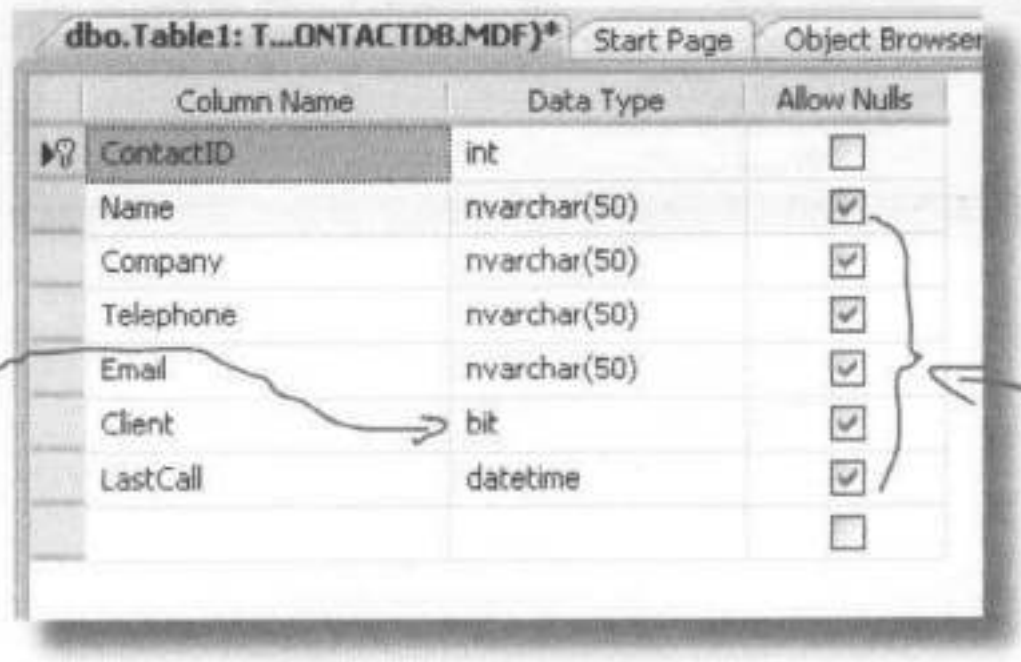


Termine de construir a tabela

Volte para onde inseriu a coluna ContactID e adicione as outras cinco colunas do cartão de contato. Aqui vemos com o que sua tabela de dados deveria se parecer quando você terminar:



Você está aqui

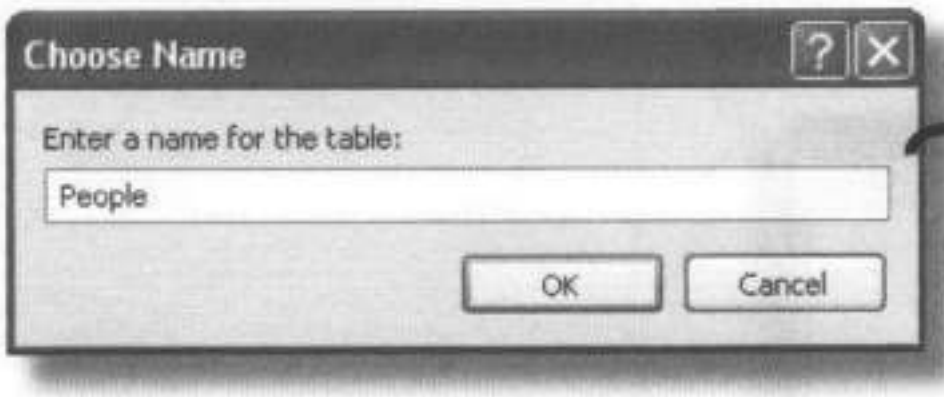


Os campos bit possuem valores Verdadeiro ou Falso e podem ser representados como uma caixa de checagem.

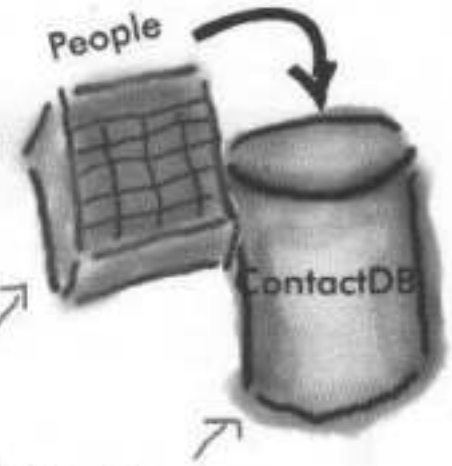
Se você cancelar a seleção em Allow Nulls, a coluna obrigatoriamente deve ter um valor.

Alguns cartões podem ter informações faltando, então deixaremos algumas colunas em branco.

Clique no botão Save na barra de ferramentas para salvar sua nova tabela. Um nome será solicitado. Chame-a de "People" e clique OK.



Temos referido-nos a essa tabela como a tabela People, mas somente agora que você lhe dará um nome oficial.

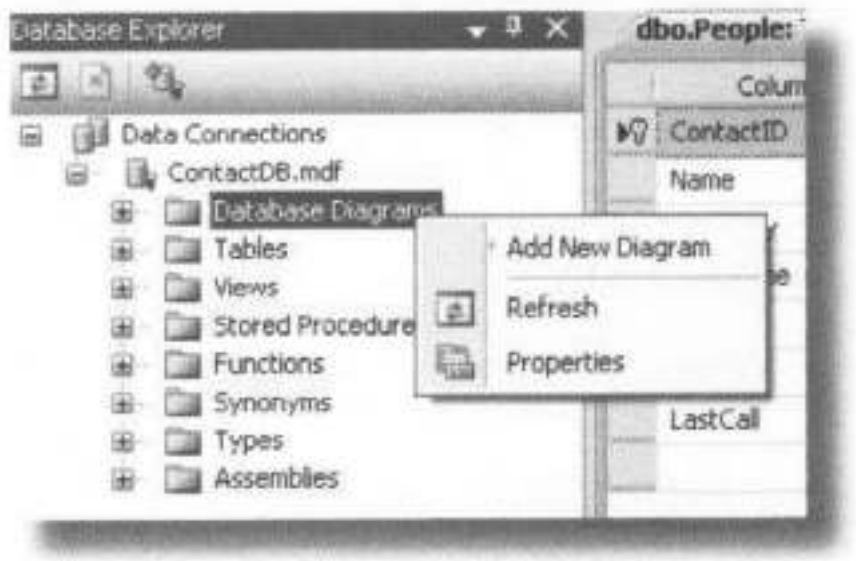


Isso cria uma tabela People, que vai para a base de dados ContactDB.

Faça um diagrama com seus dados para que seu aplicativo possa acessá-los

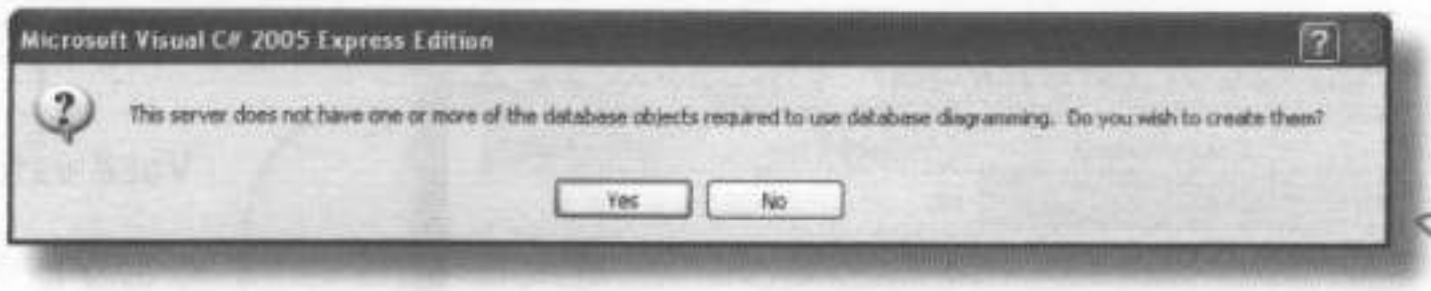
Uma vez que você tenha criado sua base de dados e tabelas, tem de fazer com que seu aplicativo saiba sobre eles. É aqui que usamos um diagrama de base de dados: uma descrição simples de sua tabela que o IDE do Visual Studio pode usar para trabalhar com os dados. Ele também permite que o IDE gere automaticamente comandos SQL para adicionar, editar e deletar linhas na tabela.

1. **Crie um novo diagrama de base de dados.**
   Vá para a janela Navegador de Base de Dados e clique com o botão direito no nó Database Diagrams (Diagramas de Base de Dados). Selecione Add New Diagram (Adicionar Novo Diagrama).



Lembre-se, estas opções estão todas dentro de ContactDB, então todas se aplicam a esta base de dados específica.

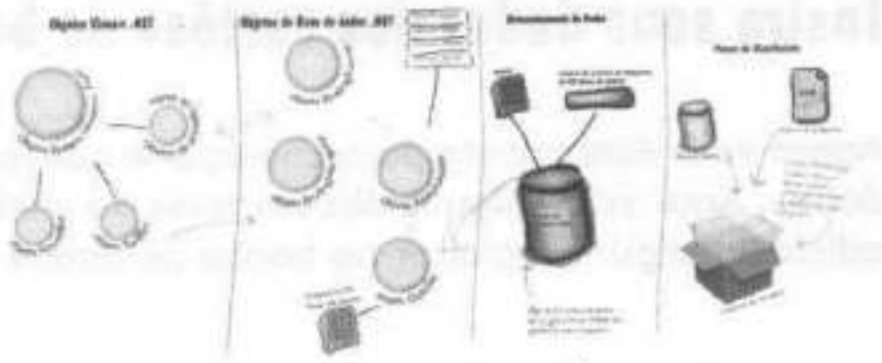
2 **Deixe que o IDE gere o código de acesso.**
Antes que você informe ao IDE sobre sua tabela específica, ele precisa criar alguns procedimentos armazenados básicos para interagir com sua base de dados. Apenas clique em Yes aqui e deixe o IDE fazer o trabalho.



O IDE gera vários procedimentos armazenados que permitem ao seu código interagir com a base de dados que você criou.

3 **Selecione as tabelas com as quais você quer trabalhar.**
Selecione a tabela People na janela que aparece e clique Add (adicionar). Agora o IDE está pronto para gerar código específico para sua tabela.

Quando você tiver bases de dados com muitas tabelas, cada uma delas aparecerá como uma linha nesta janela.

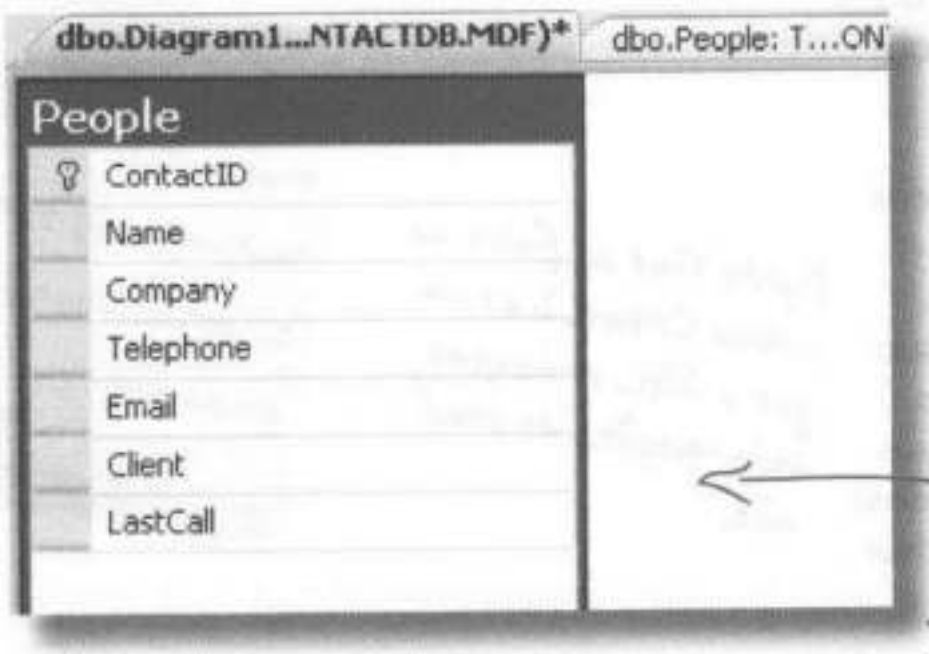


Você está aqui

4 **Chame seu diagrama de PeopleDiagram (Diagrama de Pessoas).**
Selecione File>Save Diagram. Será solicitado que você dê um nome ao seu novo diagrama de base de dados. Chame-o de PeopleDiagram e estará tudo pronto.

Se você estiver usando o Visual Studio 2005, em vez disto, selecione File>Save All.

O diagrama de base de dados é mostrado aqui visualmente. É uma representação muito simples de sua tabela.



Isto é apenas uma figura do modelo da base de dados que você acabou de fazer. Ele marca o campo ContactID como sua chave primária e lista todas as colunas na tabela.

Se você houvesse outras tabelas na base de dados que quisesse diagramar, elas apareceriam aqui também.

Um diagrama de base de dados descreve suas tabelas para o IDE do Visual Studio. O IDE então usa este diagrama para gerar código automaticamente para trabalhar com sua base de dados.

Insira seus dados dos cartões no banco de dados

Agora você está pronto para começar a adicionar os cartões no banco de dados. Aqui estão alguns dos contatos do chefe – vamos usá-los para começar a adicionar alguns registros no banco de dados.

1. Expanda Tables e então clique com o botão direito na Tabela People no Navegador de Base de Dados (Database Explorer ou Server Explorer) e selecione Show Table Data (Mostrar Dados da Tabela).

Seu trabalho é colocar os dados de todos esses seis cartões na tabela People.

2. Uma vez que você tenha visto as linhas de grade da tabela na janela principal, vá em frente e adicione todos os dados abaixo. (Você verá todos os valores NULL - nulos - primeiro – apenas digite sobre eles quando você adicionar sua primeira linha. Ignore os pontos de exclamação que aparecem juntos aos dados). Você não precisa preencher a coluna ContactID. Isto é feito automaticamente.

Digite True ou False na coluna Client. É assim que o SQL armazena informações de sim/não.

3. Uma vez que você tenha acrescentado todos os seis registros, selecione Save All no menu File novamente. Isto deve salvar os registros na base de dados.

Save All manda o IDE salvar tudo em seu aplicativo. Ele é diferente de Save, que apenas salva o arquivo no qual você está trabalhando.

não existem Perguntas Idiotas

P: Então o que aconteceu com os dados depois que eu os acrescentei? Para onde eles foram?

R: O IDE armazenou automaticamente os dados que você acrescentou na tabela People em sua base de dados. A tabela, suas colunas, os tipos de dados e todos os dados dentro dela estão todos guardados num arquivo do SQL Server Express, ContactDB.mdf. Esse arquivo é mantido como parte de seu projeto e o IDE o atualiza apenas como o faz com seus arquivos de dados, quando você os altera.

P: Certo, eu acrescentei esses seis registros. Eles serão parte do meu programa para sempre?

R: Sim, eles são tão parte de seu programa quanto o código que você escreveu e o formulário que está criando. A diferença é que, em vez de ser compilado como um programa executável, o arquivo ContactDB.mdf é copiado e armazenado junto com o executável. Quando seu aplicativo precisar acessar os dados, ele lê e escreve em ContactDB.mdf, no diretório de saída do programa.

Este arquivo é na prática uma base de dados SQL e seu programa pode usá-lo com o código que o IDE criou para você.

ContactDB.mdf

Conecte seu formulário à sua base de dados usando uma fonte de dados

Estamos finalmente prontos para desenvolver os objetos da base de dados .NET que nosso formulário usará para se comunicar com sua base de dados. Precisamos de uma **fonte de dados,** que é realmente apenas um grupo de comandos SQL que seu programa usará para se comunicar com a base de dados ContactDB.

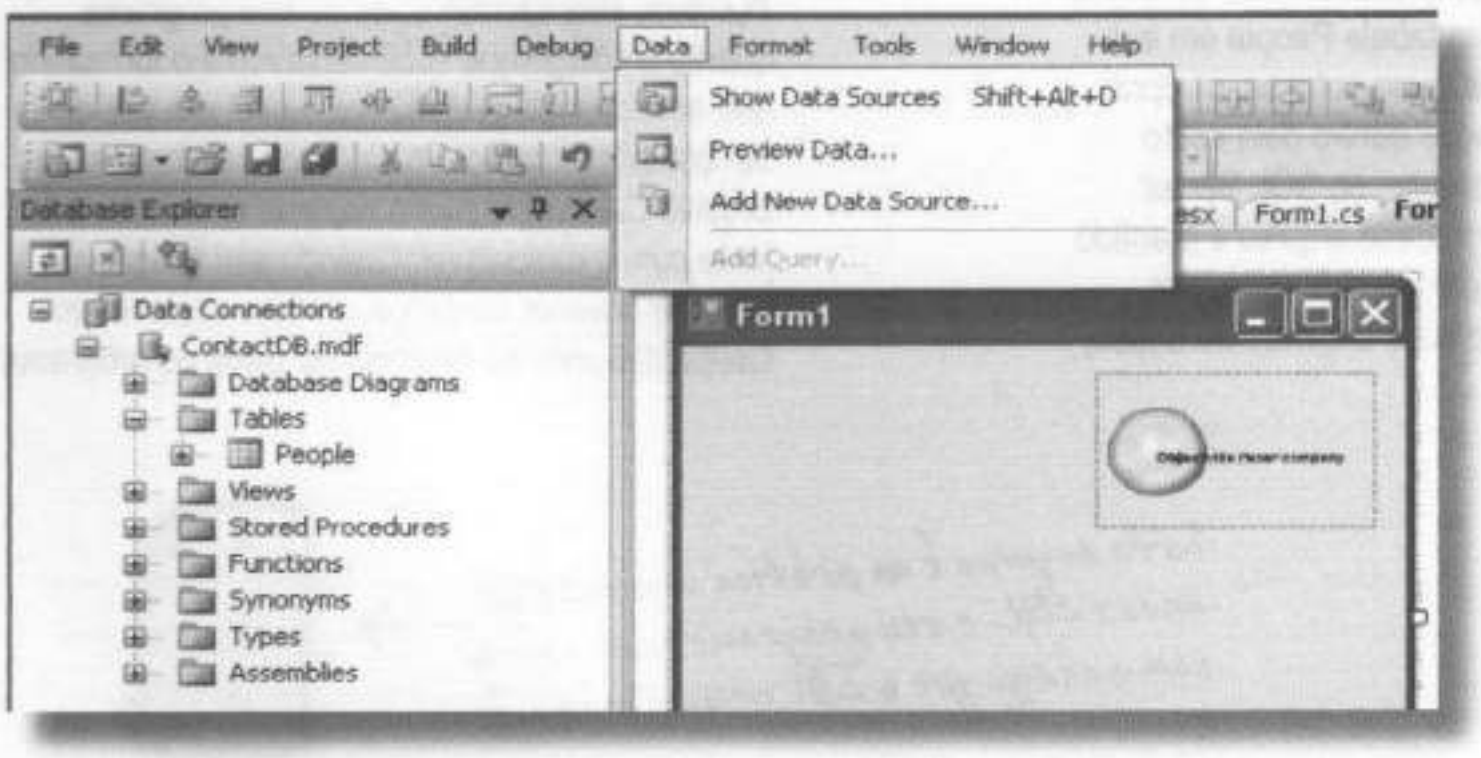
Você precisa fechar tanto a tabela de dados quanto o diagrama para voltar ao seu formulário.

1. **Volte para o formulário do seu aplicativo.**
   Feche nossa tabela People e o diagrama da base de dados ContactDB. Agora a aba Form1.cs [Design] deve estar visível.

	ContactID	Name	Company	Telephone	Email	Client	LastCall
	1	Lloyd Jones	Black Box Inc	718555638	ljones@blackbox...	True	5/26/2007 12:0...
	2	Lucinda Ericson	Ericson Events	2125559523	Lucy@ericsonev...	False	5/17/2007 12:0 ...
	3	Liz Nelson	JTP	4195552578	liznelson@jtp.org	True	3/4/2006 12.00:...
	4	Matt Franks	XYZ Industries	2125558125	matt.franks@xy...	True	5/26/2007 12:0 ...
	5	Sarah Kalter	Kalter, Riddle, a...	6145555641	sarah.kalter@K...	False	12/10/2006 12:...
	6	Laverne Smith	XYX Industries	2125558129	Laverne.Smith...	True	5/26/2007 12:0...
▶*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

2. **Adicione uma nova fonte de dados ao seu aplicativo.**
   Isso já deve ser fácil neste ponto. Clique no menu Data (Dados) e então selecione Add New Data Source (Adicionar Nova Fonte de Dados)... na caixa de seleção.

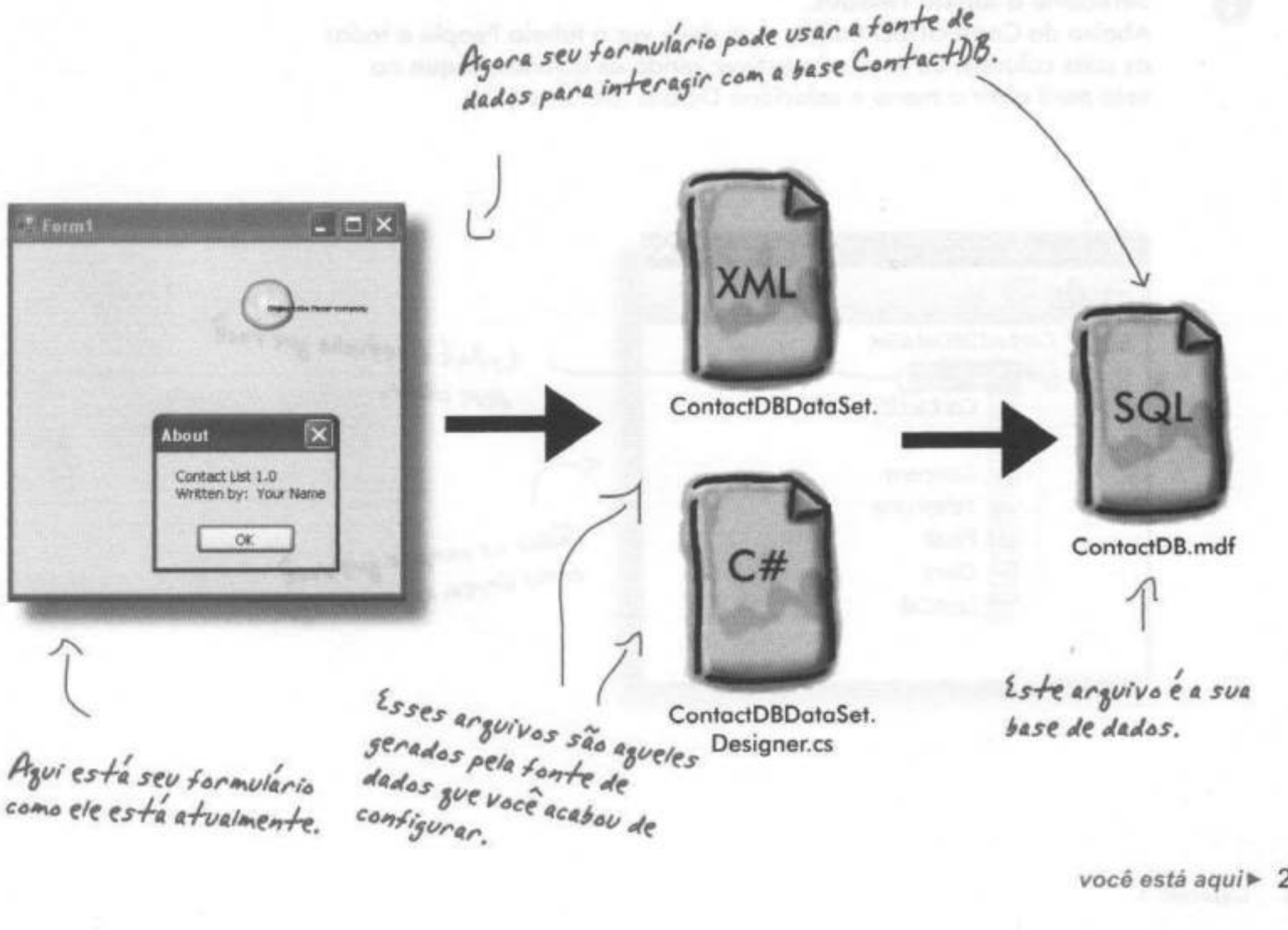
A fonte de dados que você está criando lidará com todas as interações entre seu formulário e sua base de dados.



3 **Configure sua nova fonte de dados.**
Agora você precisa configurar uma fonte de dados para usar sua base ContactDB. Eis o que você tem de fazer:

- ★ selecione Database e clique no botão Next (próximo);
- ★ clique em Next na tela "Choose your Data Connection" (Escolha sua Conexão de Dados).
- ★ Na tela "Save the Connection" (Salvar a Conexão) que segue certifique-se de que a caixa de checagem Save esteja marcada e clique em Next.
- ★ Na tela "Choose Your Objects" (Escolha seus Objetos), clique na caixa de checagem Table (Tabela).
- ★ No campo Dataset Name (Nome do Conjunto de dados), certifique-se de que esteja escrito "ContactDBDataSet" e clique em Finish (Terminar).

Estes passos conectam sua nova fonte de dados à tabela People na base ContactDB.



Adicione controles vinculados a dados em seu formulário

Agora podemos voltar ao nosso formulário e adicionar mais alguns controles. Mas estes não são controles quaisquer, são controles vinculados (bound, em inglês) à nossa base de dados e às colunas na tabela People. Isso significa que uma alteração nos dados em um dos controles no formulário automaticamente altera os dados na respectiva coluna na base de dados.

Deu um pouco de trabalho, mas agora voltamos para criar objetos de formulário que interajam com nosso armazenamento de dados.

Aqui vemos como criar diversos controles vinculados à base de dados:

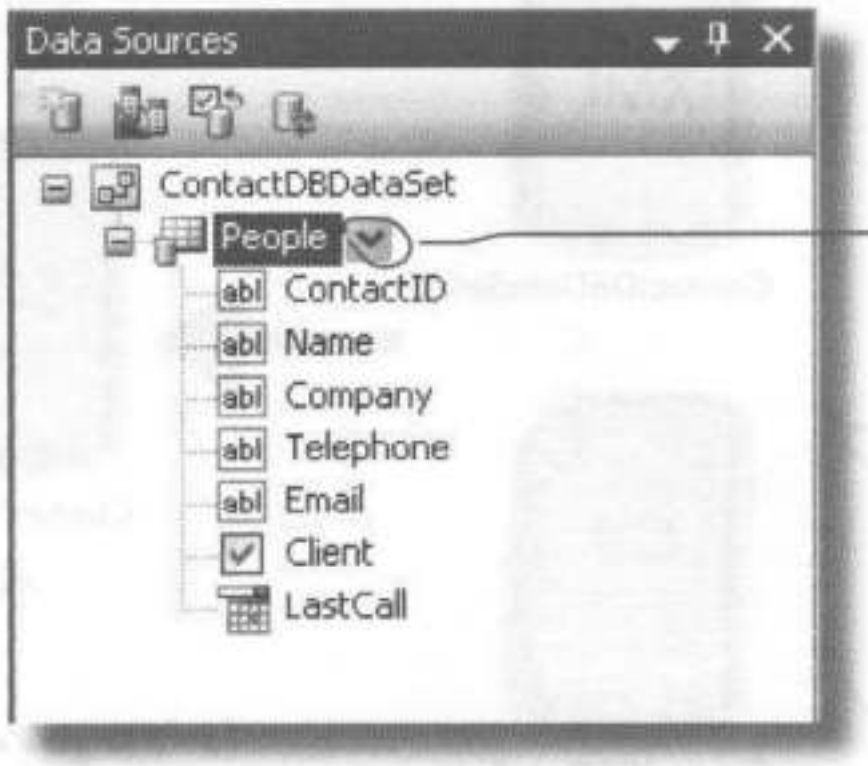
1. **Selecione a fonte de dados que você quer usar.**
   Selecione a aba Show Data Sources (Mostrar Fontes de Dados) da barra Data. Isso abrirá a janela Data Sources, mostrando as fontes que você configurou para seu aplicativo.

Se você não estiver vendo essa aba, selecione Show Data Sources, no menu Data.

Você também pode procurar e clicar na aba Data Sources na parte inferior de sua janela Navegador de Base de Dados.

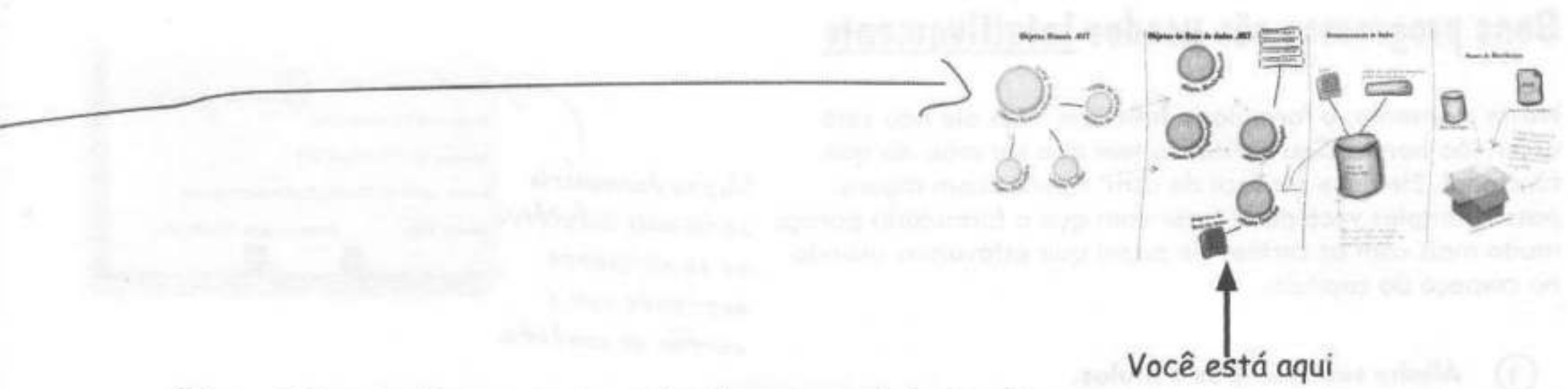
Essa janela mostra todas as fontes de dados. Temos apenas uma criada, mas você poderia ter mais para tabelas ou bases de dados diferentes.

2. **Selecione a tabela Pessoas.**
   Abaixo da ContactDBDataSet, você deve ver a tabela People e todas as suas colunas. Se você não estiver vendo as colunas, clique na seta para abrir o menu e selecione Details (Detalhes).



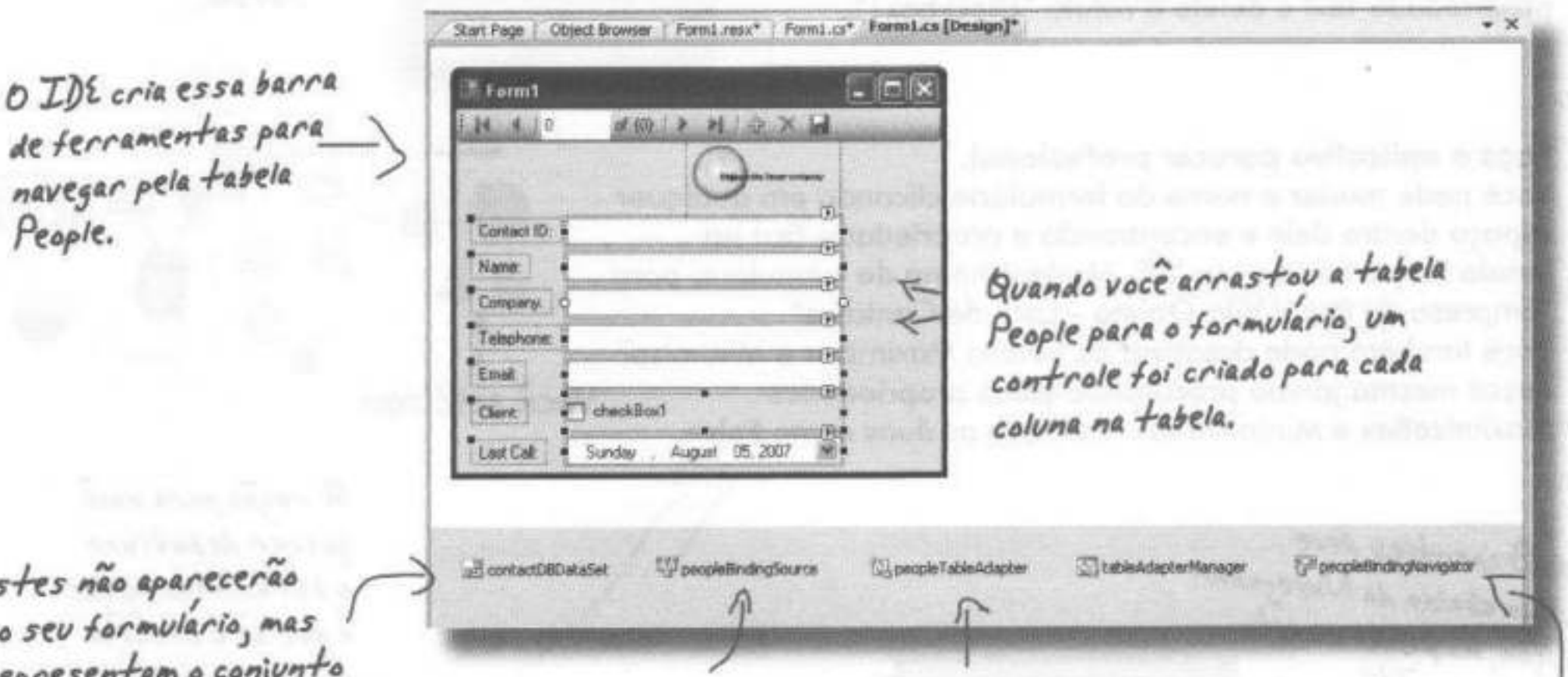
Esta é a setinha que você deve clicar.

Todos os campos que você criou devem aparecer aqui.



3 **Crie controles que tenham vínculos com a tabela People**
Arraste a tabela People até seu formulário. Você deve ver controles aparecerem para cada coluna em sua base de dados. Não se preocupe muito sobre a aparência deles agora - apenas certifique-se de que todos eles apareceram no formulário.

Se, acidentalmente, você clicar fora do formulário no qual está trabalhando, pode sempre voltar a ele selecionando na aba "Form1.cs [Design]", ou abrindo o Form1.cs no Navegador de Solução.



Bons programas são usados intuitivamente

Neste momento, o formulário funciona. Mas ele não está assim tão bonito. Seu aplicativo tem que ser mais do que funcional. Ele deve ser fácil de usar. Apenas com alguns passos simples você pode fazer com que o formulário pareça muito mais com os cartões de papel que estávamos usando no começo do capítulo.

Nosso formulário seria mais intuitivo se se parecesse bastante com o cartão de contato.

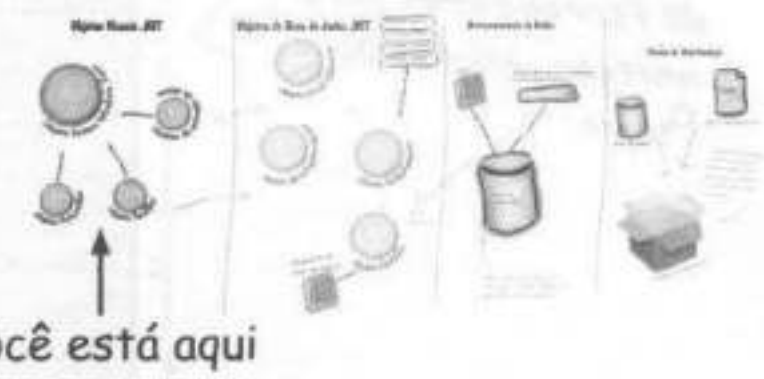
1 **Alinhe seus campos e títulos.**
Alinhe seus campos e títulos pelo canto esquerdo do formulário, para que ele se pareça com outros aplicativos e isso deixe seus usuários mais à vontade para usá-lo.

Linhas azuis aparecerão no formulário conforme você arrasta os controles em volta. Eles estão lá para ajudá-lo a alinhar os campos.

2 **Mude a Propriedade Text na caixa de seleção Client.**
Quando você arrastou primeiro os campos para o formulário, a sua caixa de seleção Client tinha um nome à direita que precisa ser apagado. Logo abaixo do Navegador de Solução, você verá a janela de propriedades. Arraste a barra para baixo até a propriedade Text e delete o rótulo "checkbox1".

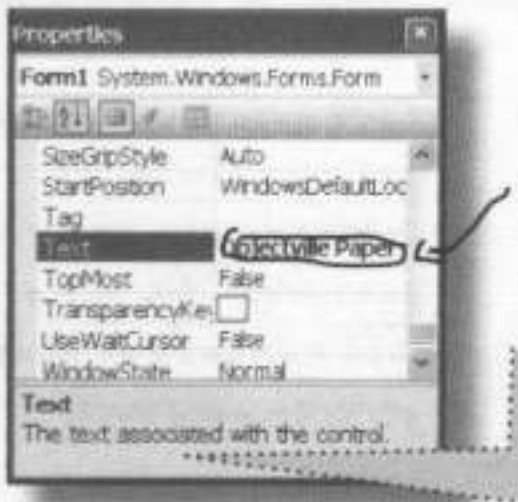
Delete esta palavra para apagar este rótulo.

3 **Faça o aplicativo parecer profissional.**
Você pode mudar o nome do formulário clicando em qualquer espaço dentro dele e encontrando a propriedade Text na janela Properties de seu IDE. Mude o nome do formulário para "Empresa de Papel Vila Objeto - Lista de Contatos".
Você também pode desativar os botões Maximizar e Minimizar nessa mesma janela procurando pelas propriedades MaximizeBox e MinimizeBox. Coloque as duas como False.



Você está aqui

A janela Properties deve estar logo abaixo do Navegador de Solução, no painel direito inferior de seu IDE.



A propriedade Text controla o início do texto na barra de título de seu formulário.

A razão para você querer desativar o botão Maximizar é que se o usuário clicasse nele seu formulário não mudaria a posição dos controles, ficando estranho.

Se você não tiver uma janela Properties, pode ativá-la selecionando-a no menu View (Visualizar).

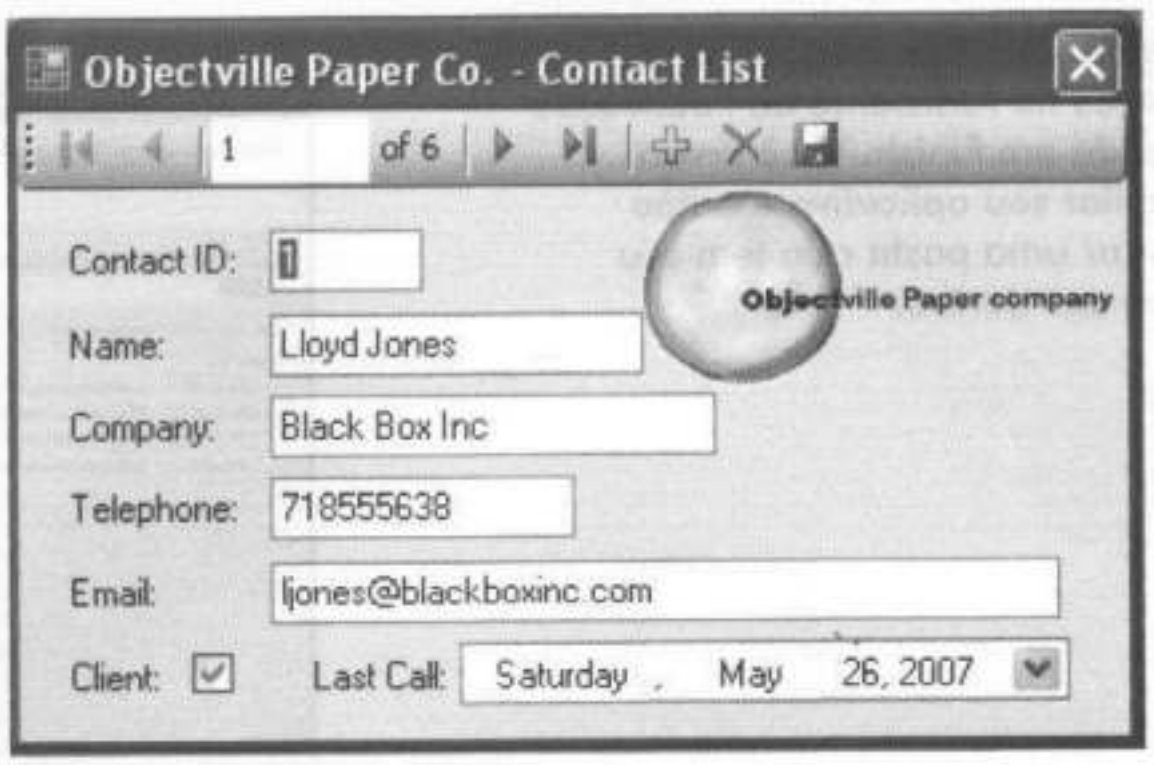
Um bom aplicativo não apenas funciona, mas é fácil de usar. É sempre uma boa idéia certificar-se de que ele se comporta como um usuário comum esperaria.

Teste drive

Tudo bem, só mais uma coisa pra fazer... executar seu programa e certificar-se de que ele funciona como o planejado! Faça como fez antes – pressione F5 em seu teclado, ou clique na seta verde ▶ na barra de ferramentas (ou escolha "Run" - executar - no menu Debug).
Você pode executar seus programas a qualquer hora, mesmo quando eles não estiverem prontos – mas se houver algum erro no código, o IDE informará e o impedirá de continuar.

Clique no X no canto da caixa para parar o programa para que você possa seguir para o próximo passo.

Estes controles permitem navegar pelos diferentes registros na base de dados.



Passaremos mais tempo falando disso no próximo capítulo.

O IDE primeiro compila, então executa.

Quando você executar seu programa no IDE, na verdade, ele faz duas coisas. Primeiro ele **compila** seu programa, depois ele o **executa**. Isso envolve algumas partes distintas. O código é compilado, ou seja, transformado num arquivo executável. Então ele coloca o código compilado, junto com quaisquer recursos e outros arquivos, num subdiretório dentro da pasta bin.

Neste caso, você encontrará o executável e o arquivo de base de dados SQL em bin/debug. Já que ele copia a base de dados todas as vezes, qualquer mudança feita será perdida na próxima vez que executar dentro do IDE. Mas se você ativar o executável do Windows, ele salvará seus dados – até que você compile de novo, ponto no qual o IDE sobrescreverá a base de dados SQL com uma nova cópia com os dados incluídos dentro do Navegador de Base de Dados.

Compilar seu programa sobrescreve os dados em sua base de dados.

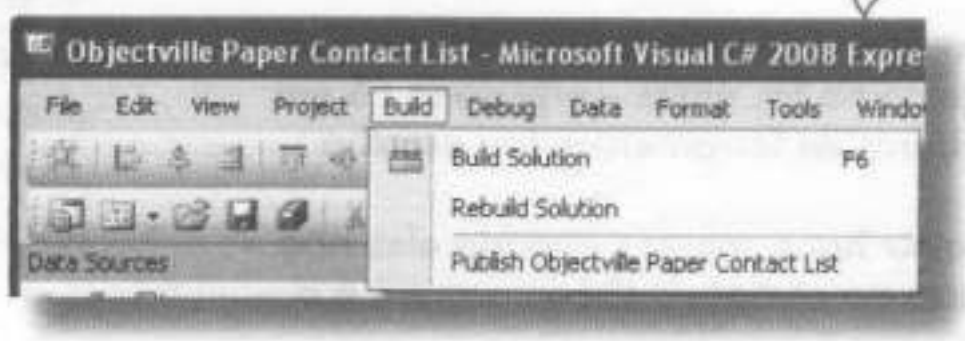
Como transformar o SEU aplicativo num aplicativo para TODOS

Neste ponto, você já tem um ótimo programa. Mas ele só funciona na sua máquina. Ou seja, ninguém mais pode usar seu aplicativo, comprá-lo de você, ver o quão genial você é e contratar você... e seu chefe e clientes não podem ver os relatórios que você está gerando a partir da base de dados.

O C# facilita escolher um aplicativo que você criou e **distribui-lo**. Distribuição é pegar um aplicativo e instalá-lo em outras máquinas. E com o IDE do Visual C#, você pode fazer uma distribuição com apenas dois passos.

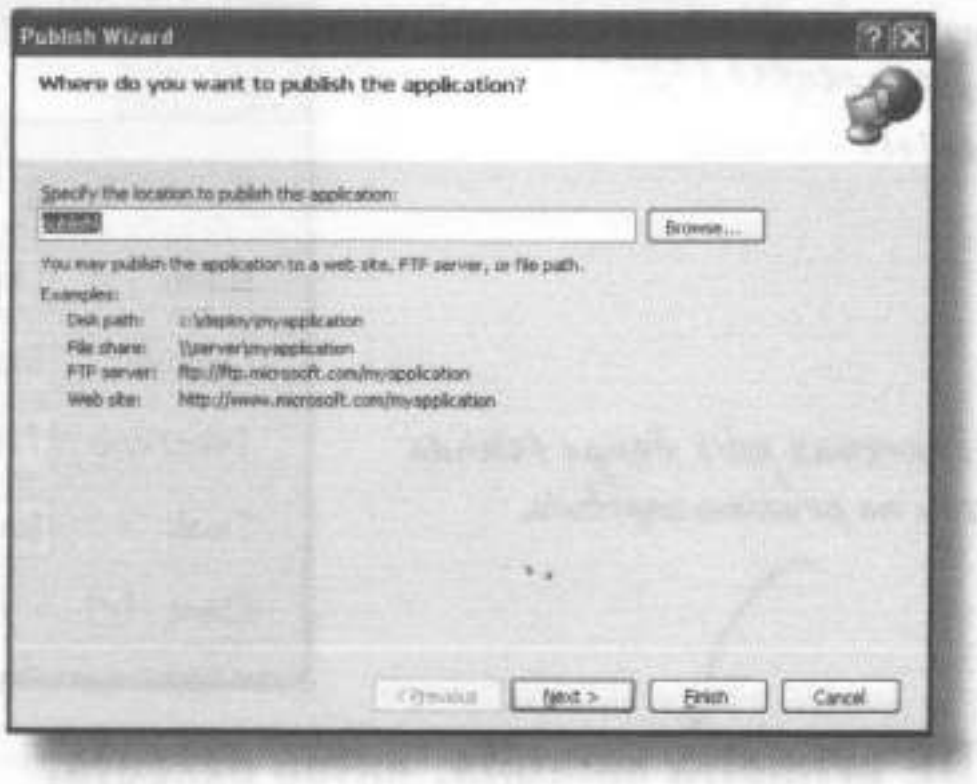
Você está aqui

1. Selecione *Publish Contacts* (Publicar Contatos) do menu Build.



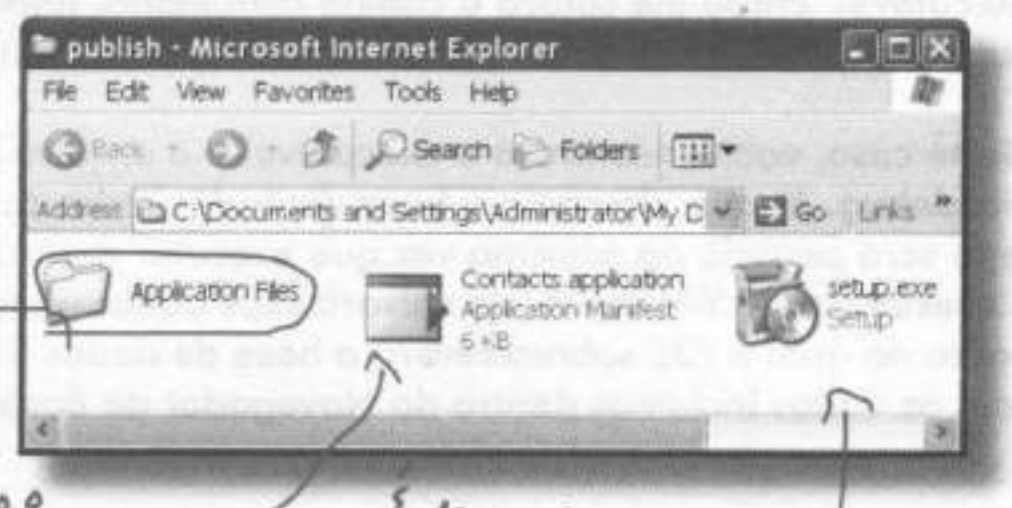
Build solution (Montar a solução) apenas copia os arquivos em sua máquina local. Publish cria um executável Setup e um arquivo de configuração para que então qualquer máquina possa instalar seu programa.

2. Apenas concorde com todos os padrões no Assistente de Publicação clicando em Finish. Você o verá compilar seu aplicativo e então mostrar uma pasta que tem seu Setup.exe dentro.



Dê o aplicativo aos seus usuários

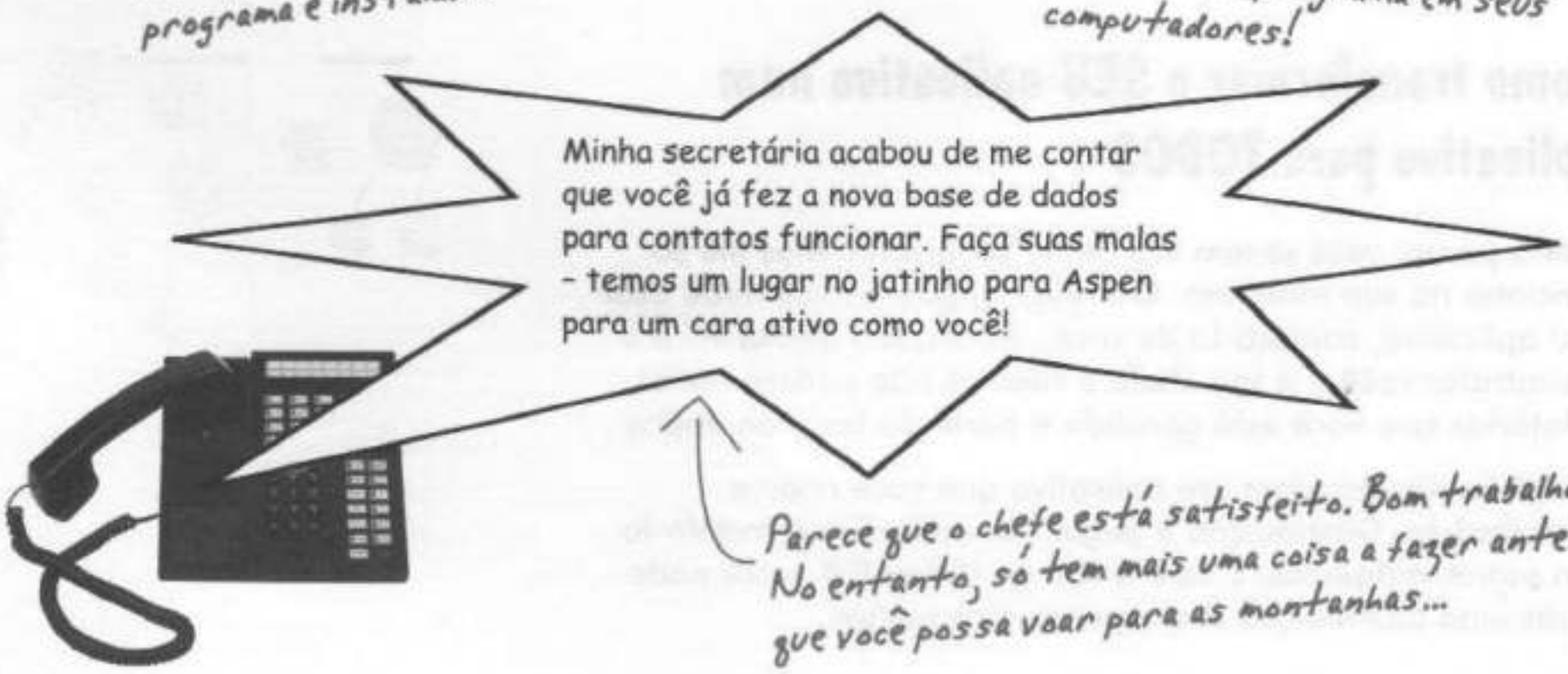
Uma vez que tenha criado uma distribuição, você terá uma nova pasta chamada publish/. Ela contém muitas coisas, todas usadas para a instalação. A mais importante para seus usuários é o setup, um executável que os permitirá instalar seu programa em seus próprios computadores.



É aqui onde todos os arquivos de suporte para o instalador estão armazenados.

Este arquivo informa ao instalador tudo que precisa ser incluído quando o programa é instalado.

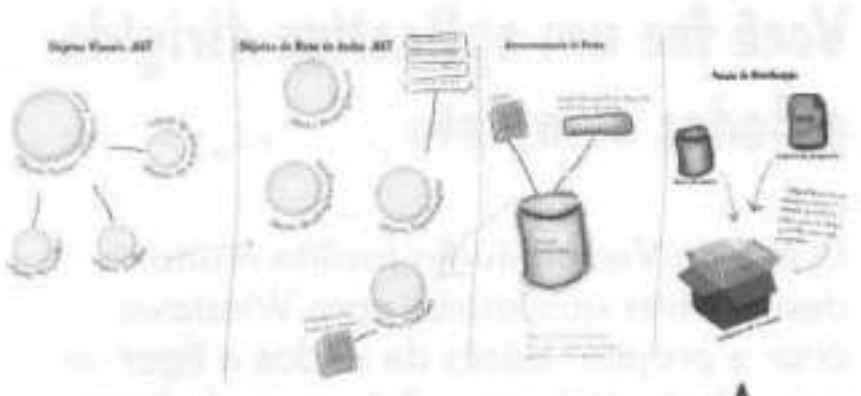
É dessa forma que seus usuários instalarão o programa em seus computadores!



Parece que o chefe está satisfeito. Bom trabalho! No entanto, só tem mais uma coisa a fazer antes que você possa voar para as montanhas...

Você NÃO terminou: teste sua instalação

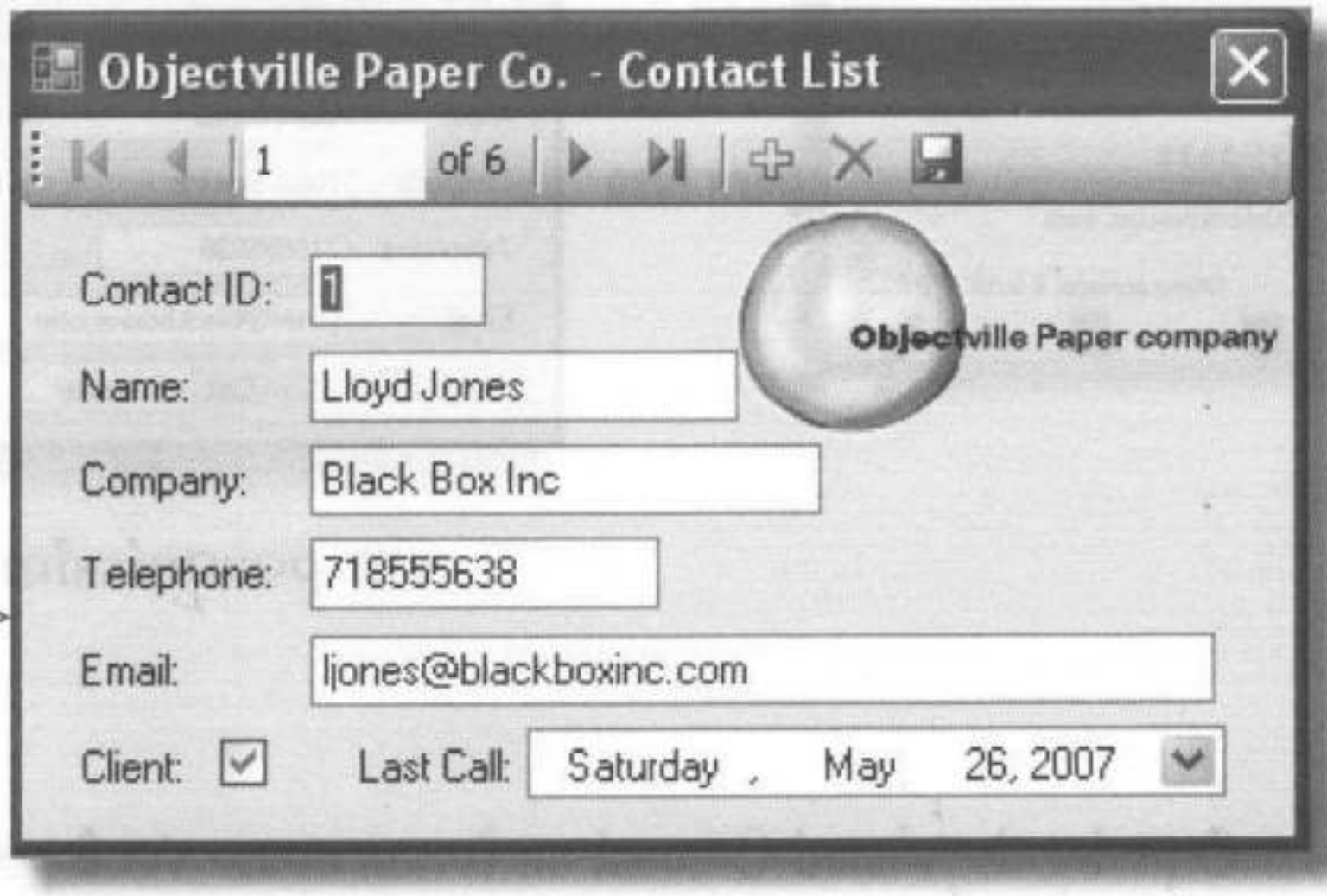
Antes que você estoure alguma garrafa de champanhe, precisa testar sua distribuição e instalação. Você não daria seu programa pra ninguém antes de executá-lo primeiro, certo?
Feche o IDE do Visual Studio. Clique no executável de instalação e selecione um local em seu computador para instalar o programa. Agora o execute de lá e tenha certeza de que ele funciona como você esperava. Você pode adicionar e alterar registros também e eles serão salvos na base de dados.



Você está aqui

Agora você pode adicionar, alterar e deletar registros; e eles serão salvos na base de dados.

Você pode usar as setas e o campo de texto para trocar de registro.



Vá em frente... faça algumas alterações. Você fez a distribuição, então dessa vez elas serão mantidas.

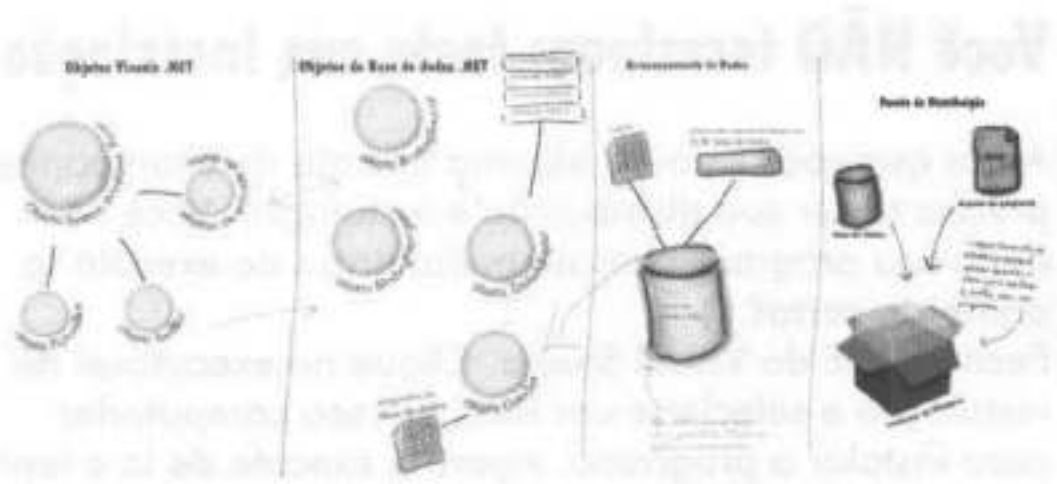
Os seis registros que você acrescentou inicialmente estão todos lá.

TESTE TUDO!

Teste seu programa, sua distribuição e os dados em seu aplicativo.

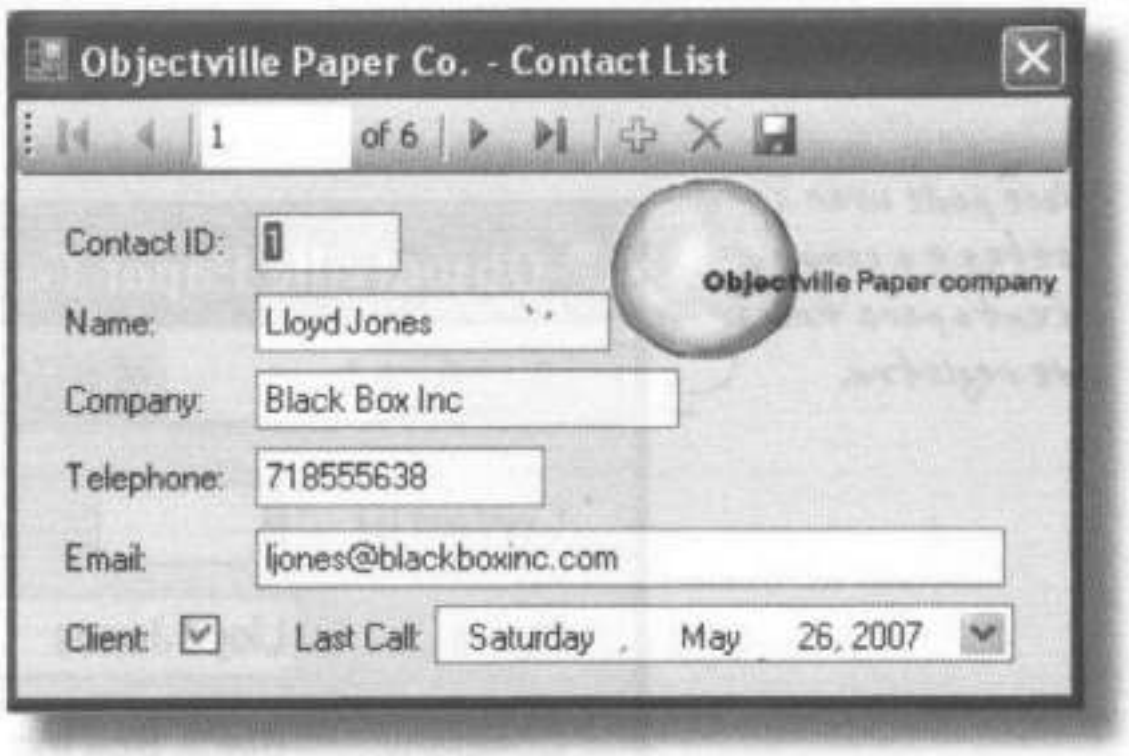
Você fez um aplicativo dirigido a dados completo

O IDE do Visual Studio facilita muito desenvolver aplicativos para Windows, criar e projetar bases de dados e ligar os dois. Você até mesmo foi capaz de fazer um instalador com alguns cliques extras.



Disso

para isso



em pouquíssimo tempo.

O poder do Visual C# está na forma como você pode rapidamente aprender e produzir com ele e, então, focar-se no que seu programa deve fazer... não em um monte de janelas, botões e código de acesso SQL.

Nos bastidores

Você é um programador, não apenas usuário de IDE

Você pode fazer muito usando o IDE. Mas existe um limite para o que o IDE pode fazer por você. Claro, existem muitas tarefas repetitivas que devem ser feitas quando se desenvolve um aplicativo. E o IDE é ótima para fazer essas coisas. Mas trabalhar com o IDE é apenas o começo. Pode-se fazer um aplicativo realizar muito mais - e escrever código C# é a forma de conseguir-se isto. Uma vez que se tenha pegado o jeito da coisa com relação a programar, não há nada que seus programas não possam fazer.

Enquanto você está fazendo isso...

O IDE é uma ferramenta poderosa - mas é tudo o que ele é, uma ferramenta para seu uso. Toda vez que você mudar seu projeto ou arrastar e soltar alguma coisa no IDE, ele cria código automaticamente. E ele é muito bom em escrever código de **suporte**, ou seja, aquele que pode ser reutilizado sem exigir muita customização.

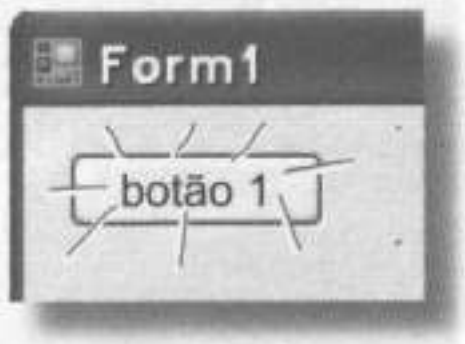
Vamos examinar o que o IDE faz em desenvolvimento de aplicativos típicos, quando estamos...

Todas essas tarefas são relacionadas com ações padrão e código de suporte. Essas são as coisas nas quais o IDE é ótimo para ajudar.

1. **Criando uma nova solução que seja um aplicativo do Windows**
   Existem muitos tipos de aplicativos que o IDE permite que se desenvolva, mas nos concentraremos em aplicativos Windows por agora. Estes são programas com elementos visuais, como formulários e botões.

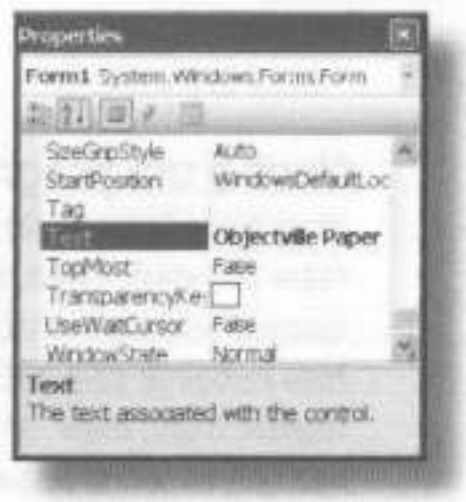
*Certifique-se de sempre criar um projeto para **aplicativo Windows Forms** - isto diz ao IDE para dar-nos um formulário vazio e adicioná-lo ao novo projeto.*

2. **Arrastando um botão da barra de ferramentas para o rmulário e clicando nele duas vezes**
   Botões são o meio pelo qual você faz as coisas acontecerem no seu formulário. Usaremos muitos botões para explorar várias partes da linguagem C#. Eles também são parte de quase todos os aplicativos C# que você escreverá.



3. **Atribuindo uma propriedade no seu formulário**
   A **janela de propriedades** no IDE é uma ferramenta realmente poderosa usada para mudar atributos de quase todas as coisas em seus programas: todas as propriedades visuais e funcionais dos controles nos formulários, atributos de bases de dados e mesmo opções do próprio projeto.

*A **janela de propriedades** no IDE é apenas uma forma muito fácil de editar uma parte específica do código em Form1.Designer.cs automaticamente. Seria bem mais difícil fazer isso manualmente.*

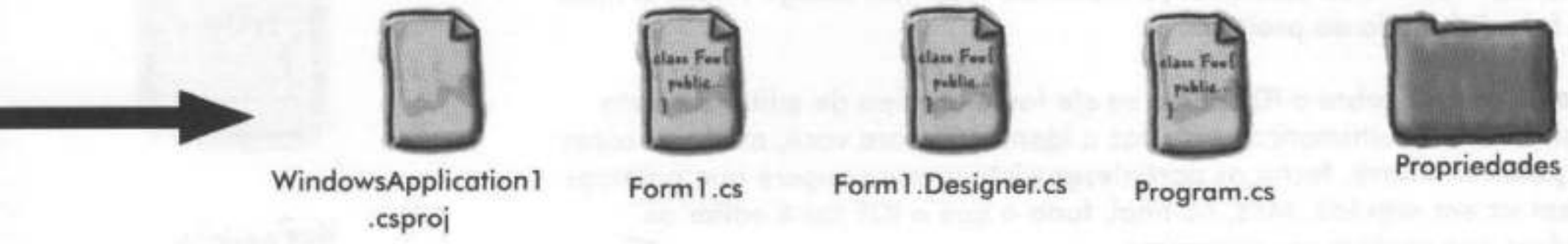


...o IDE está fazendo isso

Toda vez que altera o IDE, ele altera uma parte do código. Ou seja, alteram-se os arquivos que o contém. Algumas vezes apenas algumas poucas linhas são modificadas, mas em outras arquivos inteiros são adicionados ao projeto.

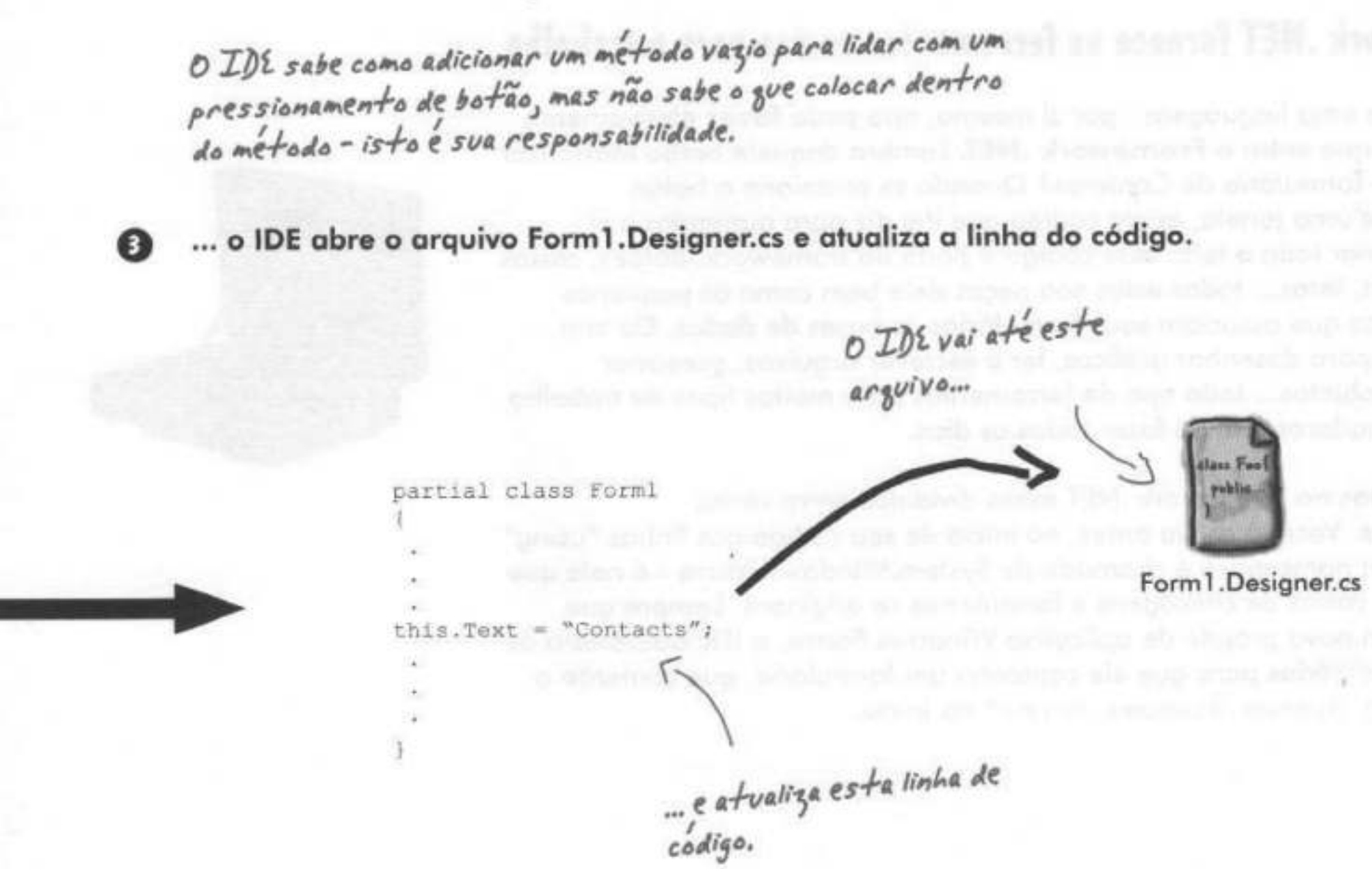
Esses arquivos são criados a partir de um modelo predefinido que contém o código básico para criar e exibir um formulário.

❶ ... o IDE cria os arquivos e pastas para o projeto.



❷ ... o IDE adiciona código ao arquivo Form1.Designer.cs, o que insere botões no formulário, e então adiciona código ao arquivo Form1.cs para lidar com o pressionamento desse botão.

❸ ... o IDE abre o arquivo Form1.Designer.cs e atualiza a linha do código.



legal, "aquela" conversa

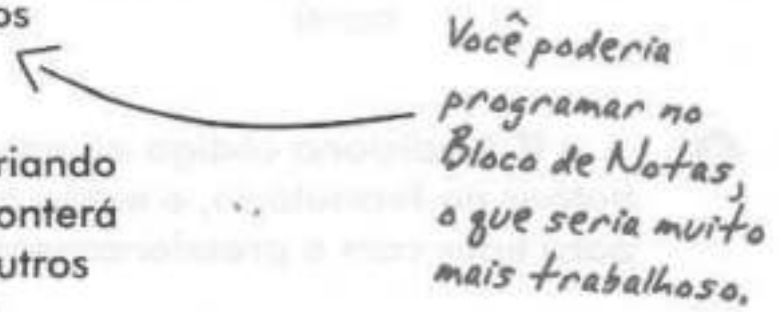
De onde vêm os programas

Um programa C# pode começar como comandos em vários arquivos, mas ele termina como um programa executando no seu computador. Eis como ele chega lá.

Todos os programas começam como arquivos de código fonte

Você já viu como editar um programa e como o IDE o salva em arquivos e pastas. Estes arquivos **são** o seu programa - você pode copiá-los para outra pasta, abri-los e tudo estará lá: formulários, recursos, código e tudo o mais que foi adicionado ao projeto.

Pode-se pensar sobre o IDE como se ele fosse um tipo de editor de texto sofisticado. Ele automaticamente faz a identação para você, muda as cores das palavras-chave, fecha os parênteses e até mesmo sugere que palavras devem vir em seguida. Mas, no final, tudo o que o IDE faz é editar os arquivos que contêm seu programa.



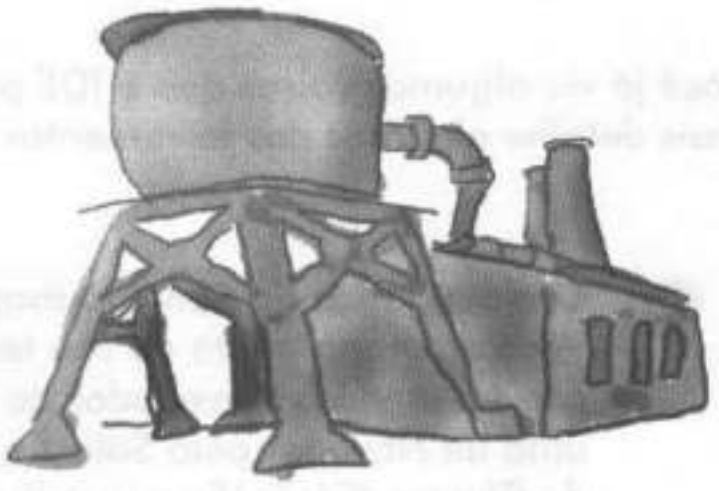
O IDE agrupa todos os arquivos de seu programa numa **solução**, criando um arquivo próprio para tanto (.sln, de solution) e uma pasta que conterá todos os demais arquivos. O arquivo de solução tem uma lista de outros que contêm projetos (os com extensão .csproj), e estes, por sua vez, contêm listas de todos os demais associados com o programa. Neste livro, desenvolveremos soluções que tem apenas um único projeto dentro delas, mas pode-se facilmente adicionar outros projetos nas suas soluções usando o Navegador de Soluções (Solution Explorer) do IDE.

O Framework .NET fornece as ferramentas certas para o trabalho

C# é apenas uma linguagem - por si mesma, não pode **fazer** efetivamente nada. E é aí que entra o **Framework .NET.** Lembra daquele botão Maximizar removido do formulário de Contatos? Quando se pressiona o botão Maximizar de uma janela, existe código que lhe diz para aumentar a si mesma e tomar toda a tela. Esse código é parte do framework. Botões, caixas de checagem, listas... todas estas são peças dele bem como as pequenas peças internas que associam seus formulários às bases de dados. Ele tem ferramentas para desenhar gráficos, ler e escrever arquivos, gerenciar coleções de objetos... todo tipo de ferramentas para muitos tipos de trabalho que programadores tem de fazer todos os dias.

As ferramentas no Framework .NET estão divididas entre vários **namespaces**. Você já os viu antes, no início de seu código nas linhas "using" (usando). Um namespace é chamado de System.Windows.Forms - é nele que seus botões, caixas de checagens e formulários se originam. Sempre que você criar um novo projeto de aplicativo Windows Forms, o IDE adicionará os arquivos necessários para que ele contenha um formulário, que conterão a linha `"using System.Windows.Forms"` no início.

Compile o programa para criar um executável



Quando você seleciona "compilar a solução" (build Solution) no menu Compilar (Build), o IDE **compila** seu programa. Ele faz isso executando o **compilador**, uma ferramenta que lê o código fonte de seu programa e o transforma num **executável**. Este, por sua vez, é um arquivo com extensão .exe - é nele que você clica duas vezes para executar seus programas. Quando um programa é compilado, o executável é criado dentro da pasta bin, que fica dentro da pasta do projeto. Quando você publica sua solução, o executável é copiado (juntamente com qualquer outro arquivo necessário) para a pasta de publicação.

Quando se seleciona "Iniciar depuração" (Start Debugging) no menu Depurar (Debug), o IDE compila o programa e inicia o executável. Ele tem mais algumas ferramentas avançadas para **depurar** o seu programa, o que apenas quer dizer executá-lo e ser capaz de colocá-lo em pausa (ou "interrompê-lo", "break" em inglês que significa, literalmente, "quebrar") de forma que se possa determinar o que está acontecendo.

Seu programa executa dentro do CLR

Quando você clica duas vezes no executável, o Windows inicia o programa. Mas existe uma "camada" extra entre os dois chamada de **Runtime* de Linguagem Comum (Common Language Runtime)**, ou CLR. Era uma vez, num passado não tão distante (mas antes do C# existir) um mundo onde escrever programas era mais difícil; tínhamos que lidar com o hardware e outras coisas de baixo nível nos computadores. O CLR - freqüentemente chamado de **máquina virtual (virtual machine)** - cuida de todos esses detalhes para você fazendo um tipo de "tradução" do seu programa para o computador que o está executando.

Você vai aprender sobre uma série de coisas que o CLR pode fazer. Por exemplo, ele gerencia rigorosamente a memória do computador determinando, por exemplo, quando um programa foi encerrado deixando pra trás parte dos dados. Nesse caso, ele elimina-os para você. Em geral, isto era algo feito pelos próprios programadores. Agora você não precisa mais se preocupar com isto. Você pode não ter percebido antes, mas o CLR vai facilitar a tarefa de aprender C#.

Você não precisa se preocupar muito com o CLR nesse momento. É o suficiente saber que ele existe e que toma conta do seu programa automaticamente. Aprenderemos mais sobre ele conforme avançarmos.

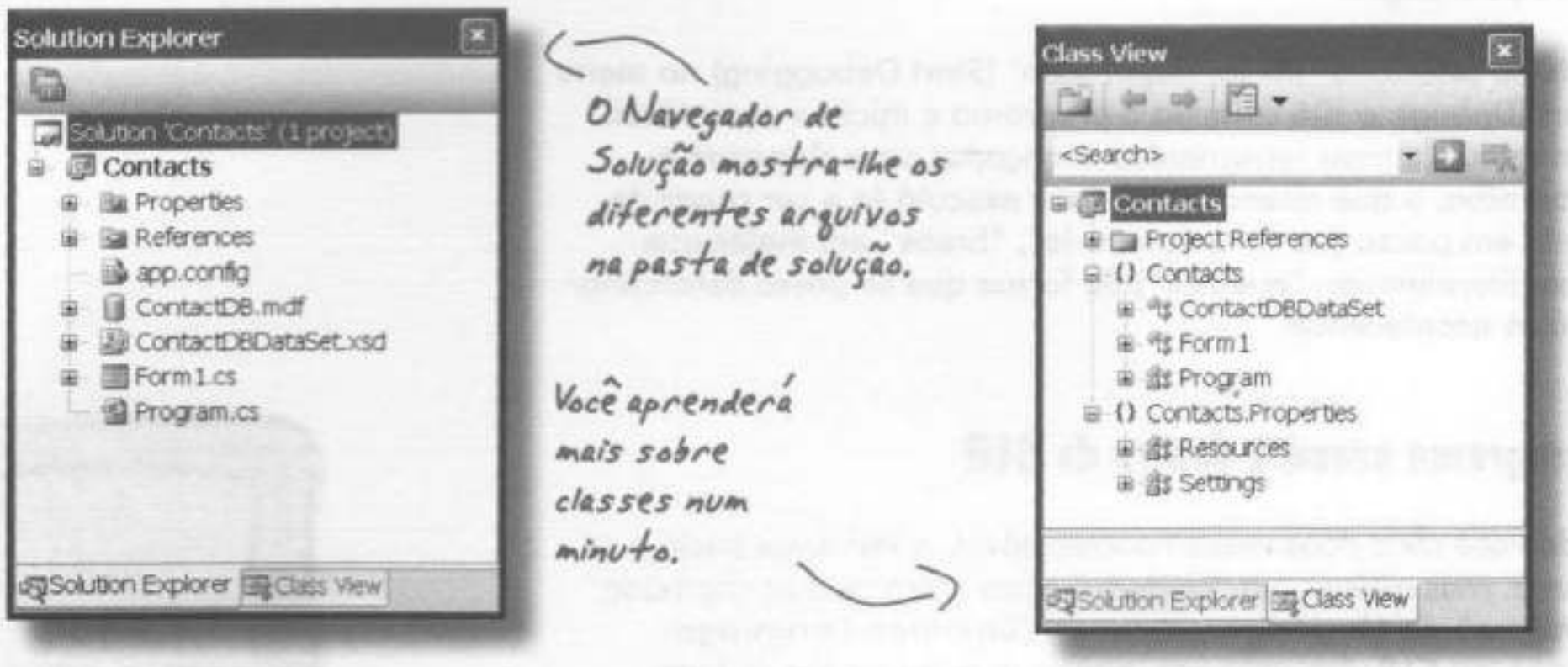
* runtime pode ser traduzido como "módulo responsável pela execução". Significa literalmente "tempo de execução".

O IDE ajuda você a fazer seu código

Você já viu algumas coisas que o IDE pode fazer. Vamos examinar com mais detalhe algumas das ferramentas que ele fornece.

O Navegador de Solução mostra tudo no seu projeto

Você emprega muito do seu tempo indo de uma classe para outra. A forma mais fácil de fazer isto é usando o Navegador de Solução (Solution Explorer). Ele tem duas abas de visualização: uma de Navegar pela Solução (que lhe mostra os arquivos num projeto) e uma de Visualização de Classes (Class View, que lhe mostra como o código está dividido logicamente em classes).

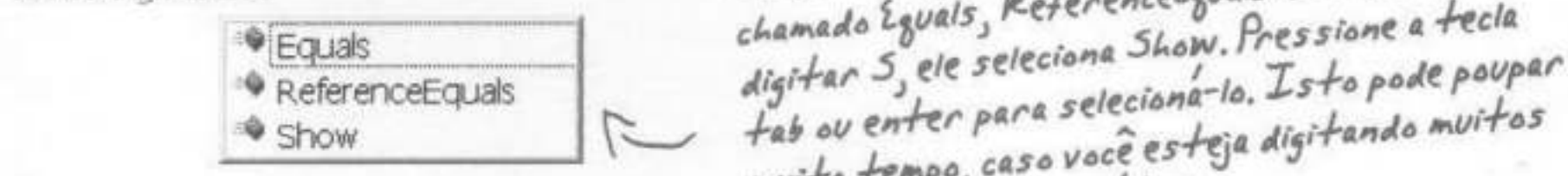


Use as abas para alternar entre arquivos abertos

Uma vez que seu programa está dividido em mais de um arquivo, normalmente, você terá muitos arquivos de código aberto ao mesmo tempo. Quando for o caso, cada um deles terá sua própria aba no editor de código. O IDE mostra um asterisco (*) ao lado de um nome de arquivo que ainda não tenha sido salvo.

O IDE ajuda você a criar seu código

Você notou pequenas janelas aparecendo conforme você vai escrevendo código no IDE? Isto é uma característica chamada IntelliSense que é realmente útil. Uma das coisas que ela faz é mostrar-lhe formas possíveis de completar a linha de código que está digitando. Se você digita MessageBox e um ponto, ele sabe que há três formas válidas de completar essa linha:



Se você selecionar Show e digitar "(", o IntelliSense do IDE mostrará informações sobre como completar a linha:

Isso significa que há 21 formas diferentes de chamar o método Show de MessageBox (como, por exemplo, formas diferentes de exibir botões e ícones).

```
MessageBox.Show(
```

▲ 3 of 21 ▼ DialogResult MessageBox.Show (**string text**, string caption)
text: The text to display in the message box.

O IDE também tem atalhos chamados trechos (snippets) que permitem se digitar uma abreviação e que o restante do código seja preenchido automaticamente. Eis um útil: digite mbox e pressione a tecla tab duas vezes, e o IDE vai preencher o método `MessageBox.Show` para você:

```
MessageBox.Show("Test");
```

A Lista de Erros ajuda você a resolver problemas de compilação

Se você ainda não descobriu como é fácil cometer erros de digitação num programa C#, vai descobrir muito em breve! Por sorte, o IDE tem uma ótima ferramenta para resolvê-los. Quando se compila uma solução, qualquer problema que tenha impedido a finalização do processo aparecerá na janela Lista de Erros, na parte inferior do IDE.

Quando você usa Iniciar Depuração (Start Debugging) para executar seus programas dentro do IDE, a primeira coisa feita é a compilação. Se ela for bem sucedida, então o programa executa. Se não, ele não vai executar e os erros vão ser exibidos na Lista de Erros (Error List).

Error List

2 Errors | 0 Warnings | 0 Messages

		Description	File	Line	Column	Project
✖	1	; expected	Form1.cs	26	48	Contacts
ⓧ	2	Only assignment, call, increment, decrement, and new object expressions can be used as a statement	Form1.cs	16	13	Contacts

Clique duas vezes num erro e o IDE se deslocará até a posição no código onde ele ocorreu:

```
private void pictureBox1_Click(object sender, EventArgs e)
{
    MessageBox.Show("Contact List 1.0")
}
```

O IDE vai sublinhar em vermelho onde ele determinou que falta um ponto e vírgula.

Quando coisas são alteradas no IDE, o código é mudado também

O IDE é ótimo para escrever código visual. Mas você não precisa acreditar na nossa palavra. Abra o Visual Studio, **crie um novo projeto de aplicativo Windows** e veja por você mesmo.

Quando você notar um Faça Isso!, abra o IDE e siga as instruções. Diremos a você exatamente o que fazer e indicaremos o que prestar atenção para obter o máximo proveito do exemplo mostrado.

1 **Abra o painel de design de código**
Abra o arquivo `Form1.Designer.cs` no IDE. Mas dessa vez, em vez de abrir o painel de design de formulário, abra o código clicando com o botão direito no Navegador de Solução, selecionando "ver código" (view code). Procure pela declaração de classe:

```
public partial class Form1 : Form
```

Notou que é uma classe parcial?

2 **Abra o painel de design de formulário e adicione uma caixa de imagem**
Acostume-se com trabalhar com mais de uma aba. Vá ao Navegador de Solução e abra o painel de design de formulário clicando duas vezes em `Form1.cs`. **Arraste uma nova caixa de imagem** para um novo formulário.

3 **Encontre e abra o código gerado pelo painel de design para o controle de caixa de imagem**
Então retorne para a aba de `Form1.Designer.cs` no IDE. Procure pela seguinte linha de código:

Clique no sinal de mais

```
+ Windows Form Designer generated code
```

Clique no sinal de + no lado esquerdo da linha para expandir o código. Procure pelas seguintes linhas:

```
//
// pictureBox1
//
this.pictureBox1.Location = new System.Drawing.Point(276, 28);
this.pictureBox1.Name = "pictureBox1";
this.pictureBox1.Size = new System.Drawing.Size(100, 50);
this.pictureBox1.TabIndex = 1;
this.pictureBox1.TabStop = false;
```

Não se preocupe se os números em seu código para as linhas Localização (Location) e Tamanho (Size) forem um pouco diferentes destes...

Pare, pare! O que isso quer dizer?

Procure no código por um minuto. Ali está, na parte de cima da seção de código gerado pelo Painel de Design de Formulários.

```
/// <summary>
/// Required method for Designer support - do not modify
/// the contents of this method with the code editor.
/// </summary>
```

A maioria dos comentários começa com duas barras (//). Mas o IDE algumas vezes insere estes comentários com três delas.

Nada mais atraente para uma criança do que um grande cartaz que diz "não toque nisso". Admita, você está tentado... vamos modificar o conteúdo desse método com o editor de código. **Adicione um botão a seu formulário e então faça o seguinte:**

1. **Modifique o código que atribui o valor da propriedade button1.Text. O que você pensa que vai acontecer com a janela de propriedades do IDE?**
   Tente - veja o que acontece! Agora volte ao painel de design de formulário e verifique a propriedade Text. Ela mudou?

2. **Continue o painel de design, e use a janela de propriedades para modificar a propriedade Name.**
   Veja se você consegue fazer com que o IDE modifique a propriedade Name. Ela aparece na janela de propriedades logo acima de todas as demais, descrita como "(Name)". O que aconteceu com o código? E com os comentários nele?

3. **Modifique o código que atribui a propriedade Location (localização) para (0,0) e a propriedade Size (tamanho) para deixar o botão bem grande.**
   Funcionou?

4. **Volte para o painel de design e modifique a propriedade BackColor (cor de fundo) do botão.**
   Olhe com atenção para o código em Form1.Designer.cs. Foram adicionadas linhas?

Você não precisa salvar o formulário ou executar o programa para ver as modificações. Apenas faça as alterações no editor de código e clique na aba do painel de design para trazê-lo para o primeiro plano - as alterações devem ser refletidas imediatamente.

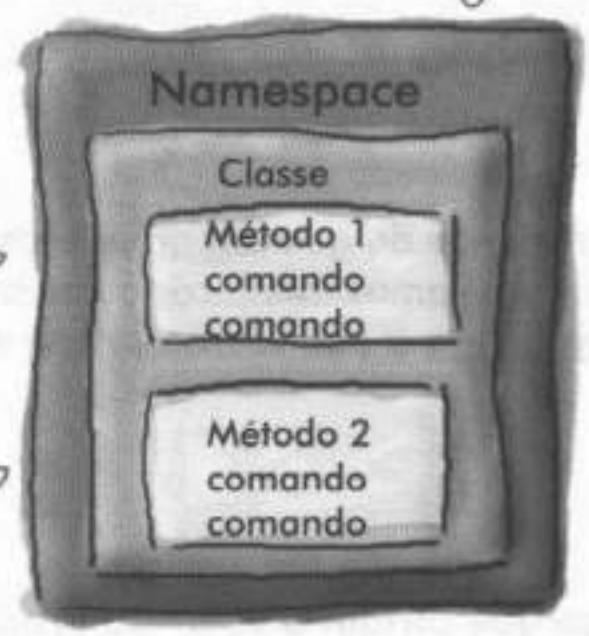
É sempre mais fácil usar o IDE para alterar o código gerado pelo painel de design para seu formulário. Mas quando isto é feito, reflete-se como uma alteração no código do seu projeto.

Anatomia de um programa

Cada programa C# é estruturado sempre exatamente da mesma forma. Todos os programas usam **namespaces, classes e métodos** para deixar o código mais fácil de gerenciar.

Toda vez que você cria um novo programa, define um namespace para ele de forma que seu código fique separado das classes do Framework .NET.

Uma classe contém uma parte de seu programa (embora alguns programas bem pequenos possam ter apenas uma única classe).



Uma classe tem um ou mais métodos. Seus métodos sempre têm de existir dentro de classes. E eles são feitos de comandos - como aqueles que já vimos.

Vamos examinar seu código mais de perto

Abra o código de seu arquivo `Form1.cs` no projeto de Contatos para que possamos examiná-lo passo a passo.

1 **O arquivo de código inicia usando as ferramentas do Framework .NET**
Você vai notar um grupo de linhas `using (usando)` no início de cada arquivo de programa. Elas dizem ao C# quais partes do Framework .NET usar. Se você usar outras classes que estão em outros namespaces, tem de adicionar linhas `using` para elas também. Já que os formulários com freqüência usam muitas ferramentas diferentes no Framework .NET, o IDE automaticamente adiciona várias linhas `using` quando se cria um formulário e ele é adicionado a um projeto.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
```

Existem linhas using no início de cada arquivo de programa. Elas dizem ao C# para usar certas classes do Framework .NET. Cada uma diz ao seu programa que uma classe nesse arquivo .cs em particular usará todos os itens nos namespaces indicados do Framework .NET.

2 **Programas C# são organizados usando classes**
Cada programa C# é organizado usando **classes**. Uma classe pode fazer qualquer coisa, mas a maioria delas faz algo bem específico. Quando se cria um novo aplicativo, o IDE adiciona uma classe chamada Form1 que mostra um formulário.

Quando você chamou o programa Contatos, o IDE criou um namespace para ele chamado Contacts adicionando a palavra chave namespace no início do código. Tudo dentro das chaves correspondentes a esse comando é parte deste namespace.

```
namespace Contacts
{
  public partial class Form1 : Form
  {
```

Esta é a classe Form1. Ela contém todo o código para desenhar o formulário e os controles de barras de ferramentas nele. O IDE o criou quando foi determinado que um novo projeto de aplicativo Windows Forms fosse criado.

3 **Classes contém métodos que executam tarefas**
Quando uma classe precisa fazer alguma coisa, ela usa um **método**. Eles precisam de algumas informações, executam alguma ação e, algumas vezes, produzem algum resultado. A forma de passar informações de entrada para os métodos é usando **parâmetros**. Um mesmo método pode ter comportamentos diferentes dependendo de quais parâmetros de entrada recebe. Quando os métodos produzem resultados, isto é chamado de **valor de retorno**. Se você notar a palavra chave void na frente de um método, isso significa que ele não retorna valor algum.

Observe os pares correspondentes de chaves. Cada "{" sempre tem de ter um "}" correspondente. Alguns pares podem ser contidos dentro de outros.

```
    public Form1()
    {
      InitializeComponent();
    }
```

Esta linha chama um método de nome InitializeComponent(), que o IDE também criou para você.

4 **Um comando executa uma ação individual**
Quando você adicionou a linha MessageBox.Show() ao programa, estava adicionando um **comando**. Todos os métodos são feitos de comandos. Quando seu programa chama um método, este executa seu primeiro comando e depois o próximo e assim por diante. Quando o método encontra um comando return, ele encerra e o programa continua depois do comando que chamou o método.

Este é um método chamado pictureBox1_Click(). Ele é chamado quando o usuário clica na caixa de imagem.

Esse método tem dois parâmetros chamados sender e e.

```
    private void pictureBox1_Click(object sender, EventArgs e)
    {
      MessageBox.Show("Contact List 1.0", "About");
    }
  }
}
```

Este é um comando. Você já sabe o que ele faz - uma pequena janela com uma mensagem surge em resposta a ele.

Seu comando chamou o método Show(), que é parte da classe MessageBox, que, por sua vez, está contida no namespace System.Windows.Forms.

Seu comando passou dois parâmetros para o método Show(). O primeiro foi uma seqüência de caracteres de texto para ser exibida como mensagem, e o segundo, uma outra seqüência para ser mostrada na barra de título.

Seu programa sabe por onde começar

Quando você criou uma nova solução para produzir um aplicativo Windows, um dos arquivos que o IDE adicionou foi chamado de Program.cs. Vá até o Navegador de Solução e clique duas vezes nele. Ele tem uma classe chamada Programa e dentro dela um método chamado Main(). Este método é o ponto de entrada, o que quer dizer que ele é o primeiro a ser executado no programa.

Cada programa C# pode ter apenas um único método que seja o ponto de entrada e ele sempre se chamará Main(). É assim que se determina por onde começar quando ele é executado.

Eis algum código que o IDE criou para você automaticamente no capítulo anterior. Você o encontrará em Program.cs.

Seu código de perto

```
1
using System;
using System.Linq;
using System.Collections.Generic;
using System.Windows.Forms;

2
namespace Contacts
{
3
  static class Program
  {

    /// <summary>
    /// The main entry point for the application.
    /// </summary>
    [STAThread]
    static void Main()
    {
5

      Application.EnableVisualStyles();
4     Application.SetCompatibleTextRenderingDefault(false);
      Application.Run(new Form1());
    }
  }
}
```

O namespace para todo esse código é Contacts. Discutiremos mais sobre namespaces algumas páginas adiante.

Linhas que começam com barras são comentários. Eles podem ser adicionados em qualquer lugar que se queira. As barras dizem ao C# para ignorar o que segue.

Cada vez que o programa é executado, ele começa por aqui, no ponto de entrada.

Este comando cria e mostra o formulário de Contatos e encerra o programa quando o formulário é fechado.

Eu declaro!

A primeira linha de cada classe ou método é chamada de declaração.

Lembre-se, isso é apenas o ponto de partida para que você mergulhe no código. Mas antes disto, você precisa saber o que está procurando.

1. **O C# e o .NET tem muitas características já desenvolvidas**
Você encontrará linhas como estas no topo de quase todos os arquivos de classe C#. System.Windows.Forms é um namespace. A linha using System.Windows.Forms faz com que tudo o que existe naquele namespace esteja disponível para ser usado pelo seu programa. Nesse caso, este namespace tem muitos elementos visuais como botões e formulário.

Seus programas vão usar cada vez mais namespaces como estes conforme você aprender a respeito do C# e outras características programadas previamente do .NET neste livro.

2. **O IDE escolhe um namespace para seu código**
Eis o namespace que o IDE criou para você - ele escolheu Contacts baseando-se no nome do projeto. Todo o código do seu programa está contido nesse namespace.

Namespaces permitem utilizar os mesmos nomes em diversos programas, a não ser que os programas não estejam dentro do mesmo namespace.

3. **Seu código fica contido numa classe**
Esta classe em particular é chamada de Program. O IDE a criou e adicionou o código que inicia o programa e exibe o formulário Contacts.

Pode ter-se múltiplas classes dentro de um mesmo namespace.

4. **Este código tem apenas um método e ele contém três comandos**
Um namespace tem classes e classes têm métodos. Dentro de cada método está um conjunto de comandos. Neste programa, os comandos lidam com iniciar o formulário de contatos.
Métodos estão onde a ação acontece - cada um deles **faz** algo.

5. **Cada programa tem um tipo de método especial chamado de ponto de entrada**
Cada programa C# precisa ter um único método chamado Main (principal). Mesmo que existam muitos métodos, apenas um pode ser o primeiro a ser executado e este é seu método Main(). O C# checa cada classe no código procurando por um método definido como static void Main(). Então, quando o programa executa, o primeiro comando nesse método é executado. O mesmo ocorre sucessivamente com quaisquer outros comandos que se seguirem ao primeiro.

Cada programa C# tem de ter apenas um <u>único</u> método chamado Main. Ele é o <u>ponto de entrada</u> no seu código.
Quando se executa um programa, o código no método Main é ativado PRIMEIRO.

Você pode mudar o ponto de entrada de seu programa

Desde que seu programa tenha um ponto de entrada, não importa em qual classe ele está, ou o que o método que o implementa faz. Vamos remover o método `Main` de `Program.cs` e criar um novo ponto de entrada.

Faça Isso!

1. Retorne para `Program.cs` e altere o nome do método `Main` para NotMain. Agora **tente compilar e executar** o programa. O que acontece?

_____

_____

Escreve o que acontece quando você modifica o nome deste método e porque você acha que isso acontece.

2. Vamos criar um novo ponto de entrada. **Adicione uma nova classe** chamada `AnotherClasse.cs` (outra classe). Adicionam-se classes ao programa clicando com o botão direito no nome do projeto no Navegador de Solução e selecionando-se "Add>>Class..." (adicione>>classe). Chame seu arquivo de classe de `AnotherClass.cs`. O IDE adicionará uma classe a seu programa chamada `AnotherClass`. Eis o arquivo que o IDE adicionou:

Clique com o botão direito no projeto em Propriedades e selecione Add e Class...

```
using System;
using System.Linq;
using System.Collections.Generic;
using System.Text;

namespace Contacts
{

    class AnotherClass
    {
    }
}
```

Essas quatro linhas using foram adicionadas ao arquivo.

Esta classe está no mesmo namespace Contacts que o IDE adicionou quando criamos pela primeira vez o projeto de aplicativo Windows.

O IDE automaticamente deu o nome para a classe baseando-se no nome do arquivo.

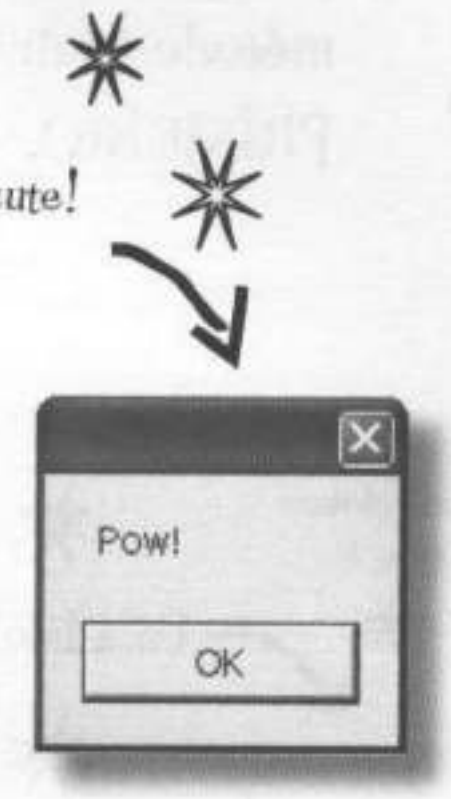
3. Adicione uma nova linha using no topo do arquivo: `using System.Windows.Forms;`
Não se esqueça de adicionar um ponto e vírgula no final da linha!

4. Adicione o seguinte método à classe AnotherClass digitando-o em algum ponto dentro das chaves:

MessageBox é uma classe contida no namespace System.Windows.Forms e este é o motivo para adicionar a linha using no passo 3. Show() é um método que integra a classe.

```
class AnotherClass
{
  public static void Main()
  {
    MessageBox.Show("Pow!");
  }
}
```

Agora execute!



Então, o que aconteceu?

Em vez de mostrar o aplicativo de Contatos, seu programa agora mostra esta caixa de mensagem. Quando você criou o novo método Main(), deu a seu programa um novo ponto de entrada. Agora a primeira coisa que ele faz é executar os comandos nesse novo método - o que quer dizer executar o comando MessageBox.Show(). Não há mais nada no método, então depois que se clicar no botão OK, o programa fica sem mais nada para executar e encerra.

5. Determine como corrigir seu programa para que ele mostre o formulário de contatos novamente.

Dica: você tem de mudar apenas duas linhas para fazer isso.

Aponte seu lápis

Preencha as notas para descrever as linhas nesse arquivo C# para onde as setas apontam. Nós preenchemos a primeira pra você.

```
using System;
using System.Linq;
using System.Text;
using System.Windows.Forms;


namespace SomeNamespace
{
     class MyClass {
          public static void DoSomething() {
               MessageBox.Show("This is a message");
          }
     }
}
```

As classes C# têm essas linhas using para adicionar métodos de outros namespaces.

.....

.....

.....

Aponte seu lápis Solução

Preencha as notas para descrever as linhas nesse arquivo C# para onde as setas apontam. Nós preenchemos a primeira pra você.

```
using System;
using System.Linq;
using System.Text;
using System.Windows.Forms;


namespace SomeNamespace
{
    class MyClass {
        public static void DoSomething() {
            MessageBox.Show("This is a message");
        }
    }
}
```

As classes C# têm essas linhas using para adicionar métodos de outros namespaces.

Todo o código está contido em classes, de forma que o programa precisa de uma aqui

Isto é um método. Cada um deles faz algo. Eles são usados para agrupar comandos.

Isto é um comando. Quando ele é executado, surge uma pequena janela com uma mensagem.

Qual meu propósito?

Associe cada um desses fragmentos de código gerados pelo IDE ao que eles fazem. (Alguns deles não foram explicados - dê um palpite para ver se acerta!)

```
partial class Form1
{
  :
  this.BackColor = Color.DarkViolet;
  :
}
```

Atribui propriedades para um rótulo (label)

```
// This loop gets executed three times
```

Nada - é um comentário que os programadores adicionam para explicar o código para qualquer um que esteja o lendo.

```
partial class Form1
{
  private void InitializeComponent()
  {
  :
  }
}
```

Desativa o ícone de maximizar ( ) na barra de título na janela Form1

```
number_of_pit_stopsLabel.Name
   = "number_of_pit_stopsLabel";
number_of_pit_stopsLabel.Size
   = new System.Drawing.Size(135, 17);
number_of_pit_stopsLabel.Text
   = "Number of pit stops:";
```

Um tipo especial de comentário que o IDE usa para explicar o que um bloco inteiro de código faz

```
/// <summary>
/// Bring up the picture of Rover when
/// the button is clicked
/// </summary>
```

Muda a cor de fundo na janela Form1

```
partial class Form1
{
   :
   this.MaximizeBox = false;
   :
}
```

Um bloco de código executado sempre que um programa abre a janela Form1

Qual meu propósito?

Associe cada um desses fragmentos de código gerados pelo IDE ao que eles fazem. (Alguns deles não foram explicados - dê um palpite para ver se acerta!)

```
partial class Form1
{
  ...
  this.BackColor = Color.DarkViolet;
  ...
}
```

```
// This loop gets executed three times
```

```
partial class Form1
{
  private void InitializeComponent()
  {
  ...
  }
}
```

```
number_of_pit_stopsLabel.Name
    = "number_of_pit_stopsLabel";
number_of_pit_stopsLabel.Size
    = new System.Drawing.Size(135, 17);
number_of_pit_stopsLabel.Text
    = "Number of pit stops:";
```

```
/// <summary>
/// Bring up the picture of Rover when
/// the button is clicked
/// </summary>
```

```
partial class Form1
{
  ...
  this.MaximizeBox = false;
  ...
}
```

Atribui propriedades para um rótulo (label)

Nada - é um comentário que os programadores adicionam para explicar o código para qualquer um que esteja o lendo.

Desativa o ícone de maximizar ( ) na barra de título na janela Form1

Um tipo especial de comentário que o IDE usa para explicar o que um bloco inteiro de código faz

Muda a cor de fundo na janela Form1

Um bloco de código executado sempre que um programa abre a janela Form1

Duas classes podem estar no mesmo namespace

Dê uma olhada nestes dois arquivos de classe de um programa chamado PetFiler2. Eles têm três classes: uma classe Dog (cachorro), uma Cat (gato) e uma Fish (peixe). Uma vez que estão todas no mesmo namespace PetFiler2, comandos no método Dog.Bark() (cão.late) podem chamar Cat.Meow() (gato.mia) e Fish.Swim() (peixe.nada). Não importa como diversos namespaces e classes são divididos em arquivos. Eles comportam-se da mesma forma quando executam.

SomeClasses.cs (algumas classes)

```
namespace PetFiler2 {

  public class Dog {
    public void Bark() {
      // statements go here
    }
  }
  public partial class Cat {
    public void Meow() {
      // more statements
    }
  }
}
```

Quando uma classe é pública isso quer dizer que todas as demais classes no programa podem acessar seus métodos.

MoreClasses.cs (mais classes)

```
namespace PetFiler2 {

  public class Fish {
    public void Swim() {
      // statements
    }
  }
  public partial class Cat {
    public void Purr() {
      // statements
    }
  }
}
```

Já que essas classes estão no mesmo namespace, todas elas podem ver umas às outras - mesmo que estejam em arquivos diferentes. Uma classe pode estar dividida em muitos arquivos também, mas você precisará usar a palavra chave partial (parcial) quando declará-la.

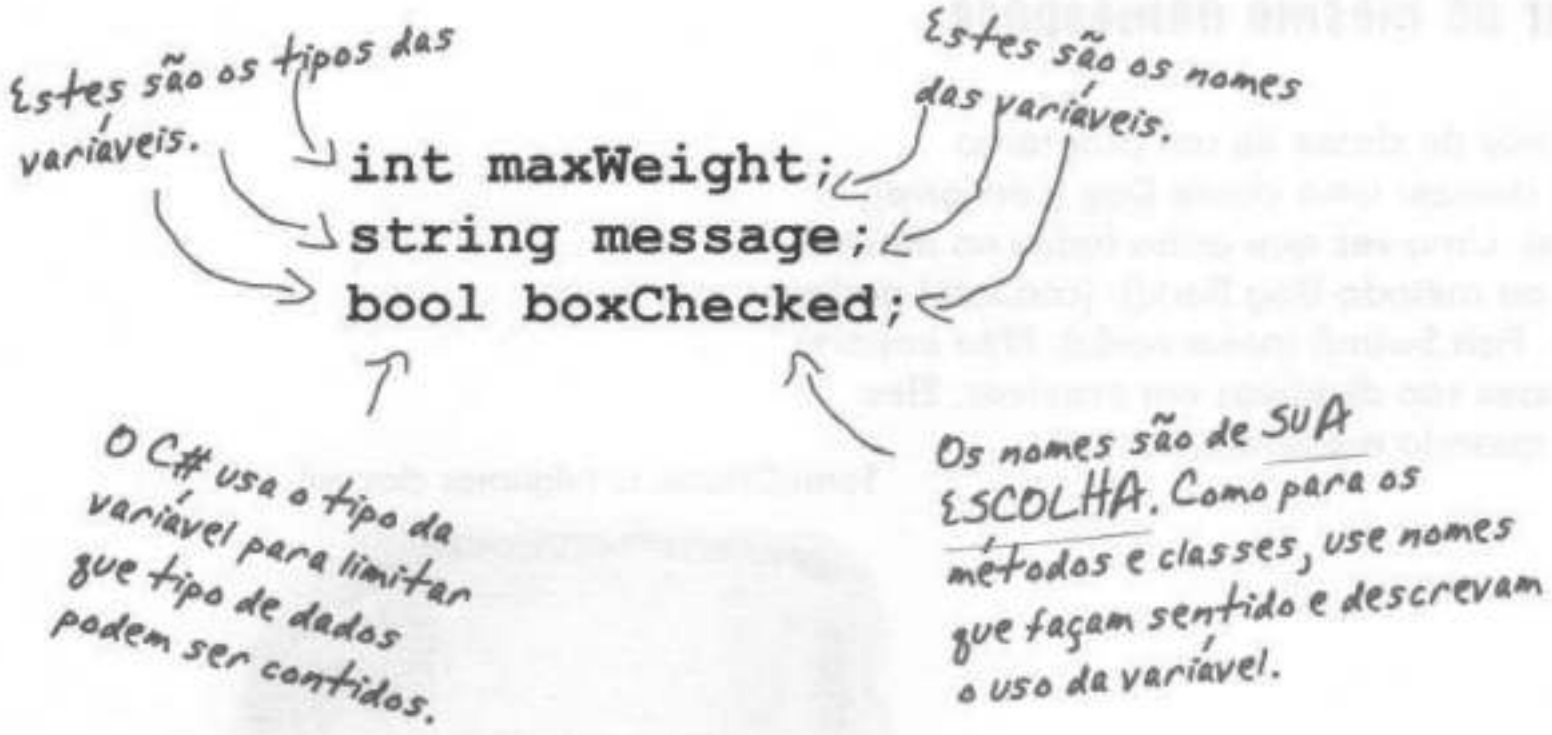
Pode-se apenas dividir uma classe entre vários diferentes arquivos se a palavra chave partial for usada. Provavelmente você não a usará em nenhum código que escreverá neste livro, mas o IDE utiliza-a para dividir seus formulários em dois arquivos, Form1.cs e Form1.Designer.cs.

Seus programas usam variáveis para trabalhar com dados

No fim das contas, todos os programas são basicamente processadores de dados. Algumas vezes os dados estão na forma de um documento, ou em uma imagem num videogame, ou em uma mensagem instantânea. Mas são apenas dados. E é aí que as **variáveis** entram. Elas são o que seu programa usa para armazenar dados.

Declare suas variáveis

Sempre que você **declarar** uma variável, você está dizendo ao seu programa seu tipo e nome. Uma vez que o C# sabe o tipo da sua variável, isto impedirá que seu programa compile se você cometer um erro e tentar fazer algo que não faz sentido, como subtrair "Totó" menos 48353.



As variáveis variam

Uma variável tem diferentes valores em momentos diferentes enquanto seu programa executa. Em outras palavras, o valor da variável ***varia*** (e é por isso que "variável" é um nome tão bom). Isto é realmente importante, porque esta idéia é vital para cada programa que você tenha escrito ou que vá escrever no futuro. Assim, por exemplo, se seu programa iguala a variável `myHeight` (minha altura) a 63:

```
int myHeight = 63;
```

a qualquer momento em que `myHeight` seja usada no código, o C# vai substitui-la pelo seu valor atual, ou seja, 63. Então, mais tarde, se você alterar seu valor para 12:

```
myHeight = 12;
```

O C# vai substitui-la por 12 - mas a variável ainda é chamada `myHeight`.

Sempre que seu programa precisar trabalhar com números, texto, valores do tipo verdadeiro/falso, ou qualquer outro tipo de dado, variáveis serão usadas para armazenar estes valores.

Você deve dar valores para as variáveis antes de usá-las

Experimente incluir os seguintes comandos num programa C#:

```
int z;
MessageBox.Show("The answer is " + z);
```

Vá em frente, teste. Um erro acontecerá e o IDE se recusará a compilar seu código. Isto ocorre porque o IDE checa cada variável para certificar-se de que você lhe atribuiu um valor antes de usá-la. A forma mais fácil de ter certeza que você não se esqueceu de fazer isso é combinar o comando de declaração com um de atribuição de valor:

```
int maxWeight = 25000;
string message = "Hi!";
bool boxChecked = true;
```

Esses valores são atribuídos às variáveis.

Alguns tipos úteis

Cada variável tem um tipo que diz ao C# que espécie de dados ela pode armazenar. Veremos muitos detalhes sobre muitos tipos diferentes do C# no capítulo 4. Enquanto isso, concentraremo-nos nos três tipos mais populares. O tipo `int` armazena números inteiros, `string` armazena texto e `bool`, valores tipo verdadeiro/falso.

va-ri-á-vel, adjetivo.
capaz de ser alterado ou adaptado
*a ponteira de velocidade **variável** da furadeira permite que Beto ajuste a ferramenta de mais lento para mais rápido, de acordo com a tarefa a ser realizada.*

Se você escreve código que usa uma variável que ainda não tenha recebido um valor, ele não vai compilar. É fácil evitar esse erro combinando suas declarações de variáveis com atribuição de valor num mesmo comando.

Depois de atribuir um valor para a sua variável, ele pode ser alterado. Assim não há desvantagem em atribuir um valor inicial a uma variável quando você declará-la.

O C# usa símbolos matemáticos familiares

Uma vez que você tenha alguns dados armazenados em variáveis, o que pode ser feito? Bem, se forem números, provavelmente, você vai querer adicionar, subtrair, multiplicar ou dividir. E é aí que entram os operadores. Você já conhece os básicos. Vamos apresentar mais alguns. Eis um bloco de código que usa operadores para realizar algumas operações matemáticas simples:

Declaramos uma nova variável inteira chamada number (número) e recebe o valor 15. Então lhes adicionamos 10. Depois do segundo comando, number terá valor igual a 25.

O terceiro comando muda o valor de number, deixando-o igual a 36 vezes 15, resultando em 540. Então ela é reiniciado novamente e o valor 12-(42/7), que resulta em 6, é atribuído-lhe

O operador =* é semelhante ao mais=, exceto que ele multiplica o valor atual em number por 3, de forma que ela acaba com o valor 48.

Este operador é um pouco diferente. O += significa pegue o valor de number e adicione-lhe 10. Já que number no momento está com o valor de 6, ele acaba com 16.

71 dividido por 3 é 23.666666. Já que number é uma variável inteira, ela pode apenas armazenar números inteiros, de forma que esse valor é alterado para 23.

```
int number = 15;
number = number + 10;
number = 36 * 15;
number = 12 - (42 / 7);
number += 10;
number *= 3;
number = 71 / 3;

int count = 0;
count ++;
count --;

string result = "hello";
result += " again " + result;
MessageBox.Show(result);
result = "the value is: " + count;
result = "";

bool yesNo = false;
bool anotherBool = true;
yesNo = !anotherBool;
```

Você usará inteiros frequentemente para fazer contagens e, quando for o caso, os operadores ++ e -- serão úteis. O ++ incrementa count (contagem) adicionando um ao valor, e o -- decrementa-o, de forma que o valor final dele será zero.

Esta caixa de mensagem mostrará uma caixa que diz hello again hello (olá novamente olá)

Quando se utiliza o operador + com uma string, ele apenas concatena duas seqüências de caracteres. Nesse caso, números serão automaticamente convertidos em string.

O "" é uma seqüência de caracteres vazia. É uma string sem nenhum caracter (é como um tipo de zero para adicionar a string).

Um bool armazena verdadeiro (true) ou falso (false). O Operador "!" significa negação e transformar verdadeiro em falso e vice-versa.

Relaxe

Não fique preocupado tentando memorizar operadores nesse ponto

Você irá conhece-los porque vai vê-los repetidamente.

Laços executam uma ação repetidamente

Eis algo peculiar sobre a maioria dos grandes programas: eles quase sempre implicam em realizar algumas tarefas repetidamente. É para isto que servem **os laços** - eles dizem ao seu programa para executar um certo conjunto de comandos enquanto alguma condição específica for verdadeira (ou falsa).

As variáveis booleanas são tão importantes, em parte, por causa disto. Um laço usa um teste para determinar se ele deve continuar executando.

```
while (x > 5)
{
    x = x - 3;
}
```

Num laço while (enquanto), todos os comandos dentro das chaves são executados enquanto a condição entre parênteses é verdadeira.

Cada laço for (para) tem a declaração de três parametros. O primeiro cria o laço em si. O comando continuará executando até que o segundo seja verdadeiro. E o terceiro é executado depois de cada volta do laço.

```
for (i = 0; i < 8; i = i + 2)
{
    MessageBox.Show("I'll pop up 4 times");
}
```

Use um trecho de código para escrever laços for simples

Você digitará alguns laços for em instantes e o IDE pode ajudar um pouco a acelerar a criação do código. Digite `for` e pressione a tecla tab duas vezes: o IDE automaticamente insere o código para você. Se você digitar o nome de uma nova variável, ele automaticamente completará o restante do trecho. Pressione a tecla tab novamente e o cursor saltará para a palavra length (comprimento, aqui usado no sentido de "tamanho").

Pressione a tecla tab para fazer o cursor ir até a palavra length. O número de vezes que esse laço executa é determinado pelo que estiver no lugar dessa palavra. Pode-se mudá-la para um número ou variável.

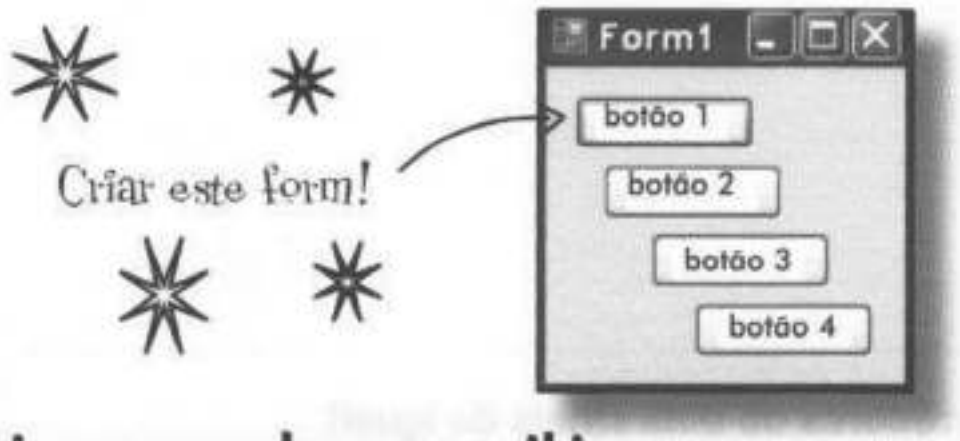
```
for (int i = 0; i < length; i++)
{

}
```

Se você alterar a variável para alguma outra coisa o trecho automaticamente altera as duas outras ocorrências dela.

Hora de começar a codificação

O verdadeiro trabalho de qualquer programa está nas suas declarações. Mas declarações não existe num vácuo. Portanto, vamos definir a fase de criação, e na obtenção alguns códigos escritos. Crie um novo aplicativo Windows Forms.

Isto dirá ao IDE para criar um novo projeto com um formulário em branco e um ponto de entrada. Você pode querer dar-lhe um nome como programa / do capítulo 2 - você criará muitos programas conforme for ler este livro.



Adicione comandos para exibir uma mensagem

Comece clicando duas vezes no primeiro botão. Então, adicione os seguintes seis comandos no método `button1_Click()`. Examine atentamente o código e o resultado que ele produz.

x é uma variável. A parte int diz ao C# que ela armazenará somente números inteiros e o restante do comando atribui-lhe o valor 3.

```
private void button1_Click(object sender, EventArgs e)
{
    // this is a comment
    String name = "Quentin";
    int x = 3;
    x = x * 17;
    double d = Math.PI / 2;
    MessageBox.Show("nome é" + name
      + "\nx is " + x
      + "\nd is " + d);
}
```

Existe uma classe predefinida chamada Math e ela tem um membro chamado PI. Math está contida no namespace System. Logo, o arquivo que armazenará este código precisará de uma linha using System; no início.

O \n é uma seqüência de escape para adicionar uma quebra de linha na caixa de mensagem.

Revisão de Sintaxe

★ Cada comando precisa terminar com um ponto e vírgula.

```
x = x + 1;
```

★ Um comentário de uma única linha começa com duas barras.

```
// esta linha será ignorada
```

★ A maioria dos espaços em branco é ignorada.

```
x   =   3 ;
```

é o mesmo que

```
x = 3;
```

★ Variáveis são declaradas com um nome e um tipo (você aprenderá sobre vários tipos no capítulo 4).

```
int peso;
// peso: numero inteiro
```

★ Classes e métodos precisam ser definidos dentro de um par de abre e fecha chaves.

```
public void vamos()
{
//codigo fantastico aqui
}
```

Comandos if/else tomam decisões

Use **comandos if/else** (se/então) para dizer aos seus programas para fazer certas coisas apenas no caso de as **condições** estabelecidas serem (ou não) verdadeiras. Muitos comandos if/else verificam se duas coisas são iguais. É neste caso que se utiliza o operador ==. Ele difere do operador sinal de igual único (=), que é usado para atribuir valores.

Cada comando if começa com um teste de uma condição.

```
if (someValue == 24)
{
    MessageBox.Show("O valor foi 24.");
}
```

Os comandos dentro das chaves são executados somente se o teste resultar em verdadeiro.

Sempre use dois sinais de igual para comparar valores.

```
if (someValue == 24)
{
    // You can have as many statements
    // as you want inside the brackets
    MessageBox.Show("O valor foi 24.");
} else {
    MessageBox.Show("O valor não foi 24.");
}
```

Comandos if/else são bastante simples. Se a condição resultar verdadeira, o programa executa os comandos entre o primeiro conjunto de chaves. Se não for o caso, ele executa os comandos dentro do segundo conjunto.

Veja bem!

Não se confunda com os operadores de dois sinais de igual!

Você usa um sinal de igual (=) para atribuir o valor para uma variável e os dois sinais (==) para comparações. Você não acreditaria em quantos erros em programas - mesmo alguns cometidos por programadores experientes - são causados por colocar = no lugar de ==. Se você observar o IDE reclamando que "não se pode converter implicitamente o tipo 'int' para 'bool'" (cannot implicitly convert type 'int' to 'bool'), provavelmente, você cometeu este erro.

Estabeleça condições e veja se elas são verdadeiras

Use comandos `if/else para dizer` ao programa fazer certas coisas só quando as `condições` estabelecidas por você forem (ou não) verdadeiras.

Quando você usa um operador condicional para comparar dois números, isto é chamado teste condicional.

Utilize operadores lógicos para checar condições

Você acabou de ver o operador == usado para testar se duas variáveis são iguais. Existem alguns outros operadores também. Não se preocupe em memorizá-los agora - nós os veremos nos próximos capítulos:

- ★ O operador != funciona de forma bem parecida ao ==, exceto pelo fato de ser verdadeiro se duas coisas que você está comparando **não são iguais**.
- ★ Você pode usar > ou < para comparar números e ver se um é maior ou menor do que o outro.
- ★ Os ==, !=, > e < são chamados **operadores condicionais**. Quando você os usa para testar duas variáveis ou valores é chamado de **teste condicional**.
- ★ Você pode combinar testes condicionais individuais num teste longo usando o operador && para AND (operação E lógica) e o operador || para OR (operação OU lógica). Então para checar se i é igual a 3 ou se j é menos do que 5, faça `(i == 3) || (j < 5)`.

Atribua um valor a uma variável e cheque-a

Aqui vemos o código para o segundo botão. É um comando `if/else` que checa uma **variável** inteira chamada x para ver se ela é igual a 10.

Certifique-se de parar seu programa antes de fazer isto - o IDE não deixará que se edite o código enquanto o programa está sendo executado. Você pode pará-lo fechando a janela, usando o botão stop (parar) na barra de ferramentas, ou selecionando Stop Debugging (parar a depuração) no menu Debug.

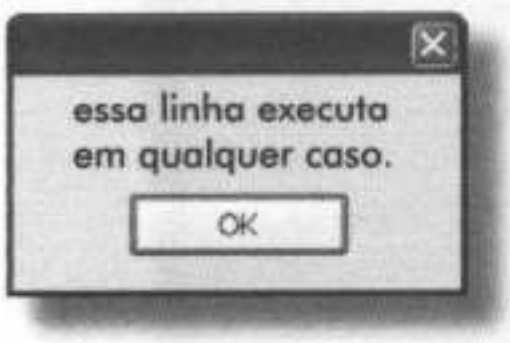
Primeiro criamos uma variável chamada x e fazemo-nas ser igual a 5. Então checamos se ela é igual a 10.

```
private void button2_Click(object sender, EventArgs e)
{
  int x = 5;
  if (x == 10)
  {
    MessageBox.Show("x necessita ser 10");
  }
  else
  {
    MessageBox.Show("x não é 10");
  }
}
```

Aqui está o resultado. Veja se consegue ajustar uma linha do código e fazer a mensagem mudar para "x must be 10" (x deve ser 10).

Acrescente outro teste condicional

O terceiro botão produz o resultado seguinte. Mude duas linhas de código para que ele faça aparecer ambas as caixas de mensagem.



Essa linha checa someValue para ver se ele é menor que 3 e então checa para ter certeza de que name (nome) é Joe.

```
private void button3_Click(object sender, EventArgs e)
   {
   int someValue = 4;
   String name = "Bobbo Jr.";
   if ((someValue < 3) && (name.Equals("Joe")))
   {
      MessageBox.Show("x é 3 e o nome é Joe");
   }
   MessageBox.Show("essa linha executa em qualquer caso.");
}
```

Adicione laços ao seu programa

Aqui está o código para o último botão. Ele possui dois laços. O primeiro é um while, que repete os comandos dentro das chaves enquanto a condição for verdadeira - faça algo enquanto isto for verdadeiro. O segundo é um laço for. Examine e veja como isto funciona.

```
private void button4_Click(object sender, EventArgs e)
{
    int count = 0;
    while (count < 10)
    {
    count = count + 1;
    }
    for (int i = 0; i < 5; i++)
    {
    count = count - 1;
    }
    MessageBox.Show("O resultado é" + count);
}
```

Este laço continua repetindo-se enquanto a variável de contagem for menor que 10.

Isto cria o laço. Ele apenas atribui um valor ao inteiro que será usado nele.

A segunda parte do comando for é o teste. Ele diz enquanto i for menor do que cinco o laço deve continuar.

Aqui é onde o laço realmente faz algo. Neste caso, ele incrementa 1 a i. Então toda vez que o laço executar, ele adicionará mais 1 a i.

Antes de clicar no botão, leia todo o código e tente descobrir o que a caixa de mensagem mostrará. Então clique e veja se você está certo!

Aponte seu lápis

Vamos praticar um pouco mais os testes condicionais e laços. Dê uma olhada no código abaixo. Circule os testes condicionais e complete os espaços para que os comentários descrevam corretamente o código em execução.

```
int result = 0; // esta variável terá o resultado final
int x = 6; // declara uma variável x e da a ela o valor de 6
while (x > 3) {
// executar esses comandos enquanto .....
result = result + x; // adiciona x .....
x = x - 1; // subtrai .....
}
for (int z = 1; z < 3; z = z + 1) {
// comece o laço ao .....
// continue o laço enquanto .....
// apos cada laço .....
result = result + z; // .....
}
// O próximo comando mostrará uma caixa de mensagem que diz
// .....
MessageBox.Show("O resultado é" + result);
```

Completamos o primeiro para você.

Mais a respeito de testes condicionais

Você pode fazer testes condicionais simples checando o valor de uma variável usando um operador comparativo. Eis como se comparam dois números, x e y:

```
x < y (menor que)
x > y (maior que)
x == y (comparação de igualdade, com dois sinais de igual)
```

Esses são os testes usados com mais freqüência.

Espere! Tem uma falha na sua lógica. O que acontece com meu laço se eu escrever um teste condicional que nunca seja falso?

Então seu laço será executado para sempre!

Toda vez que seu programa executar um teste condicional, o resultado será verdadeiro ou falso. Se for verdadeiro, então seu programa passa pelo laço mais uma vez. O código de todos os laços deveria ser feito de forma que, depois que ele executasse um certo número de vezes, retornasse falso. Mas se isso não acontecer ele continuará executando até que você desligue o aplicativo ou o computador!

Algumas vezes chama-se isto de loop infinito.

Aponte seu lápis

Eis alguns laços. Escreva se cada um deles vai se repetir para sempre ou se, eventualmente, ele terminará. Se o laço encerrar, quantas vezes ele iterará?

Laço #1

```
int count = 5;
while (count > 0) {
  count = count * 3;
  count = count * -1;
}
```

Laço #2

```
int i = 0;
while (i == 0) {
  count = count * 3;
  count = count * -1;
}
```

Laço #3

```
int j = 2;
for (int i = 1; i < 100;
     i = i * 2)
{
      j = j - i;
   while (j < 25)
   {
   j = j + 5;
   }
}
```

Laço #4

```
while (true) { int i = 1;}
```

Laço #5

```
int p = 2;
for (int q = 2; q < 32;
     q = q * 2)
{
    while (p < q)
    {
      p = p * 2;
    }
    q = p - q;
}
```

Poder do Cérebro

Você pode pensar num motivo para querer criar um laço que nunca pare de executar?

Aponte seu lápis
Solução

Vamos praticar um pouco mais os testes condicionais e laços. Dê uma olhada no código abaixo. Circule os testes condicionais e complete os espaços para que os comentários descrevam corretamente o código em execução.

```
int result = 0; // esta variável terá o resultado final
int x = 6; // declara uma variável x e dá a ela o valor de 6
while (x > 3) {
// executar esses comandos enquanto x for maior do que 3
result = result + x; // adiciona x na variável result (resultado)
x = x - 1; // subtrai 1 do valor de x
}
for (int z = 1; z < 3; z = z + 1) {
// comece o laço ao declarar uma variável z e atribuir o valor 1 a ela
// continue o laço enquanto z for menor do que 3
// apos cada laço adicione 1 em z
result = result + z; // adicione o valor de z ao resultado
}
// O próximo comando mostrará uma caixa de mensagem que diz
// the result is 18 (o resultado: 18)
MessageBox.Show("O resultado é" + result);
```

Este laço executa duas vezes - primeiro dá-se a z o valor 1 e então uma segunda iteração acontece com z igual a 2. Uma vez que ele chegue a 3, não é mais menor do que 3, então o laço é encerrado.

Aponte seu lápis
Solução

Eis alguns laços. Escreva se cada um deles vai se repetir para sempre ou se, eventualmente, ele terminará. Se o laço encerrar, quantas vezes ele iterará?

Laço #1
Este laço executa uma vez só

Laço #2
Este laço executa para sempre

Laço #3
Este laço executa 7 vezes

Laço #4
Outro laço infinito

Laço #5
Este laço executa 8 vezes

não existem Perguntas Idiotas

P: Cada linha de código que existe está sempre dentro de uma classe?

R: Sim. A cada vez que um programa C# faz alguma coisa, é porque comandos estão sendo executados. Eles são partes de classes, que, por sua vez, são partes de namespaces. Mesmo que pareça que algo não seja um comando dentro de uma classe – como, por exemplo, quando se utiliza o painel de design para atribuir um valor a uma propriedade num objeto num formulário - se procurarmos pelo código veremos que o IDE adicionou ou alterou comandos dentro de alguma classe em algum ponto.

P: Existe algum nome de namespace sem autorização de uso? Existe algum que eu seja obrigado a usar?

R: Sim, existem uns poucos namespaces que não se tem autorização para utilizar. Notou como todas as linhas using no início de seus arquivos de classe sempre começam com System? É porque existe um namespace System usado pelo Framework .NET. É nele que você encontrará todas as ferramentas importantes para adicionar capacidade a seus programas. Como, por exemplo, System.Data, que permite trabalhar com tabelas e bases de dados, e System.Math, que contém funções matemáticas. Mas na maioria dos casos, pode-se escolher qualquer nome para um namespace (desde que seja composto somente por letras, números e sublinhados). Quando você cria um novo programa, o IDE automaticamente escolhe um nome de namespace baseando-se no nome do programa.

P: Eu ainda não entendi porque preciso dessa coisa de classes parciais.

R: Classes parciais permitem distribuir o código de uma classe em mais de um arquivo. O IDE emprega essa técnica quando cria um formulário - ele mantém o código editado pelo usuário num arquivo (como, por exemplo, Form1.cs) e o código é modificado automaticamente em outro (nesse caso, Form1.Designer.cs). No entanto, não é preciso fazer isso com um namespace. Um namespace pode estar distribuído em um, dois, uma dúzia ou mais arquivos. Apenas acrescente a declaração do namespace no início do arquivo e tudo que estiver entre as chaves da declaração pertencerá a este namespace. Mais uma coisa: pode-se ter mais de uma classe num mesmo arquivo. E mais de um namespace também. Você aprenderá muito mais a respeito de classes nos próximos capitulos.

P: Digamos que eu arraste alguma coisa para um formulário e o IDE gere um monte de código automaticamente. O que acontece com este código se eu clico em "desfazer"?

R: A melhor forma de responder a esta pergunta é experimentar! Faça algo que faça com que o IDE gere código para você. Arraste um botão para um formulário, altere propriedades, etc. Então tente desfazer. O que acontece? Bem, para coisas simples o que você deve observar é que o IDE é inteligente o suficiente para desfazer por ele mesmo. Mas em situações mais complexas, como, por exemplo, adicionar uma nova base SQL no seu projeto, você receberá uma mensagem de aviso. O IDE ainda sabe como desfazer a ação, mas pode não ser capaz de refazê-la.

P: Exatamente quão cuidadoso eu preciso ser com o código gerado pelo IDE?

R: Normalmente, deve-se tomar muito cuidado. É muito útil saber o que o IDE está fazendo com o seu código e, de vez em quando, você precisará saber o que existe ali para conseguir resolver um problema sério. Mas, na maioria dos casos, deve ser possível fazer tudo o que se desejar através do IDE.

Pontos de Bala

- Você diz ao seu programa para realizar tarefas usando comandos. Eles são sempre parte de classes e cada classe sempre pertence a um namespace.
- Todos os comandos são encerrados com um ponto e vírgula (;)
- Quando se utilizam as ferramentas visuais no IDE do Visual Studio, código é automaticamente adicionado ou alterado em seu programa.
- Blocos de código são contidos dentro de chaves "`{ }`". Classes, laços, comandos if/else e muitos outros tipos de comandos usam esses blocos.
- Um teste condicional é sempre `verdadeiro` ou `falso`. Utiliza-se ele para determinar quando um laço termina e qual bloco de código executar no caso de um comando if/else.
- Sempre que um programa precisar armazenar algum dado, usa-se uma variável. Use = para atribuir valor a ela, e == para testar se duas são iguais.
- Um laço `while` executa tudo dentro de seu bloco (definido por chaves) enquanto o teste condicional for `verdadeiro`. Se um teste *condicional* for falso, o código do laço while não será executado, e o ponto de execução se deslocará para o código imediatamente após o bloco de código do laço.

Ímãs de geladeira

Parte de um programa em C# está toda embaralhada na porta de geladeira. Você pode ordenar os trechos de código para produzir um código funcional em C# que apresente a caixa de mensagem mostrada? Algumas das chaves caíram no chão e são muito pequenas para pegar, então se sinta livre para adicionar quantas precisar!

O é uma seqüência de caracteres vazia, ou seja, Result ainda não tem nenhum caractere.

Esse ímã não caiu da geladeira...

```
string Result = "";
```

```
MessageBox.Show(Result);
```

```
if (x == 1) {
Result = Result + "d";
x = x - 1;
}
```

```
if (x == 2) {
Result = Result + "b c";
}
```

```
if (x > 2) {
Result = Result + "a";
}
```

```
int x = 3;
```

```
x = x - 1;
Result = Result + "-";
```

```
while (x > 0) {
```

Resultado:

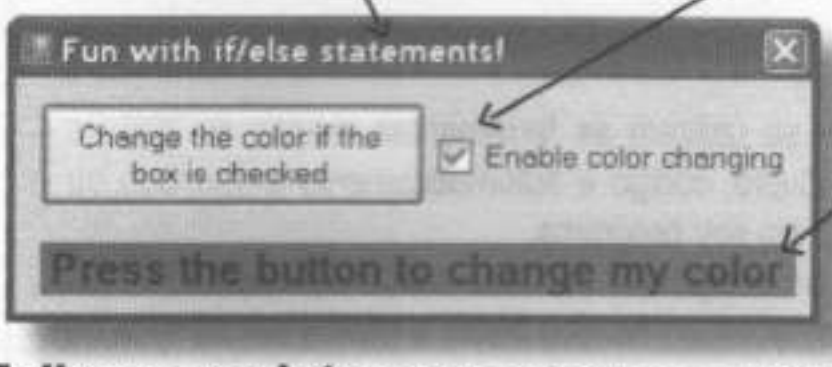
Respostas na página 74.

Passaremos muitos exercícios semelhantes a este no livro inteiro. Daremos a você a resposta após algumas páginas. Se ficar parado sem saber o que fazer, não tenha medo de dar uma olhadinha na resposta - isto não é trapaça!

Exercício

É hora de praticar um pouco com os comandos if/else. Você pode fazer este programa?

Eis o formulário.

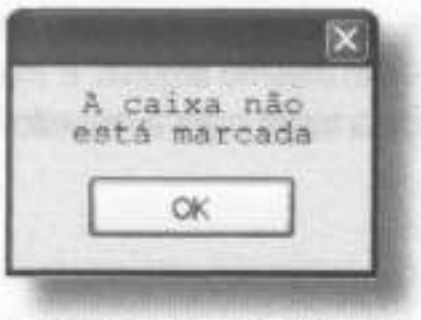


Adicione esta caixa de checagem.
Arraste-a da caixa de ferramentas para o formulário. Use a propriedade Text para mudar o texto (é também esta propriedade que se utiliza para mudar os textos de rótulos e botões).

Isto é um rótulo
Você pode usar as propriedade para alterar o tamanho da fonte e deixá-la negritada. Use a propriedade BackColor (cor de fundo) para deixar o fundo vermelho - escolha "Red" (vermelho) da seleção de cores para a Internet.

Exiba a seguinte mensagem se o usuário clicar o botão, mas a caixa de checagem NÃO estiver marcada.
Se a sua caixa de checagem tiver checkBox1 como nome (você pode mudar a propriedade Name se quiser), então eis o teste condicional para ver se ela está marcada:

```
checkBox1.Checked == true
```



Se o usuário clicar no botão e a caixa de checagem ESTIVER marcada, mude a cor de fundo do rótulo.
Se a cor de fundo do rótulo for vermelha, mude-a para azul quando o botão for clicado. Se ela for azul, mude-a de volta para vermelho. Eis o comando que altera a cor de fundo de um rótulo chamado label1:

```
label1.BackColor = Color.Red;
```

(Dica: O teste condicional para checar se a cor de fundo de um rótulo é vermelha parece muito com o código acima, mas com uma diferença importante.)

Exercício

Vamos fazer algo **chamativo!**

1 **Eis o formulário**

Dica: se você declarar uma variável dentro de um laço for - for(int c = 0; ...) ela só será válida dentro das chaves deste laço. Logo, se você tiver dois laços que usam a mesma variável, pode declará-la duas vezes, uma em cada laço, ou ter uma única declaração fora de ambos.

2 **Faça com que o fundo do formulário fique totalmente psicodélico!**
Quando o botão for clicado, faça com que o fundo do formulário fique trocando para muitas cores. Crie um laço com uma variável c de 0 a 254. Eis o bloco de código que estará dentro das chaves deste laço:

```
this.BackColor = Color.FromArgb(c, 255 - c, c);
Application.DoEvents();
```

Essa linha dirá ao programa para deixar o sistema operacional fazer outras coisas fora do seu programa. Sem ela, seu programa tomaria todos os ciclos da CPU para ele mesmo - e não prestaria atenção em eventos ocasionais (como, por exemplo, o usuário clicando no x para fechar a janela).

Um Colorir impresso!
O .NET tem várias cores predefinidas como Blue (azul) e Red (vermelho), mas ele permite que você crie suas próprias cores usando o método Color.FromArgb(), especificando três valores: um para o componente vermelho, um para o verde e um para o azul.

3 **Deixe-o mais lento**
Adicione a seguinte linha após `Application.DoEvents()`:

```
System.Threading.Thread.Sleep(3);
```

Este comando força uma espera de 3 milissegundos no laço. É parte da biblioteca de comandos do .NET e está no namespace System.Threading.

Você criará muitos aplicativos enquanto lê este livro e terá de dar a cada um deles um nome diferente. Recomendamos chamar este aqui de 2 diversão com comandos if-else, composto do número do capítulo e do texto na barra de título do formulário.

4 **Deixe-o mais suave**
Vamos fazer as cores executarem um ciclo inverso ao inicial. Adicione outro laço em que c vá de 254 até 0. Use o mesmo bloco de código entre chaves do outro laço.

5 **Faça com que ele continue**
Encapsule seus dois laços dentro de outro que os execute continuamente e não pare, de forma que quando o botão for pressionado, o fundo comece a mudar de cor e continue fazendo isso sem parar. (Dica: o laço `while (true)` vai executar para sempre!)

Quando um loop está dentro de outro chamamos de "aninhamento" de loop.

Opa! O programa não pára mais!

Execute seu programa no IDE. Inicie os laços. Agora feche a janela. Espere um pouco - o IDE não voltou para o modo de edição! Ele está agindo como se o programa ainda estivesse executando. Você precisará efetivamente mandar o programa parar usando o botão de stop no IDE (ou selecionando "stop debuggin" no menu Debug).

6 Faça-o parar

Faça o laço que você criou no passo 5 parar quando o programa é fechado. Modifique sua linha externa para o seguinte:

```
While (Visible)
```

Agora execute o programa e clique no "x" no canto superior. A janela fecha e então o programa pára! Exceto... que há um atraso de alguns segundos antes que o IDE volte para o modo de edição.

Quando você estiver checando um valor booleano como Visible (visível) num comando if dentro de um laço, algumas vezes, é tentador testar (Visible==true). Mas você pode deixar o ==true de fora - é suficiente apenas incluir o booleano.

Quando você estiver trabalhando com um formulário ou controle, Visible será verdadeiro enquanto ele estiver sendo exibido. Se você atribuir falso a ele, o formulário ou controle desaparecerão.

Dica: o operador && significa "AND" com ele que você enfileira um monte de testes condicionais para formar um único grande teste que será verdadeiro somente se o primeiro teste for & o segundo também & o terceiro, etc. Ele será útil para resolver este problema.

Você consegue imaginar o que está causando este atraso? Pode corrigir o problema de forma que o programa encerre imediatamente quando se fecha a janela?

Solução do Exercício

É hora de praticar um pouco com os comandos if/else. Você pode fazer esse programa?

Eis o arquivo Form1.cs inteiro do exercício Diversão com comandos if/else. Se mostramos um monte de código, indicaremos a parte que deve ser adicionada usando uma caixa cinza.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;

namespace Fun_with_If_Else
{
   public partial class Form1 : Form
   {
      public Form1()
      {
         InitializeComponent();
      }
```

Aqui está o código para o formulário. Chamamos a solução de Fun with If Else (Diversão com If Else), de forma que o IDE criou o namespace Fun_with_If_Else. Se você deu à sua solução um nome diferente, terá um nome de namespace diferente.

O IDE adicionou um método chamado button1_Click() no seu formulário quando você clicou duas vezes no botão. Ele executa toda vez que o botão é pressionado.

```
      private void button1_Click(object sender, EventArgs e)
      {
         if (checkBox1.Checked == true)
         {
            if (label1.BackColor == Color.Red)
            {
               label1.BackColor = Color.Blue;
            }
            else
            {
               label1.BackColor = Color.Red;
            }
         }
         else
         {
            MessageBox.Show("A caixa não está marcada");
         }
      }
   }
}
```

O comando if externo verifica a caixa de checagem para ver se ela está marcada.

O comando if interno verifica a cor do rótulo. Se no momento ele está vermelho, executa um comando que o deixa azul.

Este comando executa se a cor de fundo do rótulo não for vermelha e, neste caso, faz com que ela retorne para esta cor.

Esta caixa de mensagem aparece se a caixa de checagem não estiver marcada.

Você pode baixar o código de todas as soluções de exercícios neste livro em www.altabooks.com.br

Solução do Exercício

Vamos fazer algo **chamativo!**

Algumas vezes não mostraremos todo o código numa solução, apenas as partes alteradas. Toda a lógica do projeto FlashyThing (literalmente coisa piscante) está no método button1_Click() que o IDE adicionou automaticamente quando você clicou duas vezes no botão painel de design do formulário.

Quando o IDE adicionou este método, também acrescentou um return extra antes do fecha chave. Algumas vezes colocaremos a chave na mesma linha de um comando da mesma forma para poupar espaço - mas o C# não se importa com espaço extra, logo isto é perfeitamente válido.

```
private void button1_Click(object sender, EventArgs e) {
  while (Visible) {
    for (int c = 0; c < 254 && Visible; c++) {
      this.BackColor = Color.FromArgb(c, 255 - c, c);
      Application.DoEvents();
      System.Threading.Thread.Sleep(3);
    }
    for (int c = 254; c >= 0 && Visible; c--) {
      this.BackColor = Color.FromArgb(c, 255 - c, c);
      Application.DoEvents();
      System.Threading.Thread.Sleep(3);
    }
  }
}
```

O laço externo fica executando enquanto o formulário estiver visível. Logo que ele seja fechado, o Visible é alterado para falso e o while deve parar de executar.

O primeiro laço for faz com que as cores sejam alteradas de uma forma e o segundo reverte essa ordem para deixar a variação mais suave.

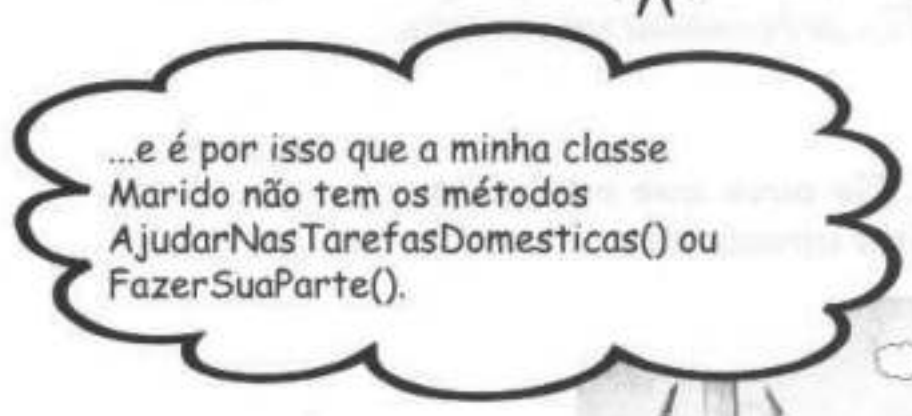
Resolvemos o problema do atraso extra usando o operador && para fazer com que cada um dos laços também chegue a propriedade Visible. Desta forma, os laços encerram assim que Visible se torna falsa.

Você conseguiu descobrir o que estava causando o atraso? Conseguiu resolver o problema de forma que o programa terminasse imediatamente quando você fechasse a janela?

O atraso acontece porque os laços for precisam se encerrar antes que o while possa checar se Visible ainda é verdadeira. Pode-se resolver o problema adicionando && Visible == true no teste condicional em cada laço for.

Seu código está um pouco diferente do nosso? Existe mais de uma forma de resolver qualquer problema de programação, como, por exemplo, usar laços while no lugar de laços for. Se seu programa funciona, então você fez o exercício corretamente!

Fazendo o código ter sentido



Cada programa que você escreve resolve um problema.

Quando você está desenvolvendo um programa é sempre uma boa idéia começar pensando sobre qual *problema* ele deveria resolver. É por isso que ***objetos*** são tão úteis. Eles permitem estruturar o código baseando-se no problema que se está resolvendo. Assim, você pode usar seu tempo *pensando* sobre o problema que precisa ser enfrentando, em vez de ficar atolado na mecânica de escrever código. Quando se usa um objeto corretamente, acaba-se com código que é *intuitivo* de escrever e fácil de ler e alterar.

Como Miguel pensa a respeito de seus problemas

Miguel é um programador que vai a uma entrevista de emprego. Ele mal pode esperar para mostrar suas habilidades com o C#, mas primeiro tem que chegar lá - e está prestes a se atrasar!

1 Miguel decide qual caminho vai usar para chegar ao local da entrevista.

Pegarei a ponte na rua 31, continuarei pela avenida Liberdade e irei pelo Campo Belo.

Miguel pensou no local que queria chegar e então determinou um caminho.

2 Ainda bem que ele estava com o rádio ligado. Ele ouve que o trânsito está muito lento no caminho escolhido e isso iria atrasá-lo!

Miguel recebe novas informações a respeito de uma das ruas que ele precisa evitar.

Aqui é Francisco Grito para o programa Relatório de trânsito nós vemos tudo. Parece que um acidente envolvendo três carros na avenida Liberdade faz com que o trânsito fique quase parado por toda a extensão da avenida até a rua 32.

3 Miguel imagina um novo caminho para chegar à entrevista em tempo.

Agora ele imaginou um novo caminho para chegar ao local da entrevista.

Sem problemas. Se eu pegar a Estrada 28, ainda chego a tempo!

Como o sistema de navegação do carro de Miguel pensa sobre este problema

Miguel desenvolveu seu próprio sistema de navegação GPS, que ele usa para ajudá-lo a andar pela cidade.

```
SetDestination("Fifth Ave & Penn Ave");
string route;
route = GetRoute();
```

O sistema de Navegação estabelece um ponto de destino e calcula uma rota.

Eis o resultado do método GetRoute() - uma string com as instruções que Miguel deveria seguir.

```
"Pegue a ponte da rua 31 até a avenida Liberdade, até o Campo Belo"
```

Navigator (Navegador)

SetDestination() (Informa Destino)
ModifyRouteToAvoid() (Modifique Rota Para Evitar)
ModifyRouteToInclude() (Modifique Rota Para Incluir)
GetRoute() (Retorna Rota)
GetTimeToDestination() (Retorna Tempo Até Destino)
TotalDistance() (Distância Total)

Eis um diagrama de classes no programa de Miguel. Ele mostra o nome em cima e os métodos embaixo.

O sistema de navegação recebe novas informações a respeito de uma rua que ele deveria evitar.

```
ModifyRouteToAvoid("Liberty Ave");
```

Agora ele pode calcular uma nova rota até o ponto de destino.

```
string route;
route = GetRoute();
```

```
"Pegue a Estrada 28 até a ponto do Parque Alto, até a praça dos Ingleses"
```

GetRoute() informa uma nova rota que não inclui a rua que Miguel quer evitar.

O sistema de Navegação de Miguel resolve o problema da mesma forma que ele

A classe de navegação de Miguel tem métodos para criar e modificar rotas

A classe de navegação de Miguel tem métodos onde toda a ação acontece. Mas diferente do método `button_Click()` nos formulários que você desenvolveu, todos eles concentram-se em torno de um único problema: criar uma rota dentro de uma cidade. É por isso que Miguel agrupou-os todos dentro de uma mesma classe e chamou-a de `Navigator (navegador)`.

Miguel projetou sua classe Navigator para facilitar criar e modificar rotas. Para conseguir um caminho a seguir, o programa de Miguel chama o método `SetDestination()` (estabelece destino) e então GetRoute() (retorna rota), para conseguir as instruções como uma seqüência de caracteres. Se for preciso modificar a rota, o programa chama o método `ModifyRouteToAvoid()` (modifique rota para evitar) para alterá-la de forma a evitar uma certa rua e então chama `GetRoute()` novamente para conseguir as novas instruções.

```
public class Navigator() {
  public void SetDestination(string destinationName) { ... };
  public void ModifyRouteToAvoid(string streetName) { ... };
  public string GetRoute() { ... };
}
```

Miguel escolheu nomes para os métodos que fariam sentido para alguém que estive pensando em como criar uma rota através de uma cidade.

*Este é o tipo de **retorno do método**. Os comandos que chamarem GetRoute() podem armazenar o valor de retorno numa variável deste tipo e ela conterá as instruções. Quando o valor de retorno é **void** (vazio) o método não retorna nada.*

```
string route;
route = GetRoute();
```

Alguns métodos têm um valor de retorno

Todos os métodos são feitos de comando que fazem coisas. Alguns apenas executam seus comandos e encerram-se. Mas outros tem um **valor de retorno**, ou seja, um valor calculado ou gerado dentro do método e enviado de volta para o comando que chamou este método. O tipo do valor de retorno (como `string` ou `int`) é chamado de **tipo de retorno**.

O comando **return** faz com que o método se encerre imediatamente. Se o seu método não tiver um valor de retorno – ou seja, ele é declarado com tipo de retorno **void** , o comando return apenas deve ser seguido de um ponto e vírgula e não é obrigatório que ele exista dentro do método. Mas se o método tiver um tipo de retorno ele obrigatoriamente deve ter o comando `return`.

*Eis um exemplo de um método que tem um tipo de retorno - ele retorna um int. O método usa os dois **parâmetros** para calcular o resultado e usa o comando **return** para passar o valor de volta para o comando que o chamou.*

```
public int MultiplyTwoNumbers(int firstNumber, int secondNumber) {
    int result = firstNumber * secondNumber;
    return result;
}
```

Eis um comando que chama um método para multiplicar dois números. Ele retorna um int:

```
int myResult = MultiplyTwoNumbers(3, 5);
```

Métodos podem ter como parâmetros valores como 3 ou 5, por exemplo. Mas você também pode passar variáveis para eles.

Use o que você aprendeu para desenvolver um aplicativo simples

Vamos associar um formulário a uma classe e fazer seus botões chamarem métodos nela.

Faça isso!

1. Crie um novo projeto de Aplicativo Windows no IDE. Então adicione um arquivo de classe chamado `Talker.cs` (falante) clicando com o botão direito no projeto no Navegador de Solução e selecionando "class..." no menu Add (adicionar). Quando você dá para a sua nova classe o nome "Talker.cs", o IDE automaticamente nomeia a nova classe dentro do arquivo como Talker. Então ele abrirá o novo arquivo numa nova aba dentro do IDE.

2. Adicione using `System. Windows.Forms;` no topo do arquivo de classe. Em seguida adicione o seguinte código na classe:

```
class Talker {
  public static int BlahBlahBlah(string thingToSay, int numberOfTimes) {
    string finalString = "";
    for (int count = 1; count <= numberOfTimes; count++) {
        finalString = finalString + thingToSay + "\n";
    }
    MessageBox.Show(finalString);
    return finalString.Length;
  }
}
```

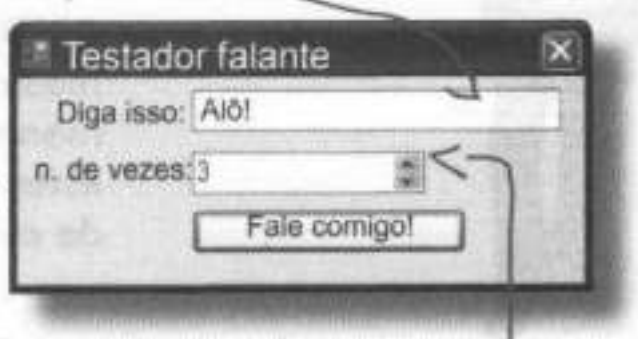
Este comando declara uma variável finalString e atribui seu valor como vazio.

O valor de retorno do método BlahBlahBlah() é um inteiro que informa o tamanho total da mensagem mostrada. Você pode chamar .Length (comprimento) em qualquer seqüência de caracteres para descobrir seu tamanho.

A nova classe tem um método chamado `BlahBlahBlah()` que precisa de dois parâmetros. O primeiro é uma string que lhe diz algo que ele deve falar e o segundo é o número de vezes que ele deve repetir o que diz. Quando ele é chamado, mostra um diálogo com a mensagem repetida o número de vezes solicitado. Seu valor de retorno é o tamanho da mensagem. Este método precisa de uma string para seu parâmetro thingToSay (coisa a dizer) e um número para o seu outro parâmetro numberOfTimes (número de vezes). Ele receberá os parâmetros de um formulário que servirá para que o usuário entre um texto usando um controle **TextBox** (caixa de texto) e um número usando um controle **NumericUpDown** (literalmente "número para cima e para baixo").

Atribua Alo! como valor padrão para a caixa de texto usando sua propriedade Text.

3 Adicione o seguinte formulário ao seu projeto:



Então clique duas vezes no botão e faça com que ele execute o seguinte código:

```
int len = Talker.BlahBlahBlah(textBox1.Text, (int) numericUpDown1.Value);
MessageBox.Show("O tamanho da mensagem é" + len);
```

Este é um controle do tipo NumericUpDown. Atribua o valor 1 para a sua propriedade Minimum, 10 para a Maximum, e 3 para o Valor.

4 Agora execute seu programa! Clique no botão e você verá ele mostrar duas caixas de mensagem. A classe mostra a primeira e o formulário, a segunda.

O método BlahBlahBlah() mostra esta caixa de mensagem baseando-se em seus parâmetros:

Quando o método retorna um valor, o formulário mostra esta:

Miguel tem uma idéia

A entrevista foi ótima! Mas o trânsito de manhã fez Miguel pensar sobre melhorias para seu navegador.

Seria ótimo se pudesse comparar algumas rotas para determinar a mais rápida...

Ele poderia criar três classes Navigator diferentes...

Miguel poderia copiar e colar o código da classe Navigator em duas outras. Assim, seu programa poderia conter três rotas ao mesmo tempo.

Navigator

SetDestination()
ModifyRouteToAvoid()
ModifyRouteToInclude()
GetRoute()
GetTimeToDestination()
TotalDistance()

Navigator2

SetDestination()
ModifyRouteToAvoid()
ModifyRouteToInclude()
GetRoute()
GetTimeToDestination()
TotalDistance()

Esta caixa é um diagrama de classe. Ele lista todos os métodos numa classe e é uma forma fácil de ver tudo o que ela faz.

Navigator3

SetDestination()
ModifyRouteToAvoid()
ModifyRouteToInclude()
GetRoute()
GetTimeToDestination()
TotalDistance()

Ei, isso não pode estar certo! E seu eu quiser alterar um método? Então terei que fazer isto três vezes.

Exato! Manter três cópias do mesmo seria código realmente complicado. Muitos problemas a ser resolvidos implicam em representar uma mesma coisa muitas vezes. Neste caso, um monte de rotas. Mas poderiam ser um monte de turbinas, cães, arquivos de música ou qualquer outro exemplo. Todos estes programas teriam algo em comum: sempre precisariam tratar o mesmo tipo de coisa da mesma forma, não importa quantas fossem em cada caso.

Miguel pode usar objetos para resolver o problema

Objetos são a ferramenta do C# para trabalhar com várias coisas semelhantes. Miguel pode usar objetos para desenvolver sua classe Navigator apenas uma única vez, mas usar *quantas cópias quiser* num programa.

Esta é a classe Navigator no programa de Miguel. Ela lista todos os métodos que um objeto Navigator pode usar.

Navigator

SetDestination()
ModifyRouteToAvoid()
ModifyRouteToInclude()
GetRoute()
GetTimeToDestination()
TotalDistance()

Miguel precisou comparar três rotas diferentes ao mesmo tempo, de forma que usou três objetos Navigator simultaneamente.

Tudo o que você precisa para criar um objeto é a palavra chave new (novo) e o nome de uma classe.

```
Navigator navigator1 = new Navigator();
navigator1.SetDestination("Fifth Ave & Penn Ave");
string route;
route = navigator1.GetRoute();
```

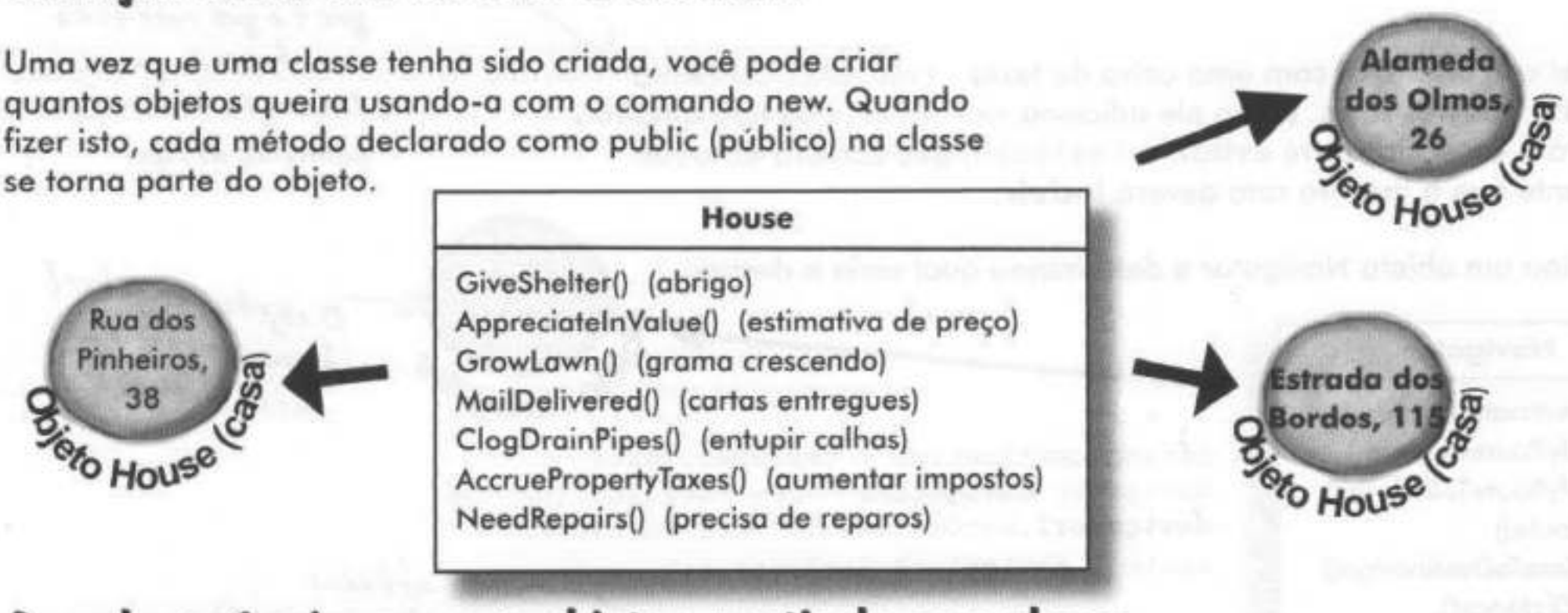
Agora você pode usar o objeto! Quando você cria um objeto a partir de uma classe, ele tem todos os métodos dela.

Você pode usar uma classe para construir um objeto

Uma classe é como uma planta de uma casa para um objeto. Se você quiser construir cinco casas idênticas num empreendimento de construção no subúrbio, você não pedirá ao arquiteto para desenhar cinco cópias idênticas de plantas. Você usará somente uma para construir as cinco casas.

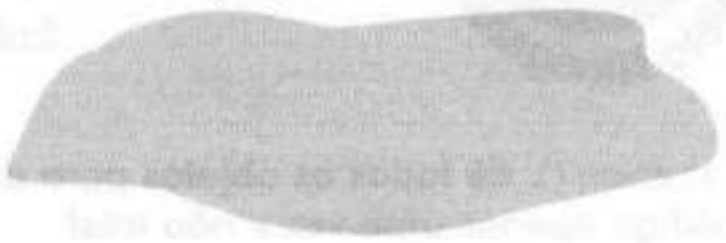
Um objeto obtém seus métodos de sua classe

Uma vez que uma classe tenha sido criada, você pode criar quantos objetos queira usando-a com o comando new. Quando fizer isto, cada método declarado como public (público) na classe se torna parte do objeto.



Quando você cria um novo objeto a partir de uma classe, ele é chamado de uma instância desta classe.

Adivinhe só... você já sabe isto! Cada item na caixa de ferramentas é uma classe: existe uma classe `Button`, uma `TextBox`, uma `Label`, etc. Quando você arrasta um botão da caixa de ferramentas, o IDE automaticamente cria uma instância da classe `Button` e a chama, por exemplo, de `button1`. Quando você arrasta um segundo botão, uma outra instância chamada de, nesse caso, `button2`, é criada. Cada instância de `Button` tem seus próprios métodos e propriedades. Mesmo assim todos os botões comportam-se exatamente da mesma forma, já que são instâncias da mesma classe.



Antes: Eis uma figura da memória de seu computador quando o seu programa inicia.

```
House 115MapleDrive = new House();
```

Seu programa executa um comando new:

Depois: Agora existe uma instância da classe House (casa) na memória.

Faça isso!

Veja você mesmo!

Abra qualquer projeto que use um botão button1 e use o IDE para procurar por todo o projeto por **"new button1"**. Você encontrará o código que o IDE adicionou ao arquivo de definição do formulário para criar a instância da classe `Button`.

ins-tância, substantivo.
Um exemplo ou ocorrência de algo. *A característica de procurar-e-substituir do IDE encontra cada* ***instância*** *de uma palavra e a altera para outra.*

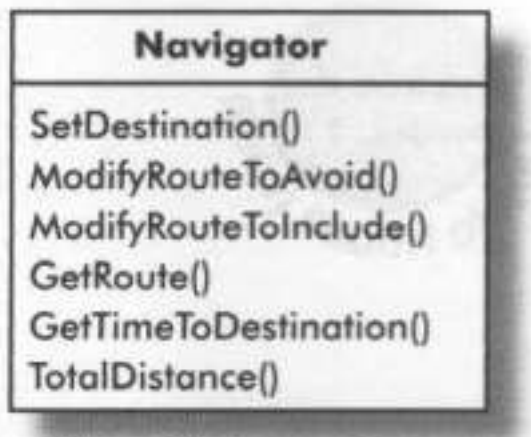
Uma solução melhor... o que os objetos conseguiram para você!

Miguel faz um novo programa de comparação de rotas que usa objetos para encontrar a menor entre três rotas para o mesmo destino. Eis como ele desenvolver o aplicativo.

GUI é a sigla para Graphical User Interface (Interface Gráfica de Usuário), que é o que você está construindo quando faz um formulário no painel de design.

1. Miguel cria uma GUI com uma caixa de texto - `textBox1` contém o **destino** das três rotas. Então ele adiciona `textBox2`, que tem uma rua cuja rota específica deve **evitar**, e `textBox3`, que conterá uma rua diferente que a terceira rota deverá **incluir**.

2. Ele criou um objeto Navigator e determinou qual seria o destino.



O objeto navigator1 é uma instância da classe Navigator.

```
String destination = textBox1.Text;
Navigator navigator1 = new Navigator()
navigator1.SetDestination(destination);
route = navigator1.GetRoute();
```

Os métodos SetDestination(), ModifyRouteToAvoid() e ModifyRouteToInclude() têm um parâmetro que é uma String.

3. Em seguida ele adicionou um segundo objeto Navigator chamado `navigator2`. Ele chama o método `SetDestination()` para determinar o destino e então o `ModifyRouteToAvoid()` (modificar rota para evitar).

4. O terceiro objeto Navigator é chamado de `navigator3`. Miguel determina o destino e então chama o método `ModifyRouteToInclude()`.

Criar um novo objeto a partir de uma classe é chamado de criar uma instância desta classe.

5. Agora Miguel pode chamar os métodos `TotalDistance()` de todos os objetos para determinar qual é a rota mais curta. E você teve que criar o código apenas uma vez e não três!

Aponte seu lápis

Siga os mesmos passos que Miguel seguiu na página anterior para escrever o código para criar objetos Navigator e chamar seus métodos.

```
String destination = textBox1.Text;
String route2StreetToAvoid = textBox2.Text;
String route3StreetToInclude = textBox3.Text;

Navigator navigator1 = new Navigator()
navigator1.SetDestination(destination);
int distance1 = navigator1.TotalDistance();
```

Adiantamos um pouco as coisas pra você. Eis o código que Miguel escreveu para obter os destinos e nomes de ruas das caixas de texto.

E aqui está o código para criar o objeto Navigator, atribuir um destino e recuperar a distância.

1. Crie o objeto **navigator2**, atribua-lhe o destino, chame seu método **ModifyRouteToAvoid()** e guarde o resultado de **TotalDistance()** numa variável inteira chamada **distance2**.

```
Navigator navigator2 = .....
navigator2. .....
navigator2. .....
int distance2 = .....
```

2. Crie o objeto **navigator3**, atribua o destino, chame **ModifyRouteToInclude()** e guarde o resultado de **TotalDistance()** numa variável inteira chamada **distance3**.

.....

.....

.....

.....

O método predefinido do C# Math.Min() compara dois números e retorna o menor. Miguel usou-o para achar a menor distância até o destino.

```
int shortestDistance = Math.Min(distance1, Math.Min(distance2, distance3));
```

Aponte seu lápis Solução

Siga os mesmos passos que Miguel seguiu na página anterior para escrever o código para criar objetos Navigator e chamar seus métodos.

```
String destination = textBox1.Text;
String route2StreetToAvoid = textBox2.Text;
String route3StreetToInclude = textBox3.Text;

Navigator navigator1 = new Navigator()
navigator1.SetDestination(destination);
int distance1 = navigator1.TotalDistance();
```

Adiantamos um pouco as coisas para você. Eis o código para obter os destinos e nomes de ruas das caixas de texto, junto com o para criar o primeiro objeto Navigator, atribuir um destino e recuperar a distância.

1. Crie o objeto **navigator2**, atribua-lhe o destino, chame seu método **ModifyRouteToAvoid()** e guarde o resultado de **TotalDistance()** numa variável inteira chamada **distance2**.

```
Navigator navigator2 = new Navigator()
navigator2.SetDestination(destination);
navigator2.ModifyRouteToAvoid(route2StreetToAvoid);
int distance2 = navigator2.TotalDistance();
```

2. Crie o objeto **navigator3**, atribua o destino, chame **ModifyRouteToInclude()** e guarde o resultado de **TotalDistance()** numa variável inteira chamada **distance3**.

```
Navigator navigator3 = new Navigator()
navigator3.SetDestination(destination);
navigator3.ModifyRouteToInclude(route3StreetToInclude);
int distance3 = navigator3.TotalDistance();
```

O método predefinido do C# Math.Min() compara dois números e retorna o menor. Miguel usou-o para achar a menor distância até o destino.

```
int shortestDistance = Math.Min(distance1, Math.Min(distance2, distance3));
```

Espere aí! Já escrevi algumas classes até agora, mas não tinha usado "new" para criar uma instância ainda! Isto quer dizer que posso chamar métodos sem ter que criar objetos?

Sim! É por isso que você usou a palavra chave `w` em seus métodos.
Dê uma outra olhada na declaração para a classe Talker que você criou algumas páginas atrás:

```
class Talker
{
  public static int BlahBlahBlah(String thingToSay, int numberOfTimes)
  {
    string finalString = "";
```

Quando você chamou os métodos não criou uma nova instância de Talker. Apenas fez isso:

```
Talker.BlahBlahBlah("Hello hello hello", 5);
```

É assim que se chamam métodos estáticos e você esteve fazendo isso até agora. Se você tirar a palavra chave `static` da declaração do método `BlahBlahBlah()`, deverá criar uma instância de Talker() para poder chamá-lo. Tirando esta distinção, métodos estáticos são exatamente iguais a métodos de objetos. Você pode passar parâmetros para eles, eles podem retornar valores e eles existem dentro de classes.
Há uma outra coisa que você pode fazer com a palavra chave `static`. Você pode marcar toda uma **classe** como static e então todos os seus métodos **deverão ser** estáticos também. Se você tentar adicionar um método não estático numa classe marcada com static, não conseguirá compilar seu código.

não existem Perguntas Idiotas

P: Quando penso em algo como "estático", penso em alguma coisa que não se modifica. Ou seja, métodos não estáticos podem mudar, mas os estáticos não? Eles comportam-se de forma diferente?

R: Não, tanto os métodos estáticos como os não estáticos são exatamente iguais. A única diferença é que os estáticos não precisam de uma instância, enquanto que os não estáticos exigem uma. Muitas pessoas têm problemas para lembrar-se disso, porque a palavra "estático" não é realmente muito intuitiva.

P: Então não posso usar minha classe até criar uma instância de um objeto?

R: Você pode usar seus métodos estáticos. Mas se tiver métodos sem esta qualidade, precisará de uma instância antes de poder usá-los.

P: Então porque eu usaria um método que precisa de uma instância? Porque não fazê-los todos estáticos?

R: Porque se você tem um objeto que está sendo usado para registrar certos dados - como as instâncias de cada um dos objetos Navigator de Miguel estavam sendo usada para manter registro de rotas diferentes - então pode usar os métodos de cada instância para trabalhar com os dados próprios de cada uma delas. Então quando Miguel chamou o método ModifyRouteToAvoid() na instância navigator2, afetou somente a rota armazenada naquele objeto em particular. Não afetou navigator1 ou navigator3. É assim que ele foi capaz de trabalhar com três diferentes rotas ao mesmo tempo - e o seu programa pode manter registro de todas elas.

P: Então como uma instância mantém registro de seus dados?

R: Vire a página e descubra!

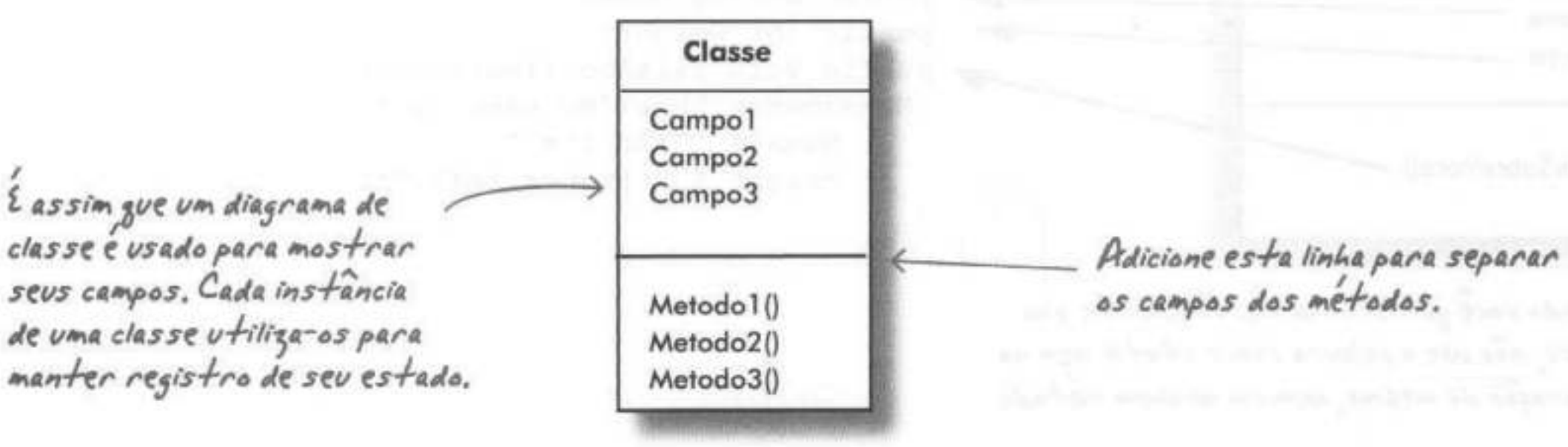
Uma instância usa campos para manter registro sobre coisas

Você mudou o texto num botão alterando sua propriedade Text no IDE. Quando você fez isto, o IDE adicionou código como o seguinte no painel de design:

```
button1.Text = "texto para o botão";
```

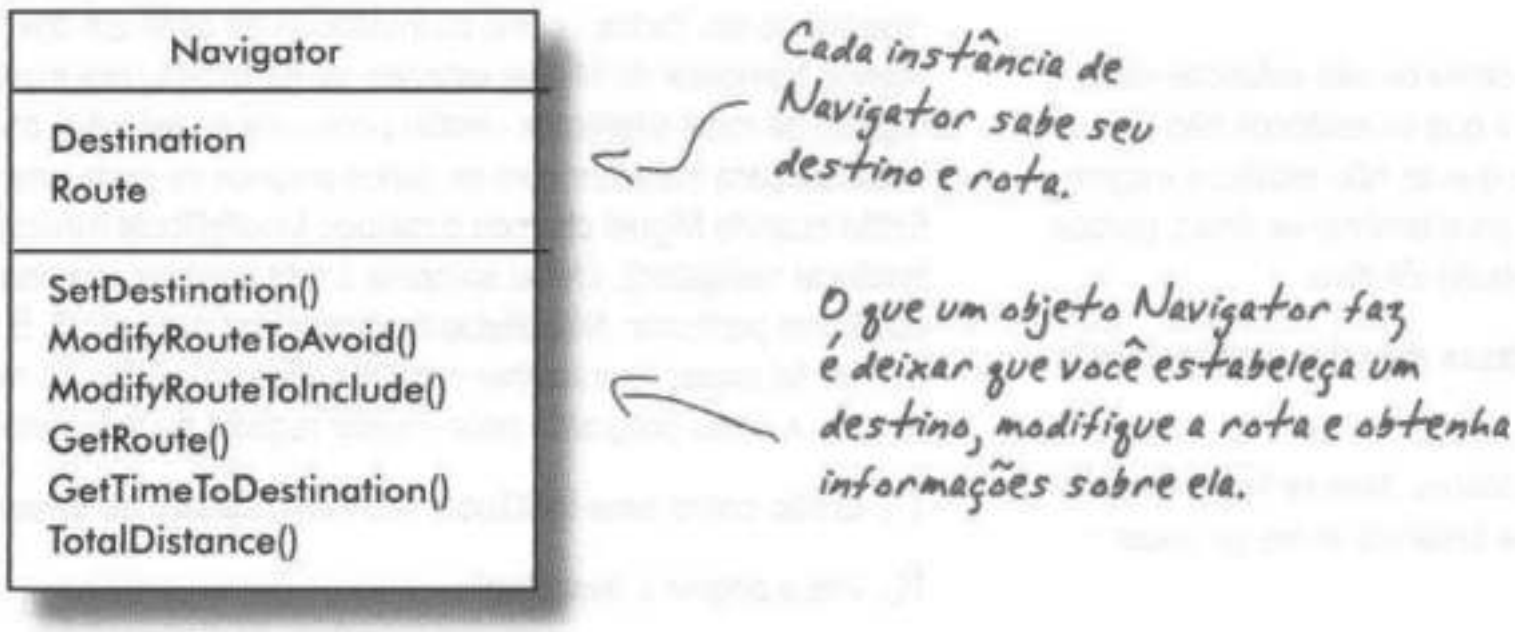
Tecnicamente se está atribuindo valor a uma **propriedade**. Estas são um tipo especial de campo - mas vamos chegar nisto daqui a pouco.

Agora você sabe que `button1` é uma instância da classe `Button`. O que esse código faz é modificar um **campo** na instância `button1`. Você pode adicionar os campos num diagrama de classe - só precisa desenhar uma linha horizontal no meio dele. Os campos são colocados acima desta linha e os métodos, abaixo.



Métodos são o que o objeto faz. Campos são o que ele sabe.

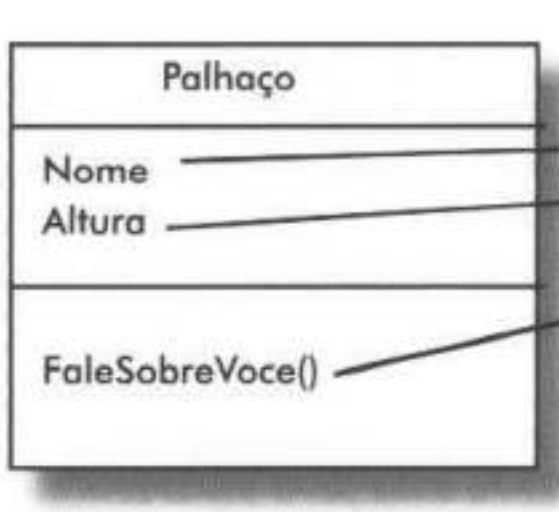
Quando Miguel criou três instâncias da classe `Navigator`, seu programa instanciou três objetos. Cada um deles foi usado para manter registro de uma rota diferente. Quando o programa criou a instância `navigator2` e chamou seu método `SetDestination()`, atribuiu o destino apenas àquela instância. Isso não afetou `navigator1` nem `navigator3`.



O comportamento de um objeto é definido pelos seus métodos e ele usa seus campos para manter registro de seu estado.

Vamos criar algumas instâncias!

É fácil adicionar campos em suas classes. Apenas declare-os fora de qualquer método. Agora cada instância tem sua própria cópia dessas variáveis.




```
public class Clown {
  public String Name;
  public int Height;
  public void TalkAboutYourself() {
   MessageBox.Show("My name is "
     + Name + " and I'm "
     + Height + " inches tall.");
  }
}
```

Lembre-se, quando você notar um void escrito na frente de um método, quer dizer que ele não retorna nada.

Quando você quiser criar instâncias de sua classe, não use a palavra chave static nem na declaração da mesma, nem em nenhum método.

*Lembre-se, o operador *= diz ao C# para pegar o que quer que esteja à esquerda dele e multiplicar pelo que estiver à direita.*

Aponte seu lápis

Escreva os conteúdos de cada mensagem que será exibida depois da execução do comando ao lado dela.

```
Clown oneClown = new Clown();
oneClown.Name = "Boffo";
oneClown.Height = 14;
oneClown.TalkAboutYourself();
```

"Meu nome é _____ e eu tenho _____ de altura."

```
Clown anotherClown = new Clown();
anotherClown.Name = "Biff";
anotherClown.Height = 16;
anotherClown.TalkAboutYourself();
```

"Meu nome é _____ e eu tenho _____ de altura."

```
Clown clown3 = new Clown();
clown3.Name = anotherClown.Name;
clown3.Height = oneClown.Height - 3;
clown3.TalkAboutYourself();
```

"Meu nome é _____ e eu tenho _____ de altura."

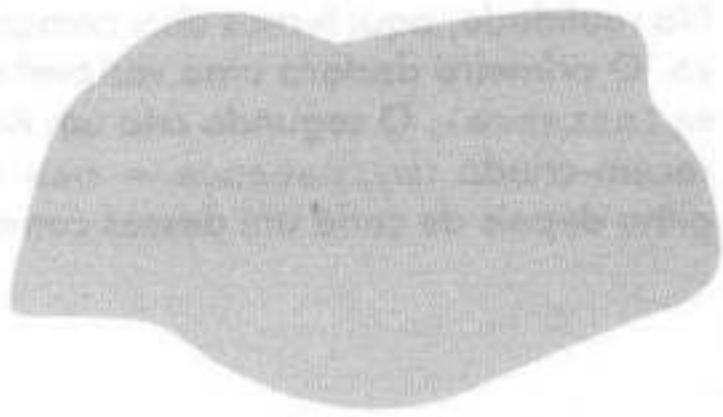
```
anotherClown.Height *= 2;

anotherClown.TalkAboutYourself();
```

"Meu nome é _____ e eu tenho _____ de altura."

Graças à memória

Quando seu programa cria um objeto, ele existirá numa parte da memória do computador chamada de pilha (heap, em inglês). Quando seu código cria um objeto com um comando new, o C# imediatamente reserva um espaço na pilha para que ele possa armazenar os dados daquele objeto.

Eis uma figura da pilha antes do programa começar. Note que ela está vazia.



Vamos olhar com cuidado o que aconteceu aqui.

Aponte seu lápis Solução

Escreva os conteúdos de cada mensagem que será exibida depois da execução do comando ao lado dela.

```
Clown oneClown = new Clown();
oneClown.Name = "Boffo";
oneClown.Height = 14;
oneClown.TalkAboutYourself();

Clown anotherClown = new Clown();
anotherClown.Name = "Biff";
anotherClown.Height = 16;
anotherClown.TalkAboutYourself();

Clown clown3 = new Clown();
clown3.Name = anotherClown.Name;
clown3.Height = oneClown.Height - 3;
clown3.TalkAboutYourself();

anotherClown.Height *= 2;

anotherClown.TalkAboutYourself();
```

Cada um dos comandos new criou uma instância da classe Clown (palhaço) reservando um pedaço da memória na pilha para ele e preenchendo esse espaço com os dados respectivos.

"Meu nome é Boffo e eu tenho 14 de altura."

"Meu nome é Biff e eu tenho 16 de altura."

"Meu nome é Biff e eu tenho 11 de altura."

"Meu nome é Biff e eu tenho 32 de altura."

Quando seu programa cria um novo objeto, ele é adicionado à pilha.

O que passa pela mente de seu programa

Aqui está como seu programa cria a nova instância da classe Clown.

```
Clown myInstance = new Clown();
```

Na realidade, aqui temos dois comandos combinados em um só. O primeiro declara uma variável do tipo Clown (`Clown myInstance`). O segundo cria um novo objeto e atribui-o à variável recém-criada (`myInstance = new Clown();`). Eis a aparência da pilha depois de cada um desses comandos:

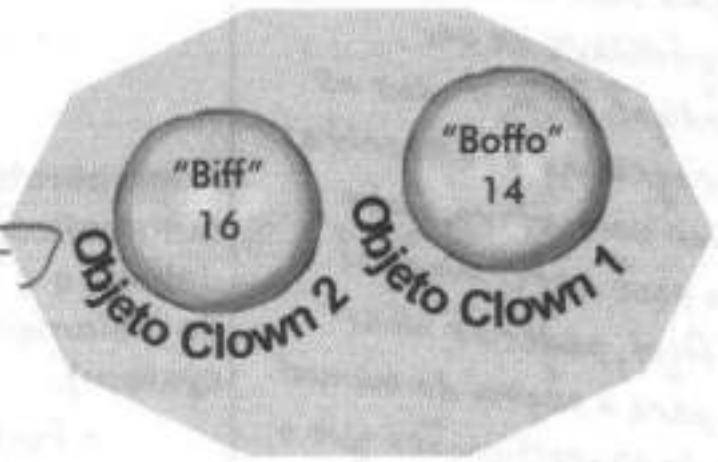
```
1  Clown oneClown = new Clown();
   oneClown.Name = "Boffo";
   oneClown.Height = 14;
   oneClown.TalkAboutYourself();
```

Este objeto é uma instância da classe Clown (palhaço).

O primeiro objeto é criado e seus campos recebem valores.

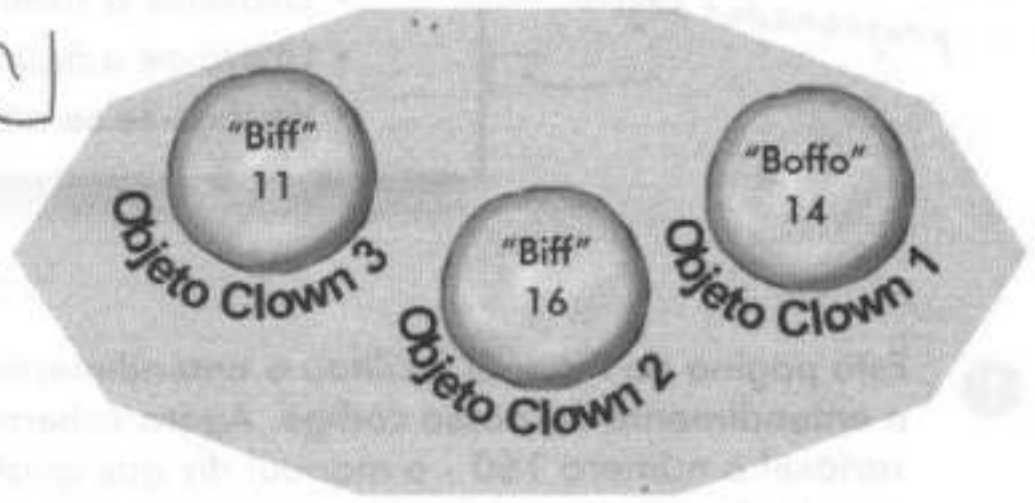
```
2  Clown anotherClown = new Clown();
   anotherClown.Name = "Biff";
   anotherClown.Height = 16;
   anotherClown.TalkAboutYourself();
```

Estes comandos criam um segundo objeto e preenchem-no com dados.



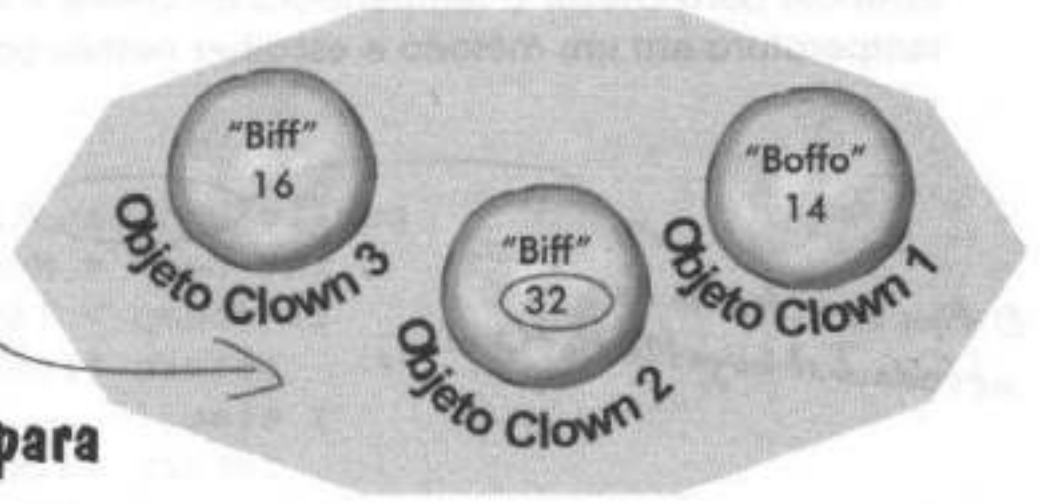

```
3  Clown clown3 = new Clown();
   clown3.Name = anotherClown.Name;
   clown3.Height = oneClown.Height - 3;
   clown3.TalkAboutYourself();
```

Então o terceiro objeto Clown é criado e povoado (ou seja, os dados são preenchidos).




```
4  anotherClown.Height *= 2;
   anotherClown.TalkAboutYourself();
```

Não existe comando new, ou seja, estes comandos não criam um novo objeto; eles apenas modificam um que já está na memória.



Você pode usar nomes de classes e métodos para deixar seu código mais intuitivo

Quando você insere código num método, está fazendo uma escolha sobre como estruturar seu programa. Vai usar um único método? Mais de um? É preciso um método, afinal de contas? As escolhas sobre métodos podem deixar seu programa muito mais intuitivo - ou, se você não tomar cuidado, muito mais confuso.

1 Eis um belo e compacto trecho de código. Ele vem de um programa de controle que faz com que uma máquina de fazer barras doces funcione.

```
int t = m.chkTemp();
if (t > 160) {
  T obj = new T();
  tb.clsTrpV(2);
  ics.Fill();
  ics.Vent();
  m.airsyschk();
}
```

O método chkTemp() retorna um inteiro... mas o que ele faz?

O método clsTrpV() tem um parâmetro, mas não sabemos para que ele serve.

"obj", "ics" e "m" são nomes péssimos! Não temos como imaginar o que eles fazem. E para que serve a classe T()?

Dê uma segunda olhada neste código. Você consegue determinar para que ele serve?

2 Estes comandos não dão nenhuma pista sobre porque o código está fazendo o que está. Neste caso, o programador ficou feliz com o resultado porque conseguiu fazer tudo o que precisava num único método. Mas fazer seu código ser tão compacto quanto possível não é realmente útil! Vamos dividi-lo em vários métodos para deixá-lo mais fácil de ler e vamos também certificar-nos de que as classes recebam nomes que façam sentido. Mas vamos começar entendendo para que esse código deveria servir.

Como você determina a utilidade de um código? Bem, todos os programas são escritos por um motivo. Cabe a você determiná-lo! Aqui, podemos olhar para a página do manual de especificações que o programador seguiu.

Produtor de Barras Doces Tipo 5 General Eletronics
Manual de Especificações

A temperatura do creme precisa ser checada a cada 3 minutos por um sistema automatizado. Se a temperatura **ultrapassar os 160°C**, o doce estará quente demais e o sistema precisará executar o procedimento de liberação do sistema de isolamento de doce para resfriamento **(CICS, de candy isolation cooling system).**

- Fecha-se a válvula de fluxo de curso na turbina #2 (trip throttle valve)
- Enche-se o sistema de isolamento totalmente com água para resfriamento
- Libera-se a água
- Verifica-se se não existe nenhum traço de ar no sistema

3 Esta página do manual facilitou o entendimento do código e dá-nos ótimas dicas sobre como fazer facilitar o entendimento do nosso código. Agora sabemos porque o teste condicional tem como limite para a variável o número 160 - o manual diz que qualquer temperatura além desta significa que o creme está quente demais. E acontece que "**m**" é uma classe que controla a máquina de fazer doce, com métodos estáticos para checar a temperatura do creme e o sistema de ar. Então vamos colocar a checagem de temperatura em um método e escolher nomes para a classe e os métodos que deixem seus objetivos óbvios.

```
public boolean IsNougatTooHot() {
  int temp = Maker.CheckNougatTemperature();
  if (temp > 160) {
    return true;
  } else {
    return false;
  }
}
```

O tipo de retorno do método IsNougatTooHot()

Ao nomear a classe como Maker (que pode ser traduzido como produtor) e o método de CheckNougatTemperature (verifique a temperatura do creme), o código fica muito mais fácil de entender.

O valor de retorno desse método é um booleano, ou seja, ele retorna verdadeiro ou falso.

4 O que as especificações dizem a respeito do que fazer quando o creme está quente demais? Elas nos dizem para empregar o procedimento de liberação do sistema de isolamento de doces para resfriamento (ou CICS). Então vamos criar outro método e escolher um nome óbvio para a classe "T" (que é a classe que controla a turbina) e para a classe "ics" (que controla o sistema de isolamento para resfriamento e que tem dois métodos, um para encher e outro para liberar o conteúdo do sistema):

```
public void DoCICSVentProcedure() {
    Turbine turbineController = new Turbine();
    turbineController.CloseTripValve(2);
    IsolationCoolingSystem.Fill();
    IsolationCoolingSystem.Vent();
    Maker.CheckAirSystem();
}
```

Um tipo void quer dizer que o método não retorna nenhum valor.

5 Agora o código está muito mais intuitivo! Mesmo que você não saiba que o procedimento de liberação do CICS precisa ser executado se o creme estiver muito quente, **é muito mais óbvio o que este código está fazendo.**

```
if (IsNougatTooHot() == true) {
    DoCICSVentProcedure();
}
```

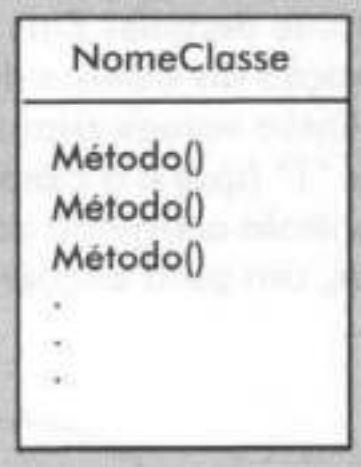
Você pode fazer seu código mais fácil de ler e entender ao pensar a respeito do problema que está sendo resolvido por ele. Se você escolher nomes para seus métodos que façam sentido para alguém que entenda o problema, então seu código será muito mais fácil de decifrar... e desenvolver!

Dê às suas classes uma estrutura natural

Pare um momento e lembre-se do motivo para que seus métodos sejam intuitivos: **porque cada programa serve para resolver um problema ou tem um propósito.** Pode não ser um problema empresarial - algumas vezes o propósito de um software (como o FlashyThing) é apenas ser divertido e legal! Mas não importa o que seu programa faça, quanto mais você puder fazer com que seu código se assemelhe ao problema que está tentando resolver, mais fácil será de escrevê-lo (e lê-lo, e fazer correções, e manutenção...).

Use diagramas de classes para planejá-las

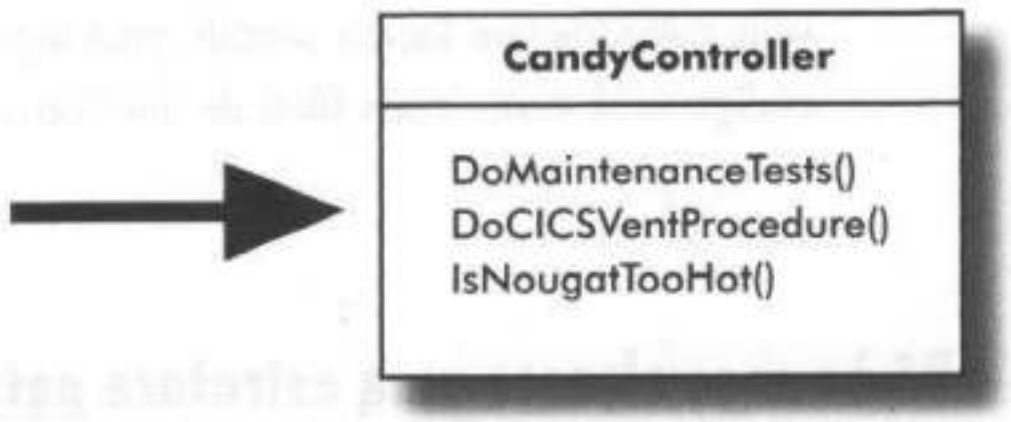
Um diagrama de classe é uma forma simples de descrevê-las usando uma folha de papel. É realmente uma ferramenta valiosa para projetar seu código ANTES de começar a escrevê-lo.
Escreva o nome da classe no topo do diagrama. Depois o nome de cada método na caixa abaixo. Agora você pode ver todas as partes de sua classe numa única olhada!



Vamos desenhar um diagrama de classe

Dê uma nova olhada no comando if no item 5 na página anterior. Você já sabe que comandos sempre são contidos dentro de métodos, que, por sua vez, estão contidos em classes, certo? Nesse caso, o comando if está num método chamado DoMaintenanceTests() (faça testes de manutenção), que é parte da classe CandyController (controlador de doces). Agora olhe para o código e para o diagrama de classe. Percebe a relação entre eles?

```
public class CandyController {
  public void DoMaintenanceTests() {
     ...
     if (IsNougatTooHot() == true) {
        DoCICSVentProcedure();
     }
     ...
  }
  public void DoCICSVentProcedure() ...
  public boolean IsNougatTooHot() ...
}
```



Aponte seu lápis

O programa para o sistema de controle de doces desenvolvido nas páginas anteriores chamava três outras classes. Volte atrás no livro, olhe para o código e preencha os diagramas de classe.

Turbine

.....

Nós preenchemos o nome desta classe para você. Que método deve estar aqui?

Uma das classes tem um método chamado Fill(). Preencha o nome da classe e seu outro método.

Fill()

.....

.....

.....

Existe mais uma classe no código das páginas anteriores. Preencha seu nome e seu método.

Diagramas de classe ajudam a organizar o código de forma que ele faça sentido

Manter diagramas de classes facilita vislumbrar problemas potenciais nelas antes de escrever código. Pensar nas suas classes de um ponto de vista mais elevado antes de entrar nos detalhes pode ajudar a projetar uma estrutura de classes que permita ao código resolver o problema para o qual é escrito. Isso permite dar um passo para trás e certificar-se de que não está planejando escrever código desnecessário ou métodos e classes pobremente estruturados e também que aqueles que você escrever sejam intuitivos e fáceis de usar.

LavaLouças

LimpaLouças()
ColoqueDetergente()
AjusteTemperaturaAgua()
EstacioneCarro()

A classe é chamada de "LavaLouças", então todos os métodos deveriam ser sobre isso. Mas um deles - EstacioneCarro() - não tem nada a ver com louças, de forma que ele deve ser retirado e colocado em outra classe.

LavaLouças

LimpaLouças()
ColoqueDetergente()
AjusteTemperaturaAgua()

Aponte seu lápis Solução

O programa para o sistema de controle de doces desenvolvido nas páginas anteriores chamava três outras classes. Volte atrás no livro, olhe para o código e preencha os diagramas de classe.

Você nota que Maker é uma classe porque ela aparece na frente do ponto em Maker.CheckAirSystem().

Turbine

CloseTripValve()

IsolationCoolingSystem

Fill()

vent()

Maker

CheckNougatTemperature()

CheckAirSystem()

Aponte seu lápis

Cada uma dessas classes tem um sério problema de projeto. Escreva o que você pensa que está errado com cada classe e como corrigi-la.

Class23

PesoBarraDoce()
ImprimirPacote()
GeraRelatorios()
Vai()

Esta classe é parte do sistema de produção de doces visto antes.

.....

.....

.....

.....

EntregadorPizza

AdicionarPizza()
PizzaEntregue()
TotalPago()
TempoViagem()

EntregadoraPizza

AdicionarPizza()
PizzaEntregue()
TotalPago()
TempoViagem()

As duas classes integram o sistema que uma pizzaria usa para controlar a entrega das pizzas.

.....

.....

.....

.....

CaixaRegistradora

VendaEfetuada()
SemVendas()
BombeieGasolina()
Reembolso()
ValorTotalNoCaixa()
RecuperaListaTransacoes()
AdicionarDinheiro()
RetiraDinheiro()

A classe `CaixaRegistradora` é parte de um programa usado por um sistema de cobrança de uma loja de conveniência automatizada.

.....

.....

.....

.....

Aponte seu lápis
Solução

Eis como corrigimos as classes. Existe apenas uma única forma de resolver os problemas - mas muitas formas de fazer o projeto destas classes dependendo de como elas são usadas.

Esta classe é parte do sistema de produção de doces visto antes.

O nome da classe não descreve seu propósito. Um programador que visse uma linha de código chamada Class23.600 não teria a menor idéia do que a linha faz. Também devemos renomear o método para ser mais descritivo - escolhemos InicieProducaoDeDoce(), mas poderia ser qualquer outra coisa.

Classe Produtora

PesoBarraDoce()
ImprimirPacote()
GeraRelatorios()
InicieProducaoDeDoce()

As duas classes integram o sistema que uma pizzaria usa para controlar a entrega das pizzas.

Parece que as duas classes fazem a mesma coisa: mantêm controle de um entregador que está fora fazendo uma entrega para clientes. Um design melhor seria substituir as duas por uma única e adicionar-lhe um campo para o sexo da pessoa.

Entregador

Sexo
AdicionarPizza()
PizzaEntregue()
TotalPago()
TempoViagem()

Adicionamos um campo Sexo porque assumimos que existe um motivo para controlar de forma separada homens e mulheres, que deveria ser o motivo original de existirem duas classes distintas.

A classe CaixaRegistradora integra um programa usado por um sistema de cobrança de uma loja de conveniência automatizada.

Todos os métodos na classe têm relação com o que uma caixa registradora faz - fazer vendas, obter listas de transações, adicionar dinheiro... exceto um: bombear gasolina. É uma boa idéia separar esse método numa outra classe.

CaixaRegistradora

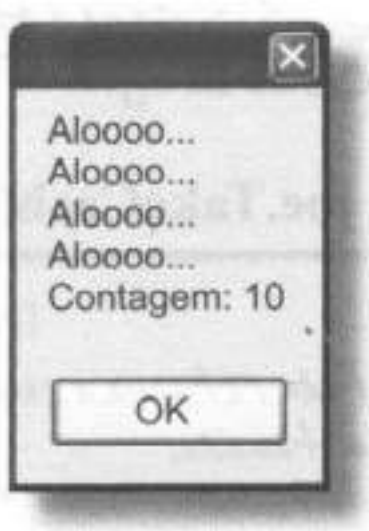
VendaEfetuada()
SemVendas()
Reembolso()
ValorTotalNoCaixa()
RecuperaListaTransacoes()
AdicionarDinheiro()
RetiraDinheiro()

```
public partial class Form1 : Form
{
  private void button1_Click(object sender, EventArgs e)
  {
    String result = "";
    Echo e1 = new Echo();
    _____________________________
    int x = 0;
    while ( ___________ ) {
      result = result + e1.hello() + "\n";
      ____________________________
      if ( ____________ ) {
        e2.count = e2.count + 1;
      }
      if ( ____________ ) {
        e2.count = e2.count + e1.count;
      }
      x = x + 1;
    }
    MessageBox.Show(result + "Contagem: " + e2.count);
  }
  public class ____________ {
    public int _________ = 0;
    public string ___________ {
      return "helloooo...";
    }
  }
}
```

Jogos de Verão

Seu trabalho é pegar os trechos de código da piscina e colocá-los nas linhas em branco no código principal. Você não pode usar o mesmo trecho mais de uma vez e não precisa usar todos eles. Seu objetivo é fazer uma classe que compile, execute e produza o resultado mostrado. Não se engane - este exercício é mais difícil do que parece.

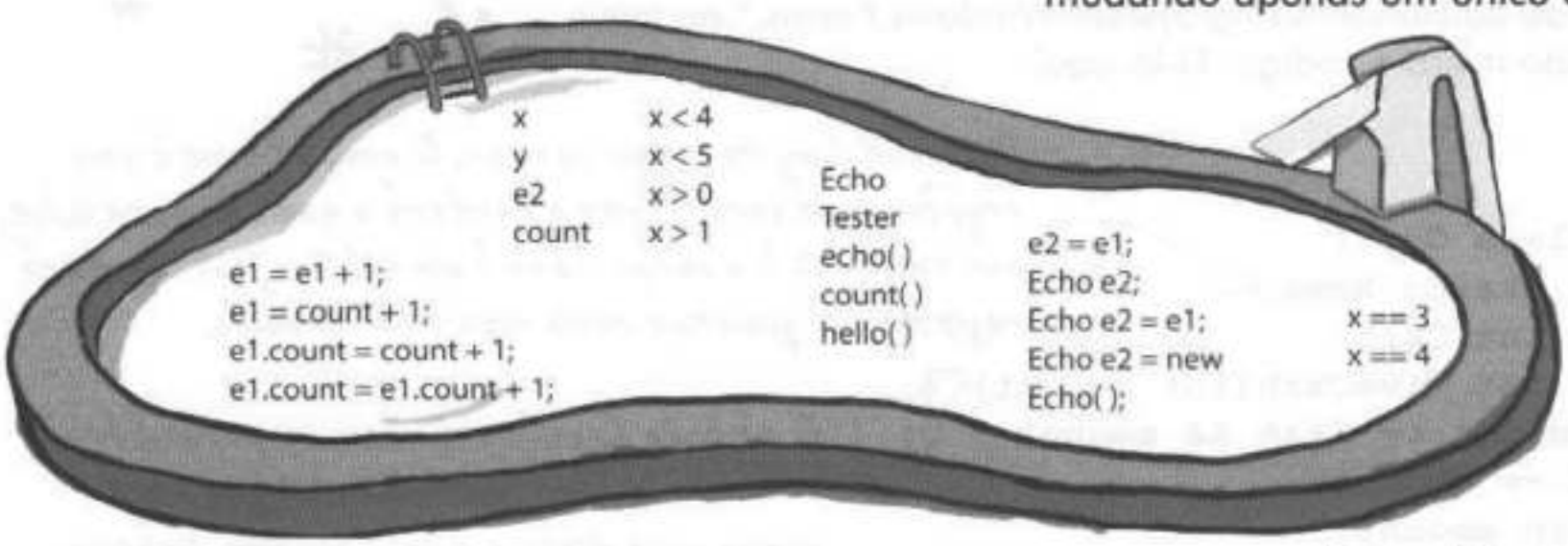
Resultado



Nota: cada trecho na piscina só pode ser usado uma vez!

Questão de Bônus!

Se a última linha da saída fosse 24, em vez de 10, como você completaria o quebra-cabeça? Você pode fazer isto mudando apenas um único comando.



→ **Resposta na página 110.**

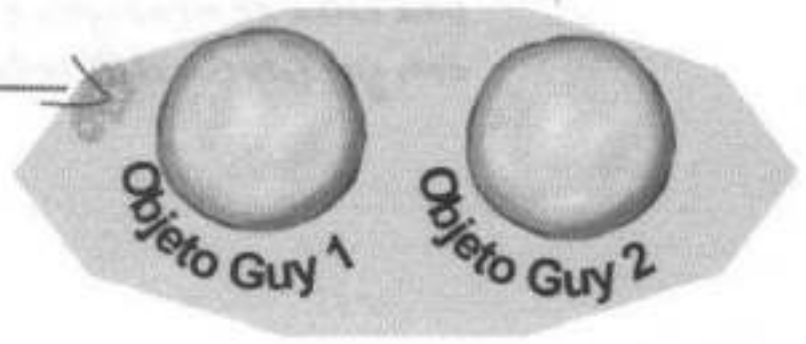
Desenvolva uma classe para trabalhar com alguns caras

Guy
Name
Cash
GiveCash()
TakeCash()

Joe e Bob emprestam dinheiro um para o outro todo tempo. Vamos desenvolver uma classe para manter registro disso.

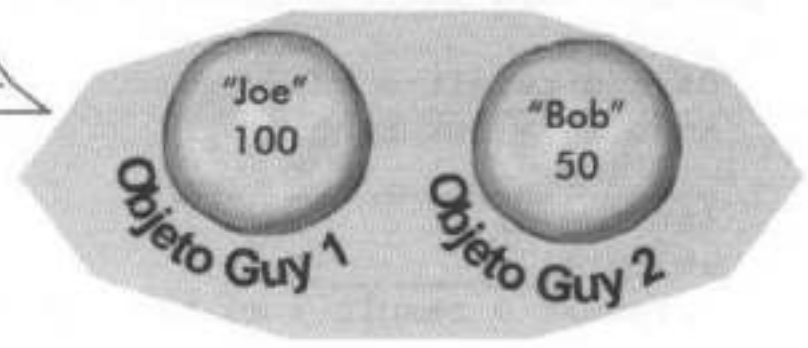
1. **Vamos criar uma classe Guy (cara) e adicionar duas instâncias dela num formulário**
   O formulário terá dois campos, um chamado joe (para associar com o primeiro objeto) e outro chamado bob (para o segundo).

O comando new, que cria as duas instâncias, está no código que é executado quando o formulário é criado. Eis como a pilha se parece depois que o formulário é carregado:



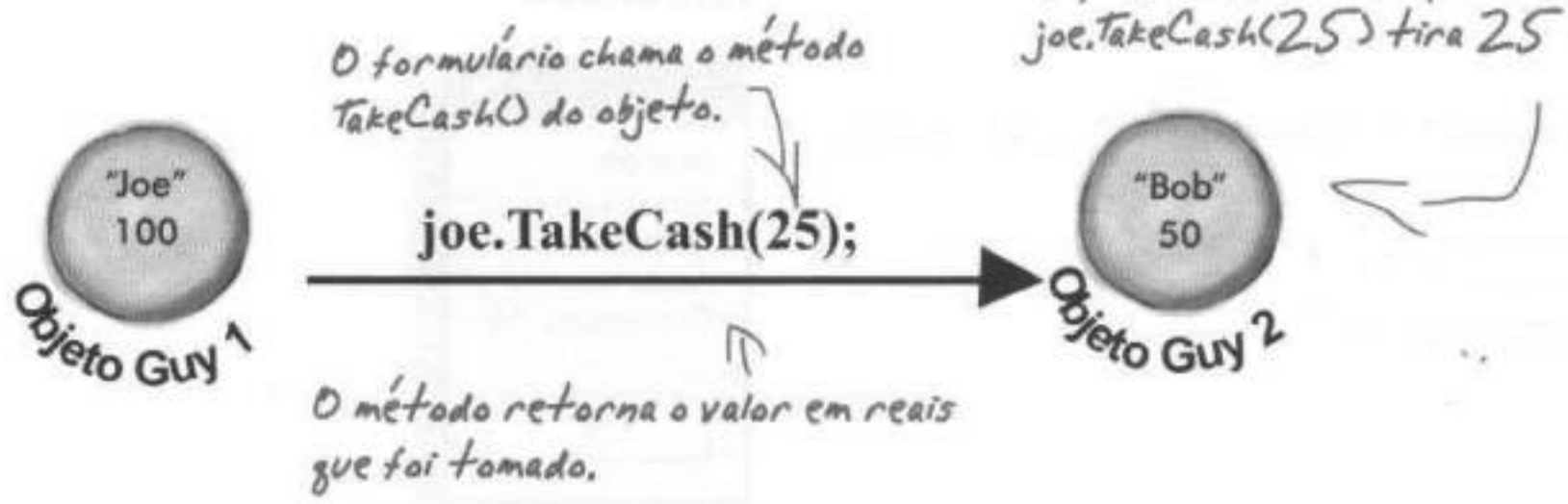
2 **Vamos atribuir a cada objeto Guy nome e dinheiro**
Os dois objetos representam dois caras diferentes. Logo, cada um deles tem um nome e uma quantidade de dinheiro no bolso.

Cada cara tem um campo Name (nome) que registra seu nome e um Cash (dinheiro) que armazena quantos reais eles tem no bolso.



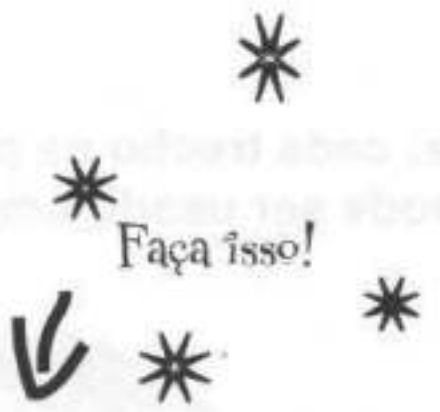
3 **Vamos dar e tirar dinheiro dos caras**
Usaremos os métodos GiveCash() (dar dinheiro) para aumentar o valor que cada cara possui e usaremos TakeCash() para tomar o dinheiro de volta deles.

Quando você pega uma instância de Guy e chama seu método TakeCash() (tome dinheiro), você passa a quantidade de dinheiro que será tomada emprestada como um parâmetro. Assim, chamar joe.TakeCash(25) tira 25 reais de joe.



Crie um projeto para seus caras

Crie um novo projeto de Aplicativo Windows Forms (porque você usará um formulário). Então use o Navegador de Solução para adicionar uma nova classe chamada Guy. Certifique-se de adicionar "using System.Windows.Forms." no início do arquivo da classe Guy. Então insira o código. Ei-lo aqui:



A classe Guy tem dois campos. O campo Name é uma seqüência de caracteres e conterá o nome do cara (Joe, por exemplo). E o campo Cash é um inteiro, que manterá registro de quantos reais eles têm no bolso.

```
public class Guy {
  public string Name;
  public int Cash;
  public int GiveCash(int amount) {
   if (amount <= Cash && amount > 0) {
    Cash -= amount;
    return amount;
   } else {
    MessageBox.Show(
     "I don't have enough cash to give you " + amount,
      Name + " says...");
    return 0;
   }
  }
}
```

O método GiveCash() tem um parâmetro chamado amount (quantidade) que será usado para dizer a o cara quanto dinheiro ele deve dar-lhe.

A classe Guy certifica-se de que você está solicitando uma quantidade positiva de dinheiro; de outra forma ela estaria somando dinheiro, em vez de subtrai-lo.

Um comando if é usado para verificar se existe dinheiro suficiente - se for o caso, esta quantidade é tirada do bolso e retornada como valor de retorno.

Se o cara não tiver dinheiro suficiente, ele dirá a você isso com uma caixa de mensagem e então fará com que GiveCash() retorne zero.

```
public int ReceiveCash(int amount) {
    if (amount > 0) {
      Cash += amount;
      return amount;
    } else {
      MessageBox.Show(amount + " isn't an amount I'll take",
        Name + " says...");
      return 0;
    }
}
```

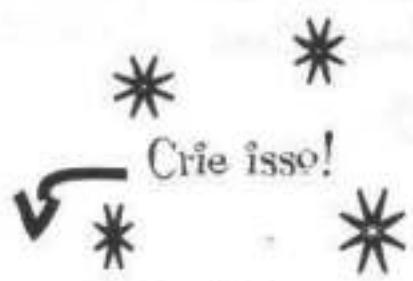
O método ReceiveCash() (receber dinheiro) funciona como GiveCash(). Uma quantidade é passada como parâmetro, checada para ver se é maior que zero e, então, adicionada ao total de dinheiro.

Se a quantidade for positiva, o método ReceiveCash() retorna a quantidade adicionada. Se for zero ou negativo, o cara mostra uma mensagem e retorna zero.

Preste atenção na colocação de suas chaves. É fácil errar o número delas - certifique-se de que cada abre-chave tenha um fecha-chave correspondente. Quando elas estão todas corretas, o IDE irá identá-las para você automaticamente quando a última delas for digitada.

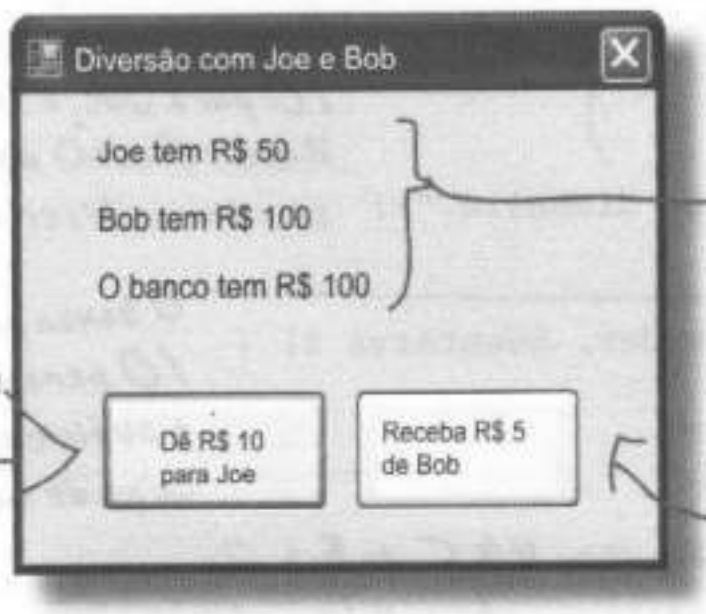
Crie um formulário para interagir com os caras

A classe Guy é ótima, mas é só o começo. Agora coloque-a em um formulário que use duas instâncias dela. Ele terá rótulos para mostrar seus nomes e quanto dinheiro eles têm; e botões para dar e tirar dinheiro deles.



1 Adicione dois botões e seis rótulos no seu formulário

Os rótulos no lado direito do formulário mostram quanto dinheiro cada cara tem. Também adicionaremos uma variável chamada bank (banco) no formulário - o terceiro rótulo à direita mostra quanto dinheiro ele tem. Você deverá alterar o nome de alguns dos rótulos que arrastará para o formulário. Você pode fazer isto clicando em cada um deles e **mudando a linha "(Name)"** na janela de propriedades. Isso fará com que seu código fique mais fácil de ler, porque você usará "joeCash" (dinheiro do joe) e "bobCash", em vez de "label4" ou "label5".



Este botão chamará o método ReceiveCash() do objeto Joe, passará 10 como quantidade e adicionará o dinheiro dado a Joe à variável bank.

Nomeie o rótulo superior como joesCash, o abaixo dele como bobsCash, e o inferior como bankCash. Você pode deixar as propriedades Text de todos inalteradas, porque adicionaremos um método que vai mudá-las.

Este botão chamará o método GiveCash() de Bob, passando 5 como quantidade, e adicionará o dinheiro que ele receberá de Joe à variável bank.

2 **Adicione variáveis no seu formulário**
Seu formulário precisará manter registro dos dois caras. Logo, você precisará de uma variável para cada um. Chame-as de joe e bob. Adicione uma outra variável ao formulário chamada bank para manter registro de quanto dinheiro o formulário deu e recebeu para os caras.

Já que estamos usando objetos Guy para manter registro a respeito de Joe e Bob, você declarará as variáveis usando Guy.

```
namespace Your_Project_Name {
  public partial class Form1 : Form {
    Guy joe;
    Guy bob;
    int bank = 100;
    public Form1() {
      InitializeComponent();
    }
```

A quantidade de dinheiro no banco sobe e desce de acordo com a quantidade de dinheiro recebida ou dada aos objetos Guy pelo formulário.

3 Adicione um método no formulário para atualizar os rótulos
Os rótulos no lado direito do formulário mostrarão quanto cada cara tem e quanto tem armazenado na variável banco. Então adicione o método UpdateForm() (atualize formulário) para mantê-los atualizados - e certifique-se de que o tipo de retorno seja void para dizer ao C# que o método não retorna um valor. Digite o seguinte método logo abaixo de onde você adicionou a variável bank:

Note como os rótulos são atualizados usando os campos Name e Cash do objeto Guy.

```
public void UpdateForm() {
  joesCash.Text = joe.Name + " tem $" + joe.Cash;
  bobsCash.Text = bob.Name + " tem $" + bob.Cash;
  bankCash.Text = "O banco tem $" + bank;
}
```

Esse novo método é simples. Ele apenas atualiza os três rótulos atribuindo valores para as suas propriedades Text. Você fará com que cada botão o chame para manter os rótulos em dia.

4 **Clique duas vezes em cada botão e adicione o código para interagir com os objetos**
Certifique-se de que o botão da esquerda seja chamado button1 e o da direita button2. Clique duas vezes em cada um deles - quando você fizer isto, o IDE adicionará dois métodos no formulário, button1_Click() e button2_Click(). Adicione o seguinte código em cada um deles:

```
private void button1_Click(object sender, EventArgs e) {
  if (bank >= 10) {
    bank -= joe.ReceiveCash(10);
    UpdateForm();
  } else {
    MessageBox.Show("O banco está sem dinheiro.");
  }
}
private void button2_Click(object sender, EventArgs e) {
  bank += bob.GiveCash(5);
  UpdateForm();
}
```

Quando o usuário clicar no botão Dê R$ 10 para Joe, o formulário chama o método ReceiveCash() do objeto Joe - mas apenas se o banco tiver dinheiro suficiente.

O banco precisa de pelo menos R$ 10 para dar para Joe. Se não tiver o suficiente, ele mostrará essa caixa de mensagem.

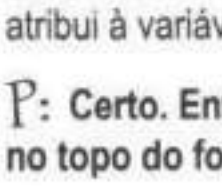
O botão Receber R$ 5 de Bob não precisa checar quanto existe no banco, porque ele apenas adiciona qualquer que seja o valor retornado por Bob.

Se Bob não tiver mais dinheiro, GiveCash() retorna zero.

5 É de sua responsabilidade determinar como Joe e Bob devem ser inicializados a respeito de suas propriedades Cash e Name. Coloque este código diretamente abaixo de InitializeComponent() no formulário. Isto integra um método especial executado somente em uma única oportunidade: quando o formulário é iniciado pela primeira vez. Depois disto, clique em ambos os botões algumas vezes - certifique-se de que um botão tira R$ 10 do banco e adiciona o dinheiro à reserva de Joe e que o outro tira R$ 5 de Bob e o dá para o banco.

```
public Form() {
  InitializeComponent();
  // Initialize joe and bob here!
}
```

Adicione as linhas de código aqui para criar os dois objetos e inicializar seus campos Name e Cash.



Exercício

não existem
Perguntas Idiotas

P: Porque a solução não inicia com "Guy bob = new Guy()"? Porque vocês deixaram de fora o primeiro "Guy"?

R: Porque você já declarou a variável bob no topo do formulário. Lembre-se de que o comando "int i = 5" é o mesmo que dois comandos, "int i" e "i=5". Aqui ocorre o mesmo. Você pode tentar declarar a variável bob numa linha como "Guy bob = new Guy();", mas você já tem a primeira parte do comando ("Guy bob;") no início do formulário. Então só precisa da segunda metade, a parte que atribui à variável bob a nova instância de Guy().

P: Certo. Então por que não nos livramos da linha "Guy bob;" no topo do formulário?

R: Porque se assim a variável existirá somente dentro daquele método específico "public Form1()". Quando você declara uma variável dentro de um método, ela é válida somente dentro dele - não pode ser acessada de nenhum outro método. Mas quando você a declara fora de um método, mas dentro de uma classe ou formulário, ela pode ser acessada por qualquer método dessa classe ou formulário.

P: O que acontece se eu tirar a primeira linha com "Guy"?

R: Você terá problemas - seu formulário não funcionará, porque ele nem mesmo atribuirá os valores iniciais para a variável bob do formulário. Pense a respeito um minuto e perceberá porque isto funciona assim. Se você tem o seguinte código no início de seu formulário:

```
public partial class Form1 : Form {
Guy bob;
```

e então adicionasse a seguinte linha mais tarde, dentro de um método:

```
Guy bob = new Guy();
```

você estaria declarando duas variáveis. Isto é confuso porque as duas tem o mesmo nome. Mas uma delas é válida para o formulário inteiro e a outra - a segunda que você adicionou - é válida apenas dentro do método. A próxima linha (bob.Name = "Bob";) atualiza somente a variável local e nem encosta na do formulário. Então quando você tenta executar seu código, ele lhe mostrará uma mensagem de erro medonha ("NullReferenceException não tratada", " NullReferenceException not handled" em inglês, sendo que NullReferenceException pode ser traduzido para "exceção de referência nula"). Este erro significa que você tentou usar um objeto antes de criá-lo com new.

Solução do Exercício

É de sua responsabilidade **determinar como Joe e Bob devem ser inicializados a respeito de suas propriedades Cash e Name.** Coloque este código diretamente abaixo de InitializeComponent()

```
public Form1() {
    InitializeComponent();

    bob = new Guy();
    bob.Name = "Bob";
    bob.Cash = 100;

    joe = new Guy();
    joe.Name = "Joe";
    joe.Cash = 50;

    UpdateForm();
}
```

Eis onde inicializamos a primeira instância de Guy. A primeira linha cria o objeto e a próxima atribui valores aos campos.

Então fazemos o mesmo com a segunda instância da classe Guy.

Tenha certeza de chamar UpdateForm() para que os rótulos estejam corretos na primeira vez que o formulário aparecer.

Certifique-se de salvar o projeto agora - voltaremos a usá-lo daqui a algumas páginas.

Existe uma forma ainda mais fácil de inicializar objetos

Quase todos os objetos criados precisam ser inicializados de alguma forma. E o objeto Guy não é exceção - ele é inútil enquanto não se atribuem valores para seus campos Name e Cash. É tão comum ter que inicializar campos que o C# oferece um atalho para isto, o inicializador de objeto. E o IntelliSense do IDE lhe ajudará a fazer isso.

Veja bem!

Inicializadores de objeto só funcionam no C# 3.0.

Se você estiver usando o Visual Studio 2005, isso não funcionará. Considere baixar a versão Express do Visual Studio 2008 - é gratuita e pode ser instalada junto com uma versão existente do VS 2005.

1. Eis o código original que escrevemos para inicializar o objeto Guy Joe.

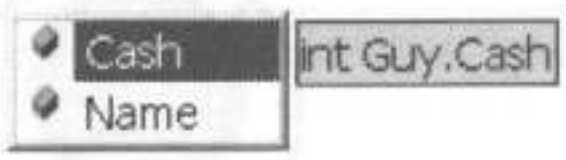
```
joe = new Guy();
joe.Name = "Joe";
joe.Cash = 50;
```

2. Apague as últimas duas linhas e o ponto e vírgula depois de "Guy()". No lugar dele, abra chaves.

```
joe = new Guy() {
```

3. Pressione espaço. Assim que você o fizer, o IDE exibe uma janela do IntelliSense que lhe mostra todos os campos que você pode inicializar.

```
joe = new Guy() {
```



4. Pressione a tecla tab para adicionar o campo Cash. Atribua o valor 50 a ele.

```
joe = new Guy() { Cash = 50
```

5. Digite uma vírgula. Assim que fizer isso o outro campo aparecerá.

```
joe = new Guy() { Cash = 50,
```

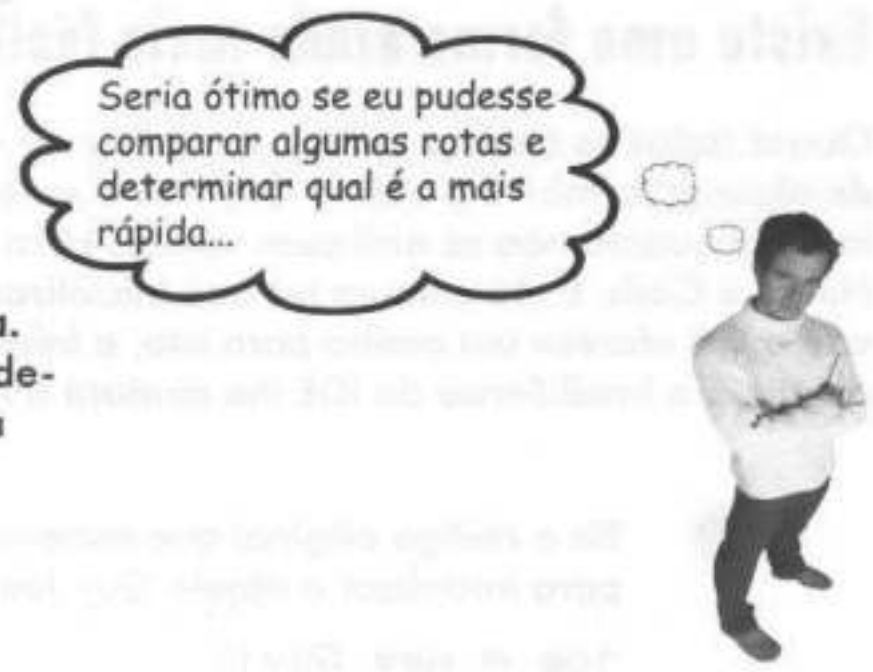
6. Encerre o inicializador de objeto. Veja que você poupou duas linhas de código!

```
joe = new Guy() { Cash = 50, Name = "Joe" };
```

A nova declaração faz exatamente a mesma coisa que as três linhas de código originalmente escritas, mas é mais curta e fácil de ler.

Os inicializadores de objeto poupam tempo e fazem seu código ser mais compacto e fácil de ler... e o IDE ajuda você a escrevê-los.

Algumas idéias para projetar classes intuitivas



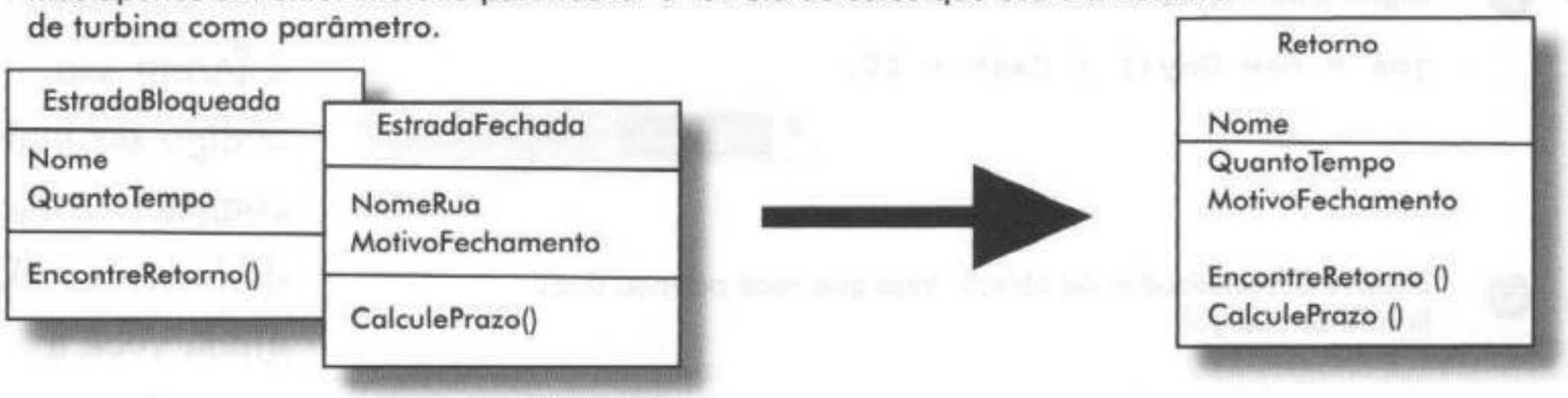
★ **Você está criando seu programa para resolver um problema.**
Passe algum tempo pensando a respeito do problema. Ele divide-se em partes distintas facilmente? Como você o explicaria para alguém? Estas são boas coisas para pensar-se quando se está projetando classes.

★ **Quais são as coisas no mundo real que o seu programa usa?**
Um programa que ajude um zoológico a manter registro de seus horários de alimentação de animais pode ter classes para diferentes tipos de alimentos e de animais.

★

Use nomes descritivos para classes e métodos
Outras pessoas deveriam ser capazes de determinar o que as classes e métodos que você projeta fazem apenas olhando para seus nomes.

★ **Procure similaridades entre classes**
Algumas vezes duas classes podem ser combinadas numa única se forem realmente semelhantes. O sistema de fabricação de doces pode ter três ou quatro turbinas, mas apenas um único método para fechar a válvula de curso que usa um número de turbina como parâmetro.



Exercício

Adicione botões no programa "Diversão com Joe e Bob" para fazer os caras darem dinheiro um para o outro.

1 **Use um inicializador de objetos para inicializar a instância de Guy Bob.**
Você já fez isto com Joe. Agora faça com que a instância de Bob use um inicializador de objetos também.

Se você já clicou o botão, pode deletá-lo, adicioná-lo novamente e renomeá-lo. Então delete o antigo método button3_Click() que o IDE adicionou anteriormente e use o novo método que ele adicionará agora.

2 **Adicone mais dois botões em seu formulário**
O primeiro botão diz que Joe deve dar R$ 10 para Bob e o segundo diz que Bob deve dar 5 de volta para Joe. Antes que você clique duas vezes no botão, vá até a janela de propriedades e mude os nomes dos botões usando a linha "(Name)" - ela fica no topo da lista. Nomeie o primeiro botão como joeGivesToBob (Joe empresta para Bob, literalmente "Joe dá para Bob") e o segundo como bobGivesToJoe.

Este botão diz a Joe para dar R$ 10 para Bob. Assim, você deve usar a linha (Name) na janela de propriedades para dar-lhe o nome joeGivesToBob.

Esse botão diz a Bob para dar dinheiro para Joe. Dê a ele o nome bobGivesToJoe.

3

Faça os botões funcionarem
Clique duas vezes no botão joeGivesToBob no painel de design. O IDE adicionará um método ao formulário chamado joeGivesToBob_Click() que executa quando o botão for clicado. Complete esse método para fazer Joe dar R$ 10 para Bob. Então clique duas vezes no outro botão e complete o novo método bobGivesToJoe_Click() criado pelo IDE para que Bob dê R$ 5 para Joe. Certifique-se de que o formulário seja atualizado depois que o dinheiro trocar de mãos.

Solução do Exercício

Adicione botões no programa "Diversão com Joe e Bob" para fazer os caras darem dinheiro um ao outro.

Eis os inicializadores de objeto para as duas instâncias da classe Guy. Bob é inicializado com R$ 100 e seu nome.

```
public partial class Form1 : Form {
  Guy joe;
  Guy bob;
  int bank = 100;
  public Form1() {
    InitializeComponent();
    bob = new Guy() { Cash = 100, Name = "Bob" };
    joe = new Guy() { Cash = 50, Name = "Joe" };
    UpdateForm();
  }
  public void UpdateForm() {
    joesCash.Text = joe.Name + " tem $" + joe.Cash;
    bobsCash.Text = bob.Name + " tem $" + bob.Cash;
    bankCash.Text = "O banco tem $" + bank;
  }
  private void button1_Click(object sender, EventArgs e) {
    if (bank >= 10) {
      bank -= joe.ReceiveCash(10);
      UpdateForm();
    } else {
      MessageBox.Show("O banco está sem dinheiro.");
    }
  }
  private void button2_Click(object sender, EventArgs e) {
    bank += bob.GiveCash(5);
    UpdateForm();
  }
  private void joeGivesToBob_Click(object sender, EventArgs e) {
    bob.ReceiveCash(joe.GiveCash(10));
    UpdateForm();
  }
  private void bobGivesToJoe_Click(object sender, EventArgs e) {
    joe.ReceiveCash(bob.GiveCash(5));
    UpdateForm();
  }
}
```

Para fazer Joe dar dinheiro a Bob, chamados o método GiveCash() de Joe e mandamos seu resultado para o método ReceiveCash() de Bob.

Dê uma boa olhada como os métodos de Guy estão sendo chamados. Os resultados retornados por GiveCash() são injetados direto como parâmetros de ReceiveCash().

O truque aqui é pensar quem está dando o dinheiro e quem está recebendo.

São 10 horas. Você sabe onde seus dados estão?

Tipos de dados, bases de dados, Tenente-Comandante Data... é tudo importante. Sem dados, seus programas são inúteis. Informações oriundas dos usuários são necessárias e são usadas para procurar ou criar informação nova para devolver a eles. Na verdade, quase tudo que você faz em programação envolve trabalho com dados de uma forma ou de outra. Neste capítulo, você aprenderá as vantagens e desvantagens dos tipos de dados C#, como trabalhar com dados em seu programa e até descobrir alguns segredos sujos sobre objetos (shhhhh...objetos são dados também).

O tipo da variável determina qual tipo de dado ela pode armazenar

Há quinze tipos de valores nativos do C# e cada um armazena um tipo diferente de dado. Você já viu alguns dos mais comuns e já sabe como usá-los. Mas há alguns poucos para ver ainda e eles realmente podem ser úteis também.

Tipos de valores que você usará sempre

Não deveria surpreender que int, string, bool e float sejam os tipos mais comuns.

Um número inteiro não possui vírgula decimal.

- **int** pode armazenar **qualquer** número inteiro de -2.147.483.648 a 2.147.483.647.
- **string** pode conter texto de qualquer tamanho (incluindo a seqüência de caracteres vazia "").
- **bool** é um valor boleano – é verdadeiro ou falso.
- float pode armazenar qualquer número **decimal** de +-1,5 x 10-45 até +- 3,4 x 1038 com até sete dígitos significativos (refere-se à precisão do número: 35.048.410.000.000, 1.743.059, 14,43857 e 0,00004374155 todos possuem sete dígitos significativos). O 1038 significa que você pode armazenar qualquer número tão grande quanto 1038 (ou 1 seguido de 38 zeros) desde que ele tenha apenas sete ou menos dígitos significativos. Na outra extremidade, 10-45 significa que você pode armazenar qualquer número tão pequeno quanto 10-45 (ou uma vírgula decimal seguida por 45 zeros seguidos de 1)... mas você adivinhou: desde que tenha apenas sete ou menos dígitos significativos.

float é uma forma abreviada para floating point (ponto flutuante) – ao contrário de um número de ponto fixo (vírgula fixa, em português), que sempre tem o mesmo número de casas decimais.

O "u" em uint significa unsigned (sem sinal), ou seja, ele não pode ser negativo (então não há sinal de menos).

Mais tipos para números inteiros

Era uma vez uma época em que a memória dos computadores era muito cara e os processadores eram muito lentos. E, acredite ou não, se você usasse o tipo errado, isso poderia deixar seu programa bem mais lento. Felizmente, os tempos mudaram e, na maioria das vezes, se você precisa armazenar um número inteiro você pode apenas usar um int. Mas, às vezes, você realmente precisa de algo maior... e, de vez em quando, algo menor também. É por isso que o C# dá mais opções:

- `byte` pode armazenar qualquer número inteiro entre 0 e 255.
- `sbyte` pode armazenar qualquer número inteiro de -127 até 128.
- `short` pode armazenar qualquer número inteiro de -32.767 até 32.768.
- `ushort` pode armazenar qualquer número inteiro de 0 até 65.535.
- `uint` pode armazenar qualquer número de 0 até 4.294.967.295.
- `long` pode armazenar qualquer número entre menos e mais 9 bilhões de bilhões.
- `ulong` pode armazenar qualquer número entre 0 e mais ou menos 18 bilhões de bilhões.

O "u" significa sem sinal

Muitas vezes, se você estiver usando esses tipos é porque está resolvendo um problema em que o efeito volta para o começo – sobre o qual você lerá daqui a pouco – realmente ajuda.

Tipos para armazenar números realmente GRANDES e realmente pequenos

Às vezes, sete dígitos significativos não é precisão o bastante. E acredite ou não, às vezes, 1038 não é grande o suficiente e 10-45 não é tão pequeno quanto se necessita. Muitos programas escritos para finanças ou para pesquisa científica apresentam estes problemas a todo o momento, então o C# nos dá mais dois tipos:

Na verdade, o tipo double é tão comum quanto o float. Muitas pessoas usam-no toda hora e raramente usam o float.

★ `double` pode armazenar qualquer número de +- 5,0 x 10-324 até +-1,7 x 10308 com 15-16 dígitos significativos.

★ `decimal` pode armazenar qualquer número de +- 1,0 x 10-28 até +- 7,9 x 1028 com 28-29 dígitos significativos.

Quando você usou a propriedade Value em seu controle numericUpDown, estava usando um decimal.

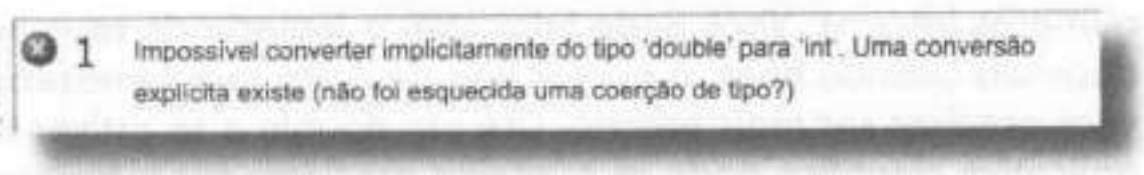
Um literal significa apenas um número que você digita em seu código. Então quando você digita `int i = 5;` o 5 é um literal.

Literais têm tipos também.

Quando você digita um número diretamente em seu programa C#, você está usando um literal... e a todo literal é automaticamente associado um tipo. Você pode checar isso por si mesmo – apenas coloque essa linha de código que associa o literal 14,7 à variável int:

```
int myInt = 14.7;
```

Agora tente compilar o programa. Você verá isso:



Esse é o mesmo erro que você encontrará se tentar fazer com que um número `inteiro` seja contido numa variável `double`. O que o IDE está informando é que o literal 14,7 tem um tipo – é um double. Você pode mudar seu tipo para um `float` colocando um F no final (`14,7F`). E `14,7M` é um decimal.

Se você tentar atribuir um float literal a um double ou um decimal literal a um float, o IDE dará a você um mensagem útil lembrando-o de adicionar o sufixo correto. Legal!

Mais alguns tipos de valores nativos úteis

Você notou como o tipo byte possui 256 valores possíveis? Bem, há outro tipo que também possui 256 valores possíveis: char. Mas ele não é numérico - você o usa para armazenar um caracter. Valores literais para char são sempre valores individuais dentro de aspas ('x', '3'). Você pode incluir **seqüências de escape** nas aspas também ('\n' é uma quebra de linha, '\t' é uma tabulação).

E finalmente, há mais um tipo importante: **object**. Um objeto pode herdar características de outro e isto pode ter acontecido com este último. No topo de toda a hierarquia de herança está a classe object – da qual todos os outros objetos são herdados. Ela é muito útil porque significa que **você pode atribuir a uma variável do tipo** `object` **qualquer valor, variável ou** `objeto`.

Você aprenderá muito mais sobre como char e byte relacionam-se no capítulo 9.

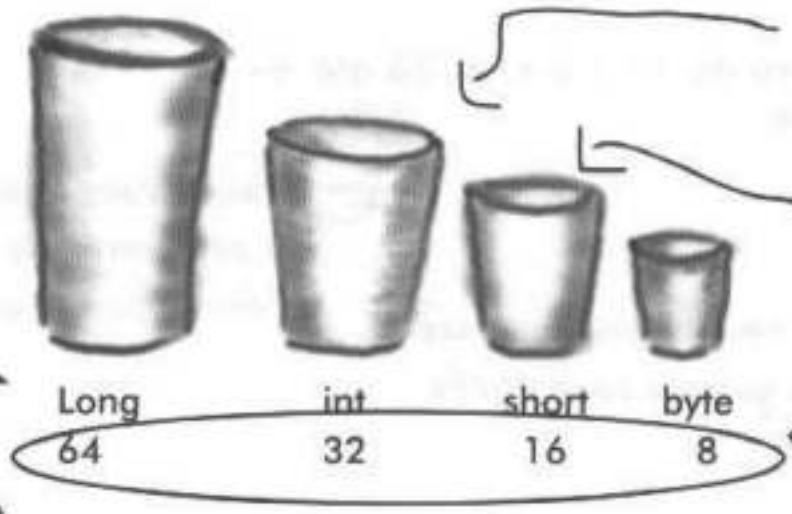
Poder do cérebro

Você pode usar a calculadora do Windows para converter números decimais (normais, base-10) e binários (números de base-2 escritos apenas com uns e zeros) – coloque-a em modo Científico, digite um número e clique no botão de rádio Bin para convertê-lo em binário. Clique então em Dec para convertê-lo de volta. Agora coloque alguns dos maiores e menores limites para os tipos de número inteiro (como -32.767 e 255) e converta-os para binário. Você consegue descobrir por que o C# tem esses limites particulares?

Uma variável é um copo cheio de dados para a viagem

Todos os seus dados ocupam lugar na memória (lembra-se da pilha do último capitulo?). Então parte do seu trabalho é pensar sobre quanto espaço você precisará sempre que usar uma seqüência de caracteres ou um número em seu programa. Essa é uma das razões pelas quais variáveis são usadas. Elas permitem reservar espaço suficiente na memória para armazenar seus dados.
Pense numa variável como um copo no qual você guarda seus dados. O C# usa um monte de tipos diferentes de copos para guardar tipos diferentes de dados. E assim como os diferentes tamanhos de copos na lanchonete, há diferentes tamanhos de variáveis também.

Você usará o long para guardar números inteiros bem grandes.



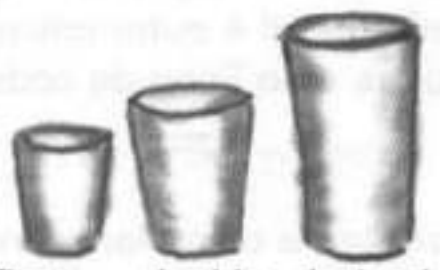
int é comumente usado para números inteiros. Ela guarda números até 2.147.483.647.

Um short guardará números inteiros até 32.767.

byte guarda números entre zero e 255.

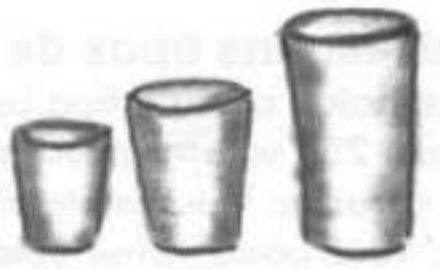
Estes são o número de bits de memória reservados para a variável quando você a declara.

Os números com casas decimais são armazenados diferentemente dos números inteiros. Você pode lidar com a maioria de seus números com casas decimais usando float, o menor tipo de dado que armazena decimais. Se você precisar ser mais preciso, use um double e se estiver escrevendo um aplicativo científico onde os números precisam ser extremamente precisos, o tipo decimal possui a maior precisão.



Estes tipos são para frações. Variáveis maiores armazenam mais casas decimais.

Mas nem sempre lidamos com números (você não gostaria de tomar café quente num copo de plástico nem café frio num de papel). O compilador do C# também pode lidar com caracteres e tipos não numéricos. O tipo char guarda um caracter e o string é usado para muitos caracteres "amarrados" (string em inglês pode ser cordão). Não há um tamanho estabelecido para uma variável string. Ela se expande para guardar quantos dados você precisar. O tipo de dados bool é usado para armazenar valores true (verdadeiro) ou false, como aqueles que você usou para os comandos if.



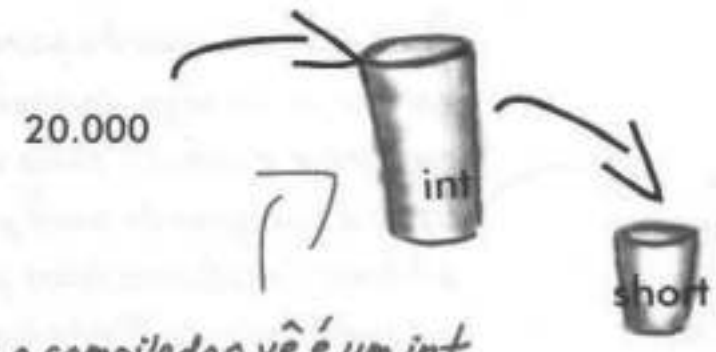
O tipo string tem características exclusivas. É o único tipo de dados sem um tamanho fixo, exceto pelos objetos (pense nisso por um momento).

Cinco quilos de dados num saco de 2,5 quilos

Quando você declara sua variável como de um tipo é assim que o seu compilador irá encará-la. Mesmo se o valor estiver muito acima do limite superior do tipo que você declarou, o compilador verá o copo no qual ele está, não o número dentro. Então isto não funcionará:

```
int leaguesUnderTheSea = 20000;
short smallerLeagues = leaguesUnderTheSea;
```

20.000 caberia em um short, sem problema. Mas já que `leaguesUnderTheSea` é declarada como um int, o compilador encara isso como a capacidade total de um int e considera-o muito grande para ser colocado dentro de um short. O compilador não fará estas conversões para você conforme o programa estiver sendo executado. Você precisa ter certeza de que está usando o tipo certo para os dados com os quais está trabalhando.



Isso faz sentido. E se você, mais tarde, puser um valor maior no copo int que não caberia no copo short? O compilador está tentando protegê-lo.

Tudo que o compilador vê é um int tentando entrar em um short (o que não funciona). Ele não se importa com o valor no copo int.

Aponte seu lápis

Três desses comandos não compilarão, ou porque estão tentando colocar muitos dados em uma variável pequena, ou porque estão colocando o tipo errado de dados dentro dela. Circule-os.

```
int hours = 24;

short y = 78000;

bool isDone = yes;

short RPM = 33;

int balance = 345667 - 567;

string taunt = "your mother";

byte days = 365;

long radius = 3;

char initial = 'S';

string months = "12";
```

Mesmo quando um número tem o tamanho certo, você não pode apenas atribui-lo a qualquer variável

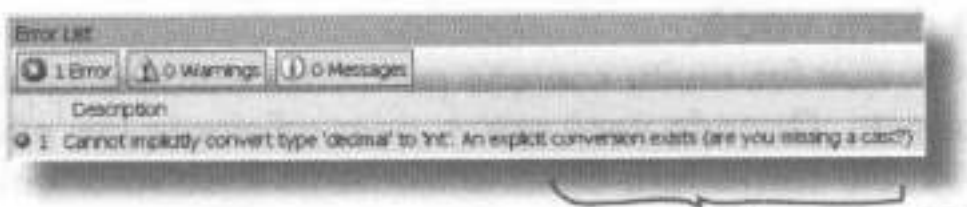
Vamos ver o que acontece quando você tenta atribuir um valor decimal a uma variável int.

Faça isso

1. Crie um novo projeto e adicione um botão a ele. Então adicione essas linhas ao método Click() do botão:

```
decimal myDecimalValue = 10;
int myIntValue = myDecimalValue;
MessageBox.Show("The myIntValue is " + myIntValue);
```

2. Tente criar seu programa. Ops – apareceu um erro que se parece com isso:



Veja como o IDE descobriu que você, provavelmente, esqueceu-se de fazer a coerção.

3. Elimine o erro fazendo a coerção (casting, em inglês, que pode ser traduzido como "elencar") do decimal em um int. Uma vez que você mudou a segunda linha para parecer com isso, seu programa compilará e rodará:

```
int myIntValue = (int) myDecimalValue;
```

É aqui que você faz a coerção do valor decimal em um int.

Então o que aconteceu?

O compilador não te deixará atribuir um valor a uma variável se esta for do tipo errado – mesmo que essa variável possa armazenar bem o valor – porque essa é a causa por detrás de um enorme número de erros. Quando você usa coerção, está dizendo ao compilador que sabe que os tipos são diferentes e nesta ocasião, em particular, não tem problema o C# espremer os dados numa nova variável.

Pare por um minuto para voltar ao começo do capítulo anterior e checar como você usou a cor quando você passou o NumericUpDown.Value para o formulário do Testador Falante.

Aponte seu lápis Solução

Três desses comandos não compilarão, ou porque estão tentando colocar muitos dados em uma variável pequena, ou porque estão colocando o tipo errado de dados dentro dela. Circule-os.

```
short y = 78000;
```

O tipo short guarda números de -32.767 até 32.768. Este número é muito grande!

```
byte days = 365;
```

Um byte pode apenas aceitar um valor até 255. Você precisará de um short para isto.

```
bool isDone = yes;
```

Você só pode atribuir um valor de true ou false a um booleano.

Quando você converte um valor grande demais, o C# ajusta-o automaticamente

Você já viu que podemos fazer coerção de um decimal em um inteiro. Acontece que qualquer número pode ser convertido em qualquer outro. Mas isto não significa que o valor fica intacto no processo. Se você converte uma variável int com o valor em 365 numa variável byte, 365 é muito grande para o byte. Mas, em vez de dar-lhe um erro, o valor apenas ajustar-se-á: por exemplo, a coerção de 256 para um byte terá como resultado 0. 257, se tornaria 1; 258, 2, etc., até 365, que acabará sendo 109. E uma vez que você tenha voltado a 255 de novo, o valor de conversão se ajusta de volta para zero.

Ajuste você mesmo!

Não há mistério em como a coerção ajusta os números – você pode fazer isto sozinho. Apenas use a calculadora do Windows, altere-a para modo Científico e calcule 365 Mod 256 (usando o botão Mod, que faz um cálculo de módulo). Você terá como resultado 109.

Ei, eu estou combinando números e seqüências de caracteres em minhas caixas de mensagens desde que eu aprendi sobre laços no capítulo 2! Eu estive fazendo coerção todo esse tempo?

Sim! O operador + faz a coerção para você.

O que você tem feito é usar o operador +, que faz muita coerção para você automaticamente – e ele é especialmente inteligente nesse aspecto. Quando você usa + para adicionar um número ou boleano em uma seqüência de caracteres, ele converterá automaticamente esse valor em uma seqüência também. Se você usar + (ou *, / ou -) com dois tipos diferentes, ele automaticamente converte o tipo menor num maior. Aqui está um exemplo:

```
int myInt = 36;
float myFloat = 16.4F;
myFloat = myInt + myFloat;
```

Quando você está atribuindo um valor numérico a um float, precisa adicionar um F no final do número para dizer ao compilador que é um float e não um double.

Já que um int pode caber em um float, mas o contrário não é verdadeiro, o operador + converte myInt num float antes de adicioná-lo a myFloat.

Aponte seu lápis

Você nem sempre pode fazer coerção de um tipo em qualquer outro. Crie um novo projeto, arraste um botão para um formulário e digite esses comandos em seu método. Então compile o programa – ele apresentará muitos erros. Risque aqueles que apresentarem erros. Isso ajudará você a descobrir com quais tipos se pode fazer coerção e com quais não!

```
int myInt = 10;
byte myByte = (byte)myInt;
double myDouble = (double)myByte;
bool myBool = (bool)myDouble;
string myString = "false";
myBool = (bool)myString;
myString = (string)myInt;
myString = myInt.ToString();
myBool = (bool)myByte;
myByte = (byte)myBool;
short myShort = (short)myInt;
char myChar = 'x';
myString = (string)myChar;
long myLong = (long)myInt;
decimal myDecimal = (decimal)myLong;
myString = myString + myInt + myByte
+ myDouble + myChar;
```

O C# faz alguma coerção automaticamente

Há duas conversões importantes que não exigem a coerção. A primeira é feita automaticamente sempre que você usar operadores aritméticos, como no exemplo:

```
long l = 139401930;
short s = 516;
double d = l - s;
d = d / 123.456;
MessageBox.Show("A resposta é" + d);
```

O operador - subtraiu o short do long e converteu o resultado em um double.

O operador + é inteligente o bastante para converter o decimal numa seqüência de caracteres.

A outra forma em que o C# converte tipos para você automaticamente é quando usa o operador + para concatenar seqüências de caracteres (o que significa apenas grudar uma seqüência no fim de outra, como você tem feito com as caixas de mensagens). Quando você usa o + para concatenar uma string com algo de outro tipo, ele automaticamente converte os números em strings para você. Abaixo vemos um exemplo. As primeiras duas linhas estão certas, mas a terceira não compilará.

```
long x = 139401930;
MessageBox.Show("A resposta é " + x);
MessageBox.Show(x);
```

O compilador do C# lança um erro que menciona algo sobre argumentos inválidos (um argumento é outro nome para um parâmetro de método). Isso porque o parâmetro para MessageBox.Show() é uma string e este código passou um long, que é o tipo errado para o método. Mas você pode convertê-lo numa string bem facilmente chamando seu método ToString(). Este método é um membro de todo tipo de valor e objeto. (Todas as classes que você mesmo criou possuem um método ToString() que retorna o nome da classe). É assim que você pode converter x em algo que MessageBox.Show() possa usar:

```
MessageBox.Show(x.ToString());
```

Aponte seu lápis
Solução

Você nem sempre pode fazer coerção de um tipo em qualquer outro. Crie um novo projeto, arraste um botão para um formulário e digite esses comandos em seu método. Então compile o programa – ele apresentará muitos erros. Risque aqueles que apresentarem erros. Isso ajudará você a descobrir com quais tipos se pode fazer coerção e com quais não!

```
int myInt = 10;
byte myByte = (byte)myInt;
double myDouble = (double)myByte;
bool myBool = (bool)myDouble;
string myString = "false";
myBool = (bool)myString;
myString = (string)myInt;
myString = myInt.ToString();
myBool = (bool)myByte;
myByte = (byte)myBool;
short myShort = (short)myInt;
char myChar = 'x';
myString = (string)myChar;
long myLong = (long)myInt;
decimal myDecimal = (decimal)myLong;
myString = myString + myInt + myByte
+ myDouble + myChar;
```

Quando você chama um método, as variáveis devem corresponder aos tipos dos parâmetros

Tente chamar MessageBox.Show(123) – passando um literal (123), em vez de uma seqüência de caracteres. O IDE não permitirá que você compile seu programa. Em vez disto, ele te mostrará um erro: "Argumento '1': não pode converter 'int' para 'string'" (Argument '1': cannot convert from 'int' to 'string').
Mas MessageBox.Show() não é o único método que provocará erros do compilador se você tentar passar uma variável cujo tipo não corresponda ao parâmetro. *Todos* os métodos farão isso, até mesmo aqueles que você mesmo escreveu. Vá em frente e tente digitar esse método completamente válido em uma classe:

```
public int MyMethod(bool yesNo) {
  if (yesNo) {
    return 45;
  } else {
    return 61;
  }
}
```

Um lembrete – o código que chama esse parâmetro não tem que passar uma variável chamada yesNo. Ele apenas tem que passar um valor ou uma variável boleana. O único lugar onde ele é chamado yesNo é dentro do código do método.

Ele funciona bem se for dado o que ele espera (`um booleano`) – chame `MyMethod(true)` ou `MyMethod(false)` e ele compila normalmente.
Mas o que acontece se você passar um inteiro ou uma seqüência de caracteres no lugar? O IDE mostra um erro igual ao quando passou 123 para a `MessageBox.Show()`. Agora tente passar um boleano, mas atribua o valor de retorno para um long ou passando-o para **MessageBox.Show()**. Isso não funcionará também – o método retorna um int, não um long nem a string que a **MessageBox.Show()** espera.

Você pode atribuir qualquer coisa a uma variável, parâmetro ou campo com o tipo object.

Comandos if sempre testam para ver se algo é verdadeiro
Você reparou como escrevemos nosso comando if sempre assim?
Não tivemos que dizer explicitamente if (yesNo == true). Isso porque um comando if sempre checa se algo é verdadeiro. Você checa se algo é falso usando ! (um ponto de exclamação, ou o operador NOT). if (!yesNo) é o mesmo que if (yesNo == false). Em nossos exemplos de código daqui em diante, você geralmente nos verá fazer if (yesNo) ou if (!yesNo) e não checar explicitamente para ver se um boleano é verdadeiro ou falso.

Quando o compilador mostra um erro "argumentos inválidos", ele quer dizer que você tentou chamar um método com variáveis cujos tipos não correspondem aos parâmetros.

Exercício

Há cerca de 77 **palavras reservadas** no C#. Estas são palavras que o compilador C# separa para uso próprio. Você não pode usá-las como nomes de variáveis. Você as conhecerá muito bem quando terminar de ler este livro. Aqui estão algumas já usadas por você. Escreva o que você acha que essas palavras fazem no C#.

namespace

for

class

public

else

new

using

if

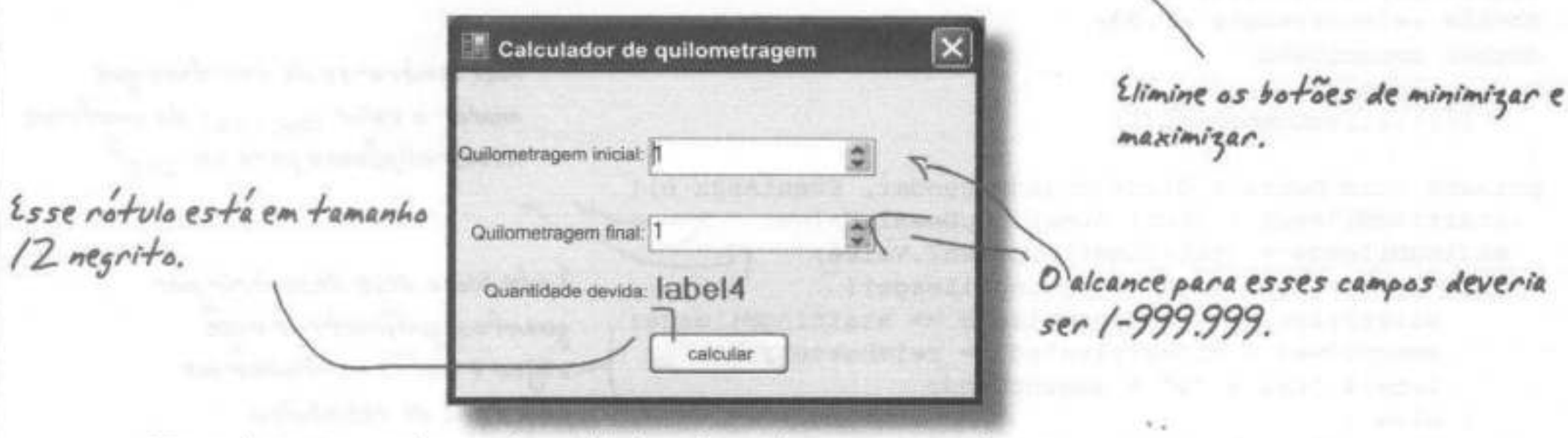
while

Respostas na página 145.

Exercício

Crie uma calculadora de reembolso para uma viagem de negócios. Ela deve permitir ao usuário colocar uma leitura inicial e uma final de hodômetro. Com os dois números, o usuário calculará quantos quilômetros viajou e descobrirá quanto deveria ter reembolsado se a empresa lhe paga **R$ 39** para cada milha gasta com seu carro.

1 **Comece com um novo projeto de aplicativo Windows.**
Faça o formulário parecer-se com isso:



Quando você terminar o Formulário, clique duas vezes no botão para acrescentar algum código ao projeto.

2 **Crie as variáveis necessárias para o calculador.**
Coloque as variáveis na definição de classe no topo de `Form1`. Você precisa de duas variáveis tipo inteiro para manter os registros da leitura inicial e final do hodômetro. Chame-as de `startingMileage` e `endingMileage` (milhagem inicial e milhagem final). Você precisa de três números que possam guardar casas decimais. Faça-os doubles e chame-os de milesTraveled (milhas Viajadas), `reimburseRate` (Taxa de reembolso) e `amountOwed` (quantia devida). Estabeleça o valor de `reimburseRate` para 0,39.

3 **Faça sua calculadora funcionar.**
Acrescente código no método button1_Click() para:

★ Certificar-se de que o número no campo Starting Mileage (Milhagem Inicial) seja menor do que o número no campo Ending Mileage (Milhagem Final). Se não for, mostre uma caixa de mensagem que diz "A quilometragem inicial deve ser menor que a quilometragem final". Escreva o título da caixa como "Impossível Calcular".

★ Subtraia o número inicial pelo final e então multiplique o resultado pela taxa de reembolso usando essas linhas:

```
milesTraveled = endingMileage -= startingMileage;
amountOwed = milesTraveled *= reimburseRate;
label4.Text = "$" + amountOwed;
```

4 **Execute.**
Certifique-se de que ela está informando os números certos. Tente mudar o valor inicial deixando-o maior que o final e veja se ela mostra a caixa de mensagem.

Solução do Exercício

Aqui vemos o código para a primeira parte do exercício.

```
public partial class Form1 : Form
{
  int startingMileage;
  int endingMileage;
  double milesTraveled;
  double reimburseRate = .39;
  double amountOwed;
  public Form1() {
      InitializeComponent();
  }
  private void button1_Click(object sender, EventArgs e){
    startingMileage = (int) numericUpDown1.Value;
    endingMileage = (int)numericUpDown2.Value;
      if (startingMileage <= endingMileage){
        milesTraveled = endingMileage -= startingMileage;
        amountOwed = milesTraveled *= reimburseRate;
        label4.Text = "$" + amountOwed;
      } else {
        MessageBox.Show(
      "A milha iniciar necessita ser menor do que a milha final",
            "Não consigo calcular a milhagem");
      }
  }
}
```

int *funciona muito bem para números inteiros. Este número poderia ir até 999.999. Então um* short *ou um* byte *não serviriam.*

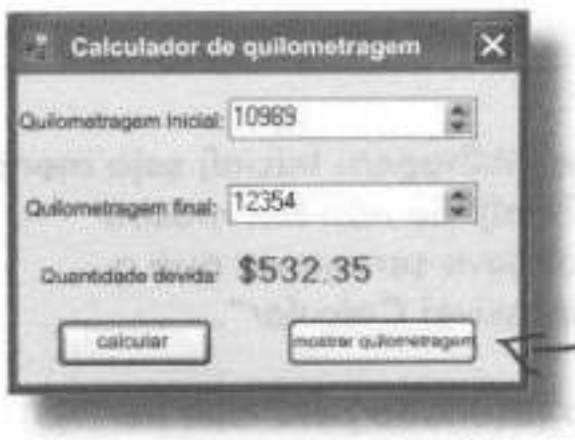
Você lembra-se de ter tido que mudar o valor decimal *do controle numericUpDown para um* int*?*

Este bloco deve descobrir por quantos quilômetros você viajou e então multiplicá-los pela taxa de reembolso.

Usamos uma forma alternativa de chamar o método MessageBox.Show() aqui. Demos a ele dois parâmetros: o primeiro é a mensagem a exibir e o segundo vai na barra de título.

Este botão parece estar funcionando, mas ele tem um grande problema. Você consegue identificá-lo?

1. **Agora adicione outro botão ao formulário.**
   Faça com que o número de quilômetros viajados seja exibido no formulário após você ter calculado o montante devido.



Clicar neste botão, após clicar em calculate (calcular), deve mostrar o número de quilômetros viajados numa caixa de mensagem.

   Quando você tiver terminado o Formulário, clique duas vezes no botão Mostrar Quilometragem para adicionar um pouco de código ao projeto.

2. **Uma linha já deve ser o suficiente.**
   Tudo que precisamos é fazer com que o formulário mostre a variável milesTraveled, certo? Então essa linha deve fazer isso:

```
private void button2_Click(object sender, EventArgs e) {
  Messagebox.Show(milesTraveled + " milhas", "Milhas Viajadas");
}
```

3. **Execute.**
   Digite alguns valores e veja o que acontece.

4. **Hum, algo não está certo...**
   O número de milhas é sempre igual ao débito. Por quê?

Combinando = com um operador

Dê uma boa olhada no operador que usamos para subtrair a quilometragem final da inicial (-=). O problema é que ele não subtrai apenas, ele também atribui um valor à variável no lado esquerdo do sinal de subtração. A mesma coisa acontece na linha onde multiplicamos número de quilômetros percorridos pela taxa de reembolso. Devemos substituir o -= e o *= por apenas – e *:

```
private void button1_Click(object sender, EventArgs e)
{
  startingMileage = (int) numericUpDown1.Value;
  endingMileage = (int)numericUpDown2.Value;
    if (startingMileage <= endingMileage){
      milesTraveled = endingMileage -= startingMileage;
      amountOwed = milesTraveled *= reimburseRate;
      label4.Text = "$" + amountOwed;
    } else {
      MessageBox.Show("A milha iniciar necessita ser menor do
            que a milha final",
            "Não consigo calcular a milhagem");
  }
```

Estes são chamados operadores compostos. Este subtrai startingMileage de endingMileage, mas também atribui um novo valor a endingMileage e milesTraveled ao mesmo tempo.

Assim é melhor – agora seu código não modificará endingMileage e milesTraveled.

```
milesTraveled = endingMileage - startingMileage;
amountOwed = milesTraveled * reimburseRate;
```

Então bons nomes de variáveis podem ajudá-lo? Com certeza! Dê uma olhada mais cuidadosa no que cada variável deve fazer. Você já tem muitas pistas no nome `milesTraveled` – você sabe que o formulário exibe esta variável incorretamente e como este valor tem que ser calculado. Então você pode tirar vantagem disso quando estiver observando seu código para tentar identificar a falha. Em vez disto, seria muito mais difícil achar o problema se as linhas incorretas se parecem com isso:

```
mT = eM -= sM;
aO = mT *= r
```

Variáveis nomeadas assim são inúteis para informá-lo sobre quais deveriam ser seus propósitos.

Objetos são variáveis também

Até agora, observamos objetos como se fossem separados de outros tipos. Mas um objeto é apenas outro tipo de dados. Seu código trata objetos exatamente como trata números, seqüências de caracteres e boleanos. Ele usa variáveis para trabalhar com eles:

Usando um int

1. Escreva um comando para declarar o inteiro.

```
int myInt;
```

2. Atribua um valor à nova variável.

```
myInt = 3761;
```

3. Use o inteiro em seu código.

```
while (i < myInt) {
```

Usando um objeto

1. Escreva um comando para declarar o objeto.

```
Dog spot;
```

2. Atribua um valor ao objeto.

```
spot = new Dog();
```

3. Cheque um dos campos do objeto.

```
while (spot.Happy) {
```

Então não importa se estou trabalhando com um objeto ou um valor. Se estiver indo para a memória e meu programa precisar usá-lo, utilizo uma variável.

Objetos são apenas mais um tipo de variável que seu programa pode usar.

Se seu programa precisa trabalhar com um número inteiro muito grande, use um long. Se ele precisa de um número inteiro pequeno, use um short. Se precisar de um valor verdadeiro/falso, use um booleano. E se ele precisa de algo que late e sente, use um Dog (cão). Não importa qual o tipo de dados com o qual seu programa precisa trabalhar, ele usará uma variável.

Refira-se aos seus objetos com variáveis de referência

Quando você cria um novo objeto, usa código como new Guy. Mas isso não é o suficiente; embora esse código crie um novo objeto Guy na pilha, ele não lhe dá uma forma de acessar o objeto. Você precisa de uma referência ao objeto. Então você cria uma variável de referência: uma variável de tipo Guy com um nome, como Joe. Então Joe é uma referência ao novo objeto Guy. Sempre que você quiser usar este cara específico, pode fazer referência a ele com a variável chamada Joe.

Isso é chamado de instanciar o objeto.

Então quando você tiver uma variável que seja um tipo de objeto, ela será uma variável de referência: uma referência a um objeto específico. Dê uma olhada:

Aqui está a pilha antes do seu código rodar. Não há nada lá.

```
public partial class Form1 : Form
{
    Guy Joe;
    public Form1()
    {
        InitializeComponent();
        Joe = new Guy();
    }
```

Esta variável chama-se Joe e irá referir-se a um objeto de tipo Guy.

Esta é a variável de referência...

...e este é o objeto ao qual Joe agora se refere.

Criar uma referência é como fazer uma etiqueta com uma etiquetadora - em vez de grudar uma nas suas coisas, você está usando-a para nomear um objeto, podendo referir-se a ele mais tarde.

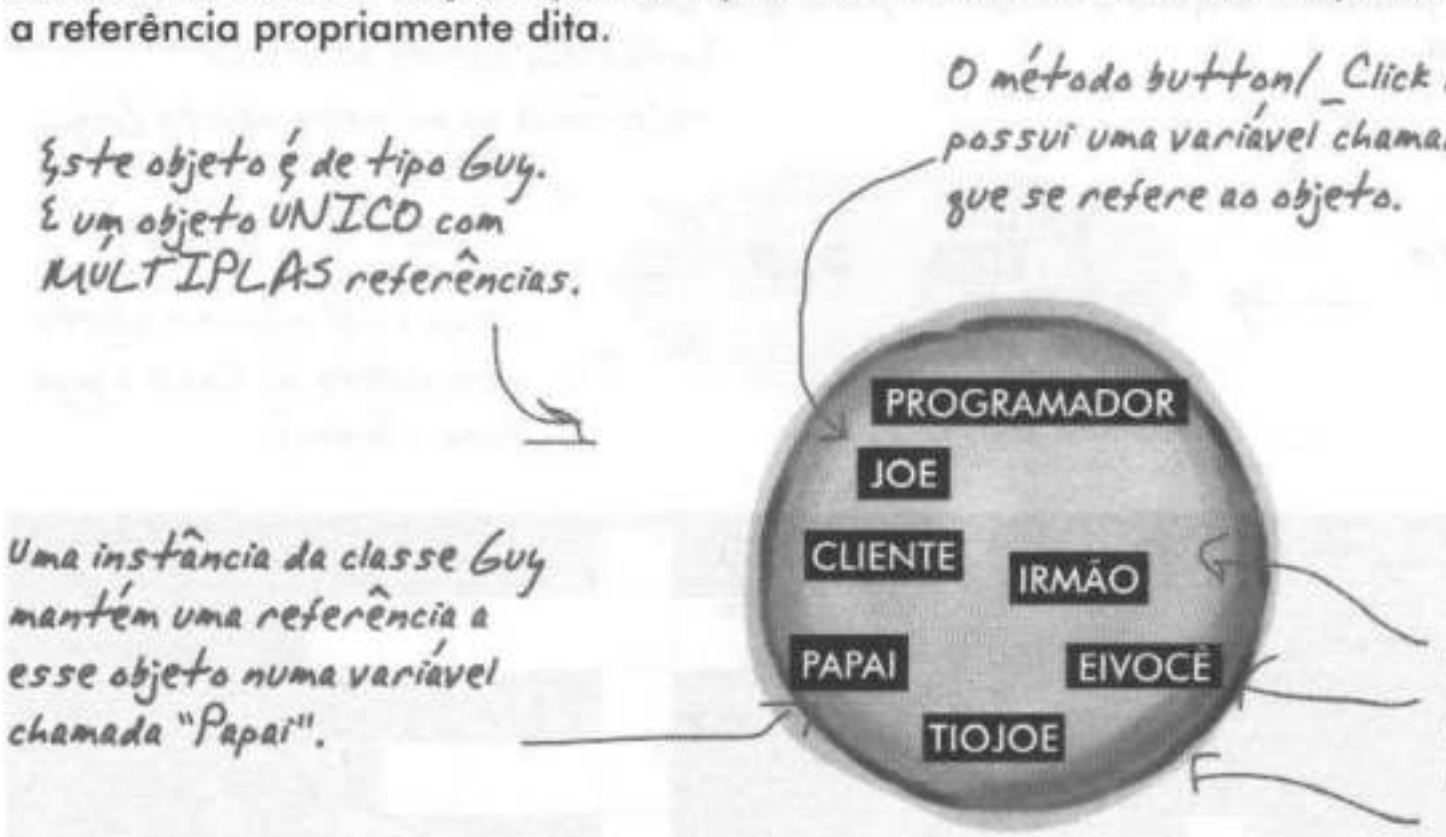
Aqui está a pilha depois que o código foi executado. Aqui vemos um objeto, com a variável Joe referindo-se a ele.

A ÚNICA forma de referir-se a este objeto guy é através da variável de referência Joe.

Referências são como etiquetas para seu objeto

Em sua cozinha, provavelmente, você tem um pote de sal e de açúcar. Se você trocasse seus nomes, faria uma refeição bem desagradável – embora os nomes tenham mudado, os conteúdos dos potes continuariam os mesmos. **Referências são como nomes**. Você pode mudar os nomes de posição, apontá-los para coisas diferentes, mas é o **objeto** que dita quais métodos e dados estão disponíveis, não a referência propriamente dita.

Quando seu código precisa funcionar com um objeto na memória, ele usa uma tonalidade, que é uma variável cujo tipo é uma classe de objeto para o qual ele se direcionará. Uma referência é como um nome que seu código usa para falar sobre um objeto específico.



Você nunca se refere ao seu objeto diretamente. Por exemplo, você não pode escrever código como `Guy.GiveCash()` se Guy for seu tipo de objeto. O compilador C# não sabe a qual Guy está se referindo, já que você pode ter muitas instâncias de Guy na pilha. Então você precisa de uma variável de referência, como joe, que você atribui a uma instância específica, como `Guy joe = new Guy()`.,

Agora você pode chamar métodos, como `joe.GiveCash()`. joe refere-se a uma instância específica da classe Guy, e seu compilador C# saberá exatamente qual instância usar. E, como você viu acima, você pode ter ***vários nomes apontando para a mesma instância***. Então você poderia dizer `Guy dad = joe` e chamar `dad.GiveCash()` (papai dá dinheiro). Isso também funciona – é isso que os filhos de Joe fazem todos os dias.

Se não há mais nenhuma referência, seu objeto é coletado como lixo

Se todos os nomes forem retirados de um objeto, mais nenhum programa pode acessá-lo. Isso quer dizer que o C# marca o objeto para coleta de lixo. Isso acontece quando o C# se livra de objetos sem referência e retoma a memória que os objetos usavam para que seu programa a aproveite.

Para um objeto continuar na pilha, ele tem que ter pelo menos uma referência. Quando a última referência ao objeto desaparece, o objeto some também.

1 **Eis aqui algum código que cria um objeto.**

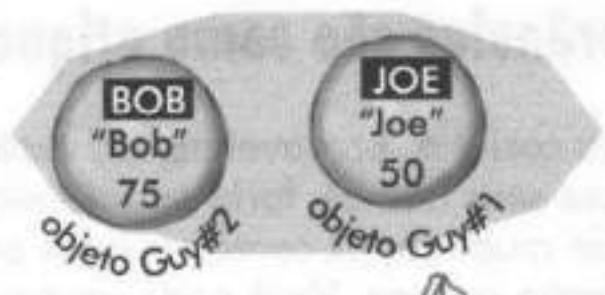
```
Guy joe = new Guy()
  { Name = "Joe", Cash = 50 };
```

A primeira linha tem uma declaração que criou a etiqueta. A segunda linha criou o objeto e colou a etiqueta nele.

2 **Agora vamos criar um segundo objeto.**

```
Guy bob = new Guy()
  { Name = "Bob", Cash = 75 };
```

Agora temos duas instâncias do objeto Guy e duas variáveis de referência: uma para cada Guy.



3 **Vamos pegar a referência ao primeiro objeto e mudá-la para que ela aponte para o segundo objeto.**

```
joe = bob;
```

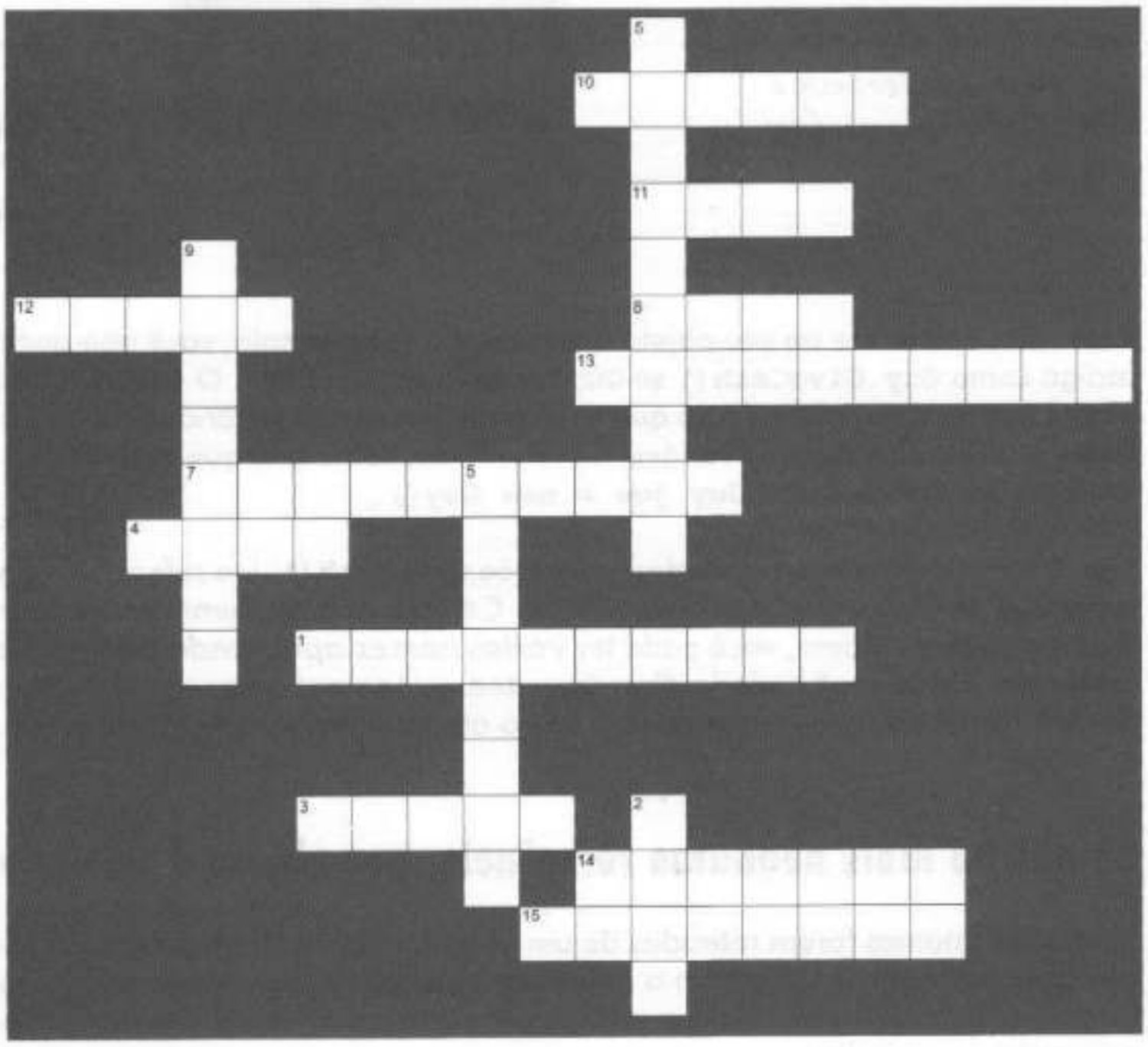
Agora joe está apontando para o mesmo objeto que bob.

Então não existe mais uma referência ao primeiro objeto Guy...

...logo o C# marca o objeto para coleta de lixo e o joga fora. Já era!

Cruzadinha de Tipo

Dê um tempo, sente-se e dê algo para o lado direito de seu cérebro fazer. É a sua cruzadinha padrão. Todas as respostas são palavras deste capítulo.
Quando terminar, vire a página e continue com o restante do capítulo.



Horizontais

1. Você pode combinar a declaração de variável e a _____ num único comando
4. Quando um objeto já não tem nenhuma referência apontando pra ele, é removido da pilha com o uso da coleta de _____
8. Uma declaração de variável sempre começa com isso
10. O tipo que guarda os maiores números
11. O tipo que guarda uma única letra ou número
12. Se você nunca estabelecer isso para uma variável, seu programa nunca compilará
13. Uma variável que aponta para um objeto
14. O que (int) faz nesta linha de código: x = (int) y;
15. Os quatro tipos de números inteiros que armazenam apenas números positivos

Verticais

2. A segunda parte de uma declaração de variável
3. Os nomes de variáveis como myBigNumber e bobTheDog usam esse estilo de maiúsculas e minúsculas
5. O que seu programa usa para trabalhar com dados na memória
6. O que você faz quando usa o operador + para ligar duas coisas
7. "namespace", "for", "while", "using" e "new" são exemplos de palavras _____
9. Todo objeto possui esse método que o converte em uma seqüência de caracteres

Respostas na página 146.

Múltiplas referências e seus efeitos colaterais

Você tem que ser cuidadoso quando começar a deslocar as variáveis de referência. Muitas vezes, pode parecer que você está apenas apontando uma variável para um objeto diferente. Mas você pode acabar removendo todas as referências para outro objeto no processo. Isso não é ruim, mas pode não ser o que você planejou. Dê uma olhada:

1
```
Dog rover = new Dog();
rover.Breed = "Greyhound";
```

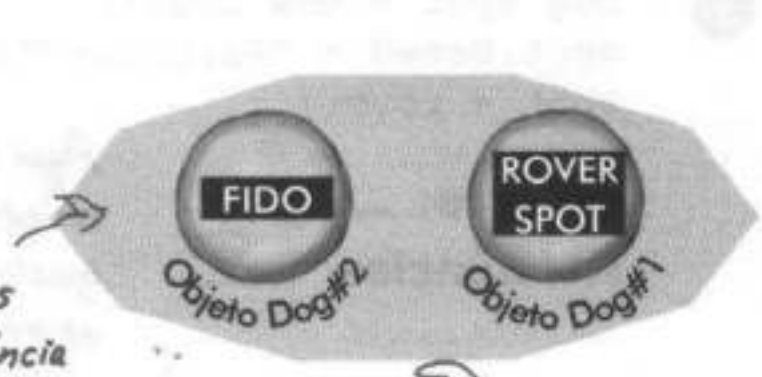
Objetos: 1
Referências: 1

Rover é um objeto Dog com um campo Breed (raça) chamado Greyhound.

2
```
Dog fido = new Dog();
fido.Breed = "Beagle";
Dog spot = rover;
```

Objetos: 2
Referências: 3

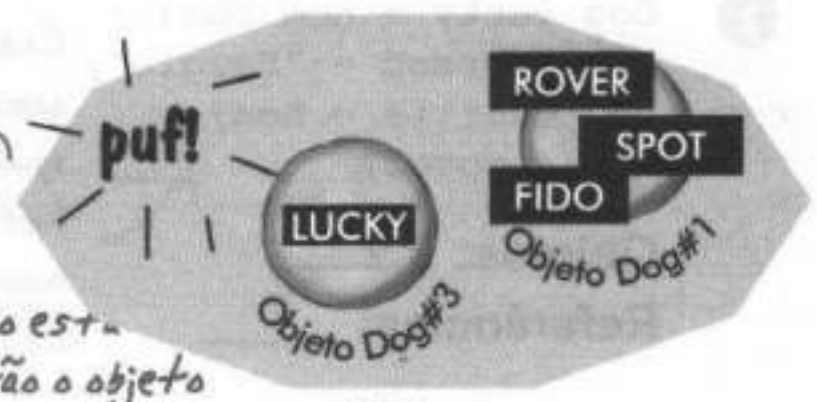
Fido é outro objeto Dog. Mas Spot é apenas outra referência ao primeiro objeto.



3
```
Dog lucky = new Dog();
lucky.Breed = "Dachshund";
fido = rover;
```

Objetos: 2
Referências: 4

Lucky é um terceiro objeto. Mas Fido está agora apontando para o objeto 1. Então o objeto 2 ficou sem referências. Ele não existe mais pelo menos em relação ao programa.



Aponte seu lápis

Agora é a sua vez. Aqui vemos um grande bloco de código. Descubra quantos objetos e referências existem em cada estágio. No lado direito, desenhe uma figura dos objetos e nomes na pilha.

1
```
Dog rover = new Dog();
rover.Breed = "Greyhound";
Dog rinTinTin = new Dog();
Dog fido = new Dog();
Dog quentin = Fido;
```

Objetos: _____
Referências: _____

2
```
Dog spot = new Dog();
spot.Breed = "Dachshund";
spot = rover;
```

Objetos: _____
Referências: _____

3
```
Dog lucky = new Dog();
lucky.Breed = "Beagle";
Dog charlie = fido;
fido = rover;
```

Objetos: _____
Referências: _____

4
```
rinTinTin = lucky;
Dog laverne = new Dog();
laverne.Breed = "pug";
```

Objetos: _____
Referências: _____

5
```
charlie = laverne;
lucky = rinTinTin;
```

Objetos: _____
Referências: _____

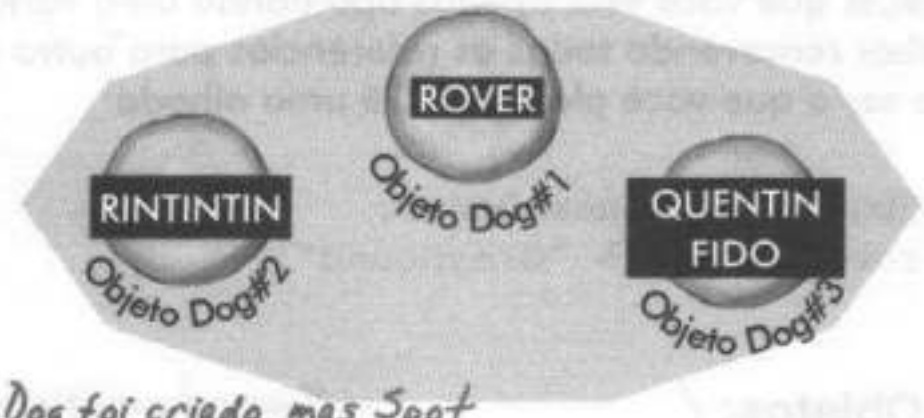
Aponte seu lápis Solução

Agora é a sua vez. Aqui vemos um grande bloco de código. Descubra quantos objetos e referências existem em cada estágio. No lado direito, desenhe uma figura dos objetos e nomes na pilha.

1

```
Dog rover = new Dog();
rover.Breed = "Greyhound";
Dog rinTinTin = new Dog();
Dog fido = new Dog();
Dog quentin = Fido;
```

Objetos: 3
Referências: 4

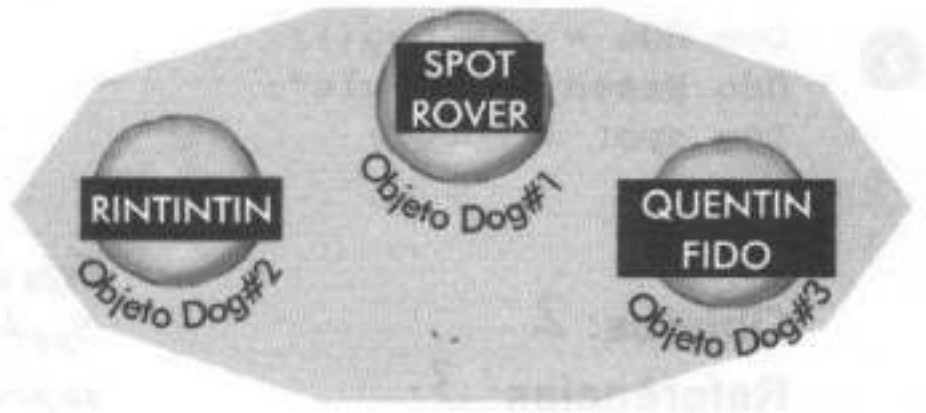


Um novo objeto Dog foi criado, mas Spot é a única referência a ele. Quando Spot é igualado a Rover, esse objeto é eliminado.

2

```
Dog spot = new Dog();
spot.Breed = "Dachshund";
spot = rover;
```

Objetos: 3
Referências: 5



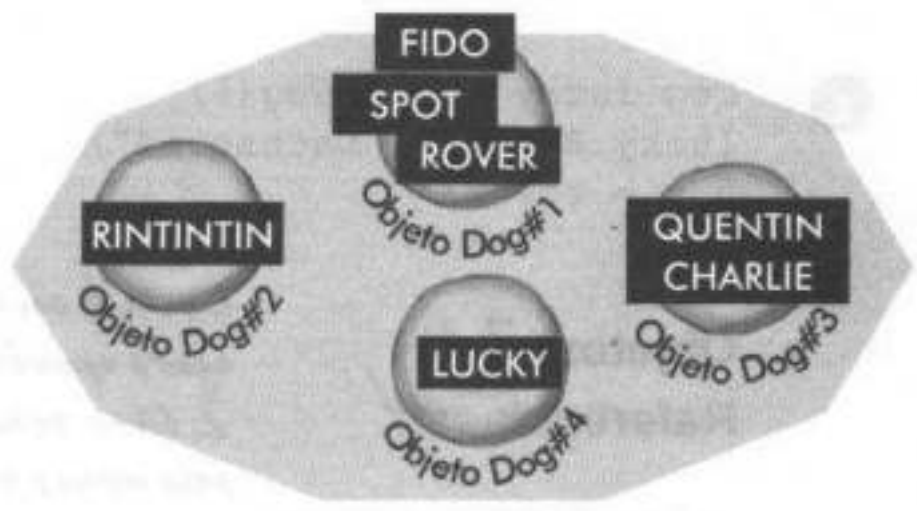
Aqui um novo objeto Dog é criado, mas quando Fido é igualado a Rover, o primeiro objeto apontado por Fido é eliminado.

3

```
Dog lucky = new Dog();
lucky.Breed = "Beagle";
Dog charlie = fido;
fido = rover;
```

Objetos: 4
Referências: 7

Charlie foi igualado a Fido quando ele ainda estava apontado para objeto 3. Então, depois disso, Fido moveu-se para o objeto 1, deixando Charlie para trás.



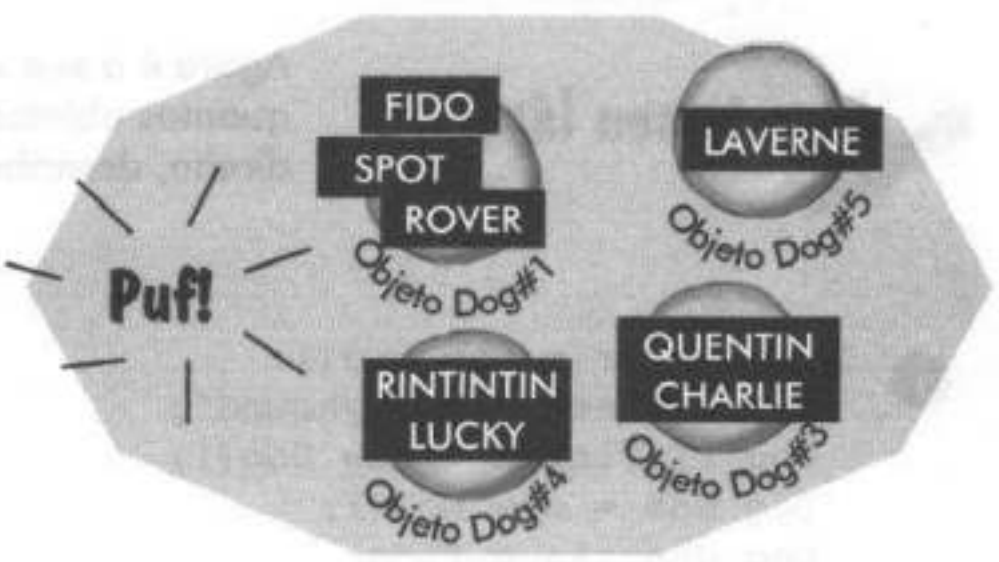
4

```
rinTinTin = lucky;
Dog laverne = new Dog();
laverne.Breed = "pug";
```

Objetos: 4
Referências: 8

Dog 2 perdeu sua última referência e foi eliminado.

Quando Lucky moveu-se para o objeto do Rin Tin Tin, o antigo objeto Lucky desapareceu.

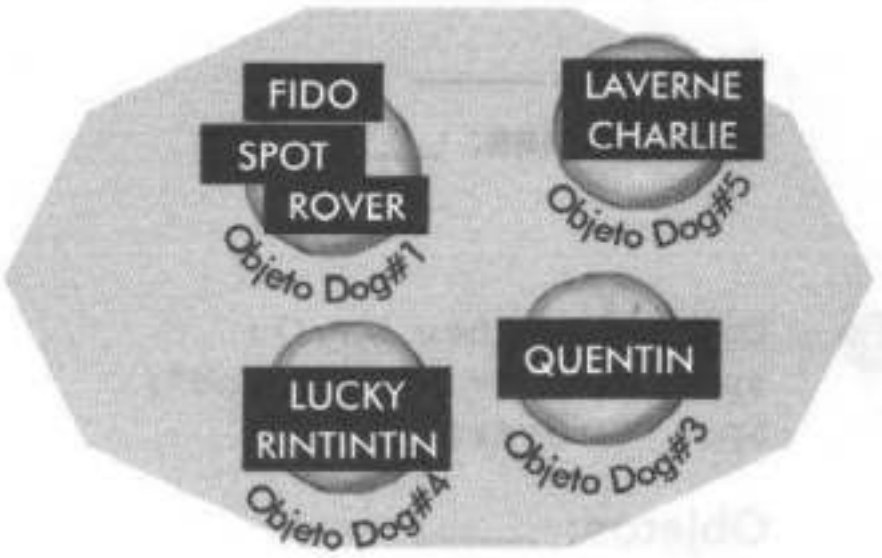


5

```
charlie = laverne;
lucky = rinTinTin;
```

Objetos: 4
Referências: 8

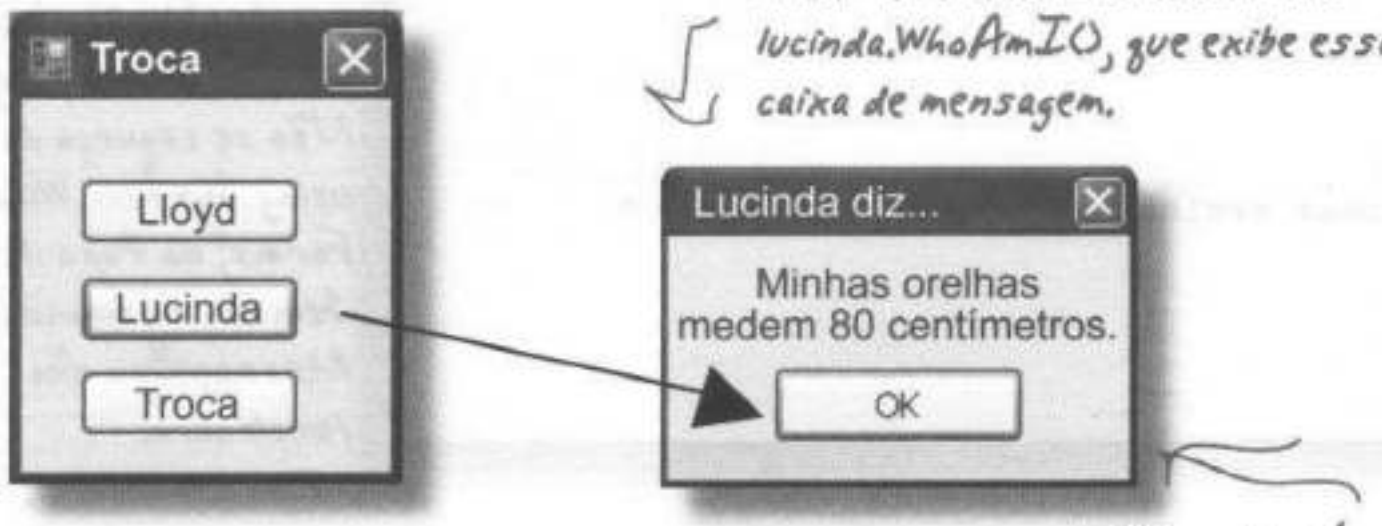
Aqui as referências movem-se por toda a parte, mas nenhum objeto novo é criado. E igualar Lucky a Rin Tin Tin não fez nada porque eles já apontavam para o mesmo objeto.



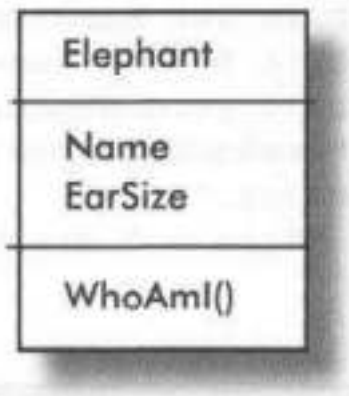
Exercício

Crie um programa com uma classe elefante. Faça duas instâncias de elefante e obrigue-as a trocarem suas propriedades, sem fazer nenhuma instância Elephant ser jogada fora.

1 **Comece com um novo projeto de aplicativo Windows.**
Faça o formulário parecer-se com isso:



Aqui está o diagrama de classe para a classe Elephant que precisa criar.



Isso te dá uma idéia sobre o que WhoAmIO deve retornar: uma única seqüência de caracteres com essa mensagem.

2 **Crie a classe Elephant.**
Adicione uma classe Elephant ao projeto. Dê uma olhada no diagrama de classe Elephant – você vai precisar de um campo int chamado EarSize (tamanho da orelha) e uma String chamada Name (certifique-se de que ambas sejam públicas). Então adicione um método chamado WhoAmI() (quem sou eu). Ele exibirá uma caixa de mensagem com informações sobre o nome e o tamanho da orelha do elefante.

3 **Crie duas instâncias de elefante e uma referência.**
Adicione dois campos Elephant na classe Form1 (na área logo abaixo da declaração de classe) chamados Lloyd e Lucinda. Inicialize-os de forma que eles tenham o nome e o tamanho de orelha corretos. Aqui estão os inicializadores do objeto Elephant para adicionar ao seu formulário:

```
lucinda = new Elephant() { Name = "Lucinda", EarSize = 33 };
lloyd = new Elephant() { Name = "Lloyd", EarSize = 40 };
```

4 **Faça os botões "Lloyd" e "Lucinda" funcionarem.**
Faça o botão Lloyd chamar lloyd.WhoAmI() e o Lucinda chamar lucinda.WhoAmI().

5 **Faça funcionar o botão swap (troca).**
Essa é a parte difícil. Faça o botão Swap trocar as duas referências, para que quando você clicar nele, as variáveis Lloyd e Lucinda troquem objetos e uma caixa "Objects swapped" (Objetos trocados) seja exibida. Teste seu programa clicando no botão Swap e, em seguida, nos outros dois botões. A primeira vez que você clicar nele, o botão Lloyd deve lançar a caixa de mensagens de Lucinda e o botão Lucinda, o inverso. Se você clicar no botão Swap novamente, tudo deve voltar ao normal.

A coleta de lixo do C# é feita com qualquer objeto sem referências. Então aqui vai uma dica: se você quiser passar cerveja de um copo para outro com água, você precisará de um terceiro para colocar a água...

Solução do Exercício

Crie um programa com uma classe elefante. Faça duas instâncias de elefante e obrigue-as a trocar propriedades sem fazer nenhuma instância Elephant ser jogada fora.

```
using System.Windows.Forms;
class Elephant {
    public int EarSize;
    public String Name;
    public void WhoAmI() {
      MessageBox.Show("Minhas orelhas medem " + EarSize + "
centimetros.",
        Name + " diz…");
    }
  }
```

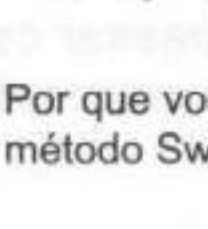
Este código define a classe Elephant no arquivo Elephant.cs que adicionamos ao projeto. Não se esqueça da linha using System.Windows.Forms; no topo da classe. Sem ela, o comando MessageBox não funcionará.

Aqui está o código da classe Form1 em Form1.cs.

```
public partial class Form1 : Form {
  Elephant lucinda;
  Elephant lloyd;
  public Form1()
  {
    InitializeComponent();
    lucinda = new Elephant()
        { Name = "Lucinda", EarSize = 33 };
    lloyd = new Elephant()
        { Name = "Lloyd", EarSize = 40 };
  }
  private void button1_Click(object sender, EventArgs e) {
    lloyd.WhoAmI();
  }
  private void button2_Click(object sender, EventArgs e) {
    lucinda.WhoAmI();
  }
  private void button3_Click(object sender, EventArgs e) {
    Elephant holder;
    holder = lloyd;
    lloyd = lucinda;
    lucinda = holder;
    MessageBox.Show("Objects swapped");
  }
}
```

Se você apenas direciona Lloyd para Lucinda, não haverá mais nenhuma referência direcionando para Lloyd e, seu objeto será descartado. É por isso que você precisa manter a referência Holder, conservando o objeto Lloyd até que Lucinda possa chegar até ela.

Não existe uma nova declaração para a referência porque nós não queremos criar uma nova instância do ELEPHANT.



PODER DO CÉREBRO

Por que você acha que não acrescentamos um método Swap() à classe Elephant?

Duas referências significam DUAS formas de mudar um dado do objeto

Além de perder todas as referências a um objeto, quando você possui múltiplas referências, você pode alterá-lo acidentalmente. Em outras palavras, uma referência a um objeto pode alterá-lo, enquanto outra referência àquele objeto nem imagina que algo mudou. Observe:

Faça isso

1. Acrescente outro botão em seu formulário.

2. Adicione esse código no botão. Você consegue adivinhar o que acontecerá quando clicar nele?

```
private void button4_Click(object sender, EventArgs e)
{
  lloyd = Lucinda;
  lloyd.EarSize = 4321;
  lloyd.WhoAmI();
}
```

Este comando diz para igualar EarSize a 4321 em qualquer objeto para o qual a referência lloyd esteja, porventura, apontando.

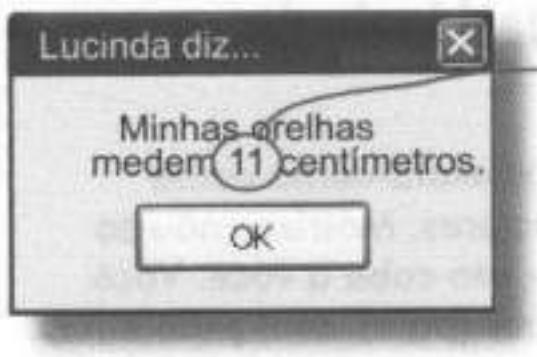
Você está chamando o método WhoAmIO do objeto lloyd.

Depois da execução deste código, as variáveis lloyd e lucinda referem-se ao MESMO objeto Elephant.

Mas lloyd aponta para a mesma coisa que lucinda.

3. OK, vá em frente e clique no novo botão. Espere um pouco, essa é a caixa de mensagem da Lucinda. Nós não chamamos o método WhoAmI() de Lloyd?

É a caixa de mensagem de lucinda...



Mas nós estabelecemos o EarSize usando a referência lloyd! O que acontece?

lloyd e lucinda agora são intercambiáveis. Mudanças em um afetam o objeto para o qual AMBOS apontam... não há mais uma diferença real entre lloyd e lucinda, pois eles apontam para o MESMO objeto.

Note que os dados NÃO estão sendo sobrescritos - as únicas coisas mudando aqui são as referências.

Um caso especial: matrizes

Se você tem que manter registro de muitos dados do mesmo tipo, como uma lista de alturas ou um grupo de cachorros, você pode fazê-lo usando uma matriz (array). O que a torna especial é que ela é um grupo de variáveis tratado como um objeto. Uma matriz é uma forma de armazenar e modificar mais de um objeto de dados sem ter que controlar cada variável individualmente. Quando você cria uma matriz, você a declara assim como qualquer outra variável, com um nome e um tipo:

Você pode combinar a declaração da variável myArray com sua inicialização - assim como com qualquer variável. Então isso ficaria assim:

```
bool[] myArray = new
bool[15];
```

Você declara uma matriz especificando seu tipo, seguido pelos colchetes.

```
bool[] myArray;
myArray = new bool[15];
myArray[4] = true;
```

Esta matriz possui 15 elementos dentro dela.

Você usa a palavra-chave new para criar uma matriz porque ela é um objeto. Então uma variável de matriz é um tipo de variável de referência.

Essa linha estabelece o valor do quinto elemento de myArray como verdadeiro. É o quinto porque o primeiro é myArray[0], o segundo é myArray[1], etc.

Use cada elemento numa matriz como se fosse uma variável normal

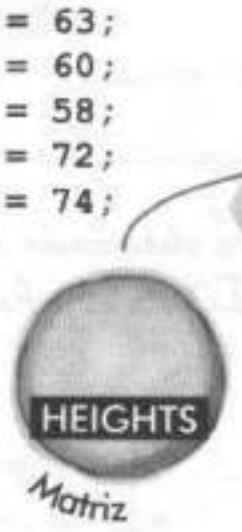
Aqui vemos um exemplo de código que declara e preenche uma matriz – e o que está acontecendo na pilha quando você faz isso. O primeiro elemento na array possui um **índice** de zero.

O tipo de cada elemento na array.

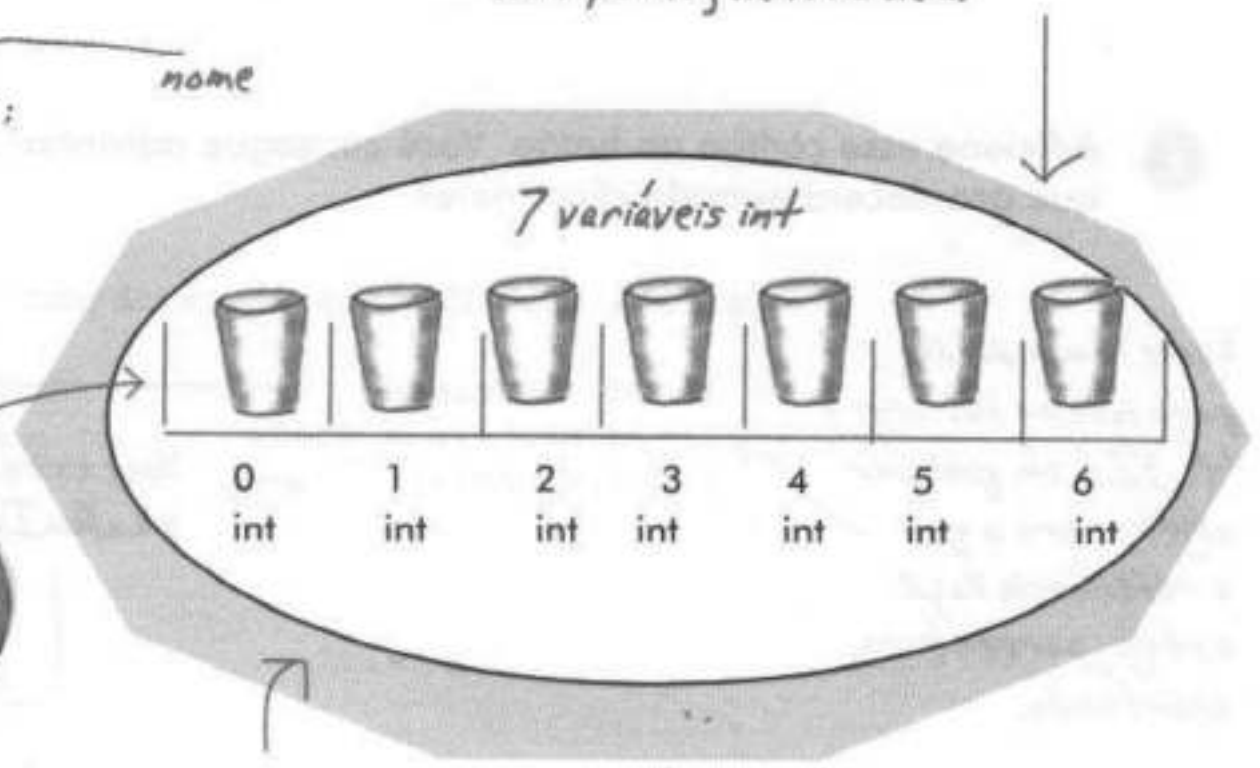
```
int[] heights;
heights = new int[7];
heights[0] = 68;
heights[1] = 70;
heights[2] = 63;
heights[3] = 60;
heights[4] = 58;
heights[5] = 72;
heights[6] = 74;
```

nome

Você refere-se a estes por índice, mas cada um funciona essencialmente como uma variável int normal.



Na memória, a matriz é armazenada como uma área contínua, mesmo que existam diversas variáveis, digamos do tipo int, dentro dela.



Note que a matriz é um objeto, embora os sete elementos sejam apenas tipos de valor – como aqueles nas primeiras duas páginas deste capítulo.

Matrizes podem conter um monte de variáveis de referência também

Você pode criar uma matriz de referências a objetos exatamente como você criou uma de números ou seqüências de caracteres. Matrizes não se importam com o tipo de variável que armazenam - isto cabe a você. Você pode ter uma matriz de ints, ou uma de objetos Duck (pato), sem problemas. Aqui vemos o código que cria uma matriz de sete variáveis Dog. A linha que inicializa a matriz apenas cria variáveis de referência. Já que há só duas linhas new Dog(), apenas duas instâncias reais da classe Dog são criadas.

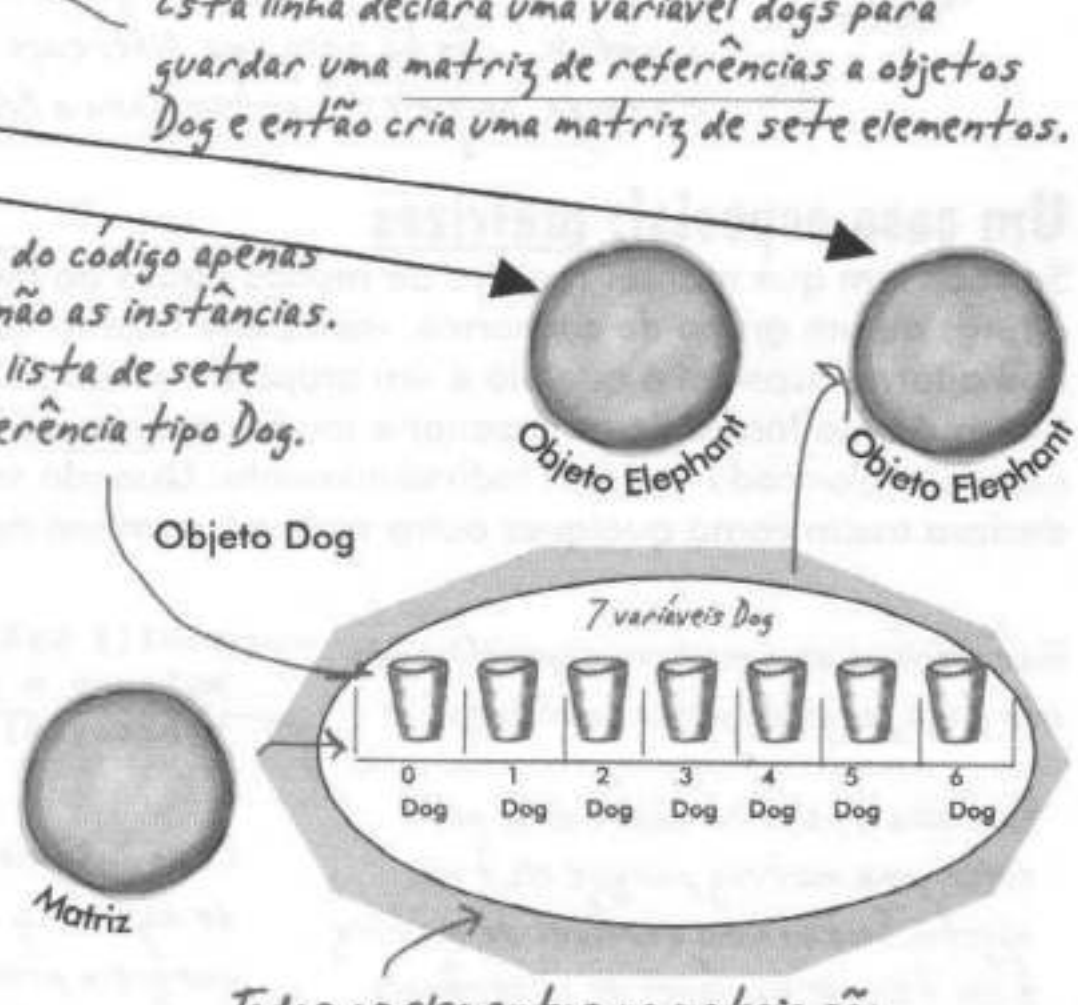
Quando você atribui ou recupera um elemento de uma matriz, o número dentro dos colchetes é chamado índice. O primeiro elemento na matriz possui um índice de zero.

```
Dog[] dogs = new Dog[7];
dogs[5] = new Dog();
dogs[0] = new Dog();
```

Esta linha declara uma variável dogs para guardar uma matriz de referências a objetos Dog e então cria uma matriz de sete elementos.

Essas duas linhas criam novas instâncias de Dog() e colocam-nas nas posições de índice 0 e 5.

A primeira linha do código apenas criou a matriz, não as instâncias. A matriz é uma lista de sete variáveis de referência tipo Dog.



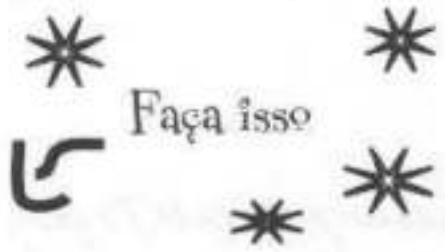
Todos os elementos na matriz são referências. Ela mesma é um objeto.

O comprimento de uma matriz

Você pode descobrir quantos elementos cabem numa array usando seu campo Lengh (comprimento, extensão). Então se você tem uma matriz chamada heights (alturas), pode usar heights.Length para descobrir o tamanho dela. Se há sete elementos na matriz, isto retornará a você 7 – ou seja, os elementos estão numerados de 0 a 6.

Bem-vindo à lanchonete de sanduíches com desconto do João Podrão!

João Podrão possui uma pilha de carne, um montão de pão e mais temperos do que você imagina. Mas o que ele não tem é um menu! Você consegue fazer um programa que crie um novo menu aleatório para ele todos os dias?



1 **Comece um novo projeto e adicione uma classe MenuMaker**
Se você precisa criar um menu, precisa de ingredientes. Matrizes são perfeitas para estas listas. Também vamos precisar escolher ingredientes aleatórios para juntar em um sanduíche. Felizmente, o Framework .NET possui uma classe embutida chamada Random (aleatória) que gera números aleatórios. Então teremos quatro campos em nossa classe: um campo Randomizer (desordemador), que armazena uma referência a um objeto Random, e três matrizes de strings, que armazenam carnes, tempero e pães.

A classe possui três campos para armazenar três matrizes diferentes de seqüências de string. Ela irá usá-los para criar os itens aleatórios do menu.

O campo Randomizer guarda uma referência a um objeto Random. Chamar seu método Next() gerará números aleatórios.

```
public class MenuMaker {
  public Random Randomizer;
  string[] Meats = { "Rosbife", "Salami", "Peru", "Presunto", "Pastrami" };
  string[] Condiments = { "mostarda amarela", "mostarda marrom",
      "mostarda com mel", "maionese", "gosto", "molho francês" };
  string[] Breads = { "centeio", "branco", "trigo", "pumpernickel",
      "pão italiano", "árabe" };
}
```

Lembre-se de usar colchetes para acessar um membro de uma matriz. O valor de Breads[2] é wheat (trigo).

2 **Acrescente um método GetMenuItem() para gerar um sanduíche aleatório**
O propósito da classe é gerar sanduíches, então adicionaremos um método para fazer exatamente isto. Ele usará o método Next() do objeto Random para escolher uma carne, condimento e pão de cada matriz aleatoriamente. Quando você passar um parâmetro int ao Next(), o método retorna um número aleatório menor do que este parâmetro. Então se seu objeto Random chama-se Randomizer, chamar Randomizer.Next(7) retornará um número aleatório entre 0 e 6.
Como saber qual parâmetro passar ao método Next()? Bem, isso é fácil – apenas passe o Length de cada matriz. Isso retornará o índice de um item aleatório nela.

O método GetMenuItem() retorna uma seqüência de caracteres com um sanduíche criado com elementos aleatórios das três matrizes.

```
public string GetMenuItem() {
  string randomMeat = Meats[Randomizer.Next(Meats.Length)];
  string randomCondiment = Condiments[Randomizer.Next(Condiments.Length)];
  string randomBread = Breads[Randomizer.Next(Breads.Length)];
  return randomMeat + " com " + randomCondiment + " no " + randomBread;
}
```

O método coloca um item aleatório da matriz Meats (carnes) em randomMeat passando Meats.Length ao método Next() do objeto Random. Já que há cinco itens na matriz Meats, Meats.Length é 5, então Next(5) retornará um número aleatório entre 0 e 4.

Como isso funciona...

O método randomizer.Next(7) gera um número aleatório menor que sete. Meats.Length retorna o número de elementos em Meats. Então randomizer.Next(Meats.Length) resulta num número aleatório maior ou igual a zero, mas menor do que o número de elementos na matriz Meats.

```
Meats[Randomizer.Next(Meats.Length)]
```

Meats é uma matriz de seqüências de caracteres. Ela tem cinco elementos, de 0 a 4. Por exemplo, Meats[0] é igual a Rosbife e Meats[3] é igual a Presunto.

3 **Crie seu formulário**

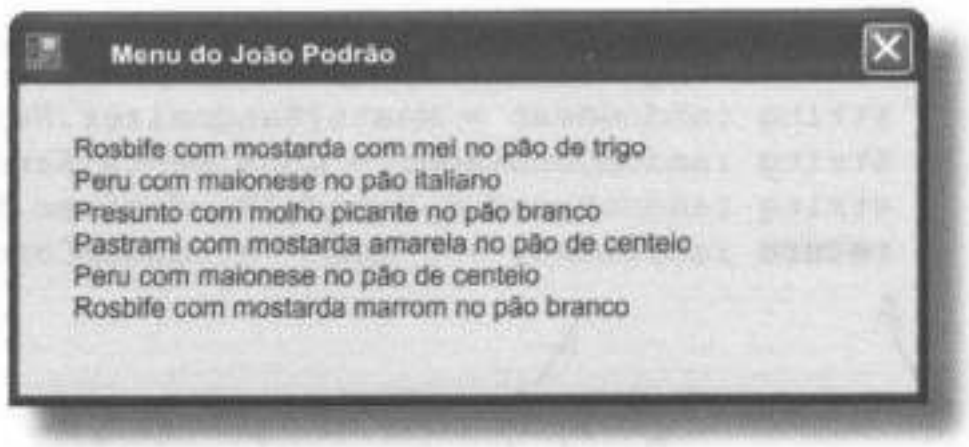
Adicione seis rótulos ao formulário, label1 até label6. Então adicione código para atribuir valor para cada propriedade Text deles usando um objeto MenuMaker. Você precisará inicializar o objeto usando uma nova instância da classe Random. Aqui está o código:

Use um inicializador de objeto para atribuir o campo Randomizer do objeto MenuMaker a uma nova instância da classe Random.

```
public Form1() {
   InitializeComponent();

   MenuMaker menu = new MenuMaker() { Randomizer = new Random() };
   label1.Text = menu.GetMenuItem();
   label2.Text = menu.GetMenuItem();
   label3.Text = menu.GetMenuItem();
   label4.Text = menu.GetMenuItem();
   label5.Text = menu.GetMenuItem();
   label6.Text = menu.GetMenuItem();
}
```

Agora você está pronto para gerar seis sanduíches diferentes e aleatórios usando o método GetMenuItem().

Quando você executar o programa, os seis nomes mostram seis sanduíches diferentes aleatórios.



Objetos usam referências para falar uns com os outros

Até aqui, você viu formulários comunicarem-se com objetos usando variáveis de referência para chamar seus métodos e checar seus campos. Objetos também podem chamar outros cada um de seus outros métodos usando referências. Na verdade, não há nada que um formulário possa fazer que seus objetos não possam, porque seu formulário é apenas outro objeto. E quando objetos se comunicam entre si, uma palavra-chave útil que possuem é this (isso). Sempre que um objeto usa a palavra-chave this, ele refere-se a ele mesmo – é uma referência que aponta para o objeto que chama.

1 **Aqui vemos um método para mandar um elefante falar**
Vamos adicionar um método à classe Elephant. Seu primeiro método é uma mensagem de um elefante. Seu segundo é o nome do elefante que disse:

```
public void TellMe(string message, Elephant whoSaidIt) {
  MessageBox.Show(whoSaidIt.Name + " diz: " + message);
}
```

Aqui está como ele fica quando é chamado:

```
Elephant lloyd = new Elephant() { Name = "Lloyd", EarSize = 40 };
Elephant lucinda = new Elephant() { Name = "Lucinda", EarSize = 33 };
lloyd.TellMe("Oi", lucinda);
```

Nós chamamos o método Lloyd deTellMe() e passamos dois parâmetros: "Oi" e uma referência para o objeto Lucinda. O método usa seu parâmetro whoSaidIt (quem disse) para acessar o parâmetro Name de qualquer elefante que tenha sido informado usando o segundo parâmetro do método.

2 **Aqui vemos um método que chama outro método**
Agora vamos adicionar um método SpeakTo() (falar com) na classe Elephant. Ele usa uma palavra-chave especial: this. Essa é uma referência que permite a um objeto falar dele mesmo.

```
public void SpeakTo(Elephant talkTo, string message) {
talkTo.TellMe(message, this);
}
```

Este método na classe Elephant chama um método TalkTo() de outra instância da mesma classe. Ele permite que um elephant se comunique com outro.

Vamos ver melhor como isso funciona.

```
lucinda.SpeakTo(lloyd, "Olá");
```

Quando o método SpeatTo() de Lucinda é chamado, ele usa seu parâmetro de referência talkTo para chamar o método TellMe() de Lloyd.

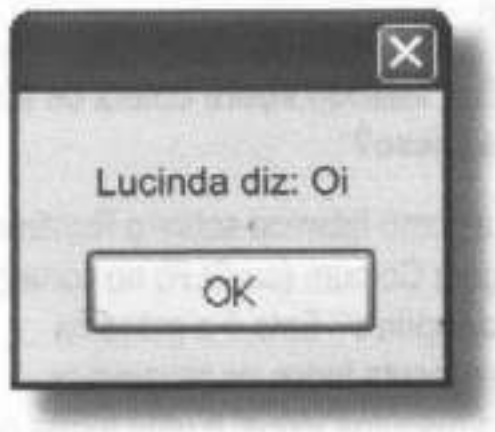
```
talkTo.TellMe(message, this);
```

Lucinda usa talkTo (que tem uma referência a Lloyd) para chamar TellMe()

this é substituído por uma referência ao objeto de Lucinda

```
lloyd.TellMe(message, [uma referência a Lucinda]);
```

Então Lloyd age como se tivesse sido chamado com ("Oi", lucinda) e mostra essa mensagem:



Onde nenhum objeto jamais esteve

Há outra palavra-chave importante que usaremos com objetos. Quando você cria uma nova referência e não a atribui a nada, ela tem um valor. Ela começa igualada a null (nulo), o que quer dizer que não está apontando para nada.

```
Dog fido;
Dog lucky = new Dog();
```

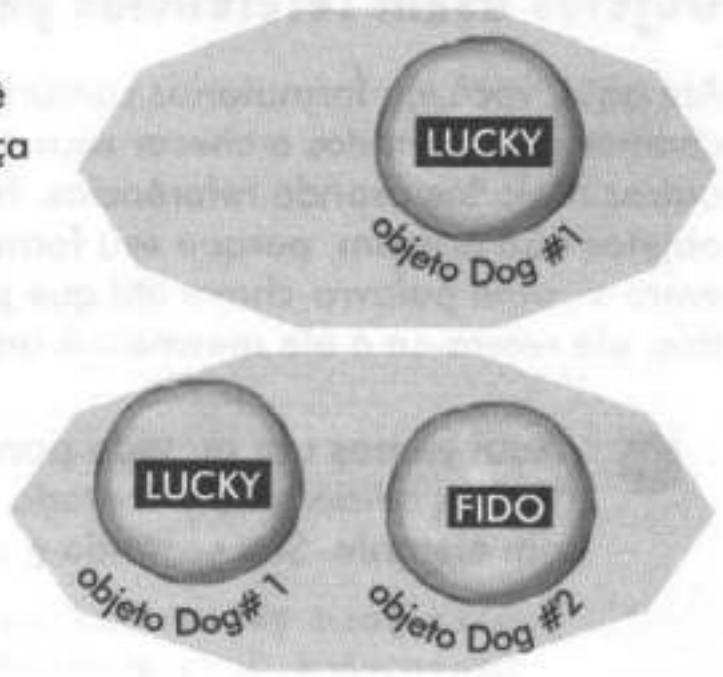
Neste momento, só há um objeto. A referência fido é igualada a null.

```
fido = new Dog();
```

Agora que fido está apontando para um objeto, ela não é mais igual a null.

```
lucky = null;
```

Quando igualamos lucky a null, ela não está mais apontando a seu objeto. Logo, ele é eliminado pela coleta de lixo.



não existem Perguntas Idiotas

P: Pergunto de novo – meu formulário é um objeto?

R: Sim! É por isso que seu código começa com uma declaração de classe. Abra o código para um formulário e veja você mesmo. Depois, abra Program.cs em qualquer programa que você tenha escrito até agora e olhe o método InitializeComponent() dentro dele – você encontrará "new Form1()".

P: Por que eu usaria null?

R: Há algumas formas típicas de usar null em programas. A mais comum é testando:

Esse teste retornará true se a referência lloyd for nula. `if (lloyd == null) {`
Outra forma de ver a palavra-chave null sendo usada é quando você quer que seu objeto seja eliminado pela coleta de lixo. Se você possui uma referência a um objeto e terminou de usá-lo, igualar a referência a null imediatamente irá marcá-lo como lixo (a não ser que exista outra referência a ele em algum outro lugar).

P: Você fica falando sobre coleta de lixo, mas o que é isso?

R: Lembra como falamos sobre o Runtime de Linguagem Comum (ou CLR) no começo do primeiro capítulo? Esta é a máquina virtual que executa todos os programas .NET. Uma máquina virtual é uma forma de isolar programas em execução do restante do sistema operacional. Uma coisa que ela faz é administrar a memória que usa. Ou seja, ela mantém registro de todos os seus objetos, descobre quando a última referência ao objeto desaparece e libera a memória que estava usando.

P: Eu ainda não sei se entendi direito como as referências funcionam.

R: Referências são a forma de você usar todos os métodos e campos num objeto. Se você criar uma referência ao objeto Dog, pode usá-la para acessar qualquer método criado no objeto Dog. Se você tem um método (não-estático) chamado Dog.Bark() (latir) ou Dog.Beg() (implorar), você pode criar uma referência chamada spot. Então você pode usá-la para acessar spot.Bark() ou spot.Beg(). Você também poderia alterar informação nos campos do objeto usando a referência. Assim você poderia alterar um campo Breed usando spot. Breed.

P: Espere, então isso significa que sempre que eu mudar um valor através de uma referência, estou mudando-o para todas as outras referências a esse objeto também?

R: Sim. Se rover é uma referência ao mesmo objeto que spot, alterar rover.Breed para "beagle" transformaria spot.Breed em "beagle".

P: Volte um pouco em tipos de valor. Por que eu não posso enfiar um número pequeno numa variável, vindo de um tipo maior, mesmo que ele seja pequeno o suficiente?

R: Certo. A questão sobre variáveis é que elas atribuem um tamanho ao seu número, não importa o quão grande é seu valor. Então se você nomeia uma variável e lhe dá um tipo long mesmo que o número seja bem pequeno (como, digamos, 5), o C# reserva memória o bastante para que ele fique bem grande. Quando você pensa nisso, percebe que é bem útil. Afinal, elas são chamadas variáveis porque mudam o tempo todo. O C# presume que você sabe o que está fazendo e não dará a uma variável um tipo de que não precisa. Então embora o número possa não ser grande agora, há uma chance de que após um pouco de matemática isso aconteça - ele pode mudar e o C# reserva para ele memória suficiente para lidar com qualquer faixa de valores que você use.

P: Repita uma coisa pra mim – o que "this" faz?

R: this é uma variável especial que você só pode usar dentro de um objeto. Quando você está dentro de uma classe, você usa this para se referir a qualquer campo ou método daquela instância em particular. Ela é especialmente útil quando você está trabalhando com um classe cujos métodos chamam outras classes. Um objeto pode usá-la para enviar uma referência que aponte para ele mesmo para outro objeto. Então se Spot chama um dos métodos de Rover passando this como um parâmetro, ele está dando a Rover uma referência ao objeto Spot.

Sempre que você tem código em um objeto que será instanciado, a instância pode usar a variável especial this – uma referencia que aponta para o próprio objeto.

Pontos de Bala

- Há tipos de valor para números que armazenam tamanhos diferentes de números.
- Os maiores números devem ser do tipo long e os menores (até 128) podem ser declarados como bytes.
- Quando você declara uma variável, você SEMPRE dá a ela um tipo. Às vezes, você combina a declaração com atribuição de valor.
- Cada tipo de valor tem um tamanho e você não pode colocar um valor de tipo maior em uma variável de tipo menor, não importa qual seja o tamanho efetivo dos dados.
- Quando você estiver usando valores literais, use o sufixo F para indicar um float (15.6F) e M para um decimal (36.12M).
- O compilador não permitirá que você iguale uma variável a um valor de um tipo diferente a não ser que haja a coerção de dados.
- Há algumas palavras reservadas pela linguagem e você não pode nomear suas variáveis com elas. Por exemplo, for, while, using, new e outras fazem coisas específicas na linguagem.
- Referências são como etiquetas. Você pode ter quantas referências a um objeto quiser e todas tratam da mesma coisa.
- Se um objeto não possui uma referência, é coletado como lixo.

Aponte seu lápis

Aqui temos uma matriz de objetos Elephant e um laço que iterará por ela e encontrará aquele com as maiores orelhas. Qual é o valor do biggestEars.Ears (maioresOrelhas.Orelhas) após cada iteração do laço for?

```
private void button1_Click(object sender, EventArgs e)
{
  Elephant[] elephants = new Elephant[7];
  elephants[0] = new Elephant() { Name = "Lloyd", EarSize = 40 };
  elephants[1] = new Elephant() { Name = "Lucinda", EarSize = 33 };
  elephants[2] = new Elephant() { Name = "Larry", EarSize = 42 };
  elephants[3] = new Elephant() { Name = "Lucille", EarSize = 32 };
  elephants[4] = new Elephant() { Name = "Lars", EarSize = 44 };
  elephants[5] = new Elephant() { Name = "Linda", EarSize = 37 };
  elephants[6] = new Elephant() { Name = "Humphrey", EarSize = 45 };
  Elephant biggestEars = elephants[0];
  for (int i = 1; i < elephants.Length; i++)
  {
    if (elephants[i].EarSize > biggestEars.EarSize)
    {
      biggestEars = elephants[i];
    }
  }
  MessageBox.Show(biggestEars.EarSize.ToString());
}
```

Estamos criando uma matriz de sete referências Elephant().

Toda matriz começa com índice 0. Logo, o primeiro elephant na array é Elephants[0].

Tome cuidado – esse laço começa com o segundo elemento da matriz (com índice 1) e itera seis vezes até que i seja igual ao tamanho da matriz.

Esta linha fawwz a referência biggestEars apontar para qualquer elefante que elephants[i] indique.

Iteração 1 biggestEars.EarsSize = _____

Iteração 2 biggestEars.EarsSize = _____

Iteração 3 biggestEars.EarsSize = _____

Iteração 4 biggestEars.EarsSize = _____

Iteração 5 biggestEars.EarsSize = _____

Iteração 6 biggestEars.EarsSize = _____

Aponte seu lápis Solução

Aqui temos uma matriz de objetos Elephant e um laço que iterará por ela e encontrará aquele com as maiores orelhas. Qual é o valor do biggestEars.Ears (maioresOrelhas.Orelhas) após cada iteração do laço for?

```
private void button1_Click(object sender, EventArgs e)
{
  Elephant[] elephants = new Elephant[7];
  elephants[0] = new Elephant() { Name = "Lloyd", EarSize = 40 };
  elephants[1] = new Elephant() { Name = "Lucinda", EarSize = 33 };
  elephants[2] = new Elephant() { Name = "Larry", EarSize = 42 };
  elephants[3] = new Elephant() { Name = "Lucille", EarSize = 32 };
  elephants[4] = new Elephant() { Name = "Lars", EarSize = 44 };
  elephants[5] = new Elephant() { Name = "Linda", EarSize = 37 };
  elephants[6] = new Elephant() { Name = "Humphrey", EarSize = 45 };
  Elephant biggestEars = elephants[0];
  for (int i = 1; i < elephants.Length; i++)
  {
    if (elephants[i].EarSize > biggestEars.EarSize)
    {
      biggestEars = elephants[i];
    }
  }
  MessageBox.Show(biggestEars.EarSize.ToString());
}
```

Você lembrou que o laço começa com o segundo elemento da matriz? Por que você acha que isso foi feito assim?

O laço for começa com o segundo elefante e compara-o a qualquer outro que biggestEars aponte. Se as suas orelhas são as maiores, ele aponta biggestEars para aquele elefante. Então ele vai para o próximo e para o próximo... no final do laço, biggestEars aponta para aquele com as maiores orelhas.

A referência biggestEars é usada para manter registro sobre qual elemento vimos enquanto executamos o laço for que possui as maiores orelhas até o momento.

Iteração 1 biggestEars.EarsSize = 40

Iteração 2 biggestEars.EarsSize = 42

Iteração 3 biggestEars.EarsSize = 42

Iteração 4 biggestEars.EarsSize = 44

Iteração 5 biggestEars.EarsSize = 44

Iteração 6 biggestEars.EarsSize = 45

Ímas de geladeira

O código para um botão está todo embaralhado na geladeira. Você consegue reorganizar os trechos para formar um método de trabalho que produz o resultado listado abaixo?

```
int y = 0;
```

```
refNum = index[y];
```

```
islands[0] = "Bermuda";
islan ds[1] = "Fiji";
islands [2] = "Azores";
islands[3] = "Cozumel";
```

```
int refNum;
while (y < 4) {
```

```
result += islands[refNum];
```

```
Message Box.Show(result);
```

```
index[0] = 1;
index[1] = 3;
index[2] = 0;
index[3] = 2;
```

```
}
```

```
}
```

```
String[] islands = new String[4];
```

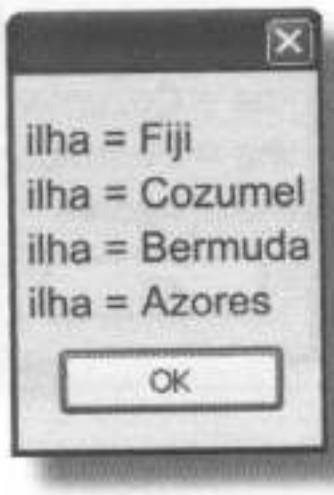
```
result += "\nisland = ";
```

```
int[] index = new int[4];
```

```
y = y + 1;
```

```
private void button1_Click (object sender, EventArgs e)
{
```

```
String result = "";
```



Solução dos ímas de geladeira

O código para um botão está todo embaralhado na geladeira. Você consegue reorganizar os trechos para formar um método de trabalho que produz o resultado listado abaixo?

```
private void button1_Click (object sender, EventArgs e)
{
```

```
String result = "";
```

```
int[] index = new int[4];
```

É aqui que a matriz index[] é inicializada.

```
index[0] = 1;
index[1] = 3;
index[2] = 0;
index[3] = 2;
```

```
String[] islands = new String[4];
```

```
islands[0] = "Bermuda";
islan ds[1] = "Fiji";
islands [2] = "Azores";
islands[3] = "Cozumel";
```

A matriz islands[] é inicializada aqui.

```
int y = 0;
```

```
int refNum;
while (y < 4) {
```

A string result é feita utilizando-se o operador += para concatenar linhas nela.

O loop while puxa um valor da matriz index[] e o usa como o índice na matriz islands[].

```
refNum = index[y];
```

```
result += islands[refNum];
```

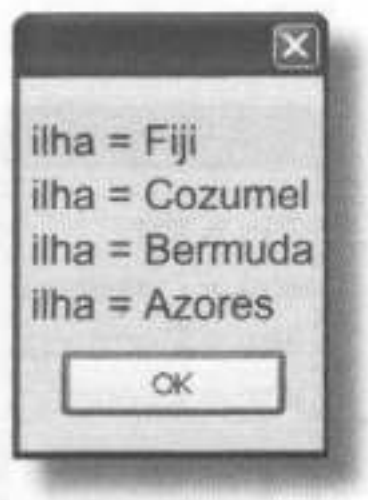
```
result += "\nisland = ";
```

```
y = y + 1;
```

```
}
```

```
Message Box.Show(result);
```

```
}
```



Jogos de Verão

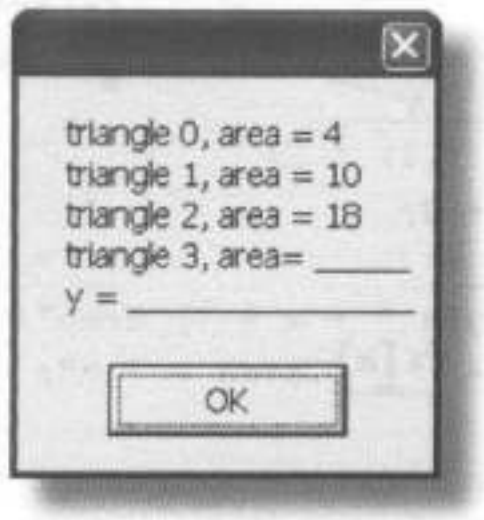
A sua tarefa é pegar trechos de código da piscina e colocá-los nas linhas em branco no código. Você **pode** usar o mesmo trecho mais de uma vez e não precisa usar todos os trechos. Seu objetivo é fazer uma classe que compile, execute e produza o resultado mostrado.

```
class Triangle
{
  double area;
  int height;
  int length;
  public static void Main(String[] args)
  {
    string results = "";
    __________
    _____________________________________
    while ( ________ )
    {
      _____________________________
      _____.height = (x + 1) * 2;
      _____.length = x + 4;
      _____________________
      results += "triangle " + x + ", area";
      results += " = " + _____.area + "\n";
      ____________
    }
    ____________
    x = 27;
    Triangle t5 = ta[2];
    ta[2].area = 343;
    results += "y = " + y;
    MessageBox.Show(results +
      ", t5 area = " + t5.area);
  }
  void setArea()
  {
    _____________ = (height * length) / 2;
  }
}
```

Aqui está o ponto de entrada para o aplicativo. Presuma que seja um arquivo com as linhas using certas no topo.

Dica: SetArea() NÃO é um método estático. Volte ao capítulo 3 para relembrar o que a palavra-chave static significa.

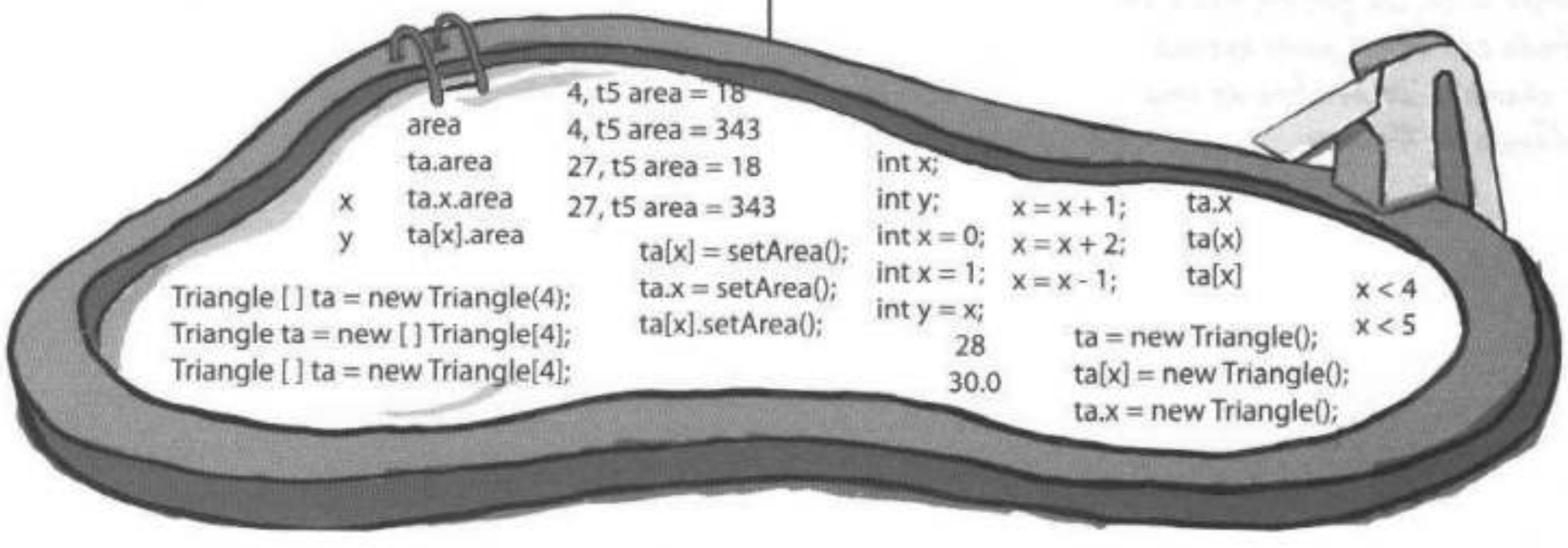
Resultado



Pergunta de Bônus!

Para pontos de bônus extras, use trechos da piscina para completar as duas lacunas do resultado.

Nota: Cada trecho da piscina pode ser usado mais de uma vez



Solução dos Jogos de Verão

Percebe como esta classe contém o ponto de entrada, mas ela também cria uma instância dela mesma? Isso é completamente válido no C#.

```
class Triangle
{
  double area;
  int height;
  int length;
  public static void Main(String[] args)
  {
    string results = "";
    int x = 0;
    Triangle [] ta = new Triangle [4];
    while ( x<4 )
    {
      ta [x] = new Triangle();
      ta [x].height = (x + 1) * 2;
      ta [x].length = x + 4;
      ta [x].setArea();
      results += "triangle " + x + ", area";
      results += " = " + ta [x].area + "\n";
      x = x + 1;
    }
    int y = x;
    x = 27;
    Triangle t5 = ta[2];
    ta[2].area = 343;
    results += "y = " + y;
    MessageBox.Show(results +
      ", t5 area = " + t5.area);
  }
  void setArea()
  {
    area = (height * length) / 2;
  }
}
```

Após esta linha, temos uma matriz de quatro referências Triangle – mas ainda não existe nenhum objeto Triangle!

O laço while cria as quatro instâncias de Triangle chamando o comando new quatro vezes.

Resposta do Bônus

O método SetArea() usa os campos height e length para criar o campo area. Já que ele não é um método estático, pode apenas ser chamado de dentro de uma instância de Triangle.

Catarina organiza aniversários também

Agora que você fez seu programa funcionar, Catarina está usando-o o tempo todo. Mas ela não organiza só jantares – ela organiza aniversários também; e os preços são um pouco diferentes. Ela precisará de você para acrescentar aniversários ao programa dela.

Acabei de receber um pedido de uma festa de aniversário para 10 pessoas. O seu programa pode lidar com isso?

Estes dois são iguais ao do jantar.

Estimativa de custo para um aniversário

- R$25 por pessoa.
- Há duas opções para o custo da decoração. Se um cliente opta por uma decoração normal, são R$7,50 por pessoa com uma taxa de R$30. Um cliente também pode sofisticar a decoração da festa com a "Opção Chique" – que custa R$15 por pessoa com uma taxa única de R$50.
- Quando a festa tem quatro pessoas ou menos, use um bolo de 20 cm (R$40). Caso contrário, ela usa um de 40 cm (R$75).
- Escrever no bolo custa R$0,25 para cada letra. O bolo de 20 cm pode ter até 16 letras escritas e o de 40, até 40 letras.

O aplicativo deve lidar com os dois tipos de festas. Use um controle de aba e uma aba para cada tipo de festa.

Grande parte das alterações tem a ver com bolos e a escrita neles.

Precisamos de uma classe BirthdayParty

Modificar seu programa para calcular o custo das festas de aniversário de Catarina significa acrescentar uma nova classe e alterar o formulário para permitir que você lide com os dois tipos de festa.

BirthdayParty
NumberOfPeople CostOfDecorations CakeSize CakeWriting
CalculateCostOfDecorations() CalculateCost()

Aqui vemos o precisamos fazer:

Você fará tudo isso em um minuto – mas primeiro você precisará ter uma idéia do que o trabalho envolve.

1. **Crie uma nova classe BirthdayParty**
   Sua nova classe `BirthdayParty` (Festa de Aniversário) precisará calcular os custos, lidar com as decorações e checar o tamanho da escrita no bolo.

2. **Adicione um TabControl ao seu formulário**
   Cada aba no formulário parece bastante com o controle `GroupBox` usado para mostrar quanto dinheiro o João e o Beto tinham lá no capítulo 3. Apenas clique na aba que deseja exibir e arraste controles para dentro dela.

3. **Dê um nome para a primeira aba e passe os controles do Dimmer (Jantar) para dentro dela**
   Você arrastará cada um dos controles que lidam com o jantar para dentro da nova aba. Eles funcionarão exatamente como antes, mas só serão exibidos quando a aba jantar for selecionada.

Solução do Exercício

Há cerca de 77 **palavras reservadas** no C#. Elas são separadas pelo compilador C# para uso próprio. Você não pode usá-las como nomes de variáveis. Você as conhecerá muito bem quando terminar de ler este livro. Aqui estão algumas que você já usou. Escreva o que você acha que essas palavras fazem no C#.

namespace

Namespaces servem para ter certeza de que os nomes em uso no seu programa não colidirão com aqueles usados no Framework .NET ou em outras classes externas usadas no seu programa. Todas as classes e métodos em um programa estão dentro de um namespace.

for

Ele permite que você faça um laço que executa três comandos. Primeiro ele declara a variável que usará, depois um comando que avalia a variável contra uma condição. O terceiro comando faz algo com o valor.

class

Uma classe é como você define um objeto. Classes possuem propriedades e métodos. Propriedades são o que elas sabem e métodos o que elas fazem.

public

Uma classe pública pode ser usada por todas as outras classes no projeto. Quando um método ou variável são declarados como públicos, podem ser usados por classes e chamados por métodos fora daquela em que estão sendo declarados.

else

Código que apareça seguindo o else será executado se o comando if anterior a ele falhar.

new

Você utiliza-o para criar uma nova instância de um objeto.

using

Essa é uma forma de listar todos os namespaces que você está usando em seu programa. Using permite que você use código do framework .NET e classes predefinidas de terceiros, assim como classes que você mesmo pode fazer.

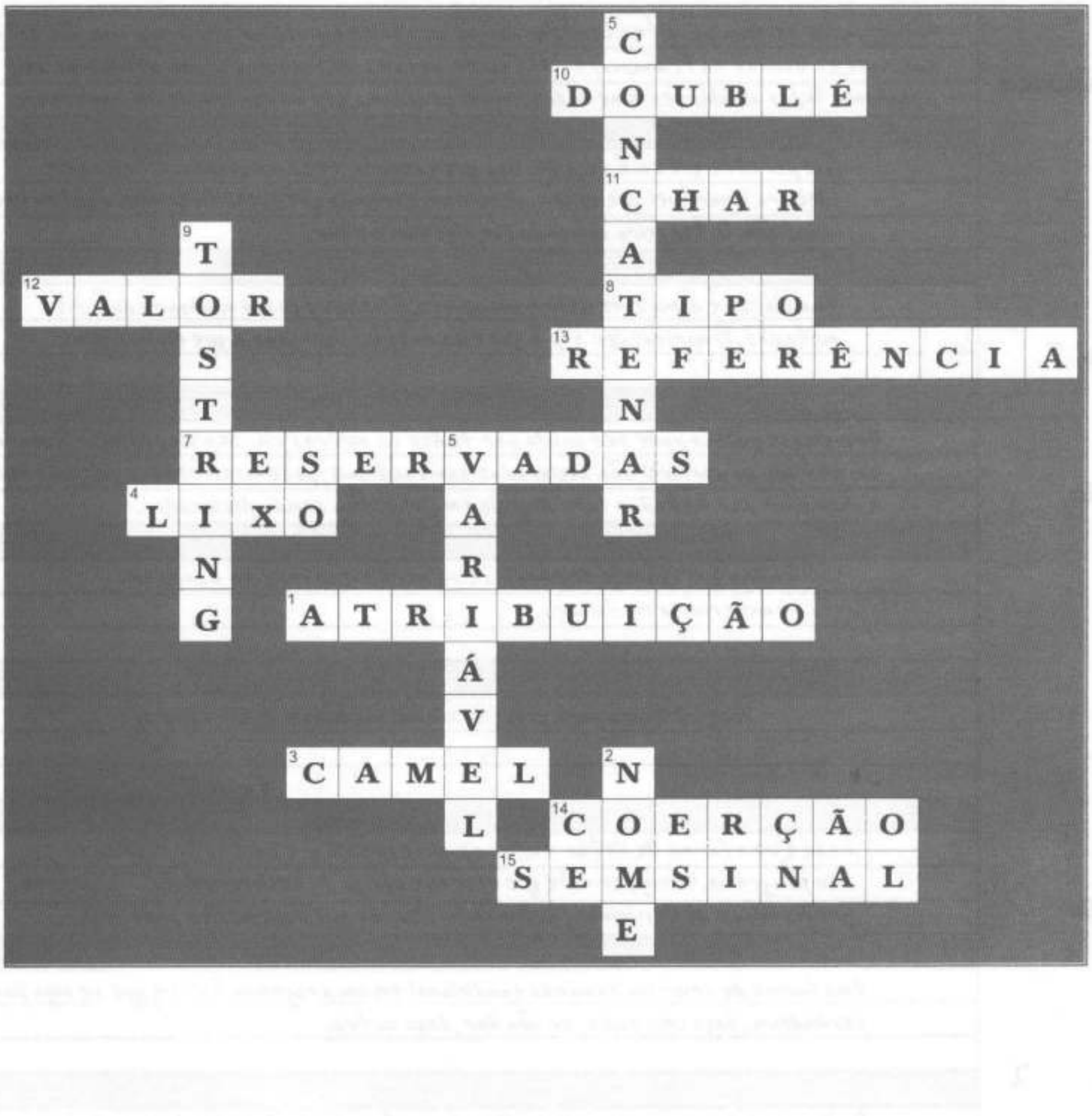
if

Uma forma de criar um comando condicional em um programa. Ele diz que se algo for verdadeiro, faça uma coisa, se não for, faça outra.

while

Os laços while continuam a executar enquanto a condição neles for verdadeira.

Solução da Cruzadinha



Nome: Data:

Laboratório C# 1

Um dia de corridas

Este laboratório dá-lhe as especificações de um programa que você deve desenvolver usando o conhecimento obtido nos últimos capítulos.

Este projeto é maior do que os já vistos até agora. Então, leia tudo antes de começar e reserve algum tempo para pensar. E não se preocupe se ficar preso em uma parte – não há nada novo aqui; você pode continuar lendo o livro e voltar ao laboratório mais tarde.

Preenchemos alguns poucos detalhes de projeto e certificamo-nos de que você tenha todas as peças necessárias... e mais nada.

É sua responsabilidade terminar o trabalho. Você pode baixar o executável deste laboratório na nossa página na Internet... mas não lhe daremos o código da resposta.

Especificações: Desenvolva um simulador de pista de corridas

João, Beto e Alfredo gostam de apostar em corridas, mas estão cansados de perder dinheiro. Eles querem que você construa um simulador, permitindo-lhes determinar os vencedores antes de colocar dinheiro na coisa. E, se você fizer um bom trabalho, eles dividirão com você os lucros. Eis como você vai desenvolver para eles...

Os caras

João, Beto e Alfredo querem apostar numa corrida de cachorros. João começa com R$ 50, Beto com R$ 75 e Alfredo com R$ 45. Antes de cada corrida, eles decidem se querem apostar e quanto cada um vai investir. Eles podem mudar as apostas até o início da corrida... mas, depois que ela começar, as apostas estarão encerradas.

O salão de apostas

O salão de apostas manterá registro do dinheiro de cada um e quais apostas fizeram. Existe um valor mínimo de apostas de R$ 5. O sistema aceita somente uma aposta por pessoa em cada corrida.
O sistema checa para se certificar de que cada apostador tenha dinheiro o suficiente para cobrir sua aposta – eles não podem apostar se não tiverem dinheiro suficiente.

Bem-vindos ao Salão
de Apostas do Crespo
Aposta mínima: R$ 5
Uma aposta por pessoa por corrida
Tem dinheiro para isso?

Bem-vindos ao Salão
de Apostas do Crespo
Aposta mínima: R$ 5
Uma aposta por pessoa por corrida
Tem dinheiro para isso?

Apostando

Todas as apostas são o-dobro-ou-nada - ou o vencedor dobra seu dinheiro, ou perde o que apostou. Existe um limite mínimo de apostas de R$ 5 e cada um pode apostar até R$ 15 num único cão. Se ele ganhar, o apostador termina com o dobro da quantidade que apostou (depois de a corrida terminar). Se ele perder, aquela quantidade desaparece de seu total.

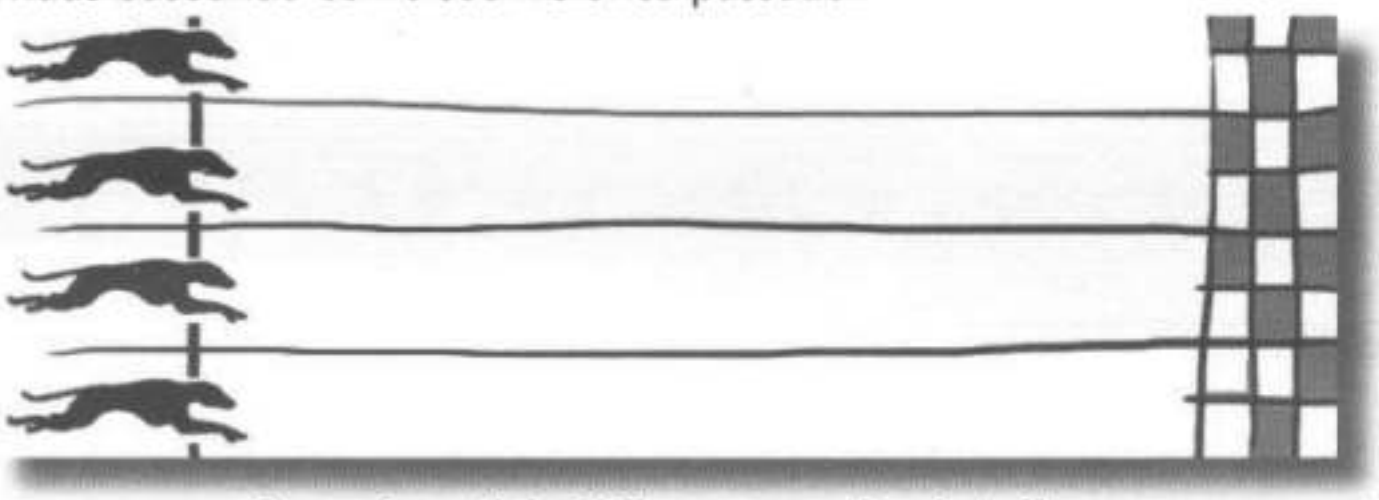
Todas as apostas: dobro-ou-nada
Aposta mínima: R$ 5
Até R$ 15 por cão
Vencedores: $$ adicionado
Perdedores: $$ removido

Digamos que um cara aposte R$ 10. No final da corrida, se o cachorro escolhido vence, ele tem mais R$ 10 (porque ele mantém os R$ 10 originais apostados e ganha mais R$ 10 por ter vencido). Se ele perder, seu total diminui R$ 10.

A corrida

Existem quatro cães que correm em linha reta. O vencedor da corrida é o primeiro a cruzar a linha de chegada. Uma corrida é totalmente aleatória, não existem características boas ou ruins para um cão e nenhum deles tem mais chance de vencer as próximas corridas baseando-se no seu histórico passado.

Se você quer desenvolver um sistema com vantagens e desvantagens para os cães, por favor faça! Será uma ótima oportunidade de praticar escrever código e ainda se divertir.



Soa divertido? Temos mais detalhes para mostrar... →

Você precisará de três classes e um formulário

Você terá que desenvolver três classes principais no seu projeto, bem como uma `GUI` para o simulador. Você deve ter uma matriz de três objetos Guy (cara) para manter registro dos três caras e seus resultados e uma matriz de quatro objetos `GreyHound` (nome de raça de cão de corrida, literalmente "cão cinza") que disputarão as corridas. Além disso, cada instância de `Guy` deve ter seu próprio objeto `Bet` (aposta, ou apostar) que mantém registro das apostas e paga ou toma dinheiro no final de cada corrida.

Será preciso adicionar "using System.Windows. Forms" no topo das classes GreyHound e Guy.

Adiantamos o seu trabalho com descrições de classes e alguns trechos de código. Você terá de terminar de desenvolvê-los.

Eis para você o esqueleto da classe que precisará desenvolver. Sua tarefa será completar os métodos.

Greyhound
StartingLocation RacetrackLength MyPictureBox Location MyRandom
Run() TakeStartingPosition()

```
public class Greyhound {
  public int StartingPosition; // Onde a minha caixa de imagem inicia
public int RacetrackLength; // O quanto a pista de corrida tem de comprimento
public PictureBox MyPictureBox = null; // Meu objeto caixa de imagem
public int Location = 0; // Minha posicao na pista
public Random MyRandom; // Uma instancia de Random
public bool Run() {
    // Mova-se para frente 1, 2, 3 ou 4 espacos aleatoriamente
    // Atualize a posicao da minha caixa de imagem no formulario
    // Retorna true se eu ganhei a corrida
}
public void TakeStartingPosition() {
    // Volte minha posicao para a linha de partida
}
}
```

Adicionamos comentários para dar a você uma idéia do que fazer.

Vê como o diagrama de classe corresponde ao código?

O inicializador do objeto GreyHound é bem simples. Apenas se certifique de passar a referência para a PictureBox certa no formulário para cada um dos objetos.

Não quebre muito a cabeça com isso... algumas vezes só é preciso atribuir um valor pra uma variável e pronto.

Seu objeto pode controlar coisas em seu formulário...

Você tem que ter certeza que o formulário passe o PictureBox correto para cada inicializador de objeto GreyHound.

A classe `GreyHound` mantém registro da posição na pista durante a corrida. Ela também atualiza a posição da `caixa de imagem` que representa o cão se movendo pela pista. Cada instância de `GreyHound` tem um campo chamado `MyPictureBox` com a referência ao controle de `caixa de imagem` no formulário que mostra a imagem de um cão. Suponha que a variável distance contenha a distância que um cão vai percorrer para frente. Então o seguinte código vai atualizar a posição de `MyPictureBox` ao adicionar `distance` ao valor de sua propriedade X:

```
Point p = MyPictureBox.Location;
p.X += distance;
MyPictureBox.Location = p;
```

Você recupera a posição atual da imagem...

... adiciona o valor de deslocamento para frente na sua coordenada X...

... e então atualiza a posição da caixa de imagem no formulário.

Guy
Name
MyBet
Cash
MyRadioButton
MyLabel
UpdateLabels()
PlaceBet()
ClearBet()
Collect()

```
public class Guy {
  public string Name; // O nome do cara
  public Bet MyBet; // Uma instancia de Bet() que mantem as apostas
  public int Cash; // Quanto dinheiro resta
  // Os ultimos dois campos sao os controles no formulario da GUI dos caras
  public RadioButton MyRadioButton; // Meu botao de radio
  public Label MyLabel; // Meu rotulo
  public void UpdateLabels() {
    // Atribua ao meu rotulo a descricao da minha aposta, e ao rotulo do
    // meu botao de radio o meu dinheiro ("Joao tem 43 reais")
  }

  public void ClearBet() { } // Reinicialize minha aposta para que ela zere
  public bool PlaceBet(int Amount, int Dog) {
    // Crie uma nova aposta e armazene-a no meu campo bet
    // Retorne verdadeiro se o cara tem dinheiro suficiente para apostar
  }
  public void Collect(int Winner) { } // Cobre minha aposta se eu ganhei
}
```

Adicione seu código aqui.

Quando inicializar o objeto Guy, certifique-se de atribuir o campo MyBet para null, e chame o método UpdateLabels() (atualiza rótulos) assim que terminar de inicializar.

A chave aqui é usar o objeto Bet... deixe que ele faça o trabalho.

Lembre-se de que as apostas são representadas por instâncias de Bet.

Este é o objeto que a classe Guy usa para representar apostas no aplicativo.

O inicializador de objeto para Bet atribui o valor, o cão e o apostador.

Bet
Amount
Dog
Bettor
GetDescription
PayOut

```
public class Bet {
  public int Amount; // A quantidade de dinheiro que foi apostada
  public int Dog; // O numero do cao em que apostamos
  public Guy Bettor; // O cara que fez a aposta
  public string GetDescription() {
    // Retorna uma sequencia de caracteres que diz quem fez a aposta, quanto
    // dinheiro foi apostado e em qual cao ("Joao apostou 8
    // no cao 4"). Se a quantidade for zero, a aposta nao foi feita
    // ("Joao nao apostou").

  }
  public int PayOut(int Winner) {
    // O parametro deve receber o vencedor da corrida. Se o cao venceu,
    // retorna a quantidade apostada. De outra forma, retorne um valor
    // negativo correspondente ao valor apostado
  }
}
```

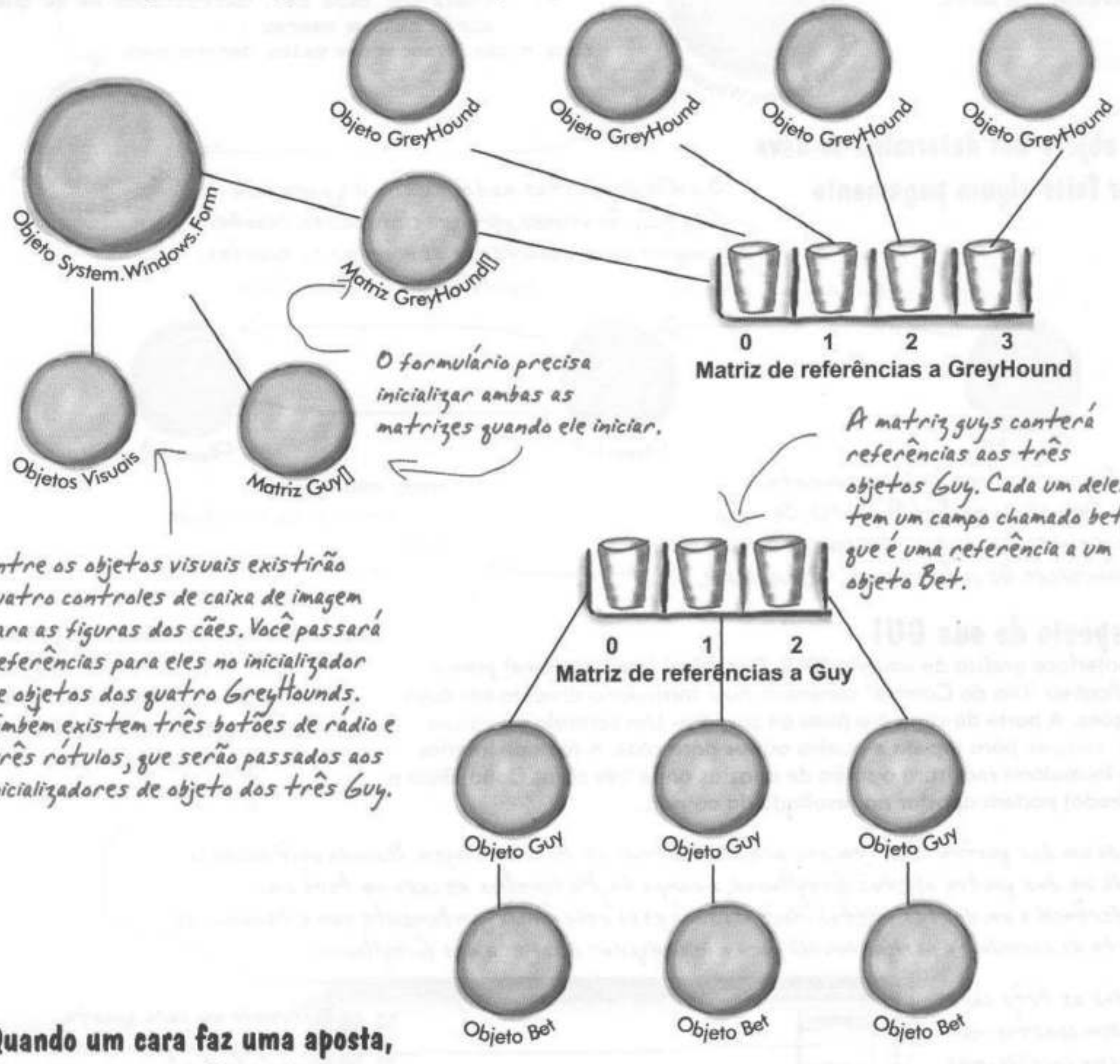
Dica: Você instanciará Bet no código de Guy, que usará a palavra-chave this para passar uma referência dele mesmo no inicializador de Bet.

Esta é uma tarefa de programação bem comum: montar uma seqüência de caracteres usando muitos dados de diferentes origens.

Eis aqui a estrutura de seu aplicativo

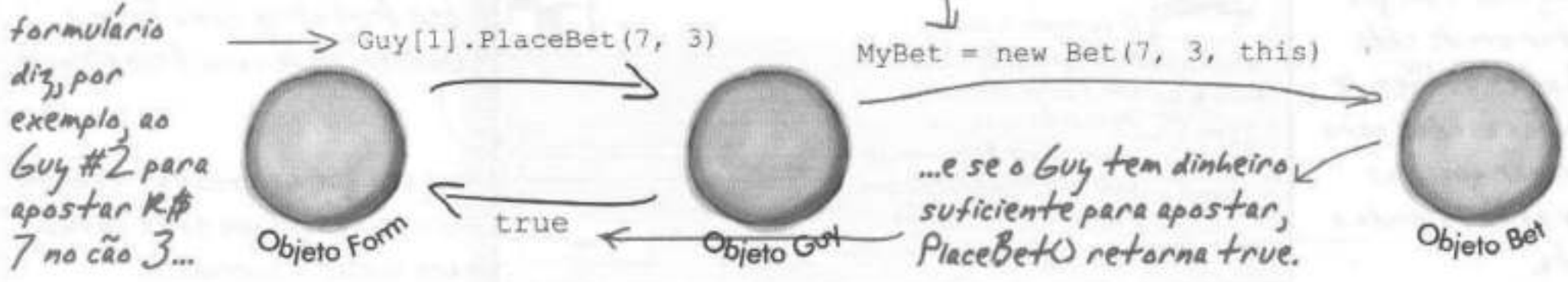
Passe algum tempo examinando cuidadosamente a estrutura. Ela parece bem complicada à primeira vista, mas não há nada aqui que você não saiba. Sua tarefa é implementar esta estrutura você mesmo, começando com as matrizes `GreyHound` e `Guy` no seu formulário principal.

A matriz dogs contém quatro referências, cada uma apontando para uma instância em separado da classe GreyHound.



Entre os objetos visuais existirão quatro controles de caixa de imagem para as figuras dos cães. Você passará referências para eles no inicializador de objetos dos quatro GreyHounds. Também existem três botões de rádio e três rótulos, que serão passados aos inicializadores de objeto dos três Guy.

Quando um cara faz uma aposta, ele cria um novo objeto Bet



O formulário diz aos cães para continuarem correndo até que um vença

Quando o usuário diz ao formulário para começar a corrida, ele inicia um laço para animar cada cão correndo pela pista.

O método RunO de cada cão checa se ele ganhou a corrida, para que o laço termine imediatamente assim que um deles o fizer.

```
while( não existe vencedor ) {
    for( itere por cada cão, certificando-se de que
         ainda nenhum venceu ) {
      faça o cão avançar um valor determinado
    }
}
```

O objeto Bet determina se deve ser feito algum pagamento

O salão de apostas no formulário diz para cada Guy qual cão venceu para que eles possam recolher qualquer valor decorrente de acertar as apostas.

```
Guy[1].Collect(winningDo)
```

```
MyBet.PayOut(winningDog)
```

O Guy vai ter seu dinheiro aumentado pelo resultado em BetPayoutO. Se o cão em que apostou ganhou, ele deve retornar Quantidade. De outra forma, -Quantidade.

```
if(meu cão ganhou) {
    return Quantidade;
} else {
    return -Quantidade;
}
```

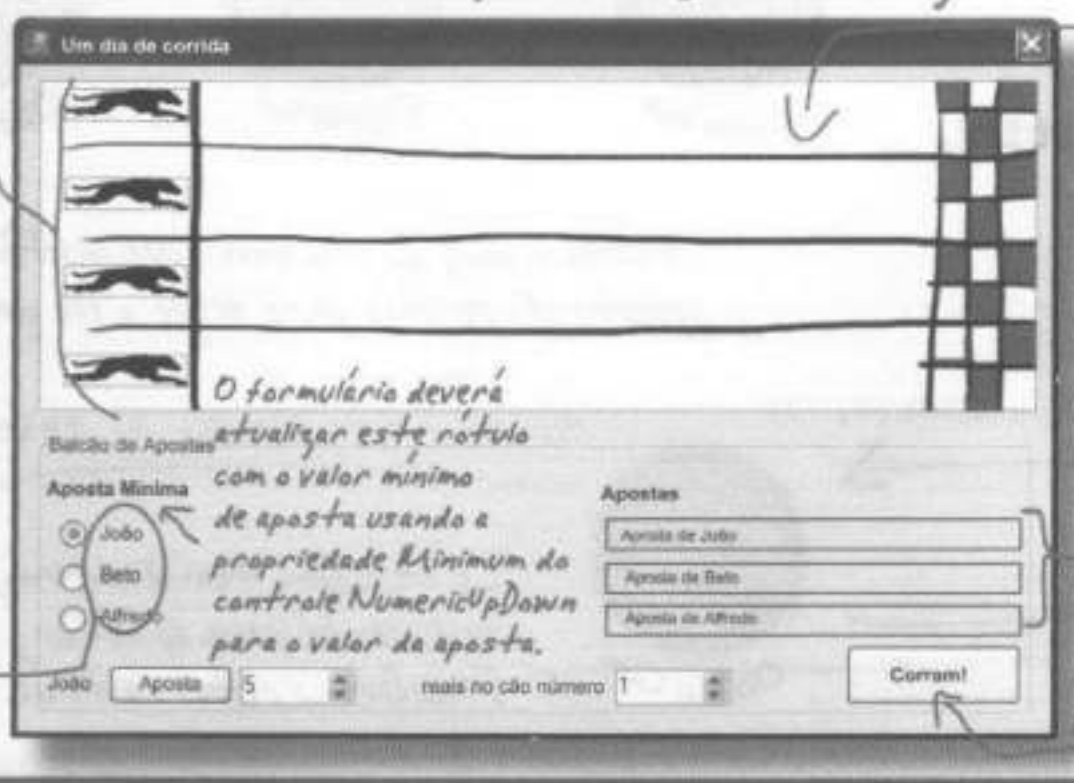
Aspecto de sua GUI

A interface gráfica de usuário (GUI, Graphical User Interface) para o aplicativo "Dia de Corrida" consistirá num formulário dividido em duas seções. A parte de cima é a pista de corridas. Um controle de `caixa de imagem` para a pista e quatro outros para cães. A metade inferior do formulário mostrará o salão de apostas onde três caras (João, Beto e Alfredo) podem apostar no resultado da corrida.

Usaremos a propriedade Length do controle de caixa de imagem da pista para determinar o comprimento da pista de corrida para os objetos Greyhound, que eles usam para descobrir quem ganhou a corrida.

Cada um dos quatro cães tem seu próprio controle de caixa de imagem. Quando você inicializa cada um dos quatro objetos Greyhound, o campo MyPictureBox de cada um terá uma referência a um destes objetos. Você passará essa referência (juntamente com o tamanho da pista de corrida e a posição inicial) para o inicializador de objeto dos Greyhound.

Todos os três caras podem apostar na mesma corrida, mas apenas uma única janela de apostas existe. Assim, eles têm que apostar um de cada vez. Estes botões de rádio são usados para selecionar qual dos caras esta fazendo a aposta.



Quando um cara aposta, ele apaga qualquer aposta anterior. A aposta atual aparece nestes controles de rótulo. Cada um deles deverá ter AutoSize como False e BorderStyle como FixedSingle.

Uma vez que as apostas tenham sido feitas, clique este botão para iniciar a corrida.

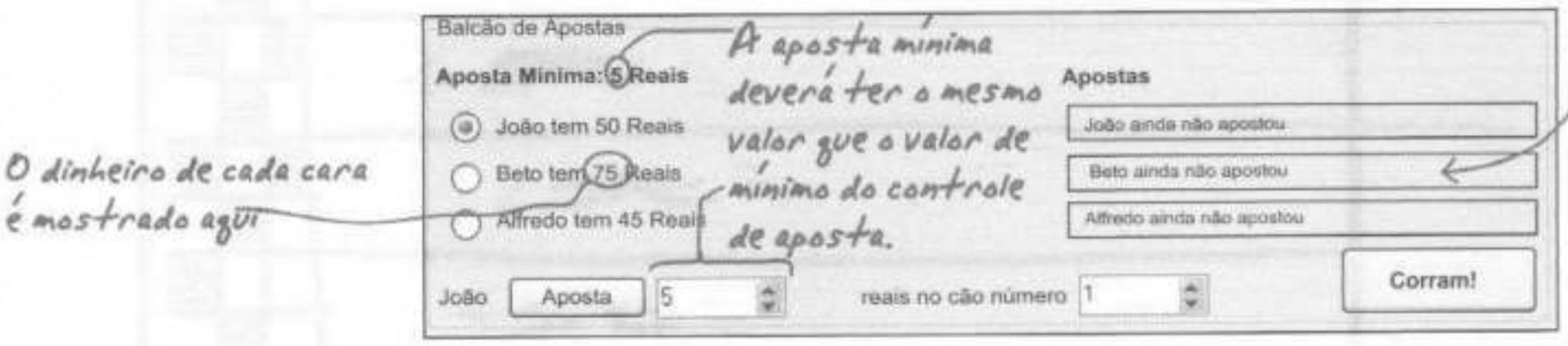
Você pode baixar os arquivos gráficos de www.altabooks.com.br

Apostando

Use os controles na caixa de agrupamento do Salão de Apostas para fazer com que cada um dos caras aposte. Existem três estágios distintos aqui:

Quando um cara aposta, seu objeto Guy atualiza este rótulo usando a referência MyLabel. Ele também atualiza o dinheiro que se tem usando a referência MyRadioButton.

1 **Nenhuma aposta ainda foi feita**
Quando o programa inicia, ou se uma corrida terminou, nenhuma aposta estará registrada no Salão. Você verá o total de dinheiro de cada cara ao lado de seu nome, à esquerda.

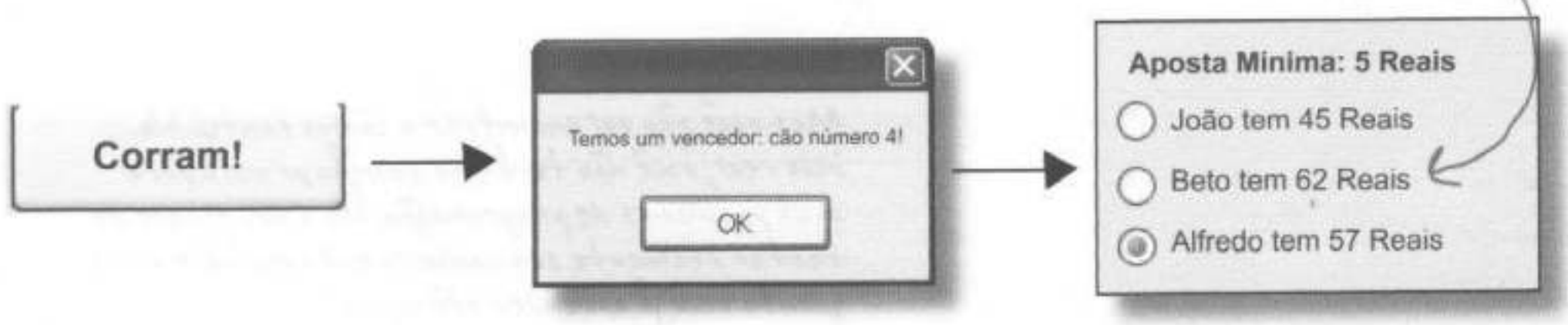


2 **Todos apostam**
Para apostar, selecione o botão do rádio de um cara qualquer, selecione uma quantidade e um cão e clique o botão "aposta". O método `PlaceBet()` vai atualizar o rótulo e o botão de rádio.

Uma vez que Beto tenha apostado, seu objeto Guy atualiza este rótulo e o texto do botão de rádio.

3 **Depois da corrida, cada cara recebe seus lucros (ou paga os prejuízos!)**
Uma vez que a corrida tenha se completado e exista um vencedor, cada objeto `Guy` chama seu método `Collet()` e adiciona seu lucro ou prejuízo ao seu total em dinheiro.

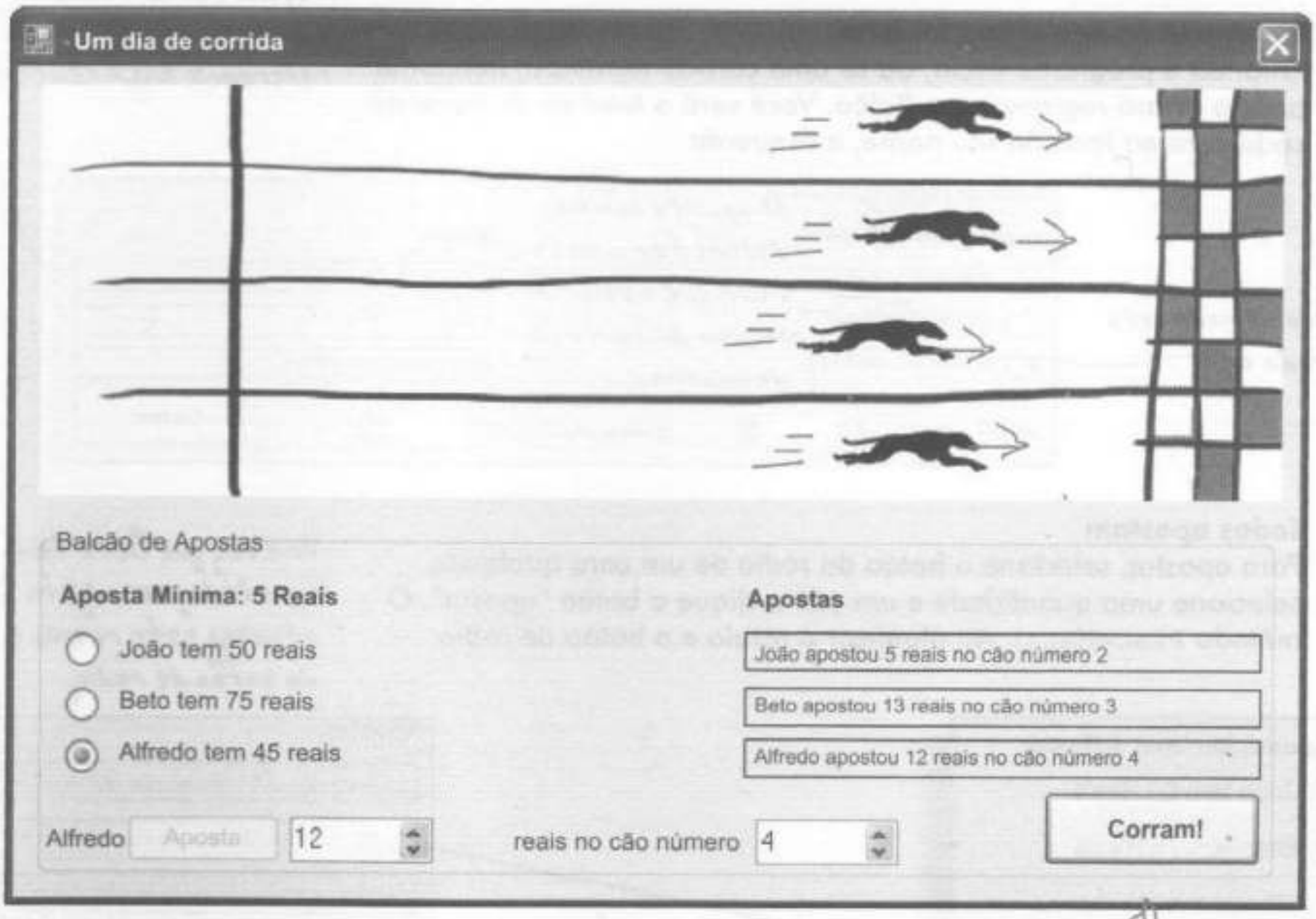
Já que Alfredo apostou 12 Reais no cão vencedor, seu total sobe 12. Os outros dois caras perdem o dinheiro que apostaram.



O Produto Final

Você saberá que seu aplicativo "Um Dia de Corrida" estará pronto quando seus caras puderem apostar e a corrida puder ser vista.

Durante a corrida, as imagens dos quatro cães correm pela pista de corrida até que um deles vença.



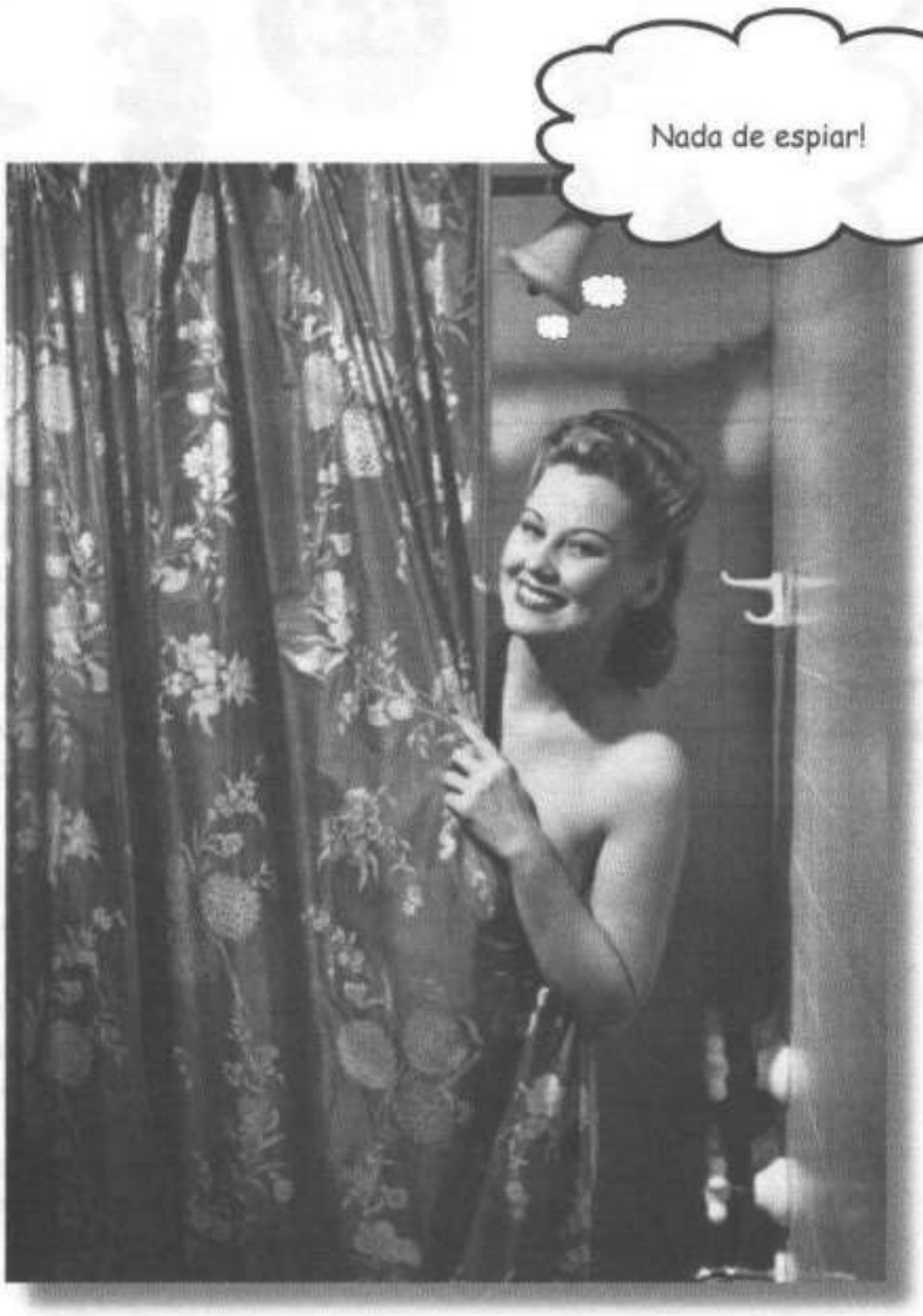
Durante a corrida, nenhuma aposta pode ser feita... e certifique-se de que uma nova corrida não possa ser iniciada enquanto os cães estiverem em movimento!

Você pode baixar um executável pronto, bem como os arquivos gráficos para os quatro cães e a pista de corrida na página de Internet do Use a Cabeça:
www.altabooks.com.br

Mas você não vai encontrar o código fonte! Na vida real, você não terá uma solução pronta para seus problemas de programação. Eis a sua chance de testar realmente seu conhecimento em C# e ver o quanto você já aprendeu até agora!

5 encapsulamento

Mantenha o que é particular... em particular



Você já quis ter um pouco mais de privacidade?

Às vezes, seus objetos sentem-se da mesma forma. Você não quer que ninguém estranho leia seu diário ou veja seus extratos bancários. Da mesma forma, um bom objeto não deixa outro xeretar seus campos. Neste capítulo, você aprenderá sobre o poder do encapsulamento. Você tornará seus dados de objeto privados e adicionará métodos para proteger o acesso a estes dados.

Catarina é uma promotora de eventos

Ela planeja jantares para seus clientes e está indo muito bem. Mas, ultimamente, anda tendo problemas para responder aos seus clientes rápido o bastante com uma estimativa de preço para seus serviços.

Catarina preferiria gastar seu tempo planejando eventos, não fazendo estimativas.

Quando um novo cliente pede à Catarina para fazer uma festa, ela precisa calcular o número de convidados, que bebidas servir e que peças de decoração deve comprar. Então, ela usa um cálculo bem complicado para descobrir o custo total, baseado em um fluxograma que ela usa há anos. A notícia ruim é que leva muito tempo para trabalhar com esse fluxograma e, enquanto ela está fazendo a estimativa, seus clientes em potencial estão conversando com outros promotores de eventos.

Cabe a você criar um estimador de preço de eventos feito em C# e salvar o negócio dela. Imagine a festa que ela fará pra você quando você conseguir!

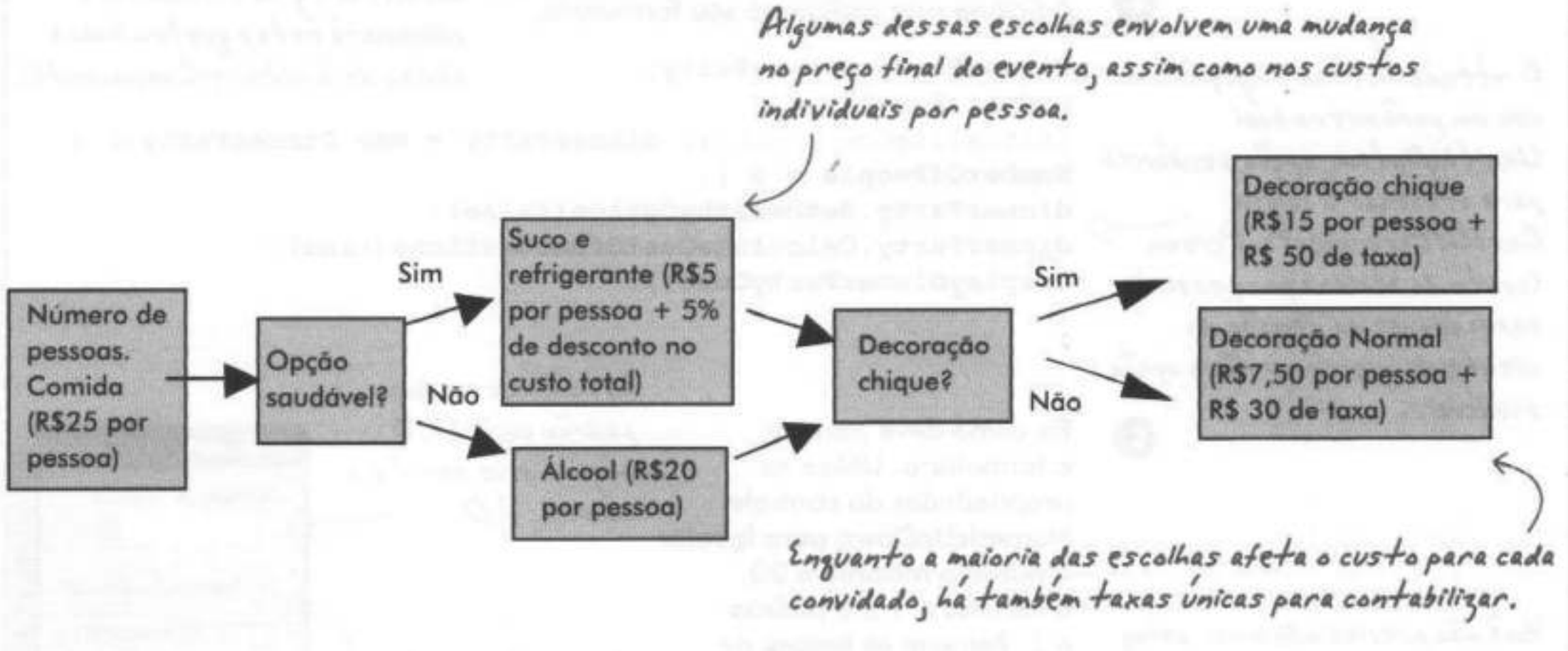
O que um estimador faz?

Catarina examina algumas partes básicas de seu sistema para descobrir os custos de um evento. Aqui vemos uma parte do que ela concluiu:

Programa de Planejamento de Festas da Catarina – Estimativa de Custo para um Jantar

- Para cada pessoa na lista de convidados há um custo de R$25 para alimentação.
- Os clientes têm uma opção em relação às bebidas. A maioria das festas serve bebidas alcoólicas, o que custa R$ 20 por pessoa. Mas eles também podem escolher fazer uma festa sem álcool. Catarina chama isso de "opção saudável". Ela custa apenas R$ 5 por pessoa, incluindo refrigerante e suco, em vez de álcool. A opção saudável é mais fácil para ela e, por isso, ela dá ao cliente um desconto de 5% no valor da festa.
- Há duas opções de custo de decoração. Se um cliente escolhe a decoração normal, sai R$7,50 por pessoa com uma taxa de R$30 do trabalho. Um cliente pode também atualizar a decoração da festa escolhendo a "opção chique" – que custa R$15 por pessoa com uma taxa única de R$50 para o trabalho.

Aqui temos uma outra visão no mesmo conjunto de custos, dividido em um pequeno gráfico de fluxo para ajudá-lo a ver como isso funciona:



Exercício

Faça um programa para resolver o problema de estimativa da festa da Catarina.

DinnerParty
NumberOfPeople CostOfBeveragesPerPerson CostOfDecorations HealthyOption
SetHealthyOption() CalculateCostOfDecorations() CalculateCost()

1. Crie um novo projeto de aplicativo Windows, adicione um arquivo de classe nele chamado `DinnerParty.cs` (festa jantar) e crie a classe DinnerParty usando o diagrama de classe à esquerda. Ele tem três métodos: `CalculateCostOfDecorations()` (calcular custo da decoração), `SetHealthyOption()` (opção saudável escolhida) e `CalculateCost()` (calcular custo). Para os campos, use decimais para os dois, int para o número de pessoas e bool para controlar se a opção saudável foi selecionada ou não. Tenha certeza de **adicionar um M depois de cada literal** que você atribuir a uma variável decimal (10,0M).

2. Eis uma ferramenta útil do C#. Já que o custo de alimentação não será alterado pelo programa, você pode declará-lo como uma constante, que é como uma variável exceto por seu valor, que nunca pode ser mudado. Esta é a declaração a ser usada:

```
public const int CostOfFoodPerPerson = 25;
```

Aqui está o diagrama para a classe DinnerParty que você precisará criar.

3. Volte à página para ter certeza de que entendeu direito a lógica para os métodos. Só um deles retorna um valor (`um decimal`) – os outros dois são void. O método `CalculateCostOfDecorations()` calcula o custo da decoração para o número de pessoas que estarão na festa. Use o método CalculateCost() para calcular o custo total acrescentando o custo da decoração ao custo das bebidas e comida por pessoa. Se o cliente quer a opção saudável, você pode colocar o desconto no método CalculateCost() depois de calcular o custo total.

4. Adicione este código ao seu formulário:

Você vai declarar o campo dinnerParty no formulário e adicionará estas quatro linhas abaixo de InitializeComponent().

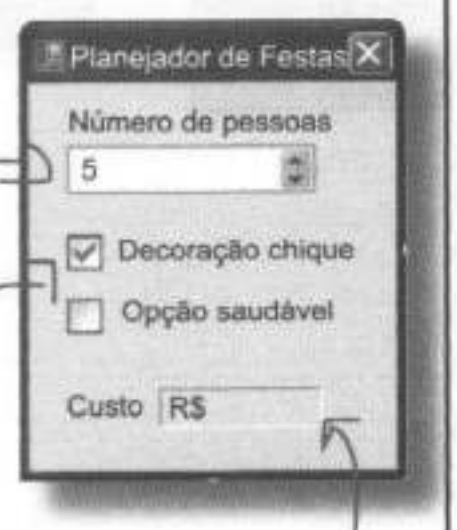
```
DinnerParty dinnerParty;
public Form1() {
InitializeComponent(); dinnerParty = new DinnerParty() {
NumberOfPeople = 5 };
dinnerParty.SetHealthyOption(false);
dinnerParty.CalculateCostOfDecorations(true);
DisplayDinnerPartyCost();
}
```

O método SetHealthyOption() usa um parâmetro bool (healthyOption, opção saudável) para atualizar o campo CostOfBeveragesPerPerson (custo de bebidas por pessoa) baseando-se na decisão do cliente de querer ou não a opção saudável.

5. Eis como deve parecer o formulário. Utilize as propriedades do controle NumericUpDown para igualar o número máximo a 20, o mínimo a 1 e o padrão a 5. Remova os botões de maximizar e minimizar também.

Você não precisa adicionar `using System.Windows.Forms;` à sua classe DinnerParty, porque ele não utiliza MessageBox.Show(), Point, ou nada mais deste namespace do Framework NET.

Estabeleça o valor padrão como 5.0 mínimo deve ser 1 e o máximo, 20.



A caixa Decoração chique deve ter Checked como true.

Este é apenas um rótulo com a propriedade Text estabelecida como , a propriedade BorderStyle como Fixed3D e a AutoSize como false.

6 Em vez de usar um botão para calcular os custos, esse formulário atualizará o rótulo custo automaticamente assim que você usar uma caixa de seleção ou o controle NumericUpDown. A primeira coisa que você precisa fazer é criar um método no formulário que exiba o custo.

Adicione este método a Form1(). Ele será chamado quando o controle NumericUpDown é clicado:

Este método será chamado por todos os outros que você criar no formulário. Ele é a forma como você atualiza o rótulo custo com o valor correto sempre que algo mudar.

```
private void DisplayDinnerPartyCost()
{
  decimal Cost = dinnerParty.CalculateCost(checkBox2.Checked);
  costLabel.Text = Cost.ToString("c");
}
```

Adicione este método ao formulário – ele recalculará o custo da festa e o colocará no rótulo Custo.

Mude o nome do rótulo que exibe o custo para costLabel.

Passar "c" ao ToString() manda-o formatar o custo como um valor monetário (c de currency). Se você está em um país que usa dólares, ele acrescenta um sinal adequado.

Isto é true se a caixa de checagem para Opção Saudável é marcada.

7 Agora associe o campo NumericUpDown à variável NumberOfPeople (número de pessoas) que você criou na classe DinnerParty e exiba o custo no formulário. Clique duas vezes no controle NumericUpDown – o IDE adicionará um tratador de evento ao seu código. Este é um método que executa toda a vez que o controle é alterado. Ele reiniciará o número de pessoas na festa. Preencha-o assim:

Quando você clica duas vezes num botão no IDE para adicionar código que rode quando o botão é clicado, isto é um tratador de evento também.

```
private void numericUpDown1_ValueChanged(
                                    object sender, EventArgs e)
{
  dinnerParty.NumberOfPeople = (int) numericUpDown1.Value;
  DisplayDinnerPartyCost();
}
```

Você precisa fazer coerção de numericUpDown.Value num int porque ele é uma propriedade Decimal.

Ops – há um problema com esse código. Você consegue identificá-lo? Não se preocupe se você ainda não consegue vê-lo. Nós nos aprofundaremos nele em apenas alguns minutos!

O valor que você envia do formulário ao método será fancyBox.Checked. Ele será passado como um parâmetro boleano ao método na classe.

Estes são métodos de apenas duas linhas. A primeira chamará o método que você criou na classe para calcular os custos e a segunda exibirá o custo total no formulário.

8 Clique duas vezes na caixa Decoração chique no formulário e certifique-se de que ele primeiro chama CalculateCostOfDecorations() e depois DisplayDinnerPartyCost() (exibir custo da festa jantar). A seguir, clique duas vezes na caixa Opção Saudável e certifique-se de que ela chama o método SetHealthyOption() na classe DinnerParty e então chame o método DisplayDinnerPartyCost().

Solução do Exercício

Aqui está o código de DinnerParty.cs.

```
public class DinnerParty {
  const int CostOfFoodPerPerson = 25;
  public int NumberOfPeople;
  public decimal CostOfBeveragesPerPerson;
  public decimal CostOfDecorations = 0;

  public void SetHealthyOption(bool healthyOption) {
    if (healthyOption) {
      CostOfBeveragesPerPerson = 5.00M;
    } else {
      CostOfBeveragesPerPerson = 20.00M;
    }
  }
  public void CalculateCostOfDecorations(bool fancy) {
    if (fancy)
    {
      CostOfDecorations = (NumberOfPeople * 15.00M) + 50M;
    } else {
      CostOfDecorations = (NumberOfPeople * 7.50M) + 30M;
    }
  }
  public decimal CalculateCost(bool healthyOption) {
    decimal totalCost = CostOfDecorations +
        ((CostOfBeveragesPerPerson + CostOfFoodPerPerson)
            * NumberOfPeople);
    if (healthyOption) {
      return totalCost * .95M;
    } else {
      return totalCost;
    }
  }
}
```

Utilizar uma constante para CostOfFoodPerPerson garante que o valor não possa ser alterado. Ela também deixa o código mais fácil de ler - fica claro que esse valor nunca muda.

Quando o formulário cria pela primeira vez o objeto, ele usa o inicializador para atribuir o valor para NumberOfPeople. Então ele chama SetHealthyOption() e CalculateCostOfDecorations() para criar outros campos.

Usamos if(Fancy) em vez de digitar if (Fancy == true) porque o comando if sempre checa se a condição é verdadeira.

Usamos parênteses para ter certeza de que a matemática funciona apropriadamente.

Isso aplica o desconto de 5% ao custo total do evento se a opção não alcoólica foi escolhida.

Fizemos você usar um decimal para os preços porque ele é projetado para valores monetários. Apenas certifique-se de sempre colocar um "M" após todo literal – então, se você quiser armazenar R$35,26, tenha certeza de escrever 35,26M.

Chamamos DisplayDinnerPartyCost para inicializar o rótulo que mostra o custo assim que o formulário é carregado.

```
public partial class Form1 : Form {
  DinnerParty dinnerParty;
  public Form1() {
    InitializeComponent();
    dinnerParty = new DinnerParty() { NumberOfPeople = 5 };
    dinnerParty.CalculateCostOfDecorations(fancyBox.Checked);
    dinnerParty.SetHealthyOption(healthyBox.Checked);
    DisplayDinnerPartyCost();
  }
  private void fancyBox_CheckedChanged(object sender, EventArgs e) {
    dinnerParty.CalculateCostOfDecorations(fancyBox.Checked);
    DisplayDinnerPartyCost();
  }
  private void healthyBox_CheckedChanged(object sender, EventArgs e) {
    dinnerParty.SetHealthyOption(healthyBox.Checked);
    DisplayDinnerPartyCost();
  }
  private void numericUpDown1_ValueChanged(object sender, EventArgs e) {
    dinnerParty.NumberOfPeople = (int)numericUpDown1.Value;
    DisplayDinnerPartyCost();
  }
  private void DisplayDinnerPartyCost() {
    decimal Cost = dinnerParty.CalculateCost(healthyBox.Checked);
    costLabel.Text = Cost.ToString("c");
  }
}
```

Mudanças nas caixas de checagem no formulário igualam os boleanos healthyOption e fancy a true ou false nos métodos SetHealthyOption() e CalculateCostOfDecorations().

O novo custo do jantar precisa ser recalculado e exibido sempre que os números mudam ou que as caixas de checagem sejam marcadas.

Chamamos nossas caixas de checagem de healthyBox e fancyBox para que você pudesse ver o que está acontecendo em seus métodos tratadores de eventos.

Formatando seqüência de caracteres

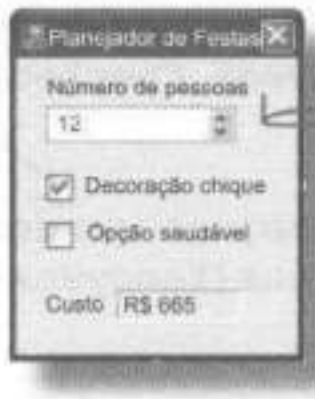
Você já viu como pode converter qualquer variável em uma seqüência de caracteres usando seu método ToString(). Se você passar c para ToString(), ele converte-o na moeda local. Você também pode passar f3 para formatá-lo como com um número decimal com três casas decimais. D (isso é um zero) para converter em um número inteiro, P0 para uma porcentagem de número inteiro, e n para exibi-lo como um número com um separador de ponto para milhares. Pare um minuto e veja como cada um desses fica em seu programa!

O teste da Catarina

Isso é demais! Fazer estimativas será muito mais fácil.

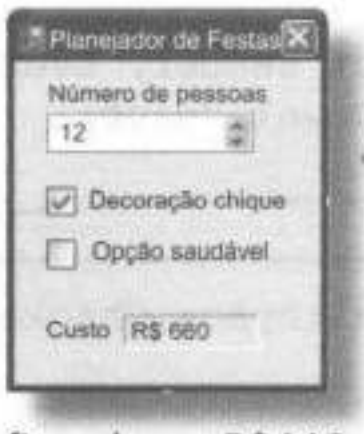
O Roberto é um dos clientes favoritos da Catarina. Ela fez o casamento dele ano passado e agora está planejando um jantar importante.

Roberto (no telefone): Oi Catarina. Como está a preparação do meu jantar?
Catarina: Tudo certo. Estávamos fora hoje de manhã procurando itens pra decoração e você vai adorar o que escolhemos.
Roberto: Que ótimo. Escuta, acabei de receber uma ligação da tia da minha mulher. Ela e o marido vão ficar conosco por algumas semanas. Você pode me dizer como fica a estimativa para aumentar de 10 para 12 pessoas na lista de convidados?
Catarina: Claro! Eu te passo isto daqui a pouco.



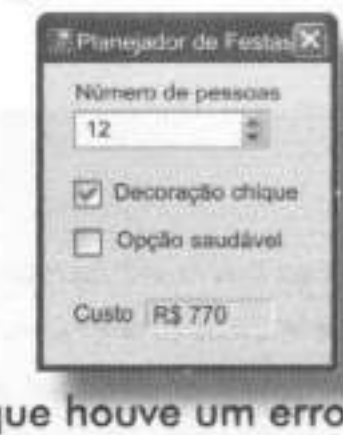
Alterando o valor no número de pessoas de 10 para 12 e apertando enter, a caixa mostra R$665 como o custo total. Hmm, isso parece um pouco baixo...

Catarina: OK. Parece que o custo total para o jantar subirá de R$575 para R$665.
Roberto: Só R$90 de diferença? Parece um ótimo negócio! E se decidirmos não fazer a decoração chique? Qual é o custo então?



Tirando a seleção da caixa Decoração Chique só reduz o total em R$5. Isto deve estar errado!

Catarina: Hum, parece que fica...hum, R$660.
Roberto: R$660? Pensei que a decoração era R$15 por pessoa. Você alterou o preço? Se fica só R$5 de diferença, devemos fazer a decoração chique. Vou te falar um negócio, seu sistema de preços é confuso.
Catarina: Acabamos de adotar um programa que faz a estimativa para nós. Mas parece que ele tem algum problema. Só um segundo enquanto eu acrescento a decoração chique de volta na conta final.



Quando você seleciona Decoração Chique de volta, o número sobe para R$770. Estes números estão todos errados.

Catarina: Roberto, eu acho que houve um erro. Parece que o custo com a decoração chique subiu para R$770. Isto parece fazer mais sentido. Mas estou começando a desconfiar deste aplicativo. Vou mandá-lo de volta para consertar alguns defeitos e fazer a sua estimativa à mão. Posso retornar para você amanhã?
Roberto: Eu não vou pagar R$770 só para acrescentar duas pessoas no jantar. O preço que você estabeleceu antes foi muito mais razoável. Vou pagar os R$665 que você falou antes; não posso pagar mais que isso!

PODER DO CÉREBRO

Por que você acha que os números estão saindo errado toda vez que a Catarina altera alguma coisa?

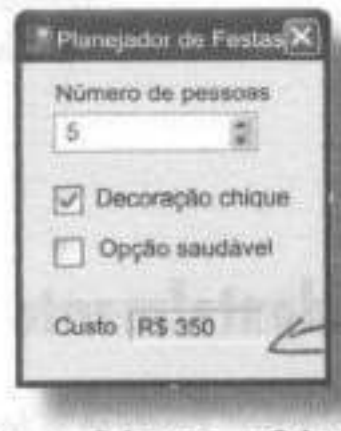
Cada opção deve ser calculada individualmente

Embora tenhamos nos certificado de calcular todos os números de acordo com o que a Catarina disse, não pensamos no que aconteceria quando as pessoas alterassem apenas uma das opções no formulário.
Quando você inicia o programa, o formulário estabelece o número de pessoas como cinco e Decoração Chique como true. Ele deixa a Opção Saudável não selecionada e calcula o custo do jantar como R$350. Aqui vemos como ele chega ao custo total inicial:

Relaxe

Não se preocupe! Dessa vez não foi sua culpa.
Criamos um defeitozinho cruel no código que lhe demos para mostrar o quanto é fácil ter problemas quando os objetos usam os campos uns dos outros... e o quanto é difícil identificar estes problemas.

5 pessoas.
R$20 por pessoa para bebidas → Custo total de bebidas = R$100
R$25 por pessoa para comida → Custo total de comida = R$125
R$15 por pessoa para decoração → Custo total da Decoração = R$125
mais R$50 de taxa.

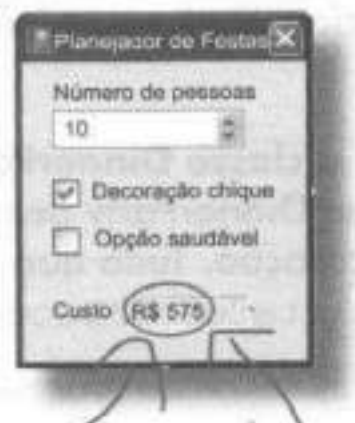


Até agora, tudo certo.

Quando você altera o número de convidados, o aplicativo deve recalcular a estimativa total da mesma forma. Mas isso não acontece:

R$100 + R$125 +R$125 = R$350

10 pessoas
R$20 por pessoa para bebidas → Custo total de bebidas = R$200
R$25 por pessoa para comida → Custo total de comida = R$250
R$15 por pessoa para decoração → Custo total da Decoração = R$200
mais R$50 de taxa.



R$200 + R$250+ R$200 = R$650

Este é o total que deveríamos ter. Mas não está certo...

Decoração antiga.

O programa está acrescentando o custo antigo de decoração com o novo custo de comida e bebida. Ele está fazendo R$200 + R$250 + R$125 = R$575

Novo custo de comida e bebida.

O problema de perto

Dê uma olhada no método que lida com alterações do valor no controle `numericUpDown`. Ele atribui o valor do campo à variável `NumberofPeople` e chama o método `DisplayDinnerPartyCost()`. Ele conta com este método para fazer o novo cálculo de todos os novos custos individuais.

```
private void numericUpDown1_ValueChanged(
                          object sender, EventArgs e) {
  dinnerParty.NumberOfPeople = (int)numericUpDown1.Value;
  DisplayDinnerPartyCost();
}
```

Essa linha atribui o valor de NumberofPeople nessa instância de DinnerParty ao valor no formulário.

Então, quando você faz uma alteração no valor do campo NumberofPeople, esse método nunca é chamado:

Este método chama CalculateCost(), mas não o método CalculateCostofDecorations().

```
public void CalculateCostOfDecorations(bool Fancy) {
  if (Fancy) {
    CostOfDecorations = (NumberOfPeople * 15.00M) + 50M;
  } else {
    CostOfDecorations = (NumberOfPeople * 7.50M) + 30M;
  }
}
```

Essa variável recebe o valor R$125, de quando o formulário chamou o método pela primeira vez. E já que esse método não é chamado novamente, o valor não muda.

É por isso que o número corrige a si próprio quando você seleciona decoração chique novamente. Clicar na caixa de checagem faz o programa executar CalculateCostOfDecorations() de novo.

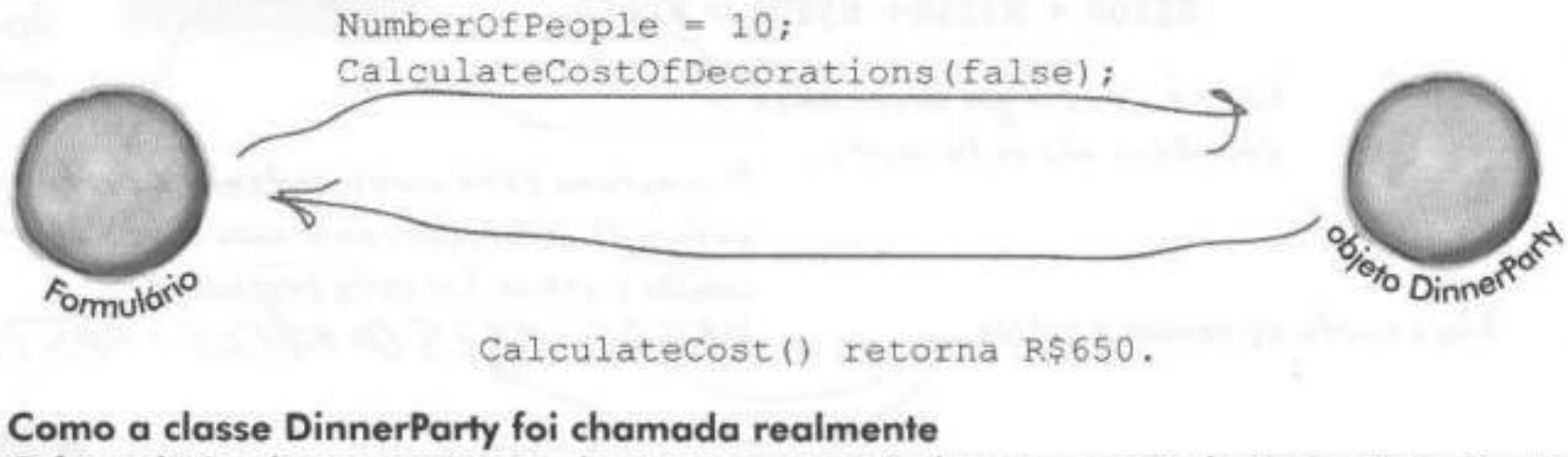
As pessoas não usarão seus programas toda vez exatamente da forma que você espera.

Felizmente, o C# possui uma ferramenta poderosa para ter certeza de que seu programa sempre funcione corretamente – mesmo quando as pessoas fazem coisas que você nunca imaginou. Isso se chama **encapsulamento** e é uma técnica muito útil para trabalhar com objetos.

É fácil usar acidentalmente seus objetos de forma errada

A Catarina teve problemas porque o formulário dela ignorou o método conveniente `CalculateCostOfDecorations()` que você criou e, em vez disso, foi diretamente aos campos na classe DinnerParty. Então, embora sua classe DinnerParty funcionasse bem, o formulário chamou-a de uma forma inesperada... e isso causou problemas.

1. **Como a classe DinnerParty esperava ser chamada**
   A classe DinnerParty deu ao formulário um método muito bom para calcular o custo total da decoração. Tudo que ele tinha de fazer era informar o número de pessoas, chamar `CalculateCostOfDecorations()` e `CalculateCost()` retornaria o custo correto.



2. **Como a classe DinnerParty foi chamada realmente**
   O formulário alterou o número de pessoas, mas só chamou o método `CalculateCost()` sem primeiro recalcular o custo da decoração. Isto prejudicou todo o cálculo e Catarina acabou dando o preço errado ao Roberto.

Encapsulamento significa manter em particular alguns dados de uma classe

Existe uma forma fácil de evitar esse tipo de problema: certifique-se de que só há uma forma de usar sua classe. Felizmente, o C# facilita isto permitindo que você declare alguns campos como **privados**. Até agora, você só viu campos públicos. Se você tem um objeto com um campo público, qualquer outro objeto pode ler ou alterar aquele campo. Mas se você torna-o privado, então este campo pode ser **acessado apenas de dentro do objeto** (ou por outro objeto da mesma classe).

Utilize sua preguiça em seu próprio benefício – se você deixar de incluir tanto o private como o public, então o C# presumirá que seu campo é privado.

Se você quer que um campo seja privado, tudo que você precisa fazer é usar a palavra-chave private quando você o declara. Isso diz ao C# que se você tem, por exemplo, uma instância de DinnerParty, seu campo numberOfPeople só pode ser lido e escrito por aquela instância. Outros objetos nem saberão que ele existe.

```
public class DinnerParty {
  private int numberOfPeople;
  ...
  public void SetPartyOptions(int people, bool fancy) {
    numberOfPeople = people;
    CalculateCostOfDecorations(fancy);
  }
  public void GetNumberOfPeople() {
    return numberOfPeople;
  }
```

Outros objetos ainda precisam de uma forma de determinar o número de pessoas para o jantar. Uma boa forma de dar-lhes acesso a isso é adicionar métodos para estabelecer ou recuperar o número de pessoas. Assim, você pode ter certeza de que o método CalculateCostOfDecorations() seja executado toda vez que o número de pessoas for alterado. Isso resolverá aquele defeito chato.

Tornando o campo que armazena o número de convidados privado, damos ao formulário apenas uma forma de dizer à classe DinnerParty quantas pessoas estão no jantar – e podemos ter certeza de que o custo da decoração seja recalculado corretamente. Quando você torna algum dado privado e escreve código para usar esses dados, isso é chamado de encapsulamento.

en-cap-su-la-do, adj.
envolvido por uma cobertura ou membrana protetora. *Os mergulhadores estavam totalmente encapsulados por seu submarino e só podiam entrar e sair pela porta pressurizada.*

Use encapsulamento para controlar acesso aos métodos e campos de sua classe

Quando você torna todos os seus campos e métodos públicos, qualquer outra classe pode acessá-los. Tudo que sua classe faz e sabe torna-se um livro aberto para todas as outras classes em seu programa... e você acabou de ver como isso pode fazer com que seu aplicativo comporte-se de formas que você nunca imaginou. O encapsulamento permite que você controle o que você compartilha e o que mantém privado dentro de sua classe. Vamos ver como isso funciona:

1. O super espião João Matoso defende a vida, liberdade e a busca pela da felicidade como um agente disfarçado na URSS. Seu objeto agenteCIA é uma instância da classe AgenteSecreto.

```
Nome verdadeiro: "João Matoso"
Codinome: "Marco Presto"
Senha: "o corvo voa à meia-noite"
```

AgenteSecreto
Codinome NomeVerdadeiro Senha
SaudacaoAgente()

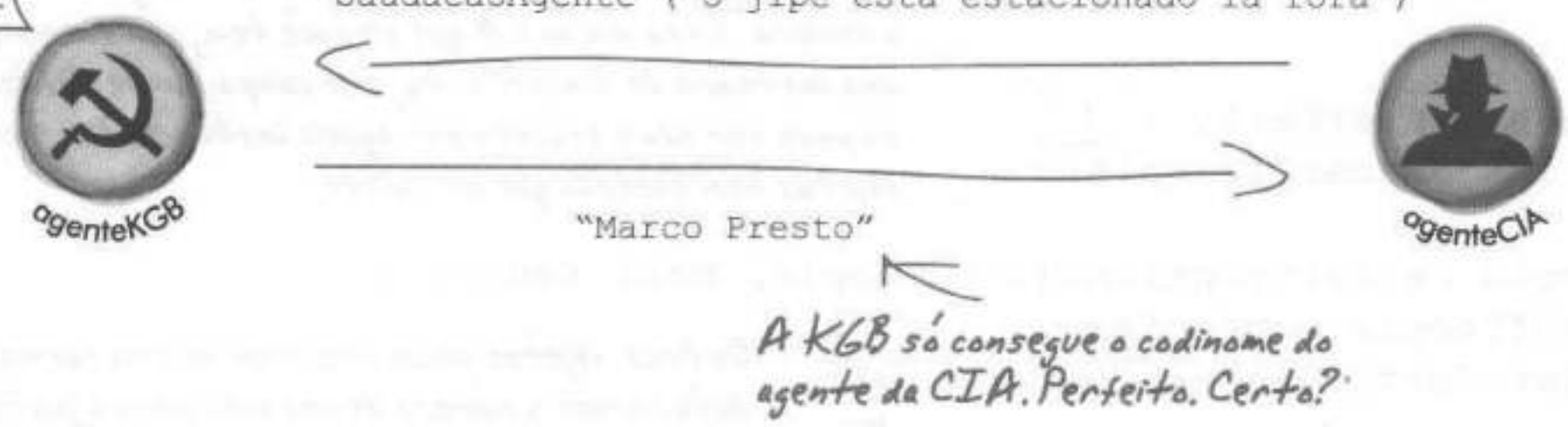
2. O agente João possui um plano para ajudá-lo a despistar os agentes inimigos da KGB. Ele adicionou um método SaudacaoAgente() que tem uma senha como parâmetro. Se ele não receber a senha correta, apenas revelará seu codinome, Marco Presto.

3. Parece uma forma à prova de falhas de proteger a identidade do agente, certo? Contanto que o objeto do agente que o chame não tenha a senha, o nome dele está seguro.

AgenteInimigo
SopaDeBeterraba Vodka
ContatarCamaradas() DerrubarCapitalistas()

O objeto agenteCIA é uma instância da classe AgenteSecreto, enquanto agenteKGB é uma instância da AgenteInimigo.

O agente da KGB usou a senha errada em seu cumprimento.



A KGB só consegue o codinome do agente da CIA. Perfeito. Certo?

Mas o campo NomeVerdadeiro fica REALMENTE protegido?

Então contanto que a KGB não saiba nenhuma das senhas dos agentes da CIA, os nomes de verdade da CIA estão seguros. Certo? Mas e a declaração de campo para o campo realName?

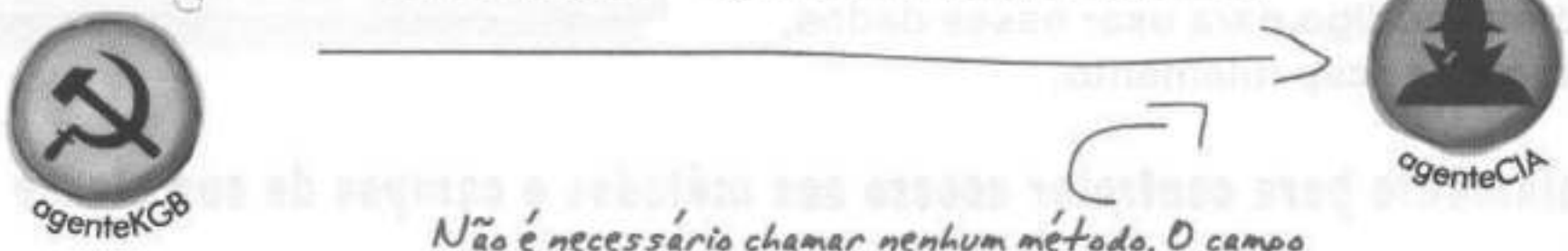
Deixar suas variáveis públicas significa que elas podem ser acessadas, e até mesmo alteradas, de fora da classe.

```
public string NomeVerdadeiro;
```

Deixar suas variáveis públicas significa que elas podem ser acessadas, e até mesmo alteradas, de fora da classe.

Ele deixou o campo como público... Por que ter todo este trabalho para descobrir a senha dele? Eu posso ler o nome diretamente!

```
string nome = agenteCIA.NomeVerdadeiro;
```



Não é necessário chamar nenhum método. O campo NomeVerdadeiro está escancarado para todo mundo ver!

O agente João pode usar campos private para impedir que sua identidade secreta seja vista por objetos do espião inimigo. Uma vez que você tenha declarado o campo NomeVerdadeiro como privado, a única forma de acessá-lo é chamando métodos que tenham acesso aos itens privados da classe. Então o agente da KGB é frustrado!

O objeto agenteKGB não consegue acessar os campos privados do agenteCIA porque eles são instâncias de classes diferentes.

Só substitua public por private e, boom, seus campos agora estão escondidos do mundo.

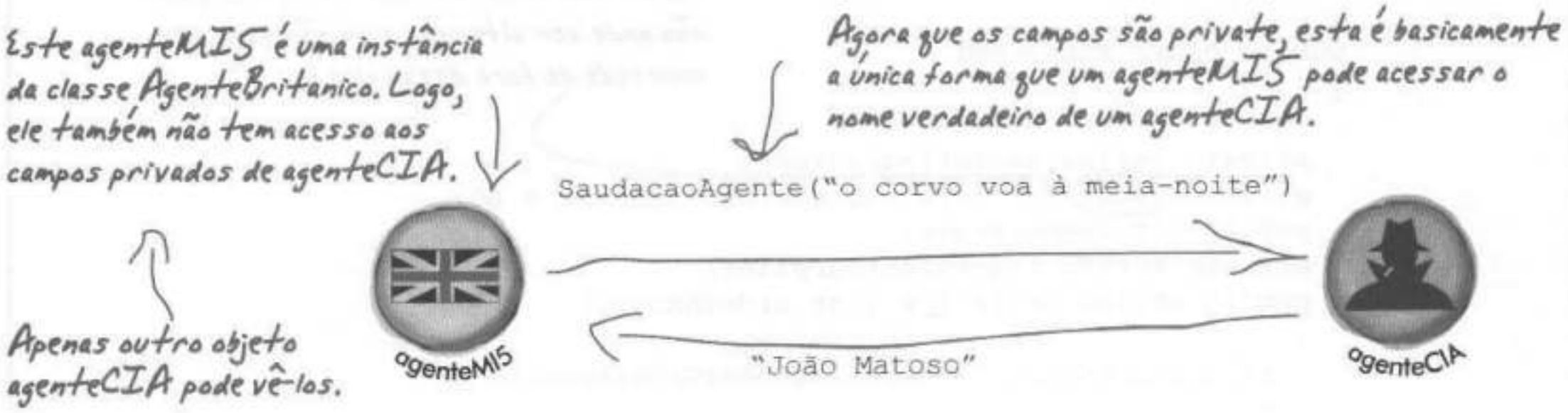
```
private string NomeVerdadeiro;
```

Você também quer garantir que o campo que armazena a senha seja private, caso contrário o agente inimigo pode descobri-la.

Manter seus campos e métodos privados garante que nenhum código de fora fará mudanças inesperadas aos valores que você está usando.

Campos e métodos privados só podem ser acessados de dentro da classe

Só há uma forma de um objeto chegar aos dados armazenados dentro de campos privados de outros objetos: usando campos e métodos públicos que retornem estes dados. Mas enquanto os agentes da KGB e do MI5 precisam usar o método SaudacaoAgente(), agentes amigáveis podem ver tudo – qualquer classe pode ver campos privados em outras instâncias da mesma classe.



Aponte seu lápis

Aqui vemos uma classe com alguns campos private. Circule os comandos abaixo que não **compilarão** se executarem de fora da classe usando **uma instância do objeto chamado mySuperChef** (meu super cozinheiro).

```
public class SuperChef
{
  public string cookieRecipe;
  private string secretIngredient;
  private const int loyalCustomerOrderAmount = 60;
  public int Temperature;
  private string ingredientSupplier;
  public string GetRecipe (int orderAmount)
  {
    if (orderAmount >= loyalCustomerOrderAmount)
    {
      return cookieRecipe + " " + secretIngredient;
    }
    else
    {
      return cookieRecipe;
    }
  }
}
```

```
1. string ovenTemp = mySuperChef.Temperature;
2. string supplier = mySuperChef.ingredientSupplier;
3. int loyalCustomerOrderAmount = 94;
4. mySuperChef.secretIngredient = "cardamomo";
5. mySuperChef.cookieRecipe = "pegue 3 ovos, 2 ½ xícaras de farinha, 1 c. de chá
de sal, 1 c. de chá de baunilha e 1 ½ xícara de açúcar e misture tudo. Cozinhe
por 10 minutos em forno alto. Hmm!";
6. string recipe = mySuperChef.GetRecipe(56);
```

7. Depois de executar todas as linhas acima que compilarão, qual é o valor da receita?

.....

.....

Aponte seu lápis Solução

Aqui vemos uma classe com alguns campos private. Circule os comandos abaixo que não **compilarão** se executarem de fora da classe usando **uma instância do objeto chamado mySuperChef** (meu super cozinheiro).

```
public class SuperChef
{
  public string cookieRecipe;
  private string secretIngredient;
  private const int loyalCustomerOrderAmount = 60;
  public int Temperature;
  private string ingredientSupplier;
  public string GetRecipe (int orderAmount)
  {
    if (orderAmount >= loyalCustomerOrderAmount)
    {
      return cookieRecipe + " " + secretIngredient;
    }
    else
    {
      return cookieRecipe;
    }
  }
}
```

Esta é uma constante private. Seu valor não pode ser alterado e ela não pode ser acessada de fora dessa classe.

A única forma de descobrir o ingrediente secreto é comprar um monte de biscoitos. Código externo não pode acessar esse campo diretamente.

#1 não compila porque você não pode atribuir um int a uma string.

```
1. string ovenTemp = mySuperChef.Temperature;

2. string supplier = mySuperChef.ingredientSupplier;

3. int loyalCustomerOrderAmount = 94;

4. mySuperChef.secretIngredient = "cardamomo";

5. mySuperChef.cookieRecipe = "pegue 3 ovos, 2 ½ xicaras de farinha, 1 c. de chá
de sal, 1 c. de chá de baunilha e 1 ½ xicara de açúcar e misture tudo. Cozinhe
por 10 minutos em forno alto. Hmm!";

6. string recipe = mySuperChef.GetRecipe(56);
```

#2 e #4 não compila, porque ingredientSupplier and secretIngredient são privados.

7. Depois de executar todas as linhas acima que compilarão, qual é o valor da receita?

pegue 3 ovos, 2 xícaras de farinha, 1 c. de chá de sal, 1 c. de chá de baunilha e 1 xícara de açúcar e misture tudo. Cozinhe por 10 minutos em forno alto. Hmm!

não existem Perguntas Idiotas

P: Certo, então eu preciso acessar os dados privados através de métodos públicos. Mas o que acontece se a classe com o campo private não me der uma forma de chegar até os dados e meu objeto precisar usá-lo?

R: Então você não poderá acessar os dados de fora do objeto. Quando você está escrevendo uma classe, deve sempre ter certeza de dar aos outros objetos alguma forma de chegar aos dados que eles precisam. Campos privados são componentes muito importantes do encapsulamento, mas eles são só uma parte da história. Escrever uma classe com bom encapsulamento significa dar a outros objetos uma forma intuitiva e fácil de usar para acessar os dados de que eles precisam sem dar-lhes acesso ao que poderia fazê-los danificar dados de que sua classe precisa.

P: Por que eu desejaria manter um campo sem uma forma de ser acessado por outra classe?

R: Às vezes, uma classe precisa controlar informação necessária para ela operar, mas que nenhum outro objeto precisa realmente ver. Aqui vemos um exemplo: quando computadores geram números aleatórios, usam valores especiais chamados sementes. Você não precisa saber como elas funcionam,

mas toda instância de Random, na verdade, contém uma matriz de várias dezenas de números que ela usa para garantir que Next() sempre dê a você um número aleatório. Se você criar uma instância de Random, não conseguirá ver aquela matriz. Isso porque você não precisa dela – mas se você tivesse acesso a ela, poderia ser capaz de inserir-lhes valores e, talvez, fazê-la resultar valores não aleatórios. Por isto, as sementes foram completamente encapsuladas de forma que você não pode acessá-las.

P: Ei, acabei de reparar que todos tratadores de eventos que eu tenho usado tem a palavra-chave private. Por que eles são privados?

R: Porque os formulários do C# são feitos de forma que só os controles neles possam desencadear tratadores de eventos. Quando você coloca a palavra-chave private na frente de qualquer método, ele pode ser usado de dentro de sua classe. Quando o IDE adiciona um tratador de evento ao seu programa, ele o declara como privado para que outros formulários ou objetos não possam chegar até ele. Mas não há regras que digam que um tratador de eventos deve ser private. Na verdade, você pode checar isso sozinho – clique duas vezes em um botão, então mude sua declaração de tratador de evento para public. O código ainda compilará e executará.

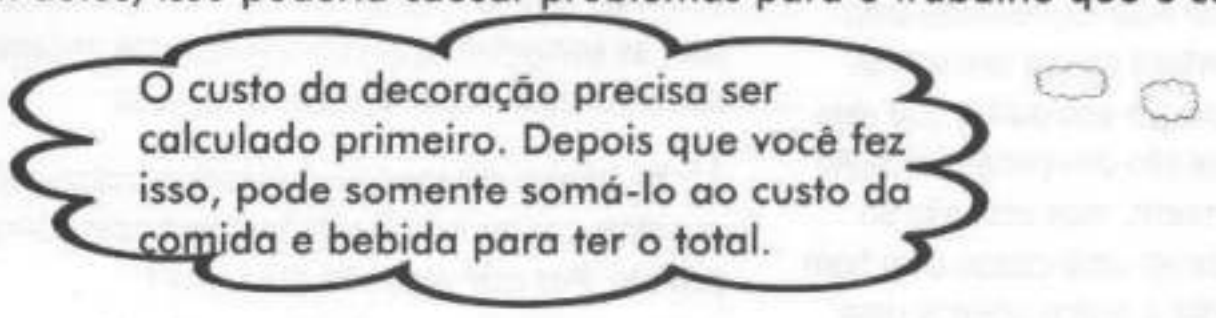
A única forma pela qual um objeto pode chegar aos dados armazenados num campo private dentro de outro objeto é usando métodos public que retornem esses dados.

Algumas idéias para encapsular classes

★ **Pense em formas nas quais os campos possam ser usados incorretamente.**
O que pode dar errado se eles não forem configurados corretamente?

★ **Tudo na sua classe é público?**
Se sua classe só possui campos e métodos públicos, provavelmente, você precisará passar um pouco mais de tempo pensando no encapsulamento.

★ **Quais campos implicam que algum processamento ou cálculo seja feito quando são alterados?**
Esses são os principais candidatos para encapsulamento. Se alguém escreve um método posterior que muda o valor em algum deles, isso poderia causar problemas para o trabalho que o seu programa está tentando fazer.



★ **Só deixe campos e métodos públicos se precisar.**
Se você não tem uma razão para declarar algo como público, não o faça. Você pode bagunçar bastante as coisas para você mesmo deixando todos os campos em seu programa públicos – mas também não vá tornando tudo privado. Passar um pouco de tempo no início do projeto pensando em quais campos realmente precisam ser públicos e quais não pode economizar muito tempo mais tarde.

O encapsulamento mantém seus dados em boas condições

Às vezes, o valor em um campo muda conforme seu programa executa. Se você não diz explicitamente ao seu programa para reiniciar o valor, você pode fazer seus cálculos usando o que estava armazenado lá. Quando for este o caso, você quer que seu programa execute alguns comandos sempre que um campo for alterado – como fazer o programa da Catarina recalcular o custo sempre que você alterar o número de pessoas. Podemos evitar o problema encapsulando os dados, utilizando campos privados. Forneceremos um método para recuperar o valor do campo e outro para configurá-lo e fazer todos os cálculos necessários.

Um exemplo rápido de encapsulamento

Uma classe Farmer (fazendeiro) usa um campo para armazenar o número de vacas e o multiplica por um número para descobrir quantos sacos de ração para gado são necessários para alimentá-las:

```
class Farmer
{
    private int numberOfCows;
}
```

É melhor deixarmos esse campo como private para que ninguém possa alterá-lo sem também mudar bagsOfFeed (sacos de comida) – se eles saírem de sincronia, isso resultará em erros!

Quando você cria um formulário para permitir que um usuário coloque o número de vacas num campo numérico, você precisa ser capaz de alterar o valor no campo numberOfCows (número de vacas). Para fazer isso, você pode criar um método que retorne o valor do campo para o formulário.

```
public int GetNumberOfCows()
{
    return numberOfCows;
}
public void SetNumberOfCows(int newNumberOfCows)
{
    numberOfCows = newNumberOfCows;
    BagsOfFeed = numberOfCows * FeedMultiplier;
}
```

Adicionaremos um método para dar a outras classes uma forma de conseguir o número de vacas.

Estes aqui fazem a mesma coisa!

E aqui temos um método para estabelecer o número de vacas que assegurem a alteração do campo BagsOfFeed também. Agora não há como os dois saírem de sincronia.

Usamos casoCamelo* para os campos privados e CasoPascal* para os públicos.

* (NT:) refere-se ao estilo de uso de maiúsculas e minúsculas. O casoCamelo (de camelCase em inglês) indica que a primeira letra deve ser minúscula e o início de cada outra palavra deve ser maiúscula. O CasoPascal (PascalCase) indica que a primeira letra de todas as palavras deve ser maiúsculas.

Propriedades facilitam o encapsulamento

O C# possui tipos especiais de métodos que facilitam encapsular seus dados. Você pode usar **propriedades**, métodos que são executados quando campo é chamado para configurar ou retornar seu valor. Esses campos, nesse caso, são chamados de **campos de armazenamento**.

Renomearemos o campo private para numberOfCows (repare a minúscula n). Este será o campo de armazenamento para a propriedade NumberOfCows.

Você cria propriedades combinando-as com uma declaração de campo normal. Aqui está a declaração para NumberOfCows.

```
private int numberOfCows;
public int NumberOfCows
{
  get
  {
    return numberOfCows;
  }
  set
  {
    numberOfCows = value;
    BagsOfFeed = numberOfCows * FeedMultiplier;
  }
}
```

Este é um acessador get (obter). Ele é um método executado sempre que o campo NumberOfCows é lido. Possui um valor de retorno que corresponde ao tipo da variável – neste caso ele retorna o valor do campo privado numberOfCows.

Este é um acessador set (atribuir), chamado sempre que se atribui um valor ao campo NumberOfCows. Embora o método não aparente ter nenhum parâmetro, na verdade ele possui um chamado value (valor) que contém qualquer valor atribuído ao campo.

Você usa acessadores get e set exatamente como campos. Aqui temos o código para um botão que estabelece o número de vacas e então mostra os sacos de ração:

Quando essa linha atribui 10 a NumberOfCows, o acessador set joga o valor no campo privado numberOfCows e então atualiza o campo público BagsOfFeed.

```
private void button1_Click(object sender, EventArgs e) {
  Farmer myFarmer = new Farmer();
  myFarmer.NumberOfCows = 10;
  int howManyBags = myFarmer.BagsOfFeed;
  myFarmer.NumberOfCows = 20;
  howManyBags = myFarmer.BagsOfFeed;
}
```

Já que o acessador set NumberOfCows atualizou BagsOfFeed, agora você pode recuperar seu valor.

Embora o código trate NumberOfCows como um campo, ele executa o acessador set, passando 20 para ele como parâmetro. E quando ele consulta o campo BagsOfFeed ele executa o acessador get, que retorna 300.

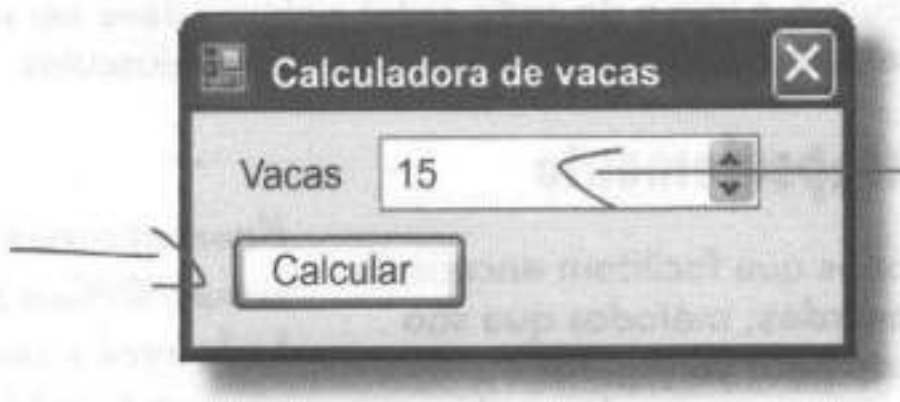
Crie um aplicativo para testar a classe Farmer

Crie um novo aplicativo Windows Forms para testar a classe Farmer e veja as propriedades em ação. Usaremos o método Console.WriteLine() (console, escrever linha) para escrever os resultados na janela de saída (output) no IDE.

1. Adicione a classe Farmer ao seu projeto:

```
public class Farmer {
  public int BagsOfFeed;
  public const int FeedMultiplier = 30;
  private int numberOfCows;
  public int NumberOfCows {
      (add the get and set accessors from the previous page)
  }
}
```

2 Crie esse formulário:



Chame esse botão de Calcular – ele usa os dados públicos de Farmer para escrever uma linha na saída.

Iguale o Value do controle NumericUpDown a 15, seu Minimum a 5 e seu Maximum a 300.

3 Aqui está o código para o formulário. Ele usa Console.WriteLine() para enviar seu resultado à janela Output (que você pode fazer aparecer selecionando "Output" no menu View - visualizar). Você pode passar vários parâmetros a WriteLine() – o primeiro é a seqüência de caracteres para escrever. Se você incluir "{0}" dentro de uma string, então writeLine() o substitui com o primeiro parâmetro. Ele substitui "{1}" pelo segundo, "{2}" pelo terceiro, etc.

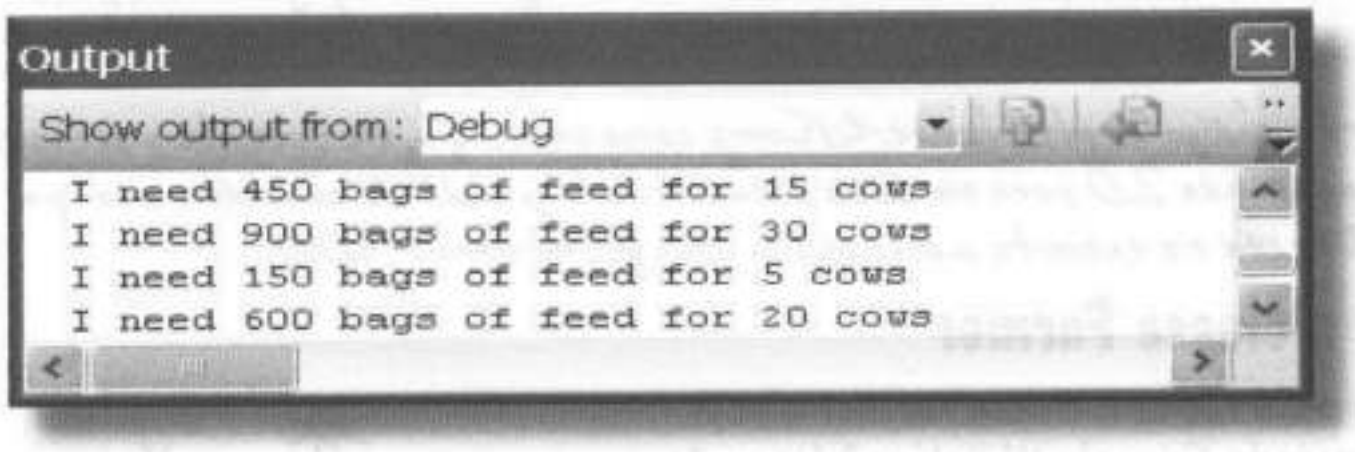
```
public partial class Form1 : Form {
  Farmer farmer;
  public Form1() {
    InitializeComponent();
    farmer = new Farmer() { NumberOfCows = 15 };
  }
  private void numericUpDown1_ValueChanged(object sender, EventArgs e) {
    farmer.NumberOfCows = (int)numericUpDown1.Value;
  }
  private void calculate_Click(object sender, EventArgs e) {
    Console.WriteLine("I need {0} bags of feed for {1} cows",
      farmer.BagsOfFeed, farmer.NumberOfCows);
  }
}
```

Use o método Console.WriteLine() para enviar uma linha de texto à janela Output do IDE.

WriteLine() substitui {0} pelo valor no primeiro parâmetro, e {1} pelo segundo parâmetro.

Use propriedades automáticas para terminar a classe

Parece que a Calculadora de Vacas funciona muito bem. Faça um teste – execute-a e clique no botão. Então altere o número de vacas para 30 e clique de novo. Faça a mesma coisa com cinco vacas e então com 20. É com isto que sua janela Output deve parecer:



Mas há um problema com a classe. Adicione um botão ao formulário que execute esse comando:

```
farmer.BagsOfFeed = 5;
```

Agora execute seu programa novamente. Ele funciona bem até você pressionar o botão novo. Mas pressione este botão e então o botão Calcular de novo. Agora seu resultado te diz que você precisa de cinco sacos de ração – não importa quantas vacas você tem!

Veja bem!

Propriedades automáticas são uma característica do C# 3.0.

Se você ainda está usando o Visual Studio 2005 e o C# 2.0, esse código não funcionará. Recomendamos que você use o Visual Studio 2008 Express. Você pode baixá-lo de graça!

Encapsule totalmente a classe Farmer

O problema é que sua classe **não está totalmente encapsulada**. Você usou propriedades para encapsular NumberOfCows, mas BagsOfFeed ainda é pública. Este é um problema comum. Na verdade, é tão comum que o C# possui uma forma de consertá-lo automaticamente: basta mudar o campo público BagsOfFeed para uma **propriedade automática**. O IDE facilita muito para você adicionar propriedades automáticas. Aqui vemos como:

O trecho de código prop-tab-tab adiciona uma propriedade automática ao seu código.

1. Remova o campo BagsOfFeed da classe Farmer. Coloque o cursor onde o campo costumava ficar, e então digite prop e pressione o tab duas vezes. O IDE adicionará essa linha ao seu código:

```
public int MyProperty { get; set; }
```

2. Aperte o tab – o cursor pula para MyProperty. Altere seu nome para BagsOfFeed:

```
public int BagsOfFeed { get; set; }
```

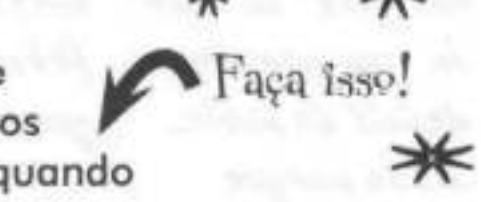
Agora você tem uma propriedade, em vez de um campo. Quando o C# vê isso, ele funciona exatamente do mesmo jeito como se você tivesse usado um campo de armazenamento (como o privado numberOfCows por trás da propriedade pública NumberOfCows).

3. Isso ainda não resolveu nosso problema. Mas há uma forma fácil de fazer isso – crie uma **propriedade apenas leitura**:

```
public int BagsOfFeed { get; private set; }
```

Tente recompilar seu código – você terá um erro na linha no botão que configura BagsOfFeed dizendo a você que o **acessador set é privado**. Agora sua classe Farmer está melhor encapsulada!

E se quisermos alterar o multiplicador de ração?



Fizemos uma Calculadora de Vacas para usar uma const no multiplicador de ração. Mas e se quisermos usar a mesma classe Farmer em programas diferentes que precisam de outros multiplicadores de ração? Você viu como encapsulamento pobre pode causar problemas quando você deixa campos em uma classe muito acessíveis a outras. É por isso que você deveria **somente deixar campos e métodos públicos se precisar**. Já que a Calculadora de Vacas nunca atualiza FeedMultiplier, não há necessidade de permitir que outra classe tenha a capacidade de alterá-la. Então vamos transformá-la em uma propriedade automática apenas leitura.

1. Remova esta linha de seu programa:

```
public const int FeedMultiplier = 30;
```

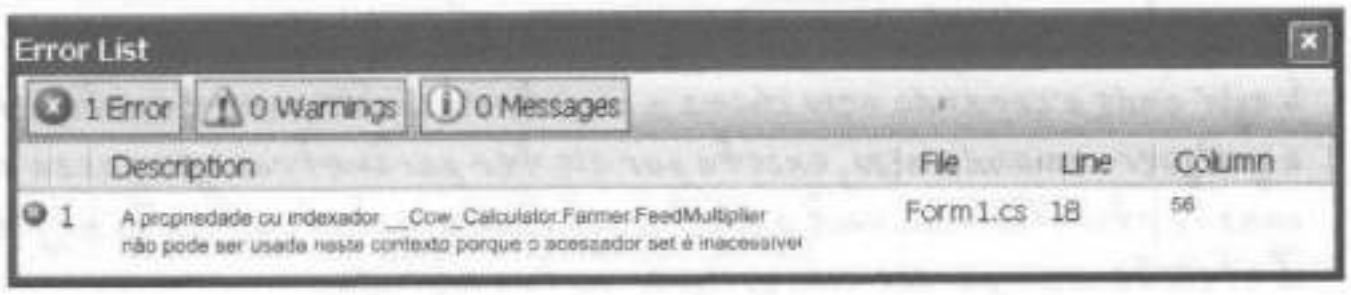
Use prop-tab-tab para adicionar uma propriedade automática apenas leitura:

```
public int FeedMultiplier { get; private set; }
```

2. Vá em frente e faça essa alteração em seu código. Depois execute-o. Ops – algo deu errado! BagsOfFedd sempre retorna 0 sacos!
Espere, isso faz sentido. FeedMultiplier nunca foi inicializado. Ele começa com o valor padrão como zero e nunca muda. Quando é multiplicado pelo número de vacas, ainda resulta em zero. Então adicione um inicializador de objeto:

```
public Form1() {
  InitializeComponent();
  farmer = new Farmer() { NumberOfCows = 15, FeedMultiplier = 30 };
```

Ops – o programa não vai compilar! Você deve ver esse erro:



Você só pode inicializar campos e propriedades públicos dentro de um inicializador de objeto. Então como você pode garantir que seu objeto seja inicializado corretamente se alguns campos que precisam ser inicializados são privados?

Use um construtor para inicializar campos privados

Tudo que você tem que fazer para adicionar um construtor a uma classe é adicionar um método que possua o mesmo nome da classe e nenhum valor de retorno.

Se você precisa inicializar seu objeto, mas alguns dos campos são privados, um inicializador de objeto apenas não dará conta. Felizmente, há um método especial que você pode acrescentar a qualquer classe, o **construtor**. Se uma classe possui um construtor, este é o primeiro método que executará quando a classe for criada com o comando new. Você pode passar parâmetros ao construtor para dar a ele valores que devam ser inicializados. Mas o construtor **não possui um valor de retorno**. Pois, na realidade, você não vai chamá-lo diretamente. Você passa seus parâmetros ao comando new e já sabe que ele retorna o objeto – então não é possível que um construtor retorne nada.

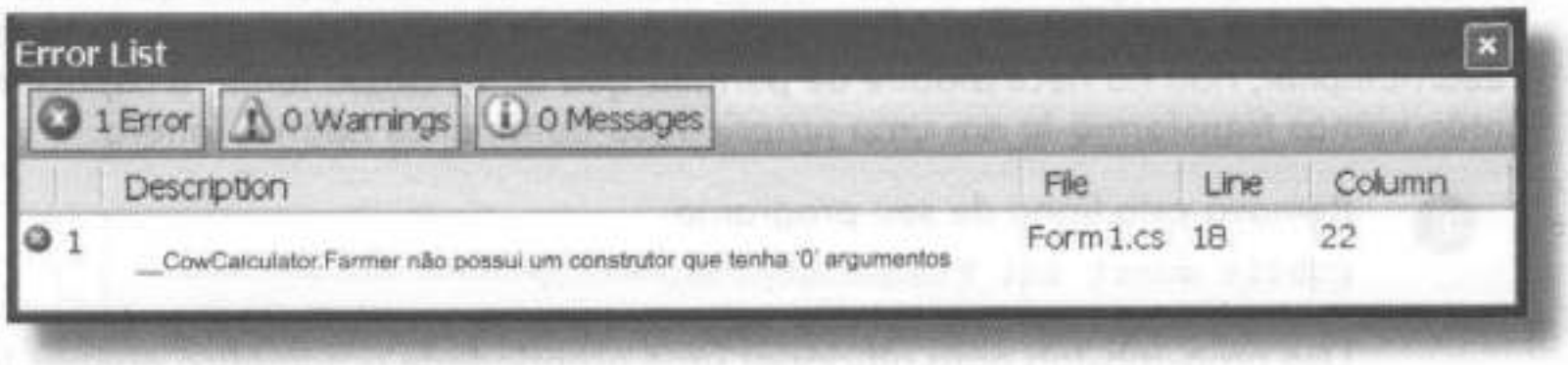
1 **Adicione um construtor à sua classe Farmer**
Este construtor possui apenas duas linhas, mas muito tem acontecendo aqui. Então vamos ver tudo passo-a-passo. Nós já sabemos que precisamos do número de vacas e um multiplicador de ração para a classe, então iremos adicioná-los como parâmetros ao construtor. Daí, criaremos as duas propriedades.

```
public Farmer(int numberOfCows, int feedMultiplier) {
  FeedMultiplier = feedMultiplier;
  this.NumberOfCows = numberOfCows;
}
```

A primeira coisa que faremos é configurar o multiplicador de ração, porque ele precisa estar pronto antes que possamos chamar o acessador set NumberOfCows.

Repare como não há "void" ou "int" ou qualquer tipo depois de public. Isso porque construtores não possuem valor de retorno.

Se criarmos apenas o campo privado numberOfCows, o acessador set NumberOfCows nunca seria chamado. Atribuindo valor para this.NumberOfCows garantimos que ele seja chamado.



Este é o erro que você verá se seu construtor tiver parâmetros, mas seu new() não tiver nenhum.

2 **Agora altere o formulário para que ele possa usar o construtor**
A única coisa que você precisa fazer agora é alterar o formulário para o comando new, que cria o objeto Farmer, usar o construtor, em vez de um inicializador de objeto.

```
public Form1() {
  InitializeComponent();
  farmer = new Farmer(15, 30);
}
```

É aqui onde o comando new chama o construtor. Ele parece **exatamente igual a qualquer comando new**, exceto por ele ter parâmetros que passa ao método construtor. Quando você o digitar no código, preste atenção na janela IntelliSense – parece com qualquer outro método.

Você já sabe que o formulário é um objeto. Bem, ele tem um construtor também! É isso que é esse método – note como seu nome é Form1 (como a classe) e não possui nenhum valor de retorno.

Construtores bem de perto

Vamos dar uma olhada mais de perto no construtor Farmer para que possamos ter uma boa idéia do que está acontecendo.

Construtores não retornam nada, então não existe um tipo de retorno.

Este construtor possui dois parâmetros, que funcionam exatamente como quaisquer outros. O primeiro informa o número de vacas e o segundo é o multiplicador de ração.

```
public Farmer(int numberOfCows, int feedMultiplier) {
  FeedMultiplier = feedMultiplier;
  this.NumberOfCows = numberOfCows;
}
```

Precisamos estabelecer o multiplicador primeiro, porque o segundo comando chama o acessador set NumberOfCows. Ele precisa que o FeedMultiplier tenha um valor para poder configurar BagsOfFeed.

Você verá com freqüência a palavra-chave this em um construtor. Chamando this.NumberOfCows, estamos falando ao C# para configurar a propriedade NumberOfCows usando seu acessador set público.

Se você não colocar o this., então o C# **chamará o acessador set privado.** Faça uma tentativa – remova this. e veja por si mesmo.

não existem

Perguntas Idiotas

P: É possível ter um construtor sem nenhum parâmetro?

R: Sim. É muito comum realmente para uma classe ter um construtor sem parâmetros. Na verdade, você já viu um exemplo disso – o construtor do seu formulário. Olhe dentro de um formulário Windows adicionado recentemente e encontre sua declaração de construtor:

```
public Form1() {
  InitializeComponent();
}
```

Esse é o construtor para seu objeto formulário. Ele não tem nenhum parâmetro, mas tem que fazer muitas coisas. Pare um minuto e abra o Form1.Designer.cs. Encontre o método InitializeComponent() clicando no "+" próximo a "Windows Form Designer generated code" (código gerado pelo Designer de formulários do Windows).
Esse método inicializa todos os controles no formulário e configura todas as propriedades deles. Se você arrastar um novo controle para o formulário no designer de formulário do IDE e alterar algumas de suas propriedades na janela Properties, verá essas mudanças refletidas dentro do método InitializeComponent().

Esse é o construtor para seu objeto formulário. Ele não tem nenhum parâmetro, mas tem que fazer muitas coisas. Pare um minuto e abra o Form1.Designer.cs. Encontre o método InitializeComponent() clicando no "+" próximo a "Windows Form Designer generated code" (código gerado pelo Designer de formulários do Windows).
Esse método inicializa todos os controles no formulário e configura todas as propriedades deles. Se você arrastar um novo controle para o formulário no designer de formulário do IDE e alterar algumas de suas propriedades na janela Properties, verá essas mudanças refletidas dentro do método InitializeComponent().

Veja bem!

Quando um parâmetro de um método tem o mesmo nome de um campo, então ele mascara o campo.

Você notou como o parâmetro numberOfCows do construtor se parece bastante com o campo de armazenamento por trás da propriedade NumberOfCows? Se você queria usar o campo de armazenamento no construtor, você usaria "this." – numberOfCows refere-se ao parâmetro e this.numberOfCows é como você acessaria o campo private.

não existem Perguntas Idiotas

P: Por que eu precisaria de lógica complicada em um acessador get ou set? Isto não é apenas uma forma de criar um campo?

R: Porque há um momento quando você sabe que ao criar um campo terá que fazer algum cálculo ou executar alguma ação. Pense no problema da Catarina – eles apareceram porque o formulário não executou o método para recalcular o custo da decoração depois de estabelecer o número na classe DinnerParty. Se substituíssemos o campo por um acessador set, então poderíamos garantir que o último recalculasse o custo. (Na verdade, você fará exatamente isso daqui a apenas algumas páginas!)

P: Espere um minuto – então qual é a diferença entre um método e um acessador get ou set?

R: Não há nenhuma! Acessadores get e set são um tipo especial de método – um que é declarado dentro de um campo e chamado sempre que o campo é ativado. Acessadores get sempre retornam um valor que é o mesmo tipo que um campo e os set sempre recebem um parâmetro chamado value cujo tipo é o mesmo que o campo. Ah, e por falar nisso, você pode apenas dizer "propriedade", em vez de "acessador get e set".

P: Então você pode ter QUALQUER tipo de comando numa propriedade?

R: Com certeza. Tudo que você pode fazer num método, pode fazer numa propriedade. Elas podem chamar outros métodos, acessar outros campos, até criar objetos e instâncias. Mas esses métodos só são chamados quando um campo é acessado. Logo, não há motivo para existir algum comando neles sem relação com acessar ou configurar o campo.

P: Se um acessador set sempre recebe um parâmetro chamado value, por que a declaração dele não tem parênteses com "int value" dentro, como você teria com qualquer outro método que leva um parâmetro chamado value?

R: Porque o C# foi feito para você não ter que digitar informação extra que o compilador não precise. O parâmetro é declarado sem você ter que digitá-lo explicitamente. Isto não parece muito quando você está digitando uma ou duas coisas, mas quando tem que digitar algumas centenas, isso pode poupar muito tempo (além de prevenir erros).
Todo acessador set sempre tem exatamente um parâmetro chamado value e o tipo desse parâmetro sempre corresponde ao tipo do campo. O C# foi criado para poupar você de escrever código redundante. Ele possui toda a informação de que precisa sobre o tipo e parâmetro assim que você digita "get {". Então não há necessidade de digitar mais nada; com o compilador do C# você não digitará mais que o necessário.

P: Espere um segundo – é por isso que eu não preciso acrescentar um valor de retorno no meu construtor?

R: Exatamente! Seu construtor não possui um valor de retorno porque todo construtor será sempre void. Seria redundante digitar "void" no início de cada construtor.

P: Eu posso ter um get sem um set e um set sem um get?

R: Sim! Quando você tem um acessador get, mas não tem um set, você cria um campo apenas leitura. Por exemplo, a classe AgenteSecreto pode ter um campo apenas leitura para o nome:

```
string name = "Dash
Martin";
public string Name {
  get { return name; }
```

E se você criar uma propriedade com um acessador set, mas não um get, seu campo poderá apenas ser escrito, mas não lido. A classe AgenteSecreto poderia usar isto para um campo Senha no qual outros espiões poderiam escrever mas não ler:

```
public string Password {
 set {
  if (value == secretCode)
{
    name = "Herb Jones";
  }
}
```

Ambas as técnicas podem vir muito a calhar quando você está fazendo encapsulamento.

Propriedades (acessadores get e set) são um tipo especial de método que só executa quando outra classe le ou escreve uma propriedade.

Aponte seu lápis

Dê uma olhada nesses acessadores get e set. O formulário que está usando essa classe possui uma nova instância de CableBill (conta da TV a cabo) chamada thisMonth (esse mês) e chama o método GetThisMonthsBill () (recupera a conta desse mês) com um clique no botão. Escreva o valor da variável amountOwed (quantia devida) depois que o código abaixo executar.

```
class CableBill
{
  private int payPerViewMovies;
  public int numberOfChannels;
  public const int serviceFee = 45;
  public int PayPerViewMovies
  {
    get
    {
      return payPerViewMovies;
    }
    set
    {
      payPerViewMovies = value;
    }
  }
  public int GetThisMonthsBill()
  {
    int payPerViewCost = PayPerViewMovies * 4;
    int channelCost = numberOfChannels * 2;
    int totalCost = payPerViewCost + channelCost + serviceFee;
    return totalCost;
  }
}
```

1.
```
thisMonth.PayPerViewMovies = 5;
thisMonth.numberOfChannels = 2;
int amountOwed = thisMonth.GetThisMonthsBill();
MessageBox.Show ("You Owe " + amountOwed);
```

Qual é o valor de amountOwed?

2.
```
thisMonth.payPerViewMovies = 5;
thisMonth.numberOfChannels = 2;
int amountOwed = thisMonth.GetThisMonthsBill();
MessageBox.Show ("You Owe " + amountOwed);
```

Qual é o valor de amountOwed?

3.
```
thisMonth.PayPerViewMovies = 5;
int amountOwed = thisMonth.GetThisMonthsBill();
MessageBox.Show ("You Owe " + amountOwed);
```

Qual é o valor de amountOwed?

não existem Perguntas Idiotas

P: Eu reparei que você usou nomes começando com letra maiúscula em alguns campos e letra minúscula para outros. Isto importa?

R: Sim – importa para você, mas não para o compilador. O C# não se preocupa com a forma como você nomeia suas variáveis, mas se você escolher nomes estranhos isto dificultará a leitura do seu código. Às vezes, ele pode ficar confuso quando você tem variáveis com o mesmo nome, exceto pelo fato de que uma começa com letra maiúscula e a outra com minúscula.

Aqui vão algumas dicas sobre nomes de variáveis para ajudá-lo a fazer isso corretamente. Elas não são regras rígidas – o compilador não liga se uma variável começa com letra maiúscula ou minúscula – mas são boas sugestões para ajudá-lo a deixar seu código mais fácil de ler.

1. Quando você declarar um campo privado, deve ser em casoCamelo - inicial minúscula e a primeira letra de cada palavra seguinte maiúsculas.
2. Propriedades e métodos públicos devem usar CasoPascal - todas as palavras começam com uma letra maiúscula.
3. Parâmetros para métodos devem estar em casoCamelo.
4. Alguns métodos, especialmente construtores, terão parâmetros com os mesmos nomes dos campos. Quando isso acontecer, o parâmetro mascara o campo, ou seja, comandos no método que usam o nome acabam se referindo ao parâmetro, não ao campo. Use a palavra-chave this para resolver o problema – adicione-a à variável para dizer ao compilador que você está falando sobre o campo, não sobre o parâmetro.

Aponte seu lápis
Solução

Escreva o valor da variável amountOwed (quantia devida) depois que o código abaixo executou.

1.
```
thisMonth.PayPerViewMovies = 5;
thisMonth.numberOfChannels = 2;
int amountOwed = thisMonth.GetThisMonthsBill();
MessageBox.Show ("You Owe " + amountOwed);
```

Qual é o valor de amountOwed?

28

2.
```
thisMonth.payPerViewMovies = 5;
thisMonth.numberOfChannels = 2;
int amountOwed = thisMonth.GetThisMonthsBill();
MessageBox.Show ("You Owe " + amountOwed);
```

Qual é o valor de amountOwed?

não vai compilar

3.
```
thisMonth.PayPerViewMovies = 5;
int amountOwed = thisMonth.GetThisMonthsBill();
MessageBox.Show ("You Owe " + amountOwed);
```

Qual é o valor de amountOwed?

42

Aponte seu lápis

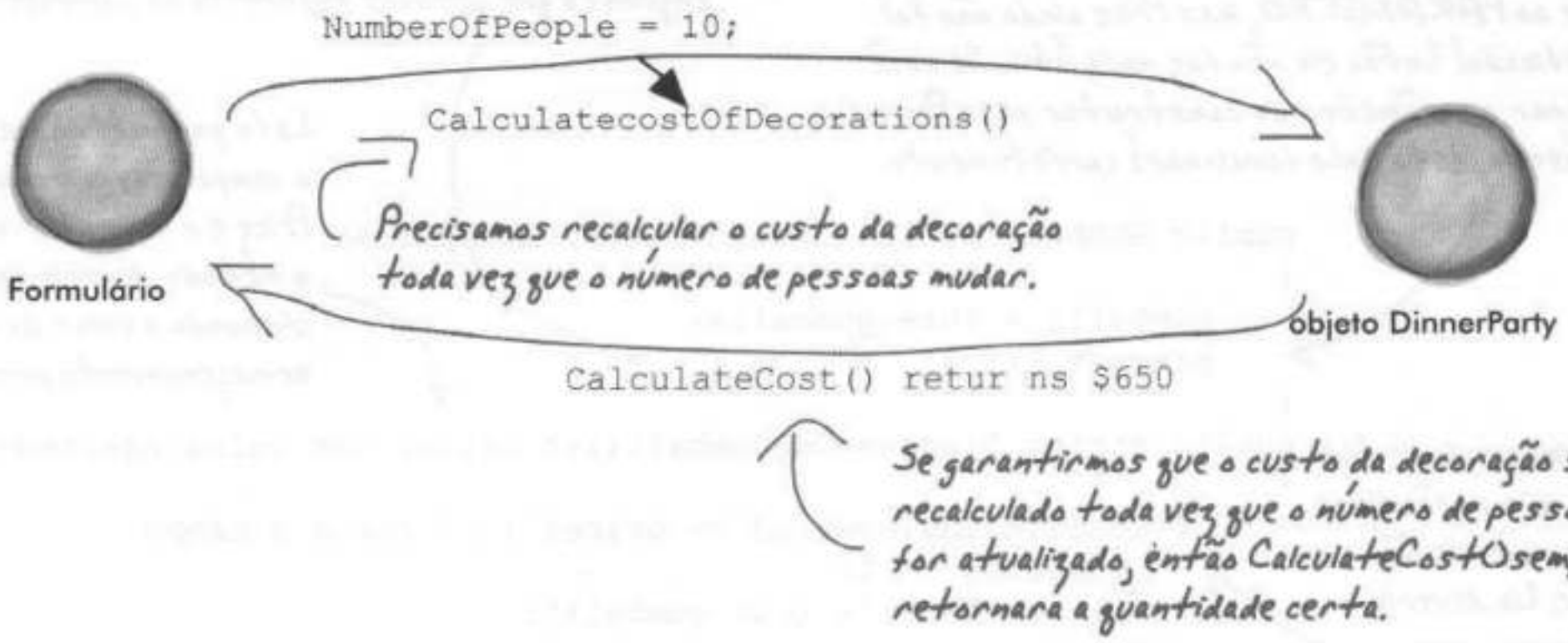
Esse código tem problemas. Escreva o que você acha que está errado com ele e o que você mudaria.

```
class GumballMachine {
    private int gumballs;
    private int price;
    public int Price
    {
        get
        {
            return price;
        }
    }
    public GumballMachine(int gumballs, int price)
    {
        gumballs = this.gumballs;
        price = Price;
    }
    public string DispenseOneGumball(int price, int coinsInserted)
    {
        if (this.coinsInserted >= price) { // check the field
            gumballs -= 1;
            return "Here's your gumball";
        } else {
            return "Please insert more coins";
        }
    }
}
```

Exercício

Use o que aprendeu sobre propriedades e construtores para corrigir o programa Planejador de Festa da Catarina.

1 **Como arrumar a calculadora do Jantar**
Se queremos consertar a classe DinnerParty, vamos precisar de garantir que o método CalculateCostODecorations() seja chamado toda vez que o NumberOfPeople mudar.



2 **Adicione propriedades e um construtor**
Tudo que você precisa fazer para resolver o problema da Catarina é garantir que a classe DinnerParty esteja bem encapsulada. Você vai começar **transformando NumberOfPeople em uma propriedade** que sempre chame CalculateCostOfDecorations() quando for chamada. Então **adicionará um construtor** que garanta que a instância seja inicializada corretamente. Finalmente, você **alterará o formulário** para que ele use o novo construtor. Se você fizer isto certo, é a única alteração que terá de fazer no formulário.

★ Você precisará criar uma nova propriedade para NumberOfPeople com um acessador set que chame CalculateCostOfDecorations(). Ele terá um campo de armazenamento, o numberOfPeople.

★ O acessador set `NumberOfPeople` precisa ter um valor para passar como o parâmetro ao método `CalculateCostOfDecorations()`. Então adicione um campo bool privado chamado fancyDecorations que você altera sempre que `CalculateCostOfDecorations()` é chamado.

★ Acrescente um construtor que configure a classe. Ele precisa ter três parâmetros para o Número de Pessoas, Opção Saudável e Decoração Chique. O formulário atualmente chama dois métodos quando inicializa o objeto DinnerParty – coloque-os no construtor.

```
dinnerParty.CalculateCostOfDecorations(fancyBox.Checked);
dinnerParty.SetHealthyOption(healthyBox.Checked);
```

★ Aqui temos o construtor para o formulário – todo o resto nele fica igual:

```
InitializeComponent();
  dinnerParty = new DinnerParty((int)numericUpDown1.Value,
                        healthyBox.Checked, fancyBox.Checked);
  DisplayDinnerPartyCost();
}
```

Aponte seu lápis
Solução

Esse código tem problemas. Escreva o que você acha que está errado com ele, e o que você mudaria.

O preço com letras minúsculas refere-se ao parâmetro do construtor, não ao campo. Esta linha atribui o valor retornado pelo acessador get Price ao PARÂMETRO, mas Price ainda não foi modificado! Então ele não faz nada útil. Se você alterar o parâmetro do construtor para Price com maiúscula, esta linha funcionará corretamente.

A palavra-chave this está no gumballs (chicletes) errado. this.gumballs refere-se à propriedade, enquanto que gumballs refere-se ao parâmetro.

Este parâmetro mascara o campo privado chamado Price e o comentário diz que o método deveria estar checando o valor do campo de armazenamento price.

```
public GumballMachine(int gumballs, int price)
{
    gumballs = this.gumballs;
    price = Price;
}

public string DispenseOneGumball(int price, int coinsInserted)
{
    if (this.coinsInserted >= price) { // checa o campo
        gumballs -= 1;
        return "Here's your gumball";
    } else {
        return "Please insert more coins";
    }
}
```

A palavra-chave this está num parâmetro, lugar onde ela não serve. Ela deveria estar em price, porque este campo é mascarado por um parâmetro.

Solução do Exercício

Use o que aprendeu sobre propriedades e construtores para corrigir o programa Planejador de Festa da Catarina.

```
public class DinnerParty {
  const int CostOfFoodPerPerson = 25;
  private int numberOfPeople;
  public int NumberOfPeople {
    get { return numberOfPeople; }
    set {
      numberOfPeople = value;
      CalculateCostOfDecorations(fancyDecorations);
    }
  }
  private bool fancyDecorations;
  public decimal CostOfBeveragesPerPerson;
  public decimal CostOfDecorations = 0;
  public DinnerParty(int numberOfPeople, bool healthyOption, bool fancyDecorations) {
    this.NumberOfPeople = numberOfPeople;
    this.fancyDecorations = fancyDecorations;
    SetHealthyOption(healthyOption);
    CalculateCostOfDecorations(fancyDecorations);
  }
  public void SetHealthyOption(bool healthyOption) {
    if (healthyOption) {
      CostOfBeveragesPerPerson = 5.00M;
    } else {
      CostOfBeveragesPerPerson = 20.00M;
    }
  }
  public void CalculateCostOfDecorations(bool fancy) {
    fancyDecorations = fancy;
    if (fancy) {
      CostOfDecorations = (NumberOfPeople * 15.00M) + 50M;
    } else {
      CostOfDecorations = (NumberOfPeople * 7.50M) + 30M;
    }
  }
  public decimal CalculateCost(bool healthyOption) {
    decimal totalCost = CostOfDecorations
      + ((CostOfBeveragesPerPerson + CostOfFoodPerPerson) * NumberOfPeople);
    if (healthyOption) {
      return totalCost * .95M;
    } else {
      return totalCost;
    }
  }
}
```

Agora que o numberOfPeople é privado, não há como o formulário mudá-lo sem também recalcular o custo da decoração. Isto resolve a falha que quase custou à Catarina um de seus melhores clientes!

Usando uma propriedade, você pode garantir que o custo da decoração seja recalculado sempre que o número de pessoas mudar.

Tome cuidado com como você usa "this". Garanta que você a coloque na frente de NumberOfPeople para que ela chame o acessador set.

Tenha certeza de que armazenou a decoração chique em um campo para que o acessador set NumberOfPeople possa usá-la.

E você vai precisar colocá-la na frente de fancyDecorations porque o parâmetro fancyDecorations mascara o campo privado com o mesmo nome.

6 herança

Sua árvore genealógica de objeto

Então lá estava eu andando com o meu objeto bicicleta na curva do homem morto quando percebi que ele foi herdado de DuasRodas e me esqueci de adicionar o método Freios()... Resumindo: vinte e seis pontos e a mamãe me pôs de castigo por um mês.

Às vezes você REALMENTE quer ser exatamente como seus pais.

Você já encontrou sem querer um objeto que faz **quase** exatamente o que você quer que o **seu** objeto faça? Você já se pegou pensando que se pudesse apenas **mudar algumas coisas**, aquele objeto seria perfeito? Bem, esta é apenas uma razão que torna a **herança** um dos conceitos e técnicas mais poderosos na linguagem C#. Antes de avançar neste capítulo, você aprenderá como criar uma subclasse a partir de um objeto para herdar seu comportamento, mas mantendo a **flexibilidade** para alterar este comportamento. Você **evitará duplicar código**, modelará o mundo real mais corretamente e terá o código mais **fácil de manter**.

4. **Dê um nome à segunda aba e adicione os controles da BirthdayParty (Aniversário) nela**
Você fará a interface para lidar com aniversários assim como fez para os jantares.

5. **Ligue sua classe de aniversário aos seus controles**
Agora tudo que você precisa fazer é acrescentar uma referência de BirthdayParty aos campos do formulário e adicionar o código para cada um de seus novos controles para que eles usem seus métodos e propriedades.

não existem Perguntas Idiotas

P: Por que não podemos apenas criar uma nova instância de `DinnerParty`, como o Miguel fez quando queria comparar as três rotas no programa de navegação dele?

R: Porque se você criasse outra instância da classe `DinnerParty`, conseguiria usá-la apenas para planejar jantares extras. Duas instâncias da mesma classe podem ser muito úteis se você precisar gerenciar dois conjuntos diferentes do mesmo tipo de dados. Mas se você precisar armazenar tipos diferentes de dados, precisará de classes diferentes.

P: Como eu sei o que colocar na nova classe?

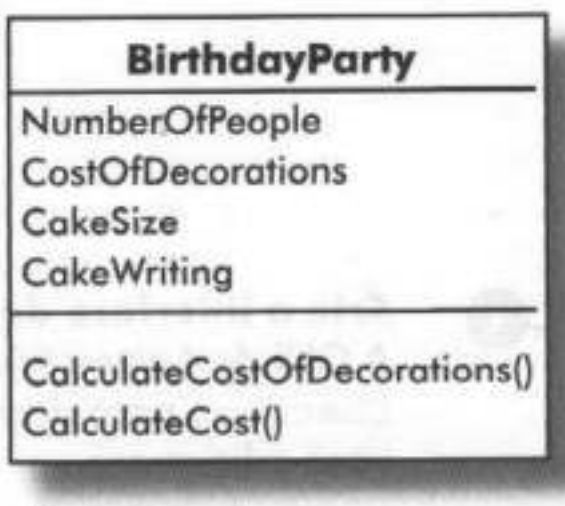
R: Antes que você possa começar a construir uma classe, precisa saber qual problema ela deve resolver. É por isso que você precisava falar com a Catarina – ela é quem usará o programa. Ainda bem que você fez um monte de anotações! Você pode inventar métodos, campos e propriedades para sua classe pensando no comportamento dela (o que ela precisa fazer) e seu estado (o que ela precisa saber).

Exercício

Adicione festas de aniversário ao programa de planejamento de festas de Catarina.

Certifique-se de usar decimal como o tipo para os campos e propriedades que funcionem com valor monetário.

1. **Adicione a classe `BirthdayParty` ao seu programa**
Você já sabe como lidar com a propriedade `NumberOfPeople` e com os métodos `CostOfDecorations` – eles são exatamente iguais às suas contrapartes em `DinnerParty`. Comece criando sua nova classe adicionando os itens idênticos. Então, acrescente o restante do comportamento:



Se a escrita for muito extensa para o bolo, o acessador set reduz o campo de armazenamento de volta ao tamanho certo. Assim, você precisará ter certeza de carregar o valor na caixa de texto sempre que ele ou o número de pessoas mudar (no caso de haver uma longa seqüência de caracteres um bolo menor será selecionado).

- ★ Adicione uma propriedade `int CakeSize` (tamanho do bolo). Faça um método privado, o `CalculateCakeSize()` que iguala `CakeSize` a 8 ou 16, dependendo do número de pessoas. Você precisará chamá-lo do construtor e do acessador `set NumberOfPeople`.

Não queremos nenhum outro método alterando o valor de CakeWriting.

- ★ Você precisará de uma propriedade `string CakeWriting` (escrito no bolo) para armazenar a escrita no bolo. Nós criaremos este código para você.

- ★ O acessador `set CakeWriting` checa `CakeSize` porque tamanhos diferentes de bolo podem conter números diferentes de letras. Então ele usa `value.Length` para checar o tamanho do sting. Se ela for muito grande, em vez de criar o campo privado, o acessador `set` lança uma caixa de mensagem que diz "Muitas letras para um bolo de 40 cm" (ou de 20 cm) ("Too many letters for a 16-inch cake"(or 8-inch cake))

- ★ Todo string possui um método `Substring()` que retorna uma parte dela. `CakeWriting` usa-o para reduzir o tamanho da escrita do bolo – se o número de pessoas mudar e reduzir o tamanho do bolo, você precisará reduzir a seqüência de caracteres também.
- ★ Finalmente, adicione o método `CalculateCost()`. Mas, em vez de pegar o custo da decoração e adicionar o custo das bebidas (que é o que acontece em `DinnerParty`), ele adicionará o custo do bolo.

2 **Atualize o formulário para adicionar abas**

Arraste um `TabControl` da toolbox para seu formulário e modifique seu tamanho para que ele abranja o formulário inteiro. Mude o texto de cada aba usando a propriedade `TabPages`: um botão "..." aparece na janela Properties perto da propriedade. Quando você clicar nele, o IDE abre uma janela que permite editar as propriedades de cada aba. Atribua à propriedade `Text` das abas "Jantar"(Dinner) e "Festa de Aniversário"(BirthdayParty).

Clique nas abas para alternar entre elas. Use a propriedade TabCollection (coleção de abas) para mudar o texto de cada uma. Clique o botão ... ao lado dela e selecione a propriedade Text de cada uma delas.

3 **Nomeie a primeira aba e passe os controles de Jantar para dentro dela**

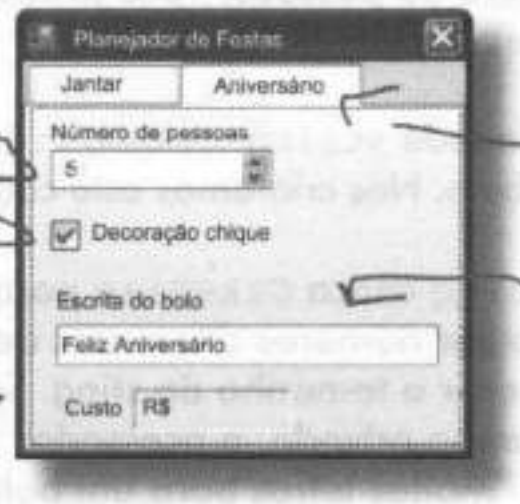
Você arrastará cada um dos controles que lidam com o jantar para dentro da nova aba. Eles funcionarão como agora, mas só serão exibidos quando a aba for selecionada.

Depois de arrastar os controles de Jantar para a aba, eles só serão visíveis quando a aba Jantar for selecionada.

4 **Crie a interface de usuário para a festa de Aniversário (Birthday Party)**

A GUI de Aniversário possui um controle `NumericUpDown` para o número de pessoas, um controle `CheckBox` para decoração chique e um controle `Label` com uma borda 3D para o custo. Além disso, você adicionará um controle `TextBox` para a escrita do bolo.

Essa aba usa os controles NumericUpDown, CheckBox e Label assim como a aba Jantar faz. Nomeie-os como numberBirthday, fancyBirthday e birthdayCost.



Clique na aba Aniversário e adicione os novos controles.

Adicione um controle TextBox chamado cakeWriting para a escrita no bolo (e um rótulo acima dele para que o usuário saiba para que ele serve). Use sua propriedade Text para dar a ele um valor padrão de Feliz Aniversário.

5 **Você precisará dessa propriedade**
Aqui temos o código para a propriedade `BirthdayParty.CakeWriting` (Aniversário.Escrita do Bolo). Ele será bem útil:

```
private string cakeWriting = "";
public string CakeWriting {
  get { return this.cakeWriting; }
  set {
    int maxLength;
    if (CakeSize == 8)
      maxLength = 16;
    else
      maxLength = 40;
    if (value.Length > maxLength) {
      MessageBox.Show("Too many letters for a " + CakeSize + " inch cake");
      if (maxLength > this.cakeWriting.Length)
        maxLength = this.cakeWriting.Length;
      this.cakeWriting = cakeWriting.Substring(0, maxLength);
    }
    else
      this.cakeWriting = value;
  }
}
```

Esta propriedade é um pouco mais complexa do que as já vistas. Ela checa o tamanho do bolo para ver se a escrita é muito grande para ele e usa a variável maxLength para armazenar o tamanho máximo. Se o texto for muito grande, ela mostra uma mensagem de erro e reduz o campo de armazenamento para o tamanho certo, permitindo que seja recarregado na caixa de texto.

Você notou como nós deixamos de usar algumas chaves? Quando você tem apenas um comando num bloco de código, você não precisa adicionar chaves em volta dele.

Toda string possui um método Substring() que retorna uma parte dela. Esta a reduz ao tamanho permitido. Assim, você terá que recarregar o texto no controle quando ele mudar (ou tamanho do bolo).

6 **Junte tudo**
Com todas as peças prontas, agora é só uma questão de escrever um pouco de código para fazer os controles funcionarem.
Adicione um objeto `BirthdayParty` ao formulário. Certifique-se de tê-lo instanciado.
Adicione código ao método tratador de evento do controle `NumericUpDown` para alterar a propriedade `NumberOfPeople` do objeto.
Faça a caixa de seleção Decoração Chique funcionar.
Acrescente um método `DisplayBirthdayPartyCost()` e o adicione a todos os tratadores de evento para que o custo seja atualizado automaticamente sempre que houver uma mudança.

7 **Execute**
Tenha certeza de que o programa funciona como deveria. Veja se ele mostra uma caixa de mensagem informando que o texto é grande demais para o bolo. Certifique-se de que o preço esteja sempre certo. Uma vez que tudo esteja funcionando, acabamos!

Solução do Exercício

Adicione festas de aniversário ao programa de planejamento de festas da Catarina.

```
public partial class Form1 : Form {
  DinnerParty dinnerParty;
  BirthdayParty birthdayParty;
  public Form1() {
    InitializeComponent();
    dinnerParty = new DinnerParty((int)numericUpDown1.Value,
                    healthyBox.Checked, fancyBox.Checked);
    DisplayDinnerPartyCost();
    birthdayParty = new BirthdayParty((int)numberBirthday.Value,
                    fancyBirthday.Checked, cakeWriting.Text);
    DisplayBirthdayPartyCost();
  }
  // Os tratadores de evento fancyBox, healthyBox e numericUpDown1 e
  // o metodo DisplayDinnerCost() são identicos àqueles no
  // exercicio do Jantar no final do Capitulo 5.
  private void numberBirthday_ValueChanged(object sender, EventArgs e) {
    birthdayParty.NumberOfPeople = (int)numberBirthday.Value;
    DisplayBirthdayPartyCost();
  }
  private void fancyBirthday_CheckedChanged(object sender, EventArgs e) {
    birthdayParty.CalculateCostOfDecorations(fancyBirthday.Checked);
    DisplayBirthdayPartyCost();
  }
  private void cakeWriting_TextChanged(object sender, EventArgs e) {
    birthdayParty.CakeWriting = cakeWriting.Text;
    DisplayBirthdayPartyCost();
  }
  private void DisplayBirthdayPartyCost() {
    cakeWriting.Text = birthdayParty.CakeWriting;
    decimal cost = birthdayParty.CalculateCost();
    birthdayCost.Text = cost.ToString("c");
  }
}
```

A instância BirthdayParty é inicializada no construtor do formulário, assim como a de DinnerParty.

Os tratadores de evento dos controles CheckBox e NumericUpDown são exatamente iguais àqueles em jantar.

A forma como o formulário lida com a escrita do bolo pode ser bem simples porque a classe BirthdayParty é bem encapsulada. Tudo que o formulário deve fazer é usar seus controles para estabelecer as propriedades no objeto - que cuida do resto.

Toda a inteligência para lidar com o número de pessoas, a escrita e o tamanho do bolo está implementada dentro dos acessadores set NumberOfPeople e CakeWriting. O formulário deve apenas atribuir e exibir os valores.

```
using System.Windows.Forms;
public class BirthdayParty {
  public const int CostOfFoodPerPerson = 25;
  public decimal CostOfDecorations = 0;
  private bool fancyDecorations;
  public int CakeSize;
  public BirthdayParty(int numberOfPeople, bool fancyDecorations, string cakeWriting) {
    this.numberOfPeople = numberOfPeople;
    this.fancyDecorations = fancyDecorations;
    CalculateCakeSize();
    this.CakeWriting = cakeWriting;
    CalculateCostOfDecorations(fancyDecorations);
  }
  private void CalculateCakeSize() {
    if (NumberOfPeople <= 4)
      CakeSize = 8;
    else
      CakeSize = 16;
  }
```

Quando o objeto BirthdayParty é inicializado, ele precisa saber o número de pessoas, se existem decoração chique e a escrita no bolo, para que então possa iniciar com o custo certo do bolo quando CalculateCost() for chamada.

O construtor está chamando o acessador set para determinar a escrita no bolo. Como o parâmetro pode ser muito grande para o bolo, é preciso calcular o tamanho do bolo primeiro.

O construtor dá valor às propriedades e então executa os cálculos.

O método CalculateCakeSize() dá valor para o campo CakeSize. Ele é chamado pelo acessador set NumberOfPeople e o método CalculateCost().

```
private string cakeWriting = "";
public string CakeWriting {
  get { return this.cakeWriting; }
  set {
    int maxLength;
    if (CakeSize == 8)
      maxLength = 16;
    else
      maxLength = 40;
    if (value.Length > maxLength) {
      MessageBox.Show("Too many letters for a " + CakeSize + " inch cake");
      if (maxLength > this.cakeWriting.Length)
        maxLength = this.cakeWriting.Length;
      this.cakeWriting = cakeWriting.Substring(0, maxLength);
    }
    else
      this.cakeWriting = value;
  }
}
```

A propriedade CakeWriting garante que a escrita do bolo nunca fique muito grande para o tamanho dele. Seu acessador set checa o tamanho do bolo, então usa a propriedade Length dos campos de armazenamento para garantir que o texto não fique muito grande. Se ficou, ele reduz a seqüência de caracteres ao tamanho certo.

Estamos utilizando decimal porque estamos lidando com preços e moedas.

```
public decimal CalculateCost() {
    decimal TotalCost = CostOfDecorations + (CostOfFoodPerPerson * NumberOfPeople);
    decimal CakeCost;
    if (CakeSize == 8)
      CakeCost = 40M + CakeWriting.Length * .25M;
    else
      CakeCost = 75M + CakeWriting.Length * .25M;
    return TotalCost + CakeCost;
  }
  private int numberOfPeople;
  public int NumberOfPeople {
    get { return numberOfPeople; }
    set {
      numberOfPeople = value;
      CalculateCostOfDecorations(fancyDecorations);
      CalculateCakeSize();
      this.CakeWriting = cakeWriting;
    }
  }
  public void CalculateCostOfDecorations(bool fancy) {
    fancyDecorations = fancy;
    if (fancy)
      CostOfDecorations = (NumberOfPeople * 15.00M) + 50M;
    else
      CostOfDecorations = (NumberOfPeople * 7.50M) + 30M;
  }
}
```

O método CalculateCost() é muito parecido com aquele de DinnerParty, exceto que este adiciona o custo do bolo, em vez da opção Escolha Saudável.

Fazer o método CakeWriting reduzir o tamanho do bolo é apenas metade da solução. A outra metade é garantir que seu acessador set execute sempre que o número de pessoas mudar.

Quando o número de pessoas muda, a classe primeiro recalcula o tamanho do bolo e, somente depois, ela usa seu acessador set CakeWriting para reduzir o texto. Assim se uma festa, de 10 pessoas vira uma de 4, uma mensagem de 36 letras será reduzida a uma que caiba num bolo menor.

Chaves são opcionais para blocos de uma linha

Muitas vezes você terá um comando if ou laço while com apenas um único comando dentro de seu bloco. Se isso acontecer muito, você pode acabar com um monte de chaves – o que pode poluir muito a visão! O C# ajuda a evitar esse problema deixando que você não use as chaves se houver apenas um comando. Então a sintaxe seguinte é perfeitamente válida para um loop e um comando if:

```
for (int i = 0; i < 10; i++)
  DoTheJob(i);
```

```
if (myValue == 30)
  myValue *= 5;
```

Mais uma coisa... Você pode acrescentar uma taxa de R$100 para festas com mais de 12 pessoas?

A Catarina está com tanto trabalhando, graças a seu programa, que ela pode começar a cobrar um pouco mais de seus clientes maiores. Então o que será necessário para mudar o software e acrescentar essa nova taxa?

- ★ Mudar `DinnerParty.CalculateCost()` para checar `NumberOfPeople` e acrescentar R$100 ao valor de retorno se ele for maior que 12.
- ★ Faça exatamente o mesmo para `BirthdayParty.CalculateCost()`.

Pare por um minuto e pense em como você adicionaria uma taxa para ambas as classes `DinnerParty` e `BirthdayParty`. Que código você poderia escrever? Onde ele deveria ser colocado?
É fácil... Mas o que acontece se houver três classes similares? Ou quatro? Ou doze? E se você tivesse que manter esse código e fizer mais alterações depois? E se você tivesse que fazer exatamente a mesma alteração em cinco ou seis classes com uma relação muito próxima?

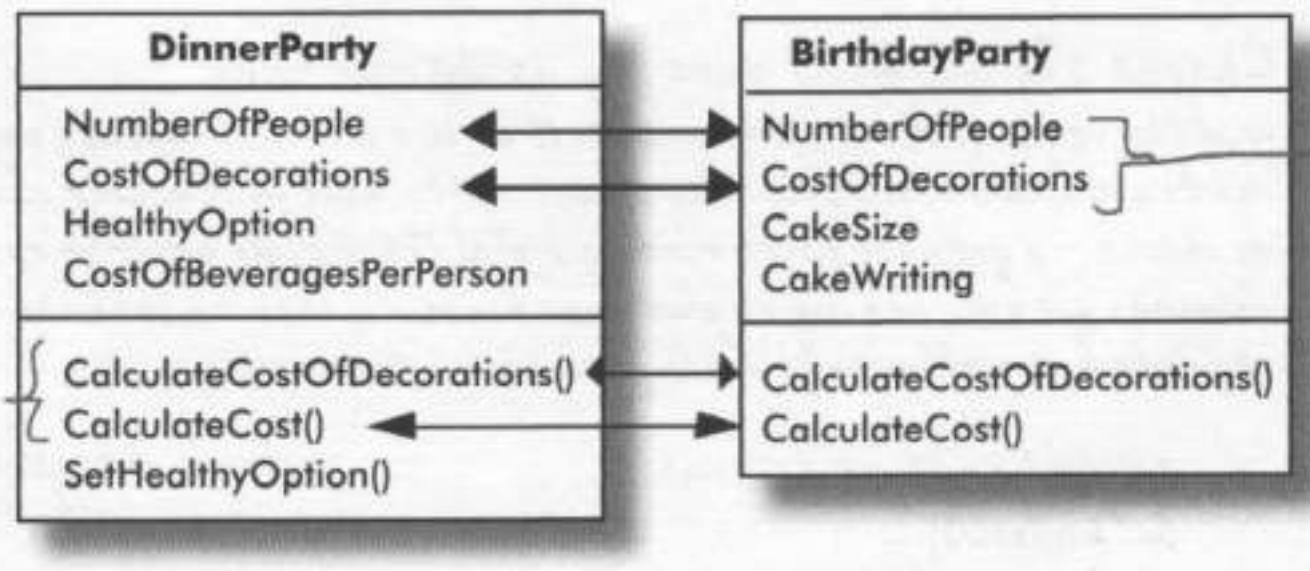
Uau, eu teria que escrever o mesmo código várias e várias vezes. Essa é uma forma muito ineficiente de se trabalhar. Deve haver uma maneira melhor!

Você está certo! Ter o mesmo código repetido em classes diferentes é ineficiente e induz a erros.
Para nossa sorte, o C# nos oferece uma forma melhor de fazer classes que sejam relacionadas e compartilhem comportamento: herança.

Quando suas classes usam herança, você só precisa escrever seu código uma vez

Não é coincidência que suas classes `DinnerParty` e `BirthdayParty` tenham muito código em comum. Quando você escreve programas C#, freqüentemente cria classes que representam coisas no mundo real – e essas coisas geralmente se relacionam entre si. Suas classes possuem código parecido porque as coisas que eles representam no mundo real – um aniversário e um jantar – possuem comportamentos parecidos.

A Catarina precisa calcular o custo de suas festas, não importa que tipo de festas seja.

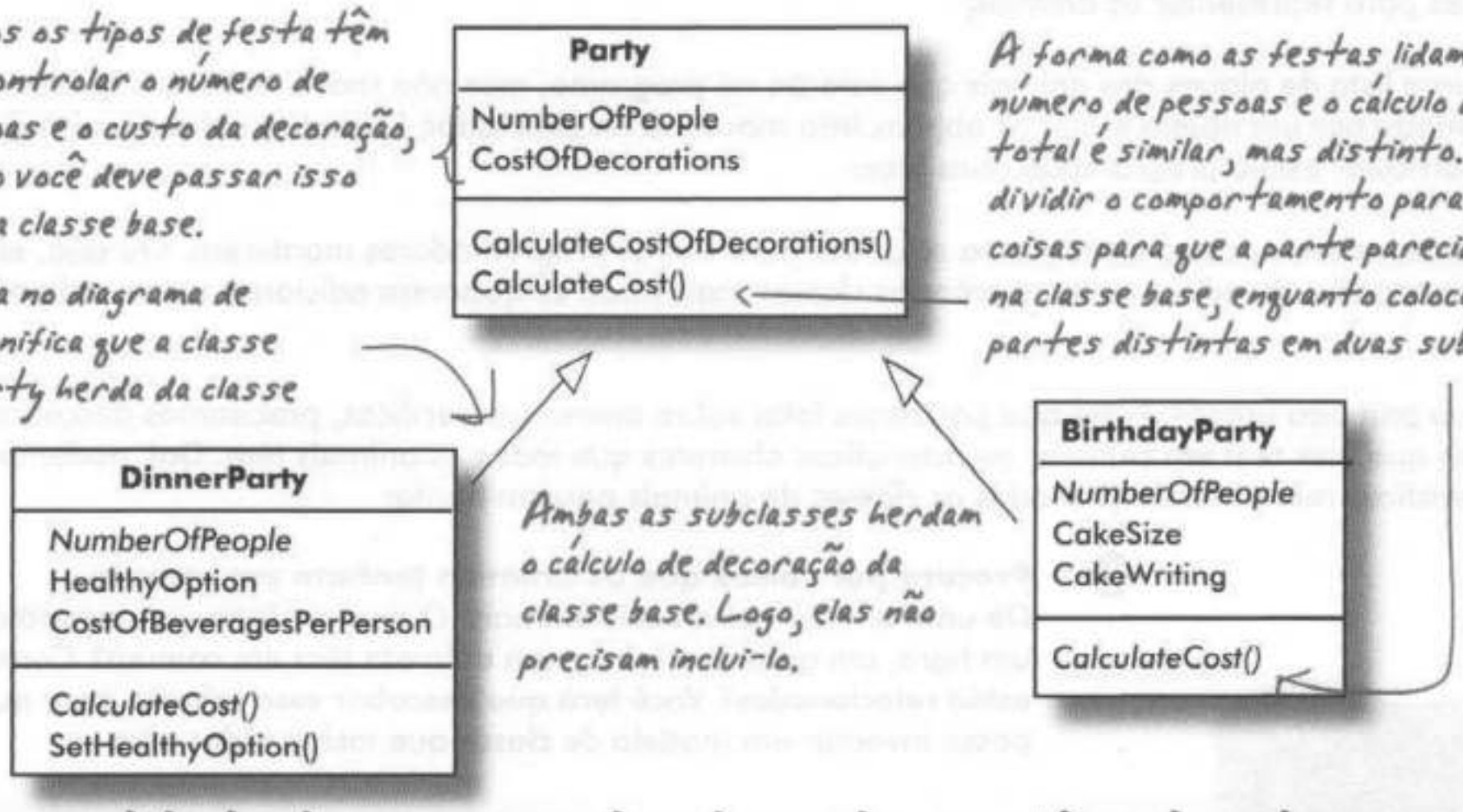


Uma festa de aniversário lida com o número de pessoas e com o custo da decoração quase da mesma forma que um jantar.

Jantares e aniversários são festas

Quando você tem duas classes, que são casos mais específicos de algo mais geral, você pode fazê-las de forma que herdem da mesma classe. Assim, cada uma delas é uma subclasse da mesma classe base.

Ambos os tipos de festa têm que controlar o número de pessoas e o custo da decoração, então você deve passar isso para a classe base.



A forma como as festas lidam com o número de pessoas e o cálculo do custo total é similar, mas distinto. Podemos dividir o comportamento para essas coisas para que a parte parecida fique na classe base, enquanto colocamos as partes distintas em duas subclasses.

Esta seta no diagrama de classe significa que a classe DinnerParty herda da classe Party.

Ambas as subclasses herdam o cálculo de decoração da classe base. Logo, elas não precisam incluí-lo.

Faça seu modelo de classe começando pelo geral e especificando cada vez mais

Os programas em C# usam herança porque ela imita a relação que seus modelos têm no mundo real. As coisas do mundo real geralmente ficam em uma hierarquia, que vai do mais geral ao mais específico, e seus programas possuem suas próprias hierarquias de classe, que fazem a mesma coisa. Em seu modelo de classe, as classes mais abaixo na hierarquia herdam daquelas mais acima.

Se você tem uma receita que precisa de queijo cheddar, pode usar o cheddar envelhecido de Vermont. Mas se a receita colocar como obrigatório este tipo, você não pode qualquer cheddar - precisa deste específico.

her-dar, verbo.
derivar uma característica dos pais ou ancestrais. *Ela queria que o bebê herdasse seus grandes olhos castanhos, e não os pequenos olhos azuis de seu marido.*

Como você criaria um simulador de zoológico?

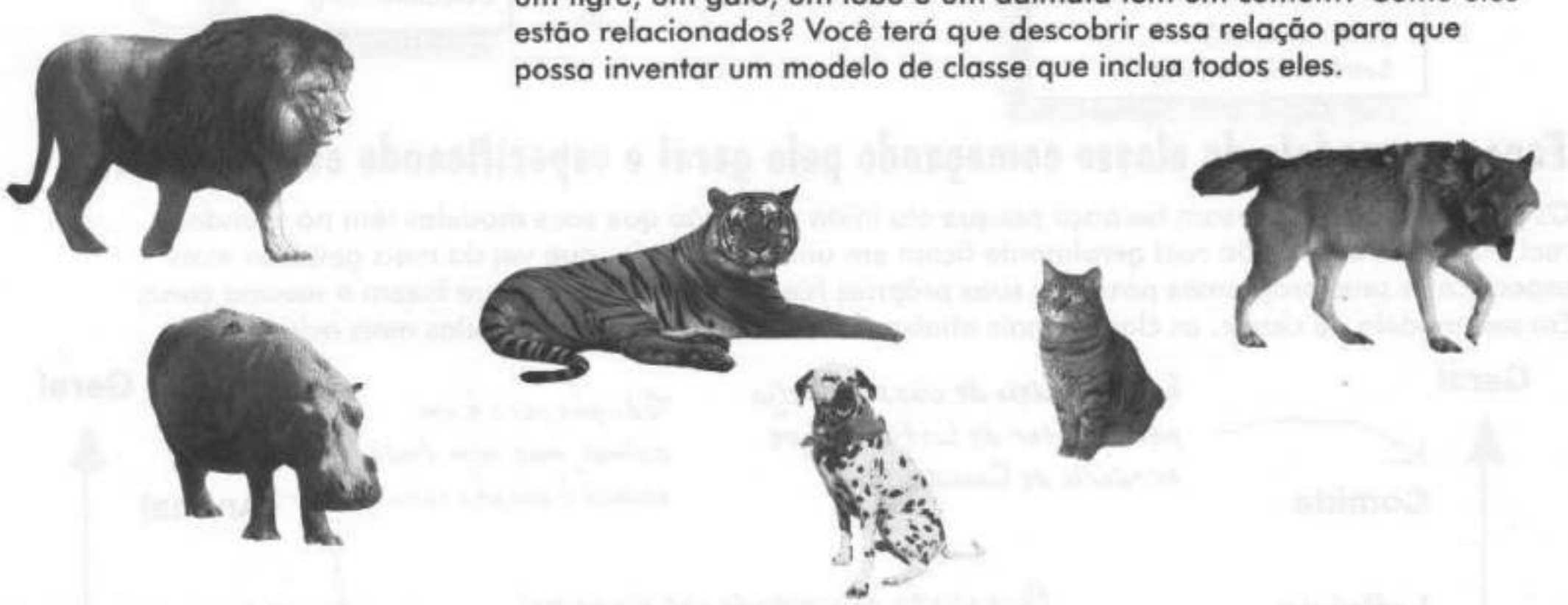
Leões e tigres e ursos... uau! Além desses, hipopótamos, lobos e um felino ocasional. Seu trabalho é criar um programa que simule um zoológico. (Não fique muito animado – não vamos fazer o código realmente, apenas criar as classes para representar os animais).

Recebemos uma lista de alguns dos animais que estarão no programa, mas não todos. Sabemos que cada animal será representado por um objeto e que os objetos irão mover-se no simulador, fazendo qualquer coisa que cada animal em particular esteja programado para fazer.

Mais importante, queremos que o programa seja fácil para outros programadores manterem. Ou seja, eles precisarão ser capazes de adicionar suas próprias classes mais tarde se quiserem adicionar novos animais ao simulador.

Então qual é o primeiro passo? Antes que possamos falar sobre animais específicos, precisamos descobrir as generalidades que eles têm em comum, características abstratas que todos os animais têm. Daí, podemos colocar essas características numa classe que todas as classes de animais possam herdar.

1 **Procure por coisas que os animais tenham em comum**
Dê uma olhada nesses seis animais. O que um leão, um hipopótamo, um tigre, um gato, um lobo e um dálmata têm em comum? Como eles estão relacionados? Você terá que descobrir essa relação para que possa inventar um modelo de classe que inclua todos eles.



Use herança para evitar duplicação de código em subclasses

Você já sabe bastante para perceber que duplicar código é ruim. É difícil de manter e sempre causa dores de cabeça no futuro. Então vamos escolher campos e métodos para uma classe base Animal que você só tenha que escrever uma vez, assim cada uma das subclasses de animais pode herdá-los. Vamos começar com os campos públicos:

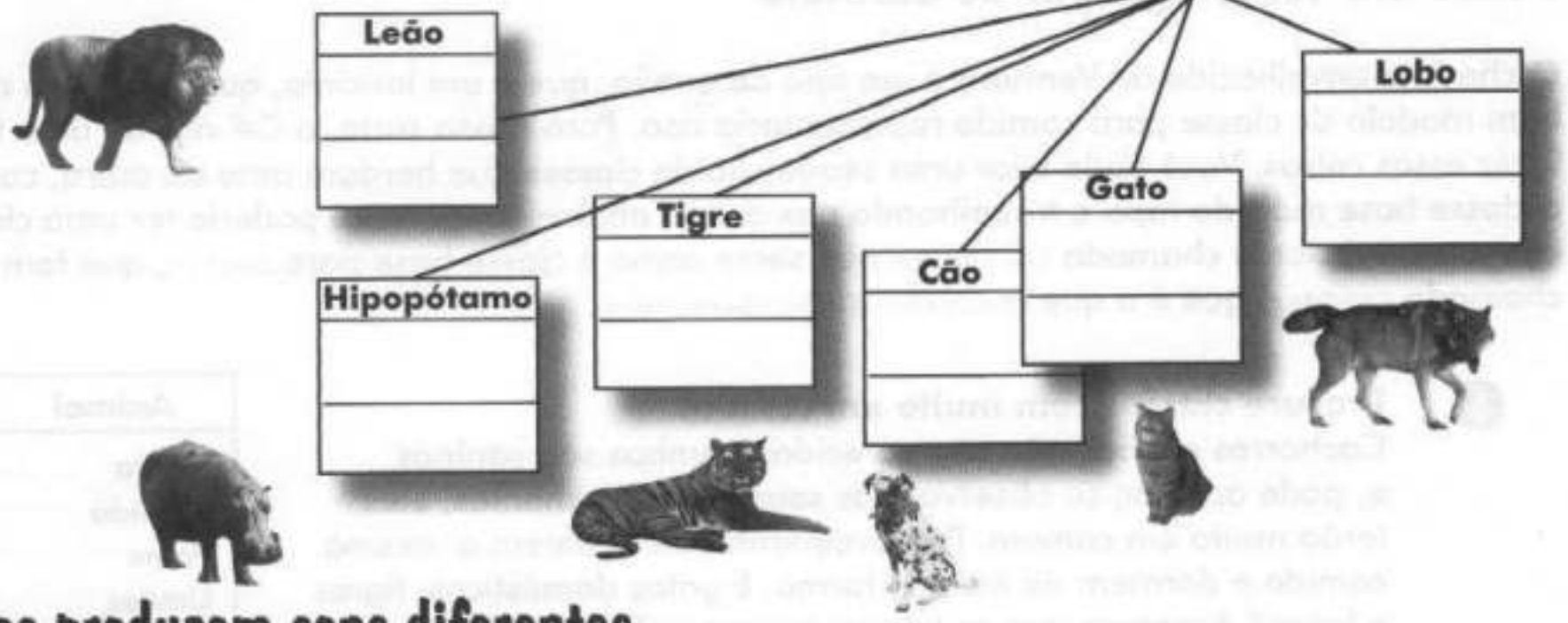
2 **Faça uma classe base para dar aos animais tudo que eles têm em comum**
Os campos, propriedades e métodos na classe base darão a todos os animais que herdam dela um estado e comportamento comum. Todos eles são animais, então faz sentido chamar a classe base Animal.

- ★ Figura: uma imagem que você pode colocar numa PictureBox.
- ★ Comida: o tipo de comida que esse animal come. Neste momento, só pode haver dois valores: carne ou grama.
- ★ Fome: um int representa o nível de fome de um animal. Ele muda dependendo de quando (e quanto) o animal come.
- ★ Limites: uma referência a uma classe que armazena a altura, comprimento e local do espaço onde o animal irá ficar.
- ★ Localização: as coordenadas X e Y onde o animal está.

E a classe Animal possui quatro métodos que os animais podem herdar:

- ★ ProduzirSom(): um método para permitir que o animal emita um som.
- ★ Comer(): comportamento para quando o animal encontra sua comida preferida.
- ★ Dormir(): um método que faz o animal deitar e tirar uma soneca.
- ★ Perambular(): os animais gostam de andar pelo espaço deles no zoológico.

Criar uma classe base quer dizer fazer escolhas. Você poderia ter decidido usar uma classe ZooOccupant (ocupante do zoológico) que define os custos da comida e manutenção, ou uma classe Attraction (atração) com métodos para como os animais entretém os visitantes do zoológico. Mas achamos que Animal faz mais sentido aqui. Você concorda?



Animais diferentes produzem sons diferentes

Leões rugem, cachorros latem e, até onde nós sabemos, hipopótamos não fazem nenhum som. Cada uma das classes que herdam de `Animal` terá um método `MakeNoise()`, mas cada um desses métodos funcionará de forma diferente e terão código diferente. Quando uma subclasse muda o comportamento de um dos métodos que herdou, dizemos que ela sobrepõe (override) o método.

Só porque uma propriedade ou um método está na classe base Animal, isso não quer dizer que toda subclasse terá que usá-los da mesma forma... nem mesmo usá-los de qualquer forma!

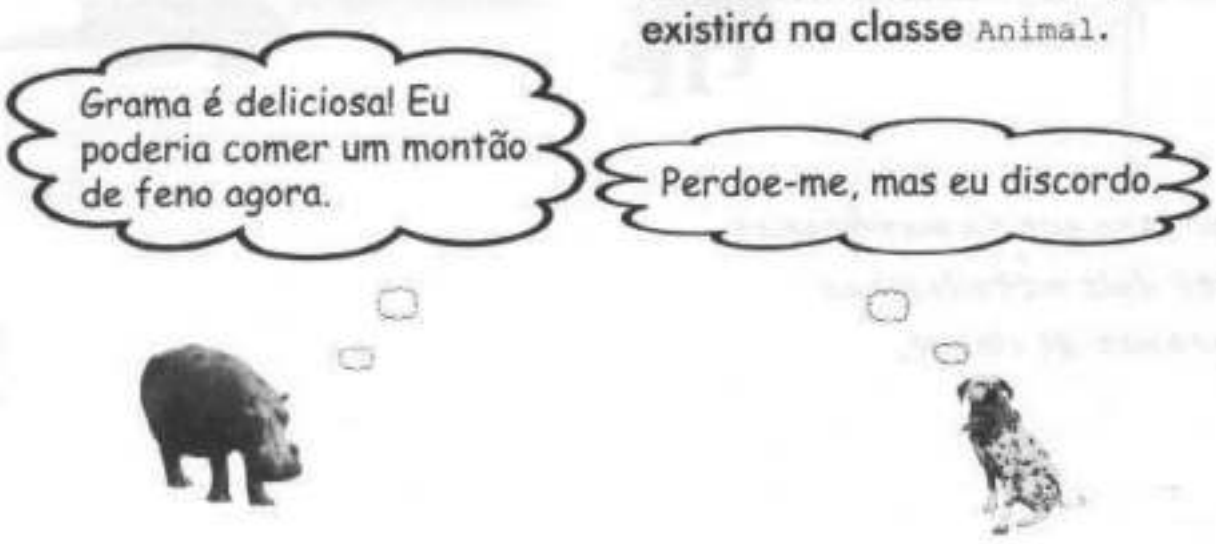
Pense o que você precisa substituir

Quando uma subclasse altera o comportamento de um método que ela herdou, chamamos de sobreposição (overriding). Todo animal precisa comer. Mas um cachorro pode comer alguns pedaços de carne, enquanto um hipopótamo come enormes quantidades de grama. Então como seria o código para este comportamento? Tanto o cachorro quanto o hipopótamo substituiriam o método `Eat()`. O método do hipopótamo o faria consumir, digamos, 10 quilogramas de feno cada vez que for chamado. O método `Eat()` do cachorro, por outro lado, reduziria a quantidade de comida para uma lata de 400 gramas de comida canina.

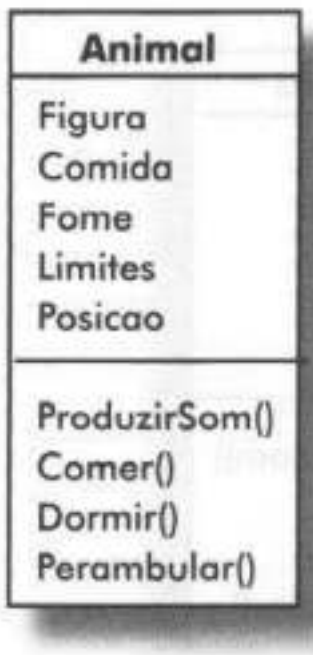
Então quando você tem uma subclasse que herda de uma classe base, ela deve herdar todos os comportamentos base... mas você pode modificá-los na subclasse para que eles não funcionem exatamente da mesma forma. É para isso que serve a sobreposição.

3 **Descubra o que cada animal faz que a classe `Animal` faz diferente – ou não faz**

O que cada tipo de animal faz que todos os outros não? Cachorros comem comida canina, então o método `Eat()` do cachorro precisará substituir o método `Animal.Eat()`. Hipopótamos nadam, então ele terá um método `Swim()` que não existirá na classe `Animal`.



aviso:** não dê comida* **aos programadores



Poder do Cérebro

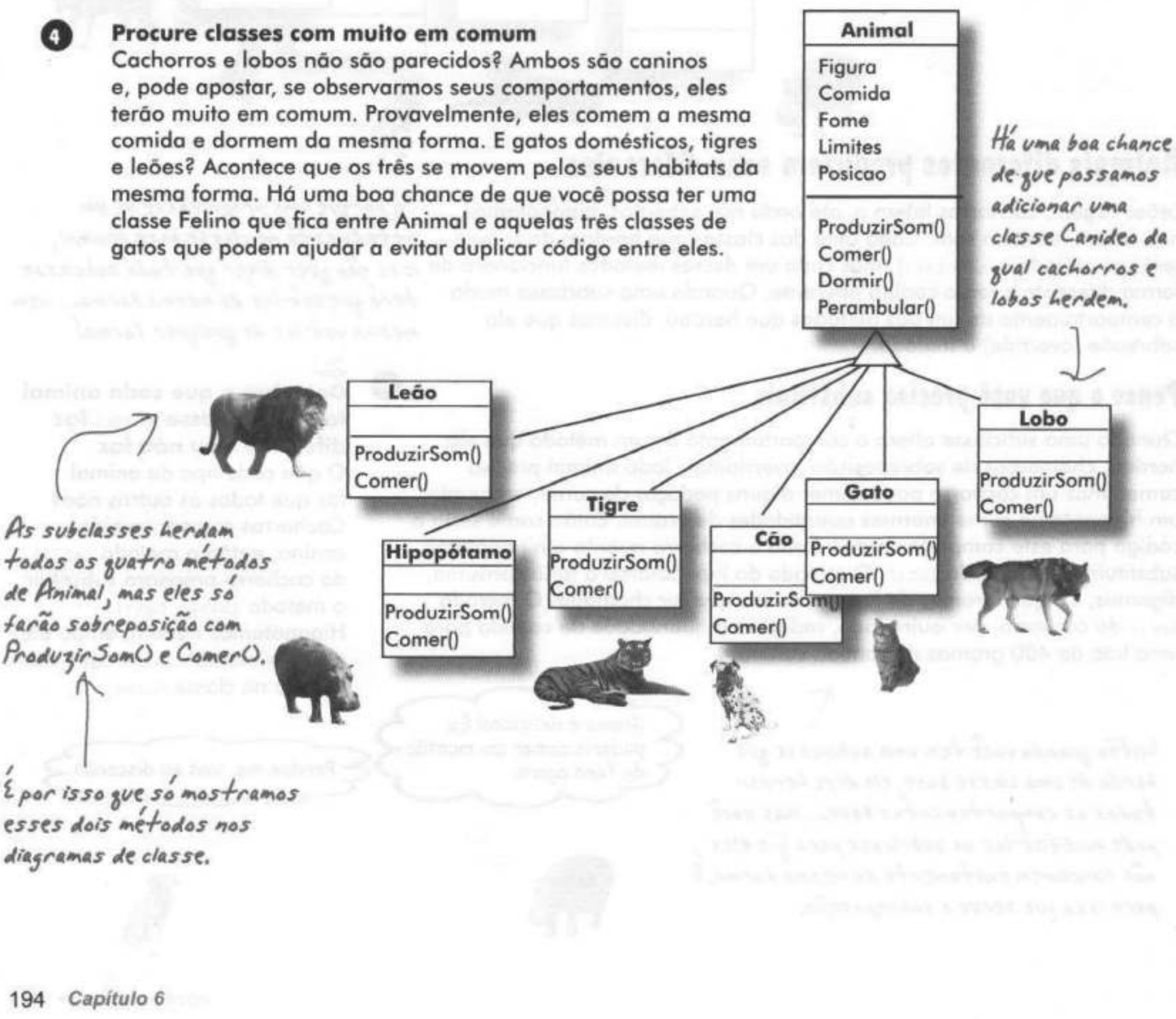
Já sabemos que alguns animais substituirão os métodos `ProduzirSom()` e `Comida()`. Quais animais substituirão `Dormir()` ou `Perambular()`? Algum deles fará isso? E as propriedades – quais animais substituirão algumas propriedades?

Pense em como agrupar os animais

O cheddar envelhecido de Vermont é um tipo de queijo, que é um laticínio, que é um tipo de comida. Um bom modelo de classe para comida representaria isso. Para nossa sorte, o C# nos dá uma forma fácil de fazer essas coisas. Você pode criar uma seqüência de classes que herdam uma da outra, começando com a classe base mais do topo e trabalhando nas outras abaixo. Então você poderia ter uma classe `Comida`, com uma subclasse chamada `Laticinio` que serve como a classe base para `Queijo`, que tem uma subclasse chamada `Cheddar`, que é o que `ChedarEnvelhecidaVermont`.

4 **Procure classes com muito em comum**

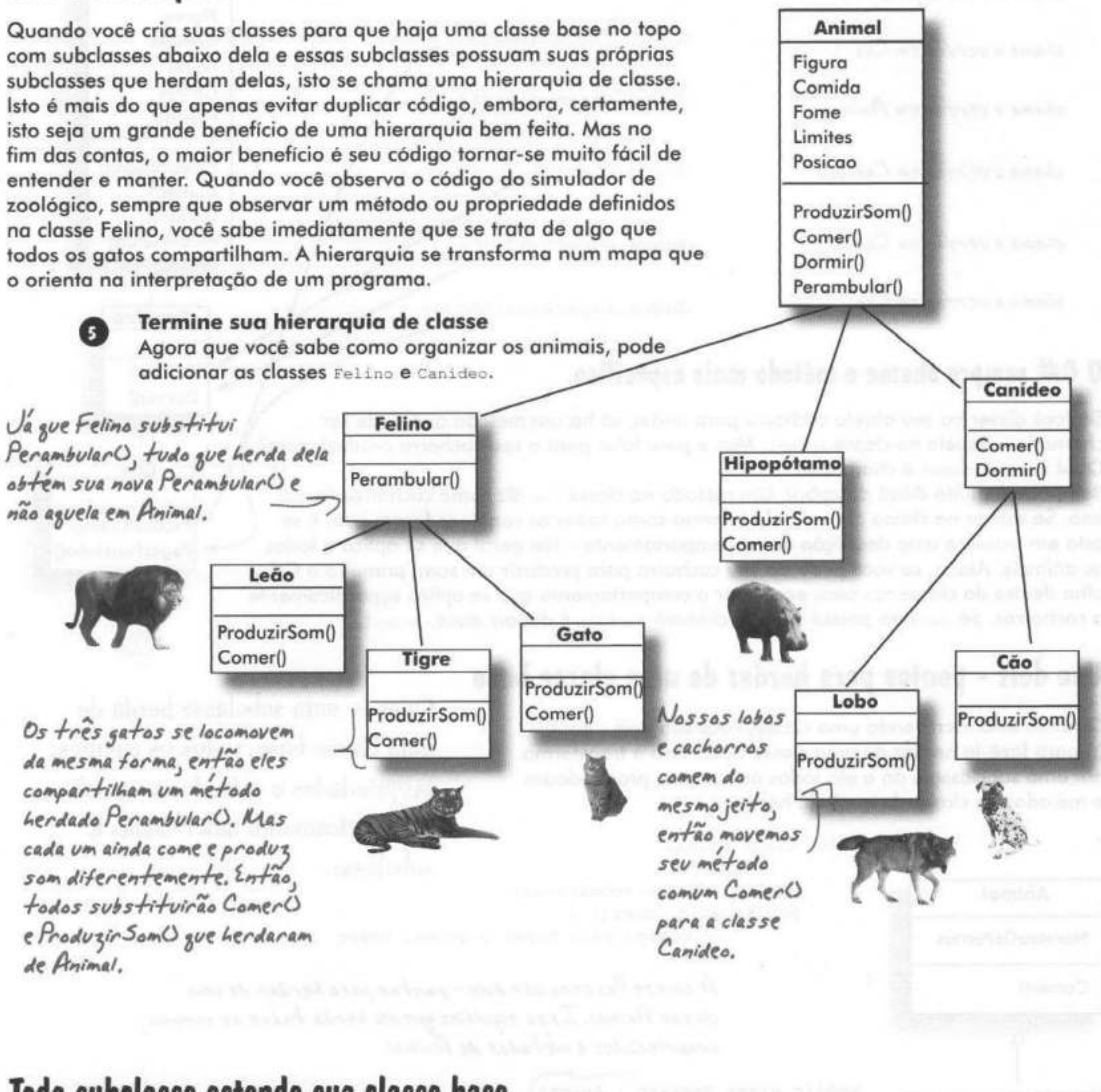
Cachorros e lobos não são parecidos? Ambos são caninos e, pode apostar, se observarmos seus comportamentos, eles terão muito em comum. Provavelmente, eles comem a mesma comida e dormem da mesma forma. E gatos domésticos, tigres e leões? Acontece que os três se movem pelos seus habitats da mesma forma. Há uma boa chance de que você possa ter uma classe Felino que fica entre Animal e aquelas três classes de gatos que podem ajudar a evitar duplicar código entre eles.



Crie a hierarquia de classe

Quando você cria suas classes para que haja uma classe base no topo com subclasses abaixo dela e essas subclasses possuam suas próprias subclasses que herdam delas, isto se chama uma hierarquia de classe. Isto é mais do que apenas evitar duplicar código, embora, certamente, isto seja um grande benefício de uma hierarquia bem feita. Mas no fim das contas, o maior benefício é seu código tornar-se muito fácil de entender e manter. Quando você observa o código do simulador de zoológico, sempre que observar um método ou propriedade definidos na classe Felino, você sabe imediatamente que se trata de algo que todos os gatos compartilham. A hierarquia se transforma num mapa que o orienta na interpretação de um programa.

5 **Termine sua hierarquia de classe**
Agora que você sabe como organizar os animais, pode adicionar as classes `Felino` e `Canideo`.



Toda subclasse estende sua classe base

Você não está limitado aos métodos que uma subclasse herda da classe base... mas você já sabia disso! Afinal, você tem feito suas próprias classes. Quando acrescenta herança a uma classe, você pega a classe construída e estende-a, adicionando todos os campos, propriedades e métodos na classe base. Então, se você queria adicionar um método `BuscarPauzinho()` ao cachorro, isto é perfeitamente normal. Ele não vai herdar ou sobrepor nada – só o cachorro terá esse método e ele não vai acabar em `Lobo`, `Canideo`, `Animal`, `Hipopotamo` ou em nenhuma outra classe.

hi-e-rar-qui-a, substantivo.
um arranjo ou classificação no qual grupos ou coisas são classificadas uma como superior à outra. *O presidente da Dynamco subiu do almoxarifado ao topo da hierarquia empresarial.*

```
Cao duque = new Cao();
duque.ProduzirSom();
duque.Perambular();
duque.Comer()
duque.Dormir();
duque.PegarPauzinho();
```

cria um novo objeto Cao

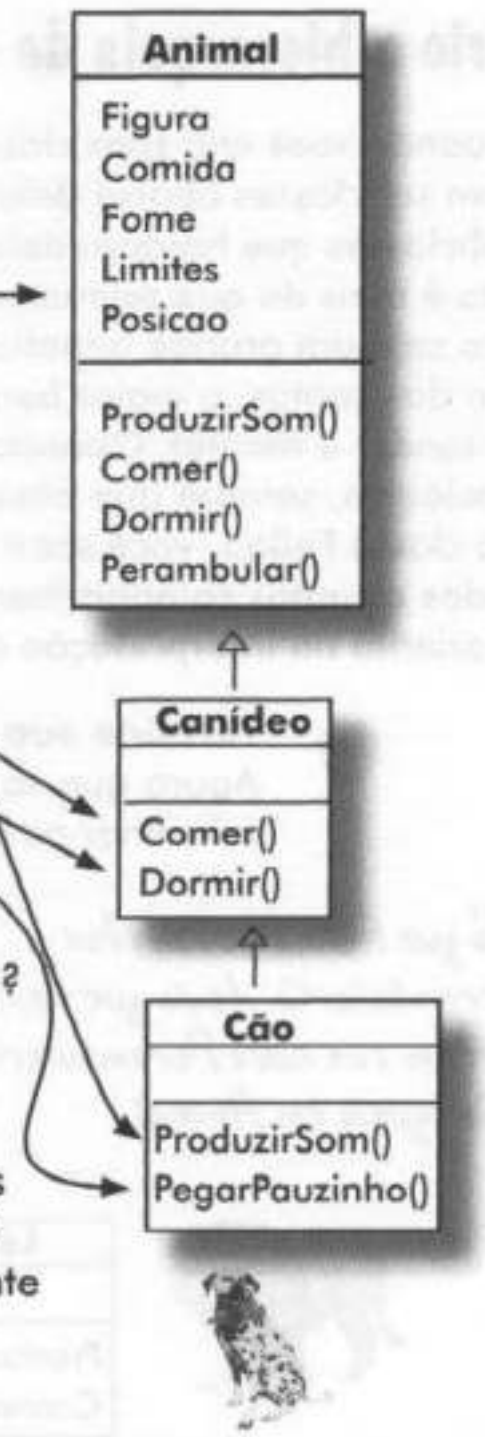
chama a versão em Cao

chama a versão em Animal

chama a versão em Canideo

chama a versão em Canideo

chama a versão em Cao



O C# sempre chama o método mais específico

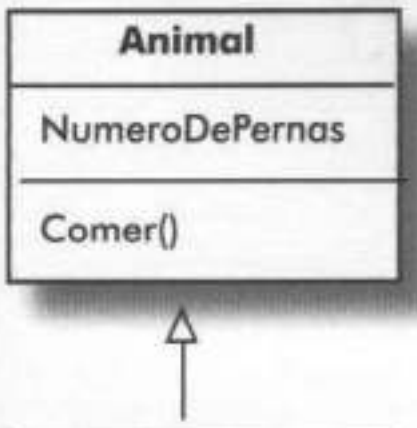
Se você disser ao seu objeto cachorro para andar, só há um método que pode ser chamado – aquele na classe `Animal`. Mas e para falar para o seu cachorro produzir som? Qual `ProduzirSom()` é chamado?

Bem, não é muito difícil descobrir. Um método na classe `Cao` diz como cachorros fazem isso. Se estiver na classe `Canideo`, ela informa como todos os canídeos fazem isso. E se está em `Animal` é uma descrição desse comportamento – tão geral que se aplica a todos os animais. Assim, se você pedir ao seu cachorro para produzir um som, primeiro o C# olha dentro da classe `Cao` para encontrar o comportamento que se aplica especificamente a cachorros. Se `Cao` não possui um, ele checará `Canideo` e depois disso, `Animal`.

Use dois - pontos para herdar de uma classe base

Quando está escrevendo uma classe, você usa dois - pontos (:) para fazê-la herdar de uma classe base. Isso a transforma em uma subclasse e dá a ela todos os campos, propriedades e métodos da classe da qual ela herda.

Quando uma subclasse herda de uma classe base, todos os campos, propriedades e métodos nesta são automaticamente adicionadas à subclasse.




```
public class Animal
{
        public int NumeroDePernas;
        public void Comer() {
                // codigo para fazer o animal comer
        }
}
```

A classe Passaro usa dois - pontos para herdar de uma classe Animal. Isso significa que ela herda todos os campos, propriedades e métodos de Animal.

```
public class Passaro : Animal
{
        public double EnvergaduraDasAsas;
        public void Voar() {
                // codigo para fazer o passaro voar
        }
}
```

Você estende uma classe adicionando dois-pontos no final da sua linha de declaração, seguindo-o com a classe base da qual herdar.

```
public button1_Click(object sender, EventArgs e) {
        Passaro tweety = new Passaro();
        tweety.EnvergaduraDasAsas = 7.5;
        tweety.Voar();
        tweety.NumeroDePernas= 2;
        tweety.Comer();
}
```

Tweety (Piu-Piu) é uma instância de Passaro, então ela tem os métodos e campos de Passaro como sempre.

Já que a classe Passaro herda de Animal, toda instância de Passaro também possui campos e métodos definidos na classe Animal.

não existem Perguntas Idiotas

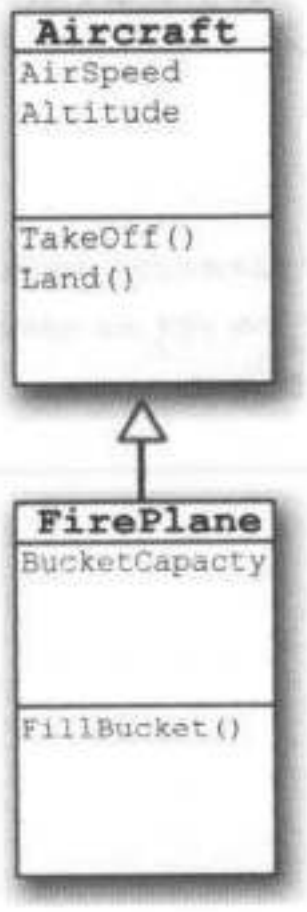
P: Por que a seta aponta para cima, da subclasse para a classe base? Em vez disso, o diagrama não ficaria melhor com uma seta apontando para baixo?

R: Pode parecer melhor, mas não seria tão certo. Quando você estabelece que uma classe herde de outra, você cria essa relação dentro da subclasse – a classe base continua a mesma. E isso faz sentido quando você pensa da perspectiva da classe base. Seu comportamento continua totalmente inalterado quando você adiciona uma classe que herda dela.

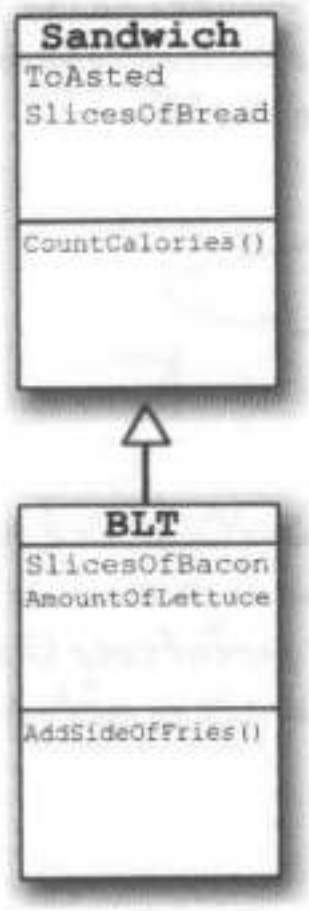
A classe base nem sequer sabe dessa nova classe que herdou dela. Seus métodos, campos e propriedades continuam totalmente intactos. Mas a subclasse certamente muda seu comportamento. Toda instância da subclasse automaticamente recebe todas as propriedades, campos e métodos da classe base e isso acontece só com o acréscimo dos dois pontos. É por isso que você desenha a seta em seu diagrama: para que ele seja parte da subclasse e aponte para a classe base da qual ela herda.

Aponte seu lápis

Dê uma olhada nestes modelos de classe e declarações e circule os comandos que não funcionarão.



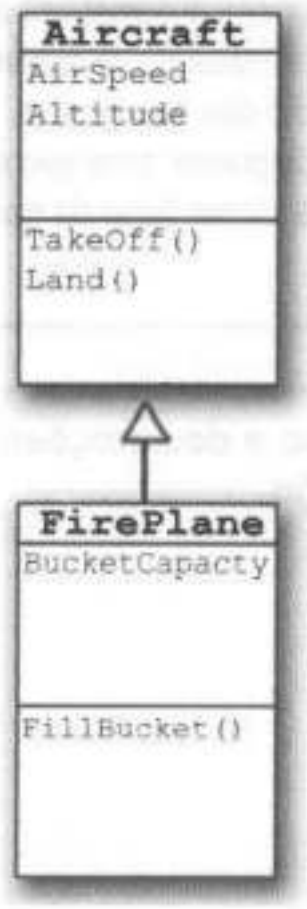

```
public class Aircraft {
  public double AirSpeed;
  public double Altitude;
  public void TakeOff() { ... };
  public void Land() { ... };
}
public class FirePlane : Aircraft {
  public double BucketCapacity;
  public void FillBucket() { ... };
}
public void FireFightingMission() {
  FirePlane myFirePlane = new FirePlane();
  new FirePlane.BucketCapacity = 500;
  Aircraft.Altitude = 0;
  myFirePlane.TakeOff();
  myFirePlane.AirSpeed = 192.5;
  myFirePlane.FillBucket();
  Aircraft.Land();
}
```




```
public class Sandwich {
  public boolean Toasted;
  public int SlicesOfBread;
  public int CountCalories() { ... }
}
public class BLT : Sandwich {
  public int SlicesOfBacon;
  public int AmountOfLettuce;
  public int AddSideOfFries() { ... }
}
public BLT OrderMyBLT() {
  BLT mySandwich = new BLT();
  BLT.Toasted = true;
  Sandwich.SlicesOfBread = 3;
  mySandwich.AddSideOfFries();
  mySandwich.SlicesOfBacon += 5;
  MessageBox.Show("My sandwich has "
    + mySandwich.CountCalories + "calories".);
  return mySandwich;
}
```

Aponte seu lápis
Solução

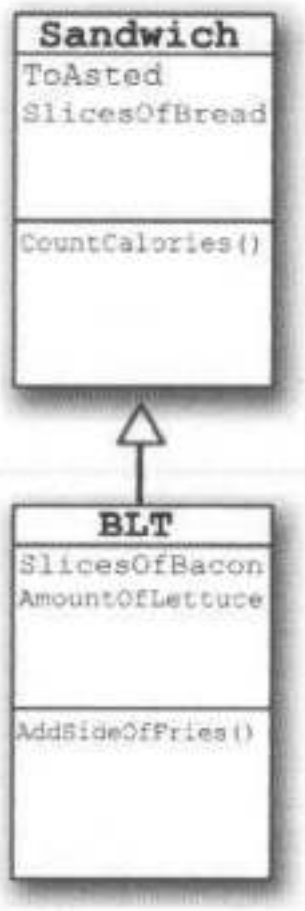
Dê uma olhada nestes modelos de classe e declarações e circule os comandos que não funcionarão.




```
public class Aircraft {
  public double AirSpeed;
  public double Altitude;
  public void TakeOff() { ... };
  public void Land() { ... };
}
public class FirePlane : Aircraft {
  public double BucketCapacity;
  public void FillBucket() { ... };
}
public void FireFightingMission() {
  FirePlane myFirePlane = new FirePlane();
  new FirePlane.BucketCapacity = 500;
  Aircraft.Altitude = 0;
  myFirePlane.TakeOff();
  myFirePlane.AirSpeed = 192.5;
  myFirePlane.FillBucket();
  Aircraft.Land();
}
```

Não é assim que você usa a palavra-chave new.

Estes comandos usam o nome da classe, em vez do nome da instância, myFirePlane.



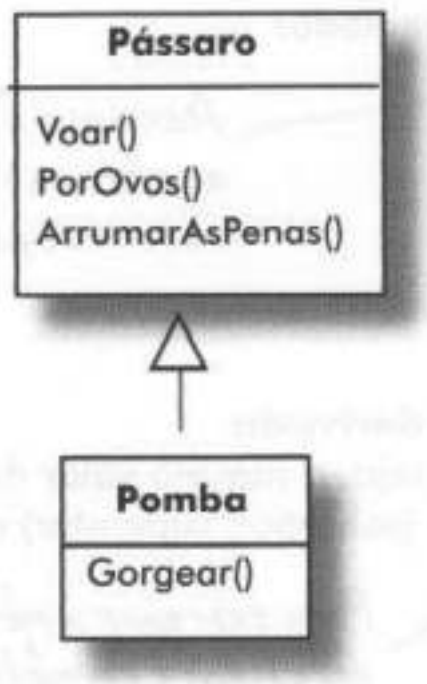

```
public class Sandwich {
  public boolean Toasted;
  public int SlicesOfBread;
  public int CountCalories() { ... }
}
public class BLT : Sandwich {
  public int SlicesOfBacon;
  public int AmountOfLettuce;
  public int AddSideOfFries() { ... }
}
public BLT OrderMyBLT() {
  BLT mySandwich = new BLT();
  BLT.Toasted = true;
  Sandwich.SlicesOfBread = 3;
  mySandwich.AddSideOfFries();
  mySandwich.SlicesOfBacon += 5;
  MessageBox.Show("My sandwich has "
    + mySandwich.CountCalories + "calories".);
  return mySandwich;
}
```

Estas propriedades são da instância, mas os comandos estão tentando chamá-las incorretamente usando os nomes de classe.

CountCalories é um método, mas esse comando não inclui os parênteses () depois da chamada ao método.

Sabemos que herança adiciona os campos, propriedades e métodos da classe base à subclasse...

Herança é simples quando sua subclasse precisa herdar todos os métodos, propriedades e campos da classe base.



Pomba é uma subclasse de Passaro, então qualquer campo e método em Passaro é automaticamente parte de Pomba também.

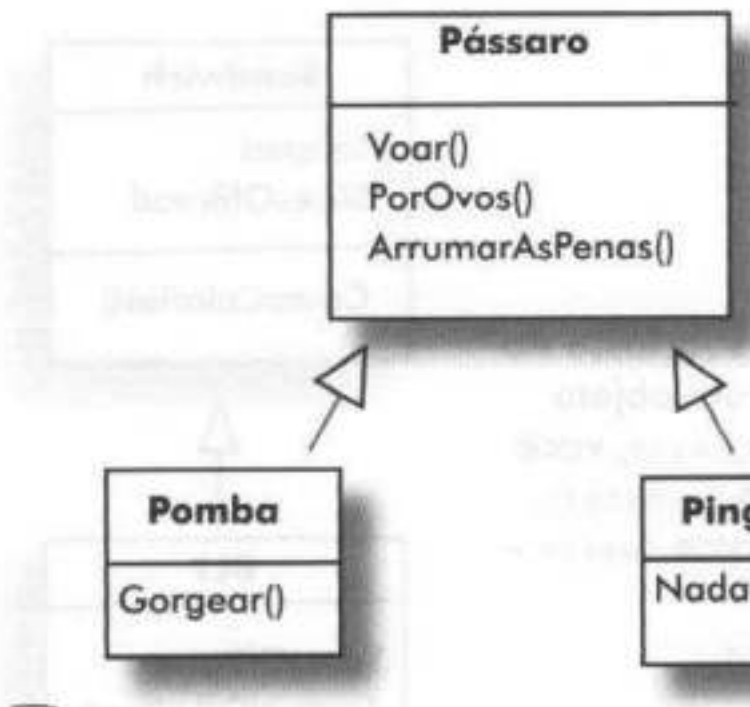
```
public class Passaro {
      public void Voar() {
            // Aqui voce fara seu passaro voar
      }
      public void PorOvos() { ... };
      public void ArrumarAsPenas() { ... };
}
public class Pomba : Passaro {
      public void Gorgear() { ... }
}
public class Pinguim : Passaro {
      public void Nadar() { ... }
}
public void SimuladorDePassaros() {
      Pomba Enriqueta = new Pomba();
      Pinguim Isidoro = new Pinguim();
   Enriqueta.Voar();
   Enriqueta. Gorgear();
      Isidoro.Fly();
}
```

Isidoro é uma instância de Pinguim. Já que ele herdou o método Voar(), não há nada que o impeça de voar.

Tanto Pomba quanto Pinguim herdam de Passaro, então os dois recebem os métodos Voar(), PorOvos() e ArrumarAsPenas().

...mas alguns pássaros não voam!

O que você faz se sua classe base possui um método que sua subclasse precisa modificar?



Pombas voam, botam ovos e ajeitam suas penas. Logo, não existirão problemas se classe Pomba herdar de Passaro.

Os objetos Pinguim não deveriam poder voar! Mas se a classe Pinguim herda de Passaro, então você terá pingüins voando por todos os lados. O que faremos então?

Poder do Cérebro

Se esse fosse seu código de Simulador de Pássaros, o que você faria para evitar que os pinguins voassem?

Uma subclasse pode substituir métodos para alterar ou mudar completamente os métodos herdados

Às vezes, você tem uma subclasse que deveria herdar a maioria dos comportamentos da classe base, mas não todos. Quando você quer alterar os comportamentos que uma classe herdou, você pode fazer sobreposição dos métodos.

1 **Adicione a palavra-chave "virtual" ao método na classe base**
Uma subclasse só pode substituir um método se for marcada com a palavra-chave virtual, que diz ao C# permitir para a subclasse sobrepor-se os métodos.

```
public class Passaro {
        public virtual void Voar() {
                // codigo para fazer o passaro
voar
        }
}
```

Adicionar a palavra-chave virtual ao método Voar() diz ao C# que uma subclasse pode substituí-lo.

2 **Adicione um método com o mesmo nome na classe derivada**
Você precisará ter exatamente a mesma assinatura – ou seja, o mesmo valor de retorno e parâmetros – e precisará usar a palavra-chave **override** (sobrepor, suplantar) na declaração.

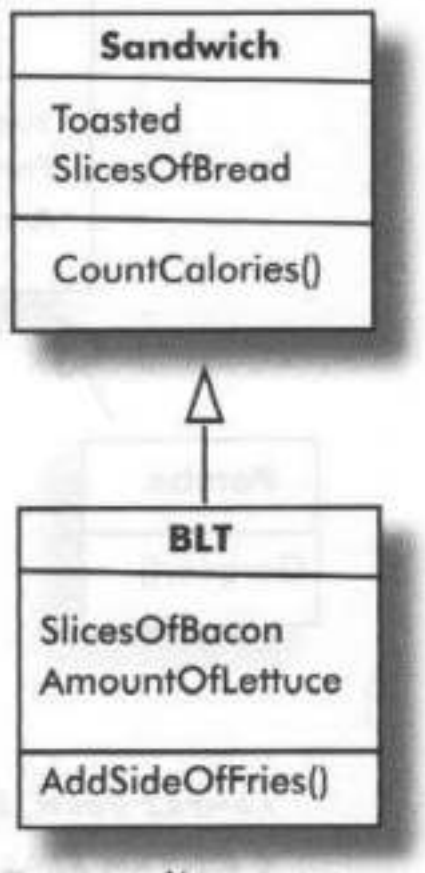
```
public class Pinguim : Passaro {
        public override void Voar() {
        MessageBox.Show("Pinguins não voam!")
    }
}
```

Para sobrepor o método Voar(), acrescente um método idêntico à subclasse e use a palavra-chave override.

Quando faz sobreposição, seu novo método precisa ter exatamente a mesma assinatura daquele na classe base ao qual está sendo sobreposto. Neste caso, isso quer dizer que ele precisa chamar-se Voar, retornar void e não ter nenhum parâmetro.

Use a palavra-chave override para adicionar um método à sua subclasse que substitua um herdado. Antes que você possa sobrepor um método, precisa marcá-lo como virtual na classe base.

Em qualquer lugar onde se pode usar uma classe base, pode-se utilizar uma de suas subclasses



Uma das coisas mais úteis que você pode fazer com herança é usar uma subclasse no lugar da classe base da qual ela herda. Se o seu método `Receita()` pega um objeto `Queijo` e você tem uma classe `CheddarEnvelhecidoDeVermont` que herda de `Queijo`, você pode passar uma instância de CheddarEnvelhecidoDeVermont ao método `Receita()`. Mas este só terá acesso aos campos, propriedades e métodos parte da classe `Queijo` – ele não tem acesso a nada específico da subclasse.

1 Digamos que temos um método para analisar objetos sandwich:

```
public void SandwichAnalyzer(Sandwich specimen) {
  int calories = specimen.CountCalories();
  UpdateDietPlan(calories);
  PerformBreadCalculations(specimen.SlicesOfBread, specimen.Toasted);
}
```

2 Você pode passar um sandwich ao método – mas você poderia passar também um BLT (sanduíche de bacon, alface e tomate). Já que um BLT é um tipo de sanduíche; nós o criamos de forma que ele herde da classe Sandwich.

```
public button1_Click(object sender, EventArgs e) {
  BLT myBLT = new BLT();
  SandwichAnalyzer(myBLT);
}
```

3. Você sempre pode se mover para baixo no diagrama de classe – uma variável de referência pode sempre ser igualada a uma instância de uma de suas subclasses. Mas você não pode se mover para cima no diagrama.

```
public button2_Click(object sender, EventArgs e) {
  Sandwich mySandwich = new Sandwich();
  BLT myBLT = new BLT();
  Sandwich someRandomSandwich = myBLT;
  BLT anotherBLT = mySandwich; // <- ISSO NÃO VAI COMPILAR!
}
```

Falaremos mais sobre isso no próximo capítulo!

Você pode atribuir myBLT a qualquer variável Sandwich porque um BLT é um tipo de sanduíche.

Mas você não pode atribuir mySandwich a uma variável BLT porque nem todo sanduíche é um BLT! É por isso que esta última linha vai dar erro.

Exercício

Mensagens Misturadas

```
a = 6;
b = 5;
a = 5;
```

56
11
65

Listamos um pequeno programa C# abaixo. Um bloco do programa está faltando! Seu desafio é associar o bloco candidato de código (à esquerda) com o resultado – na caixa de mensagem que o programa exibe – visto se o bloco fosse inserido. Nem todas as linhas do resultado serão usadas e algumas linhas do resultado podem ser usadas mais de uma vez. Desenhe linhas conectando os blocos candidatos de código com seu resultado correspondente.

Instruções:

1. Preencha as quatro lacunas no código.

2. Relacione os candidatos a código ao resultado.

```
public class A {
  public int ivar = 7;
  public _____ string m1() {
    return "A's m1, ";
  }
  public string m2() {
    return "A's m2, ";
  }
  public _____ string m3() {
    return "A's m3, ";
  }
}
public class B : A {
  public _____ string m1() {
    return "B's m1, ";
  }
}
```

```
public class C : B {
  public _____ string m3() {
    return "C's m3, " + (ivar + 6);
  }
}
public class Mixed5 {
  public static void Main(string[] args) {
    A a = new A();
    B b = new B();
    C c = new C();
    A a2 = new C();
    string q = "";

    [          ]

    System.Windows.Forms.MessageBox.Show(q);
  }
}
```

o código candidato vai aqui (três linhas)

Aqui está o ponto de partida para o programa – ele não mostra um formulário, apenas uma caixa de mensagem.

candidatos a código:

```
q += b.m1();
q += c.m2();
q += a.m3();
```

```
q += c.m1();
q += c.m2();
q += c.m3();
```

```
q += a.m1();
q += b.m2();
q += c.m3();
```

```
q += a2.m1();
q += a2.m2();
q += a2.m3();
```

resultado:

```
A's m1, A's m2, C's m3, 6
B's m1, A's m2, A's m3,
A's m1, B's m2, A's m3,
B's m1, A's m2, C's m3, 13
B's m1, C's m2, A's m3,
B's m1, A's m2, C's m3, 6
A's m1, A's m2, C's m3, 13
```

(Não apenas digite isso no IDE – você aprenderá muito mais se fizer isso no papel!)

Jogos de Verão

Sua tarefa é pegar os trechos na piscina e colocá-los nas linhas em branco do código. Você pode usar o mesmo trecho mais de uma vez e você pode não usar todos eles. Seu objetivo é fazer um conjunto de classes que compilem e executem juntas num programa. Não se engane – este exercício é mais difícil do que parece.

```
public class Rowboat ..... {
  public ..... rowTheBoat() {
    return "stroke natasha";
  }
}
```

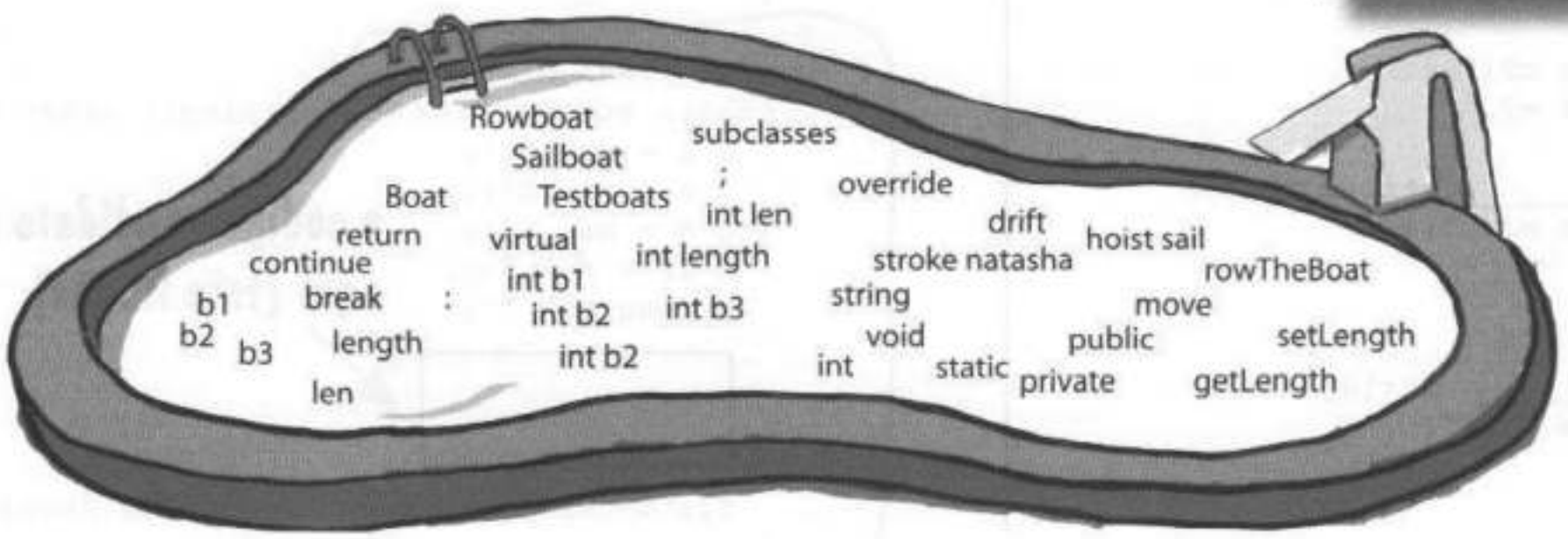
```
public class ..... {
  private int ..... ;
  ..... void ..... ( ..... ) {
    length = len;
  }
  public int getLength() {
    ..... ;
  }
  public ..... move() {
    return " ..... ";
  }
}
```

```
public class TestBoats {
 ..... Main(){
  ..... xyz = "";
  ..... b1 = new Boat();
    Sailboat b2 = new ..... ();
    Rowboat ..... = new Rowboat();
    b2.setLength(32);
    xyz = b1. ..... ();
    xyz += b3. ..... ();
    xyz += ..... .move();
    System.Windows.Forms.MessageBox.Show(xyz);
  }
}
```

Dica: Este é o ponto de entrada do programa.

```
public class ..... : Boat {
  public ..... () {
    return " ..... ";
  }
}
```

RESULTADO:



Solução do Exercício

Mensagens Misturadas

```
a = 6;  ┐ 56
b = 5;  ┐ 11
a = 5;  ┘ 65
```

```
public class A {
  public virtual string m1() {
...
  public virtual string m3() {
}
```

```
public class B : A {
  public override string m1() {
...
public class C : B {
  public override string m3() {
```

```
q += b.m1();
q += c.m2();
q += a.m3();
```

```
A's m1, A's m2, C's m3, 6
```

```
q += c.m1();
q += c.m2();
q += c.m3();
```

```
B's m1, A's m2, A's m3,
```

```
q += a.m1();
q += b.m2();
q += c.m3();
```

```
A's m1, B's m2, A's m3,
```

```
q += a2.m1();
q += a2.m2();
q += a2.m3();
```

```
B's m1, A's m2, C's m3, 13
B's m1, C's m2, A's m3,
B's m1, A's m2, C's m3, 6
A's m1, A's m2, C's m3, 13
```

---

não existem Perguntas Idiotas

P: A respeito do ponto de entrada mostrado nos Jogos de Verão – isso quer dizer que eu posso ter um programa sem um formulário Form1?

R: Sim. Quando você cria um novo projeto Windows Application, o IDE cria todos os arquivos desse projeto para você, inclusive o Program.cs (que contém uma classe estática com um ponto de entrada) e o Form1.cs (que contém um formulário vazio chamado Form1). Tente isso: em vez de criar um novo projeto Windows Application, crie um projeto vazio selecionando "Empty Project" (projeto vazio). Acrescente um arquivo de classe nele no Navegador de Solução e digite nele toda a resposta dos Jogos de Verão. Já que o seu programa usa uma caixa de mensagem, você precisa adicionar uma referência clicando com o botão direito em "References" no Navegador de Solução, selecionando "Add Reference" (adicionar referência) escolhendo System. Windows.Forms na aba .NET (isso é outra coisa que o IDE faz para você automaticamente quando você cria um Windows Application). Finalmente, selecione "Properties" (propriedades) no menu Project e escolha o tipo de projeto "Windows Application". Agora o execute... você verá os resultados! Parabéns, você acabou de criar um programa C# desde o princípio.

P: Eu posso herdar da classe que contém o ponto de entrada?

R: Sim. O ponto de entrada deve ser um método estático, mas esse método não tem que estar numa classe estática. (Lembre-se, a palavra-chave static não significa que a classe não pode ser instanciada, mas sim que seus métodos ficam disponíveis quando o programa inicia. Então no programa dos Jogos de Verão, você pode chamar TestBoats. Main() de qualquer outro método sem declarar uma variável de referência ou instanciar um objeto usando um comando new.)

P: Eu ainda não entendi por que chamamos alguns métodos de "virtuais" – eles parecem reais para mim!

R: O nome "virtual" tem a ver com como o .NET lida com os métodos virtuais nos bastidores. Ele usa algo chamando a tabela de método virtual (ou vtable). Essa é uma tabela que o .NET usa para controlar os métodos herdados substituídos. Não se preocupe – você não precisa saber como ela funciona para usar métodos virtuais!

P: O que você quis dizer com só poder se mover para cima no diagrama de classe, mas não para baixo?

R: Quando você tem um diagrama com uma classe acima de outra, a classe mais acima é mais abstrata do a mais abaixo. Classes mais específicas ou concretas (como Camiseta ou Carro, por exemplo) herdam daquelas mais abstratas (como Roupas ou Veículo, no exemplo). Quando você pensa assim, é fácil notar como. Se tudo de que você precisa é qualquer veículo, um carro, uma van ou uma moto podem servir. Mas se você precisar de um carro, uma moto não será útil para você. Herança funciona exatamente da mesma forma. Se você tem um método com Veículo como um parâmetro e se a classe Motocicleta herda da classe Veículo, você pode passar uma instância de Motocicleta ao método. Mas se o método usa Motocicleta como um parâmetro, você não pode passar qualquer objeto de Veículo, porque ele pode ser uma instância de Van, por exemplo. Caso contrário, o C# não saberia o que fazer quando um método tentar acessar a propriedade, digamos, Guidões.

Você pode sempre passar uma instância de uma subclasse a qualquer método cujos parâmetros esperam a classe da qual essa subclasse herda.

Solução dos Jogos de Verão

```
public class Rowboat ..Boat.. {
  public .string.. rowTheBoat() {
    return "stroke natasha";
  }
}

public class ..Boat.. {
  private int .length.. ;
  .public.. void setLength ( .int len. ) {
    length = len;
}
  public int getLength() {
    .return length.;
  public.virtual string..move() {
    return " ..drift.. ";
  }
}
```

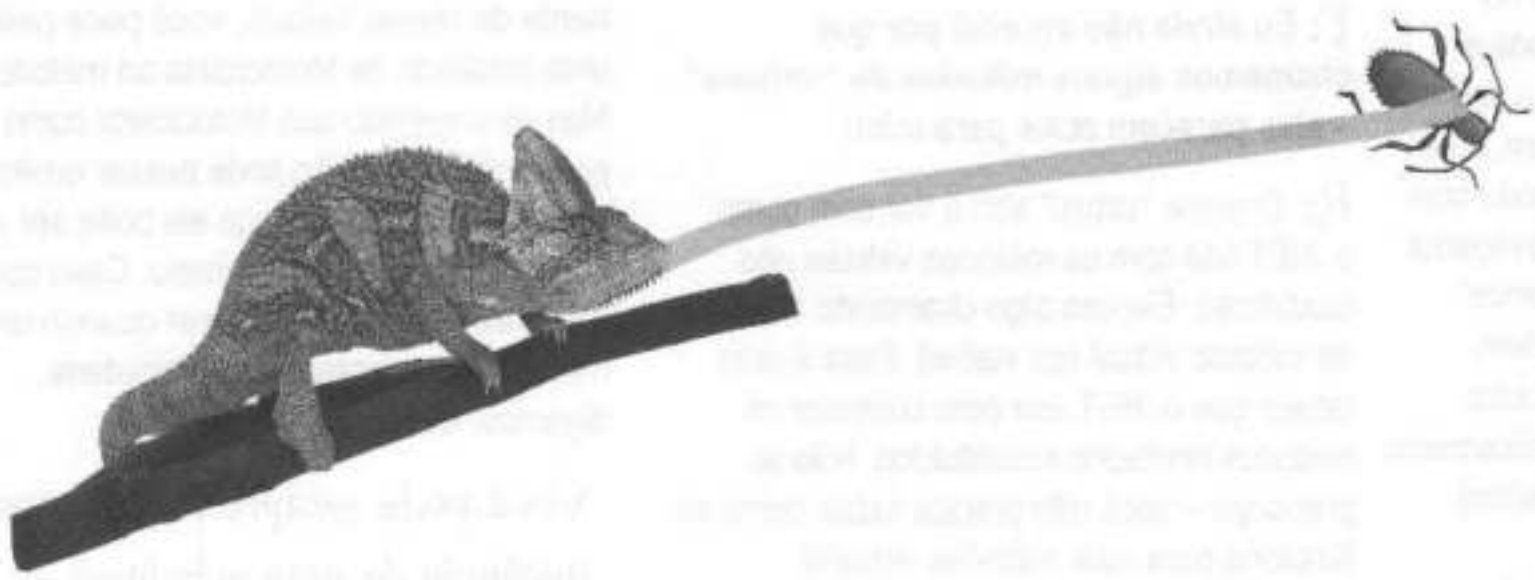
```
public class TestBoats {
  .public static void Main(){
  ..string.. xyz = "";
  ...Boat.. b1 = new Boat();
    Sailboat b2 = new Sailboat ();
    Rowboat ...b3.... = new Rowboat();
    b2.setLength(32);
    xyz = b1. .move... ();
    xyz += b3. .move.. ();
    xyz += ...b2... .move();
    System.Windows.Forms.MessageBox.Show(xyz);
  }
}

public class Sailboat : Boat {
  public ...override string move... () {
    return " hoist sail ";
  }
}
```

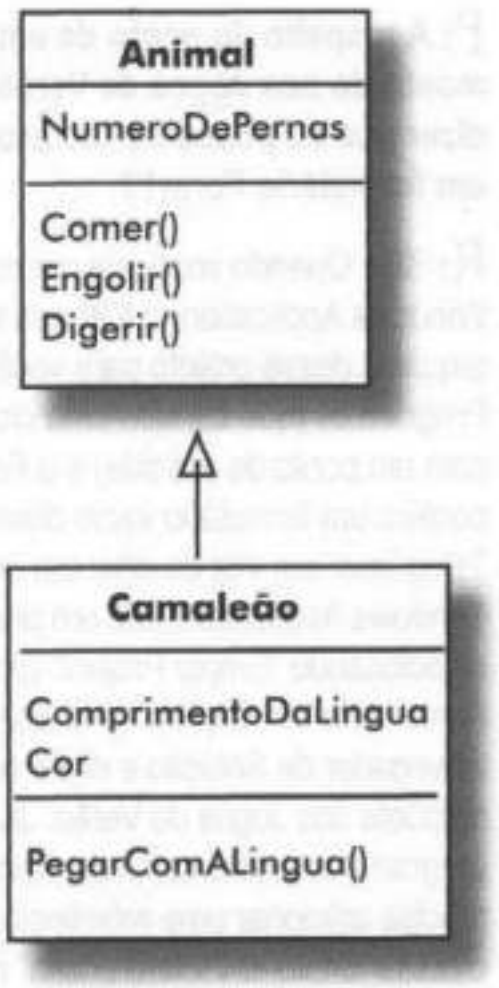
Uma subclasse pode acessar sua classe base usando a palavra-chave base

Mesmo quando você substitui um método ou propriedade em sua classe base, às vezes, você ainda quer acessá-lo. Felizmente, podemos usar base, que nos permite acessar qualquer método na classe base.

1. Todos os animais comem, então a classe `Animal` possui um método `Comer()` que tem um objeto `Comida` como seu parâmetro.

```
public class Animal {
        public virtual void Comer(Comida bocado) {
                Engolir(bocado);
                Digerir();
        }
}
```





2 **Camaleões comem pegando comida com suas línguas. Então a classe `Camaleao` herda de `Animal` mas substitui `Comer()`.**

```
public class Camaleao : Animal {
      public override void Comer(Comida bocado) {
            PegarComALingua(bocado);
            Engolir(bocado);
            Digerir();
      }
}
```

O camaleão precisa engolir e digerir a comida, assim como qualquer outro animal. Então realmente precisamos duplicar esse código?

3 **Em vez de duplicar o código, podemos usar a palavra-chave base para chamar o método substituído. Agora temos acesso a ambas versões, antiga e nova, de `Comer()`.**

```
public class Camaleao : Animal {
      public override void Comer(bocado) {
            PegarComALingua(bocado);
                  base.Comer(bocado);
      }
}
```

Esta linha chama o método Comer() na classe base da qual Camaleao herdou.

Quando uma classe base tem um construtor, sua subclasse precisa de um também

Se sua classe tem um construtor, então qualquer classe que herde dela **deve chamar esse construtor**. O construtor da subclasse pode ter parâmetros diferentes do construtor da classe base.

Adicione esta linha extra no final da declaração do construtor de sua subclasse para informar ao C# que ele precisa chamar o construtor da classe base toda vez que a subclasse for instanciada.

Aqui está o construtor para a subclasse.

```
public class Subclass : BaseClass {
  public Subclass(parameter list)
     : base(the base class's parameter list) {
     // primeiro o construtor da classe base sera executado
     // depois qualquer comando aqui tambem
  }
}
```

O construtor da classe base é executado antes do construtor da subclasse

Mas não precisa aceitar nossa palavra – veja você mesmo!

Faça isso!

1 **Crie uma classe base com um construtor que mostre uma caixa de mensagem**
Adicione um botão a um formulário que instancie essa classe base e a caixa deve ser mostrada:

```
public class MyBaseClass {
  public MyBaseClass(string baseClassNeedsThis) {
    MessageBox.Show("This is the base class: " + baseClassNeedsThis);
  }
}
```

Este é um parâmetro que o construtor da classe base precisa.

2 **Tente adicionar uma subclasse, mas não chame o construtor.**
Então adicione um botão a um formulário que instancie essa subclasse que mostra outra mensagem:

Fique atento a esse erro ligeiramente enigmático. Ele significa que sua subclasse não herdou o construtor corretamente.

Selecione Build >> Build Solution no IDE. Surgirá um erro ao usar esse código.

```
public class MySubclass : MyBaseClass{
  public MySubclass(string baseClassNeedsThis, int anotherValue) {
    MessageBox.Show("This is the subclass: " + baseClassNeedsThis
      + " and " + anotherValue);
  }
}
```

X	1	Nenhuma sobrecarga para o método 'MyBaseClass' tem '0' argumentos.

3 **Conserte o erro fazendo o novo construtor chamar aquele da classe base**
Instancie a subclasse e veja em que ordem as duas caixas de mensagem aparecem!

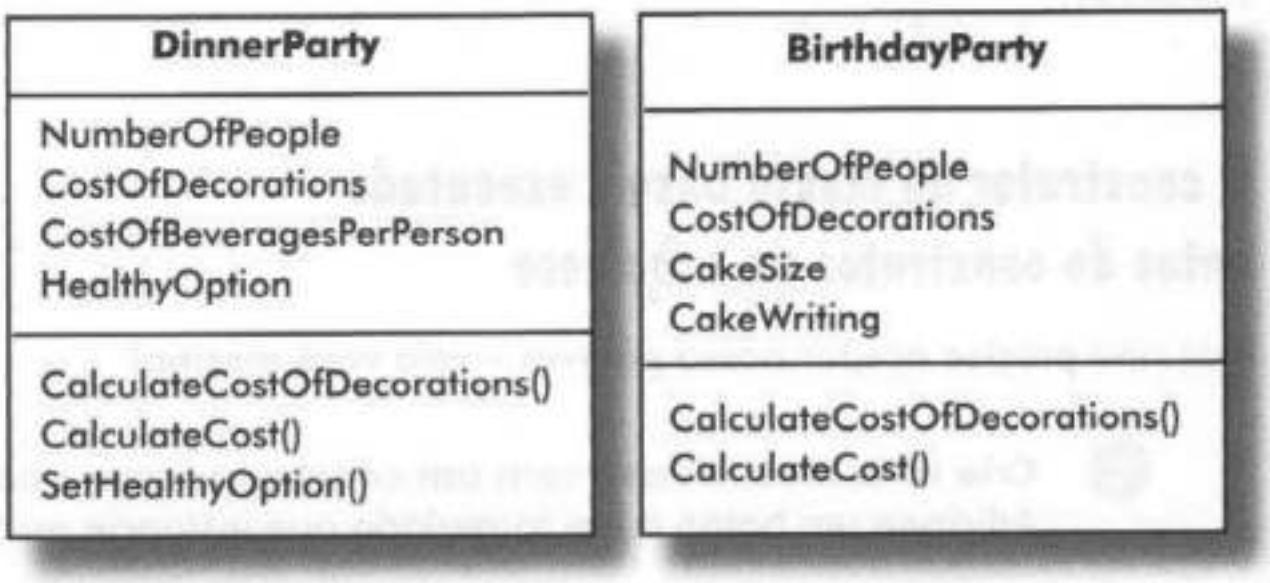
```
public class MySubclass : MyBaseClass{
  public MySubclass(string baseClassNeedsThis, int anotherValue)
    : base(baseClassNeedsThis)
  // o restante da subclasse é o mesmo
```

É assim que enviamos à classe base o parâmetro que seu construtor precisa.

Adicione esta linha para mandar o C# chamar o construtor na classe base. Ele possui uma lista de parâmetro que mostra o que é passado ao construtor da classe base. O erro será eliminado e você pode fazer um botão para ver as duas caixas de mensagem aparecerem!

Agora você está pronto para terminar o trabalho para a Catarina!

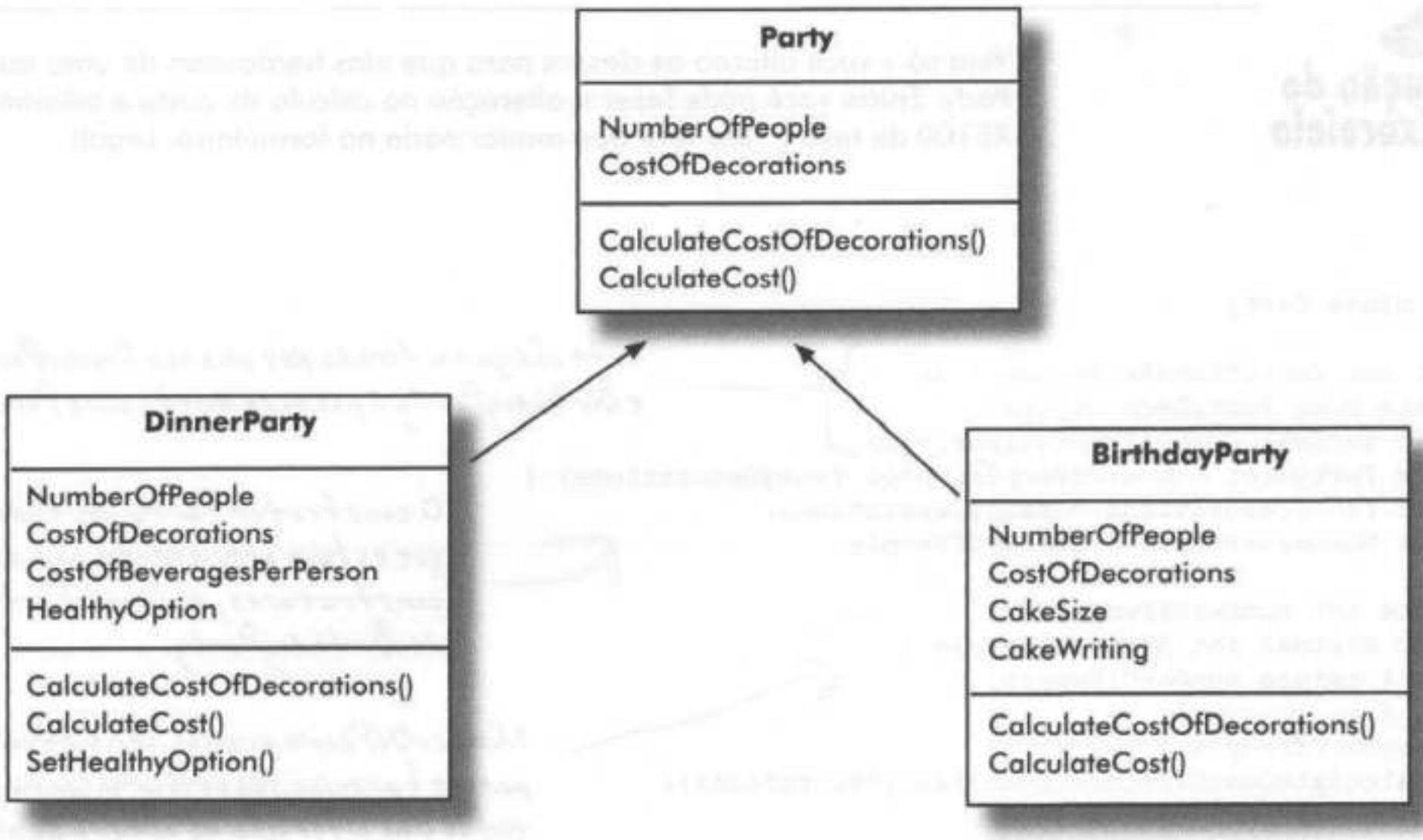
Quando deixou a Catarina pela última vez, tinha adicionado as festas de aniversário no programa dela. Ela precisa que você cobre um extra de R$100 para festas com mais de 12 pessoas. A princípio, você teria de escrever o mesmo código duas vezes, um para cada classe. Agora que você sabe como usar herança, pode fazê-las herdar da mesma classe base que contém todo o código compartilhado, escrevendo-o apenas uma vez.



Exercício

Se fizermos tudo certo, devemos poder alterar as duas classes sem mudar nada no formulário!

1 **Vamos criar o novo modelo da classe**
Ainda teremos as mesmas classes `DinnerParty` e `BirthdayParty`, mas agora elas herdarão de uma única classe `Party` (festa, grupo). Precisamos delas para ter exatamente os mesmos métodos, propriedades e campos. Assim, não teremos que fazer nenhuma alteração no formulário. Mas alguns desses métodos, propriedades e campos serão movidos para dentro da classe base `Party` e talvez tenhamos que substituir alguns deles.



2 **Crie a classe base `Party`**

Crie a classe `Party` – certifique-se de que seja pública. Você precisará olhar bem atentamente as propriedades e métodos no diagrama de classe e descobrir o que precisa mover para fora de `DinnerParty` e de `BirthdayParty` e para dentro de `Party`.

- ★ Mova as propriedades `NumberOfPeople` e `CostOfDecorations` para dentro de `Party` para que elas sejam compatíveis tanto com `DinnerParty` quanto com `BirthdayParty`.
- ★ Faça o mesmo para os métodos `CalculateCostOfDecorations()` e `CalculateCost()`. Se estes precisarem de quaisquer campos privados, você precisará movê-los também. (Lembre-se, subclasses apenas herdam campos públicos – quando você mover um campo privado para `Party`, as classes `DinnerParty` e `BirthdayParty` não terão acesso a ele).
- ★ Você também precisa de um construtor. Dê uma olhada melhor nos construtores de `BirthdayParty` e `DinnerParty` – tudo que eles tenham em comum deve ser movido para o construtor de `Party`.
- ★ Agora acrescente os R$100 de taxa para festas com mais de 12 pessoas. Afinal, é por isso que estamos fazendo tudo! É comum a ambas as festas, de aniversário e de jantar, então pertence à `Party`.

3 **Faça `DinnerParty` herdar de `Party`**

Agora que `Party` faz muitas coisas que `DinnerParty` faz, você pode eliminar a intersecção e manter apenas a parte de `DinnerParty`, exclusiva dos jantares.

- ★ Certifique-se de que o construtor esteja funcionando corretamente. Ele faz algo que o construtor de `Party` não faz? Se fizer, mantenha isso e deixe todo o restante para o construtor da classe base.
- ★ Qualquer lógica ligada à seleção da opção saudável deve ficar em `DinnerParty`.
- ★ Você precisará substituir, pelo menos, um método porque ele faz algo específico para jantares.

4 **Faça `BirthdayParty` herdar de `Party`**

Faça a mesma coisa para `BirthdayParty` – deixe qualquer coisa não específica para a classe base e mantenha a funcionalidade específica de aniversários em `BirthdayParty`.

- ★ O que o construtor `BirthdayParty` precisa fazer que não seja parte de `Party`?
- ★ Você precisará lidar com o custo do bolo dentro de `BirthdayParty`. Isso usa um método e uma propriedade, logo, você precisará fazer sobreposição.
- ★ Sim, você pode sobrepor uma propriedade! É exatamente como substituir um método. Quando você estabelece o valor de base.NumberOfPeople, ele chama o acessador `set` da propriedade na classe base. Você precisará usar a palavra-chave base tanto para recuperar quanto para atribuir o valor.

Solução do Exercício

Veja só – você alterou as classes para que elas herdassem de uma mesma, Party. Então você pôde fazer a alteração no cálculo do custo e adicionar R$100 de taxa e não teve que mudar nada no formulário. Legal!

```
public class Party
{
  const int CostOfFoodPerPerson = 25;
  private bool fancyDecorations;
  public decimal CostOfDecorations = 0;
  public Party(int numberOfPeople, bool fancyDecorations) {
    this.fancyDecorations = fancyDecorations;
    this.NumberOfPeople = numberOfPeople;
  }
  private int numberOfPeople;
  public virtual int NumberOfPeople {
    get { return numberOfPeople; }
    set {
      numberOfPeople = value;
      CalculateCostOfDecorations(fancyDecorations);
    }
  }
  public void CalculateCostOfDecorations(bool fancy) {
    fancyDecorations = fancy;
    if (fancy)
      CostOfDecorations = (NumberOfPeople * 15.00M) + 50M;
    else
      CostOfDecorations = (NumberOfPeople * 7.50M) + 30M;
  }
  public virtual decimal CalculateCost() {
    // Each person costs $25 for food plus cost of beverages
    decimal TotalCost = CostOfDecorations + (CostOfFoodPerPerson *  NumberOfPeople);
    if (NumberOfPeople > 12)
    {
      TotalCost += 100;
    }
    return TotalCost;
  }
}
public class BirthdayParty : Party {
  public int CakeSize;
  public BirthdayParty(int numberOfPeople, bool fancyDecorations, string cakeWriting)
    : base(numberOfPeople, fancyDecorations) {
    CalculateCakeSize();
    this.CakeWriting = cakeWriting;
    CalculateCostOfDecorations(fancyDecorations);
  }
  private void CalculateCakeSize() {
    if (NumberOfPeople <= 4)
      CakeSize = 8;
    else
      CakeSize = 16;
  }
```

Este código foi tirado das classes DinnerParty e BirthdayParty e passado direto para Party.

O construtor Party faz tudo que estava previamente nos dois construtores, de DinnerParty e decBirthdayParty.

NumberOfPeople precisa ser virtual porque é preciso substituí-la (para que então uma alteração no número de pessoas calcule um novo tamanho de bolo).

O cálculo da decoração é idêntico na festa de aniversário e no jantar, então faz sentido movê-lo para Party. Dessa forma, nenhuma parte do código é duplicada em múltiplas classes.

O cálculo do custo precisa ser um método virtual porque birthdayparty o substitui (e também o estende chamando o método da classe base).

O construtor conta com a classe base para fazer a maioria do trabalho. Depois chama CalculateCakeSize(), assim como o antigo construtor de BirthdayParty fazia.

O método CalculateCakeSize() é específico de festas de aniversário, então ele continua na classe BirthdayParty.

```
private string cakeWriting = "";
public string CakeWriting {
  get { return this.cakeWriting; }
  set {
    int maxLength;
    if (CakeSize == 8)
      maxLength = 16;
    else
      maxLength = 40;
    if (value.Length > maxLength) {
      MessageBox.Show("Too many letters for a " + CakeSize + " inch cake");
      if (maxLength > this.cakeWriting.Length)
        maxLength = this.cakeWriting.Length;
      this.cakeWriting = cakeWriting.Substring(0, maxLength);
    } else
      this.cakeWriting = value;
  }
}
public override decimal CalculateCost() {
  decimal CakeCost;
  if (CakeSize == 8)
    CakeCost = 40M + CakeWriting.Length * .25M;
  else
    CakeCost = 75M + CakeWriting.Length * .25M;
  return base.CalculateCost() + CakeCost;
}
public override int NumberOfPeople {
  get { return base.NumberOfPeople; }
  set {
    base.NumberOfPeople = value;
    CalculateCakeSize();
    this.CakeWriting = cakeWriting;
  }
}
}
public class DinnerParty : Party
{
  public decimal CostOfBeveragesPerPerson;
  public DinnerParty(int numberOfPeople, bool healthyOption,
                     bool fancyDecorations)
    : base(numberOfPeople, fancyDecorations) {
    SetHealthyOption(healthyOption);
    CalculateCostOfDecorations(fancyDecorations);
  }
  public void SetHealthyOption(bool healthyOption) {
    if (healthyOption)
      CostOfBeveragesPerPerson = 5.00M;
    else
      CostOfBeveragesPerPerson = 20.00M;
  }
  public decimal CalculateCost(bool healthyOption) {
    decimal totalCost = base.CalculateCost()
                      + (CostOfBeveragesPerPerson * NumberOfPeople);
    if (healthyOption)
      return totalCost * .95M;
    else
      return totalCost;
  }
}
```

A propriedade CakeWriting permanece intocada na classe BirthdayParty também.

CalculateCost() também precisa ser sobreposto, porque ele precisa primeiro calcular o custo do bolo e então adicioná-lo ao custo calculado no método CalculateCost() da classe Party.

A propriedade NumberOfPeople tem que ser sobreposta àquela em Party porque o acessador set precisa recalcular o tamanho do bolo. O acessador set precisa chamar base.NumberOfPeople para que o acessador set em Party também seja executado.

Aqui está a última classe na solução para a Catarina. (Não há nenhuma alteração no código do formulário).

Esse campo público só é usado em jantares, não em aniversários. Por isto, ele fica nessa classe.

Para fazer o que a antiga classe DinnerParty fazia, o novo construtor chama o construtor de Party e então chama SetHealthyOption().

O método SetHealthyOption() continua exatamente o mesmo.

DinnerParty tem que sobrepor CalculateCost() - ele usa o custo base de Party, adiciona o custo das bebidas e acrescenta o desconto da opção saudável.

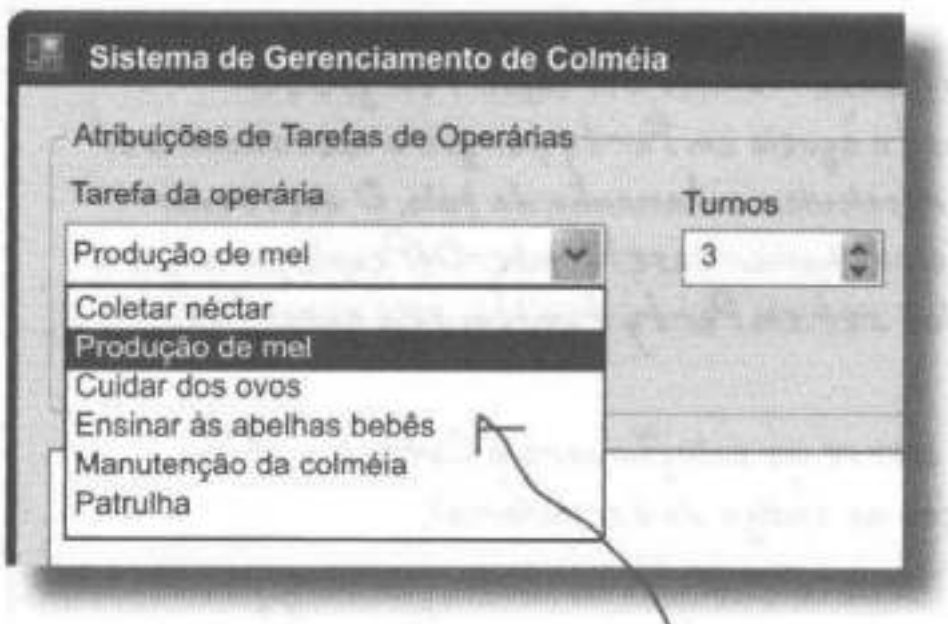
O programa é perfeito. Está muito mais fácil fazer meus negócios agora - muito obrigada!

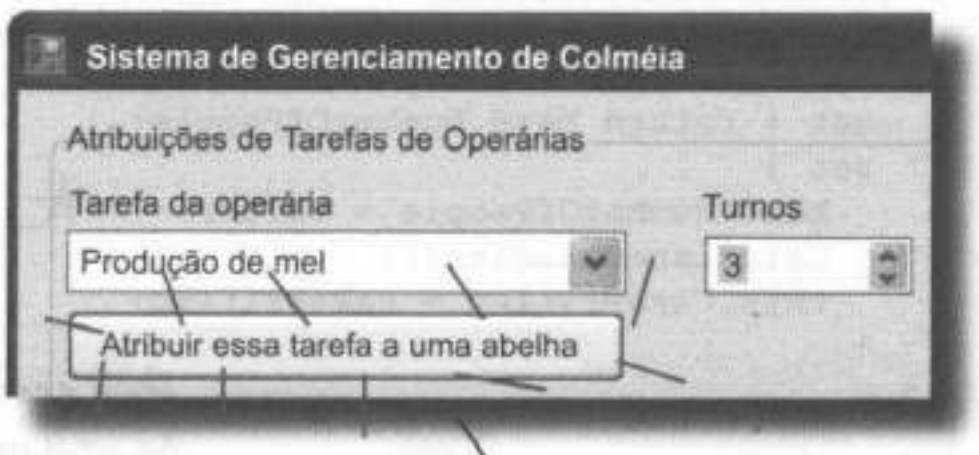
Construa um sistema de gerenciamento de colméia

Uma abelha rainha precisa de sua ajuda! A colméia dela está fora de controle e ela precisa de um programa para administrá-la. Ela tem uma colméia cheia de operárias e muitas tarefas a fazer por toda parte. Mas em algum momento ela perdeu o controle do que cada abelha está fazendo e a noção se ela tem ou não força operária para fazer as tarefas que precisam ser feitas.
Cabe a você construir um sistema de gerenciamento de colméia para ajudá-la a controlar suas operárias. É assim que ele vai funcionar:

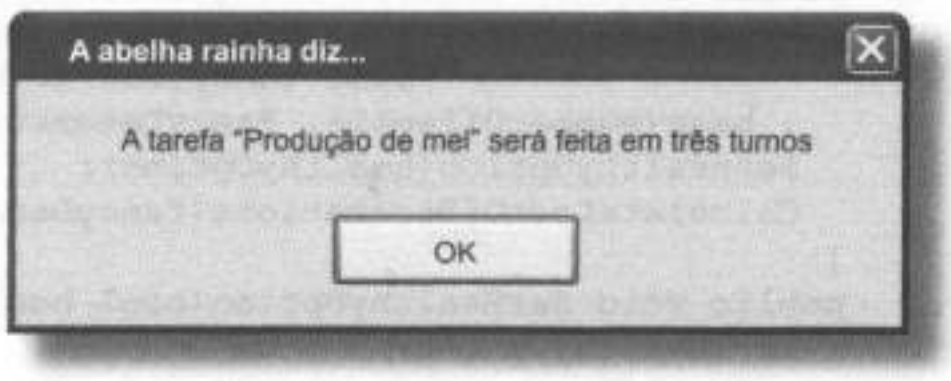
1 **A rainha atribui tarefas a suas operárias**
Há seis possíveis tarefas que as operárias podem fazer. Algumas sabem como coletar néctar e produzir mel, outras podem manter a colméia ou patrulhar procurando por inimigos. Algumas delas podem fazer qualquer tarefa na colméia. Então o seu programa dará a ela uma forma de atribuir um trabalho para qualquer abelha que esteja disponível para fazê-lo.

As abelhas trabalham em turnos e a maioria das tarefas exige mais de um turno. Então a rainha digita o número de turnos que o trabalho terá e clica no botão Atribuir essa tarefa.





A lista expansível mostra todas as seis tarefas que as operárias podem fazer. A rainha sabe qual trabalho precisa ser feito e ela não liga para qual abelha fará o quê. Então ela apenas seleciona a tarefa a ser feita – o programa deve descobrir se há uma operária disponível para fazê-la e atribuirá o trabalho a ela.



Se houver uma abelha disponível para fazer o trabalho, o programa deve atribuí-lo a ela e avisar à rainha que o trabalho será feito.

2 **Quando as tarefas estiverem todas atribuídas, é hora de trabalhar**
Quando a rainha acabar de atribuir o serviço, ela mandará as abelhas trabalharem no próximo turno clicando no botão "Próximo turno". O programa então gera um relatório de turno que informa quais abelhas trabalharam naquele turno, quais tarefas fizeram, e mais, quantos turnos elas trabalharão em cada tarefa.

Primeiro você construirá o sistema básico

Este projeto é dividido em duas partes. A primeira é uma espécie de revisão, onde você criará o sistema básico para gerenciar a colméia. Ela tem duas classes, Queen (rainha) e Worker (operária). Você fará o formulário para o sistema e vai associá-lo a duas classes. Também garantirá que as classes sejam bem encapsuladas para que sejam fáceis de alterar quando passarmos para a segunda parte.

Às vezes, diagramas de classe listam campos e tipos privados

Queen
private workers: Worker[] private shiftNumber: int
AssignWork() WorkTheNextShift()

O programa possui um único objeto Queen que gerencia o trabalho em andamento.

- ★ Queen usa uma matriz de objetos Worker para controlar cada uma das operárias e se a elas foram atribuídas tarefas. Ela fica armazenada num campo privado Worker[ ] chamado worker.
- ★ O formulário chama o método AssignWork() (atribui tarefa), passando uma string para o trabalho que precisa ser feito e um int para o número de turnos. Ele retornará true se encontrar uma operária para atribuir um trabalho, ou false se não conseguir encontrar uma operária para o trabalho.
- ★ O botão "Próximo turno" do formulário chama WorkTheNextShift() (efetivar próximo turno), que manda as operárias trabalharem e retorna um relatório de turno para ser exibido. Ele manda cada objeto Worker trabalhar um turno, e então checa o status da operária para que possa adicionar uma linha ao relatório.

CurrentJob e ShiftsLeft são propriedades apenas-leitura.

Worker
CurrentJob: string ShiftsLeft: int private jobsICanDo: string[] private shiftsToWork: int private shiftsWorked: int
DoThisJob() WorkOneShift()

Queen usa uma matriz de objetos Worker para controlar todas as operárias e quais trabalhos elas estão fazendo.

- ★ CurrentJob (tarefa atual) é uma propriedade apenas-leitura que informa ao objeto Queen qual tarefa a operária está fazendo ("Patrulha", "Manutenção da Colméia", etc.). Se a operária não está fazendo nenhuma, a propriedade retornará uma string vazia.
- ★ O objeto Queen tenta atribuir uma tarefa a uma operária usando seu método DoThisJob() (faça a seguinte tarefa). Se essa operária ainda não estiver fazendo uma tarefa e se essa for uma que ela saiba como fazer, então ela aceitará a atribuição e o método retorna true. Caso contrário, ele retorna false.
- ★ Quando o método WorkOneShift() (trabalhar um turno) é chamado, a operária trabalha um turno. Ele controla quantos turnos faltam na tarefa atual. Se o trabalho terminar, então ele reinicia o registro de seu trabalho atual para uma string vazia para que a abelha possa assumir sua próxima função.

String.IsNullOrEmpty()

Já que cada abelha armazena seu trabalho atual como uma string, a forma como a operária sabe se está ou não fazendo um trabalho no momento é checar sua propriedade CurrentJob – ela será igual a uma string vazia se a abelha estiver esperando por sua próxima tarefa. O C# oferece uma forma fácil de fazer isso: String.IsNullOrEmpty(CurrentJob) retornará true se a string CurrentJob estiver vazia ou nula e false caso contrário.

Exercício

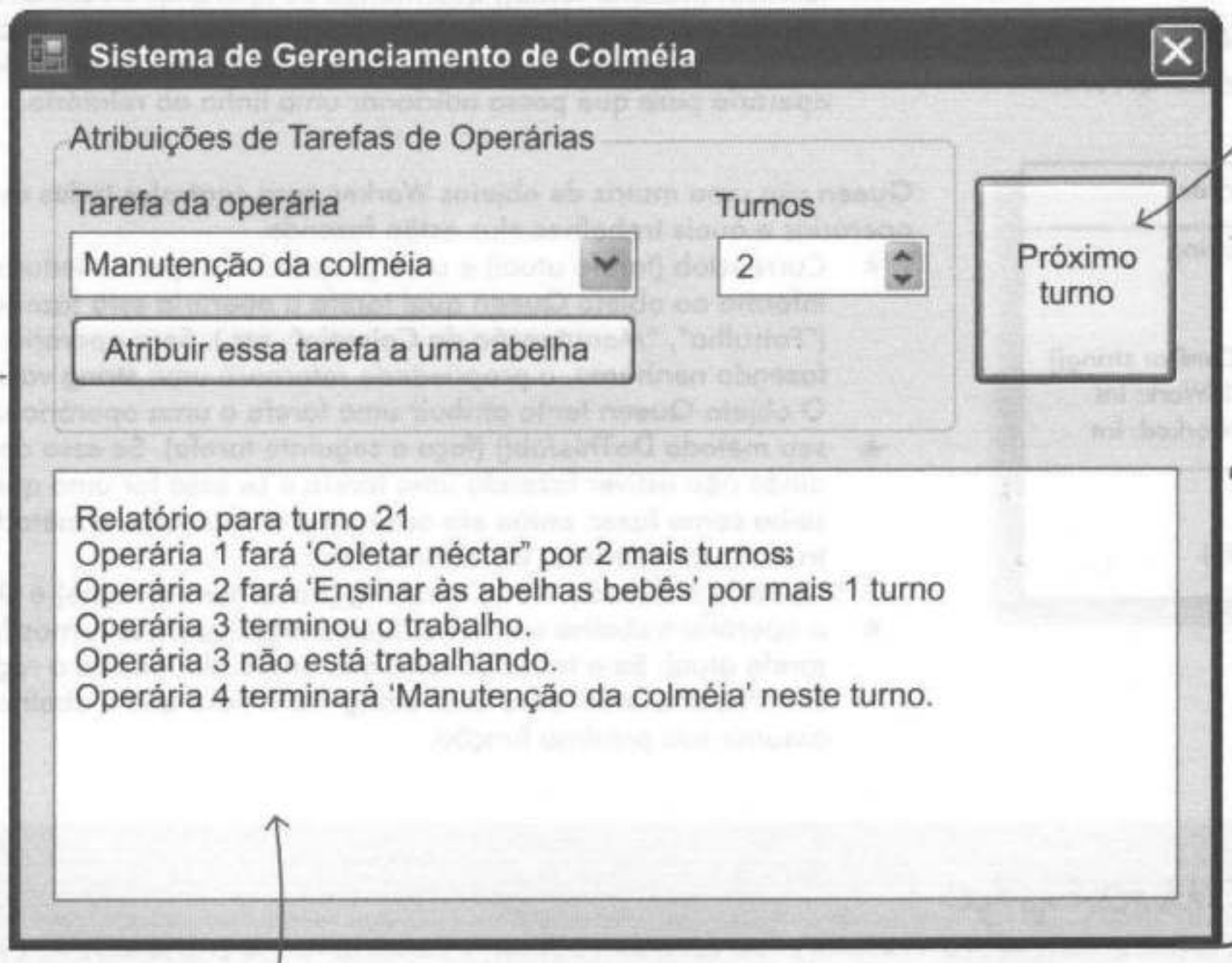
Uma abelha rainha precisa da sua ajuda! Use o que você aprendeu sobre classes e objetos para construir um sistema de gerenciamento de colméia para ajudá-la a controlar suas operárias.

1 Crie o formulário

O formulário é muito simples – toda sua inteligência está nas classes Queen e Worker. O formulário tem um campo privado Queen e dois botões chamam seus métodos AssignWork() e WorkTheNextShift(). Você precisará adicionar um controle ComboBox para as tarefas das operárias (volte à página anterior para ver os itens listados), um controle NumericUpDown, dois botões e uma caixa de texto com muitas linhas para o relatório de turno. Você também precisará do construtor do formulário – está depois da cópia dela que segue.

Este é um controle ComboBox chamado workerBeeJob (tarefa da abelha operária). Use sua propriedade Items para criar a lista e ajuste sua propriedade DropDownStyle para DropDownList para que o usuário só possa escolher itens da lista. A caixa Turnos é um controle NumericUpDown chamado shifts.

O botão nextShift chama o método WorkTheNextShift() da rainha, que retorna uma seqüência de caracteres com o relatório de turno.



Nomeie essa caixa de texto relatório e configure sua propriedade MultiLine como true.

Examine com cuidado este relatório de turno gerado pelo objeto Queen. Ele começa com o número do turno e então relata o que cada operária está fazendo. Use as seqüências de escape \r\n para adicionar uma quebra de linha no meio da seqüência de caracteres.

O construtor de cada objeto Worker recebe um parâmetro, uma matriz de seqüências de caracteres, que o informa quais trabalhos a operária sabe fazer.

```
public Form1() {
  InitializeComponent();
  Worker[] workers = new Worker[4];
  workers[0] = new Worker(new string[] { "Coletar necta", "Produção de mel" });
  workers[1] = new Worker(new string[] { "Cuidar dos olhos", "Ensinar as abelhas bebês" });
  workers[2] = new Worker(new string[] { "Manutenção da coméia", "Patrulha" });
  workers[3] = new Worker(new string[] { "Coletor de necta", "Produção de mel",
        "Cuidar dos ovos", "Ensinar as abelhas bebê", "Manutenção da coméia", "Patrulha" });
  queen = new Queen(workers);
}
```

Seu formulário precisará de uma campo Queen chamado queen. Você passará essa matriz de referências de objeto Worker ao construtor do objeto.

2 Crie as classes Worker e Queen

Você sabe quase tudo que precisa sobre as classes `Worker` e `Queen`. Só há mais alguns detalhes. `Queen.AssignWork()` itera pela matriz worker do objeto `Queen` e tenta atribuir o trabalho a cada `Worker` usando seu método `DoThisJob()`. O objeto `Worker` checa sua matriz de seqüência de caracteres `jobsICanDo` (tarefas que eu posso fazer) para ver se ela pode fazer o trabalho. Se ela puder realizar a tarefa, ela associa seu campo privado `shiftsToWork` (turnos a trabalhar) à duração do trabalho, seu `CurrentJob` ao trabalho e seu `shiftNumber` (número do turno) a zero. Quando ela trabalha um turno, diminui um de `shiftNumber`. A propriedade apenas-leitura `ShiftsLeft` retorna `shiftsToWork - shiftsWorked` (turnos a trabalhar - turnos trabalhados) - a rainha usa isso para ver quantos turnos ainda faltam para completar o trabalho.

Solução do Exercício

```
public class Worker {
  public Worker(string[] jobsICanDo) {
    this.jobsICanDo = jobsICanDo;
  }
  public int ShiftsLeft {
    get {
      return shiftsToWork - shiftsWorked;
    }
  }
  private string currentJob = "";
  public string CurrentJob {
    get {
      return currentJob;
    }
  }
  private string[] jobsICanDo;
  private int shiftsToWork;
  private int shiftsWorked;
  public bool DoThisJob(string job, int numberOfShifts) {
    if (!String.IsNullOrEmpty(currentJob))
      return false;
    for (int i = 0; i < jobsICanDo.Length; i++)
      if (jobsICanDo[i] == job) {
        currentJob = job;
        this.shiftsToWork = numberOfShifts;
        shiftsWorked = 0;
        return true;
      }
    return false;
  }
  public bool WorkOneShift() {
    if (String.IsNullOrEmpty(currentJob))
      return false;
    shiftsWorked++;
    if (shiftsWorked > shiftsToWork) {
      shiftsWorked = 0;
      shiftsToWork = 0;
      currentJob = "";
      return true;
    }
    else
      return false;
  }
}
```

O construtor apenas atribui valores para a propriedade JobsICanDo, que é uma matriz de seqüência de caracteres. Ele é privado porque queremos que a rainha pergunte se a operária pode fazer um trabalho, em vez de checar se ela sabe como fazê-lo.

ShiftsLeft é um propriedade apenas-leitura que calcula quantos turnos faltam para completar o trabalho atual.

CurrentJob é um propriedade apenas-leitura que informa a rainha qual tarefa está sendo executada.

A rainha usa o método DoThisJob() da operária para atribuir trabalho a ela - ele checa sua propriedade JobsICanDo para ver se ela sabe como fazer o trabalho.

Usamos ! - o operador NOT - para checar se a seqüência de caracteres NÃO está nula ou vazia. É como checar para ver se algo é false.

A rainha usa o método WorkOneShift() da operária para dizer a ela para trabalhar no próximo turno. O método só retorna true se esse for o último turno que ele está trabalhando. Dessa forma, a rainha pode adicionar uma linha ao relatório para dizer que a abelha terá acabado depois deste turno.

Dê uma boa olhada na lógica aqui. Primeiro ela checa o campo currentJob: se a operária não está fazendo um trabalho, ele apenas retorna false, o que pára o método. Se não, então ele incrementa ShiftsWorked e depois checa para ver se o trabalho foi terminado comparando-o com ShiftsToWork. Se for, o método retorna true. Caso contrário, ele retorna false.

```
public class Queen {
  public Queen(Worker[] workers) {
    this.workers = workers;
  }
  private Worker[] workers;
  private int shiftNumber = 0;
  public bool AssignWork(string job, int numberOfShifts) {
    for (int i = 0; i < workers.Length; i++)
      if (workers[i].DoThisJob(job, numberOfShifts))
        return true;
    return false;
  }
  public string WorkTheNextShift() {
    shiftNumber++;
    string report = "Relatório para turno #" + shiftNumber + "\r\n";
    for (int i = 0; i < workers.Length; i++)
    {
      if (workers[i].WorkOneShift())
        report += "Operária #" + (i + 1) + " terminou o trabalho\r\n";
      if (String.IsNullOrEmpty(workers[i].CurrentJob))
        report += "Operária #" + (i + 1) + " não está trabalhando\r\n";
      else
        if (workers[i].ShiftsLeft > 0)
          report += "Operária #" + (i + 1) + " fará '" + workers[i].CurrentJob
            + "' por " + workers[i].ShiftsLeft + " mais turnos\r\n";
        else
          report += "Operária #" + (i + 1) + " terminará '"
            + workers[i].CurrentJob + "' após este turno\r\n";
    }
    return report;
  }
}
```

A rainha mantém sua matriz de operárias privada porque uma vez atribuídas, nenhuma outra classe poderia conseguir alterá-las... nem mesmo vê-las, já que ela é a única que dá ordens. O construtor atribui o valor para o campo.

Quando ela quer atribuir uma tarefa às suas operárias, começa com a primeira e tenta dar um trabalho a ela. Se não conseguir, continua com a próxima. Quando uma abelha que pode fazer o trabalho é encontrada, o método retorna (o que interrompe o laço).

O método WorkTheNextShift() da rainha manda cada operária trabalhar um turno e adiciona uma linha ao relatório, dependendo do status da operária.

Nós já mostramos o código do construtor. Aqui está o restante para o formulário:

O formulário usa seu campo queen para manter uma referência no objeto Queen, que, por sua vez, tem uma matriz de referências aos objetos worker.

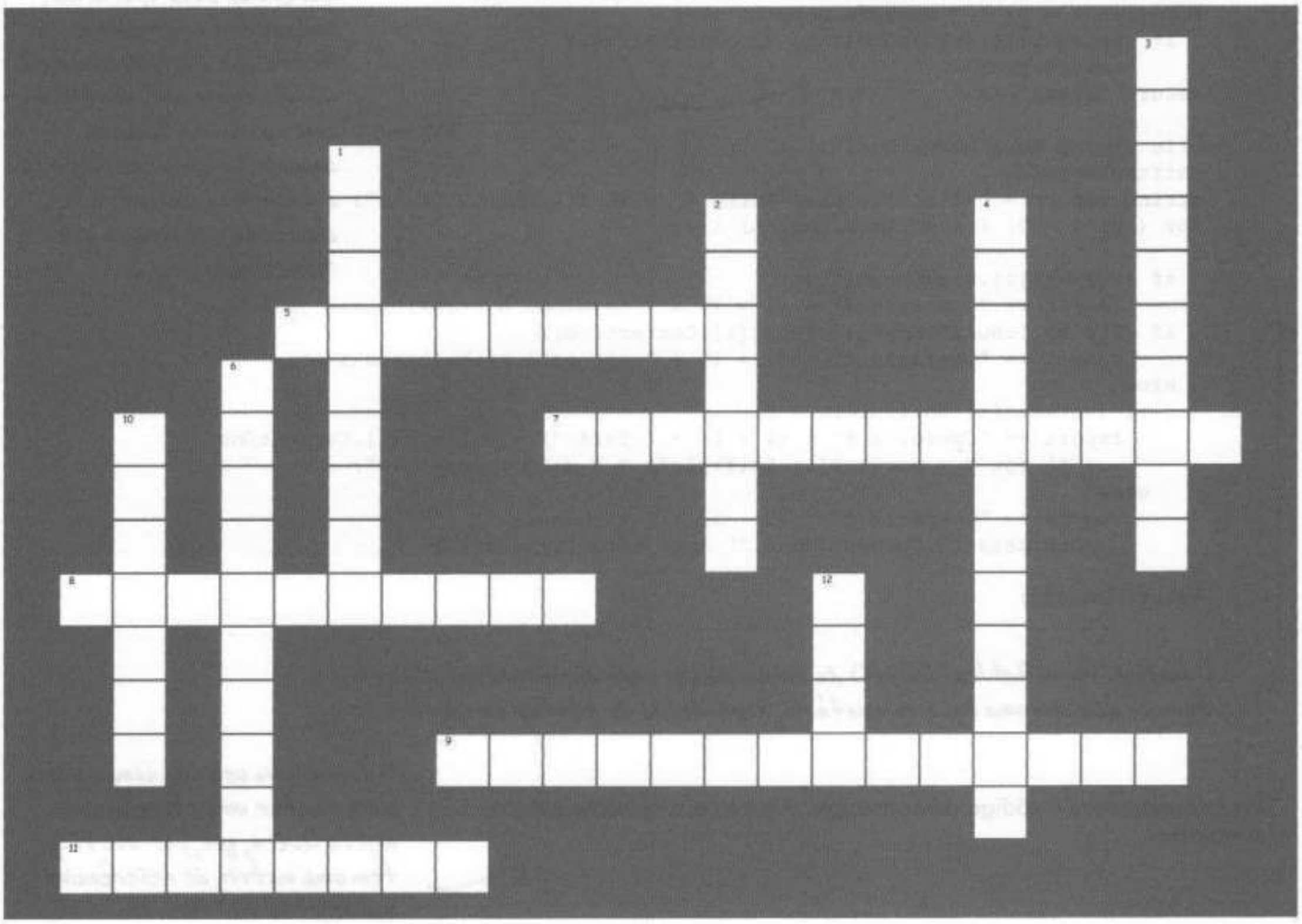
```
Queen queen;
private void assignJob_Click(object sender, EventArgs e) {
  if (queen.AssignWork(workerBeeJob.Text, (int)shifts.Value) == false)
    MessageBox.Show("Não tem operárias disponiveis para o trabalho '"
      + workerBeeJob.Text + "'", "A abelha rainha diz...");
  else
    MessageBox.Show("O trabalho '" + workerBeeJob.Text + "' irá ser feito "
      + shifts.Value + " turnos", "The queen bee says...");
}
private void nextShift_Click(object sender, EventArgs e) {
  report.Text = queen.WorkTheNextShift();
}
```

O botão assignJob chama o método AssignWork() de queen para atribuir trabalho a uma operária e exibe uma caixa de mensagem, dependendo se uma operária está ou não disponível para o trabalho.

O botão nextShift diz para a rainha fazer o próximo turno começar. Ela gera um relatório que é exibido na caixa de texto respectiva.

Cruzadinha de objeto

Antes de você ir para próxima parte do exercício, dê um descanso para o seu cérebro com uma cruzadinha rápida.



Horizontais

5. Este tipo de método obtém o valor de uma propriedade.
7. Este método retorna true se você passar "" a ele.
8. O construtor em uma subclasse não precisa dos mesmos _____ que o de sua classe base.
9. Um controle de formulário que permite criar aplicativos com abas.
11. Este tipo de classe não pode ser instanciado.

Verticais

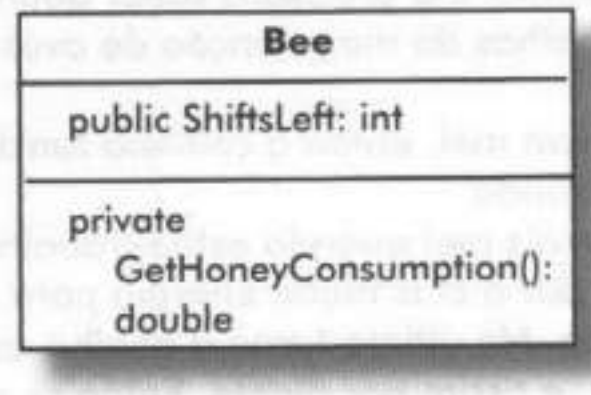
1. Uma _____ pode sobrepor métodos de sua classe base.
2. Se você quiser que uma subclasse sobreponha um método, marque-o com essa palavra-chave na classe base.
3. Um método em uma classe que é executado assim que é instanciado.
4. O que uma subclasse faz para substituir um método na classe base.
6. Ela contém classes base e subclasses.
10. O que você está fazendo quando adiciona dois pontos a uma declaração de classe.
12. Uma subclasse usa essa palavra-chave para chamar os membros da classe da qual ela herdou.

Respostas na página 222.

Use herança para estender o sistema de gerenciamento de abelhas

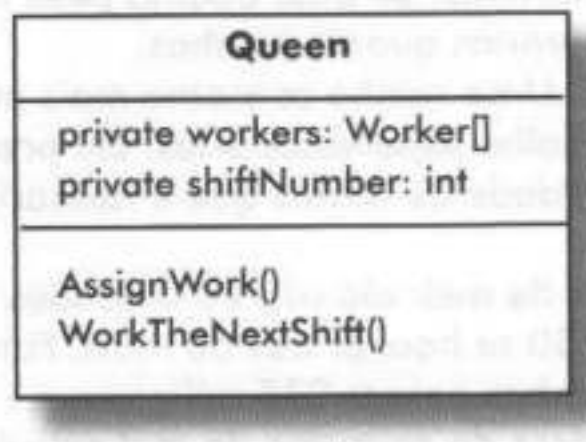
Agora que você tem um sistema básico feito, use herança para deixá-lo controlar quanto mel cada abelha consome. Abelhas diferentes consomem quantidades diferentes de mel e a rainha é a que mais consome. Então você usará o que aprendeu sobre herança para criar uma classe base Bee (abelha) da qual Queen e Worker herdam.

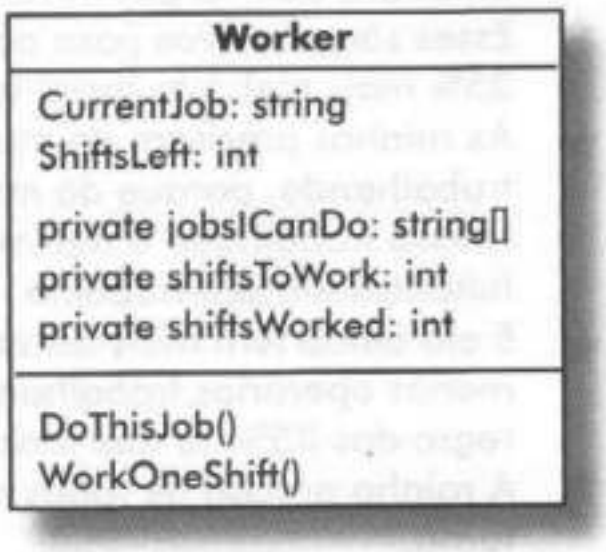
A classe Bee tem o comportamento básico de consumo de mel. Já que o consumo de mel requer o número de turnos que faltam, moveremos a propriedade ShiftsLeft para ela e a marcaremos como virtual para que Worker possa substituí-la.



A operária só precisa ser uma subclasse de Bee que sobrepõe o método ShiftsLeft com aquele que você já escreveu.

A rainha precisa alterar seu relatório para acrescentar os dados de consumo de mel. Isso significa que ela precisa somar o consumo de mel de cada operária – e já que ela mesma consome mel também, terá de herdar de Bee e sobrepor seu método GetHoneyConsumption().





Adicionar Item Existente

Sempre que você tiver um exercício em duas partes, é uma boa idéia começar um novo projeto para a segunda metade. Desta forma, você pode sempre voltar na primeira solução se precisar. Uma forma fácil de fazer isso é clicar com o botão direito no nome do projeto no Navegador de Solução do novo projeto no IDE, navegar para a pasta do projeto antigo e selecionar os arquivos que queira adicionar. O IDE fará novas cópias desses arquivos na nova pasta e vai adicioná-los ao projeto. Mas existem algumas coisas com as quais você deve tomar cuidado. O IDE NÃO vai alterar o namespace, então você precisará editar cada arquivo de classe e alterar sua linha de namespace à mão. E se você adicionar um formulário, tenha certeza de adicionar seus arquivos de design (Designer.cs) e de recurso (.resx) – e certifique-se de ter alterado os namespaces neles também.

Exercício

Ainda não terminamos! A rainha precisa controlar quanto mel a colméia está gastando com suas operárias. Aqui temos uma chance perfeita de usar suas novas habilidades sobre herança!

1 A rainha precisa saber quanto mel a colméia usa

A rainha acabou de receber um telefonema de suas abelhas contadoras, que lhe disseram que a colméia não está produzindo mel o bastante. Ela precisará saber quanto mel ela e suas operárias estão usando para que decida se transfere abelhas da manutenção de ovos para produção de mel.

- ★ Todas as abelhas consomem mel, então a colméia funciona à base de muito mel. É por isso que elas precisam continuar produzindo.
- ★ As operárias consomem mais mel quando estão trabalhando. Elas precisam da maior parte quando o trabalho começa, para dar a elas muita energia para a tarefa. Elas consomem cada vez menos conforme vão trabalhando. No último turno a abelha usa 10 unidades de mel, no penúltimo ela usa 11, no antepenúltimo 12, e assim por diante. Então se a abelha está trabalhando (o que significa que seu `ShiftsLeft` é maior que zero), você pode calcular quantas unidades de mel consumir adicionando 9 a `ShiftsLeft`.
- ★ Se uma abelha não tem uma tarefa (ou seja, seu `ShiftsLeft` é zero), ela consome apenas 7,5 unidades de mel por turno.
- ★ Esses são números para abelhas normais. Se uma abelha pesa mais que 150 miligramas, ela usa 35% mais mel. Isto inclui tanto operárias quanto rainhas.
- ★ As rainhas precisam de muito mel. Uma rainha consome mais mel quando ela tem mais operárias trabalhando, porque dá muito trabalho supervisioná-las. Ela precisa consumir tanto mel quanto se tivesse trabalhado a mesma quantidade de turnos que a operária com o maior número de turnos faltantes em seu trabalho.
- ★ E ela ainda tem mais necessidades de mel: ela usa 20 unidades extras por turno se houver duas ou menos operárias trabalhando, ou 30 se houver três ou mais. Para o consumo da rainha não vale a regra dos 35%, já que todas as rainhas pesam 275 miligramas.
- ★ A rainha precisa de todos os números do consumo de mel calculados no final de cada relatório de turno.

2 Crie uma classe `Bee` para lidar com os cálculos de mel

Já que as operárias e rainha todas fazem seus cálculos sobre mel de formas similares, você poderá evitar duplicar seu código tendo uma classe base `Bee` da qual `Worker` e `Queen` possam herdar. Você sabe que cada abelha precisa conhecer seu próprio peso (para saber se deve multiplicar seu gasto de mel por 35%).

- ★ Crie o método `GetHoneyConsumption ()` (obtenha consumo de mel) que calcule a quantidade de mel que uma operária consome. Já que as operárias e a rainha precisam fazer esse cálculo, mas a rainha precisa fazer um cálculo extra também, faz sentido worker herdá-lo e queen fazer sobreposição.
- ★ O método `GetHoneyConsumption()` precisa do número de turnos faltantes, então adicione uma propriedade virtual apenas-leitura chamada `ShiftsLeft` que retorne zero. A classe `worker` terá essa propriedade sobreposta.
- ★ O cálculo do consumo de mel precisa incluir o peso da abelha, então o construtor `Bee` precisará usar o peso como um parâmetro e armazená-lo num campo. Já que nenhuma outra classe precisa usá-lo, você deve deixá-lo privado.

Essa é uma boa regra geral. Você deve deixar campos e métodos privados por padrão e só deixá-los públicos se outra classe precisar deles. Desta forma, você evita problemas em seus programas causados por uma classe acessando propriedades ou métodos de outra incorretamente.

Dica: Você pode usar a mensagem de erro ligeiramente misteriosa sem sobrecarga para seu próprio benefício! Faça a classe Worker herdar de Bee, então compile seu código. Quando o erro for exibido, clique duas vezes nele e o IDE se moverá para o construtor Worker automaticamente. Muito conveniente!

3 **Faça a classe `Worker` herdar de `Bee`**

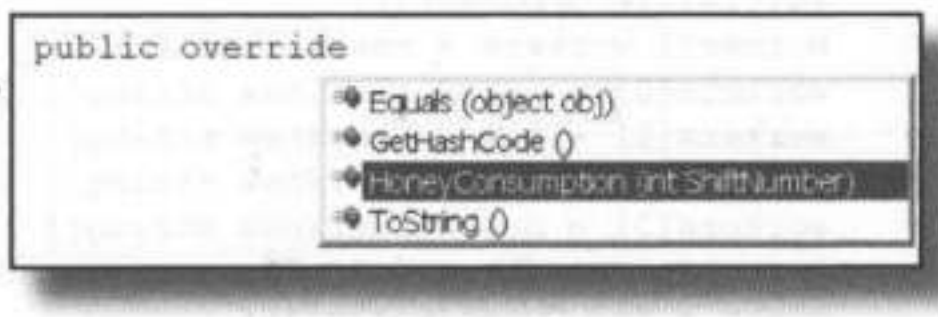
Você precisará configurar o construtor para chamar o seu equivalente da classe base, como você fez com a Catarina. Você precisará alterar o construtor `Worker` para que ele use o peso da abelha como um parâmetro; e passe esse parâmetro para o construtor da classe base. Depois, acrescente a palavra-chave override ao método `ShiftLeft` de `Worker`. Uma vez feito isso, cada `worker` poderá calcular seu consumo de mel para a rainha... e você não precisa fazer nenhuma outra alteração na classe `Worker`!

4 **Faça a classe `Queen` herdar de `Bee`**

A classe `Queen` precisa de um pouco mais de alterações que a classe `Worker`, já que efetivamente calculará o total de mel e o adicionará ao relatório de turno.

- ★ Substitua o método `Bee.GetHoneyConsumption()` e adicione o cálculo extra da rainha. Ela precisará calcular se existem duas ou menos operárias com tarefas, para saber se precisa de 20 ou 30 unidades extras. Então ela precisará adicionar isso ao número de unidades que ela usaria se tivesse o mesmo número de turnos faltantes que a operária com o maior número deles.
- ★ Atualize `WorkTheNextShift()` da rainha adicionando a linha de consumo de mel ao relatório. Inclua um laço para somar os consumos de mel para cada operária e encontre também aquela com o maior consumo – faça isso antes da rainha dizer a cada operária trabalhar o próximo turno (para que ela tenha os números de consumo para o turno atual). Ela vai somar esses números, seu próprio consumo e incluir uma linha ao final do relatório que diz "Consumo de Mel Total: xxx unidades" (xxx é o número de unidades de mel consumido).
- ★ Você precisará atualizar o construtor de `Queen` assim como fez com `Worker`.

Vá até a classe Queen e digite public override – quando você pressionar a barra de espaço, o IDE automaticamente lista todos os métodos que você pode substituir. Selecione o método que quer substituir e ele incluirá a chamada do método base automaticamente.



5 **Atualize o formulário para instanciar as abelhas corretamente**

Já que você alterou os construtores de `Queen` e `Worker`, também precisará alterar a forma como eles são chamados. Cada construtor possui um novo parâmetro `Weight`. Assim, você precisará usar pesos:

★ Operária tipo 1: 175 mg; tipo 2: 114 mg; 3: 149 mg; 4: 155 mg; Rainha: 275 mg.

Essa é a única alteração que você terá que fazer ao formulário!

Solução do Exercício

Aqui está a classe Bee. Ela faz o cálculo de consumo de mel básico usado pelas classes Worker e Queen.

A classe Bee possui um construtor que atribui valor para seu campo Weight e um método HoneyConsumption() que calcula quanto mel uma operária consome.

```
public class Bee {
  public Bee(double weight) {
      this.weight = weight;
  }
  public virtual int ShiftsLeft {
    get { return 0; }
  }
  private double weight;
  public virtual double GetHoneyConsumption() {
    double consumption;
    if (ShiftsLeft == 0)
      consumption = 7.5;
    else
      consumption = 9 + ShiftsLeft;
    if (weight > 150)
      consumption *= 1.35;
    return consumption;
  }
}
```

Se uma abelha estiver a um turno do final, ela consome 10; se faltam 2, ela consome 11, etc. Se ela não tem nenhuma tarefa, então ela consome 7,5. Se ShiftsLeft for zero, então a abelha não tem trabalho.

Se a abelha pesa mais do que 150mg, então o consumo sobe 35%.

A herança facilita atualizar o código e adicionar o novo comportamento de consumo de mel às classes Queen e Worker. Teria sido muito mais difícil fazer esta alteração se existisse muito código duplicado.

Só o construtor de formulário mudou - o restante dele é exatamente o mesmo.

```
public Form1() {
  InitializeComponent();
  Worker[] workers = new Worker[4];
  workers[0] = new Worker(new string[] { "Coletar nectar", "Produção de mel" }, 175);
  workers[1] = new Worker(new string[] { "Cuidar dos olhos", "Ensinar as abelhas bebês" }, 114);
  workers[2] = new Worker(new string[] { "Manutenção da colmeia", "Patrulha" }, 149);
  workers[3] = new Worker(new string[] { "Nectar collector", "Produção de mel",
      "Cuidar dos ovos", "Ensinar as abelhas bebês", "Manutenção da colmeia", "Patrulha" }, 155);
  queen = new Queen(workers);
}
```

A única alteração no formulário é que os pesos precisam ser adicionados aos construtores de Worker.

```
public class Worker : Bee {
  public Worker(string[] jobsICanDo, int weight)
    : base(weight) {
    this.jobsICanDo = jobsICanDo;
  }
  public override int ShiftsLeft {
  //...o restante da classe é o mesmo...
```

Tudo que a classe Worker precisa é herdar de Bee e ter seu construtor ajustado para que ele use um parâmetro Weight e passe-o ao construtor da classe base; e sobrepor a propriedade Bee.ShiftsLeft adicionando a palavra-chave override na declaração de propriedade.

A classe Queen precisou de algumas alterações, começando por herdar de Bee.

```
public class Queen : Bee {
  public Queen(Worker[] workers)
    : base(275) {
    this.workers = workers;
  }
  public string WorkTheNextShift()
  {
    double totalConsumption = 0;
    for (int i = 0; i < workers.Length; i++)
      totalConsumption += workers[i].GetHoneyConsumption();
    totalConsumption += GetHoneyConsumption();
    //...aqui fica o código original deste método...
    report += "Consumo total de mel: " + totalConsumption + " unidades";
    return report;
  }
  public override double GetHoneyConsumption() {
    double consumption = 0;
    double largestWorkerConsumption = 0;
    int workersDoingJobs = 0;
    for (int i = 0; i < workers.Length; i++) {
      if (workers[i].GetHoneyConsumption() > largestWorkerConsumption)
        largestWorkerConsumption = workers[i].GetHoneyConsumption();
      if (workers[i].ShiftsLeft > 0)
        workersDoingJobs++;
    }
    consumption += largestWorkerConsumption;
    if (workersDoingJobs >= 3)
      consumption += 30;
    else
      consumption += 20;
    return consumption;
  }
}
```

A rainha pesa 275 mg, então seu construtor chama o construtor base de Bee e passa um peso de 275 para ele.

WorkTheNextShift() tem um laço adicionado ao topo que chama cada método GetHoneyConsumption() de cada operária e chama seu próprio método GetHoneyConsumption() para calcular o consumo total.

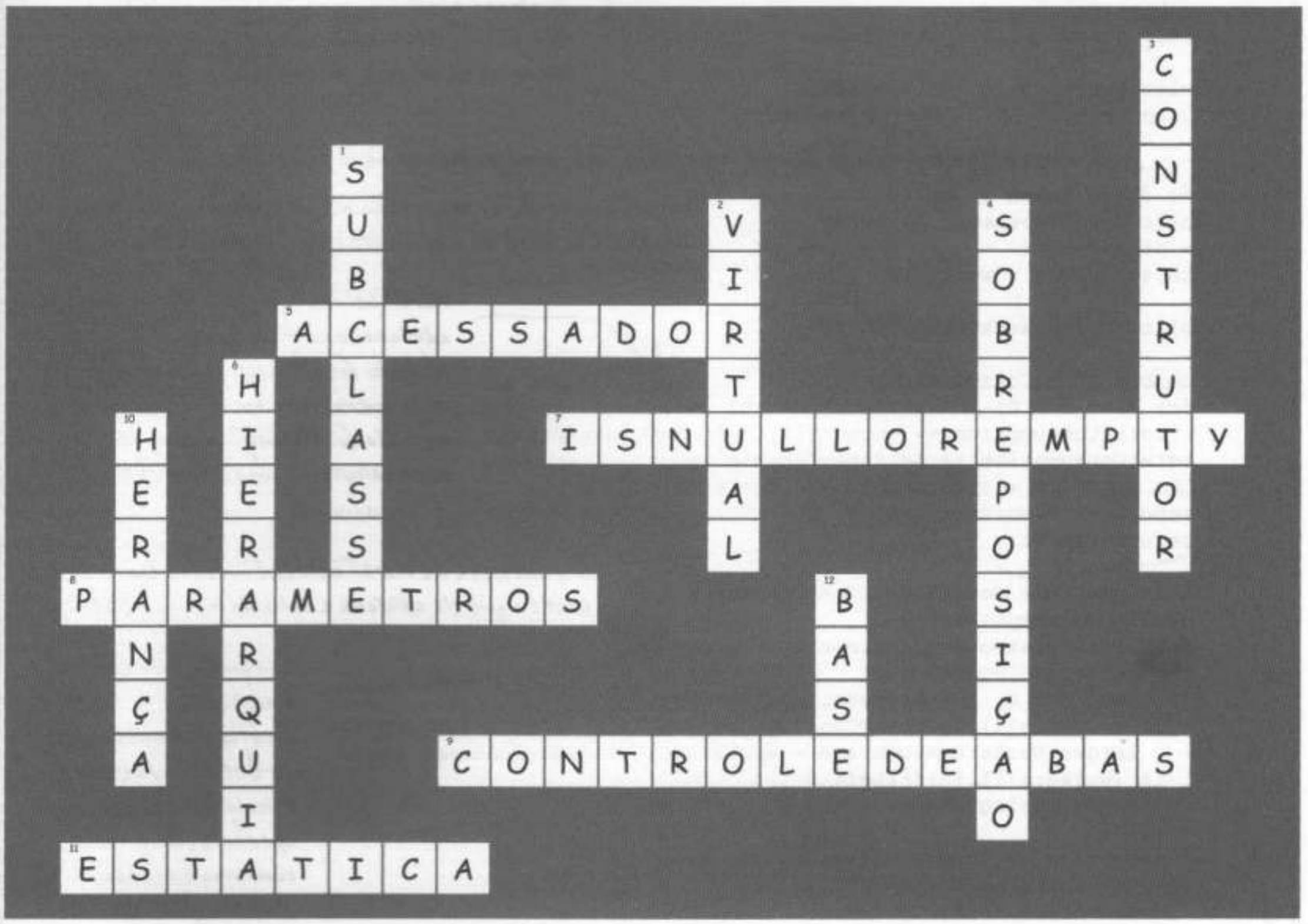
O restante de WorkTheNextShift() é o mesmo, exceto que ele adiciona a linha de mel ao relatório.

A rainha sobrepõe o método GetHoneyConsumption() de Bee para fazer seu cálculo. Ela encontra a operária com o maior consumo e acrescenta 20 ou 30 nele baseando-se em quantas operárias estão trabalhando.

Se houver três ou mais operárias trabalhando, a rainha precisa de mais 30 unidades de mel; caso contrário, ela precisa de mais 20.

Este laço observa o consumo de todas as operárias e encontra aquela com o maior consumo.

Solução da cruzadinha de objeto



Fazendo as classes manterem suas promessas

Ações são mais importantes do que palavras

Algumas vezes você precisa agrupar seus objetos baseando-se nas **coisas que eles podem fazer** em vez da relação com as classes das quais eles herdam características. É aí que entram as interfaces - elas permitem que você trabalhe com qualquer classe que possa fazer o trabalho. Mas **com o poder vem a responsabilidade**. Qualquer classe que implementar uma interface precisa se **comprometer a preencher todos os requisitos**... ou o compilador quebrará as pernas delas entende?

Vamos voltar ao colmei-ço

A corporação Bee-nâmica Geral quer que o Sistema de Gerenciamento de Colméias desenvolvido no capítulo anterior seja um Simulador de Colméias completo. Eis uma visão geral das especificações para a nova versão do programa:

Simulador de Colméias da Bee-nâmica Geral

Para melhor representar a vida na colméia, precisaremos adicionar capacidades específicas para as abelhas operárias.

- Todas as abelhas consomem mel e têm um peso.
- Rainhas distribuem tarefas, monitoram os relatórios e dizem às operárias para trabalharem no próximo turno.
- Todas as operárias trabalham em turnos.
- Abelhas patrulheiras com ferrão precisam ser capazes de afiá-los, procurar por inimigos e ferroá-los.
- Abelhas coletoras de néctar são responsáveis por encontrar flores, recolher o néctar e retornar para a colméia.

As classes Abelha e Operária parecem que não vão mudar muito. Podemos estender as classes que já temos para lidar com essas novas características.

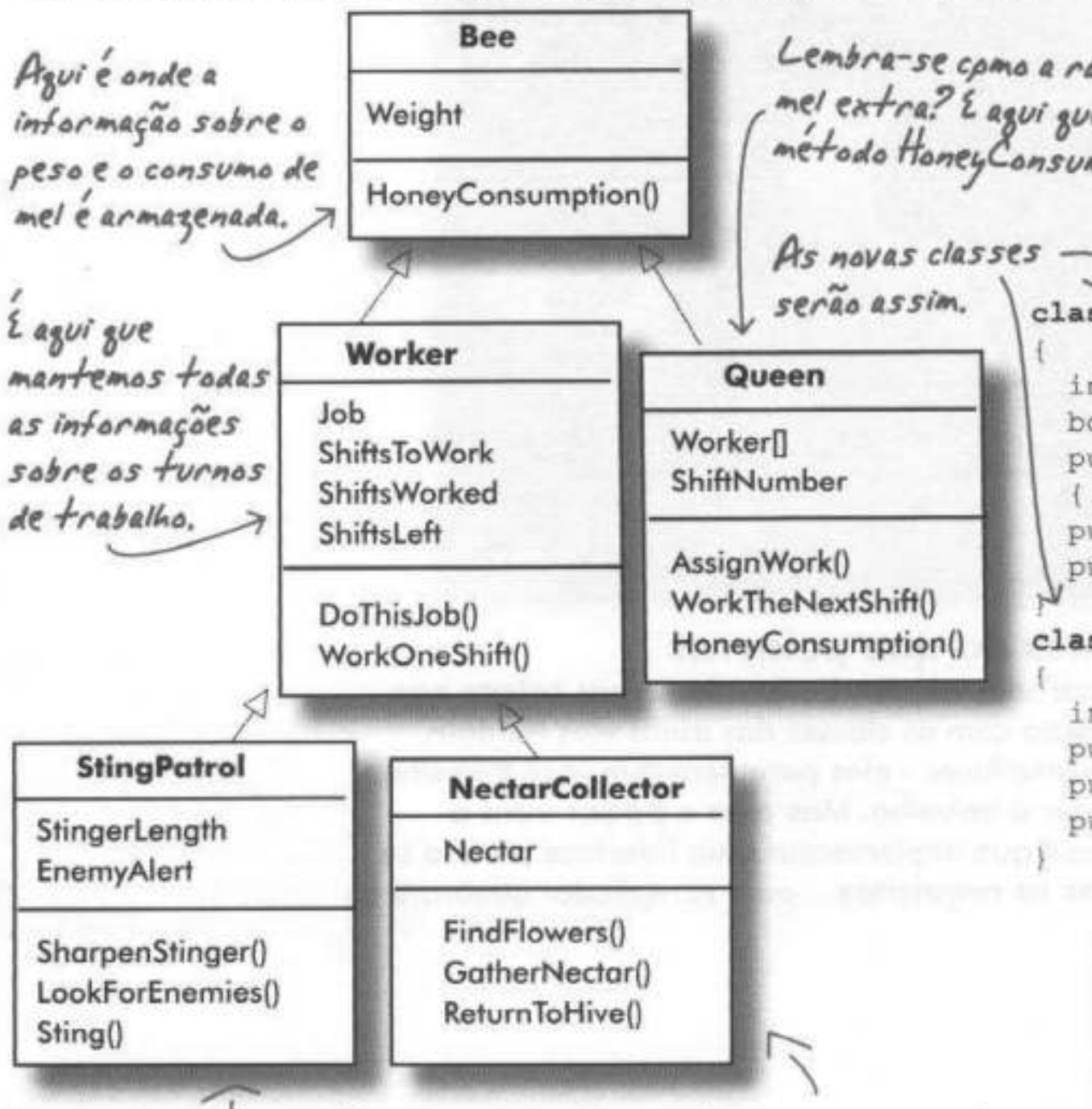
Parece que precisaremos ser capazes de armazenar dados diferentes para cada abelha operária, de acordo com o trabalho que elas façam.

Muitas coisas estão iguais

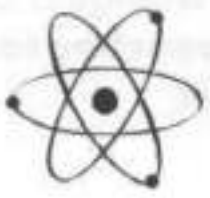
As abelhas no novo simulador de colméia ainda consomem mel como antes. A abelha rainha ainda precisa ser capaz de distribuir tarefas para as operárias e examinar os relatórios de turno que dizem quem está fazendo o quê. As operárias também trabalham em turnos como antes e apenas o trabalho delas foi mais um pouco elaborado.

Podemos usar herança para criar classes para tipos diferentes de abelhas

Mostramos aqui uma hierarquia com as classes Worker (operária) e Queen (rainha) que herdam de Bee (abelha). Worker tem subclasses `NectarCollector` e `StingPatrol` (pode ser entendido como "patrulheira com ferrão").




```
class StingPatrol : Worker
{
  int StingerLength;
  bool enemyAlert;
  public bool SharpenStinger (int Length)
  {...}
  public bool LookForEnemies(){...}
  public void Sting(string Enemy){...}
}
class NectarCollector : Worker
{
  int Nectar;
  public void FindFlowers (){...}
  public void GatherNectar(){...}
  public void ReturnToHive(){...}
}
```



Poder do Cérebro

O que fazer se tivermos uma abelha que precisa ferroar e também coletar néctar?

Uma interface diz para uma classe que ela precisa implementar certos métodos e propriedades

Uma classe pode apenas herdar de outra classe. Assim, criar duas subclasses de Bee, `StingPatrol` e `NectarCollector`, não ajudará se precisarmos de uma Bee que precisa fazer os **dois** trabalhos. Por sorte, o C# nos permite usar **interfaces** para lidar com situações como esta. As interfaces permitem definir diversos métodos que a classe **deve** ter.

Uma interface **exige** que uma classe tenha certos métodos.**Obrigando o compilador a levantar erros** se não encontrar todos os métodos necessários em qualquer classe que implemente essa interface. Os métodos podem ser desenvolvidos diretamente na classe, ou podem ser herdados de uma classe base. A interface não se importa como os métodos ou propriedades vieram parar ali, contanto que eles estejam presentes quando o código é compilado.

Você usa uma interface para exigir de uma classe que ela inclua todos os métodos e propriedades que constem na interface - se ela não tem, o compilador levantará um erro.

O método DefendeTheHive() (defender a colméia) da rainha pode apenas dizer para objetos StingPatrol para manter a colméia segura. Ela adoraria comandar as outras abelhas para usar seus ferrões, mas ela não tem nenhuma forma de ordenar que elas ataquem.

Se criarmos uma interface IStingPatrol, qualquer objeto abelha que a implementar e adicionar AlertLevel (nível de alerta), StingerLength (comprimento do ferrão), LookForEnemies() (procurar por inimigos) e SharpenStinger() (afiar ferrão) podem defender a colméia.

Ao adicionar um arquivo de classe com a interface IstingPatrol ao projeto e modificando a classe BeeCare para usá-la, você dá à rainha uma forma de ter certeza de que objetos BeeCare saibam que devem defender a colméia.

Estes são os mesmos objetos Maintenance e BeeCare. Mas agora eles implementaram a interface IStingPatrol e, por isso, sabem como afiar seus ferrões e procurar por inimigos.

Nomes de interfaces começam com I

Sempre que você criar uma interface, deve fazer seu nome começar com um I maiúsculo. Na verdade, não existe uma regra que o obrigue a isto, mas fazê-lo torna seu código mais fácil de entender. Você pode ver por você mesmo o quanto. Entre no IDE em qualquer linha em branco dentro de qualquer método e insira um I. Você verá o IntelliSense mostrar várias interfaces predefinidas do .NET. Uma vez que todas elas comecem com I, é imediatamente óbvio que são interfaces e não classes.

Use a palavra-chave interface para definir uma

Incluir uma interface em seu programa é muito parecido com incluir uma classe exceto que você nunca desenvolve nenhum método. Você apenas define seus valores de retorno e seus parâmetros. Porém em vez de um bloco de comandos dentro de chaves, você encerra a linha com um ponto e vírgula.

Interfaces nunca armazenam dados, logo você **não pode incluir nenhum campo**. Mas você pode adicionar definições para propriedades. O motivo é que uma função de acesso é apenas um tipo diferente de método e as interfaces servem para forçar as classes a terem certos métodos com nomes, tipos e parâmetros específicos. Logo, se você quer que sua interface obrigue um certo campo a existir com um nome determinado, **use uma propriedade** - você chegará ao mesmo resultado.

Você declara uma interface dessa forma:

```
public interface IStingPatrol
{
    int AlertLevel { get;}
    int StingerLength { get; set;}
    bool LookForEnemies();
    int SharpenStinger(int Length);
}
```

Qualquer classe que implemente esta interface precisa ter todos estes métodos e propriedades, ou o programa não compilará.

As interfaces não armazenam dados. Então não têm campos... mas podem ter propriedades.

Qualquer classe que implemente esta interface precisa ter um método SharpenSting() com como parâmetro um int.

```
public interface INectarCollector
{
    void FindFlowers();
    void GatherNectar();
    void ReturnToHive();
}
```

Tudo dentro de uma interface é feito para ser efetivamente usado dentro de outra classe. Então todos os métodos de uma são automaticamente públicos.

Você não escreve o código para os métodos na interface, apenas os nomes. Você desenvolverá os métodos na classe que a implementar.

Tudo numa interface pública é automaticamente público, porque você usa as interfaces para definir métodos e propriedades públicos de qualquer classe que as implemente.

Agora você pode criar uma instância de NectarStinger que realize ambas as tarefas

Você usa o **operador dois pontos** para declarar uma interface exatamente como faz com herança. Ele funciona assim: a primeira coisa depois dos dois pontos é a classe de quem se está herdando, seguido por uma lista de interfaces - exceto se não se vai herdar de classe alguma e, nesse caso, temos somente a lista (que não precisa seguir nenhuma ordem em particular).

Você implementa uma interface com um operador dois pontos exatamente como com a herança.

Esta classe herda de Worker e implementa INectarCollector e IStingPatrol.

```
class NectarStinger : Worker, INectarCollector, IStingPatrol {
  public int AlertLevel {
    get { return alertLevel; }
  }
  public int StingerLength {
    get { return stingerLength; }
    set {
      stingerLength = value;
    }
  }
  public bool LookForEnemies() {...}
  public int SharpenStinger(int Length)
  {...}
  public void FindFlowers() {...}
  public void GatherNectar() {...}
  public void ReturnToHive() {...}
}
```

Você pode usar mais de uma interface se as separar com vírgulas.

NectarStinger atribui valor para o campo de armazenamento para a propriedade AlertLevel em seu método LookForEnemies().

A abelha retrai seu ferrão quando não existem inimigos por perto, de forma que o campo de armazenamento muda de valor com o passar do tempo.

Cada método na interface corresponde a um na classe. De outra forma o código não compilará.

Quando você criar um objeto NectarStinger ele será capaz de realizar as tarefas tanto das operárias de tipo NectarCollector quanto das de StingPatrol.

Quando você tem uma classe que implementa uma interface ela age como qualquer outra classe. Você pode criar uma instância usando new e usar seus métodos:

```
NectarStinger bobTheBee = new NectarStinger();
bobTheBee.LookForEnemies();
bobTheBee.FindFlowers();
```

Classes que implementarem interfaces devem incluir TODOS os métodos

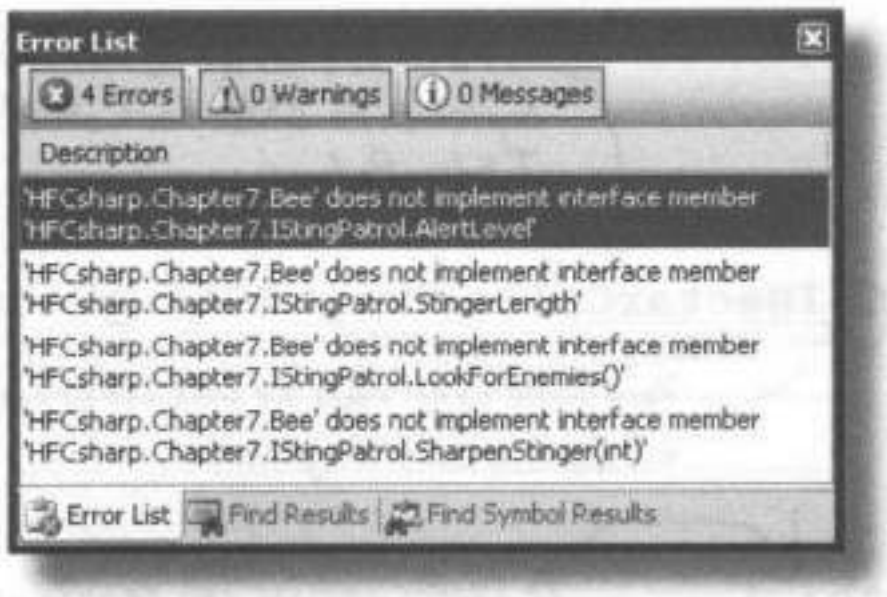
Implementar uma interface implica em ter que desenvolver um método ou propriedade na classe para cada correspondente declarado na interface - se não for o caso, o código não compilará. Se uma classe implementar mais de uma interface, será preciso fazer isso para cada uma delas. Mas não precisa aceitar nossa sugestão...

1. **Crie um novo aplicativo e adicione um novo arquivo de classe chamado IStingPatrol.cs**
   Em vez de adicionar uma classe, inclua a interface **IStingPatrol** da página anterior.

2. **Adicione uma classe Bee ao projeto**
   Não acrescente nenhum método ou propriedade ainda. Apenas implemente IStingPatrol:

```
public class Bee : IStingPatrol {
```

3. **Tente compilar o programa**
   Selecione "Rebuild" (recompilar) no menu Build. Opa, o compilador não deixará você fazer isso:



Você verá um destes erros "does not implement" (não implementa) para cada membro de IStingPatrol que não esteja implementado na classe. O compilador realmente quer que você desenvolva cada método declarado na interface.

4 **Adicione os métodos e propriedades da classe Bee**
Adicione um método `LookForEnemies` e um `SharpenStinger` - eles não precisam fazer nada, apenas precisam compilar sem erros. Então adicione um acessor do tipo get para um `int` chamado `AlertLevel` e acessores get e set para um `int` chamado `StingerLength`. Agora o programa compilará!

Pratique um pouco com interfaces

Interfaces são realmente fáceis de usar, mas a melhor forma de entendê-las é começar a usá-las. Então crie um novo projeto de Aplicativo Windows Forms, arraste um botão para o formulário e vamos lá!

1 Eis uma classe TallGuy (cara alto) e o código para um botão que cria um objeto desse tipo usando um inicializador de objeto e que chama seu método `TalkAboutYourself()` (fale de você). Nada de novo até aqui - usaremos isso num minuto.

```
public class TallGuy {
  public string Name;
  public int Height;
  public void TalkAboutYourself() {
    MessageBox.Show("My name is " + Name + " and I'm "
      + Height + " inches tall.");
  }
}
private void button1_Click(object sender, EventArgs e) {
  TallGuy tallGuy = new TallGuy() { Height = 74, Name = "Jimmy" };
  tallGuy.TalkAboutYourself();
}
```

2 Vamos criar uma interface IClown (IPalhaço) para a classe.
Você já sabe que tudo dentro de uma interface tem que ser público. Mas não aceite nossa sugestão. Crie um novo projeto e declare uma interface você mesmo, dessa forma, por exemplo:

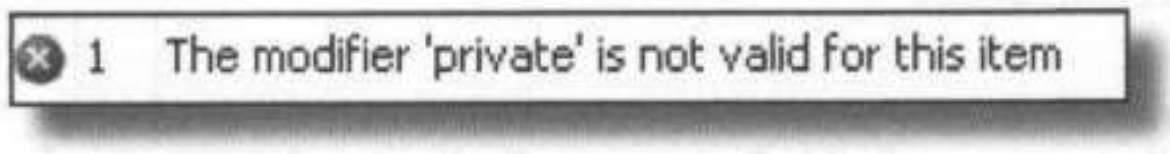
```
public interface IClown
```

Agora tente declarar um método privado dentro da interface:

```
private void Honk();
```

Você não precisa digitar public dentro da interface, porque ela automaticamente faz com que todos os métodos e propriedades sejam públicos.

Selecione Build >> Build Solution no IDE. Você verá o seguinte erro:



Agora continue e **apague o modificador de acesso privado** - o erro desaparecerá e o código compilará sem problemas.

3. Antes de seguir para a próxima página, veja se consegue criar o resto da interface Iclown e modifique a classe TallGuy para implementá-la. Adicione sua interface no seu projeto exatamente como você faria com uma classe: clique com o botão direito no projeto no Navegador de Solução e adicione um arquivo de classe chamado `IClown.cs`.
Sua nova interface IClown deve ter um método `void` chamado Honk (buzine) que não deve ter nenhum parâmetro e uma propriedade apenas-leitura tipo `string` chamada FunnyThingIHave (eu possuo uma coisa engraçada) que deve ter um acessor get, mas não um set.

4. Eis a interface - acertou?

```
public interface IClown
{
  string FunnyThingIHave { get; }
  void Honk();
}
```

Aqui está um exemplo de uma interface com um acessor get e não tem um set. Lembre-se: interfaces não podem conter campos. Mas quando você implementa esta propriedade apenas-leitura numa classe ela se parece como um campo para os demais objetos.

Certo, agora modifique a classe `TallGuy` para implementar clown. Lembre-se, o operador dois pontos é sempre seguido pela classe base de quem se herda (se existir alguma) e então uma lista de interfaces para implementar, todas separadas por vírgulas. Já que não há uma classe base e apenas uma única interface, a declaração ficará assim:

TallGuy vai implementar a interface IClown.

```
public class TallGuy : IClown
```

O IDE informa é que quando você disse que TallGuy implementaria IClown, prometeu adicionar todas as propriedades e métodos daquela interface... e depois você quebrou a promessa!

5. Tenha certeza de que o restante da classe seja o mesmo, incluindo os dois campos e o método. Selecione "Build Solution" do menu Build no IDE para compilar o programa. Você verá dois erros, incluindo este:

```
'TallGuy' does not implement interface
member 'IClown.Honk()'
```

6. Os erros desaparecerão assim que você adicionar todos os métodos e propriedades definidos na interface. Então continue e a implemente. Adicione uma propriedade tipo string apenas-leitura chamada FunnyThingIHave com um acessor get que sempre retorna a seqüência de caracteres "sapatos enormes". Adicione um método `Honk()` que mostre uma caixa de mensagem que diz "Fom Fom!".

Eis como será:

```
public string FunnyThingIHave {
  get { return "big shoes"; }
}

public void Honk() {
  MessageBox.Show("Honk honk!");
}
```

A interface exige apenas que uma classe que a implemente tenha uma propriedade FunnyThingIHave com um acessor get. Você pode colocar qualquer acessor get nela, mesmo um que sempre retorne a mesma seqüência de caracteres. A maioria dos acessores get não funcionará assim, mas funcionará bem, pois faz o que precisamos fazer aqui.

A interface diz que você precisa de um método void público chamado Honk, mas não diz o que o método precisa fazer. Ele pode fazer qualquer coisa; e não importa o que seja, o código compilará desde que o método exista e tenha a assinatura corretas.

7. Agora seu código vai compilar! Complete seu botão para que o inicializador de objeto atribua um valor para `FunnyThingIHave` e o botão deverá chamar o método `Honk()` do objeto.

Você não pode criar instâncias de uma interface, mas pode criar referências

Digamos que você tenha um método que precisa de um objeto que possa executar o método FindFlowers() (encontrar flores). Qualquer objeto que implemente a interface INectarCollector poderia. Poderia ser um objeto Worker, ou um objeto Robot ou um Dog, desde que implementasse essa interface.

É para isso que servem as **referências para interfaces**. Você pode usar uma para fazer referência para um objeto que implemente a interface necessária e sempre terá certeza de que ela tem os métodos certos para o que você quer fazer - mesmo que você não saiba muito mais a respeito do objeto.

Isso não funciona...

```
IStingPatrol dennis = new IStingPatrol();
```

Se você tentar criar uma instância de uma interface, o compilador reclamará.

```
1  Cannot create an instance of the abstract class or interface
```

Você não pode usar a palavra-chave new com uma interface, o que faz sentido - os métodos e propriedades não estão implementados. Se você pudesse criar um objeto a partir de uma interface, como ele saberia o que fazer?

... isso sim.

```
NectarStinger fred = new NectarStinger();
IStingPatrol george = fred;
```

Mesmo que esse objeto possa fazer mais coisas, quando você usa uma referência a uma interface só terá acesso aos métodos dela.

Lembra como você podia passar uma referência de BLT para qualquer classe que esperasse um sanduíche, porque BLT herdava de Sandwich? Aqui é a mesma coisa - você pode usar um NectarStinger em qualquer método ou comando que espere um IStingPatrol.

A primeira linha é um comando new comum, criando uma referência chamada Fred e apontando-a para um objeto NectarStinger. A segunda linha é onde as coisas começam a ficar interessantes, porque **cria uma nova variável de referência usando IStingPatrol**. Ela pode parecer um pouco estranha na primeira vez que você a vê. Mas observe o seguinte:

```
NectarStinger ginger = fred;
```

Você sabe o que este tipo de comando faz; cria uma nova referência para NectarStinger chamada ginger e a aponta para qualquer que seja o objeto para o qual `fred` esteja apontando. O comando que inclui `george` usa IStingPatrol da mesma forma.

Então o que aconteceu?

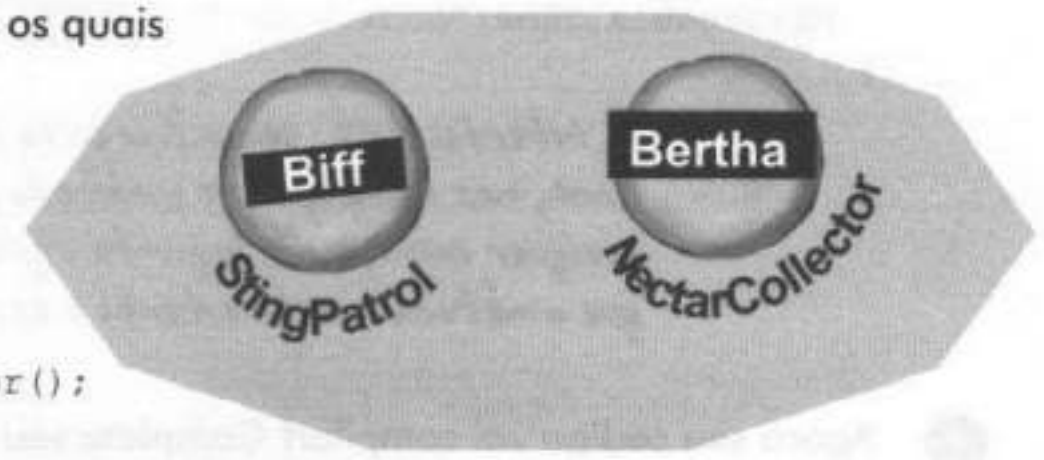
Existe apenas um único comando new, logo apenas um objeto foi criado. O segundo comando criou uma variável de referência chamada george que pode apontar para uma instância de qualquer classe que implemente IStingPatrol.

Referências de interfaces funcionam exatamente como referências de objetos

Você já sabe tudo a respeito da existência dos objetos na pilha. Quando você trabalha com uma referência de interface, isso é apenas outra forma de fazer referências aos objetos com os quais você já está lidando. Veja, é fácil!

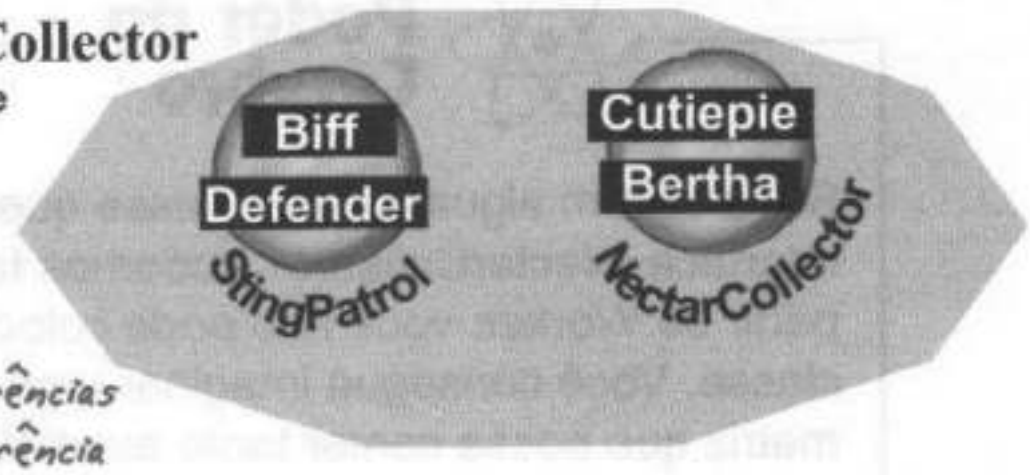
1 Crie um par de abelhas
Até aqui, tudo é bastante familiar.

```
StingPatrol biff = new StingPatrol();
NectarCollector bertha = new NectarCollector();
```



2 **Adicione referências para IStingPatrol e INectarCollector**
Você pode usar referências para interfaces exatamente como usaria qualquer outro tipo de referência.

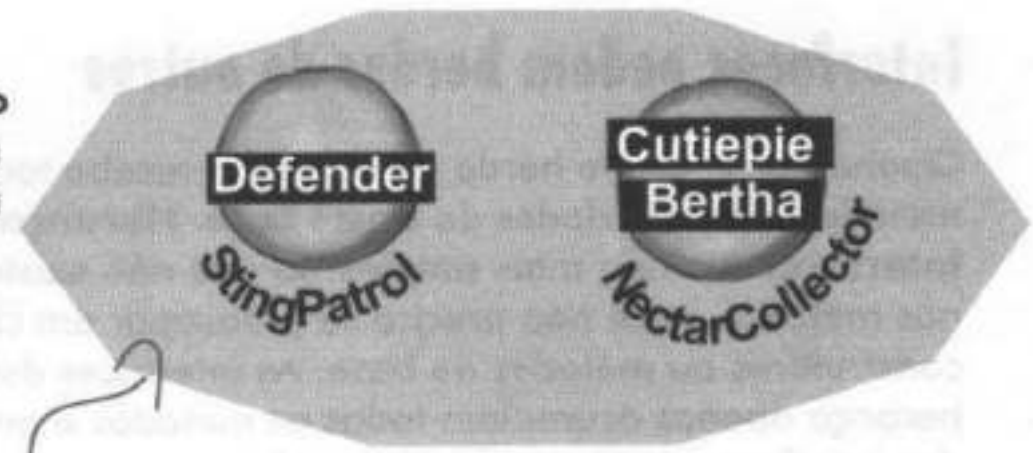
```
IStingPatrol defender = biff;
INectarCollector cutiePie = bertha;
```



Esses dois comandos usam interfaces para criar novas referências para objetos existentes. Você pode apenas apontar uma referência de interface para uma instância de uma classe que a implemente.

3 **Uma referência de interface manterá um objeto vivo**
Quando não existe mais nenhuma referência para um objeto ele desaparece. Mas não existe nenhuma regra que diga que as referências tenham que ser todas do mesmo tipo! Uma referência de interface é tão boa quanto qualquer referência de objeto no que diz respeito a manter registro a respeito dos objetos.

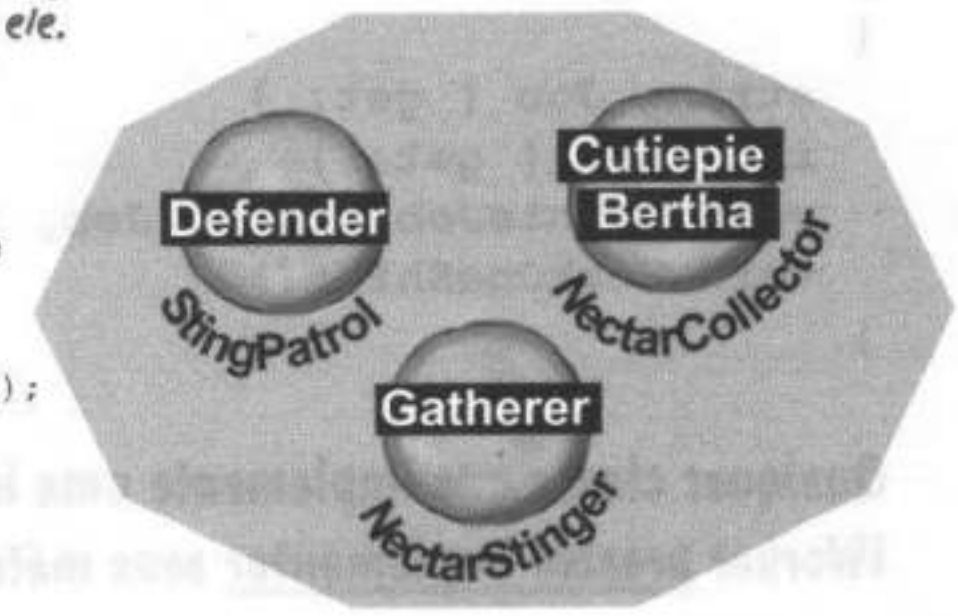
```
biff = null;
```



O objeto não desaparece porque defender ainda está apontado para ele.

4 **Atribua uma nova instância para uma referência de interface**
Você não precisa de uma referência de objeto - pode criar um objeto e atribui-lo diretamente a uma variável de referência de interface.

```
INectarCollector gatherer = new NectarStinger();
```



Você pode descobrir se uma classe implementa uma interface específica usando "is"

Algumas vezes você precisa descobrir se uma certa classe implementa uma interface. Suponha que tenhamos todas as abelhas operárias numa matriz, chamada Bees. Podemos fazer o tipo da matriz ser Worker, já que todas as abelhas operárias serão dessa classe, ou de subclasses dela.
Mas qual delas é capaz de coletar néctar? Em outras palavras, queremos saber se a classe implementa a interface INectarCollector. Podemos usar a palavra-chave **"is"** (é) para descobrir exatamente isso.

Todas as operárias estão numa matriz de tipo Worker. Usaremos "is" para determinar que tipo de operária cada elemento contém.

```
Worker[] Bees = new Worker[3];
Bees[0] = new NectarCollector();
Bees[1] = new StingPatrol();
Bees[2] = new NectarStinger();
for (int i = 0; i < Bees.Length; i++)
{
  if (Bees[i] is INectarCollector)
  {
    Bees[i].DoThisJob("Nectar Collector", 3);
  }
}
```

Temos uma matriz de abelhas operárias que são candidatas a uma missão de coletar néctar. Então iteramos pela matriz e usamos is para determinar quais delas têm os métodos e propriedades corretos para o trabalho.

Isto quer dizer: se a abelha implementa INectarCollector... faça o seguinte.

O comando is funciona como se fosse um operador de igualdade (==) para interfaces

Agora que sabemos que a abelha é uma coletora de nectar, podemos dar-lhe a tarefa de fazer isso.

Poder do Cérebro

Se você tem alguma outra classe que não herda de Worker, mas que implementa a interface INectarCollector, podemos **fazer** o mesmo! Mas já que não há herança a partir de Worker, você não pode colocá-la numa matriz com outras abelhas dessa classe. Você consegue imaginar uma forma de contornar esse problema e criar uma matriz que possa conter tanto as abelhas quanto essa nova classe?

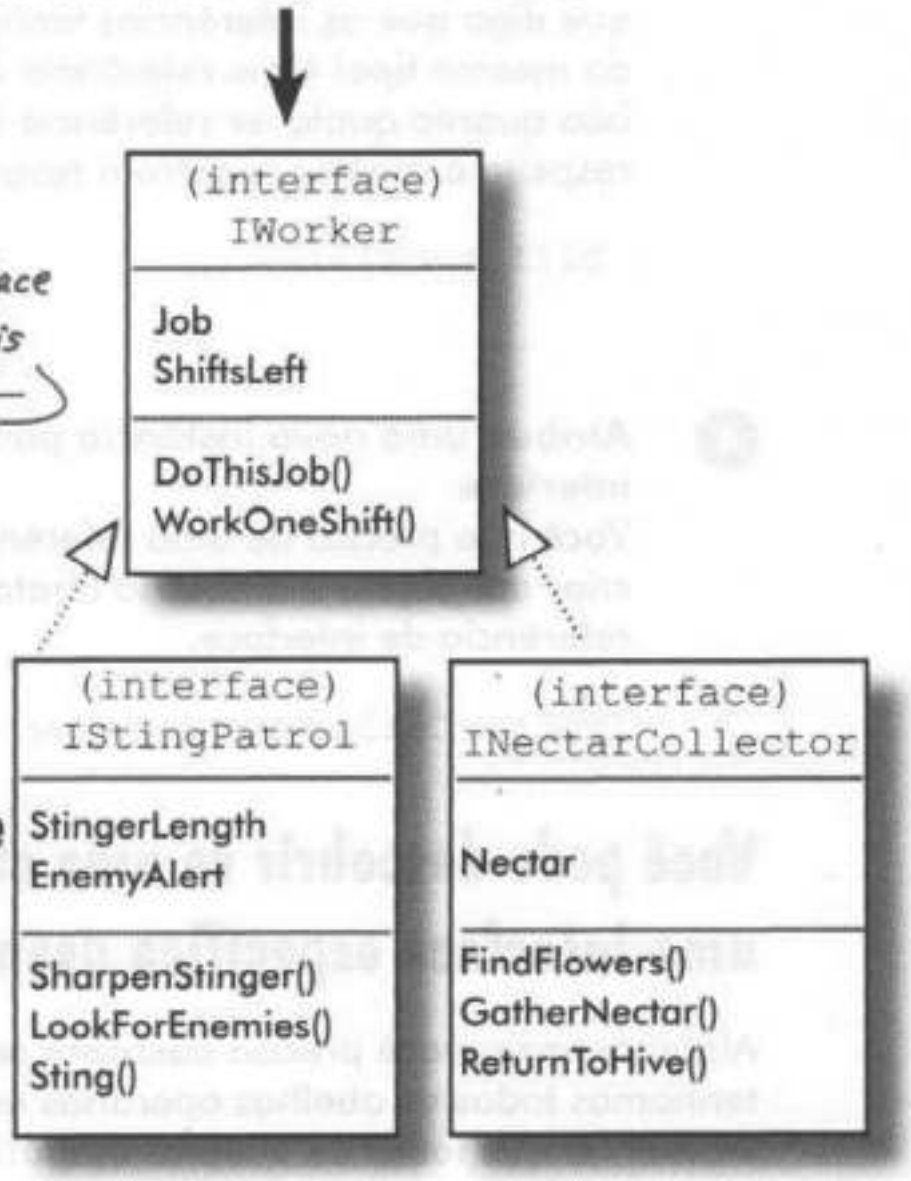
Interfaces podem herdar de outras

Quando uma classe herda de outra ela recebe todos os métodos e propriedades da classe base. **Herança entre interfaces** é ainda mais simples. Já que não existe um corpo nos métodos, você não precisa se preocupar em chamar construtores ou métodos de base. As interfaces definidas com herança apenas acumulam todos os métodos e propriedades das interfaces de hierarquia mais alta.

```
public interface IWorker
{
  string Job { get; }
  int Left { get; }
  void DoThisJob(string Job, int Shifts)
  void WorkOneShift()
}
```

Criamos uma nova interface IWorker da qual as demais devem herdar.

Quando desenharmos uma interface num diagrama de classes, mostraremos herança usando linhas pontilhadas.



Qualquer classe que implemente uma interface que herde de IWorker precisa implementar seus métodos e propriedades.

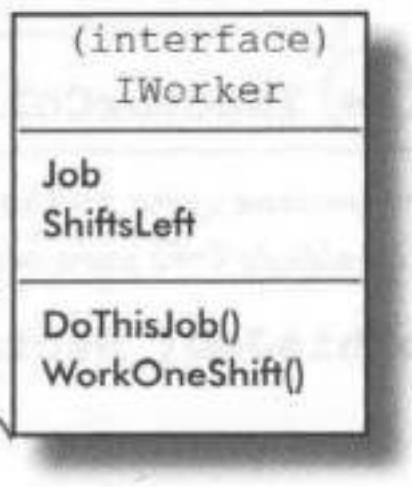
Quando uma classe implementa uma interface ela deve incluir cada propriedade e método nela. E se essa interface herda de outra então todos esses outros métodos e propriedades também devem ser implementados.

```
public interface IStingPatrol : IWorker
{
  int AlertLevel { get;}
  int StingerLength { get; set;}
  bool LookForEnemies();
  int SharpenStinger(int Length);
}
```

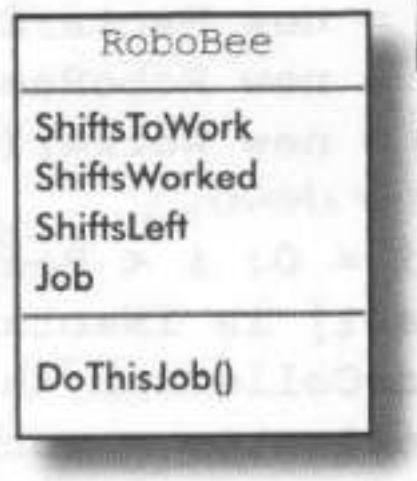
Eis aqui a mesma interface IStingPatrol, mas agora ela herda de IWorker. É apenas uma pequena mudança, mas faz uma grande diferença para a classe que implementar IStingPatrol.

Uma classe que implemente IStingPatrol não só precisa implementar esses métodos...

...mas também os métodos herdados da interface IWorker.



O RoboBee 4000 pode fazer o trabalho de uma abelha operária sem usar o precioso mel



Vamos criar uma nova abelha, a RoboBee 4000, que funcionará usando gasolina. Podemos fazer com que ela herde da interface `IWorker` tudo o que uma abelha operária faz.

Essa é a nossa classe Robo básica e os robôs podem funcionar com gasolina.

```
public class Robot
{
  public void ConsumeGas() {...}
}

public class RoboBee : Robot, IWorker
{
  private int shiftsToWork;
  private int shiftsWorked;
  public int ShiftsLeft
    {get {return ShiftsToWork - ShiftsLeft;}}
  private string job;
  public string Job;{get{return job;}}
  public bool DoThisJob(string Job, int ShiftsToWork){...}
}
```

A classe RoboBee herda de Robot e implementa IWorker. Ou seja, ela é um robô, mas pode fazer o trabalho de uma abelha operária. Perfeito!

A classe RoboBee implementa todos os métodos da interface IWorker.

Se RoboBee não implementasse tudo da interface IWorker, o código não compilaria.

Lembre-se, para as outras classes no aplicativo, não existe diferença funcional entre um RoboBee e uma abelha operária normal. Ambos implementam a interface `IWorker` então ambos agem como abelhas operárias, sob o ponto de vista do restante do programa.

Mas ainda é possível distinguir entre os tipos usando `is`:

```
if (workerBee is Robot) {
  // agora nós conhecemos o workerBee
  // este é um objeto Robot
}
```

Podemos ver quais classes ou interfaces workerBee implementa ou de quais ela deriva usando is.

Qualquer classe pode implementar QUALQUER interface desde que mantenha sua promessa de implementar todos os métodos e propriedades delas.

"is" diz a você o que um objeto implementa, "as" diz ao compilador como tratar um objeto

Algumas vezes você tem que chamar um método que um objeto tem porque implementa uma interface. Mas como saber se aquele objeto é do tipo certo? Você pode usar **is** para descobrir. Então, você pode usar **as** (que pode ser traduzido para a conjunção "como") para tratar aquele objeto - que agora você tem certeza de que é do tipo certo - como tendo aquele método que você precisa chamar.

Nome: Data:

Laboratório C#2

A Missão

Este laboratório contém uma especificação que descreve um programa para você criar, usando o conhecimento adquirido nos últimos capítulos.

Este projeto é maior do que os já vistos até agora. Então leia tudo antes de começar e reserve algum tempo para pensar. E não se preocupe se ficar preso em uma parte – não há nada novo aqui. Assim, você pode continuar lendo o livro e voltar ao laboratório mais tarde.

Preenchemos alguns poucos detalhes de projeto para você e certificamo-nos de que você tenha todas as peças necessárias... e mais nada.

É de sua responsabilidade terminar o trabalho. Você pode baixar o executável deste laboratório na nossa página de Internet... mas não daremos a você o código da resposta.

```
IWorker[] Bees = new IWorker[3];
  Bees[0] = new NectarStinger();
  Bees[1] = new RoboBee();
  Bees[2] = new Worker();
for (int i = 0; i < Bees.Length; i++) {
  if (Bees[i] is INectarCollector) {
    INectarCollector thisCollector;
    thisCollector = workers[i] as INectarCollector;
thisCollector.EmptyNectarBucket();
```

Todas essas abelhas implementam IWorker, mas não sabemos quais implementam outras interfaces, como por exemplo, INectarCollector.

Iteramos por todas as abelhas...

Não podemos chamar os métodos de INectarCollector nas abelhas. Elas são do tipo IWorker e não conhecem os métodos de INectarCollector.

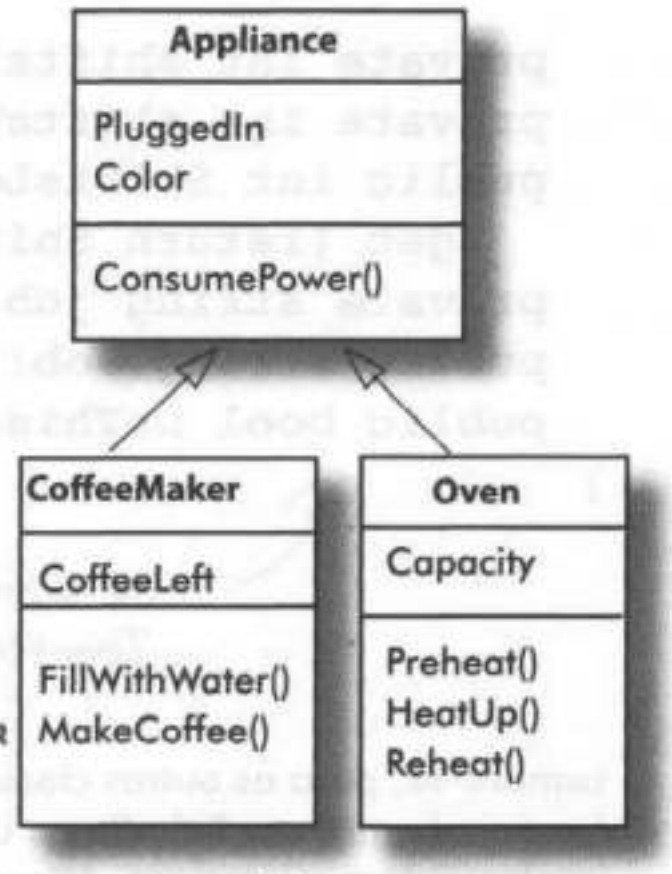
...e checamos para ver se elas implementam INectarCollector.

AGORA podemos chamar os métodos de INectarCollector.

Usamos "as" para dizer: trate este objeto COMO se ele fosse uma implementação de INectarCollector.

Uma cafeteira também é um eletrodoméstico

Se você está tentando imaginar como diminuir sua conta de energia elétrica mensal, você não se preocupará com o que cada eletrodoméstico faz e sim com a potência que eles consomem. Logo, se você estiver escrevendo um programa que monitore seu consumo de eletricidade, provavelmente, criará apenas uma classe Appliance (eletrodoméstico). Mas se você deve distinguir entre uma cafeteira e um forno, terá que montar uma hierarquia de classes. Então você adicionará os métodos e propriedades específicas de uma cafeteira ou forno em classes CoffeeMaker (cafeteira) e Oven (forno) e eles herdarão da classe Appliance que terá métodos e propriedades em comum.



Você já viu isso no capítulo anterior, quando aprendeu como passar uma referência de BLT para um método que esperasse um Sandwich.

```
public void MonitorPower(Appliance appliance) {
  // Código de acrescentar dados
  //  banco de dados consumo de energia
}

CoffeeMaker misterCoffee = new CoffeeMaker();
MonitorPower(misterCoffee);
```

Eis um método para o programa monitorar o consumo de potência numa casa.

Este código aparecerá mais tarde no programa para monitorar o consumo da cafeteira.

Mesmo que o método MonitorPower() (monitorar potência) precise de uma referência para um objeto Appliance, você pode passar uma referência a misterCoffee (senhor café) porque CoffeeMaker é uma subclasse de Appliance.

Supercoerção funciona tanto com objetos quanto com interfaces

Quando você substitui uma subclasse por uma de base - como trocar uma cafeteira por um eletrodoméstico ou um BLT por um sanduíche - você está fazendo **supercoerção** (upcasting). É uma ferramenta realmente poderosa obtida quando se criam hierarquias de classes. A única desvantagem da supercoerção é que você só pode usar as propriedades e métodos da classe base. Em outras palavras, quando você trata uma cafeteira como um eletrodoméstico, não pode dizer a ela para fazer café ou se encher de água. Mas pode dizer se ela está ou não ligada, pois isto é algo possível com qualquer eletrodoméstico (e é por isso que a propriedade PluggedIn - ligado na tomada - é parte da classe Appliance).

1 **Vamos criar alguns objetos**
Podemos criar uma cafeteira e um forno da forma tradicional.

```
CoffeeMaker misterCoffee = new CoffeeMaker();
Oven oldToasty = new Oven();
```

Começaremos criando instâncias de objetos Oven e CoffeeMaker normalmente.

2 **E o que acontece se quisermos criar uma matriz de eletrodomésticos?**
Você não pode colocar uma CoffeeMaker numa matriz Oven[], nem o inverso. Mas pode colocar ambos numa matriz Appliance[].

```
Appliance[] kitchenWare = new Appliance[2];
kitchenWare[0] = misterCoffee;
kitchenWare[1] = oldToasty;
```

Você pode usar supercoerção para criar uma matriz de eletrodomésticos que possa aceitar tanto cafeteiras quanto fornos.

3 **Mas você não pode tratar um eletrodoméstico como se fosse um forno**
Quando você recupera uma referência para um eletrodoméstico, pode **apenas** acessar os métodos e propriedades da classe Appliance. Você não pode usar os métodos e propriedades de cafeteira usando a referência a Appliance mesmo **que você tenha a certeza de com uma CoffeeMaker**. Logo, os comandos seguintes funcionarão bem, porque tratam um objeto cafeteira como um Appliance:

```
Appliance powerConsumer = new CoffeeMaker();
powerConsumer.ConsumePower();
```

powerConsumer é uma referência tipo Appliance apontando para um objeto CofeeMaker.

Mas logo que você tentar usá-la como uma CoffeeMaker:

```
powerConsumer.MakeCoffee();
```

Esta linha não compilará porque powerConsumer é uma referência tipo Appliance. Assim, só pode ser usada para fazer coisas comuns a eletrodomésticos.

Seu código não compilará e o IDE exibirá um erro:

> ✗ 'Appliance' does not contain a definition for 'MakeCoffee'

Uma vez feita a supercoerção de uma subclasse para uma classe base, você pode acessar os métodos e propriedades apenas respectivas ao **tipo de referência** que está usando para acessar o objeto.

Aponte seu lápis

Dê uma olhada na matriz à esquerda. Para cada um dos comandos à direita escreva quais valores de `i` tornariam o resultado verdadeiro (true). Além disso, dois deles não compilarão - risque as linhas onde isso acontecerá.

```
IWorker[] Bees = new IWorker[8];
Bees[0] = new NectarStinger();
Bees[1] = new RoboBee();
Bees[2] = new Worker();
Bees[3] = Bees[0] as IWorker;
Bees[4] = IStingPatrol;
Bees[5] = null;
Bees[6] = Bees[2];
Bees[7] = new INectarCollector();
```

```
1. (Bees[i] is INectarCollector)
.....
2. (Bees[i] is IStingPatrol)
.....
3. (Bees[i] is IWorker)
.....
```

Aponte seu lápis
Solução

Dê uma olhada na matriz à esquerda. Para cada um dos comandos à direita escreva quais valores de i tornariam o resultado verdadeiro (true). Além disso, dois deles não compilarão - risque as linhas onde isso acontecerá.

```
IWorker[] Bees = new IWorker[8];
Bees[0] = new NectarStinger();
Bees[1] = new RoboBee();
Bees[2] = new Worker();
Bees[3] = Bees[0] as IWorker;
Bees[4] = IStingPatrol;
Bees[5] = null;
Bees[6] = Bees[2];
Bees[7] = new INectarCollector();
```

NectarStinger() implementa a interface IStingPatrol.

Todos esses objetos herdam da classe base Worker.

1. (Bees[i] is INectarCollector)

0, 5 e 7

2. (Bees[i] is IStingPatrol)

0, 6

3. (Bees[i] is IWorker)

0, 1, 2, 3 e 6

Subcoerção deixa que você transforme seu eletrodoméstico de volta em cafeteira

A supercoerção é uma ótima ferramenta, porque permite usar uma cafeteira ou um forno em qualquer lugar onde precisaria usar um eletrodoméstico. Mas tem uma grande desvantagem - se você estiver usando uma referência a um objeto `Appliance` que aponte para um do tipo `CoffeeMaker`, pode apenas usar os métodos e propriedades que pertençam à primeira classe. E é aí que entra a **subcoerção** (downcasting): é com ela que você pega sua **referência, que previamente sofreu supercoerção**, e a transforma de volta. Você pode determinar se seu eletrodoméstico é realmente uma cafeteira usando a palavra-chave **is**. E uma vez que você tenha certeza disso, pode converter um `Appliance` de volta numa `CoffeeMaker` usando o comando **as**.

Eis nossa referência tipo Appliance que aponta para um objeto CoffeeMaker.

1 **Vamos começar com a nossa cafeteira que sofreu supercoerção.**
Eis o código que usamos:

```
Appliance powerConsumer = new CoffeeMaker();
powerConsumer.ConsumePower();
```

2 **Mas e se quisermos transformar de Appliance de volta em CoffeMaker?**
O primeiro passo para a subcoerção é usar a palavra-chave is para checar se ela é possível.

A referência javaJoe aponta para o mesmo objeto CoffeeMaker que powerConsumer. Mas como é uma referência tipo CoffeeMaker ela pode chamar o método MakeCoffee().

```
if (powerConsumer is CoffeeMaker)
   // then we can downcast!
```

3 **Agora que sabemos que é uma Cafeteira, vamos usá-la como se fosse uma.**
A palavra-chave `is` é o primeiro passo. Uma vez que você saiba uma referência tipo `Appliance` está apontando para um objeto tipo `CoffeeMaker`, pode usar "as" para fazer a subconversão. E isso permite que você use os métodos e propriedades da classe `CoffeeMaker`. E já que `CoffeeMaker` herda de `Appliance`, também os métodos e propriedades dela podem ser usados.

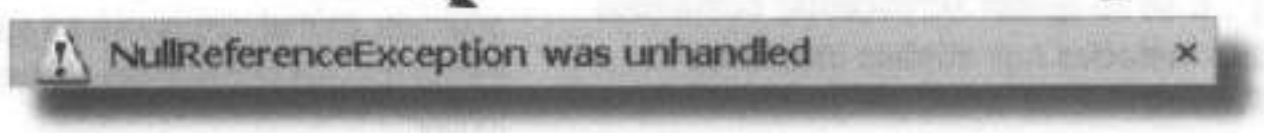
```
if (powerConsumer is CoffeeMaker) {
  CoffeeMaker javaJoe = powerConsumer as CoffeeMaker;
  javaJoe.MakeCoffee();
}
```

Quando a subcoerção falha, "as" retorna null

E o que acontece se você tentar converter um objeto tipo `Oven` numa `CoffeeMaker`? O valor de retorno será null - e se você tentar usá-lo, o .NET fará seu programa parar.

```
if (powerConsumer is CoffeeMaker) {
    Oven foodWarmer = powerConsumer as Oven;
    foodWarmer.Preheat();
}
```

Ops! Isso não está certo!



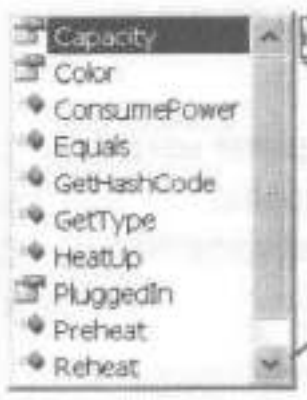
O objeto powerConsumer NÃO é do tipo Oven. Então quando você tenta fazer subcoerção com "as", a referência foodWarmer (aquecedor de comida) acaba com null como valor. E quando você tentar usar uma referência nula, isso acontece...

Supercoerção e subcoerção funcionam com interfaces, também

Você já sabe que `is` e `as` funcionam com interfaces. Bem, também funcionam todos os truques de supercoerção e subcoerção. Vamos adicionar uma classe `ICooksFood` (prepara comida) para qualquer classe que possa aquecer comida. E criaremos também uma classe `Microwave` (microondas) - e tanto `Microwave` quanto `Oven` implementarão essa interface. Agora temos três formas diferentes de acessar um objeto `Oven`. E o IntelliSense do IDE pode ajudá-lo a determinar exatamente o que você pode e o que não pode fazer com cada um deles:

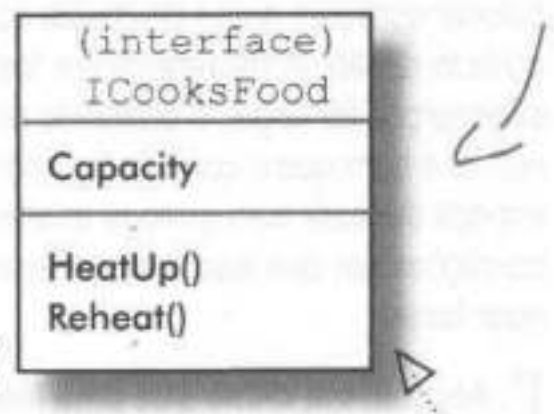
```
Oven misterToasty = new Oven();
misterToasty.
```

Assim que você digitar o ponto, a janela do IntelliSense aparecerá com uma lista de todos os membros que você pode usar.

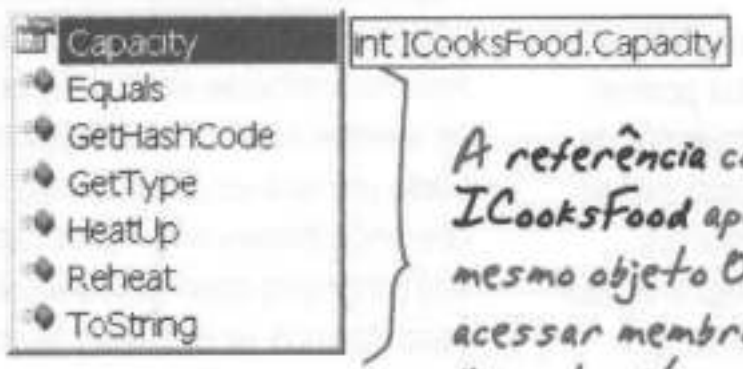


misterToasty é uma referência tipo Oven apontando para um objeto Oven. Então podemos acessar todos os métodos e propriedades... mas é o tipo menos genérico, logo só se pode apontar para objetos desse tipo.

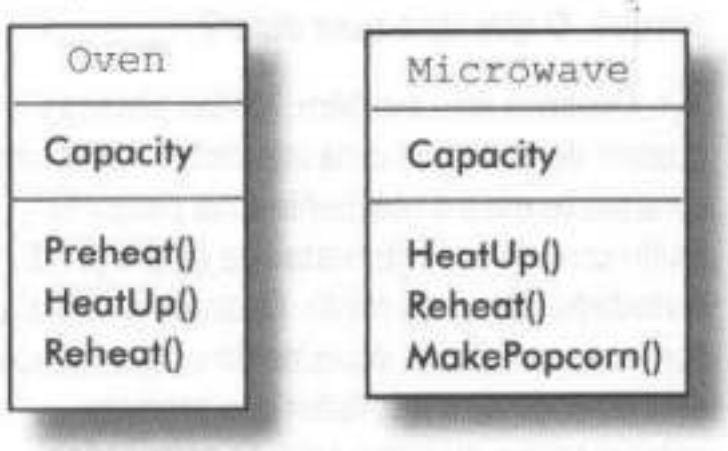
Qualquer classe que implemente ICooksFood é um eletrodoméstico que pode aquecer comida.




```
ICooksFood cooker;
if (misterToasty is ICooksFood)
  cooker = misterToasty as ICooksFood;
  cooker.
```



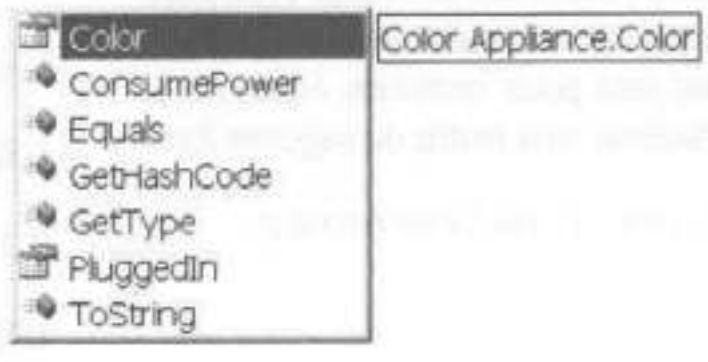
A referência cooker é do tipo ICooksFood apontando para o mesmo objeto Oven. Pode-se apenas acessar membros da interface. Mas também podemos apontar para objetos Microwave.



Três diferentes referências apontando para o mesmo objeto Oven acessam diferentes métodos e propriedades, dependendo do tipo da referência.

```
Appliance powerConsumer;
if (misterToasty is Appliance)
  powerConsumer = misterToasty;
  powerConsumer.
```

A referência powerConsumer é do tipo Appliance. Ela pode apenas acessar os campos públicos, métodos e propriedades de Appliance. Você pode apontá-la também para uma CoffeeMaker se quiser.



não existem Perguntas Idiotas

P: Volte um pouco - você me disse que eu sempre posso usar supercoerção, mas nem sempre subcoerção. Por quê?

R: Porque o compilador pode avisar quando a supercoerção está errada. A única forma de uma supercoerção não funcionar é se você estiver tentando atribuir a um objeto uma classe da qual ele não herde, ou uma interface que ele não implemente. E o compilador pode perceber imediatamente que você não fez a supercoerção corretamente e mostrar um erro.
Por outro lado, o compilador não pode saber como checar se você está fazendo subcoerção a partir de uma referência a objeto ou interface e resultar numa referência inválida. Isso porque é perfeitamente possível colocar qualquer nome de classe ou interface no lado direito da palavra-chave "as". Se a subcoerção for ilegal, o comando retornará null. E é bom que o compilador não te impeça de fazer isso, porque existem muitas condições em que isso é exatamente o que se quer fazer.

P: Alguém me disse que uma interface é como um contrato, mas não entendi bem o porquê. O que isso quer dizer?

R: Ouvimos isso também. Muitas pessoas gostam de dizer que uma interface é como um contrato (e essa é realmente uma pergunta muito comum em entrevistas de emprego). E é verdade, até certo ponto. Quando você faz com que sua classe implemente uma interface está dizendo ao compilador que promete colocar certos métodos nela. O compilador forçará você a manter a sua promessa.
Mas achamos que é mais fácil de lembrar como as interfaces funcionam se você pensar nelas como uma tipo de lista de verificação. O compilador confere na lista para ter certeza de que você realmente colocou todos os métodos da interface na sua classe. Se não for o caso ele reclamará e não deixará que você continue.

P: E se eu quiser acrescentar comandos aos métodos nas minhas interfaces? Posso?

R: Não, o compilador não deixará você fazer isso. Uma interface não tem permissão de conter nenhum comando. Mesmo que você use o operador dois pontos para implementar uma interface, não é a mesma coisa que herdar de uma classe. Implementar uma interface não acrescenta nenhum comportamento à sua classe ou a altera de qualquer forma. Tudo o que é feito é dizer ao compilador para se certificar de que a classe tenha os métodos que a interface diz que ela deveria ter.

P: Então porque eu deveria querer usar interfaces? Parece que elas apenas adicionam restrições, sem efetivamente mudar minha classe em nada.

R: Porque quando sua classe implementa uma interface, uma referência com o mesmo tipo dela pode apontar para qualquer instância daquela classe. E isso é o realmente útil – permitir criar um tipo de referência que pode funcionar com vários tipos diferentes de objetos.
Eis um exemplo rápido. Um cavalo, um boi, uma mula e um novilho, todos podem puxar uma carroça. Mas num simulador de zoológico, cada um deles seria uma classe diferente. Digamos que você tenha um passeio de carrinho no seu zoológico e quer criar uma matriz de qualquer animal que possa ser usado para isso. Ops!Você não pode simplesmente criar uma matriz que armazene todas essas classes. Se todas elas herdassem da mesma classe base, você poderia criar uma matriz desse tipo. Mas vamos supor que elas não herdem. O que fazer?
É aí que as interfaces são úteis. Você pode criar uma interface Ipuller (puxador) com métodos para puxar carrinhos. Agora você pode declarar uma matriz da seguinte forma:

```
IPuller[] pullerArray;
```

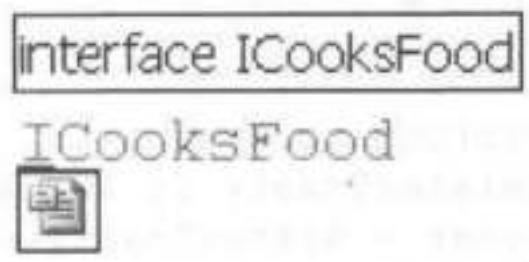
Agora você pode colocar uma referência a qualquer animal que queira na sua matriz, desde que ela implemente a interface Ipuller.

P: Existe uma forma mais fácil de implementar interfaces? São várias coisas para digitar!

R: E veja só existe! O IDE disponibiliza para você um atalho muito poderoso que automaticamente implementa uma interface. Comece a digitar a sua classe:

```
public class
  Microwave : ICooksFood
  { }
```

Clique em IcooksFood - você verá uma pequena barra aparecer debaixo do "I". Coloque o mouse sobre ela e um ícone surgirá.

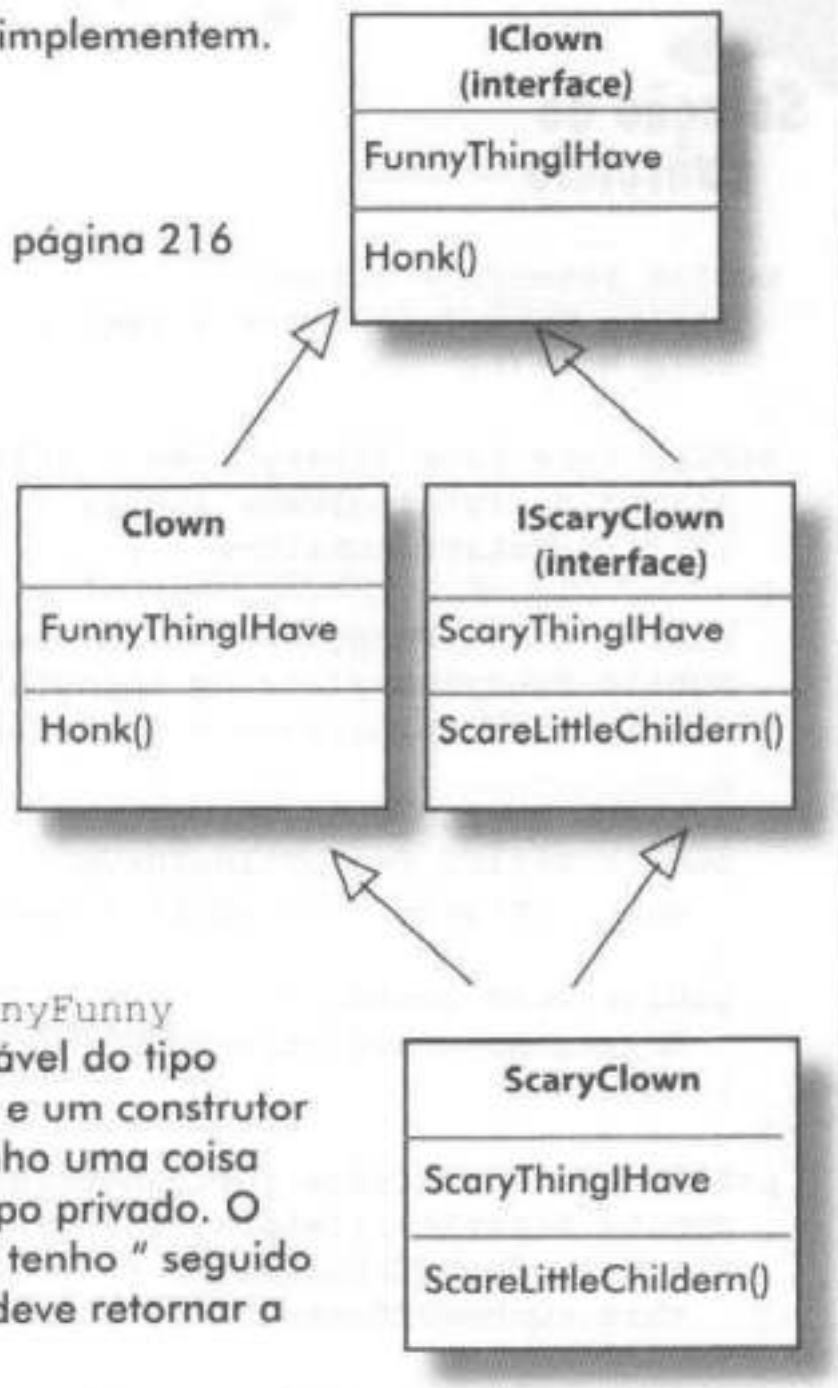


Clique nele e escolha "Implement Interface 'ICooksFood'" do menu que aparecerá. Automaticamente serão adicionados todos os membros ainda não implementados. Cada um terá uma única linha com o comando throws nela - eles fazem com que seu programa pare, como um lembrete, no caso de você se esquecer de implementar algum deles (você aprenderá mais sobre esse comando no capítulo 10).

Uma interface é como uma lista de checagem que o compilador verifica para ter certeza de que sua classe implementou um certo conjunto de métodos.

Exercício

Estenda a interface IClown e use classes que a implementem.



1. Comece com a interface `IClown` do último "Faça Isso!", na página 216

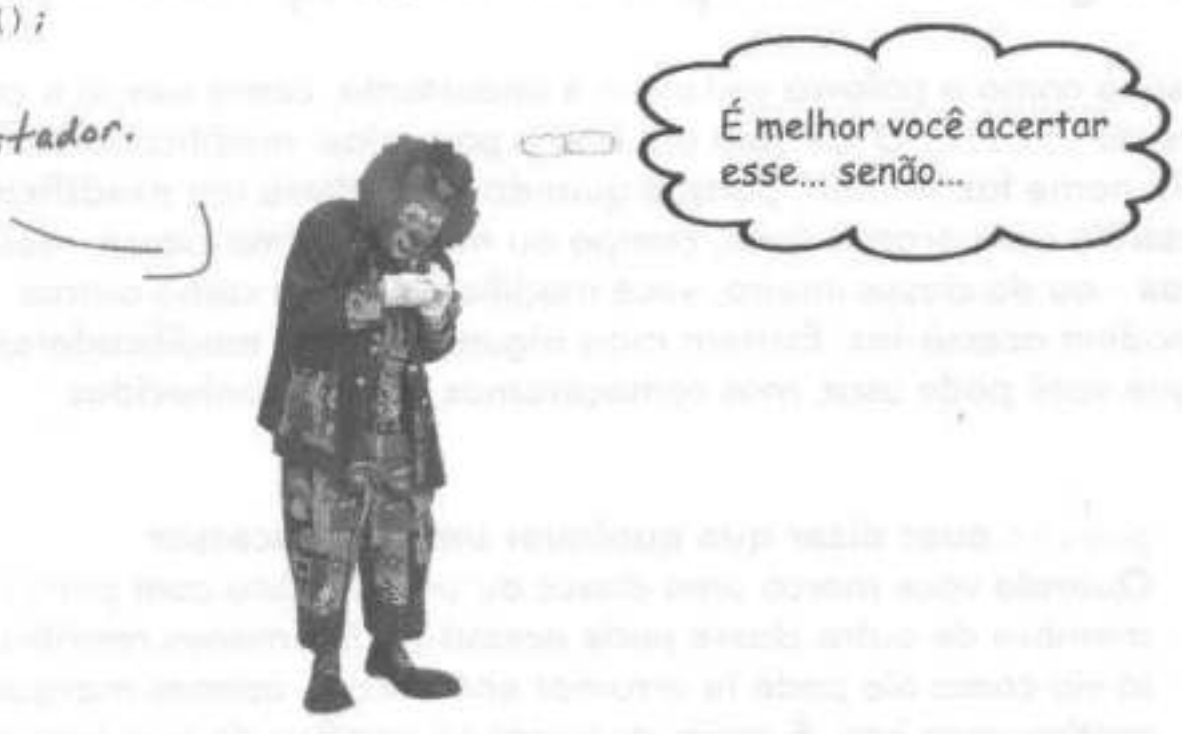
```
public interface IClown {
  string FunnyThingIHave { get; }
  void Honk();
}
```

2. Estenda `IClown` criando uma nova interface, `IScaryClown`, que herde de `IClown`. Ela deve ter uma propriedade do tipo `string` adicional chamada `ScaryThingIHave` (eu tenho uma coisa assustadora) com um acessador get, mas não um set, e um método `void` chamado `ScrayLittleChildren()` (assuste as criancinhas).

3. Crie as seguintes classes:
   - ★ Uma classe para palhaços engraçados chamada `FunnyFunny` (literalmente "divertido divertido"), que use uma variável do tipo string privada para armazenar uma coisa engraçada e um construtor que aceite um parâmetro `funnyThingIHave` (eu tenho uma coisa engraçada) – usado para atribuir o valor para o campo privado. O método `Honk()` (buzine) deve mostrar "Fom, fom! Eu tenho " seguido pela coisa engraçada que ele tem. O acessador get deve retornar a mesma coisa.
   - ★ Uma classe de palhaço assustador chamada `ScaryScary` (literalmente "assustador assustador") que use uma variável privada para armazenar um número inteiro – passado a ela pelo seu construtor num parâmetro chamado `numberOfScaryThings` (número de coisas assustadoras). O acessador get de `ScaryThingIHave` deve retornar uma seqüência de caracteres que consiste do número fornecido ao construtor seguido de "aranhas". O método `Honk()` deve mostrar uma mensagem que diz "Buh! Peguei vocês!"

4. Eis o código para um botão - mas ele não está funcionando. Você consegue descobrir como corrigir isso?

```
private void button1_Click(object sender, EventArgs e) {
  ScaryScary fingersTheClown = new ScaryScary("big shoes", 14);
  FunnyFunny someFunnyClown = fingersTheClown;
  IScaryClown someOtherScaryclown = someFunnyClown;
  someOtherScaryclown.Honk();
}
```



Solução do Exercício

Estenda a interface IClown e use classes que a implementem.

```
public interface IClown {
  string FunnyThingIHave { get; }
  void Honk();
}
public interface IScaryClown : IClown {
  string ScaryThingIHave { get; }
  void ScareLittleChildren();
}
  public class FunnyFunny : IClown {
  public FunnyFunny(string funnyThingIHave) {
    this.funnyThingIHave = funnyThingIHave;
  }
  private string funnyThingIHave;
  public string FunnyThingIHave {
    get { return "Honk honk! I have " + funnyThingIHave; }
  }
  public void Honk() {
    MessageBox.Show(this.FunnyThingIHave);
  }
}
public class ScaryScary : FunnyFunny, IScaryClown {
  public ScaryScary(string funnyThingIHave, int numberOfScaryThings)
    : base(funnyThingIHave) {
    this.numberOfScaryThings = numberOfScaryThings;
  }
  private int numberOfScaryThings;
  public string ScaryThingIHave {
    get { return "I have " + numberOfScaryThings + " spiders"; }
  }
  public void ScareLittleChildren() {
    MessageBox.Show("Boo! Gotcha!");
  }
}
private void button1_Click(object sender, EventArgs e) {
  ScaryScary fingersTheClown = new ScaryScary("big shoes", 14);
  FunnyFunny someFunnyClown = fingersTheClown;
  IScaryClown someOtherScaryclown = someFunnyClown as ScaryScary;
  someOtherScaryclown.Honk();
}
```

O método Honk(), para mostrar essa mensagem, usa apenas o acessador get - não é preciso escrever o mesmo código duas vezes.

Você poderia ter implementado o método e a propriedade de IClown de novo, mas porque não herdar de FunnyFunny?

Já que ScaryScary é uma subclasse de FunnyFunny e esta implementa IClown, ScaryScary também implementa essa interface.

Você pode atribuir para uma referência tipo FunnyFunny um objeto do tipo ScaryScary porque esta última herda da primeira. Mas você não pode fazer uma referência de tipo IScaryClown apontar para qualquer palhaço, porque você não sabe de antemão se um palhaço é assustador ou não. É por isso que você precisa usar a palavra-chave as.

Nesses lugares mais do que exatamente publico e privado

Você já sabe como a palavra `private` é importante, como usá-la e como ela difere de `public`. O C# tem um nome para elas: modificadores de acesso. O nome faz sentido, porque quando você altera um **modificador de acesso** de uma propriedade, campo ou método numa classe - seus **membros** - ou da classe inteira, você modifica a forma como outras classes podem acessá-los. Existem mais alguns tipos de modificadores de acesso que você pode usar, mas começaremos com os conhecidos:

Chamamos os métodos, campos e propriedades de uma classe como seus **membros**. Qualquer membro pode ser marcado com os modificadores de acesso public ou private.

`public` quer dizer que qualquer um pode acessar

Quando você marca uma classe ou um membro com `public` está dizendo ao C# que qualquer membro de outra classe pode acessá-lo. É o menos restritivo dos modificadores de acesso. Você já viu como ele pode te arrumar encrencas - apenas marque membros como públicos se tiver um motivo para isso. É assim que você se certifica de que suas classes estão bem encapsuladas.

- `private` **quer dizer que apenas outros membros podem acessar**
  Quando você marca um membro de uma classe como `private` ele só poderá ser acessado por outros membros dentro daquela classe ou por **outra instância da mesma classe**. E quando você marca uma classe como `private` ela só pode ser usada por outras dentro de um namespace - exceto se a classe for **definida dentro de outra** e, nesse caso, a classe `private` estará disponível apenas para a classe externa.

- `protected` **quer dizer público para subclasses, privado para todos os demais**
  Você já viu como uma subclasse está impossibilitada de acessar os membros privados de uma classe base - a primeira tem que usar a palavra-chave `base` para obter os membros públicos desta última. Não seria conveniente se as subclasses pudessem acessar campos privados? É para isso que existe o modificador de acesso `protected` (protegido). Qualquer membro de classe marcado com protected pode ser acessado por qualquer outro da mesma classe e por qualquer membro de uma subclasse.

- `internal` **quer dizer público apenas para outras classes num assembly**
  As classes do Framework .NET estão armazenadas em unidades que individualmente são chamadas de **assembly** (literalmente "montagem"). Elas são bibliotecas de classes que aparecem na lista de referências de seu projeto. Você pode ver uma lista deles clicando com o botão direito em "References" (referências) no Navegador de Solução e escolhendo "Add references..." (adicionar referências). Quando você cria um novo aplicativo Windows Forms, o IDE automaticamente inclui as referências de que você precisa. Quando você cria um assembly, no entanto, pode usar a palavra-chave `internal` para manter classes privadas dentro desse assembly, de forma a poder expor somente as classes que desejar.

- `sealed` **quer dizer que uma classe não pode ter subclasses**
  Há algumas classes que não devem ser herdadas. Muitas das do Framework .NET são assim. Vá em frente, tente fazer uma classe que herde de `String` (é a classe com o método IsEmptyOrNull() usada no capítulo anterior). O que acontece? O compilador não permitirá que seu código compile. Ele apresenta um erro "cannot derive from sealed type 'string' " (não é possível derivar do tipo selado 'string'). Você pode fazer o mesmo com suas classes - apenas adicione `sealed` (selada, vedada) depois do modificador de acesso.

Os modificadores de acesso mudam o escopo

Vamos dar uma olhada de perto nos modificadores de acesso e como eles afetam o **escopo** de vários membros de classes. Fizemos duas mudanças: o campo de armazenamento funnyThingIHave é agora `protected`; e modificaremos o método ScareLittleChildren() para usar esse campo.

Faça essas duas mudanças na solução do seu exercício. Então altere o modificador de acesso de protected de volta para private e veja que erros aparecem.

A palavra-chave "this" também muda o escopo de uma variável. Ela diz ao C# olhe na instância atual da classe para ver se acha qualquer coisa que tenha associado - mesmo que o nome seja o mesmo de um parâmetro ou variável local.

Essa é uma forma muito comum de usar o "this" já que o parâmetro e o campo de armazenamento têm o mesmo nome. O termo funnyThingIHave refere-se ao parâmetro e this.funnyThingIHave ao campo de armazenamento.

```
public interface IClown {
  string FunnyThingIHave { get; }
  void Honk();
}
public interface IScaryClown : IClown {
  string ScaryThingIHave { get; }
  void ScareLittleChildren();
}
public class FunnyFunny : IClown {
  public FunnyFunny(string funnyThingIHave) {
    this.funnyThingIHave = funnyThingIHave;
  }
  protected string funnyThingIHave;
  public string FunnyThingIHave {
    get { return "Honk honk! I have " + funnyThingIHave; }
  }
  public void Honk() {
    MessageBox.Show(this.FunnyThingIHave);
  }
}
```

Ao adicionar "this", dizemos ao C# que estamos falando sobre o campo de armazenamento, não o parâmetro com o mesmo nome.

Mudamos aqui para protected. Experimente para ver como isso afeta o método ScaryScary. ScaryLittleChildren()

Quando você usa "this" com uma propriedade, isso diz ao C# para executar os acessadores get ou set.

Modificadores de Acesso de Perto

```
public class ScaryScary : FunnyFunny, IScaryClown {
  public ScaryScary(string funnyThingIHave,
                    int numberOfScaryThings)
    : base(funnyThingIHave) {
    this.numberOfScaryThings = numberOfScaryThings;
  }
  private int numberOfScaryThings;
  public string ScaryThingIHave {
    get { return "I have " + numberOfScaryThings + " spiders"; }
  }
  public void ScareLittleChildren() {
    MessageBox.Show("You can't have my "
                    + base.funnyThingIHave);
  }
}
```

numberOfScaryThings é private, o que é típico em campos de armazenamento. Logo, apenas outra instância de ScaryScary seria capaz de vê-lo.

A palavra-chave protected diz ao C# para fazer com que algo seja **inacessível** para todos, menos para instâncias de uma subclasse.

A palavra-chave base diz ao C# para usar os valores da classe base. É outra forma de mudar o escopo.

Se deixássemos funnyThingIHave como private, o compilador exibiria um erro. Mas quando mudamos para protected, ele torna-se visível para qualquer subclasse de FunnyFunny.

```
private void button1_Click(object sender, EventArgs e) {
  ScaryScary fingersTheClown = new ScaryScary("big shoes", 14);
  FunnyFunny someFunnyClown = fingersTheClown;
  IScaryClown someOtherScaryclown = someFunnyClown as ScaryScary;
  someOtherScaryclown.Honk();
}
```

Já que esse tratador de evento clique do botão não é parte de FunnyFunny e nem de ScaryScary ele não pode acessar o campo protegido funnyThingIHave.

Estamos fora das duas classes; os comandos aqui só têm acesso aos membros públicos de objetos do tipo FunnyFunny ou ScaryScary.

não existem Perguntas Idiotas

P: Por que eu usaria interfaces, em vez de apenas escrever todos os métodos necessários diretamente na minha classe?

R: Você pode acabar com vários classes diferentes conforme escreve código para programas cada vez mais complexos. As interfaces permitem criar grupos baseando-se no tipo de trabalho feito. Elas ajudam-no a certificar-se de que cada classe fará um determinado tipo de trabalho usando os mesmos métodos. A classe pode fazer o trabalho de qualquer forma necessária e, por causa da interface, você não precisa se preocupar em entender como ela completa certas tarefas.

Eis um exemplo: você pode ter uma classe Caminhao e uma Barco que implementem `ICarregarPassageiros`. Digamos que ela estipule que qualquer classe que a implemente deva ter um método `ConsomeEnergia()`. Seu programa pode usar ambas as classes para carregar passageiros, mesmo que o método do `Barco` use energia oriunda do vento e o do caminhão use óleo diesel.

Imagine que você não tivesse a interface. Seria difícil informar ao seu programa quais veículos podem carregar pessoas e quais não. Você teria que examinar cada classe que seu programa poderia usar e tentar descobrir se existe ou não um método que sirva para carregar pessoas. Daí, teria que chamar quaisquer métodos específicos de cada veículo que estejam definidos para fazer isso. E uma vez que não existe uma interface para servir como padrão, esses métodos podem ter qualquer nome, ou a funcionalidade pode estar oculta dentro de outros métodos. Você pode ver como isso pode muito rapidamente se tornar bastante confuso.

P: Porque eu preciso usar uma propriedade? Porque não posso usar somente campos?

R: Boa pergunta. Uma interface apenas define a forma pela qual uma classe deve realizar uma determinada tarefa. Ela não é um objeto em si mesma, de forma que não pode ser instanciada e nem armazenar informações. Se você adicionasse um campo isso seria como uma declaração de variável e o C# teria que armazenar esse dado em algum lugar - e a interface não tem como fazer isso por si mesma.

P: **Qual a diferença entre uma referência comum de objeto e uma referência de interface?**

R: Você já sabe como uma referência a um objeto comum funciona. Se você criar uma instância de `Skate` chamada `ManobrasVerticais` e outra referência para esse mesmo objeto chamada `ManobrasNoHalfPipe`, os dois apontarão para a mesma coisa. Mas se `Skate` implementar a interface `IManobrasNaRua` e você criar uma referência de interface para `Skate` chamada `ManobrasDeRua` ela só terá acesso aos métodos na classe `Skate` que constem na interface.

Todas as três referências do exemplo estariam apontando para o mesmo objeto.

Se você fizesse uma chamada para o objeto usando as referências `ManobrasVerticais` ou `ManobrasEmHalfPipe`, conseguiria acessar qualquer método ou propriedade no objeto. Se fizer a chamada usando a referência `ManobrasDeRua`, pode apenas acessar métodos e propriedades da interface.

P: **Então porque eu usaria uma referência de interface se ela limita as possibilidades do objeto?**

R: uma referência de interface dá a você uma forma de lidar com vários tipos diferentes de objetos que fazem as mesmas coisas. Você pode criar uma matriz usando o tipo da referência à interface, permitindo-lhe passar e receber informações dos métodos de, no exemplo anterior, `ICarregarPassageiros` – independente de você trabalhar com um objeto Caminhao, Cavalo, Monociclo, Carro etc. A forma como cada um desses objetos faz esse trabalho é provavelmente um tanto diferente, mas com referências de interface, você sabe que todos eles têm os mesmos métodos com os mesmos parâmetros e que retornam os mesmos tipos de dados. Assim, você pode chamá-los e passar informações para eles exatamente da mesma forma.

P: **Porque eu desejaria fazer com que algo fosse protegido, em vez de público ou privado?**

R: Porque isso ajuda a encapsular melhor suas classes. Muitas vezes uma subclasse precisa acessar alguma parte interna de sua classe base. Por exemplo, se você precisar substituir uma propriedade é bastante comum usar o campo de armazenamento da classe base no acessador get, para retornar algum tipo de variação sobre o valor original. E quando você desenvolve classes, deve deixar algo público somente se houver algum motivo. Usar o modificador de acesso protected permite-lhe expor para a classe derivada somente o necessário e manter privado o restante de todos.

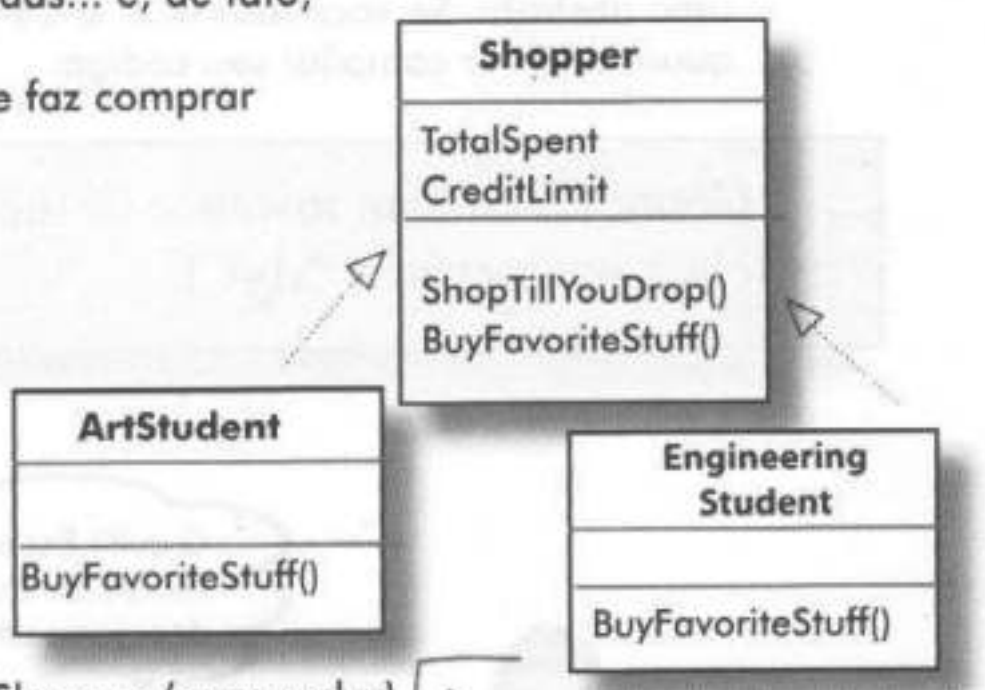
Referencias de interfaces conhecem apenas os métodos e propriedades que integram sua lista de definições.

Algumas classes nunca deveriam ser instanciadas

Lembra da nossa hierarquia de classes do simulador de zoológico? Nós, certamente, instanciamos vários hipopótamos, cães e leões. Mas o que dizer a respeito das classes Canine (canídeo) e Feline (felino)? E a classe Animal? Parece que há algumas classes que não precisam ser instanciadas... e, de fato, não faz nenhum sentido que elas sejam. Veja um exemplo:

Vamos começar com uma classe básica para um estudante que faz comprar numa livraria estudantil.

```
public class Shopper {
    public void ShopTillYouDrop()
        while (TotalSpent < CreditLimit)
            BuyFavoriteStuff();
    }
public virtual void BuyFavoriteStuff () {
      // sem implementação aqui - não sabemos
     // Aquilo que os nossos alunos gostam de comprar!
    }
}
```



Eis a classe ArtStudent (estudante de arte) - ela é derivada de Shopper (comprador).

```
public class ArtStudent : Shopper {
    public override void BuyFavoriteStuff () {
        BuyArtSupplies();
        BuyBlackTurtlenecks();
        BuyDepressingMusic();
    }
}
```

E a classe EngineeringStudent (estudante de engenharia) também herda de Shopper.

```
public class EngineeringStudent : Shopper {
    public override void BuyFavoriteStuff () {
        BuyPencils();
        BuyGraphingCalculator();
        BuyPocketProtector();
    }
}
```

Então o que acontece quando se instancia Shopper? Faz algum sentido fazer isso?

Uma classe abstrata é como um cruzamento entre uma classe e uma interface

Imagine: você precisa de algo como uma interface; as classes que a implementem têm certos métodos e propriedades. Mas você precisa incluir algum código nessa interface, de forma que alguns dos métodos não precisariam ser implementados cada vez que uma classe herda os membros. O que você quer é uma **classe abstrata**. Você tem as características de uma interface, mas pode incluir código como numa classe normal.

Um método com uma declaração, mas sem comandos nem corpo é chamado de método abstrato. Classes que o herdem precisam implementá-los, da mesma forma como quando herdam de interfaces.

Apenas classes abstratas podem ter métodos abstratos. Se você colocar um deles numa classe, terá que marcá-la como abstrata ou o código não compilará. Você aprenderá mais sobre como marcar uma classe como abstrata em alguns minutos.

- **Uma classe abstrata é <u>como</u> uma classe normal**
  Você define uma classe abstrata como faz com uma comum. Ela tem campos e métodos e pode-se herdar de outras classes também, exatamente como quando se faz uma classe normal. Há quase nada de novo para aprender aqui, porque você já sabe tudo a respeito do que uma classe abstrata faz!

- **Uma classe abstrata é <u>como</u> uma interface**
  Quando cria uma classe que implementa uma interface, você concorda em implementar todas as propriedades e métodos nela definidos. Uma classe abstrata funciona da mesma forma - ela pode incluir declarações de propriedades e métodos que, assim como os de uma interface, precisam ser implementados por qualquer classe derivada.

O oposto de uma classe **abstrata** é uma concreta. Um método concreto possui um corpo e todas as classes com as quais você tem trabalhado até agora são concretas.

- **Mas uma classe abstrata não pode ser instanciada**
  A maior diferença entre uma classe **concreta** e uma **abstrata** é que você não pode usar new para criar uma instância de uma abstrata. Se você fizer isso, o C# mostrará a você um erro quando tentar compilar seu código.

Esse erro ocorre porque você tem métodos abstratos sem código algum! O compilador não deixará você instanciar uma classe com código faltando, da mesma forma que não deixará instanciar uma interface.

> Connot create an instance of the abstract class or interface 'MyClass'

O quê? Espere! Uma classe que eu não posso instanciar? Porque eu desejaria uma coisa dessas?

Porque você quer deixar algum código já disponível, mas ainda quer que as subclasses implementem o restante do código.

Algumas vezes coisas ruins acontecem quando você cria objetos que nunca deveriam ser criados. As classes no topo de seus diagramas de classe normalmente têm alguns campos que esperam que apenas suas subclasses usem. Uma classe Animal pode ter um cálculo que dependa de um booleano chamado HasTail (tem cauda) ou Vertebrate (vertebrado), mas ela obviamente não pode dar valores para esses campos por si mesma.

Aqui está um exemplo...

Eis uma classe que o Clube de Astrofísica Vila Objeto usa para enviar seus foguetes para diferentes planetas.

Não faz sentido dar valores a esses campos na classe base, porque não sabemos quais foguetes serão usados para ir para quais planetas.

```
public class PlanetMission {
  public long RocketFuelPerMile;
  public long RocketSpeedMPH;
  public int MilesToPlanet;
  public long UnitsOfFuelNeeded() {
    return MilesToPlanet * RocketFuelPerMile;
  }
  public int TimeNeeded() {
    return MilesToPlanet / RocketSpeedMPH;
  }
public string FuelNeeded() {
  return "You'll need "
    + MilesToPlanet * RocketFuelPerMile
    + " units of fuel to get there. It'll take "
    + TimeNeeded() + " hours.";
  }
}
```

Os astrofísicos têm duas missões - uma para Marte e outra para Vênus.

```
public class Venus :PlanetMission {
  public Venus() {
    MilesToPlanet = 40000000;
    RocketFuelPerMile = 100000;
    RocketSpeedMPH = 25000;
  }
}
public class Mars : PlanetMission {
  public Mars() {
    MilesToPlanet = 75000000;
    RocketFuelPerMile = 100000;
    RocketSpeedMPH = 25000;
  }
}
```

Os construtores para as subclasses Mars (Marte) e Venus dão valores para os três campos que herdam de Planet. Mas esses campos não serão usados se você instancia Planet diretamente. Então o que aconteceria, nesse caso, se tentássemos usar FuelNeeded() (combustível necessário)?

```
private void button1_Click(object s, EventArgs e) {
  MarsMission mars = new Mars();
  MessageBox.Show(mars.FuelNeeded());
}
private void button2_Click(object s, EventArgs e) {
  VenusMission venus = new Venus();
  MessageBox.Show(venus.FuelNeeded());
}
private void button3_Click(object s, EventArgs e) {
  PlanetMission planet = new PlanetMission();
  MessageBox.Show(planet.FuelNeeded());
}
```

Antes que você vire a página, tente descobrir o que acontecerá quando o usuário clicar no terceiro botão...

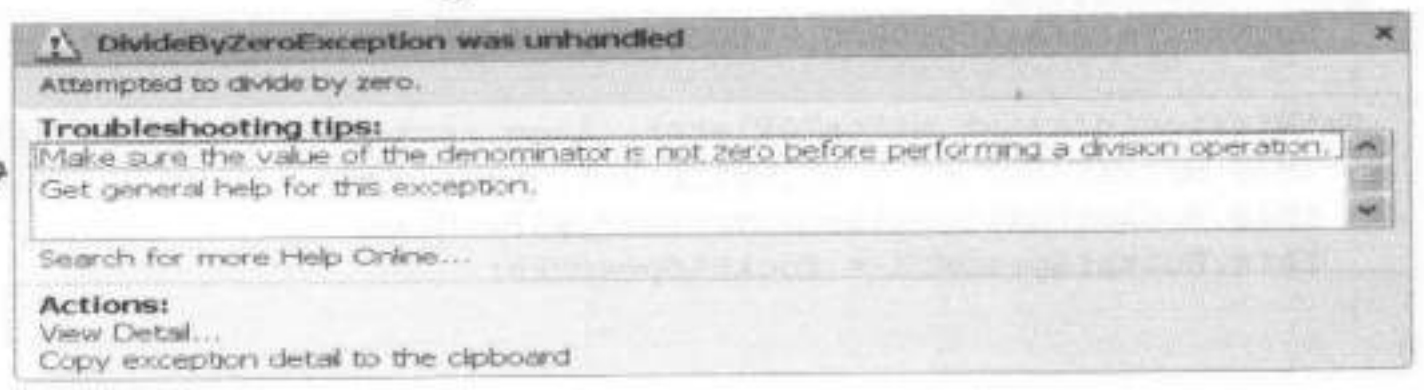
Algumas classes nunca devem ser instanciadas

Os problemas começam quando você cria uma instância da classe `PlanetMission`. Seu método `FuelNeeded()` espera que os campos sejam preenchidos numa subclasse. Mas eles não foram e têm seus valores padrão - zero. E quando o C# tenta dividir um número por zero...

```
private void button3_Click(object s, EventArgs e) {
  PlanetMission planet = new PlanetMission();
  MessageBox.Show(planet.FuelNeeded());
}
```

A classe PlanetMission não foi escrita para ser instanciada. Devemos apenas herdar dela. Mas tentamos instanciá-la e aí os problemas começaram.

Quando o método FuelNeeded() tenta dividir por RocketSpeedMPH encontra esse valor zerado. E quando você tenta dividir algo por zero, isso acontece



Solução: use uma classe abstrata

Quando você marca uma classe como abstrata, o C# não permite que seu código a instancie. É muito parecido com o que acontece com as interfaces - elas agem como um modelo para as classes que derivam dela.

Adicionar a palavra-chave "abstract" na declaração de classe diz ao C# que esta é uma classe abstrata e ela não pode ser instanciada.

```
public abstract class PlanetMission {
  public long RocketFuelPerMile;
  public long RocketSpeedMPH;
  public int MilesToPlanet;
  public long UnitsOfFuelNeeded() {
    return MilesToPlanet * RocketFuelPerMile;
  }
  // O resto da classe é definido aqui
}
```

Agora o C# se recusará a compilar seu programa até a linha que criou uma instância de PlanetMission ser removida.

Um método abstrato não tem corpo

Você já sabe como uma interface tem apenas declarações para métodos e propriedades e nenhum corpo de método. Isso ocorre porque todos os métodos numa interface são abstratos. Você pode ter métodos abstratos em classes abstratas também - mas precisa marcá-los explicitamente com a palavra-chave `abstract`, porque uma classe abstrata pode ter métodos normais também.

```
public abstract class PlanetMission {
  public abstract void SetMissionInfo(
    int MilesToPlanet, int RocketFuelPerMile,
    long RocketSpeedMPH);
// the rest of the class...
```

Este método abstrato é exatamente como seria numa interface - ele não tem um corpo e qualquer classe que herde de PlanetMission deve implementar o método SetMissionInfo() ou o programa não compilará.

Cada método numa interface é automaticamente abstrato, de forma que você não precisa usar neles a palavra-chave `abstract`; deve usá-la somente em classes abstratas. Apenas as classes abstratas podem ter métodos abstratos... mas elas podem ter métodos concretos também.

Se você adicionar esse método e tentar compilar o programa, o IDE mostra um erro.

> ⊗ 'VenusMission' does not implement inherited abstract member 'PlanetMission.SetMissionInfo (long, int, int)'

Então vamos implementá-lo! Uma vez feito, o erro desaparecerá.

```
public class Venus : PlanetMission {
  public Venus() {
    SetMissinInfo(40000000, 100000, 25000);
  }
  SetMissionInfo(int milesToPlanet, long rocketFuelPerMile, int rocketSpeedMPH) {
    this.MilesToPlanet = milesToPlanet;
    this.RocketFuelPerMile = rocketFuelPerMile;
    this.RocketSpeedMPH = rocketSpeedMPH;
  }
}
```

Aponte seu lápis

Eis a sua chance de demonstrar suas habilidades artísticas. Na esquerda você verá um conjunto de declarações de classes e interfaces. Sua tarefa é desenhar os diagramas de classe associados no lado direito. Fizemos o primeiro para você. Não se esqueça de usar uma linha tracejada para implementação de interfaces e uma linha cheia para herdar de uma classe.

Dado:

1)
```
public interface Foo { }
public class Bar : Foo { }
```

2)
```
public interface Vinn { }
public abstract class Vout : Vinn { }
```

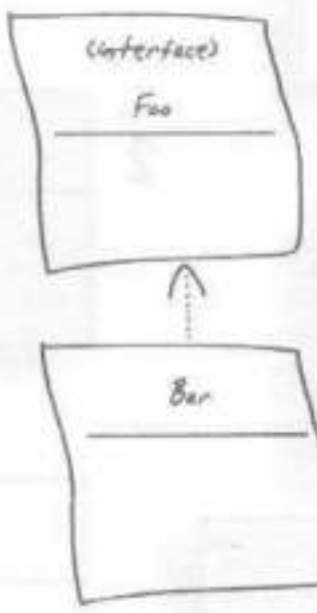
3)
```
public abstract class Muffie : Whuffie { }
public class Fluffie : Muffie { }
public interface Whuffie { }
```

4)
```
public class Zoop { }
public class Boop : Zoop { }
public class Goop : Boop { }
```

5)
```
public class Gamma : Delta, Epsilon { }
public interface Epsilon { }
public interface Beta { }
public class Alpha : Gamma,Beta { }
public class Delta { }
```

Qual a Figura?

1)



2)

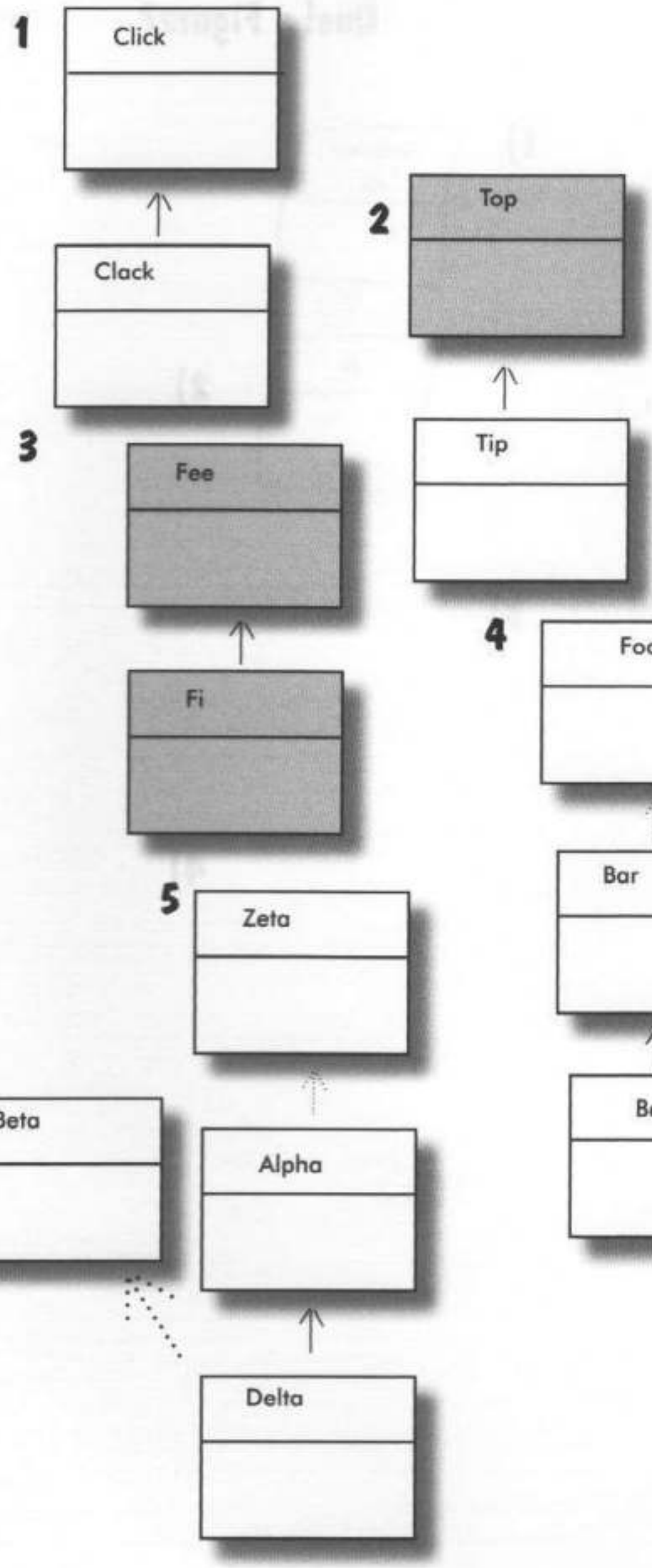
3)

4)

5)

À esquerda você encontrará grupos de diagramas de classe. Sua tarefa é transformá-los em declarações C# válidas. Fizemos o número 1 pra você.

Dado:



Qual a Figura?

1)

```
public class Click { }
public class Clack : Click { }
```

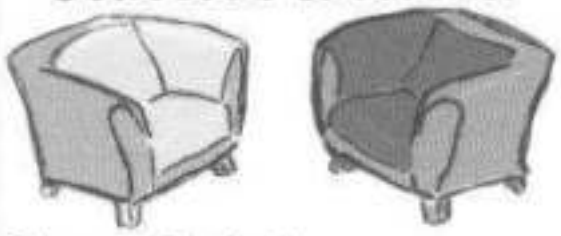
2)

3)

4)

5)

Conversa Informal



Conversa de hoje: **Um classe abstrata e uma interface batem cabeça sobre a pergunta que não quer calar: "Quem é mais importante?"**

Classe Abstrata:

Para mim, é óbvio quem é mais importante entre nós. Os programadores precisam de mim para completar suas tarefas. Vamos encarar. Você nem chega perto disso.

Você não pode realmente achar que é mais importante que eu. Você nem mesmo usa herança verdadeira - você é apenas implementada.

Melhor? Você deve estar louca. Eu sou muito mais flexível que você. Posso ter métodos abstratos ou concretos. Posso até ter métodos virtuais. Claro eu não posso ser instanciada, mas até aí, você também não. E eu posso fazer basicamente tudo o mais que uma classe comum pode fazer.

Você pode estar exagerando sua capacidade só um pouquinho.

Esse é exatamente o tipo de bobagem que eu esperaria escutar de uma interface. Código é extremamente importante! É o que faz um programa funcionar.

Você acha? Eu duvido. Programadores sempre se preocupam com seus métodos e propriedades.

Claro, com certeza. Diga para os programadores que eles não precisam programar.

Interface:

Claro. Essa é boa.

Ótimo, lá vamos nós de novo. Interfaces não usam herança real. Interfaces são apenas implementadas. Isso é apenas ignorância pura e simples. Implementação é tão boa quanto herança, de fato é até melhor!

Ah, é? E o que dizer se alguém quiser que uma classe herde de você e também de uma outra colega? **Não se pode herdar de duas classes**. É preciso escolher de que classe se vai herdar. E isso é grosseria pura! Não existe limite ao número de interfaces que uma classe pode implementar. E você falando de ser flexível! Comigo, um programador pode fazer uma classe implementar qualquer coisa.

Você pensa assim porque poder acha que, já que pode conter código, é a melhor coisa que já foi inventada desde o pão fatiado. Mas você não pode mudar o fato de uma classe só poder herdar de uma única outra por vez. Assim, você é um pouco limitada. Claro eu não posso incluir código nenhum. Mas, na realidade, o código é valorizado demais.

Nove vezes em dez, um programador quer ter certeza de que um objeto tenha certas propriedades e métodos, mas não se importa realmente como eles foram implementados.

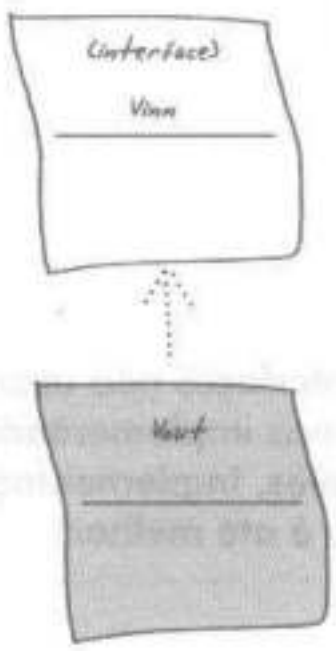
Certo, tem razão. De vez em quando. Mas pense sobre quantas vezes você já viu um programador escrever um método que usa um objeto que apenas precisa possuir um outro método qualquer e que realmente não se importa, naquele ponto, com a forma como ele foi implementado. É só importante que ele exista. Então bang! Ele precisa apenas escrever uma interface e problema resolvido!

Ah, deixa pra lá!

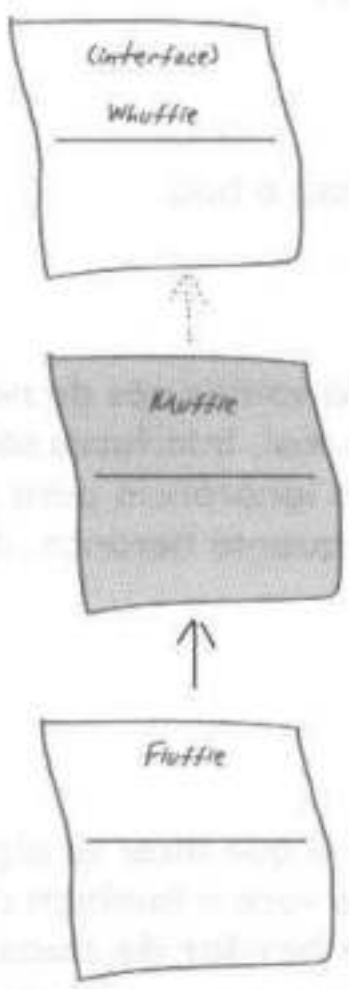
Aponte seu lápis
Solução

Qual a Figura?

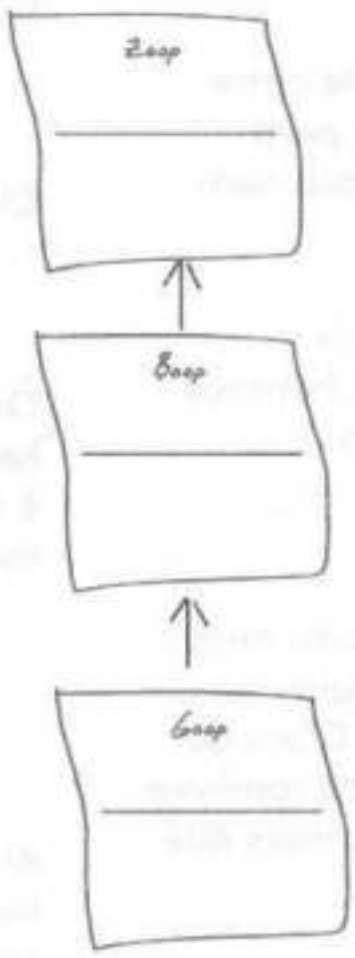
2)



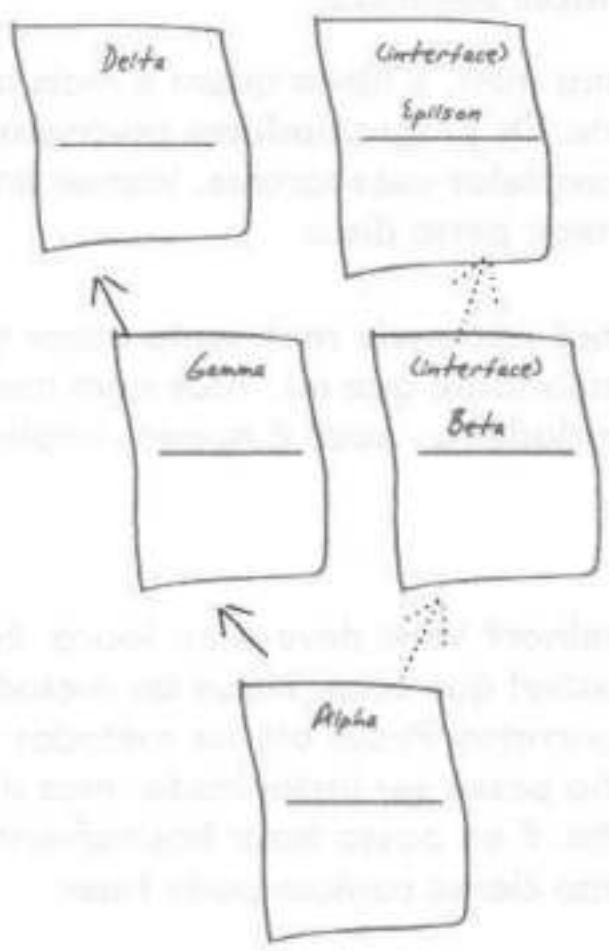
3)



4)



5)



Qual a Declaração?

2)
```
public abstract class Top { }
public class Tip : Top { }
```

3)
```
public abstract class Fee { }
public abstract class Fi : Fee { }
```

4)
```
public interface Foo { }
public class Baz : Bar { }
public class Bar : Fo { }
```

5)
```
public interface Zeta { }
public class Alpha : Zeta { }
public interface Beta { }
public class Delta : Alpha, Beta { }
```

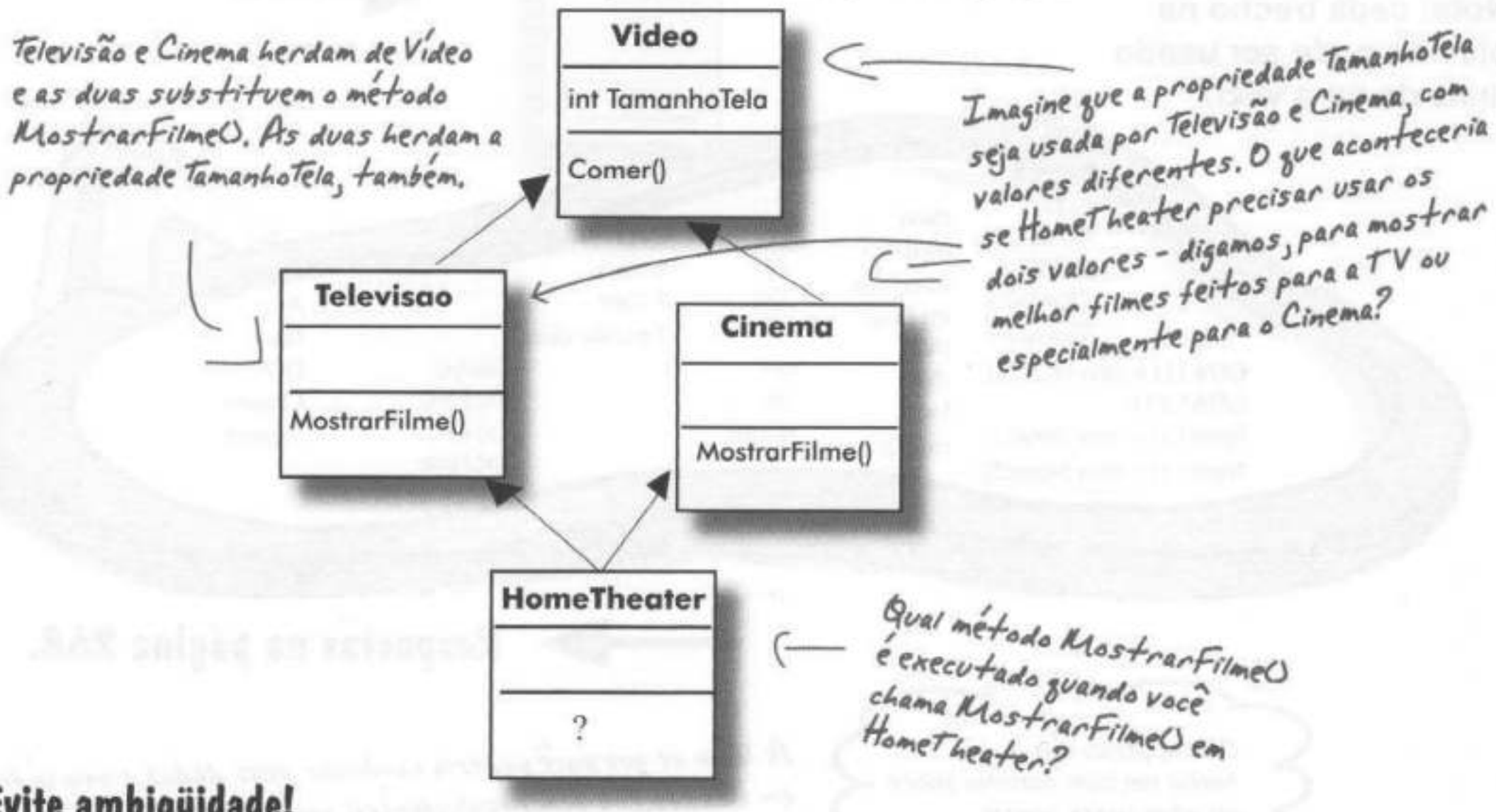
Delta herda de Alpha e implementa Beta

Ainda está me incomodando essa coisa de não poder herdar de mais de uma classe. Se eu pudesse fazer isso, não teria que usar interfaces. Essa é uma grande limitação do C#, não?

Não é uma limitação, é uma proteção.

Se o C# deixasse que você derivasse classes de mais de uma de base estaríamos abrindo uma verdadeira caixa de Pandora. Quando uma linguagem permite que uma subclasse herde de mais de uma, isso se chama herança múltipla. E ao utilizar interfaces no lugar disso, o C# poupa você de uma enorme confusão que nós gostamos de chamar de...

O Mortal Diamante da Morte!



Evite ambigüidade!

Uma linguagem que permite o Mortal Diamante da Morte pode permitir algumas situações bem horríveis, porque você precisa de regras especiais para lidar com esse tipo de ambigüidade... o que significa trabalho extra quando estiver desenvolvendo seu programa! O C# protege você de ter de lidar com isso ao impor o uso de interfaces. Se `Televisão` e `Cinema` forem interfaces, em vez de classes, um mesmo método `MostrarFilme()` pode satisfazer ambas. Tudo o que importa para a interface é que exista um método com o nome requerido.

Jogos de Verão

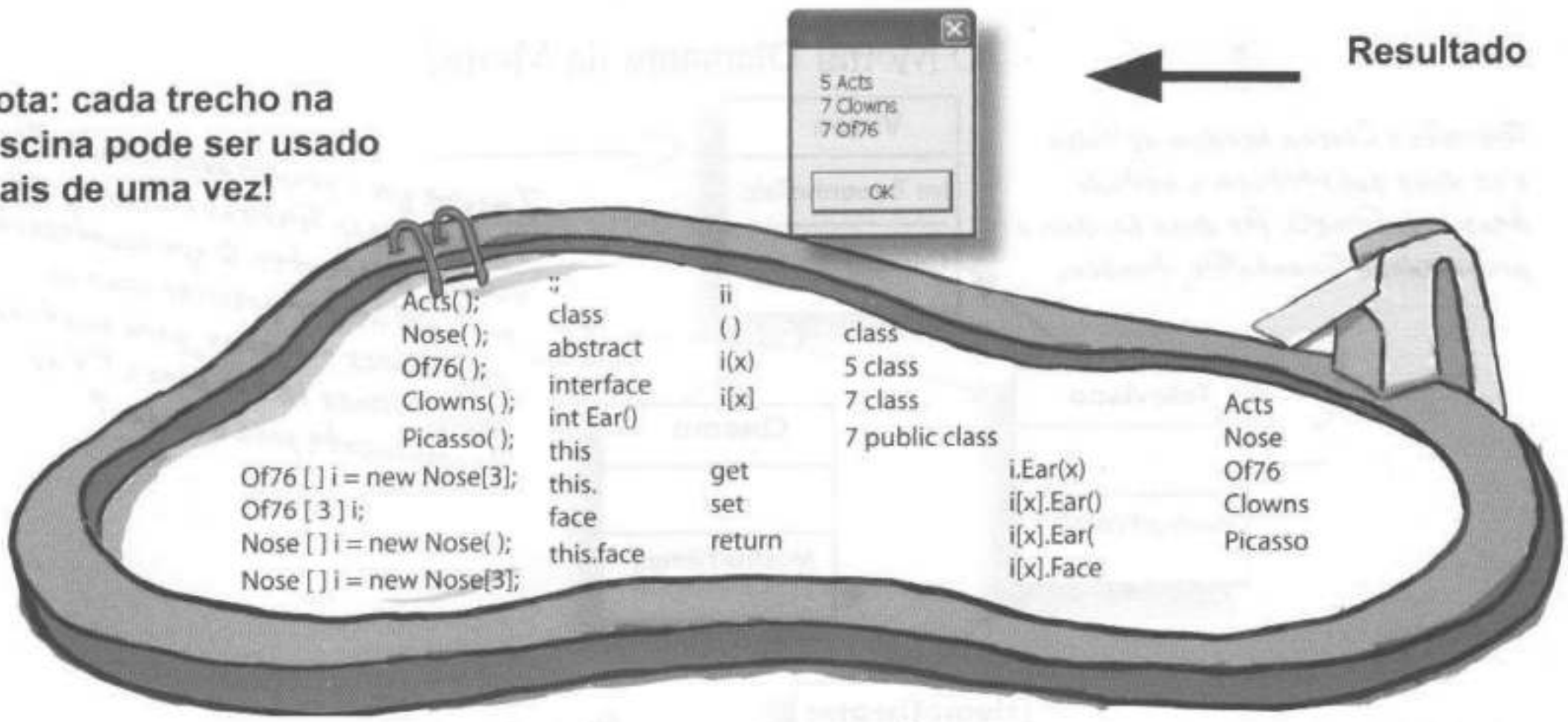
Seu **trabalho** é pegar os trechos de código da piscina e colocá-los nas linhas em branco no código principal e no resultado. Você pode usar o mesmo trecho mais de uma vez e não precisa usar todos eles. Seu **objetivo** é fazer um conjunto de classes que compile, execute e produza o resultado mostrado.

```
public.....Nose {
 .....;
  string Face { get; }
}
public abstract class..... : .....{
  public virtual int Ear()
  {
    return 7;
  }
  public Picasso(string face)
  {
  ..... = face;
  }
  public virtual string Face {
   .....{ ..... ; }
  }
  string face;
}
public class..... : .....{
  public Clowns() : base("Clowns") { }
}
```

```
public..class...        : ..... {
  public Acts() : base("Acts") { }
  public override .....{
    return 5;
  }
}
public class ..... : ..... {
  public override string Face {
   get { return "Of76"; }
  }
  public static void Main(string[] args) {
    string result = "";
    Nose[] i = new Nose[3];
    i[0] = new Acts();
    i[1] = new Clowns()
    i[2] = new Of76();
    for (int x = 0; x < 3; x++) {
      result += ( ..... + " "
                + ..... ) + "\n";
    }
    MessageBox.Show(result);
  }
}
```

Aqui está o ponto de entrada - este é um programa completo C#

Nota: cada trecho na piscina pode ser usado mais de uma vez!



Respostas na página 268.

Certo, penso que eu já tenho um bom domínio sobre objetos nesse ponto!

A idéia de que você poderia combinar seus dados e seu código em classes era bem revolucionária quando foi apresentada pela primeira vez - mas é assim que você está desenvolvendo todos os seus programas C# até agora, então pode pensar nisso como programação pura e simplesmente.

Você é um programador orientado a objetos

Existe um nome para o que você esteve fazendo. É **programação orientada a objetos**, ou POO (OOP em inglês, de Object Oriented Programming). Antes que linguagens como o C# existissem, as pessoas não usavam objetos e métodos quando criavam código. Usavam apenas funções (que é o nome do equivalente aos métodos em linguagens que não são POO) que ficavam todos juntos num único ponto - como se cada programa fosse uma única grande classe estática que tivesse apenas métodos estáticos. Era muito mais difícil criar programas que modelassem os problemas que estavam sendo resolvidos. Por sorte, você nunca teve que escrever programas sem usar POO, porque isso é um componente fundamental do C#.

Os quatro princípios da programação orientada a objetos

Quando programadores falam sobre POO eles referem-se a quatro princípios importantes. Eles devem lhe parecer familiares nesse ponto porque você esteve trabalhando com cada um deles. Você reconhecerá os primeiros três princípios pelo seus nomes: **herança**, **abstração** e **encapsulamento**. O último é chamado de polimorfismo. Isso soa meio estranho, mas no fim das contas você já sabe o que ele é também.

Encapsulamento quer dizer criar um objeto que mantém registro de seu estado internamente usando campos privados e usar propriedades e métodos públicos para permitir que outras classes trabalhem apenas com os dados internos necessários para elas.

Encapsulamento

Herança

Isso apenas quer dizer ter uma classe ou interface que herda de outra.

Abstração

Você está usando abstração quando cria um modelo de classe que começa mais geral - ou abstrato - e torna-se mais específico nas classes que herdam da primeira.

Polimorfismo

A palavra polimorfismo literalmente significa muitas formas. Você consegue pensar num momento quando um objeto tenha tomado muitas formas em seu código?

Polimorfismo quer dizer que um objeto pode tomar muitas formas diferentes

Toda vez que você usar um Mockingbird no lugar de um animal ou queijo Cheddar Vermont envelhecido numa receita que espera apenas por queijo, você está usando polimorfismo. É isso que você está fazendo toda vez que faz supercoerção ou subcoerção. É pegar um objeto e usá-lo num método ou comando que espera alguma outra coisa.

Você estará usando polimorfismo quando pegar uma instância de uma classe e usá-la num comando ou método que espere um tipo diferente, como uma classe base ou uma interface que uma classe implemente.

Mantenha os olhos abertos para o polimorfismo no próximo exercício!

Você está prestes a fazer um exercício realmente grande - o maior que viu até esse momento - e usará muito polimorfismo nele. Então mantenha os olhos abertos. Eis uma lista de quatro situações típicas de uso do polimorfismo. Mostramos um exemplo de cada uma delas (você não verá essas linhas em particular no exercício, no entanto). Assim que você notar código similar àquele que escreverá no exercício, **marque na lista**.

- ☐ Pegar qualquer variável de referência que use uma classe e atribuir uma instância de uma classe diferente.

```
NectarStinger bertha = new NectarStinger();
INectarCollector gatherer = bertha;
```

- ☐ Efetuar supercoerção usando uma subclasse num comando ou método que espere a classe base

```
spot = new Dog();
zooKeeper.FeedAnAnimal(spot);
```

Se FeedAnAnimal() (alimentar um animal) esperar um objeto tipo Animal e Dog (cão) herdar dessa classe, você pode passar Dog para o método.

- ☐ Criar uma variável de referência cujo tipo seja uma interface e apontá-la para um objeto que implemente essa interface.

```
IStingPatrol defender = new StingPatrol();
```

Isso é supercoerção também!

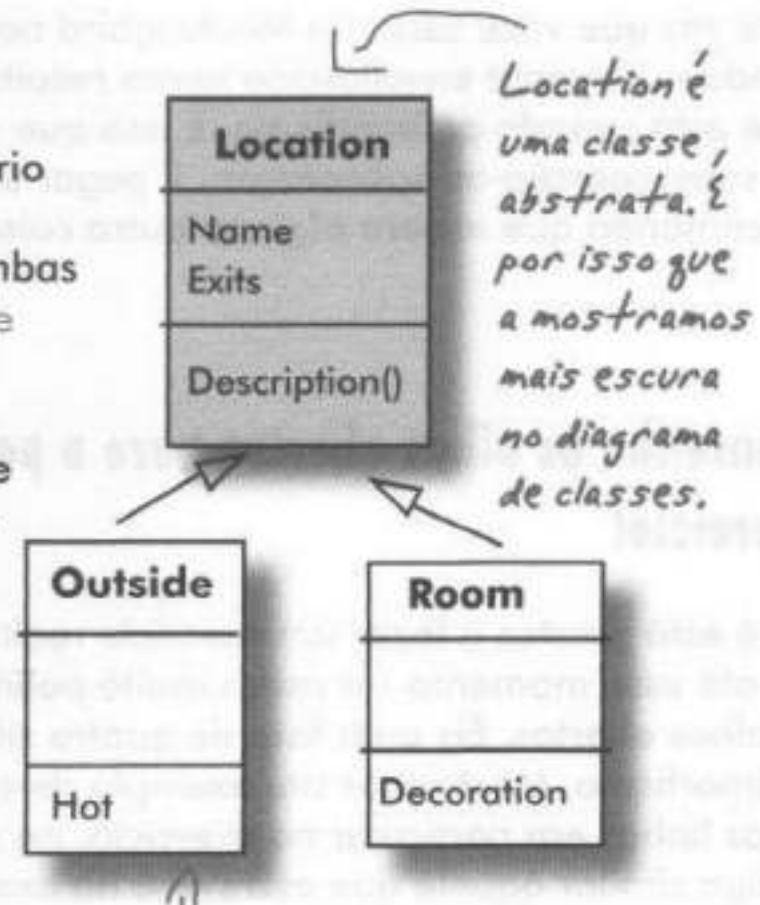
- ☐ Efetuar subcoerção usando a palavra-chave "as".

```
void MaintainTheHive(IWorker worker) {
  if (worker is HiveMaintainer) {
    HiveMaintainer maintainer = worker as HiveMaintainer;
    ...
```

O método MaintainTheHive() (manutenção da colméia) pode usar um IWorker (IOperaria) como parâmetro. Ele usa as para apontar uma referência de HiveMaintener (algo como zelador de colméia) para a operária.

Exercício Longo

Vamos construir uma casa! Crie um modelo para uma casa usando classes para representar os cômodos e outras áreas e uma interface para qualquer lugar que tenha uma porta.



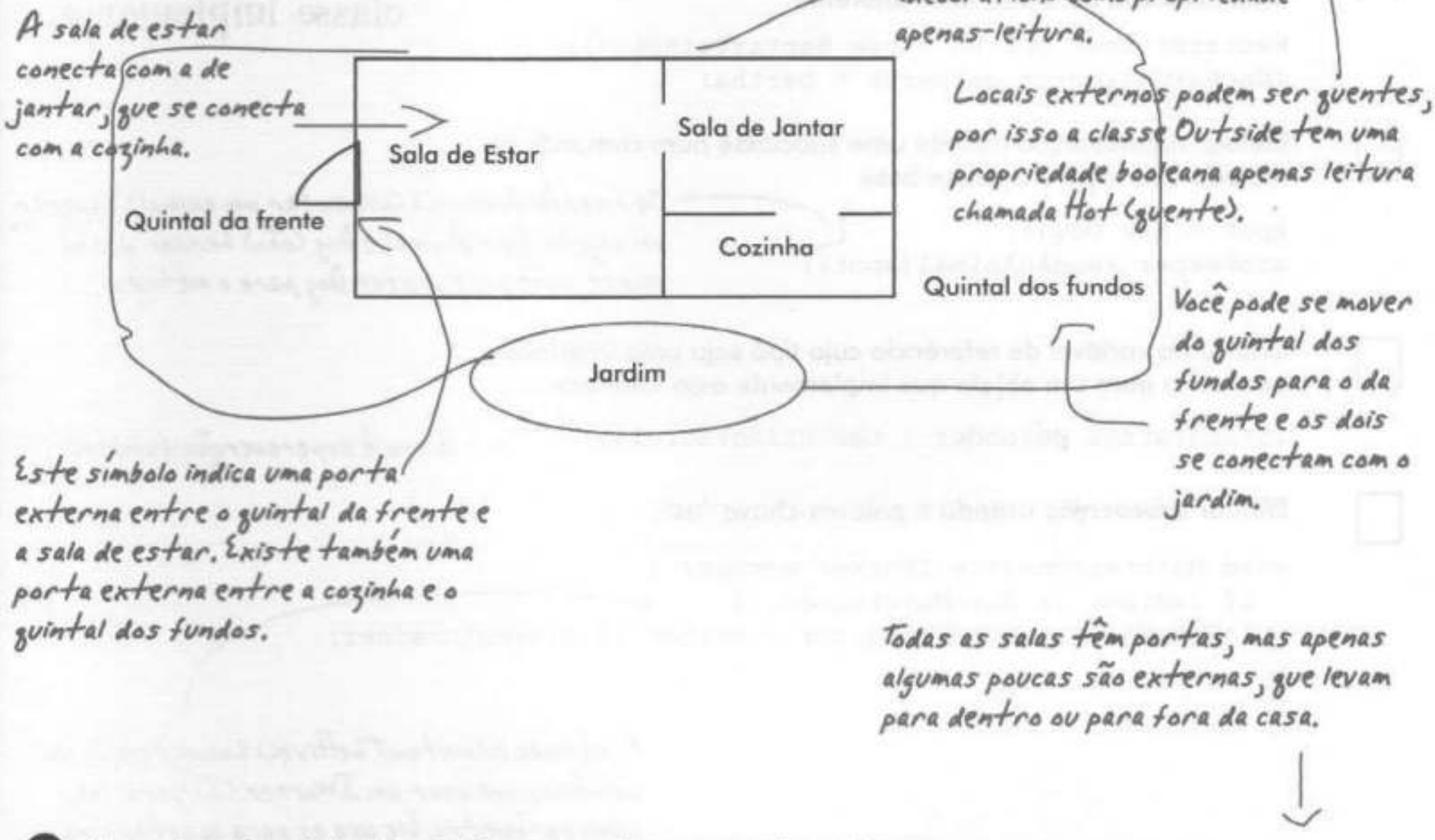
1 Comece com um modelo de classe

Cada cômodo ou local na casa será representado por seu próprio objeto. As salas internas todas herdam de `Room` (sala, cômodo espaço) e as áreas externas herdam de `Outside` (exterior) e ambas são subclasses de `Location` (local). Ela tem dois campos: `Name` é o nome da sala ("cozinha", por exemplo) e `Exits` (saídas) é uma matriz de objetos Location conectados nessa sala. Assim, `diningRoom.Name` (sala de jantar) será igual a "Dining Room" e `diningRoom.Exits` será igual à matriz { `LivingRoom` (sala de estar), Kitchen (cozinha)}.

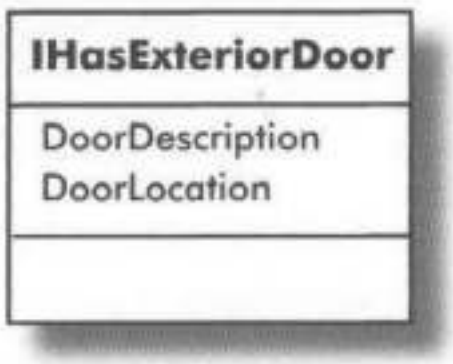
-> Crie um projeto de aplicativo Windows e adicione as classes Location, Room e Outside.

2 Você precisará de uma planta da casa

Essa casa tem três cômodos, um quintal e um jardim. Existem duas portas: a da frente conecta a sala com o quintal da frente e a porta dos fundos conecta a cozinha com o quintal nos fundos.



Todas as salas têm portas, mas apenas algumas poucas são externas, que levam para dentro ou para fora da casa.



3 Use a interface IHasExteriorDoor em salas com portas externas

Existem duas portas externas na casa: a da frente e a dos fundos. Cada local com uma (o quintal da frente, o dos fundos, a sala de estar e a cozinha) deve implementar `IHasExteriorDoor` (possui porta externa). A propriedade apenas leitura `DoorDescription` (descrição da porta) contém uma descrição da porta (a porta da frente é "uma porta de carvalho com uma maçaneta de latão" e a dos fundos é "uma porta com tela"). O campo `DoorLocation` contém uma referência aos locais para onde a porta leva (`cozinha`, por exemplo).

4 **Aqui está a classe Location**
Para dar a você um ponto de partida eis a classe Location:

```
public abstract class Location {
  public Location(string name) {
    this.name = name;
  }
  public Location[] Exits;
  private string name;
  public string Name {
    get { return name; }
  }
  public virtual string Description {
    get {
      string description = "You're standing in the " + name
        + ". You see exits to the following places: ";
      for (int i = 0; i < Exits.Length; i++) {
        description += " " + Exits[i].Name;
        if (i != Exits.Length - 1)
            description += ",";
      }
      description += ".";
      return description;
    }
  }
}
```

O construtor inicializa o campo nome, que é o campo de armazenamento apenas leitura da propriedade Name.

Description é um método virtual. Você será obrigado a substituí-lo.

O campo público Exits é uma matriz de referências tipo Location que mantém registro de todos os demais locais para onde este se conecta.

A propriedade Description retornará uma string que descreve a sala, incluindo o nome e uma lista de todos os demais cômodos conectados (o que será encontrado no campo Exits[]). As subclasses terão que alterar a descrição ligeiramente, então elas devem substituir a propriedade.

A classe Room vai substituir e estender Description para adicionar a decoração e Outside vai acrescentar a temperatura.

Lembre-se, Location é uma classe abstrata - você pode derivar outras dela e criar variáveis de referência do tipo Location, mas não pode instanciá-la.

5 **Crie as classes**
Primeiro crie as classes `Room` e `Outside` baseando-se no modelo de classes. Então crie mais duas: `OutsideWithDoor` (exterior com porta), que herda de `Outside` e implementa `IHasExteriorDoor` e `RoomWithDoor` (sala com porta), que é subclasse de `Room` e implementa `IHasExteriorDoor`.
Aqui estão as declarações de classe para ajudar um pouco.

Comece a fazer já - daremos mais detalhes sobre as classes na próxima página.

```
public class OutsideWithDoor : Outside, IHasExteriorDoor
{
  // O read-only DoorLocation propriedade vai aqui
  // A propriedade DoorDescription vai aqui
}
public class RoomWithDoor : Room, IHasExteriorDoor
{
  // Não implementa aqui - nós não sabemos
  // Aquilo que os nossos alunos gostam de comprar!
}
```

Esse será um exercício bem grande... mas prometemos que será divertido! E você com certeza aprenderá tudo conforme for fazendo.

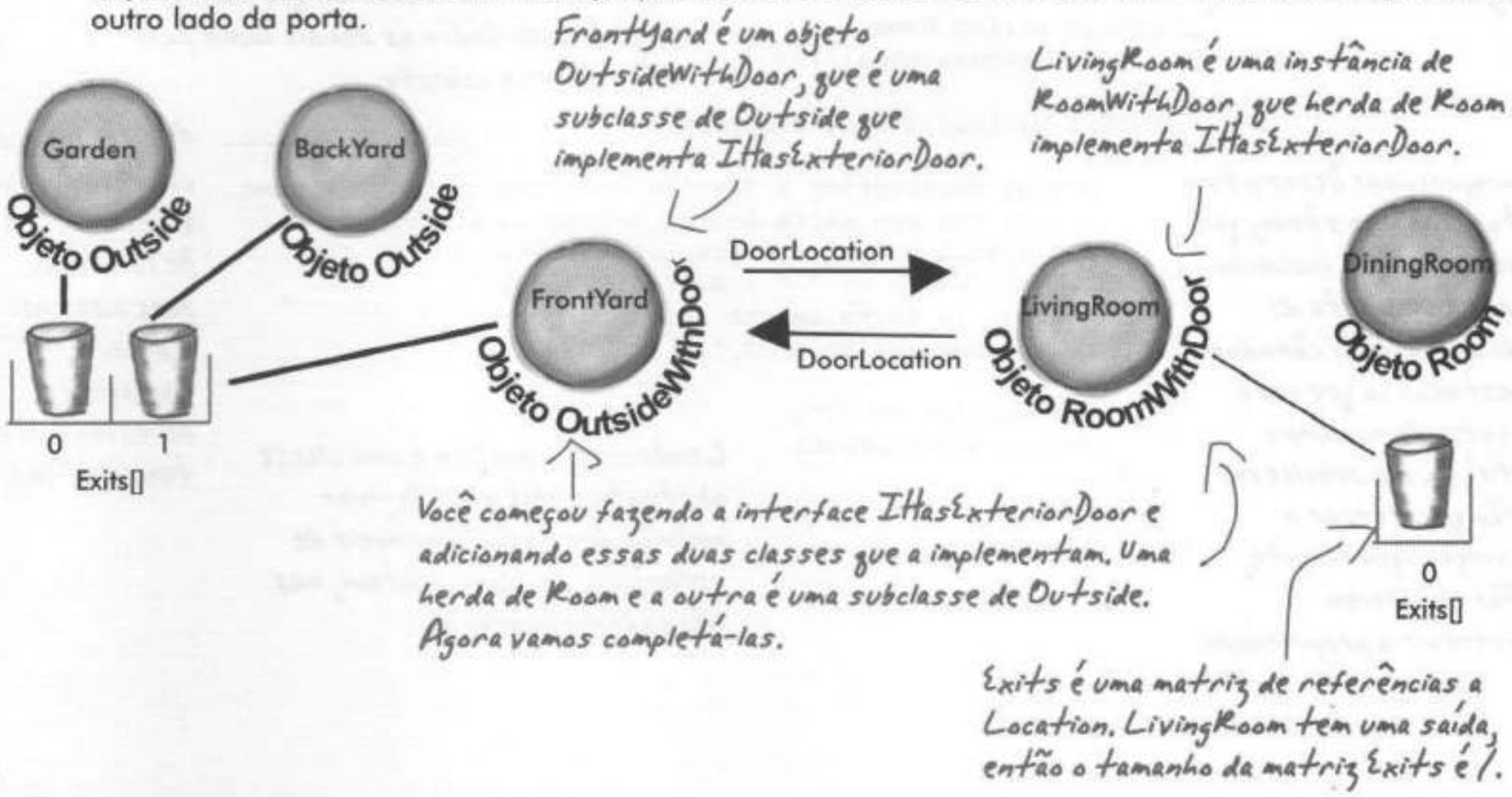
Ainda não acabou - vire a página!

Exercício Longo (continuação)

Agora que você tem o modelo de classes, pode criar os objetos para todas as partes da casa e adicionar um formulário para observá-los.

6 Como os objetos da sua casa funcionam

Eis a arquitetura para dois dos seus objetos, `frontYard` (quintal da frente) e `diningRoom` (sala de jantar). Uma vez que cada um deles tem uma porta, ambos precisam ser instâncias de uma classe que implemente `IHasExteriorDoor`. A propriedade `DoorLocation` mantém uma referência do cômodo do outro lado da porta.



7 Complete as classes e crie as instâncias

Você já sabe todas as classes de que precisará. Agora é hora de completá-las e criar seus objetos.

- ★ Você tem que se certificar que o construtor da classe `Outside` inicialize a propriedade apenas leitura `Hot` e também de substituir a propriedade `Description` para que ela mostre "está muito quente aqui", se `Hot` for `true`. Deve ser quente no quintal dos fundos (back yard), mas não no da frente (front yard) nem no jardim (garden).

- ★ O construtor para `Room` precisa inicializar `Decoration` e deve substituir a propriedade `Description` para adicionar "Você pode ver (a decoração tal) aqui". A sala de estar (living room) tem um carpete antigo, a de jantar (dining room) tem um candelabro de cristal e a cozinha (kitchen) tem detalhes em aço inox e uma porta com tela que leva para o quintal dos fundos.

Cada cômodo deve ter seu próprio campo na classe do formulário.

- ★ Seu formulário precisa criar cada um dos objetos e manter uma referência deles. Então adicione um método a ele chamado `CreateObjects()` (criar objetos) e chame-o no construtor do formulário.

Exits é uma matriz de referências do tipo Location. Assim essa linha cria uma que contém dois elementos.

- ★ Instancie cada um dos objetos para os seis locais da casa. Eis uma das linhas para isso:

```
RoomWithDoor livingRoom = new RoomWithDoor("Living Room",
    "an antique carpet" , "an oak door with a brass knob");
```

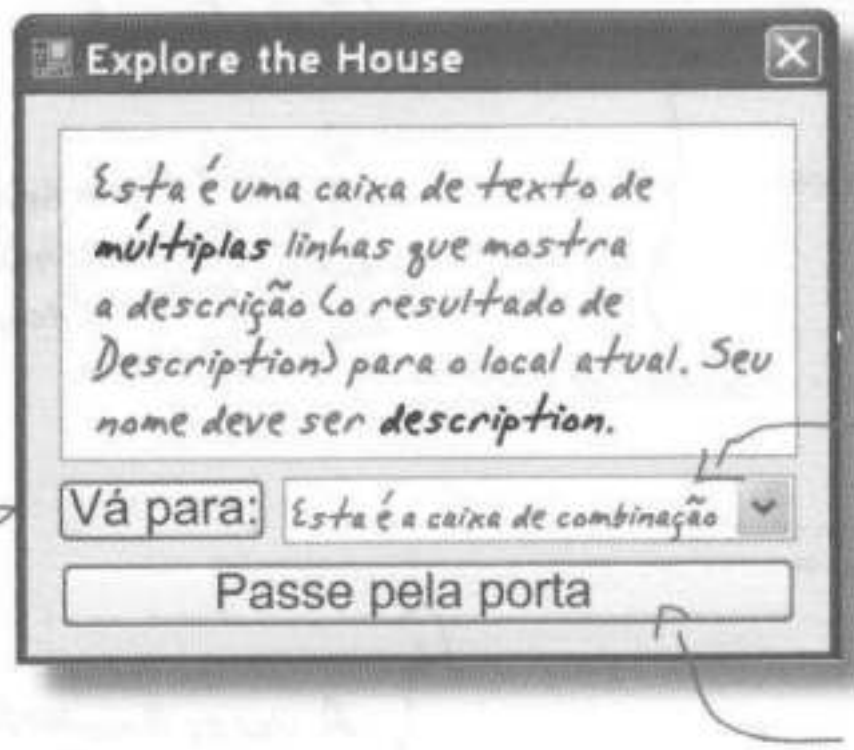
Estas são chaves. Qualquer outra coisa provocará um erro.

- ★ Seu método CreateObject() tem que preencher o campo Exits[] de cada objeto:

```
frontYard.Exits = new Location[] { backYard, garden };
```

8 **Crie um formulário para explorar a casa**
Crie um formulário simples para explorar a casa. Ele deve ter uma grande caixa de texto de várias linhas chamada description que mostrará a descrição da sala atual. Uma caixa de combinação chamada exits deve listar todas as saídas do cômodo atual. Ela tem dois botões: goHere (vá pra lá) move-se para a sala selecionada na caixa de combinação e goThroughTheDoor (atravesse a porta) só é visivel quando existe uma porta externa.

É aqui que você configurará o que vai povoar a caixa de combinação.



Clique no botão gottere para ir para outro lugar.

A caixa de combinação contém uma lista de todas as saídas, de forma que seu nome deve ser exits. Tenha certeza de que a propriedade DropDownStyle (estilo da caixa de seleção) seja DropDownList.

Este botão só deve ser visível quando você estiver numa sala com uma porta externa. Você pode deixá-lo visível ou invisível atribuindo true ou false para a propriedade Visible. O nome deve ser goThroughTheDoor.

9 **Agora você só precisa fazer o formulário funcionar!**
Você tem todas as peças, agora só precisa fazê-las funcionar todas juntas.

- ★ Você precisará de um campo no seu formulário chamado currentLocation (local atual) para manter registro da sua posição atual.
- ★ Adicione um método MoveToANewLocation() (vá pra um novo local) que tenha um parâmetro do tipo Location. Esse método deve primeiro atribuir o local passado como parâmetro para currentLocation e adicionar o nome de cada local na matriz Exits usando o método da caixa de combinação Items.Add() (Itens, Adicionar). Finalmente, reinicie a caixa de combinação para que ela mostre o primeiro item da lista, atribuindo zero para sua propriedade SelectedIndex (índice selecionado).
- ★ Atribua o texto correto para a caixa de texto exibir a descrição da sala atual.
- ★ Use a palavra-chave `is` para verificar se o local atual tem uma porta. Se tiver, faça com que o botão "Passe pela porta" fique visível usando sua propriedade Visible. Se não for o caso, faça com que ele fique invisível.
- ★ Se o botão "Vá para" for clicado, mova a posição atual para a caixa de combinação.
- ★ Se o botão "Passe pela porta" for clicado, mova-se para o local para o qual a porta leva.

Dica: Quando você escolhe um item na caixa de combinação, o índice selecionado será o mesmo que o do local correspondente na matriz Exits[].

Outra dica: o campo currentLocation do seu formulário é uma referência do tipo Location. Então mesmo que ele esteja apontado para um objeto que implemente IHasExteriorDoor, você não pode simplesmente digitar currentLocation.DoorLocation porque DoorLocation não é um campo de Location. Você precisará fazer subcoerção se quiser obter a posição da porta desse objeto.

Exercício Longo Solução

Aqui está o código para o modelo da casa. Usamos classes para representar as salas e locais e uma interface para qualquer lugar que tenha uma porta.

```
public interface IHasExteriorDoor {
  string DoorDescription { get; }
  Location DoorLocation { get; set; }
}
public class Room : Location {
  private string decoration;
  public Room(string name, string decoration)
    : base(name) {
    this.decoration = decoration;
  }
  public override string Description {
    get {
      return base.Description + " You see " + decoration + ".";
    }
  }
}
public class RoomWithDoor : Room, IHasExteriorDoor {
  public RoomWithDoor(string name, string decoration, string doorDescription)
    : base(name, decoration)
  {
    this.doorDescription = doorDescription;
  }
  private string doorDescription;
  public string DoorDescription {
    get { return doorDescription; }
  }
  private Location doorLocation;
  public Location DoorLocation {
    get { return doorLocation; }
    set { doorLocation = value; }
  }
}
```

Eis a interface IHasExteriorDoor.

A classe Room herda de Location e adiciona um campo de armazenamento para a propriedade apenas leitura Decoration. Seu construtor inicializa o campo.

A classe RoomWithDoor herda de Room e implementa IHasExteriorDoor. Ela faz tudo o que uma sala comum faz, mas adiciona uma descrição da porta externa no construtor. Ela também adiciona DoorLocation, uma referência ao local para onde a porta conduz. DoorDescription e DoorLocation são exigidas por IHasExteriorDoor.

```
public class Outside : Location {
  private bool hot;
  public bool Hot { get { return hot; } }

  public Outside(string name, bool hot)
    : base(name)
  {
    this.hot = hot;
  }
  public override string Description {
    get {
      string NewDescription = base.Description;
      if (hot)
        NewDescription += " It's very hot.";
      return NewDescription;
    }
  }
}
public class OutsideWithDoor : Outside, IHasExteriorDoor {
  public OutsideWithDoor(string name, bool hot, string doorDescription)
    : base(name, hot)
  {
  this.doorDescription = doorDescription;
  }
  private string doorDescription;
  public string DoorDescription {
    get { return doorDescription; }
  }
  private Location doorLocation;
  public Location DoorLocation {
    get { return doorLocation; }
    set { doorLocation = value; }
  }
  public override string Description {
    get {
      return base.Description + " You see " + doorDescription + ".";
    }
  }
}
```

Outside é bem parecida com Room - ela herda de Location e acrescenta um campo de armazenamento para a propriedade Hot, que é usada no método Description() estendido a partir da classe base.

OutsideWithDoor herda de Outside e implementa IHasExteriorDoor e parece bastante com RoomWithDoor.

A propriedade Description da classe base informa se o local está quente ou não. E utiliza-se da propriedade Description original de Location para adicionar a descrição principal e as saídas.

Ainda não terminamos - vire a página!

Aqui está o código do formulário. Está todo em Form1.cs, dentro da declaração de Form1.

É assim que o formulário mantém registro da sala sendo mostrada.

O formulário usa essas variáveis de referência para manter registro dos cômodos da casa.

O construtor do formulário cria os objetos e então usa o método MoveToANewLocation.

Quando o formulário cria os objetos, primeiro ele precisa instanciar as classes e passar as informações corretas para o construtor de cada uma.

É aqui que passamos a descrição da porta para os construtores de OutsideWithDoor.

É aqui que a matriz Exits[] de cada instância é povoada. Precisamos esperar até ser possível fazer isso: quando todas as instâncias estiverem criadas. De outra forma, não teríamos nada para colocar nas matrizes!

Para os objetos IHasExteriorDoor, precisamos inicializar a posição de suas portas.

```
public partial class Form1 : Form
{
  Location currentLocation;
  RoomWithDoor livingRoom;
  Room diningRoom;
  RoomWithDoor kitchen;
  OutsideWithDoor frontYard;
  OutsideWithDoor backYard;
  Outside garden;
  public Form1() {
    InitializeComponent();
    CreateObjects();
    MoveToANewLocation(livingRoom);
  }
  private void CreateObjects() {
    livingRoom = new RoomWithDoor("Living Room", "an antique carpet",
        "an oak door with a brass knob");
    diningRoom = new Room("Dining Room", "a crystal chandelier");
    kitchen = new RoomWithDoor("Kitchen", "stainless steel appliances", "a screen door");
    frontYard = new OutsideWithDoor("Front Yard", false, "an oak door with a brass knob");
    backYard = new OutsideWithDoor("Back Yard", true, "a screen door");
    garden = new Outside("Garden", false);
    diningRoom.Exits = new Location[] { livingRoom, kitchen };
    livingRoom.Exits = new Location[] { diningRoom };
    kitchen.Exits = new Location[] { diningRoom };
    frontYard.Exits = new Location[] { backYard, garden };
    backYard.Exits = new Location[] { frontYard, garden };
    garden.Exits = new Location[] { backYard, frontYard };
    livingRoom.DoorLocation = frontYard;
    frontYard.DoorLocation = livingRoom;
    kitchen.DoorLocation = backYard;
    backYard.DoorLocation = kitchen;
  }
```

O método MoveToANewLocation() mostra um novo local no formulário.

Primeiro precisamos limpar a caixa de combinação. Depois, podemos adicionar cada nome de cômodo nela. Finalmente, atribuímos zero para seu índice selecionado (ou seja, a linha destacada) para que ela mostre a primeira linha na lista. Não se esqueça de atribuir a propriedade DropDownStyle dela para DropDownList - dessa forma o usuário não poderá digitar nada na caixa.

Quando o usuário clica no botão Vá para: ele se move para o local selecionado na caixa de combinação.

Isso faz com que o botão Passe pela porta fique invisível se o local atual não implementar IHasExteriorDoor.

Precisamos usar a palavra-chave as para fazer a subcoerção de currentLocation para IHasExteriorDoor para podermos acessar o campo DoorLocation.

```
  private void MoveToANewLocation(Location newLocation) {
    currentLocation = newLocation;
    exits.Items.Clear();
    for (int i = 0; i < currentLocation.Exits.Length; i++)
      exits.Items.Add(currentLocation.Exits[i].Name);
    exits.SelectedIndex = 0;
    description.Text = currentLocation.Description;
    if (currentLocation is IHasExteriorDoor)
      goThroughTheDoor.Visible = true;
    else
      goThroughTheDoor.Visible = false;
  }
  private void goHere_Click(object sender, EventArgs e) {
    MoveToANewLocation(currentLocation.Exits[exits.SelectedIndex]);
  }
  private void goThroughTheDoor_Click(object sender, EventArgs e) {
    IHasExteriorDoor hasDoor = currentLocation as IHasExteriorDoor;
    MoveToANewLocation(hasDoor.DoorLocation);
  }
}
```

Mas ainda não acabamos!

Tudo bem criar o modelo de uma casa, mas não seria melhor transformá-lo num jogo? Vamos fazer isso! Você jogará esconde-esconde contra o computador. Precisaremos adicionar uma classe `Opponent` (oponente) e fazê-la se esconder dentro de alguma sala. E precisaremos que a casa seja bem maior. Ah e também algum lugar para que alguém se esconda! Adicionaremos uma nova interface de forma que em algumas salas haja locais para se esconder. Finalmente, atualizaremos o formulário para permitir que se olhe nesses locais e para manter registro de tentativas para achar seu oponente. Parece divertido? Claro que sim!

Vamos começar!

Exercício

Hora de brincar de esconde-esconde! Desenvolva seu próprio programa original de casa para adicionar mais cômodos esconderijos e um oponente que se esconde de você.

Crie um novo projeto e use a opção Add Existing Item (adicionar item existente) do IDE para adicionar as classes da primeira parte deste exercício.

1 Adicione uma interface IHidindPlace

Não é preciso fazer nada extravagante aqui. Qualquer subclasse de Location que implemente `IHidingPlace` (local para se esconder) tem um lugar para que o oponente se esconda. Ela apenas precisa de uma seqüência de caracteres para guardar o texto que descreve o local ("no armário", "debaixo da cama", etc.).

★ Dê a ela um acessador get, mas nenhum set - vamos atribuir o valor no construtor, já que uma vez que uma sala tenha um esconderijo, não precisaremos mudar isso nunca mais.

2 Adicione classes que implementem IHidingPlace

Você precisará de mais duas classes: `OutsideWithHidingPlace` (que herda de Outside) e `RoomWithHidindPlace` (que herda de Room). Além disso, vamos fazer que qualquer quarto com uma porta tenha um esconderijo e teremos que herdar de `RoomWithHidingPlace`, em vez de `Room`.

Assim toda sala que tiver uma porta externa terá também um esconderijo.

3 Adicione uma classe para seu oponente

O objeto oponente escolherá um local para se esconder aleatoriamente e será sua tarefa encontrá-lo.

★ Ele precisará de um campo do tipo Location privado `(myLocation)`, para determinar onde está e um campo privado `random` (aleatório) do tipo `Random` para usar quando se move para um local aleatório.

★ O construtor deve receber um local inicial e atribuir esse lugar para `myLocation` e criar uma nova instância de `Random` que deve ser colocada em `random`. Ele deve começar no jardim da frente (isso será informado a ele pelo formulário) e mover-se de um esconderijo a outro aleatoriamente. Ele deve mover-se dez vezes quando o jogo inicia. Quando ele encontrar uma porta externa, deve jogar uma moeda para ver se passa por ela.

★ Adicione um método `Move()` que desloque o oponente de seu local atual para um novo. Primeiro, se ele estiver numa sala com uma porta, deve jogar uma moeda para decidir se passa por ela ou não. Assim, se `random.Next(2)` for igual a 1 ele atravessa pela porta. Então ele deve escolher uma das saídas de seu local atual aleatoriamente e ir para lá. Se o lugar não tiver onde se esconder ele deve fazer isso de novo - escolher uma saída aleatória e ir para lá e continuar fazendo isto até que encontre um esconderijo.

★ Adicione um método `Check()` que recebe como parâmetro um objeto Location e retorna true se o oponente está escondido lá, ou false se não.

4 Adicione mais cômodos na casa

Atualize seu método `CreateObjects()` para adicionar mais salas:

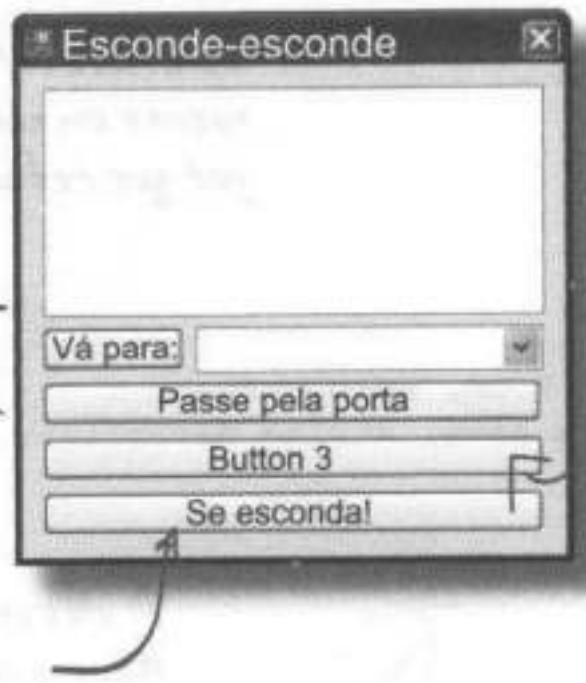
★ Adicione **escadas** (stairs) com um corrimão de madeira que conecte a sala ao **hall superior** (upstairs hallway), que terá um quadro de um cachorro e um armário onde é possível se esconder.

★ O hall superior se conecta com três quartos: o **principal** (master bedroom) com uma cama grande, um **segundo** (second bedroom) com uma cama pequena e um **banheiro** (bathroom) com uma pia e um vaso sanitário. É possível se esconder debaixo das camas em qualquer quarto ou no chuveiro.

★ O quintal da frente e o dos fundos devem conectar-se com a **calçada** (driveway), onde é possível se esconder na garagem. Além disso, deve ser possível se esconder num galpãozinho no **jardim**.

5 **Certo, hora de atualizar o formulário**
Você precisará adicionar alguns botões ao formulário. E ficará um pouco mais complicado deixá-los visíveis ou invisíveis, dependendo do estado do jogo.

Você usará os dois botões de cima e a caixa de combinação exatamente da mesma forma de antes, exceto que eles devem ser visíveis apenas enquanto o jogo estiver rodando.



O botão do meio é chamado check. Não é preciso alterar sua propriedade Text.

Este é o botão que deve ser usado para checar o esconderijo na sala. Ele deve ser visível apenas numa sala que tenha um. Quando ele for mostrado, sua propriedade Text deve ser alterada para Olhar seguido do nome do lugar. Por exemplo, para uma sala onde é possível se esconder embaixo da cama, o botão deverá mostrar Olhar embaixo da cama.

Quando o jogo começa, o botão se esconda! deve ser o único mostrado. Quando você clica, o formulário deverá contar até 10 na caixa de texto e chamar o método Move() do oponente 10 vezes. Então o botão deverá ficar invisível.

6 Faça os botões funcionarem
Existem dois botões novos para adicionar ao formulário.

★ O botão do meio deve checar o local onde é possível se esconder na sala atual e deve ser visível apenas quando você estiver numa sala que possua um local assim. Use o método `Check()` do oponente para isso. Se ele for encontrado, o jogo deve ser encerrado.

Volte ao capítulo 2 para relembrar sobre DoEvents() e Sleep() - eles serão úteis.

★ O botão abaixo deverá ser usado para iniciar o jogo. Ele fará a contagem até dez mostrando "1..." esperando 200 milissegundos e então "2...", "3..." e assim por diante, na caixa de texto. Depois de mostrar cada número ele diz ao oponente para se mover chamando seu método `Move()`. Então no final deve ser mostrado "Pronto não, aí vou eu!" por meio segundo e então o jogo inicia.

7 **Adicione um método para repintar o formulário e outro para reiniciar o jogo**
Adicione um método `RedrawForm()` (redesenhar o formulário) que coloque o texto correto na caixa, torne os botões visíveis ou invisíveis e que coloque o texto corretos no botão do meio. Depois adicione um método `ResetGame()` (reiniciar jogo) que deve ser executado quando o oponente for encontrado. O objeto oponente deve ser reiniciado recomeçando no quintal da frente - ele se esconderá novamente quando o usuário pressionar "Se esconda!" de novo. Ele deve deixar o formulário com tudo invisível (menos a caixa de texto) e o botão "Se esconda!". A caixa de texto deve mostrar quando o oponente for encontrado e quantos movimentos foram feitos até achá-lo.

8 **Mantenha registro de quantos movimentos o jogador efetuou**
Certifique-se de que a caixa de texto mostre o número de vezes que o jogador verificou um esconderijo ou se moveu entre salas. Quando o oponente for encontrado, uma caixa de mensagem deve ser mostrada com "Você me achou em X movimentos!".

9 **Faça com que as coisas pareçam certas quando o jogo começar**
Quando você iniciar o programa pela primeira vez, tudo o que deve ver é uma caixa de texto vazia e um botão "Se esconda!". Quando você clicar o botão, a brincadeira começa!

Solução do Exercício

Faça seu próprio programa original de simulação adicionando mais cômodos esconderijos e um oponente que se esconde de você.

Eis a nova interface IHidingPlace. Ela tem apenas um campo string com um acessador get que retorna o nome desse lugar.

```
public interface IHidingPlace {
  string HidingPlaceName { get; }
}
public class RoomWithHidingPlace : Room, IHidingPlace {
  public RoomWithHidingPlace(string name, string decoration, string hidingPlaceName)
    : base(name, decoration)
  {
    this.hidingPlaceName = hidingPlaceName;
  }
  private string hidingPlaceName;
  public string HidingPlaceName {
    get { return hidingPlaceName; }
  }
  public override string Description {
    get {
      return base.Description + " Someone could hide " + hidingPlaceName + ".";
    }
} }
public class RoomWithDoor : RoomWithHidingPlace, IHasExteriorDoor {
  public RoomWithDoor(string name, string decoration,
                      string hidingPlaceName, string doorDescription)
    : base(name, decoration, hidingPlaceName)
  {
    this.doorDescription = doorDescription;
  }
  private string doorDescription;
  public string DoorDescription {
    get { return doorDescription; }
  }
  private Location doorLocation;
  public Location DoorLocation {
    get { return doorLocation; }
    set { doorLocation = value; }
  }
}
```

A classe RoomWithHidingPlace herda de Room e implementa IHidingPlace adicionando a propriedade HidingPlaceName. O construtor inicializa o seu campo de armazenamento.

Como decidirmos que cada sala com uma porta também precisará de um esconderijo, RoomWithDoor herdará de RoomWithHidingPlace. A única mudança nela é que seu construtor recebe o nome de um esconderijo e o envia para o construtor de RoomWithHidingPlace.

A classe OutsideWithHidingPlace herda de Outside e implementa IHidingPlace, exatamente como RoomWithHidingPlace faz.

```
public class OutsideWithHidingPlace : Outside, IHidingPlace {
  public OutsideWithHidingPlace(string name, bool hot, string hidingPlaceName)
    : base(name, hot)
  { this.hidingPlaceName = hidingPlaceName; }
  private string hidingPlaceName;
  public string HidingPlaceName {
    get { return hidingPlaceName; }
  }
  public override string Description {
    get {
      return base.Description + " Someone could hide " + hidingPlaceName + ".";
    }
  }
} }
```

O construtor da classe Opponent recebe como parâmetro um local inicial. Ele cria uma nova instância de Random, que usa pra se mover aleatoriamente pelas salas.

```
public class Opponent {
  private Random random;
  private Location myLocation;
  public Opponent(Location startingLocation) {
    myLocation = startingLocation;
    random = new Random();
  }
  public void Move() {
    if (myLocation is IHasExteriorDoor) {
      IHasExteriorDoor LocationWithDoor =
                          myLocation as IHasExteriorDoor;
      if (random.Next(2) == 1)
        myLocation = LocationWithDoor.DoorLocation;
    }
    bool hidden = false;
    while (hidden) {
      int rand = random.Next(0, myLocation.Exits.Length);
      myLocation = myLocation.Exits[rand];
      if (myLocation is IHidingPlace)
        hidden = true;
    }
  }
  public bool Check(Location locationToCheck) {
    if (locationToCheck != myLocation)
      return false;
    else
      return true;
  }
}
```

O método Move() primeiro checa para ver se a sala atual tem uma porta usando a palavra-chave "is" - se for o caso existe uma chance de 50% de atravessá-la. Então ele faz a mudança para um local aleatório e continua se movendo até encontrar um lugar para se esconder.

A parte principal do método Move() é o laço while. Ele continua executando até que a variável hidden seja verdadeira - e ela só fica nesse estado quando um lugar para se esconder é encontrado.

O método Check() apenas checa a posição do oponente contra o local que é passado como parâmetro - uma referência do tipo Location. Se as duas apontam para o mesmo objeto, o oponente foi encontrado!

Não acabamos ainda - vire a página!

Solução do Exercício (continuação)

Aqui estão os campos para a classe Form1. Ela os utiliza para manter registro dos locais, do oponente e do número de movimentos que o jogador fez.

```
int Moves;
Location currentLocation;
RoomWithDoor livingRoom;
RoomWithHidingPlace diningRoom;
RoomWithDoor kitchen;
Room stairs;
RoomWithHidingPlace hallway;
RoomWithHidingPlace bathroom;
RoomWithHidingPlace masterBedroom;
RoomWithHidingPlace secondBedroom;
OutsideWithDoor frontYard;
OutsideWithDoor backYard;
OutsideWithHidingPlace garden;
OutsideWithHidingPlace driveway;
Opponent opponent;
```

O construtor de Form1 cria os objetos e o oponente e reinicia o jogo. Adicionamos um parâmetro booleano em ResetGame() para que ele mostre a mensagem apenas quando você ganhar, assim ela não aparecerá quando o programa for ativado pela primeira vez.

```
public Form1() {
  InitializeComponent();
  CreateObjects();
  opponent = new Opponent(frontYard);
  ResetGame(false);
}
private void MoveToANewLocation(Location newLocation) {
  Moves++;
  currentLocation = newLocation;
  RedrawForm();
}
private void RedrawForm() {
  exits.Items.Clear();
  for (int i = 0; i < currentLocation.Exits.Length; i++)
    exits.Items.Add(currentLocation.Exits[i].Name);
  exits.SelectedIndex = 0;
  description.Text = currentLocation.Description + "\r\n(move #" + Moves + ")";
  if (currentLocation is IHidingPlace) {
    IHidingPlace hidingPlace = currentLocation as IHidingPlace;
    check.Text = "Check " + hidingPlace.HidingPlaceName;
    check.Visible = true;
  }
  else
    check.Visible = false;
  if (currentLocation is IHasExteriorDoor)
    goThroughTheDoor.Visible = true;
  else
    goThroughTheDoor.Visible = false;
}
```

O método MoveToANewLocation() (mover para um novo lugar) estabelece um novo local e então repinta o formulário.

RedrawForm() povoa a lista da caixa de combinação, mostra o texto (adicionando o número de movimento) e torna os botões visíveis ou invisíveis, dependendo de existir ou não uma porta ou se a sala tem um esconderijo.

Precisamos do nome do esconderijo, mas temos apenas o objeto CurrentLocation (local atual) que não tem uma propriedade HidingPlaceName (nome do esconderijo). Assim, temos que usar as para fazer a subcoerção (downcast) da referência para uma variável do tipo IHidingPlace.

Uau, você adicionou toda uma ala na casa apenas com poucas linhas! É por isso que classes e objetos corretamente encapsulados são realmente uteis.

```
private void CreateObjects() {
  livingRoom = new RoomWithDoor("Living Room", "an antique carpet",
        "inside the closet", "an oak door with a brass handle");
  diningRoom = new RoomWithHidingPlace("Dining Room", "a crystal chandelier",
        "in the tall armoire");
  kitchen = new RoomWithDoor("Kitchen", "stainless steel appliances",
        "in the cabinet", "a screen door");
  stairs = new Room("Stairs", "a wooden bannister");
  hallway = new RoomWithHidingPlace("Upstairs Hallway", "a picture of a dog",
        "in the closet");
  bathroom = new RoomWithHidingPlace("Bathroom", "a sink and a toilet",
        "in the shower");
  masterBedroom = new RoomWithHidingPlace("Master Bedroom", "a large bed",
        "under the bed");
  secondBedroom = new RoomWithHidingPlace("Second Bedroom", "a small bed",
        "under the bed");
  frontYard = new OutsideWithDoor("Front Yard", false, "a heavy-looking oak door");
  backYard = new OutsideWithDoor("Back Yard", true, "a screen door");
  garden = new OutsideWithHidingPlace("Garden", false, "inside the shed");
  driveway = new OutsideWithHidingPlace("Driveway", true, "in the garage");
  diningRoom.Exits = new Location[] { livingRoom, kitchen };
  livingRoom.Exits = new Location[] { diningRoom, stairs };
  kitchen.Exits = new Location[] { diningRoom };
  stairs.Exits = new Location[] { livingRoom, hallway };
  hallway.Exits = new Location[] { stairs, bathroom, masterBedroom, secondBedroom };
  bathroom.Exits = new Location[] { hallway };
  masterBedroom.Exits = new Location[] { hallway };
  secondBedroom.Exits = new Location[] { hallway };
  frontYard.Exits = new Location[] { backYard, garden, driveway };
  backYard.Exits = new Location[] { frontYard, garden, driveway };
  garden.Exits = new Location[] { backYard, frontYard };
  driveway.Exits = new Location[] { backYard, frontYard };
  livingRoom.DoorLocation = frontYard;
  frontYard.DoorLocation = livingRoom;
  kitchen.DoorLocation = backYard;
  backYard.DoorLocation = kitchen;
}
```

O novo método CreateObjects() cria todos os objetos necessários para montar a casa. Ele parece bastante com o antigo, mas acrescenta muitos lugares para onde podemos ir.

Não acabamos ainda - vire a página!

Solução do Exercício (continuação)

Aqui está o resto do código do formulário. Os tratadores de evento dos botões gottere e goThroughTheDoor são idênticos àqueles da primeira parte deste exercício; volte umas páginas se quiser vê-los.

O método ResetGame() reinicia o jogo. Ele mostra a mensagem e faz com que todos os botões, exceto Se esconda!, fiquem invisíveis.

```
private void ResetGame(bool displayMessage) {
  if (displayMessage) {
    MessageBox.Show("You found me in " + Moves + " moves!");
    IHidingPlace foundLocation = currentLocation as IHidingPlace;
    description.Text = "You found your opponent in " + Moves
       + " moves! He was hiding " + foundLocation.HidingPlaceName + ".";
  }
  Moves = 0;
  hide.Visible = true;
  goHere.Visible = false;
  check.Visible = false;
  goThroughTheDoor.Visible = false;
  exits.Visible = false;
}
private void check_Click(object sender, EventArgs e) {
  Moves++;
  if (opponent.Check(currentLocation))
    ResetGame(true);
  else
    RedrawForm();
}
private void hide_Click(object sender, EventArgs e) {
  hide.Visible = false;
  for (int i = 1; i <= 10; i++) {
    opponent.Move();
    description.Text = i + "... ";
    Application.DoEvents();
    System.Threading.Thread.Sleep(200);
  }
  description.Text = "Ready or not, here I come!";
  Application.DoEvents();
  System.Threading.Thread.Sleep(500);
  goHere.Visible = true;
  exits.Visible = true;
  MoveToANewLocation(livingRoom);
}
```

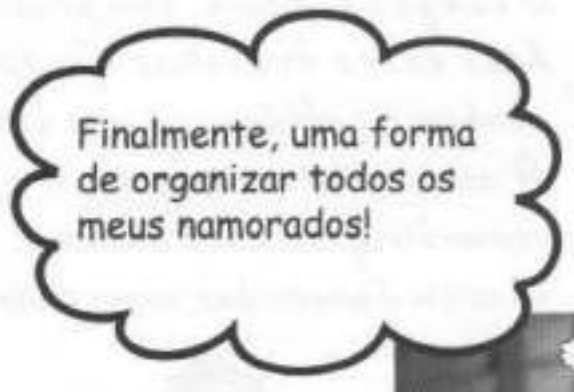
Queremos mostrar o nome do esconderijo, mas CurrentLocation é uma referência a Location que não nos dá acesso ao campo HidingPlaceName. Por sorte, podemos usar a palavra-chave as para fazer a subcoerção (downcast) para uma referência a IHidingPlace que aponte para o mesmo objeto.

Quando você clica o botão check ele verifica se o oponente está escondido ou não na sala atual. Se ele está, isso encerra o jogo. Se não, o formulário é repintado (para atualizar o número de movimentos).

O botão hide é o que começa o jogo. A primeira coisa que ele faz é deixar ele mesmo invisível. Então ele conta até 10 e diz ao oponente para se mover. Finalmente ele faz com que o botão superior e a caixa de combinação figuem visíveis e coloca o jogador na sala de estar. O método MoveToANewLocation() chama RedrawForm().

8 enumerações e coleções

Guardando muitos dados



Quando chove, pode ser garoa ou tempestade.

No mundo real, você não costuma lidar com seus dados em pequenas quantidades, pois eles surgem em **quantidade, aos montes, em pilhas**. Você precisará de ferramentas muito poderosas para organizá-los e é aí que as coleções entram. Elas permitem **armazenar, ordenar e gerenciar** todos os dados necessários ao seu programa para se concentrar. Assim, você pode pensar sobre escrever programas que trabalhem com seus dados e deixar as coleções tomarem conta deles pra você.

Strings nem sempre servem para armazenar categorias de dados

Suponha que você tenha muitas abelhas operárias, todas representadas pela classe `Worker` (trabalhador, operário). Como você escreveria um construtor que esperasse a tarefa a ser realizada como parâmetro? Se você usar uma string para a descrição, pode acabar com código que se parece com o seguinte:

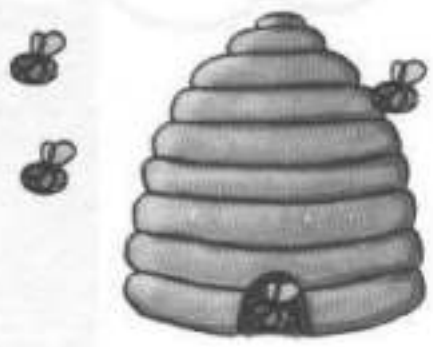
Nosso programa de gerenciamento de abelhas mantém registro das tarefas atribuídas a cada uma usando uma seqüência de caracteres como patrulheira ou coletora de néctar.

Nosso código permitiria que todos esses valores fossem passados para um construtor mesmo que o programa suporte apenas Patrulheira, Coletora de Néctar e outros trabalhos que as abelhas fazem.

```
Worker buzz = new Worker( "Procuradora do Estado");
Worker clover = new Worker("Tratadora de Cães");
Worker gladys = new Worker("Apresentadora de Notícias");
```

O código compilará, sem problemas. Mas esses trabalhos não fazem nenhum sentido para uma abelha. A classe Worker não deveria permitir que esses valores fossem fornecidos como válidos.

Provavelmente, você pode adicionar código ao construtor de `Worker` para checar se cada string fornecida é um trabalho válido para abelhas. Mas, caso você adicione novos trabalhos que as abelhas podem fazer, terá de alterar esse código e compilar o programa novamente. Mas essa seria uma solução ruim. O que aconteceria se você tivesse outras classes que precisassem checar o tipo de abelhas que elas poderiam ser? Você teria que duplicar esse código de checagem e isso é não é um bom caminho a seguir. O que você precisa é uma forma de dizer "Ei, existem apenas alguns valores permitidos aqui". Você precisa **enumerar** os valores que podem ser usados.



Enums permitem enumerar um conjunto de valores válidos

Um enum é um tipo de dados que permite apenas certos valores. Assim poderíamos definir uma matriz chamada `Tarefas` e definir as tarefas permitidas:

Este é o nome da enum.

```
public enum Tarefa {
    ColetoraDeNectar,
    Patrulheira,
    ManutencaoColmeia,
    EnsinarLarvas,
    CuidarDosOvos,
    FazerMel,
}
```

Cada uma dessas é uma tarefa válida. Qualquer uma pode ser usada como valor para Tarefas.

O último enumerador não precisa terminar com uma vírgula, mas isto facilita rearranjar os itens usando copiar e colar.

Separe cada valor com uma vírgula e termine a coisa toda com uma chave.

*O contido entre as chaves é chamado de lista de **enumeração** e cada um dos itens é um **enumerador**. A coisa toda é chamada de **enumeração**.*

*Mas a maioria das pessoas chama-as apenas de **enums**.*

Agora, você pode fazer referências a ela com tipos como o seguinte:

Este é o nome da enum.

Finalmente, o valor que você quer dentre os enumerados.

```
Worker baba = new Worker(Tarefas.CuidarDosOvos);
```

Mudamos o construtor de Worker para aceitar Worker.Tarefas como tipo de parâmetro.

Qualquer outro valor não é permitido. Você não pode simplesmente inventar um novo valor para a enumeração. Se fizer isso, o programa não compilará.

```
private void button1_Click(object sender EventArgs e)
{
    Worker buzz = new Worker(Tarefas.ProcuradorDoEstado);
}
```

Tarefas não contêm uma definição para ProcuradorDoEstado

```
'Jobs' does not contain a definition for
'AttorneyGeneral'
```

Enums permitem representar números usando nomes

Algumas vezes é mais fácil trabalhar com números se você tiver nomes para eles. Você pode atribuir números para os valores numa enum e usar os nomes para referir-se a eles. Dessa forma, você não terá um monte de números sem significado espalhados pelo seu código. Aqui está uma enumeração para manter registro de pontuação numa competição canina. Ela é definida **dentro da classe CompeticaoCanina**. Assim, se você quiser usá-la fora da classe deverá escrever `CompeticaoCanina.PontosParaTruques`.

Esta é a enum CompeticaoCanina. PontosParaTruques.

Eles não precisam aparecer em nenhuma ordem em particular e se pode dar múltiplos nomes para o mesmo número.

Forneça um nome, depois o "=" e a seguir o número.

```
class CompeticaoCanina{
  public enum PontosParaTruques {
    Sentar = 7,
    DarPata = 25,
    Rolar = 50,
    Buscar = 10,
    Junto = 5,
    Latir = 20,

  }

  ...
  // codigo mais adiante na classe
  int score = (int) PontosParaTruques.Buscar * 3;
  MessageBox.Show(score.ToString());


  MessageBox.Show(PontosParaTruques.Junto.ToString())
```

A (int) assim diz o compilador para a transformar a número representado. Portanto, uma vez que tem PontosParaTrugues.Falar um índice de 20, (int) PontosParaTrugues.Falar. coloca o valor para 20.

mostra "30", 10*3

ToString() retorna o nome... Nesse caso, "Junto".

Pode-se fazer coerção da enum para um número e fazer cálculos com o resultado, ou você pode usar o método `ToString()` para lidar com o nome como uma seqüência de caracteres. Se você não atribuir nenhum número para os nomes, os itens na lista serão numerados pelos seus índices como padrão. O primeiro item terá o valor 0, o segundo 1, etc.

Você pode colocar uma enum dentro de uma classe dessa forma, ou ela pode existir por si só fora de uma classe.

Mas o que acontece se você quiser usar números realmente grandes num dos enumeradores? O tipo padrão para números numa enumeração é `int`, então você precisará especificar o tipo se tiver que usar outro, usando o operador :, como no exemplo que segue:

```
public enum PontosParaTruques: long {
    Sentar = 7,
    DarPata = 2500000000025
}
```

Isso diz ao compilador para tratar os números dentro da enum PontosParaTrugues como do tipo long, não int.

Se você tentar compilar seu código sem especificar long como tipo, verá a seguinte mensagem:

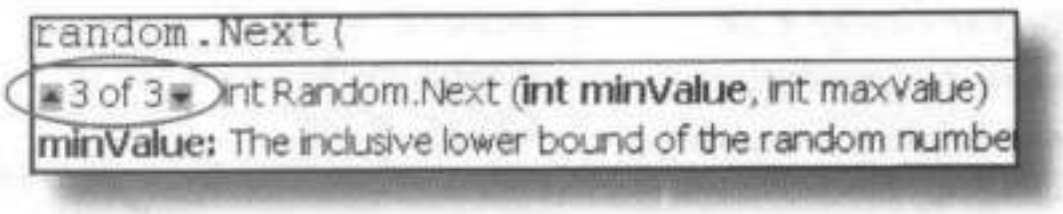
```
Cannot implicitly convert type 'long' to 'int'.
```

não existem Perguntas Idiotas

P: Espere um pouco. Quando eu estava digitando este código, notei que uma janela do IntelliSense apareceu dizendo algo como "3 de 3" (3 of 3) quando eu estava usando o construtor de Random.Next(). O que isso quer dizer?

R: O que você viu foi um construtor sobrecarregado. Quando uma classe tem um método que você pode chamar de mais de uma forma, isso é chamado de sobrecarga (overloading). Quando você utiliza uma classe com um método sobrecarregado, o IDE informa todas as opções que você tem. Nesse caso, a classe Random tem três métodos Next() possíveis.

Assim que você digita "random.Next(" na janela de código, o IDE mostra uma janela do IntelliSense que exibe os parâmetros para os diferentes métodos sobrecarregado. As setas para cima e para baixo em torno do "3 de 3" deixam que você navegue entre as opções. Isso é muito útil quando você está lidando com um método com uma dezena ou mais de definições sobrecarregadas. Quando estiver fazendo isso, certifique-se de escolher o método Next() certo! Mas não se preocupe muito com isso agora - falaremos mais de sobrecarga mais adiante no capítulo.



Exercício

Use o que você aprendeu sobre enums para desenvolver uma classe que represente cartas de baralho.

1 **Crie um novo projeto e adicione uma classe Card**
Você precisará de dois campos públicos: Suit (naipe), que pode ser Spades (espadas), Clubs (paus), Diamonds (ouros) ou Hearts (copas) e Value (valor), que pode ser Ace (Ás), Two (dois), Three (três)... Ten (dez), Jack (valete), Queen (damas) e King (rei). E você deve criar uma propriedade apenas leitura Name ("Ás de Espadas", "Cinco de Ouros", etc.).

2 **Use duas enumerações para definir naipes e valores**
Certifique-se de que `(int) Card.Suits.Spades` seja igual a zero, seguido de `Clubs` (1), `Diamonds` (2) e `Hearts` (3). Faça os valores idênticos aos nominais: `(int) Card.Values.Ace` deve ser 1, `Two` deve ser 2, etc. `Jack` deve ser igual a 11, `Queen` igual a 12 e `King` igual a 13.

3 **Adicione uma propriedade para o nome da carta**
`Name` deve ser uma propriedade apenas leitura. O acessador get deve retornar uma seqüência de caracteres que descreva a carta. Este código executará num formulário que chama a propriedade `Name` de uma carta e a exibe:

```
Card card = new Card(Card.Suits.Spades, Card.Values.Ace);
string cardName = card.Name;
```

Para isto funcionar, sua classe Card precisará de um construtor com dois parâmetros.

O valor de card.Name deve ser "Ace of Spades" (ás de espadas).

4 **Adicione um botão ao formulário que mostre o nome de uma carta aleatoriamente**
Seu programa pode criar uma carta com um naipe e valor aleatórios fazendo a coerção de um número entre 0 e 3 para `Cards.Suits` e outro número aleatório entre 1 e 13 como `Cards.Values`. Para isto, você pode se aproveitar de uma característica da classe predefinida `Random` que permite ao método `Next()` ser chamado de três formas diferentes:

*Quando você tem mais de uma forma de chamar um método, isso é chamado de **sobrecarga**. Veremos mais sobre isso mais adiante...*

```
Random random = new Random();
int numberBetween0and3 = random.Next(4);
int numberBetween1and13 = random.Next(1, 14);
int anyRandomInteger = random.Next();
```

Isso diz a Random para retornar um valor que seja pelo menos 1, mas menor que 14.

Solução do Exercício

Um baralho é um ótimo exemplo em que limitar os valores é importante. Ninguém quer mostrar suas cartas e ver um Coringa de Paus ou um 13 de Copas. Aqui está como escrevemos a classe Cards.

```
public class Card {
  public enum Suits {
    Spades,
    Clubs,
    Diamonds,
    Hearts
  }
  public enum Values {
    Ace = 1,
    Two = 2,
    Three = 3,
    Four = 4,
    Five = 5,
    Six = 6,
    Seven = 7,
    Eight = 8,
    Nine = 9,
    Ten = 10,
    Jack = 11,
    Queen = 12,
    King = 13
  }
  public Suits Suit;
  public Values Value;
  public Card(Suits suit, Values value) {
    this.Suit = suit;
    this.Value = value;
  }
  public string Name {
    get { return Value.ToString() + " of " + Suit.ToString(); }
  }
}
private void button1_Click(object sender, EventArgs e) {
  Random random = new Random();
  Card card = new Card((Card.Suits)random.Next(4), (Card.Values)random.Next(1, 14));
  MessageBox.Show(card.Name);
}
```

Quando você não especifica valores, o primeiro item na lista tem valor zero, o segundo 1, e assim por diante.

Eis onde atribuímos o valor de Cards.Values.Ace para 1.

Quando você cria seus campos Suit e Value, pode usar apenas Suit, em vez de Cards.Suit, uma vez que você está dentro da classe Card.

O acessador get da propriedade Name pode aproveitar-se da forma como o método ToString() de uma enum retorna seu nome convertido numa seqüência de caracteres.

Aqui é onde usamos o método sobrecarregado Random.Next() para gerar um número aleatório para fazer a coerção para a enum.

Nós <u>poderíamos</u> ter usado uma matriz para criar um baralho...

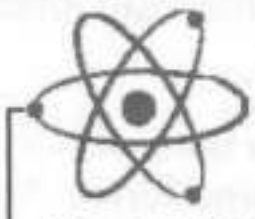
E se você quiser criar uma classe para representar um baralho? Será preciso uma forma de manter registro de cada carta no baralho e uma maneira de saber a ordem em que elas estão. Uma array do tipo Card poderia servir—a carta mais acima do baralho estaria no indice 0, a seguinte no 1 e assim por diante. Eis um ponto de partida—uma classe Deck (baralho) que começa com um conjunto completo de 52 cartas.

```
public class Deck {
  private Card[] cards = {
    new Card(Card.Suits.Spades, Card.Values.Ace),
    new Card(Card.Suits.Spades, Card.Values.Two),
    new Card(Card.Suits.Spades, Card.Values.Three),
    // ...
    new Card(Card.Suits.Diamonds, Card.Values.Queen),
    new Card(Card.Suits.Diamonds, Card.Values.King),
  };
  public void PrintCards() {
    for (int i = 0; i < cards.Length; i++)
      Console.WriteLine(cards[i].ToString());
  }
}
```

Essa declaração de matriz continuaria por todas as cartas. Apenas abreviamos aqui para poupar espaço.

... mas e se você quisesse fazer mais que isso?

Pense em tudo o que você poderia precisar fazer com um baralho. No entanto, se você estiver jogando com as cartas, precisa mudar freqüentemente a ordem delas e adicionar ou remover cartas do baralho. Não é muito fácil fazer isso com uma matriz.



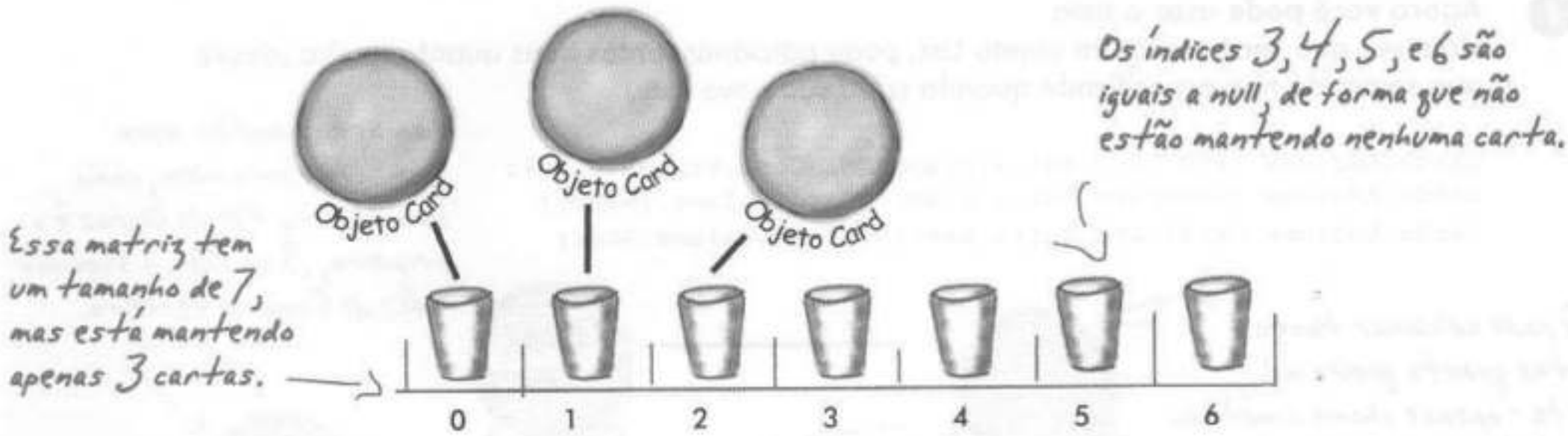
Poder do Cérebro

Como você poderia adicionar um método Shuffle() (embaralhar) na classe Deck para reordenar as cartas em posições aleatórias? E como fazer um método para remover a primeira carta de cima do baralho? E como adicionar uma carta?

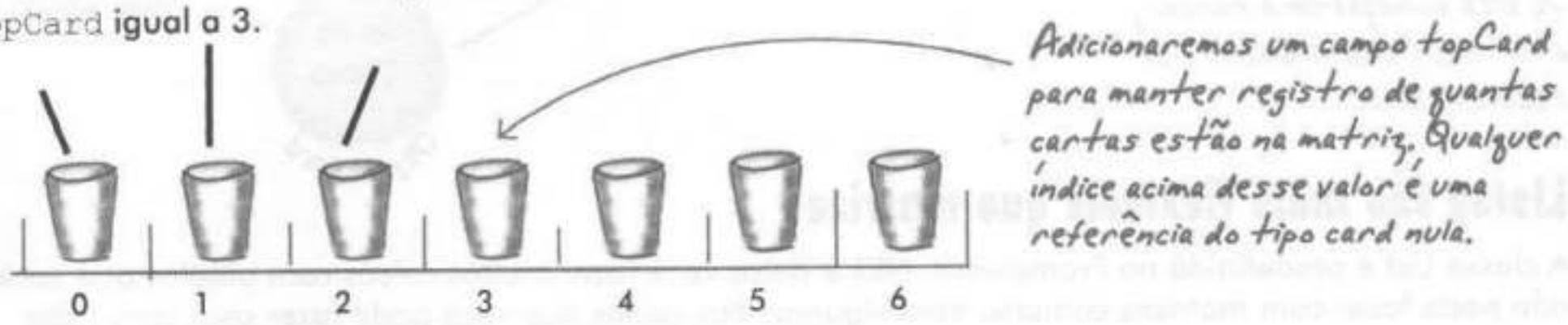
Matrizes são difíceis de trabalhar

Uma matriz é ótima para armazenar uma lista fixa de valores ou referências. Mas uma vez que você precisa mover os elementos, ou adicionar mais, as coisas começam a ficar meio chatas.

1 Todas as matrizes têm um tamanho e você precisa conhecê-lo para trabalhar com ela. Você pode usar referências nulas para manter alguns dos elementos vazios.



2 Você precisa manter registro de quantas cartas existem no baralho. Então precisa de um campo int, que você poderia chamar de `topCard` (carta no topo), que mantém o índice da última carta no baralho. De forma que nossa matriz de 3 cartas tem um tamanho de 7, mas teríamos `topCard` igual a 3.



Existe um método Array.Resize() predefinido no Framework .NET que faz exatamente isso.

3 Mas as coisas começam a ficar complicadas. É fácil colocar um método Peek() (literalmente "espiar") que retornasse uma referência à carta em cima do baralho - de forma que você pode espiar o topo das cartas. Mas e se você quiser adicionar uma carta? Se topCard for menor que o tamanho da matriz, você pode apenas colocar a carta num dos índices nulos e somar 1 no indicador. Mas se a matriz estiver cheia, você terá de criar uma matriz maior e copiar as cartas para ela. Remover é fácil - mas antes de subtrair 1 de topCard, você precisa certificar-se de ter igualado a referência da carta removida para null. E se você precisar tirar uma carta **do meio do baralho**? Se você remover a quarta carta, terá de mover a quinta para seu lugar e a sexta para o lugar da quinta e assim por diante... puxa, que bagunça!

Listas facilitam guardar coleções de... qualquer coisa

O Framework .NET tem diversas classes de **coleções** que lidam com as situações complicadas ocorridas entre você e um elemento a ser removido. O tipo mais comum de coleção é uma **Lista**. Uma vez que você cria um objeto do tipo Lista, é fácil adicionar, remover, observar e mesmo mover itens de um lugar da lista para outro. Eis como uma Lista funciona:

1 **Primeiro você cria uma nova instância de List**
Todas as matrizes têm um tipo - você não usa somente uma matriz, você usa uma matriz de inteiros, de objetos Card, etc. Listas funcionam da mesma forma. Você precisa especificar o tipo de objeto ou valor a ser armazenado na lista colocando-o entre sinais de menor e maior < > quando a palavra chave `new` for usada.

```
List cards = new List<Card>();
```

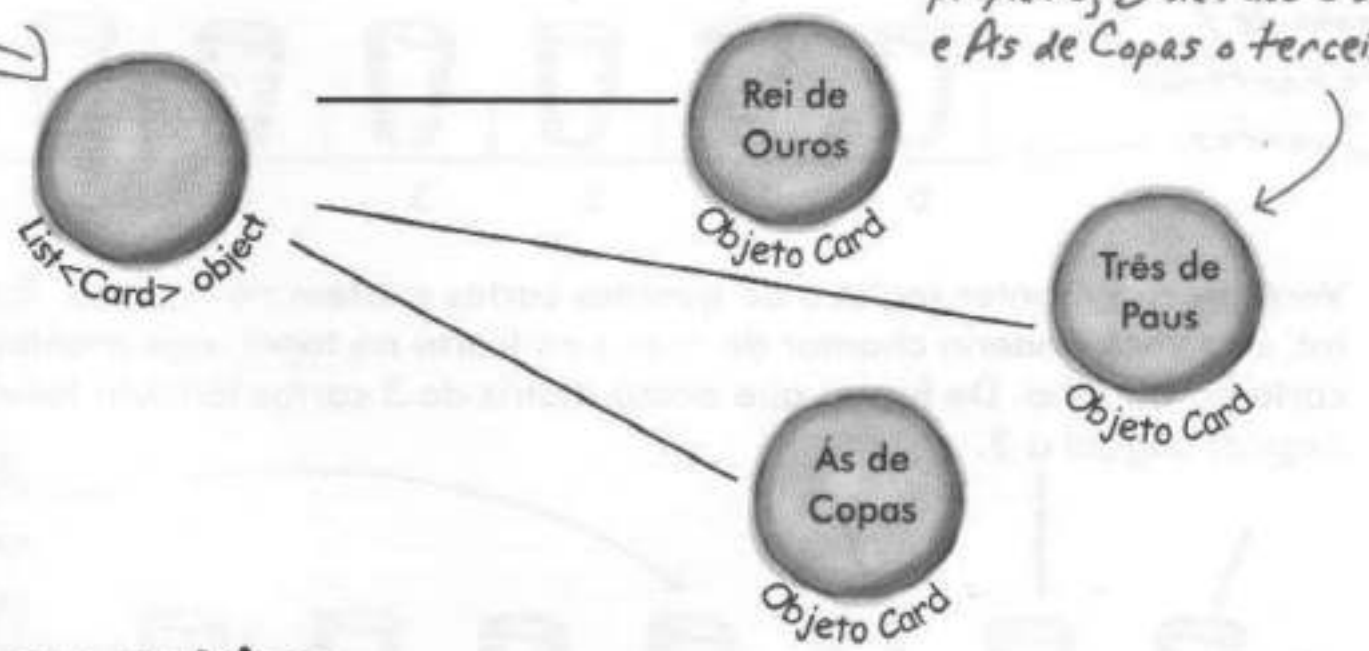
Você especificou <Card> quando criou a lista, então agora essa lista mantém referências somente do tipo Card.

2 **Agora você pode usar a lista**
Uma vez que você tenha um objeto List, pode adicionar tantos itens quanto queira (desde que sejam do tipo especificado quando criou sua nova lista).

```
cards.Add(new Card(Card.Suits.Diamonds, Card.Values.King);
cards.Add(new Card(Card.Suits.Clubs, Card.Values.Three);
cards.Add(new Card(Card.Suits.Hearts, Card.Values.Ace);
```

Uma lista mantém seus elementos em ordem, como uma matriz. Rei de Ouros é o primeiro, 3 de Paus o segundo e Ás de Copas o terceiro.

Você pode adicionar tantas cartas quanto queira na Lista - apenas chame o método Add(). Ele se certificará de que você tem espaços suficientes para os itens. Se eles começarem a rarear, a lista automaticamente se redimensiona.



Listas são mais flexíveis que matrizes

A classe List é predefinida no Framework .NET e deixa você fazer muitas coisas com objetos que você não pode fazer com matrizes comuns. Veja algumas das coisas que você pode fazer com uma Lista.

1 **Você pode criar uma.**

```
List<Egg> myCarton = new List<Egg>();
```

2 **Adicione alguma coisa nela.**

```
Egg x = new Egg();
myCarton.Add(x);
```

3 **Adicione mais alguma coisa nela.**

```
Egg y = new Egg();
myCarton.Add(y);
```

4 **Saiba quantas coisas estão dentro dela**

```
int theSize = myCarton.Count;
```

5 **Descubra se algum item em particular está dentro dela.**

Você pode procurar por qualquer Egg dentro da lista. Isso com certeza retorna true.

```
bool Isin = myCarton.Contains(x);
```

6 **Veja onde essa coisa está guardada.**

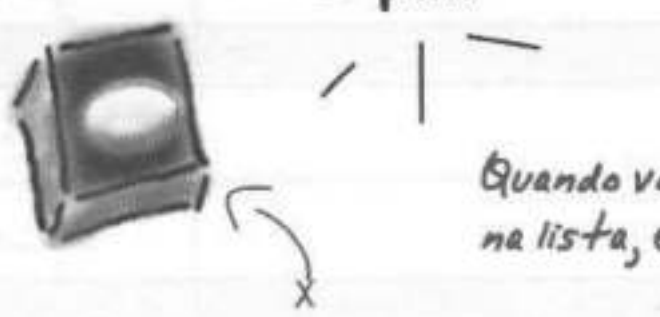
A lista deve retornar o índice 0 para x e 1 para y.

```
int idx = myCarton.IndexOf(y);
```

7 **Determine quanto vai caber na lista.**

Isso vai dizer a você o número de objetos que a lista pode suportar antes de ter que se redimensionar.

```
int limit = myCarton.Capacity;
```



Quando você removeu y, deixamos apenas x na lista, então ela encolheu!

8 **Tire algo dela.**

```
myCarton.Remove(y)
```

Aponte seu lápis

Preencha o restante da tabela abaixo olhando o código para as listas no lado esquerdo e colocando o que você acha que seria o equivalente usando uma matriz comum. Não esperamos que você acertasse todos eles, mas faça o melhor que puder.

Assuma que esses comandos são todos executados na ordem, um após o outro.

Preenchemos alguns para você...

Lista	Matriz comum
`List<String> myList = new List <String>();`	String[ ] myList = new String [2]
`String a = "Yay!";`	String a = Yay!;
`myList.Add(a);`	
`String b = "Bummer";`	String b = Bummer;
`myList.Add(b);`	
`object o = myList[1];`	
`int theSize = myList.Count;`	
`bool isIn = myList.Contains(b);` Dica: você vai precisar de mais de uma linha de código aqui. →	

Aponte seu lápis
Solução

Sua tarefa foi preencher o restante da tabela abaixo olhando o código para as listas no lado esquerdo e colocando o que você acha que seria o equivalente usando uma matriz comum.

`List<String> myList = new List <String>();`	`String[] myList = new String[2];`
`String a = "Yay!";`	`String a = "Yay!";`
`myList.Add(a);`	`myList[0] = a;`
`String b = "Bummer";`	`String b = "Bummer";`
`myList.Add(b);`	`myList[1] = b;`
`object o = myList[1];`	`int theSize = myList.Length;`
`int theSize = myList.Count;`	`object o = myList[1];`

`bool isIn = myList.Contains(b);`	`bool isIn = false;`
	`for (int i = 0; i < myList.`
	`Length; i++) {`
	`if (b == myList[i]) {`
	`isIn = true;`
	`}`
	`}`

Listas são objetos que usam métodos exatamente como qualquer outra classe utilizada até agora. Você pode ver a lista de métodos disponíveis usando o ISE, apenas digitando um ponto depois do nome de qualquer lista. Passar parâmetros para ela também funciona da mesma forma que para uma classe que você mesmo tenha escrito.

Com matrizes você fica muito mais limitado. Quando criá-la você precisa definir seu tamanho e qualquer lógica a lhe ser aplicada deve ser escrita à parte.

O Framework .NET tem uma classe Array (matriz) que facilitam algumas coisas ... mas estamos nos concentrando em objetos do tipo List porque eles são muito mais fáceis de usar.

Listas encolhem e aumentam dinamicamente

Uma coisa ótima nas listas é que você não precisa saber qual vai ser o tamanho dela quando você a cria. Ela automaticamente aumenta e diminui para se adequar ao conteúdo. Eis um exemplo de alguns dos métodos que fazem com que trabalhar com o tipo List seja mais fácil do que fazer o mesmo com matrizes:

Estamos declarando uma lista de objetos Shoe (sapato) chamada ShoeCloset (sapateira).

```
List<Shoe> shoeCloset = new List<Shoe>();
shoeCloset.Add(new Shoe()
  { Style = Style.Sneakers, Color = "Black" });
shoeCloset.Add(new Shoe()
  { Style = Style.Clogs, Color = "Brown" });
shoeCloset.Add(new Shoe()
  { Style = Style.Wingtips, Color = "Black" });
shoeCloset.Add(new Shoe()
  { Style = Style.Loafers, Color = "White" });
shoeCloset.Add(new Shoe()
  { Style = Style.Loafers, Color = "Red" });
shoeCloset.Add(new Shoe()
  { Style = Style.Sneakers, Color = "Green" });
int numberOfShoes = shoeCloset.Count;
foreach (Shoe shoe in shoeCloset) {
  shoe.Style = Style.Flipflops;
  shoe.Color = "Orange";
}
```

Você pode usar um comando new dentro do método List.Add().

Um tipo especial de laço para listas é o `foreach` (para cada). Ele vai executar um comando para cada objeto na lista. Esse laço cria um identificador chamado `shoe`. Conforme o laço vai avançando pelos itens, ele faz com que `shoe` seja igual ao primeiro item, depois ao segundo, e assim por diante até o laço terminar.

Isso retorna o número total de objetos Shoe na lista.

Esse laço foreach examina cada um dos sapatos na sapateira.

Os laços foreach funcionam em matrizes também! De fato, eles funcionam em qualquer coleção.

O método RemoveO vai retirar um objeto por referência e RemoveAtO (remover em) faz o mesmo mas usa o índice.

O método ClearO (limpar) remove todos os objetos na lista.

Salvamos as referências de dois sapatos antes de limpar a lista. Adicionamos uma delas de volta, mas a outra ainda está faltando.

```
shoeCloset.RemoveAt(4);

Shoe thirdShoe = shoeCloset[3];
Shoe fifthShoe = shoeCloset[5];
shoeCloset.Clear();

shoeCloset.Add(thirdShoe);
if (shoeCloset.Contains(fifthShoe))
  Console.WriteLine("That's surprising.");
```

Essa linha nunca executa, porque ContainsO (contém) vai retornar false. Apenas adicionamos thirdShoe (terceiro sapato) na lista depois de limpá-la, não fifthShoe (quinto sapato).

Aqui está a classe Shoe que estamos usando...

```
public class Shoe {
  public Style Style;
  public string Color;
}
public enum Style {
  Sneakers,
  Loafers,
  Sandals,
  Flipflops,
  Wingtips,
  Clogs,
}
```

Objetos List podem armazenar qualquer tipo

Você já viu que uma lista pode armazenar strings ou sapatos. Você pode também fazer listas de inteiros, ou qualquer outro tipo de objeto que criar. Isso faz com que a lista seja chamada de **coleção genérica**. Quando você cria um novo objeto List, você o associa com um tipo específico: pode ser inteiro, string, ou objetos Shoe. Isso facilita trabalhar com listas - uma vez ela criada, você sempre sabe o tipo de dados que armazena.

Na realidade, isso não quer dizer que você está adicionando algo chamado T. É a notação para que você entenda quando uma classe ou interface é capaz de trabalhar com qualquer tipo. A parte <T> quer dizer que você pode colocar um tipo aqui, como, por exemplo, List<Sapato>, e isso limita o uso para aquele tipo.

```
List name = new List<T>();
```

Listas podem ser muito flexíveis (permitindo o uso de qualquer tipo) ou muito restritivas. Ou seja, elas podem fazer o que as matrizes fazem e muitas coisas mais.

O Framework .NET possui algumas interfaces genéricas. Elas permitem às coleções desenvolvidas por você funcionarem com qualquer tipo também. O objeto List implementa essas interfaces e é por isso que você pode criar uma lista de inteiros e trabalhar com ela quase como faria com uma lista de objetos Shoe, por exemplo.

→ **Veja você mesmo**. Digite a palavra "List" no IDE e clique nela com o botão direito e selecione "Go To Definiton" (vá para a definição). Isso vai levar você para a declaração da classe List. Ela implementa algumas interfaces:

É daqui que vêm RemoveAt(), Clear(), CopyTo(), Remove(), etc.

```
public class List<T> : IList<T>,
ICollection<T>, IEnumerable<T>, IList,
ICollection, IEnumerable
```

Esta interface permite que você use o foreach, entre outras coisas.

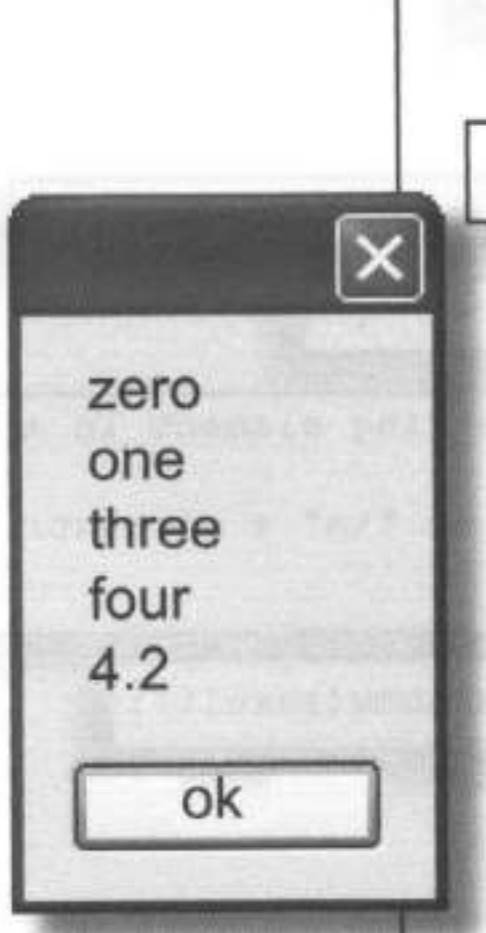
Isto é de onde Add(), Clear(), CopyTo() e Remove() vêm. É a base para todas as coleções genéricas.

Pontos de Bala

- **List** é uma classe do Framework .NET.
- Uma lista se **redimensiona dinamicamente** para qualquer tamanho necessário. Ela tem uma capacidade - sempre que você adicionar dados em quantidade suficiente, ela vai aumentar para acomodá-los.
- Para colocar algo numa lista, use **Add**(). Para remover, use **Remove**().
- Você declara o tipo da lista usando um **parâmetro de tipo**, que é um nome de um tipo entre sinais de maior e menor. Por exemplo, List<Sapo> significa que a lista será capaz de armazenar somente objetos do tipo Sapo.
- Para descobrir onde algo está guardado (e se está) numa lista, use **indexOf()**.
- Para saber quantos elementos estão contidos na lista, use a propriedade **count**.
- Você pode utilizar o método **Contains()** para descobrir se um objeto em particular está contido numa lista.
- O laço **foreach** é um tipo especial que vai iterar por todos os elementos numa lista e executar código para cada um. A sintaxe para ele é **foreach (strings in stringList)**, por exemplo. Você não tem que dizer o tamanho do incremento para esse tipo de laço - ele vai iterar por toda a lista por si mesmo.

Ímãs de Geladeira

Você pode reconstruir o programa usando os trechos para criar um formulário Windows funcional que mostre a caixa de mensagem abaixo quando você clica num botão?




```
private void button1_Click(object sender,
EventArgs e){
```

```
a.RemoveAt(2);
```

```
List<string> a = new List<string>();
```

```
public void printL (List<string> a){
```

```
if (a.contains("two")) {
  a.Add(twopointtwo);
}
```

```
a.Add(zilch);
a.Add(fi rst);
a.Add(second);
a.Add(third);
```

```
}
```

```
string result = "";
```

```
if (a.Contains("three")){
    a.Add("four");
  }
```

```
foreach (string element in a)
{
    result += "\n" + element;
}
```

```
MessageBox.Show(result);
```

```
}
```

```
if (a.IndexOf("four") != 4) {
    a.Add(fourth);
  }
```

```
}
```

```
printL(a);
```

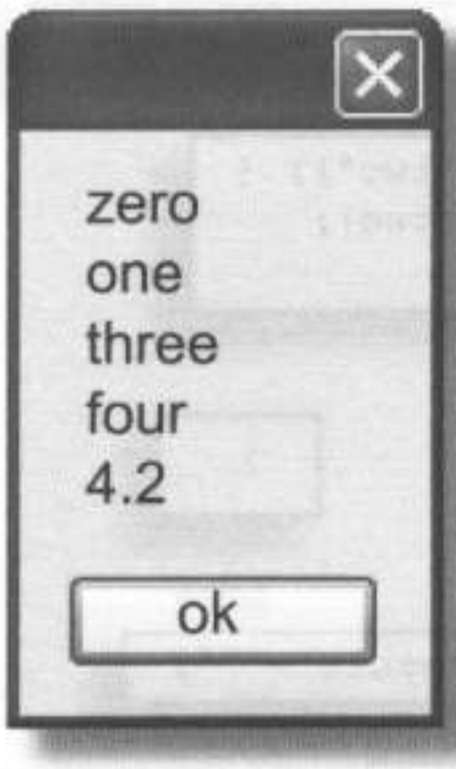
```
string zilch = "zero";
string fi rst = "one";
string second = "two";
string third = "three";
string fourth = "4.2";
string twopointtwo = "2.2";
```

Ímãs de Geladeira Solução

```
private void button1_Click(object sender,
EventArgs e){
```

```
List<string> a = new List<string>();
```

```
string zilch = "zero";
string fi rst = "one";
string second = "two";
string third = "three";
string fourth = "4.2";
string twopointtwo = "2.2";
```




```
a.Add(zilch);
a.Add(fi rst);
a.Add(second);
a.Add(third);
```

Você percebe porque o "2.2" nunca é adicionado à lista, mesmo tendo sido declarado aqui?

```
if (a.Contains("three")){
    a.Add("four");
  }
```

```
a.RemoveAt(2);
```

RemoveAt() remove o elemento de índice 2 - que é o terceiro na lista.

```
if (a.IndexOf("four") != 4) {
    a.Add(fourth);
  }
```

```
if (a.contains("two")) {
  a.Add(twopointtwo);
}
```

```
printL(a);
```

```
}
```

O método printL() usa um laço foreach para iterar por uma lista de strings, adiciona todas elas a uma única seqüência grande de caracteres e mostra-a numa caixa de mensagem.

```
public void printL (List<string> a){
```

```
string result = "";
```

O laço foreach itera por todos os elementos na lista e exibe-os.

```
foreach (string element in a)
{
    result += "\n" + element;
}
```

```
MessageBox.Show(result);
```

```
}
}
```

não existem Perguntas Idiotas

P: Então porque eu usaria uma enum em vez de uma lista? Elas não resolvem o mesmo problema?

R: Enums são um pouco diferentes das listas. Você pode pensar nelas como uma forma prática de armazenar **listas de constantes** para poder referir-se a estas pelo nome. Elas são ótimas para manter a legibilidade do seu código e para que se tenha certeza de sempre usar os nomes certos de variáveis para acessar valores efetivamente usados com freqüência.
Já uma lista pode armazenar praticamente qualquer coisa. Uma vez que ela é uma lista de **objetos**, cada elemento tem seus próprios métodos e propriedades. Enums, por outro lado, devem ser criadas usando um dos **tipos de valor** do C# (como os mostrados na primeira página do capítulo 4). Assim, você não pode armazenar variáveis de referência nelas.
Além disso, Enums não podem mudar de tamanho dinamicamente. Elas não podem implementar interfaces nem ter métodos. Você tem que fazer coerção delas para outro tipo para retirar um valor delas e colocar o resultado numa variável. Junte tudo isso e perceberá que existem algumas grandes diferenças entre as duas formas de armazenar dados. Mas ambas são úteis e têm suas próprias aplicações.

P: Certo, parece que as listas são bem poderosas. Então por que eu usaria uma matriz?

R: As matrizes usam menos memória e exigem menos da CPU. Se você estiver fazendo algo realmente pesado em termos de processamento – como, por exemplo, repetir as mesmas operações milhares de vezes, pode descobrir que as listas vão deixar seu programa mais lento significativamente. Por sorte, você pode facilmente converter qualquer lista numa matriz usando o método `ToArray()` (para matriz)... e converter uma matriz numa lista usando um dos construtores sobrecarregados do objeto `List`.

Ok. Honestamente, estamos falando de uma melhora de performance muito, muito pequena. Como, por exemplo, se você tiver que fazer a mesma coisa milhões de vezes num segundo, use uma matriz, em vez de uma lista.

P: Não entendi o termo "genérico". Por que chamar algo de coleção genérica? Por que uma matriz não pode ser chamada assim?

R: Uma coleção genérica é um objeto de coleção (ou seja, um objeto predefinido que permite armazenar e gerenciar diversos outros objetos) configurado para armazenar um tipo específico.

P: Tudo bem. Isto explica a parte "coleção". Mas o que a torna genérica?

R: Supermercados têm o costume de vender itens genéricos embalados com pacotes com suas próprias marcas explicando o tipo de produto (por exemplo, "batatas fritas", "refrigerante", "sabão", etc.). As marcas genéricas destacam o que existe dentro do pacote, não tem tanta preocupação com o visual.
A mesma coisa acontece com os tipos de dados genéricos. Sua List<> vai trabalhar exatamente da mesma forma, independente do que ela contiver. Uma lista de objetos Sapato, Carta, de inteiros, longs ou mesmo outras listas ainda vai se comportar como um container. Assim você pode sempre adicionar, remover, inserir, etc., não importa o que efetivamente estiver contido na lista.

O termo genérico deve-se ao fato de que, mesmo que uma instância específica de uma lista só possa trabalhar com um tipo de dado, a classe Lista em si é genérica e pode trabalhar com qualquer tipo.

É pra isso que serve a coisa do <T>. É a forma pela qual você associa uma instância específica de List a um tipo. Mas a classe List em si é genérica o suficiente para trabalhar com QUALQUER tipo. É por isso que coleções genéricas são diferentes de qualquer coisa que você viu até agora.

P: Eu posso fazer uma lista sem um tipo?

R: Não. Cada lista - de fato, todas as coleções genéricas (e você vai aprender sobre as demais em breve) - precisam ter um tipo associado a elas. O C# tem listas não genéricas chamadas ArrayLists (listas de matrizes) que podem armazenar qualquer tipo de objeto. Se você quiser usar uma delas, terá de incluir um linha "`using.System.Collections;`" no seu código.
Coleções genéricas são, na realidade, uma característica recente no C# - elas não existiam em versões anteriores da linguagem. Mas elas são tão úteis que as pessoas raramente usam coleções não genéricas... o que é o motivo de não estarmos falando muito delas.

Quando você cria um novo objeto List, sempre fornece um tipo - isso diz ao C# que modalidade de dado vai ser armazenado. Uma lista pode armazenar um tipo de valor (como int, bool, string, etc.) ou uma classe.

Inicializadores de Coleções funcionam exatamente como os de objeto

O C# dá a você um ótimo atalho para diminuir o trabalho de digitação quando você precisa criar uma lista e imediatamente lhe adicionar vários itens. Quando você cria um novo objeto List, pode usar um **inicializador de coleção** para conseguir um conjunto inicial de elementos. Eles vão ser adicionados assim que a lista for criada.

Você viu esse código algumas páginas atrás - ele cria uma nova List<Shoe> e a preenche com novos objetos Shoe.

```
List<Shoe> shoeCloset = new List<Shoe>();
shoeCloset.Add(new Shoe() { Style = Style.Sneakers, Color = "Black" });
shoeCloset.Add(new Shoe() { Style = Style.Clogs, Color = "Brown" });
shoeCloset.Add(new Shoe() { Style = Style.Wingtips, Color = "Black" });
shoeCloset.Add(new Shoe() { Style = Style.Loafers, Color = "White" });
shoeCloset.Add(new Shoe() { Style = Style.Loafers, Color = "Red" });
shoeCloset.Add(new Shoe() { Style = Style.Sneakers, Color = "Green" });
```

O mesmo código reescrito utilizando um inicializador de coleção

Você pode criar um inicializador de coleção pegando cada item que está sendo adicionado usando Add() e acrescentando-o ao comando que cria a lista.

Inicializadores de coleção são uma característica exclusiva do C# 3.0. Se você ainda está usando o Visual Studio 2005, precisará baixar gratuitamente da Microsoft o 2008 Express, senão o código não vai funcionar.

```
List<Shoe> shoeCloset = new List<Shoe>() {
  new Shoe() { Style = Style.Sneakers, Color = "Black" },
  new Shoe() { Style = Style.Clogs, Color = "Brown" },
  new Shoe() { Style = Style.Wingtips, Color = "Black" },
  new Shoe() { Style = Style.Loafers, Color = "White" },
  new Shoe() { Style = Style.Loafers, Color = "Red" },
  new Shoe() { Style = Style.Sneakers, Color = "Green" },
};
```

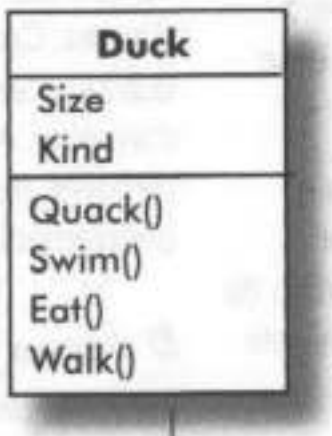
O comando para criar a lista é seguido de chaves com comandos new separados por vírgulas.

Você não está limitado a usar comandos new no inicializador - pode usar variáveis também.

Um inicializador de coleção faz com que seu código seja mais compacto ao permitir que você combine a criação de uma lista com um conjunto inicial de itens.

Vamos criar uma lista de Patos

Eis aqui uma classe pato que mantém registro de sua extensa coleção de patos. (Você tem uma coleção de patos, não?)



A classe tem dois campos públicos. Ela tem também alguns métodos, mas não estamos mostrando aqui.

```
public class Duck {
  public int Size;
  public KindOfDuck Kind;
}

public enum KindOfDuck {
  Mallard,
  Muscovy,
  Decoy
}
```

Usaremos uma enum chamada KindOfDuck para manter registro dos tipos de patos que temos na coleção.

Eis o inicializador

Temos seis patos, então criaremos uma List<Duck> com um inicializador de coleção com seis comandos. Cada um deles cria um pato novo, usando um inicializador de objeto para dar valor aos campos Size (tamanho) e Kind (raça).

```
List<Duck> Ducks = new List<Duck>() {
  new Duck() { Kind = KindOfDuck.Mallard, Size = 17 },
  new Duck() { Kind = KindOfDuck.Muscovy, Size = 18 },
  new Duck() { Kind = KindOfDuck.Decoy, Size = 14 },
  new Duck() { Kind = KindOfDuck.Muscovy, Size = 11 },
  new Duck() { Kind = KindOfDuck.Mallard, Size = 14 },
  new Duck() { Kind = KindOfDuck.Decoy, Size = 13 },
};
```

Listas são fáceis, mas ORDENAR pode ser complicado

Não é difícil pensar em formas de ordenar números ou letras. Mas baseando-se em que você ordena dois objetos, especialmente se eles tiverem múltiplos campos? Em alguns casos você pode querer ordenar objetos pelo valor num campo de nome e em outros pode fazer sentido ordená-los por altura ou data de nascimento, por exemplo. Existem muitas formas de ordenar elementos e as listas suportam todas elas.

Você pode ordenar uma lista de patos por tamanho...

Ordenado do menor para o maior...

...ou por tipo.

Ordenado por tipo de pato...

As listas sabem como ordenar

Toda lista tem um método `Sort()` (ordena) que reorganiza todos os itens, colocando-os em ordem. As listas já sabem como ordenar a maior parte dos tipos nativos e classes e é fácil ensiná-las como ordenar as classes que você mesmo criar.

Duas formas de ordenar seus patos

O método List.Sort()sabe como ordenar qualquer tipo ou classe que implemente a interface `IComparable` (comparável). Essa interface tem apenas um membro - um método chamado CompareTo() (comparar com). O método Sort() usa o método CompareTo() de um objeto para compará-lo com outro e usa seu valor de retorno (um int) para determinar quem é o primeiro.

Você pode fazer com que qualquer classe use o método nativo Sort() das listas se implementar IComparable e adicionar um método CompareTo().

Mas algumas vezes precisamos ordenar uma lista de objetos que não implementa `IComparable` e o .NET tem outra interface para ajudar com isso. Você pode passar a Sort() uma instância de uma classe que implemente `IComparer` (comparador). Essa interface também tem um único método. O método Sort() da Lista usa o método Compare() (compara) do objeto comparador para avaliar pares de objetos, para determinar qual deles virá primeiro numa lista.

O método CompareTo() de um objeto compara-o com outro objeto.

Uma forma de ordenar sua lista é modificar a classe Duck para que ela implemente `IComparable`. Para isso, adicionaríamos um método CompareTo() que receberia uma referência a Duck como parâmetro. Se o pato a ser comparado deve aparecer depois do pato atual na lista ordenada, CompareTo() deve retornar um número positivo.

Eis aqui uma classe Duck que ordena a si mesma baseando-se no tamanho do pato:

Quando você implementa IComparable, especifica o tipo sendo comparado no momento em que faz a implementação na classe.

A maioria dos métodos CompareTo() parecem-se muito com isso. Este método primeiro compara o campo Size (tamanho) com o de outro pato. Se este pato é maior, ela retorna 1. Se ele é menor, retorna -1. Se forem iguais, retorna zero.

```
public class Duck : IComparable<Duck> {
  public int Size;
  public KindOfDuck Kind;
  public int CompareTo(Duck duckToCompare) {
    if (this.Size > duckToCompare.Size)
      return 1;
    else if (this.Size < duckToCompare.Size)
      return -1;
    else
      return 0;
  }
}
```

Se você quer ordenar sua lista do menor para o maior, faça com que CompareTo() retorne um número positivo se um pato estiver sendo comparado com um menor e um número negativo caso contrário.

Use IComparer para dizer às suas listas como se ordenar

As listas têm uma interface especial predefinida do Framework .NET que lhe permite criar sua própria lógica de ordenação. Ao implementar a interface `IComparer`, você pode dizer ao seu objeto `List` exatamente como ele deve ordenar seus elementos. Você faz isso implementando o método `Compare()` da interface. Ele precisa de dois parâmetros, x e y, e retorna um `int`. Se x é menor que y, ele deve retornar um número negativo. Se eles forem iguais, ele deve retornar zero. E se x é maior que y, o valor retornado deve ser positivo.

Eis um exemplo de como você declararia uma classe comparadora para avaliar os objetos Duck por tamanho:

Essa classe implementa IComparer e especifica o tipo de objeto que pode ser ordenado: Duck.

Esses vão sempre ser equivalentes: o mesmo tipo em cada um.

```
public class DuckComparer_bySize : IComparer<Duck>
{
    public int Compare(Duck x, Duck y)
    {
        if (x.Size < y.Size)
            return -1;
        if (x.Size > y.Size)
            return 1;
        return 0;
} }
```

Você pode fazer qualquer tipo de comparação que queira no método.

O método Compare() retorna um int e tem dois parâmetros: os dois devem ser do tipo que você está ordenando.

0 quer dizer que os dois objetos devem ser tratados como iguais (no que se refere a este tipo de cálculo comparativo).

1 positivo quer dizer que x deve ser colocado depois do y. Ele é maior que y.

1 quer dizer que o objeto x deve ser colocado antes do y. Ele é menor que y.

Sua lista vai ser ordenada de formas diferentes de acordo com a forma como você implementar IComparer.

Crie uma instância de seu objeto de comparação

Quando você quiser ordenar usando `IComparer`, precisará criar uma nova instância da classe que a implementa. Esse objeto existe por um motivo - para auxiliar List.Sort() na forma de ordenar os elementos. Mas como qualquer outra classe (não estática), você precisa instanciá-la antes de usar.

```
List<Duck> ducks = new List<Duck>() { ... };
DuckComparer_bySize dc = new DuckComparer_bySize();
ducks.Sort(dc);
```

Deixamos de mostrar o código de inicialização da lista, visto há algumas páginas atrás.

Você passa para Sort() uma referência para o novo objeto DuckComparer_bySize (comparador de patos por tamanho) como parâmetro.

Ordenado do menor para o maior...

Múltiplas implementações de IComparer, muitas formas de ordenar seus objetos

Você pode criar múltiplas classes que implementem `IComparer` com diferentes lógicas de comparação, dependendo do que precisa fazer. Você poderá chamar o comparador que precisar quando for necessário ordenar a lista de uma forma em particular. Eis aqui outra implementações para um comparador de patos:

```
class DuckComparer_byKind : IComparer<Duck> {
 public int Compare(Duck x, Duck y) {
  if (x.Kind < y.Kind)
      return -1;
  if (x.Kind > y.Kind)
      return 1;
  else
      return 0;
   }
}
```

Esse comparador ordena por tipo de pato. Lembre-se, quando você compara usando a enum Kind, você está avaliando usando os índices na enum.

Usamos o valor de Type que vem dos índices na enum Duck.Type.

Então patos Reais vêm antes de Crioulos, que vêm antes dos de madeira.

Note como maior que e menor que têm significados diferentes aqui. Um tipo de pato é maior do que o outro, para que assim tenhamos uma ordem para usar.

Eis um exemplo de como enums e listas podem trabalhar juntas. Uma Enum serve para armazenar números e aqui está sendo usada para ordenar a lista.

```
DuckComparer_byKind dcKind = new DuckComparer_byKind();
ducks.Sort(dcKind);
```

Ordenado por tipo de pato...

IComparer pode fazer comparações complexas

Uma vantagem de criar uma classe separada para ordenar seus patos é poder criar uma lógica mais complexa nessa classe—e pode adicionar membros que ajudem a determinar como a lista será ordenada.

Eis uma classe mais complexa para comparar patos. Seu método CompareTo() usa os mesmos parâmetros, mas ele examina o campo público SortBy (ordenar por) para determinar como os patos serão ordenados.

```
public class DuckComparer : IComparer<Duck> {
 public enum SortCriteria {
  SizeThenKind,
  KindThenSize,
}
public SortCriteria SortBy = SortCriteria.SizeThenKi
nd;
```

A enum diz ao objeto de que forma ordenar os patos.

```
public int Compare(Duck x, Duck y) {
 if (SortBy == SortCriteria.SizeThenKind)
   if (x.Size > y.Size)
       return 1;
   else if (x.Size < y.Size
       return -1;
   else
       if (x.Kind > y.Kind)
         return 1;
       else if (x.Kind < y.Kind)
         return -1;
       else
         return 0;
 else
   if (x.Kind > y.Kind)
       return 1;
   else if (x.Kind < y.Kind)
       return -1;
   else
       if (x.Size > y.Size)
         return 1;
       else if (x.Size < y.Size)
         return -1;
       else
         return 0;
  }
}
DuckComparer dc = new DuckComparer();
dc.SortBy = DuckComparer.SortCriteria.KindThenSize;
ducks.Sort();
dc.SortBy = DuckComparer.SortCriteria.SizeThenKind;
ducks.Sort();
```

O comando if checa o campo SortBy. Se ele tiver o valor SizeThenKind (tamanho e depois raça), primeiro os patos vão ser ordenados por tamanho e, depois, patos do mesmo tamanho serão ordenados por raça.

Em vez de apenas retornar 0 quando os dois patos são do mesmo tamanho, o comparador checa a raça e retorna 0 somente se os dois patos são do mesmo tamanho e da mesma raça.

Se SortBy não estiver com o valor SizeThenKind, o comparador primeiro ordena pela tipo de pato. Se dois patos forem da mesma raça, ele usa o tamanho.

Eis como usaríamos o objeto Comparer. Primeiro instanciaríamos o objeto normalmente. Então poderíamos atribuir o valor do campo SortBy antes de chamar ducks.Sort(). Agora você pode mudar a forma como a lista ordena seus patos apenas mudando um campo no objeto.

Exercício

Crie cinco cartas aleatórias e depois as ordene.

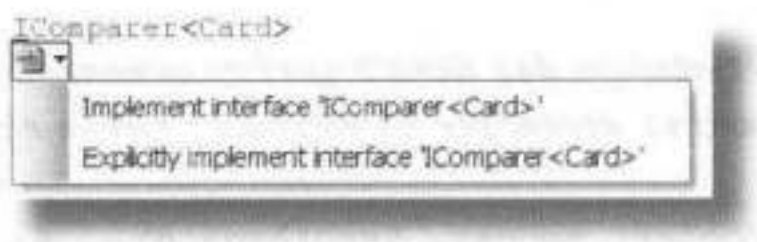
Aplicativos de console que você executa na linha de comando usam muito Console.WriteLine(), mas você pode usá-lo num aplicativo Windows também.

1 **Crie código para criar um conjunto de cartas misturado**
Adicione um botão para criar cinco objetos Card aleatórios. Depois disto, use o método predefinido `Console.WriteLine()` para escrever seus nomes na janela **output**. Você pode ver tudo o que foi escrito nessa janela selecionando "Output" no menu View (visualizar) enquanto o programa está executando.

2 **Crie uma classe que implemente IComparer<List> para ordenar as cartas**
Eis uma boa chance de usar aquele atalho do IDE que implementa interfaces:

```
public class CardComparer_byValue : IComparer<Card>
```

Então clique em IComparer e passe o mouse sobre o "I". Você deve ver uma caixa aparecer debaixo dele. Quando você clicar nela, o IDE mostra uma janela:

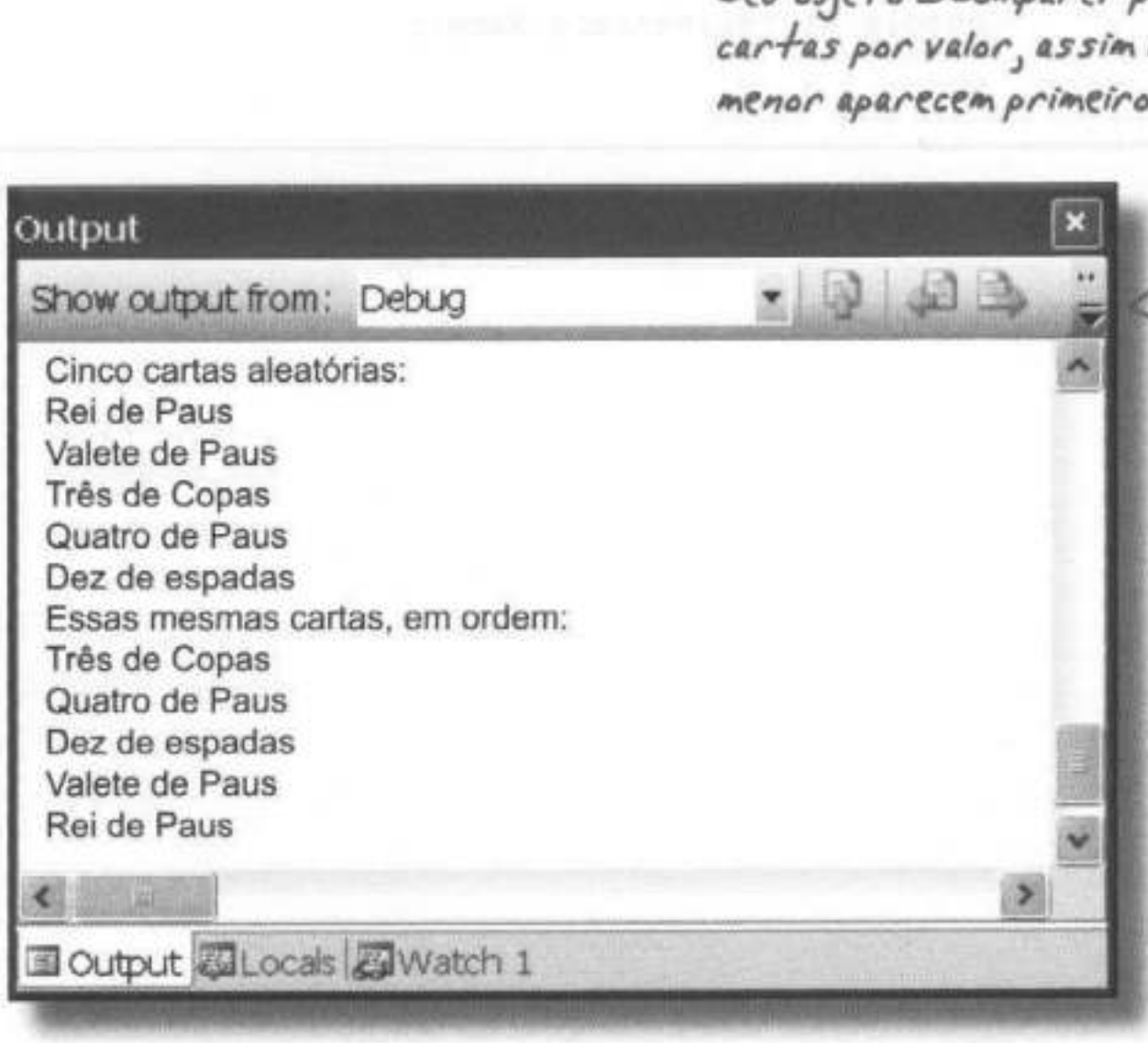


Se você clicar em "Implementar a interface 'IComparer<Card>'", o IDE automaticamente preenche todos os métodos e propriedades necessários para a implementação. Nesse caso, ele cria um método Compare() vazio para comparar duas cartas, x e y. Escreva o método para que ele retorne 1 se x for maior que y, -1 se for menor e 0 se forem a mesma carta. Nesse caso, certifique-se de que qualquer rei venha antes de qualquer valete, que vem antes de qualquer quatro, que vem antes de qualquer ás.

3 **Certifique-se de que o resultado pareça correto**
Eis aqui como a sua janela de output deve parecer depois que você clicar no botão.

Seu objeto IComparer precisa ordenar as cartas por valor, assim as cartas de valor menor aparecem primeiro na lista.

Quando você usa o método predefinido Console.WriteLine(), ele adiciona uma linha na janela output.



Solução do Exercício

Crie cinco cartas aleatórias e depois as ordene.

Eis as entranhas do ordenador de cartas, que usa o método predefinido List.Sort(). Ele usa um objeto IComparer, que tem um único método: Compare(). Essa implementação pega duas cartas e primeiro compara seus naipes, depois seus valores.

```
public class CardComparer_byValue : IComparer<Card> {
    public int Compare(Card x, Card y) {
        if (x.Value < y.Value) {
            return -1;
        }
        if (x.Value > y.Value) {
            return 1;
        }
        if (x.Suit < y.Suit) {
            return -1;
        }
        if (x.Suit > y.Suit) {
            return 1;
        }
        return 0;
    }
}
```

Se x for de naipe maior, retorna /. Se for de naipe menor, retorna -/. Lembre-se, ambos os comandos encerram o método imediatamente.

Esses comandos apenas são executados se x e y são do mesmo naipe – ou seja, os dois primeiros comandos não foram executados.

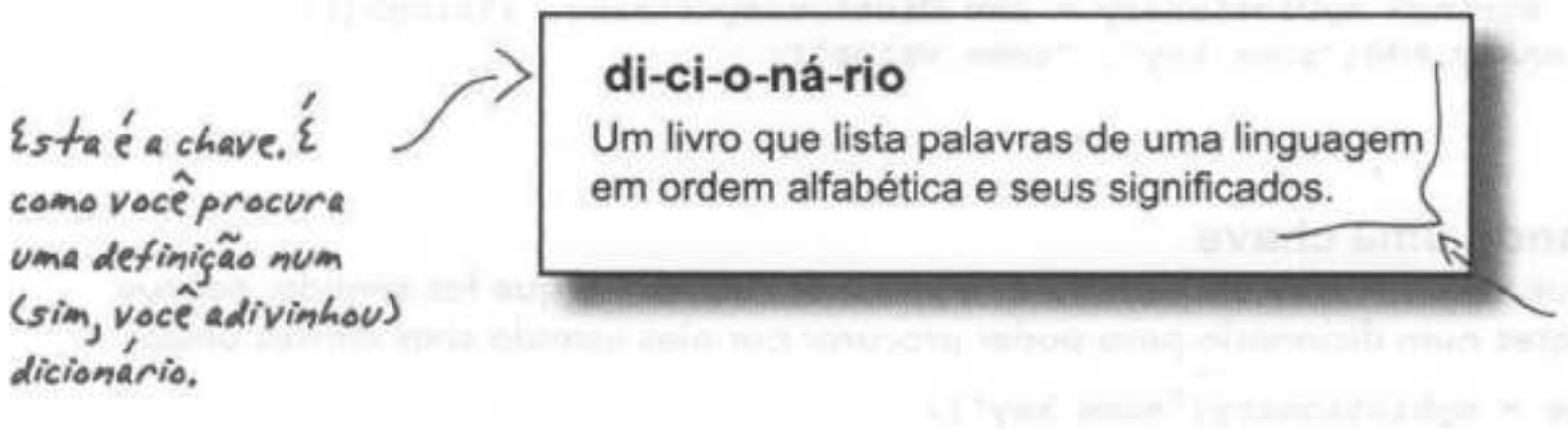
Se nenhum dos demais quatro comandos for executados, as cartas devem ser as mesmas - então retorne zero.

```
private void button1_Click(object sender, EventArgs e)
{
    Console.WriteLine("Cinco cartas aleatórias:");
    List<Card> cards = new List<Card>();
    for (int i = 0; i < 5; i++)
    {
        cards.Add(new Card((Card.Suits)random.Next(4),
                           (Card.Values)random.Next(1, 14)));
        Console.WriteLine(cards[i].Name);
    }
    Console.WriteLine("Essas mesmas cartas, em ordem:");
    cards.Sort(new CardComparer_byValue());
    foreach (Card card in cards)
    {
        Console.WriteLine(card.Name);
    }
}
```

Eis um objeto genérico lista de cartas. Uma vez que as cartas estão na lista, é fácil ordená-las usando um IComparer.

Um dicionário para armazenar chaves e valores

Uma lista é como uma grande página cheia de nomes. Mas e se você quiser adicionar para cada nome um endereço, por exemplo? Ou detalhes para cada carro numa listagem de uma garagem? Para isso você pode usar um **dicionário**. Eles permitem associar um valor especial—a **chave**—com vários dados—o **valor**. E mais uma coisinha: uma chave específica pode **aparecer somente uma única vez** num dicionário.



Eis como declarar um `dicionário` em C#.

```
Dictionary <Tkey, TValue> kv = new Dictionary <TKey, TValue>();
```

Esses são como o List<T>. O <T> quer dizer que um tipo vai aqui. Então você pode declarar um tipo para a chave e outro para o valor.

O primeiro item entre <> é sempre a chave e o segundo sempre o valor.

E aqui está um dicionário em ação:

```
private void button1_Click(object sender, EventArgs e)
{
  Dictionary<string, string> wordDefinition =
      new Dictionary<string, string>();
  wordDefinition.Add ("Dictionary", "A book that lists the words of a "
                    + "language in alphabetical order and gives their meaning");
  wordDefinition.Add ("Key", "A thing that provides a means of gaining access to "
                    + "our understanding something.");
  wordDefinition.Add ("Value", "A magnitude, quantity, or number.");

  if (wordDefinition.ContainsKey("Key"))
      MessageBox.Show(wordDefinition["Key"]);
}
```

Este dicionário tem strings como chave e valor. É como um dicionário de verdade: termo e definição.

Add() tem como parâmetros uma chave e um valor.

O método Add() é a forma de adicionar chaves e valores num dicionário.

Eis como você recupera um valor para uma chave. Parece com o uso de um índice de matriz - recupere o valor para a chave nesse índice.

ContainsKey() (contém a chave) diz a você se uma chave existe num dicionário. Prático, não?

Lista de Funcionalidades de um Dicionário

Dicionários parecem bastante com listas. Os dois são flexíveis ao permitir trabalhar com muitos tipos de dados e também possuem muita funcionalidades predefinidas. Eis o básico sobre um dicionário:

★ Adicione um item

Você pode adicionar um item num dicionário passando a ele uma chave e um valor usando o método `Add()`.

```
Dictionary<string, string> myDictionary = new Dictionary<string, string>();
myDictionary.Add("some key", "some value");
```

★ Procure um valor usando uma chave

A coisa mais importante que você faz com um dicionário é procurar valores - o que faz sentido, porque você armazenou esses valores num dicionário para poder procurar por eles usando suas chaves únicas

```
string lookupValue = myDictionary["some key"];
```

★ Remova um item

Exatamente como uma lista, você pode remover um item de um dicionário usando o método `Remove()`. Tudo que você precisa passar para ele é a chave para que tanto ela quanto o valor sejam removidos.

```
myDictionary.Remove("some key");
```

Uma chave é única num dicionário. Qualquer uma só pode aparecer uma única vez. Valores podem aparecer qualquer número de vezes - duas ou mais chaves podem ter o mesmo valor. Dessa forma, quando você procura ou remove uma chave, o dicionário sabe com qual item trabalhar.

★ Recupere uma lista de chaves

Você pode conseguir uma lista de todas as chaves num dicionário usando `KeyCollection` (coleção de chaves) e iterando por ela usando um laço foreach. Você normalmente usa KeyCollection da seguinte forma:

```
foreach (string key in myDictionary.Keys) { ... };
```

Keys (chaves) é uma propriedade de um objeto dicionário. Esse dicionário em particular tem chaves que são seqüências de caracteres, de forma que Keys é uma coleção de strings.

★ Obtenha uma lista de valores

Você pode recuperar uma lista de todos os valores num dicionário usando `ValueCollection`. Na maior parte das vezes, você usará um loop foreach com ValueCollection também:

```
foreach (string value in myDictionary.Values) { ... };
```

Já que o dicionário tem valores string, o identificador do foreach será uma seqüência de caracteres também.

Chaves e valores também podem ter tipos diferentes

Dicionários são realmente versáteis e pode armazenar praticamente qualquer coisa, desde strings até números e objetos. Eis aqui um exemplo de um dicionário que armazena um inteiro como chave e um objeto pato como valor. O objeto `Duck` tem um campo `Size` e uma enum `Types` definidas dentro de seu construtor.

É aqui que o dicionário é declarado. Ele armazenará números e patos. Adicionamos cada um dos patos ao dicionário, dando a eles um número como ID único para servir de chave.

```
Dictionary<int, Duck> duckDictionary = new Dictionary<int, Duck>();
duckDictionary.Add(5155, new Duck() { Kind = KindOfDuck.Mallard, Size = 15 });
duckDictionary.Add(6256, new Duck() { Kind = KindOfDuck.Mallard, Size = 14 });
duckDictionary.Add(2799, new Duck() { Kind = KindOfDuck.Mallard, Size = 13 });
```

A propriedade Count diz quantos pares de chave/valor existem no dicionário.

```
int howMany = duckDictionary.Count;
Console.WriteLine("There are {0} ducks.", howMany);
```

Se você precisar usar chaves e valores do dicionário ao mesmo tempo, pode usar um objeto KeyValuePair<>.

Esse loop atribui o par chave/valor atual para idDuck, um de cada vez.

```
foreach (KeyValuePair<int, Duck> idDuck in duckDictionary)
    Console.WriteLine("Key/value pair: {0}: {1}, {2}",
      idDuck.Key, idDuck.Value.Size, idDuck.Value.Kind.ToString() );
```

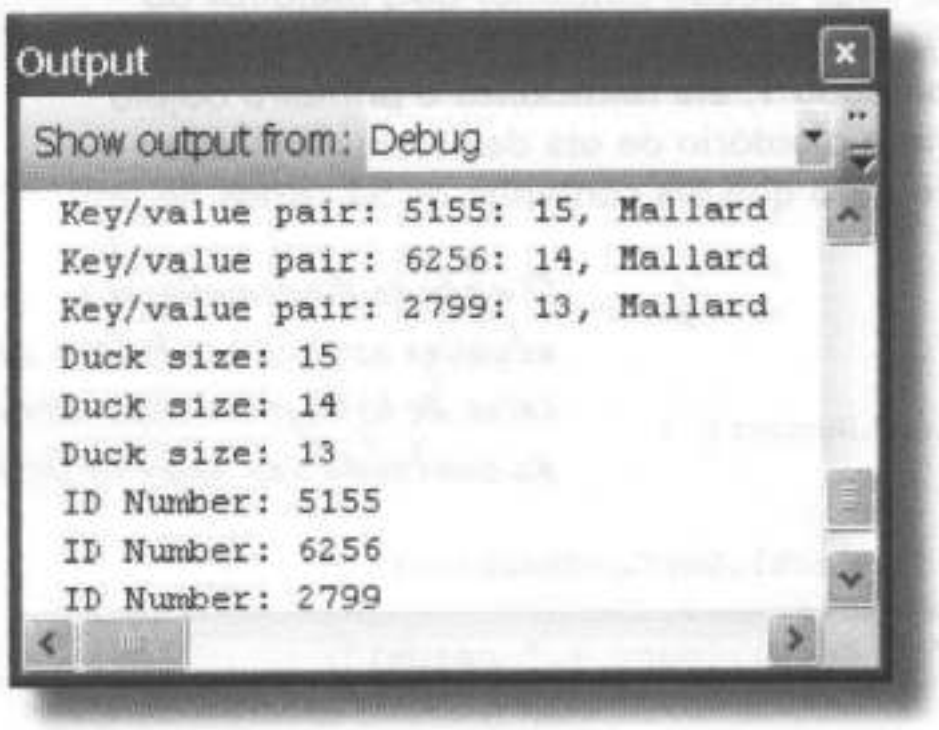
Cada KeyValuePair tem uma chave e um valor. Uma vez que o valor é um Duck, você pode usar seus campos.

```
foreach (Duck duck in duckDictionary.Values)
  Console.WriteLine("Duck size: {0}", duck.Size);
```

Esse loop foreach usa todos os valores do dicionário.

```
foreach (int key in duckDictionary.Keys)
  Console.WriteLine("ID Number: {0}", key);
```

E este loop foreach recupera cada uma das chaves do dicionário.



Eis o que este código escreve no console.

Exercício

1 **Crie um formulário que permita você mover cartas entre baralhos**

Você já fez uma classe que representa uma carta. Agora é hora de fazer uma para armazenar qualquer quantidade delas, que chamaremos `Deck` (baralho). Um baralho na vida real tem 52 cartas, mas a classe `Deck` deverá ser capaz de armazenar qualquer número de cartas - até mesmo nenhuma.

Então você deve fazer um formulário que mostre a você o conteúdo de dois objetos `Deck`. Quando você iniciar o programa, o baralho 1 deve ter 10 cartas aleatórias e o 2 deve ser um baralho completo com 52 cartas, ambos ordenados por naipe e valor. Você deverá poder reiniciar qualquer um deles para sua condição inicial usando dois botões. O formulário também deverá ter botões (marcados com "<<" e ">>") para mover cartas entre os baralhos.

Esses botões devem chamar moveToDeck2 (o de cima) e moveToDeck1 (o de baixo). Eles movem cartas de um baralho para o outro.

Você pode usar a propriedade Name do botão para dar a ele um nome que tornará seu código mais fácil de ler. Então quando você clicar duas vezes no botão, o tratador de evento terá um nome correspondente.

Use dois controles ListBox para mostrar os dois baralhos. Quando moveToDeck1 for clicado, a carta selecionada deve ser movida do baralho 2 para o 1.

Esses botões deverão ser chamados shuffle1 e shuffle2. Eles devem chamar o método Shuffle() (embaralhar) do baralho apropriado e então exibi-lo novamente.

Cada um desses botões (reset1 e reset2) chamarão primeiro o método RedrawDeck() (exibe baralho) e depois o método ResetDeck().

Adicionalmente aos tratadores de evento para os seis botões, você precisa adicionar dois métodos ao formulário. Primeiro crie um ResetDeck() (reiniciar o baralho), que retorna o baralho para seu estado inicial. Ele deve receber um inteiro como parâmetro: se for passado 1, ele reinicializa o primeiro objeto Deck primeiro esvaziando-o e depois incluindo nele um número aleatório de até dez cartas também aleatórias. Se for passado 2, ele reinicializa o segundo objeto para que ele contenha as 52 cartas de um baralho completo. Então adicione o seguinte método:

O método RedrawDeck() atualiza os dois controles de caixa de listagem independente do conteúdo dos objetos Deck.

```
private void RedrawDeck(int DeckNumber) {
  if (DeckNumber == 1) {
    listBox1.Items.Clear();
    foreach (string cardName in deck1.GetCardNames())
      listBox1.Items.Add(cardName);
    label1.Text = "Deck #1 (" + deck1.Count + " cards)";
  } else {
    listBox2.Items.Clear();
    foreach (string cardName in deck2.GetCardNames())
      listBox2.Items.Add(cardName);
    label2.Text = "Deck #2 (" + deck2.Count + " cards)";
  }
}
```

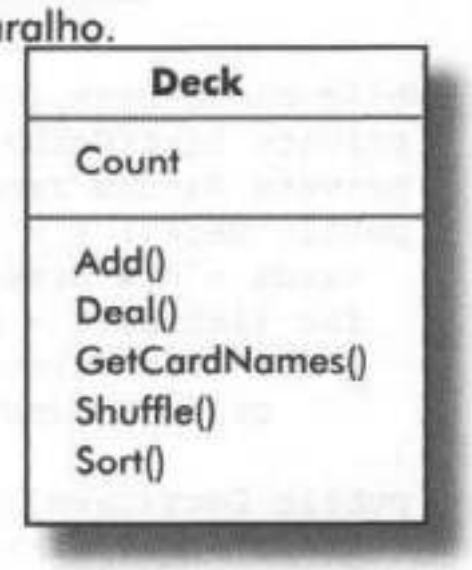
Dê uma olhada em como usamos o loop foreach para adicionar cada carta do baralho à caixa de listagem.

2 Crie a classe Deck

Quando você tiver as declarações para uma classe sem a implementação, isso é chamado de um esqueleto.

Eis o esqueleto para a classe `Deck`. Já preenchemos vários métodos para você. Você deve terminar o trabalho escrevendo os métodos `Shuffle()` (embaralhar) e `GetCardNames()` (recuperar nome das cartas) e terá de fazer o método `Sort()` funcionar. Nós também adicionamos dois **construtores sobrecarregados** que serão úteis: um cria um baralho completo de 52 cartas e outro usa uma matriz de objetos `Card` e os inclui no baralho.

O baralho guarda suas cartas numa Lista - mas ele a mantém privada, para ter certeza de que o encapsulamento foi bem feito.




```
public class Deck {
  private List<Card> cards;
  private Random random = new Random();
  public Deck() {
    cards = new List<Card>();
      for (int suit = 0; suit <= 3; suit++)
          for (int value = 1; value <= 13; value++)
              cards.Add(new Card((Card.Suits)suit, (Card.Values)value));
  }
  public Deck(Card[] initialCards) {
    cards = new List<Card>(initialCards);
  }
  public int Count { get { return cards.Count; } }
  public void Add(Card cardToAdd) {
    cards.Add(cardToAdd);
  }
  public Card Deal(int index) {
    Card CardToDeal = cards[index];
    cards.RemoveAt(index);
    return CardToDeal;
  }
  public void Shuffle() {
    // Esse método embaralha (shuffles) as cartas, um deles em ordem aleatória
  }
  public string[] GetCardNames() {
    // Esse método retorna uma string array que contém o nome de cada cartão
  }
  public void Sort() {
    cards.Sort(new CardComparer_bySuit());
  }
}
```

Se você não passar parâmetros para o construtor, ele cria um baralho completo de 52 cartas.

Esse construtor sobrecarregado recebe um único parâmetro - uma matriz de cartas, que ele carrega como o baralho inicial.

O método Deal retira uma carta do baralho - remove-a e retorna uma referência para ela. Você pode dar cartas do topo do baralho passando zero a ele, ou puxar cartas do meio passando o índice da carta que deve ser dada.

Você precisará escrever os métodos Shuffle(), GetCardNames() e adicionar uma classe que implemente IComparer para ter certeza de que o método Sort() funcione. E vai precisar adicionar a classe Card que você já fez. Se você usar Add Existing Item (adicionar item existente) para adicioná-la, não se esqueça de mudar seu namespace.

Dica: A propriedade SelectedIndex do controle de caixa de listagem terá o mesmo valor que o índice da carta na lista. Você pode passá-lo diretamente para o método Deal() (dar carta). Se nenhuma carta estiver selecionada, ele será menor do que zero. Nesse caso, os botões moveToDeck não devem fazer nada.

Outra dica: O formulário torna realmente fácil testar seu método Shuffle(). Clique seguidas vezes em Organizar baralho / até que você consiga um baralho com três cartas. Isso vai tornar facilitar se o seu método de embaralhar está funcionando.

Solução do Exercício

Crie uma classe para armazenar um baralho, juntamente com um formulário para usá-la.

```
public class Deck {
  private List<Card> cards;
  private Random random = new Random();
  public Deck() {
    cards = new List<Card>();
    for (int suit = 0; suit <= 3; suit++)
      for (int value = 1; value <= 13; value++)
        cards.Add(new Card((Card.Suits)suit, (Card.Values)value));
  }
  public Deck(Card[] initialCards) {
    cards = new List<Card>(initialCards);
  }
  public int Count { get { return cards.Count; } }
  public void Add(Card cardToAdd) {
    cards.Add(cardToAdd);
  }
  public Card Deal(int index) {
    Card CardToDeal = cards[index];
    cards.RemoveAt(index);
    return CardToDeal;
  }
  public void Shuffle() {
    List<Card> NewCards = new List<Card>();
    while (cards.Count > 0) {
      int CardToMove = random.Next(cards.Count);
      NewCards.Add(cards[CardToMove]);
      cards.RemoveAt(CardToMove);
    }
    cards = NewCards;
  }
  public string[] GetCardNames() {
    string[] CardNames = new string[cards.Count];
    for (int i = 0; i < cards.Count; i++)
      CardNames[i] = cards[i].Name;
    return CardNames;
  }
  public void Sort() {
    cards.Sort(new CardComparer_bySuit());
  }
}
```

Eis o construtor que cria um baralho completo com 52 cartas. Ele usa laços for aninhados. O Loop externo usado pelos quatro naipes. Isso implica que o Loop interno que é usado pelos 13 valores executa quatro vezes, uma por naipe.

Aqui está o outro construtor - essa classe tem dois construtores sobrecarregados, cada um com um conjunto de parâmetros diferentes.

Os métodos Add e Deal são bem simples - eles usam os métodos da lista de cartas. O método Deal remove uma carta da lista e o Add adiciona uma.

Seu método GetCardNames() precisa criar uma matriz grande o suficiente para armazenar o nome de todas as cartas. Este aqui usa um loop for, mas poderia ser um foreach também.

O método Shuffle cria uma nova instância de List<Cards> chamada NewCards. Ele então puxa cartas aleatórias do campo Cards e as coloca em NewCards até que Cards esteja vazio. Uma vez que isso é feito, ele reinicializa o campo Cards fazendo-o apontar para essa nova instância. A instância anterior não terá mais nenhuma referência apontando para ela, então é eliminada pelo coletor de lixo.

```
class CardComparer_bySuit : IComparer<Card>
{
  public int Compare(Card x, Card y)
  {
    if (x.Suit > y.Suit)
      return 1;
    if (x.Suit < y.Suit)
      return -1;
    if (x.Value > y.Value)
      return 1;
    if (x.Value < y.Value)
      return -1;
    return 0;
  }
}
```

Ordenar por naipe é parecido com ordenar por valor. A única diferença é que nesse caso os naipes são comparados primeiro e então os valores apenas se os naipes forem iguais.

Em vez de usar if/else if, usamos uma série de comandos if. Isso funciona porque cada um deles executará apenas se o anterior não - o primeiro que executar encerrará o método.

```
  Deck deck1;
  Deck deck2;
  Random random = new Random();
  public Form1() {
    InitializeComponent();
    ResetDeck(1);
    ResetDeck(2);
    RedrawDeck(1);
    RedrawDeck(2);
  }
  private void ResetDeck(int deckNumber) {
    if (deckNumber == 1) {
      int numberOfCards = random.Next(1, 11);
      deck1 = new Deck(new Card[] { });
      for (int i = 0; i < numberOfCards; i++)
        deck1.Add(new Card((Card.Suits)random.Next(4),
                  (Card.Values)random.Next(1, 14)));
      deck1.Sort();
    } else
      deck2 = new Deck();
  }
```

O construtor do formulário precisa reinicializar os dois baralhos e então exibi-los.

Para reiniciar o baralho 1, esse método primeiro usa random.Next(). Ele escolhe quantas cartas serão acrescentadas e cria um novo objeto Deck vazio, usando um Loop for para adicionar essa quantidade de cartas aleatórias. Ele encerra colocando as cartas em ordem. Reiniciar o baralho 2 é fácil - apenas cria-se uma nova instância de Deck.

Você já tem o método RedrawDeck() feito na parte das instruções.

Eis o resto do código para o formulário.

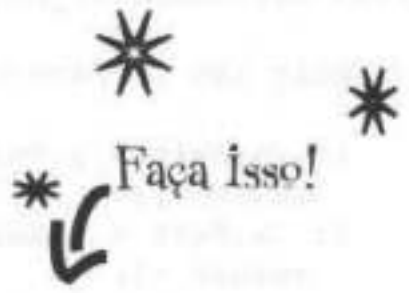
```
private void reset1_Click(object sender, EventArgs e) {
    ResetDeck(1);
    RedrawDeck(1);
  }
  private void reset2_Click(object sender, EventArgs e) {
    ResetDeck(2);
    RedrawDeck(2);
  }
  private void shuffle1_Click(object sender, EventArgs e) {
    deck1.Shuffle();
    RedrawDeck(1);
  }
  private void shuffle2_Click(object sender, EventArgs e) {
    deck2.Shuffle();
    RedrawDeck(2);
  }
  private void moveToDeck1_Click(object sender, EventArgs e) {
    if (listBox2.SelectedIndex >= 0)
      if (deck2.Count > 0) {
        deck1.Add(deck2.Deal(listBox2.SelectedIndex));
      }
    RedrawDeck(1);
    RedrawDeck(2);
  }
  private void moveToDeck2_Click(object sender, EventArgs e) {
    if (listBox1.SelectedIndex >= 0)
      if (deck1.Count > 0)
        deck2.Add(deck1.Deal(listBox1.SelectedIndex));
    RedrawDeck(1);
    RedrawDeck(2);
  }
}
```

Nomear bem seus controles facilita a leitura do código. Se estes tivessem como nome button1_Click, button2_Click, etc., você não poderia dizer de qual botão é esse código!

Esses botões são bem fáceis - primeiro reiniciar ou embaralhar, depois exibir.

Você pode usar a propriedade SelectedIndex do controle de caixa de listagem para determinar qual carta o usuário selecionou e então movê-la de um baralho para outro (se o valor for menor que zero, nenhuma carta estará selecionada, então o botão não faz nada). Uma vez que a carta seja movida, os dois baralhos precisam ser exibidos novamente.

Você pode fazer seus próprios métodos sobrecarregados



Você tem usado **métodos sobrecarregados** e mesmo construtores sobrecarregados que são parte das classes e objetos do Framework .NET, logo já viu como eles são úteis. Não seria legal se você pudesse fazer o mesmo nas suas próprias classes? Bom, você pode—e é fácil! Tudo o que você precisa fazer é escrever dois ou mais métodos com o mesmo nome, mas parâmetros diferentes.

1. **Crie um novo projeto e adicione a classe Card nele**
   Você pode fazer isso facilmente clicando com o botão direito no projeto no Navegador de Solução e selecionando "Existing Item" (item existente) no menu. O IDE fará uma cópia da classe e a adicionará ao seu novo projeto. O arquivo **ainda tem o namespace do projeto anterior**; vá para o topo de Card.cs e altere a linha `namespace` para colocar a classe no mesmo do projeto atual.

   Se você não fizer isso, só poderá acessar a classe Card especificando seu namespace (como, por exemplo, projetoAntigo.Card).

2. **Adicione alguns métodos sobrecarregados na classe de carta**
   Crie dois métodos estáticos `DoesCardMatch()` (as cartas são equivalentes?). O primeiro deve checar o naipe de uma carta. O segundo, seu valor. Os dois devem retornar verdadeiro se as cartas forem equivalentes.

```
public static bool DoesCardMatch(Card CardToCheck, Card.Suits Suit) {
  if (CardToCheck.Suit == Suit) {
    return true;
  } else {
    return false;
  }
}
public static bool DoesCardMatch(Card CardToCheck, Card.Values Value) {
  if (CardToCheck.Value == Value) {
    return true;
  } else {
    return false;
  }
}
```

Métodos sobrecarregados não precisam ser estáticos, mas é bom praticar um pouco escrever esse tipo de método.

3. **Adicione um botão no formulário para usar os novos métodos**
   Adicione o seguinte botão:

```
Card cardToCheck = new Card(Card.Suits.Clubs, Card.Values.Three);
bool doesItMatch = Card.DoesCardMatch(cardToCheck, Card.Suits.Hearts);
```

Tão logo você digite "`DoesCardMatch(`" o IDE vai mostrar a você que realmente um método sobrecarregado foi desenvolvido:

```
Card.DoesCardMatch(
▣1 of 2▣ bool Card.DoesCardMatch (Card CardToCheck, Card.Suits Suit)
```

Passe um minuto brincando com os dois métodos para você se acostumar com o conceito de sobrecarga.

Exercício Longo

Desenvolva um jogo **vá pescar**! que você possa jogar contra o computador

Esse exercício é um pouco diferente...
Há uma boa chance de que você esteja aprendendo C# porque quer um emprego como desenvolvedor profissional. É por isso que modelamos esse exercício como se fosse uma tarefa dada a um profissional. Quando você trabalha como um programador numa equipe, normalmente não cria um programa inteiro do início até o fim; você desenvolve uma parte de um programa maior. Então vamos dar a você um quebra cabeças com algumas das peças já montadas. O código para o formulário está inteiro na próxima página, no passo 3. Você só terá que digitá-lo - o que pode parecer uma grande ajuda da nossa parte, mas, na verdade, implica que suas classes **devem trabalhar com aquele código**. E isso pode ser um desafio!

1 Comece com as especificações

Todo projeto de desenvolvimento profissional de software começa com uma especificação e este não será exceção. Você desenvolverá um clássico jogo chamado Vá Pescar! Algumas pessoas jogam com regras ligeiramente diferentes. Então eis aqui um lembrete das que você usará:

- ★ O jogo começa com um baralho de 52 cartas. Cinco são dadas para cada jogador. As restantes são chamadas de monte. Cada jogador, na sua vez, pergunta por um **valor** ("alguém tem sete?"). Qualquer outro jogador que tenha cartas com aquele valor deve entregá-las. Se ninguém tiver uma, o jogador precisa "ir pescar" no monte, pegando uma carta dele.
- ★ O objetivo do jogo é fazer "livros", que são conjuntos completos de todas as quatro cartas com o mesmo valor. O jogador com a maior quantidade deles no fim do jogo é o vencedor. Tão logo um jogador tenha montado um livro, ele o coloca virado para cima na mesa para que todos os demais jogadores possam ver que livros cada um tem.
- ★ Quando um jogador coloca um livro na mesa, isso pode fazer com que ele fique sem cartas. Se for o caso, ele tem que pegar mais cinco do monte. Se sobraram menos de cinco nela, ele tem que pegar todas. O jogo acaba assim que o monte acabar. O vencedor é então aquele que tiver mais livros.
- ★ Para essa versão computadorizada de "vá pescar", deve existir dois jogadores programados e um jogador humano. Todos os turnos começam com o jogador humano selecionando uma das cartas em sua mão, que deverá ser visível sempre para ele. Ele faz isso escolhendo uma de suas cartas e indicando que ele vai perguntar por um valor. Os dois jogadores programados então devem, em seguida, pedir seus valores também. O resultado de cada turno deve ser exibido. Isso se repetirá até que um dos jogadores vença.
- ★ O jogo deve cuidar de todos os detalhes de troca de cartas e montagem de livros automaticamente. Uma vez que um jogador tenha vencido, o jogo termina. O nome do vencedor deve então ser exibido (ou dos vencedores, se houver empate). Nenhuma outra ação pode ser feita – o jogador deve reiniciar o programa para começar outro jogo.

Se você não souber o que está desenvolvendo antes de começar, como saberá quando terminou? É por isso que a maior parte dos projetos profissionais de desenvolvimento de software começa com uma especificação que lhe diz o que vai ser feito.

2 **Crie o formulário**

Crie o formulário para o jogo Vá Pescar. Ele deve ter um controle de caixa de listagem para a mão dos jogadores, duas caixas de texto para o progresso do jogo e um botão para que o jogador pergunte por uma carta. Para jogar, o usuário tem que selecionar uma das cartas de sua mão e clicar o botão para perguntar se os jogadores computadorizados têm cartas com aquele valor.

Exercício Longo (continuação)

3 **Eis o código do formulário**
Digite-o exatamente como você o vê aqui. O restante do código a ser escrito você mesmo deve interagir com ele.

```
public partial class Form1 : Form {
  public Form1() {
    InitializeComponent();
  }
  private Game game;
  private void buttonStart_Click(object sender, EventArgs e) {
    if (String.IsNullOrEmpty(textName.Text)){
      MessageBox.Show("Please enter your name", "Can't start the game yet");
      return;
    }
    game = new Game(textName.Text, new string[] { "Joe", "Bob" }, textProgress);
    buttonStart.Enabled = false;
    textName.Enabled = false;
    buttonAsk.Enabled = true;
    UpdateForm();
  }
  private void UpdateForm() {
    listHand.Items.Clear();
    foreach (String cardName in game.GetPlayerCardNames())
      listHand.Items.Add(cardName);
    textBooks.Text = game.DescribeBooks();
    textProgress.Text += game.DescribePlayerHands();
    textProgress.SelectionStart = textProgress.Text.Length;
    textProgress.ScrollToCaret();
  }
```

Esta é a única classe com a qual o formulário interage. Ela executa todo o jogo.

A propriedade Enabled ativa ou desativa um controle no formulário.

Quando você começa um novo jogo, ele cria uma nova instância da classe Game, ativa o botão pergunte, desativa o iniciar o jogo e repinta o formulário.

Esse método limpa e repovoa a caixa de listagem que mantém a mão do jogador, e então atualiza as caixas de texto.

Usar SelectionStart() (início da seleção) e ScrollToCaret() (role até o cursor). Assim, a caixa rolará até exibir a parte final de seu texto, de modo que se ele for muito grande aparecerá só o seu final.

A linha com SelectionStart move o cursor da caixa de texto até o final. Uma vez que tenha feito isso, o método ScrollToCaret() rola a caixa até a posição do cursor.

```
  private void buttonAsk_Click(object sender, EventArgs e) {
    textProgress.Text = "";
    if (listHand.SelectedIndex < 0) {
      MessageBox.Show("Please select a card");
      return;
    }
    if (game.PlayOneRound(listHand.SelectedIndex)) {
      textProgress.Text += "The winner is... " + game.GetWinnerName();
      textBooks.Text = game.DescribeBooks();
      buttonAsk.Enabled = false;
    } else
      UpdateForm();
  }
}
```

O jogador seleciona uma das cartas e clica no botão pergunte para ver se algum dos outros jogadores tem uma carta com o mesmo valor da selecionada. A classe Game joga um turno usando o método PlayOneRound() (jogar um turno).

4 **Você vai precisar deste código aqui também**
Você vai precisar do código escrito para a classe `Card`, a `Deck` e a `CardComparer_byValue`. Mas você deverá adicionar mais alguns métodos na classe `Deck`... e tem que entendê-los para por usá-los.

```
public Card Peek(int cardNumber) {
    return cards[cardNumber];
}
public Card Deal() {
    return Deal(0);
}
public bool ContainsValue(Card.Values value) {
    foreach (Card card in cards)
      if (card.Value == value)
        return true;
    return false;
}
```

O método Peek() (espiar) permite que você dê uma espiada numa carta do baralho sem ter de pegá-la para você.

Alguém sobrecarregou Deal() (dar carta) para deixá-lo um pouco mais fácil de ler. Se você não passar nenhum parâmetro, ele dá a carta do topo do baralho.

O método ContainsValue() procura pelo baralho todo por cartas um valor e retorna true se encontrar alguma. Você pode adivinhar como usará esse método nesse jogo?

```
public Deck PullOutValues(Card.Values value) {
    Deck deckToReturn = new Deck(new Card[] { });
    for (int i = cards.Count - 1; i >= 0; i--)
      if (cards[i].Value == value)
        deckToReturn.Add(Deal(i));
    return deckToReturn;
}
public bool HasBook(Card.Values value) {
    int NumberOfCards = 0;
    foreach (Card card in cards)
      if (card.Value == value)
        NumberOfCards++;
    if (NumberOfCards == 4)
      return true;
    else
      return false;
}
public void SortByValue() {
    cards.Sort(new CardComparer_byValue());
}
```

Você usará o método PullOutValues() (retirar valores) quando fizer o código para conseguir um livro de cartas do baralho. Ele procura por qualquer carta com um determinado valor, tira-as do baralho e retorna um baralho novo sem aquelas cartas.

O método HasBook() (tem livro) checa um baralho para ver se ele contém um livro de quatro cartas de qualquer valor que seja passado como parâmetro. Ele retorna true se existir um livro no baralho e false se não.

O método SortByValue() ordena o baralho usando a classe Comparer_byValue.

5 Agora vem a parte DIFÍCIL: desenvolva a classe Player

Existirá uma instância de Player (jogador) para cada um dos três jogadores. Elas serão criadas pelo tratador de eventos do botão `buttonStart`.

Olhe atentamente para cada comentário - eles dizem a você o que o método deve fazer. Sua tarefa é desenvolvê-los.

```
public class Player
{
  private string name;
  public string Name { get { return name; } }
  private Random random;
  private Deck cards;
  private TextBox textBoxOnForm;
  public Player(String name, Random random, TextBox textBoxOnForm) {
    // o construtor para a classe Player inicializa quatro campos private, e entao
    // adiciona uma linha no controle TextBox no formulario que diz "Joao entrou
    // no jogo" - mas use o nome no campo privado e nao se esqueca de
    // adicionar uma quebra ("\r\n") no fim de cada linha adicionada na caixa de texto
  }
  public List<Card.Values> PullOutBooks() { } // veja o código no final da página
  public Card.Values GetRandomValue() {
    // esse metodo retorna uma valor aleatorio - mas devem ser um que exista no baralho!
  }
  public Deck DoYouHaveAny(Card.Values value) {
    // neste metodo um oponente pergunta se o jogador tem cartas de um certo valor
    // usando Deck.PullOutValues() para retirar os valores. Adicione uma linha na
    // caixa de texto que diz "Joao tem 3 Seis" - use o novo metodo estatico Card.Plural()
  }
  public void AskForACard(List<Player> players, int myIndex, Deck stock) {
    // Aqui temos uma versao sobrecarregada de AskForACard() - escolha um valor
    // aleatorio do baralho usando GetPandomValue() e pergunte por ele usando AskForACard()
  }
  public void AskForACard(List<Player> players, int myIndex, Deck stock, Card.Values value) {
    // pergunte para outros jogadores se eles tem um dado valor. Primeiro adicione
    // uma linha na caixa de texto que diz "Joao pergunta se alguem tem alguma Rainha",
    // por exemplo. Entao itere pela lista de jogadores passada como parametro e para
    // cada um deles pergunte se ele tem o valor (usando seu método DoYouHaveAny).
    // Ele deve passar a você um baralho - adicione este ao seu. Mantenha registro
    // de quantas cartas foram adicionadas. Se forem zero, teremos que pegar uma do
    // monte (que também foi passado como parametro), e nesse caso uma linha
    // "joao tem que pegar uma carta da pilha" deve ser adicionada
  }
  // Eis aqui uma propriedade e alguns metodos pequenos que ja foram escritos para voce'
  public int CardCount { get { return cards.Count; } }
  public void TakeCard(Card card) { cards.Add(card); }
  public string[] GetCardNames() { return cards.GetCardNames(); }
  public Card Peek(int cardNumber) { return cards.Peek(cardNumber); }
  public void SortHand() { cards.SortByValue(); }
}
```

Ainda não terminou - vire a página!

Exercício Longo (continuação)

Esse método PeekO acrescentado à classe Deck será útil. Ele permite ao programa olhar uma das cartas do baralho dado um índice, mas diferente de DealO, ele não remove a carta.

```
public List<Card.Values> PullOutBooks() {
  List<Card.Values> Books = new List<Card.Values>();
  for (int i = 1; i <= 13; i++) {
    Card.Values value = (Card.Values)i;
    int howMany = 0;
    for (int card = 0; card < cards.Count; card++)
      if (cards.Peek(card).Value == value)
        howMany++;
    if (howMany == 4) {
      Books.Add(value);
      for (int card = cards.Count - 1; card >= 0; card--)
        cards.Deal(card);
    }
  }
  return Books;
}
```

Você terá que desenvolver DUAS versões sobrecarregadas do método AskForACardO. A primeira será usada pelos oponentes quando eles perguntarem por valores - ela procura nas próprias mãos por uma carta para originar a pergunta. A segunda será usada quando o jogador perguntar por uma carta. As duas perguntam para TODOS os demais jogadores (tanto computadorizados quanto humanos) por qualquer carta que tenha o mesmo valor.

6 **Você precisará adicionar o seguinte método na classe Card**
É um método estático que recebe um valor e retorna-o no plural - dessa forma um "cinco" retornará "cincos", mas um "seis" deve retornar "seis". Uma vez que ele é estático, deve ser chamado diretamente com o nome da classe - `Card.Plural()` - e não de uma instância.

```
public partial class Card {
  public static string Plural(Card.Values value) {
    if (value == Values.Six)
      return "Sixes";
    else
      return value.ToString() + "s";
  }
}
```

7 **O resto do trabalho: desenvolva a classe Game**
O formulário mantém uma instância de Game. Ela gerencia o jogo. Examine atentamente como ela é usada no formulário.

```
public class Game {
  private List<Player> players;
  private Dictionary<Card.Values, Player> books;
  private Deck stock;
  private TextBox textBoxOnForm;
  public Game(string playerName, string[] opponentNames, TextBox textBoxOnForm) {
    Random random = new Random();
    this.textBoxOnForm = textBoxOnForm;
    players = new List<Player>();
    players.Add(new Player(playerName, random, textBoxOnForm));
    foreach (string player in opponentNames)
      players.Add(new Player(player, random, textBoxOnForm));
    books = new Dictionary<Card.Values, Player>();
    stock = new Deck();
    Deal();
    players[0].SortHand();
  }
  private void Deal() {
    // sera aqui o metodo A PARTIR DO QUAL o jogo comeca - ele sera chamado apenas no
    // inicio do jogo. Ele embaralha o monte, dá cinco cartas para cada jogador e
    // usa um laco foreach para chamar o metodo PullOutBooks() de cada um.
  }
  public bool PlayOneRound(int selectedPlayerCard) {
    // Execute um turno do jogo. O parametro sera a carta que o jogador selecionou
    // na sua mao - recupera seu valor. Entao itere por todos os jogadores e chame o
    // metodo AskForACard() de cada um, começando pelo jogador humano (que deve
    // estar no indice zero na lista de jogadores. Certifique-se de que ele pergunte pelo valor
```

```
      // da carta que selecionou). Entao chame PullOutBooks() - se ele retornar true,
      // o jogador ficou sem cartas e precisa comprar uma nova mao. Depois
      // que todos jogaram, ordene a mao do jogador humano (para que ela apareca bem
      // arrumada no formulario). Verifique se o monte ainda tem cartas. Se nao tem,
      // apague o texto na caixa e escreva "O monte esta sem cartas. O jogo acabou!" e
      // retorne true. Se nao for o caso, o jogo ainda nao acabou, logo retorne false.
}
public bool PullOutBooks(Player player) {
      // Monte um livro para um jogador. Retorne true se o jogador ficar sem cartas. Se
      // nao for o caso, retorne false. Cada livro deve ser adicionado ao dicionario Books.
      // Um jogador fica sem cartas quando usar todas as suas restantes para montar um
      // livro - e assim tentar ganhar o jogo.
}
public string DescribeBooks() {
      // Retorne uma longa string que descreve os livros de todos, composta examinando
      // o dicionario Books. "Joao tem um livro de seis. (quebra de linha) Edu tem um livro
      de Ases."
}

public string GetWinnerName() {
      // este metodo sera chamado no final do jogo. Ele usa seu proprio dicionario
      // (Dictionary<string, int> winners) para determinar quantos livros cada jogador
      // acrescentou no dicionario Books. Primeiro ele usa um laco foreach em Book.Keys
      // foreach(Card.Values value in Books.Keys) - para povoar seu dicionario winners
      // (vencedores) com a quantidade de livros de cada um. Entao ele itera por esse
      // dicionário para encontrar o maior numero de livros, o que determina o vencedor.
      // E finalmente ele faz uma ultima passagem por winners para montar uma string
      // com o nome dos vencedores ("Joao e Edu", por exemplo). Se temos um unico
      // vencedor,  ele deve retornar uma string como "Edu com tres livros". Se temos mais
      // de um, ele deve retornar algo como "um empate entre Joao e Beto com 2 livros"
}
// Eis aqui dois pequenos metodos que ja foram escritos para você
public string[] GetPlayerCardNames() {
  return players[0].GetCardNames();
}
public string DescribePlayerHands() {
  string description = "";
  for (int i = 0; i < players.Count; i++) {
    description += players[i].Name + " has " + players[i].CardCount;
    if (players[i].CardCount == 1)
        description += " card.\r\n";
    else
        description += " cards.\r\n";
  }
  description += "The stock has " + stock.Count + " cards left.";
  return description;
}
```

Exercício Longo

Aqui estão os métodos da classe Game.

```
public class Game {
  private void Deal() {
    stock.Shuffle();
    for (int i = 0; i < 5; i++)
      foreach (Player player in players)
        player.TakeCard(stock.Deal());
    foreach (Player player in players)
      PullOutBooks(player);
  }
  public bool PlayOneRound(int selectedPlayerCard) {
   Card.Values cardToAskFor = players[0].Peek(selectedPlayerCard).Value;
    for (int i = 0; i < players.Count; i++) {
      if (i == 0)
        players[0].AskForACard(players, 0, stock, cardToAskFor);
      else
        players[i].AskForACard(players, i, stock);
    if (PullOutBooks(players[i])) {
      textBoxOnForm.Text += players[i].Name + " drew a new hand\r\n";
      int card = 1;
      while (card <= 5 && stock.Count > 0) {
        players[i].TakeCard(stock.Deal());
        card++;
      }
    }
  players[0].SortHand();
      if (stock.Count == 0) {
        textBoxOnForm.Text = "The stock is out of
cards. Game over!\r\n";
        return true;
      }
    }
    return false;
  }
  public bool PullOutBooks(Player player)
  {
    List<Card.Values> BooksPulled =
player.PullOutBooks();
    foreach (Card.Values value in BooksPulled)
      books.Add(value, player);
    if (player.CardCount == 0)
      return true;
    return false;
  }
  public string DescribeBooks() {
    string whoHasWhichBooks = "";
    foreach (Card.Values value in books.Keys)
      whoHasWhichBocks += books[value].Name + " has a book of "
        + Card.Plural(value) + "\r\n";
    return whoHasWhichBooks;
  }
  public string GetWinnerName() {
    Dictionary<string, int> winners = new Dictionary<string, int>();
    foreach (Card.Values value in books.Keys) {
      string name = books[value].Name;
      if (winners.ContainsKey(name))
        winners[name]++;
      else
        winners.Add(name, 1);
    }
```

O método Deal() é chamado quando o jogo começa. Ele embaralha o monte e então dá cinco cartas para cada jogador. Então ele monta qualquer livro que por acaso os jogadores já tenham.

Tão logo o jogador clique no botão pergunte por uma carta, o jogo chama AskForACard() com essa carta. Então ele chama o mesmo método para cada oponente.

Depois que o jogador ou um oponente pergunta por uma carta, o jogo monta qualquer livro que ele possa ter feito. Se o jogador ficar sem cartas, ele compra uma nova mão de cinco cartas do monte.

Depois que o turno termina, o jogo ordena a mão do jogador, para se certificar que as cartas apareçam em ordem no formulário. Então ele checa para ver se o jogo acabou. Se é o caso, PlayOneRound() retorna true.

PullOutBooks() olha as cartas de um jogador para ver se ele conseguiu quatro do mesmo valor. Se for o caso, elas são adicionadas ao dicionário de livros. E se não sobrar nenhuma carta depois disso, ela retorna true.

O formulário precisa mostrar uma lista de livros, então ele usa DescribeTheBooks() para transformar um dicionário de livros de um jogador em texto.

Depois que a última carta foi comprada, o jogo precisa determinar quem ganhou. E o que GetWinnerName() faz. E ele usa um dicionário chamado winners para fazer isso. O nome de cada jogador é uma chave no dicionário, e o valor de cada chave é o número de livros que aquele jogador conseguiu durante o jogo.

```
    int mostBooks = 0;
    foreach (string name in winners.Keys)
      if (winners[name] > mostBooks)
        mostBooks = winners[name];
    bool tie = false;
    string winnerList = "";
    foreach (string name in winners.Keys)
      if (winners[name] == mostBooks)
      {
        if (!String.IsNullOrEmpty(winnerList))
        {
          winnerList += " and ";
          tie = true;
        }
        winnerList += name;
      }
    winnerList += " with " + mostBooks + " books";
    if (tie)
      return "A tie between " + winnerList;
    else
      return winnerList;
  }
}
```

Em seguida o jogo itera pelo dicionário para determinar o número de livros que o jogador com a maior quantidade deles tem. Ele coloca esse valor numa variável chamada mostBooks.

Agora que nós sabemos qual jogador tem mais livros, o método pode criar uma string que indica o vencedor (ou vencedores).

Aqui estão os métodos da classe Game.

```
public Player(String name, Random random, TextBox textBoxOnForm) {
    this.name = name;
    this.random = random;
    this.textBoxOnForm = textBoxOnForm;
    this.cards = new Deck( new Card[] {} );
    textBoxOnForm.Text += name + " has just joined the game\r\n";
}
public Card.Values GetRandomValue() {
    Card randomCard = cards.Peek(random.Next(cards.Count));
    return randomCard.Value;
}
public Deck DoYouHaveAny(Card.Values value) {
    Deck cardsIHave = cards.PullOutValues(value);
    textBoxOnForm.Text += Name + " has " + cardsIHave.Count + " "
        + Card.Plural(value) + "\r\n";
  return cardsIHave;
}
public void AskForACard(List<Player> players, int myIndex, Deck stock) {
  Card.Values randomValue = GetRandomValue();
  AskForACard(players, myIndex, stock, randomValue);
}
public void AskForACard(List<Player> players, int myIndex,
    Deck stock, Card.Values value) {
  textBoxOnForm.Text += Name + " asks if anyone has a " + value + "\r\n";
  int totalCardsGiven = 0;
  for (int i = 0; i < players.Count; i++) {
    if (i != myIndex) {
      Player player = players[i];
      Deck CardsGiven = player.DoYouHaveAny(value);
      totalCardsGiven += CardsGiven.Count;
      while (CardsGiven.Count > 0)
        cards.Add(CardsGiven.Deal());
    }
  }
  if (totalCardsGiven == 0) {
    textBoxOnForm.Text += Name + " must draw from the stock.\r\n";
    cards.Add(stock.Deal());
  }
}
```

Aqui está o construtor da classe Player. Ele atribui valores para os campos privados e adiciona uma linha à caixa de texto de progresso dizendo que o jogador se juntou ao jogo.

O método GetRandomValue() usa Peek() para olhar uma carta aleatória na mão do jogador.

DoYouHaveAny() usa o método PullOutValues() para retirar e retornar todas as cartas que sejam equivalentes ao parâmetro.

Existem dois métodos AskForACard() sobrecarregados. Este é usado pelos oponentes - ele pega uma de suas cartas aleatoriamente e chama o outro método AskForACard().

Este método AskForACard() itera por todos os jogadores (exceto aquele que está perguntando), chamando o método DoYouHaveAny(), e adiciona qualquer carta que seja entregue na mão do jogador que chama.

Se nenhuma carta foi entregue, o jogador tem que comprar uma do monte usando o método Deal().

E ainda MAIS outros tipos de coleções...

`List` e `Dictionary` são duas das **coleções genéricas nativas** que são parte do Framework .NET. Listas e dicionários são bastante flexíveis - você pode acessar qualquer dos dados em qualquer ordem. Mas algumas vezes você precisa restringir a forma como um programa trabalha com os dados porque a coisa que você está representando dentro do seu programa funciona dessa forma no mundo real. Para situações como essas, você pode usar uma **Queue** (fila) ou **Stack** (pilha). Essas são as outras duas coleções genéricas que se parecem com listas, mas são especialmente boas para se certificar de que os dados sejam processados seguindo uma ordem.

Existem ainda mais outros tipos de coleções - mas estes são os que você provavelmente vai encontrar mais vezes.

Use uma fila quando o primeiro objeto que você armazenou será o primeiro a ser usado, como por exemplo:

- ★ Carros movendo-se por uma rua estreita de mão única;
- ★ Pessoas numa fila;
- ★ Clientes esperando para serem atendidos por um setor de suporte por telefone;
- ★ Qualquer outra coisa que precise ser tratada na base do primeiro-a-chegar, primeiro-a-atender.

Uma fila é primeiro-a-entrar, primeiro-a-sair, o que quer dizer que o primeiro objeto colocado nela é o primeiro que se retira para usar.

Use uma pilha quando você sempre quiser usar o objeto mais recente, como por exemplo:

- ★ Mobília colocada num caminhão de mudanças;
- ★ Uma pilha de livros onde se quer ler aquele que foi acrescentado mais recentemente;
- ★ Uma pirâmide humana onde os mais acima devem descer primeiro (imagine a bagunça se alguém embaixo sair antes!).

Uma pilha é primeiro-a-entrar, último-a-sair - o primeiro objeto que entra na pilha é o último que sai dela.

Coleções genéricas são uma parte importante do Framework .NET

Elas são realmente úteis - tanto que o IDE automaticamente adiciona o seguinte comando no topo de qualquer classe adicionada a um projeto:

```
using System.Collections.Generic;
```

Quase todo projeto grande inclui algum tipo de coleção genérica, porque seus programas precisam armazenar dados. E quando você estiver lidando com grupos de coisas similares no mundo real, elas quase sempre caem numa categoria que corresponde bastante bem a um dos tipos de coleções.

Uma fila é como uma lista que deixa que você coloque objetos no final e use os do começo. Uma pilha só deixa você acessar o último objeto colocado nela.

Você pode, no entanto, usar foreach para iterar pelos elementos de uma pilha ou fila, porque elas implementam IEnumerable!

Uma fila é PEPS* - Primeiro a Entrar, Primeiro a Sair

Uma **queue** (fila) parece muito com uma lista, exceto que você não pode adicionar ou remover elementos em qualquer lugar. Para adicionar um elemento a uma fila, você o **enfileira**. Isso adiciona o objeto ao final da fila. Você pode **desenfileirar** o primeiro objeto, que fica no início da fila. Quando você faz isso, o objeto é removido da fila e o restante dos objetos avançam nela uma posição.

Cria uma nova fila de strings.

Aqui adicionamos quatro itens na fila. Quando os retiramos dela, eles vêm na mesma ordem em que foram enfileirados.

Peek() permite espiar o primeiro item da fila sem removê-lo.

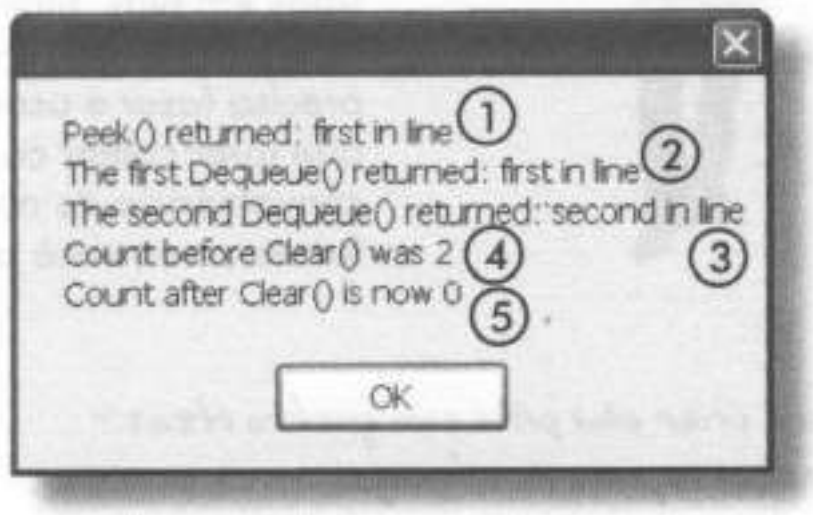
O primeiro Dequeue() (desenfileire) retira o primeiro item da fila. Então o segundo item avança uma posição e vai para o primeiro lugar - a próxima chamada ao mesmo método recuperará esse item.

O método Clear() remove todos os objetos da fila.

A propriedade Count da fila retorna o número de itens ainda nela.

```
Queue<string> myQueue = new Queue<string>();
myQueue.Enqueue("first in line");
myQueue.Enqueue("second in line");
myQueue.Enqueue("third in line");
myQueue.Enqueue("last in line");
string takeALook = myQueue.Peek();          (1)
string getFirst = myQueue.Dequeue();        (2)
string getNext = myQueue.Dequeue();         (3)
int howMany = myQueue.Count;                (4)
myQueue.Clear();
MessageBox.Show("Peek() returned: " + takeALook + "\n"
  + "The first Dequeue() returned: " + getFirst + "\n"
  + "The second Dequeue() returned: " + getNext + "\n"
  + "Count before Clear() was " + howMany + "\n"
  + "Count after Clear() is now " + myQueue.Count);   (5)
```

Objetos numa fila precisam esperar cada um a sua vez. O primeiro



Uma pilha é UEPS* - Último a Entrar, Primeiro a Sair

Uma **stack** (pilha) é realmente similar a uma fila - com uma grande diferença. Você **empilha** um item numa pilha e quando quer retirar um dela você puxa esse item. Quando você puxa um item de uma pilha, você **acaba** com o que mais recentemente foi empilhado nela. É exatamente como uma pilha de pratos, revistas ou qualquer outra coisa - você pode colocar algo em cima da pilhas, mas precisa primeiro tirar os de cima antes de conseguir pegar o que está embaixo.

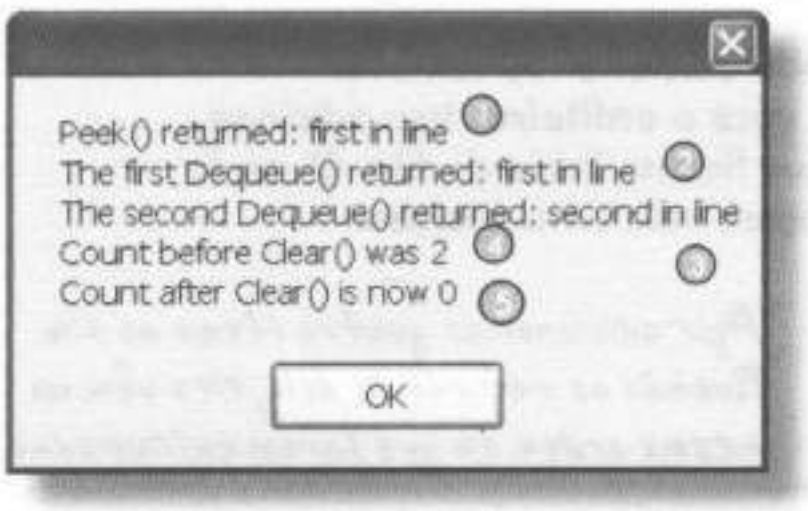
Criar uma pilha é exatamente como criar qualquer outra coleção genérica.

Quando você empilha algo, os demais itens ficam em uma posição para trás daquele item no topo.

Quando você puxa um item de uma pilha, recupera o que mais recentemente foi empilhado.

```
Stack<string> myStack = new Stack<string>();
myStack.Push("first in line");
myStack.Push("second in line");
myStack.Push("third in line");
myStack.Push("last in line");
(1) string takeALook = myStack.Peek();
(2) string getFirst = myStack.Pop();
(3) string getNext = myStack.Pop();
(4) int howMany = myStack.Count;
myStack.Clear();
MessageBox.Show("Peek() returned: " + takeALook + "\n"
  + "The first Pop() returned: " + getFirst + "\n"
  + "The second Pop() returned: " + getNext + "\n"
  + "Count before Clear() was " + howMany + "\n"
  + "Count after Clear() is now " + myStack.Count);
                                                  (5)
```

* (NT) Em inglês, FIFO, first in, first out.



O último objeto que você colocar numa pilha é o primeiro que vai sair dela.

Espere um pouco, tem algo me incomodando. Você não me mostrou nada que eu possa fazer com uma pilha ou fila que não possa ser feito usando uma lista - elas apenas me poupam umas poucas linhas de código. Mas eu não posso acessar os itens no meio de uma pilha ou fila. E posso fazer com uma lista facilmente! Então eu desistiria disso somente por causa dessas pequenas conveniências?

Não se preocupe - você não desiste de nada quando usa uma fila ou pilha

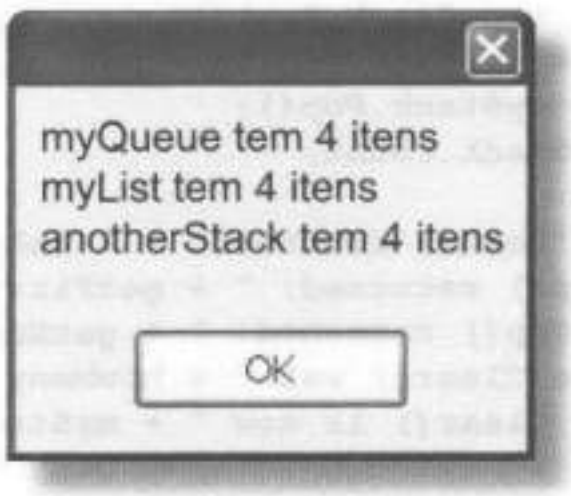
É muito fácil converter um objeto Queue num List. E é assim também para converter listas em filas, filas em pilhas... de fato, você pode criar uma lista, fila ou pilha a partir de qualquer outro objeto que implemente a interface `IEnumerable`. Tudo o que você precisa fazer é usar o construtor sobrecarregado que deixa você passar a coleção que quer copiar como um parâmetro. Isso quer dizer que você tem a flexibilidade e a conveniência de representar seus dados com a coleção que melhor se adéqüe com a forma como você precisa usar os dados.

Vamos criar uma pilha com quatro itens - nesse caso, uma de strings.

É fácil converter essa pilha numa fila, e então copiar a fila para uma lista, e então copiar a lista para outra pilha.

```
Stack<string> myStack = new Stack<string>();
myStack.Push("first in line");
myStack.Push("second in line");
myStack.Push("third in line");
myStack.Push("last in line");
Queue<string> myQueue = new Queue<string>(myStack);
List<string> myList = new List<string>(myQueue);
Stack<string> anotherStack = new Stack<string>(myList);
MessageBox.Show("myQueue has " + myQueue.Count + " items\n"
+ "myList has " + myList.Count + " items\n"
+ "anotherStack has " + anotherStack.Count + " items\n");
```

Todos os quatro itens foram copiados nas novas coleções.



...e você pode sempre usar um laço foreach para acessar todos os membros numa pilha ou fila!

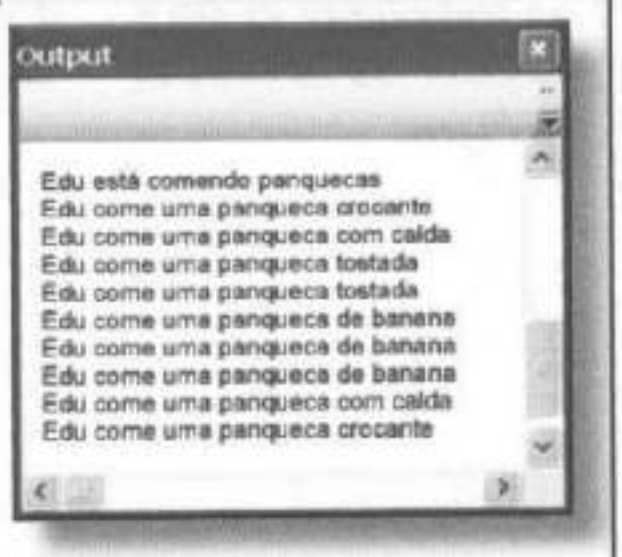
Exercício

Escreva um programa para ajudar uma cafeteria cheia de lenhadores (lumberjack) a comer algumas panquecas (apelidadas de "flapjack" em inglês, algo como "cara achatado"). Comece com a classe Lumberjack, preenchendo o código que falta. Então crie o formulário e adicione os tratadores de eventos nos botões.

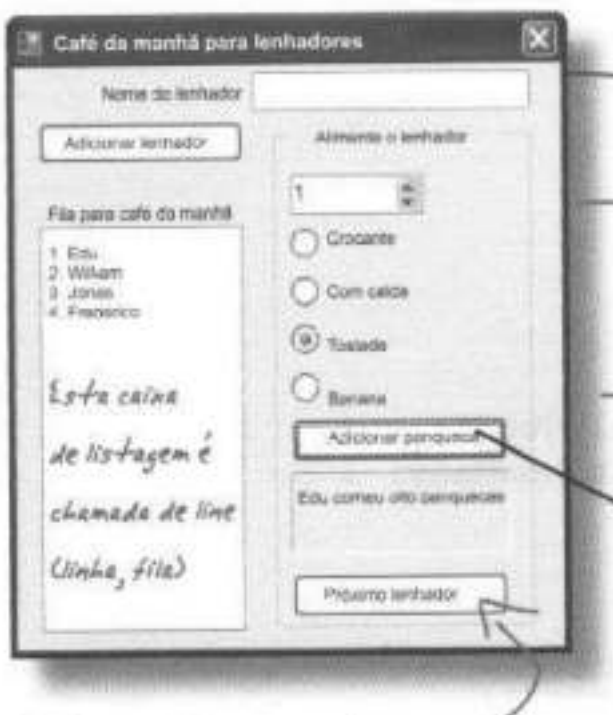
1 Aqui está a classe Lumberjack. Preencha o acessador get para `FlapjackCount` e os métodos `TakeFlapjacks` (pegar panquecas) e `EatFlapJack` (comer panqueca).

```
public class Lumberjack {
  private string name;
  public string Name { get { return name; } }
  private Stack<Flapjack> meal;
  public Lumberjack(string name) {
    this.name = name;
    meal = new Stack<Flapjack>();
  }
  public int FlapjackCount { get { // retorne a quantidade } }
  public void TakeFlapjacks(Flapjack Food, int HowMany) {
    // adicione algumas panquecas na pilha Meal (refeicao)
  }
  public void EatFlapjacks() {
    // escreva o resultado no console
} }
```

```
public enum Flapjack {
  crispy,
  soggy,
  browned,
  banana
}
```



2 Crie o formulário. Ele deve permitir inserir o nome do lenhador numa caixa de texto para ser colocado na fila do café da manhã. Você pode dar ao lenhador na frente da fila um prato de panquecas e então dizer-lhe para comê-las, usando o botão "próximo lenhador". Fizemos o tratador de evento clique do botão "Adicionar panquecas" para você. Use uma fila chamada **breakfastLine** (fila de café da manhã) para controlar os lenhadores.



Quando o usuário clicar Adicionar lenhador, coloque o nome informado na caixa de texto na fila BreakfastLine.

Quando você arrastar esses controles botões de rádio para dentro da caixa de grupo, o formulário automaticamente os conecta e permite apenas ao usuário selecionar um deles de cada vez. Olhe o método addFlapjacks_Click para descobrir qual deve ser o nome deles.

Este botão deve tirar da fila o próximo lenhador, chamar seu método EatFlapjacks e então repintar a caixa de listagem.

Você precisa adicionar um método RedrawList() (repintar lista, literalmente redesenhar lista) para atualizar o conteúdo da fila. Todos os três botões devem chamá-lo. Eis uma dica: use um laço foreach.

```
private void addFlapjacks_Click(...) {
  Flapjack food;
  if (crispy.Checked == true)
    food = Flapjack.crispy;
  else if (soggy.Checked == true)
    food = Flapjack.soggy;
  else if (browned.Checked == true)
    food = Flapjack.browned;
  else
    food = Flapjack.banana;
  Lumberjack currentLumberjack = breakfastLine.Peek();
  currentLumberjack.TakeFlapjacks(food,
        (int)howMany.Value);
  RedrawList();
}
```

Note a sintaxe específica do else if.

Peek() retorna uma referência para o primeiro lenhador na fila.

O controle NumericUpDown deve chamar howMany (quantos) e o rótulo nextInLine (próximo da fila).

Solução do Exercício

```
private Queue<Lumberjack> breakfastLine = new Queue<Lumberjack>();
private void addLumberjack_Click(object sender, EventArgs e) {
  breakfastLine.Enqueue(new Lumberjack(name.Text));
  name.Text = "";
  RedrawList();
}
private void RedrawList() {
  int number = 1;
  line.Items.Clear();
  foreach (Lumberjack lumberjack in breakfastLine) {
    line.Items.Add(number + ". " + lumberjack.Name);
    number++;
  }
  if (breakfastLine.Count == 0) {
    groupBox1.Enabled = false;
    nextInLine.Text = "";
  } else {
    groupBox1.Enabled = true;
    Lumberjack currentLumberjack = breakfastLine.Peek();
    nextInLine.Text = currentLumberjack.Name + " has "
        + currentLumberjack.FlapjackCount + " flapjacks";
} }
private void nextLumberjack_Click(object sender, EventArgs e) {
  Lumberjack nextLumberjack = breakfastLine.Dequeue();
  nextLumberjack.EatFlapjacks();
  nextInLine.Text = "";
  RedrawList();
}
public class Lumberjack {
  private string name;
  public string Name { get { return name; } }
  private Stack<Flapjack> meal;
  public Lumberjack(string name) {
    this.name = name;
    meal = new Stack<Flapjack>();
  }
  public int FlapjackCount { get { return meal.Count; } }
  public void TakeFlapjacks(Flapjack food, int howMany) {
    for (int i = 0; i < howMany; i++) {
      meal.Push(food);
  } }
  public void EatFlapjacks() {
    Console.WriteLine(name + "'s eating flapjacks");
    while (meal.Count > 0) {
      Console.WriteLine(name + " ate a "
        + meal.Pop().ToString() + " flapjack");
} } }
```

Chamamos a caixa de listagem de line e o rótulo entre os dois botões nextInLine.

O método RedrawList() usa um laço foreach para puxar os lenhadores da fila e adicioná-los na caixa de listagem.

Este comando if atualiza o rótulo com informação sobre o primeiro lenhador da fila.

O método TakeFlapjacks atualiza a pilha Meal.

O método EatFlapjacks use um laço while para exibir a refeição do lenhador.

A especificação: fazer um jogo de aventura

Seu trabalho é criar um jogo de aventura em que um aventureiro importante está em uma missão para derrotar inimigos mortalmente perigosos nível após nível. Você criará um `sistema de turnos`: o jogador faz um movimento e os inimigos fazem também. O jogador pode se mover ou atacar e cada inimigo tem a mesma chance. O jogo continua até que o jogador ou derrote todos os inimigos nos sete níveis ou morra.

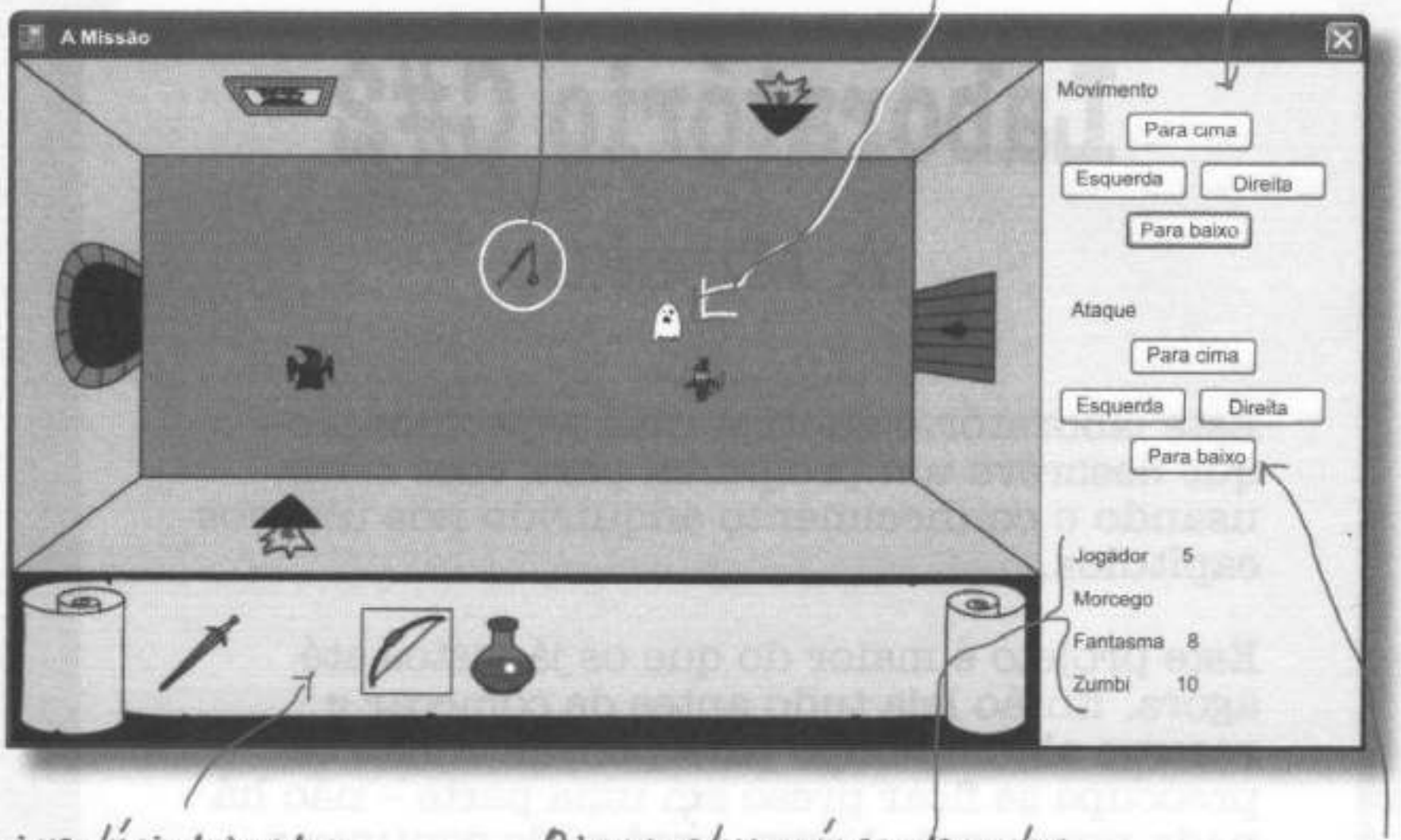
Os inimigos têm uma pequena vantagem – eles podem mover-se em toda jogada e depois disso atacarão se o jogador estiver ao alcance deles.

A janela do jogo dá uma visão de cima da masmorra onde o jogador enfrenta seus inimigos.

O jogador pode pegar armas e poções pela caminho.

O jogador e os inimigos movem-se pela masmorra.

O jogador move-se usando os quatro botões direcionais.



Este é o inventário do jogador. Ele mostra os itens que o jogador pegou e uma caixa em torno do item em uso no momento. O jogador clica em um item para prepará-lo e usa o botão Ataque para usá-lo.

O jogo mostra o número de pontos de vida para o jogador e inimigos. Quando o jogador ataca um inimigo, os **pontos de ataque** do inimigo caem. Quando os pontos de vida chegam em zero, o inimigo ou o jogador morre.

Estes quatro botões são usados para atacar inimigos e beber poções.

O jogador recolhe armas...

Há armas e poções espalhadas pela masmorra que o jogador pode pegar e usar para derrotar seus inimigos. Tudo a fazer é ir até a arma; ela desaparece do chão e aparece em seu inventário.

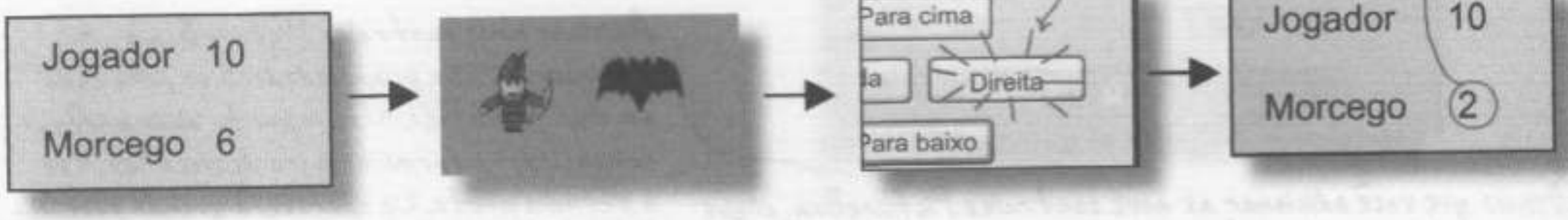
Uma caixa preta em volta da arma significa que ela está **preparada**. Armas diferentes funcionam de forma diferente – elas têm alcances diversos, algumas só atacam em uma única direção enquanto outras têm um alcance maior, e causam diferentes níveis de dano aos inimigos.

...e ataca inimigos com elas

Cada nível no jogo tem uma arma que o jogador pode pegar e usar para derrotar seus inimigos. Quando a arma é recolhida, deve desaparecer do chão do jogo.

O morcego está à direita do jogador, então ele ataca com o botão de ataque Direita.

O ataque faz os pontos de vida do morcego caírem, neste caso, de 6 para 2.



Níveis mais avançados têm mais inimigos

Há três tipos diferentes de inimigos: um morcego, um fantasma e um zumbi. O primeiro nível tem apenas um morcego. O sétimo, que é o último, tem os três inimigos.

Um zumbi move-se rapidamente na direção do jogador e causa um grande dano quando ataca.

O morcego move-se um tanto aleatoriamente. Quando está perto do jogador, causa um pequeno dano.

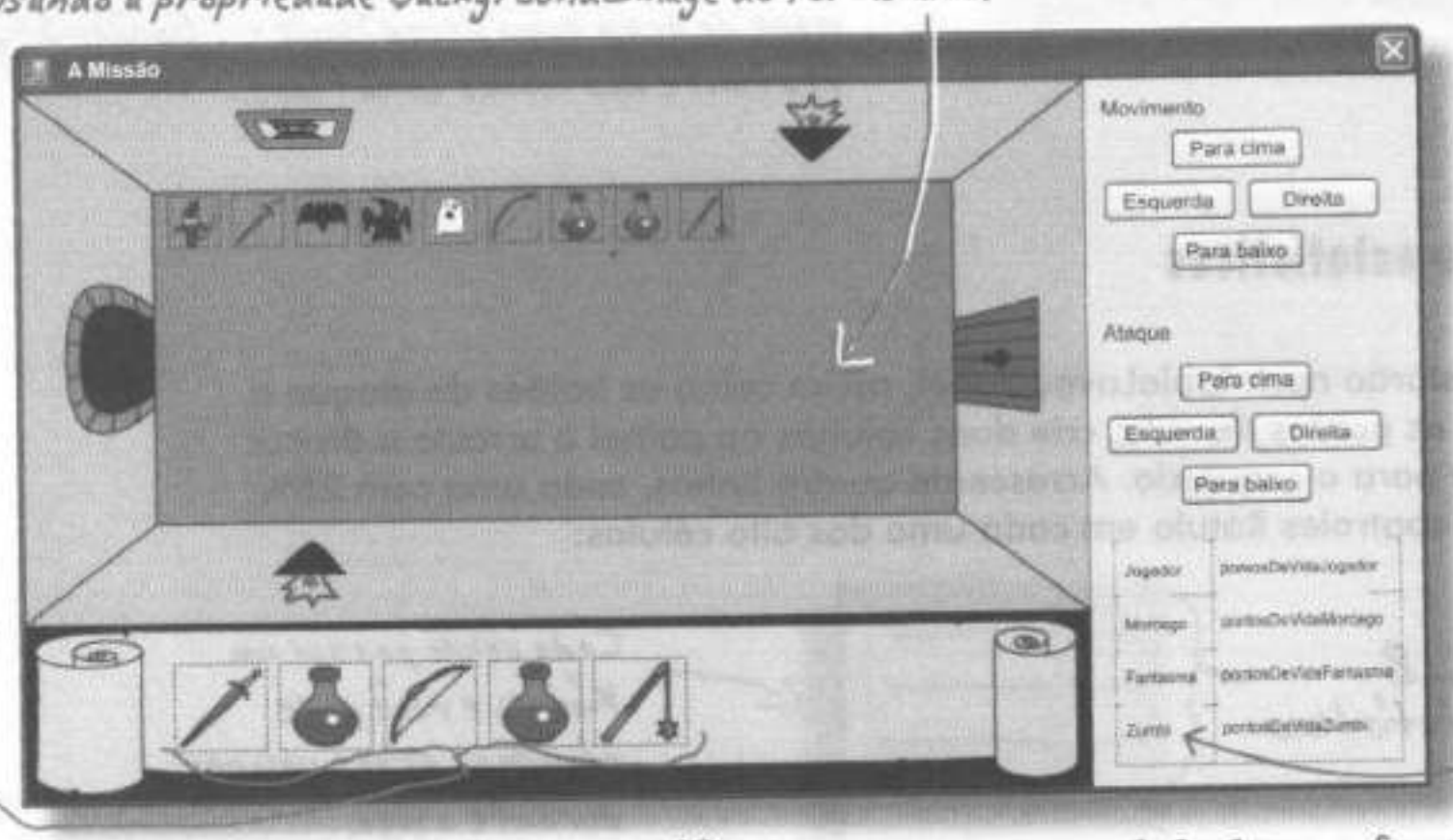
O fantasma move-se vagarosamente na direção do jogador. Assim que chega perto dele, ele ataca e causa dano médio.

O projeto: criando o formulário

O formulário dá ao jogo seu visual único. Use a propriedade `BackgroundImage` dele para exibir a imagem da masmorra e do inventário e uma série de controles `PictureBox` para mostrar o jogador, armas e inimigos na masmorra. Você usará um controle `TableLayoutPanel` para exibir os pontos de vida do jogador, morcego, fantasma e zumbi assim como botões para andar e atacar.

A masmorra em si é uma imagem estática, exibida usando a propriedade BackgroundImage do formulário.

Certifique-se de que a propriedade BackgroundImageLayout seja None.



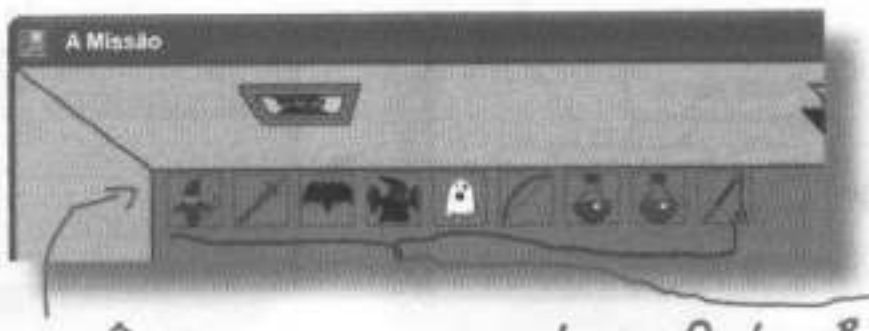
Cada um destes ícones é um PictureBox.

Pontos de vida, botões de movimento e de ataque são todos exibidos em um TableLayoutPanel.

Baixe a imagem de fundo e os gráficos para as armas, inimigos e jogador do site dos Laboratórios do Use a Cabeça: www.altabooks.com.br

Tudo na masmorra é uma caixa de imagem

Jogadores, armas e inimigos devem todos ser representados por ícones. Adicione nove controles PictureBox e dê às suas propriedades Visible valor False. Então, seu jogo pode deslocar os controles pelo formulário e alterar suas propriedades Visible conforme precisar.



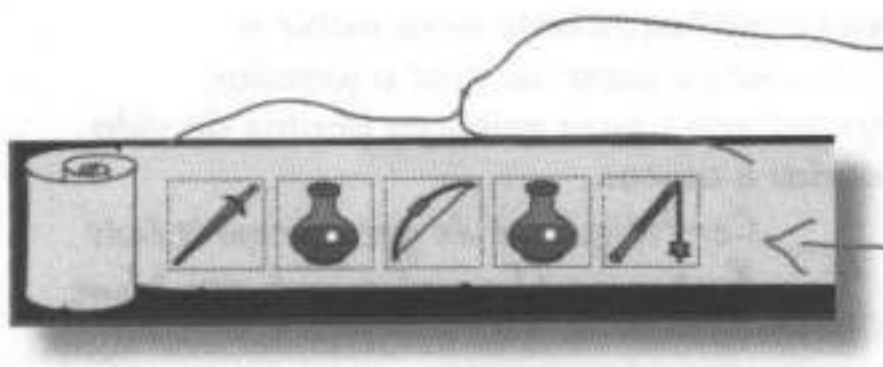
Depois que você adicionar os nove controles PictureBox, clique com o botão direito no ícone do jogador e selecione Bring to Front (trazer para frente), então mande os três ícones de arma para trás (send to back). Isso assegura que o ícone do jogador fique acima de qualquer outro item que ele pegar.

Adicione nove controles PictureBox à masmorra. Use a propriedade Size para cada um ter 30x30. Não importa onde você vai colocá-los – o formulário irá deslocá-los. Use a setinha preta. Ela aparecerá quando você clicar nas caixas de imagem para atribuir a cada uma delas uma imagem. Baixe-a do site dos Laboratórios do Use a Cabeça.

Os controles passam um sobre o outro no IDE. Assim, o formulário precisa saber quais estão na frente e quais estão atrás. É isso que os comandos do designer de formulário Trazer para frente e Mandar para trás fazem.

O inventário contém controles de caixa de imagem também

Você pode representar o inventário do jogador como controles PictureBox 50x50. Estabeleça a propriedade BackColor de cada um como `Transparent`. Já que os arquivos de figura têm um fundo transparente, você verá a barra de rolagem e a masmorra atrás deles:



Você vai precisar de mais cinco controles de caixa de imagem de 50x50 para o inventário.

Quando o jogador prepara uma das armas, o formulário deve configurar o BorderStyle daquele ícone da arma como FixedSingle e o BorderStyle do restante dos ícones como None.

Crie sua janela de estatísticas

Os pontos de vida estarão num TableLayoutPanel, assim como os botões de ataque e de movimento. Para os pontos de vida, crie duas colunas no painel e arraste o divisor da coluna um pouco para a esquerda. Acrescente quatro linhas, cada uma com 25% de altura e adicione controles Rótulo em cada uma das oito células:

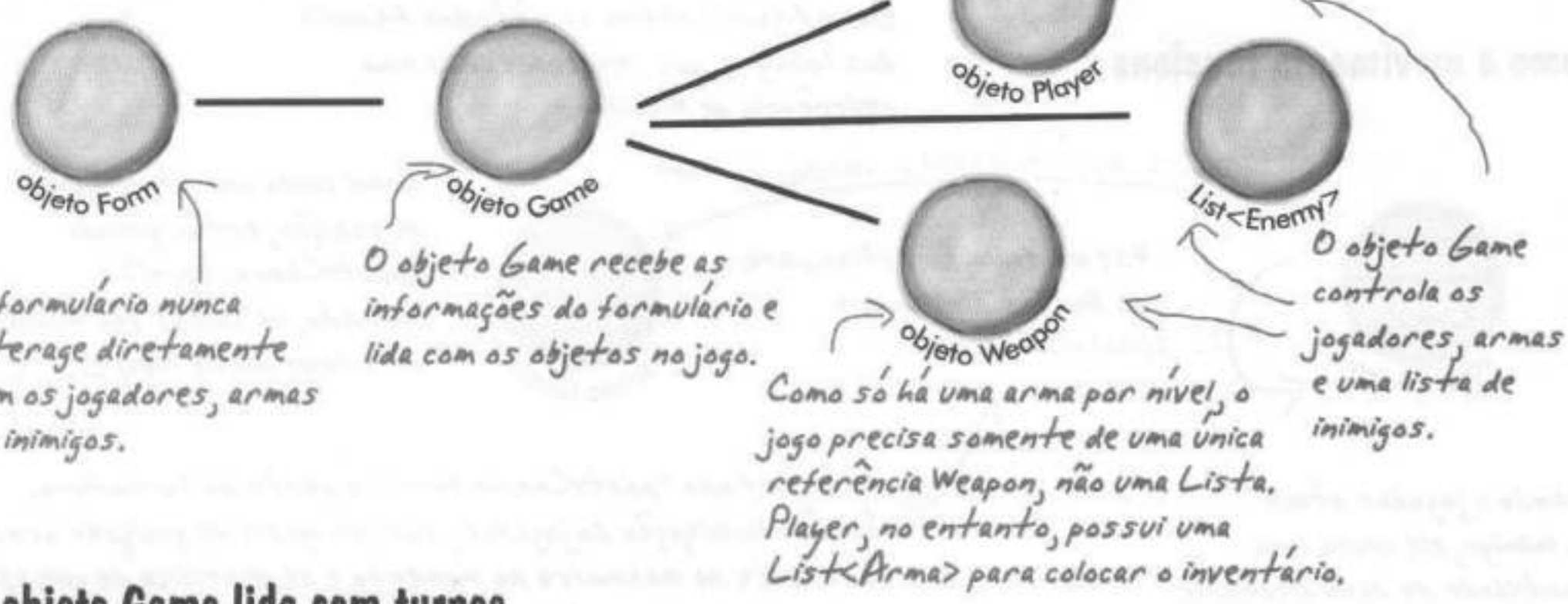
2 colunas, 4 linhas... 8 células para suas estatísticas de pontos de vida.

Cada célula possui um Rótulo e você pode atualizar esses valores durante o jogo.

A arquitetura: usando os objetos

Você precisará de vários tipos de objeto em seu jogo: um objeto `Player` (jogador), muitos subtipos de um objeto Enemy (inimigo) e muitos subtipos de um objeto Weapon (arma). E você também precisa de um objeto para acompanhar tudo que está acontecendo: o objeto Game.

Esta é só uma visão geral. Daremos **mais detalhes** sobre como o jogador e os inimigos se movem, como o inimigo descobre se está perto do jogador, etc.

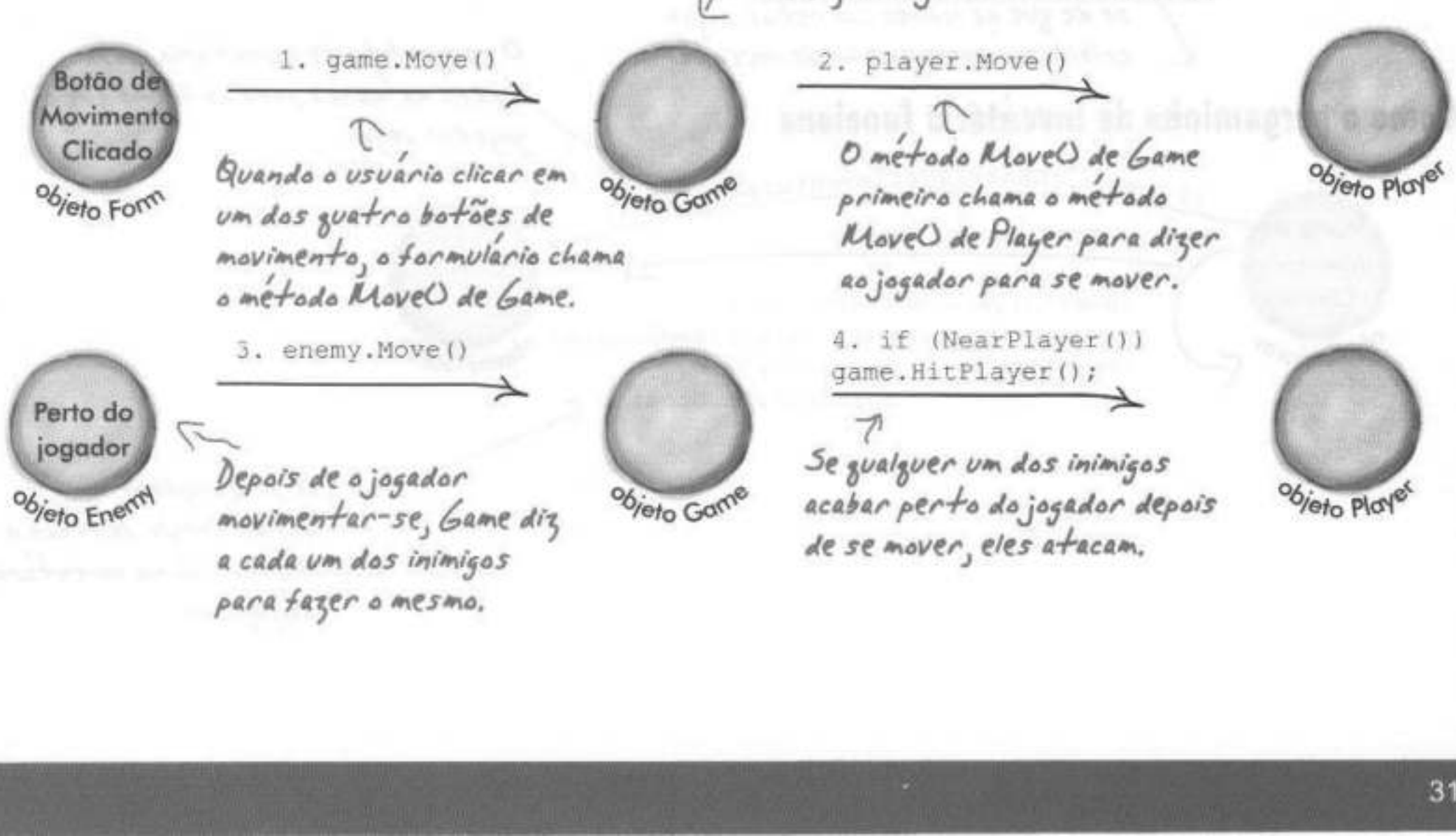


O objeto Game lida com turnos

Quando um dos botões de Movimento de seu formulário for clicado, este chamará o método `Move()` do objeto `Game`. Esse método permitirá que o jogador tenha sua rodada e então que todos os inimigos se movam. Logo é função de `Game` ligar-se com a parte baseada em turnos do movimento no jogo.

Por exemplo, assim é que funciona os botões de movimento:

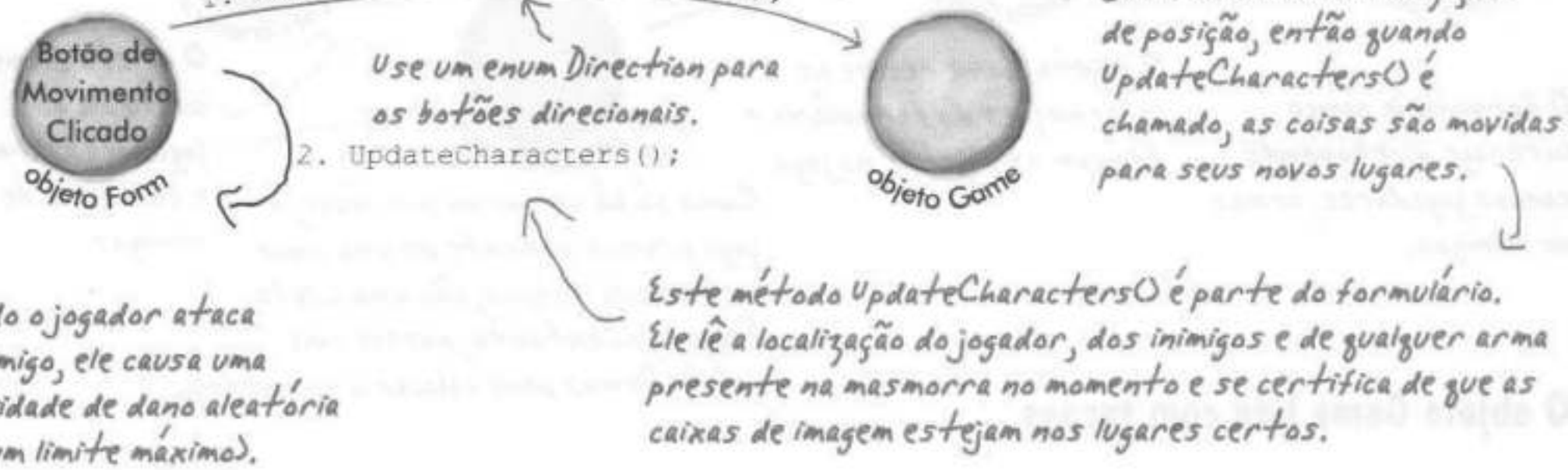
Deixamos os parâmetros fora deste diagrama. Cada método Move() recebe uma direção e alguns deles, um objeto Random também.



O formulário delega atividade ao objeto Game

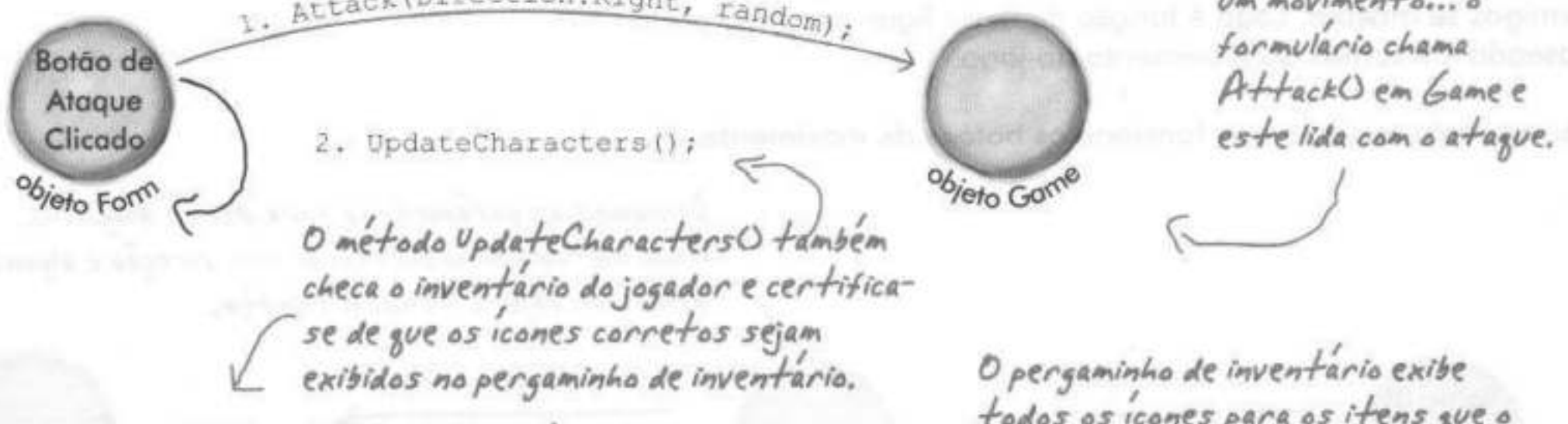
O movimento, o ataque e lidar com o inventário: tudo começa no formulário. Então clicar no botão de movimento ou de ataque, ou em um item no inventário, dispara código em seu formulário. Mas é o objeto `Game` que controla os objetos no jogo. Logo, o formulário deve passar tudo que acontece para o objeto `Game` e este continua daí:

Como o movimento funciona



Quando o jogador ataca um inimigo, ele causa uma quantidade de dano aleatória (até um limite máximo).

Como o ataque funciona



Como o pergaminho de inventário funciona

Criando a classe Game

Começamos a classe Game para você - vide código abaixo. Há muito a ser feito, por isso, leia este código com atenção, entre no IDE e prepare-se para trabalhar:

Você precisará da classe Rectangle do System.Drawing. Portanto, certifique-se de adicionar isto ao topo de sua classe.

Tudo bem essas propriedades permanecerem públicas se Enemy e Weapon estiverem bem encapsulados... em outras palavras, apenas garanta que o formulário não possa fazer nada inapropriado com elas.

O jogo mantém um objeto Player privado. O formulário apenas interagirá com ele através de métodos em Game, em vez de diretamente.

```
using System.Drawing;
public class Game {
  public List<Enemy> Enemies;
  public Weapon WeaponInRoom;
  private Player player;
  public Point PlayerLocation { get { return player.Location; } }
  public int PlayerHitPoints { get { return player.HitPoints; } }
  public List<string> PlayerWeapons { get { return player.Weapons; } }
  private int level = 0;
  public int Level { get { return level; } }
  private Rectangle boundaries;
  public Rectangle Boundaries { get { return boundaries; } }
  public Game(Rectangle boundaries) {
    this.boundaries = boundaries;
    player = new Player(this,
      new Point(boundaries.Left + 10, boundaries.Top + 70),
      boundaries);
  }
  public void Move(Direction direction, Random random) {
    player.Move(direction);
    foreach (Enemy enemy in Enemies)
      enemy.Move(random);
  }
  public void Equip(string weaponName) {
    player.Equip(weaponName);
  }
  public bool CheckPlayerInventory(string weaponName) {
    return player.Weapons.Contains(weaponName);
  }
  public void HitPlayer(int maxDamage, Random random) {
    player.Hit(maxDamage, random);
  }
  public void IncreasePlayerHealth(int health, Random random) {
    player.IncreaseHealth(health, random);
  }
  public void Attack(Direction direction, Random random) {
    player.Attack(direction, random);
    foreach (Enemy enemy in Enemies)
      enemy.Move(random);
  }
  private Point GetRandomLocation(Random random) {
    return new Point(boundaries.Left +
      random.Next(boundaries.Right / 10 - boundaries.Left / 10) * 10,
        boundaries.Top +
      random.Next(boundaries.Bottom / 10 - boundaries.Top / 10) * 10);
  }
  public void NewLevel(Random random) {
    level++;
    switch (level) {
     case 1:
      Enemies = new List<Enemy>();
      Enemies.Add(new Bat(this, GetRandomLocation(random), boundaries));
      WeaponInRoom = new Sword(this, GetRandomLocation(random));
      break;
    }
  }
}
```

O objeto Rectangle possui um campo Top (em cima), Bottom (embaixo), Left (Esquerda) e Right (Direita) e funciona perfeitamente para a área geral do jogo.

Game começa criando uma caixa que envolve a masmorra e depois cria um novo objeto Player nela.

O movimento é simples: mova o jogador na direção que o formulário informa e mova cada inimigo em uma direção aleatória.

Estes são todos bons exemplos de encapsulamento... Game não sabe como Player lida com essas ações, ele apenas passa a informação necessária e deixa Player fazer o resto.

Attack() é quase igual a Move(). O jogador ataca e os inimigos todos têm sua vez de jogar.

GetRandomLocation() será útil para o método NewLevel(), usado para determinar onde colocar os inimigos e as armas.

Isso é só um truque matemático para conseguir um local aleatório dentro do retângulo que representa a área da masmorra.

Só acrescentamos o comando case para o nível 1. E sua função adicionar comandos case para outros níveis.

Termine o restante dos níveis

É sua função terminar o método NewLevel(). Aqui estão os detalhes para cada nível.

Só teremos espaço no inventário para uma poção azul e uma vermelha. Então se o jogador já possui uma vermelha, o jogo não deve adicionar uma vermelha ao nível (e o mesmo vale para a azul).

Então se a poção azul ainda estiver no inventário do jogador do nível 2, nenhuma aparecerá neste.

Nível	Inimigos	Armas
2	Fantasma	Poção azul
3	Zumbi	Arco
4	Morcego, Fantasma	Arco, se não foi pego no 3; caso contrário, poção azul
5	Morcego, Zumbi	Poção vermelha
6	Fantasma, Zumbi	Bastão
7	Morcego, Fantasma, Zumbi	Bastão, se não foi pega no 6; caso contrário, poção vermelha
8	N/A	N/A - termine o jogo com Application.Exit()

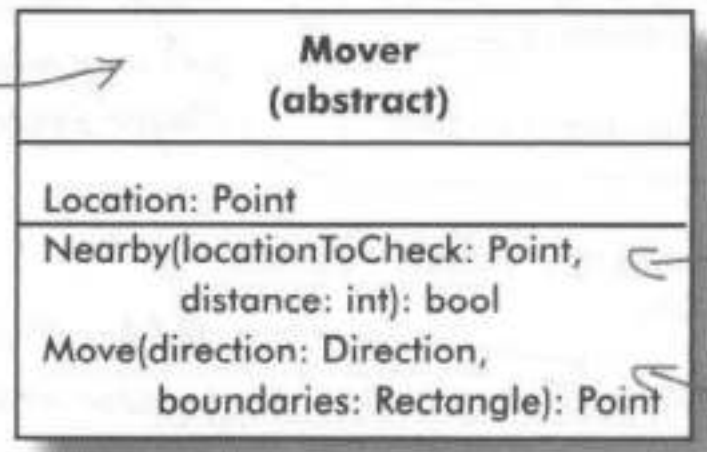
Esta apenas aparece se a poção vermelha do Nível 5 já foi utilizada.

Encontrando comportamento comum: movimento

Você já sabe que duplicar código é ruim. E o código duplicado normalmente aparece quando dois ou mais objetos compartilham o mesmo comportamento. Este é o caso no jogo da masmorra também... tanto os inimigos quanto jogadores movimentam-se.

Vamos criar uma classe `Mover`, para abstrair esse comportamento comum em um único lugar. `Player` e `Enemy` herdarão de `Mover`. E embora as armas não se movam, elas herdam de `Mover` também, porque precisam de algumas de suas propriedades e métodos. `Mover` possui um método `Move()` para se mover e uma propriedade `Location` apenas-leitura que o formulário pode usar para posicionar uma subclasse de `Mover`.

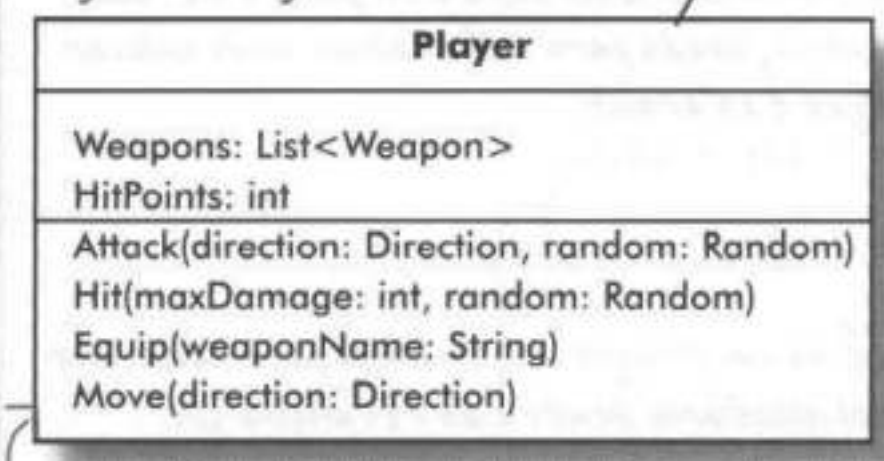
Mover é abstrata, então não pode ser instanciada. Você só vai instanciar Player e Enemy, que herdam dela.

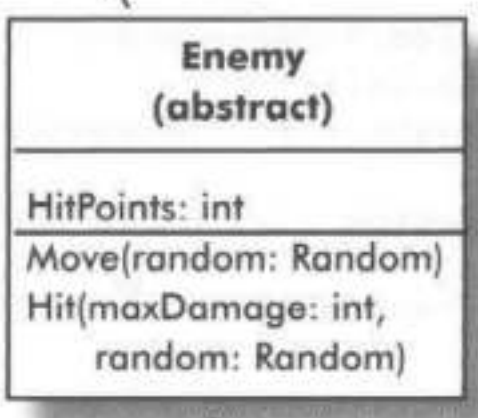


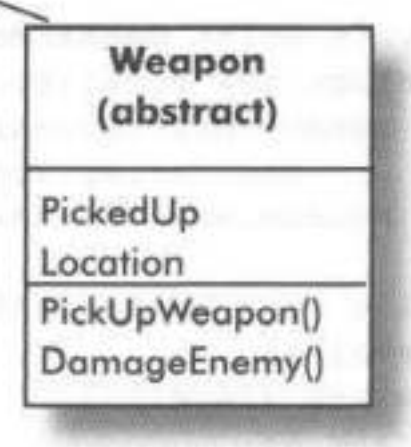
NearbyO (próximo, a curta distância) pega um ponto e calcula se ele está a uma certa distância do objeto.

Move recebe uma direção, assim como os limites da masmorra, e calcula onde seria o ponto final desse movimento.

Player e Enemy herdam de Mover.







A classe Player sobrepõe o método MoveO.

Você pode chamar NearbyO e MoveO tanto em Enemy quanto em Player agora.

Enemies não tem um método AttackO porque seu ataque foi incluído em MoveO.

Acrescente uma enum Direction

A classe `Move`, assim como muitas outras classes, precisa de uma enum `Direction`. Crie essa enum e dê a ela quatro valores enumerados: `Up`, `Down`, `Left` e `Right`.

O código fonte da classe Mover

Aqui está o código para Mover:

Já que propriedades protegidas ficam disponíveis apenas para subclasses, o objeto formulário não consegue atribuir a posição... apenas lê-lo através do método get público que definimos.

```
public abstract class Mover {
  private const int MoveInterval = 10;
  protected Point location;
  public Point Location { get { return location; } }
  protected Game game;
  public Mover(Game game, Point location) {
    this.game = game;
    this.location = location;
  }
  public bool Nearby(Point locationToCheck, int distance) {
    if (Math.Abs(location.X - locationToCheck.X) < distance &&
        (Math.Abs(location.Y - locationToCheck.Y) < distance)) {
      return true;
    } else {
      return false;
    }
  }
  public Point Move(Direction direction, Rectangle boundaries) {
    Point newLocation = location;
    switch (direction) {
      case Direction.Up:
        if (newLocation.Y - MoveInterval >= boundaries.Top)
          newLocation.Y -= MoveInterval;
        break;
      case Direction.Down:
        if (newLocation.Y + MoveInterval <= boundaries.Bottom)
          newLocation.Y += MoveInterval;
        break;
      case Direction.Left:
        if (newLocation.X - MoveInterval >= boundaries.Left)
          newLocation.X -= MoveInterval;
        break;
      case Direction.Right:
        if (newLocation.X + MoveInterval <= boundaries.Right)
          newLocation.X += MoveInterval;
        break;
      default: break;
    }
    return newLocation;
  }
}
```

Instâncias de Mover recebem como parâmetro o objeto Game e uma posição atual.

O método Nearby checa a distância de um Point em relação à posição atual do objeto. Se eles estiverem a uma distância - definida por distance - de cada um, ele retorna true; se não, retorna false.

O método Move() tenta mover-se um passo em uma direção. Se ele conseguir, retorna o novo Point. Se chegar em um dos limites, retorna o Point original.

Se o local final está fora dos limites, a nova posição permanece a mesma do ponto de partida.

Finalmente, esse novo local é retornado (ainda pode ser o mesmo que o ponto de partida!)

A classe Player controla o jogador

Aqui vemos o início da classe Player. Comece com esse código no IDE e prepare-se para continuá-lo.

```
public class Player : Mover {
  private Weapon equippedWeapon;
  private int hitPoints;
  public int HitPoints { get { return hitPoints; } }
  private List<Weapon> inventory = new List<Weapon>();
  public List<string> Weapons {
    get {
      List<string> names = new List<string>();
      foreach (Weapon weapon in inventory)
        names.Add(weapon.Name);
      return names;
    }
  }
  public Player(Game game, Point location, Rectangle boundaries)
          : base(game, location) {
    hitPoints = 10;
  }
  public void Hit(int maxDamage, Random random) {
    hitPoints -= random.Next(1, maxDamage);
  }
  public void IncreaseHealth(int health, Random random) {
    hitPoints += random.Next(1, health);
  }
  public void Equip(string weaponName) {
    foreach (Weapon weapon in inventory)
      if (weapon.Name == weaponName)
        equippedWeapon = weapon;
    }
  }
}
```

Todas as propriedades de Player estão ocultas para o acesso direto.

O jogador pode guardar muitas armas no inventário, mas só pode usar uma de cada vez.

O construtor de player atribui 10 para o valor de seu hitPoints e chama o construtor da classe base.

Player herda de Mover, então este método passa Game e o local para aquela classe base.

Quando um inimigo ataca o jogador, ele causa uma quantidade de dano aleatória. E quando a poção aumenta os pontos do jogador, a quantidade aumentada também é aleatória.

O método Equip() diz ao jogador para usar uma de suas armas. O objeto Game chama esse método quando um dos ícones no inventário é clicado.

Um objeto Player só pode ter um objeto Weapon em uso por vez.

Embora as poções ajudem o jogador e não causem dano ao inimigo, ainda assim são consideradas armas por game. Desta forma, o inventário pode ser uma List<Weapon> e game pode apontar para uma com sua referência WeaponInRoom (arma na sala).

Escreva o método Move() para Player

Game chama o método `Move()` de `Player` para dizer ao jogador para mover-se em uma certa direção. `Move()` recebe como parâmetro uma direção para se mover (usando o enum `Direction` que você já devia ter adicionado). Aqui está o começo desse método:

Isso acontece quando um dos botões de movimento no formulário é clicado.

```
public void Move(Direction direction) {
  base.location = Move(direction, game.Boundaries);
  if (!game.WeaponInRoom.PickedUp) {
    // see if the weapon is nearby, and possibly pick it up
  }
}
```

Move está na classe base Mover.

Você precisa completar o restante deste método. Cheque e veja se a arma está próxima ao jogador (num raio de uma única unidade de distância). Se estiver, pegue a arma e adicione-a ao inventário.

Quando o jogador pega uma arma, ela precisa desaparecer da masmorra e aparecer no inventário.

Se a arma for a única que o jogador possuir, vá em frente e prepare-a imediatamente. Assim, o jogador pode usá-la na próxima jogada.

Weapon e o formulário serão responsáveis por deixar a Picturebox da arma invisível quando o jogador a pegar .. isso não é função da classe Player.

Acrescente um método Attack() também

O próximo é o método `Attack()`. Ele é chamado quando um dos botões de ataque do formulário é clicado e é executado numa direção (novamente, uma enum `Direction`). Aqui está a assinatura do método:

```
public void Attack(Direction direction, Random random) {
  // Your code goes here
}
```

Se o jogador não tiver uma arma preparada, esse método não fará nada. Se o jogador tiver uma, ele deve chamar o método Attack da arma.

Mas poções são um caso especial. Se uma poção for usada, remova-a do inventário, já que ela não está mais disponível.

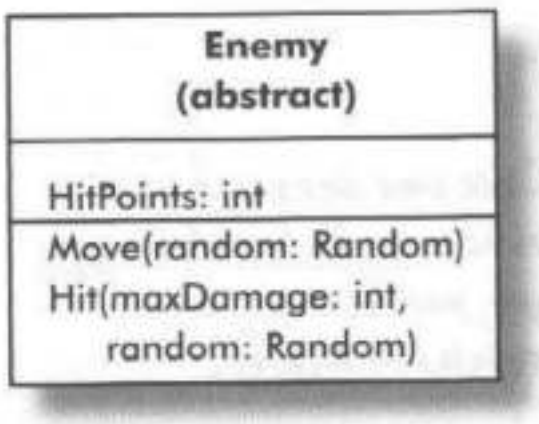
Todas as armas têm um método Attack() que possui um enum Direction e um objeto Random. O Attack() de player verificará qual arma está preparada e chamará o Attack() da arma.

Se a arma for uma poção, Attack() remove-a do inventário após o jogador bebê-la.

Potions implementará uma interface IPotion (mais sobre isso daqui a pouco), então você pode usar a palavra is para ver se Weapon é uma implementação de IPotion.

Morcegos, fantasmas e zumbis herdam da classe Enemy

Daremos a você outra classe abstrata útil: `Enemy`. Cada tipo diferente de inimigo possui sua própria classe que herda da classe `Enemy`. Os diferentes tipos de inimigos movem-se de maneiras diferentes. Assim, a classe abstrata `Enemy` deixa o método `Move` como um método abstrato – as três classes de inimigos precisarão implementá-lo diferentemente, dependendo de como eles se movem.




```
public abstract class Enemy : Mover {
  private const int NearPlayerDistance = 25;
  private int hitPoints;
  public int HitPoints { get { return hitPoints; } }
  public bool Dead { get {
      if (hitPoints <= 0) return true;
      else return false;
    }
  }
  public Enemy(Game game, Point location, Rectangle boundaries, int
hitPoints)
    : base(game, location) { this.hitPoints = hitPoints; }
  public abstract void Move(Random random);
  public void Hit(int maxDamage, Random random) {
    hitPoints -= random.Next(1, maxDamage);
  }
  protected bool NearPlayer() {
    return (Nearby(game.PlayerLocation,
                   NearPlayerDistance));
  }
  protected Direction FindPlayerDirection(Point playerLocation) {
    Direction directionToMove;
      if (playerLocation.X > location.X + 10)
        directionToMove = Direction.Right;
    else if (playerLocation.X < location.X - 10)
        directionToMove = Direction.Left;
    else if (playerLocation.Y < location.Y - 10)
        directionToMove = Direction.Up;
    else
        directionToMove = Direction.Down;
    return directionToMove;
  }
}
```

O formulário pode usar essa propriedade apenas-leitura para ver se o inimigo deve ser visível na masmorra do jogo.

Quando o jogador ataca um inimigo, ele chama o método Hit() (golpe) de enemy, que subtrai um número aleatório dos pontos de vida.

Cada subclasse de Enemy implementa este método.

O método NearPlayer() usa o método estático Mover.Nearby() para descobrir se o inimigo está perto do jogador.

Se você forneceu para FindPlayerDirection() a localização do jogador, ele usará o campo location da classe base para descobrir onde o jogador está em relação ao inimigo e retornará uma enum Direction que informa em qual direção o inimigo precisa se mover para se aproximar do jogador.

Escreva as diferentes subclasses de Enemy

As três diferentes subclasses de `Enemy` (inimigo) são bem fáceis. Cada inimigo possui um número diferente de pontos de vida no começo, eles movem-se de forma diferente e provocam uma quantidade de dano diferente quando atacam. Você precisará fazer cada um passar um parâmetro `startingHitPoints` diferente para o construtor base de Enemy e terá de escrever métodos `Move()` diferentes para cada subclasse.

Aqui vemos um exemplo de como uma dessas classes pode ficar:

```
public class Bat : Enemy {
  public Bat(Game game, Point location, Rectangle boundaries)
      : base(game, location, boundaries, 6)
  { }

  public override void Move(Random random) {
      // Seu código vai aqui
  }
}
```

O morcego começa com seis pontos de vida, logo ele passa 6 ao construtor da classe base.

Provavelmente, você não precisará de nenhuma ação nesse construtor. A classe base cuida de tudo.

Cada uma dessas é uma subclasse da base Enemy, que, por sua vez, cria a subclasse Mover.

O morcego voa por aí um tanto aleatoriamente; ele usa Random para voar em uma direção aleatória na metade do tempo.

Quando o inimigo não tiver mais pontos de vida, o formulário não vai mais exibi-lo. Mas ele ainda consta na lista Enemies do jogo até que o jogador termine o nível.

Devemos garantir que o formulario verifique se um inimigo deve ser visível em todos os turnos.

O morcego começa com seis pontos de vida. Ele continua se aproximando do jogador e atacando, desde que tenha um ou mais pontos de vida. Quando ele se move, existe uma chance de 50% de seja na direção do jogador e 50% de seja numa direção aleatória. Depois que o morcego se moveu, ele checa se está perto do jogador – se estiver, então ele ataca com até dois pontos de dano.

O fantasma é mais difícil de derrotar que o morcego, mas assim como este, ele só vai mover-se e atacar se seus pontos de vida forem maiores que zero. Ele começa com oito pontos. Quando ele tem a oportunidade de se mover, existe uma chance em três de que ele vá na direção do jogador e duas chances em três de permanecer parado. Se estiver perto do jogador, ele ataca com até três pontos de dano.

O fantasma e o zumbi usam Random para fazê-los se mover mais lentamente que o jogador.

O zumbi é o inimigo mais difícil. Ele começa com 10 pontos de vida e só se move e ataca se seus pontos forem maiores que zero. Quando ele tem a oportunidade de se mover, há duas chances em três de que vá em direção ao jogador e uma em três de ficar parado. Se estiver perto do jogador, ele ataca com até quatro pontos de dano.

Weapon herda de Mover e cada tipo de arma herda de Weapon

Precisamos de uma classe base `Weapon` (arma), assim como tínhamos uma classe base `Enemy`. E cada arma tem uma posição, assim como uma propriedade, indicando se ela foi pega ou não. Aqui está a classe base `Weapon`:

Weapon herda de Mover porque ela usa seus métodos NearbyO e MoveO em DamageEnemyO.

Uma arma recolhida (picked up, literalmente pega) não deveria mais ser exibida... o formulário pode usar este acessador get para descobrir isso.

```
public abstract class Weapon : Mover {
  protected Game game;
  private bool pickedUp;
  public bool PickedUp { get { return pickedUp; } }
  private Point location;
  public Point Location { get { return location; } }
```

Toda arma possui uma posição na masmorra do jogo.

```
  public Weapon(Game game, Point location) {
    this.game = game;
    this.location = location;
    pickedUp = false;
  }
```

O construtor atribui valores para os campos game e location e iguala pickedUp a false (porque a arma ainda não foi pega).

A propriedade Name de cada arma retorna seu nome (Sword - espada, Mace - bastão, Bow - arco).

Cada classe de arma precisa implementar uma propriedade Name e um método Attack() que determina como essa arma ataca.

```
  public void PickUpWeapon() { pickedUp = true; }
  public abstract string Name { get; }
  public abstract void Attack(Direction direction, Random random);
  protected bool DamageEnemy(Direction direction, int radius,
                             int damage, Random random) {
    Point target = game.PlayerLocation;
    for (int distance = 0; distance < radius; distance++) {
      foreach (Enemy enemy in game.Enemies) {
        if (Nearby(enemy.Location, target, radius)) {
          enemy.Hit(damage, random);
          return true;
        }
      }
      target = Move(direction, target, game.Boundaries);
    }
    return false;
  }
}
```

Cada arma possui um alcance e padrão de ataque próprios e, por isso, elas implementam o método Attack() de forma diferente.

O método DamageEnemy() é chamado por Attack(). Ele tenta achar um inimigo em uma certa direção e raio. Se ele achar, chama o método Hit() do inimigo e retorna true. Se nenhum inimigo for encontrado, ele retorna false.

Armas diferentes atacam de formas diferentes

Cada subclasse de `Weapon` possui seu próprio nome e lógica de ataque. Seu trabalho é implementar essas classes. Aqui está a estrutura básica para uma subclasse de Weapon:

Cada subclasse representa uma das três armas: uma espada, arco ou bastão.

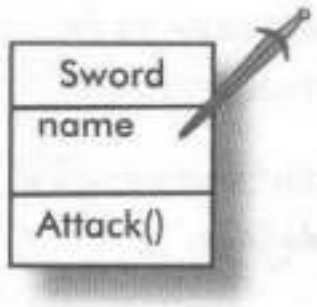
```
public class Sword : Weapon {
  public Sword(Game game, Point location)
    : base(game, location) { }
  public override string Name { get { return "Sword"; } }
  public override void Attack(Direction direction, Random random) {
    // Seu código vai aqui
  }
}
```

Cada subclasse conta com a classe base para fazer o trabalho de inicialização.

Aqui basicamente deixamos o nome de cada arma fixo no código.

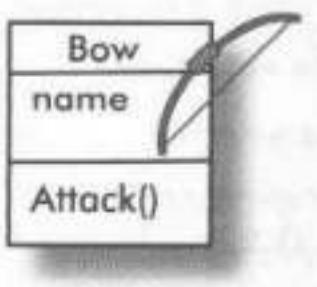
O objeto Game passará a direção na qual o ataque deve ser feito.

O jogador pode usar as armas infinitas vezes – elas nunca são descartadas ou param de funcionar.

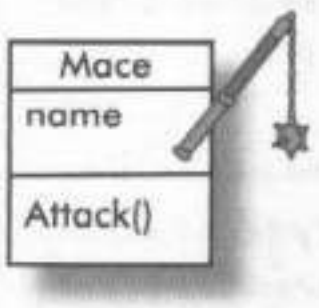


A espada (sword) é a primeira arma que o jogador pega. Ela tem um grande ângulo de ataque: primeiro atingirá um inimigo que está na direção do ataque – se não existir nenhum inimigo lá, ela continua o ataque no sentido horário da direção original e atinge qualquer inimigo ali; e se ainda assim não acertar, continua no sentido anti-horário da direção original do ataque. Ela tem um raio de 10 e causa três pontos de dano.

Pense cuidadosamente sobre isso... o que é a direita da esquerda? O que é a esquerda da direção para cima?



O arco (bow) possui um ângulo muito restrito de ataque, mas tem um alcance bem grande – o raio de ataque é 30, mas ele só causa um ponto de dano. Ao contrário da espada, que ataca em três direções (porque o jogador a movimenta num grande arco), quando o jogador usa o arco em uma direção, ele apenas atira nessa única direção.



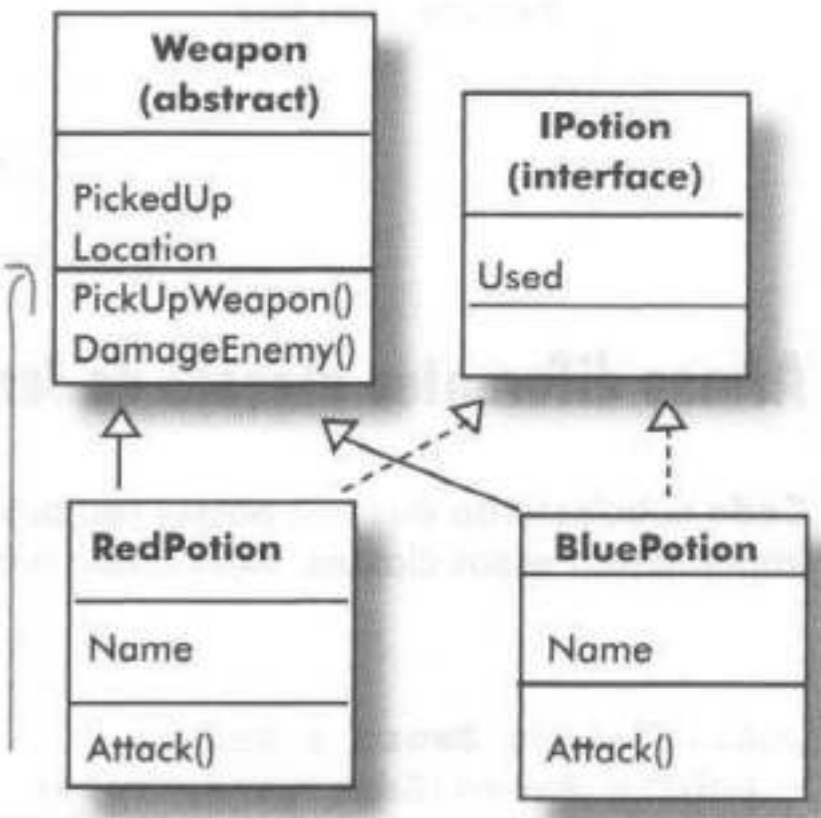
A bastão (mace) é a arma mais poderosa na masmorra. Não importa em qual direção o jogador atacar com ela – já que ela faz um circulo completo, atacará qualquer inimigo num raio de 20 e causará até seis pontos de dano.

As diferentes armas chamarão DamageEnemy() de muitas formas. A bastão ataca em todas as direções. Assim, se o jogador estiver atacando para a direita, ele chamará DamageEnemy(Direction.Right, 20, 6, random). Se não acertar nenhum inimigo, atacara para frente. Se não houver nenhum inimigo lá, tentará à esquera e então para trás – um movimento de círculo completo.

Poções implementam a interface IPotion

Existem duas poções, uma azul e uma vermelha, que melhoram a saúde o jogador. Elas funcionam como as armas – o jogador as pega na masmorra, colocam-nas em posição no inventário e **usam-nas clicando em um dos botões de ataque**. Então faz sentido elas herdarem da classe abstrata `Weapon`.

Porém poções funcionam um pouco diferente. Assim você precisará adicionar uma interface `IPotion` para que possam ter comportamento extra: aumentar a vida do jogador. Essa interface é realmente simples. Poções só precisam adicionar uma propriedade apenas-leitura chamada `Used` que retorna `false` se o jogador não tiver usado a poção e `true` se tiver. O formulário a usará para determinar se deve ou não mostrar a poção no inventário.




```
public interface IPotion {
  bool Used { get; }
}
```

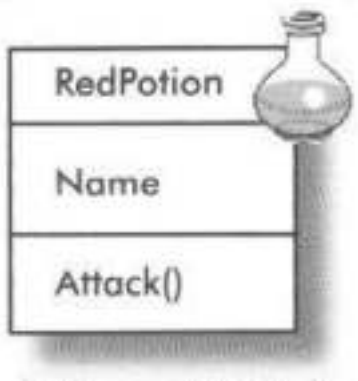
IPotion faz com que as poções possam ser usadas só uma vez. Também é possível descobrir se uma Weapon é uma poção com if (weapon is IPotion), graças à interface.

As poções herdam da classe Weapon porque elas são usadas como armas – o jogador clica na poção no inventário para prepará-la e então clica em qualquer botão de atague para usá-la.

Você deve ser capaz de escrever essas classes usando esse diagrama de classes e a informação que segue.

A propriedade Name da classe `BluePotion` (poção azul) deve retornar a seqüência de caracteres "Blue Potion". Seu método `Attack()` será chamado quando o jogador usar a poção azul – ela deve aumentar até cinco pontos na vida do jogador chamando o método `IncreasePlayerHealth()` (aumentar saúde do jogador). Depois que o jogador usa a poção, o método `Used()` (usada) dela deve retornar `true`.

Se o jogador pegar uma poção azul no nível 2, usá-la, e pegar outra no nível 4, o jogo criará duas instâncias de BluePotion diferentes.



A classe `RedPotion` é muito parecida com `BluePotion`, exceto que sua propriedade Name retorna a seqüência de caracteres "Red Potion" e seu método `Attack()` aumenta a vida do jogador em até 10 pontos.

O formulário junta tudo

Existe uma instância do objeto `Game` e ela encontra-se em seu formulário como um campo privado. Ela é criada no evento `Load` do formulário e os vários tratadores de evento no formulário usam os campos e métodos do objeto `Game` para manter o jogo em andamento.

Tudo começa com o tratador de evento `Load` do formulário, que passa um `Rectangle` para `Game`, definindo os limites da área de jogo da masmorra. Aqui está uma parte do código do formulário para ajudar um pouco:

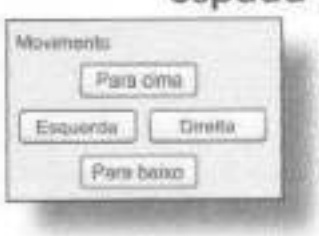
```
private Game game;
private Random random = new Random();
private void Form1_Load(object sender,
                        EventArgs e) {
  game = new Game(new Rectangle(78, 57, 420, 155));
  game.NewLevel(random);
  UpdateCharacters();
}
```

Estes são os limites da masmorra na imagem de fundo que você vai baixar e adicionar ao formulário.

Usando um Rectangle
Você encontrará muitos Rectangles sempre que trabalhar com formulários. Você pode criar um passando a ele valores X, Y, Width (largura) e Height (altura), ou dois Points (para cantos opostos). Uma vez que você tenha uma instância de retângulo, pode também acessar seus valores Left, Right, Top e Bottom, assim como seus valores X, Y, Width e Height.

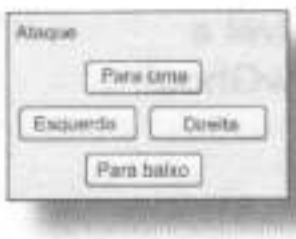
*Lembre-se de dar duplo clique em **cada** PictureBox para que o IDE adicione um tratador de evento separado para cada uma delas.*

O formulário possui um tratador de evento separado para cada evento Click das caixas de imagem. Quando o jogador clicar na espada, primeiro verifica-se se ela está no inventário usando o método `CheckPlayerInventory()` (verificar inventário do jogador) do objeto `Game`. Se o jogador estiver de posse da espada, o formulário chama `game.Equip()` para que o jogador a segure. Ele então configura cada propriedade `BorderStyle` de PictureBox para desenhar uma caixa em volta da espada e certifica-se de que nenhum dos outros ícones têm uma caixa em volta deles.



Há um tratador de evento para cada um dos quatro botões direcionais. Eles são bem simples. Primeiro, o botão chama `game.Move()` com o valor de `Direction` apropriado, e então chama o método `UpdateCharacters()` (atualizar personagens) do formulário.

Certifique-se de alterar os botões de volta quando o jogador segurar a espada, o arco ou a bastão.



Os quatro tratadores de evento dos botões de ataque também são muito simples. Cada botão chama `game.Attack()` e então o método `UpdateCharacters()` do formulário. Se o jogador prepara uma poção, ela ainda é usada da mesma forma – chamando `game.Attack()` – mas elas não têm direção. Então deixe os botões Esquerda, Direita e Para trás invisíveis quando o jogador preparar uma poção e altere o texto no botão Para frente para "Beber".

O método UpdateCharacters() do formulário posiciona as caixas de imagem

A última parte do jogo é o método `UpdateCharacters()` do formulário. Quando todos os objetos tenham se movido e atuado uns sobre os outros, o formulário atualiza tudo... então as armas que foram pegas têm a propriedade Visible de suas caixas de imagem igualadas a false, inimigos e jogadores são colocados em seus novos lugares (e os mortos ficam invisíveis) e o inventário é atualizado.

Isso é o que você tem que fazer:

1. **Atualize a posição e estatísticas do jogador**
   A primeira coisa a fazer é atualizar a localização da PictureBox do jogador e o rótulo que mostra seus pontos de vida. Você precisará de algumas variáveis para determinar se mostrará cada um dos vários inimigos.

```
public void UpdateCharacters() {
  Player.Location = game.PlayerLocation;
  playerHitPoints.Text =
    game.PlayerHitPoints.ToString();

  bool showBat = false;
  bool showGhost = false;
  bool showGhoul = false;
  int enemiesShown = 0;
  // aqui vai mais codigo...
```

A variável showBat terá true atribuído a ela se tornarmos a PictureBox do morcego visível. O mesmo vale para showGhost e showGhouli.

2. **Atualize a localização e pontos de vida de cada inimigo**
   Cada inimigo pode estar em uma nova posição e ter um valor diferente de pontos de vida. Você precisa atualizar isso para cada inimigo depois de atualizar a localização do jogador:

```
foreach (Enemy enemy in game.Enemies) {
  if (enemy is Bat) {
    bat.Location = enemy.Location;
    batHitPoints.Text = enemy.HitPoints.ToString();
    if (enemy.HitPoints > 0) {
      showBat = true;
      enemiesShown++;
    }
  }
  // etc...
```

Este aqui vai logo após o código acima.

Isto afetará a visibilidade dos controles da PictureBox do inimigo daqui a pouco.

Você vai precisar de mais dois comandos if como este em seu laço foreach – um para o fantasma e um para o zumbi.

Depois que você iterou por todos os inimigos no nível, cheque a variável showBat. Se o morcego foi morto, então showBat ainda será false, logo deixe sua PictureBox invisível e apague seu rótulo de pontos de vida. Depois faça o mesmo para showGhost e showGhoul.

3 **Atualize as caixas de imagem de arma**
Declare uma variável `weaponControl` e use um grande comando `switch` para igualá-la à PictureBox que corresponde à arma na masmorra.

```
sword.Visible = false;
bow.Visible = false;
redPotion.Visible = false;
bluePotion.Visible = false;
mace.Visible = false;
Control weaponControl = null;
switch (game.WeaponInRoom.Name) {
    case "Sword":
      weaponControl = sword; break;
```

Certifique-se de que os nomes de seus controles correspondam a estes nomes. É fácil ter falhas difíceis de controlar se eles não corresponderem.

Você terá mais cláusulas case para cada tipo de arma.

O restante das cláusulas case devem atribuir à variável `weaponControl` o controle correto no formulário. Depois do `switch`, iguale `weaponControl.Visible` a `true` para mostrá-la.

4 **Altere a propriedade Visible de cada PictureBox dos ícones do inventário**
Use o método `CheckPlayerInventory()` do objeto Game para descobrir se os diversos ícones no inventário devem ser exibidos.

5 **Aqui está o restante do método**
A primeira coisa que você fará é atualizar a localização da PictureBox do jogador e o rótulo que mostra seus pontos de vida. Então você precisará de algumas variáveis para determinar se mostrará cada um dos vários inimigos.

```
weaponControl.Location = game.WeaponInRoom.Location;
if (game.WeaponInRoom.PickedUp) {
  weaponControl.Visible = false;
} else {
  weaponControl.Visible = true;
}
if (game.PlayerHitPoints <= 0) {
  MessageBox.Show("You died");
  Application.Exit();
}
if (enemiesShown < 1) {
  MessageBox.Show("You have defeated the enemies on this level");
  game.NewLevel(random);
  UpdateCharacters();
}
```

Todo nível tem uma arma. Se ela foi recolhida, precisamos deixar seu ícone invisível.

Application.Exit() imediatamente fecha o programa. Ele é parte de System.Windows.Forms, então você vai precisar do comando using apropriado se quiser usá-lo dentro de um formulário.

Se não houver mais inimigos no nível, o jogador derrotou todos eles e é hora de ir para o próximo estágio.

A diversão está apenas começando!

Sete níveis, três inimigos... este é um jogo bem legal. Mas você ainda pode melhorá-lo. Aqui temos algumas idéias para você começar...

Deixe os inimigos mais inteligentes
Você consegue descobrir como alterar os métodos Move() dos inimigos para que seja mais difíceis derrotá-los? Veja se consegue transformar suas constantes em propriedades e deixe uma forma de alterá-las usando o jogo.

Acrescente mais níveis
O jogo não tem que acabar depois de sete níveis. Veja se consegue colocar mais... você consegue descobrir como fazer o jogo durar indefinidamente? Se o jogador ganhar, faça um final legal animado com fantasmas e morcegos dançando! E o jogo termina abruptamente se o jogador morre. Veja se você consegue pensar em mais algum final legal para o usuário – talvez deixá-lo recomeçar o jogo ou jogar seu último nível de novo.

Acrescente tipos diferentes de inimigos
Você não precisa limitar a masmorra a zumbis, fantasmas e morcegos. Veja se consegue adicionar mais inimigos ao jogo.

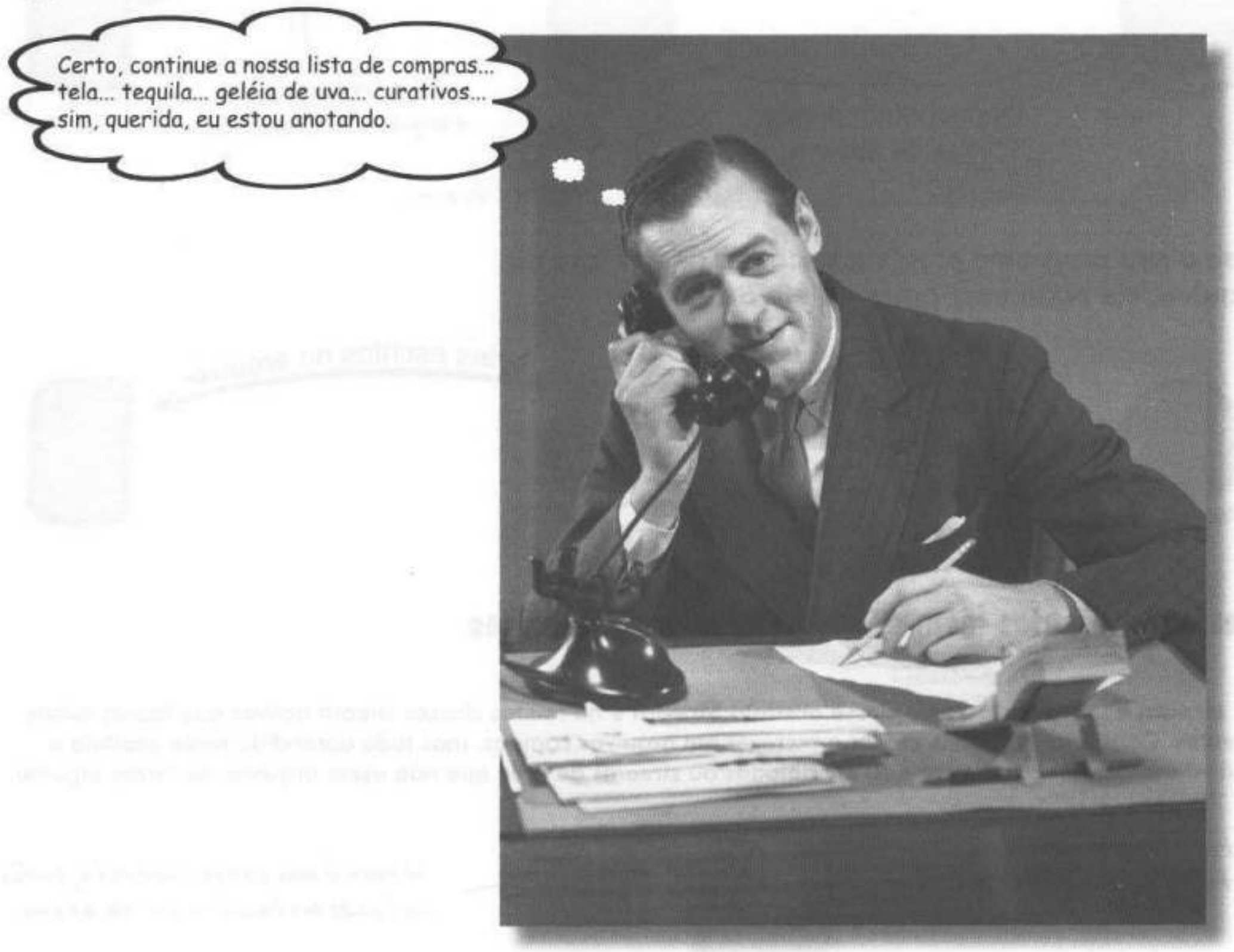
Acrescente mais armas
O jogador definitivamente precisará de mais ajuda para derrotar qualquer novo inimigo acrescentado. Pense em novas maneiras a respeito de como armas possam atacar, ou coisas diferentes que poções possam fazer. Tire vantagem do fato de Weapon ser uma subclasse de Mover – crie armas mágicas que o jogador deva perseguir!

Acrescente mais gráficos
Você pode ir até www.altabooks.com.br para encontrar mais arquivos de gráficos para inimigos, armas e outras imagens adicionais para ajudá-lo a ativar sua imaginação.

Essa é a sua chance de se exibir! Você inventou uma nova versão legal do jogo? Junte-se ao fórum do Use a Cabeça. faça valer seus direitos e conte vantagem: www.altabooks.com.br

9 Lendo e escrevendo arquivos

Salve a matriz de bytes, salve o mundo



Às vezes vale a pena ser um pouco persistente.

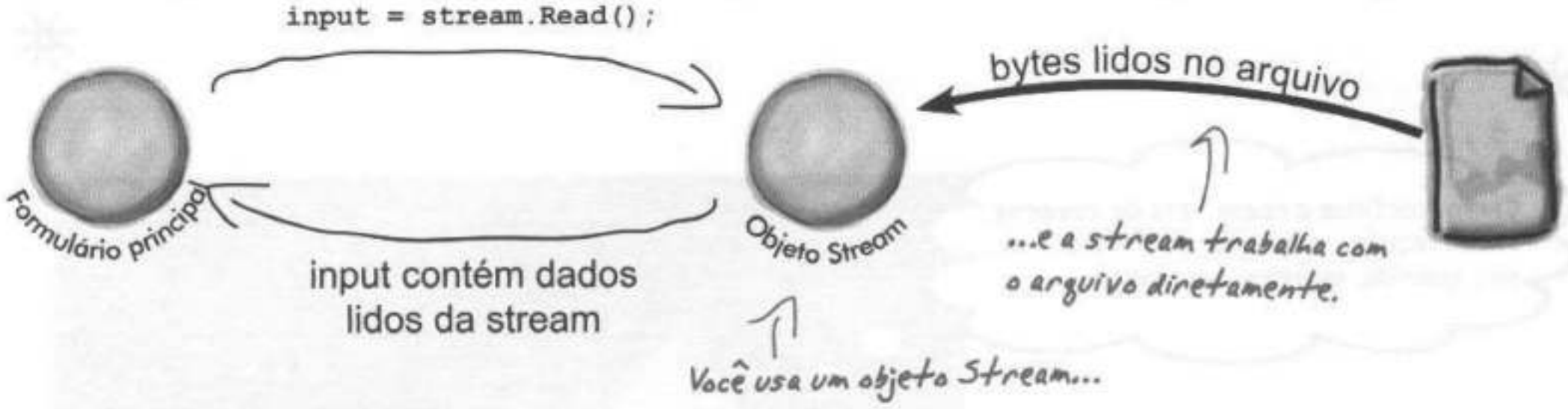
Até agora, todos os seus programas têm funcionado por muito pouco tempo. Eles são ativados, executam por um curto intervalo e desligam. Mas isso nem sempre é o bastante, principalmente quando você está lidando com informações importantes. Você precisa ser capaz de **salvar seu trabalho**. Neste capítulo, veremos como **escrever dados em um arquivo** e como **ler esta informação de volta** de um arquivo. Você aprenderá sobre as **classes stream** (fluxo, literalmente riacho) do .NET e também verá os mistérios do **hexadecimal** e do **binário**.

O C# usa streams para ler e escrever dados

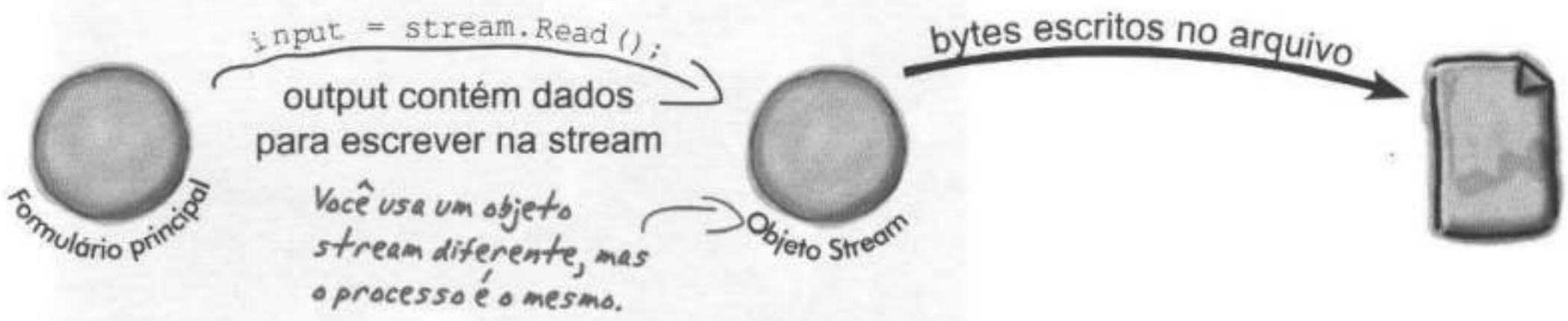
Uma stream é como o Framework .NET transfere dados para e do seu programa. Quando seu programa C# lê ou escreve em arquivos, conecta-se a outro computador em rede; ou quando faz qualquer coisa que **envie ou receba** bytes transferidos de um lugar para outro, ele está usando streams.

Sempre que quiser ler ou escrever dados em um arquivo, use um objeto Stream.

Digamos que você tem um programa simples – um formulário com um tratador de eventos que precisa ler dados de um arquivo. Você usará um objeto Stream para isto.

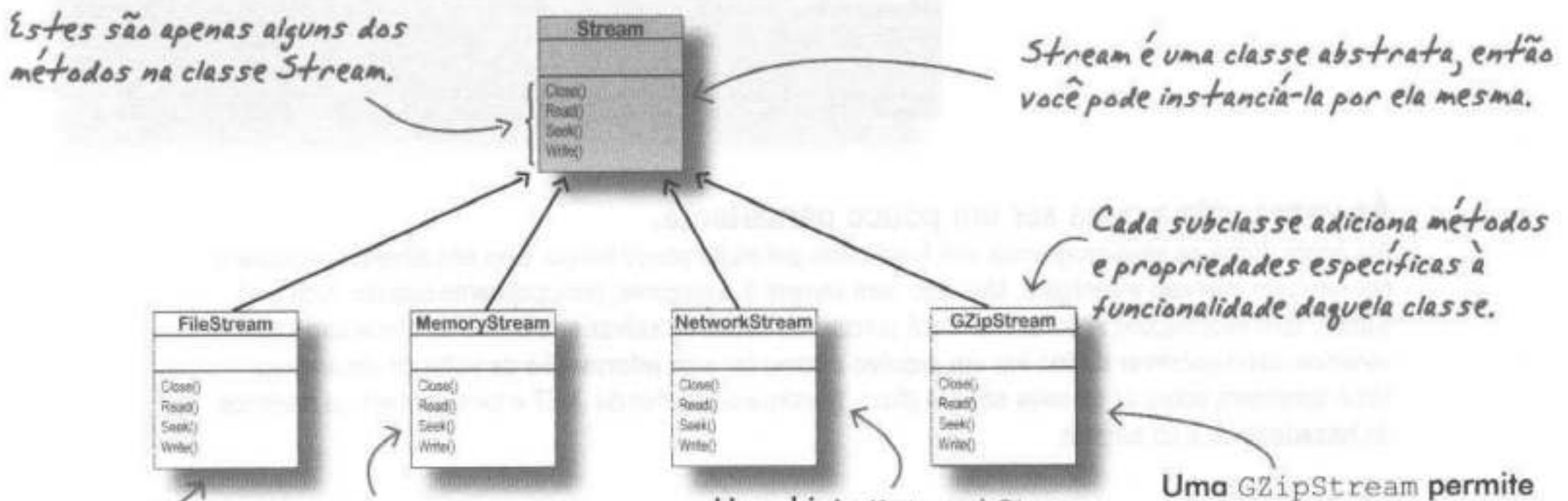


E se o seu programa precisar escrever dados fora do arquivo, ele pode usar outro objeto Stream.



Streams diferentes lêem e escrevem coisas diferentes

Toda **stream** é uma subclasse da classe abstrata **Stream** e há muitas classes stream nativas que fazem coisas diferentes. Concentraremo-nos em ler e escrever em arquivos comuns, mas tudo aprendido neste capítulo é aplicável a arquivos compactados ou encriptados ou streams de rede que não usem arquivos de forma alguma.



Coisas que você pode fazer com uma stream:

1. **Escrever.**
   Você pode escrever texto ou dados binários em uma stream através do método `Write()` (escrever).

2. **Ler dados.**
   Você pode usar o método `Read()` (ler) para recuperar dados de um arquivo, de uma rede, da memória, ou praticamente de qualquer outro lugar, usando uma stream.

3. **Alterar sua posição na stream.**
   A maioria das streams suporta um método `Seek()` (procurar) que permite encontrar uma posição dentro dela, permitindo inserir dados num local específico.

Stream permitem ler e escrever dados. Use o tipo certo de stream para os dados com os quais você está trabalhando.

Um FileStream escreve bytes em um arquivo

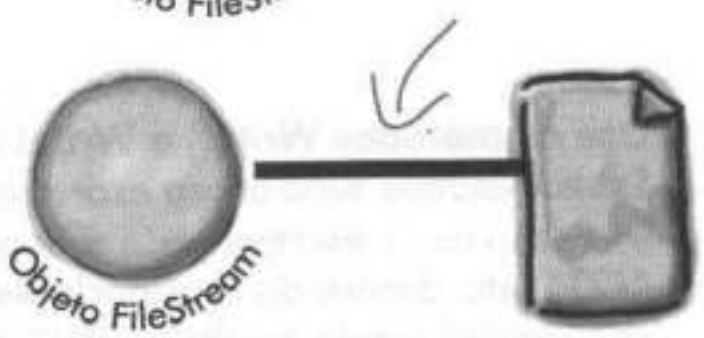
Quando seu programa precisa escrever algumas linhas de texto em um arquivo, muitas coisas devem acontecer:

1. Crie um novo objeto FileStream e mande-o escrever no arquivo.

Certifique-se de adicionar `using System.IO;` em qualquer programa que use streams.

Um FileStream só pode ser anexado a um arquivo por vez.

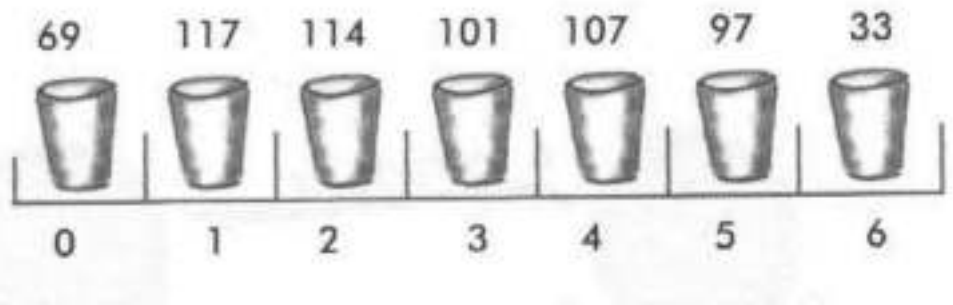
2. O FileStream associa-se a um arquivo.



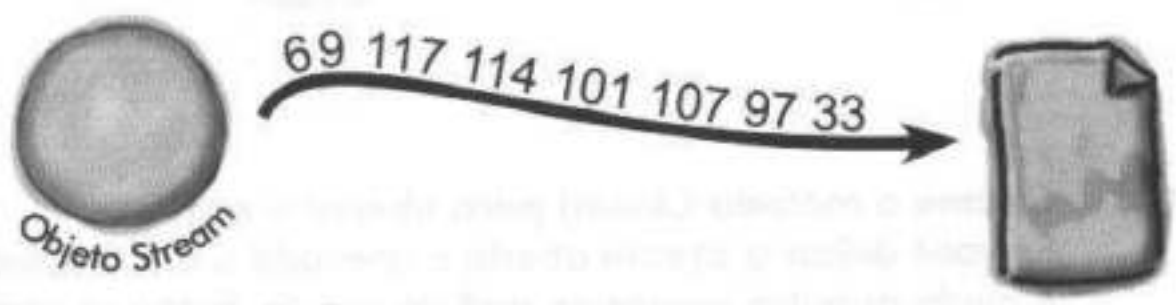
3. Streams escrevem bytes em arquivos. Logo, você precisará converter a seqüência de caracteres que deseja escrever numa matriz de `bytes`.

Isso se chama codificação (encoding). Falaremos sobre isso mais tarde...

Eureka!

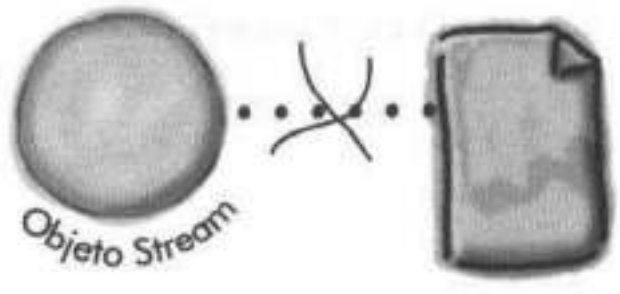


4. Chame o método Write() de stream e passe a matriz de `bytes` para ele.



5. Feche o arquivo para que outros programas possam acessá-lo.

Esquecer de fechar uma stream é um grande problema. Se isso acontecer, o arquivo travará e outros programas não poderão usá-lo até você fechar sua stream.



Como escrever texto em um arquivo em três passos simples

O C# possui uma classe conveniente chamada **StreamWriter** (escritor de stream) que faz todas essas coisas num único passo fácil. Tudo a fazer é criar um novo objeto `StreamWriter` e dar-lhe um nome de arquivo. Ele **automaticamente** cria um `FileStream` e conecta-o ao arquivo. Então você pode usar os métodos `Write()` e `WriteLine()` (escrever um linha) de StreamWriter para escrever tudo que quiser em um arquivo.

StreamWriter cria e gerencia um objeto FileStream para você automaticamente.

1 **Use o construtor de StreamWriter para abrir ou criar um arquivo**
Você pode passar um nome de arquivo ao construtor de `StreamWriter()`. Quando você faz isso, ele automaticamente abre o arquivo. `StreamWriter` também possui um construtor sobrecarregado que recebe um true se você quiser adicionar texto no final de um arquivo existente (ou apensar texto, append em inglês), ou false se você quiser deletar um arquivo existente e criar um novo com o mesmo nome.

```
StreamWriter writer = new StreamWriter(@"C:\newfiles\toaster oven.txt", true);
```

2 **Use os métodos Write() e WriteLine() para escrever em um arquivo**
Estes métodos funcionam exatamente como aqueles do `Console: Write()` escreve texto e `WriteLine()` escreve texto e acrescenta uma quebra de linha no final. Se você incluir "{0}", "{1}", "{2}", etc. dentro da seqüência de caracteres que está escrevendo, os métodos incluem parâmetros nas seqüências sendo escritas: "{0}" é substituído pelo primeiro parâmetro após a seqüência sendo escrita, "{1}" é substituído pelo segundo, etc.

```
writer.WriteLine("The {0} is set to {1} degrees.", appliance, temp.ToString());
```

3 **Chame o método Close() para liberar o arquivo**
Se você deixar a stream aberta e anexada a um arquivo, ela deixará o arquivo aberto travado e nenhum outro programa poderá usá-lo. Então se certifique de sempre fechar seus arquivos!

```
writer.Close();
```

O Trapaceiro prepara outro plano diabólico

Os cidadãos da vila Objeto vivem a muito tempo com medo do Trapaceiro. Agora ele está usando um `StreamWriter` para implementar outro plano maligno. Vamos ver o que está acontecendo:

Esta linha cria o objeto StreamWriter e o informa onde o arquivo estará.

O caminho começa com um sinal @ para que StreamWriter não interprete \ como o início de uma seqüência de escape.

```
StreamWriter sw = new StreamWriter(@"c:\plano_secreto.txt");
sw.WriteLine("Como eu vou derrotar o Capitão Maravilhoso");
sw.WriteLine("Outro plano secreto genial feito pelo Trapaceiro");
sw.Write("Eu criarei um exército de clones e ");
sw.WriteLine("fazê-los atacar os cidadãos da vila Objeto.");
string location = "o shopping";
for (int number = 0; number <= 6; number++){
      sw.WriteLine("Clone #{0} atacará {1}", number, location);
      if (location == "o shopping") { location = "o centro da cidade"; }
      else { location = "o shopping"; }
}
sw.Close();
```

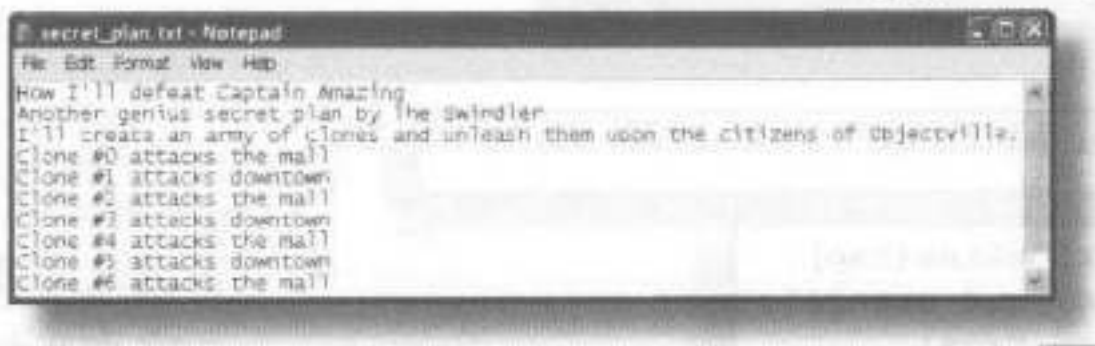
WriteLine() adiciona um caractere de nova linha depois de escrever. Write() só envia texto, sem encerrar a linha no fim.

Close() libera qualquer conexão para o arquivo e quaisquer recursos que StreamWriter esteja usando. O texto não é escrito se você não fechar a stream.

Você consegue descobrir o que está acontecendo com a variável location neste código?

Você pode usar {} no texto para passar variáveis à string sendo escrita. {0} é substituído pelo primeiro parametro depois da seqüência, {1} pelo segundo, e assim por diante.

Isto é o que o código acima produz.



Ímãs de Geladeira de StreamWriter

Suponha que você tem o código para button1_Click() exibido abaixo. Sua tarefa é usar os ímãs para construir o código para a classe Flobbo para que, quando o tratador de evento for chamado, ele produza o resultado mostrado no final da página. Boa sorte!

```
private void button1_Click(object sender, EventArgs e) {
  Flobbo f = new Flobbo("blue yellow");
  StreamWriter sw = f.Snobbo();
  f.Blobbo(f.Blobbo(f.Blobbo(sw), sw), sw);
}
```

```
sw.WriteLine(Zap);
Zap = "red orange";
return true;
```
}

```
sw.WriteLine(Zap);
sw.Close();
return false;
```

```
public bool Blobbo
    (bool Already, StreamWriter sw) {
```

```
public bool Blobbo(StreamWriter sw) {
```

```
sw.WriteLine(Zap);
Zap = "green purple";
return false;
```
}

```
return new
StreamWriter("macaw.txt");
```
} } }

```
private string Zap;
public Flobbo(string Zap) {
    this.Zap = Zap;
}
```

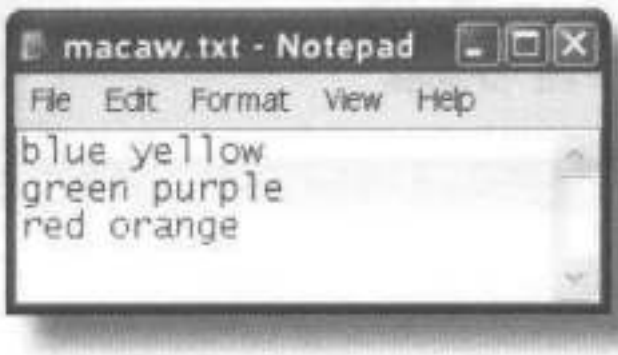
```
if (Already) {
```

```
public class Flobbo {
```

```
} else {
```

```
public StreamWriter Snobbo() {
```

Resultado:



Solução dos Ímãs e Geladeira de StreamWriter

Sua tarefa foi construir a classe Flobbo com os ímãs para conseguir o resultado esperado.

```
private void button1_Click(object sender, EventArgs e) {
  Flobbo f = new Flobbo("blue yellow");
  StreamWriter sw = f.Snobbo();
  f.Blobbo(f.Blobbo(f.Blobbo(sw), sw), sw);
}
```

```
public class Flobbo {
```

```
private string Zap;
public Flobbo(string Zap) {
    this.Zap = Zap;
}
```

```
public StreamWriter Snobbo() {
```

```
return new
StreamWriter("macaw.txt");
```

```
}
```

```
public bool Blobbo(StreamWriter sw) {
```

```
sw.WriteLine(Zap);
Zap = "red orange";
return true;
```

```
}
```

O método Blobbo() é sobrecarregado - ele tem duas declarações com parâmetros diferentes.

```
public bool Blobbo
    (bool Already, StreamWriter sw) {
```

```
if (Already) {
```

```
sw.WriteLine(Zap);
sw.Close();
return false;
```

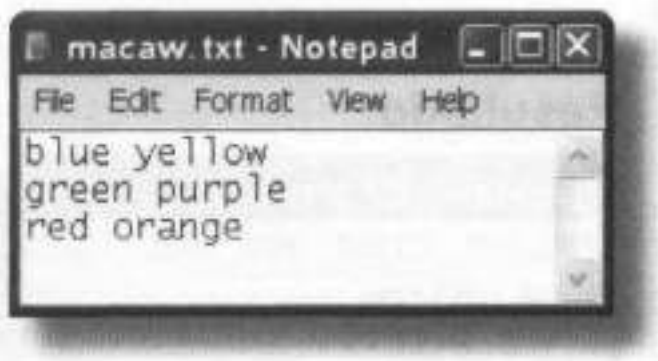
Certifique-se de fechar os arquivos quando terminar de usá-los.

```
} else {
```

```
sw.WriteLine(Zap);
Zap = "green purple";
return false;
```

```
}
}
}
```

Resultado:



Ler e escrever implica em dois objetos

Vamos ler os planos secretos de Swindler com outra stream, uma `StreamReader` (stream leitora). Ela funciona exatamente como uma `StreamWriter`, mas, em vez de escrever num arquivo, informa-se a ela o nome do arquivo que deve ser lido em seu construtor. O método `ReadLine()` (ler uma linha) retorna uma seqüência de caracteres com a próxima linha do arquivo. Você pode escrever um laço que leia linhas até que seu campo `EndOfStream` (fim da stream) seja true – é quando ele não tem mais linhas para ler:

```
StreamReader reader =
 new StreamReader(@"d:\plano_secreto.txt");
StreamWriter writer =
 new StreamWriter(@"e:\emailParaCapMaravilhoso.txt");
```

Passe o arquivo que você quer ler para o construtor de StreamReader.

Este programa usa um StreamReader para ler o plano do Trapaceiro e um StreamWriter para escrever um arquivo que será enviado por e-mail ao Capitão Maravilhoso

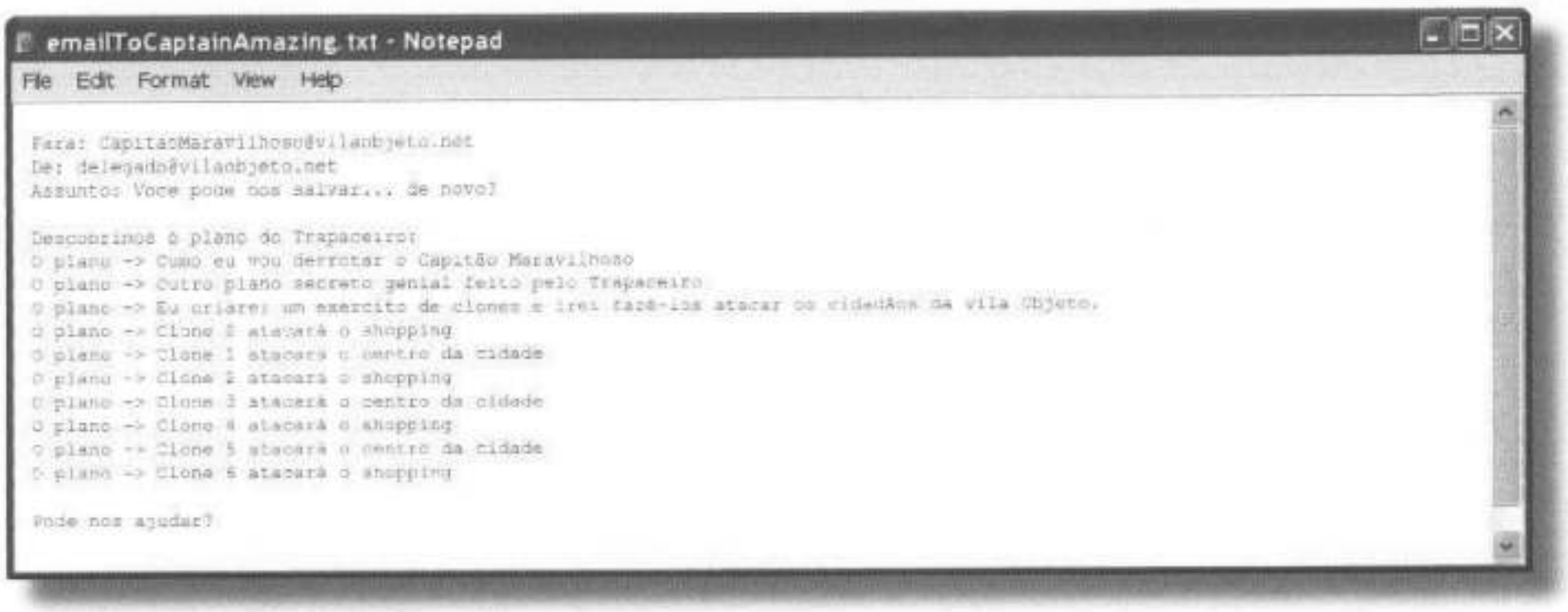
```
writer.WriteLine("Para: CapitaoMaravilhoso@vilaobjeto.net");
writer.WriteLine("De: delegado@vilaobjeto.net");
writer.WriteLine("Assunto: Voce pode nos salvar... de novo?");
writer.WriteLine();
writer.WriteLine("Descobrimos o plano do Trapaceiro:");
while (!reader.EndOfStream) {
      string lineFromThePlan = reader.ReadLine();
      writer.WriteLine("O plano -> " + lineFromThePlan);
}
writer.WriteLine();
writer.WriteLine("Pode nos ajudar?");
writer.Close();
reader.Close();
```

Um método WriteLine() vazio escreve uma linha em branco.

EndOfStream é a propriedade que informa se não há mais dados restando para ler no arquivo.

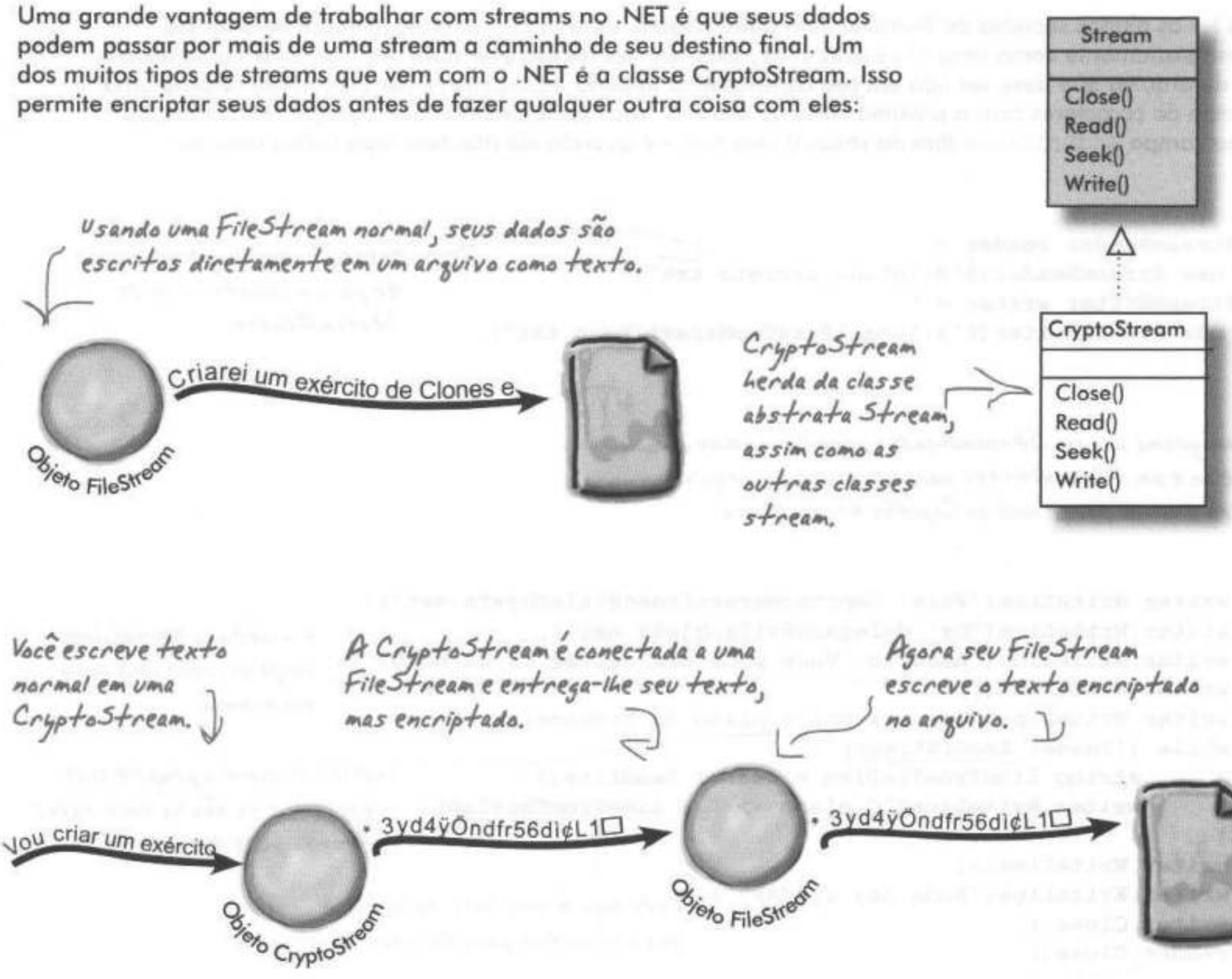
Este laço lê uma linha do leitor e usa o escritor para escrevê-la.

Certifique-se de fechar toda stream que abrir, mesmo se você estiver só lendo um arquivo.



Os dados podem passar por mais de uma stream

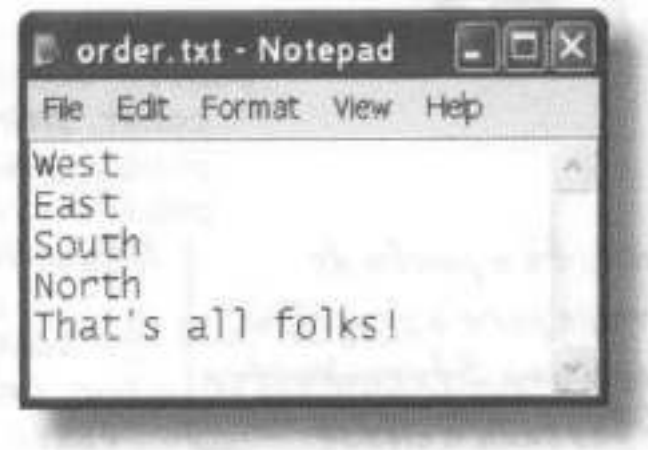
Uma grande vantagem de trabalhar com streams no .NET é que seus dados podem passar por mais de uma stream a caminho de seu destino final. Um dos muitos tipos de streams que vem com o .NET é a classe CryptoStream. Isso permite encriptar seus dados antes de fazer qualquer outra coisa com eles:



Voce pode ENCADEAR streams. Uma stream pode escrever em outra, que pode escrever em outra... freqüentemente terminando numa stream de rede ou de arquivo.

Jogos de Verão

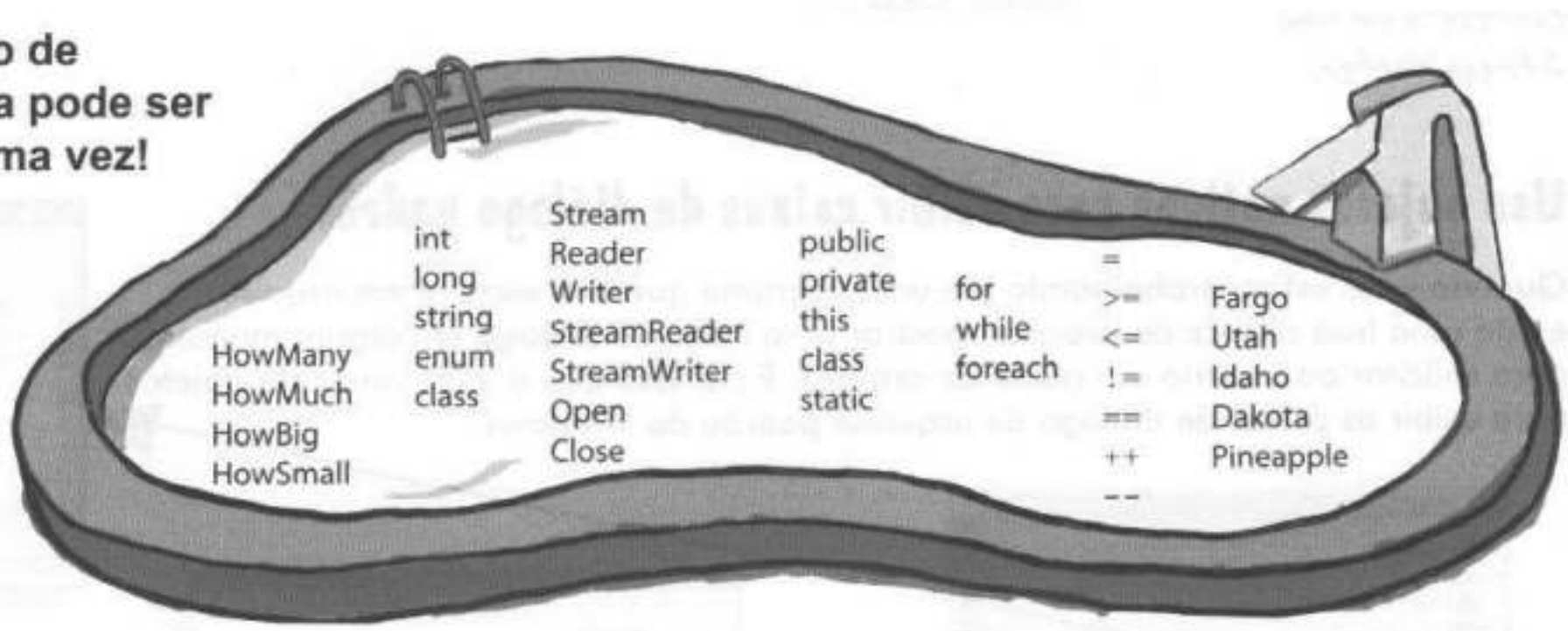
Seu trabalho é pegar trechos de código da piscina e colocá-los nas lacunas no programa. Você pode usar o mesmo trecho mais de uma vez e não precisa usar todos eles. Seu objetivo é fazer o programa produzir o resultado mostrado à direita.




```
public class Pineapple {
  const _______ d = "delivery.txt";
  public _____ _______
      { North, South, East, West, Flamingo }
  public static void Main() {
   ___________ o = new _____________("order.txt");
    Pizza pz = new Pizza(new ___________(d, true));
    pz._________(Fargo.Flamingo);
    for (_____ w = 3; w >= 0; w--) {
      Pizza i = new Pizza
        (new _____________(d, false));
      i.Idaho((Fargo)w);
      Party p = new Party(new ___________(d));
      p._____________(o);
    }
    o._____________("That's all folks!");
    o._____________();
  }
}
```

```
public class Pizza {
  private _____________ _______;
  public Pizza(_____________ _______) {
   _______.Writer = Writer;
  }
  public void _______(_______.Fargo f) {
    Writer._________(f.ToString());
    Writer._________();
  }
}
public class Party {
  private _____________ Reader;
  public Party(_____________ Reader) {
   ___________.Reader = Reader;
  }
  public void HowMuch(_____________ q) {
    q._________(Reader._________());
    Reader._________();
  }
}
```

Nota: cada trecho de código na piscina pode ser usado mais de uma vez!



Solução dos Jogos de Verão

Essa enum é usada com seu método ToString() para mostrar grande parte do resultado.

```
public class Pineapple {
  const string d = "delivery.txt";
  public enum Fargo { North, South, East, West, Flamingo }
  public static void Main() {
    StreamWriter o = new StreamWriter("order.txt");
    Pizza pz = new Pizza(new StreamWriter(d, true));
    pz.Idaho(Fargo.Flamingo);
    for (int w = 3; w >= 0; w--) {
      Pizza i = new Pizza(new StreamWriter(d, false));
      i.Idaho((Fargo)w);
      Party p = new Party(new StreamReader(d));
      p.HowMuch(o);
    }
    o.WriteLine("That's all folks!");
    o.Close();
  }
}
public class Pizza {
  private StreamWriter Writer;
  public Pizza(StreamWriter Writer) {
    this.Writer = Writer;
  }
  public void Idaho(Pineapple.Fargo f) {
    Writer.WriteLine(f.ToString());
    Writer.Close();
  }
}
public class Party {
  private StreamReader Reader;
  public Party(StreamReader Reader) {
    this.Reader = Reader;
  }
  public void HowMuch(StreamWriter q) {
    q.WriteLine(Reader.ReadLine());
    Reader.Close();
  }
}
```

Aqui está o ponto de entrada para o programa. Ele cria uma StreamWriter de é passada à classe Party (festa). Então ele itera pelos membros Fargo, passando cada um deles para o método Pizza.Idaho() mostrar.

A classe Pizza mantém uma StreamWriter como um campo privado e seu método Idaho() escreve enums Fargo no arquivo usando o método ToString().

A classe Party possui um campo StreamReader e seu método HowMuch() (quanto) lê uma linha de StreamReader e escreve-o em uma StreamWriter.

Use objetos nativos para exibir caixas de diálogo padrão

Quando você estiver trabalhando em um programa que lê e escreve em arquivos, existe uma boa chance de precisar mostrar uma caixa de diálogo em algum momento para solicitar ao usuário um nome de arquivo. É por isso que o .NET vem com objetos para exibir as caixas de diálogo de arquivos padrão do Windows.

O .NET possui caixas de diálogo nativas, como a OpenFileDialog, para selecionar um arquivo para abrir.

Esta é a caixa de diálogo FolderBrowseDialog.

ShowDialog() exibe uma caixa de diálogo

Vamos ver estes passos com você em um minuto.

Exibir uma caixa de diálogo é fácil. Tudo que você deve fazer é:

1. Crie uma instância do objeto caixa de diálogo. Você pode fazer isso com código usando `new`, ou você pode arrastá-lo para o seu formulário de dentro da caixa de ferramentas.

2. Altere as propriedades do objeto da caixa de diálogo. Algumas úteis incluem `Title` (que atribui o texto à barra de título), `InitialDirectory` (diretório inicial, que o informa qual diretório abrir primeiro) e `FileName` (nome de arquivo, para as caixas de diálogo abrir e salvar).

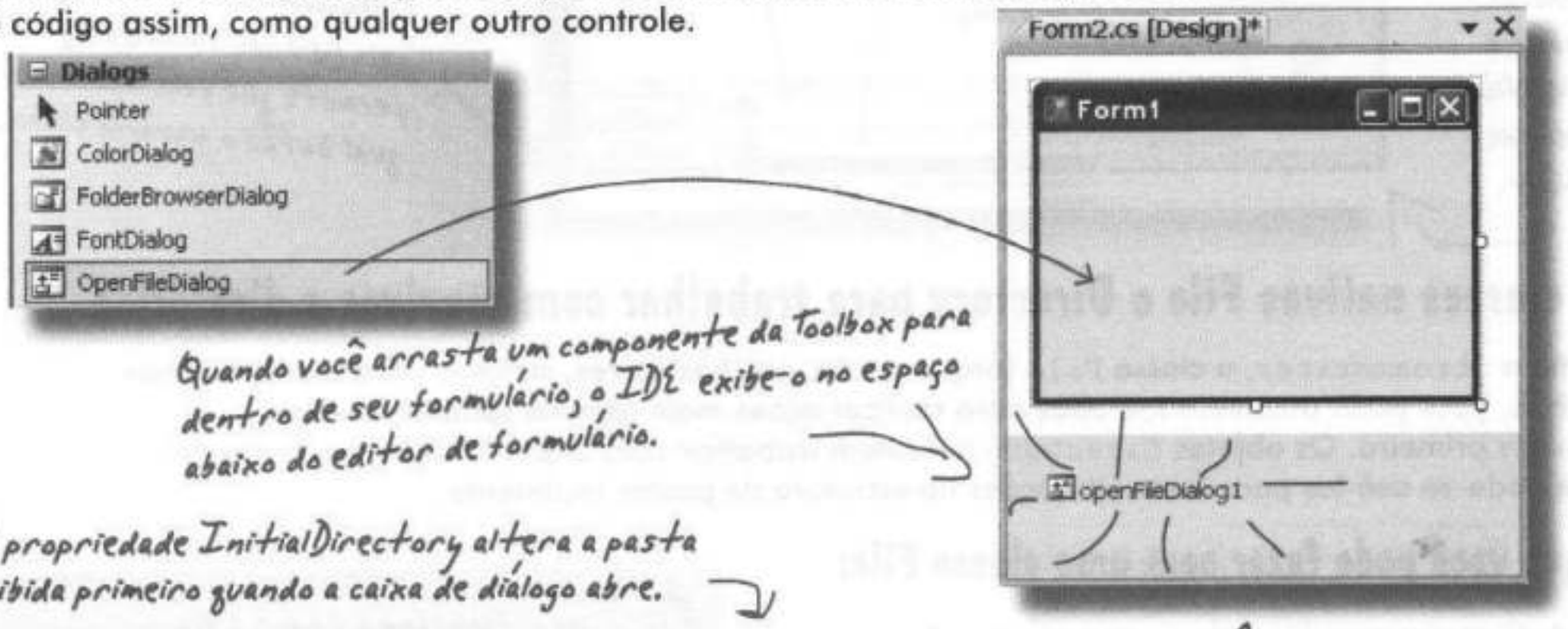
3. Chame o método `ShowDialog()` (exibir diálogo) do objeto. Isso lança a caixa de diálogo e não retorna até que o usuário clique no botão OK ou no Cancelar ou até que ele feche a janela.

4. O método `ShowDialog()` retorna um `DialogResult`, que é uma enum. Alguns de seus membros são `OK` (o que significa que o usuário clicou OK), `Cancel`, `Yes` e `No` (para caixas de diálogo Yes/No).

Caixas de diálogo são apenas mais um controle do .NET

Não visual quer dizer apenas que ele não aparece em seu formulário quando você o arrasta da caixa de ferramentas.

Você pode acrescentar caixas de diálogo padrão do Windows de arquivos em seu programa apenas arrastando-as para seu formulário –arraste um controle OpenFileDialog (diálogo abrir arquivos) da toolbox para seu formulário. Em vez de aparecer como um controle visual, você verá ele aparecer no espaço abaixo de seu formulário. Isso porque ele é um **componente**, um tipo especial de **controle não visual da caixa de ferramentas** que não aparece diretamente no formulário, mas utilizável no código assim, como qualquer outro controle.



Quando você arrasta um componente da Toolbox para dentro de seu formulário, o IDE exibe-o no espaço abaixo do editor de formulário.

A propriedade InitialDirectory altera a pasta exibida primeiro quando a caixa de diálogo abre.

```
openFileDialog1.InitialDirectory = @"c:\MyFolder\Default\";
openFileDialog1.Filter = "Text Files (*.txt)|*.txt|"
  + "Comma-Delimited Files (*.csv)|*.csv|All Files (*.*)|*.*";
openFileDialog1.FileName = "default_file.txt";
openFileDialog1.CheckFileExists = true;
openFileDialog1.CheckPathExists = false;
DialogResult result = openFileDialog1.ShowDialog();
if (result == DialogResult.OK){
  OpenSomeFile(openFileDialog1.FileName);
}
```

A propriedade Filter permite alterar os filtros que aparecem abaixo na caixa de diálogo, tais como os tipos de arquivos mostrados.

Essas propriedades mandam a caixa de diálogo exibir uma mensagem de erro se o usuário tentar abrir um arquivo ou caminho que não exista no disco.

Exiba a caixa de diálogo usando seu método ShowDialog(); que retornará uma DialogResult. Isso é uma enum que você pode usar para checar se o usuário clicou ou não no botão OK. Ela será atribuída a DialogResult.OK se o usuário clicar OK e a DialogResult.Cancel se ele clicar Cancelar.

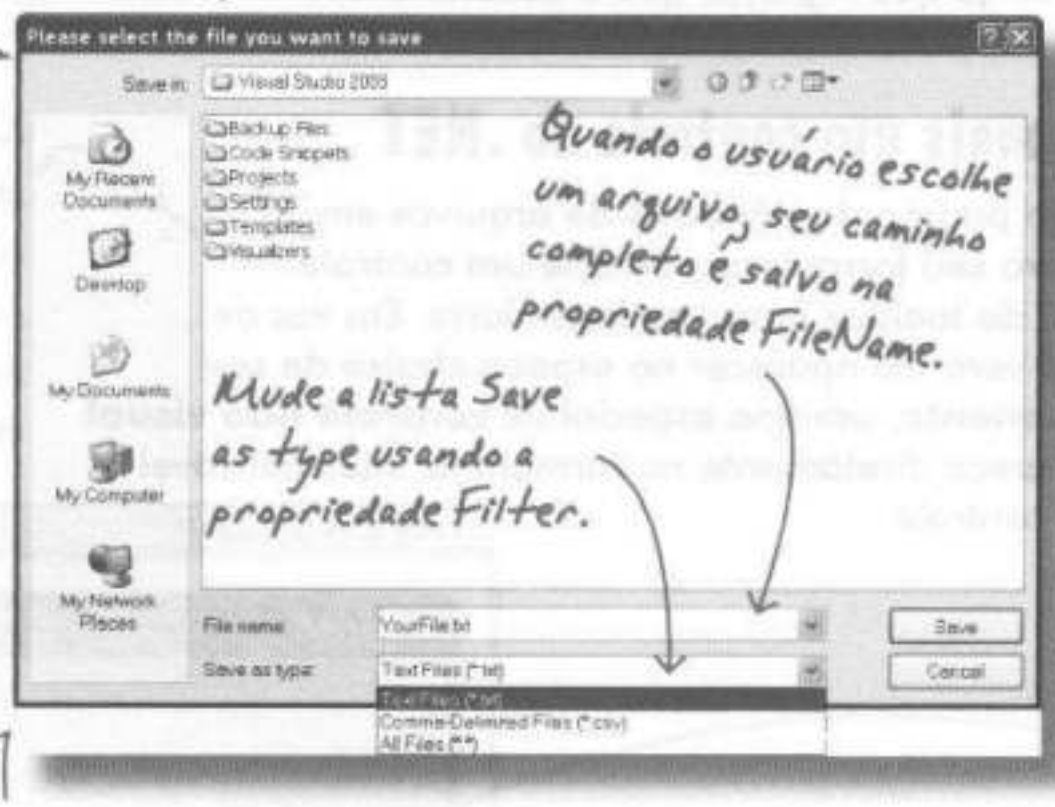
Caixas de diálogo também são objetos

Um objeto **OpenFileDialog** mostra a janela "Abrir" padrão do Windows e **SaveFileDialog** (diálogo salvar arquivo) mostra a janela "Salvar". Você pode exibi-los criando uma instância com new, alterando as propriedades do objeto e chamando seu método `ShowDialog()`. Esse método retorna uma enum `DialogResult` (como algumas caixas de diálogo têm mais de dois botões ou resultados, um simples não seria o suficiente).

Quando você arrasta um objeto de diálogo salvar da toolbox para seu formulário, o IDE apenas adiciona uma linha como esta no método InitializeComponent() de seu formulário.

```
saveFileDialog1 = new SaveFileDialog();
saveFileDialog1.InitialDirectory = @"c:\MyFolder\Default\";
saveFileDialog1.Filter = "Text Files (*.txt)|*.txt|"
  + "Comma-Delimited Files (*.csv)|*.csv|All Files (*.*)|*.*";
DialogResult result = saveFileDialog1.ShowDialog();
if (result == DialogResult.OK){
  SaveTheFile(saveFileDialog1.FileName);
}
```

As propriedades ShowDialog() e FileName funcionam exatamente da mesma forma como no objeto OpenFileDialog.



A propriedade Title deixa você alterar este texto.

O método ShowDialog() lança a caixa de diálogo e abre a pasta especificada na propriedade InitialFolder.

O objeto SaveFileDialog lança a caixa de diálogo Salvar como... padrão do Windows.

Quando o usuário escolhe um arquivo, seu caminho completo é salvo na propriedade FileName.

Mude a lista Save as type usando a propriedade Filter.

O DialogResult retornado pelo método ShowDialog permite que você descubra qual botão o usuário clicou.

Use as classes nativas File e Directory para trabalhar com arquivos e diretórios

Assim como a `StreamWriter`, a classe `File` (arquivo) cria, nos bastidores, streams para você trabalhar com arquivos. Você pode usar seus métodos para realizar ações mais comuns sem ter de criar `FileStreams` primeiro. Os objetos `Directory` permitem trabalhar com diretórios inteiros cheios de arquivos e pode-se usá-los para fazer alterações na estrutura de pastas facilmente.

Coisas que você pode fazer com uma classe File:

1. **Descobrir se um arquivo existe**

   Você pode checar para ver se um arquivo existe usando o método `Exists()`. Ele retornará true se existir e false se não.

2. **Ler e escrever no arquivo**

   Você pode usar o método `OpenRead()` (abrir para leitura) para acessar dados de um arquivo, ou o método `Create()` (criar) ou `OpenWrite()` (abrir para escrita) para escrever nele.

3. **Anexar texto em um arquivo**

   O método `AppendAllText()` (anexar todo o texto) permite anexar texto a um arquivo criado. Inclusive, ele cria o arquivo se este não estiver lá quando o método executar.

4. **Obtendo informações**

   O GetLastAccessTime() e GetLastWriteTime() retornam a data e hora do último acesso e alteração do arquivo.

FileInfo funciona como File

Se você trabalhará muito com um arquivo, pode criar uma instância da classe FileInfo, em vez de usar os métodos estáticos da classe File. A classe FileInfo faz quase exatamente o mesmo que a classe File, exceto pelo fato de ter de instanciá-la para usá-la. Você pode criar uma nova instância de FileInfo e acessar seu método Exists(), ou seu método OpenRead(), exatamente da mesma forma.
A única diferença é que File é mais rápida para um pequeno número de ações e FileInfo é mais apropriada para trabalhos maiores.

Coisas que se pode fazer com uma class Directory:

1. **Criar um novo diretório**
   Crie um diretório usando o método `CreateDirectory()`. Tudo a fazer é fornecer o caminho - o método faz o resto.

2. **Obter uma lista de arquivos no diretório**
   Você pode criar uma matriz de arquivos em um diretório usando o método `GetFiles()` (obter arquivos) - apenas informe ao método um diretório e ele se encarregará de tudo.

3. **Apagar um diretório**
   Apagar um diretório é bem simples também. Use o método `Delete()`.

não existem Pertungas Idiotas

P: Eu ainda não entendi esse negócio de {0} e {1} apareceu no StreamWriter.

R: Quando imprimir seqüências de caracteres num arquivo, freqüentemente você se encontrará tendo de incluir os conteúdos de várias variáveis. Por exemplo, você pode ter de escrever algo assim:

```
writer.WriteLine("My name is " + name +
  "and my age is " + age);
```

Usar + para combinar seqüências de caracteres é muito tedioso e propenso a erro. É mais fácil tirar vantagem de {0} e {1}:

```
writer.WriteLine(
  "My name is {0} and my age is {1}",
  name, age);
```

É muito mais fácil ler esse código, especialmente se muitas variáveis forem incluídas na mesma linha.

P: Por que você colocou um @ na frente da seqüência de caracteres do nome de arquivo?

R: Os métodos `Write()` e `WriteLine()` suportam seqüências de escape como `\n` e `\r`. Isso dificulta digitar nomes de arquivos, que possuem muitos caracteres com barras invertidas antes deles. Se você colocar @ na frente da seqüência, isso diz ao C# para não interpretar seqüências de escape e também para embutir as quebras de linha em sua seqüência. Assim, por exemplo, você pode pressionar Enter no meio da string e uma quebra de linha será inserida no resultado:

```
string twoLine = @"this is a string
that spans two lines.";
```

P: O que significa \n e \t mesmo?

R: São seqüências de escape. `\n` é uma nova linha e `\t` é uma tab. `\r` é um caracter de retorno de carro, ou metade do retorno de carro de Windows – em arquivos de texto Windows as linhas devem terminar com `\r\n`. Se você quiser usar uma barra invertida de verdade em sua seqüência de caracteres, sem que o C# interprete-a como o início de uma seqüência de escape, basta digitar uma dupla barra invertida: `\\`.

P: O que você falou no começo sobre converter uma seqüência de caracteres em uma matriz de bytes? Como isso funciona?

R: Provavelmente, muitas vezes, você ouviu que arquivos num disco são representados em bits e bytes. Ou seja, quando você escreve um arquivo em um disco, o sistema operacional o trata como uma longa seqüência de bytes. Lembra-se do capítulo 4, como uma variável `byte` pode armazenar qualquer número entre 0 e 255? Todo arquivo no disco rígido é uma longa seqüência de números entre 0 e 255. Cabe aos programas que lêem e escrevem esses arquivos interpretarem os bytes como dados significativos. Quando você abre um arquivo no Bloco de Notas, ele converte cada byte individual em um caracter – por exemplo, `E` é 69 e `a` é 97. E quando você digita um texto nele e o salva, o Bloco de Notas converte de volta cada um dos caracteres em um byte e o salva no disco. E se você quiser escrever uma `string` em uma stream, precisará fazer o mesmo.

P: Se eu estiver usando apenas uma StreamWriter para escrever em um arquivo, por que devo me preocupar se ela está criando uma FileStream para mim?

R: Se você estiver apenas lendo ou escrevendo linhas em ordem em arquivos de texto, necessitará apenas de `StreamReader` e `StreamWriter`. Mas quando precisar fazer qualquer coisa mais complexa que isso, terá de começar a trabalhar com outras streams. Se for preciso escrever dados como números, matrizes, coleções ou objetos em um arquivo, uma `StreamWriter` apenas não será o suficiente. Mas não se preocupe, veremos como isso funciona muito mais detalhadamente daqui a pouco.

P: E se eu quiser criar minhas próprias caixas de diálogo? Posso fazer isso?

R: Sim, claro que pode. Você pode adicionar um novo formulário em seu projeto, projetando-o como quiser. Então pode criar uma nova instância dele com new (assim como quando criou um objeto `OpenFileDialog()`. Depois, pode chamar seu método `ShowDialog()` e ele funcionará como qualquer outra caixa de diálogo. Falaremos mais sobre adicionar outros formulários em seu programa no capítulo 13.

P: Por que eu preciso me preocupar em fechar streams depois de ter trabalhado com elas?

R: Alguma vez um processador de texto informou que não pode abrir um arquivo porque estava "ocupado"? Quando um programa usa um arquivo, o Windows o trava e evita que outros programas o usem. E ele fará isso para seu programa quando abrir um arquivo. Se você não chamar o método `Close()`, é possível que seu programa mantenha um arquivo aberto e travado até que se encerre.

Aponte seu lápis

O .NET possui duas classes nativas com vários métodos estáticos para trabalhar com arquivos e pastas. A classe **File** fornece métodos para trabalhar com arquivos e a **Directory** permite lidar com diretórios. Escreva o que você acha que fazem essas linhas de código.

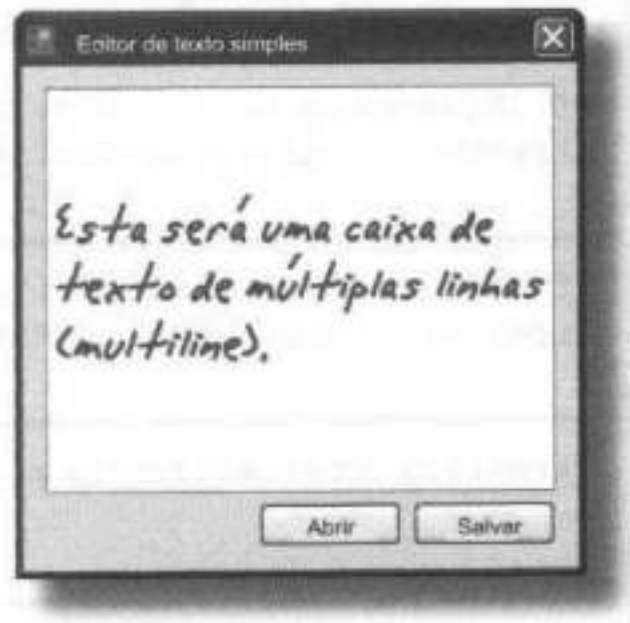
Código	O que o código faz
`if (Directory.Exists(@"c:\SYP")) { Directory.CreateDirectory(@"c:\SYP"); }`	
`if (Directory.Exists(@"c:\SYP\Bonk")) { Directory.Delete(@"c:\SYP\Bonk"); }`	
`Directory.CreateDirectory(@"c:\SYP\Bonk");`	
`Directory.SetCreationTime(@"c:\SYP\Bonk", new DateTime(1976, 09, 25));`	
`string[] files = Directory.GetFiles(@"c:\windows\", "*.log", SearchOption.AllDirectories);`	
`File.WriteAllText(@"c:\SYP\Bonk\weirdo.txt", @"This is the first line and this is the second line and this is the last line");`	
`File.Encrypt(@"c:\SYP\Bonk\weirdo.txt");` *Veja se consegue descobrir o que esta linha faz - você ainda não leu sobre isso*	
`File.Copy(@"c:\SYP\Bonk\weirdo.txt", @"c:\SYP\copy.txt");`	
`DateTime myTime = Directory.GetCreationTime(@"c:\SYP\Bonk");`	
`File.SetLastWriteTime(@"c:\SYP\copy.txt", myTime);`	
`File.Delete(@"c:\SYP\Bonk\weirdo.txt");`	

Use diálogos especializados para abrir e salvar arquivos (tudo com apenas poucas linhas de código)

Você pode criar um programa que abre um arquivo de texto. Ele permitirá que você faça alterações no arquivo e salve-as com muito pouco código, tudo utilizando controles .NET padrão. Veja como:

Faça isso

1 **Crie um formulário simples.**
Tudo de que você precisa é de uma caixa de texto e dois botões. Arraste os controles OpenFileDialog (diálogo abrir arquivo) e SaveFileDialog (diálogo salvar arquivo) para o formulário também. Clique duas vezes nos botões para criar seus tratadores de eventos e **adicione um campo string privado name no formulário**. Não se esqueça de colocar um comando using no topo do arquivo para `System.IO`.



2 **Associe o botão Abrir a openFileDialog.**
O botão Abrir mostra uma OpenFileDialog e então usa File.ReadAllText() (arquivo, ler todo o texto) para ler o arquivo na caixa de texto:

```
private void open_Click(object sender, EventArgs e) {
  if (openFileDialog1.ShowDialog() == DialogResult.OK) {
    name = openFileDialog1.FileName;
    textBox1.Clear();
    textBox1.Text = File.ReadAllText(name);
  }
}
```

Clicar em Abrir mostra o controle OpenFileDialog.

Os métodos ReadAllText() e WriteAllText() são parte da classe File. Vamos falar disso na próxima página e vamos vê-los com mais detalhes daqui a algumas poucas páginas.

3 **Agora, desenvolva o botão Salvar.**
O botão Salvar usa o método File.WriteAllText() (escrever todo o texto) para salvar o arquivo:

```
private void save_Click(object sender, EventArgs e) {
  if (saveFileDialog1.ShowDialog() == DialogResult.OK) {
    name = saveFileDialog1.FileName;
    File.WriteAllText(name, textBox1.Text);
  }
}
```

4 **Brinque com as outras propriedades das caixas de diálogo**

- ★ Use a propriedade Title de saveFileDialog para alterar o texto na barra de título.
- ★ Configure a propriedade initialFolder para fazer o diálogo OpenFileDialog iniciar num diretório específico.
- ★ Filtre OpenFileDialog para que ele mostre apenas arquivos de texto usando a propriedade Filter.

Se você não adicionar um filtro, as listas de seleção no final das caixas de diálogo abrir e salvar ficarão vazias. Tente usar este filtro: Arquivos Texto (*.txt)|*.txt

Aponte seu lápis
Solução

O .NET possui duas classes embutidas com vários métodos estáticos para trabalhar com arquivos e pastas. A classe **File** fornece métodos para trabalhar com arquivos e a **Directory** permite lidar com diretórios. O seu trabalho era escrever o que cada parte do código fez.

Código	O que o código faz
`if (Directory.Exists(@"c:\SYP")) { Directory.CreateDirectory(@"c:\SYP"); }`	Checa se a pasta C:\SYP existe. Se não, ele a cria.
`if (Directory.Exists(@"c:\SYP\Bonk")) { Directory.Delete(@"c:\SYP\Bonk"); }`	Checa se a pasta C:\SYP\Bonk existe. Se existir, é deletada.
`Directory.CreateDirectory(@"c:\SYP\Bonk");`	Cria o diretório C:\SYP\Bonk.
`Directory.SetCreationTime(@"c:\SYP\Bonk", new DateTime(1976, 09, 25));`	Altera o tempo de criação da pasta C:\SYP\Bonk para 25 de setembro de 1976.
`string[] files = Directory.GetFiles(@"c:\windows\", "*.log", SearchOption.AllDirectories);`	Recupera uma lista de todos os arquivos em C:\Windows que correspondam ao padrão *.log, incluindo todos os arquivos correspondentes em qualquer subdiretório.
`File.WriteAllText(@"c:\SYP\Bonk\weirdo.txt", @"This is the first line and this is the second line and this is the last line");`	Cria um arquivo chamado weirdo.txt (se ele já não existir) na pasta C:\SYP\Bonk e escreve três linhas de texto nele.
`File.Encrypt(@"c:\SYP\Bonk\weirdo.txt");` ← Esta é uma alternativa ao uso de CryptoStream.	Aproveita a encriptação nativa do Windows para codificar o arquivo weirdo.txt usando as credenciais da conta logada.
`File.Copy(@"c:\SYP\Bonk\weirdo.txt", @"c:\SYP\copy.txt");`	Copia o arquivo C:\SYP\Bonk\weirdo.txt para C:\SYP\Copy.txt.
`DateTime myTime = Directory.GetCreationTime(@"c:\SYP\Bonk");`	Declara a variável myTime e iguala-a ao tempo de criação da pasta C:\SYP\Bonk.
`File.SetLastWriteTime(@"c:\SYP\copy.txt", myTime);`	Altera a data de última escrita do arquivo copy.txt em C:\SYP\ para que ela seja igual a qualquer data armazenada na variável myTime.
`File.Delete(@"c:\SYP\Bonk\weirdo.txt");`	Deleta o arquivo C:\SYP\Bonk\weirdo.txt.

IDisposable garante o descarte apropriado de seus objetos

Declare um objeto em um bloco using e o método Dispose() do objeto é chamado automaticamente.

Muitas classes do .NET implementam uma interface particularmente útil, `IDisposable`. **Ela possui apenas um membro: Dispose()**. Quando uma classe implementa essa interface, ela diz que há coisas importantes a serem feitas antes de ela se encerrar a si mesma, geralmente porque alocou recursos que não serão devolvidos ao sistema até que você mande. O método `Dispose()` é como você diz ao objeto para liberar esses recursos.
Você pode usar a característica "Go To Definition" (vá para a definição) no IDE na definição oficial do C# de `IDisposable`. Vá até seu projeto e digite IDisposable em qualquer lugar dentro do código. Então clique com o botão direito nele e selecione "Go To Definition" no menu. Ele abrirá uma nova aba com código dentro. Expanda todo o código você verá isto:

Você aprenderá mais sobre Go To Definition mais tarde.

```
namespace System
{
  // Sumario:
  //   Define um metodo para liberar recursos alocados.
  public interface IDisposable
  {
    // Sumario:
    //   Efetua tarefas definidas pelo aplicativo
    //   associadas com liberar
    //   ou reiniciar recursos nao gerenciados.
    void Dispose();
  }
}
```

Muitas classes alocam recursos importantes, como memória, arquivos e outros objetos. Elas passam a controlá-los e não os liberam até que você diga que elas já terminaram de trabalhar com eles.

Qualquer classe que implemente IDisposable deverá liberar imediatamente todos os recursos que tomou conta quando você chamar seu método Dispose(). É quase sempre a última coisa feita antes de terminar de trabalhar com o objeto.

a-loc-ar, verbo.
distribuir recursos ou deveres para um propósito específico. *A equipe de programação estava irritada com seu gerente de projeto porque ele* ***alocou*** *todas as salas de conferência para um seminário de gerenciamento inútil.*

Go To Definition

Há uma característica útil no IDE que lhe permite ir automaticamente para a definição de qualquer variável, objeto ou método. Basta clicar nele com o botão direito e selecionar Go To Definition e o IDE irá diretamente para o código que o define.

Evite erros no sistema de arquivos com comandos using

Estivemos falando durante todo o capítulo que é preciso **fechar suas streams**. Isso porque as falhas mais comuns enfrentadas pelos programadores quando lidam com arquivos são causada por streams não fechadas corretamente. Felizmente, o C# oferece uma ótima ferramenta que impede isto de acontecer com você: `IDisposable` **e o método** `Dispose()`. Quando você **embute seu código stream dentro de um bloco using**, ele automaticamente fecha suas streams para você. Tudo a fazer é **declarar sua referência de stream** utilizando um comando using, seguido por um bloco de código (dentro de chaves) que use essa referência. Quando você fizer isso, o comando using **automaticamente chama o método Dispose()** da stream assim que termina de executar o bloco de código. Eis como isso funciona:

Esses comandos using são diferentes daqueles na parte de cima de seu código.

...e então por um bloco de código dentro de chaves.

Um comando using é sempre seguido por uma declaração de objeto...

```
using (StreamWriter sw = new StreamWriter("secret_plan.txt")) {
  sw.WriteLine("How I'll defeat Captain Amaing");
  sw.WriteLine("Another genius secret plan");
  sw.WriteLine("by The Swindler");
}
```

Estes comandos podem usar o objeto criado no comando using acima como qualquer objeto normal.

Quando o comando using termina, o método Dispose() do objeto em uso é executado.

Neste caso, o objeto em uso é o Log Stream, então o método Dispose() da classe Stream é executado... o que fecha a stream.

Toda stream possui um método Dispose() que a fecha. Então se você declarar sua stream num comando using, ela sempre fechará a si própria!

Utilize diversos comandos using para múltiplos objetos

Você pode aninhar comandos using – não é preciso usar conjuntos extras de chaves ou identação.

```
using (StreamReader reader = new StreamReader("secret_plan.txt");
using (StreamWriter writer = new StreamReader("email.txt");
{
  // comandos que usam reader e writer
}
```

Você não precisa chamar Close() nas streams aqui, porque o comando using as fechará automaticamente.

Quando você usar uma stream, SEMPRE a declare dentro de um comando using. Isso assegura que ela sempre seja fechada!

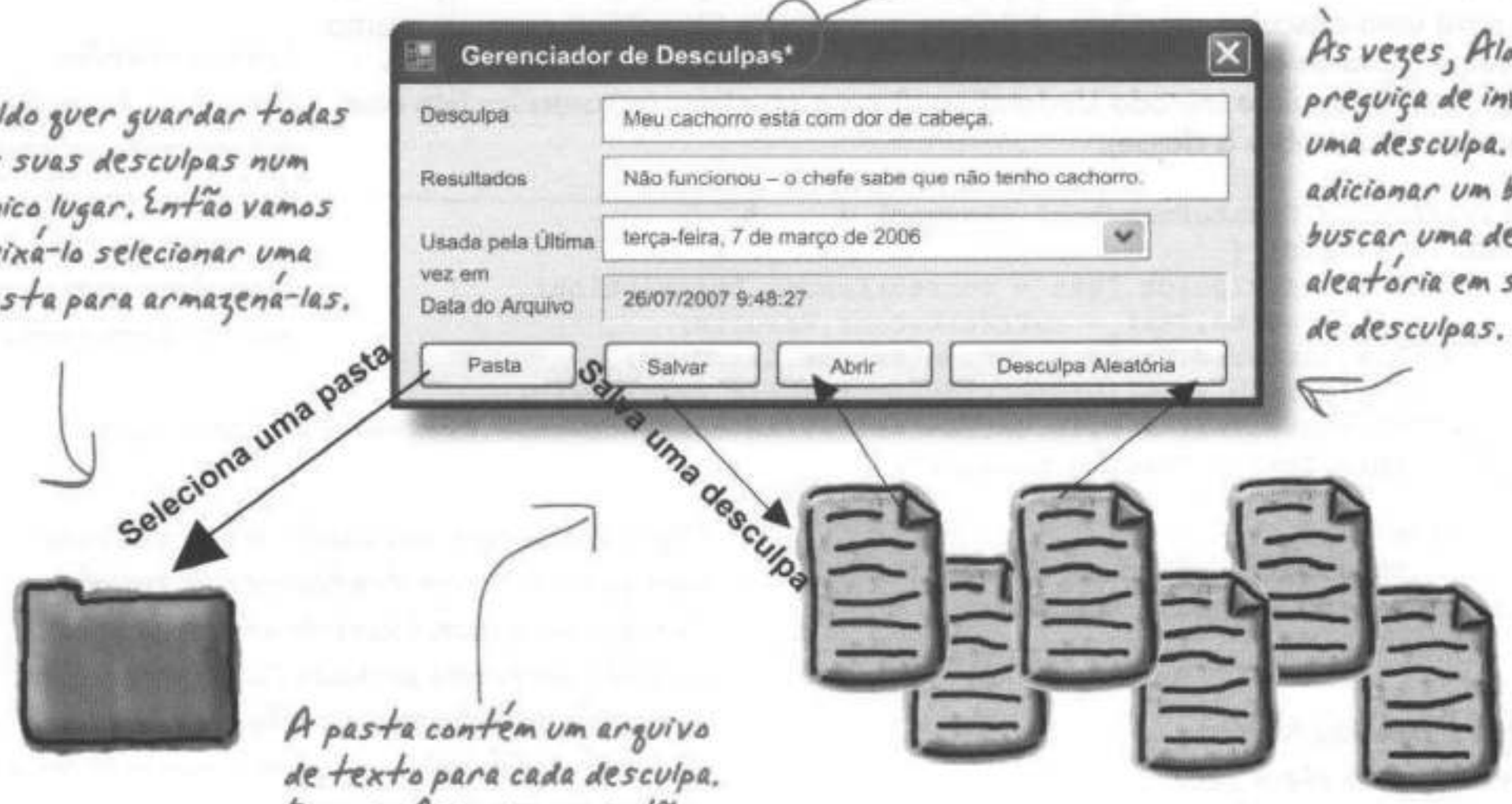
Problemas no trabalho

Conheça o Aldo. Ele gosta de seu trabalho como um desenvolvedor C#, mas ele adora tirar um ocasional dia de folga. Mas seu chefe **odeia** quando as pessoas tiram dias de folga, então Aldo deve inventar uma boa desculpa.

Você pode ajudar o Aldo criando um programa para gerenciar suas desculpas

Use o que você sabe sobre ler e escrever arquivos para criar um gerenciador utilizável por Aldo para controlar quais desculpas ele já usou recentemente e como elas funcionaram com o chefe.

Este asterisco aparece quando um formulário possui dados não salvos.



Exercício

Crie um gerenciador para que Aldo possa controlar suas desculpas no trabalho.

Excuse
Description: string LastUsed: DateTime ExcusePath: string
OpenFile(string) Save(string)

1 **Crie o formulário**

Este formulário possui algumas características especiais:

- ★ Assim que o formulário é carregado, **apenas o botão Pasta deve ser habilitado** – desative os outros três até que o usuário selecione uma pasta.
- ★ Quando o formulário abre ou salva uma desculpa, ele exibe a data do arquivo de desculpas usando um controle Label com AutoSize estabalecido como False e BorderStyle como Fixed3D.
- ★ Depois que uma desculpa é salva, o formulário mostra uma caixa de mensagem "Desculpa Salva".
- ★ O botão Pasta exibe uma caixa de diálogo de navegação de pasta. Se o usuário selecionar uma pasta, ele habilita os botões Salvar, Abrir e Desculpa Aleatória.
- ★ O formulário sabe quando há alterações não salvas. Quando não for o caso, o texto na barra de título do é "Gerenciador de Desculpas". Mas quando o usuário alterar qualquer um dos três campos, um asterisco (*) será acrescentado à barra de título. Ele é retirado quando os dados forem salvos ou quando uma nova desculpa for carregada
- ★ O formulário precisará manter registro sobre a pasta atual e se a desculpa atual foi ou não salva. Você pode descobrir quando a desculpa não foi salva **usando os tratadores de evento Changed** para os três controles de entrada.

Quando você arrasta uma caixa de texto para um formulário e clica duas vezes nela, cria um tratador de evento Changed (alterado) para aquele campo.

2 **Crie uma classe Excuse e armazene uma instância dela no formulário**

Agora adicione um campo CurrentExcuse (desculpa atual) no formulário com a desculpa atual. Você precisará de **três construtores sobrecarregados:** um para quando o formulário é carregado pela primeira vez, um para abrir um arquivo e um para uma desculpa aleatória. Adicione os métodos OpenFile() para abrir uma desculpa (para os construtores usarem) e Save() para salvar a desculpa. Então acrescente o seguinte método UpdateForm() para atualizar os controles (ele dará algumas **dicas** sobre a classe):

Este parâmetro indica se o formulário foi alterado ou não. Você precisará de um campo em seu formulário para manter este estado.

```
private void UpdateForm(bool Changed) {
  if (!Changed) {
     this.description.Text = currentExcuse.Description;
     this.results.Text = currentExcuse.Results;
     this.lastUsed.Value = currentExcuse.LastUsed;
     if (!String.IsNullOrEmpty(currentExcuse.ExcusePath))
      FileDate.Text = File.GetLastWriteTime(currentExcuse.ExcusePath).ToString();
      this.Text = "Excuse Manager";
    }
    else
      this.Text = "Excuse Manager*";
    this.formChanged = Changed;
  }
```

Clique duas vezes nos controles de entrada para que o IDE crie tratadores de evento Changed para você. Esses tratadores para os três controles primeiro fazem alterações na instância de Excuse e então chamam UpdateForm(true) – logo cabe a você alterar os campos em seu formulário.

Lembre-se, o ! significa NÃO – então este comando checa se o caminho para a desculpa NÃO está nulo ou vazio.

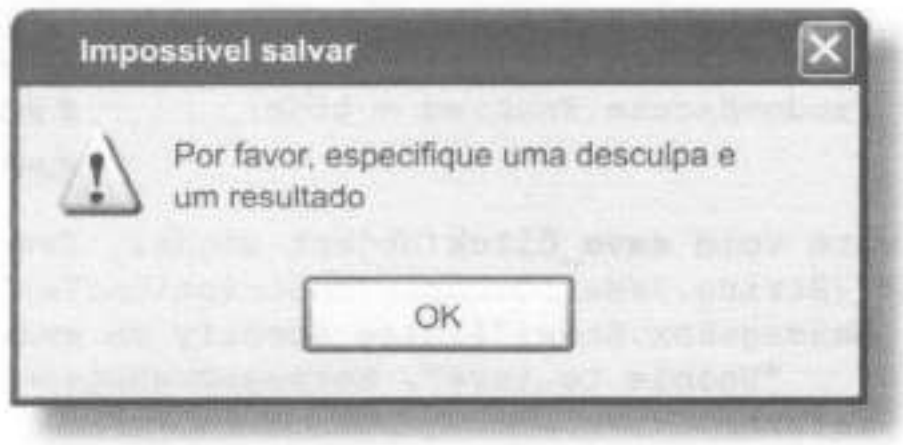
3 **Faça o botão Pasta abrir um navegador de pasta**

Quando o usuário clicar no botão Pasta, o formulário deve exibir uma caixa de diálogo "Navegador de Pastas" (Browse for Folder). O formulário precisará armazenar a pasta em um campo para que outras caixas de diálogo possam usá-la. Quando o formulário carrega pela primeira vez, os botões Salvar, Abrir e Desculpa Aleatória são **desabilitados**, mas se o usuário selecionar uma pasta o botão Pasta os ativa.

4 **Faça o botão respectivo salvar a desculpa atual num arquivo**
Clicar no botão Salvar deve fazer uma caixa de diálogo Salvar Como aparecer.

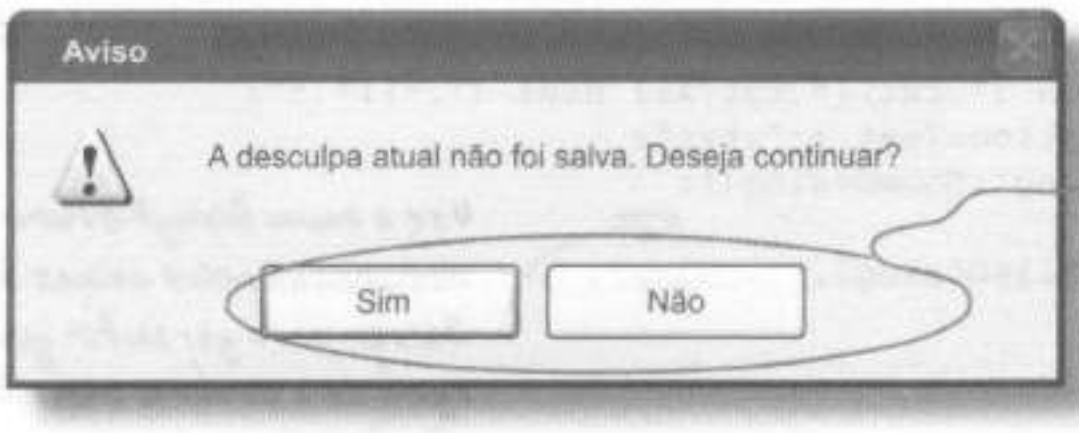
- ★ Cada desculpa é salva num arquivo texto separado. A primeira linha do arquivo é uma desculpa, a segunda é o resultado e a terceira é a data quando foi usada pela última vez (use o método `ToString()` de DateTimePicker(). A classe Excuse deve ter um método `Save()` para salvar uma desculpa num arquivo específico.
- ★ Quando a caixa de diálogo Salvar Como for aberta, sua pasta deve ser atribuída àquela selecionada pelo usuário usando o botão Pasta e o nome do arquivo deve ser atribuído à desculpa com um extensão ".txt".
- ★ A caixa de diálogo deve ter dois filtros: Arquivos de Texto (*.txt) e Todos os Arquivos (*.*).
- ★ Se o usuário tentar salvar a desculpa atual, mas tiver deixado a desculpa ou o resultado em branco, o formulário deve exibir um aviso:

Você pode exibir esse ícone de Exclamação usando o método MessageBox.Show() sobrecarregado que permite especificar um parâmetro MessageBoxIcon.



5 **Providencie para que o botão respectivo abra uma desculpa salva**
Clicar no botão Salvar deve fazer aparecer uma caixa de diálogo Abrir.

- ★ Quando essa caixa de diálogo for aberta, sua pasta deve ser atribuída àquela que o usuário selecionou usando o botão Pasta.
- ★ Adicione um método `Open()` na classe Excuse para abrir uma desculpa de um arquivo dado.
- ★ Use `Convert.ToDateTime()` para carregar a data salva no controle DateTimePicker.
- ★ Se o usuário tentar abrir uma desculpa, mas a desculpa atual não estiver salva, esta caixa de diálogo aparecerá:



Para mostrar uma caixa de diálogo com Sim/Não usa-se o método MessageBox.Show() sobrecarregado que permite especificar o parâmetro MessageBoxButtons. Se o usuário clicar em Não, Show() retorna DialogResult.No.

6 **Finalmente, faça o botão respectivo gerar uma desculpa aleatória.**
Quando o usuário clica no botão Desculpa Aleatória, ele olha na pasta de desculpas, escolhe uma delas aleatoriamente e a abre.

- ★ O formulário precisará salvar um objeto Random em um campo e passá-lo para um dos construtores sobrecarregados do objeto `Excuse`.
- ★ Se a desculpa atual não foi salva, este botão deve lançar o mesmo aviso que o do botão Abrir.

Solução do Exercício

Crie um gerenciador para que Aldo possa administrar suas desculpas no trabalho.

```
private Excuse currentExcuse = new Excuse();
private string selectedFolder = "";
private bool formChanged = false;
Random random = new Random();
private void folder_Click(object sender, EventArgs e) {
  folderBrowserDialog1.SelectedPath = selectedFolder;
  DialogResult result = folderBrowserDialog1.ShowDialog();
  if (result == DialogResult.OK) {
    selectedFolder = folderBrowserDialog1.SelectedPath;
    save.Enabled = true;
    open.Enabled = true;
    randomExcuse.Enabled = true;
  }
}
private void save_Click(object sender, EventArgs e) {
  if (String.IsNullOrEmpty(description.Text) || String.IsNullOrEmpty(results.Text)) {
    MessageBox.Show("Please specify an excuse and a result",
        "Unable to save", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
    return;
  }
  saveFileDialog1.InitialDirectory = selectedFolder;
  saveFileDialog1.Filter = "Text files (*.txt)|*.txt|All files (*.*)|*.*";
  saveFileDialog1.FileName = description.Text + ".txt";
  DialogResult result = saveFileDialog1.ShowDialog();
  if (result == DialogResult.OK) {
    currentExcuse.Save(saveFileDialog1.FileName);
    UpdateForm(false);
    MessageBox.Show("Excuse written");
  }
}
private void open_Click(object sender, EventArgs e) {
  if (CheckChanged()) {
    openFileDialog1.InitialDirectory = selectedFolder;
    openFileDialog1.Filter = "Text files (*.txt)|*.txt|All files (*.*)|*.*";
    openFileDialog1.FileName = description.Text + ".txt";
    DialogResult result = openFileDialog1.ShowDialog();
    if (result == DialogResult.OK) {
      currentExcuse = new Excuse(openFileDialog1.FileName);
      UpdateForm(false);
    }
  }
}
private void randomExcuse_Click(object sender, EventArgs e) {
  if (CheckChanged()) {
    currentExcuse = new Excuse(random, selectedFolder);
    UpdateForm(false);
  }
}
```

O formulário usa campos para armazenar o objeto Excuse atual, a pasta selecionada, lembrar se a desculpa atual foi ou não alterada e para manter um objeto Random para o botão Desculpa Aleatória.

Se o usuário selecionou uma pasta, o formulário salva seu caminho e habilita os outros três botões.

As duas barras verticais significam OU – isso é verdadeiro se description OU se results estiverem vazias.

É aqui que os filtros são atribuídos ao diálogo Salvar Como.

Isso fará com que duas linhas apareçam na caixa de opções Arquivos de Tipo na parte de baixo do diálogo Salvar: uma para Arquivos de Texto (*.txt) e uma para Todos os Arquivos (*.*).

Use a enum DialogReturn retornada pelas caixas Abrir e Salvar para garantir que você só agirá se o usuário clicar OK, e não Cancelar.

```
private bool CheckChanged() {
  if (formChanged) {
    DialogResult result = MessageBox.Show(
          "The current excuse has not been saved. Continue?",
          "Warning", MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
    if (result == DialogResult.No)
      return false;
  }
  return true;
}
private void description_TextChanged(object sender, EventArgs e) {
  currentExcuse.Description = description.Text;
  UpdateForm(true);
}
private void results_TextChanged(object sender, EventArgs e) {
  currentExcuse.Results = results.Text;
  UpdateForm(true);
}
private void lastUsed_ValueChanged(object sender, EventArgs e) {
  currentExcuse.LastUsed = lastUsed.Value;
  UpdateForm(true);
}
public class Excuse {
  public string Description;
  public string Results;
  public DateTime LastUsed;
  public string ExcusePath;
  public Excuse() {
    ExcusePath = "";
  }
  public Excuse(string excusePath) {
    OpenFile(excusePath);
  }
  public Excuse(Random random, string folder) {
    string[] fileNames = Directory.GetFiles(folder, "*.txt");
    OpenFile(fileNames[random.Next(fileNames.Length)]);
  }
  private void OpenFile(string excusePath) {
    this.ExcusePath = excusePath;
    using (StreamReader reader = new StreamReader(excusePath)) {
      Description = reader.ReadLine();
      Results = reader.ReadLine();
      LastUsed = Convert.ToDateTime(reader.ReadLine());
    }
  }
  public void Save(string fileName) {
    using (StreamWriter writer = new StreamWriter(fileName))
    {
      writer.WriteLine(Description);
      writer.WriteLine(Results);
      writer.WriteLine(LastUsed.ToString());
} } }
```

MessageBox.Show() também retorna uma enum DialogResult que podemos examinar.

Aqui estão os três tratadores de evento Changed para os três campos de entrada no formulário. Se qualquer um deles for acionado, a desculpa mudou. Então, primeiro atualizamos a instância Excuse e chamamos UpdateForm(), depois, adicionamos o asterisco à barra de título do formulário e igualamos Changed a true.

Passar true a UpdateForm() apenas avisa para marcar o formulário como alterado, mas não atualizar os controles de entrada.

Certificamo-nos de utilizar um comando using sempre que abrirmos uma stream. Dessa forma nossos arquivos sempre serão fechados.

O botão Desculpa Aleatória usa Directory.GetFiles() para ler todos os arquivos de texto na pasta selecionada dentro de uma matriz e então escolhe um índice de matriz aleatório para abrir.

É aqui que o comando using entra. Declaramos a StreamWriter dentro de um comando using e seu método Close() é chamado automaticamente!

Escrever arquivos geralmente envolve tomar muitas decisões

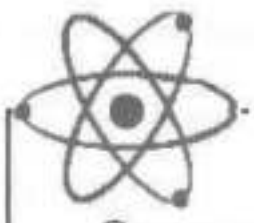
Você escreverá muitos programas com apenas uma entrada, talvez de um arquivo, e deverá decidir o que fazer baseando-se somente nela. Aqui está o código que usa um grande comando `if` – que é muito comum. Ele checa a variável `part` e imprime diferentes linhas no arquivo baseado em que elemento de uma enum é usado. Há muitas escolhas possíveis, logo há muitos `else ifs`:

```
enum BodyPart {
    Head,
    Shoulders,
    Knees,
    Toes
}

private void WritePartInfo(BodyPart part, StreamWriter writer) {
  if (part == BodyPart.Head)
    writer.WriteLine("the head is hairy");
  else if (part == BodyPart.Shoulders)
    writer.WriteLine("the shoulders are broad");
  else if (part == BodyPart.Knees)
    writer.WriteLine("the knees are knobby");
  else if (part == BodyPart.Toes)
    writer.WriteLine("the toes are teeny");
  else
    writer.WriteLine("some unknown part is unknown");
  }
}
```

Aqui temos uma enum – vamos querer comparar uma variável com cada um dos quatro membros e escrever uma linha diferente na StreamWriter dependendo de qual ela corresponder. Também vamos escrever algo diferente se não houver equivalência.

Se usarmos uma série de comandos if/else, escreveremos este if (part == [Option]) muitas vezes.

Teremos um else final no caso de não encontrarmos uma correspondência.



Poder do Cérebro

O que pode acontecer de errado quando você escreve código com muitos comandos if/else? Pense sobre erros de digitação e falhas causadas por chaves, um único sinal de igual, etc.

Use um comando switch para escolher a opção correta

Comparar uma variável com vários valores diferentes é um padrão muito comum, especialmente quando você está lendo e escrevendo arquivos. É tão comum, na verdade, que o C# tem um tipo de comando especial específico para esta situação.

Não há nada sobre um comando switch especificamente relacionado a arquivos. Ele é apenas uma ferramenta útil do C# utilizável aqui.

Um comando switch comparar uma variável com muitos valores de forma fácil de ler e compacta. Aqui vemos um comando switch que faz o mesmo que a série de comandos `if/else` na página anterior:

Um comando switch compara UMA variável com VÁRIOS valores possíveis.

```
enum BodyPart
{
  Head,
  Shoulders,
  Knees,
  Toes,
}

private void WritePartInfo(BodyPart part, StreamWriter writer)
{
  switch (part) {
    case BodyPart.Head:
      writer.WriteLine("the head is hairy");
      break;
    case BodyPart.Shoulders:
      writer.WriteLine("the shoulders are broad");
      break;
    case BodyPart.Knees:
      writer.WriteLine("the knees are knobby");
      break;
    case BodyPart.Toes:
      writer.WriteLine("the toes are teeny");
      break;
    default:
      writer.WriteLine("some unknown part is unknown");
      break;
  }
}
```

Você começa com a palavra-chave switch seguida de uma variável que será comparada com muitos valores possíveis diferentes.

O corpo do comando switch é uma série de casos que comparam o que quer que siga a palavra-chave switch com diversos valores em particular.

Todo case deve terminar com break.

Cada um dos casos consiste da palavra-chave case seguida do valor a ser comparado e dois pontos. Depois disso, há uma serie de comandos seguidos por break;. Esses comandos serão executados se o case corresponder ao valor de comparação.

Comandos switch podem terminar com um **default**. bloco que é executado se nenhum dos outros cases corresponderem.

Use um comando switch para deixar seu baralho ler de um arquivo ou escrever a si próprio em um

Escrever uma carta de baralho num arquivo é bem simples – basta um laço que salve o nome de cada carta. Aqui temos um método que você pode adicionar ao objeto Deck que faz exatamente isso:

```
public void WriteCards(string Filename) {
  using (StreamWriter writer = new StreamWriter(Filename)) {
    for (int i = 0; i < Cards.Count; i++) {
      writer.WriteLine(Cards[i].Name);
  } }
}
```

O comando switch permite testar se um valor se enquadra em um de muitos casos e executa diferentes comandos, dependendo de em qual deles a comparação for verdadeira.

Mas e quanto a ler o arquivo? Não é tão simples. É aí que o comando switch pode ser útil.

```
Card.Suits suit;
switch (suitString) {
  case "Spades":
    suit = Card.Suits.Spades;
    break;
  case "Clubs":
    suit = Card.Suits.Clubs;
    break;
  case "Hearts":
    suit = Card.Suits.Hearts;
    break;
  case "Diamonds":
    suit = Card.Suits.Diamonds;
    break;
  default:
    MessageBox.Show(suitString + " isn't a valid suit!");
}
```

O comando switch começa com um **valor** alvo de comparação. Este, em particular, é chamado de um método com um naipe armazenado em uma seqüência de caracteres.

Cada uma dessas linhas **case** comparam algum valor ao alvo declarado na linha switch. Se eles corresponderem, ela executa todos os comandos a seguir até chegar em um **break**.

A linha **default** vem no final. Se nenhum dos cases corresponderem, os comandos depois de **default** são executados.

Acrescente um construtor Deck() sobrecarregado que leia um baralho de um arquivo

Você pode usar um comando switch para criar um novo construtor para a classe Deck escrita no último capítulo. Este construtor lê em um arquivo e checa cada linha procurando por uma carta. Qualquer carta válida é adicionada ao baralho.

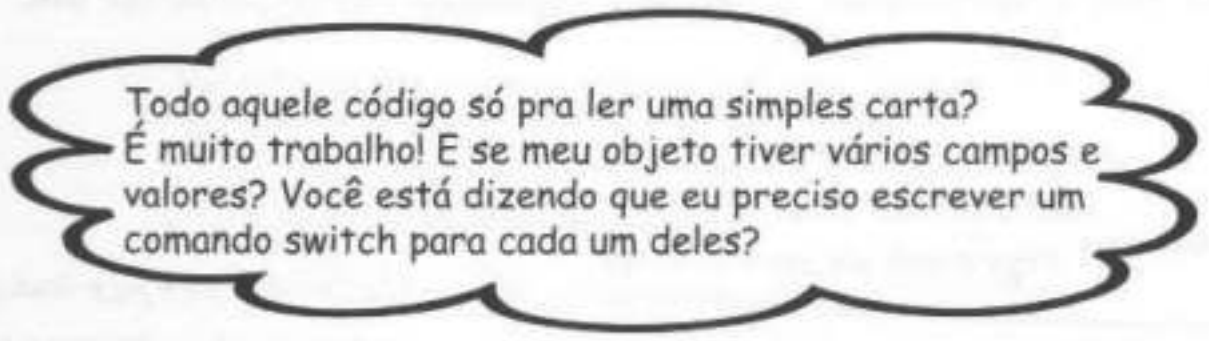
Aqui está um método encontrável em toda string que pode ser útil:
`Split()`. Permite dividir a seqüência de caracteres em uma matriz de subseqüências passando a ela uma matriz `char[]` de caracteres separadores que serão usados para dividir a seqüência original.

```
public Deck(string Filename) {
  Cards = new List<Card>();
  StreamReader reader = new StreamReader(Filename);
  while (!reader.EndOfStream) {
    bool invalidCard = false;
    string nextCard = reader.ReadLine();
    string[] cardParts = nextCard.Split(new char[] { ' ' });
    Card.Values value = Card.Values.Ace;
    switch (cardParts[0]) {
      case "Ace": value = Card.Values.Ace; break;
      case "Two": value = Card.Values.Two; break;
      case "Three": value = Card.Values.Three; break;
      case "Four": value = Card.Values.Four; break;
      case "Five": value = Card.Values.Five; break;
      case "Six": value = Card.Values.Six; break;
      case "Seven": value = Card.Values.Seven; break;
      case "Eight": value = Card.Values.Eight; break;
      case "Nine": value = Card.Values.Nine; break;
      case "Ten": value = Card.Values.Ten; break;
      case "Jack": value = Card.Values.Jack; break;
      case "Queen": value = Card.Values.Queen; break;
      case "King": value = Card.Values.King; break;
      default: invalidCard = true; break;
    }
    Card.Suits suit = Card.Suits.Clubs;
    switch (cardParts[2]) {
      case "Spades": suit = Card.Suits.Spades; break;
      case "Clubs": suit = Card.Suits.Clubs; break;
      case "Hearts": suit = Card.Suits.Hearts; break;
      case "Diamonds": suit = Card.Suits.Diamonds; break;
      default: invalidCard = true; break;
    }
    if (!invalidCard) {
      Cards.Add(new Card(suit, value));
    }
  }
}
```

Esta linha diz ao C# dividir a seqüência nextCard usando um espaço como caracter separador. Isto divide a seqüência Six of Diamonds na matriz { "Six", "of", "Diamonds" }.

Este comando switch checa a primeira palavra na linha para ver se ela corresponde a uma carta. Se corresponder, o valor certo é atribuído à variável value.

Fazemos a mesma coisa para a terceira palavra na linha, mas convertemos essa em uma suit.



Existe uma forma mais fácil de armazenar seus objetos em arquivos. Chama-se serialização.

Em vez de escrever cuidadosamente cada campo e valor num arquivo, linha por linha, você pode salvar seu objeto facilmente serializando-o em uma stream. **Serializar** um objeto é como **achatá-lo** para que possa inseri-lo num envelope. E por outro lado, você pode reverter a serialização, que é como retirá-lo do arquivo e **inflá-lo** de novo.

O que acontece com um objeto quando ele é serializado?

Parece que algo misterioso deve acontecer com um objeto para copiá-lo da pilha e colocá-lo em um arquivo mas, na verdade, é bem simples.

1 Objeto na pilha

Quando você cria uma instância de um objeto, ele tem um **estado**. Tudo que um objeto "sabe" é o que torna uma instância de uma classe diferente de outra da mesma classe.

2 Objeto serializado

Quando o C# serializa, **ele salva o estado completo do objeto**, para que uma instância (objeto) idêntica possa ser trazida de volta à vida na pilha mais tarde.

3 E mais tarde...

Mais tarde – talvez depois de dias – e em um programa diferente você pode voltar ao arquivo e **reverter a serialização**. Isso puxa a classe original de volta do arquivo e a restaura **exatamente como era**, com todos os seus campos e valores intactos.

Objeto na pilha de novo

Mas o que É exatamente um estado de objeto? O que precisa ser salvo?

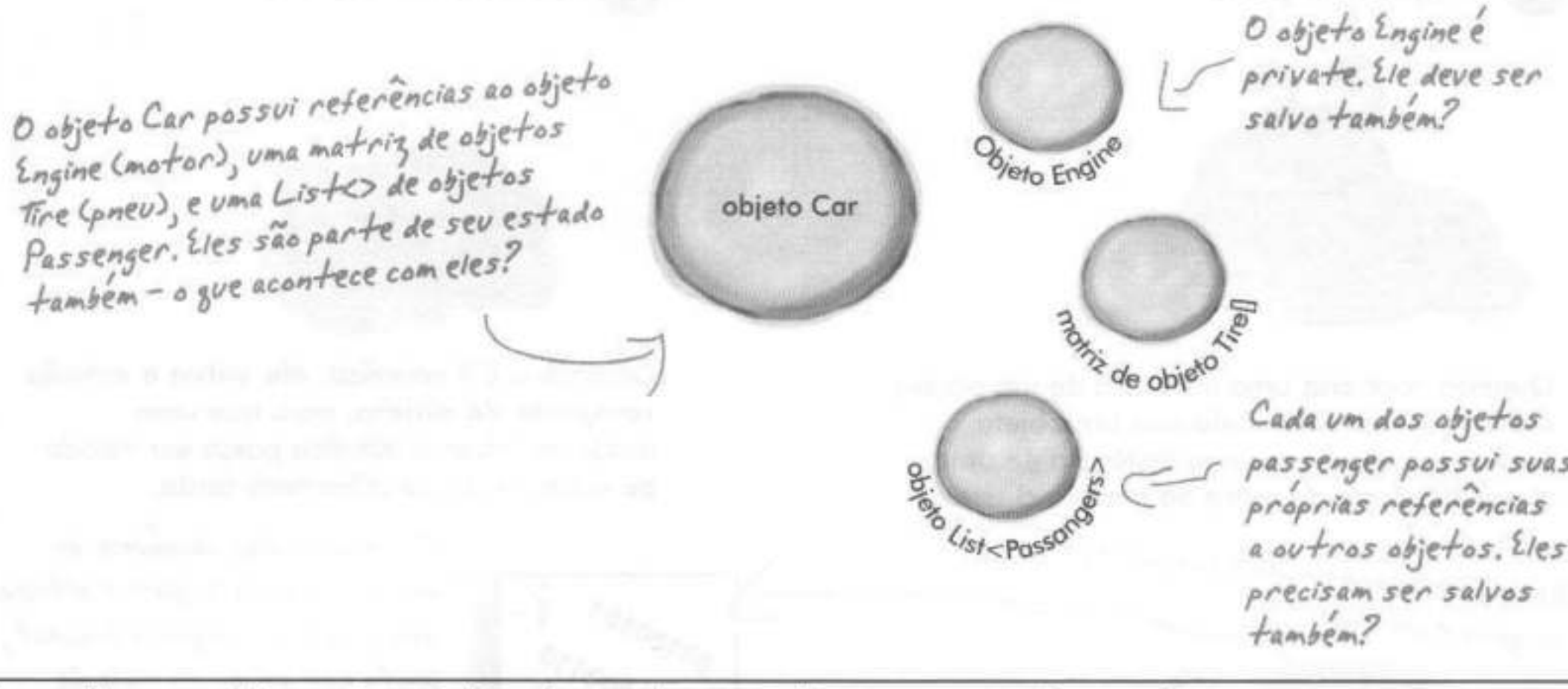
Já sabemos que **um objeto armazena seu estado em seus campos**. Então quando um objeto é serializado, cada um desses campos precisa ser salvo no arquivo.

A serialização começa a ficar interessante quando você tem objetos mais complicados. 37 e 70 são bytes – eles são tipos de valores, então podem apenas ser escritos em um arquivo literalmente. Mas e se um objeto possuir uma variável de instância que for uma referência? E se for um objeto com cinco variáveis de instância que são referências? E se essas mesmas variáveis de instância de objeto possuírem variáveis de instância?

Pense nisso por um minuto. Qual parte de um objeto é potencialmente única? Imagine o que precisa ser restaurado para obter-se um objeto idêntico àquele salvo. De alguma forma tudo na pilha deve ser escrito no arquivo.

Exercitando o Cérebro

O que deve acontecer para o objeto Car ser salvo para que seja restaurado ao seu estado original? Digamos que o carro possui três passageiros, um motor 3.0 e pneus radiais para todo tipo de terreno... essas coisas não são parte do estado? O que deve acontecer com elas?



Quando um objeto é serializado, todos os objetos ao qual se refere são serializados também...

...e todos os objetos aos quais eles se referem, e todos aqueles a que os últimos se referem, e assim por diante. Mas não se preocupe – pode soar complicado, mas tudo isso acontece automaticamente. O C# começa com o objeto que você deseja serializar e procura em seus campos por outros objetos. Então ele faz o mesmo para todos eles. Cada objeto é escrito no arquivo, junto com todas as informações que o C# precisa para reconstituir todos eles quando se reverte a serialização do objeto.

Serialização permite ler ou escrever um objeto inteiro de uma vez

Você não está limitado a ler e escrever linhas de texto em seus arquivos. Você pode usar **serialização** para permitir seus programas copiarem objetos inteiros em arquivos e depois lerem-nos de volta... tudo em apenas algumas linhas de código! Há um pouco de trabalho de preparação a ser feito – **adicione uma linha [Serializable] no topo da classe a ser serializada** – mas depois disto, tudo está pronto para ser escrito.

É rápido salvar um objeto num arquivo ou le-lo de um. Você pode serializá-lo ou reverter uma serialização.

Você precisará de um objeto BinaryFormatter

Para serializar um objeto – qualquer um – a primeira coisa a fazer é criar uma instância de `BinaryFormatter`. É bem simples: tudo de que é preciso é um linha de código (e uma linha `using` extra no topo do arquivo da classe).

```
using System.Runtime.Serialization.Formatters.Binary;
...
BinaryFormatter formatter = new BinaryFormatter();
```

Agora apenas crie uma stream e leia ou escreva seus objetos

Use o método **Serialize()** do objeto BinaryFormatter para escrever qualquer objeto em uma stream.

```
Stream output = File.Create(filenameString);
formatter.Serialize(output, objectToSerialize);
output.Close();
```

O método File.Create() cria um novo arquivo. Você pode abrir um já existente usando File.OpenWrite().

O método Serialize() possui um objeto e escreve-o na stream. Isso é muito mais fácil do que você mesmo criar um método para escrevê-lo!

Se você utilizar um comando using, Close() será chamado automaticamente.

E uma vez que você tenha um objeto serializado em um arquivo, use o método **Deserialize()** (a tradução seria algo como "desserializar") do objeto BinaryFormatter para lê-lo de volta. O método retorna uma referência. Logo, você precisa fazer coerção do resultado para que ele corresponda ao tipo da variável de referência para a qual você está copiando.

```
Stream input = File.OpenRead(filenameString);
SomeObj obj = (SomeObj)formatter.Deserialize(input);
input.Close();
```

Quando você usa Deserialize() para ler um objeto de volta de uma stream, não se esqueça de fazer a coerção do valor de retorno para corresponder ao tipo de objeto que você está lendo.

Para sua classe ser serializada, marque-a com o atributo [Serializable]

Um **atributo** é um marcador especial que você pode adicionar ao topo de qualquer classe C#. É assim que o C# armazena **metadados** sobre seu código, ou informação sobre como o código deve ser usado ou tratado. Quando você **adiciona [Serializable] no topo de uma classe logo acima da declaração**, você está dizendo ao C# que ela pode ser serializada com segurança. E você só faz isso com classes que incluam campos tipos de valores (como `int`, `string` ou `enum`) ou outras classes serializáveis. Se você não adicionar o atributo à classe que você quer serializar, ou se incluir um campo com um tipo não serializável, ocorrerá um erro em seu programa quando você tentar executá-lo. **Veja você mesmo**...

Faça isso

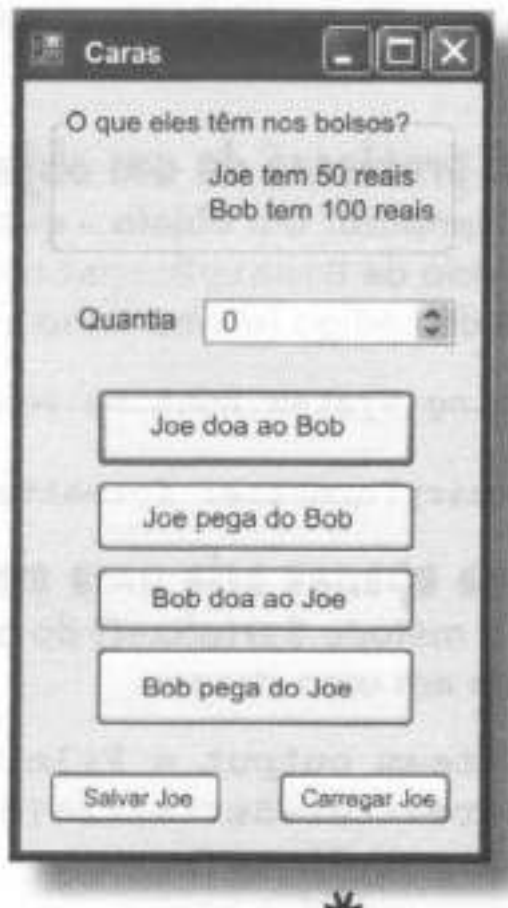
1. **Crie uma classe e a serialize**
   Lembra-se da classe Guy do capítulo 3? Vamos serializar Joe para que mantenhamos um arquivo que registre quanto dinheiro ele tem nos bolsos mesmo depois de fechar seu programa.

```
[Serializable]
  public class Guy
```

Aqui está o código para serializá-la para um arquivo chamado "Guy_file.dat" – adicione um botão "Salvar Joe" e um "Carregar Joe" ao Formulário

```
using System.IO;
using System.Runtime.Serialization.Formatters.Binary;
...
private void saveJoe_Click(object sender, EventArgs e)
{
  using (Stream output = File.Create("Guy_File.dat")) {
    BinaryFormatter formatter = new BinaryFormatter();
    formatter.Serialize(output, Joe);
  }
}
private void loadJoe_Click(object sender, EventArgs e)
{
  using (Stream input = File.OpenRead("Guy_File.dat")) {
    BinaryFormatter bf = new BinaryFormatter();
    Joe = (Guy)bf.Deserialize(input);
  };
  CheckGuys();
}
```

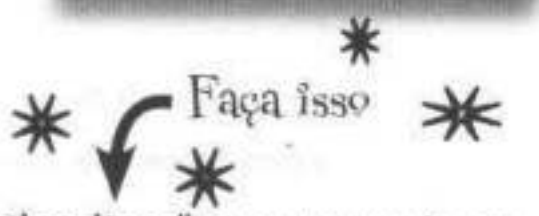
Você precisará dessas duas linhas using. A primeira é para os métodos do arquivo e da stream e a segunda, para a serialização.



2. **Execute o programa e brinque com ele.**
Se o Joe tivesse R$ 200 guardados de suas transações com o Bob enquanto se executa o programa, seria uma pena perdê-lo só porque você precisou desligar. Agora seu programa pode salvar Joe em um arquivo e restaurá-lo quando você quiser.

Vamos serializar e reverter um baralho

Pegue um baralho e escreva-o em um arquivo. O C# facilita muito a serialização de objetos. Tudo a fazer é criar uma stream e escrever seus objetos.



1. **Crie um novo projeto e adicione as classes Deck e Card**
Clique com o botão direito no Navegador de Soluções e escolha "Add/Existing Item", e acrescente as classes `Card` e `Deck` criadas no capítulo 7. Você também precisa adicionar as duas classes comparadoras de carta, já que `Deck` as usa. O IDE copiará os arquivos em um novo projeto – certifique-se de que você alterou a linha `namespace` no topo de cada arquivo de classe para corresponderem ao novo namespace de seu projeto.

2. **Marque todas as classes como serializáveis**
Adicione o atributo [`Serializable`] em todas as classes adicionadas ao projeto.

Se você não fizer isso, o C# não permitirá serializar as classes em um arquivo.

3. **Adicione alguns métodos úteis ao formulário**
O método RandomDeck cria uma baralho aleatório e o método PrintCards distribui todas as cartas e imprime-as no console.

```
Random random = new Random();
private Deck RandomDeck(int Number) {
  Deck myDeck = new Deck(new Card[] { });
  for (int i = 0; i < Number; i++)
  {
    myDeck.Add(new Card(
      (Card.Suits)random.Next(4),
      (Card.Values)random.Next(1, 14)));
  }
  return myDeck;
}
private void DealCards(Deck DeckToDeal, string Title) {
  Console.WriteLine(Title);
  while (DeckToDeal.Count > 0)
  {
    Card nextCard = DeckToDeal.Deal();
    Console.WriteLine(nextCard.Name);
  }
  Console.WriteLine("------------------");
}
```

Isso cria um baralho vazio e adiciona algumas cartas aleatórias a ele usando a classe Card do último capítulo.

O método DealCards() distribui cada uma das cartas do baralho e imprime-as no console.

4 **Certo, o trabalho de preparação está pronto... agora serialize o baralho**
Comece adicionando botões para serializar um baralho aleatório em um arquivo e lê-lo de volta. Cheque a saída do console, assegurando-se de que o baralho escrito é o mesmo lido.

```
private void button1_Click(object sender, EventArgs e) {
  Deck deckToWrite = RandomDeck(5);
  using (Stream output = File.Create("Deck1.dat")) {
    BinaryFormatter bf = new BinaryFormatter();
    bf.Serialize(output, deckToWrite);
  }
  DealCards(deckToWrite, "What I just wrote to the file");
}
private void button2_Click(object sender, EventArgs e) {
  using (Stream input = File.OpenRead("Deck1.dat")) {
    BinaryFormatter bf = new BinaryFormatter();
    Deck deckFromFile = (Deck)bf.Deserialize(input);
  }
  DealCards(deckFromFile, "What I read from the file");
}
```

BinaryFormatter pega qualquer objeto marcado com o atributo Serializable – neste caso um objeto Deck – e escreve-o numa stream usando seu método Serialize().

O método Deserialize() de BinaryFormatter retorna um Object, que é apenas o tipo geral do qual todo objeto C# herda – razão pela qual precisamos fazer a coerção dele em um objeto Deck.

5 **Agora serialize vários baralhos no mesmo arquivo**
Depois de abrir uma stream, você pode escrever o quanto quiser nela. Você pode serializar quantos objetos precisar no mesmo arquivo. Então agora adicione mais dois botões para escrever um número aleatório de baralhos no arquivo. Cheque a saída para ter certeza de que tudo está bem.

```
private void button3_Click(object sender, EventArgs e) {
  using (Stream output = File.Create("Deck1.dat")) {
    BinaryFormatter bf = new BinaryFormatter();
    for (int i = 1; i <= 5; i++) {
      Deck deckToWrite = RandomDeck(random.Next(1,10));
      bf.Serialize(output, deckToWrite);
      DealCards(deckToWrite, "Deck #" + i + " written");
    }
  }
}
private void button4_Click(object sender, EventArgs e) {
  using (Stream input = File.OpenRead("Deck1.dat")) {
    BinaryFormatter bf = new BinaryFormatter();
    for (int i = 1; i <= 5; i++) {
      Deck deckToRead = (Deck)bf.Deserialize(input);
      DealCards(deckToRead, "Deck #" + i + " read");
    }
  }
}
```

Você pode serializar um objeto após outro na mesma stream.

Note como a linha que lê um único baralho do arquivo usa (Deck) para fazer coerção da saída de Deserialize() para um Deck. Isso porque Deserialize() retorna um objeto, mas não necessariamente sabe que tipo de objeto.

Contanto que você faça coerção para o tipo certo dos objetos que ler na stream, não há limites para o número deles que se pode serializar.

6 **Veja o arquivo que você escreveu**
Abra Deck1.dat no Bloco de Notas. Pode não ser o tipo de leitura que você faria na praia, mas ele possui todas as informações para restaurar o seu baralho.

Espere um pouco. Não sei se gosto da idéia de escrever vários objetos em um arquivo estranho, que parece lixo quando eu abro. Quando escrevi o baralho como seqüências de caracteres, pude abrir a saída no Bloco de Notas e ver tudo nele. O C# não deveria facilitar o entendimento de tudo que estou fazendo?

Quando você serializa objetos em um arquivo, eles são escritos em formato binário.

Mas isso não significa indecifrável – ele é apenas compacto. É por isso que você pode reconhecer as strings quando abre um arquivo com objetos serializados: essa é a forma mais compacta que o C# pode escrever strings em um arquivo – como strings. Mas escrever um número como uma seqüência realmente seria desperdício de recursos. Qualquer int pode ser armazenado em quatro bytes. Então seria estranho se o C# armazenasse, digamos, o número 49.369.144 como uma seqüência de 8 caracteres que você pudesse ler – 10 caracteres se você incluir os pontos. Isso seria desperdício de espaço!

Nos Bastidores

O .NET usa Unicode para converter um caracter ou uma seqüência deles em bytes. Felizmente, o Windows tem uma ferramenta útil para ajudar-nos a entender como o unicode funciona. Abra o Mapa de Caracteres (menu Iniciar/Acessórios; ou Iniciar/Executar e digite "charmap.exe").

Quando você vê todas as letras e símbolos usados em linguagens pelo mundo, você percebe quantas coisas diferentes precisam ser escritas em um arquivo só para armazenar texto. É por isso que o .NET **encoda** todas essas strings e caracteres com um formato chamado Unicode. Encodar apenas quer dizer obter os dados lógicos (como a letra H) e transformá-los em bytes (o número 72). Ele precisa fazer isso porque letras, números, enums e outros dados são transformados em bytes no disco ou na memória. E é por isso que o Mapa de Caracteres é útil – ele mostra como as letras são encodadas em números.

Selecione a fonte Arial e arraste a barra para baixo até chegar às letras hebraicas. Encontre a letra Shin e clique nela.

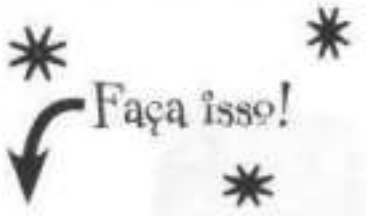
Assim que clicar na letra, seu número Unicode aparecerá na barra de status. A letra hebraica Shin é o número 05E9. Isso é um número hexadecimal – hex para abreviar.

Você pode convertê-lo em decimal usando a calculadora do Windows: abra-a, coloque no modo científico, clique no botão de rádio Hex, digite 05E9 e então clique em Dec – o resultado é 1.513.

Unicode é um padrão industrial desenvolvido por um grupo sem fins lucrativos chamado Consórcio Unicode (Unicode Consortium). Ele funciona em programas e plataformas de computadores diferentes. Reserve um minuto para visitar o site deles: **http://www.unicode.org/**

O .NET converte texto em Unicode automaticamente

Os dois tipos de valor do C# para armazenar texto – `string` e `char` – mantêm seus dados na memória em Unicode. Quando eles são escritos como bytes em um arquivo, cada um dos números em Unicode é escrito no arquivo. Comece um projeto novo e arraste três botões para o formulário. Usaremos os métodos File.WriteAllBytes() (escrever todos os bytes) e ReadAllBytes() (ler todos os bytes) para ter uma noção exata de como os dados Unicode são escritos em um arquivo.



1 **Escreva uma seqüência de caracteres normal em um arquivo e leia de volta**
Use o mesmo método WriteAllText() do editor de texto para fazer o primeiro botão escrever "Eureka!" no arquivo "eureka.txt". Então crie uma nova matriz de byte chamada eurekaBytes, leia o arquivo nela e imprima todos os bytes lidos:

```
File.WriteAllText("eureka.txt", "Eureka!");
byte[] eurekaBytes = File.ReadAllBytes("eureka.txt");
foreach (byte b in eurekaBytes)
  Console.Write("{0} ", b);
Console.WriteLine();
```

O método ReadAllBytes() retorna uma referência em uma nova matriz com todos os bytes lidos do arquivo.

Você verá os seguintes bytes: `69 117 114 101 107 97 33`. Agora abra o arquivo no Editor de Texto Simples escrito neste mesmo capítulo. Ele diz "Eureka!".

2 **Faça o segundo botão exibir os bytes como números hexadecimais**
Não é apenas o Mapa de Caracteres que mostra os números como hexa. Quase tudo que você ler se relaciona com encodar dados que os exibirão em hexa. Por isto, é útil saber como trabalhar com eles. Faça o código para o tratador de evento do segundo botão em seu programa idêntico ao primeiro; altere apenas a linha Console.Write() para que ela fique assim:

```
Console.Write("{0:x2} ", b);
```

Hexa usa números do 0 até o 9 e as letras de A a F para representar números de base 16, então 6B é igual a 107.

Isso diz ao Write() para imprimir o parâmetro #0 (o primeiro após a seqüência de caracteres para imprimir) como um código hexa de dois caracteres. Então ele escreve os mesmos sete bytes em hexa, em vez de decimal: `45 75 72 65 6b 61 21`

3 **Faça o terceiro botão escrever em letras hebraicas**
Volte ao Mapa de Caracteres e clique duas vezes no caractere Shin (ou clique no botão Selecionar). Isso irá adicioná-lo à caixa "Caracteres a copiar". Então faça o mesmo com o restante das letras em "Shalom": Lamed (U+05DC), Vav (U+05D5) e o Mem final (U+05DD). Agora adicione o código para tratador de evento do terceiro botão. Ele será idêntico ao do botão 2, exceto por uma mudança. Clique no botão "Copiar", no Mapa de Caracteres, e cole as letras sobre "Eureka!" para que a linha fique assim:

```
File.WriteAllText("eureka.txt", "שלום");
```

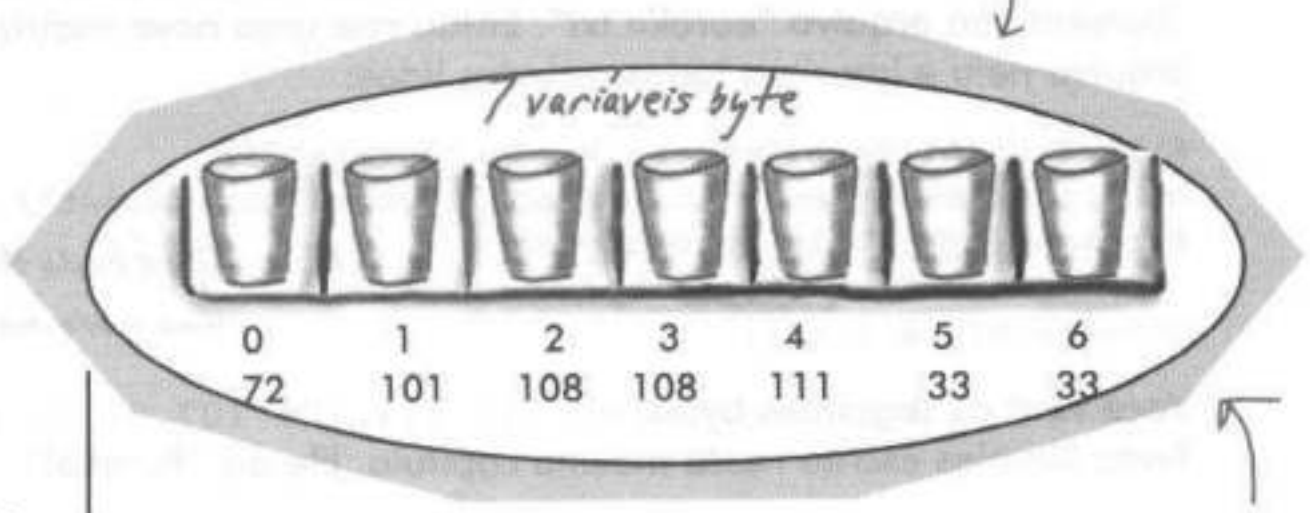
Você notou que o IDE colou as letras de trás pra **frente**? É porque ele sabe que a escrita hebraica é lida da direita para a esquerda. Assim, quando ele encontra letras hebraicas em Unicode, exibe-as da direita para a esquerda. Coloque o cursor no meio das letras – as setas direcionais esquerda e direita invertem-se! Isso facilita muito se você precisar escrever em hebraico. Agora execute o código e observe atentamente o resultado: `ff fe e9 05 dc 05 d5 05 dd 05`. Os primeiros dois caracteres são "FF FE", que é uma forma em Unicode de dizer que teremos uma seqüência de caracteres de dois bytes. O restante dos bytes são as letras hebraicas – mas elas estão invertidas, então U+05E9 aparece como **e9 05**. Agora abra o arquivo em seu Editor de Texto Simples – tudo parece certo!

O C# pode usar matrizes de byte para mover dados

Já que todos os seus dados são encodados como bytes, faz sentido pensar em um arquivo como uma grande matriz de bytes. E você já sabe como ler e escrever matrizes de byte.

Aqui está o código para criar uma. Abra uma stream de entrada e leia os dados em bytes para as posições de 0 a 6 da matriz.

```
byte[] greeting;
greeting = File.ReadAllBytes(filename);
```

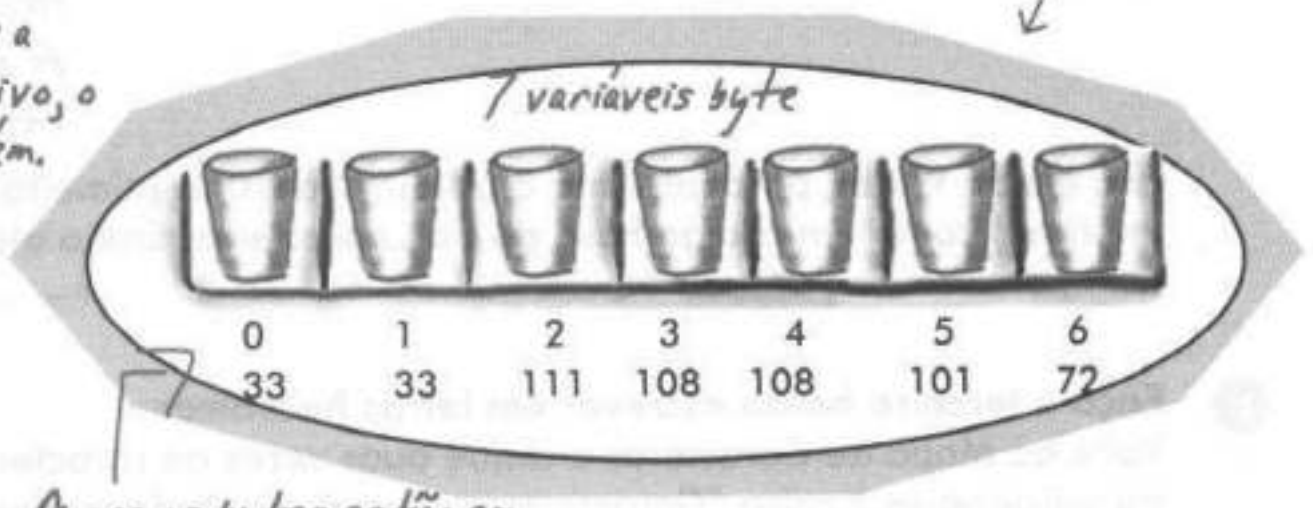


Esses números são os códigos Unicode para os caracteres em Olá!!

Este é um método estático para matrizes que inverte a ordem dos bytes. Só estamos usando-o para mostrar que as alterações na matriz de byte são escritas de forma idêntica no arquivo.

```
Array.Reverse(greeting);
File.WriteAllBytes(filename, greeting);
```

Quando o programa escreve a matriz de byte em um arquivo, o texto fica invertido também.



Agora os bytes estão em ordem inversa.

Use um BinaryWriter para escrever dados em binário

Você poderia encodar todas as suas seqüências de caracteres, gráficos, ints e floats em matrizes de byte antes de escrevê-los em arquivos, mas isso seria muito tedioso. É por isso que o .NET tem uma classe muito útil: **BinaryWriter encoda dados automaticamente** e escreve-os em um arquivo. Tudo a fazer é criar uma FileStream e passá-la ao construtor da BinaryWriter. Então você pode chamar seus métodos para escrever os dados. Acrescente outro botão ao seu programa e mostraremos como usar BinaryWriter().

1 Comece criando dados para escrever num arquivo.

```
int intValue = 48769414;
string stringValue = "Hello!";
byte[] byteArray = { 47, 129, 0, 116 };
float floatValue = 491.695F;
char charValue = 'E';
```

Se você usar File.Create(), um novo arquivo será criado- se já houver um lá, ele irá apaga-lo e iniciar um novo. Há também o método File.OpenWrite(), que abre o arquivo existente e começa a sobrescrever o que foi feito desde o início.

2 Para usar uma BinaryWriter, primeiro você precisa abrir uma nova stream com File.Create():

```
using (FileStream output = File.Create("binarydata.dat")) {
  BinaryWriter writer = new BinaryWriter(output);
```

3 Agora basta chamar seu método Write(). Cada vez que você o fizer, ele adiciona novos bytes ao final do arquivo com uma versão encodada de qualquer dado passado como um parâmetro.

```
writer.Write(intValue);
writer.Write(stringValue);
writer.Write(byteArray);
writer.Write(floatValue);
writer.Write(charValue);
```

Cada comando Write() encoda um valor em bytes e envia-o ao objeto FileStream. Você pode passar qualquer tipo de valor e o comando irá encodá-lo automaticamente.

FileStream escreve os bytes no final do arquivo.

Aponte seu lápis

4 Agora use o mesmo código de antes para ler o arquivo que acabou de escrever.

```
  byte[] dataWritten = File.ReadAllBytes("binarydata.dat");
  foreach (byte b in dataWritten)
    Console.Write("{0:x2} ", b);
  Console.WriteLine(" - {0} bytes", dataWritten.Length);
}
```

Uma dica: strings podem ter tamanhos variados, por isto a seqüência deve começar com um numero para informar ao .NET qual é o seu tamanho. Além disso, você pode procurar pelos valores Unicode da seqüência e caracteres usando o Mapa de Caracteres.

Escreva o resultado nas lacunas abaixo. Você pode determinar quais bytes correspondem a cada um dos cinco comandos Write()? Marque cada grupo de bytes com o nome da variável.

__ - ___ bytes

Aponte seu lápis
Solução

Valores float e int ocupam quatro bytes quando você os escreve em um arquivo. Se você usou long ou double, eles ocuparão oito bytes cada.

```
86 29 e8 02 06 48 65 6c 6c 6f 21 2f 81 00 74 f6 d8 f5 43 45 - 20 bytes
```

intValue stringValue byteArray floatValue charValue

O primeiro byte na seqüência é 6 – esse é o comprimento dela. Você pode usar o Mapa de Caracteres para procurar cada um dos caracteres em Olá! – ele começa com U+0048 e termina com U+0021.

Se você usar a calculadora do Windows para converter esses bytes de hexa para decimal, você verá que estes são os números em byteArray.

Um char armazena um caracter Unicode e E ocupa apenas um byte – é encodado como U+0045.

Use BinaryReader para ler os dados de volta

A classe BinaryReader funciona exatamente como BinaryWriter. Você cria uma stream, anexa-lhe o objeto BinaryReader e chama seus métodos. **Mas o leitor não sabe quais dados estão no arquivo!** E não tem como saber. Seu valor de ponto flutuante de `491.695F` foi encodado como `d8 f5 43 45`. Mas esses mesmos bytes são uma int perfeitamente válida – 1.140.185.334. Então você precisará dizer à BinaryReader exatamente quais tipos ela deve ler do arquivo. Acrescente mais um botão ao seu formulário e faça-o ler os dados que você acabou de escrever.

Não precisa acreditar na gente. Substitua a linha que lê o float com uma chamada para ReadInt32. (Você precisará alterar o tipo de floatRead para int). Então você pode ver por você mesmo o que ele lê do arquivo.

1. Comece configurando os objetos FileStream e BinaryReader:

```
using (FileStream input = File.OpenRead("binarydata.dat")) {
  BinaryReader reader = new BinaryReader(input);
```

2. Você informa a BinaryReader que tipo de dados ler chamando seus diferentes métodos.

```
  int intRead = reader.ReadInt32();
  string stringRead = reader.ReadString();
  byte[] byteArrayRead = reader.ReadBytes(4);
  float floatRead = reader.ReadSingle();
  char charRead = reader.ReadChar();
}
```

Cada tipo de valor possui seu próprio método em BinaryReader() que retorna os dados no tipo correto. A maioria não precisa de nenhum parâmetro, mas ReadBytes() possui um que informa quantos bytes ela deve ler.

3. Você informa a BinaryReader que tipo de dados ler chamando seus diferentes métodos.

```
Console.Write("int: {0} string: {1} bytes: ", intRead, stringRead);
foreach (byte b in byteArrayRead)
  Console.Write("{0} ", b);
Console.Write(" float: {0} char: {1} ", floatRead, charRead);
```

Aqui está o resultado que é impresso no console:

```
int: 48769414 string: Hello! bytes: 47 129 0 116 float: 491.695 char: E
```

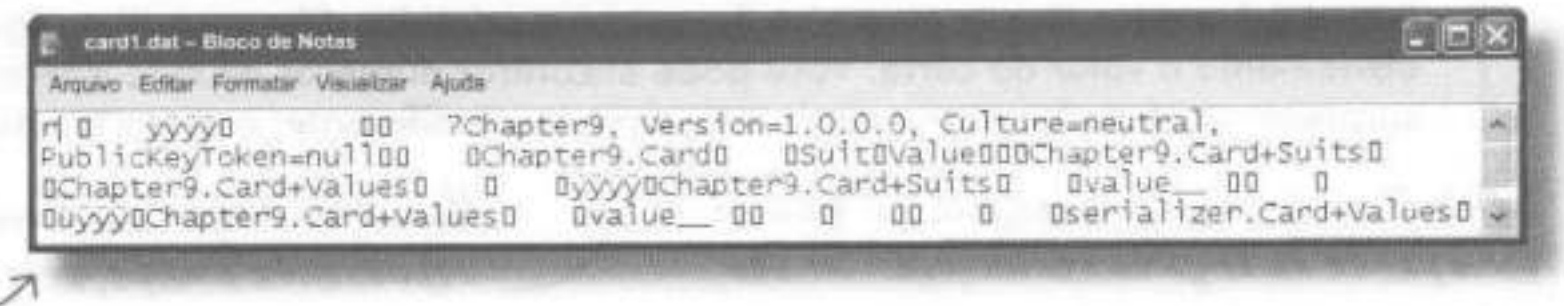
Você pode ler e escrever arquivos serializados manualmente também

Arquivos serializados não parecem tão bonitos quando você os abre no Bloco de Notas. Você encontrará todos os arquivos já escritos na pasta "bin/Debug" de seu projeto – vamos parar por um minuto para nos familiarizar com a estrutura interna de um arquivo serializado.

Faça Isso!

1 **Serialize dois objetos Card em arquivos diferentes**
Use o código de serialização já escrito para serializar os **Três de Paus** em `card1.dat` e o **Seis de Copas** em `card2.dat`. Cheque para ter certeza de que ambos foram escritos, estejam agora em uma pasta e ambos tenham o mesmo tamanho de arquivo. Então, abra um deles no Bloco de Notas:

Há algumas palavras no arquivo (como Chapter9, o namespace usado), mas a maioria é ilegível.



2 **Escreva um laço para comparar os dois arquivos binários**
Usamos o método ReadByte() para ler o próximo byte de uma stream – ele retorna um int com o valor desse byte. Também usamos o campo Length da stream para com certeza ler todo o arquivo.

```
byte[] firstFile = File.ReadAllBytes("card1.dat");
byte[] secondFile = File.ReadAllBytes("card2.dat");
for (int i = 0; i < firstFile.Length; i++)
  if (firstFile[i] != secondFile[i])
    Console.WriteLine("Byte #{0}: {1} versus {2}",
      i, firstFile[i], secondFile[i]);
```

Esse loop examina o primeiro byte de cada um dos arquivos e compara-os. A partir daí, examina o segundo, o terceiro e etc. Quando ele encontra uma diferença, escreve uma linha no console.

Os dois arquivos são lidos em duas matrizes de byte diferentes e, por isso, podem ser comparados byte por byte. Já que a mesma classe foi serializada em dois arquivos diferentes, eles serão quase idênticos... mas vejamos o QUÃO idêntico eles são.

Veja bem!

Quando você escreve em um arquivo, nem sempre começa do zero!

Tome cuidados se você usar File. OpenWrite(). Ele não apaga o arquivo – apenas começa a reescrever os dados partindo do inicio. É por isso que estamos usando o File.Create() – ele cria um novo arquivo.

Ainda não terminamos – vire a página!

Encontre as diferenças entre os arquivos e use essa informação para alterá-los

O laço que você acabou de escrever identifica exatamente onde os dois arquivos serializados Card diferem. Já que as únicas diferenças entre os dois objetos eram seus campos Suit e Value, então essa deveria ser a única diferença entre seus arquivos também. Logo, se encontrarmos os bytes responsáveis pelo naipe e valor, seremos capazes de **alterá-los para criar um novo card** com qualquer suit e value que desejarmos!

3 **Dê uma olhada no resultado do console para ver como os dois arquivos diferem**
O console deve mostrar que dois bytes diferem:

```
Byte #218: 1 versus 3
Byte #266: 3 versus 6
```

Isso faz muito sentido! Volte ao enum `Card.Suits` do capítulo anterior e verá que o valor para Clubs (paus) é 1 e para Hearts (copas) é 3 – esta é a primeira diferença. E a segunda – seis versus três – é obviamente o valor da carta. Você pode encontrar números de byte diferentes, o que não é nenhuma surpresa: você pode estar usando um namespace diferente, o que alteraria o tamanho do arquivo.

Lembra-se de como o namespace foi incluído como parte do arquivo serializado? Se seu namespace for diferente, os números de byte serão outros também.

Humm, se o byte 218 no arquivo serializado representa o naipe, deveríamos poder alterá-lo lendo esse arquivo, alterando o único byte e escrevendo-o novamente. (Lembre-se, seu próprio arquivo serializado pode armazenar o naipe num local diferente).

4 **Escreva código para criar manualmente um novo arquivo com um Rei de Espadas**
Usaremos umas das matrizes lidas. Vamos alterá-la para que contenha uma nova carta e depois escrevê-la de volta.

Se você encontrar números de byte diferentes no passo 3, substitua-os aqui.

```
firstFile[227] = (byte)Card.Suits.Spades;
firstFile[275] = (byte)Card.Values.King;
File.Delete("card3.dat");
File.WriteAllBytes("card3.dat", firstFile);
```

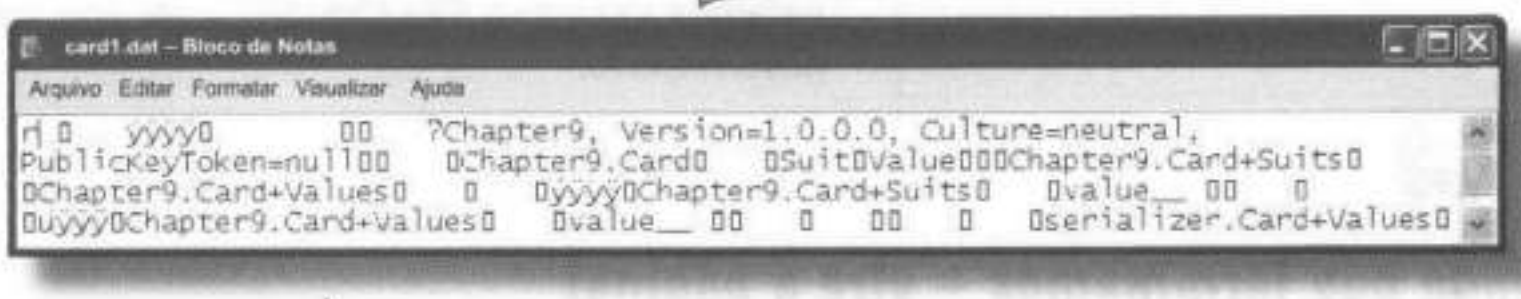
Agora **reverta a serialização da carta em card3.dat** e veja se ela é um Rei de Espadas!

Agora que você sabe quais bytes contêm naipe e valor, pode alterar somente esses bytes na matriz antes que ela seja escrita em card3.dat.

O trabalho com arquivos binários pode ter armadilhas

O que você faz se tem um arquivo cujo conteúdo você desconhece? Você não sabe qual aplicativo o criou e precisa saber algo sobre ele – mas quando você o abre no Bloco de Notas, ele parece um monte de lixo. O que aconteceria se você não tivesse mais opções e realmente precisasse olhar dentro dele? Olhando para a figura, fica bem claro que o Bloco de Notas não é a ferramenta certa.

Aqui está o card serializado, aberto no Bloco de Notas. Ele não será útil de jeito nenhum.



Você pode entender algumas coisas – como os nomes das enum (Suit e Value) e o nome do namespace que usamos (Chapter9). Mas isso não é tão útil assim.

Há outra opção – é um formato chamado de descarga hexa (hex dump, em inglês), uma forma padrão de observar dados binários. É definitivamente mais informativo que observar o arquivo no Bloco de Notas. Hexadecimal – ou "hexa" – é uma forma convencional de exibir bytes em um arquivo. Todo byte pode ser escrito com dois caracteres em hexa. Assim, você pode ver muitos dados em um espaço bem pequeno e em um formato que facilita a identificação de padrões. Além disso, é útil exibir dados binários em linhas com 8, 16 ou 32 bytes de extensão porque a maioria desses tipos de dados tendem a agrupar-se em conjuntos de 4, 8, 16 ou 32... como todos os tipos no C#. Por exemplo, um int ocupa 4 bytes e fica com 4 bytes de extensão quando é serializado no disco. Eis como o mesmo arquivo se parece quando mostrado como uma descarga hexa, usando um dos inúmeros programas gratuitos disponíveis para Windows que imprimem nesse formato:

Você pode ver imediatamente o valor numérico de cada byte no arquivo.

O número no início de cada linha é o deslocamento (offset, ou a distância do início do arquivo) do primeiro byte na linha.

Você ainda pode ver o texto original, mas os caracteres sem sentido não são mostrados.

Use streams de arquivo para criar um descarregador hexa

Uma **descarga hexa** (hex dump) é uma visualização `hexadecimal` do conteúdo de um arquivo; uma forma muito comum para programadores analisarem a fundo sua estrutura interna. A maioria dos sistemas operacionais tem um utilitário de descarga hexa embutido. Infelizmente, o Windows não. Então vamos fazer um!

Como fazer um descarregador hexa

Comece com um trecho de texto conhecido:

```
Nós, povo dos Estados Unidos, para formar uma união mais perfeita...
```

Aqui vemos como ficaria uma descarga hexa desse texto:

Repetimos, você pode ver imediatamente o valor numérico de cada byte no arquivo.

```
0000: 57 65 20 74 68 65 20 70 -- 65 6f 70 6c 65 20 6f 66   We the people of
0010: 20 74 68 65 20 55 6e 69 -- 74 65 64 20 53 74 61 74    the United Stat
0020: 65 73 2c 20 69 6e 20 6f -- 72 64 65 72 20 74 6f 20   es, in order to
0030: 66 6f 72 6d 20 61 20 6d -- 6f 72 65 20 70 65 72 66   form a more perf
0040: 65 63 74 20 75 6e 69 6f -- 6e 2e 2e 2e               ect union...
```

Acrescentaremos o número no começo de cada linha usando o deslocamento do primeiro byte nela.

E vamos precisar substituir os caracteres sem sentido por pontos.

Cada um desses números – 57, 65, 6F – é o valor de um byte no arquivo. O motivo de alguns dos "números" terem valores expressos com letras é que são `hexadecimais` (ou hexas). Essa é apenas outra forma de escrever um número. Em vez de usar dez dígitos de 0 a 9, usam-se dezesseis de 0 a 9 mais as letras de A a F.

Cada linha em nossa descarga hexa representa dezesseis caracteres na entrada usada para gerá-la. Em nossa descarga, os primeiros quatro caracteres são o deslocamento no arquivo – a primeira linha começa no caracter 0, a próxima no 16 (ou hexa 10), depois 32 (hexa 20) etc. (Outros descarregadores hexa são um pouco diferentes, mas este servirá para nós).

Trabalhando com hexa

Você pode colocar números hexa diretamente em seu programa – apenas acrescente o prefixo 0x na frente do número:

```
int j = 0x20;
MessageBox.Show("The value is " + j);
```

Quando você usar o operador + para concatenar um número numa seqüência de caracteres, ele é convertido em decimal. Você pode usar o método estático `String.Format()` para, em vez disso, converter seu número em uma seqüência de caracteres formatada em hexa:

String.Format() usa parâmetros exatamente iguais a Console.WriteLine(), ou seja, você não precisa aprender nada novo para usá-lo.

```
string h = String.Format("{0:x2}", j);
```

StreamReader e StreamWriter servirão muito bem

Nosso aplicativo escreverá sua descarga em um arquivo e, já que ele está apenas escrevendo texto, uma StreamWriter servirá. Mas também podemos aproveitar o método **ReadBlock()** (ler bloco) de StreamReader. Ele lê um bloco de caracteres em uma matriz – você especifica o número que se quer ler e ele vai lê-los ou, se houver menos do que esse número no arquivo, vai ler até o fim dele. Já que estamos exibindo 16 caracteres por linha, leremos blocos de 16 caracteres.

Acrescente mais um botão ao seu programa – adicione essa descarga hexa nele. Altere as primeiras duas linhas para que apontem para os verdadeiros arquivos em seu disco rígido. Comece com um arquivo Card serializado. Então veja se pode modificá-lo para usar as caixas de diálogo Abrir e Salvar Como.

```
using (StreamReader reader = new StreamReader(@"c:\files\inputFile.txt"))
using (StreamWriter writer = new StreamWriter(@"c:\files\outputFile.txt", false))
{
  int position = 0;
  while (!reader.EndOfStream) {
    char[] buffer = new char[16];
    int charactersRead = reader.ReadBlock(buffer, 0, 16);
    writer.Write("{0}: ", String.Format("{0:x4}", position));
    position += charactersRead;
    for (int i = 0; i < 16; i++) {
      if (i < charactersRead) {
        string hex = String.Format("{0:x2}", (byte)buffer[i]);
        writer.Write(hex + " ");
      }
      else
        writer.Write("   ");
      if (i == 7) { writer.Write("-- "); }
      if (buffer[i] < 32 || buffer[i] > 250) { buffer[i] = '.'; }
    }
    string bufferContents = new string(buffer);
    writer.WriteLine(" " + bufferContents.Substring(0, charactersRead));
  }
}
```

O campo EndofStream da StreamReader retorna false se ainda houver caracteres a serem lidos no arquivo.

Esse ReadBlock() lê até 16 caracteres em uma matriz char.

O método estático String.Format converte números em strings. {0:x4} manda imprimir o segundo parâmetro - neste caso, position - como um número hexa de 4 caracteres.

Alguns caracteres com um valor abaixo de 32 não imprimem. Assim, substituiremos todos eles por um ponto.

Você pode converter uma matriz char[] em uma string passando-a ao construtor sobrecarregado de strings.

Este laço passa pelos caracteres e imprime cada um deles em uma linha na saída.

Toda seqüência de caracteres possui um método substring que retorna uma parte da seqüência. Neste caso, ele retorna os primeiros charactersRead caracteres começando pela parte inicial - posição 0 (olhe de novo a parte superior do laço para ver onde charactersRead está - o método ReadBlock() retorna o número de caracteres que ele lê da matriz).

não existem Perguntas Idiotas

P: Por que eu não tive de usar o método Close() para fechar o arquivo depois de ter usado File.ReadAllText() e File.WriteAllText()?

R: A classe File possui diversos métodos estáticos úteis que abrem automaticamente o arquivo, lêem ou escrevem dados e então o **fecham automaticamente**. Além dos métodos `ReadAllText()` e `WriteAllText()`, há os `ReadAllBytes()` e `WriteAllBytes()` que funcionam com matrizes de byte, e os `ReadAllLines()` (ler todas as linhas) e `WriteAllLines()`, que lêem e escrevem matrizes de strings, sendo que cada seqüência na matriz será uma linha separada no arquivo. Todos eles métodos automaticamente abrem e fecham as streams, permitindo executar toda a sua operação de arquivo em um único comando.

P: Se FileStream possui métodos para ler e escrever, por que eu preciso usar StreamReader e StreamWriter?

R: A classe `FileStream` é bem útil para ler e escrever bytes em arquivos binários. Seus métodos para ler e escrever operam com bytes e matrizes de bytes. Mas muitos programas funcionam exclusivamente com arquivos de texto – como a primeira versão do Gerador de Desculpas, que só escrevia seqüências de caracteres em arquivos. É aí que `StreamReader` e `StreamWriter` vêm realmente a calhar. Eles possuem métodos específicos para ler e escrever linhas de texto. Sem eles, se você quisesse ler uma linha de texto de um arquivo, deveria primeiro ler uma matriz de bytes e então escrever um laço para procurar quebras de linha pela matriz – logo, é fácil notar como eles facilitam a sua vida.

P: Quando eu devo usar File e quando devo usar FileInfo?

R: A principal diferença entre as classes `File` e `FileInfo` é que os métodos em `File` são estáticos, não precisando, portanto, criar uma instância. Por outro lado, `FileInfo` exige que você a instancie com um nome de arquivo. Em alguns casos, isso seria mais incômodo, como quando você precisa apenas executar uma única operação de arquivo (como, por exemplo, apenas deletar ou mover um arquivo). Mas se você precisa fazer muitas operações no mesmo arquivo, é mais eficiente usar `FileInfo`, porque só é preciso passar o filename uma vez. Deve-se decidir qual usar baseado em uma situação particular que se encontre. Em outras palavras, se você estiver fazendo uma operação de arquivo, use File. Se estiver fazendo muitas operações de arquivo de uma vez, use FileInfo.

P: Espere um minuto. Por que "Eureka!" foi escrito com um byte por caracter, mas quando eu escrevi as letras hebraicas elas precisaram de dois bytes? E o que era aquele negócio de "FF FE" no começo dos bytes?

R: O que você está vendo é a diferença entre duas codificações Unicode bem parecidas. Letras, números, marcas de pontuação normais e alguns caracteres padrão (como chaves, & e outras coisas que você encontra no teclado) da língua portuguesa são todos números Unicode baixos – entre 0 e 127. (Se você já usou ASCII antes, em Unicode eles são os mesmos que os caracteres ASCII). Se um arquivo contém somente esses caracteres Unicode com números baixos, apenas seus bytes são salvos.

As coisas ficam mais complicadas quando você acrescenta caracteres Unicode com números mais altos no conjunto. Um byte só pode conter um número entre 0 e 255. Mas dois bytes em seqüência podem armazenar números entre 0 e 65.535 – o que, em hexa, é FFFF. O arquivo precisa ser capaz de informar a qualquer programa que o abra que ele conterá esses caracteres com números altos. Então ele coloca uma seqüência de bytes reservada especial no começo do arquivo: "FF FE". Isso é chamado de "marca de ordem de byte" (byte order mark). Assim que o programa vê isso, ele sabe que todos os caracteres são codificados com dois bytes cada. (Então um E seria codificado como 00 45 – com zeros no começo).

P: Por que isso é chamado de marca de ordem de byte?

R: Lembra-se de como seus bytes foram invertidos? O valor Unicode de Shin de U+05E9 foi escrito no arquivo como E9 05. Isso é chamado de "little endian" *. Volte ao código que escreveu esses bytes e acrescente um terceiro parâmetro em WriteAllText(): Encoding.BigEndianUnicode. Isso diz a ele para escrever os dados em "big endian" *, o que não troca a posição dos bytes. Você verá os bytes aparecerem como "05 E9" desta vez. Você também verá uma marca de ordem de byte diferente: "FE FF". E seu Editor de Texto Simples é inteligente o bastante para ler ambos!

Se você estiver escrevendo uma string com apenas caracteres Unicode com números baixos, ela é escrita com um byte por caracter. Mas se ela possuir caracteres de números altos, eles serão escritos usando dois bytes cada.

* (NT): Trocadilho em inglês com "indiozinho" e "grande índio", oriundo de música infantil tradicional da cultura norte americana.

Uau, isso foi muito fácil! Todo o código para salvar e abrir desculpas estava dentro da classe Excuse. Eu só tive de alterar a classe – quase não precisei mexer no formulário. É como se o formulário nem ligasse sobre como a classe salva seus dados. Ela só passa o nome de arquivo e sabe que tudo será salvo corretamente.

É isso aí! Foi muito fácil alterar seu código porque a classe foi bem encapsulada.

Quando você tem uma classe que esconde suas operações internas do restante do programa e só mostra o comportamento que precisa ser exposto, ela é chamada de **uma classe bem encapsulada**. No programa Gerenciador de Desculpas, o formulário não possui nenhuma informação sobre como as desculpas são salvas nos arquivos. Ele apenas passa um nome de arquivo para a classe de desculpas e ela cuida do resto. Isso facilita muito fazer grandes alterações na forma como a sua classe trabalha com arquivos. Quanto melhor você encapsular suas classes, mais fácil será para alterá-las posteriormente.

Lembra-se de como encapsulamento era um dos quatro princípios centrais da POO? Aqui temos um exemplo de como esses princípios melhoram seus programas.

Exercício

Altere o Gerador de Desculpas do Brian para que ele use arquivos binários com objetos Excuse serializados, em vez de arquivos de texto.

1. **Faça que a classe Excuse seja serializável**
   Marque a classe Excuse com o atributo [Serializable] para deixá-la serializável. Você também deverá acrescentar a linha using:

2. **Altere o método Excuse.Save() para serializar a desculpa**
   Quando o método Save() escreve um arquivo na pasta, em vez de usar StreamWriter para escrevê-lo, faça-o abrir um arquivo e serializar a própria instância da classe. Você precisará descobrir como a classe atual pode reverter a própria serialização sozinha.

Dica: Qual palavra-chave você pode usar dentro de uma classe que retorna uma referência a ela mesma?

3. **Altere o método Excuse.OpenFile() para reverter a serialização de uma desculpa**
   Você precisará criar um objeto Excuse temporário para reverter a serialização no arquivo e então copiar seus campos na instância atual.

4. **Altere o formulário para que ele use uma nova extensão de arquivo**
   Há apenas uma alteração muito pequena a ser feita no formulário. Já que não estamos mais trabalhando com arquivos de texto, não podemos mais usar a extensão .txt. Altere as caixas de diálogo, nomes de arquivos padrão e código de busca de diretório para que, em vez disso, eles funcionem com arquivos *.excuse.

Solução do Exercício

Altere o Gerador de Desculpas do Brian para que ele use arquivos binários com objetos Excuse serializados, em vez de arquivos de texto.

Você só precisa alterar essas quatro linhas no formulário: duas linhas no evento Click do botão de salvar e duas no do botão de abrir – eles só alteram os diálogos para usar a extensão .excuse.

```
private void save_Click(object sender, EventArgs e) {
 // código existente
 saveFileDialog1.Filter = "Excuse files (*.excuse)|*.excuse|All files (*.*)|*.*";
 saveFileDialog1.FileName = description.Text + ".excuse";
 // código existente
}
private void save_Click(object sender, EventArgs e) {
 // código existente
 openFileDialog1.Filter =
  "Excuse files (*.excuse)|*.excuse|All files (*.*)|*.*";
 openFileDialog1.FileName = description.Text + ".excuse";
 // código existente
}
[Serializable]
public class Excuse {
  public string Description;
  public string Results;
  public DateTime LastUsed;
  public string ExcusePath;
  public Excuse() {
    ExcusePath = "";
  }
  public Excuse(string excusePath) {
    OpenFile(ExcusePath);
  }
  public Excuse(Random random, string folder) {
    string[] fileNames = Directory.GetFiles(folder, "*.excuse");
    OpenFile(fileNames[random.Next(fileNames.Length)]);
  }
  private void OpenFile(string excusePath) {
    this.ExcusePath = excusePath;
    BinaryFormatter formatter = new BinaryFormatter();
    Excuse tempExcuse;
    using (Stream input = File.OpenRead(excusePath)) {
      tempExcuse = (Excuse)formatter.Deserialize(input);
    }
    Description = tempExcuse.Description;
    Results = tempExcuse.Results;
    LastUsed = tempExcuse.LastUsed;
  }
  public void Save(string fileName) {
    BinaryFormatter formatter = new BinaryFormatter();
    using (Stream output = File.OpenWrite(fileName)) {
      formatter.Serialize(output, this);
    }
  }
}
```

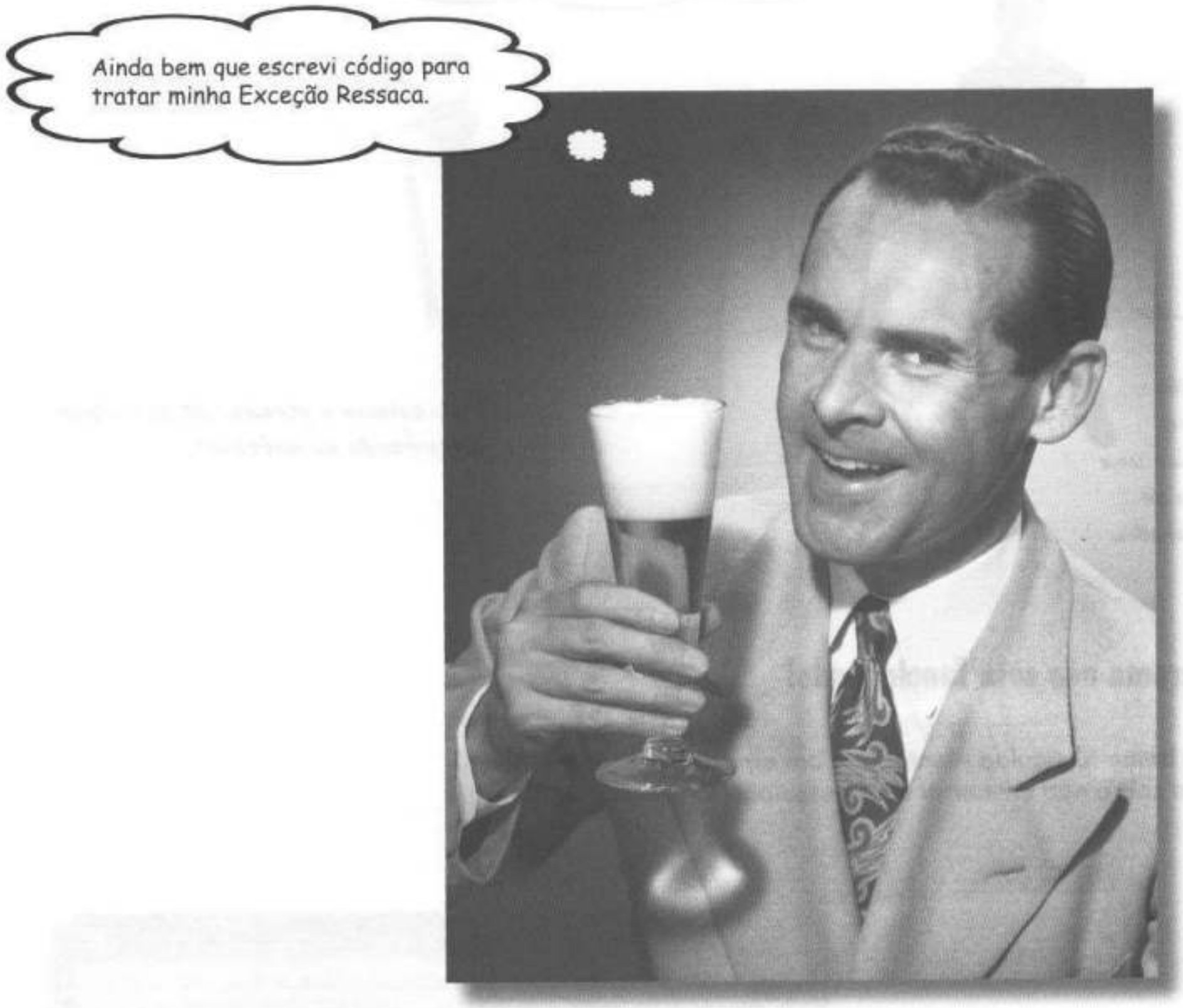
Os diálogos padrão de salvar e abrir funcionam bem aqui.

A única alteração no formulário é fazê-lo trocar a extensão do arquivo que ele passa à classe Excuse.

O construtor para carregar desculpas aleatórias precisa procurar arquivos com a extensão .excuse, em vez de .txt.

Passamos this porque queremos que esta classe seja serializada.

Apagar incêndios é coisa do passado



Programadores não são treinados para serem bombeiros.

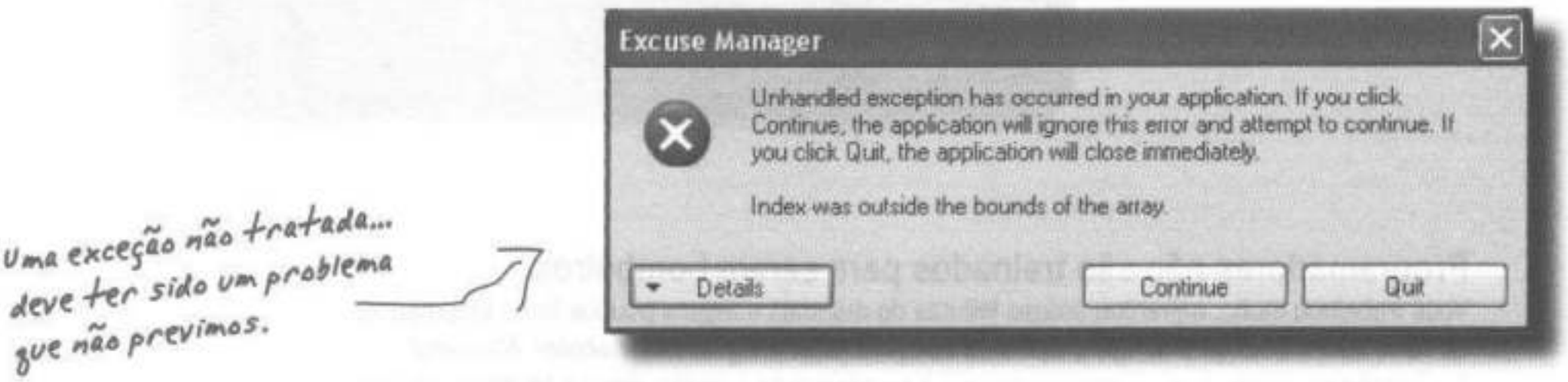
Você trabalhou muito, enfrentou árduas leituras de manuais e alguns poucos livros empolgantes da série Use a Cabeça e atingiu o ápice da sua profissão: **programador sênior**. Mas você fica recebendo notas da sua empresa porque **seu programa levanta erros e fecha** ou não **se comporta como deveria**. Nada tira mais você do caminho da programação do que ter que corrigir um erro estranho... mas com **tratamento de exceções**, você pode escrever código para **lidar com problemas** que aparecem. Ainda melhor, você pode até mesmo reagir a esses problemas e **manter as coisas funcionando**.

Aldo precisa de desculpas para passear

Aldo foi transferido recentemente para a divisão internacional. Então ele passou a viajar por todo o mundo. Mas ele ainda precisa manter controle sobre suas desculpas, por isso ele instalou o programa que você fez no notebook e leva-o para todo lado.

Mas o programa não está funcionando!

Aldo clica no botão "Desculpa Aleatória" e um erro de aparência terrível aparece. Algo sobre não encontrar suas desculpas. O que acontece?



Aponte seu lápis

Eis outro exemplo de código com problemas. Esse código lança cinco tipos diferentes de exceção e as mensagens de erro são mostradas à direita. Sua tarefa é identificar as linhas com problemas e associá-las à exceção que elas geram. Leia as mensagens das exceções para ter boas pistas do que fazer.

```
public static void BeeProcessor() {
  object myBee = new HoneyBee(36.5, "Zippo");
  float howMuchHoney = (float)myBee;

  HoneyBee anotherBee = new HoneyBee(12.5, "Buzzy");
  double beeName = double.Parse(anotherBee.MyName);

  double totalHoney = 36.5 + 12.5;
  string beesWeCanFeed = "";
  for (int i = 1; i < (int) totalHoney; i++) {
      beesWeCanFeed += i.ToString();
  }
  float f =
    float.Parse(beesWeCanFeed);

  int drones = 4;
  int queens = 0;
  int dronesPerQueen = drones / queens;

  anotherBee = null;
  if (dronesPerQueen < 10) {
      anotherBee.DoMyJob();
  }
}
```

Chamar doubleParse("32"), por exemplo, vai analisar a string e retornar um valor double, nesse caso, 32.

Quando você tem uma referência que não aponta para um objeto, ela recebe um valor especial chamado null. Ao atribuir null para uma referência indicamos ao C# que não estamos apontando para nada.

1 OverflowException não foi tratada

O valor, ou é muito grande, ou muito pequeno, para um Single.

2 NullReferenceException não foi tratada

A referência não foi apontada para uma instância de objeto.

3 InvalidCastException não foi tratada

Coerção especificada não é válida.

4 DivideByZeroException não foi tratada

Tentativa de dividir por zero

Dicas de Resolução:

Certifique-se de que o valor do denominador não seja zero antes de efetuar uma operação de divisão.

Ler textos de ajuda a respeito dessa exceção.

Procurar por mais ajuda na Internet...

5 FormatException não foi tratada

Seqüência de caracteres de entrada não está no formato correto.

Dicas de Resolução:

Make sure your method arguments are in the right format.

Quando converter uma string para DateTime, analise a string para obter a data antes de colocar cada variável no objeto DateTime.

Ler textos de ajuda a respeito dessa exceção.

Procurar por mais ajuda na Internet...

Aponte seu lápis Solução

Seu trabalho foi encontrar a linha de código com um problema e associá-la à exceção levantada.

```
object myBee = new HoneyBee(36.5, "Zippo");
float howMuchHoney = (float)myBee;
```

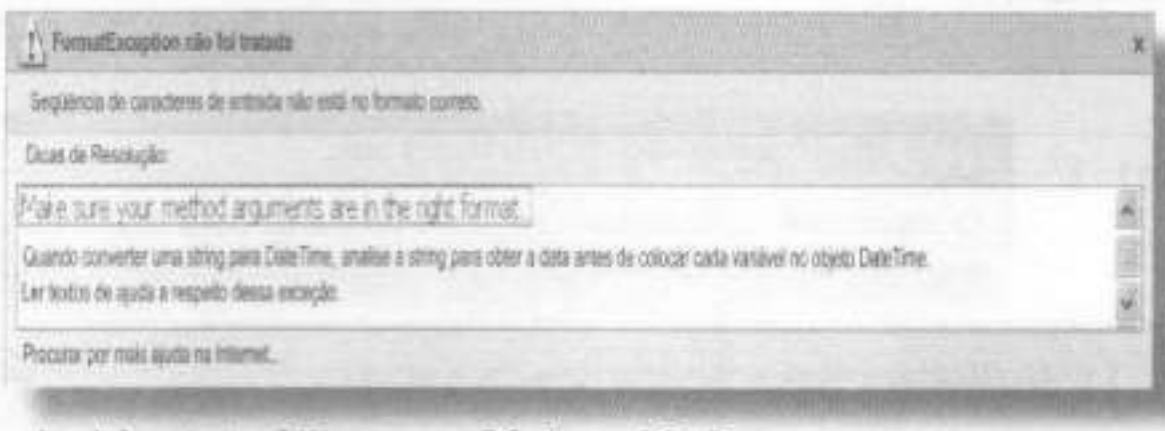
O C# não tem a menor idéia sobre como fazer coerção de HoneyBee para um float. Tentar fazer isso lançará uma InvalidCastException (exceção de coerção inválida).

InvalidCastException não foi tratada

Coerção especificada não é válida.

```
HoneyBee anotherBee = new HoneyBee(12.5, "Buzzy");
double beeName = double.Parse(anotherBee.MyName);
```

O método Parse() quer que lhe dê uma string com um certo formato. "Buzzy" não é uma seqüência de caracteres que ele saiba como converter num número. É por isso que ele lança uma FormatException (exceção de formato).




```
double totalHoney = 36.5 + 12.5;
tring beesWeCanFeed = "";
for (int i = 1; i < (int) totalHoney; i++) {
    beesWeCanFeed += i.ToString();
}
float f = float.Parse(beesWeCanFeed);
```

O laço for criará uma string chamada beesWeCanFeed (abelhas que podemos alimentar). Ela contém um número com mais de 60 dígitos. Não há forma de um float conter um número assim tão grande. Tentar espremê-lo na variável lançará uma OverflowException (exceção de estouro de limite).

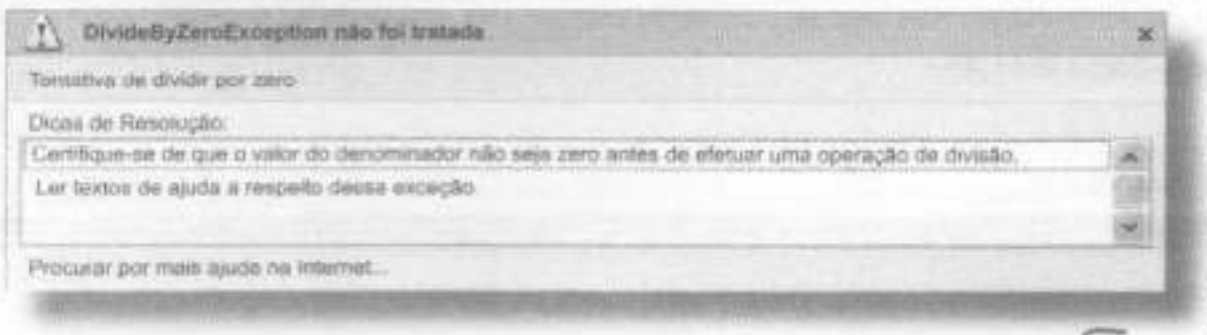
OverflowException não foi tratada

O valor, ou é muito grande, ou muito pequeno, para um Single.

Na prática não acontecem tantas exceções seguidamente. O programa lançará a primeira e então parará. Você somente verá a segunda numa seqüência se corrigir a primeira.

```
int drones = 4;
int queens = 0;
int dronesPerQueen = drones / queens;
```

É realmente fácil lançar uma DivideByZeroException (exceção divisão por zero). Apenas divida qualquer número por zero.



Dividir um número por zero sempre faz com que essa exceção seja lançada. Até sem saber o valor (de queens, no exemplo), pode impedi-la de acontecer apenas checando, para se certificar de que ele não é zero antes de dividir por ele (drones - zangão - nesse caso).

```
anotherBee = null;
if (dronesPerQueen < 10) {
    anotherBee.DoMyJob();
}
```

Atribuir null à variável de referência anotherBee (outra abelha) é o mesmo que dizer ao C#: ela não aponta para nada. Logo, em vez de apontar para um objeto, ela aponta para coisa alguma. Lançar uma NullReferenceException (exceção de referência nula) é como o C# diz-lhe que não existe um objeto com um método DoMyJob() (fazer meu trabalho), sendo impossível chamá-lo.

NullReferenceException não foi tratada

A referência não foi apontada para uma instância de objeto.

O erro de divisão por zero não deveria ocorrer nunca. Você pode ver apenas que algo está errado examinando o código. O mesmo vale para as demais exceções. Todos estes problemas poderiam ter sido evitados - e quanto mais você souber a respeito de exceções, mais será capaz de impedir seu código de levantar erros e parar.

Quando seu programa lança uma exceção, o .NET gera um objeto Exception

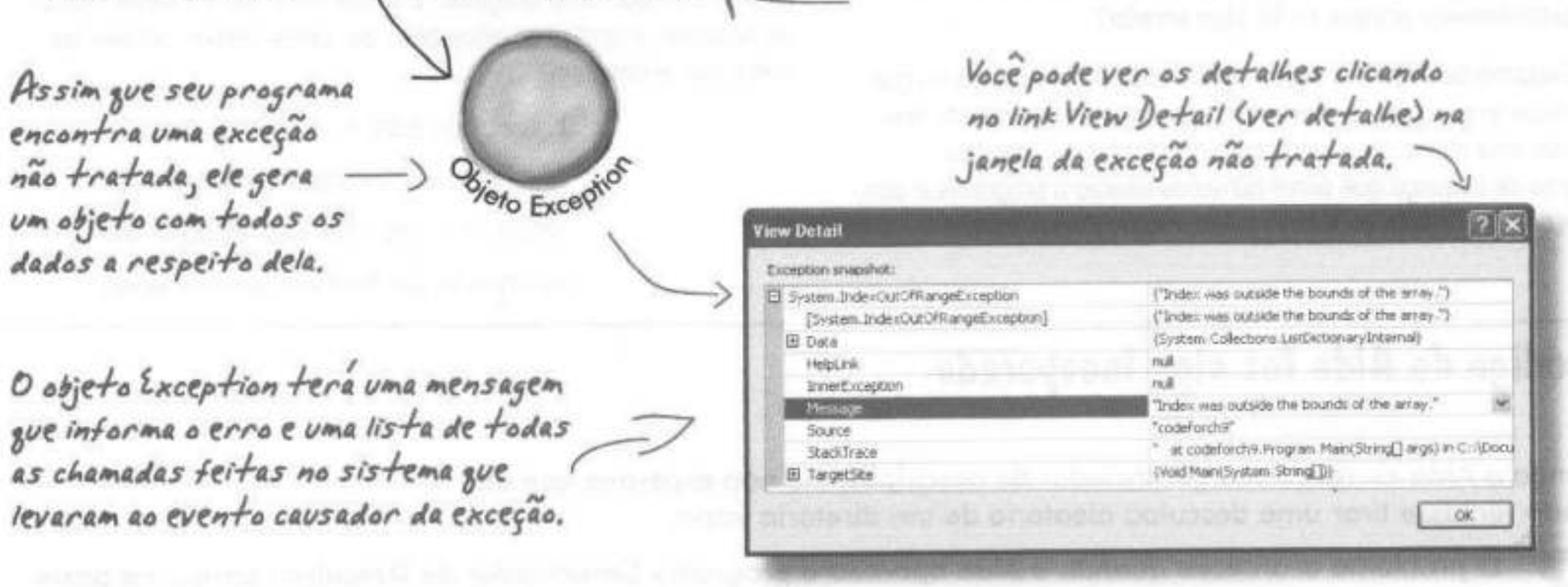
Você esteve olhando para como o .NET diz-lhe que algo de errado ocorreu com seu programa: uma **exceção**. Em C#, quando uma exceção ocorre, um objeto é criado para representar o problema. Ele é chamado, de forma nada surpreendente, de Exception.

Por exemplo, suponha que você tenha uma matriz com quatro itens. Então tenta acessar o décimo sexto (índice 15, uma vez que ela é zero-based, literalmente "baseada em zero", ou seja, começa em zero).

> **ex-ce-ção, substantivo**
> Uma pessoa ou coisa excluída do contexto geral ou não segue uma regra. *Embora Jaime normalmente detestasse doce de leite, para ela, a cobertura de doce de leite de Cinésio era uma exceção.*

```
int anArray = {3, 4, 1, 11};
int aValue = anArray[15];
```

Esse código obviamente causará problemas.



O .NET tem todo esse trabalho de criar um objeto porque ele quer lhe dar todas as informações sobre o motivo da exceção. Você pode ter de corrigir código, ou apenas alterar a forma como lida com uma situação em particular em seu programa.

Nesse caso, o objeto **IndexOutOfRangeException** (exceção índice fora dos limites) indica um erro de programação: você está tentando acessar um índice na matriz fora de seus limites. Você também recupera informações sobre exatamente onde no código o problema ocorreu, o que facilita rastreá-lo (mesmo com milhares de linhas de código).

não existem Perguntas Idiotas

P: Porque há tantos tipos de exceção?

R: Há muitas formas de escrever-se código que o C# não sabe como lidar. Seria difícil resolver os problemas se o programa apenas mostrasse uma mensagem de erro genérica (por exemplo, "um erro ocorreu na linha 37"). É muito mais fácil rastrear e corrigir um problema no código quando você sabe especificamente o tipo de erro ocorrido.

P: Na verdade, o que é uma exceção?

R: É um objeto que o .NET cria quando um problema ocorre (falaremos mais sobre isso em um minuto).

P: O quê? É um objeto?

R: Sim, uma exceção é um objeto. As suas propriedades revelam informações a respeito da exceção. Por exemplo, ele tem uma propriedade Message (mensagem) com uma seqüência de caracteres útil como "coerção especificada é inválida" ou "valor é grande demais, ou pequeno demais para um Single", que é usado para gerar a janela que mostra a exceção. O .NET gera este objeto para dar-lhe o máximo de informação possível sobre o que estava acontecendo quando o programa executou o comando responsável pela exceção.

P: Certo, mas ainda não entendi. Porque há tantos tipos de exceção; você pode explicar de novo?

R: Porque há uma mesma quantidade de formas que seu código pode executar de maneiras inesperadas. Há muitas situações que simplesmente farão seu código lançar um erro e parar. Seria

realmente muito difícil de rastrear esses problemas se você não soubesse porque o programa parou. Ao lançar diferentes tipos de exceção em circunstâncias diferentes, o .NET está dando-lhe informação de muita importância para ajudá-lo a encontrar e corrigir o problema.

P: Então exceções há para ajudarem-me, não apenas para me perturbarem?

R: Sim! Exceções têm tudo a ver com ajudar você a esperar o inesperado. Muitas pessoas ficam frustradas quando vêem o código lançar uma exceção. Mas se você pensar sobre elas como uma forma do .NET de ajudá-lo a encontrar e depurar seu programa, isso realmente ajudará quando você estiver tentando achar o que fez o código lançar um erro.

P: Então quando meu código lança uma exceção, não é obrigatoriamente porque eu fiz algo errado?

R: Exatamente. Algumas vezes seus dados são diferentes do que você esperava – como, por exemplo, se você tem um método que lida com uma matriz mais longa ou mais curta que o previsto. E não se de esqueça que seres humanos usarão o programa e eles, quase sempre, agirão de formas inesperadas. Exceções são a forma do .NET de ajudá-lo a lidar com as situações inesperadas que seu código pode encontrar para que ainda possa continuar sem problemas, em vez de fechar ou exibir alguma mensagem de erro misteriosa e inútil.

P: Uma vez que eu saiba o que estou procurando, fica bem claro que o código na página anterior lançaria erros. Todas as exceções indicam erros fáceis de achar?

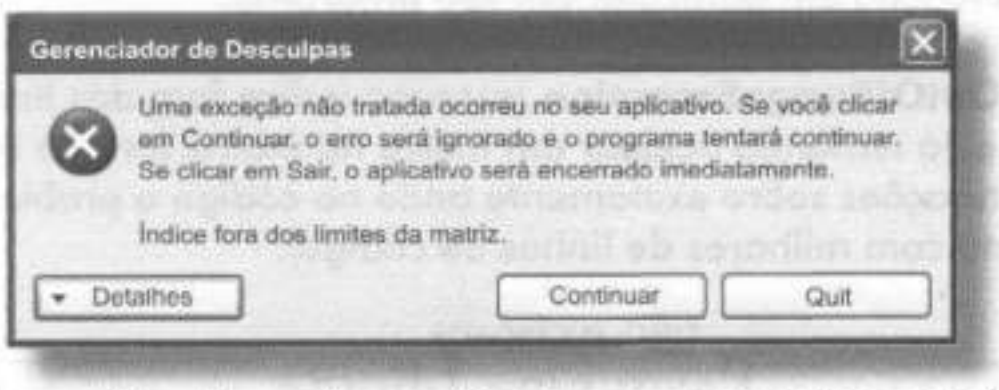
R: Não. Infelizmente, algumas vezes, quando o seu código tem problemas, será bem difícil determinar a causa apenas o examinando. É por isso que o IDE tem uma ferramenta realmente útil chamada depurador (debugger). Ela permite parar o programa e executar os comandos passo a passo, inspecionando, enquanto isso, o valor de cada variável e campo individualmente. Isso facilita muito determinar onde seu código está se comportando de uma forma diferente do que você imaginou. É aí que você tem a melhor chance de descobrir e corrigir as exceções - ou, ainda melhor, preveni-las antes que aconteçam.

Exceções tem tudo a ver com ajudar a encontrar e corrigir situações em que seu código se comporta de formas inesperadas.

O código do Aldo fez algo inesperado

Quando o Aldo escreveu seu gerenciador de desculpas, ele não esperava que um usuário tentasse tirar uma desculpa aleatória de um diretório vazio.

1. O problema aconteceu quando o Aldo apontou o programa Gerenciador de Desculpas para uma pasta vazia no notebook e clicou o botão "Aleatória". Vamos dar uma olhada e ver se conseguimos perceber o erro. Eis a janela da exceção não tratada exibida quando executamos o programa fora do IDE.



2. Certo, é um bom ponto de partida. Ele está nos dizendo que um índice está fora dos limites da matriz, certo? Então vamos procurar por uma matriz no código do tratador de evento do botão "Desculpa Aleatória":

```
private void RandomExcuseButton_Click(object sender, EventArgs e) {
  if (CheckChanged() == true) {
    CurrentExcuse = new Excuse(random, Folder);
    UpdateForm(false);
  }
}
```

3. Hum, não temos matrizes aqui. Mas ele cria um novo objeto Excuse usando um dos construtores sobrecarregados. Talvez exista uma matriz no código do construtor:

```
public Excuse(Random random, string Folder) {
  string[] fileNames = Directory.GetFiles(Folder, "*.excuse");
  OpenFile(fileNames[random.Next(fileNames.Length)]);
}
```

Isso! Existe uma matriz. Devemos estar tentando usar um índice que ultrapassa o final dela.

4 Parece que `Directory.GetFiles()` retorna uma matriz vazia quando você lhe dá um diretório vazio. Ei, podemos testar para evitar isto! Tudo a **fazer é ter de certeza que o diretório não está vazio** antes de tentar abrir um arquivo. Assim, aquela incômoda janela de exceção não tratada será substituída por uma caixa de mensagem informativa.

```
private void RandomExcuseButton_Click(object sender, EventArgs e) {
  string[] fileNames = Directory.GetFiles(Folder, "*.excuse");
  if (fileNames.Length == 0) {
    MessageBox.Show("Please specify a folder with excuse files in it",
                    "No excuse files found");
  } else {
    if (CheckChanged() == true) {
      CurrentExcuse = new Excuse(random, Folder);
      UpdateForm(false);
    }
  }
}
```

Checando para ver se há arquivos de desculpas na pasta antes de criar um objeto Excuse, podemos evitar que a exceção seja lançada - e mostrar uma janela útil, também.

Ah, entendi. Exceções nem são sempre ruins. Algumas vezes elas identificam erros, mas muitas vezes apenas estão informam que o acontecido é diferente do esperado.

Isso mesmo. Uma exceção é uma ferramenta realmente útil usada para encontrar lugares onde seu código se comporta de maneiras inesperadas.

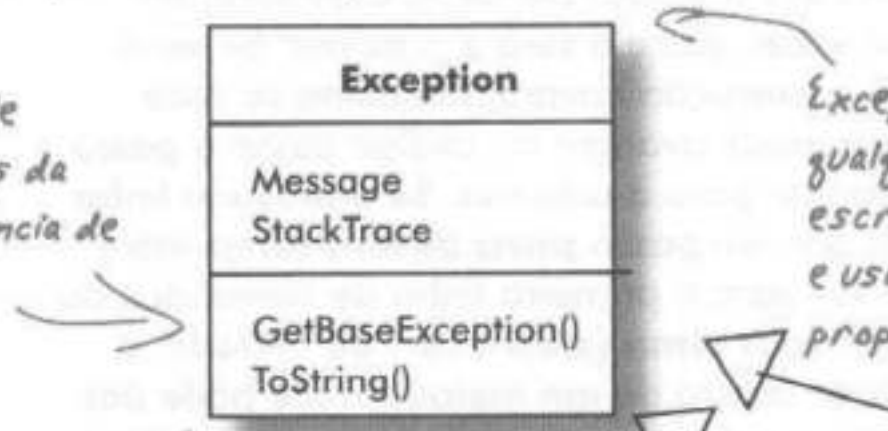
Muitos programadores ficam frustrados quando lidam com exceções pela primeira vez. Mas elas são realmente úteis e você pode usá-las em benefício próprio. Quando acontecer uma exceção, você terá muitas pistas que o ajudarão a entender quando seu código se comportar de formas não previstas. E isso é bom: você descobre um novo cenário que seu programa terá que lidar e tem a oportunidade de **fazer algo a respeito**.

Todos os objetos de exceção herdam de Exception

Há muitos tipos de exceções diferentes no .NET que eventualmente devem ser lançadas. Como muitas delas têm características semelhantes, herança será usada. Uma classe base Exception é definida no .NET e todas as exceções específicas herdam dela.

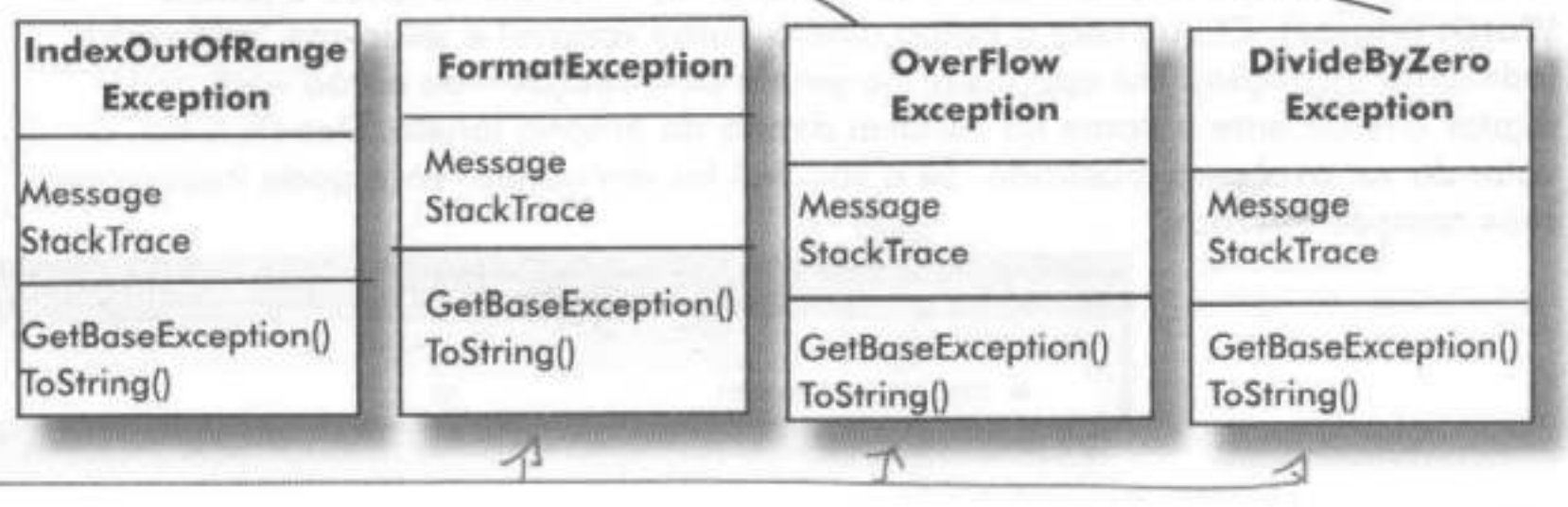
A classe Exception tem alguns membros úteis. As propriedades `Message` armazenam uma mensagem fácil de entender sobre o erro. E `StackTrace` (rastreamento da pilha) diz-lhe o que estava acontecendo na memória quando a exceção ocorreu e o que levou a ela (há outros membros, mas usaremos esses primeiro).

ToString() gera um sumário de toda a informação nos campos da exceção e retorna uma seqüência de caracteres com ele.



Exception pode ser estendida como qualquer outra classe. Então você pode escrever suas próprias classes de exceção e usar Message ou qualquer outra propriedade ou método.

É realmente útil que o .NET tenha tantos tipos de exceção, pois cada uma delas aparece em situações diferentes. Você pode entender bastante a respeito da ação inesperada causadora da exceção apenas olhando qual delas foi lançada.



A barra de ferramentas de depuração ajuda a rastrear e prevenir exceções

Antes que você adicione tratamento de exceções aos seus programas, precisa saber quais comandos no seu código as estão lançando. É por isso que o **depurador** nativo do IDE pode ser muito útil. Quando você executa o depurador, o IDE exibe uma barra de ferramentas com alguns botões realmente úteis. Reserve um minuto para passar o mouse sobre cada um dos botões para ver seu nome e tecla de atalho (o que, aliás, é um recurso bem prático).

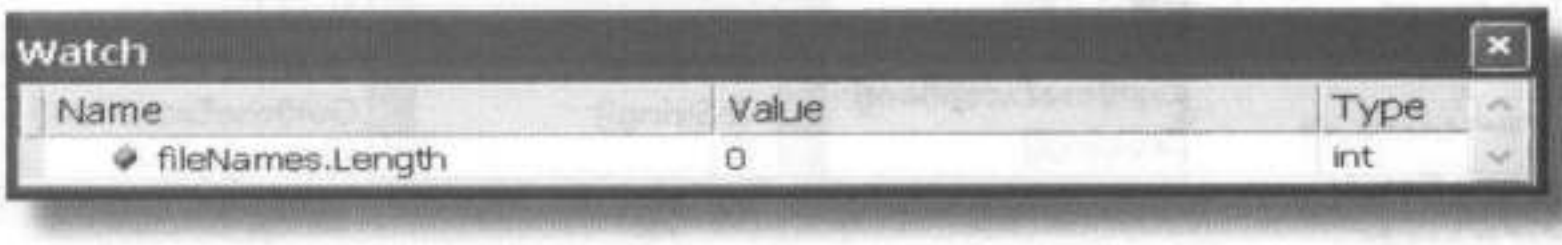
A barra de ferramentas de depuração aparece somente quando você está depurando seu programa no IDE. Logo, você deverá executar um programa para conseguir ver os ícones mostrados.

Depurar quer dizer executar seu código linha por linha para ver o que acontece

Sempre que você executa seu programa dentro do IDE, pode pausá-lo a qualquer momento clicando no botão **Break All** (interromper totalmente, literalmente "quebrar tudo") na barra de ferramentas (ou escolhendo o mesmo comando no menu Debug, depurar). Isso faz seu programa **parar o que está fazendo** e mostrar a linha de código a ser executada em seguida. Ele deixa essa linha de código destacada em amarelo para você saber que ela será a próxima. Se você clicar em **Continue**, o programa iniciará a execução novamente como se você nunca o tivesse parado. Mas você também pode avançar no código passo a passo, ou seja, executar a linha destacada e avançar para a próxima. Se a próxima linha for uma chamada de método, você pode dar um passo **para dentro (step into)** do seu método, fazendo o depurador saltar para a primeira linha de comandos do método e destacá-la. Ou você pode passar **por cima (step over)** do método, o que executa o método todo. Se você estiver dentro de um método, você pode dar um passo para **fora dele (step out)**; o depurador executará todos os comandos restantes no método e parará na primeira linha depois de retornar.

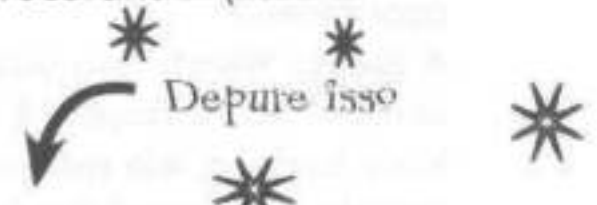
Você também pode inspecionar e alterar variáveis e campos usando a janela **Watch (vigiar)**. Clique com o botão direito numa variável e selecione "add watch" (adicionar inspeção). Ela aparecerá na janela de inspeção - ou então você pode digitar diretamente o nome da variável dentro da própria janela. Depois disso, o valor da variável será mostrado. Se a variável for um objeto, você pode inspecionar seus campos internos.

Quando você interrompe seu programa dentro do depurador, o IDE pausa a execução e mostra a próxima linha de código a ser executada deixando-a destacada num fundo amarelo. Você pode avançar linha por linha até encontrar o problema.

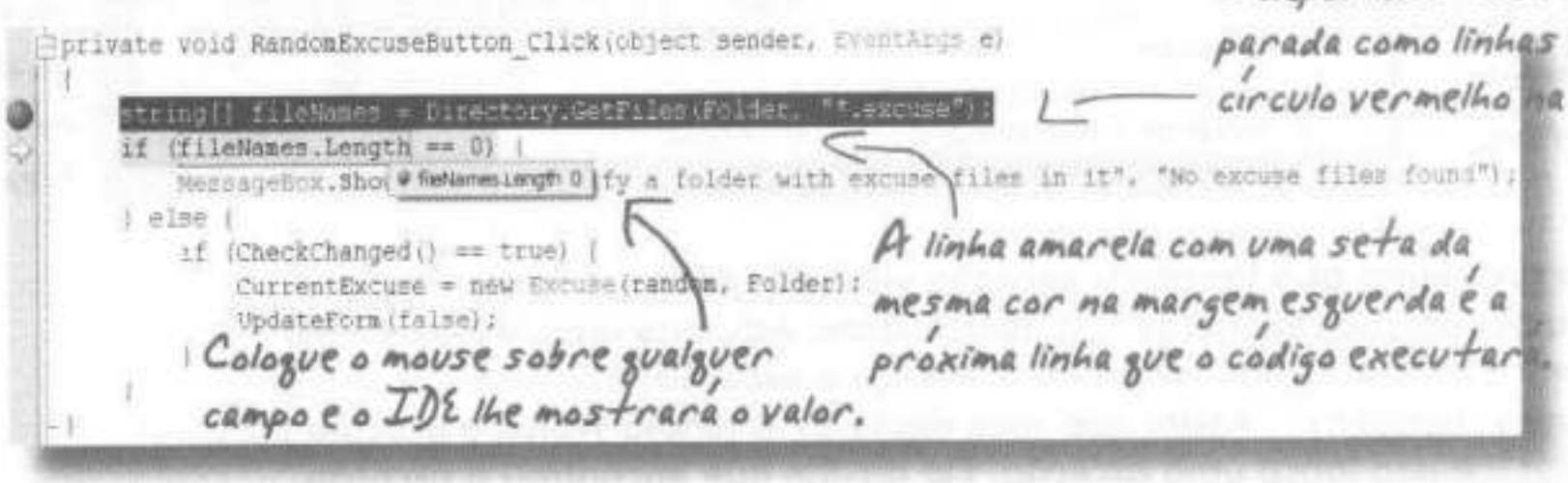


Use o depurador do IDE para desentocar o erro no gerenciador de desculpas

Vamos usar o depurador para o problema encontrado no gerenciador de desculpas. É um bom lugar para praticar um pouco com o depurador, pois você sabe exatamente o que está procurando. (Esse é um luxo inexistente na maioria das vezes!)



1 **Use um ponto de parada para interromper seu programa**
Você tem um ponto de partida - a exceção ocorre quando o botão Desculpa Aleatória é clicado depois que uma pasta vazia é selecionada. Então abra o código do botão, clique em qualquer lugar na primeira linha do método e selecione "toggle breakpoint" (colocar/retirar ponto de parada) no menu Debug (ou pressione F9):



O IDE deixou a linha vermelha e colocou um círculo na margem esquerda. Esta é a forma como o depurador diz que há um ponto de parada (breakpoint) na linha. Agora, quando você executar o programa no IDE, a execução parará nessa linha. Vamos lá, experimente - execute o programa no IDE (usando o mesmo comando "Start Debugging" - iniciar depuração - utilizado desde o início). Quando você tentar reproduzir o problema, a linha deve aparecer destacada em amarelo, com uma seta apontando para o código. Agora seu programa está temporariamente parado. O item de menu "Start Debugging" é alterado para "Continue Debugging" também. Clique nele - o programa continuará exatamente de onde parou, começando com a linha onde há o ponto de parada.

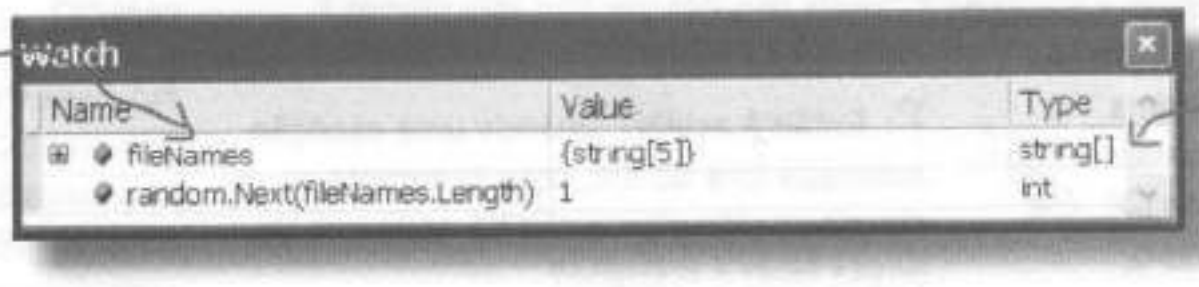
Agora você sabe porque o comando do IDE para executar um programa é "iniciar depuração" - porque você está usando o depurador nativo do IDE.

Iniciar depuração quando não se tem nenhum ponto de parada equivale a executar o programa.

2 **Avance passo a passo pelo aplicativo**
Use o comando passo para dentro (da barra de ferramentas ou a tecla F11) para mover-se linha por linha pelo aplicativo. Quando o ponto de execução atinge a linha que cria o novo objeto `Excuse`, ele saltará direto para o construtor que você corrigiu. Avance da primeira linha para que ele atribua valor para a variável `fileNames`. Então deixe o cursor do mouse parado sobre a variável para ver o valor.

3 **Use a janela Watch para reproduzir o problema**
Uma característica realmente poderosa do depurador é a janela **Watch**. Ela permite inspecionar o valor de variáveis e campos em seus objetos. Movimente o cursor do mouse sobre "Length" em "`fileNames.Length`". Selecione a expressão inteira, clique com o botão direito e selecione "Add watch" do menu. Ela será adicionada à janela Watch (localizada no mesmo painel que Output - você pode exibi-la selecionando "Watch", no menu Debug, subitem de Windows). Então adicione cada parte do comando: primeiro `random.Next(fileNames.Length)`. O resultado deve parecer com o mostrado aqui, dependendo de quantos arquivos houver na pasta para a qual você esteja apontando - nesse caso, temos cinco, de forma que `fileNames` tem cinco elementos:

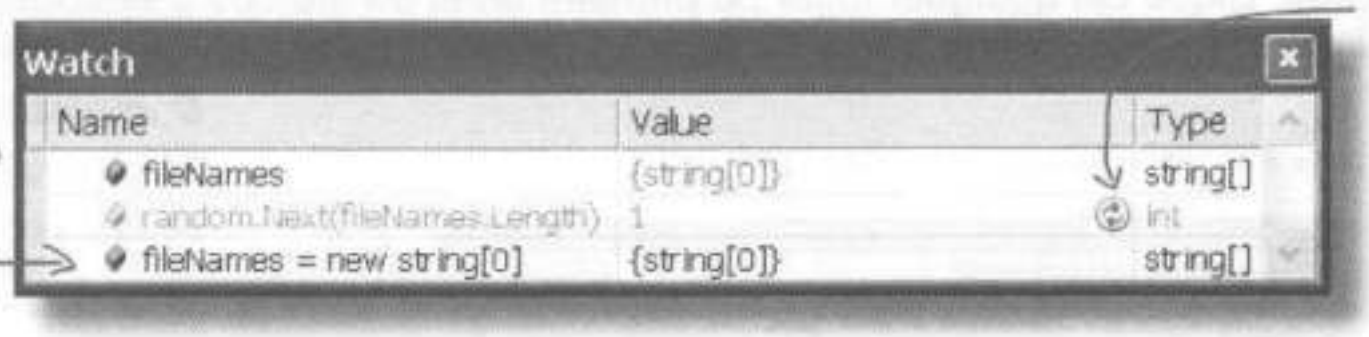
Usaremos a janela Watch para reproduzir o problema que causou a exceção. Começaremos adicionando a matriz fileNames.



Você precisa pausar (em inglês, break, quebrar) um programa para poder adicionar uma inspeção de variável.

4 **Iguale fileNames a uma matriz de strings vazia**
Clique duas vezes no espaço em branco na janela Watch abaixo das duas inspeções de variável. Você verá um cursor. Digite: **fileNames = new string[0]**. Observe a linha superior da janela - assim que você pressiona Enter, o valor de `fileNames` muda para `{string[0]}`. Um ícone de reavaliação aparece ao lado da linha do `random.Next` - clique nele e você verá o valor mudando para zero. O que aconteceu?
A janela Watch tem outra característica muito poderosa - ela permite **alterar o valor** de variáveis e campos em exibição. E ela permite que você mesmo **execute métodos e crie novos objetos**. Quando você faz isso, ela mostra um ícone de reavaliação ( ) clicável, para dizer à janela para recalcular aquela linha também (algumas vezes executar o mesmo método duas vezes gerará resultados diferentes, como no caso de Random).

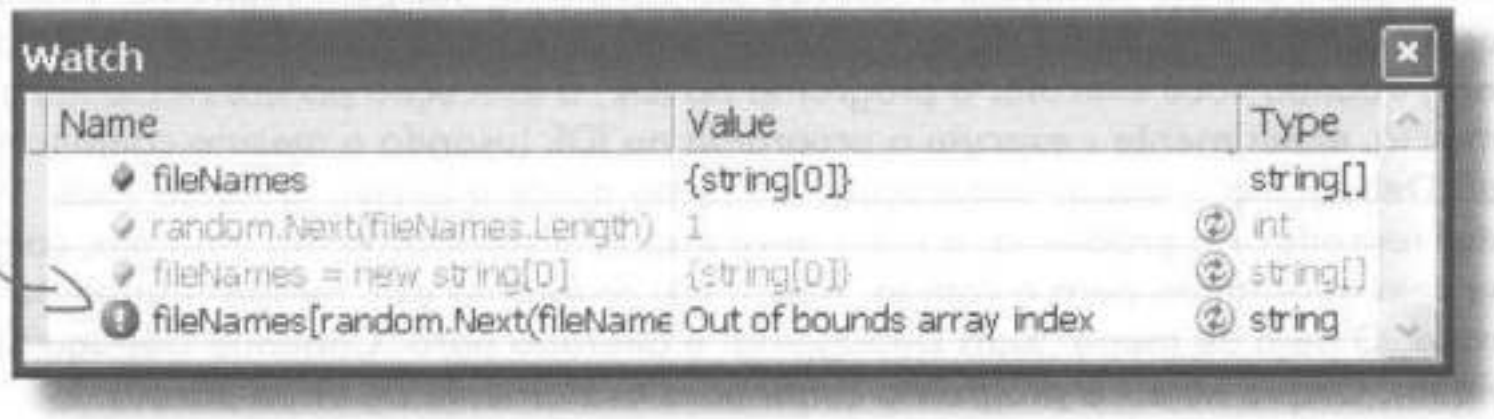
Sabemos que o problema ocorreu quando tínhamos uma matriz fileNames vazia, então usaremos a janela de Watch para esvaziar essa matriz.



Esse ícone diz à janela Watch para recalcular o valor do método Next().

5 **Reproduza o problema que lançou a exceção vista por Aldo no início**
É aqui onde a depuração realmente fica interessante. Adicione uma linha a mais no depurador - o comando que efetivamente lança a exceção: `fileNames[random.Next(fileNames.Length)]`. Assim que você digitá-lo, a janela Watch o executa para calcular o valor... e isso lança uma exceção. Ela diz-lhe que encontrou a exceção mostrando um ponto de exclamação e mostra o texto da exceção na coluna Value (valor).

Esse ponto de exclamação é a forma da janela Watch informar que encontrou uma exceção.



Quando você notar uma exceção, pode voltar atrás e reproduzi-la no depurador. Essa é outra forma pela qual mensagens de exceção bem descritivas podem ajudar a corrigir o código.

não existem Perguntas Idiotas

P: Porque a janela de exceção não tratada do Aldo parece diferente daquela no IDE?

R: Quando você executa um programa dentro do IDE, está usando o depurador. Ele **interrompe o programa** (como se você tivesse pressionado o botão Break All ou inserido um ponto de parada) quando intercepta uma exceção e a exibe numa janela útil. Isso permite inspecionar o objeto Exception e os campos e variáveis de seu programa, possibilitando-lhe rastrear o problema. Quando Aldo executou o programa, ele não estava fazendo isso de dentro do IDE. Ele publicou seu programa e instalou-o, exatamente como você fez no capítulo 1 com o programa de lista de contatos. Você pode executar seu programa fora do IDE quando quiser sem publicá-lo; apenas compile o programa e o Visual Studio criará um arquivo executável. Basta olhar dentro do seu diretório de projeto na pasta `"bin"` – um dos subdiretórios deve ter o arquivo `exe` do seu aplicativo. Se você executar, qualquer exceção lançada não terá sido tratada e mostrará a mesma janela que Aldo viu.

P: Então é assim? Quando uma exceção acontece fora do IDE, meu programa simplesmente pára e não há nada que eu possa fazer a respeito?

R: Bom, seu programa pára quando existe uma exceção não `tratada`. Mas nem todas as exceções precisam ser não tratadas! Vamos falar muito mais a respeito de como você pode tratar exceções em seu código. Não existe motivo para que seus usuários vejam uma exceção não tratada.

P: Como eu sei onde colocar um ponto de parada?

R: Essa é uma ótima pergunta e não existe uma resposta certa. Quando seu código lança uma exceção é sempre uma boa idéia começar pelo comando que a lançou. Mas normalmente, o problema começou em algum

ponto anterior no programa e a exceção é apenas uma conseqüência. Por exemplo, o comando que lançar uma exceção de divisão por zero pode estar dividindo algo por um valor gerado 10 comandos antes, se nada que lance uma exceção foi feito com ele antes. Então não existe uma única boa resposta a respeito de onde você deve colocar um ponto de parada, porque cada situação é diferente. Mas desde que você tenha uma boa idéia sobre como seu código funciona, você deve ser capaz de determinar um bom ponto de partida.

P: **Posso executar qualquer método na janela Watch?**

R: Sim. Qualquer comando válido no seu programa funcionará dentro da janela Watch; mesmo coisas sem sentido algum de se fazer dentro dessa janela. Eis um exemplo: abra um programa, comece a executá-lo, interrompa-o e adicione `System.Threading.Thread.Sleep(2000)` na janela Watch (lembre-se: este método faz seu programa pausar por dois segundos). Não há nenhum motivo para você fazer isso na vida real, mas é interessante ver o que acontece: você fica vendo o cursor de ampulheta durante dois segundos enquanto o método executa. Então, uma vez que `Sleep` (dormir) não tem valor de retorno, **a janela de Watch mostrará o resultado** `"Expression has been evaluated and has no value"` **(a expressão foi calculada e não tem valor), para que você saiba que ela não retornou nada, mas foi executada. Além disto, na janela Watch as caixas do IntelliSense também funcionam, aparecendo para ajudá-lo a digitar código.** Isso é útil porque lhe diz quais métodos estão disponíveis num objeto quando seu programa está executando.

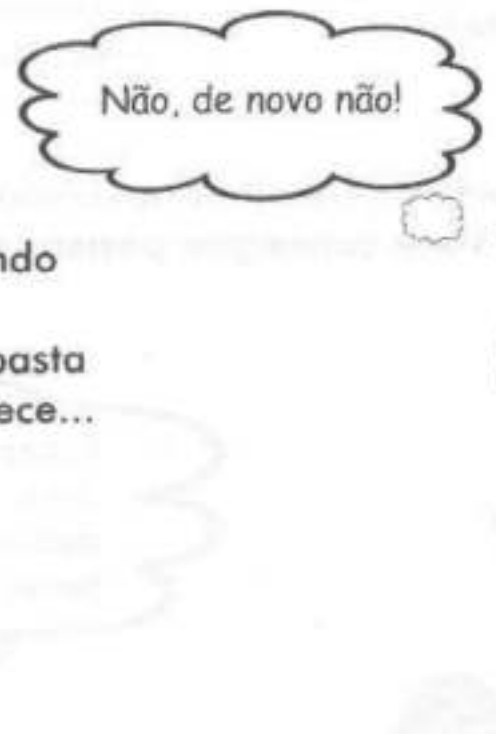
P: **Espere. Então é possível executar algo na janela Watch que muda a forma como meu programa funciona?**

R: Sim! Não permanentemente, mas pode afetar o resultado de seu programa. E ainda pior, apenas parar o **cursor do mouse** sobre campos dentro do depurador pode fazer seu programa mudar seu comportamento, pois isto sobre uma propriedade **faz seu acessador get executar**. Se você tem uma propriedade com um acessador get que executa um método, deixar o cursor do mouse sobre ela executará o método. E se ele alterar algum valor em seu programa, o valor continuará o mesmo se você permanecer executando o programa. E isso pode causar alguns resultados bastante imprevisíveis dentro do depurador. Programadores dão nomes para resultados que parecem imprevisíveis e aleatórios: **heisenbugs*** (uma piada que faz sentido para físicos e gatos dentro de caixas).

Quando você executa seu programa dentro do IDE, uma exceção não tratada fará com que ele seja interrompido, exatamente como se tivesse atingido um ponto de parada.

* (NT) de Heinsenberg e seu Princípio da Incerteza e "bug", que significa literalmente "inseto", mas é usado para "erro no código".

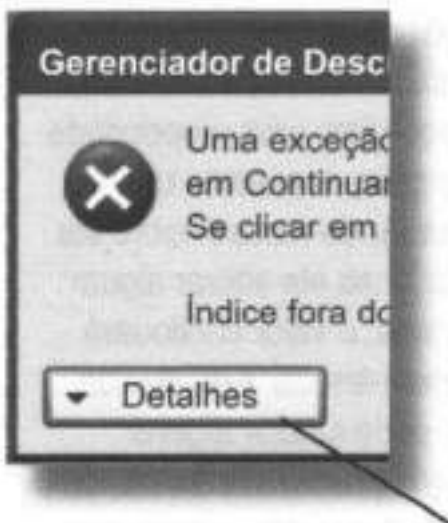
Opa, o código ainda está com problemas...



Aldo estava usando alegremente seu Gerenciador de Desculpas, quando lembrou que tinha uma pasta cheia de desculpas criadas quando desenvolveu o programa - mas ele se esqueceu de que criou aquela pasta **antes** de adicionar serialização ao programa. Vamos ver o que acontece...

1. Você pode reproduzir o problema que o Aldo enfrentou - crie seu próprio arquivo de desculpas em texto puro, usando o bloco de notas. A primeira linha deve ser a descrição, a segunda os resultados e a terceira da data da última utilização ("15/10/2007 12:08:13 PM", por exemplo).

2. Execute o Gerenciador de Desculpas e abra o arquivo de desculpas recém-criado. Uma exceção é lançada! Mas, dessa vez, clique no botão Detalhes (Details) para que possamos ver o que está acontecendo. Preste atenção na **pilha de chamada** - que é a lista de chamadas: quando um método chama outro, e foi chamado por outro, e assim por diante.



O programa lança um SerializationException. Podemos determinar qual linha lançou o objeto a partir dos detalhes da exceção?

Parece que temos um problema com o BinaryFormatter - o que faz sentido, pois estamos tentando desfazer a serialização de um arquivo texto.

Você pode descobrir bastante examinando a **pilha de chamadas**. Ela diz-lhe quais métodos estavam executando. Você pode ver que o método OpenFile() da classe Excuse foi chamado de seu construtor (".ctor"), que foi chamado do tratador de evento de clique do botão "Desculpa Aleatória".

```
********** Exception Text **********

System.Runtime.Serialization.SerializationException: End of Stream encountered before
parsing was completed.

at System.Runtime.Serialization.Formatters.Binary.__BinaryParser.Run()

at System.Runtime.Serialization.Formatters.Binary.ObjectReader.Deserialize (HeaderHandler
handler, __BinaryParser serParser, Boolean fCheck, Boolean isCrossAppDomain,
IMethodCallMessage methodCallMessage)

at System.Runtime.Serialization.Formatters.Binary.BinaryFormatter.Deserialize (Stream
serializationStream, HeaderHandler handler, Boolean fCheck, Boolean isCrossAppDomain,
IMethodCallMessage methodCallMessage)

at System.Runtime.Serialization.Formatters.Binary.BinaryFormatter.Deserialize (Stream
serializationStream)

at Chapter10.Excuse.OpenFile(String ExcusePath) in C:\Documents and Settings\
Administrator\My Documents\Visual Studio 2005\Projects\Chapter10\Chapter10\Excuse.
cs:line 40

at Chapter10.Excuse..ctor(Random random, String Folder) in C:\Documents and Settings\
Administrator\My Documents\Visual Studio 2005\Projects\Chapter10\Chapter10\Excuse.
cs:line 30

at Chapter10.Form1.RandomExcuseButton_Click(Object sender, EventArgs e) in C:\
Documents and Settings\Administrator\My Documents\Visual Studio 2005\Projects\
Chapter10\Chapter10\Form1.cs:line 146
```

3. Então o botão de Detalhes na janela de exceção não tratada lhe diz muito sobre o que causou o problema. **Você consegue pensar em como tomar alguma providência a respeito?**

Espere aí. Claro que o programa vai dar erro - eu dei a ele um arquivo que não serve. Usuários fazem besteiras a toda hora. Você não espera que eu possa fazer algo a respeito disso, certo?

Na verdade, existe algo que pode ser feito

Sim, é verdade que os usuários fazem besteiras a toda hora. Isso é um fato da vida. Mas isso não quer dizer que você não possa fazer nada a respeito. Há um nome para programas que lidam com dados ruins, entradas mal formadas e outras situações inesperadas de forma elegante: eles são chamados de **robustos**. E o C# dá-lhe algumas ferramentas de tratamento de exceção realmente poderosas para ajudar a fazer seus programas mais robustos. Porque você não pode controlar o que os usuários farão; é possível apenas certificar-se de que seu programa não feche quando eles fizerem algo errado.

ro-bus-to, adjetivo.
de construção resistente; capaz de agüentar ou superar condições adversas. *Depois do desastre da ponte do desfiladeiro de Tacoma, a equipe de engenharia civil procurou por um projeto mais **robusto** para a ponte que substituiria a que caiu.*

Veja bem!

BinaryFormatter lançará uma exceção se houver qualquer coisa errada com um arquivo de serialização.

É fácil fazer o Gerenciador de Desculpas lançar uma exceção do tipo SerializationException - apenas mande-o abrir qualquer arquivo que não seja um objeto Excuse serializado. Quando você tenta desfazer a serialização de um objeto serializado num arquivo, BinaryFormatter espera que ele contenha efetivamente um objeto equivalente à classe que está tentando ler. Se o arquivo contiver qualquer outra coisa, o método Deserialize() lançará a referida exceção.

Trate de exceções com try e catch

Em C#, você pode basicamente dizer "**tente** executar esse código e se uma exceção for lançada, **capture-a** com este outro trecho de código". A parte do código que está **tentando executar é o bloco try** (tentar) e a parte onde se lida com a exceção é chamada de **bloco catch** (capturar, literalmente catar), ou bloco de captura. No bloco de captura, você pode fazer coisas como apresentar uma mensagem de erro amigável, em vez de deixar seu programa ter um final desastroso:

```
private void RandomExcuseButton_Click(object sender,
EventArgs e) {
  // ... o codigo que voce adicionou algumas paginas atras vem aqui...
  try {
    if (CheckChanged() == true) {
      CurrentExcuse = new Excuse(random, Folder);
      UpdateForm(false);
    }
  }
  catch {
    MessageBox.Show(
      "Your excuse file was invalid.",
      "Unable to open a random excuse");
  }
}
```

Este é o bloco try. Você começa o tratamento de exceções com try.

Quando uma exceção é lançada, o programa imediatamente vai para o comando catch e inicia a execução de seu bloco.

A palavra-chave catch implica que o bloco a seguir contém um tratador de exceção.

Esta é a forma mais simples de tratamento de exceção: pára o programa, mostra a mensagem de exceção, depois continua executando.

Coloque o código que pode lançar exceções dentro do bloco try. Se nenhuma exceção for lançada, ele executará exatamente como sempre e os comandos no bloco de captura serão ignorados. Mas se algum comando dentro do bloco try lançar uma exceção, o restante dele não será executado.

Poder do Cérebro

Se lançar uma exceção faz com que o ponto de execução imediatamente salte para o bloco catch, o que acontece aos objetos e dados com os quais você estava trabalhando antes da exceção acontecer?

O que fazer quando um método que você quer chamar apresenta riscos?

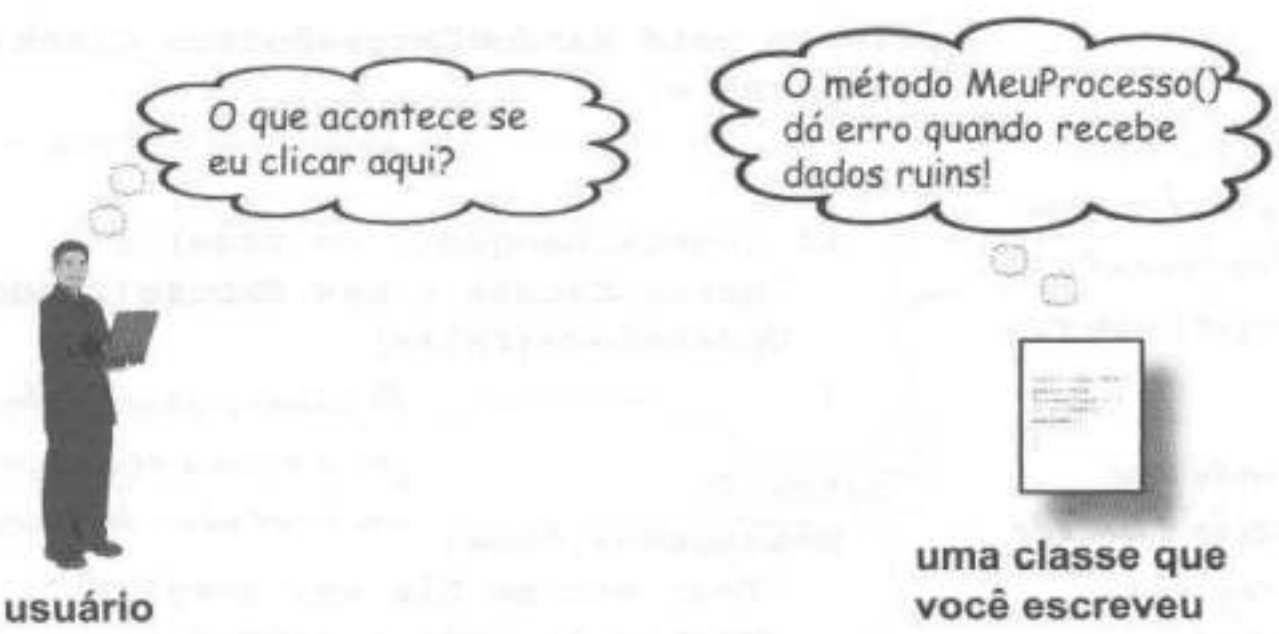
Usuários são imprevisíveis. Eles informam todo o tipo de dados estranhos para seu programa e clicam em coisas de formas que você nunca imaginou. Tudo bem, porque você pode lidar com dados de entrada inesperados com um bom tratamento de exceção.

1 **Digamos que seu usuário está trabalhando com seu programa e dá a ele algum dado de entrada inesperado.**

2 **O método imediatamente faz algo arriscado, algo que pode não funcionar em tempo de execução.**

Tempo de execução significa enquanto o mesmo está rodando.

3 **Você precisa perceber que o método chamado apresenta riscos.**



4 **Você então escreve código que pode lidar com a falha caso ela aconteça. Você precisa estar preparado para qualquer eventualidade.**

não existem Perguntas Idiotas

P: Então quando devo usar try e catch?

R: Quando estiver escrevendo código em qualquer momento que apresente riscos, ou seja, que poderia lançar exceções. O truque é saber quando o código é seguro e quando apresenta riscos.
Você sabe o que código que usa dados informados pelo usuário apresenta riscos. Usuários informam arquivos errados, palavras no lugar de números, nomes no lugar de datas e eles clicam em qualquer lugar possível e imaginável. E um bom programa deve lidar com todas essas coisas de forma calma e previsível. Pode ser que ele não consiga apresentar aos usuários uma resposta útil, mas ele deve fazer com que percebam encontrou um problema e, quando possível, sugerir uma solução.

P: Como um programa sugere uma solução para um problema que ele não conhece com antecedência?

R: É para isso que o bloco catch serve. Um bloco de captura só é executado quando o código que se tentou fazer funcionar no bloco try lançou uma exceção. É a sua chance de certificar-se de que o usuário saiba que algo de errado aconteceu e informá-lo que isso é uma situação a ser evitada.
Se, por exemplo, o Gerenciador de Desculpas simplesmente fechasse quando recebesse dados de entrada ruins, isso não seria útil. Mas se ele ainda continuar tentando a entrada e mostrar lixo no formulário, isso também não seria bom - de fato, algumas pessoas dizem que é pior. Mas se você fizer o programa mostrar uma mensagem de erro dizendo que um arquivo não pôde ser lido, o usuário terá uma noção que algo de errado ocorreu e terá informações para corrigir o problema.

P: O depurador só é usado para resolver problemas com exceções?

R: Não. O depurador de fato é uma ferramenta muito poderosa que você pode usar para examinar qualquer código que tenha escrito. Algumas vezes é útil avançar passo a passo pelo programa e checar o valor de certas variáveis e campos – como, por exemplo, quando você tem um método complexo e quer ter certeza de que está funcionando adequadamente. Mas como você provavelmente já adivinhou por causa do nome "depurador", o uso mais comum dele é rastrear e remover erros (bugs, em inglês). Algumas vezes esses erros são exceções lançadas. Mas, muitas vezes, você usará o depurador para tentar encontrar outros tipos de problemas, como, por exemplo, código que dá um resultado inesperado.

P: Eu não estou certo de ter entendido completamente a janela de Watch. Para que ela serve mesmo?

R: Quando você estiver depurando um programa, geralmente, desejará prestar atenção na forma como algumas variáveis e campos mudam. É para isso que serve a janela Watch. Se você adicionar inspeções para algumas variáveis, a janela de Watch atualiza seus valores a cada passo que você avançar no código. Isso permite monitorar exatamente o que acontece com elas depois de cada comando, o que pode ser útil quando você está tentando rastrear um problema.
A janela Watch também permite entrar qualquer comando que queira e ela vai executá-lo. Se esse comando alterar qualquer um dos campos ou variáveis no seu programa, isso acontece normalmente. Isso possibilita mudar valores durante a execução do programa, o que pode ser outra ferramenta muito útil para reproduzir exceções e outros erros.

Qualquer alteração na janela Watch afeta somente os dados na memória e apenas enquanto o programa está executando. Reinicie o programa e as suas alterações serão desfeitas.

O bloco catch só é executado quando código no bloco try lança uma exceção. Isso lhe dá a chance de certificar-se que seu usuário seja informado para corrigir o problema.

Use o depurador para seguir o fluxo do try/catch

Uma parte importante de tratamento de exceção é que, quando um comando no bloco try lança uma exceção, o restante do código entra em **curto-circuito**. O ponto de execução do programa imediatamente salta para a primeira linha do bloco catch. **Mas não precisa aceitar nossa sugestão**...

1. Certifique-se de ter incluído todo o código deste capítulo no botão de desculpa aleatória no seu Gerenciador de Desculpas. Coloque um ponto de parada na primeira linha do tratador de evento de clique do botão. Então execute seu programa no IDE. Clique o botão Pasta e especifique uma com um único arquivo de desculpas dentro - e assegure-se de **não ser um arquivo válido** (qualquer outro tipo de arquivo fará uma exceção ser lançada). Pressione o botão de desculpa aleatória. O depurador deve interromper o programa no ponto de parada que você colocou. Pressione o botão "passar por cima" (step over, ou F10) seis vezes para chegar ao comando que chama o construtor de Excuse. Eis como a sua tela do depurador deve estar:

Coloque um ponto de parada na primeira linha do tratador de evento do botão de Desculpa Aleatória.

Passe por cima dos comandos até que sua barra amarela de "próximo comando" mostre que a próxima a ser executada é a linha que cria o novo objeto Excuse.

```
private void RandomExcuseButton_Click(object sender, EventArgs e) {
    string[] fileNames = Directory.GetFiles(Folder, "*.excuse");
    if (fileNames.Length == 0) {
        MessageBox.Show("Please specify a folder with excuse files in it",
                        "No excuse files found");
    } else {
        try {
            if (CheckChanged()) {
                CurrentExcuse = new Excuse(random, Folder);
                UpdateForm(false);
            }
        }
        catch {
            MessageBox.Show(
              "Your excuse file was invalid.",
              "Unable to open a random excuse");
        }
    }
}
```

Certifique-se de usar o comando "passar por cima" (step over, ou F10) do depurador para não entrar no método CheckChanged().

2. Use "passo para dentro" (step into, ou F11) para entrar no comando `new`. O depurador saltará para o construtor de Excuse e posicionará sua barra amarela de "próximo comando" na linha de declaração no código. Continue usando "passo para dentro" para entrar no método OpenFile(). Observe o que acontece quando você chega à linha `Deserialize()`.

Assim que passar pelo comando new, que cria o objeto Excuse, o depurador salta para o código do construtor.

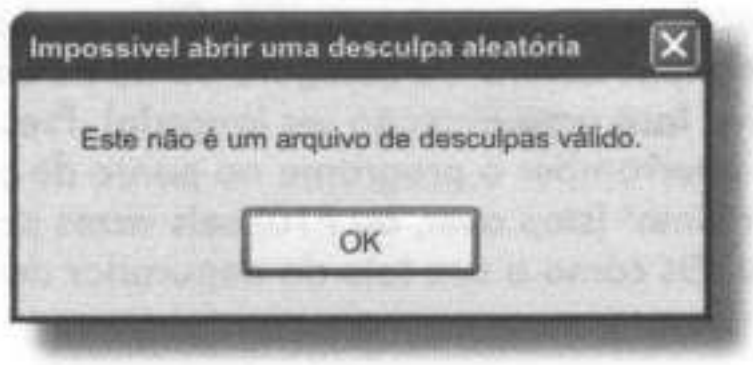
```
public Excuse(Random random, string folder)
{
    string[] fileNames = Directory.GetFiles(folder, "*.excuse");
    OpenFile(fileNames[random.Next(fileNames.Length)]);
}
```

3. Assim que o depurador executar o comando `Deserialize()`, uma exceção é lançada e o programa salta direto para a primeira linha do bloco de captura. Ele fez um **curto-circuito** direto da chamada de `UpdateForm()` e **saltou imediatamente para o bloco catch**.

O depurador destacará o comando catch com a barra amarela de "próximo comando", mas ele exibe o restante do bloco em cinza para mostrar-lhe que está prestes a executar o bloco todo.

```
private void RandomExcuseButton_Click(object sender, EventArgs e) {
    string[] fileNames = Directory.GetFiles(Folder, "*.excuse");
    if (fileNames.Length == 0) {
        MessageBox.Show("Please specify a folder with excuse files in it",
                        "No excuse files found");
    } else {
        try {
            if (CheckChanged()) {
                CurrentExcuse = new Excuse(random, Folder);
                UpdateForm(false);
            }
        }
        catch {
            MessageBox.Show(
              "Your excuse file was invalid.",
              "Unable to open a random excuse");
        }
    }
}
```

4. Permita ao programa continuar a execução pressionando o botão Continue (ou F5). Ele funcionará sem supervisão de novo, começando com qualquer coisa destacada pela barra amarela de "próximo comando" - nesse caso, o bloco catch.



Veja bem!

Cuidado com exceções em construtores!

Você já sabe que um construtor não tem um valor de retorno, nem mesmo `void`. *Isso é porque ele realmente não retorna nada. Seu único propósito é inicializar um objeto - o que é um problema para o tratamento de exceção feito dentro de um construtor. Quando uma exceção é lançada dentro de um construtor, o comando que tentou* ***instanciar uma classe fica sem uma cópia do objeto****. É por isso que você teve de mover o bloco try/catch para o tratador de evento do botão. Dessa forma, se ocorrer uma exceção no construtor, o código não esperará que CurrentExcuse contenha um objeto Excuse válido.*

Eis uma dica para sua carreira: muitas entrevistas de emprego como programador C# incluem perguntas sobre como lidar com exceções num construtor.

Se você tem código que SEMPRE deve executar, use um bloco finally

Quando seu programa lança uma exceção, algumas coisas podem acontecer. Se a exceção não for tratada, ele parará de processar e encerrará com erro. Se ela for tratada, seu código salta para o bloco `catch`. Mas o que fazer sobre o restante do código no bloco `try`? E se, por exemplo, você estiver fechando uma stream ou liberando recursos importantes? O código precisa executar, mesmo que uma exceção ocorra, ou o estado do seu programa virará uma bagunça. É aí que o bloco **finally** (finalmente) é realmente útil. Ele vem depois dos blocos `try` e `catch`. O **bloco finally sempre executa**, tenha ou não uma exceção sido lançada. Eis como você poderia usá-lo para encerrar o tratamento de eventos no botão de desculpa aleatória:

```
private void RandomExcuseButton_Click(object sender, EventArgs e) {
  string[] fileNames = Directory.GetFiles(Folder, "*.excuse");
  if (fileNames.Length == 0) {
    MessageBox.Show("Please specify a folder with excuse files in
it",
                    "No excuse files found");
  } else {
    try {
      if (CheckChanged() == true) {
        CurrentExcuse = new Excuse(random, Folder);
      }
    }
    catch (Exception) {
      CurrentExcuse = new Excuse();
      CurrentExcuse.Description = "";
      CurrentExcuse.Results = "";
      CurrentExcuse.LastUsed = DateTime.Now;
      MessageBox.Show(
        "Your excuse file was invalid.",
        "Unable to open a random excuse");
    }
    finally {
      UpdateForm(false);
    }
  }
}
```

Se o construtor de Excuse lançar uma exceção, não temos como saber o que acontece com CurrentExcuse. Mas sabemos que nenhuma instância de Excuse foi criada. Logo o bloco catch cria um novo objeto Excuse e esvazia todos os seus campos.

O bloco finally certifica-se de que UpdateForm() seja executado, independente de uma exceção ser lançada. Assim, se o construtor de Excuse ler uma desculpa com sucesso, chamará UpdateForm(). Ele também fará isso caso o construtor lance uma exceção e deixe a desculpa em branco.

Você notou que depois do catch vem "(Exception)"? Quando você usa um comando catch, pode segui-lo com um tipo específico de exceção – e isso lhe diz para capturar somente esse tipo. Se você especificar **(Exception)** ou não colocar nada depois de catch, ele captura qualquer exceção. Mas se quiser apenas capturar, digamos, um `SerializationException`, poderia especificar isso dentro dos parênteses ou usar **IOException** (o que **capturaria qualquer problema de entrada ou saída de arquivo)**.

Agora depure isso

1. Atualize o tratador de evento do botão de desculpa aleatória com o código da página anterior. Coloque um ponto de parada na primeira linha do método e depure o programa.

2. Execute o programa normalmente e certifique-se de que o botão de desculpa aleatória funciona quando você aponta o aplicativo para uma pasta com um monte de arquivos de desculpa válidos. O depurador deve interromper o programa no ponto de parada estabelecido:

Quando a barra de "próximo comando" e o ponto de parada estão na mesma linha, o IDE mostra a seta amarela sobre o grande ponto vermelho na margem.

3. Avance passo a passo pelo restante do tratador de evento do botão e tenha certeza de que ele executa como você espera. Ele deve encerrar o bloco `try`, pular o `catch` (porque nenhuma exceção foi lançada) e executar o `finally`.

4. Agora aponte o programa para uma pasta com um arquivo de desculpa inválido e clique no botão de desculpa aleatória. Ele deve começar a executar o bloco `try` e saltar para o bloco `catch` quando a exceção for lançada. Depois de terminar todos os comandos do bloco de `captura`, ele deve executar o bloco `finally`.

não existem Perguntas Idiotas

P: Volte um pouco. Então toda vez que meu programa encontrar uma exceção, ele parará, independente do que estiver fazendo, a não ser que e escreva código para capturá-la. Por que isso é bom?

R: Uma das melhores coisas sobre exceções é que elas informam quando você enfrenta problemas. Imagine como seria fácil num aplicativo complexo perder a noção a respeito do funcionamento de todos os muitos objetos em uso. Exceções chamam a atenção para seus problemas e ajudam a determinar suas causas para que você saiba que o código está fazendo o que deveria fazer.
Sempre que há uma exceção no seu programa, alguma coisa esperada não aconteceu. Talvez uma referência de objeto não esteja apontando para onde deveria, ou um usuário forneça um dado que você não imaginou possível, ou um arquivo com o qual o código estava trabalhando subitamente se torna inacessível. Se alguma coisa assim acontecer e você não souber, é provável que seu programa exiba um resultado errado e o comportamento desse ponto em diante seja bem diferente do esperado quando desenvolveu o código.
Agora imagine que você não tenha a menor idéia de que um erro ocorreu e os usuários começassem a telefonar dizendo que os dados estão errados e que o programa é instável. É por isso que as exceções interromperem tudo o que o programa está fazendo é algo bom. Isso força a você a lidar com o problema enquanto é fácil encontrá-lo e corrigi-lo.

P: Certo, então o que é uma exceção tratada e uma não tratada?

R: Quando seu programa lançar uma exceção, o ambiente de runtime procurará no código por um bloco de captura equivalente ao tipo de exceção. Se você escreveu um, o bloco catch será executado e o que foi especificado para aquela exceção em particular. Uma vez que você tenha escrito um bloco catch para lidar com aquele tipo de erro antes dele ocorrer, essa exceção é considerada tratada. Se o runtime não puder encontrar um bloco de captura para lidar com a exceção, ele pára tudo o que seu programa estiver fazendo e levanta um erro. Nesse caso, você diz que a exceção não foi tratada.

P: O que significa especificar um tipo particular de exceção para capturar? Por que eu desejaria fazer isso?

R: Normalmente, você não quer capturar qualquer tipo de exceção. De fato, você deve fazer o melhor possível para evitar capturar `Exception` e, em vez disto, tentar lidar com exceções de tipos particulares. Por exemplo, desejo que sua classe Excuse fique prevenida contra uma `FileNotFoundException` (exceção arquivo não encontrado) e que não a lance para o formulário. Imaginemos que você

queira fazer de tal forma que, se alguém tentar abrir um arquivo inexistente, automaticamente fosse criado um novo arquivo de desculpas com o nome de arquivo informado e alguns valores padrão seriam salvos nele.
Você poderia adicionar um bloco `try` seguido de um `catch(FileNotFoundException)` e se um arquivo não fosse encontrado, a classe poderia lidar com isso - mas uma `IOException` ou `SerializationException` não seriam capturadas e assim o tratado de exceção inserido no formulário receberia essas exceções. Mas se você fizer isso, deve certificar-se de que há algum método na pilha de chamada com um bloco catch de captura para qualquer tipo de exceção, caso contrário a exceção continuará não tratada. E isso faria com que o usuário visse aquele erro feio de "exceção não tratada" fechando o programa.

P: O que acontece quando você tem um catch que não especifica um tipo de exceção em particular?

R: Um bloco assim captura qualquer tipo de exceção que o bloco try possa lançar.

P: Se um bloco catch sem nenhuma exceção especificada capturará qualquer coisa, eu desejarei especificar algum tipo em algum momento?

R: Boa pergunta. Porque certas exceções podem exigir tipos diferentes de ações para o programa continuar funcionando. Uma exceção ocorrida quando você divide por zero pode ter um bloco catch onde você retorna e atribui alguns valores numéricos para preservar alguns dos dados com os quais estava trabalhando. Uma exceção de referência nula pode exigir que você crie novas instâncias de um objeto se quiser se recuperar do erro.

P: Todo o tratamento de erros sempre acontece dentro de uma seqüência try/catch/finally?

R: Não. Você pode lidar com as coisas de várias formas. Você pode ter múltiplos blocos catch se quiser lidar com muitos tipos diferentes de erros. Você pode também não incluir o bloco catch. É perfeitamente aceitável ter um conjunto de blocos try/finally. Isso não lidaria com nenhuma exceção, mas você teria certeza de que o código no bloco finally executará mesmo que o do bloco try tenha parado no meio caminho por um erro. Mas falaremos muito mais sobre isso num minuto.

Uma exceção não tratada implica em seu programa executar **imprevisivelmente**. É por isso que o programa pára sempre que encontra uma.

Jogos de Verão

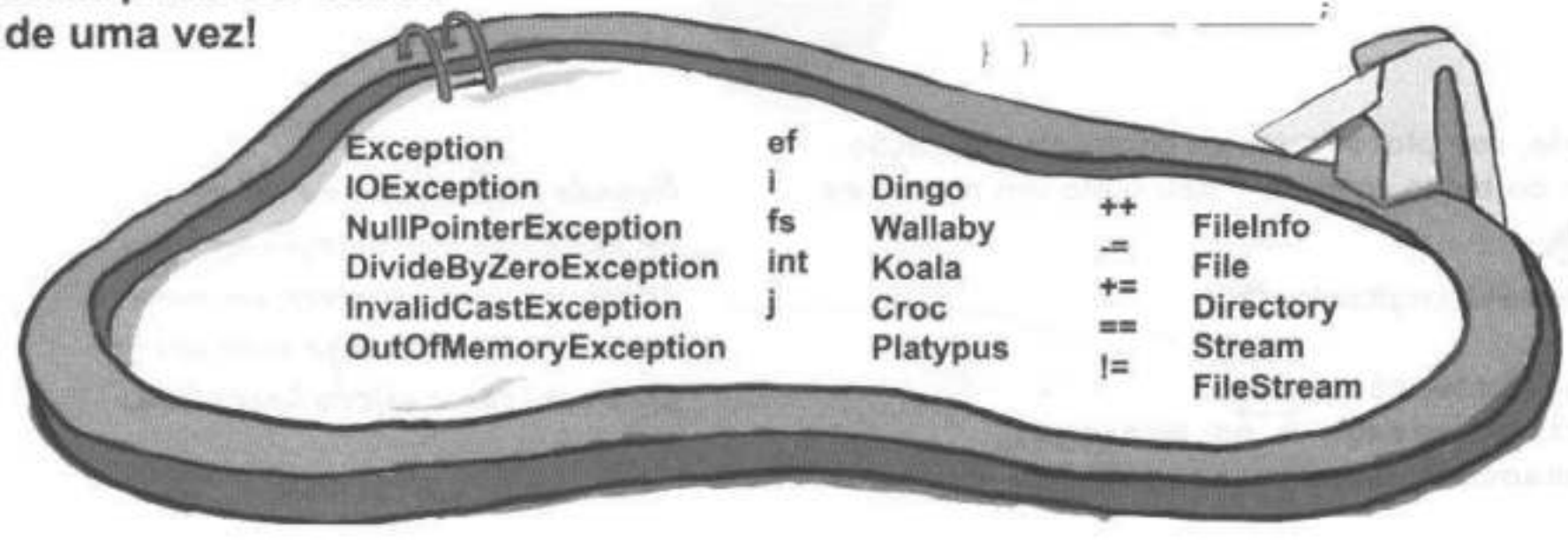
Sua tarefa é pegar trechos de código da piscina e colocá-los nas lacunas no programa. Você pode usar o mesmo trecho mais de uma vez e não precisa usar todos eles. Seu objetivo é fazer o programa produzir o resultado mostrado.

Resultado ⟶ G'day Mate!

```
using System.IO;
public static void Main() {
  Kangaroo Joey = new Kangaroo();
  int Koala = Joey.Wombat(
        Joey.Wombat(Joey.Wombat(1)));
  try {
      Console.WriteLine((15 / Koala)
          + " eggs per pound");
  }
  catch (_____________________) {
      Console.WriteLine("G'Day Mate!");
} }
```

```
public class Kangaroo {
  _____________ fs;
  int Croc;
  int Dingo = 0;
  public int Wombat(int Wallaby) {
    _______ __;
    try {
      if (________ > 0) {
        __ = _____.OpenWrite("wobbiegong");
        Croc = 0;
      } else if (________ < 0) {
        Croc = 3;
      } else {
        ___ = _____.OpenRead("wobbiegong");
        Croc = 1;
    } }
    catch (IOException) {
        Croc = -3;
    }
    catch {
        Croc = 4;
    }
    finally {
      if (______ > 2) {
        Croc ___ Dingo;
    } }
    ________ ______;
} }
```

Nota: cada trecho dentro da piscina pode ser usado mais de uma vez!



Solução dos Jogos de Verão

Joey.Wombat() é chamado três vezes e na terceira retorna zero. Isso faz WriteLine() lançar uma DivideByZeroException.

A pista dada pelo fato disso ser um FileStream é que ele tem um método OpenRead(), que, por sua vez, lança uma IOException.

```
public static void Main() {
  Kangaroo Joey = new Kangaroo();
  int Koala = Joey.Wombat(Joey.Wombat(Joey.Wombat(1)));
  try {
    Console.WriteLine((15 / Koala) + " eggs per pound");
  }
  catch (DivideByZeroException) {
    Console.WriteLine("G'Day Mate!");
} }
public class Kangaroo {
  FileStream fs;
  int Croc;
  int Dingo = 0;
  public int Wombat(int Wallaby) {
    Dingo ++;
    try {
      if (Wallaby > 0) {
        fs = File.OpenWrite("wobbiegong");
        Croc = 0;
      } else if (Wallaby < 0) {
        Croc = 3;
      } else {
        fs = File.OpenRead("wobbiegong");
        Croc = 1;
      } }
    catch (IOException) {
      Croc = -3;
    }
    catch {
      Croc = 4;
    }
    finally {
      if (Dingo > 2) {
        Croc -= Dingo;
      } }
    return Croc;
} }
```

Este bloco catch apenas captura exceções de divisão por zero.

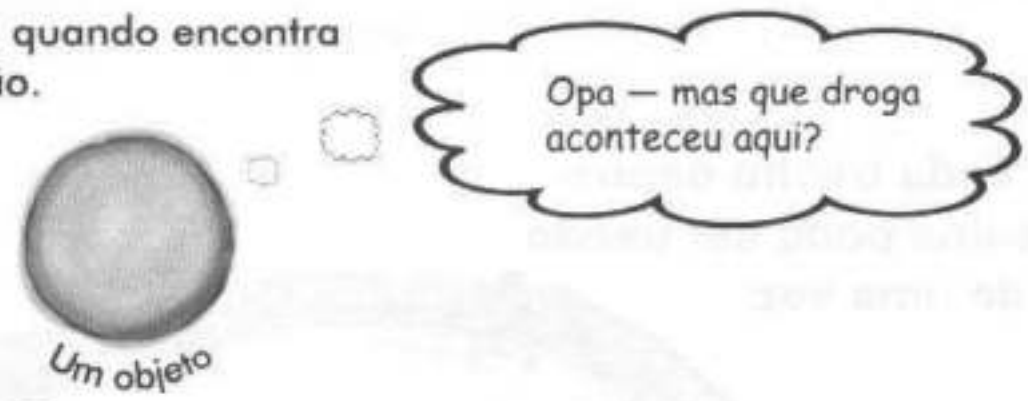
Este código abre um arquivo "wobbiegong" e o mantém aberto na primeira vez em que é chamado. Mais tarde, ele abre o arquivo de novo. Mas como ele nunca o fecha, isso faz lançar uma IOException.

Você já sabe que sempre deve fechar os arquivos depois de usá-los. Se não fizer isso, o arquivo continuará bloqueado por estar aberto e se você tentar abri-lo novamente uma IOException será lançada.

Use o objeto Exception para obter informações sobre o problema

Desde o começo dissemos que o .NET gera um objeto Exception quando uma exceção é lançada. Quando você escreve um bloco catch, tem acesso a esse objeto. Veja aqui como isso funciona:

1. Um objeto está fazendo o que tem que fazer, quando encontra alguma coisa inesperada e lança uma exceção.

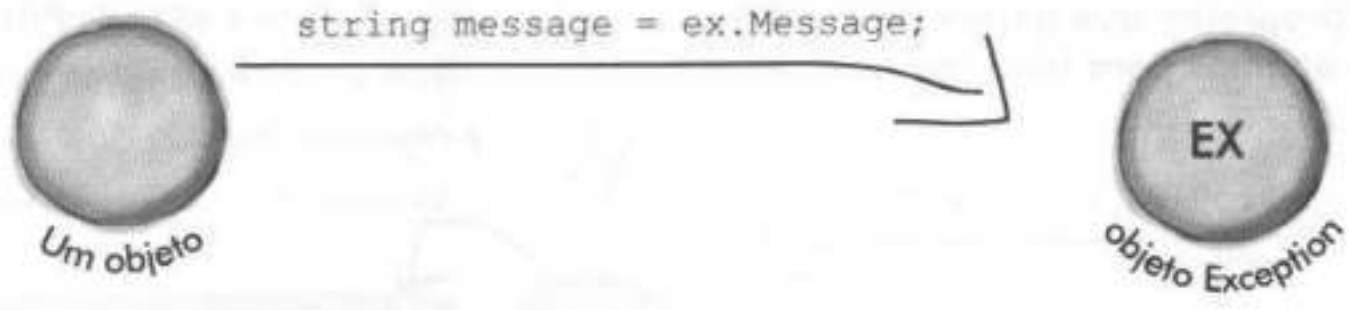


2. Por sorte, seu bloco try/catch capturou a exceção. Dentro do bloco catch, ele deu a ela um nome: **ex**.

```
try {
  DoSomethingRisky();
}
catch (Exception ex) {
  string message = ex.Message;
  MessageBox.Show(message, "An error occurred");
}
```

Quando você especifica um tipo em particular de exceção no bloco catch, se você fornecer um nome de variável, seu código pode usá-la para acessar o objeto Exception.

3. O objeto da exceção fica por ali até que o bloco catch tenha encerrado. Então a referência **ex** desaparece e é recolhida pelo coletor de lixo.



Use mais de um bloco catch para lidar com múltiplos tipos de exceção.

Você sabe que pode capturar um tipo específico de exceção... mas o que acontece se você escrever código onde mais de um problema pode ocorrer? Nesses casos, você pode querer escrever blocos que lidem com cada tipo de exceção possível. É para isso que usar mais de um bloco catch serve. Eis um exemplo de código para o sistema de controle de processamento de néctar da colméia. Você pode ver como ele captura muitos tipos de exceções. Em alguns casos ele usa propriedades do objeto Exception. É bastante comum usar a propriedade **Message**, que geralmente contém uma descrição da exceção.

Você também pode chamar o método ToString() da exceção para obter muitos dados pertinentes para sua caixa de mensagem.

```
public void ProcessNectar(NectarVat vat, Bee worker, HiveLog log) {
  try {
    NectarUnit[] units = worker.EmptyVat(vat);
    for (int count = 0; count < worker.UnitsExpected, count++) {
      stream hiveLogFile = log.OpenLogFile();
      worker.AddLogEntry(hiveLogFile);
    }
  }
  catch (IndexOutOfRangeException) {
    vat.Emptied = true;
  }
  catch (IOException ex) {
    worker.AlertQueen("Log file is corrupted: " + ex.Message);
  }
  catch (Exception ex) {
    worker.AlertQueen("An unspecified error happened: "
      + "Message: " + ex.Message + "\r\n"
      + "Stack trace: " + ex.StackTrace + "\r\n"
      + "Data: " + ex.Data + "\r\n");
  }
  finally {
    vat.Seal();
    worker.FinishedJob();
  }
}
```

Se você não usar o objeto Exception, não há necessidade de declará-lo.

Quando você tem muitos blocos catch, eles são executados na ordem em que aparecem. Nesse código, primeiro verifica-se se ocorre um índice fora dos limites. Se não for o caso, checa-se para ver se uma exceção de entrada/saída de arquivo ocorreu. O último bloco catch é um bloco geral que captura qualquer exceção e executará caso a exceção lançada não tenha sido capturada.

O bloco catch atribui a exceção para a variável ex, que ele usa para recuperar informações do objeto Exception.

Esse comando usa três propriedades do objeto Exception. Message, que tem a mensagem normalmente vista na janela de exceção no IDE ("Tentativa de dividir por zero"), StackTrace, que informa um sumário da pilha de chamadas, e Data, que algumas vezes contém dados pertinentes associados à exceção.

Sem problemas se dois blocos usam o mesmo nome ("ex", nesse caso) para a exceção.

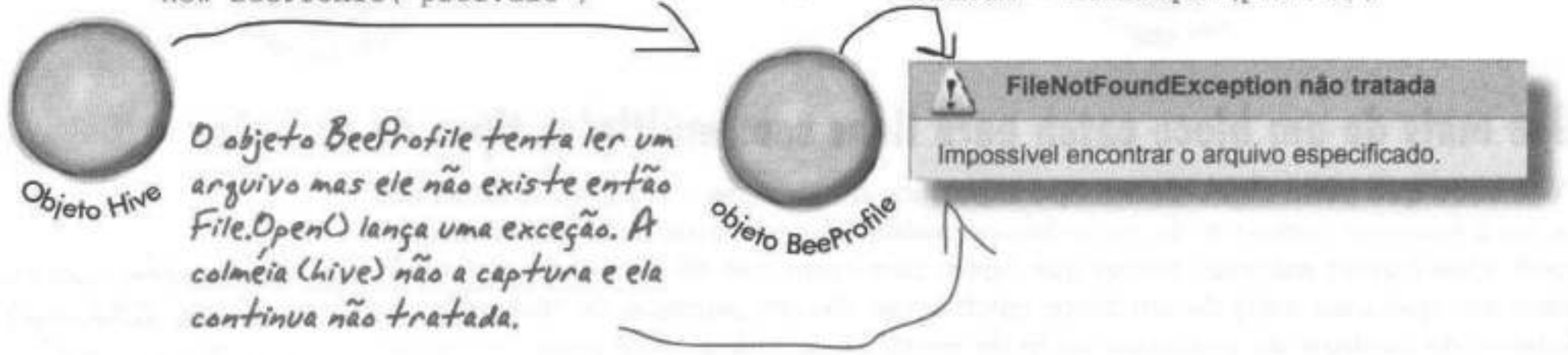
Uma classe lança uma exceção, outra a captura

Logicamente, um método dentro de uma única classe pode lançar e capturar suas próprias exceções.

Quando você está projetando uma classe, nem sempre sabe como ela será usada. Algumas vezes, outras pessoas usarão seus objetos de uma forma que causará problemas e, em outras, você mesmo fará isso! As exceções ajudam nisso também. Todo o propósito em lançar exceções é tentar perceber o que deu errado. Assim, você pode por para funcionar algum tipo de plano de contingência. Você normalmente não vê um método que lança uma exceção e a captura. Um exceção é normalmente lançada num método e então capturada num outro totalmente diferente - normalmente por um objeto diferente.

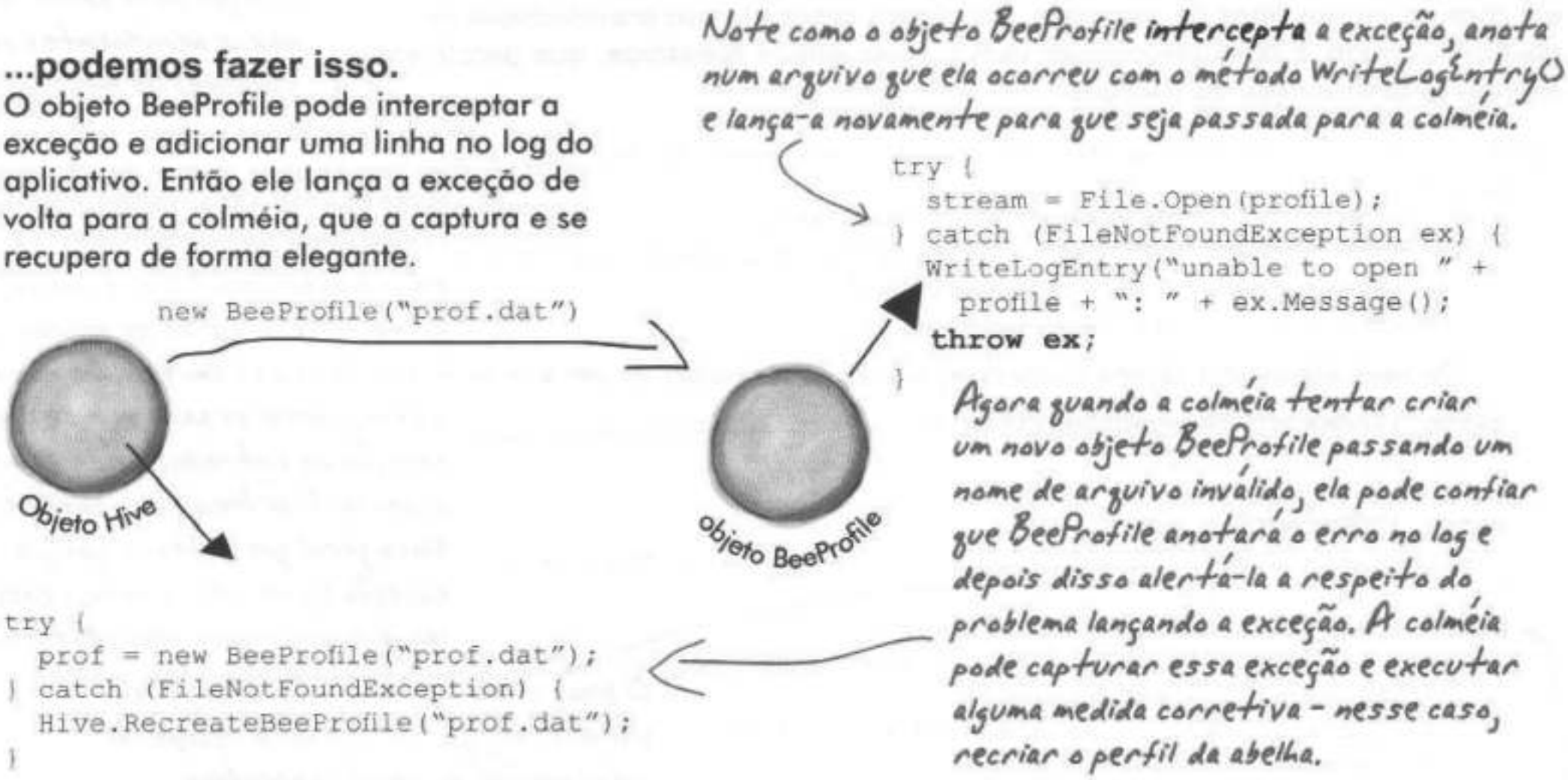
Em vez disso...

Sem um bom tratamento de exceções, uma delas pode parar o programa inteiro. Eis como funcionaria um programa que gerenciasse perfis de descrição de abelhas para uma rainha.



...podemos fazer isso.

O objeto BeeProfile pode interceptar a exceção e adicionar uma linha no log do aplicativo. Então ele lança a exceção de volta para a colméia, que a captura e se recupera de forma elegante.

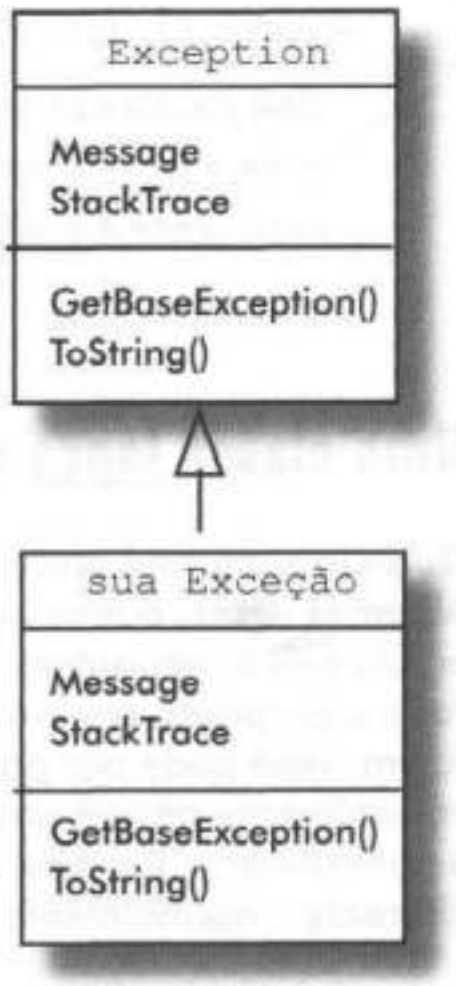


As abelhas precisam de uma exceção para esgotamento de mel

Suas classes podem lançar suas próprias exceções. Por exemplo, se você receber um parâmetro nulo num método que espera um valor, é bem comum lançar a mesma exceção que um método nativo do .NET usaria:

```
throw new ArgumentNullException();
```

Mas algumas vezes você gostaria que seu programa lançasse uma exceção quando uma condição especial é encontrada. As abelhas que criamos na colméia, por exemplo, consomem mel numa taxa diferente de acordo com seu peso. Se não existir mel para continuar, faz sentido fazer a colméia lançar uma exceção. Você pode criar uma exceção personalizada para lidar com essa condição de erro específica derivando sua própria classe de `Exception` e então lançá-la quando encontrar esse erro em particular.




```
public class OutOfHoneyException : System.Exception {
  public OutOfHoneyException(string message) : base(message) { }
}
public class HoneyDeliverySystem {
  ...
```

Você precisa criar uma classe para sua exceção e certificar-se de que ela herde de System.Exception. Note como estamos sobrecarregando o construtor para poder passar uma mensagem de exceção.

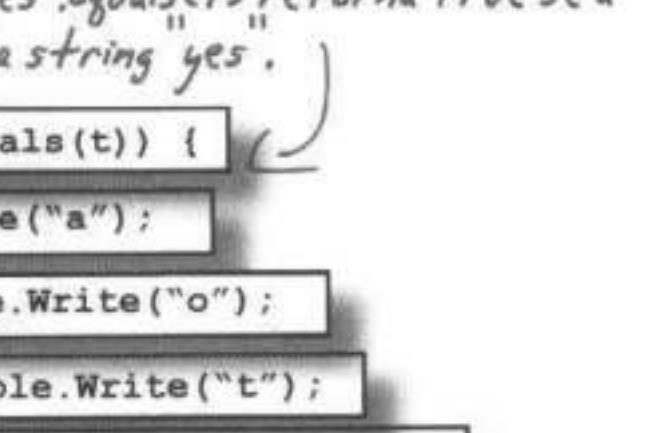
```
  public void FeedHoneyToEggs() {
    if (honeyLevel == 0) {
        throw new OutOfHoneyException("The hive is out of honey.");
    } else {
        foreach (Egg egg in Eggs) {
          ...
}
public partial class Form1 : Form {
...
  private void consumeHoney_Click(object sender, EventArgs e) {
    HoneyDeliverySystem delivery = new HoneyDeliverySystem();
    try {
        FeedHoneyToEggs()
    }
    catch (OutOfHoneyException ex){
      MessageBox.Show(ex.Message, "Warning: Resetting Hive");
      Hive.Reset();
    }
  }
}
```

Se há mel na colméia, a exceção nunca será lançada e esse código executará.

Isso lança uma nova instância do objeto de exceção.

Você pode capturar uma exceção personalizada pelo nome como qualquer outra e fazer qualquer coisa que queira para tratá-la.

Nesse caso, se a colméia estiver sem mel nenhuma das abelhas poderá trabalhar e o simulador não poderá continuar. A única forma de manter o programa funcionando nessas condições é reiniciá-la. Podemos colocar o código para fazer isso no bloco catch.



Ímãs de geladeira de exceção

Ordene os ímãs para que o aplicativo escreva o resultado no console.

resultado:
when it thaws it throws (que significa "quando ele derrete ele lança").

```
public static void Main() {
  Console.Write("when it ");
  ExTestDrive.Zero("yes");
  Console.Write(" it ");
  ExTestDrive.Zero("no");
  Console.WriteLine(".");
}
class MyException : Exception { }
```

Cada tipo de valor - incluindo constantes como yes (sim), ou 37, ou true - tem métodos nativos próprios. Assim "yes".Equals(t) retorna true se a variável t contém a string "yes".

```
if ("yes".Equals(t)) {
Console.Write("a");
Console.Write("o");
Console.Write("t");
Console.Write("w");
Console.Write("s");
try {
} catch (MyException) {
throw new MyException();
} finally {
Console.Write("r");
}
}
doRisky(test);
public class ExTestDrive {
  public static void Zero(string test) {
static void doRisky(String t) {
  Console.Write("h");
}
}
}
}
```

Solução dos Ímãs de geladeira de exceção

Ordene os ímãs para que o aplicativo escreva o resultado no console.

resultado:
when it thaws it throws (que significa "quando ele derrete ele lança").

```
public static void Main() {
  Console.Write("when it ");
  ExTestDrive.Zero("yes");
  Console.Write(" it ");
  ExTestDrive.Zero("no");
  Console.WriteLine(".");
}
class MyException : Exception { }
```

Essa linha define uma exceção personalizada chamada MyException, que é capturada por um bloco catch no código.

```
public class ExTestDrive {
  public static void Zero(string test) {
    try {
      Console.Write("t");
      doRisky(test);
      Console.Write("o");
    } catch (MyException) {
      Console.Write("a");
    } finally {
      Console.Write("w");
    }
    Console.Write("s");
  }
  static void doRisky(String t) {
    Console.Write("h");
    if ("yes".Equals(t)) {
      throw new MyException();
    }
    Console.Write("r");
  }
}
```

O método Zero() ou imprime "thaws" (derrete) ou "throws" (lança), dependendo de receber ou não "yes" como seu parâmetro test.

O bloco finally faz com que o método com certeza sempre imprima w. E o s é impresso fora do tratador de exceção e, por isso, é sempre impresso, também.

O método doRisky() somente lança a exceção se a ele for passada a string yes.

Essa linha é executada se doRisky() (fazer arriscado) não lançar a exceção.

Pontos de Bala

- Qualquer comando pode lançar uma exceção se alguma coisa falhar em tempo de execução
- Use um bloco `try/catch` para tratar exceções. Exceções não tratadas farão seu programa parar a execução e exibir uma janela de erro.
- Qualquer exceção no bloco de código depois do comando `try` fará com que o ponto de execução do programa salte imediatamente para o primeiro comando do bloco `catch`.
- O objeto `Exception` informa sobre a exceção capturada. Se você especificar uma variável de tipo `Exception` no comando `catch`, ela conterá essas informações a respeito de qualquer exceção lançada no bloco `try`:

```
try {
  // statements that might
  // throw exceptions
} catch (IOException ex) {
  // if an exception is thrown,
  // ex has information about it
}
```

- Há muitos tipos diferentes de exceções que você pode capturar. Cada uma tem seu próprio objeto que herda de Exception. Tente evitar sempre capturar Exception - capture exceções específicas.
- Cada `try` pode ter mais de um catch:

```
try { ... }
catch (NullReferenceException ex) {
  // these statements will run if a
  // NullReferenceException is thrown
}
catch (OverflowExceptiCn ex) { ... }
catch (Exception ex) {
  // Any exception that hasn't been
  // caught will jump to this block
}
```

- Seu código pode lançar uma exceção usando `throw`:

```
throw;
throw new Exception("Exception message");
```

- Você pode criar uma exceção personalizada herdando da classe base `Exception`.

```
class CustomException : Exception;
```

- Na maior parte do tempo, você precisa apenas lançar exceções nativas do .NET, como ArgumentException (exceção argumento). O motivo para usar tipos diferentes de exceções é poder **dar informações melhores a seus usuários**. Mostrar uma janela que diz "um erro desconhecido ocorreu" não é nem de longe tão útil quanto uma "a pasta de desculpas está vazia. Por favor escolha uma pasta diferente se você quiser carregar desculpas".

Uma forma fácil de evitar muitos problemas: using providencia um try e um finally para você

Você já sabe que using é uma forma fácil de certificar-se de que seus arquivos sempre sejam fechados. Mas o que você não sabia é que ele é **apenas um atalho do C#** para um `try` e um `finally`!

Lembre-se, quando você declara uma referência num comando using, seu método Dispose() (descartar) é automaticamente chamado no final do bloco.

```
using (YourClass c
       = new YourClass() ) {
  // código
}
```

é o mesmo que

```
YourClass c = new
YourClass();
try {
  // código
} finally {
  c.Dispose()
}
```

Quando você emprega um comando using, está se aproveitando do finally para se certificar de que o método Dispose() seja sempre chamado.

Evitando exceções: implemente IDisposable para fazer sua própria limpeza

IDisposable é uma forma efetiva de evitar exceções e problemas muito comuns. Certifique-se de usar o comando using quando estiver trabalhando com qualquer classe que a implemente.

Streams são ótimas, porque elas já têm código próprio para fechar a si mesmas quando o objeto é descartado. Mas e se você tiver o seu próprio objeto personalizado e ele sempre precisar que algo seja feito quando for descartado? Não seria ótimo se você pudesse escrever seu próprio código que executasse se você o utilizasse dentro de um bloco `using`?

O C# permite fazer exatamente isso com a interface `IDisposable`. Implementa-a e escreve seu código de finalização no método `Dispose()`, como no exemplo:

Você somente pode usar uma classe dentro de um bloco using se ela implementar IDisposable, caso contrário seu programa não compilará.

```
class Nectar : IDisposable {
  private double amount;
  private BeeHive hive;
  private Stream hiveLog;
  public Nectar(double amount, BeeHive hive, Stream hiveLog) {
    this.amount = amount;
    this.hive = hive;
    this.hiveLog = hiveLog;
  }
  public void Dispose() {
    Hive.Add(amount);
    Hive.WriteHiveLogEntry(hiveLog,
      amount + "mg of nectar was added");
  }
}
```

Seu objeto precisa implementar IDisposable se você quiser usá-lo num bloco using.

A interface IDisposable tem um único membro: o método Dispose(). O que for posto nele será executado no final do bloco using.

Esse código, em particular, sempre anota a quantidade de néctar adicionado. Ele é importante e precisa sempre executar, portanto colocamo-nos no método Dispose().

Você pode usar múltiplos comandos using. Primeiro, vamos usar um objeto antigo que implementa `IDisposable`, `Stream`. Então, trabalharemos com nosso objeto atualizado `Nectar`, que também implementará essa interface:

Freqüentemente você verá blocos using dentro uns dos outros.

```
using (Stream Log = new File.Write("log.txt"))
using (Nectar nect = new Nectar(16.3, hive, Log)) {
  Bee.FlyTo(flower);
  Bee.Harvest(nect);
  Bee.FlyTo(hive);
}
```

O objeto Nectar usa a stream Log, que fecha automaticamente no final do bloco using externo.

Então o objeto Bee usa o objeto nect, que automaticamente faz uma anotação no fim do bloco using interno.

não existem Perguntas Idiotas

P: Só posso usar objetos que implementem IDisposable num bloco using?

R: Sim. `IDisposable` foi criada para trabalhar com o comando `using` e usar um destes é como criar uma nova instância de uma classe, exceto que seu método `Dispose()` é sempre chamado.

P: Pode-se colocar qualquer comando dentro de um bloco using?

R: Com certeza. O propósito do `using` é ajudá-lo a certificar-se que todo objeto criado seja descartado. Mas o que será feito com esses objetos depende inteiramente de você. De fato, você pode criar um objeto com um comando `using` e nem mesmo criar um bloco para ele. Mas isso seria bastante inútil. Por isso, não recomendamos fazê-lo.

P: Pode-se chamar Dispose() fora de um comando using?

R: Sim. Você não é **obrigado** a usar um bloco `using`. Você pode chamar `Dispose()` por si mesmo quando terminar de usar um objeto. Ou pode fazer qualquer procedimento de encerramento que seja necessário - como chamar o método `Close()` de uma stream manualmente. Mas se você usar um bloco `using`, isso deixará seu código mais fácil de entender e prevenirá problemas decorrentes de esquecer-se de descartar seus objetos.

P: Você mencionou um bloco "try/finally". Isso significa tudo bem se eu tiver um bloco try sem um catch?

R: Sim! Você pode ter um bloco try sem um catch e ter apenas um finally. Ele se parecerá com isso:

```
try {
    DoSomethingRisky();
    SomethingElseRisky();
}
finally {
    AlwaysExecuteThis();
}
```

Se `SomethingRisky()` (algo arriscado) lançar uma exceção, o bloco finally executará imediatamente.

P: Dispose() só funciona com arquivos e streams?

R: Não, há muitas classes que implementam IDisposable e quando você estiver usando uma sempre use um comando using (você verá algumas classes assim nos próximos capítulos). E se você escrever uma classe que deva descartada de uma certa forma, poderá implementar essa interface também.

Se o try/catch é tão bom, porque o IDE não o coloca em volta de todos os comandos? Assim eu não teria de escrever eu mesmo esse monte de blocos de captura, certo?

Você desejará saber que tipos de exceção estão sendo lançados, para poder lidar com elas.

Existe mais a respeito de tratamento de exceções do que somente apresentar mensagens de erro genéricas. Por exemplo, no buscador de desculpas, se descobrirmos um `FileNotFoundException`, podemos exibir um erro que sugira onde os arquivos certos devem estar. Se tivermos uma exceção relacionada à base de dados, podemos enviar um e-mail para o administrador da base e etc. Tudo isso depende de você capturar tipos específicos de exceções.

É por isso que tantas classes herdam de Exception; e também o motivo de você querer escrever suas próprias classes de exceção.

A pior captura: bloco de comentários

Um bloco de captura manterá seu programa executando, se assim você quiser. Uma exceção é lançada, você captura-a e, em vez de desligar e apresentar uma mensagem de erro, você continua. Mas algumas vezes, isso não é tão bom assim.
Veja o DivisorCalculator que parece agir estranhamente a toda hora. O que está acontecendo?

```
public class Calculator {
...
  public void Divide(float dividend, float divisor) {
    try {
      this.quotient = dividend / divisor;
    } catch {
      // Nota do Jaime: precisamos descobrir uma forma de evitar
      // que as pessoas entrem um zero num problema com divisao
    }
  }
}
```

Aqui está o problema. Se o divisor for zero, isso lançará uma DivideByZeroException.

Mas existe um bloco catch. Então porque ainda temos erros?

O programador pensou que poderia esconder essa exceção usando um bloco catch vazio, mas apenas causou dor de cabeça para quem for rastrear os problemas ocorridos depois.

Você deve tratar suas exceções, não escondê-las

Só porque você pode manter o programa funcionando isso não quer dizer que você tratou suas exceções. No código acima, a calculadora não fechará ... pelo menos, não no método `Divide()`. Mas e se algum outro código chamar esse método e tentar imprimir os resultados? Se o divisor foi zero, o método provavelmente retornará um valor incorreto (e inesperado).

Em vez de apenas adicionar comentários e esconder suas exceções, **você precisa tratá-las**. E se você não conseguir eliminar o problema, `não deixe blocos catch vazios ou com comentários`! Isso só dificultará para outras pessoas descobrirem o que está acontecendo. É melhor deixar o programa continuar a lançar as exceções, pois aí, pelo menos, será fácil entender o problema.

Tudo bem usar soluções temporárias (temporariamente)

Algumas vezes você encontra um problema e sabe que ele é um problema, mas não está certo do que fazer a respeito. Nesses casos, você pode querer anotar o que está acontecendo. Isso não é tão bom quanto tratar a exceção, mas é melhor do que não fazer nada.

Eis aqui uma solução temporária para a calculadora:

```
public class Calculator {
...
  public void Divide(float dividend, float divisor) {
    try {
      this.quotient = dividend / divisor;
    } catch (Exception ex) {
      StreamWriter sw = new StreamWriter(@"C:\Logs\errors.txt");
      sw.WriteLine(ex.getMessage());
      sw.Close();
    }
  }
}
```

Isso ainda precisará se corrigido, mas a curto prazo, esclarece o local onde o problema ocorreu.

Entendi. É como usar o tratamento de exceções para colocar uma marca em volta da área com problemas.

Lidar com exceções nem sempre é o mesmo que CORRIGIR problemas.

Nunca é bom ter seu programa fechando sem querer. Mas é muito pior não ter a menor idéia do que o faz fechar ou o que está acontecendo com os dados dos usuários. É por isso que você precisa sempre lidar com os erros previsíveis e logar os imprevisíveis.

Algumas idéias simples para tratamento de exceções

Projete seu código para lidar com falhas de forma ELEGANTE.

Dê a seus usuários mensagens de erro ÚTEIS.

Lance exceções nativas do .NET sempre que puder. Use apenas tipos personalizados se precisar dar informações personalizadas aos usuários.

Pense em código para seus blocos try que POSSAM ser sofrer curto-circuito.

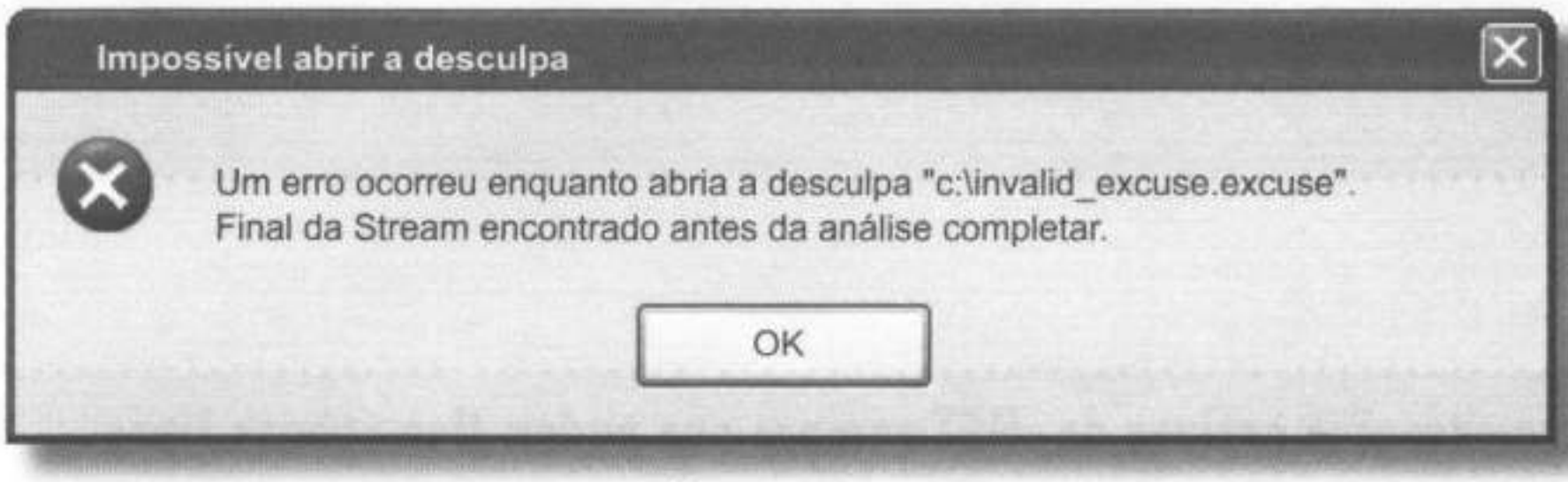
...e principalmente...

Evite erros desnecessários do sistema de arquivos... SEMPRE USE UM BLOCO USING QUANDO UTILIZAR UMA STREAM! SEMPRE SEMPRE SEMPRE!

Exercício

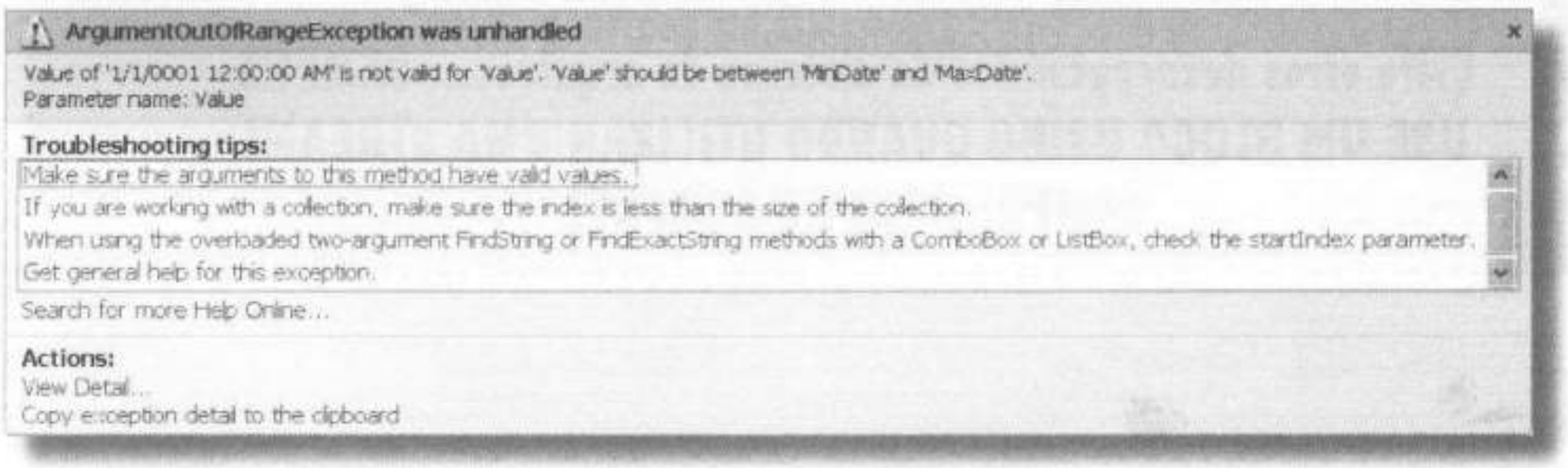
Use o que você sabe sobre `try/catch/finally` para melhorar o tratamento de exceções no gerenciador de desculpas do Aldo.

1. Adicione tratamento de exceção no método que responde ao clique do botão Open. Faça somente um bloco try/catch simples que mostre uma caixa de mensagem. Eis o que ela deveria mostrar se você tentar abrir um arquivo que não seja realmente um de desculpas:



Você verá essa mensagem de exceção com um arquivo bem pequeno, mas um maior levantaria outro erro ("The input stream is not a valid binary format...", ou "a stream de entrada não está num formato binário válido"). Sua caixa de mensagem deve funcionar em qualquer caso.

2. Você ainda não acabou. Abra o gerenciador de desculpas, selecione uma pasta, entre dados nas caixas "Descrição" e "Últimos Resultados", mas não digite uma data de última utilização. Agora selecione um diretório e tente salvar a desculpa. Acontece com você essa exceção de ArgumentOutOfRange (argumento fora dos limites)?



Use o depurador para rastrear a exceção. Essa, em particular, pode ser totalmente evitada - você pode resolver o problema e certificar-se de que a exceção nunca mais ocorra? (Dica: isso não tem nenhuma relação com adicionar um bloco try/catch. Você deverá deduzir o motivo de a data de "última utilização" causar um problema. Examine cuidadosamente a mensagem de exceção procurando por pistas.)

3. Um último detalhe. Antes que seu programa lance a exceção ArgumentOutOfRange, ele salvou um arquivo. Carregue-o – a mesma exceção ocorrerá. E você verá que se tentar abrir outro arquivo que não é um de desculpas, ocorrerá uma exceção diferente. Adicione um bloco de tratamento de exceção aninhado dentro daquele que você já fez no passo 2 para se certificar de que ele não falhará quando tentar carregar um arquivo de desculpas inválido (possível em muitas situações). Eis o que fazer:

   1. Declare uma variável booleana chamada clearForm (limpar formulário) acima do bloco try/catch. Você dará a ela valor true se ocorrer uma exceção e verificará mais adiante se o formulário deve ser limpo.
   2. Adicione outro bloco try/catch dentro daquele que você acabou de adicionar ao botão Open.
   3. Adicione um bloco finally no try/catch externo para reiniciar o formulário para seu estado original vazio. Reinicie LastUsed. Value to DateTime.Now (que retorna a data atual) se a variável clearForm for verdadeira.

Solução do Exercício

Use o que você sabe sobre try/catch/finally para melhorar o tratamento de exceções no gerenciador de desculpas do Aldo.

Aqui está o bloco try/catch para criar uma caixa de mensagem que mostre um erro, no caso de algum problema ocorrer quando o formulário chamar o construtor de Excuse para carregar uma desculpa.

```
private void open_Click(object sender, EventArgs e) {
  if (CheckChanged()) {
    openFileDialog1.InitialDirectory = selectedFolder;
    openFileDialog1.Filter =
      "Excuse files (*.excuse)|*.excuse|All files (*.*)|*.*";
    openFileDialog1.FileName = description.Text + ".excuse";
    DialogResult result = openFileDialog1.ShowDialog();
    if (result == DialogResult.OK) {
      bool clearForm = false;
      try {
        currentExcuse = new Excuse(openFileDialog1.FileName);
        try {
          UpdateForm(false);
        }
        catch {
          MessageBox.Show("The excuse file '"
            + openFileDialog1.FileName + "' is invalid",
            "Unable to open the excuse");
          clearForm = true;
        }
      }
      catch (Exception ex) {
        MessageBox.Show("An error occurred while opening the excuse '"
          + openFileDialog1.FileName + "'\n" + ex.Message,
          "Unable to open the excuse", MessageBoxButtons.OK,
          MessageBoxIcon.Error);
        clearForm = true;
      }
      finally {
        if (clearForm) {
          description.Text = "";
          results.Text = "";
          lastUsed.Value = DateTime.Now;
        }
      }
    }
  }
}
```

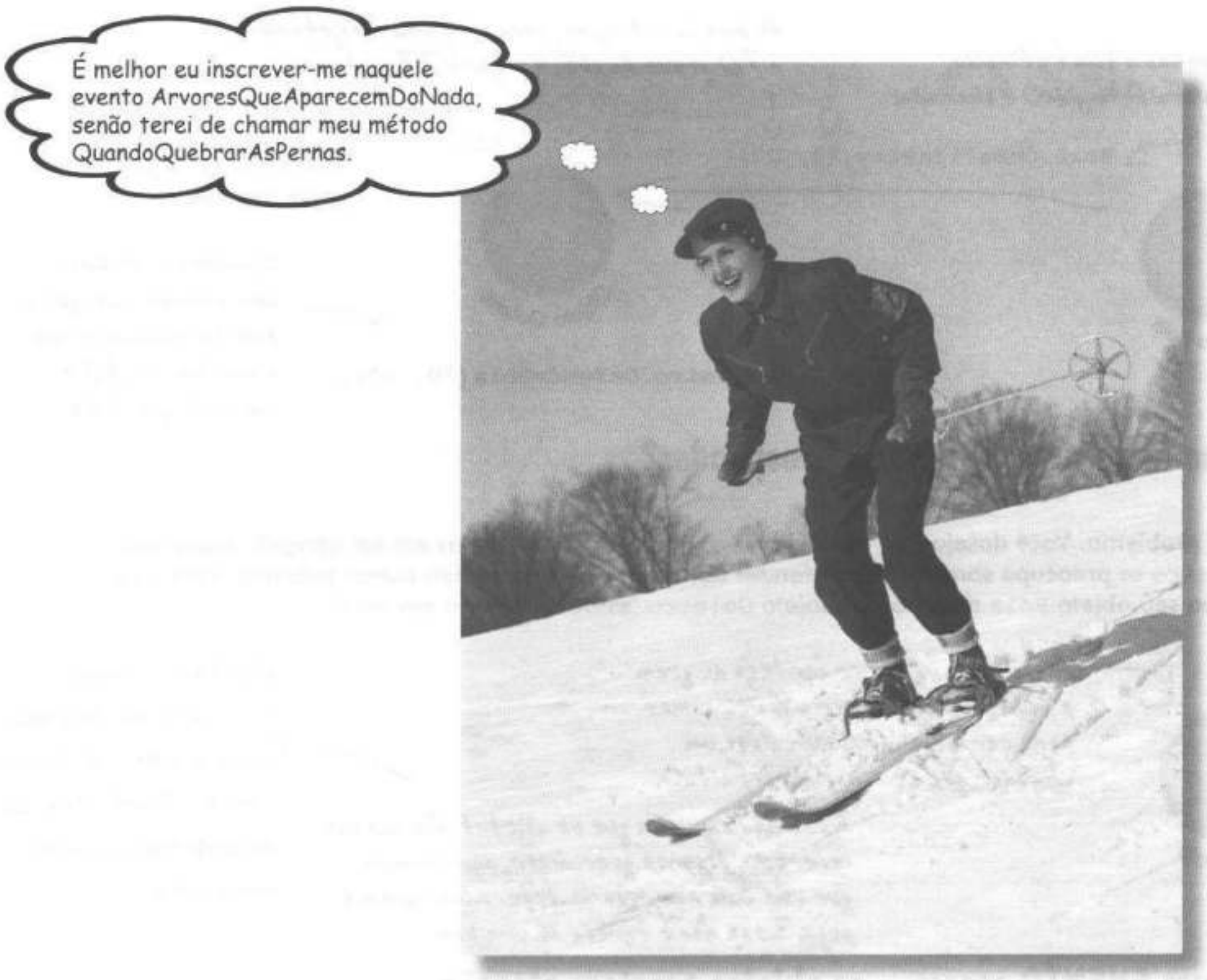
Eis o try/catch aninhado. Ele trata exceções que acontecem se o arquivo é carregado, mas contém dados inválidos. Isso não é o mesmo que os problemas do construtor de Excuse.

Aqui temos a caixa de mensagem do bloco try/catch externo. Ele mostra a mensagem de exceção.

*Ambos os blocos de captura atribuem true para clearForm para que esse bloco finally **saiba** que o formulário deve ser reiniciado. Tudo bem ter código que interage com seu bloco finally, pois você sabe que ele sempre será executado.*

11 eventos e delegados

O que o seu código faz quando você não está olhando



Seus objetos estão começando a pensar por eles mesmos.

Você nem sempre pode controlar o que um objeto está fazendo. Algumas coisas... simplesmente acontecem. E quando acontecem, você desejará que seus objetos sejam inteligentes o suficiente para **responder a qualquer** coisa que apareça. Esta é a finalidade dos eventos. Um objeto publica um evento, outro objeto inscreve-se nele e todos trabalham juntos para manter as coisas funcionando. O que é ótimo, até que você tenha vários objetos respondendo ao mesmo evento. Nesses casos os **callbacks** serão úteis.

Já desejou que seus objetos pensassem por si mesmos?

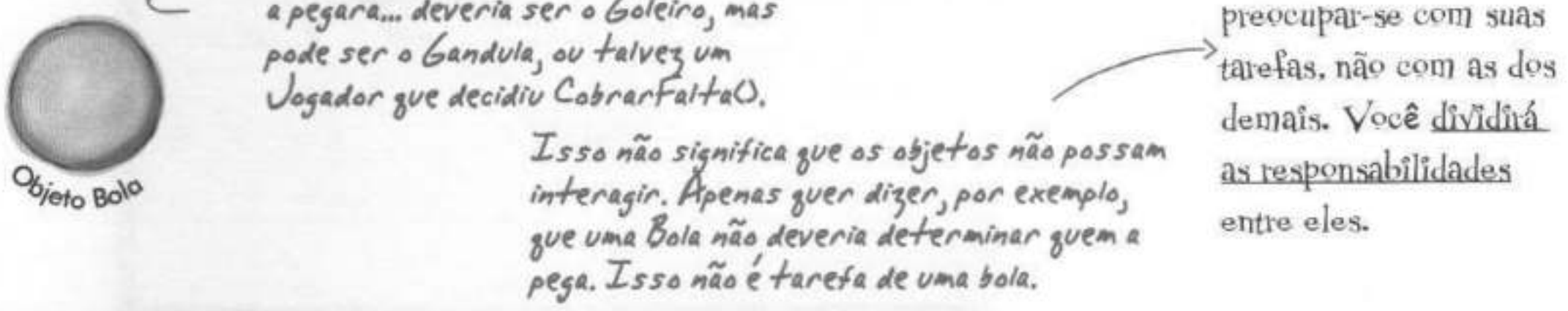
Suponha que você esteja escrevendo um simulador de futebol. Você modelará o jogo, vender o programa para um grande time (eles tem muito dinheiro, certo?) e ficar rico. Você cria seus objetos `Bola`, `Goleiro`, `Juiz`, `Torcedor` e muitos outros. Você até escreveu código para que Goleiro possa defender uma bola.

Esta é uma forma padronizada de nomear métodos - falaremos mais sobre isso daqui a pouco.

Agora você só precisa unir tudo. Coloque um método `OnBolaEmJogo()` (on é uma das preposições "em" do inglês e a expressão pode ser entendida como "quando a bola estiver em jogo"). Agora, você deseja que seu objeto `Goleiro` responda com seu método de tratamento de evento. Uma vez que os métodos tenham sido escritos, você precisa associá-los.

Mas como um objeto SABE como responder?

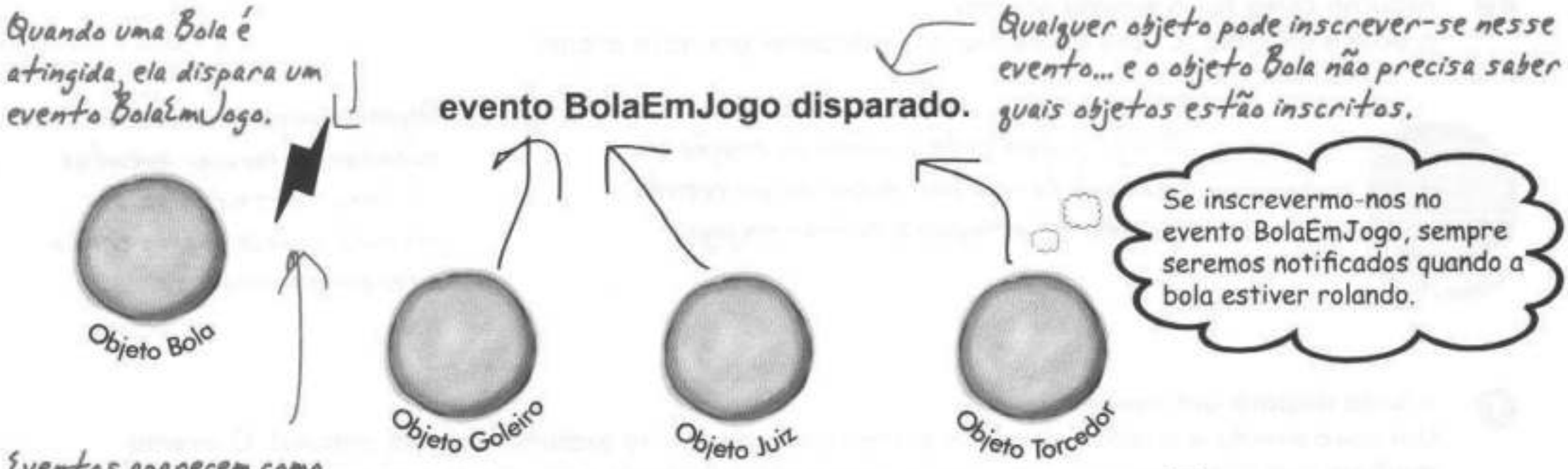
Aqui está o problema. Você deseja que seu objeto `Bola` se preocupe apenas em ser atingido e que seu objeto `Goleiro` se preocupe somente em defender bolas em sua direção. Em outras palavras, você não desejará que seu objeto `Bola` diga ao seu objeto `Goleiro` "estou chegando em você".



Quando um EVENTO ocorre... os objetos prestam atenção

O que você precisa fazer quando a bola é atingida é usar um evento. Um evento é simplesmente **algo que acontece** em seu programa. Os objetos podem responder a eventos – como, por exemplo, nosso objeto `Goleiro`.
Ainda melhor, mais de um objeto pode prestar atenção num evento. Assim o Goleiro pode ficar atento para um evento bola-sendo-atingida, bem como o `Gandula`, outros objetos `Jogador`, o `Juiz` e até mesmo um `Torcedor`. E cada um deles pode responder ao evento de forma diferente. Então o que queremos é um objeto `Bola` que possa **disparar um evento**.
Além disso, queremos que outros objetos se **inscrevam naquele tipo particular de evento**... o que apenas quer dizer para prestar atenção nele e ser notificado quando o evento ocorre.

evento, substantivo.
algo que acontece, especialmente algo importante. *O eclipse solar foi um evento espetacular de se assistir.*



Quer FAZER ALGO com um evento?

Você precisa de um tratador de eventos.

Uma vez que seu objeto "ouviu" que o evento ocorreu, ele pode executar algum código, que será chamado de **tratador de evento**. Esse código recebe informações sobre o evento, e executa quando o evento ocorre.

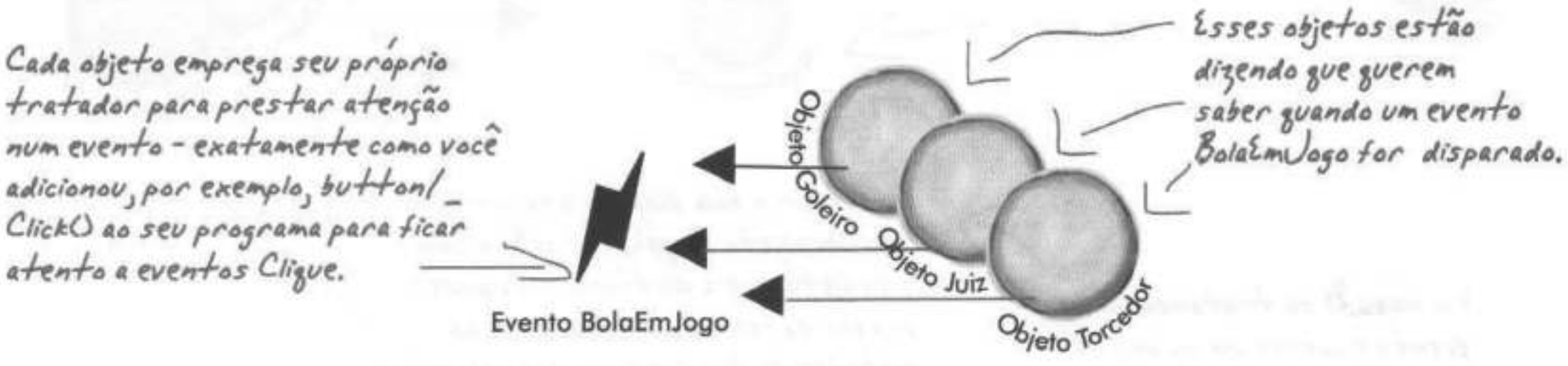
Nós estamos fazendo isso desde o início. Toda vez que você clica num botão, um evento é disparado e seu código responde a ele.

Lembre-se de que tudo isso **ocorre sem a sua intervenção** em tempo de execução. Então você escreve código para disparar um evento, código para tratar dele quando ele ocorre e ativa seu aplicativo. Assim, quando o evento ocorrer, seus tratadores são acionados... **sem você ter de fazer qualquer coisa**. E, melhor de tudo, seus objetos têm responsabilidades distintas. Eles preocupam-se com eles mesmos, não com os outros.

Um objeto dispara seus eventos e os outros ficam atentos

Vamos examinar como eventos, tratadores e inscrições funcionam em C#.

1 **Primeiro, outros objetos inscrevem-se no evento**
Antes que `Bola` possa disparar seu evento `BolaEmJogo`, outros objetos precisam inscrever-se nele. Essa é a forma deles dizerem "quando um evento `BolaEmJogo` ocorrer quero avisado".

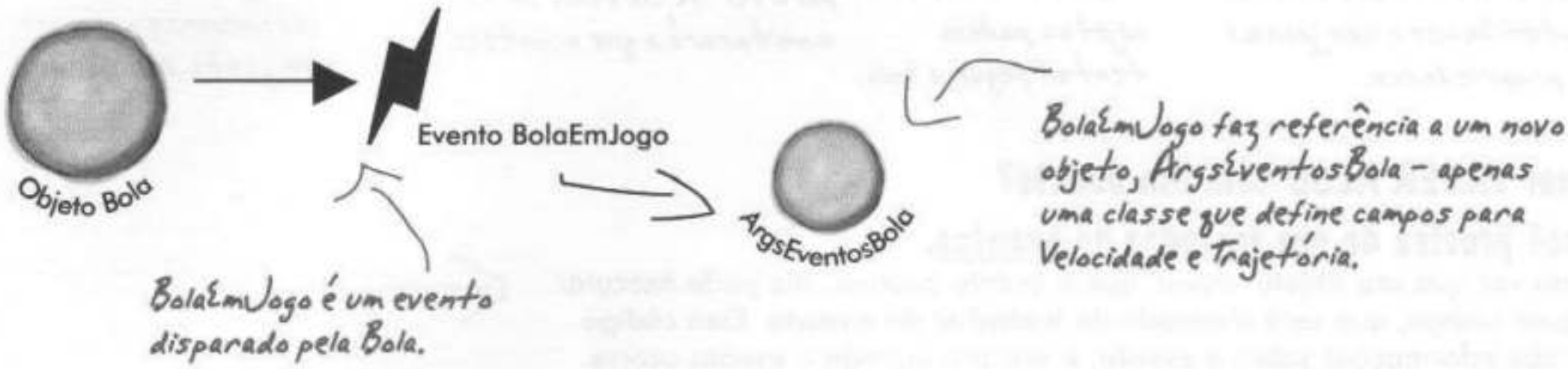


2 **Alguma coisa faz o evento ocorrer**
A bola é atingida. É hora do objeto `Bola` disparar um novo evento.

O objeto Bola está no início da reação em cadeia. É tarefa dele disparar um evento quando for atingido e entrar em jogo.

Algumas vezes falamos em levantar ou invocar eventos - é tudo a mesma coisa. As pessoas apenas usam termos diferentes para isso.

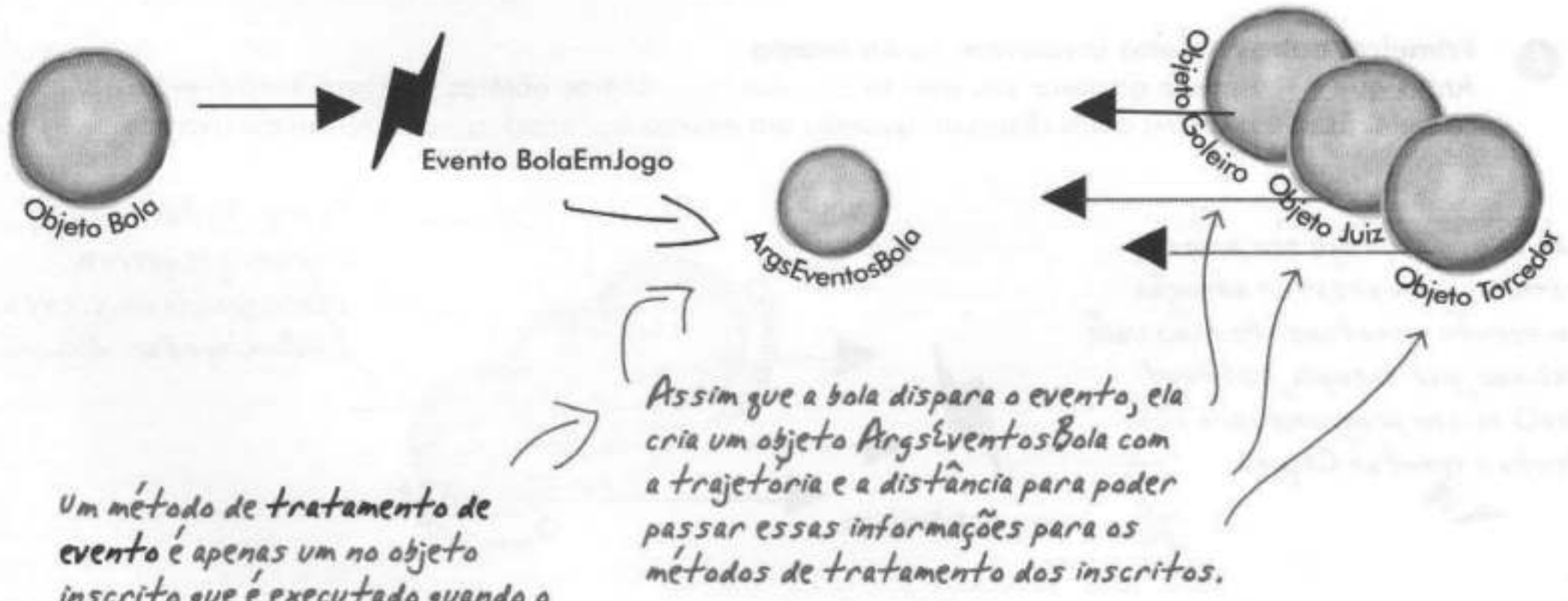
3 **A bola dispara um evento**
Um novo evento é criado (falaremos sobre como isso é feito exatamente num minuto). O evento também tem alguns argumentos, nesse caso a velocidade da bola, assim com sua trajetória. Eles são anexados ao evento como uma instância do objeto ArgsEventosBola e então o evento é enviado, ficando disponível para qualquer um que estiver prestando atenção nesse tipo de evento.



Então outros objetos tratam o evento

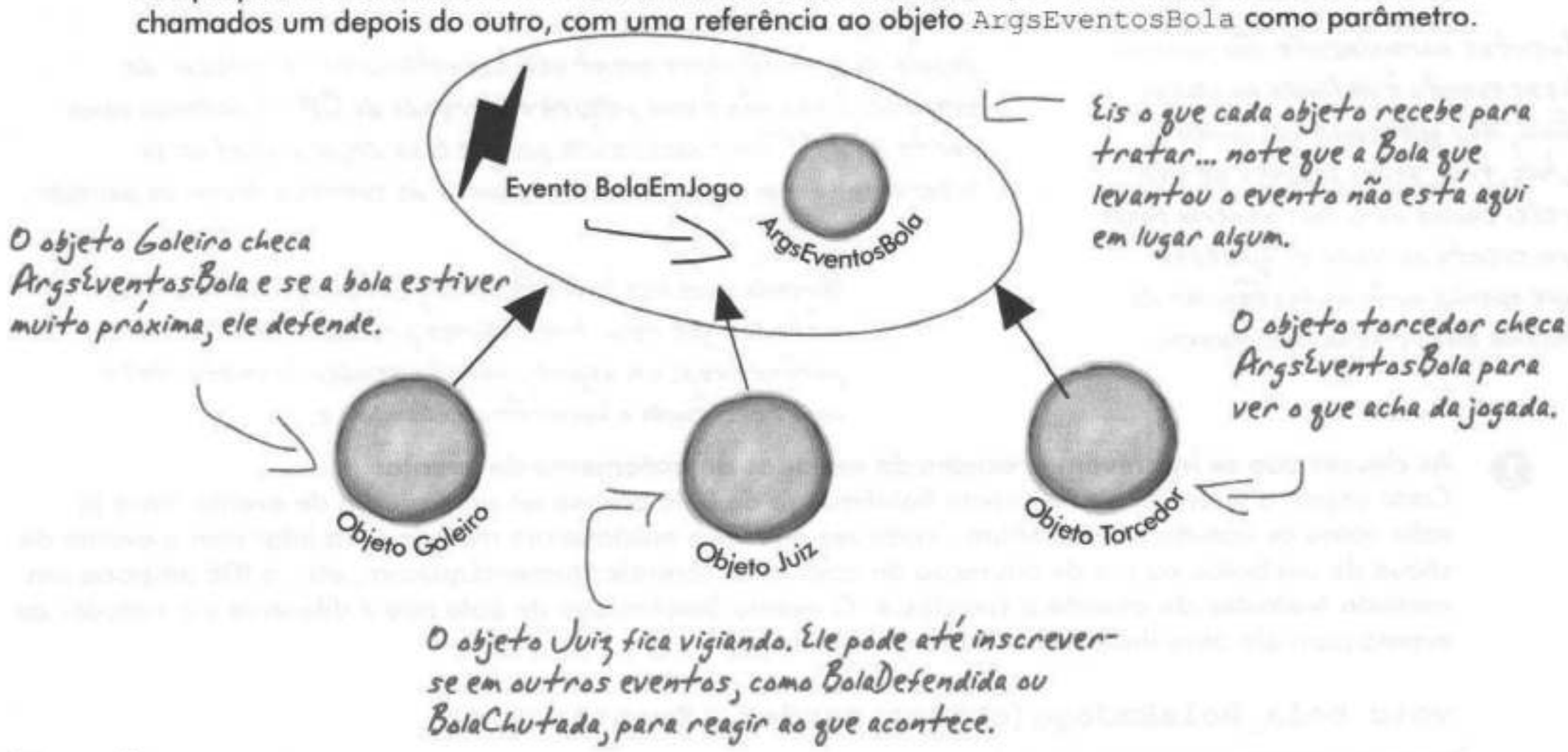
Uma vez que um evento tenha sido disparado, todos os objetos que se inscreveram são notificados e podem fazer algo em resposta.

4 **Inscritos são notificados**
Uma vez que o `Goleiro`, o `Juiz` e o `Torcedor` se inscreveram no objeto `Bola` para o evento `BolaEmJogo`, todos são notificados - os métodos tratadores de evento de todos eles são chamados um depois do outro.



Eventos são tratados numa seqüência do tipo primeiro-a-se-inscrever, primeiro-a-chamar - o objeto que se inscreve primeiro é notificado antes.

5 **Cada objeto trata o evento**
Agora o `Goleiro`, o `Juiz` e o `Torcedor` podem todos tratar o evento `BolaEmJogo`, cada uma da sua própria forma. Mas eles não fazem isso todos ao mesmo tempo - seus tratadores de eventos são chamados um depois do outro, com uma referência ao objeto `ArgsEventosBola` como parâmetro.



Ligando os pontos

Agora que você já entendeu o que está acontecendo, vamos examinar melhor como as peças encaixam-se. Por sorte, temos apenas algumas poucas peças móveis.

É uma boa idéia (embora não seja obrigatório) que seus argumentos de eventos herdem de `EventArgs`. Essa é uma classe vazia - ela não tem membros públicos.

Isso implica que você poderá fazer **supercoerção** do seu objeto de argumentos de evento para `EventArgs`, no caso de ter de passá-lo para um evento que não sabe como lidar com ele.

1 **Precisamos de um objeto para os argumentos do evento**
Lembre-se, nosso evento `BolaEmJogo` tem alguns argumentos que carrega consigo. Logo, precisamos de um objeto bem simples para conter esses argumentos. O .NET tem uma classe padrão para isso chamada **EventArgs**, mas ela **não tem membros**. Seu único propósito é permitir aos seus argumentos de evento serem passados para tratadores que possam lidar com eles. Eis aqui a declaração de classe:

```
public class ArgsEventosBola: EventArgs
```

A bola usará essas propriedades para passar informações sobre ela para os tratadores de eventos para que estes, por sua vez, tentem determinar onde ela estará.

2 **Depois deveremos definir o evento na classe que irá dispará-lo**
A classe Bola terá uma linha com a **palavra-chave event** - é assim que ela informa outros objetos sobre o evento, para que eles possam se inscrever. Essa linha pode estar em qualquer lugar na classe – normalmente, perto das declarações de propriedades. Mas desde que esteja dentro da classe Bola, outros objetos podem inscrever-se no evento declarado. Isso se parecerá com:

```
public event EventHandler BolaEmJogo;
```

Eventos normalmente são públicos. Esse evento é definido na classe Bola, mas queremos que Goleiro, Juiz, etc., sejam capazes de ter referências dele. Você poderia fazer um evento privado se quisesse que apenas outras instâncias da mesma classe se inscrevessem.

Depois da palavra-chave event vem EventHandler (tratador de evento). Isso **não é uma palavra reservada do C#** - é definida como parte do .NET. Ela é necessária porque isso diz aos objetos se inscrevendo com o que seus tratadores de eventos devem se parecer.

Quando você usa EventHandler, está dizendo a outros métodos que seus tratadores precisam ter dois parâmetros: um object chamado sender (remetente) e uma referência a EventArgs chamada e.

3 **As classes que se inscrevem precisam de métodos de tratamento de eventos**
Cada objeto a inscrever-se no evento BolaEmJogo de Bola precisa ter um tratador de evento. Você já sabe como os tratadores funcionam - cada vez que você adiciona um método para lidar com o evento de clique de um botão ou um de alteração de valor num controle NumericUpDown, etc., o IDE adiciona um **método tratador de evento** à sua classe. O evento BolaEmJogo de Bola não é diferente e o tratador de evento para ele deve lhe parecer bastante familiar.

```
void bola_BolaEmJogo(object sender, EventArgs e)
```

Não existe regra no C# que obrigue os tratadores de eventos terem nomes especiais, mas existe uma forma tácita, quase padronizada, de convenção de nomeação: o nome da referência do objeto, seguido por um caractere de sublinhado, seguido pelo nome do evento.

A declaração do evento BolaEmJogo lista o tipo de evento como EventHandler, ou seja, ele precisa de dois parâmetros - um object chamado sender e um EventArgs chamado e - e nenhum valor de retorno.

A classe com esse método de tratamento de evento em particular tem uma variável de referência tipo Bola chamada "bola", logo seu evento BolaEmJogo começa com "bola", seguido pelo nome do evento sendo tratado, "BolaEmJogo".

4 **Cada objeto individual inscreve-se no evento**
Uma vez nossos tratadores de eventos prontos, os diversos objetos, Goleiro, Juiz, Jogador e Torcedor, precisam associar seus tratadores ao evento. Cada um deles terá seu próprio método específico `bola_BolaEmJogo` que responderá de forma diferente ao evento. Então se existir uma variável ou campo de referência a um objeto tipo Bola chamada `ball`, o operador += fará essa associação:

```
bola.BolaEmJogo += new EventHandler(bola_BolaEmJogo);
```

Isso diz ao C# para associar o tratador com o evento BolaEmJogo qualquer que seja o objeto que a referência "bola" esteja apontando.

O operador += diz ao C# para inscrever um tratador num evento.

Essa parte especifica qual método de tratamento inscrever no evento.

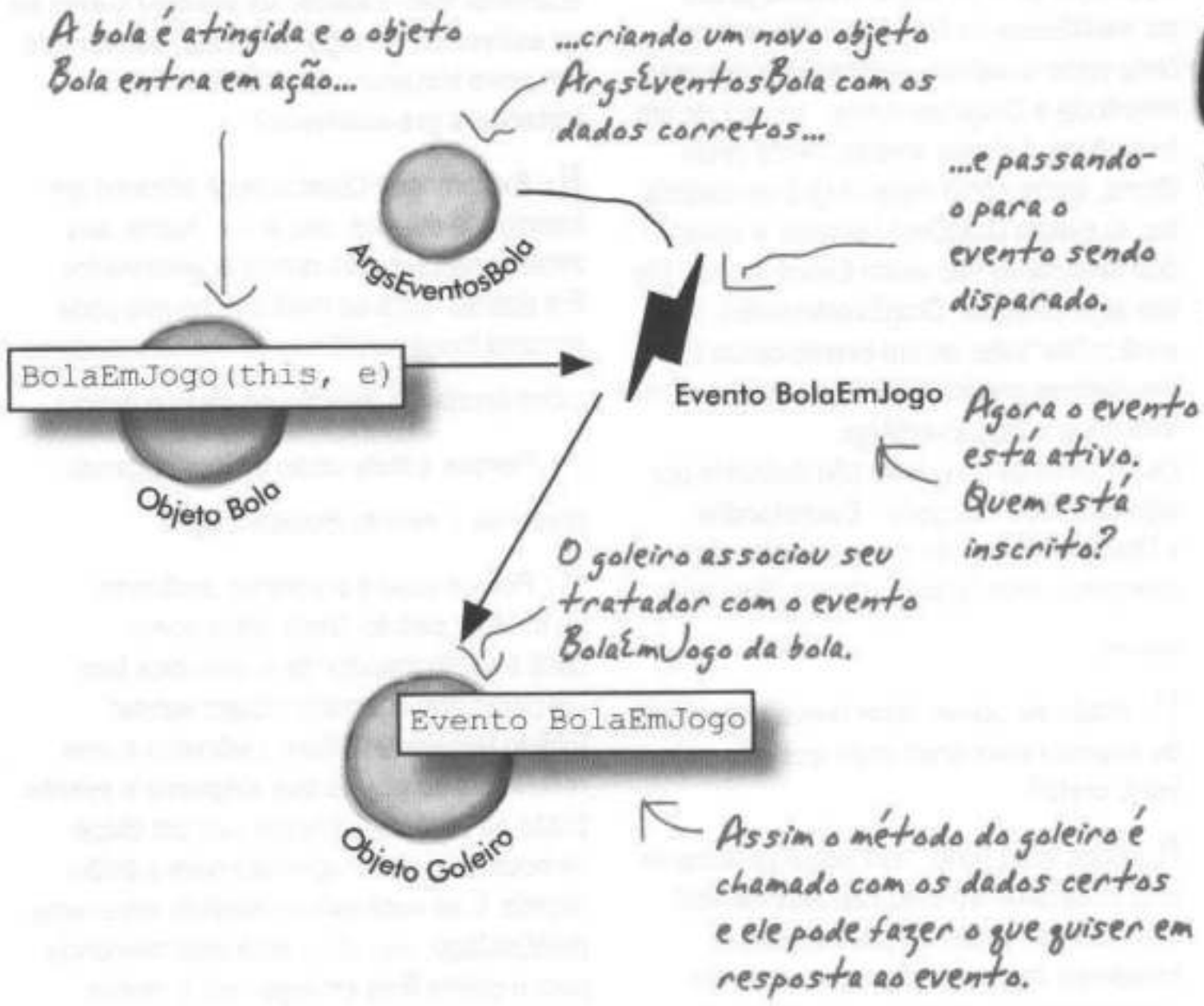
A assinatura do método tratador de evento (seus parâmetros e valor de retorno) deve **equivaler** àquela definida por EventHandler ou o programa **não compilará**.

Vire a página, ainda há alguns detalhes... ⟶

5 **O objeto Bola dispara seus eventos para notificar os inscritos que está em jogo**
Agora que o sistema de eventos está todo no lugar, a Bola **pode disparar seus eventos** em resposta a alguma outra coisa que acontece no simulador. Disparar um evento é fácil - basta chamar o evento BolaEmJogo, por exemplo.

```
if (BolaEmJogo != null)
    BolaEmJogo(this, e);
```

"e" é um novo objeto ArgsEventosBola.



Veja bem!

Se você disparar um evento sem tratadores, uma exceção será lançada.

Se nenhum objeto adicionou tratadores ao evento, ele será nulo. Por isso, cheque sempre para ter certeza de que o evento não é nulo antes de dispará-lo. Se você não fizer isso, uma exceção NullReferenceException poderá ser lançada.

Use um nome padronizado quando você adicionar um método para disparar um evento

Tire um minuto para olhar o código de qualquer formulário e digite a palavra-chave override em qualquer lugar onde você declararia um método. Assim que você pressionar o espaço, uma janela do IntelliSense aparecerá:

```
override
```

Veja como cada um desses métodos usa um parâmetro EventArgs. Eles todos passam esse parâmetro adiante para um evento que eles dispararão.

Existe um enorme número de eventos que um objeto Form pode disparar e cada um deles tem seus próprios métodos que o disparam. O método OnDoubleClick() (quando do duplo clique) dispara o evento DoubleClick e é para isso que ele existe. Assim o evento de Bola deve seguir a mesma convenção: certificaremo-nos de **haver um método OnBolaEmJogo** que tenha como parâmetro um ArgsEventosBola. O simulador de futebol irá chamá-lo quando precisar que a bola dispare seu evento BolaEmJogo - assim quando o simulador detectar que alguém chutou a bola, criará uma nova instância de ArgsEventosBola com a trajetória da bola e a distância e a passará para OnBolaEmJogo().

não existem Perguntas Idiotas

P: Por que devo incluir o termo EventHandler quando declaro um evento? Pensei que um EventHandler (tratador de evento) fosse o que o outro objeto usa para se inscrever em eventos.

R: É isso mesmo - quando você precisa se inscrever num evento, usa um método tratador de eventos. Mas você notou como usamos o termo `EventHandler` na declaração (passo 2) e também na linha que inscreve o tratador de evento (passo 4)? O que `EventHandler` está fazendo é definir a **assinatura** do evento - isso diz aos objetos para assinar o evento exatamente como eles precisam para definir seus métodos de tratamento. Especificamente, ele implica que se você quer inscrever um método num evento, o primeiro precisará receber dois parâmetros (um `object` e uma referência a `EventArgs`) e ter valor de retorno `void`.

P: O que acontece se eu tentar usar um método não equivalente àqueles definidos por EventHandler?

R: Seu programa não compilará. O compilador vai certificar- se de que você não inscreveu acidentalmente um método de tratamento incompatível com um evento. É por isso que o tratador padrão, `EventHandler`, é tão útil - assim que você notá-lo, saberá exatamente com o quê seu método de tratamento de evento se parece.

P: Espere, tratador de evento "padrão"? Existem outros tipos?

R: Sim! Seus eventos não precisam enviar um objeto e uma referência a EventArgs. De fato, eles podem enviar qualquer coisa - ou nada! Olhe na última linha na janela do IntelliSense no final da página anterior. Note como o método OnDragDrop usa uma referência a DragEventsArgs, em vez de um EventArgs. A classe anterior herda desta última, assim como nosso ArgsEventosBola faz. O evento DragDrop (arrastar e soltar) dos formulários não usam EventHandler. Ele usa algo diferente, DragEventHandler, e se você quiser tratar de um evento desse tipo seu método precisa receber um object e uma referência a DragEventArgs.
Os parâmetros do evento são definidos por algo chamado delegado - EventHandler e DragEventArgs são dois exemplos de delegados. Mas falaremos mais disso num minuto.

P: Então eu posso fazer meus tratadores de evento retornarem algo que não seja void, certo?

R: Bom, você pode, mas isso é geralmente uma idéia ruim. Se você não retornar void num tratador, você não pode encadear tratadores. Isso quer dizer que você não poderá conectar mais de um tratador em cada evento. Uma vez que encadeamento é uma característica útil, é melhor você sempre retornar void nos seus métodos de tratamento.

P: Encadeamento? O que é isso?

R: É como mais de um objeto pode se inscrever no mesmo evento - eles podem encadear seus tratadores num evento, um depois do outro. Falaremos mais sobre isso num minuto, também.

P: É por isso que eu usei o += quando adicionei meu tratador de evento? Como se eu estivesse, de alguma forma, adicionado um novo tratador a um conjunto de tratadores pré-existente?

R: Exatamente! Quando você adiciona um tratador de eventos, usa o `+=`. Assim, seu tratador não substitui outros já adicionados. Ele apenas torna-se mais um, no que pode ser uma longa seqüência de outros tratadores, todos prestando atenção no mesmo evento.

P: Porque a Bola usou o "this" quando disparou o evento BolaEmJogo?

R: Porque esse é o primeiro parâmetro do tratador padrão. Você notou como cada método tratador de evento click tem um parâmetro chamado "object sender" (objeto remetente)? Esse parâmetro é uma **referência ao objeto que disparou o evento**. Então se você está lidando com um clique de botão, `sender` apontará para o botão clicado. E se você estiver tratando um evento BolaEmJogo, `sender` será uma referência para o objeto Bola em jogo - eis o motivo para a Bola colocar `this` como primeiro parâmetro quando levanta o evento.

Um ÚNICO evento é sempre disparado por um ÚNICO objeto. Mas um ÚNICO evento pode ser atendido por MÚLTIPLOS objetos.

O IDE cria tratadores de eventos para você automaticamente

A maioria dos programadores segue as mesmas convenções para nomear seus tratadores de evento. Se existir um objeto `Bola` com um evento `BolaEmJogo` e o nome da referência para o objeto for `bola`, o tratador de evento tipicamente será chamado de `bola_BolaEmJogo()`. Essa não é uma regra rígida, mas se você escrever código assim, será mais fácil para outros desenvolvedores entenderem.

Por sorte, o IDE facilita nomear seus tratadores de evento de forma apropriada. Ele tem uma característica de **adicionar métodos de tratamento automaticamente para você** quando está trabalhando com classes que disparam eventos. Não deveria ser surpresa que o IDE pode fazer isso por você - afinal de contas, isso é exatamente o que ele faz quando você clica duas vezes num botão de um formulário.

1 **Inicie um novo aplicativo Windows e adicione Ball (bola) e BallEventArgs**
Eis a classe Ball:

```
public class Ball {
  public event EventHandler BallInPlay;
  public void OnBallInPlay(BallEventArgs e) {
    if (BallInPlay != null)
      BallInPlay(this, e);
  }
}
```

E aqui está a classe BallEventArgs:

```
public class BallEventArgs : EventArgs {
  public int Trajectory { get; private set; }
  public int Distance { get; private set; }
  public BallEventArgs(int Trajectory, int Distance) {
    this.Trajectory = Trajectory;
    this.Distance = Distance;
  }
}
```

2 **Comece a criar o construtor de Pitcher***
Adicione uma nova classe Pitcher em seu projeto. Crie um construtor que use uma referência a `Ball` chamada `ball` como parâmetro. Deve existir uma linha de código no construtor para adicionar seu tratador de eventos em `ball.BallInPlay` (bola em jogo). Comece a digitar o comando, mas não digite o += ainda.

```
public Pitcher(Ball ball) {
  ball.BallInPlay
}
```

3 **Digite += e o IDE completará o comando para você**
Assim que você digitar += no comando, o IDE mostrará uma pequena caixa muito útil:

```
public Pitcher(Ball ball) {
  ball.BallInPlay +=
}
```

new EventHandler(ball_BallInPlay); (Pressione TAB para inserir)

E logo depois que você pressionar a tecla tab, o IDE completará o comando para você. Ele deve ficar parecido com o que segue:

```
public Pitcher(Ball ball) {
  ball.BallInPlay += new EventHandler(ball_BallInPlay);
}
```

* (NT) A explicação original deste capítulo em inglês empregava baseball, não futebol. Pitcher (lançador) foi usado pelo autor onde usamos "Goleiro".

Quando você clica duas vezes num botão no designer de formulário, o IDE faz exatamente a mesma coisa - adicionar um tratador de eventos automaticamente - exceto que ele adiciona o código ao método InitializeComponents() de Form/Designer.cs, em vez de apenas colocá-lo no final do arquivo de classe.

4 O IDE adicionará o tratador de evento também.
Ainda não acabamos - você ainda precisa adicionar um método para associar-se ao evento. Por sorte, o IDE também se encarrega disso pra você.

```
new EventHandler(ball_BallInPlay);
```

```
Pressione TAB para gerar o tratador 'ball_BallInPlay' nesta classe.
```

Pressione a tecla tab novamente para fazer o IDE adicionar esse novo tratador de evento à sua classe Pitcher. O IDE sempre seguirá a convenção nomeObjeto_NomeTratador():

```
void ball_BallInPlay(object sender, EventArgs e) {
  throw new NotImplementedException();
}
```

O IDE sempre preenche o método com NotImplementedException() (exceção não implementada) para o caso de você executar o código. Assim uma exceção é lançada para lembrá-lo de que ainda é preciso implementar algo criado automaticamente.

5 Finalize o tratador de evento da classe
Agora que você tem o esqueleto do tratador de evento adicionado à sua classe, complete-o com o restante do código. O lançador deve pegar qualquer bola baixa, senão cobrirá a primeira base.

Já que BallEventArgs é uma subclasse de EventArgs, vamos fazer subcoerção usando a palavra-chave `as` para podermos acessar suas propriedades.

```
void ball_BallInPlay(BallEventArgs e) {
  if (e is BallEventArgs) {
    BallEventArgs ballEventArgs = e as BallEventArgs;
    if ((ballEventArgs.Distance < 95) && (ballEventArgs.Trajectory < 60))
      CatchBall();
    else
      CoverFirstBase();
  }
}
```

Criaremos esses métodos num minuto.

Exercício

É hora de colocar o que você aprendeu até agora em prática. Sua tarefa é completar as classes Ball e Pitcher, acrescentar uma classe Fan (Torcedor) e certifique-se de todas trabalhem em conjunto numa versão bastante básica de um simulador de baseball.

1 Complete a classe Pitcher.
Abaixo está o que já sabemos a respeito de Pitcher. Adicione os métodos CatchBall() (pegar bola) e CoverFirstBase() (cobrir a primeira base). Ambos devem imprimir se o lançador pegou a bola ou correu para a primeira base.

```
public class Pitcher {
  public Pitcher(Ball ball) {
    ball.BallInPlay += new EventHandler(ball_BallInPlay);
  }
  void ball_BallInPlay(object sender, EventArgs e) {
    if (e is BallEventArgs){
      BallEventArgs ballEventArgs = e as BallEventArgs;
      if ((ballEventArgs.Distance < 95) && (ballEventArgs.Trajectory < 60))
        CatchBall();
      else
        CoverFirstBase();
    }
  }
}
```

Precisaremos implementar esses dois métodos para escrever uma linha no console.

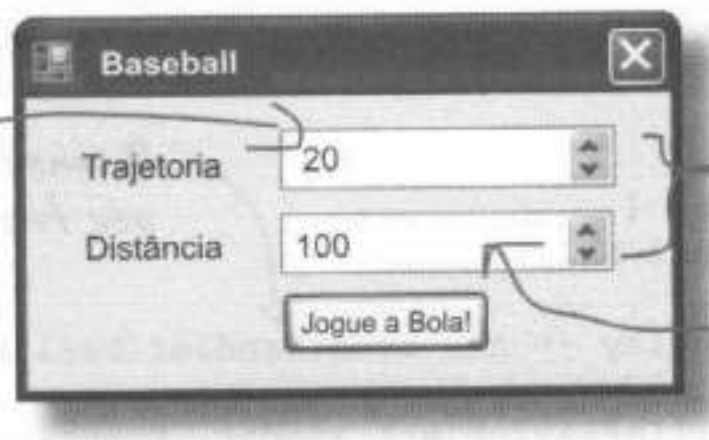
2 **Escreva uma classe Fan**
Crie outra classe Fan. Ela deve ser capaz de inscrever-se no evento BallInPlay no seu construtor. O tratador de evento dos torcedores deve ser se a distância é maior do que 400 pés (120m) e se a trajetória é maior do que 30 graus (o que permitiria a jogada no baseball chamada de "home run", quando o batedor corre sem poder ser impedido em torno do campo). Se for o caso, o torcedor deve pegar uma luva e tentar pegar a bola para levar como lembrança. Se não, ele deve gritar e torcer. Escreva o que o torcedor está fazendo no console.

Olhe para a imagem da janela
Output na proxima pagina para ver
exatamente o que deve ser impresso.

Objeto Fan

3 **Crie um simulador bem simples**
Crie um novo aplicativo. Ele deve ter dois controles NumericUpDown: um para a distância da bola, outro para sua trajetória. Adicione um botão, com o texto "Jogue a Bola!". Quando ele for clicado, a bola será atingida e os valores nos controles NumericUpDown passados. Seu formulário deve se parecer com este:

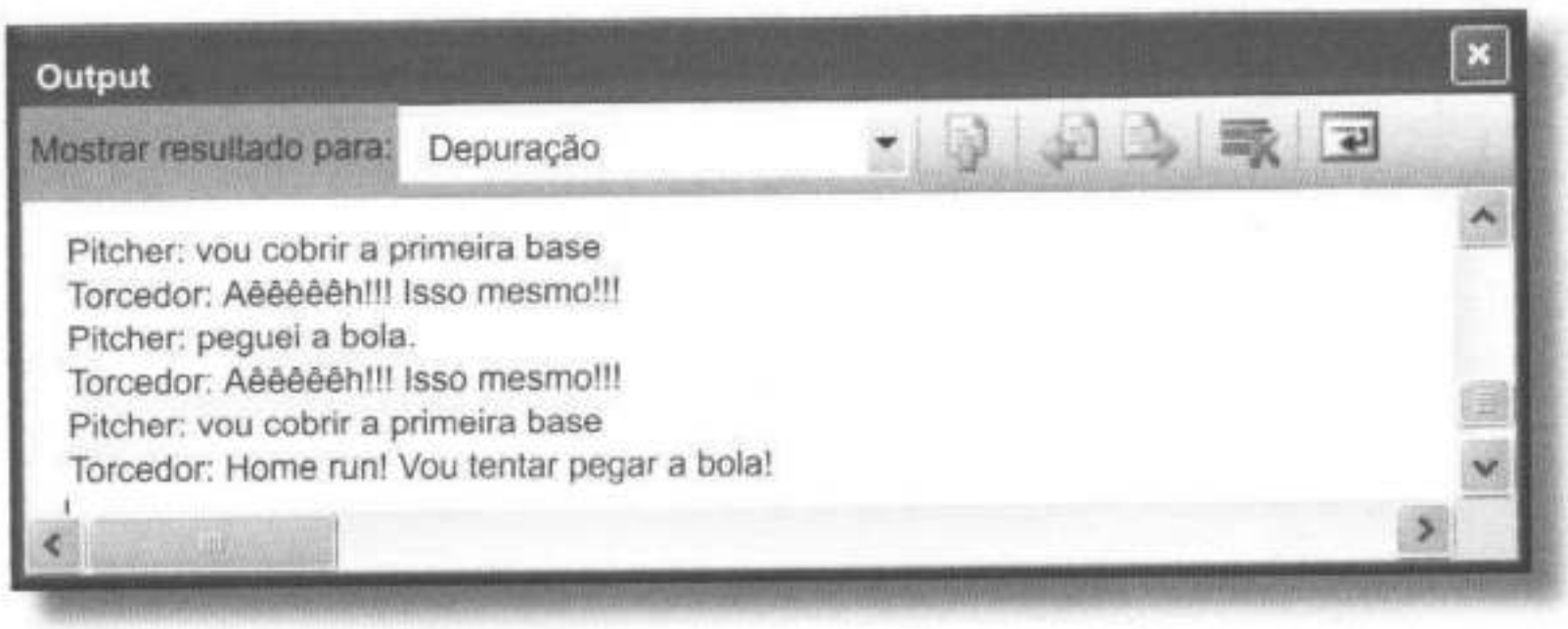
O valor para a trajetória pode ir de 0 a 100, então deixe a propriedade Minimum em 0, a Maximum em 100 e a Value em 20.



Não se esqueça de fazer coerção das propriedades Value para int antes de usá-las.

A distância pode variar de 0 a 500, com um valor padrão de 100.

4 **Reproduza os seguintes resultados**
Veja se você consegue fazer seu simulador gerar o resultado que segue com três bolas sucessivas colocadas em jogo. Escreva os valores usados para conseguir os resultados abaixo:



Bola 1:

Trajetória: _____

Distância: _____

Bola 2:

Trajetória: _____

Distância: _____

Bola 3:

Trajetória: _____

Distância: _____

Solução do Exercício

É hora de colocar o que você aprendeu até agora em prática. Sua tarefa é completar as classes Ball e Pitcher, acrescentar uma classe Fan (Torcedor) e certificar-se de que todas trabalhem em conjunto numa versão bastante básica de um simulador de baseball.

```
public class Ball
{
  public event EventHandler BallInPlay;
  public void OnBallInPlay(BallEventArgs e)
  {
    if (BallInPlay != null)
        BallInPlay(this, e);
  }
}
public class BallEventArgs {
{
  public int Trajectory { get; private set; }
  public int Distance { get; private set; }
  public BallEventArgs(int Trajectory, int Distance)
  {
    this.Trajectory = Trajectory;
    this.Distance = Distance;
  }
}
public class Fan {
  public Fan(Ball ball)
  {
    ball.BallInPlay += new EventHandler(ball_BallInPlay);
  }
  void ball_BallInPlay(object sender, EventArgs e)
  {
    if (e is BallEventArgs) {
      BallEventArgs ballEventArgs = e as BallEventArgs;
      if (ballEventArgs.Distance > 400 && ballEventArgs.Trajectory > 30)
        Console.WriteLine("Fan: Home run! I'm going for the ball!");
      else
        Console.WriteLine("Fan: Woo-hoo! Yeah!");
    }
  }
}
```

O método OnBallInPlay() apenas dispara o evento BallInPlay - mas ele deve checar para ter certeza que não é nulo, para evitar o risco de lançar uma exceção.

Propriedades apenas leitura automáticas funcionam muito bem como argumentos de eventos porque os tratadores lêem apenas os dados passados para eles.

O construtor do objeto Fan encadeia seu tratador no evento BolaEmJogo.

O tratador de evento BolaEmJogo do torcedor procura por qualquer bola que vá bem alto e longe.

```
public class Pitcher {
  public Pitcher(Ball ball) {
    ball.BallInPlay += new EventHandler(ball_BallInPlay);
  }
  void ball_BallInPlay(object sender, EventArgs e) {
    if (e is BallEventArgs) {
        BallEventArgs ballEventArgs = e as BallEventArgs;
        if ((ballEventArgs.Distance < 95) && (ballEventArgs.Trajectory < 60))
          CatchBall();
        else
          CoverFirstBase();
    }
  }
  private void CatchBall() {
    Console.WriteLine("Pitcher: I caught the ball");
  }
  private void CoverFirstBase() {
    Console.WriteLine("Pitcher: I covered first base");
  }
}
public partial class Form1 : Form {
  Ball ball = new Ball();
  Pitcher pitcher;
  Fan fan;
  public Form1() {
    InitializeComponent();
    pitcher = new Pitcher(ball);
    fan = new Fan(ball);
  }
  private void playBall_Click(object sender, EventArgs e) {
    BallEventArgs ballEventArgs = new BallEventArgs(
        (int)trajectory.Value, (int)distance.Value);
    ball.OnBallInPlay(ballEventArgs);
  }
}
```

Você já tem o tratador de evento BolaEmJogo do Pitcher. Ele procura por qualquer bola baixa.

O formulário precisa de uma bola, um torcedor e um lançador. Ele associa o torcedor e o lançador à bola no seu construtor.

Quando o botão é clicado, o formulário diz ao lançador para lançar a bola para o batedor, que diz à bola para disparar seu evento BolaEmJogo, que chama os tratadores de evento dos objetos lançador e torcedor.

Aqui estão os valores usados para conseguir o resultado. Seus números podem ser um pouco diferentes.

Bola 1

Trajetória: 75

Distância: 105

Bola 2

Trajetória: 48

Distância: 80

Bola 3

Trajetória: 40

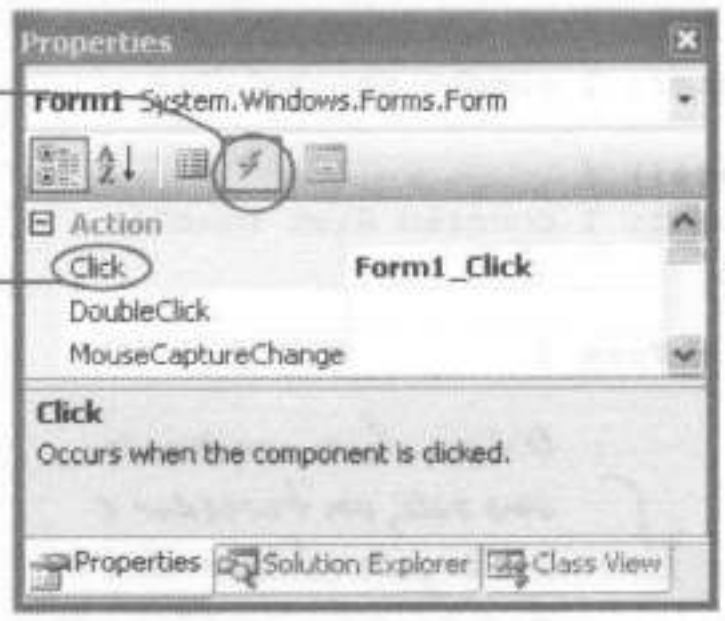
Distância: 435

Os formulários que você esteve fazendo usaram eventos

Toda vez que você criou um botão, clicou duas vezes nele no designer e escreveu código num método como `button1_Click()`, você esteve trabalhando com eventos.

1. Crie um novo projeto de aplicativo Windows. Entre no formulário e vá para a janela de propriedades. Lá, você verá ícones no topo da janela - clique num que tem um pequeno raio. Isso fará surgir a **página de eventos da janela de propriedades**:

Você pode ver todos os eventos pré-existentes para um controle clicando nele e então, no botão de eventos, na janela de propriedades.



Você pode criar um evento que será disparado quando alguém clicar no formulário selecionando Form/_Click ao lado de Click, na janela de eventos.

Role a página para baixo e clique duas vezes na palavra "Click". Quando você fizer isso, o IDE adicionará um novo tratador de evento ao seu formulário que será disparado quando você clicar no elemento alvo. Uma linha também será adicionada em Form/Designer.cs para associar o tratador ao evento.

2. Clique duas vezes na linha "Click" na página de eventos. O IDE automaticamente adicionará um método tratador ao seu formulário chamado `Form1_Click`. Acrescente a seguinte linha de código a ele:

```
private void Form1_Click(object sender, EventArgs e) {
  MessageBox.Show("You just on clicked the form");
}
```

3. O Visual Studio faz mais do que simplesmente acrescentar uma pequena linha de declaração de método para você. Abra Form1.Designer.cs e use o Quick Find (busca rápida) do IDE para procurar pelo texto `"Form1_Click"` no projeto atual. Você encontrará essa linha:

```
this.Click += new System.EventHandler(this.Form1_Click);
```

Agora execute o programa para ter certeza de que o código funciona!

Tratadores de eventos sempre precisam ser "amarrados"

*Se você arrastar um botão para seu formulário e adicionar um método `button1_Click()`, com os parâmetros corretos, mas que não esteja **registrado para prestar atenção ao botão**, o método nunca será chamado. Clique duas vezes no botão no designer - o IDE verá que o nome padrão de tratador já foi usado, então adicionará outro com o nome `button1_Click_1()`.*

Um evento, múltiplos tratadores

Eis algo bem útil que você pode fazer com eventos: **encadeá-los** para que um único evento ou delegado possa chamar muitos métodos, um depois do outro. Vamos adicionar alguns botões no aplicativo e ver como isso funciona.

4. Adicione esses dois métodos ao seu formulário:

```
private void SaySomething(object sender, EventArgs e) {
  MessageBox.Show("Something");
}
private void SaySomethingElse(object sender, EventArgs e) {
  MessageBox.Show("Something else");
}
```

5. Agora adicione dois botões a seu formulário. Clique duas vezes em cada um deles para adicionar seu tratador de eventos. Eis o código para os dois métodos:

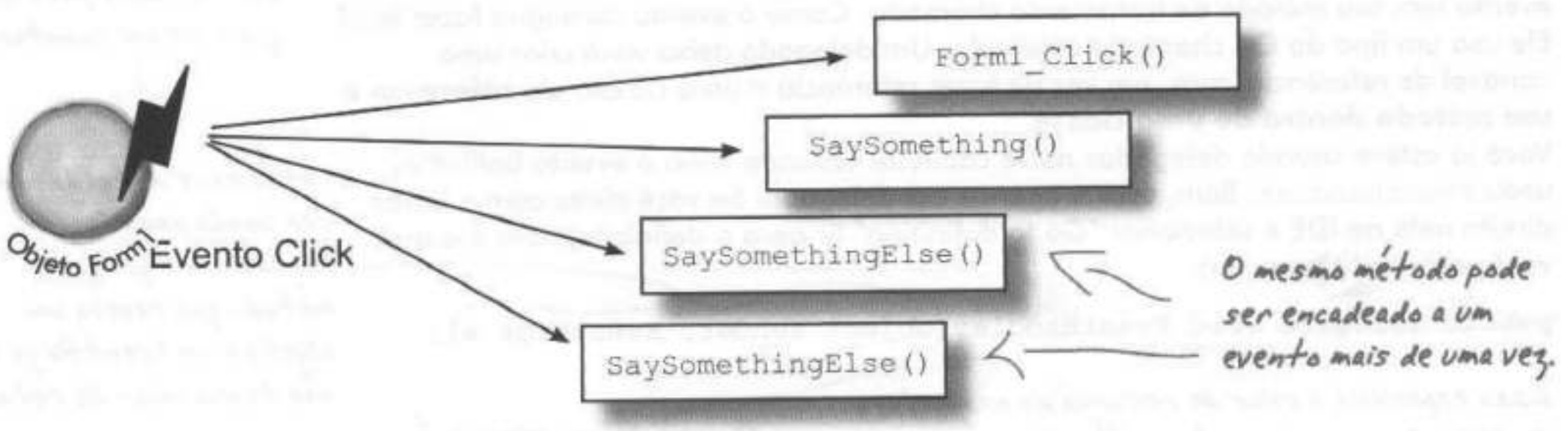
```
private void button1_Click(object sender, EventArgs e) {
  this.Click +=new EventHandler(SaySomething);
}
private void button2_Click(object sender, EventArgs e) {
  this.Click +=new EventHandler(SaySomethingElse);
}
```

6. Agora execute seu programa e faça o seguinte:

- ★ **Clique no formulário** - você verá uma caixa de mensagem dizendo "You just clicked on the form" (você acabou de clicar no formulário).
- ★ Agora **clique em button1** e depois no **formulário de novo**. Você verá duas caixas mostrando "você acabou de clicar no formulário" e então "something" (alguma coisa).
- ★ **Clique em button2 duas vezes** e no **formulário de novo**. Você verá quatro caixas: "você acabou de clicar no formulário", "alguma coisa", "something else" (mais alguma coisa) e "something else" novamente.

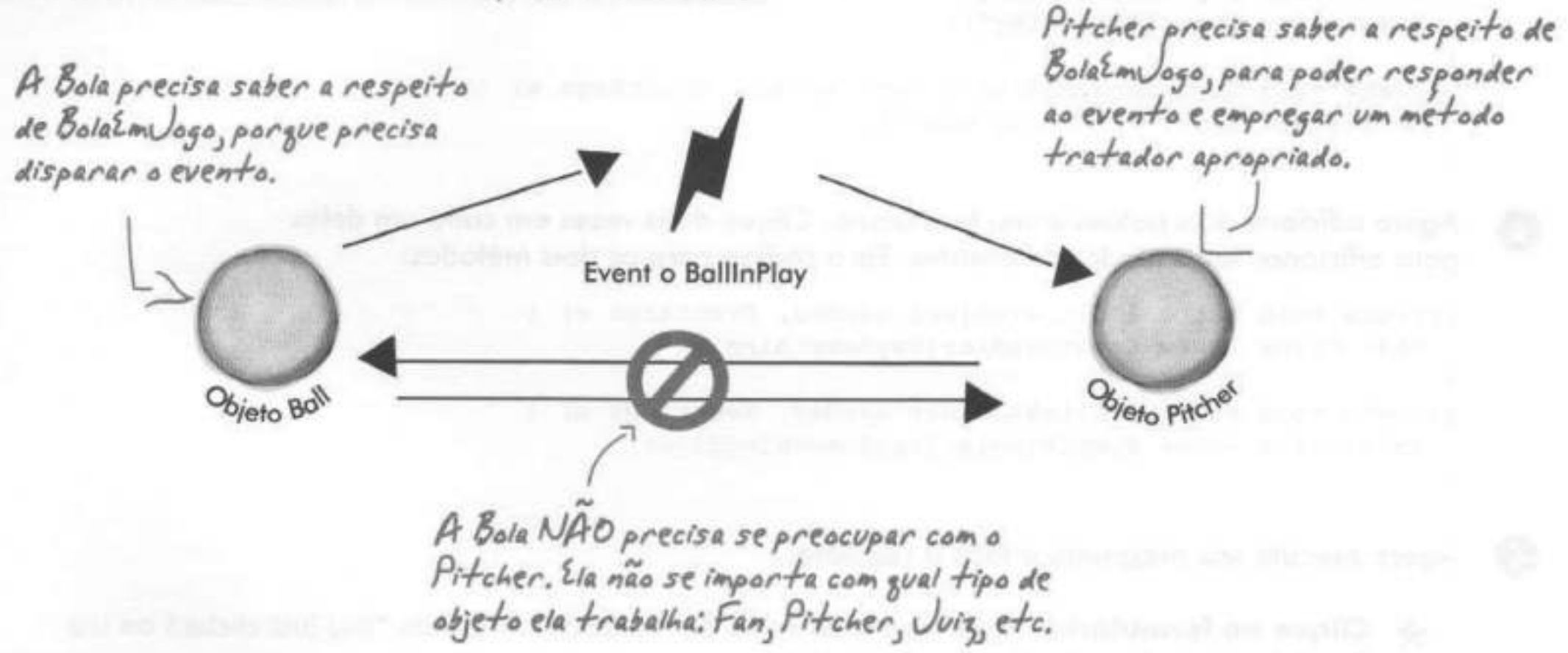
O que aconteceu?

Toda vez que você clicou um dos botões, encadeou outro método - seja Something() ou SomethingElse() - no evento Click do formulário. Você pode continuar clicando neles e continuará **encadeando os mesmos métodos** ao evento. Não importa com quantos métodos estejam encadeados, nem se os mesmos métodos aparecem mais de uma vez na seqüência; ele chama todos cada vez que for disparado, um depois do outro e na ordem em que eles foram adicionados.



Conectando remetentes com destinatários de eventos

Uma das coisas complicadas a respeito de eventos é o **remetente** dever saber o tipo de eventos a enviar - incluindo os argumentos a passar. E o **destinatário** deve saber a respeito do tipo de retorno e dos argumentos de que o método de tratamento precisará.
Mas - e aí é que está a parte complicada - você não pode `unir` o remetente com o destinatário. Você quer que o remetente envie o evento e `não se preocupe com quem o recebe`. E o remetente deve se preocupar com o evento, `não com o objeto que o disparou`. Assim tanto o remetente quanto o destinatário concentram-se no evento, não um no outro.



"Meu pessoal entrará em contato com o seu"

Você sabe o que o código seguinte faz:

```
Ball currentBall;
```

Ele cria uma **variável de referência** que aponta para um objeto Bola. Ela não está associada a uma bola específica. Em vez disto, ela pode apontar para qualquer objeto Bola - ou pode ser nula e não apontar para nada.

Um evento precisa de um tipo de referência similar - exceto que, em vez de apontar para um objeto, ele precisa de uma que **aponte para um método**. Todos os eventos precisam manter registro a respeito de uma lista de métodos inscritos nele. Você já viu que eles podem estar em outras classes e até ser privados. Então como ele controla a lista de todos os métodos de tratamento que precisa chamar?

Um delegado COLOCA-SE NO LUGAR de um método real

Um dos aspectos mais úteis dos eventos é que quando eles são disparados, não sabem quem são os tratadores que estão chamando. Qualquer um inscrito num evento tem seu método de tratamento chamado. Como o evento consegue fazer isso? Ele usa um tipo do C# chamado delegado. Um delegado deixa você criar uma variável de referência, mas, em vez de fazer referência a uma classe, ela **refere-se a um método dentro de uma classe**.
Você já esteve usando delegados neste capítulo! Quando criou o evento BallInPlay, usou `EventHandler`. Bem, esse é apenas um delegado. Se você clicar com o botão direito nele no IDE e selecionar "Go to definition" (ir para a definição), isso é o que você verá (teste para ver):

Quando você cria um delegado, tudo a fazer é especificar a assinatura dos métodos para os quais ele vai apontar.

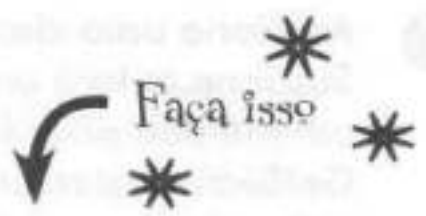
```
public delegate void EventHandler(object sender, EventArgs e);
```

Então esse delegado pode ser usado para fazer referência a qualquer método que receba um objeto e um EventArgs e não tenha valor de retorno.

Isso especifica o valor de retorno da assinatura do delegado - ou seja, um EventHandler só pode apontar para métodos com valores de retorno void.

O nome desse delegado é EventHandler.

Um delegado adiciona um novo tipo ao seu projeto



Quando você adiciona um delegado a seu projeto, está criando um **tipo delegado**. E quando você o usa para definir um campo ou variável, está criando uma **instância** do tipo delegado. **Então crie um novo projeto**. Depois adicione um novo arquivo de classe ao projeto chamado ReturnsAString.cs. Mas, em vez de colocar uma classe dentro dele, adicione uma única linha:

```
public delegate string ReturnsAString();
```

ReturnsAString é um tipo delegado que você adicionou a seu projeto. Agora você pode usá-lo para declarar variáveis e campos.

Vá para o código do formulário e adicione o campo:

```
ReturnsAString someMethod;
```

someMethod é uma instância do tipo delegado ReturnsAString.

Agora compile seu programa - funciona! (Deve aparecer um aviso porque você não usa o campo para nada - tudo bem.) Assim que você adicionou seu novo delegado ao programa, um novo tipo chamado `ReturnsAString` foi criado. Se você usá-lo para declarar uma variável, pode atribuir-lhe qualquer método que não receba parâmetros e retorne uma string. Tente - adicione o seguinte método ao seu código:

```
private string HiThere() {
  return "Hi there!";
}
```

A assinatura desse método é equivalente à de ReturnsAString.

Adicione um botão e inclua essas três linhas. Clique e veja o que acontece:

```
someMethod = new ReturnsAString(HiThere);
string message = someMethod();
MessageBox.Show(message);
```

Você pode atribuir someMethod como faria para qualquer outra variável. Mas quando você a chama como se fosse um método, ela chama qualquer método para o qual eventualmente esteja apontado.

delegado, substantivo.
Pessoa autorizada ou enviada para representar outras. *O presidente enviou um **delegado** ao encontro.*

Delegados em ação

Não há nada de misterioso sobre delegados - de fato, não se emprega muito código para usá-los. Vamos utilizá-los para ajudar um dono de restaurante a ordenar os ingredientes secretos de seus famosos chefes de cozinha.

1. **Crie um novo projeto e adicione um delegado**
   Delegados geralmente aparecem fora de qualquer outra classe. Então adicione um novo arquivo de classe ao seu projeto e chame-o de AddSecretIngredient.cs (adicionar ingrediente secreto). Ele terá uma única linha de código:

   ```
   public delegate string GetSecretIngredient(int amount);
   ```

   Este delegado pode ser usado para criar uma variável que possa apontar para qualquer método com um parâmetro inteiro e retornar uma string.

2 **Adicione uma classe para a primeira Chef, Suzanne**
Suzanne.cs terá uma classe que manterá registro do ingrediente secreto da primeira chef. Ela terá um método privado chamado SuzannesSecretIngredient() com uma assinatura equivalente a de GetSecretIngredients. Mas ela também terá uma propriedade apenas-leitura - e adivinhe qual será o tipo dessa propriedade. Ela retorna um GetSecretIngredient. Assim outros objetos podem usar essa propriedade para conseguir uma referência ao método citado.

```
public class Suzanne {
  public GetSecretIngredient MySecretIngredientMethod {
    get {
      return new GetSecretIngredient(SuzannesSecretIngredient);
    }
  }
  private string SuzannesSecretIngredient(int amount) {
    return amount.ToString() + " ounces of cloves";
  }
}
```

O método do ingrediente secreto de Suzanne recebe um parâmetro inteiro chamado amount (quantidade) e retorna uma string que descreve o ingrediente.

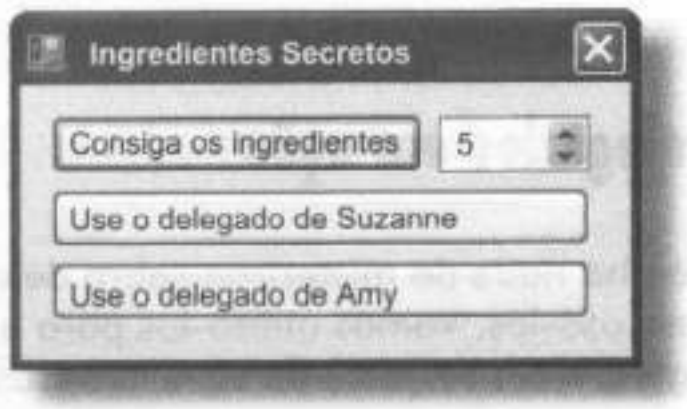
A propriedade GetSecretIngredients de Amy retorna uma nova instância do delegado GetSecretIngredients que aponta para o seu método de ingrediente secreto.

3 **Então adicione uma classe para a segunda Chef, Amy**
O método de Amy funciona de forma muito semelhante ao de Suzanne:

```
public class Amy {
  public GetSecretIngredient AmysSecretIngredientMethod {
    get {
      return new GetSecretIngredient(AmysSecretIngredient);
    }
  }
  private string AmysSecretIngredient(int amount) {
    if (amount < 10)
      return amount.ToString()
          + " cans of sardines -- you need more!";
    else
      return amount.ToString() + " cans of sardines";
  }
}
```

O método do ingrediente secreto de Amy também recebe uma quantidade e retorna uma string, mas algo diferente do de Suzanne.

4 **Crie um novo projeto e adicione um delegado**
Crie esse formulário →



Eis o código para ele:

```
GetSecretIngredient ingredientMethod = null;
Suzanne suzanne = new Suzanne();
Amy amy = new Amy();
private void useIngredient_Click(object sender, EventArgs e) {
  if (ingredientMethod != null)
    Console.WriteLine("I'll add " + ingredientMethod((int)amount.Value));
  else
    Console.WriteLine("I don't have a secret ingredient!");
}
private void getSuzanne_Click(object sender, EventArgs e) {
  ingredientMethod = new GetSecretIngredient(suzanne.MySecretIngredientMethod);
}
private void getAmy_Click(object sender, EventArgs e) {
  ingredientMethod = new GetSecretIngredient(amy.AmysSecretIngredientMethod);
}
```

5 Use o depurador para ver como os delegados funcionam

Você tem uma ótima ferramenta - o depurador do IDE - que realmente ajuda a perceber como os delegados funcionam. Faça o seguinte:

- ★ Comece executando seu programa. Primeiro clique no botão "Consiga os ingredientes" - ele deve escrever uma linha no console que diz "I don't have a secret ingredient" (não tenho um ingrediente secreto).
- ★ Clique no botão "Use o delegado de Suzanne" - isso atribui o campo ingredientMethod ao formulário (que é um delegado do tipo GetSecretIngredient) independente do que a propriedade GetSecretIngredient de Suzanne retornar. Essa propriedade retorna uma nova instância do tipo GetSecretIngredient que está apontando para o método SuzanneSecretIngredient().
- ★ Clique novamente no botão "Consiga o ingrediente". Agora que o campo ingredientMethod do formulário está apontando para SuzanneSecretIngredient(), ele chama esse método, passando o valor no controle NumericUpDown e escrevendo o resultado no console.
- ★ Clique no botão "Use o delegado de Amy". Ele usa a propriedade Amy.GetSecretIngredient para atribuir valor ao campo ingredientMethod do formulário de forma que ele aponte para o método AmySecretIngredient().
- ★ Clique no botão "Consiga os ingredientes" mais uma vez. Agora ele chamará o método de Amy.
- ★ Agora **use o depurador** para ver exatamente o que está acontecendo. Coloque um ponto de parada na primeira linha de cada um dos três métodos no formulário. **Reinicie o programa** (o que reiniciará o campo ingredientMethod de forma que ele seja novamente igual a null), e comece de novo com os cinco passos acima. Use o Step Into (F11) para avançar por cada linha de código. Observe o que acontece quando você clica o botão "consiga os ingredientes". O ponto de execução pula direto para dentro das classes Suzanne e Amy, dependendo para qual dos métodos o campo ingredientMethod esteja apontando.

Jogos de Verão

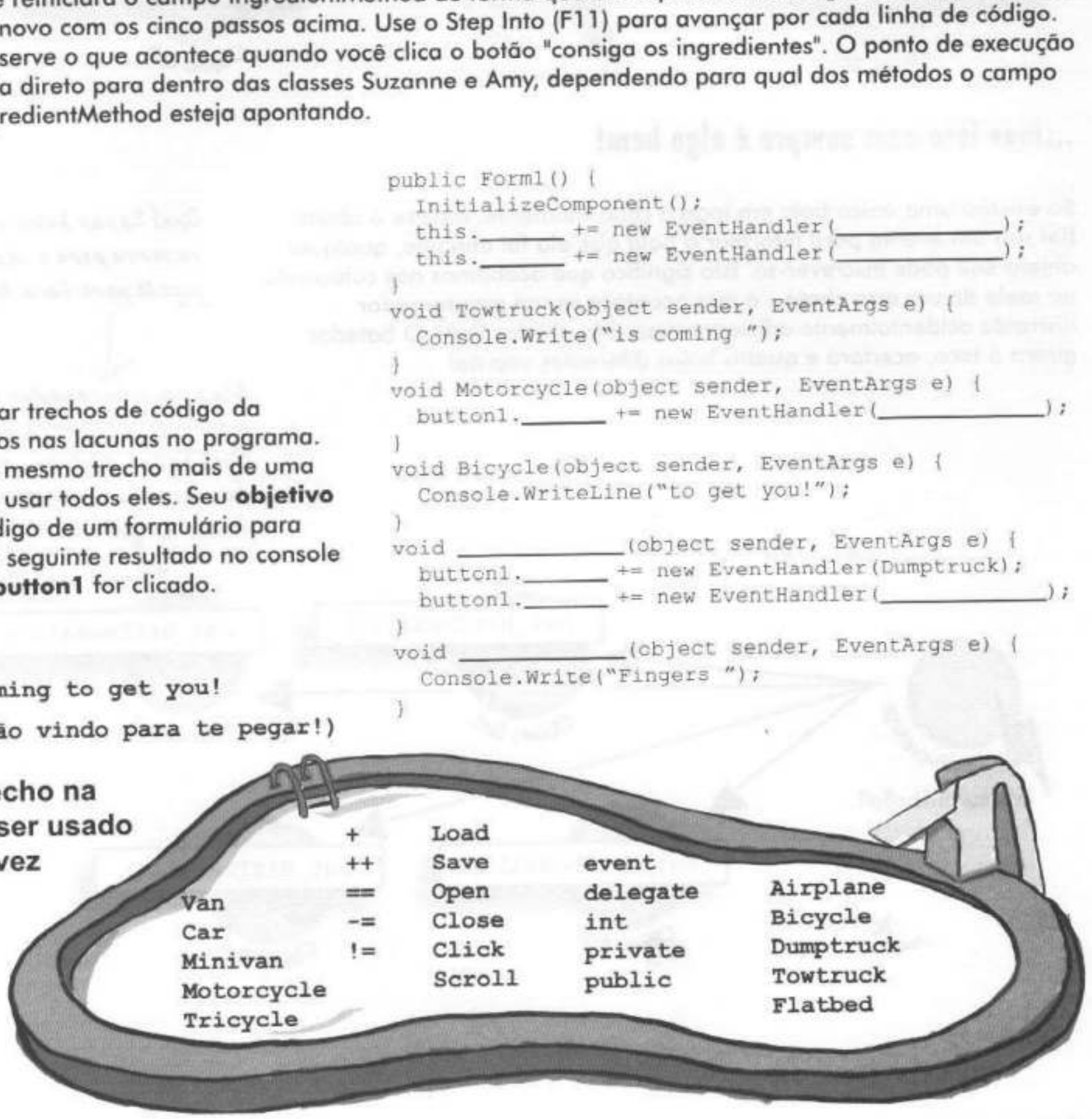
Sua **tarefa** é pegar trechos de código da piscina e colocá-los nas lacunas no programa. Você pode usar o mesmo trecho mais de uma vez e não precisa usar todos eles. Seu **objetivo** é completar o código de um formulário para que ele escreva o seguinte resultado no console quando o botão **button1** for clicado.

Resultado

```
Fingers is coming to get you!
(Dedinhos estão vindo para te pegar!)
```

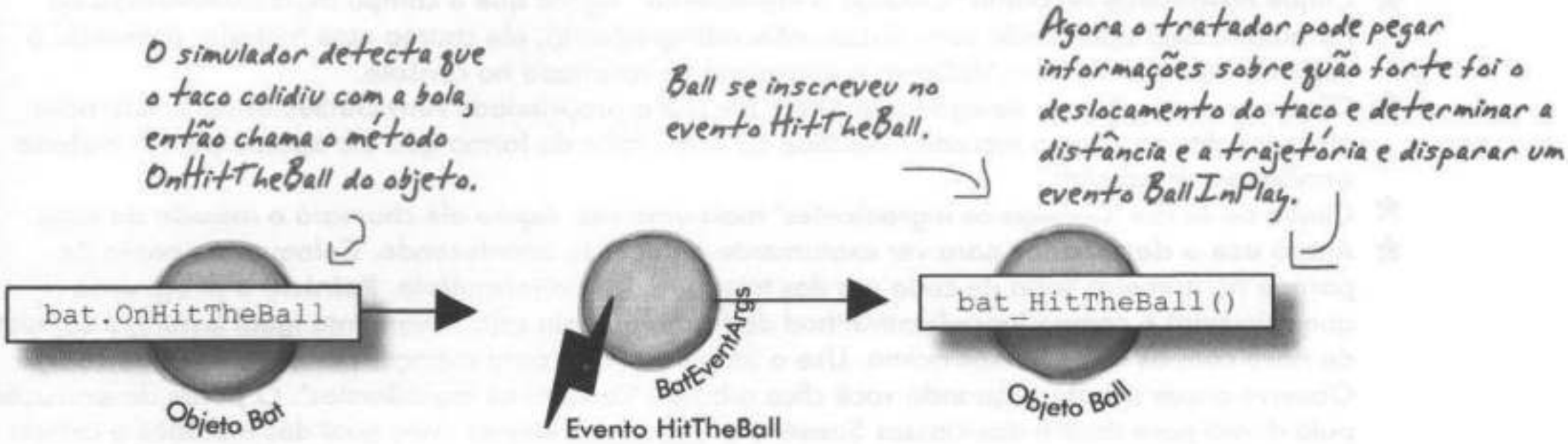
Nota: cada trecho na piscina pode ser usado mais de uma vez

```
public Form1() {
  InitializeComponent();
  this._____ += new EventHandler(_____);
  this._____ += new EventHandler(_____);
}
void Towtruck(object sender, EventArgs e) {
  Console.Write("is coming ");
}
void Motorcycle(object sender, EventArgs e) {
  button1._____ += new EventHandler(_____);
}
void Bicycle(object sender, EventArgs e) {
  Console.WriteLine("to get you!");
}
void _____(object sender, EventArgs e) {
  button1._____ += new EventHandler(Dumptruck);
  button1._____ += new EventHandler(_____);
}
void _____(object sender, EventArgs e) {
  Console.Write("Fingers ");
}
```



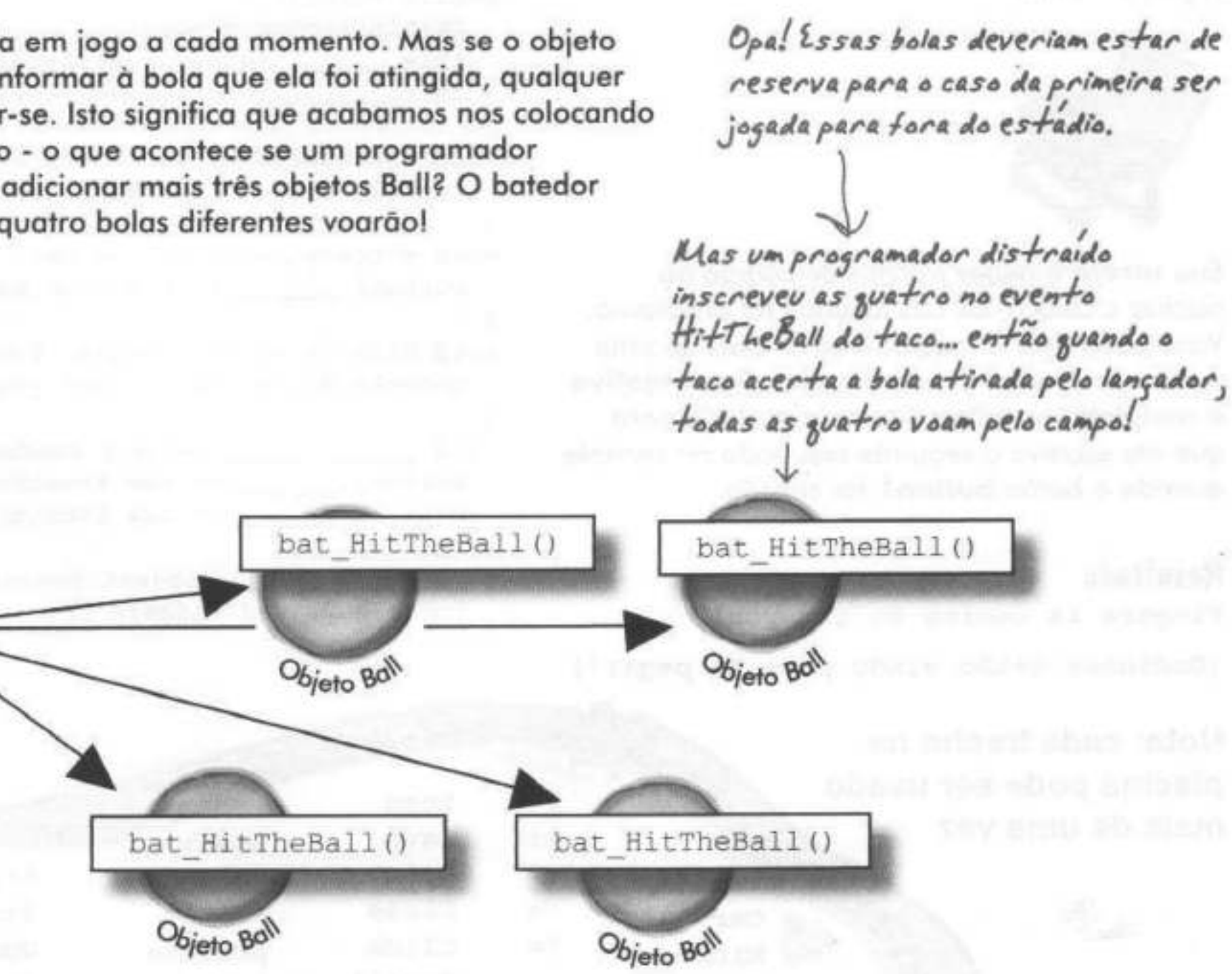
Qualquer objeto pode inscrever-se num evento público

Suponha que adicionemos uma nova classe ao nosso simulador, uma classe `Bat` (taco, em inglês e homógrafo da palavra "morcego") e ela adicione um evento `HitTheBall` (atingir a bola). Eis como funcionaria: se o simulador detecta que o jogador atingiu a bola, ele chama o método `OnHitTheBall` de `Bat`, que dispara o evento `HitTheBall`.
Então agora podemos adicionar um método `bat_HitTheBall` na classe `Ball` que se inscreve no evento `HitTheBall`. Assim, quando a bola for atingida, seu próprio tratador chama seu método `OnBallInPlay()` para disparar seu próprio evento, `BallInPlay`, e a reação em cadeia começa. Jogadores jogam, torcedores torcem, juízes gritam... temos um jogo de bola.



...mas isso nem sempre é algo bom!

Só existirá uma única bola em jogo a cada momento. Mas se o objeto Bat usa um evento para informar à bola que ela foi atingida, qualquer objeto Ball pode inscrever-se. Isto significa que acabamos nos colocando no meio de um erro chato - o que acontece se um programador distraído acidentalmente adicionar mais três objetos Ball? O batedor girará o taco, acertará e quatro bolas diferentes voarão!



O caso do Crustáceo Dourado

Enrique "pé chato" Hodgkins é um CacadorDeTesouro. Ele está na trilha de uma das mais valiosas posses no raro e exótico mercado de jóias de tema marinho: um caranguejo dourado incrustado com jade. Mas muitos outros objetos CacadorDeTesouro também estão. Todos eles têm uma referência para o mesmo caranguejo nos seus construtores, mas Enrique quer pegar o prêmio antes.

Mistério de Cinco Minutos

Num diagrama de classe roubado, Enrique descobriu que a classe CaranguejoDourado dispara o evento CorrerParaSeEsconder toda vez que alguém tenta chegar perto dele. Ainda melhor, o evento inclui ArgsNovaPosicao, com detalhes a respeito da rota do caranguejo. Como nenhum dos outros caçadores de tesouro sabe sobre esses eventos, Enrique imagina que pode ficar rico.

Ele adiciona código no seu construtor para registrar seu método AtencaoParaPistas() como um novo tratador para o evento CorrerParaSeEsconder, na referência do caranguejo que ele possui. Então ele envia um capanga de má reputação atrás do caranguejo, sabendo que isso vai fazê-lo fugir, esconder-se e disparar o evento CorrerParaSeEsconder - o que dá ao método AtencaoParaPistas() de Enrique toda a informação necessária.

Tudo corre como planejado, até que Enrique chega à nova posição e corre para agarrar o caranguejo. Ele fica espantado ao ver que três outros objetos CacadorDeTesouro estão lá, brigando pelo caranguejo.

Como os outros caçadores de tesouro chegaram na frente de Enrique?

Resposta na página 435.

Solução dos Jogos de Verão

O construtor encadeia dois tratadores no evento de carga (load). Eles são chamados assim que o formulário é carregado.

```
public Form1() {
  InitializeComponent();
  this.Load += new EventHandler(Minivan);
  this.Load += new EventHandler(Motorcycle);
}
void Towtruck(object sender, EventArgs e) {
  Console.Write("is coming ");
}
void Motorcycle(object sender, EventArgs e) {
  button1.Click += new EventHandler(Bicycle);
}
void Bicycle(object sender, EventArgs e) {
  Console.WriteLine("to get you!");
}
void Minivan(object sender, EventArgs e) {
  button1.Click += new EventHandler(Dumptruck);
  button1.Click += new EventHandler(Towtruck);
}
void Dumptruck(object sender, EventArgs e) {
  Console.Write("Fingers ");
}
```

Os dois tratadores do evento load associam mais três tratadores ao evento Click do botão.

Quando o botão é clicado, ele chama três tratadores de evento que estão encadeados.

Use um callback, em vez de um evento, para associar exatamente um único objeto a um delegado

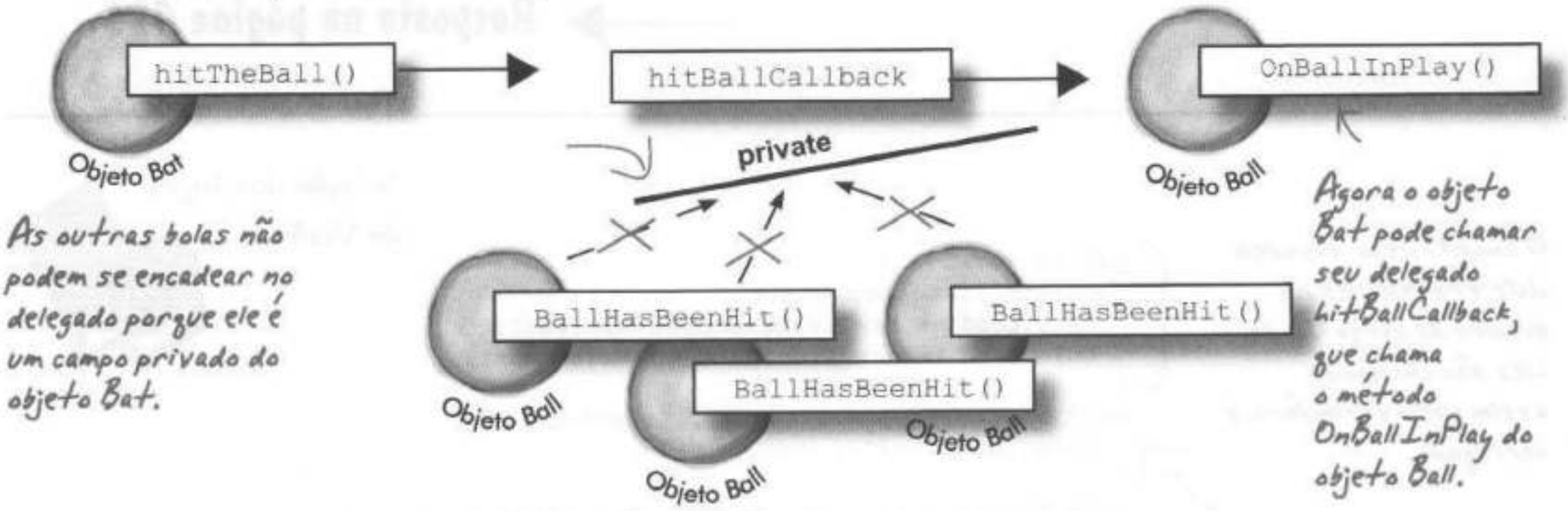
Nosso sistema de eventos só funcionará se tivemos apenas um único `bat` e uma única `ball`. Mas se tivermos vários objetos `Ball` e todos se inscreverem no evento público `HitTheBall`, todas elas sairão voando quando o evento for disparado. Mas isso não faz nenhum sentido... apenas uma única bola foi realmente atingida. Precisamos de uma maneira de fazer com que a única `bola` que está sendo lançada se associe ao taco, mas de uma forma que não permita que outras bolas façam o mesmo. E é isso que é um `callback` (chamada de retorno, literalmente chamar para trás ou chamar de volta) - uma maneira de usar um delegado de forma que o objeto que chama com certeza seja o único cujo método necessário e nenhum outro método possa encadear-se no delegado.

Eis como um callback funciona:

1. **O taco manterá seu campo delegado privado**
   A forma mais fácil de impedir os objetos Ball errados de encadearem-se no delegado de Bat é mantê-los privados. Dessa forma, ele tem controle sobre qual objeto Ball será chamado.

2. **O construtor de Ball recebe um delegado que aponta para um método na bola**
   Quando a bola está em jogo, ela cria uma nova instância do taco e passa-lhe um ponteiro para o método OnBallInPlay(). Isso é chamado de método callback porque o taco vai utilizá-lo para chamar de volta o objeto que o instanciou.

3. **Quando o taco acerta a bola, ele chama o método callback**
   Mas uma vez que o taco mantém seu delegado privado, ele pode ter certeza de que outras bolas não foram atingidas. Isso resolve o problema!



Callbacks usam delegados, mas NÃO eventos

Um callback é **uma forma diferente de usar um delegado**. Não é uma nova palavra-chave ou operador. É apenas uma forma de descrever um **padrão** - uma maneira de usar delegados em suas classes de forma que um objeto possa dizer a outro "notifique-me quando isso acontecer - e não conte a mais ninguém!".

Faça isso

1. **Adicione um novo delegado no seu projeto de Baseball**
   Uma vez que o taco terá um campo delegado privado que aponte para o método OnBallInPlay() do objeto Ball, precisaremos de um delegado que indique uma assinatura equivalente a:

```
public delegate void BatCallback(BallEventArgs e);
```

O callback do objeto Bat apontará para o método OnBallInPlay() de Ball, assim o seu delegado precisa indicar uma assinatura equivalente à deste método - logo ele precisa receber um parâmetro BallEventArgs e ter um valor de retorno void.

2 **Adicione a classe Bat ao projeto**
A classe Bat é simples. Ela tem o método HitTheBall() que o simulador chamará quando uma bola for atingida. Esse método usa o delegado hitBallCallback() para chamar o método OnBallInPlay() da bola (ou o que quer que seja que tenha sido passado no construtor).

Certifique-se de checar cada delegado para ter certeza de que ele não é nulo, senão uma exceção de referência nula pode ser lançada.

```
public class Bat {
  private BatCallback hitBallCallback;
  public Bat(BatCallback callbackDelegate) {
    this.hitBallCallback = new BatCallback(callbackDelegate);
  }
  public void HitTheBall(BallEventArgs e) {
    if (hitBallCallback != null)
      hitBallCallback(e);
  }
}
```

Usamos = no lugar de += porque não existe necessidade de encadeamento. Esse delegado só aponta para um único método. Mas se você achar melhor usar o +=, funcionará da mesma forma.

3 **Precisamos associar o taco à bola**
Como o construtor de Bat consegue uma referência de um método OnBallInPlay() de uma bola em particular? Fácil - basta chamar o método GetNewBat() do objeto Ball, que você deverá adicionar:

Atribuímos o callback ao construtor do objeto Bat. Mas, em alguns casos, fará mais sentido fazer isso usando um elemento público, um método ou acessador set de uma propriedade.

```
public Bat GetNewBat()
{
  return new Bat(new BatCallback(OnBallInPlay));
}
```

O método GetNewBat() cria um novo objeto Bat e usa o delegado BatCallBack para passar uma referência de seu próprio método OnBallInPlay() para o novo taco. Esse será o método de callback que o taco usará quando atingir a bola.

4 **Agora podemos encapsular a classe Ball um pouco melhor**
Não é comum que os métodos começados em `On...` seja `públicos`. Você pode ver por você mesmo - entre no código do formulário e tente chamar o método OnClick. Você não poderá fazer isso, porque ele é `protected` (ou seja, uma subclasse pode sobrepor-se ao método). Então vamos seguir esse padrão com nossa bola também, deixando o método OnBallInPlay() protegido:

```
protected void OnBallInPlay(BallEventArgs e)
{
  if (BallInPlay != null)
    BallInPlay(this, e);
}
```

Esse é um **padrão realmente comum** que você verá muitas vezes enquanto estiver trabalhando com as classes do .NET. Quando uma delas tem um evento que é disparado, quase você encontrará um método protegido equivalente começado com "On".

5 **Tudo a fazer agora é ativar o formulário**
O formulário não pode mais chamar o método OnBallInPlay() – exatamente o nosso desejo. É por isso que criamos o novo método GetNewBat(). Agora o formulário precisa pedir à bola um novo taco para poder atingi-la. E quando ele faz isso, o objeto Ball se certificará de que seu método OnBallInPlay() esteja associado ao callback do taco.

```
private void playBall_Click(object sender, EventArgs e)
{
  Bat bat = ball.GetNewBat();
  BallEventArgs ballEventArgs = new BallEventArgs(
    (int)trajectory.Value, (int)distance.Value);
  bat.HitTheBall(ballEventArgs);
}
```

Se o formulário (ou o simulador) quiser atingir um objeto Ball, ele precisa obter um novo objeto Bat dela. A bola se certificará de que o callback esteja associado ao taco. Agora quando o formulário chamar o método HitTheBall() do taco, ele chama o método OnBallInPlay() da bola, que por sua vez, dispara o evento BallInPlay.

Agora **execute o programa** - ele deve funcionar exatamente como antes. Mas agora ele está **protegido** contra qualquer problema que poderia ser causado por mais de uma bola prestando atenção ao mesmo evento.

Mas não precisa aceitar nossa palavra - use o depurador!

Pontos de Bala

- Quando você adiciona um delegado a seu projeto, está **criando um novo tipo** que armazena referências a métodos.
- Eventos usam delegados para notificar objetos que certas ações ocorreram.
- Objetos inscrevem-se nos eventos de objetos se quiserem reagir a alguma coisa ocorrida naquele objeto.
- Um EventHandler é um tipo de delegado muito comum quando você trabalha com eventos.
- Você encadeia muitos tratadores num evento único. É por isso que você usa o += para fazer isso.
- Sempre cheque se um evento ou delegado são nulos antes de usá-los para evitar uma NullReferenceException.
- Todos os controles na caixa de ferramentas usam eventos para fazer coisas acontecerem nos seus programas.
- Quando um objeto passa uma referência de um método para outro objeto para que ele - e somente ele - possa retornar informações, esse método é chamado de callback.
- Eventos permitem a qualquer método inscreverem-se anonimamente, enquanto que callbacks permitem exercer mais controle sobre quais delegados são aceitáveis.
- Tanto callbacks quanto eventos usam delegados para fazer referência e chamar métodos de outros objetos.
- O depurador é uma ferramenta realmente útil para ajudar a entender como eventos, delegados e callbacks funcionam. Aproveite isso!

não existem Perguntas Idiotas

P: Como os callbacks diferem dos eventos?

R: Eventos são parte do C#. Eles a forma de um objeto avisar aos demais que algo específico ocorreu. Quando um objeto deixa um evento público, qualquer quantidade de outros objetos podem se inscrever sem que o objeto de origem saiba ou se importe. Quando um objeto dispara um evento, se por acaso alguém tiver se inscrito, os métodos de tratamento de cada um deles serão chamados. Callbacks não são públicos. Em vez disto, uma relação entre dois objetos é estabelecida e um objeto automaticamente reage ao outro. Geralmente, um callback é mantido privado e a classe que o armazena tem controle sobre a permissão de acesso. Um callback é freqüentemente atribuído dentro do construtor de um objeto.

P: Então um callback não é, na realidade, um tipo nativo do C#?

R: Não. Um callback é um **padrão** - é apenas uma nova forma de usar os tipos, palavras- chave e ferramentas do C#. Volte e olhe o código de callback que acabou de escrever para as classes Bat e Ball. Você não vê qualquer palavra nova que nós já usamos antes? Não! Há muitos padrões possíveis. De fato, existe uma área inteira da programação chamada padrões de projeto **(design patterns)**. Há muitos problemas que você encontrará e que já foram resolvidos no passado; e os que se repetem com muita freqüência têm seus próprios padrões de projeto – e você pode tirar proveito disso.

Veja o "Use a Cabeça Padrões de Projeto" no site dos Laboratórios Use a Cabeça. É uma ótima forma de aprender sobre os diferentes padrões aplicáveis aos seus programas.

www.altabooks.com.br

P: Então callbacks são apenas eventos privados?

R: Não exatamente. É fácil pensar assim, mas um evento privado é um bicho totalmente diferente. Lembra-se do que o modificador de acesso `private` realmente significa? Quando você marca um membro de uma classe como `privado`, instâncias da mesma classe ainda podem acessá-lo. Então se você marcar um evento como privado, outras instâncias da mesma classe ainda podem se inscrever nele. Isso é diferente de um callback, porque eles ainda implicam que um objeto anonimamente se inscreva num evento.

P: Mas ele parece exatamente com um evento, exceto pela palavra-chave event, certo?

R: O motivo pelo qual um callback parece tanto com um evento é que ambos usam **delegados**. E isso faz sentido, já que estes últimos são a ferramenta do C# que permite a um objeto passar para outro uma referência de um de seus métodos.
Mas a grande diferença entre eventos normais e callbacks é que um evento é uma forma de uma classe publicar para o mundo que algo específico ocorreu. Um callback, por outro lado, nunca é público. Ele é privado e o método que faz a chamada mantém um controle estrito sobre quem ele está chamando.

O primeiro padrão que você verá é chamado de padrão de "publicação-inscrição" e ele deve parecer-lhe bastante familiar. Um objeto publica informação e os outros objetos inscrevem-se para obtê-la. Hummm...

O Caso do Crustáceo Dourado

Como os outros caçadores de tesouro chegaram antes de Enrique?

Mistério de Cinco Minutos Resolvido

O cerne do mistério reside na forma como o caçador de tesouro buscou sua recompensa. Mas primeiro precisamos ver exatamente o que Enrique achou nos diagramas roubados.

Num diagrama de classe roubado, Enrique descobriu que a classe CaranguejoDourado dispara o evento CorrerParaSeEsconder toda vez que alguém tenta chegar perto dele. Ainda melhor, o evento inclui ArgsNovaPosicao, com detalhes a respeito da rota do caranguejo. E nenhum dos outros caçadores de tesouro sabe sobre esses eventos, então Enrique imagina que pode ficar rico.

```
public class GoldenCrab {
  public delegate void Escape(NewLocationArgs e);
  public event Escape RunForCover;
  public void SomeonesNearby() {
    NewLocationArgs e = new NewLocationArgs("Under the rock");
    RunForCover(e);
  }
}
public class NewLocationArgs {
  public NewLocationArgs(HidingPlace newLocation) {
    this.newLocation = newLocation;
  }
  private HidingPlace newLocation;
  public HidingPlace NewLocation { get { return newLocation; } }
}
```

Quando alguém se aproxima do caranguejo dourado, seu método SomeonesNearby() (alguém por perto) dispara o evento RunForCover (correr para se esconder) e ele encontra um local para se esconder.

Então como Enrique se aproveitou dessa nova informação secreta?

Enrique adiciona código ao seu construtor para registrar seu método AtencaoParaPistas() como um novo tratador para o evento CorrerParaSeEsconder, na referência do caranguejo que ele possui. Então ele envia um capanga de má reputação atrás do caranwguejo, sabendo que isso vai fazê-lo fugir, se esconder e disparar o evento CorrerParaSeEsconder - o que dá ao método AtencaoParaPistas() de Enrique toda a informação necessária...

```
public class TreasureHunter {
  public TreasureHunter(GoldenCrab treasure) {
    treasure.RunForCover += new GoldenCrab.Escape(treasure_RunForCover);
  }
  void treasure_RunForCover(NewLocationArgs e) {
    MoveHere(e.NewLocation);
  }
  void MoveHere(HidingPlace Location) {
    // ... código mover para nova localização ...
  }
}
```

Enrique pensou que estava sendo inteligente alterando o construtor de sua classe para adicionar um tratador de evento que chamaria seu método MoveHere() (mover até aqui) quando o caranguejo levantasse o evento RunForCover. Mas ele esqueceu-se de que os outros caçadores de tesouro herdam da mesma classe e seu código inteligente encadeou seus tratadores de evento também!

Isso explica porque o plano de Enrique saiu pela culatra. Quando ele adicionou o tratador de evento no construtor de TreasureHunter (caçador de tesouros), inadvertidamente, **fez o mesmo para todos os outros caçadores!** Ou seja, os métodos tratadores de todos eles foram encadeados no mesmo evento RunForCover. Assim, quando o crustáceo dourado correu para se esconder, todos foram notificados a respeito do evento. E tudo isso seria perfeito se Enrique fosse o primeiro a receber a mensagem. Mas ele não tinha como saber quando os demais caçadores foram chamados - se eles inscreveram-se antes dele, foram chamados antes.

Aponte seu lápis

Preencha as lacunas para fazer com que esse jogo de acerte-a-toupeira funcione. Você deverá criar o código que faz os callbacks. Uma vez feito isso, vá em frente e digite tudo no IDE. Ou você pode tentar fazê-lo funcionar direto no IDE e então preencher as lacunas depois. É divertido!

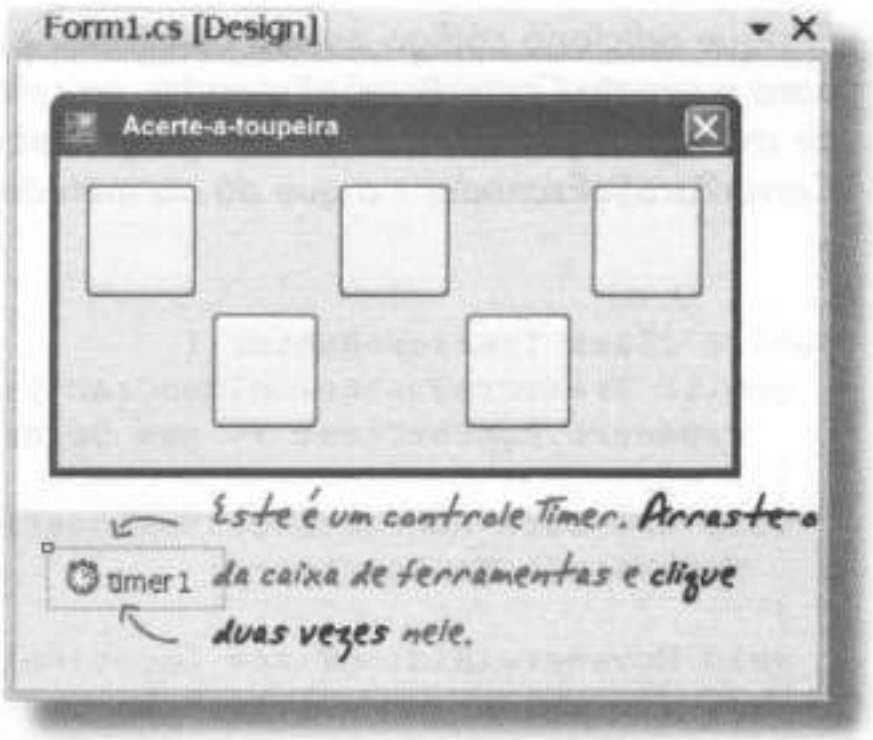
```
public partial class Form1 : Form {
  Mole mole;
  Random random = new Random();
  public Form1() {
    InitializeComponent();
    mole = new Mole(random, new Mole._____(_____));
    timer1.Interval = random.Next(500, 1000);
    timer1.Start();
  }
  private void timer1_Tick(object sender, EventArgs e) {
    timer1.Stop();
    ToggleMole();
  }
  private void ToggleMole() {
    if (mole.Hidden == true)
      mole.Show();
    else
      mole.HideAgain();
    timer1.Interval = random.Next(500, 1000);
    timer1.Start();
  }
  private void MoleCallBack(int MoleNumber, bool Show) {
    if (MoleNumber < 0) {
      timer1.Stop();
      return;
    }
    Button button;
    switch (MoleNumber) {
      case 0: button = button1; break;
      case 1: button = button2; break;
      case 2: button = button3; break;
      case 3: button = button4; break;
      default: button = button5; break;
    }
    if (Show == true) {
      button.Text = "HIT ME!";
      button.BackColor = Color.Red;
    } else {
      button.Text = "";
      button.BackColor = SystemColors.Control;
    }
    timer1.Interval = random.Next(500, 1000);
    timer1.Start();
  }
  private void button1_Click(object sender, EventArgs e) {
    mole.Smacked(0);
  }
}
```

O formulário passa um delegado apontando para um método de callback para as toupeiras.

Quando você clicar duas vezes no timer no formulário (depois que você arrastá-lo da caixa de ferramentas), o IDE criará esse tratador de evento para você. Os timers disparam o evento Tick continuamente. Você aprenderá a respeito deles nos próximos capítulos.

Esse método é chamado para mostrar ou ocultar a toupeira quando o tempo marcado pelo timer tiver passado.

Esse switch certifica-se de que o botão certo mude de cor e texto.



Este é um controle Timer. Arraste-o da caixa de ferramentas e clique duas vezes nele.

Adicione esses tratadores de evento da forma normal clicando duas vezes no designer.

Quando você digitar o código, **adicione cinco tratadores de evento de botão.** Faça com que button2_click() chame mole.Smacked() [toupeira.amassada()], button3_click() chame mole.Smacked(2), button4_click(), mole.Smacked(3) e button5_click() mole.Smacked(4).

```
public class Mole {
  public _____ void PopUp(int hole, bool show);
  private _____ popUpCallback;
  private bool hidden;
  public bool Hidden { get { return hidden; } }
  private int timesHit = 0;
  private int timesShown = 0;
  private int hole = 0;
  Random random;
  public Mole(Random random, PopUp popUpCallback) {
    if (popUpCallback == null)
      throw new ArgumentException("popUpCallback can't be null");
    this.random = random;
    this._____ = _____;
    hidden = true;
  }
  public void Show() {
    timesShown++;
    hidden = false;
    hole = random.Next(5);
    _____(hole, true);
  }
  public void HideAgain() {
    hidden = true;
    _____(hole, false);
    CheckForGameOver();
  }
  public void Smacked(int holeSmacked) {
    if (holeSmacked == hole) {
      timesHit++;
      hidden = true;
      CheckForGameOver();
      _____(hole, false);
    }
  }
  private void CheckForGameOver() {
    if (timesShown >= 10) {
      popUpCallback(-1, false);
      MessageBox.Show("You scored " + timesHit, "Game over");
      Application.Exit();
    }
  }
}
```

Complete o delegado e o campo que o armazenará - ambos no topo da classe Mole (toupeira).

É aqui que nos certificamos de que o callback não é nulo - se for, o objeto Mole lança uma ArgumentException.

Quando o formulário cria um novo objeto Mole, ele passa uma referência para esse método de callback. Dê uma olhada no formulário para ver como o construtor é chamado e preencha essa lacuna.

Depois que a toupeira se mostrar, ela precisa chamar o método no formulário para exibi-la, e para isso deixa o botão vermelho e mostra o texto "HIT ME!" (me acerte).

Os métodos HideAgain() (esconder de novo) e Smacked() também usam o callback delegado para chamar o método no formulário.

O jogo usa o timer para esperar um intervalo de tempo aleatório entre 1s e 1,5 s. Quando este intervalo passa, ele diz à toupeira para se mostrar. O formulário dá ao objeto Mole um callback usado para dizer ao formulário que ele aparecerá ou se esconderá num dos cinco buracos. O formulário usa seu timer para esperar entre 0.5 s e 1.5 s mais uma vez e então diz à toupeira para se esconder.

O jogo acaba depois que a toupeira se mostrou 10 vezes. Sua pontuação é o número de vezes que você a atingiu.

Aponte seu lápis
Solução

Preencha as lacunas para fazer com que esse jogo de acerte-a-toupeira funcione. Você precisará criar o código que faz os callbacks. Uma vez feito isso, vá em frente e digite tudo no IDE. É divertido!

```
public partial class Form1 : Form {
  private void Form1_Load(object sender, EventArgs e) {
    mole = new Mole(random, new Mole.  Pop Up   ( MoleCallBack      ));
    timer1.Interval = random.Next(500, 1000);
    timer1.Start();
  }
.}
public class Mole {
  public   delegate   void PopUp(int hole, bool show);
  private  Pop Up    popUpCallback;
...
  public Mole(Random random, PopUp popUpCallback) {
    this.random = random;
    this.  popUpCallback     =  popUpCallback     ;
    hidden = true;
  }
  public void Show() {
    timesShown++;
    hidden = false;
    hole = random.Next(5);
      popUpCallback    (hole, true);
  }
  public void HideAgain() {
    hidden = true;
      popUpCallback    (hole, false);
    CheckForGameOver();
  }
  public void Smacked(int holeSmacked) {
    if (holeSmacked == hole) {
      timesHit++;
      hidden = true;
      CheckForGameOver();
        popUpCallback    (hole, false);
    }
  }
```

É aqui que o formulário passa uma referência de seu método MoleCallBack() para o objeto Mole. Isso permite que esse último chame o método do primeiro.

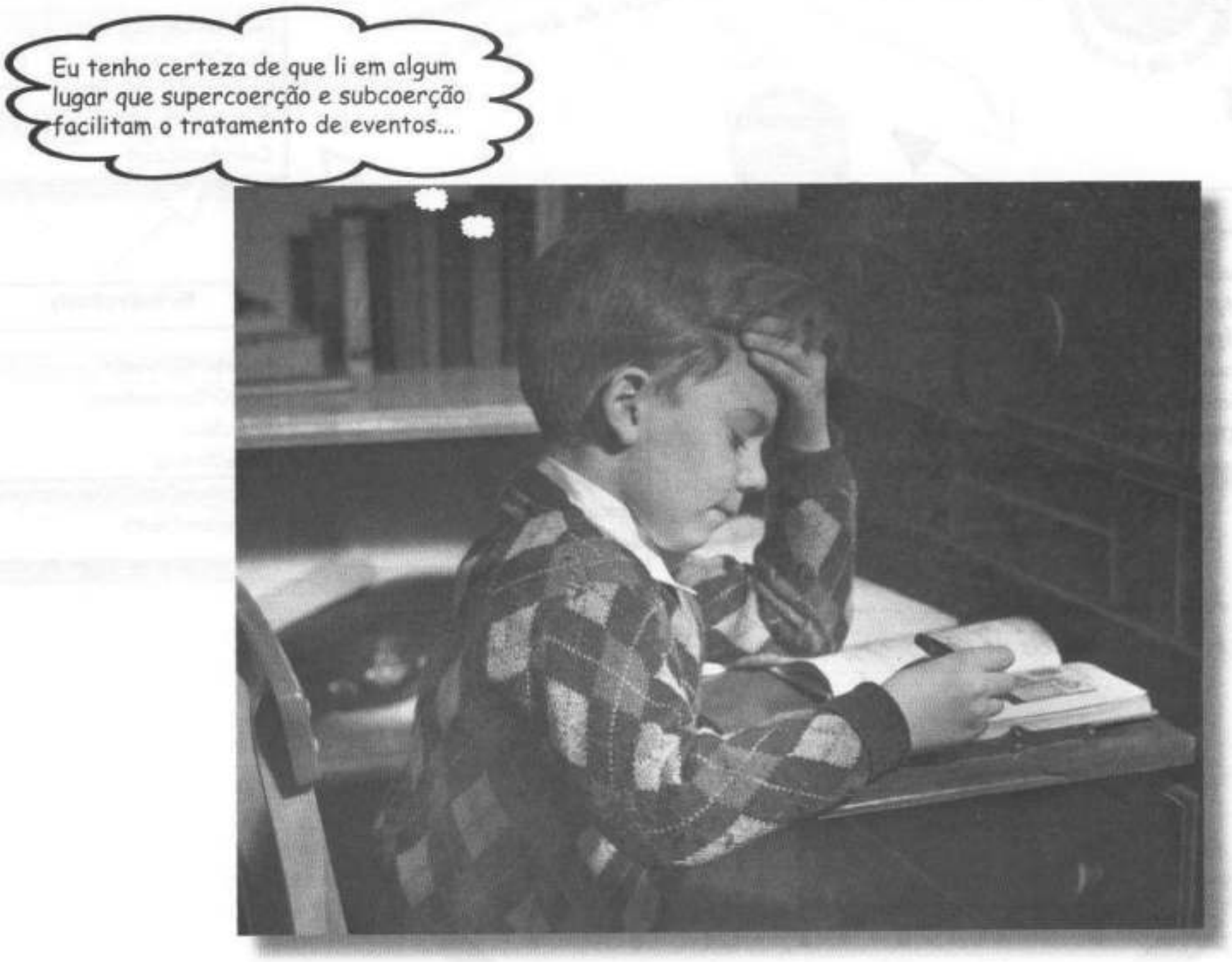
Eis onde a toupeira define seu delegado e o utiliza para criar um campo privado para armazenar uma referência ao método no formulário que muda as cores dos botões.

Quando o formulário cria uma nova instância do objeto Mole, ele passa uma referência do seu método MoleCallBack() para o construtor como um parâmetro. Essa linha no construtor copia a referência para seu campo popUpCallback. Seus métodos podem usar esse campo para chamar o método MoleCallBack() do formulário.

Quando a toupeira se mostra, depois se esconde ou é amassada, o objeto Mole usa seu campo delegado popUpCallback para chamar o método no formulário que muda a cor e o texto de um dos botões.

12 revisão e prévia

✳Conhecimento, poder e desenvolver coisas legais✳



Aprender não serve muito até que você CRIE algo.

Até que você tenha realmente escrito código funcional, é difícil ter certeza se você realmente sabe alguns dos conceitos mais complexos do C#. Nesse capítulo, aprenderemos sobre novos conceitos e amarraremos algumas pontas soltas: falaremos sobre timers e como lidar com coleções usando LINQ, entre outras coisas. Nós também vamos desenvolver a fase 1 de um aplicativo realmente complexo e certificarmo-nos de que você tenha um bom domínio sobre o que já aprendeu nos capítulos anteriores. Então ponha o cinto de segurança... é hora de desenvolver um software legal.

Você percorreu um longo caminho, ~~baby~~

[nota do RH: "baby" não é mais politicamente correto. Por favor, usar as expressões "prejudicado quanto à idade" ou "infante" para evitar ofender os leitores.]

Percorremos um longo caminho desde que você usou o IDE para ajudar-nos a prestar um favor para a empresa de papel da Vila Objeto. Aqui estão apenas algumas das coisas que você fez ao longo de várias centenas de páginas:

Você criou formulários, usou o Framework .NET e até mesmo utilizou bases de dados.

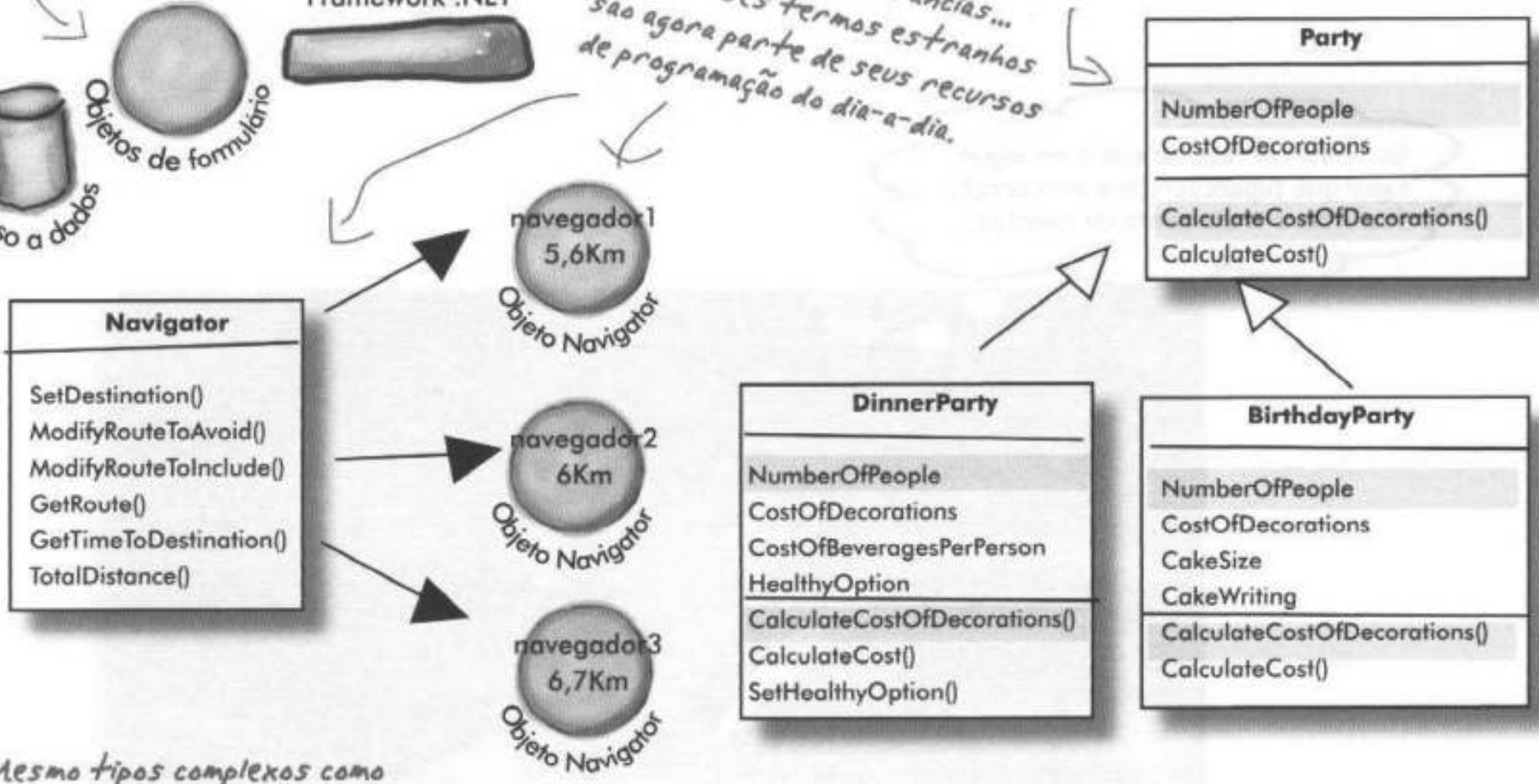
Soluções do Framework .NET

Objetos de formulário

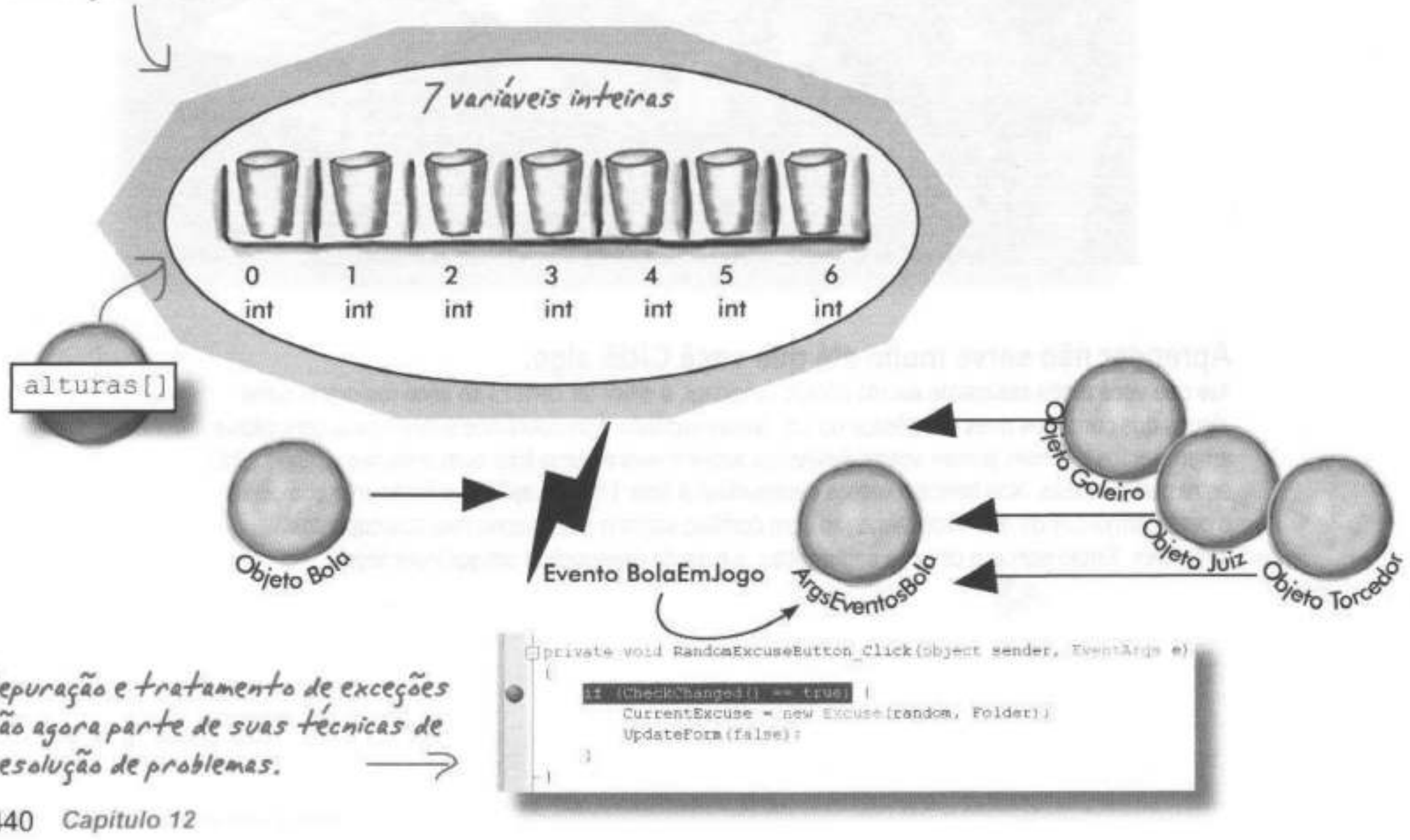
Acesso a dados

Objetos, classes, instâncias... todos esses termos estranhos são agora parte de seus recursos de programação do dia-a-dia.

Você empregou herança, bem como interfaces e subclasses, para montar árvores de objetos.



Mesmo tipos complexos como matrizes não são difíceis de lidar.

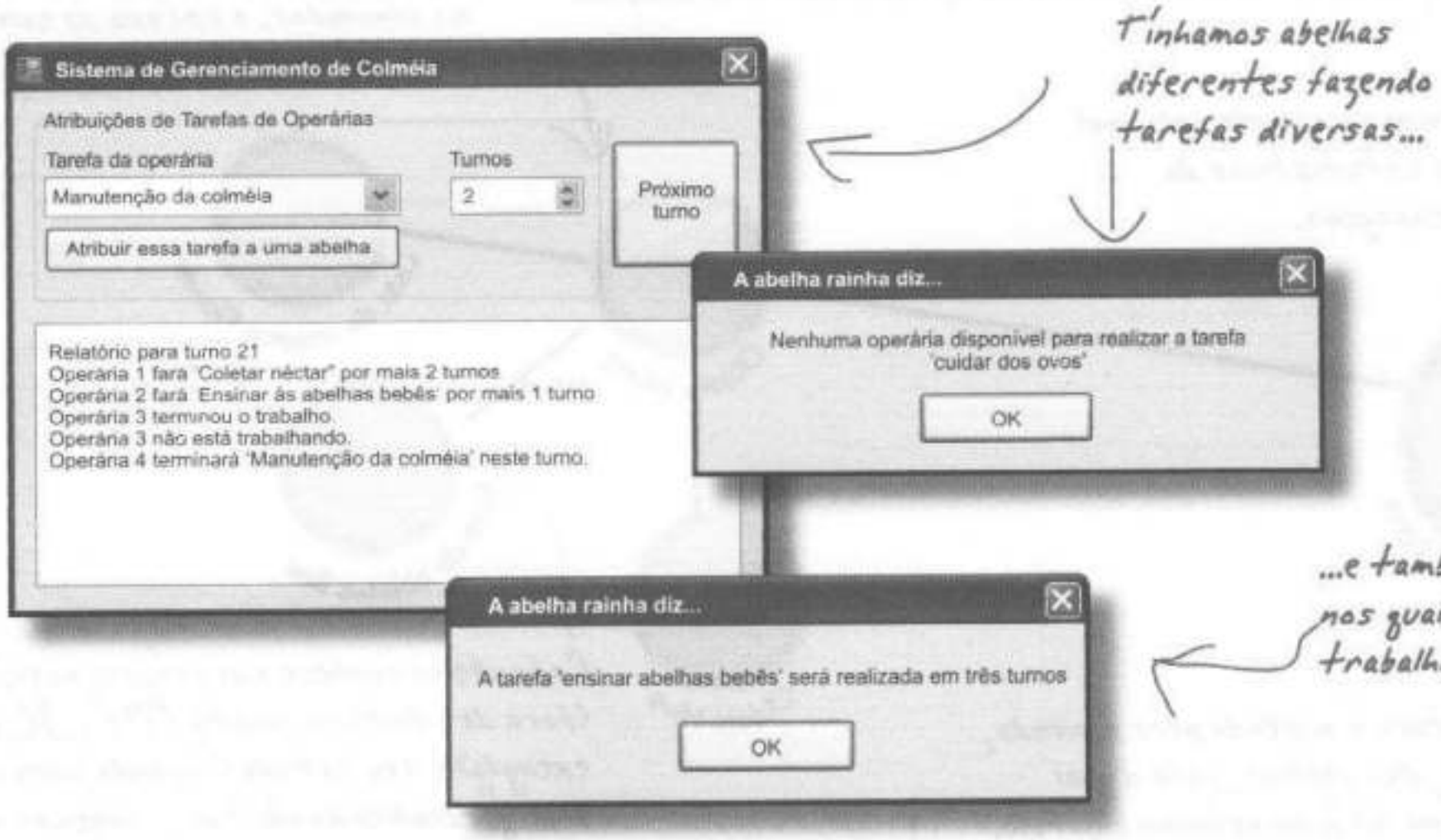


Depuração e tratamento de exceções são agora parte de suas técnicas de resolução de problemas.

```
private void RandomExcuseButton_Click(object sender, EventArgs e)
{
    if (CheckChanged() == true) {
        CurrentExcuse = new Excuse(random, Folder);
        UpdateForm(false);
    }
}
```

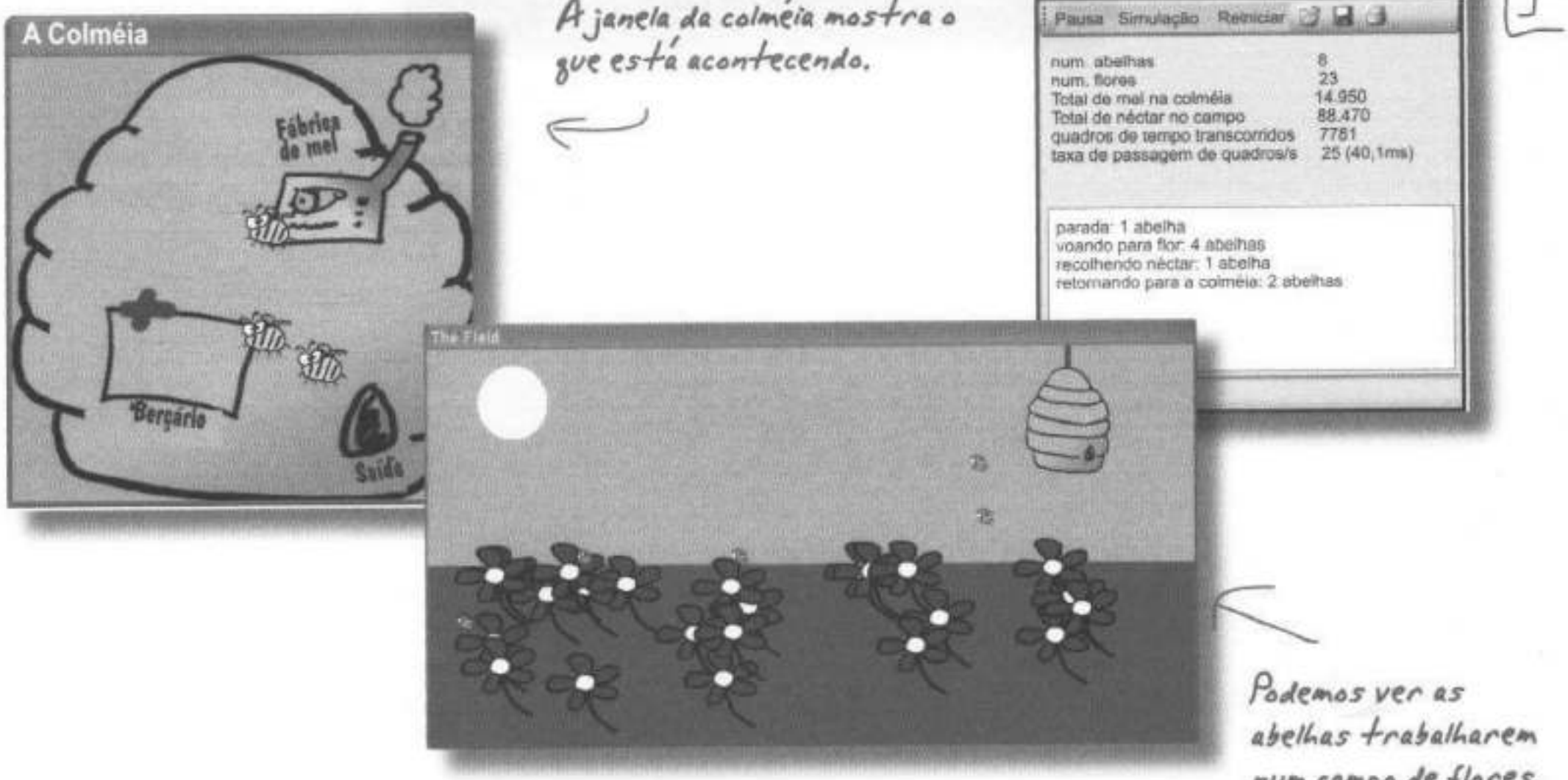
Nós já fomos apicultores

Lá no capítulo 6, criamos algumas classes de abelhas. Lembra-se delas?



Mas podemos fazer muito melhor agora...

No entanto, você aprendeu muito desde o capítulo 6. Então vamos começar do zero e criar um **simulador de colméia animado** nos próximos capítulos. Terminaremos com uma interface de usuário que mostrará a colméia e as abelhas mantidas e uma janela de estatísticas para os usuários acompanharem o que as abelhas estão fazendo.

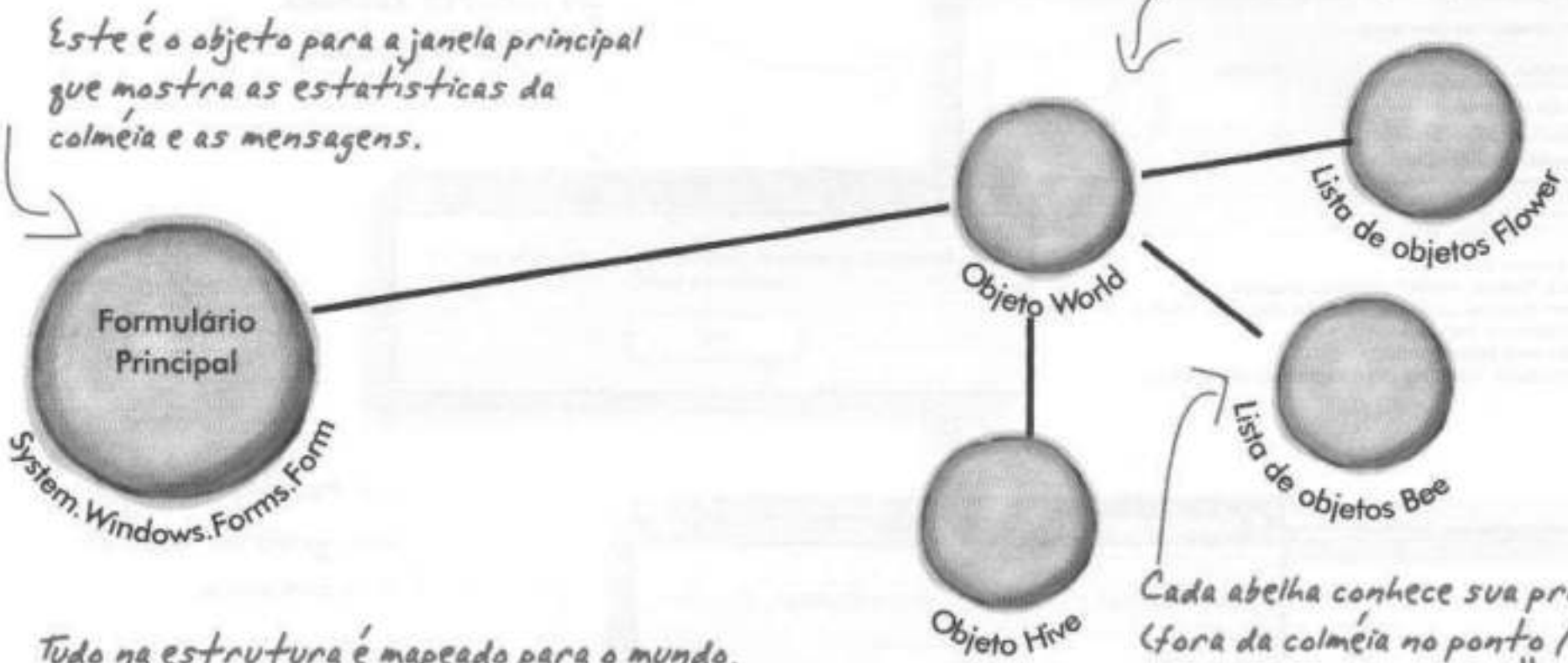


A estrutura do simulador de colméia

Aqui está a estrutura para o simulador de colméia. Mesmo que ele controle muitas abelhas diferentes, o modelo geral de objetos é bastante simples.

O objeto World (mundo) mantém registro de tudo o que acontece no simulador: o estado da colméia, cada abelha e cada flor.

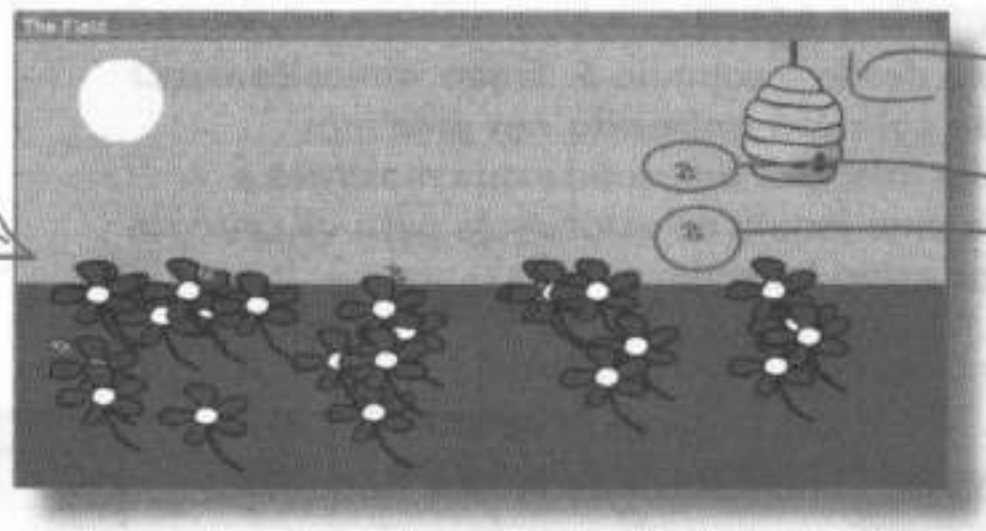
Este é o objeto para a janela principal que mostra as estatísticas da colméia e as mensagens.



Cada abelha conhece sua própria posição (fora da colméia no ponto 174, 36, por exemplo) e seu estado ("voando para uma flor", "recolhendo néctar", "fazendo mel").

Tudo na estrutura é mapeado para o mundo, de forma geral, das abelhas, para o qual montaremos uma GUI no próximo capítulo.

World representará a coisa toda.



A colméia é o quartel general das abelhas.

E, é claro, precisamos de uma classe Bee.

Precisaremos de objetos Flower para cada flor.

Desenvolvendo o simulador de colméia

Como nunca fizemos algo tão complexo antes, levará alguns capítulos para juntar todas as peças. Pelo caminho, adicionaremos timers, LINQ e muita habilidade em lidar com gráficos no seu currículo.
Aqui está o que você fará neste capítulo (faremos mais no próximo):

1. **Desenvolver uma classe Flower que envelhece, produz néctar e, eventualmente, murcha e morre.**

2. **Criar uma classe Bee com diversos estados diferentes (recolher néctar de uma flor, retornar à colméia, etc.) e saberá o que fazer de acordo com esse estado.**

3. **Produzir uma classe Hive que terá uma entrada, uma saída, um local para as abelhas que nascem e uma área para fabricação para converter néctar coletado em mel.**

4. **Fazer uma classe World (mundo) que gerenciará a colméia, as flores e as abelhas em qualquer momento.**

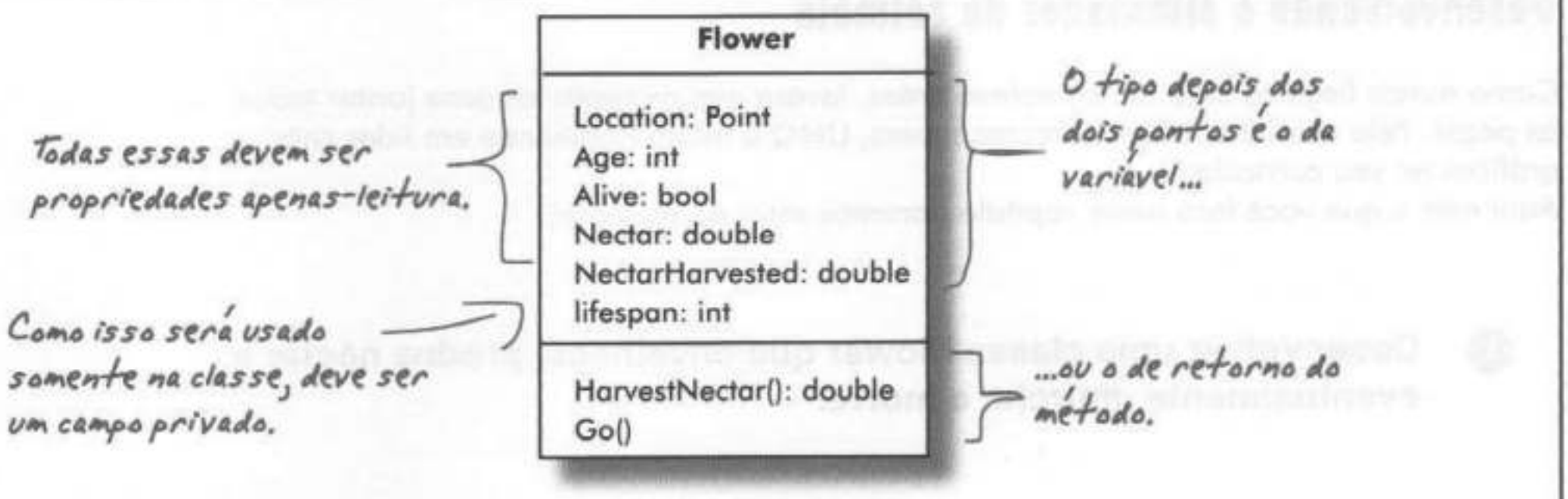
5. **Associar tudo num formulário principal que calculará estatísticas se baseando nas demais classes e manterá o mundo funcionando.**

Exercício

Vamos direto desenvolver algum código. Primeiro, precisaremos de uma classe Flower. Ela terá uma posição, definida por um Point, uma idade e um tempo de vida. Conforme o tempo passa, a flor fica mais velha. Então, quando a idade atinge o tempo de vida, a flor morre. É sua tarefa fazer tudo isso funcionar.

Um "esqueleto" de classe é apenas as suas declarações de campos, propriedades e métodos, sem implementação.

1. **Escreva o esqueleto do código para flor**
Abaixo está o diagrama de classe para `Flower`. Escreva o esqueleto básico. `Location` (posição), `Age` (idade), `Alive` (viva), `Nectar` e `NectarHarvested` (néctar colhido) são **propriedades automáticas.** Esta última pode ser escrita e as demais são **apenas-leitura.** Nesse momento, deixe os métodos em branco - vamos voltar a falar deles num minuto.



2 **Adicione diversas constantes à classe**
Precisaremos de muitas constantes para flores. Adicione essas seis:

- `LifeSpanMin` (tempo de vida mínimo), o intervalo mais curto que uma flor pode viver.
- `LifeSpanMax` (tempo máximo de vida), o maior tempo de vida possível para uma flor.
- `InitialNectar` (néctar inicial), com quanto néctar uma flor inicia.
- `MaxNectar`, o máximo de néctar que uma flor pode conter.
- `NectarAddedPerTurn` (néctar adicionado por turno), quanto néctar é adicionado a cada vez que a flor envelhece um pouco.
- `NectarGatheredPerTurn` (néctar recolhido por turno), quanto néctar pode ser coletado durante um ciclo.

Só para você saber, normalmente, não se mostram constantes em diagramas de classe.

Você deve ser capaz de determinar os tipos de cada constante baseando-se em seus valores. Flores vivem entre 15.000 e 30.000 ciclos e tem 1,5 unidades de néctar quando iniciam. Podem armazenar até 5 unidades. Em cada ciclo durante a vida da flor, ela adiciona 0,01 unidades de néctar e num único ciclo, 0,3 unidades podem ser coletadas.

Você precisará adicionar using System.Drawing; no início de qualquer arquivo de classe que use Point.

3 **Crie o construtor**
O construtor para Flower deve receber um Point, indicando a posição da flor, e uma instância da classe Random. Você deve ser capaz de usar esses argumentos para determinar a posição da flor e então atribuir zero para sua idade, verdadeiro para a propriedade que indica se ela está viva e a quantidade inicial de néctar. Já que nenhum néctar foi ainda recolhido, atribua também o valor correto para a variável que indica essa quantidade. Finalmente, determine o tempo de vida da flor. Eis uma linha de código para ajudar:

```
lifeSpan = random.Next(LifeSpanMin, LifeSpanMax + 1);
```

Isso só funcionará se você tiver suas variáveis e constantes nomeadas de acordo, bem como os argumentos do construtor de Flower.

4 **Escreva código para o método HarvestNectar()**
Cada vez que esse método for chamado, ele deve checar para ver se o néctar colhido no ciclo é maior do que a quantidade restante. Se for o caso, ele deve retornar zero. Se não, deve retirar a quantidade coletada num ciclo do total restante da flor e retornar quanto foi coletado. Ah, e não se esqueça de adicionar a quantidade colhida a NectarHarvested, que manterá o total de néctar coletado de cada flor em particular.

* Dica: Nós usamos NectarGatheredPerTurn, néctar e NectarHarvested nesse método e nada mais.

5 **Escreva código para o método Go()**
Esse será o método que fará as flores "funcionarem" (go é ir em inglês). Assuma que cada vez que esse método for chamado, um ciclo se passará, então atualize a idade da flor corretamente. Você também deverá ver se a idade é maior do que o tempo de vida, e se for o caso, a flor deve morrer.

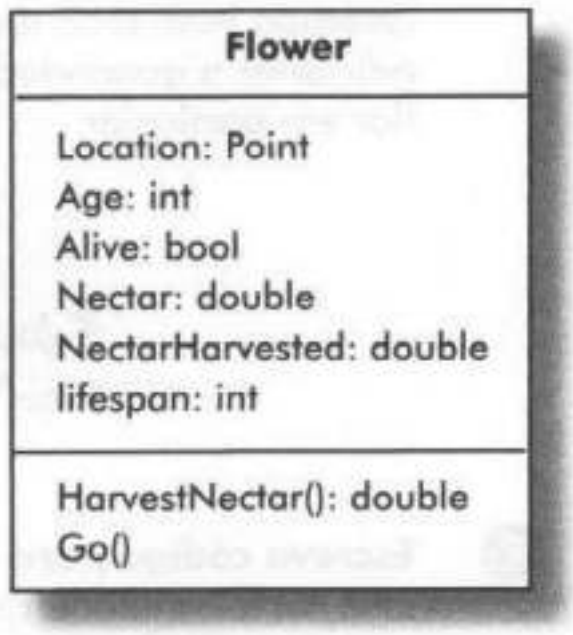
Assumindo que a flor continue viva, será preciso adicionar a quantidade de néctar que cada flor produz num ciclo. Certifique-se de checar para não ultrapassar o limite de néctar que uma flor pode manter.

O produto final será animado, com pequenas imagens de abelhas voando por aí. O método GoO será chamado uma vez por quadro de tempo e executaremos muitos desses por segundo.

Respostas na próxima página... tente finalizar e compilar seu código antes de olhar.

Solução do Exercício

Sua tarefa foi desenvolver a classe Flower para o simulador de colméia.




```
public class Flower {
  private const int LifeSpanMin = 15000;
  private const int LifeSpanMax = 30000;
  private const double InitialNectar = 1.5;
  private const double MaxNectar = 5.0;
  private const double NectarAddedPerTurn = 0.01;
  private const double NectarGatheredPerTurn = 0.3;
  public Point Location { get; private set; }
  public int Age { get; private set; }
  public bool Alive { get; private set; }
  public double Nectar { get; private set; }
  public double NectarHarvested { get; set; }
  private int lifeSpan;
  public Flower(Point location, Random random) {
    Location = location;
    Age = 0;
    Alive = true;
    Nectar = InitialNectar;
    NectarHarvested = 0;
    lifeSpan = random.Next(LifeSpanMin, LifeSpanMax + 1);
  }
  public double HarvestNectar() {
    if (NectarGatheredPerTurn > Nectar)
      return 0;
    else {
      Nectar -= NectarGatheredPerTurn;
      NectarHarvested += NectarGatheredPerTurn;
      return NectarGatheredPerTurn;
    }
  }
  public void Go() {
    Age++;
    if (Age > lifeSpan)
      Alive = false;
    else {
      Nectar += NectarAddedPerTurn;
      if (Nectar > MaxNectar)
        Nectar = MaxNectar;
    }
  }
}
```

Location, Age, Alive e Nectar são propriedades automaticas apenas-leitura.

NectarHarvested precisa ser acessado pelas demais classes.

Flores têm tempos de vida aleatórios para que o campo de flores não mude todo no mesmo instante.

Uma abelha chama HarvestNectar() para coletar néctar da flor. Uma abelha pode apenas recolher um pouco do néctar de cada vez. Por isto, ela deverá ficar em torno da flor por muitos turnos até colher todo o néctar.

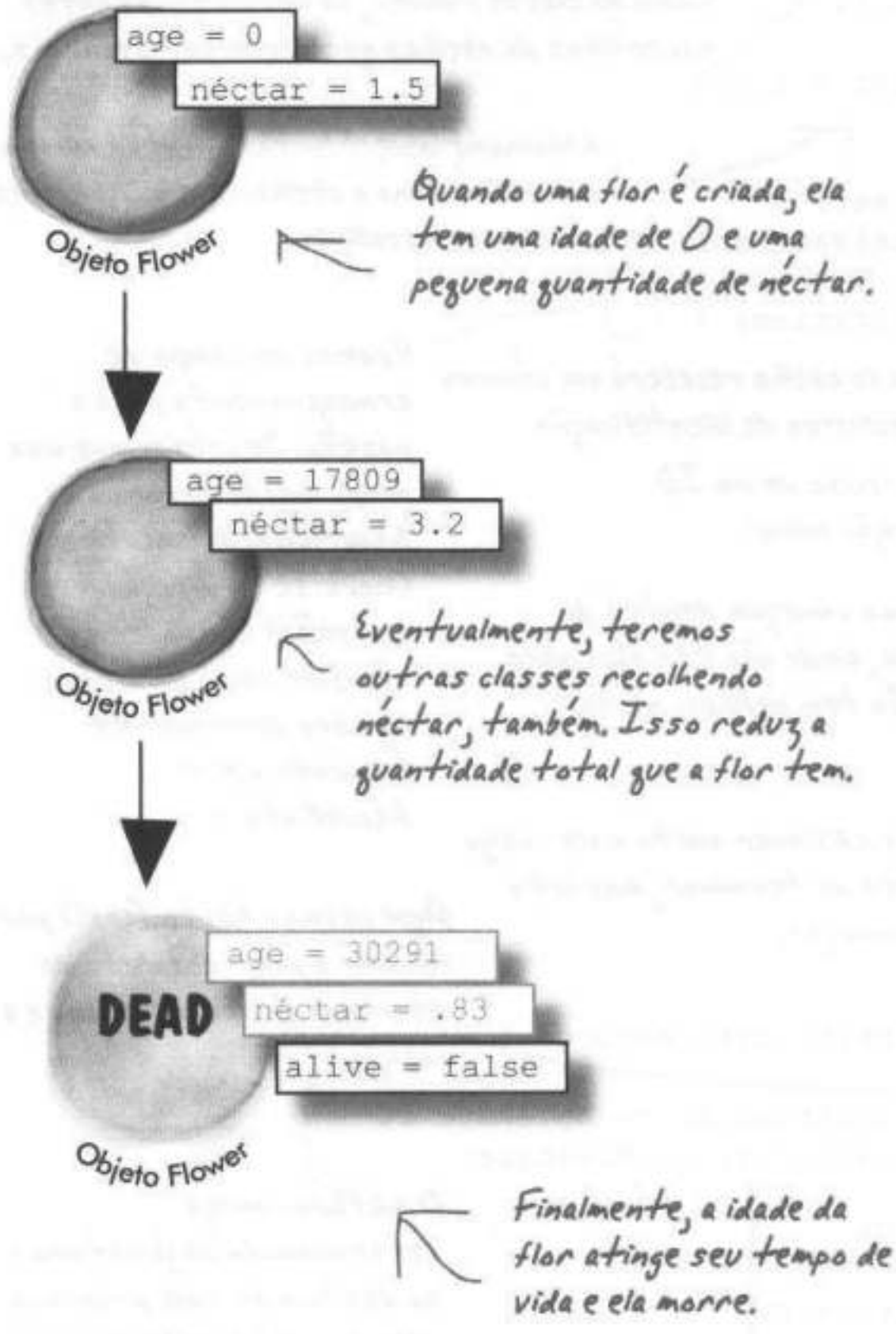
Certifique-se de que a flor pare de adicionar néctar se estiver morta.

Como parte da animação do simulador, o método Go() será chamado a cada quadro de tempo. Isso fará a flor envelhecer apenas uma pequena fração de cada vez - conforme o simulador for executando, essas pequenas frações vão se acumulando.

`Point` está contido no namespace System.Drawing, então tenha certeza de adicionar `using System.Drawing;` no início do arquivo de classe.

Vida e morte de uma flor

Nossa flor tem apenas uma forma básica de funcionar a cada turno: ela vive, adiciona néctar, tem ele recolhido e, eventualmente, morre.



não existem Perguntas Idiotas

P: Parece que NectarHarvested não é usado para nada na classe, exceto quando somamos nele. Para que ele serve?

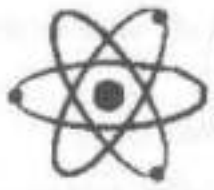
R: Boa pergunta! Estamos planejando um pouco para o futuro. Eventualmente o simulador vigiará as flores e o total de néctar recolhido delas para nossa janela de estatística. Então deixe a variável assim mesmo e nossas outras classes vão usá-la em breve.

P: Qual a razão das propriedades automáticas apenas-leitura?

R: Lembra-se do capítulo 5 e como escondíamos nossos elementos privados? Sempre é uma boa prática. As flores podem tomar conta desses valores, então fizemos com que eles fossem apenas-leitura. Outros objetos, como abelhas e a colméia, devem ser capazes de ler essas propriedades, mas não de alterá-las.

P: Meu código está diferente do seu. Fiz alguma coisa errada?

R: Você pode ter código nos seus métodos numa ordem diferente, mas desde que ele funcione da mesma forma que o nosso, está tudo bem. Esse é outro aspecto do encapsulamento: os detalhes internos de cada classe não são importantes para as demais, desde que cada uma delas faça o que deve fazer.



Poder do Cérebro

Se Go() aumenta a idade da flor em 1 e o tempo de vida está entre 15.000 e 30.000, Go() será chamada pelo menos 15.000 vezes para cada flor antes que ela morra. Como você pode lidar com uma quantidade de chamadas tão grande? E se existirem 10 flores? 100? 1000?

Agora precisamos de uma classe Bee

Com as flores prontas para fornecerem néctar, precisamos de uma classe `Bee`. Abaixo está o código básico para ela. Ela sabe sua própria idade, se ela está na colméia ou não, e quanto néctar ela pode coletar. Também adicionamos um método para mover a abelha na direção de um ponto de destino específico.

```
public class Bee {
  private const double HoneyConsumed = 0.5;
  private const int MoveRate = 3;
  private const double MinimumFlowerNectar = 1.5;
  private const int CareerSpan = 1000;
  public int Age { get; private set; }
  public bool InsideHive { get; private set; }
  public double NectarCollected { get; private set; }
  private Point location;
  public Point Location { get { return location; } }
  private int ID;
  private Flower destinationFlower;
  public Bee(int id, Point location) {
    this.ID = ID;
    Age = 0;
    this.location = location;
    InsideHive = true;
    destinationFlower = null;
    NectarCollected = 0;
  }
  public void Go(Random random) {
    Age++;
  }
```

Como na classe Flower, há várias constantes específicas de abelhas que devem ser definidas.

MinimumFlowerNectar (néctar mínimo na flor) é como a abelha sabe quais flores podem ser escolhidas.

Usamos um campo de armazenamento para a posição. Se usássemos uma propriedade automática, MoveTowardsLocation() (move-se em direção a posição) não poderia atribuir valor para esse membro diretamente ("Location.X -= MoveRate").

Cada abelha receberá um número exclusivo de identificação.

Uma abelha precisa de um ID e de uma posição inicial.

Abelhas começam dentro da colméia, ainda não tem flor para ir e não tem nenhum néctar.

Precisaremos adicionar muito mais código em Go() antes de terminar, mas isto serve para começar.

```
  private bool MoveTowardsLocation(Point destination) {
    if (destination != null) {
      if (Math.Abs(destination.X - location.X) <= MoveRate &&
          Math.Abs(destination.Y - location.Y) <= MoveRate)
          return true;
      if (destination.X > location.X)
        location.X += MoveRate;
      else if (destination.X < location.X)
        location.X -= MoveRate;
      if (destination.Y > location.Y)
        location.Y += MoveRate;
      else if (destination.Y < location.Y)
        location.Y -= MoveRate;
    }
    return false;
  }
}
```

Aqui usamos Math.Abs() para calcular o valor absoluto da diferença entre o destino e a posição atual.

O método começa determinando se já estamos no destino ou mais próximos dele do que MoveRate (taxa de deslocamento).

Se a abelha não tem um local de destino, o campo será nulo. Logo só nos movemos na direção de um destino se ele NÃO for nulo.

Se não estamos pertos o suficiente, movemo-nos na direção do destino uma distância igual à taxa de deslocamento.

Retornamos false, pois ainda não estamos no ponto de destino. Precisamos continuar nos movendo.

O ponto final onde MoveTowardLocation() deixa a abelha é a posição atual modificada nos seus componentes X e Y, o que fica armazenado no campo location. Ela retorna true se a abelha atingiu o ponto de destino.

Exercício

Abelhas podem fazer muitas coisas. Abaixo mostramos uma lista. Crie uma nova enum que `Bee` usará chamada `BeeState` (estado da abelha). Você pode também criar uma propriedade automática chamada `CurrentState` (estado atual) para cada abelha para manter registro do estado de cada uma delas. Deixe o estado inicial das abelhas em idle (desocupada) e no método `Go()`, adicione um comando switch com uma opção para cada item na enum.

O item da enum	O que ele significa
Idle	A abelha não está fazendo nada
FlyingToFlower (Voando para flor.)	A abelha está voando para uma flor.
GatheringNectar (Recolhendo néctar.)	A abelha está recolhendo néctar de uma flor.
ReturningToHive (Retornando para a colméia.)	A abelha está indo de volta na direção da colméia.
MakingHoney (Fazendo mel.)	A abelha está fazendo mel.
Retired (Aposentada.)	A abelha pendurou suas seis chuteiras.

Solução do Exercício

Abelhas podem fazer muitas coisas. Abaixo mostramos uma lista. Crie uma nova enum que `Bee` usará chamada `BeeState` (estado da abelha). Você também pode criar uma propriedade automática chamada `CurrentState` (estado atual) para cada abelha para manter registro do estado de cada uma delas. Deixe o estado inicial das abelhas em idle (desocupada) e no método `Go()`, adicione um comando switch com uma opção para cada item na enum.

```
public enum BeeState {
    Idle,
    FlyingToFlower,
    GatheringNectar,
    ReturningToHive,
    MakingHoney,
    Retired
  }
public class Bee {
    // declaração de constantes
    // declaração de variáveis
  public BeeState CurrentState { get; private set; }
public Bee(int ID, Point InitialLocation) {
  this.ID = ID;
  Age = 0;
  location = InitialLocation;
  InsideHive = true;
  CurrentState = BeeState.Idle;
  destinationFlower = null;
  NectarCollected = 0;
}
```

Aqui está a enum com todos os diferentes estados.

Precisamos também de uma variável para manter registro do estado de cada abelha.

Você lembrou-se de colocar using `System.Drawinf;` no topo do arquivo de classe (pois ela usa `Point`)?

Preenchemos alguns dos estados. Tudo bem se você não conseguiu chegar até este código, mas vá em frente e inclua-o no projeto.

```
public void Go(Random random) {
    Age++;
    switch (CurrentState) {
      case BeeState.Idle:
        if (Age > CareerSpan) {
          CurrentState = BeeState.Retired;
        } else {
          // o que nós fazemos quando estamos desocupados ?
        }
        break;
      case BeeState.FlyingToFlower:
        // vão no sentido de uma flor !
        break;
      case BeeState.GatheringNectar:
        double nectar = destinationFlower.HarvestNectar();
        if (nectar > 0)
          NectarCollected += nectar;
        else
          CurrentState = BeeState.ReturningToHive;
        break;
      case BeeState.ReturningToHive:
        if (!InsideHive) {
          // avançar no sentido da colmeia !
        } else {
          // o  que fazemos quando estamos no interior da colmeia?
        } break;
```

Eis o comando switch para lidar com o estado de cada abelha.

Se a idade atinge o tempo de vida da abelha, ela aposenta-se. Mas ela primeiro termina a tarefa atual.

Vamos inserir código aqui mais tarde.

Aqui, coletamos néctar da flor onde estamos trabalhando...

...e se ainda existe néctar sobrando, adicione ao que já coletamos...

...mas se não há mais, volte para a colméia.

Retornar para a colméia verifica se já não estamos lá.

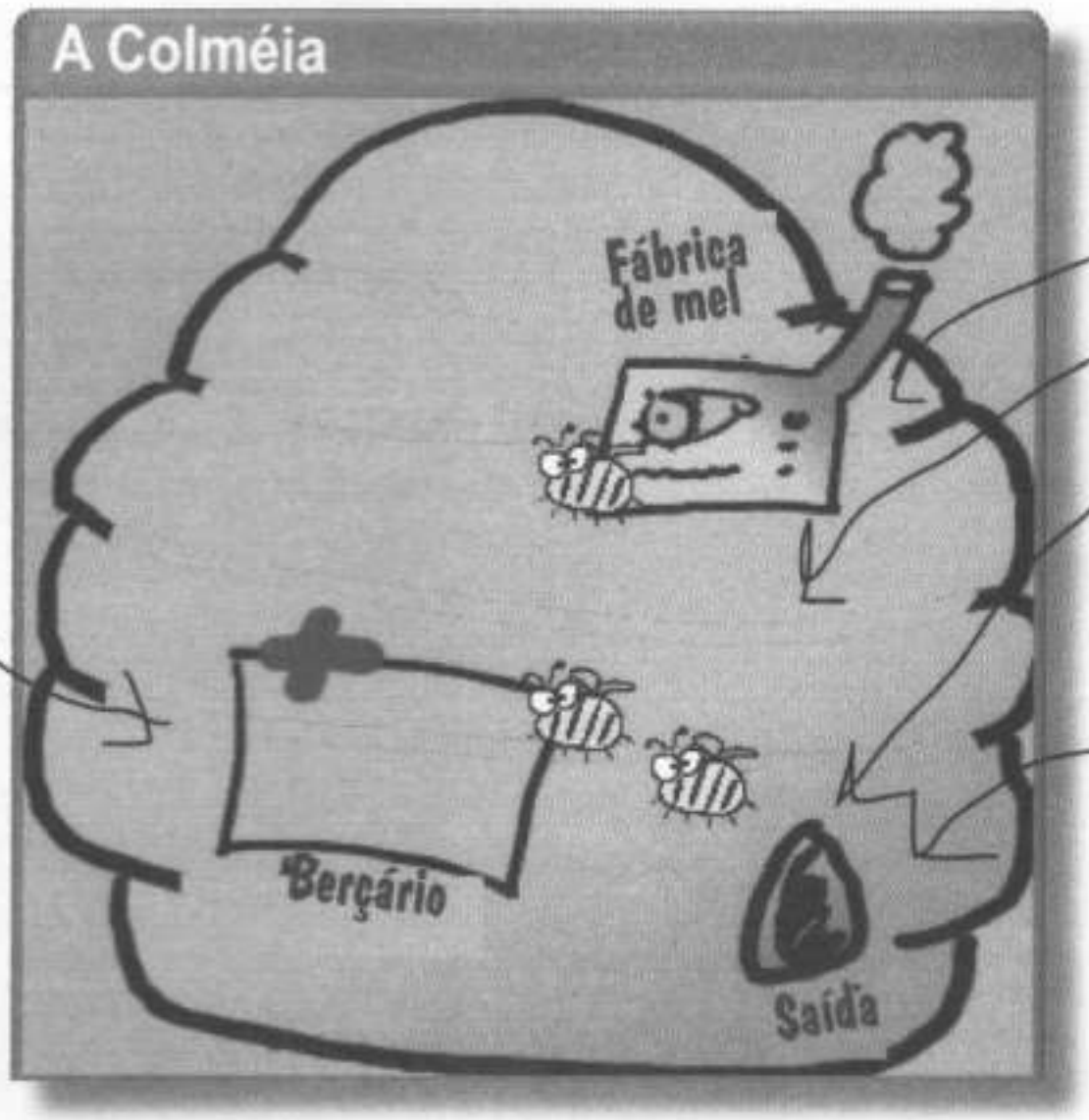
Você precisará ter todos esses estados funcionando.

```
      case BeeState.MakingHoney:
        if (NectarCollected < 0.5) {
          NectarCollected = 0;
          CurrentState = BeeState.Idle;
        } else {
          // uma vez que temos um enxame, vamos tornar o néctar em mel
        }
        break;
      case BeeState.Retired:
        // Não fazer nada! Nós somos aposentados!
        break;
    }
  }
}
```

A abelha adiciona meia unidade de néctar à fábrica de mel de cada vez. Se não existe néctar suficiente para adicionar, a fábrica não pode usá-lo e a abelha apenas o joga fora.

P.A.S.T. (Programadores em Prol das Abelhas Sem Teto)

Temos abelhas e flores cheias de néctar. Precisamos escrever código para que elas possam coletar. Mas, antes disso, para começar, onde as abelhas são criadas? E para aonde elas levam todo o néctar? É para isso que serve a classe Hive.
A colméia não é apenas um lugar para as abelhas voltarem. Ela tem diversos locais no seu interior, todos com posições diferentes no mundo. Existe a entrada e a saída, bem como um berçário para trazer mais abelhas ao mundo e uma fábrica para transformar néctar em mel.



Novas abelhas são criadas e iniciam-se no berçário da colméia.

Cada local é distinto e as abelhas podem ir de um para outro exatamente como fazem com uma flor.

As abelhas voltam pela entrada e deixam a colméia pela saída. E tudo muito organizado.

A colméia funciona à base de mel

Outra tarefa importante que a colméia realiza é controlar quanto mel está armazenado. É preciso usar mel para a colméia continuar funcionando e se novas abelhas precisarem ser criadas, isso exige mel também. Além disso, a fábrica deve empregar o néctar coletado pelas abelhas e transformá-lo em mel. Para cada unidade de néctar que entra, 0,25 unidades de mel podem ser criadas.

Pense a respeito disso um momento... conforme o tempo passa, a colméia usa mel para funcionar e para criar novas abelhas. Enquanto isso, outras abelhas estão trazendo néctar, que é transformando em mel, o que faz com que as coisas continuem funcionando.

Cabe a você (com alguma ajuda) modelar todo esse comportamento no simulador da colméia.

Exercício

Cabe a você escrever o código da Hive.

Hive
Honey: double locations: Dictionary<string, Point> beeCount: int
InitializeLocations() AddHoney(Nectar: double): bool ConsumeHoney(amount: double): bool AddBee(random: Random) Go(random: Random) getLocation(location: string): Point

1 **Crie o esqueleto da Hive**
Como fizemos com a classe `Flower`, você deve começar com um esqueleto básico para a `Hive`. O diagrama de classe é mostrado ao lado. Faça `Honey` (mel) uma propriedade automática apenas-leitura e locations privada e `beeCount` só é usada internamente, então pode ser um campo privado.

2 **Defina as constantes para a colméia**
Precisaremos de constantes para o número inicial de abelhas (6), a quantidade de mel com que a colméia começa (3,2), a quantidade máxima de mel que pode ser armazenada (15), a proporção entre a quantidade de néctar usada e a de mel produzida (0,25), o número máximo de abelhas (8) e o mínimo de mel para a colméia criar uma nova abelha (4).

Você deverá pensar em bons nomes para cada uma delas, bem como nos seus tipos. Para os tipos, não pense somente nos valores iniciais, mas também nos valores possíveis para as variáveis com as quais essas constantes serão usadas. Um double é uma boa escolha para usar com variáveis double, um inteiro com inteiros, etc.

3 **Escreva o código para fazer Locations funcionar**
Primeiro, escreva o método `GetLocation()`. Ele deve receber uma string. Procure por essa string no dicionário Locations e retorne ao ponto associado. Se não existir, lance uma ArgumentException. Depois escreva o método `InitializeLocations()`, que deve criar as seguintes posições na colméia:

- Entrada, em (600, 100)
- Berçário, em (95, 174)
- Fábrica de Mel, em (157, 98)
- Saída, em (194, 213)

Cada uma dessas posições é mapeada para uma posição num espaço 2D ocupado por nossa colméia. Mais tarde, deveremos certificar-nos de que o simulador faça com que a colméia preencha todos esses pontos.

Nessa simulação, estamos lidando com apenas uma única colméia, com pontos fixos. Se você tivesse muitas delas, deveria fazer com que os pontos fossem relativos à colmeia, não ao espaço do mundo.

4 **Crie o construtor da colméia**
Quando uma colméia é criada, ela deve atribuir sua quantidade de mel inicial. Deve criar as posições dentro da colméia e também uma nova instância de `Random`. Então `AddBee()` (adicionar abelha) deve ser chamado - e passa-se essa instância de `Random` - uma vez para cada abelha na colméia no início da simulação.

AddBee() precisa de um objeto Random porque ele adiciona um valor aleatório à posição do berçário - assim as abelhas não começam sobrepondo-se.

Solução do Exercício

Sua tarefa era iniciar o desenvolvimento da classe Hive.

Garanta que você adicionou "usando systemDrawing".

Você pode nomear suas constantes de diversas formas. Sem problemas, desde que seja consistente no restante do código.

```
public class Hive {
  private const int InitialBees = 6;
  private const double InitialHoney = 3.2;
  private const double MaximumHoney = 15.0;
  private const double NectarHoneyRatio = .25;
  private const double MinimumHoneyForCreatingBees = 4.0;
  private const int MaximumBees = 8;
  private Dictionary<string, Point> locations;
  private int beeCount = 0;
  public double Honey { get; private set; }
  private void InitializeLocations() {
    locations = new Dictionary<string, Point>();
    locations.Add("Entrance", new Point(600, 100));
    locations.Add("Nursery", new Point(95, 174));
    locations.Add("HoneyFactory", new Point(157, 98));
    locations.Add("Exit", new Point(194, 213));
  }
  public Point GetLocation(string location) {
    if (locations.Keys.Contains(location))
      return locations[location];
    else
      throw new ArgumentException("Unknown location: " + location);
  }
  public Hive() {
    Honey = InitialHoney;
    InitializeLocations();
    Random random = new Random();
    for (int i = 0; i < InitialBees; i++)
      AddBee(random);
  }
  public bool AddHoney(double Nectar) { return true; }
  public bool ConsumeHoney(double Amount) { return true; }
  private void AddBee(Random random) { }
  public void Go(Random random) { }
}
```

Nós deixamos MaximumHoney como double, pois este tipo pode ir do valor inicial - InitialHoney (3,2) - até esse valor. Já que esta última deverá ser double, é melhor fazer a primeira ser do mesmo tipo.

Lembra-se dos dicionários? O nosso armazena posições e as chaves são strings.

Não se esqueça de criar uma nova instância do dicionário, ou isso não funcionará.

O restante do método é bastante simples.

Ele impede outras classes de trabalhar com nosso dicionário de posições, o que poderia modificar alguma coisa indevidamente. É um exemplo de encapsulamento.

Você deve chamar AddBee() uma vez para cada abelha que inicia com a colméia.

Não temos o código para estes aqui ainda, mas você deve criar métodos vazios somente para deixar a classe completa.

Não é uma forma estranha de desenvolver código? Nossas abelhas não sabem nada sobre flores ainda e nossa colméia está cheia de declarações de métodos vazios. Nada funciona ainda, certo?

Código de verdade é feito passo a passo

Seria ótimo se pudéssemos escrever o código de uma classe inteira de uma vez só, compilá-lo, testá-lo, colocá-lo à parte e **então** continuar com a próxima classe. Infelizmente, isso é quase sempre impossível.

É mais comum escrever código exatamente como estamos fazendo neste capítulo: passo a passo. Pudemos fazer quase toda a classe `Flower`, mas no que se refere à abelha, ainda temos algum trabalho a fazer (a maior parte é dizer o que fazer em cada estado).

E agora, com a `colméia`, temos muitos métodos vazios para completar. Além disso, não associamos as `abelhas` à `colméia`. E ainda temos o problema de como chamar o método `Go()` em todos esses objetos milhares de vezes...

Mas nós não começamos completando as classes! Primeiro pensamos na estrutura e **depois** começamos a desenvolver.

Primeiro você projeta, depois desenvolve

Começamos o projeto sabendo exatamente o objetivo: um simulador de colméia. E sabemos muito sobre abelhas, flores, colméia e o mundo onde todas trabalham juntas. É por isso que começamos pela **estrutura**, que nos diz como as classes deveriam trabalhar umas com as outras. Depois podemos concentrar-nos em cada classe, projetando-as individualmente.

Projetos sempre correm melhor se você tiver uma boa idéia do que está criando **antes** de começar a desenvolver. Isso parece óbvio, senso comum. Mas faz toda a diferença no produto final.

Preenchendo a classe Hive

Vamos voltar à classe da `colméia` e preencher alguns dos métodos vazios:

```
public class Hive {
  // constant declarations
  // variable declarations
  // InitializeLocations()
  // GetLocation()
  // Hive constructor
  public bool AddHoney(double nectar) {
    double honeyToAdd = nectar * NectarHoneyRatio;
    if (honeyToAdd + Honey > MaximumHoney)
        return false;
    Honey += honeyToAdd;
    return true;
  }
  public bool ConsumeHoney(double amount) {
    if (amount > Honey)
      return false;
    else {
      Honey -= amount;
      return true;
    }
  }
  private void AddBee(Random random) {
    beeCount++;
    int r1 = random.Next(100) - 50;
    int r2 = random.Next(100) - 50;
    Point startPoint = new Point(locations["Nursery"].X + r1,
                                 locations["Nursery"].Y + r2);
    Bee newBee = new Bee(beeCount, startPoint);
    // Once we have a system, we need to add this bee to the system
  }
  public void Go(Random random) { }
}
```

Primeiro, determinamos quanto mel pode resultar de converter uma quantidade de néctar...

...e então vemos se há espaço na colméia para armazenar essa quantidade.

Se houver, adicionamos o mel à colméia.

Esse método recebe uma quantidade de mel e tenta consumir a quantidade da reserva da colméia.

Se não tivermos mel suficiente na colméia para as exigências do método, retornamos false.

Se houver, removemos da reserva e retornamos true.

Isso cria um ponto dentro de 50 unidades tanto na direção X quanto na Y da posição do berçário.

Adicione uma nova abelha, na posição indicada.

Completaremos AddBee() e preencheremos Go() um pouco mais em breve...

Esse aqui é privado... apenas instâncias de Hive podem criar abelhas.

O método Go() da colméia

Já escrevemos um método `Go()` para as `flores` e um para as abelhas (mesmo que ele ainda deva ser completado). Eis aqui o método Go() para a `colméia`:

```
public void Go(Random random) {
  if (Honey > MinimumHoneyForCreatingBees)
    AddBee(random);
}
```

A única restrição (pelo menos por enquanto) é que a colméia precisa ter mel suficiente para criar mais abelhas.

A mesma instância de Random passada para Go() é enviada ao método AddBee().

Infelizmente, isso não é muito realista. Muitas vezes, numa colméia atarefada, uma rainha não tem tempo de criar mais abelhas. Não temos uma classe QueenBee (abelha rainha), mas vamos assumir que quando existe mel suficiente para criar abelhas, há 10% de chance de que uma seja criada. Podemos modelar isso da seguinte forma:

```
public void Go(Random random) {
  if (Honey > MinimumHoneyForCreatingBees
    && random.Next(10) == 1) {
    AddBee(random);
  }
}
```

Essa é uma forma fácil de simular uma chance em dez de uma abelha ser criada. Gera-se um número aleatório entre 0 e 9. Se ele for 1, cria-se a abelha.

Um motivo para deixar isso de fora é você poder salvar a semente (seed) dos números aleatórios - assim você pode executar novamente uma determinada simulação... se quiser fazer isso no futuro!

não existem Perguntas Idiotas

P: Então a colméia pode criar um número infinito de abelhas?

R: Nesse momento, sim - ou, pelo menos, o limite é bem grande. Mas você está certo, isso não é muito realista. Mais tarde, voltaremos a isso e adicionaremos uma restrição para que somente uma quantidade de abelhas existam por vez no nosso mundo simulado.

P: Não poderíamos associar essa instância de Random como uma propriedade da classe, em vez de passá-la para AddBee()?

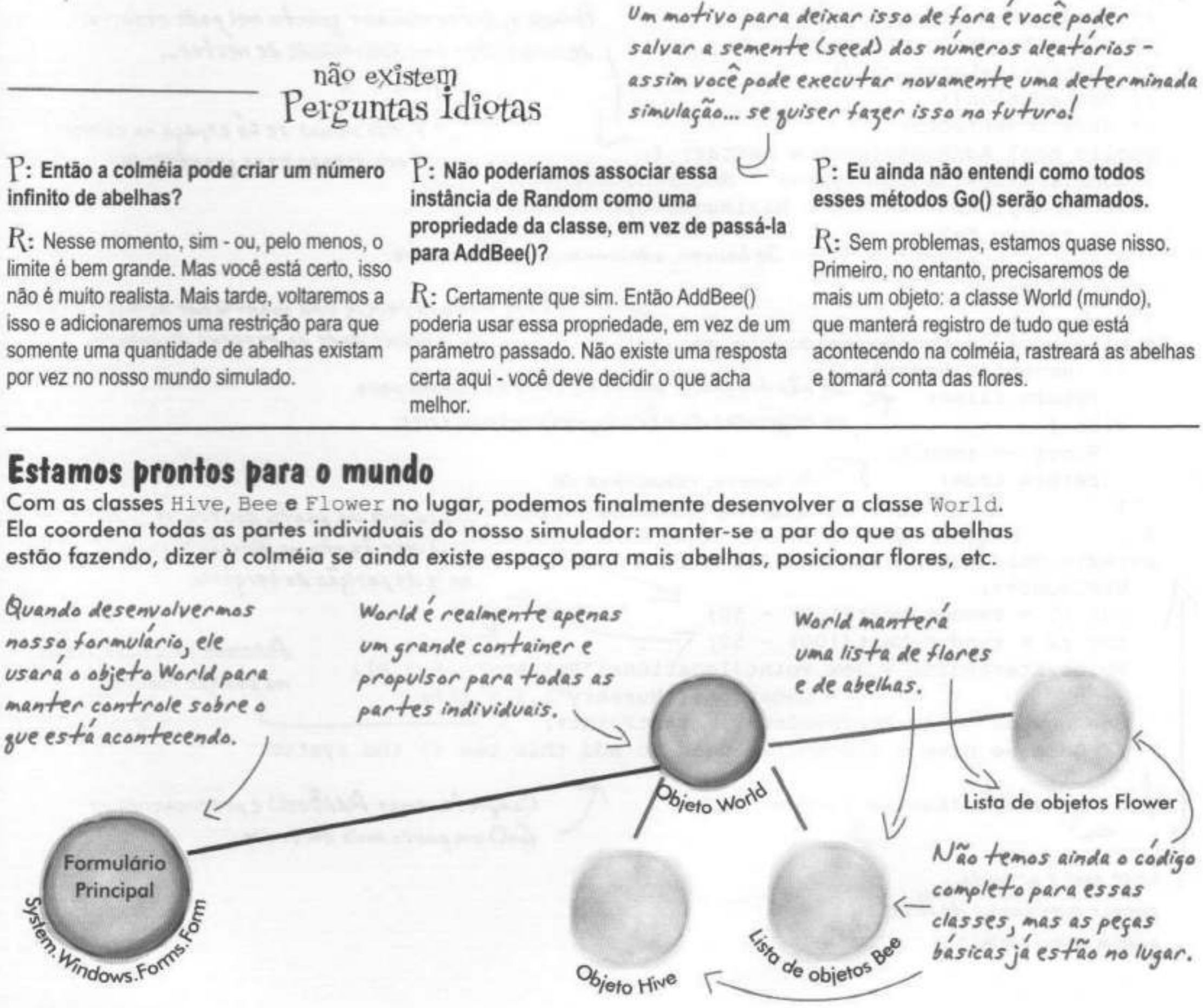
R: Certamente que sim. Então AddBee() poderia usar essa propriedade, em vez de um parâmetro passado. Não existe uma resposta certa aqui - você deve decidir o que acha melhor.

P: Eu ainda não entendi como todos esses métodos Go() serão chamados.

R: Sem problemas, estamos quase nisso. Primeiro, no entanto, precisaremos de mais um objeto: a classe World (mundo), que manterá registro de tudo que está acontecendo na colméia, rastreará as abelhas e tomará conta das flores.

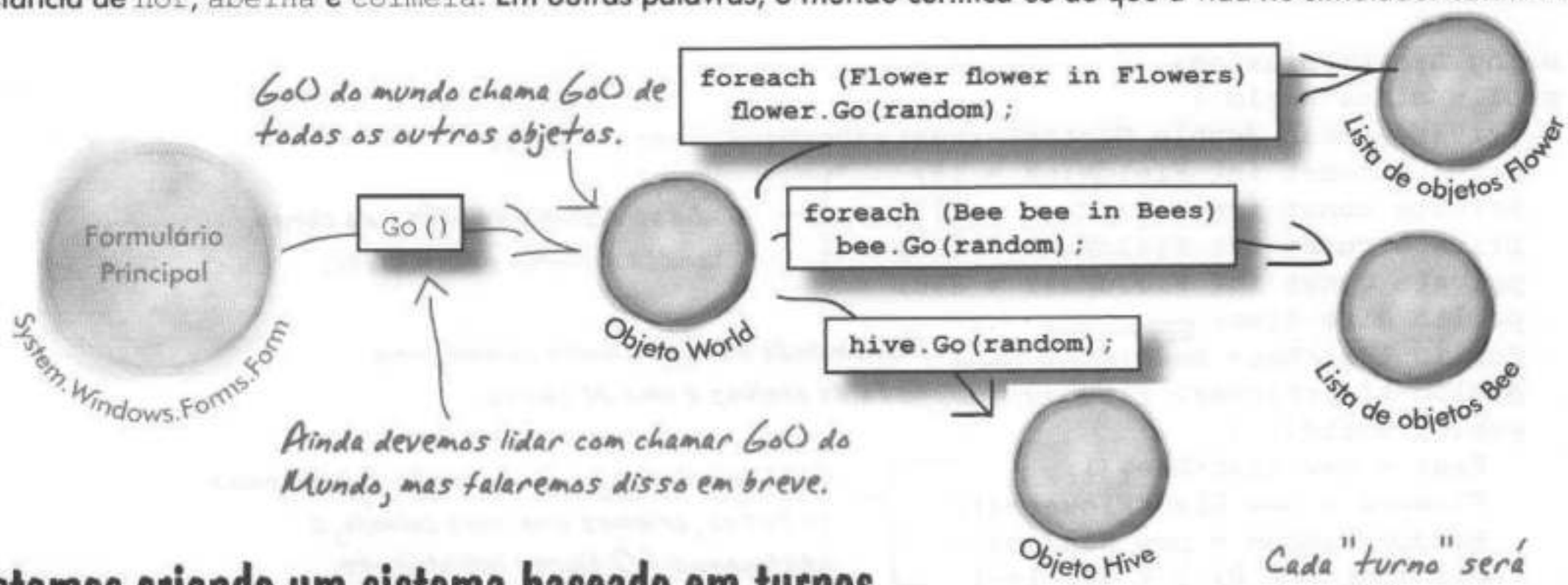
Estamos prontos para o mundo

Com as classes `Hive`, `Bee` e `Flower` no lugar, podemos finalmente desenvolver a classe `World`. Ela coordena todas as partes individuais do nosso simulador: manter-se a par do que as abelhas estão fazendo, dizer à colméia se ainda existe espaço para mais abelhas, posicionar flores, etc.



O objeto World faz com que tudo funcione

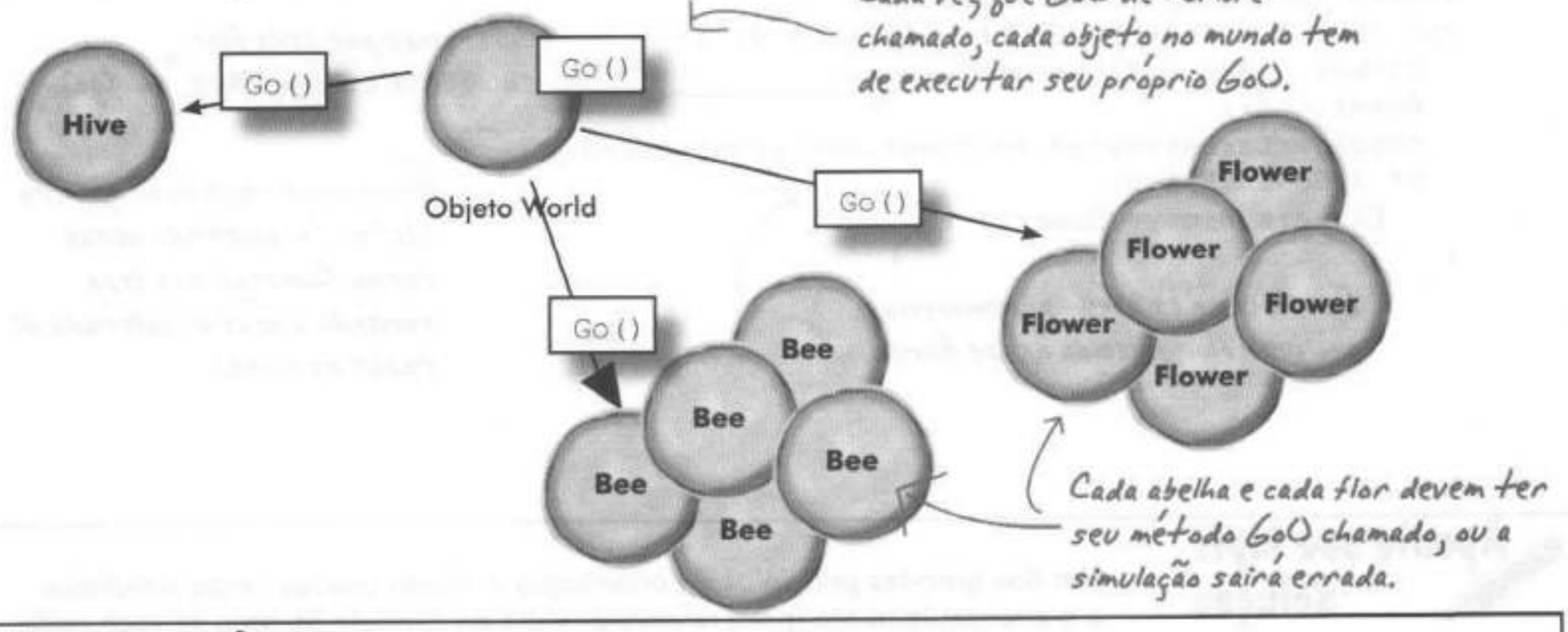
Uma das nossas maiores tarefas para o objeto `World` é, para cada turno do simulador, chamar `Go()` de cada instância de `flor`, `abelha` e `colméia`. Em outras palavras, o mundo certifica-se de que a vida no simulador continue.



Estamos criando um sistema <u>baseado em turnos</u>

Nossos métodos `Go()` de cada objeto devem executar cada um seu turno, ou ciclo, da simulação. Um turno, nesse caso, significa apenas uma quantidade arbitrária de tempo... por exemplo, um turno pode acontecer a cada 10 segundos, 60, 10 minutos, etc. O principal: um turno afeta todos os objetos no mundo. A colméia envelhece um turno e verifica se precisa adicionar novas abelhas. Cada abelha tem o seu turno, movendo-se uma distância pequena até seu destino ou fazendo alguma pequena ação e envelhecendo também. Cada flor tem seu turno, fazendo um pouco de néctar e envelhecendo. E é isso que `World` faz: ele certifica-se de que, quando seu método `Go()` for chamado, cada objeto no mundo tenha seu turno para agir.

Cada "turno" será desenhado como um único quadro de animação, assim o mundo só precisa mudar um pouquinho a cada turno.



Aponte seu lápis

Um dos grandes princípios da orientação a objeto usados nesse simulador é o encapsulamento (para relembrar volte ao capítulo 5). Veja se você pode examinar o código desenvolvido até aqui e encontrar dois exemplos para cada classe desenvolvida.

Hive

1. .....
.....
2. .....
.....

Bee

1. .....
.....
2. .....
.....

Flower

1. .....
.....
2. .....
.....

Eis o código para o Mundo

A classe World é uma das mais simples no nosso simulador. Aqui está o código:

```
using System.Drawing;
public class World {
  private const double NectarHarvestedPerNewFlower = 50.0;
  private const int FieldMinX = 15;
  private const int FieldMinY = 177;
  private const int FieldMaxX = 690;
  private const int FieldMaxY = 290;
  public Hive Hive;
  public List<Bee> Bees;
  public List<Flower> Flowers;
  public World() {
    Bees = new List<Bee>();
    Flowers = new List<Flower>();
    Random random = new Random();
    for (int i = 0; i < 10; i++)
      AddFlower(random);
  }
  public void Go(Random random) {
    Hive.Go(random);
    for (int i = Bees.Count - 1; i >= 0; i--) {
      Bee bee = Bees[i];
      bee.Go(random);
      if (bee.CurrentState == BeeState.Retired)
        Bees.Remove(bee);
    }
    double totalNectarHarvested = 0;
    for (int i = Flowers.Count - 1; i >= 0; i--) {
      Flower flower = Flowers[i];
      flower.Go();
      totalNectarHarvested += flower.NectarHarvested;
      if (!flower.Alive)
        Flowers.Remove(flower);
    }
```

Isso define os limites do campo, onde as flores podem viver.

Cada mundo terá uma única colméia, uma lista de abelhas e uma de flores.

Quando criamos um novo mundo, inicializamos as listas, criamos uma nova colméia, e adicionamos 10 flores inicialmente.

Isso é fácil... apenas dizemos à colméia para ir (go), passando-lhe uma instância de Random.

Iteramos por cada abelha existente e dizemo-lhes "vá" (go).

Se uma abelha está aposentada, devemos retirá-la do mundo.

Iteramos por cada flor existente e dizemo-lhes "vá" (go).

Precisamos registrar quanto néctar foi coletado nesse turno. Conseguimos isso somando o néctar coletado de todas as flores.

Assim como as abelhas, removemos qualquer flor falecida nesse turno.

Aponte seu lápis Solução

Um dos grandes princípios da orientação a objeto usados nesse simulador é o encapsulamento (para relembrar volte ao capítulo 5). Veja se você pode examinar o código desenvolvido até aqui e encontrar dois exemplos para cada classe desenvolvida.

Hive

1. *O dicionário Locations da colméia é privado*
2. *Ela providencia um método para adicionar mel às abelhas*

Bee

1. *A posição da abelha é apenas-leitura*
2. *Sua idade também. Assim outras classes não podem alterá-las.*

Flower

1. *A flor tem um método para recolher néctar*
2. *E mantém seu booleano alive como privado*

```
    if (totalNectarHarvested > NectarHarvestedPerNewFlower) {
      foreach (Flower flower in Flowers)
        flower.NectarHarvested = 0;
      AddFlower(random);
    }
  }
  private void AddFlower(Random random)
  {
    Point location = new Point(random.Next(FieldMinX, FieldMaxX),
                               random.Next(FieldMinY, FieldMaxY));
    Flower newFlower = new Flower(location, random);
    Flowers.Add(newFlower);
  }
}
```

Abelhas polinizam flores conforme recolhem néctar. Uma vez que recolheram uma quantidade de néctar, polinizaram o suficiente para que o mundo adicione uma nova flor.

Isto é, se existir néctar suficiente no campo o mundo acrescenta uma nova flor.

Isso serve para conseguir uma posição aleatória no campo...

...para adicionar uma nova flor.

Perguntas Idiotas

P: Porque vocês não usaram um laço foreach para remover flores mortas e abelhas aposentadas?

R: Porque você não pode remover itens de uma coleção dentro de um laço foreach que itera por ela. Se você fizer isso, o .NET lançará uma exceção.

P: Certo, então porque cada um desses laços começa no fim da lista e conta de trás para frente até zero?

R: Porque em cada laço precisamos preservar a numeração dos elementos da lista. Digamos que você começasse no inicio de uma lista de cinco abelhas e seu laço num certo momento encontrasse uma flor numa posição intermediária que estivesse morta. Se ele remover a flor no índice 3, por exemplo, a lista terá agora somente quatro flores, e uma nova flor entra no lugar da outra e recebe o índice 3 - e essa flor será pulada, porque da próxima vez que iterarmos examinaremos a flor no índice 4.
Se o laço começar pelo fim, a flor que se mover para um espaço vazio sempre já terá sido examinada, logo não há chance de pular flores.

Exercício

Com nossas quatro classes principais no lugar, temos algum trabalho a fazer para amarrar as pontas. Siga os passos abaixo e você deverá ter classes Bee, Hive, Flower e World funcionando. Mas cuidado: você deverá alterar vários pontos em quase todas as classes antes de terminar.

1. **Atualiza Bee para receber uma referência de uma colméia e do mundo.**
   Agora que temos uma classe para a `colméia` e uma para o `mundo`, os objetos `Bee` precisam saber sobre eles. Atualize seu código para receber uma referência para a colméia e o mundo no construtor e salvar essas referências para uso futuro.

2. **Atualize a colméia para receber uma referência de World.**
   Assim como as abelhas precisam saber sobre sua `colméia`, a `colméia` precisa saber a respeito do `mundo`. Atualize `Hive` para receber uma referência de `World` no seu construtor, e salve essa referência. Você também deve atualizar o código na `colméia` que cria novas abelhas para passar-lhes uma referência da própria `colméia` e do `mundo`.

3. **Atualize World para passar a si mesmo para uma nova colméia.**
   Atualize a classe `World` para que quando ele criar uma nova `colméia`, passe uma referência de si mesmo.

PARE! Nesse ponto, você deve ser capaz de compilar todo o código. Se não for o caso, examine-o e corrija qualquer erro antes de prosseguir.

4. **Coloque um limite superior para o número de abelhas que a colméia pode criar**
   A classe Hive tem uma constante MaximumBees que determina quantas abelhas ela pode sustentar (contando dentro e fora da colméia). Agora que a colméia tem acesso ao mundo, você deve ser capaz de implementar essa restrição.

5. **Quando a colméia criar abelhas, informe o mundo.**
   A classe World mantém registro das abelhas que existem. Quando a colméia cria uma nova abelha, certifique-se de que o objeto Bee seja adicionado à lista geral que o mundo mantém.

Dica: examine o código próximo de onde você cria e adiciona abelhas. Existem dois lugares no código relacionados a isso ocorrendo na colméia, então tome cuidado.

não existem Perguntas Idiotas

P: Por que vocês lançam uma exceção no método GetLocation() da classe Hive?

R: Para lidar com os dados passados como parâmetros. A colméia tem alguns locais definidos, mas o parâmetro de GetLocation() é uma string qualquer. O que aconteceria se existisse um erro no programa que fizesse com que uma seqüência de caracteres inválida (como uma string vazia, ou o nome de um local inexistente no dicionário) fosse passada? O que o método deveria retornar?
Quando temos um parâmetro inválido e não é muito claro o que fazer com ele, é sempre uma boa idéia lançar uma ArgumentException. Eis aqui como o método GetLocation() faz isso:

```
throw new ArgumentException(
      "Unknown location: " + location);
```

Esse comando faz a colméia lançar uma ArgumentException com a mensagem "Unknown Location" (local desconhecido) com o texto não encontrado.
Isto é útil porque alertar imediatamente se um parâmetro com um local inválido é passado ao método. E ao incluir esse parâmetro na mensagem da exceção, você está dando-se uma informação importante que o ajudará a corrigir o problema.

P: Por que armazenar todos os locais num objeto Point se não estamos desenhando nada?

R: Toda abelha tem uma posição, quer você a desenhe na tela ou não. A função do objeto Bee é manter controle sobre o local da abelha no mundo. Cada vez que seu método Go() é chamado, ela precisa se mover um pouquinho mais para perto de seu destino. Agora, mesmo que não estejamos desenhando uma figura da abelha até esse momento, ainda precisamos manter registro do local dentro da colméia ou campo onde ela está para determinarmos se ela atingiu seu destino.

P: Então por que usar Point para armazenar a posição, e não alguma outra forma? Os `Points` não são específicos para desenho?

R: Sim, um Point é o que todos os controles visuais usam nas suas propriedades Location. No entanto, só porque o .NET emprega-o assim, não quer dizer que ele também não seja útil para manter registro de posições. Sim, poderíamos ter criado a nossa própria classe `BeeLocation` com campos inteiros chamados X e Y. Mas não existe nenhum motivo para reinventar a roda quando o C# nos fornece um **Point** de bandeja!

É quase sempre mais fácil reorientar ou estender uma classe que já exista e faça QUASE TUDO o que você quer do que criar uma nova do zero.

Solução do Exercício

Com nossas quatro classes principais no lugar, temos algum trabalho a fazer para amarrar as pontas. Siga os passos abaixo e você deverá ter classes Bee, Hive, Flower e World funcionando. Eis como fizemos as alterações para conseguir isso.

1 Atualiza Bee para receber uma referência de uma colméia e do mundo.
Agora que temos uma classe para a `colméia` e uma para o `mundo`, os objetos `Bee` precisam saber sobre eles. Atualize seu código para receber uma referência para a colméia e o mundo no construtor e salvar essas referências para uso futuro.

```
public class Bee {
   // existing constant declarations
   // existing variable declarations
   private World world;
   private Hive hive;
   public Bee(int ID, Point InitialLocation, World world, Hive hive)
{
      // existing code
      this.world = world;
      this.hive = hive;
   }
}
```

Isso é bem simples... receba esses valores e atribua-os às variáveis locais.

2 **Atualize a colméia para receber uma referência de World.**
Assim como as abelhas precisam saber sobre sua `colméia`, a `colméia` precisa saber a respeito do `mundo`. Atualize `Hive` para receber uma referência de `World` no seu construtor e salve-a. Você também deve atualizar o código na `colméia` que cria novas abelhas para passar a elas uma referência da própria `colméia` e do `mundo`.

```
public class Hive {
  private World world;
  public Hive(World world) {
    this.world = world;
    // existing code
  }
  public void AddBee(Random random) {
    // other bee creation code
    Bee newBee = new Bee(beeCount, startPoint, world, this);
  }
}
```

Mais código bem básico... pegue a referência, mantenha uma cópia local.

Você vai querer armazenar World ANTES porque o restante do construtor precisa utilizá-lo.

Agora novas abelhas precisam de uma referência ao mundo e à colméia.

Se você tiver problemas para fazer as coisas funcionarem, baixe o código deste exercício (e de todos os outros, aliás) de **www.altabooks.com.br**

3 **Atualize World para passar a si mesmo para uma nova colméia.**
Atualize a classe `World` para quando ele criar uma nova `colméia`, passar uma referência de si mesmo.

```
public World() {
  Bees = new List<Bee>();
  Flowers = new List<Flower>();
  Hive = new Hive(this);
  Random random = new Random();
    for (int i = 0; i < 10; i++)
      AddFlower(random);
}
```

Aqui se passa a referência da colméia.

4 **Coloque um limite superior para o número de abelhas que a colméia pode criar**
A classe Hive tem uma constante MaximumBees que determina quantas abelhas ela pode sustentar (dentro e fora da colméia). Agora que a colméia tem acesso ao mundo, você deve ser capaz de implementar essa restrição.

```
public void Go(Random random) {
  if (world.Bees.Count < MaximumBees
      && Honey > MinimumHoneyForCreatingBees
      && random.Next(10) == 1) {
    AddBee(random);
  }
}
```

Podemos usar o objeto World para ver quantas abelhas existem e comparar isso ao máximo para a colméia.

Colocamos a comparação primeiro. Se não houver mais espaço para criar abelhas, não faz sentido verificar se existe mel suficiente para isso.

5 **Quando a colméia criar abelhas, informe o mundo.**
A classe World mantém registro das abelhas. Quando a colméia criar uma nova abelha, certifique-se de que o objeto Bee seja adicionado à lista geral mantida pelo mundo.

```
private void AddBee(Random random) {
  beeCount++;
  // Calculate the starting point
  Point startPoint = // start the near the nursery
  Bee newBee = new Bee(beeCount, startPoint, world, this);
  world.Bees.Add(newBee);
}
```

Adicionamos a nova abelha à lista geral do mundo.

Isso demonstra um dos motivos que torna uma referência do mundo necessária na classe Hive.

Dê comportamento para as abelhas

Grande parte do código faltando nas nossas classes é a que completa o método `Go()` de `Bee`. Conseguimos desenvolver alguns dos estados antes, mas ainda temos vários por fazer (Idle está incompleto, `FlyingToFlower` e parte do `MakingHoney`). Vamos finalizar esses estados agora:

```
public void Go(Random random) {
  Age++;
  switch (CurrentState) {
    case BeeState.Idle:
      if (Age > CareerSpan)
        CurrentState = BeeState.Retired;
      else if (world.Flowers.Count > 0
          && hive.ConsumeHoney(HoneyConsumed)) {
        Flower flower =
          world.Flowers[random.Next(world.Flowers.Count)];
        if (flower.Nectar >= MinimumFlowerNectar && flower.Alive) {
          destinationFlower = flower;
          CurrentState = BeeState.FlyingToFlower;
        }
      }
      break;
    case BeeState.FlyingToFlower:
      if (!world.Flowers.Contains(destinationFlower))
        CurrentState = BeeState.ReturningToHive;
      else if (InsideHive) {
        if (MoveTowardsLocation(hive.GetLocation("Exit"))) {
          InsideHive = false;
          location = hive.GetLocation("Entrance");
        }
      }
      break;
    else
      if (MoveTowardsLocation(destinationFlower.Location))
        CurrentState = BeeState.GatheringNectar;
      break;
    case BeeState.GatheringNectar:
      double nectar = destinationFlower.HarvestNectar();
      if (nectar > 0)
        NectarCollected += nectar;
      else
        CurrentState = BeeState.ReturningToHive;
      break;
    case BeeState.ReturningToHive:
      if (!InsideHive) {
        if (MoveTowardsLocation(hive.GetLocation("Entrance"))) {
          InsideHive = true;
          location = hive.GetLocation("Exit");
        }
      }
```

Se estivermos sem fazer nada, vamos querer encontrar outra flor para colher néctar.

Veja se ainda existem flores e então consuma mel suficiente para a viagem. Se algo estiver errado, não podemos agir.

Precisamos de outra flor viva e com néctar.

Assumindo que tudo funcionou até agora, vá para a nova flor.

Verificamos se a flor não morreu enquanto nos dirigíamos até ela.

É por isso que passamos uma referência da colméia para o construtor de Bee.

Se pudermos chegar à saída, estaremos fora da colméia. Atualizamos a posição. Já que agora estamos no formulário do campo, devemos estar voando perto da entrada.

Esta é a saída. Quando a colméia armazena a posição "Exit" (saída), corresponde ao ponto no formulário Colméia que mostra a figura de uma saída.

Essa é a entrada. Quando as abelhas voam de volta para a colméia, elas vão em direção a esta entrada para a colméia no formulário do campo.

É por isso que o dicionário de posições tem dois pares de posições "Exit" e "Entrance" (entrada) diferentes.

Se estamos fora da colméia e a flor está viva, vá até ela para começar a coletar néctar.

Se chegamos à colméia, atualize nossa posição e assuma o status insidehive (dentro da colméia).

```
  else
    if (MoveTowardsLocation(hive.GetLocation("HoneyFactory")))
        CurrentState = BeeState.MakingHoney;
  break;
case BeeState.MakingHoney:
  if (NectarCollected < 0.5) {
    NectarCollected = 0;
    CurrentState = BeeState.Idle;
  }
  else
    if (hive.AddHoney(0.5))
        NectarCollected -= 0.5;
    else
        NectarCollected = 0;
  break;
case BeeState.Retired:
  // Do nothing! We're retired!
  break;
}
}
```

Se já estamos na colméia, vamos em direção à fábrica de mel.

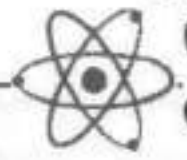
Tente dar seu néctar para a colméia.

Se a colméia pode usar o néctar para fazer mel...

...remova-o da abelha.

Se a colméia está cheia, AddHoney() retornará false e, nesse caso, a abelha joga fora o restante de néctar para poder voar para outra missão.

Uma vez que uma abelha se aposenta, ela fica parada até que a colméia a remova da lista. Então ela viaja para Miami!



O Poder do Cérebro

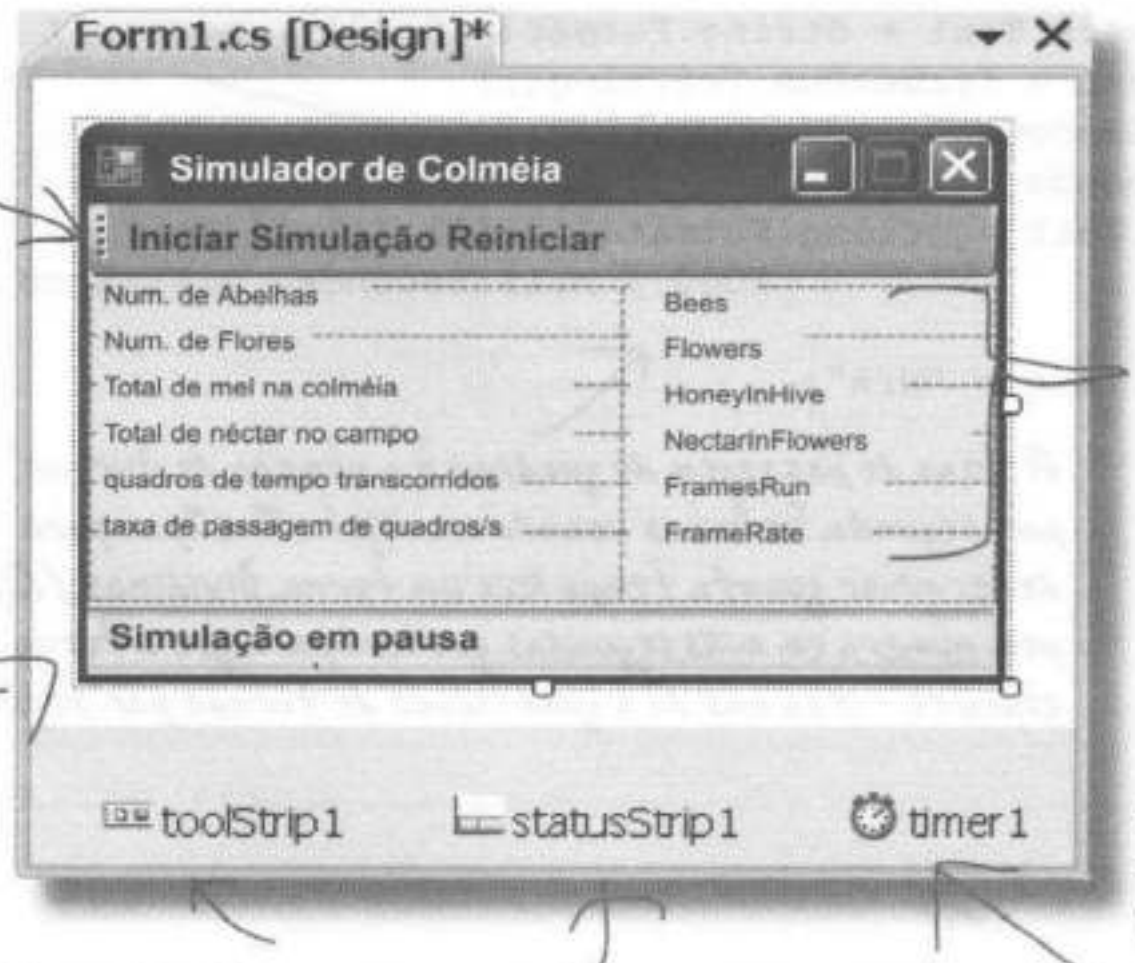
Suponha que você queira mudar o simulador para que leve dois turnos para chegar numa flor e dois para ir de volta para a colméia. Sem escrever código nenhum, quais **métodos** de quais classes você deveria mudar para colocar esse novo comportamento em funcionamento?

O formulário principal diz ao mundo para funcionar

Certo, você sabe que o mundo avança um turno a cada vez que seu método `Go()` é chamado. Mas o que chama esse método `Go()`? Óbvio, o formulário principal! É hora de desenvolvê-lo.
Vá em frente e adicione um novo formulário ao seu projeto. Faça com que ele pareça com o formulário abaixo. Estamos usando alguns controles novos, mas vamos explicar todos eles nas próximas páginas.

O controle ToolStrip coloca uma faixa de ferramentas no topo do seu formulário. Você pode adicionar os dois botões usando a caixa de combinação que aparecerá no formulário. Atribua a propriedade DisplayStyle de cada botão para Text.

Adicione uma StatusStrip para colocar uma barra de status no rodapé. Utilize a caixa de combinação para adicionar um StatusLabel.

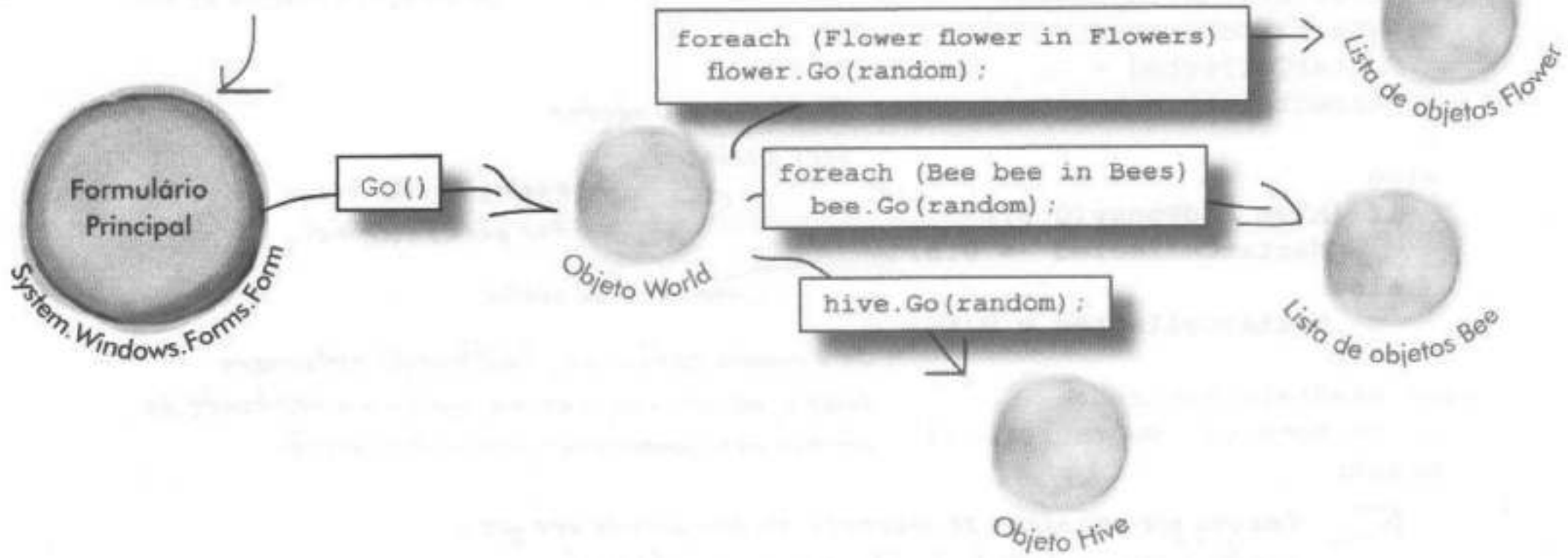


Cada um desses rótulos existe numa célula de um controle TableLayoutPanel. Você dimensiona-o exatamente como faria com uma tabela no Microsoft Word. Clique na pequena seta escura para adicionar, remover e redimensionar colunas e linhas.

Adicione um controle Timer ao formulário. Ele nunca fica visível - é um componente não visual que o designer mostra como um ícone no espaço abaixo do formulário.

O controle ToolStrip adiciona uma barra de ferramentas no topo do seu formulário e o StatusStrip uma barra de status no rodapé. Mas eles também aparecem como ícones abaixo do formulário para que você possa editar suas propriedades.

Estamos finalmente chegando ao código que faz o objeto World funcionar.



Podemos usar o mundo para conseguir estatísticas

Agora queremos que todos esses controles funcionem. Mas não precisamos de tratadores de clique para todos eles. Em vez disso, vamos usar um único método que atualizará todas as diversas estatísticas na janela do simulador:

Isso indica quanto tempo se passou num turno... devemos conseguir esse parâmetro em algum lugar, mas veremos isso daqui a algumas páginas.

```
private void UpdateStats(TimeSpan frameDuration) {
  Bees.Text = world.Bees.Count.ToString();
  Flowers.Text = world.Flowers.Count.ToString();
  HoneyInHive.Text = String.Format("{0:f3}", world.Hive.Honey);
  double nectar = 0;
  foreach (Flower flower in world.Flowers)
    nectar += flower.Nectar;
  NectarInFlowers.Text = String.Format("{0:f3}", nectar);
  FramesRun.Text = framesRun.ToString();
  double milliSeconds = frameDuration.TotalMilliseconds;
  if (milliSeconds != 0.0)
    FrameRate.Text = string.Format("{0:f0} ({1:f1}ms)",
                                   1000 / milliSeconds, milliSeconds);
  else
    FrameRate.Text = "N/A";
}
```

A maior parte disso envolve conseguir dados do mundo e atualizar os rótulos.

Certifique-se de usar os nomes corretos dos controles no código.

Imprima o primeiro parâmetro como um número sem casas decimais, depois um espaço e o segundo parâmetro com uma casa decimal seguido das letras "ms" (entre parênteses).

Adicione este método em Form1.

A taxa de passagem de quadros é o número de turnos por segundo. Estamos usando um objeto TimeSpan para determinar quanto tempo leva um turno. Dividimos 1000 pelo número em milissegundos que leva para que o turno execute - isso nos dá a quantidade de turnos por segundo.

Falaremos mais sobre isso quando criarmos o objeto TimeSpan.

Opa! De onde veio o objeto World? Ainda não o criamos, não? E o que é essa coisa toda de tempo e quadros?

Esse código usa o mesmo método String.Format() usado no dump hexadecimal. Mas, em vez de imprimir usando "x2", você usa "f3" para mostrar o número com três casas decimais.

Vamos criar um mundo

Você está certo, precisamos criar um objeto `World`. Adicione essa linha ao construtor de seu formulário.

```
public Form1() {
  InitializeComponent();
  world = new World();
}
```

Vá em frente e adicione um campo privado do tipo `World` ao seu formulário, chamado `world`.
Isso deixa-nos apenas com todo o código relacionado com o tempo. Dizemos desde o início que precisaríamos de uma forma de executar `Go()` no `mundo` de novo e de novo... parece que precisamos de algum tipo de timer.

Timers disparam eventos repetidamente

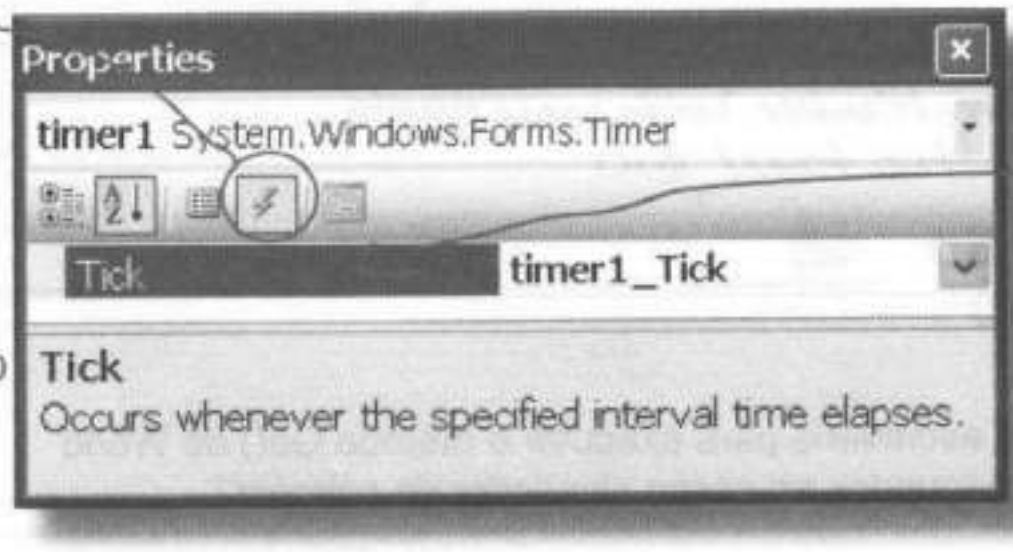
Lembra-se de como você usou um laço para animar os cães? Bom, existe uma forma melhor de fazer isso. Um **Timer**, que é a palavra em inglês para temporizador, é um componente especialmente útil que dispara um evento repetidamente até mil vezes por segundo.

Pare um minuto e crie um novo projeto para ver como os timers funcionam. Então voltaremos para o simulador e colocaremos esse novo conhecimento em uso.

1. **Crie um novo projeto com um timer e três botões**
Você não precisa fechar seu projeto atual - apenas abra uma nova cópia do Visual Studio e nele, outro projeto. Arraste um timer e três botões para o formulário. Clique no ícone do timer na parte inferior da janela do designer e atribua 1000 para a sua propriedade Interval. Esse número é medido em milissegundos - ele diz ao timer para disparar seu evento tick uma vez por segundo, nesse caso.

2. **Abra a janela de propriedades do IDE e clique no botão para eventos**
(Lembre-se, o botão parece com um raio e permite que você gerencie os eventos de qualquer um dos controles do seu formulário.) O controle timer tem apenas um único evento, Tick. Clique no ícone do **timer** no designer e então **duas vezes na linha** na página de eventos; o IDE criará um novo método tratador de evento para você e irá associá-lo à propriedade automaticamente.

O botão de Eventos na janela de propriedades permite trabalhar com todos os eventos de cada um dos seus controles.



O controle Timer tem um evento Tick. Se você clicar aqui duas vezes, o IDE cria um método tratador de evento automaticamente.

A parte de baixo da janela tem a descrição do evento.

3 **Adicione código para o evento Tick e em seus botões**
Eis aqui um código que vai ajudá-lo a ter uma noção de como o timer funciona:

Esse botões permitem-lhe brincar com a propriedade Enabled (ativo) e com os métodos Start() e Stop(). O primeiro botão troca Enabled de true para false e os outros dois chamam os métodos, respectivamente.

```
private void timer1_Tick(object sender, EventArgs e) {
  Console.WriteLine(DateTime.Now.ToString());
}
private void toggleEnabled_Click(object sender, EventArgs e) {
  if (timer1.Enabled)
    timer1.Enabled = false;
  else
    timer1.Enabled = true;
}
private void startTimer_Click(object sender, EventArgs e){
  timer1.Start();
  Console.WriteLine("Enabled = " + timer1.Enabled);
}
private void stopTimer_Click(object sender, EventArgs e) {
  timer1.Stop();
  Console.WriteLine("Enabled = " + timer1.Enabled);
}
```

Esse comando escreve a data e a hora atual na saída. Verifique a janela Output para ter certeza de que o evento tick é disparado uma vez a cada segundo (uma vez a cada 1000 milissegundos).

A propriedade Enabled do Timer inicia e para a contagem.

O método Start() do timer inicia a contagem e atribui true para Enabled. O método Stop() para a contagem e atribui false para Enabled.

O timer está usando um delegado nos bastidores

Como o C# e o .NET dizem ao timer o que fazer a cada ciclo? Como o método `timer1_Tick()` é executado a cada vez que o timer conta um determinado intervalo? Bom, estamos de volta aos **eventos** e **delegados**, exatamente o que discutimos no capítulo anterior. Use o "Go To Definition" (vá para a definição) do IDE para lembrar como o delegado EventHandler funciona:

O evento Tick do timer é um evento comum tratado por um tratador normal, exatamente como aqueles que lidam com os cliques de botão.

4 **Clique com o botão direito na variável timer1 e selecione "Go To Definition"**
O "Go To Definition" fará com que o IDE automaticamente salte para o local no código onde a variável timer1 é definida. O IDE parará no código que gerou para criar timer1 como uma propriedade do objeto Fomr1 em Form1.Designer.cs. Suba pelo arquivo até achar esta linha:

```
this.timer1.Tick += new System.EventHandler(this.timer1_Tick);
```

Este é o evento Tick do seu controle timer. Você estabeleceu que ele deveria ocorrer a cada 1000 milissegundos.

Eis aqui um dos delegados em System: o tratador de eventos básico. É um delegado... um ponteiro para um ou mais métodos.

Aqui está o método que você acabou de escrever, timer1_Tick(). Você está dizendo ao delegado para apontar para este método.

5 **Agora clique com o botão direito em EventHandler e selecione "Go To Definition"**
O IDE pulará automaticamente para o código que define EventHandler. Dê uma olhada no nome da nova aba que se abre para exibir o código: "EventHandler [from metadata]". Ou seja, o código que define EventHandler não é o seu código. Ele é nativo do Framework .NET e o IDE gerou uma linha "falsa" de código para mostrar-lhe como ele é representado:

```
public delegate void EventHandler(object sender, EventArgs e);
```

Os eventos são do tipo EventHandler. Então nosso evento Tick agora aponta para o método timer1_Tick().

Eis aqui porque todo evento no C# geralmente recebe como parâmetros um Object e um EventArgs - é essa é a forma do delegado que o C# define para tratamento de eventos.

Poder do Cérebro

Que código você escreveria para executar o método Go() de World a cada 500 milissegundos no nosso simulador de colméia?

Adicione um Timer ao simulador

Vamos adicionar um timer ao simulador. Você já tem um controle timer, provavelmente chamado `timer1`. Em vez de usar o IDE para gerar um método `timer1_Tick()`, no entanto, vamos associar o timer a um método tratador de eventos chamado RunFram() (executar quadro) manualmente.

DateTime e TimeSpan
O .NET usa a classe DateTime (data e hora) para armazenar informações sobre um momento no tempo e sua propriedade Now (agora) retorna a data e hora atual. Se você quiser encontrar uma diferença entre duas datas, use um objeto TimeSpan (lapso de tempo): apenas subtraia um objeto DateTime de outro e isso retornará um objeto TimeSpan que representará a diferença entre elas.

TimeSpan tem propriedades como Days (dias), Hours (horas), Seconds (segundos) e Milliseconds que permitem medir o lapso de tempo usando várias unidades.

```
public partial class Form1 : Form {
  World world;
  private Random random = new Random();
  private DateTime start = DateTime.Now;
  private DateTime end;
  private int framesRun = 0;
  public Form1() {
    InitializeComponent();
    world = new World();
    timer1.Interval = 50;
    timer1.Tick += new EventHandler(RunFrame);
    timer1.Enabled = false;
    UpdateStats(new TimeSpan());
  }
  private void UpdateStats(TimeSpan frameDuration) {
    // Code from earlier to update the statistics
  }
  public void RunFrame(object sender, EventArgs e) {
    framesRun++;
    world.Go(random);
    end = DateTime.Now;
    TimeSpan frameDuration = end - start;
    start = end;
    UpdateStats(frameDuration);
  }
}
```

Você já deve ter criado uma propriedade World antes.

Estes serão usados para determinar há quanto tempo o simulador está executando até um determinado momento.

Queremos manter registro a respeito de quantos quadros - ou turnos - passaram-se.

Executa a cada 50 milissegundos.

Estabeleceremos como tratador um método personalizado, RunFrame().

O Timer inicia desativado.

Também iniciamos atualizando as estatísticas, com um novo TimeSpan (com zero como lapso de tempo).

Como um segundo tem 1000 milissegundos, nosso timer disparará 200 vezes por segundo.

Aumente a contagem de quadros e execute o método Go() do mundo.

Depois, determinamos o tempo decorrido desde a execução do último quadro.

Finalmente, atualize as estatísticas novamente, com o novo lapso de tempo.

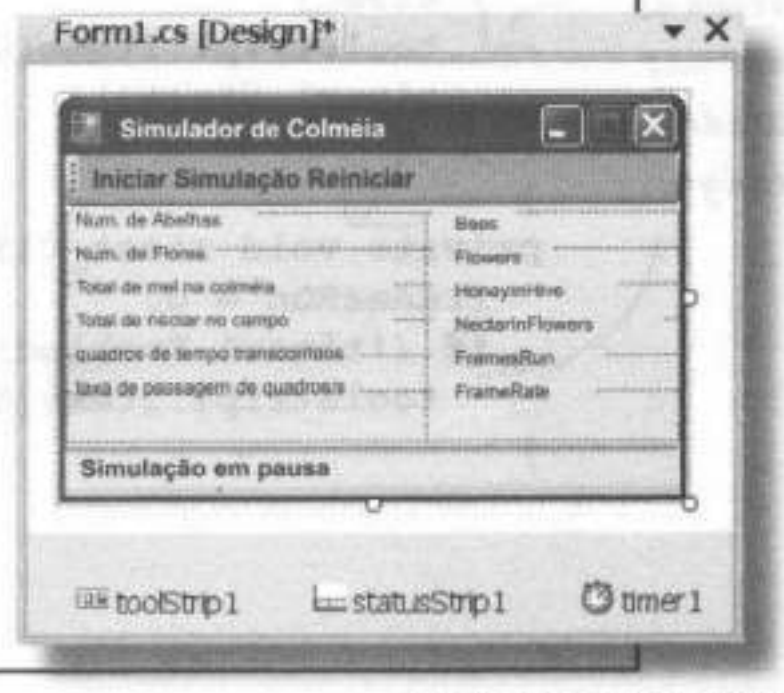
Exercício

Sua tarefa é escrever os tratadores de evento para os botões **startSimulation** (iniciar simulação) e **reset** (reiniciar) na ToolStrip (faixa de ferramentas). Eis aqui o que cada botão deve fazer:

Se você ainda não arrastou uma ToolStrip e uma StatusStrip da caixa de ferramentas até o formulário, faça agora.

1. Inicialmente, o primeiro botão deve apresentar o texto "Iniciar simulação". Pressioná-lo deverá fazer a simulação iniciar e o texto deve mudar para "Pausar simulação". Se a simulação estiver em pausa, o botão deve ter o texto "Recomeçar simulação".

2. O segundo botão deve ter o texto "Reiniciar". Quando ele for pressionado, o mundo deve ser recriado. Se o timer estiver em pausa, o texto do primeiro botão deve mudar de "Recomeçar simulação" para "Iniciar simulação".



Não existe uma única resposta para essa pergunta - queremos apenas que você pense a respeito do que ainda falta fazer.

Aponte seu lápis

Para você, o que ainda falta fazer nessa fase do simulador? Tente executar o programa. Liste tudo o que ainda deve ser implementado antes de ir para a parte gráfica do código.

.....

.....

.....

.....

.....

não existem
Perguntas Idiotas

P: Usamos o termo "turno", mas estamos agora falando de quadros. Qual a diferença?

R: Semântica, apenas. Ainda estamos lidando com turnos: pequenos lapsos de tempo em que cada objeto do mundo age. Mas já que em breve colocaremos alguns gráficos intensivos para funcionar, começamos a usar o termo "quadros", porque ele e "taxa de quadros" (frame rate) são usados em jogos gráficos.

Solução do Exercício

Sua tarefa foi escrever os tratadores de evento para os botões de Iniciar simulação e Reiniciar:

```
public partial class Form1 : Form {
  // variable declarations
  public Form1() {
    InitializeComponent();
    world = new World();
  }
  private void Form1_Load(object sender, tArgs e) {
    // code to start simulator
  }
  private void UpdateStats(TimeSpan frameDuration) {
    // Code from earlier to update the statistics
  }
  public void RunFrame(object sender, EventArgs e) {
    // event handler for timer
  }
  private void startSimulation_Click(object sender, EventArgs e) {
    if (timer1.Enabled) {
      toolStrip1.Items[0].Text = "Resume simulation";
      timer1.Stop();
    } else {
      toolStrip1.Items[0].Text = "Pause simulation";
      timer1.Start();
    }
  }
  private void reset_Click(object sender, EventArgs e) {
    framesRun = 0;
    if (!timer1.Enabled)
      toolStrip1.Items[0].Text = "Start simulation";
  }
}
```

Certifique-se de que os nomes dos controles em seu formulário sejam os mesmos usados no código.

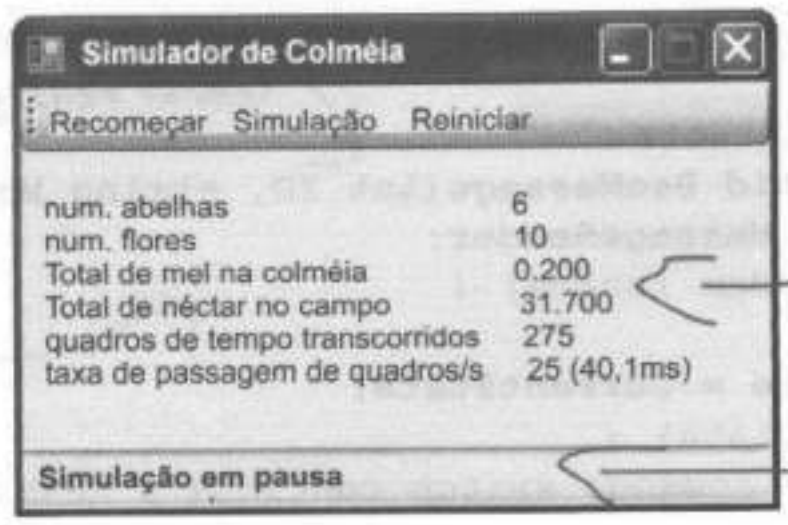
Ativa/desativa o timer e atualiza a mensagem.

Reiniciar o simulador é feito apenas recriando a instância de World e reiniciando framesRun (quadros executados).

O único momento em que precisamos mudar o rótulo do primeiro botão é quando ele diz Recomeçar simulação. Se ele diz Pausar simulação, isso não precisa ser alterado.

Teste Preliminar

Você já fez muita coisa. Compile seu código, corrija qualquer erro de digitação e execute o simulador. Como ele se comporta?



Exercício

Eis aqui a sua chance de juntar tudo o que aprendeu até agora. Precisamos permitir que as abelhas digam ao simulador o que estão fazendo. Quando elas fizerem isso, o simulador deve atualizar as mensagens de estado.

Dessa vez, deixaremos para você não só escrever a maior parte do código, mas também determinar que código precisa ser escrito. Como você pode ter um método no simulador que seja chamado quando uma abelha mudar de estado?

Para ajudar um pouco, escrevemos o método para adicionar ao formulário. A classe Bee deve chamá-lo sempre que seu estado mudar:

* Certo, mais uma dica. Você deverá modificar todas as classes, menos uma, para fazer isso funcionar.

```
private void SendMessage(int ID, string Message) {
  statusStrip1.Items[0].Text = "Bee #" + ID + ": " + Message;
}
```

Solução do Exercício

Sua tarefa era descobrir uma forma de as abelhas informarem ao simulador sobre o que elas estão fazendo.

Eis o que adicionamos na classe Bee.

Usamos um **callback** para associar cada objeto abelha individual ao método SendMessage() (enviar mensagem) do formulário.

```
public class Bee {
  // o codigo ja existente
  public delegate void BeeMessage(int ID, string Message);
  public BeeMessage MessageSender;
  public void Go(Random random) {
    Age++;
    BeeState oldState = CurrentState;
    switch (currentState) {
    // o restante do comando switch continua o mesmo
    }
    if (oldState != CurrentState
      && MessageSender != null)
      MessageSender(ID, CurrentState.ToString());
  }
}
```

BeeMessage (mensagem de abelha) é nosso delegado. Ele equivalente ao método SendMessage() que escrevemos para o formulário.

Se o estado da abelha mudar, chamamos o método que nosso delegado BeeMessage aponta e informamos esse método da alteração de estado.

Aqui estão as mudanças feitas na colméia.

A colméia precisa de um delegado também, assim ela pode passar para as abelhas os métodos que devem chamar quando são criadas em AddBee()

```
public class Hive {
  // o codigo ja existente
  public Bee.BeeMessage MessageSender;
  public Hive(World world, Bee.BeeMessage MessageSender) {
    this.MessageSender = MessageSender;
    // codigo ja existente no construtor
  }
  public void AddBee(Random random) {
    // codigo ja existente em AddBee()
    Bee newBee = new Bee(beeCount, startPoint, world, this);
    newBee.MessageSender += this.MessageSender;
    world.Bees.Add(newBee);
  }
```

AddBee() agora deve certificar-se de que cada abelha nova receba o método para o qual deve apontar.

O mundo precisa de algumas mudanças também.

```
public class World {
  // o codigo ja existente
  public World(Bee.BeeMessage messageSender) {
    Bees = new List<Bee>();
    Flowers = new List<Flower>();
    Hive = new Hive(this, messageSender);
    Random random = new Random();
    for (int i = 0; i < 10; i++)
      AddFlower(random);
  }
}
```

World não precisa de um delegado próprio. Ela apenas passa o método a ser chamado para a instância de Hive.

```
public partial class Form1 : Form {
  // declaracao de variaveis
  public Form1() {
    InitializeComponent();
    world = new World(new Bee.BeeMessage(SendMessage));
    // o restante do construtor de Form1
  }
  private void reset_Click(object sender, EventArgs e) {
    framesRun = 0;
    world = new World(new Bee.BeeMessage(SendMessage));
    if (!timer1.Enabled)
      toolStrip1.Items[0].Text = "Start simulation";
  }
  private void SendMessage(int ID, string Message) {
    statusStrip1.Items[0].Text = "Bee #" + ID + ": " + Message;
  }
}
```

Por último, mas não menos importante, eis o formulário atualizado. Tudo que não está sendo mostrado continua inalterado.

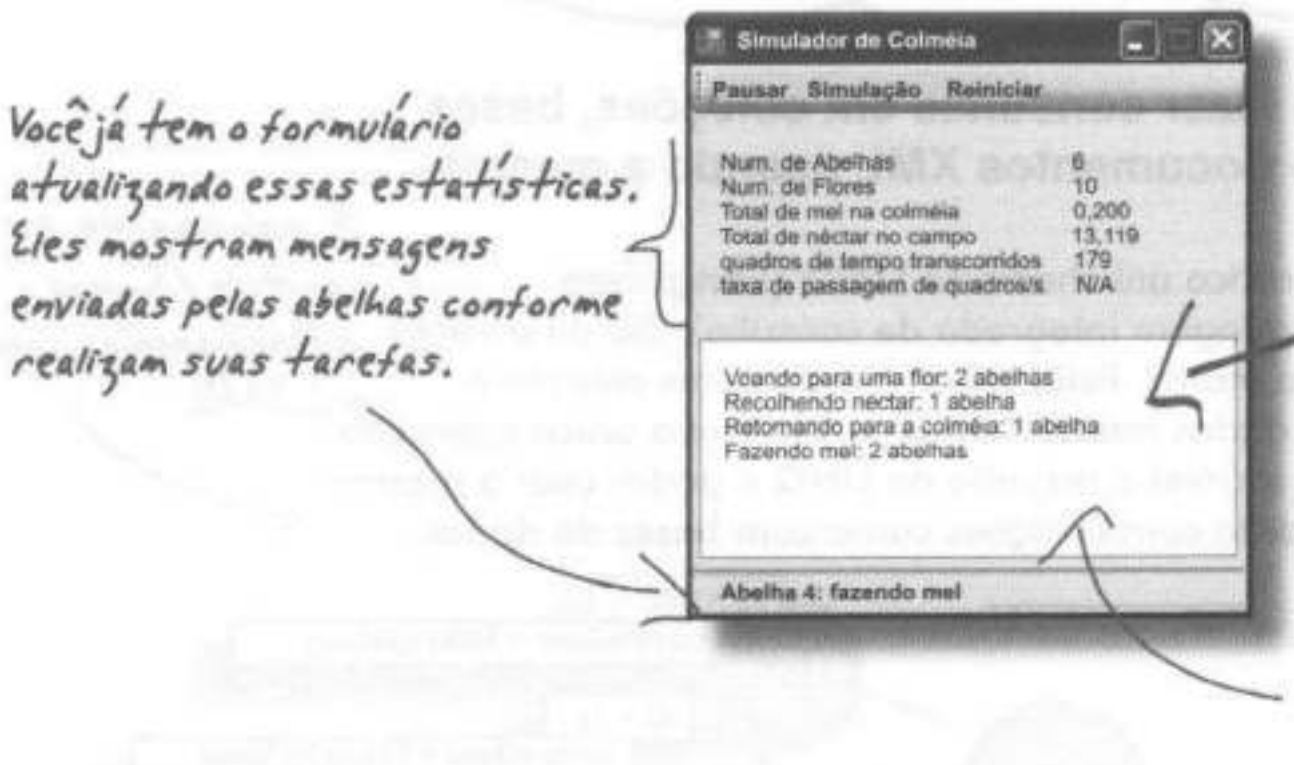
Criamos um novo delegado para a classe Bee (certifique-se de ter declarado BeeMessage como público) e apontamos para nosso método SendMessage().

Mesma coisa aqui... cria o mundo com o método que as abelhas chamarão.

Esse é o método que escrevemos para você... mas certifique-se de tê-lo adicionado, também.

Vamos trabalhar com grupos de abelhas

Suas abelhas devem estar zumbindo por aí em volta da colméia e a simulação deve estar funcionando! Não é legal? Mas como ainda não temos a parte visual do simulador - é o que faremos no próximo capítulo - tudo que temos são as mensagens enviadas pelas abelhas com seus callbacks de volta ao formulário principal. Então vamos adicionar mais alguma coisa a respeito do que as abelhas estão fazendo.



Você já tem o formulário atualizando essas estatísticas. Eles mostram mensagens enviadas pelas abelhas conforme realizam suas tarefas.

Vá em frente e adicione uma caixa de listagem ao formulário. Vamos usá-la para mostrar algumas estatísticas adicionais sobre as abelhas no mundo.

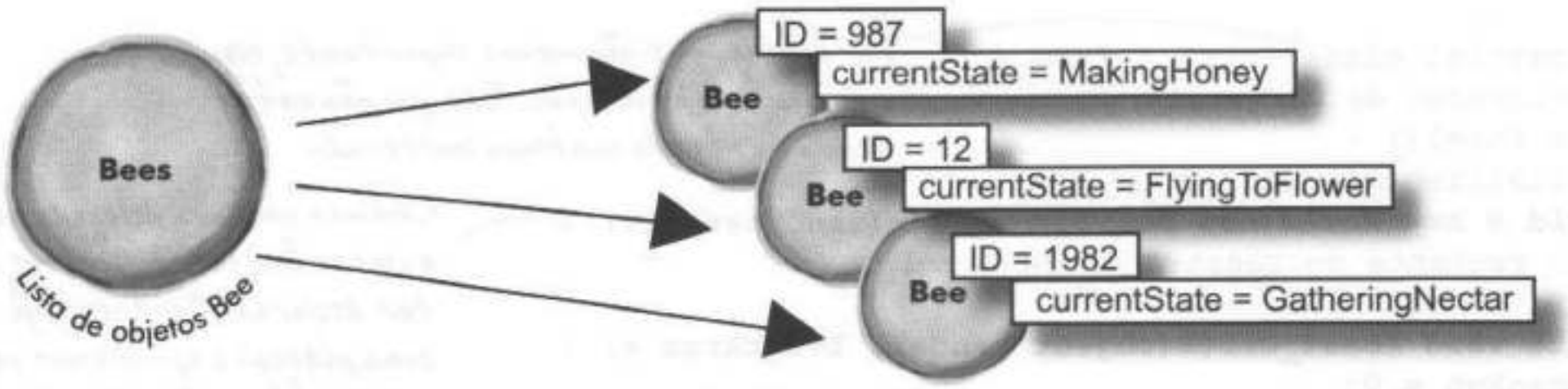
A todo momento há abelhas voando por aí. A nova caixa de listagem mostrará quantas delas estão fazendo quais trabalhos. Nesse caso, duas estão voando para flores (numa destas já está uma abelha recolhendo néctar) e duas estão na fábrica transformando o néctar em mel.

Uma coleção coleciona... DADOS

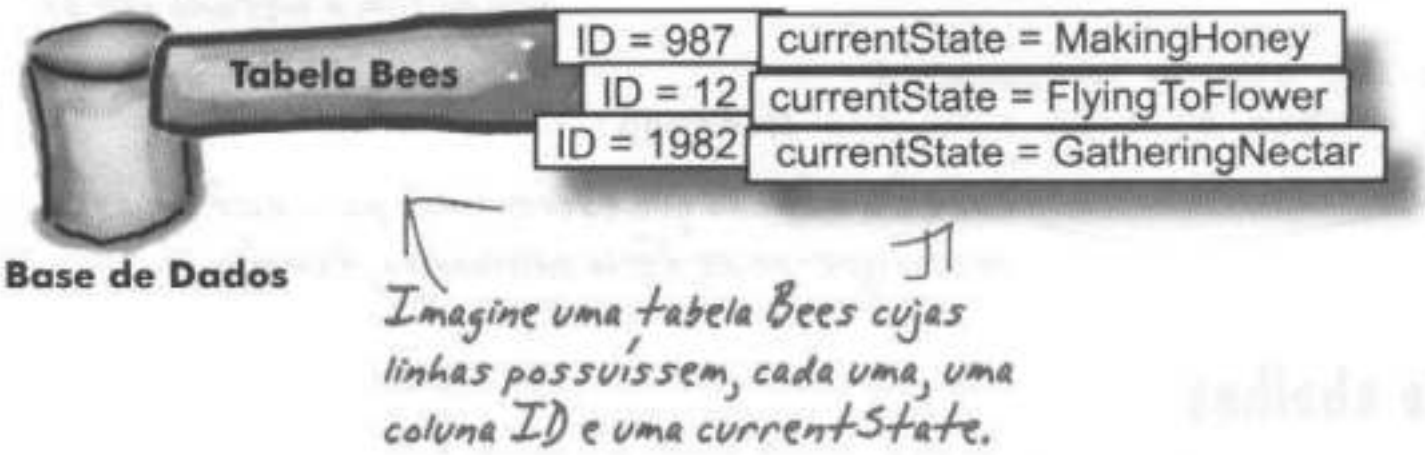
Nossas abelhas estão armazenadas numa List, que é um dos tipos de coleção do C#. E os tipos de coleção servem somente para armazenar dados... como uma base de dados. Assim cada abelha é como uma linha de dados, completa com seu estado, ID e etc. Eia como as abelhas são representadas como uma coleção de objetos:

Poder do Cérebro

Você já sabe o suficiente para recolher as informações necessárias para preencher a caixa de listagem - pare um minuto e pense como isso poderia funcionar. É um pouco mais complexo do que parece. O que você precisa fazer para determinar quantas abelhas estão nos vários estados indicados por Bee.State?



Há muitos dados armazenados nos campos de Bee. Você pode quase pensar numa coleção de objetos como pensa a respeito de linhas numa base de dados. Cada objeto mantém dados em seus campos, como cada linha numa base de dados mantém dados em suas colunas.



Imagine uma tabela Bees cujas linhas possuíssem, cada uma, uma coluna ID e uma currentState.

Muitas coleções, especialmente quando mantém objetos, podem ser encaradas como armazenadoras de dados. exatamente como uma base.

Quem se importa se eu posso pensar numa coleção como se ela fosse uma base de dados se eu não posso usá-la como uma? Que perda de tempo...

E se você pudesse fazer consultas em coleções, bases de dados e mesmo documentos XML usando a mesma sintaxe básica?

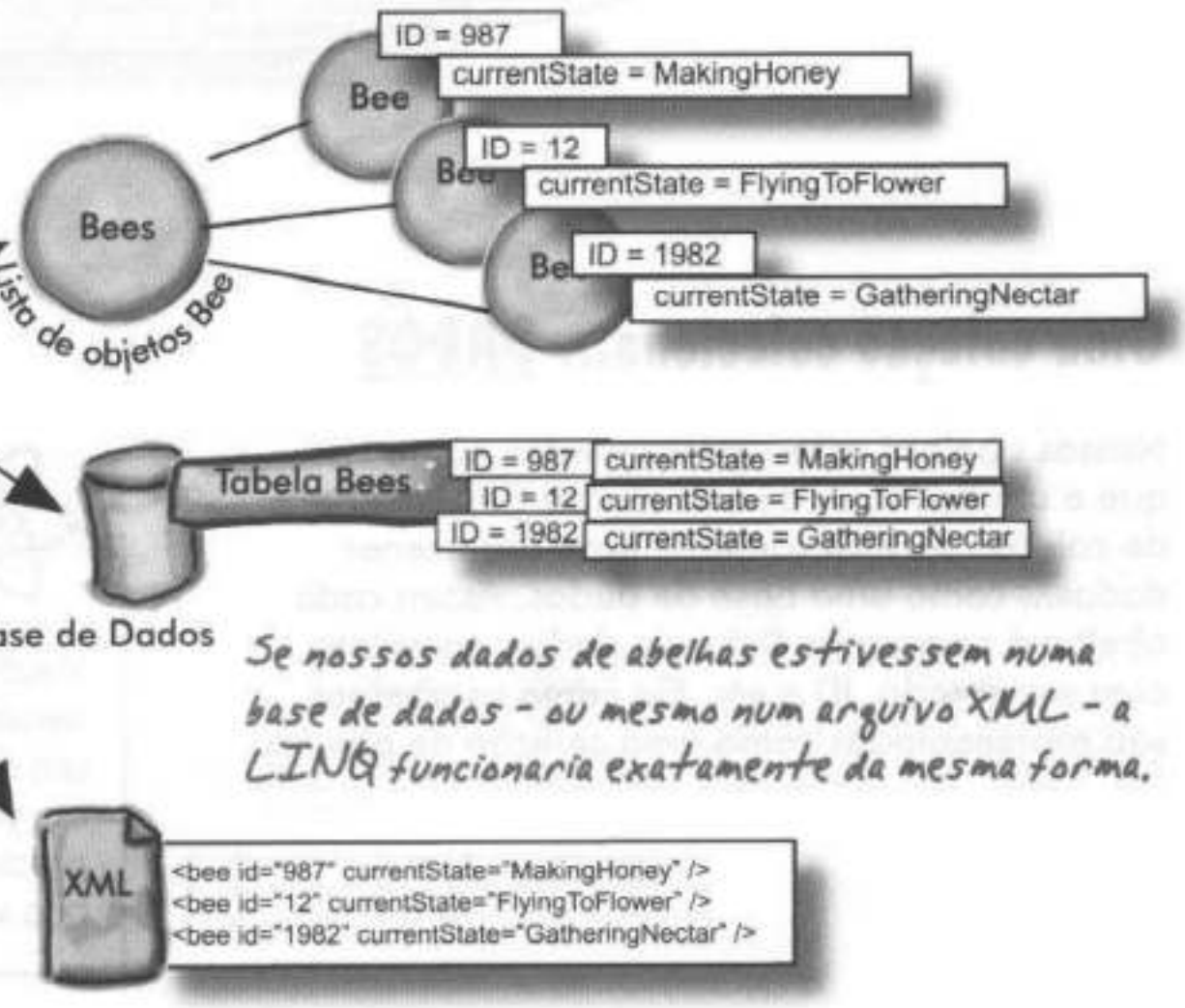
O C# tem uma característica útil chamada **LINQ (Language Integrated Query**, linguagem integrada de consulta). Ela dá a você uma forma de usar uma matriz, lista, pilha, fila ou outra coleção e trabalhar com todos os dados internos de uma vez numa única operação. Mas o que é realmente incrível a respeito da LINQ é poder usar a mesma sintaxe – ela funciona tanto com coleções como com bases de dados.

A maior parte do capítulo 15 nós trabalharemos com a LINQ.

Uma consulta QUERY funciona essencialmente da mesma forma com dados de uma coleção ou numa base de dados.

```
var beeGroups =
  from bee in world.Bees
  group bee by bee.CurrentState
  into beeGroup
  orderby beeGroup.Key
  select beeGroup;
```

LINQ



Se nossos dados de abelhas estivessem numa base de dados - ou mesmo num arquivo XML - a LINQ funcionaria exatamente da mesma forma.

LINQ facilita trabalhar com dados em coleções e bases de dados

Empregaremos um capítulo inteiro para aprender LINQ, em breve. Por agora, podemos usá-la juntamente com algum código pré-fabricado para adicionar algumas características extras ao nosso simulador. Código pré-fabricado é algo que você deve digitar e não importa se você não entendê-lo. Você aprenderá sobre como ele funciona no capítulo 15.

Código pré-fabricado

```
private void SendMessage(int ID, string Message) {
  statusStrip1.Items[0].Text = "Bee #" + ID + ": " + Message;
  var beeGroups =
    from bee in world.Bees
    group bee by bee.CurrentState into beeGroup
    orderby beeGroup.Key
    select beeGroup;
  listBox1.Items.Clear();
  foreach (var group in beeGroups) {
    string s;
    if (group.Count() == 1)
      s = "";
    else
      s = "s";
    listBox1.Items.Add(group.Key.ToString() + ": "
            + group.Count() + " bee" + s);
    if (group.Key == BeeState.Idle
      && group.Count() == world.Bees.Count()
      && framesRun > 0) {
        listBox1.Items.Add("Simulation ended: all bees are idle");
        toolStrip1.Items[0].Text = "Simulation ended";
        statusStrip1.Items[0].Text = "Simulation ended";
        timer1.Enabled = false;
    }
  }
}
```

Esta é a consulta LINQ. Ela verifica todas as abelhas na coleção Bees e agrupa-as pela sua propriedade CurrentState.

A Key (chave) do grupo é o CurrentState da abelha. Logo, essa será a ordem em que os estados serão mostrados no formulário.

Certifique-se de que este nome seja o mesmo da caixa de listagem no seu formulário.

beeGroup é derivado da consulta LINQ. Podemos contar seus membros e iterar por eles.

Esse pedaço do código certifica-se de que seja escrito "1 abelha" e "3 abelhas", mantendo o plural correto.

Finalmente, adicione o estado do grupo (sua chave) e contagem na caixa de listagem.

Eis aqui outra característica boa. Já que sabemos quantas abelhas estão paradas...

...podemos ver se TODAS estão. Se for o caso, a colméia está sem mel, então pararemos a simulação.

Relaxe

Aprenderemos muito a respeito da LINQ nos próximos capítulos

Você não precisa memorizar a sintaxe da LINQ e tentar enfiar tudo isso na sua cabeça agora. Praticaremo-nas bem mais no capítulo 15.

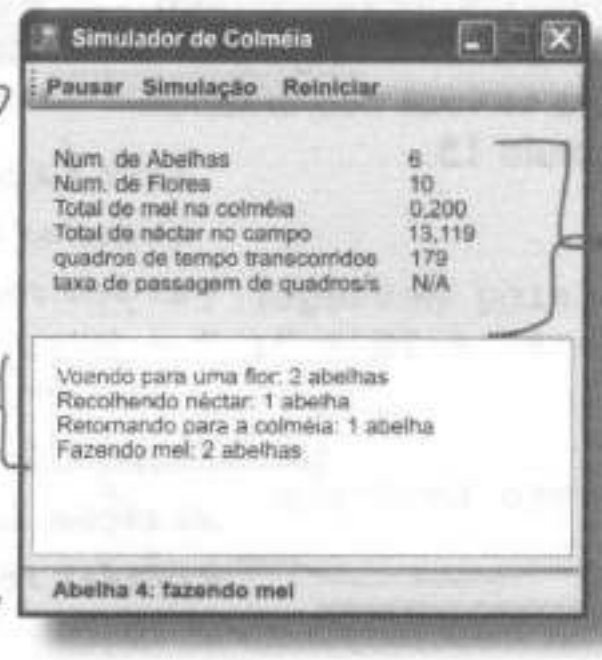
Exercício

Adicione código para fazer os botões de salvar e abrir funcionarem.

1. **Faça que o botão salvar serialize o mundo num arquivo.** Pare o timer (você pode recomeçar a contagem depois de salvar). Atribua null para MessageSender para a colméia e todas as abelhas. Assim o .NET não tentará serializar o código apontado pelo delegado, que seria o próprio formulário. Mostre uma caixa de diálogo de Salvar e então serialize o objeto World e o número de quadros executados até o momento. Não se esqueça de reassociar `SendMessage()` depois de salvar.
2. **Faça o botão abrir reverter a serialização de um mundo salvo num arquivo.** Faça com o timer o mesmo feito no botão salvar, mostre um diálogo Abrir e reverta a serialização do objeto World salvo no arquivo indicado e do número de quadros também. Aí você pode associar o delegado `MessageSender` novamente e recomeçar a contagem do timer (se for necessário).
3. **Não se esqueça do tratamento de exceções!** Certifique-se de que o mundo esteja em boa forma se acontecer algum problema ao ler ou escrever no arquivo. Considere mostrar uma caixa de erro com informações compreensíveis indicando o que aconteceu de errado.

Teste preliminar (parte 2)

Vá em frente, compile o código e execute o projeto. Se qualquer erro for levantado, faça uma dupla checagem na sua sintaxe, especialmente no novo código LINQ. Depois, inicie o simulador!



O timer no seu formulário controla a execução da simulação.

Essas estatísticas vêm do formulário que, por sua vez, retira-as do objeto World.

A LINQ consulta suas coleções para resultar nesses dados a cada turno.

Abelhas chamam o formulário do simulador para atualizá-lo quando mudam de estado.

Um desafio final: Abrir e Salvar

Estamos quase prontos para começar com os gráficos e tornar nosso simulador agradável aos olhos. Primeiro, no entanto, faremos mais uma coisa com essa versão: permitir carregar, salvar e imprimir as estatísticas das abelhas.

1. **Adicione os ícones de Abrir, Salvar e Imprimir**
   O controle ToolStrip tem uma característica muito útil: pode inserir botões com figura para ícones padrão automaticamente: novo, abrir, salvar, cortar, copiar, colar e ajuda. Apenas clique duas vezes no ícone da faixa de ferramentas na parte de baixo da janela do Designer e selecione "Insert Standard Items" (inserir itens padronizados). Então clique no primeiro item - é o ícone para "novo" - e apague-o. Mantenha os próximos três, porque são os de que precisamos (abrir, salvar e imprimir). Depois vem um separador e você pode apagá-lo ou movê-lo para ficar entre o botão Reiniciar e o Salvar. Após fazer isso apague o restante dos botões.

Você adicionará o botão Imprimir agora, mas só faremos com que ele imprima a página de estados para a colméia no próximo capítulo.

2. **Adicione os tratadores de eventos dos botões**
   Os novos botões padronizados são chamados openToolStripButton, saveToolStripButton e printToolStripButton. Clique duas vezes neles para adicionar seus tratadores de eventos.

Solução do Exercício

Não se esqueça dos comandos using extras.

```
using System.IO;
using System.Runtime.Serialization.Formatters.Binary;
```

Você deverá fazer com que as classes World, Hive, Flower e Bee sejam serializáveis. Quando você serializar o mundo, o .NET encontrará as referências a Hive, Flower e Bee e irá serializá-las também.

```
[Serializable]
public class World {
[Serializable]
public class Hive {
[Serializable]
public class Flower {
[Serializable]
public class Bee {
```

Eis o código para o botão Salvar.

```
private void saveToolStripButton_Click(object sender, EventArgs e) {
  bool enabled = timer1.Enabled;
  if (enabled)
    timer1.Stop();
  world.Hive.MessageSender = null;
  foreach (Bee bee in world.Bees)
    bee.MessageSender = null;
  SaveFileDialog saveDialog = new SaveFileDialog();
  saveDialog.Filter = "Simulator File (*.bees)|*.bees";
  saveDialog.CheckPathExists = true;
  saveDialog.Title = "Choose a file to save the current simulation";
  if (saveDialog.ShowDialog() == DialogResult.OK) {
    try {
      BinaryFormatter bf = new BinaryFormatter();
      using (Stream output = File.OpenWrite(saveDialog.FileName)) {
        bf.Serialize(output, world);
        bf.Serialize(output, framesRun);
      }
    }
    catch (Exception ex) {
      MessageBox.Show("Unable to save the simulator file\r\n" + ex.Message,
        "Bee Simulator Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
  }
  world.Hive.MessageSender = new Bee.BeeMessage(SendMessage);
  foreach (Bee bee in world.Bees)
    bee.MessageSender = new Bee.BeeMessage(SendMessage);
  if (enabled)
    timer1.Start();
}
```

Se não atribuirmos null para todos os delegados, quando tentássemos serializar o mundo eles agiriam como referências para Form1, que o BinaryFormatter tentaria serializar.

Decidimos usar ".bees" como extensão para os arquivos salvos pelo simulador.

É aqui que o mundo é salvo num arquivo.

Lembre-se, quando serializamos World, os objetos apontados por todas as referências que ele mantém também são serializados... todas as abelhas, flores e a colméia.

Depois que salvamos o arquivo, precisaremos associar os delegados MessageSender de novo e então recomeçar a contagem do timer (se nós paramos).

solução *do exercício*

Aqui está o código para o botão Abrir.

```
private void openToolStripButton_Click(object sender, EventArgs e) {
  World currentWorld = world;
  int currentFramesRun = framesRun;
  bool enabled = timer1.Enabled;
  if (enabled)
    timer1.Stop();
  OpenFileDialog openDialog = new OpenFileDialog();
  openDialog.Filter = "Simulator File (*.bees)|*.bees";
  openDialog.CheckPathExists = true;
  openDialog.CheckFileExists = true;
  openDialog.Title = "Choose a file with a simulation to load";
  if (openDialog.ShowDialog() == DialogResult.OK) {
    try {
      BinaryFormatter bf = new BinaryFormatter();
      using (Stream output = File.OpenRead(openDialog.FileName)) {
        world = (World)bf.Deserialize(output);
        framesRun = (int)bf.Deserialize(output);
      }
    }
    catch (Exception ex) {
      MessageBox.Show("Unable to read the simulator file\r\n" + ex.Message,
      "Bee Simulator Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
      world = currentWorld;
      framesRun = currentFramesRun;
    }
  }
  world.Hive.MessageSender = new Bee.BeeMessage(SendMessage);
  foreach (Bee bee in world.Bees)
    bee.MessageSender = new Bee.BeeMessage(SendMessage);
  if (enabled)
    timer1.Start();
}
```

Antes de abrir o arquivo e ler algo dele, salve uma referência ao mundo atual e framesRun (quadros executados). Se ocorrer um problema, você pode reverter para esses e manter tudo funcionando.

Configure a caixa de dialogo Abrir e mostre-a.

O comando using garante que a stream seja fechada.

Eis onde serializamos o mundo e o número de quadros executados para o arquivo.

Se as operações com arquivo lançarem uma exceção, restauramos o mundo atual e também framesRun.

Uma vez que tudo esteja carregado, associamos novamente os delegados e recomeçamos a contagem do timer.

Você precisará que seu simulador esteja pronto e funcionando antes de iniciar o próximo capítulo. É possível baixar uma versão funcional da página de Internet dos Laboratórios do Use a Cabeça: **www.altabooks.com.br**

13 controles e gráficos

Deixe bonito

Algumas vezes você precisa fazer os gráficos você mesmo.

Por um tempo, acreditamos que os controles lidavam com tudo o que fosse visual nos nossos aplicativos. Mas algumas vezes isso não é suficiente, como, por exemplo, quando você deve **animar uma imagem**. E uma vez que comece a lidar com animação, você acabará **criando seus próprios controles** para seus programas .NET, ou adicionando um **buffer duplo**, ou **desenhando diretamente nos formulários**. Tudo começa com o objeto **Graphics**, os **Bitmaps** e uma determinação de não aceitar os gráficos como eles são.

Você esteve usando controles até agora para interagir com seus programas

`Caixas de texto`, de `imagem`, `rótulos`... nesse ponto você já tem um bom domínio sobre como usar os controles na caixa de ferramentas do IDE. Mas o que você **realmente** sabe a respeito deles? Há muito mais a respeito de controles do que apenas arrastar um ícone num formulário.

- **Você pode criar seus próprios controles**
  Os controles da caixa de ferramentas são realmente úteis para construir formulários e aplicativos, mas não há nada de mágico neles. Eles são apenas classes, como as outras que você vêm escrevendo por si próprio. De fato, o C# torna fácil a criação de controles personalizados, apenas herdando da classe base certa.

- **Seus controles personalizados aparecem na caixa de ferramentas**
  Também não há nada de misterioso sobre a caixa de ferramentas no IDE. Ela apenas olha as classes em seu projeto e as classes nativas do .NET que sejam controles. Se houver alguma - uma classe que implemente as interfaces corretas, exibe-a como um ícone. Se você adicionar seus próprios controles, eles aparecerão na caixa de ferramentas também.

Você pode criar uma classe que herde de qualquer uma das classes de controles existentes - mesmo se não for incluir mais nenhum código - e ela automaticamente aparecerá na caixa de ferramentas.

- **Você pode escrever código para adicionar e remover controles nos formulários enquanto o programa está executando.**
  Projetar um formulário na janela Designer do IDE, não significa que as coisas acabam por aí. Você já fez `caixas de imagem` andarem por aí (quando desenvolveu o aplicativo de corrida de cães). Mas você também pode adicionar e remover controles. De fato, quando você cria um formulário usando o IDE, tudo o que ele faz é escrever código que adiciona os controles... ou seja, você pode escrever código similar e executá-lo quando quiser.

Controles de formulário são apenas objetos

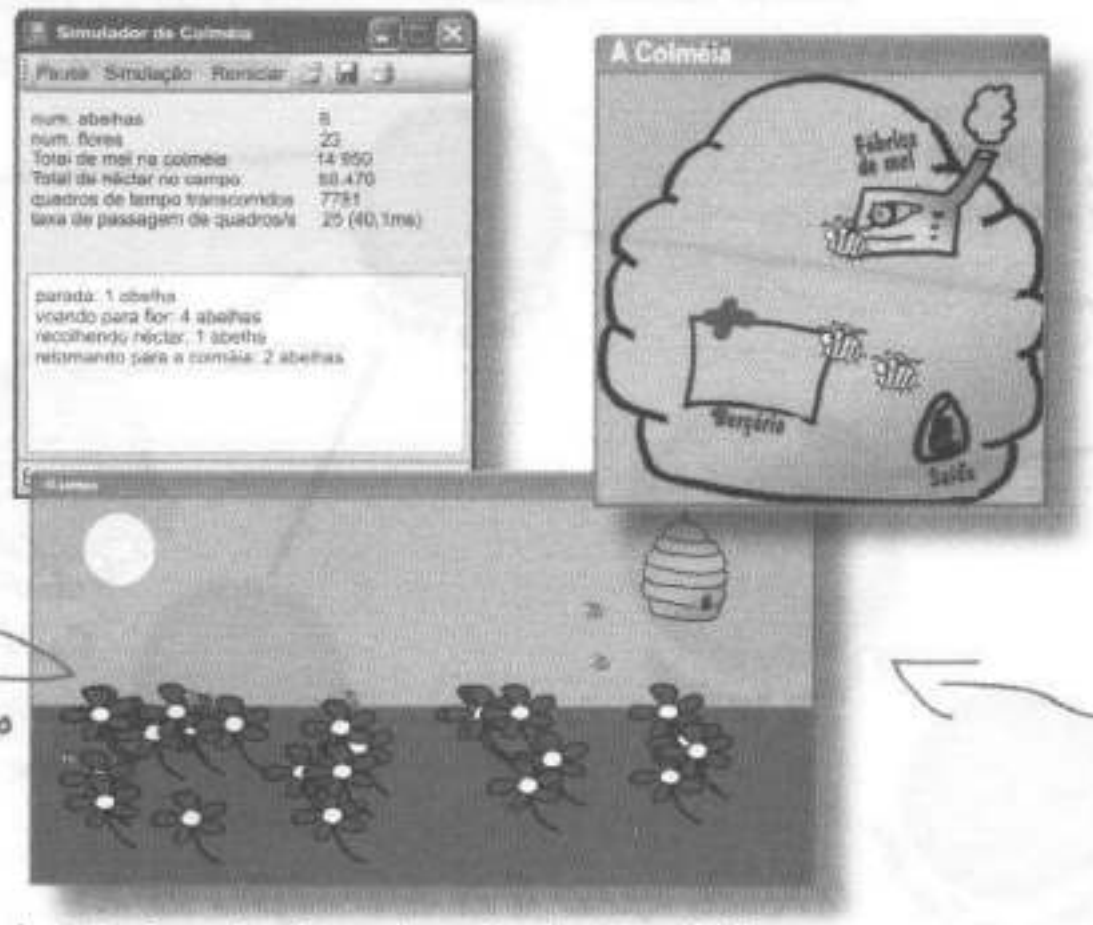
Você já sabe como os **controles** são importantes para seus formulários. Até aqui, você usou botões, caixas de texto, de imagem, de checagem e de grupo, rótulos e outros formulários. Mas estes controles são apenas objetos, exatamente como tudo com o que você esteve trabalhando. Um controle é apenas um objeto, como qualquer outro – porém, ele sabe como desenhar a si próprio. O objeto `Form` mantém registro sobre seus controles usando uma coleção especial chamada **Controls**, usada para adicionar ou remover controles usando código.

Use controles para animar o simulador de colméia

Você desenvolveu um simulador ótimo mas não muito bonito. É hora de criar uma visualização realmente impressionante que mostre aquelas abelhas trabalhando. Você está prestes a construir um renderizador (neologismo a partir da palavra renderer, sem tradução, que quer dizer algo como "desenhador", neste contexto) que animará a colméia... e controles são a chave.

1 **A interface de usuário mostra-lhe tudo o que está acontecendo**

Seu simulador terá três janelas diferentes. Você já fez a principal, um "mostrador instantâneo". Ela mostra estatísticas sobre a simulação no momento atual atualizadas pelas abelhas. Agora você adicionará uma janela para mostrar o que está acontecendo dentro da colméia e outra para mostrar o campo e as flores onde as abelhas recolhem néctar.



O formulário desenvolvido no capítulo anterior será o mostrador instantâneo para o simulador.

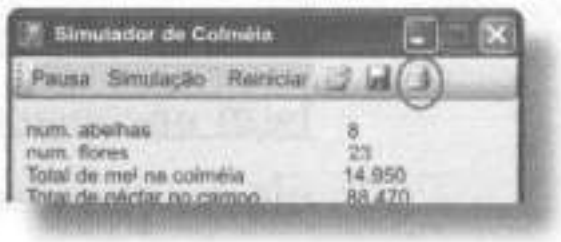
Essa janela mostra o que está acontecendo na colméia.

Esta janela mostra o campo de flores e as abelhas recolhendo néctar.

Essas duas janelas serão janelas filhas - quando você minimizar a principal, as duas desaparecerão. E quando você movimentar a principal, elas seguirão.

2 **Colocaremos um botão Imprimir na janela de estatísticas**

A janela de estatísticas já tem um botão Abrir e um Salvar. Uma vez que adicionemos os gráficos, poderemos acrescentar um botão na barra de ferramentas para imprimir uma página de informações a respeito do que está acontecendo.



3 **A janela de colméia mostra-lhe o que está acontecendo dentro da colméia**

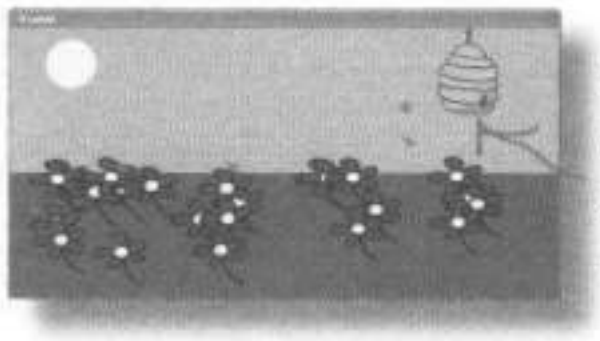
Você deverá animar cada abelha conforme elas voam pelo mundo. Algumas vezes, elas estarão dentro da colméia e, quando for o caso, elas aparecerão nesta janela.

A colméia tem três importantes locais. As abelhas nascem no berçário, devem voar até a saída para deixar a colméia para recolher néctar das flores e, quando elas voltam, precisam ir à fábrica para fazer mel.

4 **Abelhas coletam o néctar na janela do campo**

Abelhas têm uma tarefa importante: coletar néctar das flores e trazê-lo de volta à colméia para fazer mel. Então, elas comem mel para dar-lhes energia para voar e conseguir mais néctar.

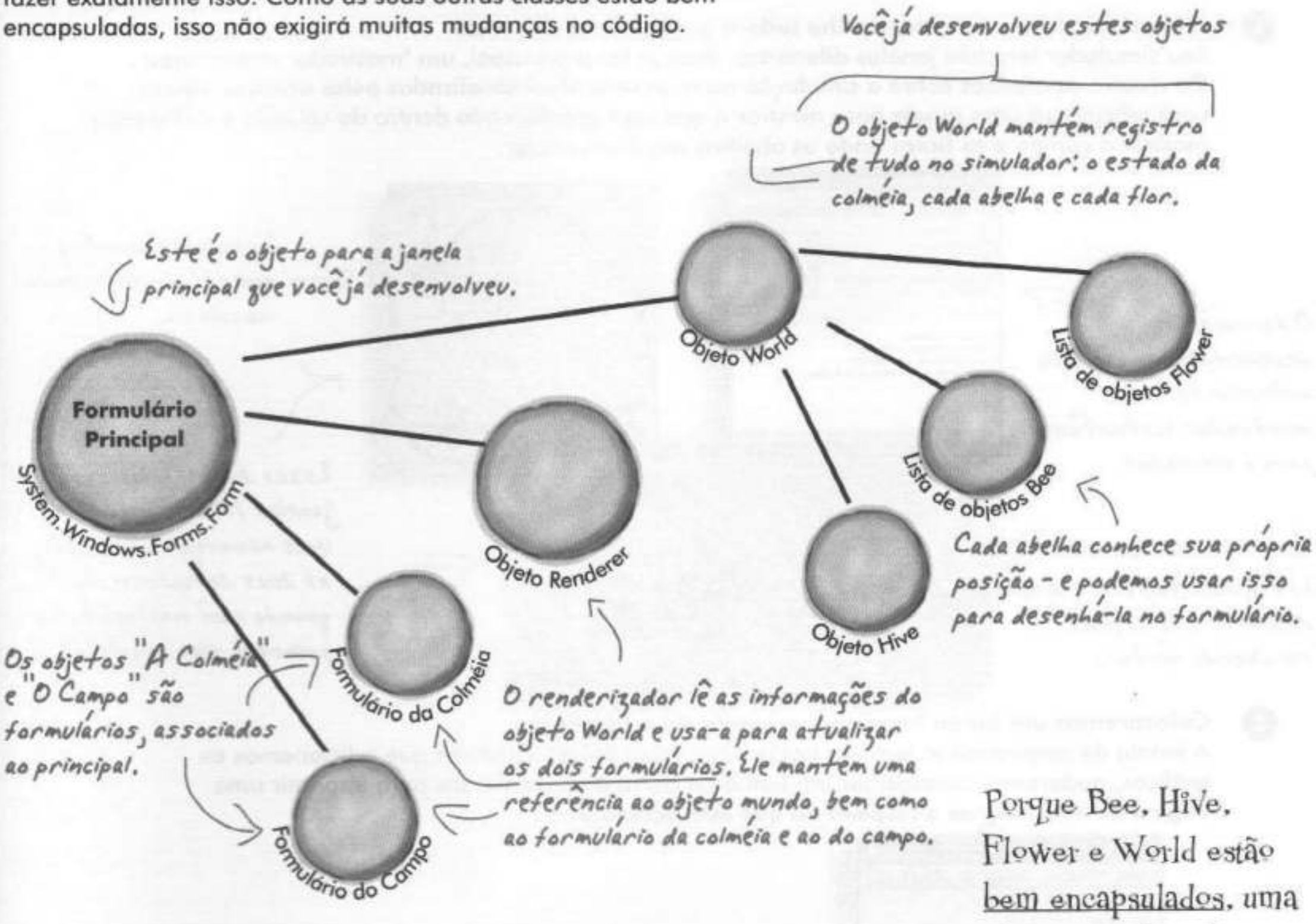
A saída é no formulário da colméia e a entrada é no do campo. (E por isso que colocamos ambos no dicionário de locais da colméia.)



Eis a entrada da colméia. Quando as abelhas voam para ela, elas desaparecem do formulário do campo e reaparecem perto da saída no formulário da colméia.

Adicione um renderizador à sua estrutura

Precisamos de outra classe que leia as informações no mundo e as utilize para desenhar a colméia, abelhas e flores em dois novos formulários. Adicionaremos uma classe **Renderer** para fazer exatamente isso. Como as suas outras classes estão bem encapsuladas, isso não exigirá muitas mudanças no código.

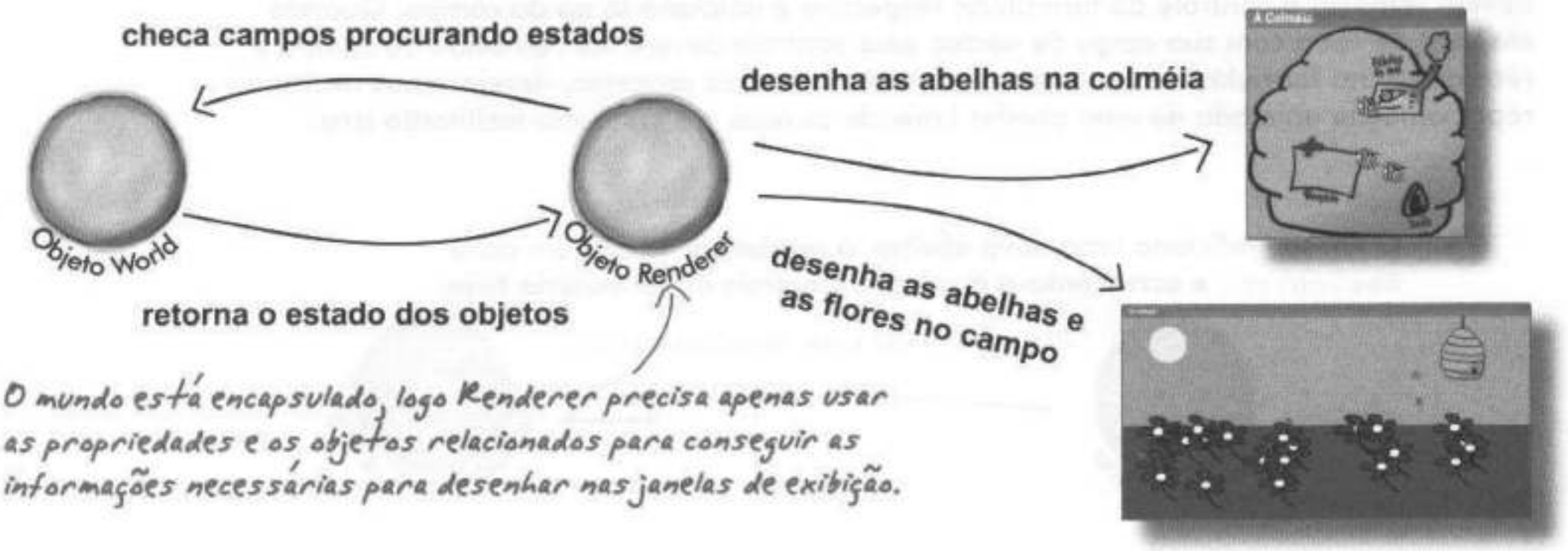


ren-de-ri-zar, verbo, neologismo, do inglês "render".
Representar artisticamente.
O professor de arte de Soraia disse para a classe examinar todas as sombras e linhas no modelo e ***renderizá-lo*** *numa folha de papel.*

Porque Bee, Hive, Flower e World estão bem encapsulados, uma classe que desenhe esses objetos pode ser adicionada sem provocar muitas alterações no código existente.

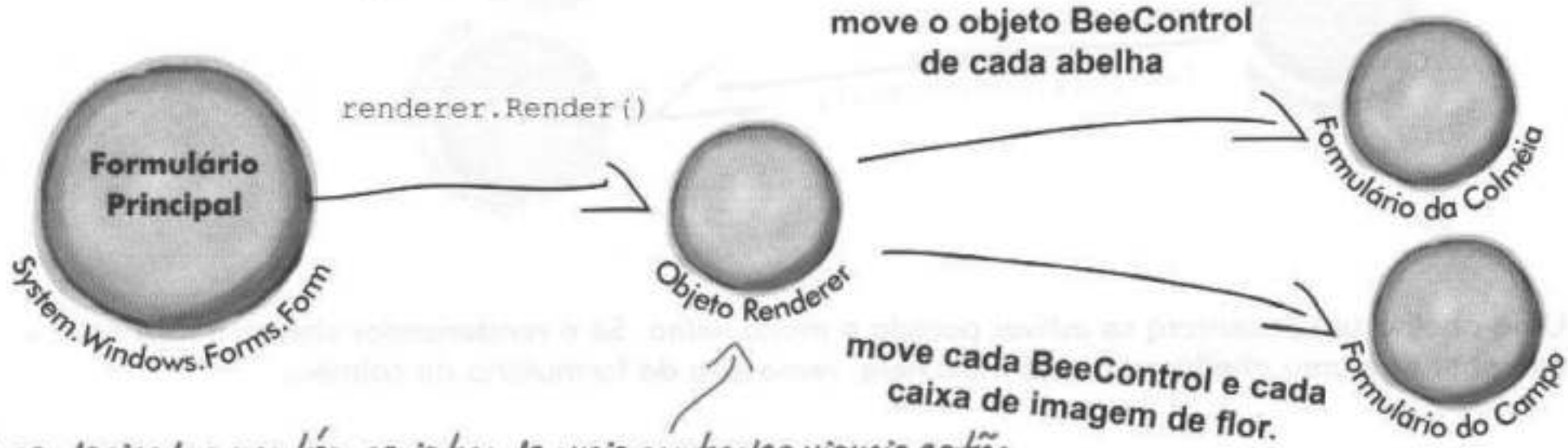
O renderizador desenha tudo no mundo nos dois formulários

O objeto `World` mantém registro de tudo na simulação: a colméia, as abelhas e as flores. Mas ele não desenha nada ou produz nenhum resultado. Isto é trabalho do objeto `Renderer`. Ele lerá todas as informações de World, Hive, dos diversos Bee e Flower e os desenhará nos formulários.



O simulador desenha o mundo a cada quadro

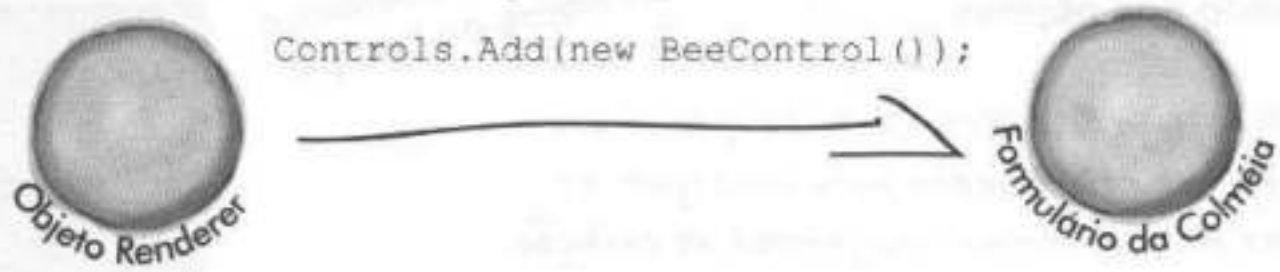
Depois que o formulário principal chamar o método `Go()` de `World`, ele deve chamar o método `Render()` do renderizador para redesenhar as janelas de exibição. Por exemplo, cada flor será mostrada usando um controle de `caixa de imagem`. Mas, por enquanto, continuaremos com as abelhas e criaremos um controle animado. Você pode criar esse novo controle, o `BeeControl`, e definir você mesmo seu comportamento.



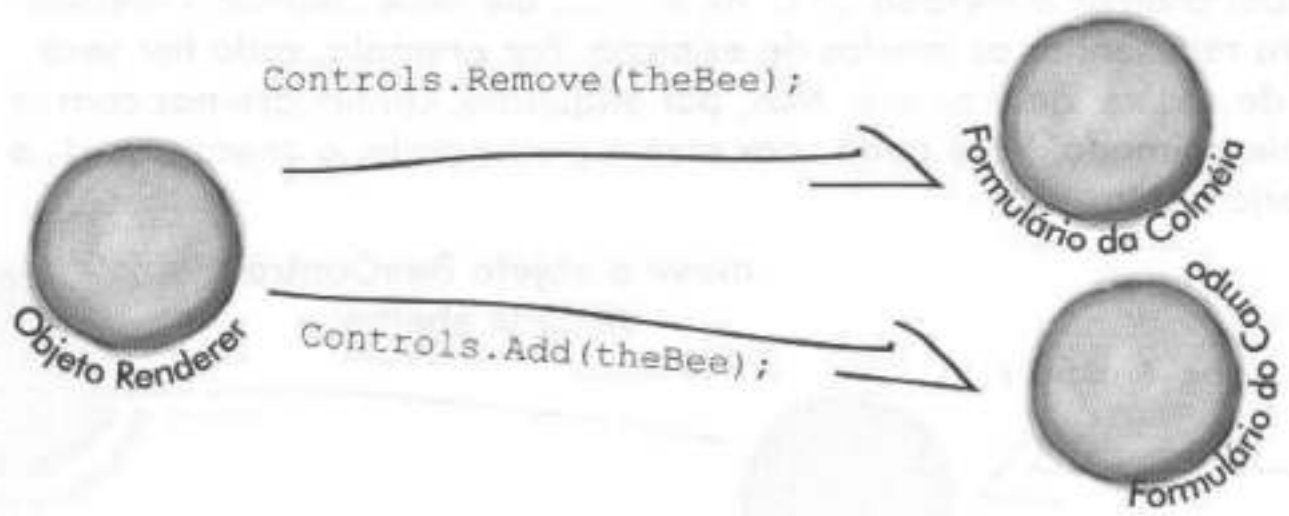
Controles são adequados como elementos visuais

Quando uma nova abelha é adicionada à colméia, queremos que nosso simulador adicione um novo `BeeControl` ao formulário da colméia e mude sua posição conforme ela se movimentar pelo mundo. Quando essa abelha voa para fora da colméia, nosso simulador deverá remover o controle do formulário respectivo e adicioná-lo ao do campo. Quando ela voar de volta com sua carga de néctar, esse controle deverá ser removido do campo e recolocado no formulário da colméia. E durante todo esse processo, desejaremos usar uma representação animada de uma abelha batendo as asas... e controles facilitarão isso.

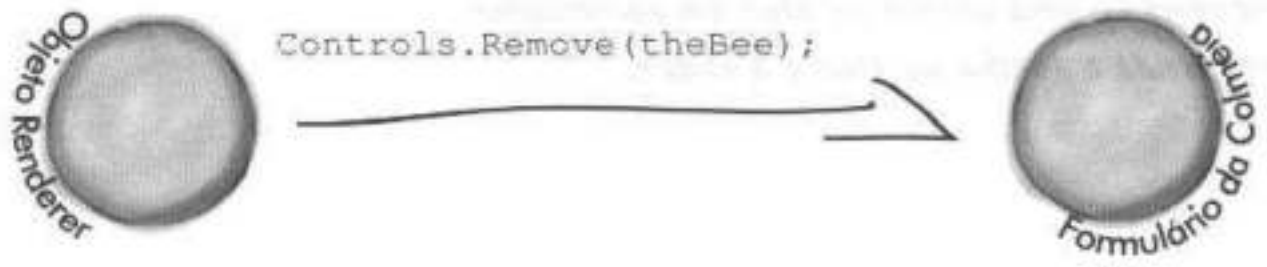
1. O mundo adiciona uma nova abelha, o renderizador cria um novo `BeeControl` e acrescenta-o à coleção Controls do formulário Hive.



2. Quando a abelha voa para fora da colméia e entra no campo, o renderizador remove o `BeeControl` da coleção `Controls` do formulário da colméia e adiciona-o à coleção semelhante do formulário do campo.



3. Uma abelha se aposentará se estiver parada e muito velha. Se o renderizador checar a lista Bees e perceber que uma abelha não está mais nela, remove-a do formulário da colméia.



Aponte seu lápis

Você consegue descobrir o que cada um desses trechos de código faz? Assuma que cada um deles esteja num formulário e anote o seu melhor palpite.

```
this.Controls.Add(new Button());
```

```
Form2 childWindow = new Form2();
childWindow.BackgroundImage =
      Properties.Resources.Mosaic;
childWindow.BackgroundImageLayout =
      ImageLayout.Tile;
childWindow.Show();
```

Se você tiver uma caixa de listagem no seu formulário, pode usar o método AddRange() para adicionar itens.

```
Label myLabel = new Label();
myLabel.Text = "What animal do you like?";
myLabel.Location = new Point(10, 10);
ListBox myList = new ListBox();
myList.Items.AddRange( new object[]
    { "Cat", "Dog", "Fish", "None" } );
myList.Location = new Point(10, 40);
Controls.Add(myLabel);
Controls.Add(myList);
```

Você não precisa anotar linha por linha, pode só fazer um resumo do que está acontecendo no bloco de código.

```
Label controlToRemove = null;
foreach (Control control in Controls) {
  if (control is Label
      && control.Text == "Bobby")
    controlToRemove = control as Label;
}
Controls.Remove(controlToRemove);
controlToRemove.Dispose();
```

Questão de bônus: porque você acha que não colocamos o comando Controls.Remove() dentro do loop foreach?

Tente se quiser e escreva o motivo pelo qual o resultado mostrado pelo .NET acontecerá.

Aponte seu lápis Solução

Você consegue descobrir o que cada um desses trechos de código faz? Assuma que cada um deles esteja num formulário e anote o seu melhor palpite.

```
this.Controls.Add(new Button());
```

Cria um novo botão e adiciona-o ao formulário. Ele terá valores padrão (ou seja, a propriedade Text estará vazia).

```
Form2 childWindow = new Form2();
childWindow.BackgroundImage =
      Properties.Resources.Mosaic;
childWindow.BackgroundImageLayout =
      ImageLayout.Tile;
childWindow.Show();
```

Existirá um segundo formulário no aplicativo chamado Form2. Esse trecho cria-o, atribui uma imagem para seu fundo a partir de um recurso chamado "Mosaic" e faz essa imagem repetir-se, em vez de esticar-se, e exibe-a para o usuário.

```
Label myLabel = new Label();
myLabel.Text = "What animal do you like?";
myLabel.Location = new Point(10, 10);
ListBox myList = new ListBox();
myList.Items.AddRange( new object[]
    { "Cat", "Dog", "Fish", "None" } );
myList.Location = new Point(10, 40);
Controls.Add(myLabel);
Controls.Add(myList);
```

Esse código cria um novo rótulo, atribui seu texto e coloca-o numa nova posição. Então se cria uma nova caixa de listagem, adiciona quatro itens a ela e desloca-a para logo abaixo do rótulo. O código adiciona o rótulo e a caixa de listagem ao formulário, então eles são exibidos imediatamente.

```
Label controlToRemove = null;
foreach (Control control in Controls) {
  if (control is Label
      && control.Text == "Bobby")
    controlToRemove = control as Label;
}
Controls.Remove(controlToRemove);
controlToRemove.Dispose();
```

Esse laço itera por todos os controles no formulário até achar um rótulo com o texto "Bobby". Uma vez que ele acha o rótulo, remove-o do formulário.

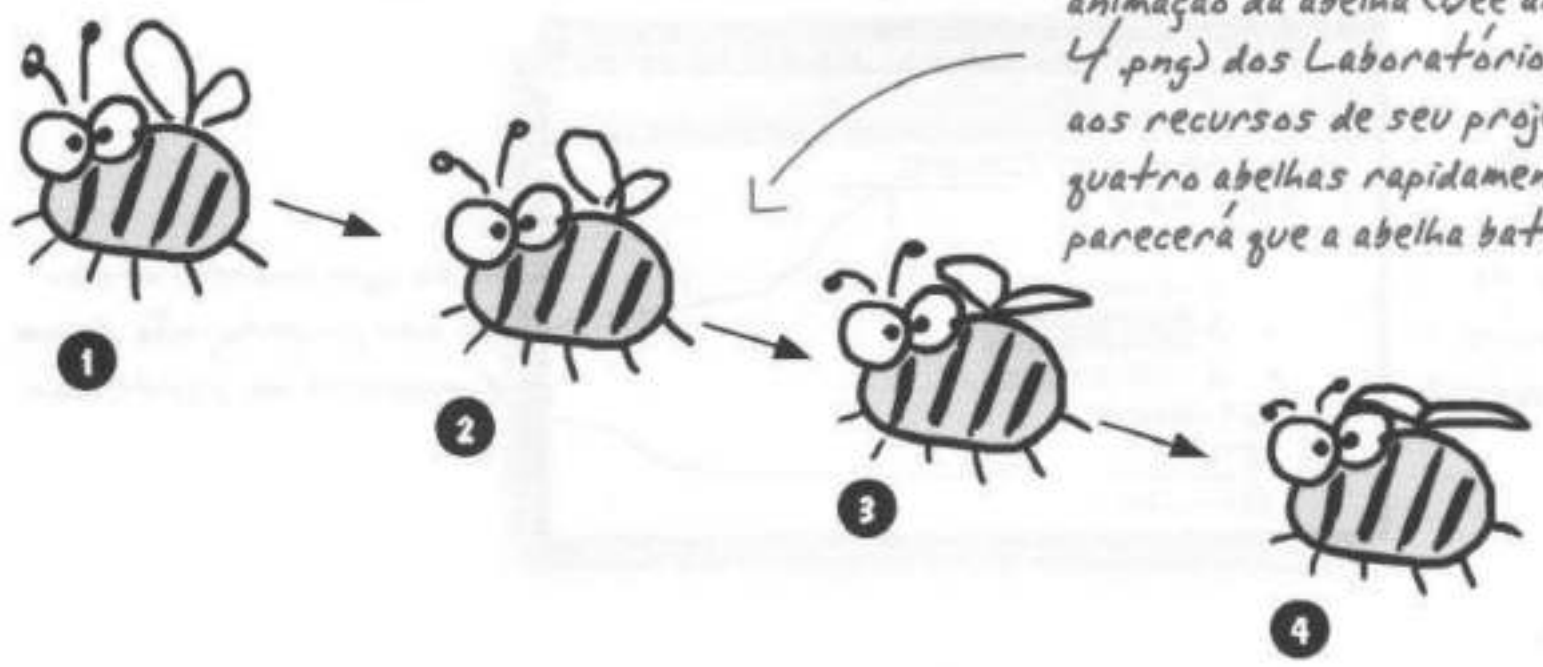
Se você tentar, o .NET lançará uma exceção. Para ele, a coleção deve permanecer intacta; de outra forma ele se perde e os resultados são imprevisíveis. É por isso que usamos um laço for no lugar.

Questão de bônus: por que você acha que não colocamos o comando Controls.Remove() dentro do laço foreach?

Você não pode modificar a coleção Controls (ou qualquer outra) no meio de um laço foreach que itere por ela.

Desenvolva seu primeiro controle animado

Você **desenvolverá seu próprio controle** que desenha uma figura animada de abelha. Se você nunca fez animações, não é tão difícil quanto parece: desenhe uma seqüência de figuras e mostre uma após a outra; isto produzirá a ilusão de movimento. Por sorte, a forma como o C# e o .NET lidam com recursos realmente facilita fazer animação.

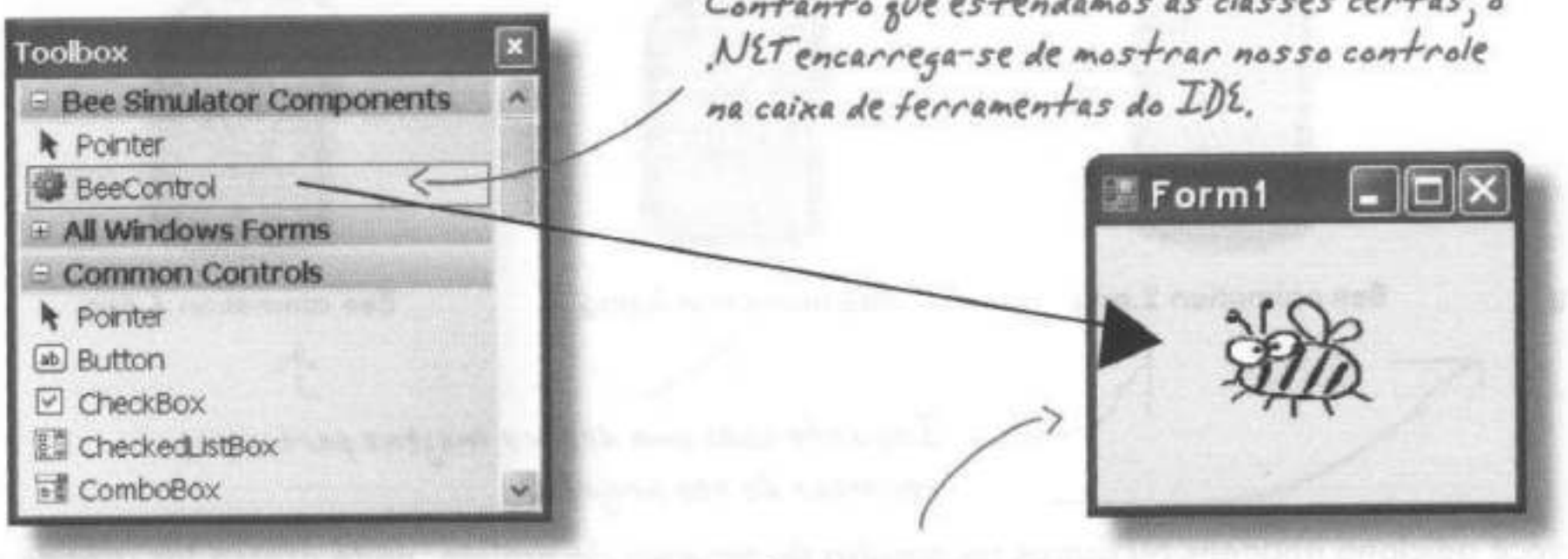


Uma vez que você tenha baixado as quatro figuras da animação da abelha (Bee animation 1.png até Bee animation 4.png) dos Laboratórios Use a Cabeça, você as adicionará aos recursos de seu projeto. Quando você exibir essas quatro abelhas rapidamente uma depois da outra, parecerá que a abelha bate as asas.

Queremos um controle na caixa de ferramentas

Se você criar `BeeControl` direitinho, ele aparecerá como um controle que poderá ser arrastado da caixa de ferramentas para o formulário. Ela parecerá como uma `caixa de imagem` mostrando uma figura de abelha, exceto que terá asas animadas.

Baixe as imagens para esse capítulo da página de Internet dos Laboratórios Use a Cabeça: www.altabooks.com.br

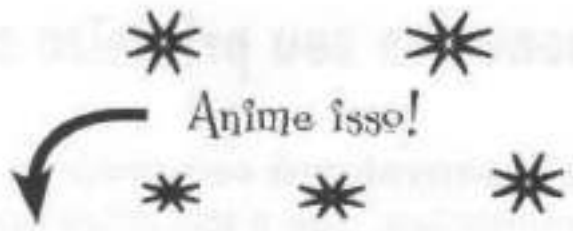


Contanto que estendamos as classes certas, o .NET encarrega-se de mostrar nosso controle na caixa de ferramentas do IDE.

Isso é como uma caixa de imagem, mas a imagem é fixa e há a animação que incluímos nela. Algum palpite a respeito de qual será a classe base?

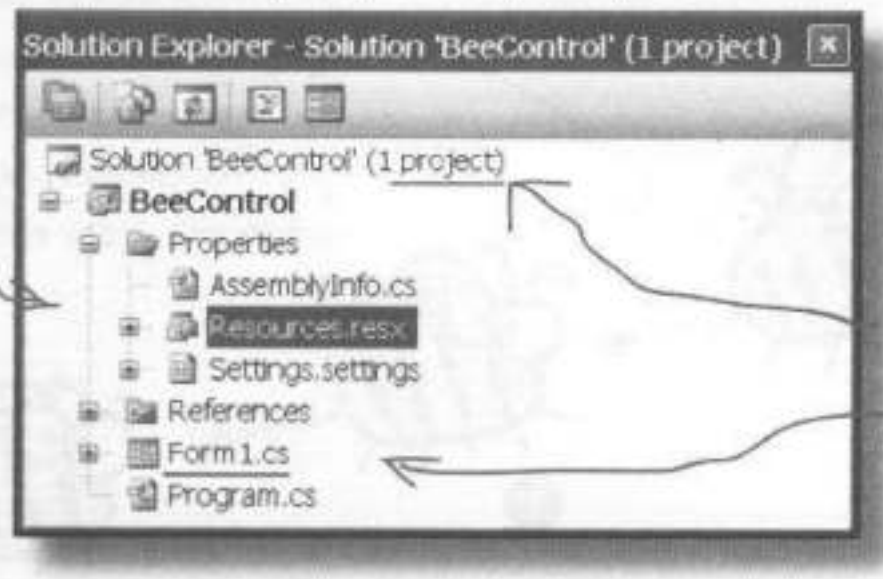
BeeControl é <u>COMO</u> uma caixa de imagem... então comecemos <u>HERDANDO</u> de uma PictureBox

Já que cada controle na caixa de ferramentas é apenas um objeto, é fácil fazer um novo controle. Tudo a fazer é adicionar uma nova classe ao seu projeto e herdar de um controle existente, e adicionar novo comportamento a ele.
Queremos um controle - vamos chamar de **BeeControl** - que mostre uma figura animada de uma abelha batendo as asas, mas começaremos com um controle que mostre uma imagem não animada e adicionaremo-lhes a capacidade para a animação. Então começaremos com uma `PictureBox` e acrescentaremos código para desenhar uma abelha animada nela.



1. **Crie um novo projeto** e adicione quatro células de animação aos recursos, assim como adicionou o logo da Empresa de Papel da Vila Objeto ao seu projeto no capítulo 1. Mas, em vez de adicioná-los aos recursos do formulário, faça com que sejam recursos de projeto.

No capítulo 1, adicionamos o gráfico para o logo ao arquivo de recursos do formulário. Dessa vez, faremos isso usando a coleção global de recursos do projeto, para que tudo fique disponível a todas as classes no projeto (através da coleção Properties.Resources).



Essa aqui aparece abaixo do seu projeto, não de um formulário em particular.

Pare um minuto e volte ao capítulo 1 para lembrar como fazer isso.

2. Desenhamos quatro células de animação de abelha que você pode importar para seus recursos. Você pode baixá-las de **www.altabooks.com.br**. Vá até a página de recursos, selecione "Images" na primeira caixa de seleção no topo da tela e selecione "Add Existing File..." (adicionar arquivo existente) na caixa de seleção "Add Resource" (adicionar recurso).

Bee animation 1.png

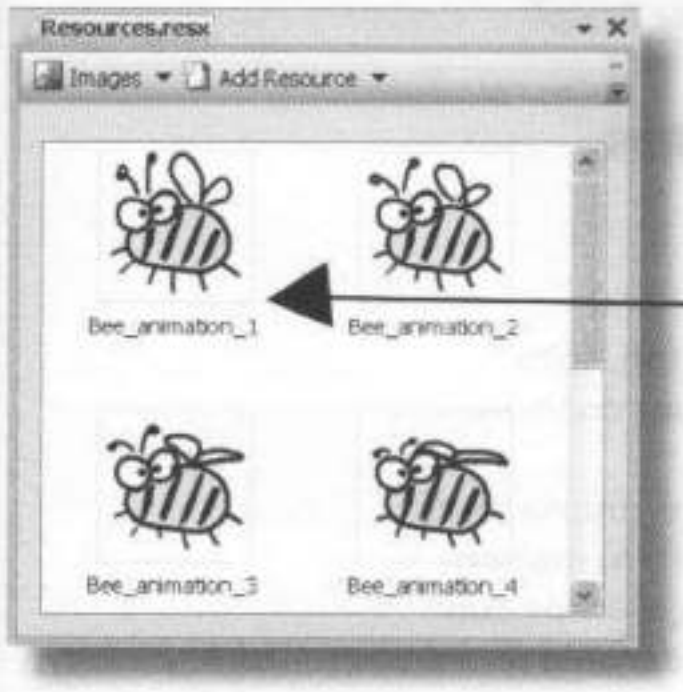
Bee animation 2.png

Bee animation 3.png

Bee animation 4.png

Importe cada uma dessas imagens para os recursos de seu projeto.

3. Quando você adiciona imagens ou outros ao arquivo de recursos do projeto, pode acessá-los usando a classe Properties.Resources. Basta digitar o nome da classe em qualquer lugar do seu código. Assim que você isso, o IntelliSense mostrará uma caixa de seleção com uma lista com todas as imagens importadas.



Note aquele ". no final... é isso que diz ao IDE para mostrar as propriedades e métodos da classe que você incluiu.

```
pictureBox1.Image =
  Properties.Resources.Bee_animation_1;
```

Essas imagens são armazenadas como propriedades públicas da classe Properties.Resources.

Isso atribui a figura usada como imagem de uma PictureBox em particular (e, no caso, serve como nossa imagem inicial).

Você deverá acrescentar um "using System.Windows.Forms" para a caixa de imagem e o timer.

4 Agora inclua seu BeeControl! Adicione a classe para ele no projeto:

```
public class BeeControl : PictureBox {
  private Timer animationTimer = new Timer();
  public BeeControl() {
    animationTimer.Tick += new EventHandler(animationTimer_Tick);
    animationTimer.Interval = 150;
    animationTimer.Start();
    BackColor = System.Drawing.Color.Transparent;
    BackgroundImageLayout = ImageLayout.Stretch;
  }
  private int cell = 0;
  void animationTimer_Tick(object sender, EventArgs e) {
    cell++;
    switch (cell) {
      case 1: BackgroundImage = Properties.Resources.Bee_animation_1; break;
      case 2: BackgroundImage = Properties.Resources.Bee_animation_2; break;
      case 3: BackgroundImage = Properties.Resources.Bee_animation_3; break;
      case 4: BackgroundImage = Properties.Resources.Bee_animation_4; break;
      case 5: BackgroundImage = Properties.Resources.Bee_animation_3; break;
      default: BackgroundImage = Properties.Resources.Bee_animation_2;
      cell = 0; break;
    }
  }
}
```

É aqui que você inicializa seu timer ao instanciá-lo, atribuindo sua propriedade Interval e depois, o tratador para seu evento Tick.

Cada vez que o evento Tick do timer disparar, cell é incrementada e um switch baseado nela atribui a figura correta à propriedade Image (herdada de PictureBox).

Uma vez que tenhamos voltado ao quadro 1, reiniciamos cell de volta para 0.

Quando você muda o código de um controle, precisa compilar novamente seu programa para fazer com que suas mudanças apareçam no designer.

Então **compile novamente seu programa**. Volte ao designer de formulário e olhe na caixa de ferramentas: BeeControl deve estar lá. Arraste-o para seu formulário - e você terá uma abelha **animada!**

Crie um botão para adicionar BeeControl ao seu formulário

É fácil adicionar um controle a um formulário - basta acrescentá-lo na coleção Controls. E é fácil da mesma forma removê-lo retirando-o dessa coleção. Mas os controles implementam IDisposable, então certifique-se sempre de descartar seus controles depois de removê-los.

Agora faça isso

1 Remova o BeeControl do seu formulário e adicione um botão
Vá para o designer de formulário e **apague o controle BeeControl do seu formulário**. Depois adicione um botão. Faremos com que ele adicione e remova BeeControls.

2 Acrescente um botão para adicionar e remover um controle de abelha
Eis o tratador de evento para fazer isso:

Você pode usar um inicializador de objetos para atribuir valores às propriedades do BeeControl depois que ele for instanciado.

```
BeeControl control = null;
private void button1_Click(object sender, EventArgs e) {
  if (control == null) {
    control = new BeeControl() { Location = new Point(100, 100) };
    Controls.Add(control);
  } else {
    Controls.Remove(control);
    control.Dispose();
    control = null;
  }
}
```

Quando você adiciona um controle na coleção Controls, ele aparece no formulário imediatamente.

Tudo a fazer para remover o controle é retirá-lo da coleção Controls. Mas certifique-se de descartá-lo - senão você estará desperdiçando recursos!

Agora quando você executar seu programa, se clicar no botão uma vez ele adicionará um novo BeeControl no formulário. Clique novamente e ele vai apagá-lo. Ele usa o campo privado `control` para manter uma referência ao controle (e atribui null para a referência quando não há nenhum controle no formulário).

Você pode adicionar seus próprios controles na caixa de ferramentas apenas criando uma classe que herda de Control.

Nos Bastidores

Cada um dos controles visuais na sua caixa de ferramentas herda de System.Windows.Forms.Control. Essa classe tem alguns membros que devem ser muito familiares nesse ponto: Visible, Width, Height, Text, Location, BackColor, BackgroundImage... você observa todas essas propriedades na janela de propriedades do formulário.

Seus controles precisam descartar os controles deles também!

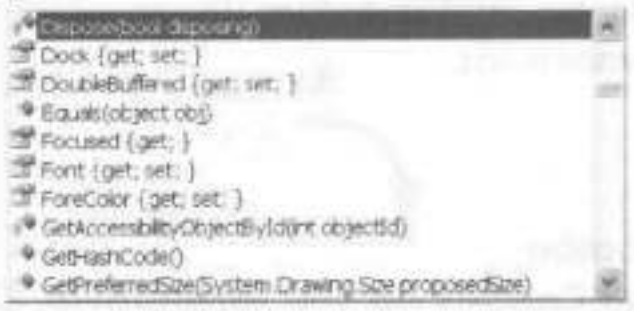
Há um problema com BeeControl. Controles precisam ser descartados depois de usados. Mas o BeeControl cria uma nova instância de Timer, que é um controle, aparece na caixa de ferramentas... e nunca chega a ser descartado! Isso é um problema, por sorte, fácil de corrigir - apenas sobreponha o método Dispose().

A classe dos controles implementa IDisposable, logo assegure-se de que cada controle usado seja descartado.

3 Sobreponha o método Dispose() e descarte o timer

Já que BeeControl herda de um controle, este último deve ter um método Dispose(). Logo podemos simplesmente sobrepor e estender esse método para descartar nosso timer. É só ir até o código do controle e digitar override:

```
public class BeeControl : PictureBox {

override
```



Quando você digita "override" dentro de uma classe, o IDE mostra uma janela do IntelliSense com todos os métodos que podem ser sobrepostos. Selecione o método Dispose() e o IDE criará um para você!

Assim que você clicar em Dispose(), o IDE criará o método com uma chamada a base.Dispose():

```
protected override void Dispose(bool disposing) {
  base.Dispose(disposing);
}
```

4 Adicione o código para descartar o timer

Uma vez que BeeControl herda de um controle, este último deve ter um método Dispose(). Assim, podemos simplesmente sobrepor e estender esse método para descartar nosso timer. Basta adicionar uma linha no método Dispose() que o IDE criou para você:

```
protected override void Dispose(bool disposing) {
  animationTimer.Dispose();
  base.Dispose(disposing);
}
```

Agora o BeeControl descartará seu timer como parte de seu método Dispose(). Ele limpa a própria bagunça!

Agora o BeeControl descartará o timer que criou em seu construtor. O IDE também incluiu uma chamada ao método Dispose() da classe base PictureBox - deixe assim para que a caixa de imagem possa descartar a si mesma.

Qualquer controle que você escreva a partir do zero é responsável por descartar qualquer outro controle (ou outros objetos descartáveis) que ele criar.

Um UserControl é uma forma fácil de desenvolver um controle

Há uma forma mais fácil de criar seus próprios controles para a caixa de ferramentas. Em vez de criar uma classe que herde de um controle existente, **use o IDE para adicionar um UserControl (controle de usuário) ao seu projeto**. Você trabalha com seu controle de usuário exatamente como faria com um formulário. Você arrasta outros controles da caixa de ferramenta para dentro dele - e utiliza o próprio designer de formulário no IDE. E você pode usar os eventos da mesma forma que faria com um formulário. Então refaçamos BeeControl usando um UserControl.

Faça isso

1. Clique com o botão direito em BeeControl.cs no IDE e renuncie para OldBeeControl.cs (old significa "antigo", "velho"). O IDE mostrará uma **janela pedindo um sim ou não**: *"You are renaming a file. Would you also like to perform a rename in this project of all references to the code element 'BeeControl'?" (você está renomeando um arquivo. Você gostaria de renomear, nesse projeto, todas as referências ao elemento de código 'BeeControl'?)* - essa janela está perguntado se você quer renomear a classe BeeControl para OldBeeControl e depois mudar todo o código do projeto para refletir essa alteração. **Clique no botão "Yes"** para renomear seu BeeControl para OldBeeControl.

2. Clique com o botão direito no Navegador de Solução e selecione "Add>>UserControl..." (adicionar -> controle de usuário). Faça com que o IDE **adicione um controle de usuário chamado BeeControl**. O IDE abrirá o novo controle no designer de formulário.

Use o método animationTimer_Tick (você vai ter de renomeá-lo) e o campo cell do antigo controle de abelha - mas não copie mais nada.

3. Arraste um controle timer no seu controle de usuário. Ele aparecerá na parte de baixo do designer, exatamente como acontece num formulário. Use a janela de propriedades para atribuir 150 para `Interval` e true para Enabled. Então clique duas vezes nele - o IDE adicionará o tratador para o evento Tick. Use o mesmo código usado antes para animar o primeiro controle de abelha.

4. Agora atualize o construtor de BeeControl.

```
public BeeControl() {
  InitializeComponent();
  BackColor = System.Drawing.Color.Transparent;
  BackgroundImageLayout = ImageLayout.Stretch;
}
```

Você pode também fazer isso usando a página de propriedades no IDE, em vez de usar código.

5. Volte para o tratador de evento do botão no formulário. Quando você renomeou o antigo BeeControl para OldBeeControl, o código do formulário mudou também. Então mude as duas linhas de código de volta para BeeControl para usar seu novo controle, em vez de uma caixa de imagem:

```
BeeControl control = null;
private void button1_Click(...) {
  if (control == null) {
    control = new BeeControl() { ...
```

Um UserControl é uma fácil de adicionar um controle à caixa de ferramentas. Edite um deles exatamente como faria com um formulário - você pode arrastar outros controles da caixa de ferramentas para ele e usar eventos exatamente da mesma forma.

Depois **execute seu programa** - ele deve funcionar exatamente como antes. O botão agora adiciona e remove seu BeeControl, que é baseado num UserControl.

Você não descartou seus controles porque seus formulários fazem isso no seu lugar.

Mas não aceite nossa palavra. Use a função de pesquisa do IDE para encontrar em seu projeto o termo "Dispose". Você verá que o IDE adiciona um método em Form1.Designer.cs que sobrepõe o Dispose() e que chama o mesmo método da `classe base`. Quando o formulário é descartado, **ele automaticamente descarta tudo na sua coleção Controls** para que você não precise se preocupar com isso. Mas quando você começa a remover controles ou cria novas instâncias (como o timer no BeeControl) fora dessa coleção, então deve descartar você mesmo.

não existem Perguntas Idiotas

P: Por que o código do formulário para o BeeControl baseado numa caixa de imagem funciona como o baseado no UserControl?

R: Porque o código não se importa com a forma como o BeeControl foi implementado. Ele apenas se importa com ser capaz de adicionar o controle à coleção de controles do formulário.

P: Eu cliquei duas vezes na minha classe OldBeeControl no Navegador de Solução e surgiu uma mensagem sobre adicionar componentes na minha classe. O que isso quer dizer?

R: Quando você cria um controle adicionando ao seu projeto uma classe que herda de PictureBox ou outro controle, o IDE faz algumas coisas inteligentes. Uma delas é permitir o trabalho com **componentes**, os controles não visuais como o Timer e o OpenFileDialog que aparecem no espaço abaixo do formulário quando você está editando.
Tente - crie uma classe vazia que herde de PictureBox, compile novamente seu projeto e clique duas vezes nele no IDE. Você receberá a seguinte mensagem:

To add components to your class, drag them from the Toolbox and use the Properties window to set their properties. (Para adicionar componentes à sua classe, arraste-os da caixa de ferramentas e use a janela de propriedades para configurá-los.)

Arraste um OpenFileDialog da caixa de ferramentas até essa nova classe. Ele aparecerá como um ícone. Você pode clicar nele e atribuir valores às suas propriedades. Faça isso com algumas. Agora volte para o código da sua classe. Veja no construtor - o IDE adicionou código para instanciar o objeto OpenFileDialog e para alterar as propriedades.

P: Quando eu mudei as propriedades do OpenFileDialog, notei uma mensagem de erro no IDE "You must rebuild your project for the changes to show up in any open designers" (você precisa compilar novamente seu projeto para que qualquer alteração em designers abertos sejam visíveis). Porque ocorre esse erro?

R: Porque o designer executa seu controle e até que você compile novamente seu código ele não executará a última versão dele. Lembra como as asas da abelha estavam batendo quando você criou pela primeira vez seu BeeControl, mesmo quando você o arrastou da caixa de ferramentas e o largou no designer? Você não estava executando seu programa ainda, mas o código que você escreveu estava. O timer estava disparando seu evento Tick e seu tratador de eventos estava alterando a imagem. A única forma de o IDE fazer isso é o código estar efetivamente compilado e executando na memória em algum lugar. Assim ele lembra você para atualizar o código para que ele possa mostrar seus controles corretamente.

O renderizador usará seu BeeControl para desenhar abelhas animadas nos formulários

Com uma classe BeeControl e dois formulários, você precisa apenas de uma forma de posicionar as abelhas, movê-las de um formulário para outro e acompanhar suas alterações. Você também precisará posicionar as flores no formulário do campo, mas como elas não se movem, isso será bem simples. Tudo isso é código que podemos colocar numa nova classe, `Renderer`. Aqui está o que essa classe faz:

Você desejará que os formulários de colméia e de campo sejam "ligados" ao de estatística - para conseguir coisas úteis, como minimizá-los todos juntos. Você pode fazer isso dizendo ao Windows que o formulário de estatística é o dono dos demais.

1 O formulário de estatísticas será o antecessor do de colméia e do de campo

O primeiro passo para adicionar gráficos ao Simulador de Colméia é incluir dois formulários ao projeto. Você deve acrescentar um chamado HiveForm (para mostrar o interior da colméia) e outro FieldForm (que mostrará o campo de flores). Então você deve inserir linhas no construtor do formulário principal para mostrar seus dois formulários-filhos, dizendo ao Windows que o formulário de estatísticas é seu dono (**owner**).

```
public Form1() {
  // outro código no construtor Form1
  hiveForm.Show(this);
  fieldForm.Show(this);
}
```

Todo objeto formulário tem um método Show(). Para permitir a um formulário ser dono de outro, passe uma referência do primeiro para o método Show() do outro.

2 O renderizador manterá referências para o mundo e para cada formulário filho

Logo no início da classe `Renderer` você deverá ter alguns campos importantes. A classe precisa saber a localização de cada abelha e flor. Logo, ela precisa de uma referência para `World`. Como deverá adicionar, mover e remover controles dos dois formulários, precisará de uma referência para cada um deles.

```
public class Renderer {
  private World world;
  private HiveForm hiveForm;
  private FieldForm fieldForm;
```

Inicie sua classe Renderer com estas linhas. Completaremos essa classe durante este capítulo.

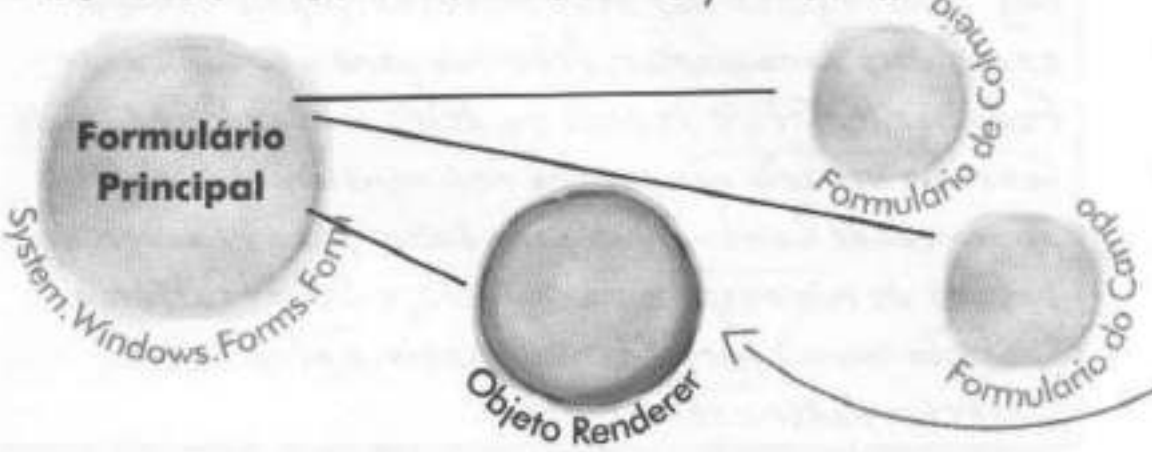
3 O renderizador usará dicionários para manter registro dos controles

World mantém registro de seus objetos `Bee` usando um `List<Bee>` e para armazenar suas flores usa um `List<Flower>`. O renderizador precisa ser capaz de examinar cada um desses objetos Bee e Flower e determinar a qual `BeeControl` e `PictureBox` eles correspondem - ou, se ele não puder achar um controle correspondente, precisa poder criar um. Então temos a oportunidade perfeita para usar dicionários. Precisaremos de mais dois campos privados em `Renderer`:

```
private Dictionary<Flower, PictureBox> flowerLookup =
  new Dictionary<Flower, PictureBox>();
private Dictionary<Bee, BeeControl> beeLookup =
  new Dictionary<Bee, BeeControl>();
```

Essas duas coleções dicionários permitem ao renderizador armazenar exatamente um único controle para cada abelha ou flor no mundo.

Esses dicionários serão mapas um-para-um entre uma abelha ou flor e um controle correspondente.



O renderizador atuará em dois formulários, bem como em todos os objetos desenvolvidos no capítulo anterior.

4 **As flores e abelhas já sabem suas posições**
Há um motivo para termos armazenado as posições de flores e abelhas usando um `Point`: uma vez que tenhamos um objeto `Bee`, podemos facilmente procurar seu `BeeControl` e colocá-lo na posição correta.

```
beeControl = beeLookup[bee];
beeControl.Location = bee.Location;
```

Para cada abelha ou flor, podemos procurar um controle correspondente. Então podemos posicioná-lo usando o local informado naquele objeto bee ou flower.

5 **Se uma abelha não possuir um BeeControl, o renderizador adiciona um ao formulário de colméia**
É bem fácil para o renderizador descobrir se uma abelha ou flor em particular possui um controle. Se o método `ContainsKey()` do dicionário retornar `false` para um objeto `Bee` em particular, não há um controle no formulário para aquela abelha. Nesse caso `Renderer` deverá criar um `BeeControl`, adicioná-lo ao dicionário e inserir esse controle no formulário (ele também chamará o método `BringToFront` - trazer para a frente - do controle, para certificar-se de que ele não fique oculto atrás das `caixas de imagens` das flores):

```
if (!beeLookup.ContainsKey(bee)) {
  if (!beeLookup.ContainsKey(bee)) {
    beeControl = new BeeControl() { Width = 40, Height = 40 };
    beeLookup.Add(bee, beeControl);
    hiveForm.Controls.Add(beeControl);
    beeControl.BringToFront();
  } else
    beeControl = beeLookup[bee];
```

ContainsKey() diz-nos se a abelha existe no dicionário. Se não, precisamos acrescentá-la, juntamente com um controle correspondente.

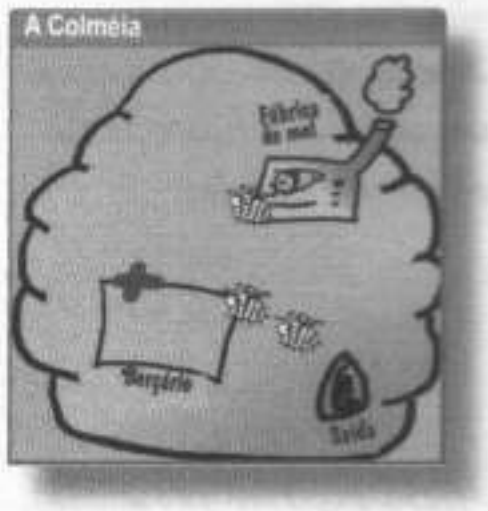
Lembre-se como um dicionário pode usar qualquer coisa como chave? Bem, este usa um objeto Bee. O renderizador precisa saber qual BeeControl no formulário pertence a cada abelha em particular. Então ele procura por um objeto abelha no dicionário, o que resulta no controle correto. Depois disso o renderizador pode mover esse controle.

BringToFront() garante que a abelha apareça "em cima" das flores no formulário do campo e acima da imagem de fundo do formulário da colméia.

Adicione os formulários de colméia e de campo ao projeto

Agora você precisará de formulários para colocar abelhas. Então **comece com seu projeto pré-existente de simulador de colméia** e use "Add>>Existing Item..." para **acrescentar seu novo controle de usuário BeeControl**. Depois adicione mais dois formulários **clicando com o botão direito no projeto** usando o Navegador de Solução e escolhendo "Windows Form..." no menu Add. Se você nomear os arquivos HiveForm.cs e FieldForm.cs, o IDE automaticamente atribuirá suas propriedades Name para `HiveForm` e `FieldForm`. Como formulários são apenas objetos, esses dois serão somente mais duas classes.

Isto é um controle PictureBox com sua propriedade BackgroundImagem apontando para a imagem do exterior da colméia. Quando você carregar as imagens da colmeia no Designer de Recursos, elas aparecerão nas listas quando o símbolo "..." ao lado da propriedade referida for clicado.



Certifique-se de redimensionar ambos os formulários para que eles se pareçam com essas imagens.

Atribua à propriedade BackgroundImagem do formulário a imagem do interior da colméia.

O campo

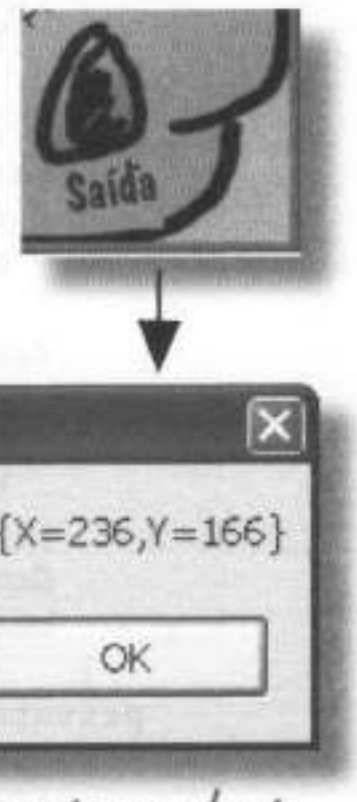
Você deverá ter as imagens do interior e do exterior da colméia - "Hive (inside).png" e "Hive (outside).png" - carregadas nos seus recursos. Depois adicione esses dois formulários. Atribua para a propriedade FormBorderStyle de cada um deles o valor FixedSingle (assim o usuário não poderá redimensioná-los), a propriedade ControlBox para false (para remover os botões de minimizar e maximizar), e StartPosition (posição inicial) para Manual (assim a propriedade Location poderá ser escrita).

Lembre-se, vá para a janela de propriedades, clique no ícone de raio para trazer a janela de eventos, procure a linha com MouseClick e clique duas vezes nele. O IDE adicionará o tratador de eventos para você.

Descubra suas posições

Você precisa descobrir onde a colméia está no seu formulário de campo. Usando a janela de propriedades, crie um tratador para o evento MouseClick e adicione o seguinte código:

```
private void Form1_MouseClick(object sender, MouseEventArgs e) {
  MessageBox.Show(e.Location.ToString());
}
```



Agora você pode executar o formulário, e clicar na posição da colméia. Isso mostrará a você um par de coordenadas para sua colméia no campo.

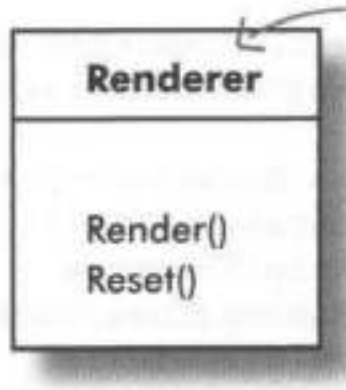
Adicione o mesmo tratador no formulário da colméia e execute-o. Então, clicando, consiga as coordenadas da saída, do berçário e da fábrica de mel. Usando todas essas posições, você pode atualizar o método ResetLocations() que escreveu na classe Hive no capítulo anterior.

```
private void ResetLocations()
{
  locations = new Dictionary<string, Point>();
  locations.Add("Entrance", new Point(600, 100));
  locations.Add("Nursery", new Point(95, 174));
  locations.Add("HoneyFactory", new Point(157, 98));
  locations.Add("Exit", new Point(194, 213));
}
```

Remova o tratador de evento de clique depois de terminar... você só precisa dele para conseguir as coordenadas em seu formulário.

Essas são as coordenadas que funcionaram conosco, mas se seus formulários forem um pouco maiores ou menores, as suas serão diferentes.

Desenvolva o Renderizador



Todos os campos no renderizador são privados porque nenhuma outra classe precisa alterar nenhuma propriedade. Ela é totalmente encapsulada. O mundo apenas chama Render() para desenhar nos formulários e Reset() para eliminar todos os controles, quando necessário.

Eis aqui a classe Renderer completa. O formulário principal chama o método Render() dessa classe logo depois de chamar World.Go(), para desenhar as abelhas e flores nos formulários. Você deve certificar-se de que o gráfico das flores (Flower.png) esteja carregado no projeto, exatamente como as imagens da abelha animada.

O renderizador mantém referências ao mundo e aos dois formulários onde desenha as abelhas.

```
public class Renderer {
  private World world;
  private HiveForm hiveForm;
  private FieldForm fieldForm;
  private Dictionary<Flower, PictureBox> flowerLookup =
    new Dictionary<Flower, PictureBox>();
  private List<Flower> deadFlowers = new List<Flower>();
  private Dictionary<Bee, BeeControl> beeLookup =
    new Dictionary<Bee, BeeControl>();
  private List<Bee> retiredBees = new List<Bee>();
```

O mundo usa objetos Bee e Flower para manter registro de cada abelha e flor existentes. Os formulários usam caixas de imagens para exibir cada flor e BeeControls para mostrar cada abelha. O renderizador usa estes dicionários para conectar cada abelha e flor com seus próprios controles respectivos.

Quando uma flor morre ou uma abelha aposenta-se, ele usa as listas deadFlowers (flores mortas) e retiredBees (abelhas aposentadas) para limpar os dicionários.

```
public Renderer(World world, HiveForm hiveForm, FieldForm fieldForm) {
  this.world = world;
  this.hiveForm = hiveForm;
  this.fieldForm = fieldForm;
}
public void Render() {
    DrawBees();
    DrawFlowers();
    RemoveRetiredBeesAndDeadFlowers();
}
public void Reset() {
  foreach (PictureBox flower in flowerLookup.Values) {
    fieldForm.Controls.Remove(flower);
    flower.Dispose();
  }
  foreach (BeeControl bee in beeLookup.Values) {
    hiveForm.Controls.Remove(bee);
    fieldForm.Controls.Remove(bee);
    bee.Dispose();
  }
  flowerLookup.Clear();
  beeLookup.Clear();
}
private void DrawFlowers() {
  foreach (Flower flower in world.Flowers)
    if (!flowerLookup.ContainsKey(flower)) {
      PictureBox flowerControl = new PictureBox() {
        Width = 45,
        Height = 55,
        Image = Properties.Resources.Flower,
        SizeMode = PictureBoxSizeMode.StretchImage,
        Location = flower.Location
      };
      flowerLookup.Add(flower, flowerControl);
      fieldForm.Controls.Add(flowerControl);
    }
  foreach (Flower flower in flowerLookup.Keys) {
    if (!world.Flowers.Contains(flower)) {
      PictureBox flowerControlToRemove = flowerLookup[flower];
      fieldForm.Controls.Remove(flowerControlToRemove);
      flowerControlToRemove.Dispose();
      deadFlowers.Add(flower);
} } }
private void DrawBees() {
  BeeControl beeControl;
  foreach (Bee bee in world.Bees) {
    beeControl = GetBeeControl(bee);
    if (bee.InsideHive) {
      if (fieldForm.Controls.Contains(beeControl))
        MoveBeeFromFieldToHive(beeControl);
    } else if (hiveForm.Controls.Contains(beeControl))
      MoveBeeFromHiveToField(beeControl, bee);
    beeControl.Location = bee.Location;
  }
  foreach (Bee bee in beeLookup.Keys) {
    if (!world.Bees.Contains(bee)) {
      beeControl = beeLookup[bee];
      if (fieldForm.Controls.Contains(beeControl))
        fieldForm.Controls.Remove(beeControl);
      if (hiveForm.Controls.Contains(beeControl))
        hiveForm.Controls.Remove(beeControl);
      beeControl.Dispose();
      retiredBees.Add(bee);
    }
  }
}
```

O timer no formulário principal que executa a animação chama o método Render(), ele atualiza as abelhas e flores e limpa seus dicionários.

Se o simulador for reiniciado, ele chama o método RemoveAllControls() (remover todos os controles) para limpar completamente os dois formulários. Ele acha todos os controles em cada um de seus dois dicionários e remove-os dos formulários, chamando Dispose() para cada um deles. Então ele limpa os dois dicionários.

Precisamos de dois laços foreach para desenhar as flores. O primeiro procura por flores novas e adiciona suas caixas de imagem. O segundo procura por flores mortas e remove as caixas.

O primeiro laço foreach usa o dicionário flowerLookup para checar cada flor e determinar se ela tem um controle no formulário. Se não, ele cria uma nova PictureBox usando um inicializador de objeto, adiciona essa caixa ao formulário e também ao dicionário.

DrawFlowers() usa a propriedade Location no objeto Flower para determinar a posição da caixa de imagem no formulário.

O segundo laço foreach procura por qualquer PictureBox no dicionário flowerLookup que não esteja mais no formulário e o remove.

Depois de remover a caixa de imagem, o método Dispose() é chamado. Então o objeto Flower é colocado em deadFlowers para ser eliminado posteriormente.

DrawBees() também usa dois laços foreach e segue os mesmos procedimentos básicos de DrawFlowers(). Como ele é um pouco mais complexo, dividimos seu comportamento em outros métodos para deixá-lo mais simples de entender.

Uma vez que o BeeControl é removido, precisamos chamar seu método Dispose() - o controle de usuário descartará o timer em nosso lugar.

DrawBees() checa se uma abelha está na colmeia, mas seu controle está no formulário do campo e vice-versa. Ele usa dois métodos extras para mover BeeControls entre os formulários.

O segundo laço foreach funciona como o de DrawFlowers(), exceto que ele precisa remover o BeeControl do formulário correto.

Você precisa certificar-se de incluir using System.Drawing e using System.Windows.Forms no início do arquivo de classe do renderizador.

```
private BeeControl GetBeeControl(Bee bee) {
    BeeControl beeControl;
    if (!beeLookup.ContainsKey(bee)) {
      beeControl = new BeeControl() { Width = 40, Height = 40 };
      beeLookup.Add(bee, beeControl);
      hiveForm.Controls.Add(beeControl);
      beeControl.BringToFront();
    }
    else
      beeControl = beeLookup[bee];
    return beeControl;
}
private void MoveBeeFromHiveToField(BeeControl beeControl, Bee bee) {
  hiveForm.Controls.Remove(beeControl);
  beeControl.Size = new Size(20, 20);
  fieldForm.Controls.Add(beeControl);
  beeControl.BringToFront();
}
private void MoveBeeFromFieldToHive(BeeControl beeControl) {
  fieldForm.Controls.Remove(beeControl);
  beeControl.Size = new Size(40, 40);
  hiveForm.Controls.Add(beeControl);
  beeControl.BringToFront();
}
private void RemoveRetiredBeesAndDeadFlowers() {
  foreach (Bee bee in retiredBees)
    beeLookup.Remove(bee);
  retiredBees.Clear();
  foreach (Flower flower in deadFlowers)
    flowerLookup.Remove(flower);
  deadFlowers.Clear();
}
```

GetBeeControl() procura uma abelha no dicionário beeLookup e retorna seu objeto. Se ele não está lá, um novo BeeControl de 40 x 40 é criado e adicionado ao formulário de colméia (pois é lá que as abelhas nascem).

Não se esqueça: "!" quer dizer NÃO.

MoveBeeFromHiveToField() pega um BeeControl específico, tira-o da coleção Controls do formulário da colméia e adiciona-o à coleção equivalente do formulário do campo.

Como as abelhas no formulário do campo são menores do que as no da colméia, o método precisa alterar a propriedade Size do BeeControl.

MoveBeeFromFieldToHive() movimenta um BeeControl de volta para o formulário da colméia. Ela deve ser aumentada.

Sempre que DrawBees() e DrawFlowers() percebem que uma abelha ou flor não existe mais no mundo, elas são adicionadas nas listas deadFlowers ou retiredBees para serem removidas no final do quadro.

Depois que todos os controles são movidos, o renderizador chama esse método para eliminar qualquer flor morta ou abelha aposentada dos dois dicionários.

Agora conecte o formulário principal aos seus novos dois formulários, HiveForm e FieldForm

É ótimo ter um renderizador, mas, até agora, não há nenhum formulário para se desenhar. Podemos resolver isso voltando à classe `Form` principal (provavelmente, chamada `Form1`) e fazendo algumas alterações de código:

```
public partial class Form1 : Form {
  HiveForm hiveForm = new HiveForm();
  FieldForm fieldForm = new FieldForm();
  Renderer renderer;
```

Quando o formulário principal carrega, ele cria uma instância de cada um dois outros dois. Eles são apenas objetos na pilha, no momento - não serão mostrados até que seus métodos Show() sejam chamados.

```
  // o restante dos campos
```

Mova o código para instanciar World para o método ResetSimulator().

```
  public Form1() {
    InitializeComponent();

    MoveChildForms();
    hiveForm.Show(this);
    fieldForm.Show(this);
    ResetSimulator();

    // o resto do codigo permanece o mesmo
  }
```

O formulário passa uma referência dele mesmo para Formulários.Show() para tornar-se o formulário pai.

O construtor do formulário principal move os dois formulários filhos para seus lugares e depois exibe-os. Então ele chama ResetSimulator(), que instancia Renderer.

Tudo mais que costumava ficar no construtor deve ser movido para o método ResetSimulator().

```
  private void MoveChildForms() {
    hiveForm.Location = new Point(Location.X + Width + 10, Location.Y);
    fieldForm.Location = new Point(Location.X,
      Location.Y + Math.Max(Height, hiveForm.Height) + 10);
  }
  public void RunFrame(object sender, EventArgs e) {
    framesRun++;
    world.Go(random);
    renderer.Render();
    // codigo anterior
  }
  private void Form1_Move(object sender, EventArgs e) {
    MoveChildForms();
  }
```

Este código move os dois formulários de forma que o de colméia fique ao lado do de estatísticas e o do campo fique logo abaixo deles.

Adicionar essa linha em RunFrame faz com que o simulador atualize os gráficos depois de cada vez que o método Go() do mundo for chamado.

O evento Move é disparado a cada vez que o formulário principal é movido. Chamar MoveChildForms() garante que os formulários filhos sempre se movam juntamente com o principal.

Use o botão Events na janela de propriedades para adicionar o tratador de evento à Move.

Certifique-se de atribuir a propriedade StartPosition dos formulários de campo e colméia para Manual, senão MoveChildForms() não funcionará.

É aqui que **criamos novas instâncias** das classes World e Renderer, o que reinicia o simulador.

```
  private void ResetSimulator() {
    framesRun = 0;
    world = new World(new Bee.BeeMessage(SendMessage));
    renderer = new Renderer(world, hiveForm, fieldForm);
  }
  private void reset_Click(object sender, EventArgs e) {
    renderer.Reset();
    ResetSimulator();
    if (!timer1.Enabled)
      toolStrip1.Items[0].Text = "Start simulation";
  }
  private void openToolStripButton_Click(object sender, EventArgs e) {
    // O restante do codigo neste botao permanece exatamente o mesmo
    renderer.Reset();
    renderer = new Renderer(world, hiveForm, fieldForm);
  }
}
```

O botão Reiniciar precisa chamar Reset() para eliminar todos os BeeControls e caixas de imagens de flores e então reiniciar o simulador.

Finalmente, é preciso adicionar código no botão Abrir, na caixa de ferramentas, para usar o método Reset() para remover as abelhas e flores das coleções Controls dos dois formulários e criar um novo renderizador usando um mundo recém-carregado.

não existem Perguntas Idiotas

P: Eu vi que vocês exibem o formulário usando o método `Show()`, mas não entendi como passar `this` como parâmetro.

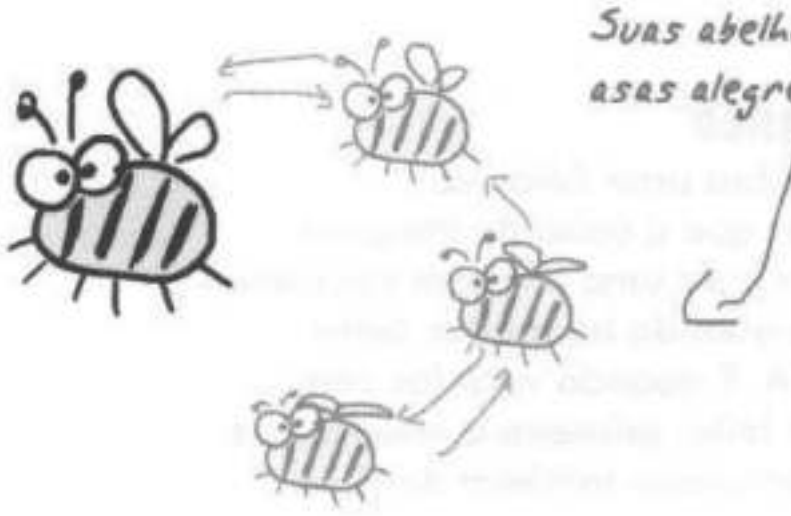
R: Tudo isso relaciona-se com a idéia de que um formulário é apenas mais uma classe. Quando você exibe um deles você apenas instanciou essa classe e chamou seu método `Show()`. Há uma versão sobrecarregada desse método que recebe um único parâmetro. Ela informa uma janela que será a ancestral de outra. Quando um formulário é pai de outro, isso faz com que o Windows crie um relacionamento especial entre eles - por exemplo, quando você minimiza a janela pai, isso automaticamente minimiza todas as janelas filhas daquele formulário.

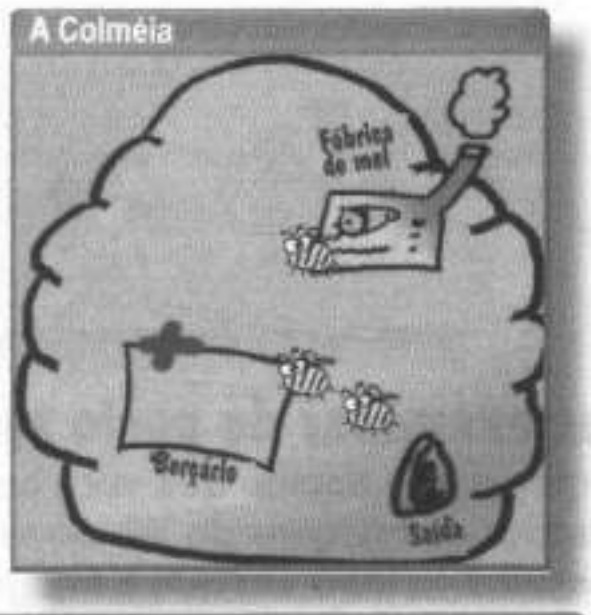
P: Podemos alterar os controles nativos e bagunçar o código deles?

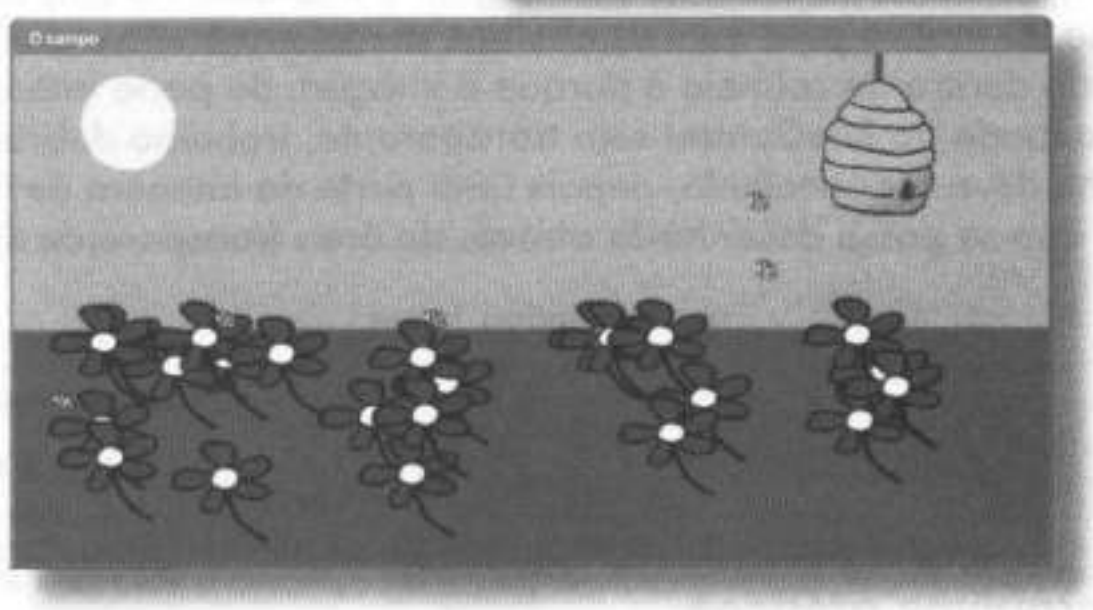
R: Não, na realidade não é possível acessar o código dos controles nativos do Visual Studio. No entanto, cada um deles é uma classe em que se pode empregar herança, exatamente como fizemos com `PictureBox` para criar `BeeControl`. Para adicionar ou alterar comportamentos em qualquer um dos controles, você pode adicionar seus próprios métodos e propriedades que manipulem os da classe base.

Aquecendo os motores... ahn... as asas

Compile todo o seu código, elimine qualquer erro e execute o simulador.







É bonito, mas alguma coisa não está muito certa...

Olhe de perto para as abelhas zumbindo em torno da colméia e para as flores e notará alguns problemas com a forma como elas estão sendo exibidas. Lembra como você atribuiu a propriedade `BackColor` de cada `BeeControl` para `Color.Transparent`? Infelizmente, isso não é o suficiente para fazer com que o simulador evite alguns problemas que são, na realidade, bem típicos de programas gráficos.

1. **Há alguns problemas de performance sérios**
   Você notou como o simulador inteiro fica mais lento quando todas as abelhas estão dentro da colméia? Se não, tente adicionar mais abelhas aumentando as constantes na classe `World`. Preste atenção na taxa de quadros - acrescente mais abelhas e ela começará a cair significativamente.

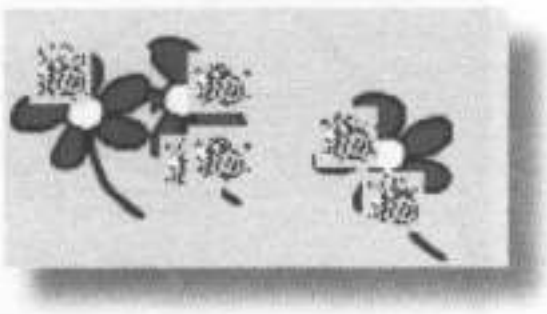
2. **O fundo das flores não é transparente de verdade**
   E há outro problema, totalmente **à parte**. Quando salvamos o gráfico para as flores, deixamos seus fundos transparentes. Mas, embora isso tenha garantido a cada flor permitir à cor de fundo do formulário aparecer, não temos um resultado tão bom quando as flores se sobrepõem.

Quando você atribui Transparent para a cor de fundo de uma caixa de imagem, ela pinta qualquer pixel transparente na imagem com a cor de fundo do **formulário**... o que nem sempre é a coisa certa a se fazer.

Quando uma caixa de texto passa por cima de outra, o C# pinta os pixels transparentes com a cor de fundo do formulário, não com as cores do controle que fica embaixo, fazendo com que apareçam "recortes" retangulares sempre que duas flores se sobrepõem.

3 **O fundo das abelhas também não é transparente**

Aparentemente `Color.Transparent` realmente tem algumas limitações. Quando as abelhas estão sobrevoando as flores, o mesmo problema de "recorte" aparece. A transparência funciona um pouco melhor com o formulário da colméia, onde a imagem de fundo efetivamente aparece nas áreas transparentes do gráfico das abelhas. Mas quando as abelhas se sobrepõem, o mesmo erro ocorre. E se você observar de perto as abelhas se movendo pela colméia, verá ocasionalmente algumas falhas em que as imagens das abelhas aparecem distorcidas quando elas se movem.



Vamos examinar de perto esses problemas de performance

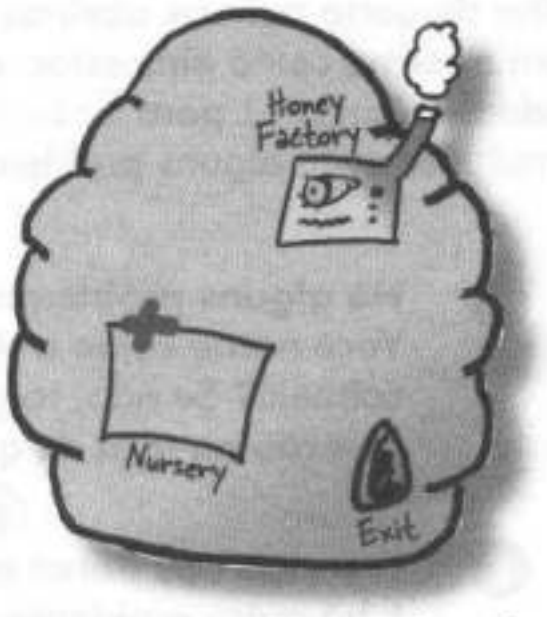
Cada imagem de abelha que você baixou é grande. Realmente grande. Abra uma delas no Visualizador de Imagens do Windows e veja por si próprio. Isso quer dizer que a caixa de imagens deve encolhê-la toda vez que a imagem é alterada. Porém, alterar a escala de uma imagem consome tempo. O motivo pelo qual as abelhas se movem muito mais lentamente quando há muitas delas voando dentro da colméia é porque a imagem da parte interna é IMENSA. E quando você faz com que o fundo do BeeControl seja transparente, trabalho dobrado deve ser feito: primeiro a imagem da abelha deve ser encolhida, depois uma parte da imagem de fundo do formulário também deve ser para que se possa desenhá-lo através da área transparente da abelha.

Bee animation 1.png

A imagem da abelha é realmente grande e a caixa de imagens precisa de tempo para encolhê-la toda vez que mostra um novo quadro da animação.

Os arquivos gráficos para as abelhas são realmente GRANDES. A caixa de imagem precisa alterar a escala da imagem para o tamanho certo cada vez que um novo quadro da animação é mostrado. Isso toma muito tempo...

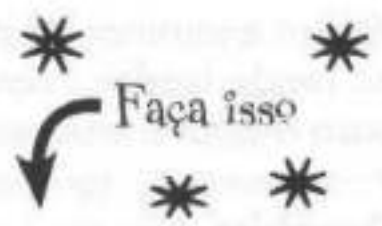
Hive (Inside).png



A imagem da parte interna da colméia é imensa. Cada vez que uma abelha voa na frente dela, sua caixa de imagem precisa diminuir seu tamanho para corresponder ao do controle. Ela precisa fazer isso para mostrar a parte da figura que aparece através de qualquer parte transparente da imagem da abelha.

...assim tudo a fazer para melhorar a performance do simulador é encolher todas as imagens antes de tentar mostrá-las.

Tudo a fazer para acelerar a performance dos gráficos é adicionar um método ao renderizador para alterar a escala de qualquer imagem para algo diferente. Então podemos **redimensionar cada figura uma única vez quando ela for carregada** e usar apenas a versão alterada da imagem no controle da abelha e no fundo do formulário da colméia.



1 **Adicione o método ResizeImage ao renderizador**
Todas as figuras no seu projeto (como Properties.Resources.Flower) são armazenadas como objetos Bitmap (mapa de bits). Eis um método estático que os redimensiona - adicione-o à classe Renderer:

```
public static Bitmap ResizeImage(Bitmap picture, int width, int height) {
  Bitmap resizedPicture = new Bitmap(width, height);
  using (Graphics graphics = Graphics.FromImage(resizedPicture)) {
    graphics.DrawImage(picture, 0, 0, width, height);
  }
  return resizedPicture;
}
```

Vamos examinar mais detalhadamente o que o objeto Graphics é e como esse método funciona nas próximas páginas.

2 **Acrescente o seguinte método ResizeCells no seu BeeControl**
Seu BeeControl pode armazenar seus próprios objetos Bitmap - uma matriz de quatro deles. Eis um controle que povoa essa matriz, redimensionando cada item para que ele tenha exatamente o tamanho do controle:

```
private Bitmap[] cells = new Bitmap[4];
private void ResizeCells() {
  cells[0] = Renderer.ResizeImage(Properties.Resources.Bee_animation_1, Width, Height);
  cells[1] = Renderer.ResizeImage(Properties.Resources.Bee_animation_2, Width, Height);
  cells[2] = Renderer.ResizeImage(Properties.Resources.Bee_animation_3, Width, Height);
  cells[3] = Renderer.ResizeImage(Properties.Resources.Bee_animation_4, Width, Height);
}
```

3 **Modifique o comando switch para que ele utilize a matriz de células, não os recursos**
O tratador do evento Tick do BeeControl tem um comando switch que configura BackgroundImage:

```
BackgroundImage = Properties.Resources.Bee_animation_1;
```

Substitua `Properties.Resources.Bee_animation_1` **por** `cells[0]`. Agora faça substituições no restante das linhas **case**, para que o case 2 use `cells[1]`, case 3 use `cells[2]`, case 4 use `cells[3]`, case 5 use `cells[2]` e a cláusula default use `cells[1]`. Dessa forma, apenas as imagens redimensionadas serão exibidas.

4 **Inclua chamadas para ResizeCells() no BeeControl**
Você deverá inserir duas chamadas para o novo método ResizeCells(). Primeiro, adicione uma no final do construtor - e **mude a propriedade BackgroundImageLayout para** `None`. Então volte ao designer no IDE clicando duas vezes no BeeControl na janela de propriedades. Vá para a página de Eventos (clicando no ícone de raio), procure por Resize, e clique duas vezes nele. **Adicione um tratador de evento de redimensionamento**. Faça com que esse novo tratador de Resize chame ResizeCells() também - dessa forma as figuras da animação serão redimensionadas sempre que o formulário for.

5 **Configure manualmente a imagem de fundo do formulário**
Vá até a janela de propriedades e configure como (none) a imagem de fundo do formulário. Então vá para o construtor e atribua para essa imagem uma que tenha sido apropriadamente redimensionada.

```
public partial class HiveForm : Form {
  public HiveForm() {
    InitializeComponent();
    BackgroundImage = Renderer.ResizeImage(
      Properties.Resources.Hive__inside_,
      ClientRectangle.Width, ClientRectangle.Height);
  }
}
```

Seu formulário tem uma propriedade ClientRectangle com um Rectangle que tem as dimensões da área de exibição.

Agora execute o simulador - ele estará muito mais rápido!

Você redimensionou seus Bitmaps usando um objeto Graphics

Vamos examinar de perto o método `ResizeImage()` que você adicionou ao renderizador. Primeiro, ele cria um novo objeto `Bitmap` com o tamanho para o qual a imagem será redimensionada. Então ele usa `Graphics.FromImage()` (gráficos, a partir da imagem) para **criar um novo objeto Graphics**. Ele usa o método `DrawImage()` desse novo objeto para desenhar a figura no `Bitmap`. Note como você deve passar `width` e `height` como parâmetros para `DrawImage()` - é assim que você diz a ele para alterar a escala da imagem para a do novo tamanho. Finalmente o novo `Bitmap` criado é retornado, assim ele pode ser usado como imagem de fundo do formulário ou uma das quatro células de animação.

Formulários e controles têm um método CreateGraphics() que retorna um novo objeto Graphics. Você verá muito mais a respeito disso em breve.

Você passa uma imagem para o método, junto com novas largura e altura que servirão para redimensionamento.

```
public static Bitmap ResizeImage(Bitmap picture, int width, int height) {
  Bitmap resizedPicture = new Bitmap(width, height);
  using (Graphics graphics = Graphics.FromImage(resizedPicture)) {
    graphics.DrawImage(picture, 0, 0, width, height);
  }
  return resizedPicture;
}
```

O método FromImage() retorna um novo objeto Graphics. Isso permite desenhar sobre essa imagem. Pare um minuto e use o IntelliSense do IDE para examinar os métodos na class Graphics. Quando você chama DrawImage(), ela copia a imagem dentro do bitmap resizedPicture na posição (0,0) e o redimensiona usando os parâmetros width e height.

Vamos ver redimensionamento de imagens em ação

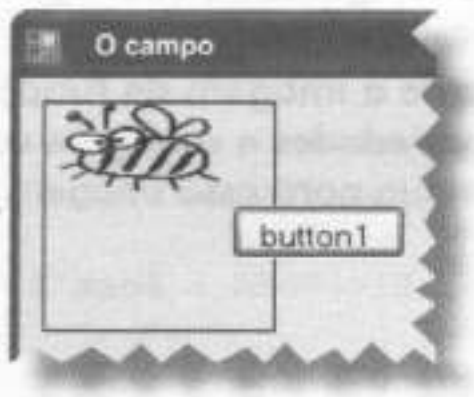
Arraste um botão para o formulário de campo e adicione o código seguinte. Ele cria um novo controle PictureBox com 100 x 100 pixels e configura sua borda para uma linha negra, para que você possa ver seu tamanho. Então ele usa `ResizeImage()` para compactar a imagem da abelha para 80 x 40 pixels e atribui essa nova figura para a propriedade `Image`. Quando a caixa de imagem é adicionada ao formulário, a abelha é exibida.

Faça isso apenas de maneira temporária. Apague o botão e o código quando tiver terminado.

```
private void button1_Click(object sender, EventArgs e)
{
  PictureBox beePicture = new PictureBox();
  beePicture.Location = new Point(10, 10);
  beePicture.Size = new Size(100, 100);
  beePicture.BorderStyle = BorderStyle.FixedSingle;
  beePicture.Image = ResizeImage(
    Properties.Resources.Bee_animation_1, 80, 40);
  Controls.Add(beePicture);
}
```

O método ResizeImage() cria um objeto Graphics para desenhar num objeto Bitmap invisível. Ele retorna esse bitmap para que ele possa ser exibido num formulário ou numa caixa de imagem.

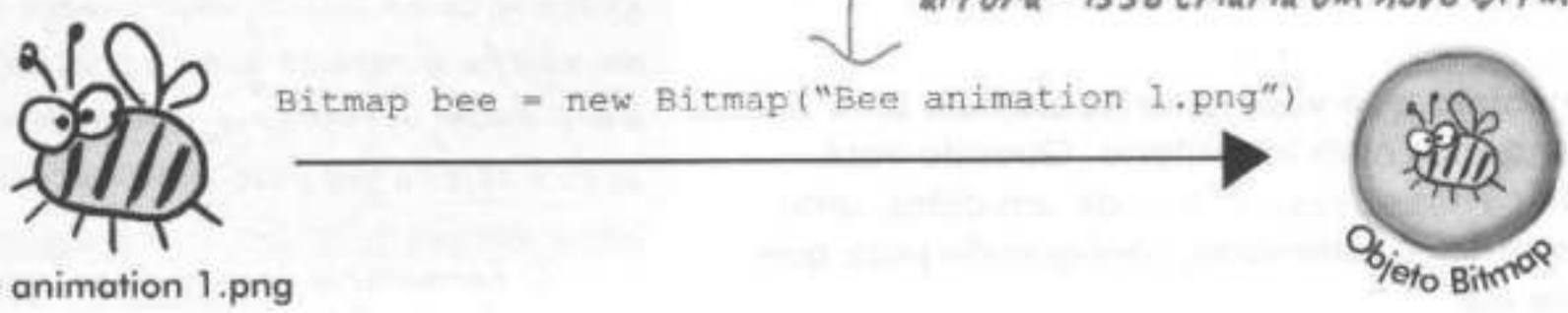
Você pode ver o redimensionamento de imagens em ação - a imagem compactada da abelha é muito menor do que a caixa de imagem. ResizeImage() foi quem a compactou.



Os seus recursos que são imagens ficam armazenados em objetos Bitmap

Quando você importa arquivos gráficos para os recursos de seu projeto, o que acontece com eles? Você já sabe que pode acessá-los usando `Properties.Resources`. Mas o que, exatamente, seu programa está fazendo com as imagens, uma vez que elas são importadas?
O .NET transforma suas imagens em novos objetos **Bitmap**.

A classe Bitmap tem muitos construtores sobrecarregados. Este aqui carrega um arquivo do disco. Você também pode passar somente inteiros para largura e altura - isso criaria um novo Bitmap sem figura.



Então cada bitmap é desenhado na tela

Depois que as imagens estão em Bitmap , o seu formulário é desenhado na t ela, como a chamada aseguir:

Essa chamada obtém um objeto Graphics para desenhar no formulário. Empregamos um comando using para certificar-nos de que o objeto Graphics seja descartado.

```
using (Graphics g = CreateGraphics()) {
  g.DrawImage(myBitmap, 30, 30, 150, 150);
}
```

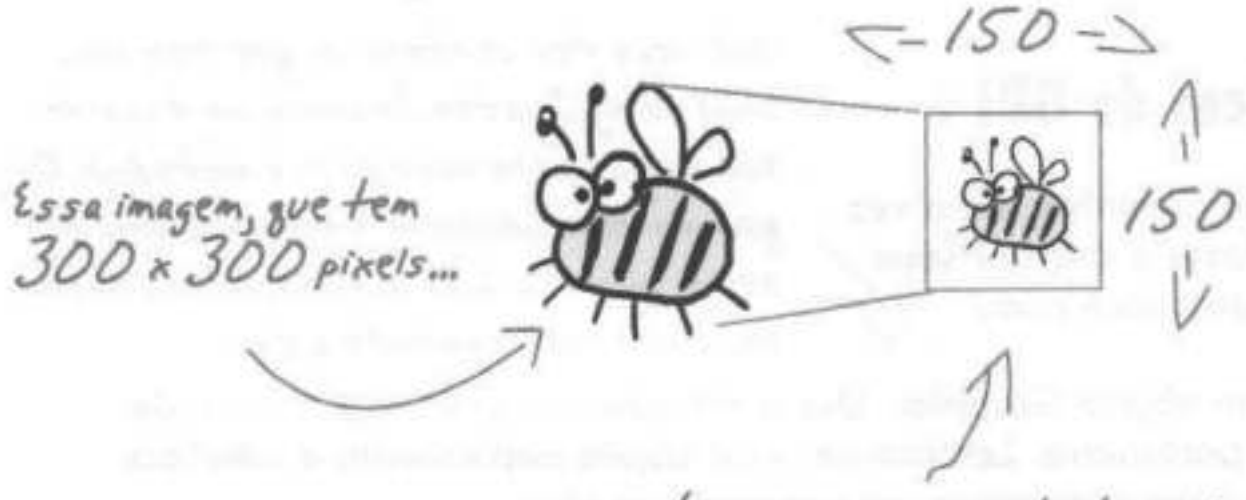
DrawImage() recebe um Bitmap, a imagem a ser desenhada...

...uma coordenada X,Y inicial...

...e um tamanho, 150 x 150 pixels.

Quanto maiores eles são...

Você viu aqueles dois últimos parâmetros de `DrawImage()`? E o que aconteceria se a imagem no `Bitmap` tivesse 175 por 175 pixels? A biblioteca gráfica teria que redimensionar a imagem para se encaixar nesses 150 por 150. E se o `Bitmap` contivesse uma figura de 1500 por 2025? Aí alterar a escala fica mais complicado...



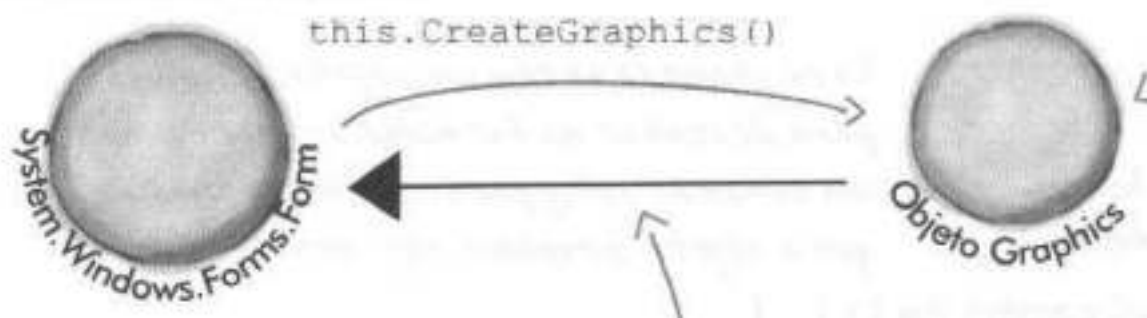
Redimensionar imagens usa muito poder de processamento! Se você faz isso uma única vez, não é tão sério. Mas se você faz isso A CADA QUADRO, seu programa ficará mais lento. Nós fornecemos imagens REALMENTE GRANDES para as abelhas e a colméia. Quando o renderizador as movimenta por aí (especialmente na frente da imagem do interior da colméia), ele deve redimensioná-las de novo e de novo. E era isso que causou os problemas de performance!

Use System.Drawing para ASSUMIR O CONTROLE dos gráficos você mesmo

O objeto Graphics é parte do namespace System.Drawing. O Framework .NET vem com algumas ferramentas gráficas bem poderosas que vão bem além do controle de caixa de imagem simples da caixa de ferramentas. Você pode desenhar formas geométricas, usar fontes e fazer todo tipo de gráficos complexos... e tudo isso começa com um objeto Graphics. Sempre que você quiser adicionar ou modificar qualquer gráfico ou imagem de algum objeto, vai criar um objeto Graphics que é **associado ao objeto que quer desenhar**, e então usará seus métodos para desenhar nesse alvo.

SystemDrawing
Os métodos gráficos no namespace SystemDrawing, às vezes, são chamados de GDI+, que é sigla para Graphics Device Interface (interface de dispositivo gráfico). Quando você desenha com a GDI+, você começa com um objeto Graphics que é associado a um Bitmap, formulário, controle ou outro objeto que você queira desenhar.

1. **Comece com o objeto que você quer desenhar**
Por exemplo, pensemos num formulário. Quando você chama o método `CreateGraphics()` de um deles, uma instância de **Graphics** é retornada, configurada para que se desenhe sobre ele.



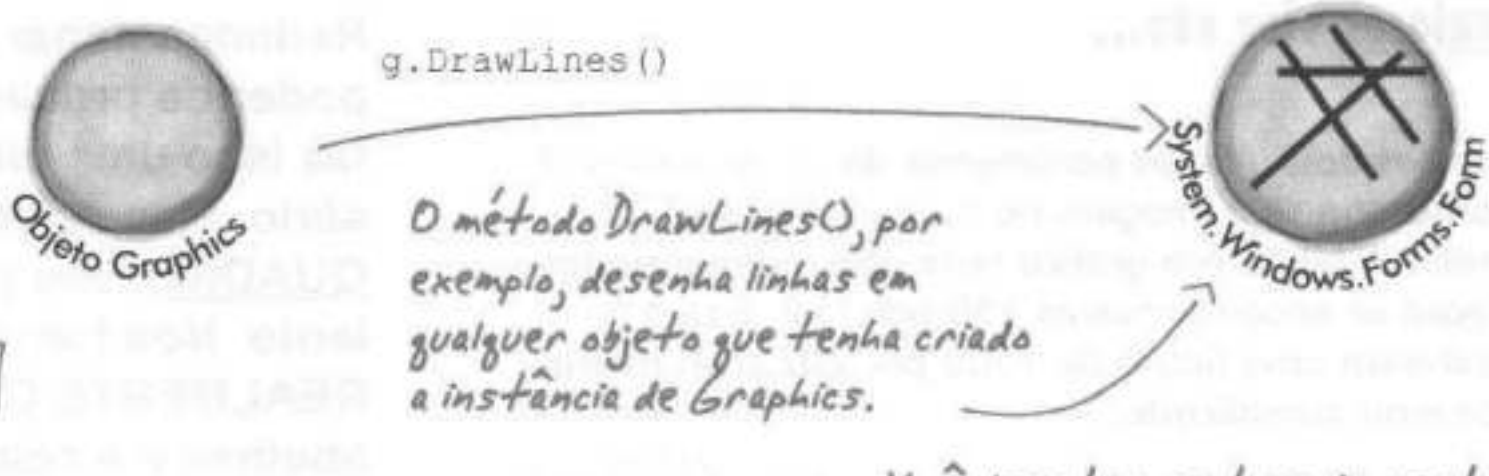
O formulário pode chamar seu próprio método CreateGraphics() e outros objetos também. De qualquer forma, uma referência a um objeto Graphics cujos métodos serão usados para desenhar é retornada.

Chamadas nessa instância de Graphics afetarão o formulário que o criou.

Você não desenha no objeto Graphics em si. Você utiliza-o apenas para desenhar em outros objetos.

2. **Use os métodos do objeto Graphics para desenhar no outro objeto**
Cada objeto `Graphics` tem métodos que permitem desenhar no objeto que o criou. Quando você chama esses métodos para desenhar linhas, círculos, retângulos, textos e imagens, eles aparecerão no formulário.

Mesmo que você esteja chamando métodos nesse objeto Graphics, os gráficos em si aparecerão no objeto que o criou.



O método DrawLines(), por exemplo, desenha linhas em qualquer objeto que tenha criado a instância de Graphics.

Um tour de 30 segundos pelos gráficos da GDI+

Há todo tipo de figuras e imagens que você pode desenhar, uma vez que tenha criado um objeto `Graphics`. Tudo a fazer é chamar seus métodos; ele desenhará diretamente no objeto que você criou.

Você deve ter certeza de que tem uma linha using SystemDrawing no topo de sua classe para usar esses métodos. Ou, quando você adicionar um formulário no seu projeto, o IDE acrescentará essa linha automaticamente a ele.

1. O primeiro passo é sempre conseguir um objeto Graphics. Use o método `CreateGraphics()` do formulário, ou um deles passado como parâmetro. Lembre-se, este objeto implementa a interface `IDisposable()`, logo se você criar um novo empregue um comando `using`:

```
using (Graphics g = this.CreateGraphics()) {
```

Lembre-se, isso desenha no objeto que criou essa instância.

2. Se você quer desenhar uma linha, chame `DrawLine()` com um ponto de início e fim, cada um representado por coordenadas X e Y:

```
g.DrawLine(Pens.Blue, 30, 10, 100, 45);
```

A coordenada inicial...

...e a coordenada final.

ou você pode fazer isso usando um par de pontos:

```
g.DrawLine(Pens.Blue, new Point(30, 45), new Point(100, 10));
```

3. Eis código que desenha um retângulo preenchido com cor de granito e então uma borda azul celeste. Ela usa um `retângulo` para definir suas dimensões - nesse caso, o canto superior esquerdo está em (150, 15) e terá 140 pixels de largura e 90 de altura.

```
g.FillRectangle(Brushes.SlateGray, new Rectangle(150, 15, 140, 90));
g.DrawRectangle(Pens.SkyBlue, new Rectangle(150, 15, 140, 90));
```

Há muitas cores para usar - basta digitar "Color", "Pens" ou "Brushes" seguido de um ponto e a janela do IntelliSense as mostrará.

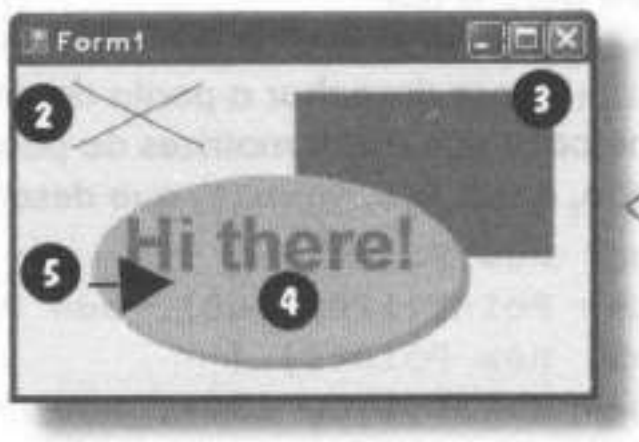
4. Você pode desenhar uma elipse ou um círculo usando os métodos `DrawCircle()` ou `FillCircle()`, que também usam um `retângulo` para especificar o tamanho da forma geométrica. O código seguinte desenha duas elipses ligeiramente deslocadas uma da outra para criar um efeito de sombreamento:

```
g.FillEllipse(Brushes.DarkGray, new Rectangle(45, 65, 200, 100));
g.FillEllipse(Brushes.Silver, new Rectangle(40, 60, 200, 100));
```

5. Use o método `DrawString()` para desenhar texto com qualquer fonte e cor. Para isso, você deverá criar um objeto Font, que implementa `IDisposable`. Então empregue um comando `using`:

```
using (Font arial24Bold = new Font("Arial", 24, FontStyle.Bold)) {
  g.DrawString("Hi there!", arial24Bold, Brushes.Red, 50, 75);
}
```

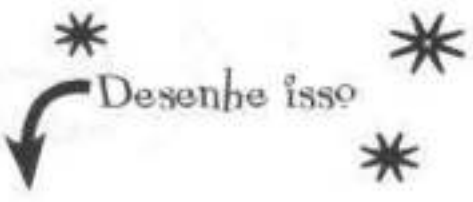
Se os comandos acima forem executados na ordem, este será o resultado final no formulário. Cada um dos comandos acima corresponde aos números mostrados. O canto superior esquerdo é a coordenada (0,0).



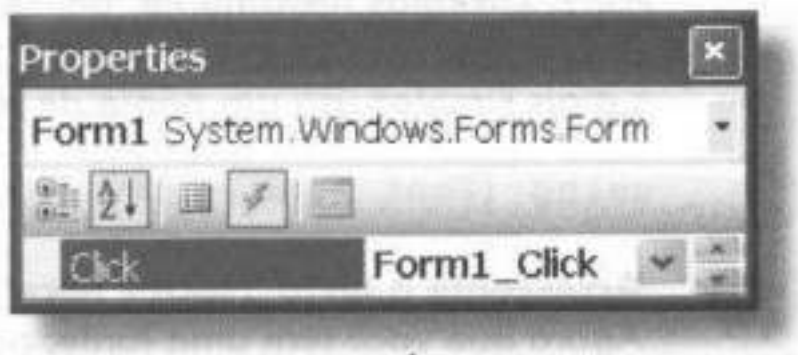
Não aparece passo 1 nessa imagem, já que ele foi o de criação do objeto Graphics em si.

Use gráficos para desenhar uma figura num formulário

Vamos criar um novo aplicativo Windows que desenha uma figura num formulário **quando você clica nele**.



1. **Comece adicionando um evento Click ao formulário**
Vá até a **página de eventos na janela de propriedades** (clicando no ícone de raio), depois até o evento Click e clique duas vezes nele.
Inicie o tratador de evento com uma linha `using` que cria um objeto Graphics. Quando você trabalha com a GDI+, você usa muitos objetos que implementam `IDisposable`. Se você não descartá-los, eles lentamente roubarão recursos do seu computador até que o programa feche. Então no final você acabará utilizando muitos comandos `using`:




```
using (Graphics g = CreateGraphics()) {
```

Aqui está a primeira linha do seu método de tratamento de evento Form1_Click(). Mostraremos a você todas as linhas dele - junte todas para desenhar a figura.

2. **Preste atenção na ordem em que você desenha coisas no formulário**
Como queremos um fundo azul celeste para essa figura, você desenhará um grande retângulo azul primeiro - depois qualquer outra coisa será desenhada **sobre dele**. Você vai aproveitar-se de uma das propriedades do formulário chamada `ClientRectangle`. Ela é um `Rectangle` que define os limites da área de desenho de um formulário. `Retângulos` são muito úteis - você pode criar um `novo` especificando um `Point` para seu canto superior esquerdo e sua largura e altura. Uma vez feito isso, ele automaticamente calculará suas propriedades `Top`, `Left`, `Right` e `Bottom`. E ele tem **métodos úteis como** `Contains()`, **que retornará verdadeiro se um determinado ponto estiver dentro dele**.

```
g.FillRectangle(Brushes.SkyBlue, ClientRectangle);
```

Isso será muito útil mais tarde no livro! O que você pensa que pode ser feito com Contains()?

3 **Desenhe a abelha e a flor**
Você já sabe como o método `DrawImage()` funciona:

```
g.DrawImage(Properties.Resources.Bee_animation_1, 50, 20, 75, 75);
g.DrawImage(Properties.Resources.Flower, 10, 130, 100, 150);
```

Canetas são para desenhar linhas, que, por sua vez, têm uma espessura. Para desenhar uma figura geométrica preenchida ou algum texto, você deverá ter um pincel.

4 **Adiciona uma caneta com a qual possa desenhar**
Cada vez que você desenha uma linha, usa um objeto Pen (caneta) que determina sua cor e espessura. Há uma classe nativa `Pens` que fornece muitas canetas (`Pens.Red` é uma caneta vermelha fina, por exemplo). Mas você pode criar suas próprias canetas usando o construtor da classe Pen, que recebe um objeto `Brush` (pincel) e uma espessura (informada em ponto flutuante, logo, certifique-se de acrescentar um `"F"` no final). Pincéis são o meio pelo qual você desenhará gráficos sólidos (como retângulos e elipses preenchidos). Há uma classe `Brushes` que fornece pincéis em várias cores.

```
using (Pen thickBlackPen = new Pen(Brushes.Black, 3.0F)) {
```

Isso vai dentro do comando using interno que cria a caneta.

5 **Adicione uma flecha que aponte para a flor**
Há alguns métodos gráficos que usam uma matriz de Points e ligam-nos usando uma série de curvas. Usaremos o método `DraweLines()` para desenhar a ponta de uma flecha e `DrawCurve()` para desenhar sua base. Há outros métodos que usam matrizes de pontos também (como `DrawPolygon()`, que desenha um polígono fechado, e `FillPolygon()`, que desenha um preenchido).

```
g.DrawLines(thickBlackPen, new Point[] {
  new Point(130, 110), new Point(120, 160), new Point(155, 163)});
g.DrawCurve(thickBlackPen, new Point[] {
  new Point(120, 160), new Point(175, 120), new Point(215, 70) });
}
```

Eis onde o bloco using termina - não precisamos mais de thickBlackPen, então descartamo-nas.

Quando você passa uma matriz de pontos para DrawCurve(), ela desenha uma curva suave que os conecta na ordem informada.

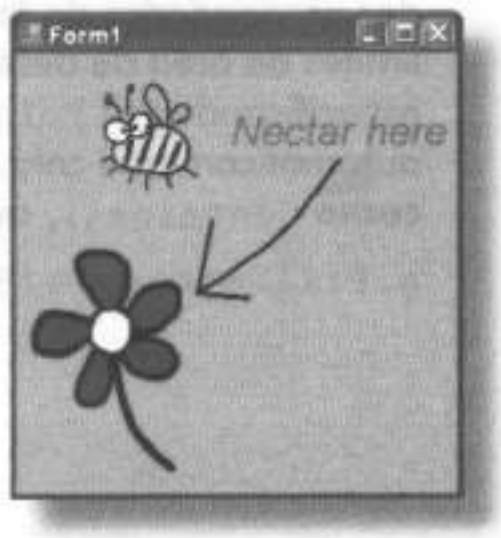
6 **Inclua uma fonte para desenhar o texto**
Quando você for desenhar texto, a primeira coisa a fazer é criar um objeto `Font`. Dizemos novamente, utilize um comando using porque `Font` implementa `IDisposable`. Criar uma fonte é simples. Há muitos construtores sobrecarregados - o mais simples usa um nome de fonte, um tamanho e uma enumeração `FontStyle`.

```
using (Font font = new Font("Arial", 16, FontStyle.Italic)) {
```

7 **Acrescente o texto "Nectar here"**
Agora que você tem uma fonte, pode descobrir onde desenhar a string determinando o tamanho que ela terá quando for desenhada. O método `MeasureString()` (mede a seqüência de caracteres) retorna um `SizeF` que define seu tamanho (`SizeF` é apenas uma versão em ponto `flutuante` de `Size` - tamanho - e ambos apenas definem uma largura e uma altura). Uma vez que sabemos onde a flecha termina, usaremos essa medida de string para posicionar seu centro logo acima da flecha.

```
    SizeF size = g.MeasureString("Nectar here", font);
    g.DrawString("Nectar here", font, Brushes.Red, new Point(
      215 - (int)size.Width / 2, 70 - (int)size.Height));
  }
}
```

Certifique-se de ter fechado ambos os blocos using.

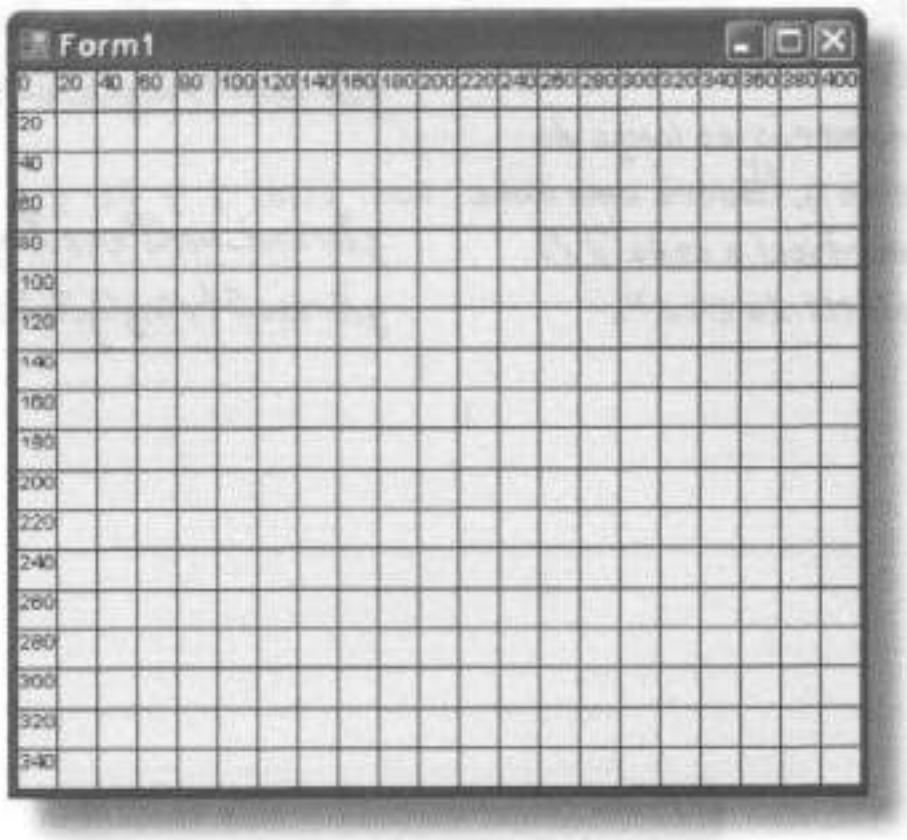


Você pode criar um retângulo informando um ponto e um Size (ou largura e altura). Uma vez que você tenha seu retângulo, pode determinar seus limites e checar com o seu método Contains() se ele contém um Point.

Aponte seu lápis

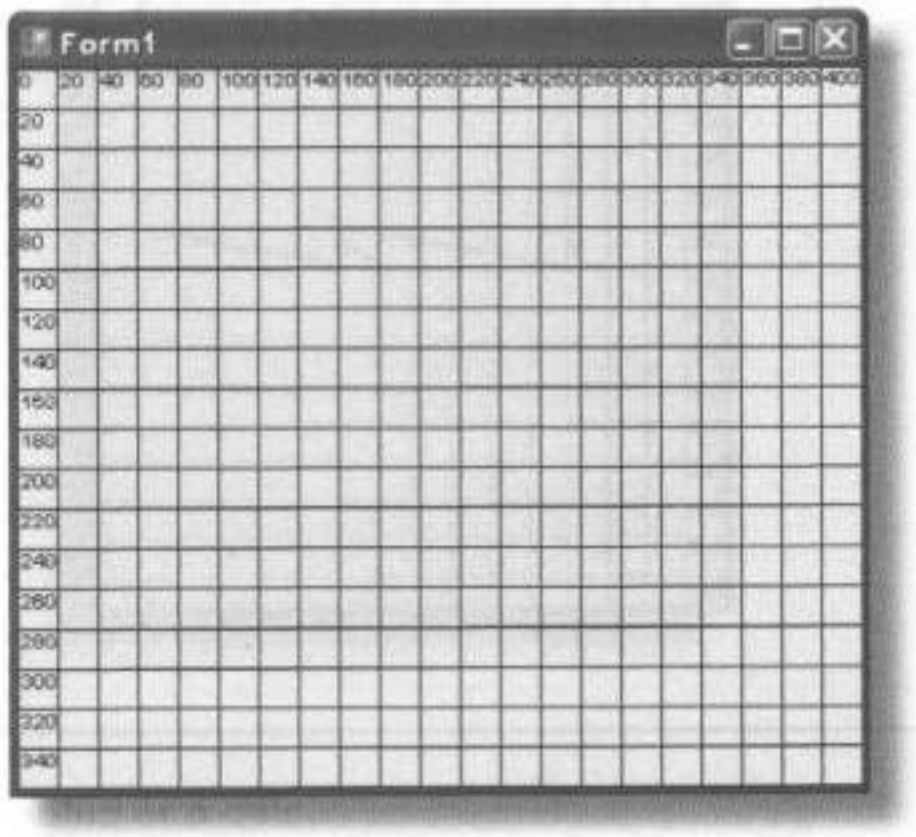
1. A maior parte do seu trabalho com Graphics implicará em pensar em seus formulários como uma grade de coordenadas X e Y. Eis aqui o código para construir a grade mostrada abaixo. Sua tarefa é preencher as partes faltantes.

```
using (Graphics g = this.CreateGraphics())
using (Font f = new Font("Arial", 6, FontStyle.Regular)) {
  for (int x = 0; x < this.Width; x += 20) {
  ..........
  } ..........
  for (int y = 0; y < this.Height; y += 20) {
  ..........
  }
} ..........
```



2. Você consegue imaginar o que acontece quando você executar o código abaixo? Desenhe o resultado no formulário, usando a grade que você acabou de desenhar para localizar pontos específicos.

```
using (Pen pen =
       new Pen(Brushes.Black, 3.0F)) {
  g.DrawCurve(pen, new Point[] {
    new Point(80, 60),
    new Point(200,40),
    new Point(180, 60),
    new Point(300,40),
  });
  g.DrawCurve(pen, new Point[] {
    new Point(300,180), new Point(180, 200),
    new Point(200,180), new Point(80, 200),
  });
  g.DrawLine(pen, 300, 40, 300, 180);
  g.DrawLine(pen, 80, 60, 80, 200);
  g.DrawEllipse(pen, 40, 40, 20, 20);
  g.DrawRectangle(pen, 40, 60, 20, 300);
  g.DrawLine(pen, 60, 60, 80, 60);
  g.DrawLine(pen, 60, 200, 80, 200);
}
```



3. Aqui está mais algum código gráfico, que lida com figuras irregulares. Determine o que será desenhado usando a grade dada abaixo.

FillPolygon(), DrawLines() e outros poucos métodos gráficos possuem construtores que aceitam matrizes de Points que definem os vértices de uma série de linhas conectadas.

```
g.FillPolygon(Brushes.Black, new Point[] {
    new Point(60,40), new Point(140,80), new Point(200,40),
    new Point(300,80), new Point(380,60), new Point(340,140),
    new Point(320,180), new Point(380,240), new Point(320,300),
    new Point(340,340), new Point(240,320), new Point(180,340),
    new Point(20,320), new Point(60, 280), new Point(100, 240),
    new Point(40, 220), new Point(80,160),
  });
using (Font big = new Font("Times New Roman", 24, FontStyle.Italic)) {
  g.DrawString("Pow!", big, Brushes.White, new Point(80, 80));
  g.DrawString("Pow!", big, Brushes.White, new Point(120, 120));
  g.DrawString("Pow!", big, Brushes.White, new Point(160, 160));
  g.DrawString("Pow!", big, Brushes.White, new Point(200, 200));
  g.DrawString("Pow!", big, Brushes.White, new Point(240, 240));
}
```

Aponte seu lápis
Solução

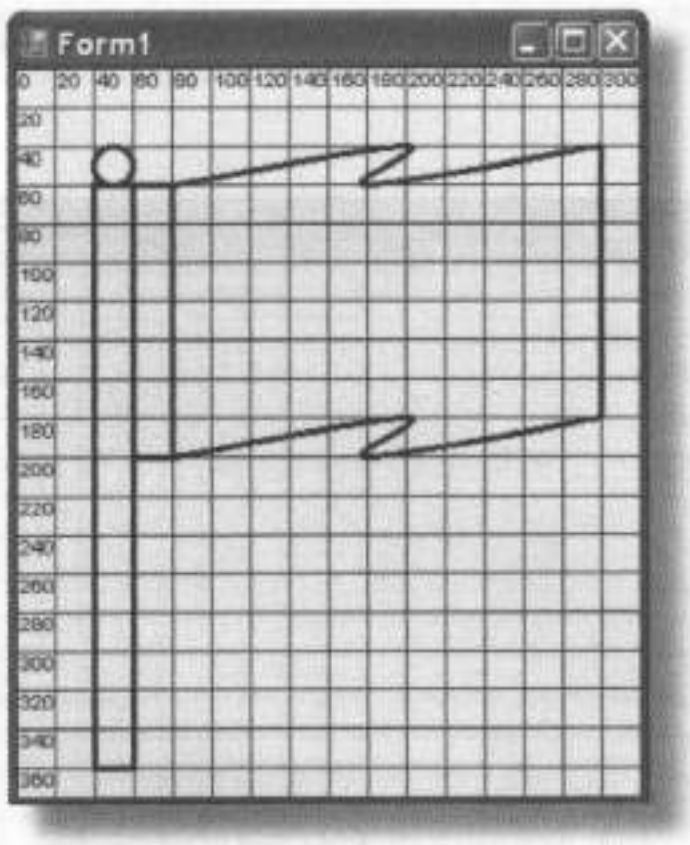
Sua tarefa era preencher o código faltante para desenhar uma grade e desenhar o resultado de dois trechos de código nas grades.

```
using (Graphics g = this.CreateGraphics())
using (Font f = new Font("Arial", 6, FontStyle.Regular)) {
  for (int x = 0; x < this.Width; x += 20) {
    g.DrawLine(Pens.Black, x, 0, x, this.Height);
    g.DrawString(x.ToString(), f, Brushes.Black, x, 0);
  }
  for (int y = 0; y < this.Height; y += 20) {
    g.DrawLine(Pens.Black, 0, y, this.Width, y);
    g.DrawString(y.ToString(), f, Brushes.Black, 0, y);
  }
}
```

Primeiro desenhe as linhas verticais e os números ao longo do eixo y. Haverá uma linha vertical a cada 20 pixels do eixo x.

Empregamos comandos using para ter certeza de que os objetos Graphics e Font seriam descartados depois de desenhar no formulário.

Depois desenhamos as linhas horizontais e os números no eixo x. Para desenhar uma linha horizontal, você escolhe um valor de y e desenha de (0, y) no lado esquerdo do formulário até (this.Width, y) no lado direito.



Gráficos podem resolver nosso problema de transparência

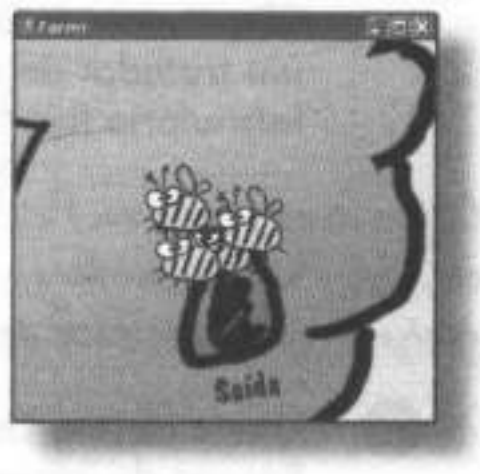
Você provavelmente já imaginou nesse ponto que `DrawImage()` é a chave para resolver o problema das caixas desenhadas em torno das abelhas e flores no renderizador quando ocorrem sobreposições. Então vamos acabar com ele! Volte para seu aplicativo Windows com a figura.

O renderizador desenhou as abelhas de forma que elas parecem estranhas quando sobrepostas.

1. Adicione um método `DrawBee()` que desenhe uma abelha em qualquer objeto Graphics. Ele usará o construtor sobrecarregado de `DrawImage()` que recebe um Rectangle para determinar onde desenhar e o tamanho da imagem.

```
public void DrawBee(Graphics g, Rectangle rect) {
  g.DrawImage(Properties.Resources.Bee_animation_1, rect);
}
```

Muito melhor - clique no formulário e as abelhas vão sobrepor-se perfeitamente.



2. Aqui está o novo **tratador de evento** `Click` **para o formulário**. Examine atentamente seu funcionamento: ele desenha a colméia de forma que o canto superior esquerdo esteja bem fora do formulário, na posição (`-Width, -Height`), e ela terá o dobro da largura e altura do formulário - assim você poderá redimensionar o formulário e ela continuará sendo desenhada corretamente. Então ele desenha quatro abelhas usando o método `DrawBee()`.

```
private void Form1_Click(object sender, EventArgs e) {
  using (Graphics g = CreateGraphics()) {
    g.DrawImage(Properties.Resources.Hive__inside_,
      -Width, -Height, Width * 2, Height * 2);
    Size size = new Size(Width / 5, Height / 5);
    DrawBee(g, new Rectangle(
        new Point(Width / 2 - 50, Height / 2 - 40), size));
    DrawBee(g, new Rectangle(
        new Point(Width / 2 - 20, Height / 2 - 60), size));
    DrawBee(g, new Rectangle(
        new Point(Width / 2 - 80, Height / 2 - 30), size));
    DrawBee(g, new Rectangle(
        new Point(Width / 2 - 90, Height / 2 - 80), size));
  }
}
```

Primeiro desenhamos o fundo da colméia, com um canto bem fora da página para que vejamos somente uma pequena parte. Então desenhamos quatro abelhas de forma que elas se sobreponham - se não acontecer isso, aumente seu formulário e clique novamente.

...mas temos um porém

Mas veja o que acontece se você arrastar o formulário para fora da tela e para dentro de novo! Oh, não!

1. Execute seu programa, clique no formulário e observe como ele desenha as abelhas! Mas alguma coisa está errada. Quando você arrasta o formulário para fora da tela e para dentro de novo, a **imagem desaparece**! Agora volte atrás e verifique o programa "Néctar aqui" que você escreveu algumas páginas atrás - **ele tem o mesmo problema!**

O que você acha que está acontecendo?

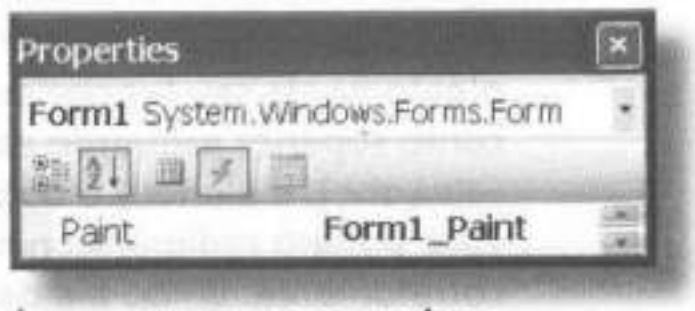
Use o evento Paint para fazer seus gráficos durarem

De que servem os gráficos se eles desaparecem assim que uma parte do seu formulário é coberta? Para nada. Por sorte, há uma forma fácil de ter certeza de que seus gráficos durem: basta **escrever um tratador de evento Paint**. Seu formulário dispara um evento Paint (pintar) toda vez que precisa ser redesenhado - como quando ele é arrastado para fora da tela. Uma das propriedades de seu parâmetro `PaintEventArgs` é um objeto Graphics e qualquer coisa que você `desenhar` usando-o vai "ficar grudado" no formulário.

Formulários e controles tem um evento Paint que disponibiliza um objeto Graphics. Qualquer coisa que você desenhar nele será repintada automaticamente.

1. **Adicione um tratador de evento Paint**
   Clique duas vezes em "Paint" na página de Eventos na janela de Propriedades para adicionar um tratador de evento de pintura. O evento Paint é disparado sempre que uma imagem no seu formulário fica "suja" (dirty). Então desenhar seus gráficos dentro dele fará a imagem durar.

Clique duas vezes em Paint para adicionar um tratador de eventos para ele. Seu PaintEventArgs tem uma propriedade chamada Graphics - e qualquer coisa que você desenhar a usando permanecerá no formulário.



2. **Use o objeto Graphics fornecido por EventArgs do evento Paint**
   Em vez de iniciar com um comando using, faça com que seu tratador de evento comece assim:

```
private void UserControl1_Paint(object sender, PaintEventArgs e) {
  Graphics g = e.Graphics;
```

   Você **não** precisa utilizar um comando **using** – como não foi **você quem criou o objeto, não precisa descartá-lo**.

3. **Copie o código que desenha as abelhas sobrepostas e a colméia**
   Adicione o novo método DrawBees() da página anterior no seu novo controle de usuário. Então copie o código do evento Click no seu novo evento Paint - **exceto a primeira linha com o comando using**, já que você já tem um objeto Graphics chamado g (como você não usará mais o comando using, certifique-se de eliminar o fecha-chaves). Agora execute seu programa. **Os gráficos não somem!**

Faça o mesmo com o "Néctar aqui" para que os gráficos não sumam também.

Formulários e controles redesenham-se a todo momento

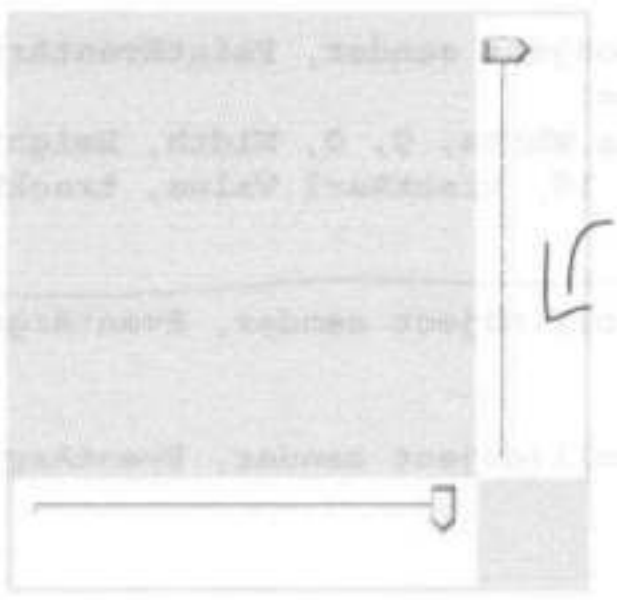
Pode não parecer, mas os formulários devem redesenhar-se toda hora. Sempre que houver controles sobre um formulário, eles estarão mostrando gráficos - rótulos mostram texto, botões mostram uma imagem de botão, caixas de checagem tem aquela pequena caixa com um X dentro delas. Você trabalha com elas como se fossem controles que se arrasta por aí, mas, na verdade, cada um deles desenha sua própria imagem. Sempre que você arrastar um formulário pela tela ou deixá-lo atrás de outra janela e então arrastá-lo de volta ou descobri-lo, a parte coberta é agora inválida, ou seja, ela não estará mais mostrando a imagem que deveria. É nesse momento que o .NET envia uma mensagem ao formulário dizendo que ele precisa se redesenhar. O formulário dispara um evento Paint sempre que estiver "sujo" e precisar ser repintado. Se em algum momento você quiser forçar seu formulário ou controle a redesenhar-se, pode dizer ao .NET para deixá-lo "sujo" chamando seu método Invalidate() (invalidar).

Exercício

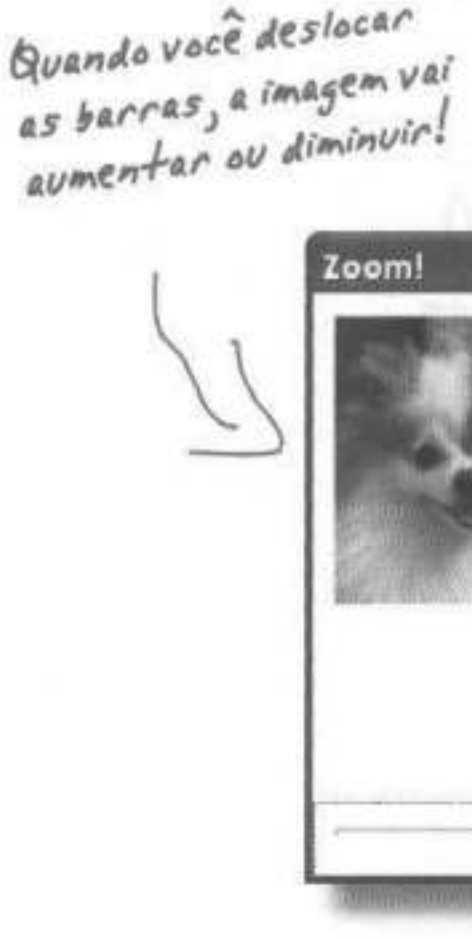
Veja se você pode combinar os seus conhecimentos de formulários e controles de usuários - e consiga um pouco mais de prática usando objetos Bitmap e o método DrawImage (), construindo controle de usuários que utiliza TrackBars para ampliar uma imagem dentro e fora.

1 **Adicione dois controles TrackBar a um novo controle de usuário**
Crie um novo projeto de aplicativo Windows. **Adicione um controle de usuário** - chame-o de Zoomer - e atribua (300, 300) para sua propriedade `Size`. Arraste para ele dois controles TrackBar da caixa de ferramentas. Coloque `TrackBar1` na parte inferior do controle. Depois arraste `TrackBar2` para o lado direto e atribua `Horizontal` à sua propriedade `Orientation`. Ambos devem ter a propriedade `Minimum` em 1, `Maximum` e `Value` em 175 e `TickStyle` em `None`. Deixe o fundo dos dois controles brancos. Finalmente, clique duas vezes em cada um deles para adicionar um tratador de evento `Scroll`. Faça com que ambos os tratadores chamem o método `Invalidade()` do controle.

Seu controle de usuário tem um evento Paint e ele funciona exatamente como o que você usou no formulário. Basta usar o parâmetro e de seu `PaintEventArgs`. Ele tem uma propriedade chamada `Graphics` e qualquer coisa que você desenhar usando esse objeto será pintada na instância do controle assim que você arrastá-lo da caixa de ferramentas.



2 **Carregue uma imagem num objeto Bitmap e desenhe-o no controle**
Adicione um campo privado do tipo `Bitmap` chamado photo ao seu controle Zoomer. Quando você criar a instância do `Bitmap`, use seu construtor para carregar uma imagem qualquer que você queira - nós usamos uma de um cachorrinho peludo. Então adicione um evento `Paint` ao controle. O tratador de evento deve criar um objeto Graphics para desenhar no controle, desenhar um retângulo branco sólido por cima do controle inteiro e então usar `DrawImage()` para desenhar o conteúdo de seu campo `photo` de forma que seu canto superior esquerdo esteja em (10, 10), sua largura seja a propriedade `Value` de `trackBar1` e sua altura seja o de `trackBar2`. Depois arraste seu controle para um formulário - certifique-se de redimensionar este último para que os TrackBars fiquem nas bordas.



Sempre que o usuário arrastar um dos cursores nas TrackBars, elas chamarão o método Invalidate() do controle. Isso fará com que este dispare seu evento Paint, redimensionando a imagem. Lembre-se, já que você não criou o objeto Graphics - ele foi passado para você como parâmetro em PaintEventArgs - não é preciso descartá-lo. Assim, você não precisa utilizar um comando using. Apenas desenhe a imagem dentro do tratador do evento Paint.

Solução do Exercício

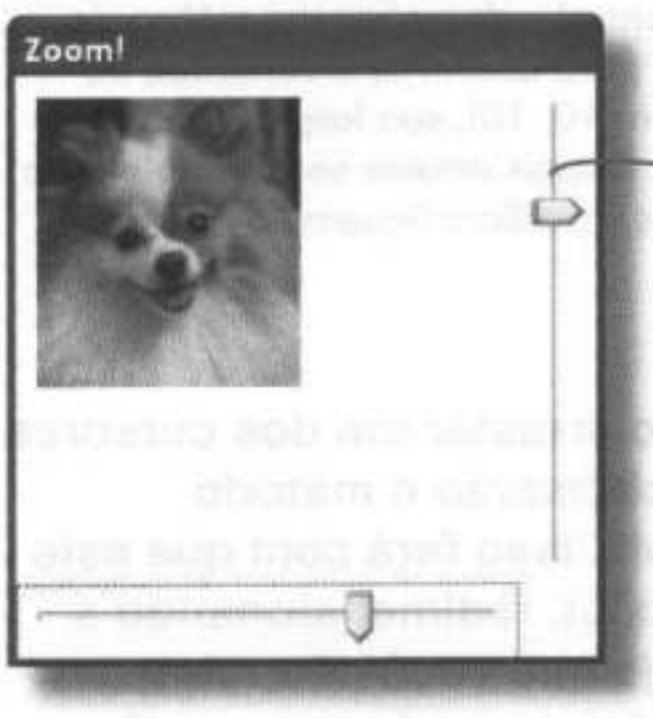
Pratique um pouco mais o uso de objetos `Bitmap` e do método `DrawImage()` desenvolvendo um formulário que os utilize para carregar uma imagem de um arquivo e depois ampliá-la ou reduzi-la.

```
public partial class Zoomer : UserControl {
  Bitmap photo = new Bitmap(@"c:\Graphics\fluffy_dog.jpg");
  public Zoomer() {
    InitializeComponent();
  }
  private void Zoomer_Paint(object sender, PaintEventArgs e) {
      Graphics g = e.Graphics;
      g.FillRectangle(Brushes.White, 0, 0, Width, Height);
      g.DrawImage(photo, 10, 10, trackBar1.Value, trackBar2.Value);
    }
  }
  private void trackBar1_Scroll(object sender, EventArgs e) {
    Invalidate();
  }
  private void trackBar2_Scroll(object sender, EventArgs e) {
    Invalidate();
  }
}
```

Este construtor de Bitmap em particular carrega uma imagem de um arquivo. Há outros construtores sobrecarregados, incluindo um que permite que se especifique apenas uma largura e altura - e esse cria um bitmap vazio.

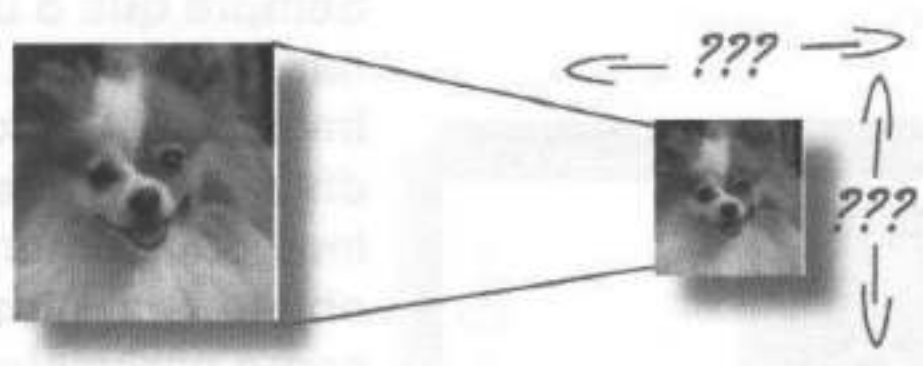
Primeiro desenhamos um grande retângulo branco para preencher o controle inteiro, depois desenhamos a foto em cima. Os dois últimos parâmetros determinam o tamanho da imagem a ser desenhada - trackBar1 indica a largura e trackBar2 a altura.

Cada vez que o usuário arrasta um dos cursores das TrackBars, ele dispara um evento Scroll. Ao fazer com que o tratador desse evento chame o método Invalidate() do controle, nós o forçamos a ser repintado... e quando ele faz isso, desenha uma nova cópia da imagem com um tamanho diferente.



Cada vez que arrastamos um desses forçamos um novo redimensionamento com DrawImage().

```
g.DrawImage(myBitmap, 30, 30, 150, 150);
```



Examinando de perto como os formulários e controles repintam-se

Nos Bastidores

Lembre-se, há pouco tempo, dissemos que quando você começa a trabalhar com objetos Graphics, assumirá o controle dos gráficos. É como se você dissesse ao .NET "Ei, eu sei o que estou fazendo, posso lidar com essa responsabilidade extra". No caso de estar desenhando e redesenhando, você pode não querer que um formulário se repinte quando está minimizando ou maximizando... ou pode querer repintar com mais **freqüência**. Uma vez que você saiba o que está acontecendo nos bastidores com seu formulário ou controle, pode assumir o comando no que se refere a redesenhar.

1 **Cada formulário tem um evento Paint que desenha os seus gráficos**
Vá para a lista de eventos de qualquer formulário e você encontrará um chamado **Paint**. Sempre que o formulário tiver que se repintar, esse evento é disparado. Cada formulário e controle usa um evento `Paint` internamente para decidir quando deve se redesenhar. Mas o que dispara esse evento? Isso é feito por um método chamado **OnPaint** que o formulário ou controle de usuário herdam da classe Control (esse método segue o padrão visto no capítulo 11, no qual métodos que disparam eventos são nomeados com "On" seguido do nome do evento). Você pode sobrepor o método OnPaint em qualquer formulário:

Faça isso exatamente como você fez antes com Dispose()

Sobreponha OnPaint em qualquer formulário e adicione esta linha.

```
protected override void OnPaint(PaintEventArgs e) {
  Console.WriteLine("OnPaint {0} {1}", DateTime.Now, e.ClipRectangle);
  base.OnPaint(e);
}
```

Arraste seu formulário por ai - coloque-o metade para fora da tela, minimize-o, deixe-o atrás de outras janelas. Observe atentamente o resultado escrito. Você verá que seu método OnPain dispara um evento Paint sempre que uma parte esteja "suja" - ou **inválida** - e precisar ser redesenhada. E se você observar atentamente ClipRectangle (retângulo de recorte), verá que ele descreve a parte do formulário que precisa ser redesenhada. Ela é passada para `PaintEventArgs` do evento Paint para permitir uma melhor performance redesenhando apenas a parte inválida.

2 **Invalidade() controla quando redesenhar e o QUE deve ser repintado**
O .NET dispara o evento Paint quando alguma coisa interfere ou cobre algo num formulário e quando ele é movido para fora da tela e depois é exibido novamente. `Invalidate()` é chamado e é passado-lhe um `retângulo`. Este diz ao método que parte do formulário precisa ser redesenhada... que parte dele está "suja". O .NET chama OnPaint para dizer ao formulário para disparar um evento `Paint` e recriar a área suja.

Invalidade() essencialmente diz: algum pedaço do formulário deve ser inválido, então redesenhe essa parte para ter certeza de que as coisa certas sejam exibidas.

3 **O método Update() dá à sua requisição de invalidação prioridade máxima**
Você pode não perceber, mas seu formulário recebe mensagens a todo momento. O mesmo sistema que lhe diz que foi coberto e chama OnPaint, que tem todo tipo de outras mensagens que precisam ser repassadas. Veja você mesmo: digite `override` e observe todos os métodos que começam com "On" - cada um deles é uma mensagem que o formulário precisa responder. O método `Update()` move a mensagem de invalidação para o topo da lista de mensagens.

Então quando você chama esse método manualmente, está dizendo ao .NET que todo o controle ou formulário é inválido e tudo deve ser redesenhada. Você pode passar seu próprio retângulo de recorte se quiser - ele será passado para PaintEventArgs do evento Paint.

4 **O método Refresh() do formulário é um Invalidate() seguido de um Update()**
Formulários e controles têm um atalho. Eles têm um método Refresh() que primeiro chama Invalidate() para deixar toda a área de cliente inválida, depois chama Update() para ter certeza de que a mensagem se mova para o topo da lista.

não existem Perguntas Idiotas

P: Ainda parece que redimensionar os gráficos num programa como o Paint ou o PhotoShop é melhor. Por que eu não posso fazer isso?

R: Você pode, se for você o responsável pelas imagens que usar em seus aplicativos e elas sempre forem usadas com as mesmas dimensões. Mas, freqüentemente, esse não é o caso. Muitas vezes, você recebe imagens de outras fontes, seja da Internet, de um colega de trabalho no grupo de projeto, etc. Ou você pode ler imagens de uma fonte apenas-leitura e assim ser obrigado a redimensioná-la via programação.

P: Mas seu eu puder redimensionar fora do .NET, é melhor, não?

R: Se você tiver certeza de que nunca deverá aumentar, pode ser. Mas se o programa puder exibir a imagem em várias dimensões enquanto executar, você deverá fazer redimensionamento em algum momento, de qualquer forma. Além disso, se a sua imagem tiver de ser exibida em qualquer momento maior do que o tamanho que você a deixou, terá problemas. É muito mais fácil diminuir do que aumentar.
É mais comum que redimensionar uma imagem via programação seja melhor do que ser limitado por um programa externo ou, por exemplo, do que ser obrigado a ler de uma fonte somente leitura.

P: Entendi que CreateGraphics() cria o objeto Graphics para desenhar num formulário, mas para que serve aquela chamada para FromImage() no método ResizeImage()?

R: `FromImage()` (a partir de imagem) recupera um objeto `Graphics()` a partir de um `Bitmap`. E exatamente como o `CreateGraphics()` chamado num formulário retorna um objeto `gráfico` para desenhar nele, `FromImage()` retorna um objeto `Graphics` para desenhar no `bitmap` que faz a chamado ao método.

P: Então um objeto Graphics não serve apenas para desenhar num formulário?

R: Na realidade, um objeto Graphics é para desenhar em qualquer coisa que retorne um objeto desse tipo. Um bitmap retorna um que você usar para desenhar numa imagem invisível que pode ser usada depois. E você encontrará objetos desse tipo em muitos outros lugares além de formulários. Arraste um botão da caixa de ferramentas, então entre no código e digite seu nome seguido de um ponto. Examine a janela do IntelliSense que aparecerá - ela tem um método CreateGraphics() que retorna um objeto Graphics. Qualquer coisa que você desenhar usando esse objeto aparecerá no botão! O mesmo serve para rótulos, caixas de imagem, caixas de status... praticamente todos os controles na caixa de ferramentas retornam um objeto gráfico.

P: Espere, eu pensei que using fosse uma coisa usada apenas com streams. Porque estou fazendo o mesmo com gráficos?

R: A palavra chave `using` é útil com streams, mas é algo que você pode usar com qualquer classe que implemente a interface `IDisposable()`. Quando uma classe implementa essa interface, se você instanciá-la deverá chamar seu método `Dispose()` posteriormente. Com as streams, esse método garante que qualquer arquivo aberto seja fechado.
Objetos `Graphics`, `Pen` e `Brush` também são descartáveis. Quando você cria qualquer um deles, eles ocupam um pequeno pedaço da memória e outros recursos e nem sempre devolvem-nos imediatamente. Se você está desenhando algo uma vez só, não notará nenhuma diferença. Mas, na maior parte das vezes, seu código gráfico será chamado de novo e de novo – como, por exemplo, num tratador de evento `Paint`, que pode ser chamado muitas vezes por segundo para um formulário particularmente atarefado. É por isso que você sempre deve chamar `Dispose()` nos seus objetos relacionados a gráficos. E a forma mais fácil de ter certeza de que você fez isso é utilizando uma linha com `using` e deixar o .NET se preocupar com o descarte. Qualquer objeto criado com using terá automaticamente seu método `Dispose()` chamado no final do bloco que segue o comando `using`. Isso garantirá que seu programa não vai tomar mais e mais memória lentamente se executar por muito tempo.

P: Se eu estiver criando um novo controle, devo usar UserControl ou criar uma classe que herde de um dos controles da caixa de ferramentas?

R: Isso depende do que você quer que seu novo controle faça. Se você está desenvolvendo um realmente parecido com um dos que já estão na caixa de ferramenta, provavelmente, achará mais fácil herdar desse controles. Mas na maior parte das vezes, quando programadores criam novos controles em C#, eles usam um UserControl. Uma vantagem significativa de um destes é que você pode **arrastar controles da caixa de ferramentas para ele.** Ele funciona de forma muito parecida com uma `caixa de grupo` ou outro agrupador de controles - você pode arrastar um botão ou caixa de checagem para dentro de seu controle de usuário e trabalhar com eles como se estivesse trabalhando com controles num formulário. O designer de formulário do IDE torna-se uma ferramenta poderosa para ajudá-lo a projetar controles de usuário. Você verá mais a respeito disso num minuto.

Um UserControl pode conter outros controles. O designer de formulário do IDE permite-lhe arrastar controles da caixa de ferramentas para dentro de seu novo controle de usuário.

Eu notei que o meu controle Zoomer ficou piscando muito. Com toda essa conversa sobre gráficos, aposto que há alguma coisa que possamos fazer a respeito disso! Mas por que isso acontece?

Mesmo sem redimensionamento, desenhar uma imagem num formulário leva tempo

Suponha que você tivesse redimensionado externamente todas as imagens do simulador. Ainda assim levaria tempo desenhar todas as abelhas, flores e a colméia. E nesse momento, estamos desenhando diretamente sobre o objeto Graphics no formulário. Então se seu olho é capaz de perceber o trabalho do renderizador, você acabará vendo o formulário piscando.

Como muitos desenhos estão sendo feitos, há uma boa chance de isso acontecer, mesmo sem redimensionamento. E é por isso que você nota problemas na maioria dos jogos feitos por amadores... o olho humano percebe o fim de um ciclo de renderização, parecendo que a tela pisca um pouco.

Poder do Cérebro

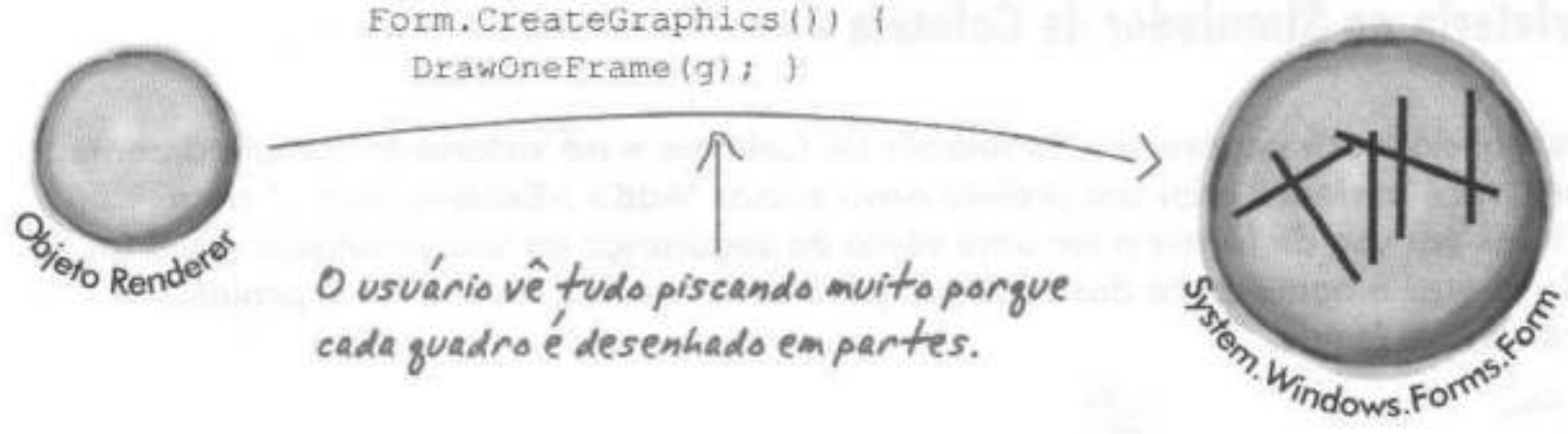
Como você poderia livrar-se desse problema? Se desenhar várias imagens no formulário faz ele piscar e você deve desenhá-las, como você poderia contornar isso?

Buffer duplo suaviza sua animação

Volte para o Zoomer e brinque com as trackbars. Você percebe como o formulário pisca? Isso ocorre porque o tratador de evento `Paint` primeiro deve desenhar o retângulo branco e então a imagem cada vez que os cursores se movem um pouquinho. Quando seus olhos vêem um retângulo branco e em seguida uma imagem se alternando muitas vezes por segundo, eles interpretam isso como se o formulário estivesse piscando. É irritante... e pode ser evitado usando uma técnica chamada **buffer** duplo. Ou seja, desenhar cada quadro ou célula de animação num bitmap invisível (um "buffer", palavra que pode ser traduzida para "área de armazenamento de reserva") e exibir o resultado somente depois de ele ter sido inteiramente composto. Eis aqui como isso funcionaria usando um `Bitmap`:

1. Aqui está um programa típico que desenha alguns gráficos num formulário usando o objeto Graphics.

```
using (graphics g =
  Form.CreateGraphics()) {
   DrawOneFrame(g); }
```

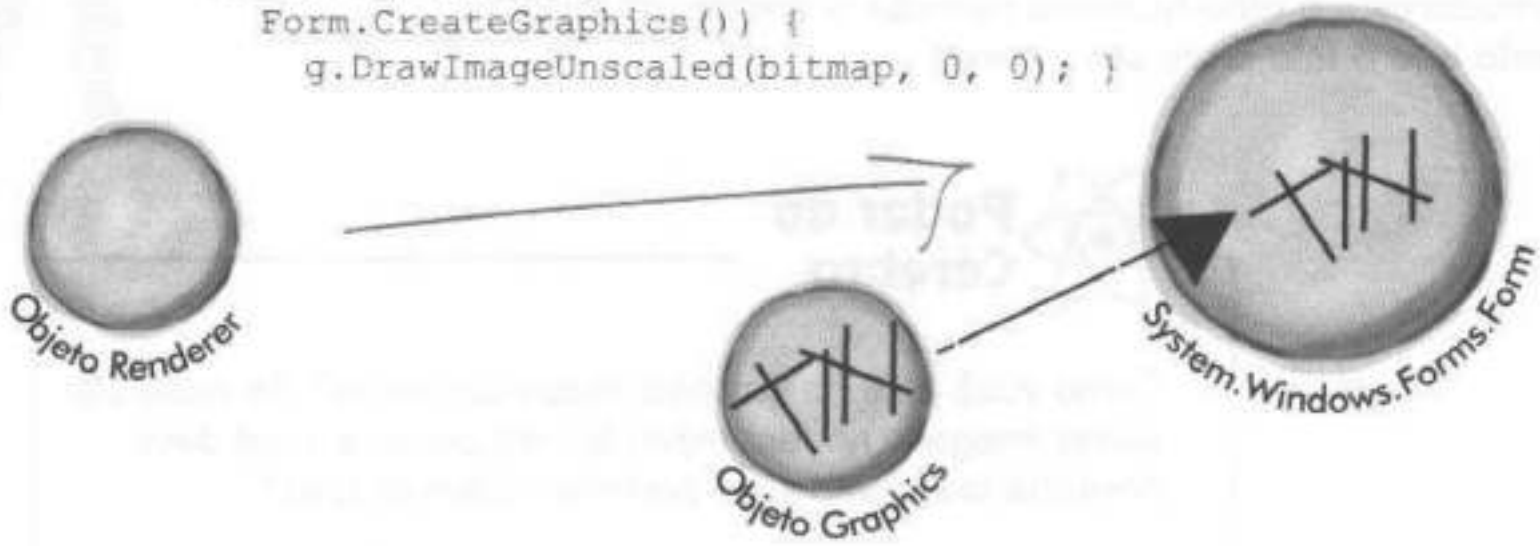


2. Para implementar o buffer duplo, podemos adicionar um objeto `Bitmap` de forma que o programa o utilize como um buffer. Cada vez que nosso formulário ou controle precisar ser repintado, em vez de desenhar diretamente nele, usaremos o buffer em seu lugar.

```
using (graphics g =
  Graphics.FromImage(bitmap)) {
    DrawOneFrame(g); }
```

3. Agora que o frame foi todo desenhado no objeto `Bitmap` invisível, podemos usar `DrawImageUnscaled()` para copiar tudo de volta para o objeto `Graphics` do formulário. Tudo é copiado de uma só vez e isso elimina o efeito de piscada.

```
using (graphics g =
Form.CreateGraphics()) {
  g.DrawImageUnscaled(bitmap, 0, 0); }
```



Buffer duplo está implementado nos formulários e controles

Você pode implementar um buffer duplo você mesmo usando um `Bitmap`, mas o C# e o .NET deixam tudo ainda mais fácil porque têm suporte nativo para essa técnica. **Tudo a fazer é deixar como verdadeira a propriedade DoubleBuffered**. Tente isso com o controle de usuário Zoomer - vá até a janela de propriedades, atribua true à DoubleBuffered e seu controle parará de piscar! Agora **volte ao seu controle BeeControl** e tente o mesmo. Isso não vai corrigir todos os problemas dos gráficos - faremos isso num minuto - mas faz diferença.

Quando você usa o evento Paint para todos os gráficos, pode ativar pintura usando buffers apenas alterando uma propriedade.

Agora você está pronto para corrigir os problemas com gráficos no simulador!

Faça uma vistoria no Simulador de Colméia

No próximo exercício, você pegará seu Simulador de Colméia e irá vistoriá-lo completamente. Provavelmente, você preferirá criar um projeto novo e usar "Add>>Existing Item..." para adicionar os itens em uso de forma a ter uma cópia de segurança do seu simulador atual (não se esqueça de alterar o namespace dos arquivos para ser o mesmo do seu novo projeto). Eis o que você deverá fazer:

1. **Você começará removendo o controle de usuário BeeControl**
   Não existirá controle nenhum na colméia e nem no campo. Nenhum BeeControl, nenhuma caixa de imagem, nada. As abelhas, flores e imagem da colméia serão todas desenhadas usando gráficos da GDI+. Então clique com o botão direito em BeeControl.cs (e também em OldBeeControl.cs) no Navegador de Solução e clique Delete - eles serão removidos do projeto e apagados permanentemente.

2 **Você deverá ter um timer para lidar com o batimento das asas das abelhas**
As abelhas batem suas asas bem mais devagar do que a taxa de quadros do simulador, então você deverá ter um segundo timer mais lento. Isso não deve ser surpresa, já que o BeeControl tinha um para fazer exatamente a mesma coisa.

3 **O passo importante: vistorie o renderizador**
Você deverá descartar o renderizador atual inteiramente, porque ele faz tudo com controles. Você não deverá ter dicionários de pesquisa, porque não haverá nenhuma caixa de imagens ou BeeControl para procurar. Em vez disso, teremos dois métodos importantes: DrawHive(g) (desenhar colméia) pintará o formulário de colméia num objeto Graphics e DrawFiels(g) (desenhar campo) fará o mesmo com o de campo.

4 **Por último, você deverá incorporar o novo renderizador**
Os formulários de campo e de colméia precisarão de tratadores de evento Paint. Cada um deles chamará os métodos DrawFiled(g) ou DrawHive(g) do objeto Renderer. Os dois timers - um para dizer ao simulador para desenhar o próximo quadro e outro para bater as asas das abelhas - chamará o método Invalidate() dos formulários para repintá-los. Quando isso acontecer, os tratadores de evento Paint pintarão o quadro.

Vamos começar!

Exercício

É hora de se livrar dos problemas dos gráficos no simulador de colméia. Para fazê-lo parecer melhor acabado, use Graphics e buffer duplo.

1 Modifique o método RunFrame() do formulário principal
Você deverá remover a chamada para `Renderer.Render()` e adicionar dois comandos `Invalidate()`.

```
public void RunFrame(object sender, EventArgs e) {
  framesRun++;
  world.Go(random);
  end = DateTime.Now;
  TimeSpan frameDuration = end - start;
  start = end;
  UpdateStats(frameDuration);
  hiveForm.Invalidate();
  fieldForm.Invalidate();
}
```

Você deverá remover a chamada para o método Render() já que ele será removido.

Desde que você mantenha o mundo atualizado e os dois formulários tenham uma referência para o objeto renderizador, para animá-los basta chamar seus métodos Invalidate(). Os seus tratadores de evento Paint tomarão conta do restante.

2 Adicione um segundo timer ao formulário principal para fazer as asas das abelhas baterem
Arraste um novo timer para o formulário principal, atribua 150ms ao seu `intervalo` e true à propriedade `Enabled`. Então clique duas vezes nele e adicione o seguinte tratador de evento:

```
private void timer2_Tick(object sender, EventArgs e) {
  renderer.AnimateBees();
}
```

Depois acrescente este método `AnimateBees()` no renderizador para fazer as asas das abelhas baterem:

```
private int Cell = 0;
private int Frame = 0;
private void AnimateBees() {
  Frame++;
  if (Frame >= 6)
  Frame = 0;
  switch (Frame) {
    case 0: Cell = 0; break;
    case 1: Cell = 1; break;
    case 2: Cell = 2; break;
    case 3: Cell = 3; break;
    case 4: Cell = 2; break;
    case 5: Cell = 1; break;
    default: Cell = 0; break;
  }
  hiveForm.Invalidate();
  fieldForm.Invalidate();
}
```

A idéia por trás do código aqui é alterar um campo chamado Cell (célula) que você pode usar quando o renderizador estiver desenhando as abelhas. Certifique-se de que sempre esteja desenhando BeeAnimationLarge[Cell] no formulário da colméia e BeeAnimationSmall[Cell] no formulário do campo. O timer constantemente chamará o método AnimateBees(), o que fará com que o campo Cell sempre mude e isso terá como conseqüência suas abelhas baterem as asas.

3 **Configure os formulários da colméia e do campo para animação com buffer duplo**
Remova o código do construtor do formulário da colméia que configura a imagem de fundo. Então remova todos os controles dos dois formulários e **atribua** `true` **para suas propriedades** `DoubleBuffer`. Por fim, adicione um tratador de evento Paint em cada um deles. Eis aqui o tratador para o formulário da colméia - o equivalente do campo será idêntico, exceto que ele chamará `renderer.PaintField()` em vez de `renderer.PaintHive()`.

```
private void HiveForm_Paint(object sender, PaintEventArgs e) {
  renderer.PaintHive(e.Graphics);
}
```

Certifique-se de ter ativado o buffer duplo, ou seus formulários piscarão!

4 **Os formulário de colméia e de campo precisarão de uma propriedade pública renderer**
Adicione um campo público nos formulários de campo e de colméia:

```
public Renderer renderer;
```

Há dois lugares onde se criará um novo Renderer(): no botão abrir (substituindo as chamadas a renderer.Reset) e no método ResetSimulator(). Remova todas as chamadas para renderer.Reset() e substitua cada uma delas por novos comandos renderer com uma chamada ao novo método CreateRenderer() que você adicionará:

```
private void CreateRenderer() {
  renderer = new Renderer(world, hiveForm, fieldForm);
  hiveForm.renderer = renderer;
  fieldForm.renderer = renderer;
}
```

Você não precisará mais de um método Reset() no renderizador. Tudo o que ele fazia era remover os controles dos formulários e não há mais controles para remover.

5 **Revise o renderer removendo código baseado em controles e adicionando gráficos**

Eis o que você precisa fazer para corrigir o renderizador:

- ★ Remova os dois dicionários, uma vez que não há mais controles. E enquanto você fizer isto, como não precisa mais de `BeeControl`, ou `Render()`, `DrawBees()` e `DrawFlowers()`, remova-os também.

- ★ Adicione alguns campos `Bitmap` chamados `HiveInside` (dentro da colméia), HiveOutside (fora da colméia) e Flower para armazenar imagens. Crie duas matrizes de `bitmaps` chamadas `BeeAnimationLarge` (animação grande de abelha) e `BeeAnimationSmall` (animação pequena de abelha). Cada uma delas manterá quatro imagens de abelhas - as maiores terão 40x40 e as menores 20x20. Crie um método chamado `InitializeImages()` para redimensionar os recursos e guardá-los nesses campos, e chame-o no construtor da classe Renderer.

- ★ Adicione o método `PaintHive()` que receba um objeto `Graphics` como parâmetro e pinte a colméia nele. Primeiro desenhe um retângulo azul celeste, depois use `DrawImageUnscaled()` para desenhar o interior da colméia e então use o mesmo método para desenhar cada uma das abelhas que estiverem dentro dela.

- ★ Por último, adicione o método `PaintField()`. Ele deve desenhar um retângulo azul celeste na metade superior do formulário, um verde na inferior. Você descobrirá que duas propriedades dos formulários são úteis isso: ClientSize e `ClientRectangle` dizem a você o tamanho da área de desenho, de forma que você pode achar a metade de sua altura usando ClientSize.Height / 2. Depois use `FillEllipse()` para desenhar um sol amarelo no céu, `DrawLine()` para desenhar um galho onde a colméia estará pendurada, e `DrawImageUnscaled()` para desenhar o lado de fora da colméia. Então desenhe cada flor. Finalmente, desenhe cada abelha (usando a imagem pequena delas) - elas devem ser pintadas por último para ficarem na frente das flores.

- ★ Enquanto estiver desenhando as abelhas, lembre-se de que `AnimateBees()` altera o campo Cell.

Solução do Exercício

É hora de se livrar dos problemas dos gráficos no simulador de colméia. Para fazê-lo parecer melhor acabado, use Graphics e buffer duplo.

Eis aqui a classe Renderer completa, incluindo o método AnimateBees() que mostramos a você. Certifique-se de ter feito todas as modificações nos três formulários - especialmente nos métodos tratadores de eventos Pain no formulário de campo e de colméia. Eles devem chamar os métodos PaintHive() e PaintField() do renderizador e eles cuidarão de toda a animação.

```
using System.Drawing;
public class Renderer {
  private World world;
  private HiveForm hiveForm;
  private FieldForm fieldForm;
  public Renderer(World TheWorld, HiveForm hiveForm, FieldForm fieldForm) {
    this.world = TheWorld;
    this.hiveForm = hiveForm;
    this.fieldForm = fieldForm;
    InitializeImages();
  }
  public static Bitmap ResizeImage(Image ImageToResize, int Width, int Height) {
    Bitmap bitmap = new Bitmap(Width, Height);
    using (Graphics graphics = Graphics.FromImage(bitmap)) {
      graphics.DrawImage(ImageToResize, 0, 0, Width, Height);
    }
    return bitmap;
  }
  Bitmap HiveInside;
  Bitmap HiveOutside;
  Bitmap Flower;
  Bitmap[] BeeAnimationSmall;
  Bitmap[] BeeAnimationLarge;
  private void InitializeImages() {
    HiveOutside = ResizeImage(Properties.Resources.Hive__outside_, 85, 100);
    Flower = ResizeImage(Properties.Resources.Flower, 75, 75);
    HiveInside = ResizeImage(Properties.Resources.Hive__inside_,
      hiveForm.ClientRectangle.Width, hiveForm.ClientRectangle.Height);
    BeeAnimationLarge = new Bitmap[4];
    BeeAnimationLarge[0] = ResizeImage(Properties.Resources.Bee_animation_1, 40, 40);
    BeeAnimationLarge[1] = ResizeImage(Properties.Resources.Bee_animation_2, 40, 40);
    BeeAnimationLarge[2] = ResizeImage(Properties.Resources.Bee_animation_3, 40, 40);
    BeeAnimationLarge[3] = ResizeImage(Properties.Resources.Bee_animation_4, 40, 40);
    BeeAnimationSmall = new Bitmap[4];
    BeeAnimationSmall[0] = ResizeImage(Properties.Resources.Bee_animation_1, 20, 20);
    BeeAnimationSmall[1] = ResizeImage(Properties.Resources.Bee_animation_2, 20, 20);
    BeeAnimationSmall[2] = ResizeImage(Properties.Resources.Bee_animation_3, 20, 20);
    BeeAnimationSmall[3] = ResizeImage(Properties.Resources.Bee_animation_4, 20, 20);
  }
```

O método InitializeImages() redimensiona todos os recursos que forem imagens e os armazena em campos Bitmap dentro do objeto Renderer. Assim, os métodos PaintHive() e PaintField() podem desenhar as imagens sem ter de redimensioná-las usando o método DrawImageUnscaled() dos objetos Graphics.

```
  public void PaintHive(Graphics g) {
    g.FillRectangle(Brushes.SkyBlue, hiveForm.ClientRectangle);
    g.DrawImageUnscaled(HiveInside, 0, 0);
    foreach (Bee bee in world.Bees) {
      if (bee.InsideHive)
        g.DrawImageUnscaled(BeeAnimationLarge[Cell],
                            ee.Location.X, bee.Location.Y);
    }
  }
  public void PaintField(Graphics g) {
```

A propriedade ClientSize de um formulário é um retângulo que informa o tamanho da área de desenho.

```
public void PaintField(Graphics g) {
using (Pen brownPen = new Pen(Color.Brown, 6.0F)) {
    g.FillRectangle(Brushes.SkyBlue, 0, 0,
        fieldForm.ClientSize.Width, fieldForm.ClientSize.Height / 2);
    g.FillEllipse(Brushes.Yellow, new RectangleF(50, 15, 70, 70));
    g.FillRectangle(Brushes.Green, 0, fieldForm.ClientSize.Height / 2,
        fieldForm.ClientSize.Width, fieldForm.ClientSize.Height / 2);
    g.DrawLine(brownPen, new Point(643, 0), new Point(643, 30));
    g.DrawImageUnscaled(HiveOutside, 600, 20);
    foreach (Flower flower in world.Flowers) {
      g.DrawImageUnscaled(Flower, flower.Location.X, flower.Location.Y);
    }
    foreach (Bee bee in world.Bees) {
      if (!bee.InsideHive)
        g.DrawImageUnscaled(BeeAnimationSmall[Cell],
                            bee.Location.X, bee.Location.Y);
    }
  }
}
```

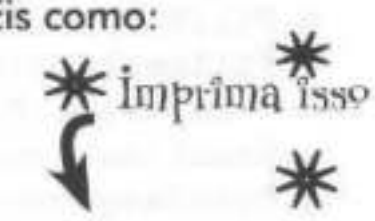
O método PaintField() procura pelas abelhas e flores no mundo e desenha o campo usando suas posições. Primeiro ele desenha o céu e o chão, em seguida o sol, e então a colméia. Depois disso, ele desenha as flores e as abelhas. É importante que tudo seja desenhado na ordem certa - se as flores fossem desenhadas depois das abelhas, estas pareceriam voar por trás daquelas.

```
private int Cell = 0;
private int Frame = 0;
public void AnimateBees()
{
  Frame++;
  if (Frame >= 6)
      Frame = 0;
  switch (Frame) {
      case 0: Cell = 0; break;
      case 1: Cell = 1; break;
      case 2: Cell = 2; break;
      case 3: Cell = 3; break;
      case 4: Cell = 2; break;
      case 5: Cell = 1; break;
      default: Cell = 0; break;
  }
  hiveForm.Invalidate();
  fieldForm.Invalidate();
}
}
```

Eis o mesmo método AnimateBees() mostrado num exercício. Ele itera pelas animações usando o campo Frame - primeiro ele mostra a célula 0, depois a 1, a 2, a 3, e então ele volta para 2 e para 1 novamente. Dessa forma a animação de bater asas da abelha fica mais suave.

Use um objeto Graphics e um tratador de eventos para imprimir

Os métodos de Graphics que você tem usado para desenhar nos formulários são **os mesmos que você usará para imprimir**. Os objetos de impressão do .NET em `System.Drawing.Printing` realmente facilitam adicionar impressão e visualização em seus aplicativos. Tudo a fazer é **criar um objeto** `PrintDocument`. Ele tem um evento chamado `PrintPage` (imprimir página), que você pode usar exatamente como usou o evento `Tick` dos timers. Então chame o método `Print()` (imprimir) do objeto `PrintDocument` e o documento será impresso. E lembre-se, o IDE torna especialmente fácil adicionar tratadores de eventos. Eis como:



1. Inicie um novo projeto de aplicativo Windows e adicione um botão no formulário. Vá para o código e acrescente uma linha using `System.Drawing.Printing;` no início. Clique duas vezes e adicione o tratador de eventos. Observe o que acontece assim que você digita `+=`:

```
private void button1_Click(object sender, EventArgs e) {
  PrintDocument document = new PrintDocument();
  document.PrintPage +=
```

```
new PrintPageEventHandler(document_PrintPage)   (Pressione TAB para inserir)
```

2. Pressione a tecla Tab e o IDE automaticamente preenche o restante da linha. Isso é exatamente o mesmo que você fez para acrescentar tratadores de evento no capítulo 11:

```
private void button1_Click(object sender, EventArgs e) {
  PrintDocument document = new PrintDocument();
  document.PrintPage += new PrintPageEventHandler(document_PrintPage);
```

```
Pressione TAB para gerar o tratador 'document_PrintPage' nesta classe
```

3. Assim que você pressionar Tab, o IDE gera um método tratador de evento e acrescenta-o ao formulário.

```
void document_PrintPage(object sender, PrintPageEventArgs e) {
  throw new NotImplementedException();
}
```

Agora você pode incluir QUALQUER código gráfico aqui - basta substituir a linha com o throw e usar e.Graphics para fazer todo o desenho. Mostraremo-lhes isso num minuto...

O parâmetro e de `PrintPageEventArgs` tem uma propriedade Graphics. Basta substituir o comando `throw` com código que chame os métodos de desenho de `e.Graphics`.

4. Agora complete o tratador de evento `button1_Click` chamando **`document.Print()`**. Quando esse método for chamado, PrintDocument cria um objeto `Graphics` e o informa como parâmetro ao disparar o evento `PrintPage`. Qualquer coisa que o tratador de evento desenhar nesse objeto `gráfico` será enviado para a impressora.

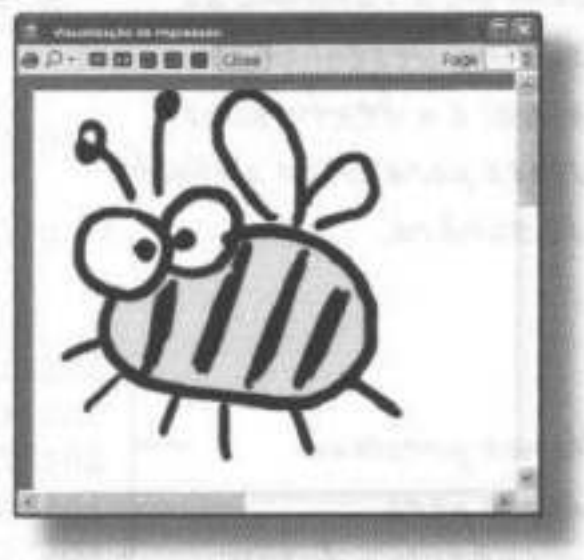
```
private void button1_Click(object sender, EventArgs e) {
  PrintDocument document = new PrintDocument();
  document.PrintPage += new PrintPageEventHandler(document_PrintPage);
  document.Print();
}
```

PrintDocument trabalha em conjunto com o diálogo de impressão e o de visualização do Windows

Uma vez que você tem um PrintDocument e um tratador de evento para imprimir a página, para mostrar uma janela de visualização, basta criar um novo objeto PrintPreviewDialog.

Adicionar uma janela de visualização ou uma de impressão é muito parecido com acrescentar uma caixa de diálogo de abrir ou salvar. Tudo a fazer é criar um objeto `PrintDialog` ou `PrintPreviewDialog` (diálogo de visualização de impressão), configurar sua propriedade `Document` para ser igual a seu objeto `Document` e chamar o método `Show()`. O diálogo vai cuidar de enviar o documento para a impressora - não é preciso chamar o método `Print()`. Então vamos adicionar o seguinte ao botão criado no passo 1:

```
private void button1_Click(object sender, EventArgs e) {
  PrintDocument document = new PrintDocument();
❺ document.PrintPage += new PrintPageEventHandler(document_PrintPage);
  PrintPreviewDialog preview = new PrintPreviewDialog();
  preview.Document = document;
  preview.ShowDialog(this);
}
void document_PrintPage(object sender, PrintPageEventArgs e) {
  DrawBee(e.Graphics, new Rectangle(0, 0, 300, 300));
}
```



Use e.HasMorePages para imprimir documentos de múltiplas páginas

Se você tiver a necessidade de imprimir mais de uma página, tudo a fazer é usar o seu tratador de evento `PrintPage` para atribuir true à `e.HasMoraPages` (tem mais páginas). Isso diz ao `Document` que você tem outra página para imprimir. Ele chamará o tratador de evento de novo e de novo, uma vez por página, enquanto este continuar mantendo a propriedade como true. Modifique o tratador de evento do seu documento para imprimir duas páginas:

```
bool firstPage = true;
void document_PrintPage(object sender, PrintPageEventArgs e) {
  DrawBee(e.Graphics, new Rectangle(0, 0, 300, 300));
  using (Font font = new Font("Arial", 36, FontStyle.Bold)) {
    if (firstPage) {
      e.Graphics.DrawString("First page", Font, Brushes.Black, 0, 0);
      e.HasMorePages = true;
      firstPage = false;
    } else {
      e.Graphics.DrawString("Second page", Font, Brushes.Black, 0, 0);
      firstPage = true;
    }
  }
}
```

Se você atribuir true para e.HasMorePages, o objeto Document vai chamar o tratador de evento novamente para imprimir a próxima página.

Agora execute novamente o seu programa e certifique-se de que ele esteja exibindo duas páginas na visualização de impressão.

Exercício

Escreva o código para o botão Imprimir no simulador de forma que ele mostre uma janela de visualização de impressão das estatísticas das abelhas e imagens da colméia e do campo.

❶ **Faça o botão mostrar uma janela de visualização de impressão**
Adicione um tratador para o evento Click do botão que pause o simulador, mostre o diálogo de visualização de impressão e continue a simulação depois de encerrado (se o simulador já estiver em pausa quando o botão for clicado, certifique-se de que continue em pausa depois que a visualização for mostrada).

2 **Crie o método de tratamento de evento PrintPage do documento**
Ele deve criar uma página que se pareça exatamente como a mostrada adiante.
Vamos começar para você.

```
private void document_PrintPage(object sender, PrintPageEventArgs e) {
  Graphics g = e.Graphics;
  Size stringSize;
  using (Font arial24bold = new Font("Arial", 24, FontStyle.Bold)) {
    stringSize = Size.Ceiling(
          g.MeasureString("Bee Simulator", arial24bold));
    g.FillEllipse(Brushes.Gray,
        new Rectangle(e.MarginBounds.X + 2, e.MarginBounds.Y + 2,
        stringSize.Width + 30, stringSize.Height + 30));
    g.FillEllipse(Brushes.Black,
        new Rectangle(e.MarginBounds.X, e.MarginBounds.Y,
        stringSize.Width + 30, stringSize.Height + 30));
    g.DrawString("Bee Simulator", arial24bold,
        Brushes.Gray, e.MarginBounds.X + 17, e.MarginBounds.Y + 17);
    g.DrawString("Bee Simulator", arial24bold,
        Brushes.White, e.MarginBounds.X + 15, e.MarginBounds.Y + 15);
  }
  int tableX = e.MarginBounds.X + (int)stringSize.Width + 50;
  int tableWidth = e.MarginBounds.X + e.MarginBounds.Width - tableX - 20;
  int firstColumnX = tableX + 2;
  int secondColumnX = tableX + (tableWidth / 2) + 5;
  int tableY = e.MarginBounds.Y;
  // sua tarefa: preencher o restante do método para fazê-lo imprimir
```

Criamos a oval com texto dentro usando o método MeasureString(), que retorna um Size que contém o tamanho da string. Desenhamos a oval e o texto duas vezes para criar o efeito de sombra.

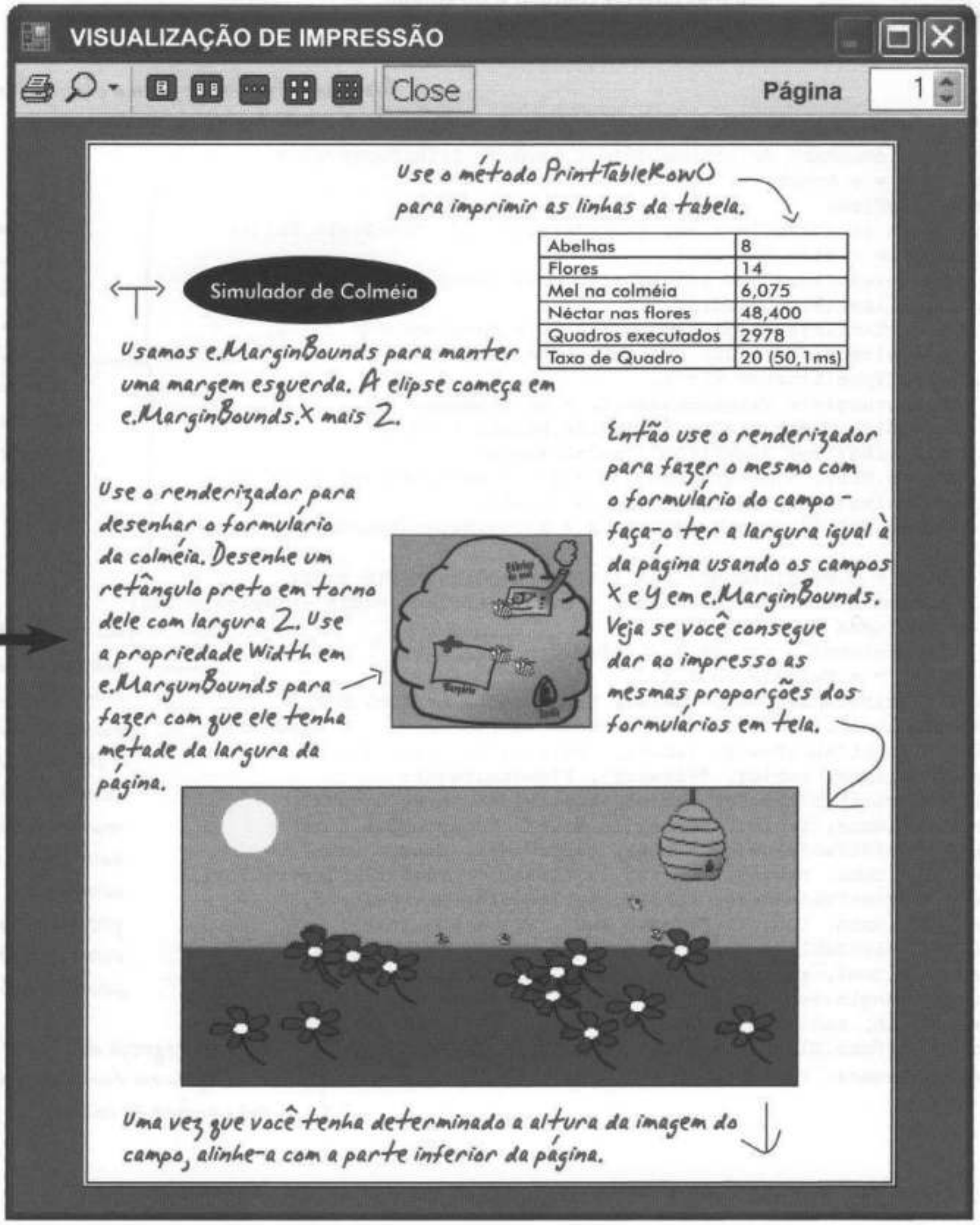
Vamos precisar disto para construir a tabela.

3 **Esse método PrintTableRow() será útil**
Você vai achar esse método útil quando criar a tabela de estatísticas de abelhas que aparecerá no início da página.

```
private int PrintTableRow(Graphics printGraphics, int tableX,
                int tableWidth, int firstColumnX, int secondColumnX,
                int tableY, string firstColumn, string secondColumn) {
  Font arial12 = new Font("Arial", 12);
  Size stringSize = Size.Ceiling(printGraphics.MeasureString(firstColumn, arial12));
  tableY += 2;
  printGraphics.DrawString(firstColumn, arial12, Brushes.Black,
                firstColumnX, tableY);
  printGraphics.DrawString(secondColumn, arial12, Brushes.Black,
                secondColumnX, tableY);
  tableY += (int)stringSize.Height + 2;
  printGraphics.DrawLine(Pens.Black, tableX, tableY, tableX + tableWidth, tableY);
  arial12.Dispose();
  return tableY;
}
```

Cada vez que PrintTableRow() for chamado, ele adiciona a altura da linha que imprime em tableY e retorna esse novo valor.

Examine com cuidado as notas que escrevemos na página. Elas são um tanto complexas - não tenha pressa!



Eis uma dica: para achar a altura para cada formulário, determine a razão entre sua altura e sua largura e multiplique a largura final por isso. Você pode posicionar o topo do formulário de campo subtraindo da margem inferior da página a altura (e.MarginBounds.Y + e.MarginBounds.Height - fieldHeight).

Solução do Exercício

Escreva o código para o botão Imprimir no simulador de forma que ele mostre uma janela de visualização de impressão das estatísticas das abelhas e imagens da colméia e do campo.

Eis aqui o tratador para o evento PrintPage do documento. Ele vai no formulário.

```
using System.Drawing.Printing;
private void document_PrintPage(object sender, PrintPageEventArgs e) {
  Graphics g = e.Graphics;
  Size stringSize;
  using (Font arial24bold = new Font("Arial", 24, FontStyle.Bold)) {
    stringSize = Size.Ceiling(
      g.MeasureString("Bee Simulator", arial24bold));
    g.FillEllipse(Brushes.Gray,
      new Rectangle(e.MarginBounds.X + 2, e.MarginBounds.Y + 2,
      stringSize.Width + 30, stringSize.Height + 30));
    g.FillEllipse(Brushes.Black,
      new Rectangle(e.MarginBounds.X, e.MarginBounds.Y,
      stringSize.Width + 30, stringSize.Height + 30));
    g.DrawString("Bee Simulator", arial24bold,
      Brushes.Gray, e.MarginBounds.X + 17, e.MarginBounds.Y + 17);
    g.DrawString("Bee Simulator", arial24bold,
      Brushes.White, e.MarginBounds.X + 15, e.MarginBounds.Y + 15);
  }
  int tableX = e.MarginBounds.X + (int)stringSize.Width + 50;
  int tableWidth = e.MarginBounds.X + e.MarginBounds.Width - tableX - 20;
  int firstColumnX = tableX + 2;
  int secondColumnX = tableX + (tableWidth / 2) + 5;
  int tableY = e.MarginBounds.Y;
  tableY = PrintTableRow(g, tableX, tableWidth, firstColumnX,
    secondColumnX, tableY, "Bees", Bees.Text);
  tableY = PrintTableRow(g, tableX, tableWidth, firstColumnX,
    secondColumnX, tableY, "Flowers", Flowers.Text);
  tableY = PrintTableRow(g, tableX, tableWidth, firstColumnX,
    secondColumnX, tableY, "Honey in Hive", HoneyInHive.Text);
  tableY = PrintTableRow(g, tableX, tableWidth, firstColumnX,
    secondColumnX, tableY, "Nectar in Flowers", NectarInFlowers.Text);
  tableY = PrintTableRow(g, tableX, tableWidth, firstColumnX,
    secondColumnX, tableY, "Frames Run", FramesRun.Text);
  tableY = PrintTableRow(g, tableX, tableWidth, firstColumnX,
    secondColumnX, tableY, "Frame Rate", FrameRate.Text);
  g.DrawRectangle(Pens.Black, tableX, e.MarginBounds.Y,
    tableWidth, tableY - e.MarginBounds.Y);
  g.DrawLine(Pens.Black, secondColumnX, e.MarginBounds.Y,
    secondColumnX, tableY);
```

Já tínhamos mostrado-lhe essa parte. Ela desenha o título numa elipse e atribui valores para variáveis que você usará para desenhar a tabela de estatísticas de abelhas.

Você entendeu como o método PrintTableRow() funciona? Tudo a fazer é chamá-lo uma vez para cada linha; ele então imprime o texto que você quiser nas duas colunas. Um truque é que ele retorna o novo valor de tableY para a próxima linha.

Não se esqueça de desenhar um retângulo em torno da tabela e a linha entre as colunas.

Você deverá ter uma caneta preta com dois pixels de espessura para desenhar as linhas em torno das cópias de tela.

Os bitmaps precisam ter o mesmo tamanho da área de desenho dos formulários, então ClientSize mostra-se útil.

Como a caneta e os dois bitmaps precisam ser descartados, nós colocamo-nos dentro de um grande bloco using.

O método PaintHive() precisa de um objeto Graphics para desenhar, então o código cria um Bitmap vazio e o passa como parâmetro.

e.MarginBounds.Width tem a largura da área imprimível da página. É essa a largura que a cópia de tela do campo deve ter.

Aqui é onde a altura da cópia de tela é calculada usando a proporção altura/largura do formulário.

Eis o código para o botão Print. Ele pausa o simulador (se ele estiver executando), cria um PrintDocument, associa-o ao tratador de evento PrintPage, mostra o diálogo e então reinicia o simulador.

```
using (Pen blackPen = new Pen(Brushes.Black, 2))
using (Bitmap hiveBitmap = new Bitmap(hiveForm.ClientSize.Width,
                                      hiveForm.ClientSize.Height))
using (Bitmap fieldBitmap = new Bitmap(fieldForm.ClientSize.Width,
                                       fieldForm.ClientSize.Height))
{
  using (Graphics hiveGraphics = Graphics.FromImage(hiveBitmap))
  {
    renderer.PaintHive(hiveGraphics);
  }
  int hiveWidth = e.MarginBounds.Width / 2;
  float ratio = (float)hiveBitmap.Height / (float)hiveBitmap.Width;
  int hiveHeight = (int)(hiveWidth * ratio);
  int hiveX = e.MarginBounds.X + (e.MarginBounds.Width - hiveWidth) / 2;
  int hiveY = e.MarginBounds.Height / 3;
  g.DrawImage(hiveBitmap, hiveX, hiveY, hiveWidth, hiveHeight);
  g.DrawRectangle(blackPen, hiveX, hiveY, hiveWidth, hiveHeight);
  using (Graphics fieldGraphics = Graphics.FromImage(fieldBitmap))
  {
    renderer.PaintField(fieldGraphics);
  }
  int fieldWidth = e.MarginBounds.Width;
  ratio = (float)fieldBitmap.Height / (float)fieldBitmap.Width;
  int fieldHeight = (int)(fieldWidth * ratio);
  int fieldX = e.MarginBounds.X;
  int fieldY = e.MarginBounds.Y + e.MarginBounds.Height - fieldHeight;
  g.DrawImage(fieldBitmap, fieldX, fieldY, fieldWidth, fieldHeight);
  g.DrawRectangle(blackPen, fieldX, fieldY, fieldWidth, fieldHeight);
}
}
private void printToolStripButton_Click(object sender, EventArgs e) {
  bool stoppedTimer = false;
  if (timer1.Enabled) {
    timer1.Stop();
    stoppedTimer = true;
  }
  PrintPreviewDialog preview = new PrintPreviewDialog();
  PrintDocument document = new PrintDocument();
  preview.Document = document;
  document.PrintPage += new PrintPageEventHandler(document_PrintPage);
  preview.ShowDialog(this);
  if (stoppedTimer)
    timer1.Start();
}
```

Há tanto a ser feito...

Você desenvolveu um pequeno simulador que é bonito e bem feito, mas por que parar? Há muito mais a fazer por si próprio. Aqui estão algumas idéias - veja se você consegue implementar algumas delas.

Adicione um painel de controle

Converta as constantes das classes World e Hide em propriedades. Depois adicione um novo formulário que seja um painel de controle com controles para ajustar esses valores.

Acrescente inimigos

Inclua inimigos que atacam a colméia. Quanto mais flores existirem, mais inimigos serão atraídos para a colméia. Depois crie abelhas patrulheiras com ferrões para defender a colméia deles e também abelhas de manutenção que defendam e reparem a colméia. Essas abelhas vão consumir mel extra.

Um bom simulador vai ter muitos aspectos em conflito e dará ao usuário formas de decidir qual caminho escolher para influenciar o progresso da colmeia.

Implemente atualizações na colméia

Se a colméia tiver mel suficiente, ela deve ser aumentada. Uma colméia maior pode manter mais abelhas, mas exige mais mel e atrai mais inimigos. Se estes causarem muitos danos, a colméia deve encolher novamente.

Inclua uma abelha rainha que põe ovos

Os ovos precisam de abelhas-babás para cuidar deles. Mais mel na colméia faz a abelha colocar mais ovos, que exigem mais operárias para serem cuidadas, as quais consomem mais mel.

Adicione Animação

Anime o fundo dos formulários de forma que o sol lentamente viaje pelo céu. Faça com que escureça de noite e desenhe estrelas e a lua. Implemente alguma perspectiva - faça as abelhas ficarem menores quanto mais longe elas se afastam da colméia, para dentro do campo, atrás de flores.

Use sua imaginação!

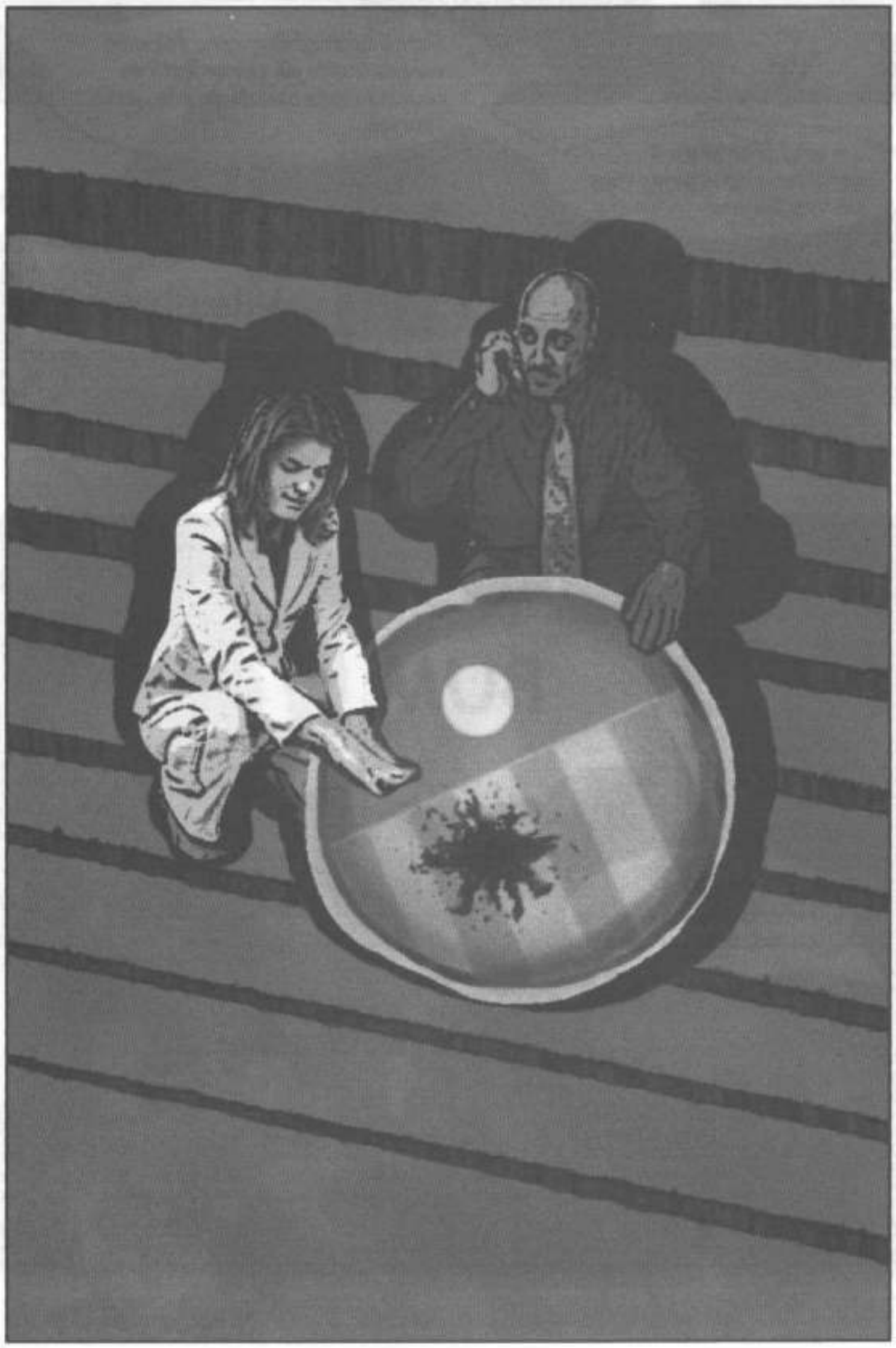
Tente pensar em formas diferentes de tornar a simulação mais interessante ou interativa.

Você imaginou uma modificação legal para o simulador? É sua chance de se exibir - envie o código fonte do seu projeto para os fóruns do Use a Cabeça C# em www.altabooks.com.br.

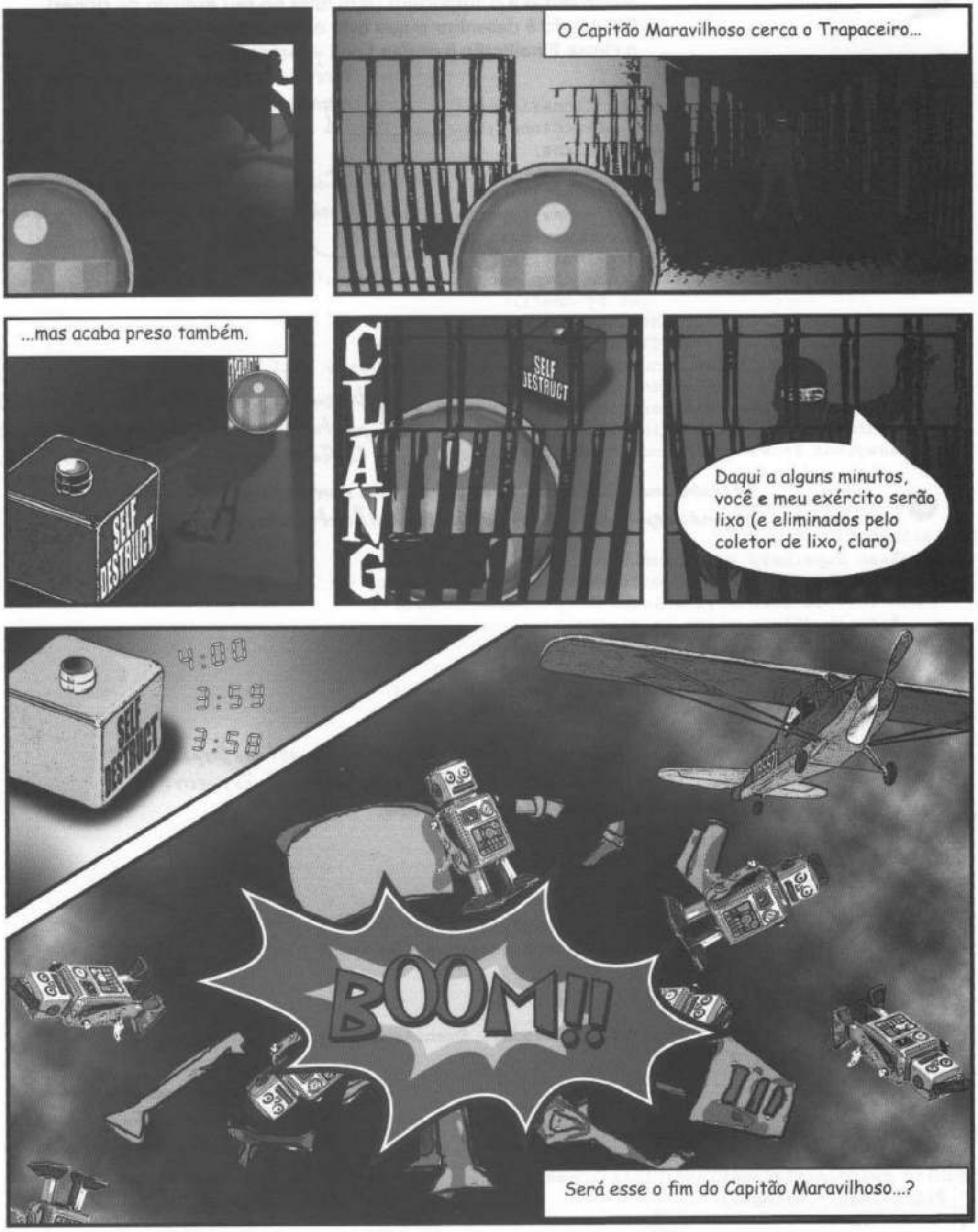
CAPITÃO MARAVILHOSO

A MORTE DO OBJETO

Laboratórios Use a Cabeça	
R$ 2,98	Capítulo 14



Capitão Maravilhoso, o objeto mais maravilhoso da Vila Objeto, persegue seu arquiinimigo...
PEGUEI VOCÊ, TRAPACEIRO.
Tarde demais! Enquanto falamos meu exército de clones está se reunindo logo abaixo de nós nessa fábrica...
...e está pronto para espalhar o caos pelas ruas da Vila Objeto!
Bem-vindo à
Vila objeto
Lar do Polimorfismo
BUM !!
Destruirei as referências dos clones, uma por uma.


O Capitão Maravilhoso cerca o Trapaceiro...
...mas acaba preso também.
SELF DESTRUCT
CLANG
SELF DESTRUCT
Daqui a alguns minutos, você e meu exército serão lixo (e eliminados pelo coletor de lixo, claro)
SELF DESTRUCT
4:00
3:59
3:58
BOOM!!
Será esse o fim do Capitão Maravilhoso...?

Aponte seu lápis

A seguir está o código detalhando o combate entre o Capitão Maravilhoso e o Trapaceiro (sem falar no seu exército de clones). Sua tarefa é desenhar o que está acontecendo na memória quando a classe FinalBattle (batalha final) é instanciada.

```
public class FinalBattle {
  public CloneFactory Factory = new CloneFactory();
  public List<Clone> Clones = new List<Clone>();
  public SwindlersEscapePlane escapePlane;
  public FinalBattle() {
    public Villain swindler = new Villain(this);
    using (Superhero captainAmazing = new Superhero()) {
      Factory.PeopleInFactory.Add(captainAmazing);
      Factory.PeopleInFactory.Add(swindler); ❶
      captainAmazing.Think("I'll take down each of the clones' references,
                            one by one");
      captainAmazing.IdentifyTheClones(Clones);
      captainAmazing.RemoveTheClones(Clones);
      swindler.Think("A few minutes from now, you AND my army will be garbage");
      swindler.Think("(collected, that is!)");
      escapePlane = new SwindlersEscapePlane(swindler) ❷
      swindler.TrapCaptainAmazing(Factory);
      MessageBox.Show("The Swindler escaped");
    }
  }
} ❸
[Serializable]
public class Superhero : IDisposable {
      private List<Clone> clonesToRemove = new List<Clone>();
      public void IdentifyTheClones(List<Clone> clones) {
        foreach (Clone clone in clones)
          clonesToRemove.Add(clone);
      }
      public void RemoveTheClones(List<Clone> clones) {
        foreach (Clone clone in clonesToRemove)
          clones.Remove(clone);
      ...
      }
}
public class Villain {
  private FinalBattle finalBattle;
  public Villain(FinalBattle finalBattle) {
    this.finalBattle = finalBattle;
  }
  public void TrapCaptainAmazing(CloneFactory factory) {
    factory.SelfDestruct.Tick += new EventHandler(SelfDestruct_Tick);
    factory.SelfDestruct.Interval = 600;
    factory.SelfDestruct.Start();
  }
  private void SelfDestruct_Tick(object sender, EventArgs e) {
    finalBattle.factory = null;
  }
}

public class SwindlersEscapePlane {
  public Villain PilotsSeat;
  public SwindlersEscapePlane(Villain escapee) {
    PilotsSeat = escapee;
  }
}
public class CloneFactory {
  public Timer SelfDestruct = new Timer();
  public List<object> PeopleInFactory = new List<object>();
  ...
}
```

Você pode assumir que Clones usou um inicializador de objeto.

Começamos aqui, com o que está acontecendo no objeto factory (fábrica).

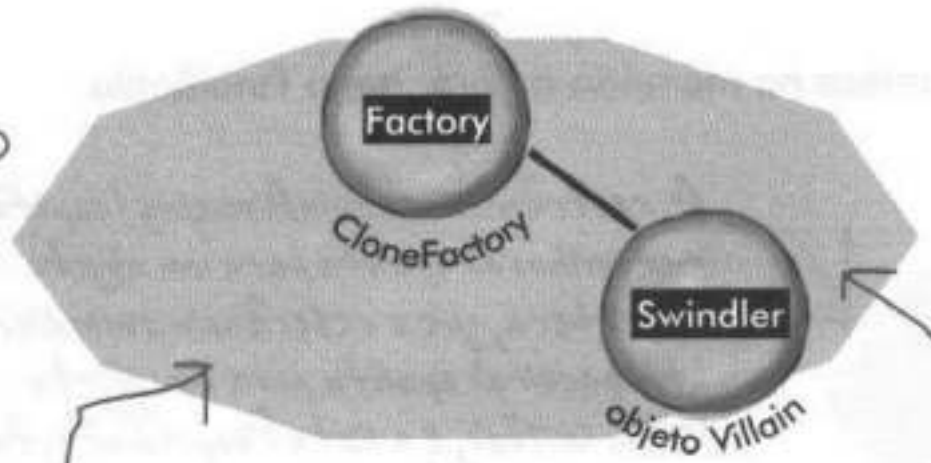
❷ Desenhe o que está acontecendo bem nesse ponto, quando o objeto SwindlersEscapePlane (avião de fuga do Trapaceiro) é instanciado.

❸ Desenhe uma representação de como a pilha deve se parecer exatamente um segundo depois que o construtor de FinalBattle executar.

Existe mais código aqui (incluindo o método Dispose) que não estamos mostrando, mas você não precisa dele para fazer o exercício.

Existe uma classe clone e também não a estamos mostrando nesse código para você. Você não precisa delas para responder.

Certifique-se de acrescentar rótulos aos seus objetos para mostrar as variáveis de referência que apontam para eles.



Começamos o primeiro para você. Tenha certeza de desenhar linhas mostrando a estrutura - fizemos uma da fábrica de clones até o objeto Superhero, porque a fábrica tem referências dele (através de seu campo PeopleInFactory).

Deixamos espaço, já que há mais a ser desenhado nessa fase.

Nesse desenho você pode deixar de fora os objetos Clone e List, e concentrar-se nos objetos de primeira linha.

Sua tarefa é desenhar o que está acontecendo com a memória nessas duas fases também.

Baseando-se nos seus diagramas, onde no código o Capitão Maravilhoso morre?

.....

.....

.....

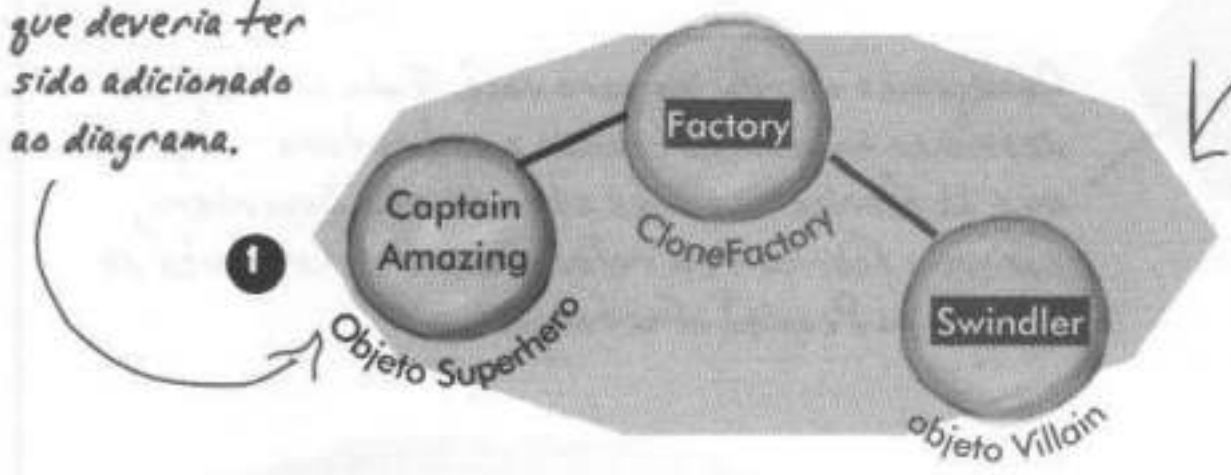
.....

Tenha certeza de anotar isso no diagrama.

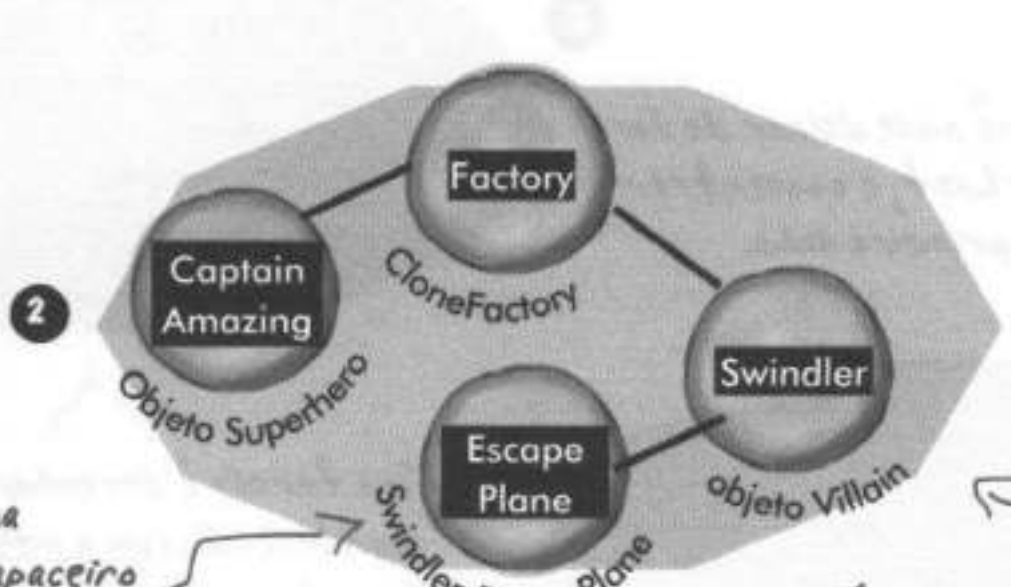
Aponte seu lápis
Solução

Desenhe o que acontece na memória no programa FinalBattle.

Este é o objeto que deveria ter sido adicionado ao diagrama.



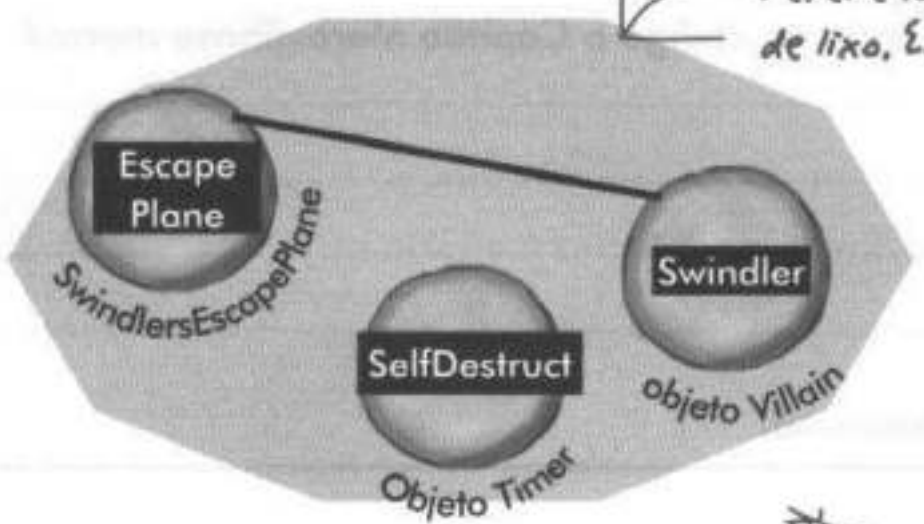
A referência captainAmazing (capitão maravilhoso) aponta para um objeto Superhero, que a referência swindler (trapaceiro) aponta para um objeto Villain (vilão), e a lista PeopleInFactory (pessoas na fábrica) da fabrica de clones contém referências para ambos.



A referência escapePlane (avião de fuga) agora aponta para uma nova instância de um objeto SwindlersEscapePlane e seu campo PilotSeat (assento do piloto) aponta para o objeto Villain.

Enquanto existir uma referência para o Trapaceiro no avião de fuga, ele não será eliminado pela coleta de lixo.

Quando selfDestruct (autodestruir) dispara, o valor de referência da fabrica é anulado e ela torna-se candidata à coleta de lixo. Então ela não aparece neste desenho.



Assim que a referência da fábrica desaparece, ela leva consigo o objeto CloneFactory - e isso faz com que o objeto Lista referenciado pelo seu campo PeopleInFactory desapareça... e era isso a única coisa que mantinha o objeto Superhero vivo. Agora ele será destruído na próxima vez que a coleta de lixo executar.

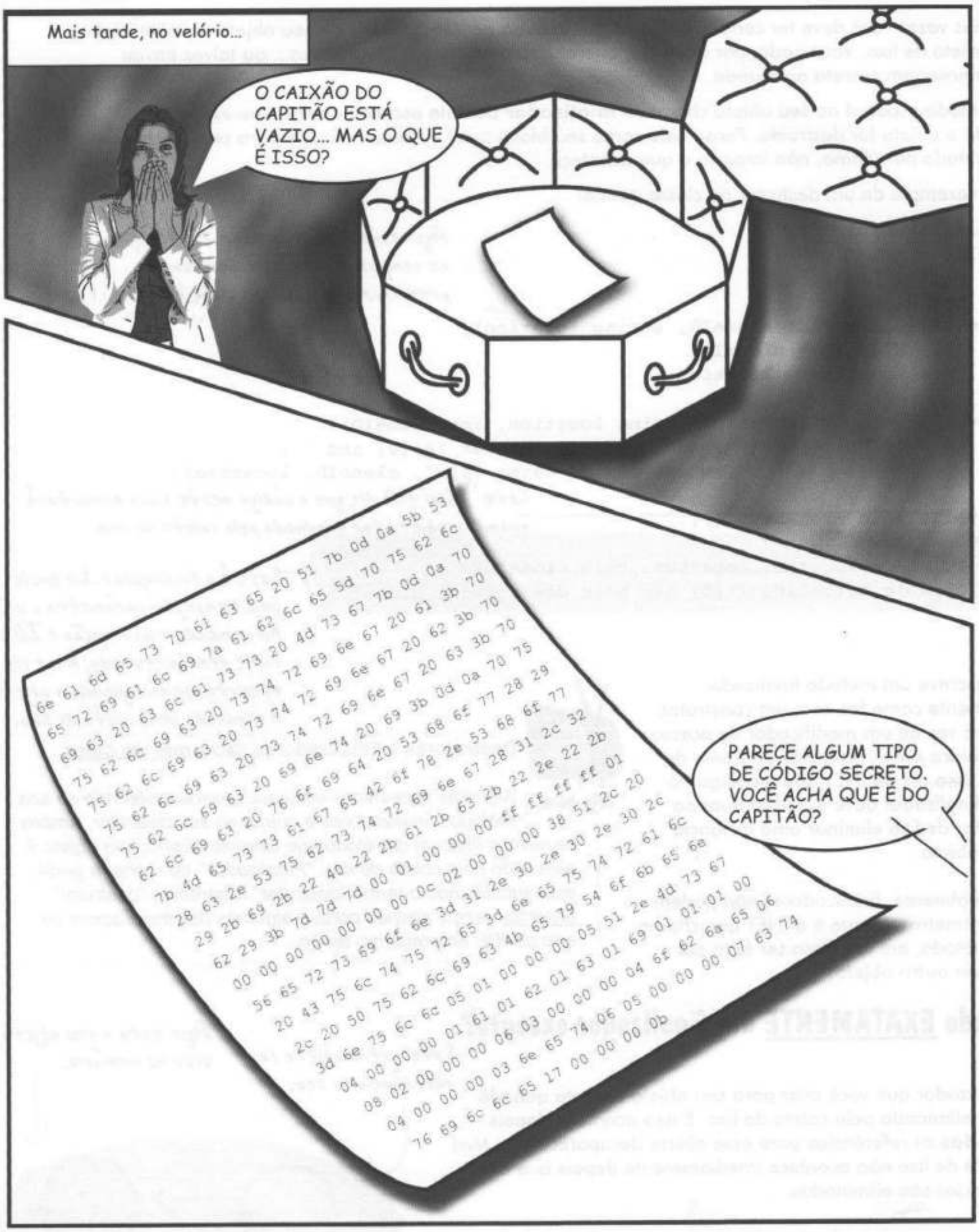
Baseando-se nos seus diagramas, onde no código o Capitão Maravilhoso morre?

```
void SelfDestruct_Tick(object sender, EventArgs e) {
    finalBattle.factory = null;
}
```

Um segundo depois que o construtor de FinalBattle executa, nosso herói se foi.

Uma vez que finalBattleFactory teve null atribuído como seu novo valor, a instância fica pronta para ser eliminada pela coleta de lixo. E isso elimina a última referência do Capitão!

Uma vez que a instância de Superhero não tem mais a fábrica de clones fazendo referência a ela, é marcada para coleta de lixo também.


Mais tarde, no velório...
O CAIXÃO DO CAPITÃO ESTÁ VAZIO... MAS O QUE É ISSO?
PARECE ALGUM TIPO DE CÓDIGO SECRETO. VOCÊ ACHA QUE É DO CAPITÃO?

Sua última chance de FAZER alguma coisa... o finalizador de seu objeto

Algumas vezes você deve ter certeza de que alguma coisa aconteça **antes de** seu objeto ser eliminado pela coleta de lixo. Você pode, por exemplo, querer liberar conexões ou recursos... ou talvez enviar uma mensagem secreta ao mundo.

Um método especial no seu objeto chamado **finalizador** permite escrever código que executará quando o objeto for destruído. Pense nele como seu bloco `finally` personalizado para seu objeto: ele é executado por último, não importa o que aconteça.

Eis um exemplo de um destrutor na classe `Clone`:

```
[Serializable]
public class Clone {
    string Location;
    int CloneID;
  public Clone (int cloneID, string location){
    this.CloneID = cloneID;
    this.Location = location;
  }
  public void TellLocation(string location, int cloneID){
    Console.WriteLine("My Identification number is {0} and " +
                      "you can find me here: {1}.", cloneID, location);
  }
  public void WreakHavoc(){...}
  ~Clone() {
    TellLocation(this.Location, this.CloneID);
    Console.WriteLine ("{0} has been destroyed", CloneID);
  }
}
```

Aqui está o construtor. Parece que os campos CloneID e Location são preenchidos quando um clone é criado.

Este ~ (ou til) diz que o código neste bloco executará quando o objeto for eliminado pela coleta de lixo.

Este é o finalizador. Ele envia uma mensagem parâmetro o vilão informando a localização e ID do clone desafortunado. Mas ele executa apenas quando o objeto é eliminado pela coleta de lixo.

Você escreve um método finalizador exatamente como faz com um construtor, mas em vez de um modificador de acesso, você coloca um ~ na frente do nome da classe. Isso diz ao .NET que o código no bloco finalizador deve executar quando o coletor de lixo eliminar uma instância desse objeto.

Adicionalmente, finalizadores não podem ter parâmetros, já que é o .NET que chama esse método, em vez disso ser feito por qualquer outro objeto.

Veja bem!

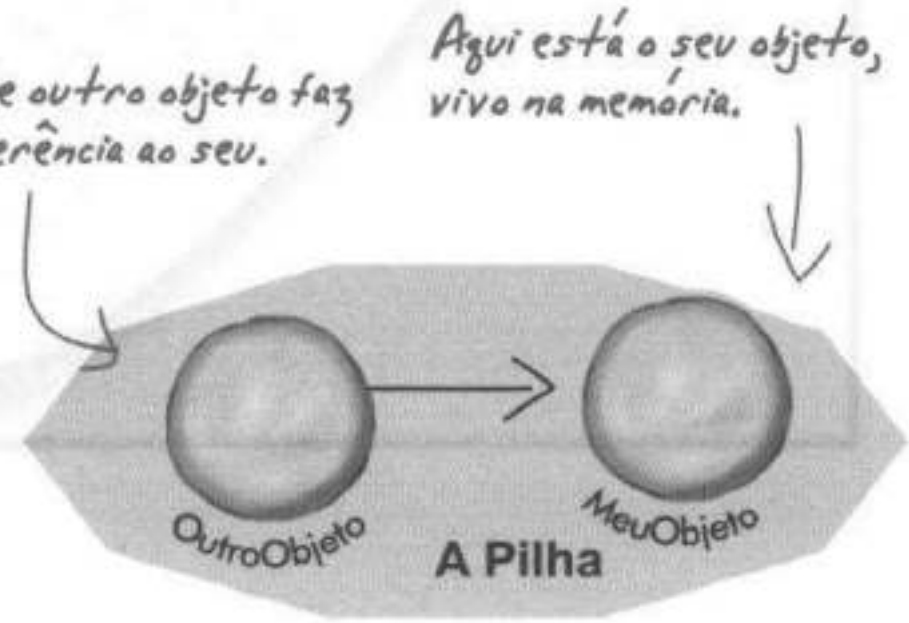
Destrutores e Finalizadores são a mesma coisa.

Algumas vezes você escutará pessoas referindo-se aos métodos finalizadores e outras ao seu destrutor. Ambos os termos indicam o método que executa quando um objeto é eliminado pela coleta de lixo. "Finalizador", de maneira geral, está substituindo o termo "destrutor". Usaremos "destrutor" algumas vezes, apenas porque algumas das mensagens de erro do IDE empregam o termo.

Quando EXATAMENTE um finalizador executa?

O finalizador que você criar para seu objeto executa quando ele for eliminado pela coleta de lixo. E isso acontece depois que **todas** as referências para esse objeto desaparecerem. Mas a coleta de lixo não acontece imediatamente depois que as referências são eliminadas.

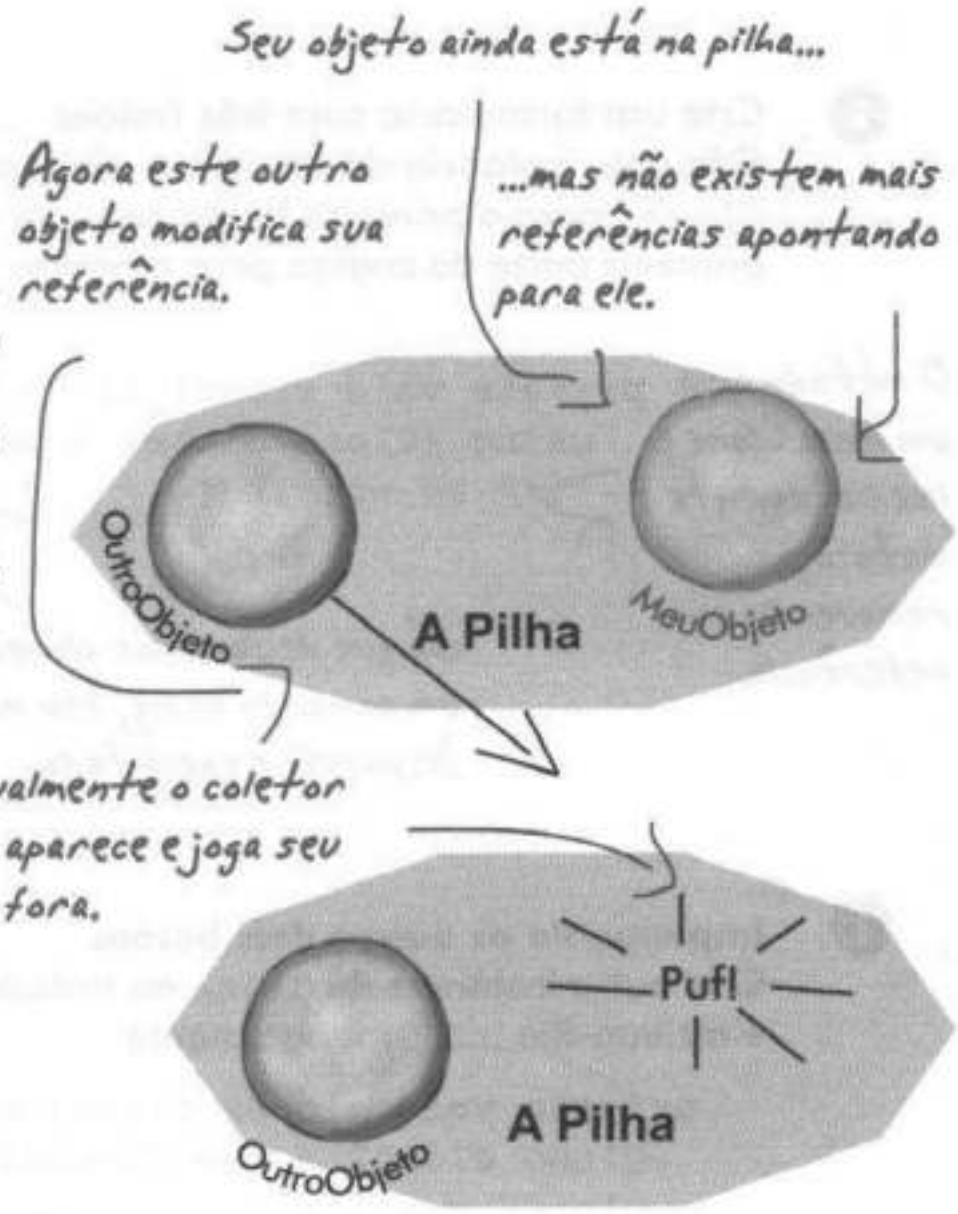
Imagine que você tenha um objeto com uma referência. O .NET faz com que o coletor de lixo execute e ele verifica seu objeto. Mas já que existem referências para ele, o coletor ignora esse objeto e continua o trabalho. Seu objeto continua vivo na memória.



Então, algo ocorre. O último objeto que mantinha uma referência para o seu decide que é hora de ir. Agora, seu objeto está lá na memória, mas não tem referências. Ele não pode mais ser acessado. Ele é basicamente um **objeto morto**.

Mas tem um detalhe. **A coleta de lixo é algo que o .NET controla**, não seus objetos. Então se o coletor de lixo ainda não foi ativado por, digamos, alguns segundos ou mesmo minutos, o objeto continua vivo na memória. Ele não pode ser usado, mas ainda não foi coletado. **E o finalizador do objeto (ainda) não foi executado.**

Finalmente, o .NET envia o coletor de lixo. Dessa vez, seu objeto está morto e o coletor joga-o fora. Aqui, finalmente, o destrutor executa... possivelmente até vários minutos depois que a última referência para o objeto tenha sido removida ou alterada.



Você pode SUGERIR ao .NET que é hora de coletar o lixo

O .NET permite **sugerir-lhe** que coletar o lixo seria uma boa idéia. **Na maioria das vezes, você não usará esse método, porque a coleta de lixo é ajustada para responder a muitas condições no CLR e chamá-lo não é realmente uma boa idéia**. Mas apenas para ver como um finalizador funciona, pode-se pedir que a coleta de lixo seja realizada. Quando quiser fazer isso, basta chamar o método `GC.collect()`.

Tome cuidado, no entanto. Esse método não **força** o .NET a coletar o lixo imediatamente. Ele apenas diz "faça a coleta de lixo assim que for possível".

```
public void RemoveTheClones(
                List<Clone> clones) {
  foreach (Clone clone in clonesToRemove)
  clones.Remove(clone);
  GC.Collect();
}
```

Dispose() funciona com `using`, finalizadores com o coletor de lixo

`Dispose()` executa quando um objeto criado com um comando using é anulado ou perde todas as suas referências. Se você não quiser utilizar um comando using, apenas atribuir null para a última referência não fará com que `Dispose()` seja chamado - você precisaria, nesse caso, chamá-lo diretamente. Um finalizador de objeto executa quando o coletor de lixo elimina aquele objeto em particular. Vamos criar alguns objetos e ver como essas duas técnicas diferem.

Faça isso!

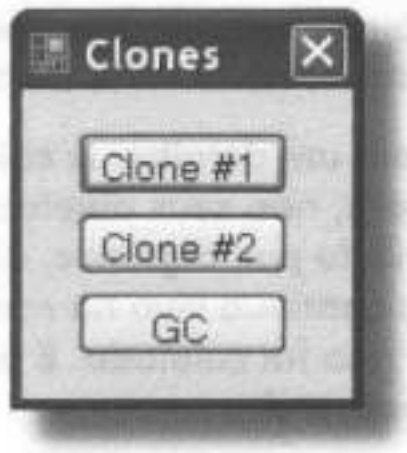
1 **Crie uma classe Clone e certifique-se de que ela implemente IDisposable**
A classe deve ter uma propriedade automática chamada ID. Ela deve ter um construtor, um método `Dispose()` e um finalizador:

```
public class Clone : IDisposable {
  public int ID { get; private set; }
  public Clone(int ID) {
    this.ID = ID;
  }
  public void Dispose() {
    MessageBox.Show("I've been disposed!",
                    "Clone #" + ID + " says...");
  }
  ~Clone() {
    MessageBox.Show("Aaargh! You got me!",
                    "Clone #" + ID + " says...");
  }
}
```

Já que a classe implementa IDisposable, ela deve ter o método Dispose().

Aqui está o finalizador. Ele executará quando o objeto for eliminado pelo coletor de lixo.

Eis o formulário que você deve criar.



2 **Crie um formulário com três botões**
Crie uma instância de um `Clone` dentro de um tratador de evento `Click1` para o primeiro botão com um comando `using`. Aqui está a primeira parte do código para o botão:

O método cria um novo Clone e imediatamente mata-o, removendo sua referência.

```
private void clone1_Click(object sender, EventArgs e)
  using (Clone clone1 = new Clone(1)) {
    // Do nothing!
  }
}
```

Já que declaramos clone1 utilizando um comando using, seu método Dispose() é executado.

Assim que o bloco using termina e o método Dispose() de Clone é chamado, nenhuma outra referência existe para o objeto e ele é marcado para coleta de lixo.

3 **Implemente os outros dois botões**
Crie outra instância de `Clone` no tratador de evento `Click` do segundo botão e atribua-lhe `null` manualmente:

```
private void clone2_Click(object sender, EventArgs e) {
  Clone clone2 = new Clone(2);
  clone2 = null;
}
```

Já que aqui não temos um comando using, Dispose() nunca executará, mas o finalizador sim.

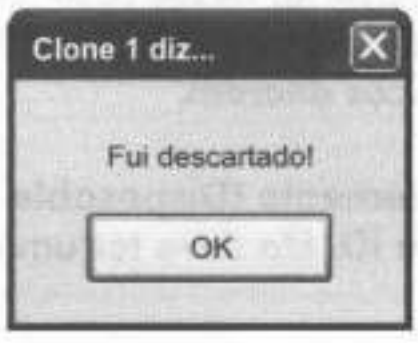
No terceiro botão, adicione uma chamada para `GC.Collect()` para sugerir ao coletor de lixo que ele deve executar.

```
private void gc_Click(object sender, EventArgs e) {
  GC.Collect();
}
```

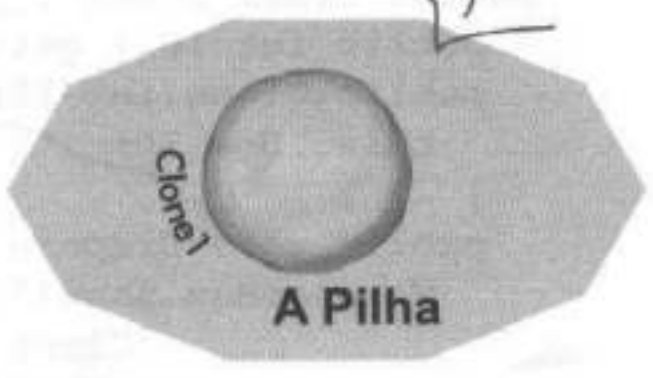
Isto sugere ao coletor de lixo que ele deve executar.

Lembre-se, normalmente não é uma boa idéia fazer isso. Mas aqui tudo bem, já que é uma boa forma de aprender sobre coleta de lixo.

4 **Execute o programa e brinque com Dispose() e com finalizadores**
Clique no primeiro botão e dê uma olhada nas caixas de mensagem. `Dispose()` executará primeiro.



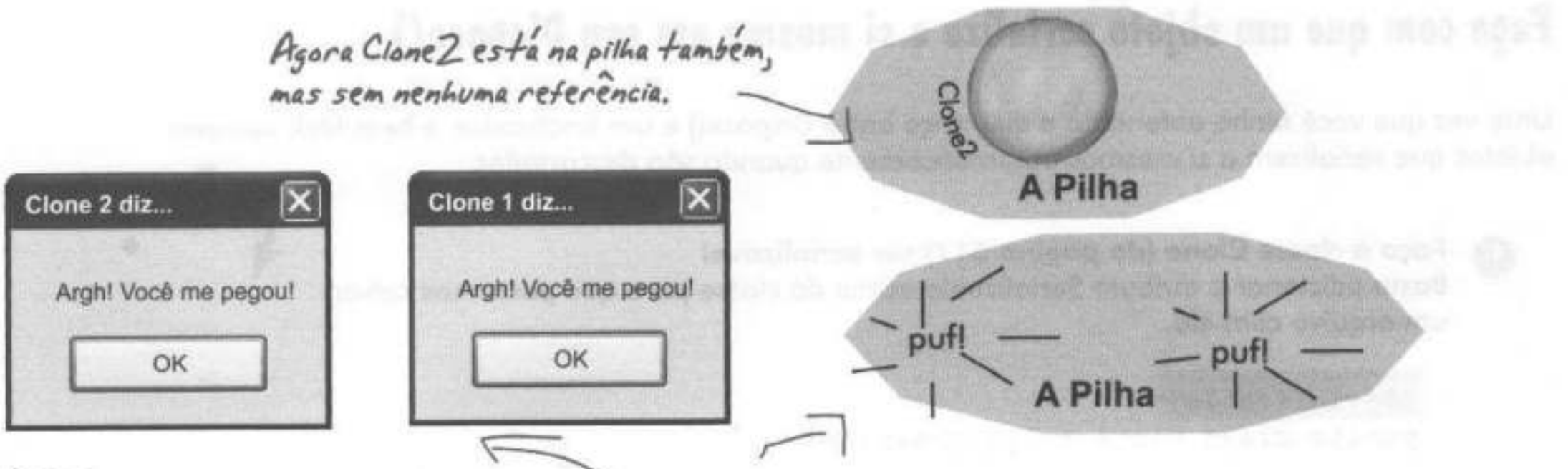
Mesmo que o objeto Clone1 tenha sido anulado e seu método Dispose() executado, ele ainda está na pilha esperando pela coleta de lixo.



Na maioria dos casos, você **não** verá uma caixa de mensagem indicando a coleta de lixo, porque seu objeto foi anulado, mas o coletor de lixo ainda não executou.

Agora clique no segundo botão... nada acontece. Certo? Como não utilizamos um comando `using`, nada de método `Dispose()`. E até que o coletor de lixo execute, você não verá a caixa de mensagem no finalizador.

Agora pressione o terceiro botão, para sugerir a coleta de lixo. Você deve ver os finalizadores tanto de `Clone1` quanto de `Clone2` dispararem caixas de mensagem.



Brinque com o programa. Clique no botão Clone 1, depois no Clone 2, então no GC. Faça isso algumas vezes. De vez em quando clone 1 é coletado primeiro, outras vezes é o clone 2. Eventualmente o coletor executa mesmo que você não tenha solicitado a coleta com GC.Collect().

Finalizadores não podem depender de estabilidade

Quando você escreve um finalizador, não pode depender de executá-lo num tempo determinado. Mesmo que você chame `GC.Collect()` - o que deve ser evitado, a não ser que você tenha realmente um bom motivo para fazer isso - o método apenas **sugere** que o coletor de lixo deve executar. Ele não é uma garantia que a coleta ocorrerá em seguida. E quando ela ocorrer, você não tem como saber em que ordem os objetos serão coletados.

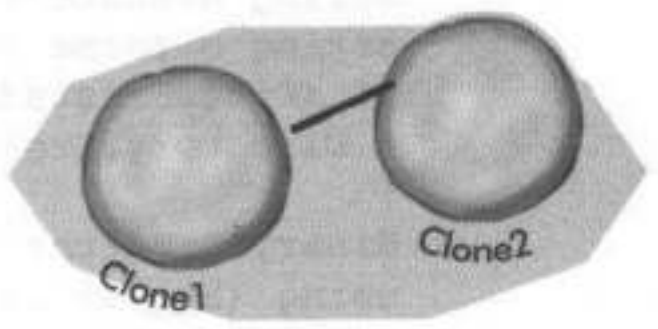
Então o que isso quer dizer, em termos práticos? Bem, pense no que acontece se você tem dois objetos com referências um do outro. Se o objeto 1 for coletado primeiro, a referência do objeto 2 estará apontando para um objeto que não existe mais. Mas se o objeto 2 é coletado primeiro, a referência do objeto 1 é que é inválida. Isso quer dizer que **você não pode depender de referências no seu finalizador** de objeto. Ou seja, é uma péssima idéia tentar fazer algo dentro de um finalizador que dependa de referências válidas.

Serialização é um ótimo exemplo de algo que você **não deveria fazer dentro de um finalizador**. Se seu objeto tem várias referências para outros objetos, a serialização depende de todos eles ainda existirem na memória... e todos os que forem referenciados por estes, e assim por diante. Então se você tentar serializar quando a coleta de lixo estiver acontecendo, você pode acabar com partes vitais faltando no programa porque alguns objetos podem ter sido coletados antes do finalizador executar.

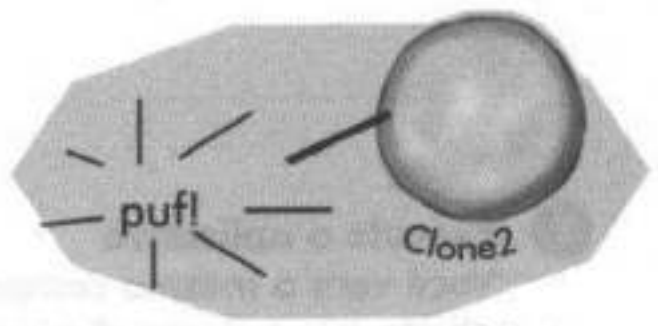
Por sorte, o C# fornece uma solução muito boa para isso: `IDisposable`. Qualquer coisa que possa modificar seus dados principais ou que dependa de outros objetos ainda existirem em memória precisa acontecer como parte do método `Dispose()`, não num finalizador.

Algumas pessoas gostam de pensar em finalizadores como um tipo de proteção contra falhas no método Dispose(). E isso faz sentido - você viu com seu objeto Clone que só porque você implementa IDisposable, isso não quer dizer que o método Dispose() será chamado. Mas você precisa ter cuidado - se seu método Dispose() depender de outros objetos ainda na pilha, chamar Dispose() dentro do finalizador pode causar problemas. A melhor forma de contornar esse problema é certificar-se de **utilizar sempre um comando using** quando criar um objeto que implemente IDisposable.

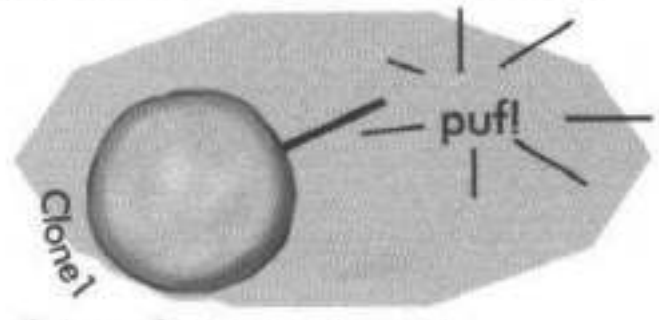
Digamos que você tenha dois objetos com referências um do outro...



...se ambos estiverem marcados para coleta de lixo ao mesmo tempo, o objeto 1 pode desaparecer primeiro...



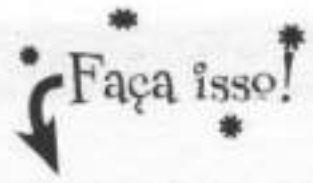
...por outro lado, o objeto 2 pode desaparecer antes do objeto 1. Você não tem como saber essa ordem...



...e é por isso que um finalizador de objeto não pode depender da existência de outro objeto na pilha.

Faça com que um objeto serialize a si mesmo em seu Dispose()

Uma vez que você tenha entendido a diferença entre Dispose() e um finalizador, é bem fácil escrever objetos que serializem a si mesmos automaticamente quando são descartados.



1 Faça a classe Clone (da página 517) ser serializável
Basta adicionar o atributo Serializable acima da classe para que possamos salvar um arquivo com ela.

```
[Serializable]
public class Clone : IDisposable
```

2 Modifique o método Dispose() de Clone para serializar ele mesmo.
Vamos usar um `BinaryFormatter` para escrever o `Clone` num arquivo em `Dispose()`:

```
using System.IO;
using System.Runtime.Serialization.Formatters.Binary;
// codigo existente
public void Dispose() {
  string filename = "C:\\Temp\\Clone.dat";
  string dirname = "C:\\Temp\\";
  if (File.Exists(filename) == false) {
    Directory.CreateDirectory(dirname);
  }
  BinaryFormatter bf = new BinaryFormatter();
  using (Stream output = File.OpenWrite(filename)) {
    bf.Serialize(output, this);
  }
  MessageBox.Show("This is " + this.ID +
                  " must.. serialize..object.");
}
```

Você vai precisar de mais alguns comandos using para acessar as classes de E/S que usaremos.

Isso é serialização simples, embora tenhamos deixado o nome do arquivo fixo no código, já que Dispose() não pode receber parâmetros.

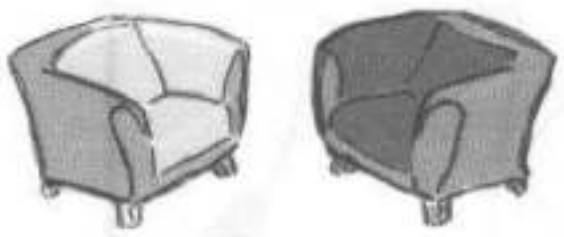
Você pode fazer com que uma classe como essa atribua um nome de arquivo para serializar a si mesma em seu construtor.

3 Execute o aplicativo
Você verá o mesmo comportamento visto nas páginas anteriores... mas antes do objeto `Clone1` ser eliminado pelo coletor de lixo, ele é serializado. Olhe o arquivo resultante e veja a representação binária do objeto.

Poder do Cérebro

O que você pensa que deve ser o restante do código da classe Superhero? Mostramo-lhes parte dele na página 512. Você pode completar o restante com o que sabe agora?

Conversa Informal



Conversa desta noite: **o método Dispose() e o finalizador se enfrentam para decidir quem é melhor.**

Dispose:

Para ser honesto, fiquei um pouco surpreso de ter sido convidado para este evento. Eu pensava que o mundo da programação tivesse chegado num consenso. Quero dizer, eu sou muito melhor do que você. Na verdade, você é bem fraquinho. Não pode nem mesmo serializar você mesmo, alterar dados importantes, nada. Você é bem instável, não?

Existe a interface precisamente **porque** eu sou tão importante. De fato, sou o único método nela!

Ok, você está certo, programadores precisar saber o que estão fazendo quando precisam de mim, ou me chamar diretamente, ou empregar um comando using. Mas eles sempre sabem quando eu vou executar e podem me usar para fazer qualquer coisa que precisem antes de liberar recursos para finalizar o objeto. Sou poderoso, confiável e fácil de usar. Três em um. E você? Ninguém sabe exatamente quando você executará ou qual será o estado do aplicativo quando você finalmente decidir aparecer.

Então não há nada que você possa fazer que eu não possa. Mas você pensa que é importante porque executa quando o coletor de lixo aparece.

Finalizador:

Como é? Essa é boa... Certo. Bem, eu não queria dizer isso, mas já que você já chegou tão baixo... pelo menos, eu não preciso de uma interface para funcionar. Sem `IDisposable`, você é apenas mais um método inútil.

Sim, sim... continue dizendo isso para você mesmo. E o que acontece quando alguém se esquece de utilizar um comando using quando instanciam seus objetos? Então você não pode ser encontrado em lugar algum.

Handles são o que seu programa usa quando ele contorna o .NET e o CLR e interagem diretamente com o Windows. Já que o .NET não sabe nada a respeito deles, não pode fazer limpeza de recursos automaticamente neles para você.

Certo, mas se você precisar fazer algo exatamente no último momento quando um objeto é eliminado pela coleta de lixo, não há como fazer isso sem mim. Pode liberar recursos de rede e handles de janelas e streams ou qualquer outra coisa que possa causar problemas para o restante do programa se não for feita uma limpeza nos recursos. Posso garantir que seus objetos lidem com o descarte de forma mais elegante – e isso não é algo desprezível.

Eu prefiro isso a essa sua pose de exibido, amigo.

não existem Perguntas Idiotas

P: Um finalizador pode usar todos os métodos e campos de um objeto?

R: Certamente. Embora você não possa passar parâmetros para um finalizador, pode usar qualquer dos campos do objeto, diretamente ou usando `this` - mas tenha cuidado, porque se esses campos fazem referência a outros objetos, estes podem já ter sido coletados. Mas, com certeza, você pode chamar qualquer método no objeto sendo finalizado (desde que eles não dependam de outros objetos).

P: O que acontecem com exceções lançadas dentro de um finalizador?

R: Boa pergunta. É completamente aceitável colocar blocos `try/catch` dentro de um método finalizador. Tente para ver. Crie uma exceção de divisão por zero dentro de um bloco try no programa `Clone` que acabamos de escrever. Capture-a e exiba uma caixa de mensagem dizendo "Acabei de capturar uma exceção" logo antes da caixa do "... acabei de ser destruído" que já implementamos. Agora execute o programa e clique no primeiro botão e então no GC. Você verá tanto a caixa da exceção quanto a da destruição do objeto (claro, é geralmente uma `idéia muito ruim` mostrar caixas de mensagem em finalizadores para objetos que forem mais do que simples brinquedos... e essas caixas podem nunca ser exibidas).

P: Com que freqüência o coletor de lixo executa?

R: Não existe resposta para essa pergunta. Ele não executa com nenhum período previsível e você não tem nenhuma forma de controle efetivo sobre ele. Você pode ter certeza de que ele executará quando seu programa encerrar. Mas para ter certeza de que ele funcione antes disso, terá de usar GC.Collect()...e mesmo assim, quando ele executar depois disso ainda será um problema.

P: Após quanto tempo depois de chamar GC.Collect() o .NET vai iniciar a coleta de lixo?

R: Quando você executa `GC.Collect()`, está dizendo ao .NET para coletar o lixo assim que possível – normalmente logo depois que o .NET acabar o que quer que esteja fazendo. Isso pode querer dizer que será bem rápido, mas você não tem realmente nenhum controle sobre isso.

P: Se eu precisar que algo com certeza execute, coloco-o num finalizador, certo?

R: É possível que um finalizador não execute. É possível impedir os finalizadores no momento da coleta de lixo. Ou os processos podem cair sem executar mais nada. Mas como regra geral, seu finalizador deve executar.

Enquanto isso, nas ruas da Vila Objeto...

CAPITÃO MARAVILHOSO... ELE VOLTOU!

MAS ALGO ESTÁ ERRADO. ELE NÃO PARECE O MESMO... E SEUS PODERES ESTÃO ESTRANHOS.

O CAPITÃO MARAVILHOSO LEVOU TANTO TEMPO PARA CHEGAR AQUI QUE O SR. FELPUDO SALVOU-O DE CAIR DE CIMA DA ÁRVORE...

Miau!

Mais tarde...

Ainda mais tarde...

Esconderijo da Coleção do Capitão Maravilhoso

ULTRA SECRETO

PUF... PUF... ARGH! ESTOU EXAUSTO.

O que está errado? Por que os poderes do Capitão estão se comportando de forma diferente? Será o seu fim?

Uma estrutura parece-se com um objeto...

Um dos tipos do .NET sobre o qual não falamos muito é o `struct`. A palavra-chave é uma abreviação de **structure** (estrutura) e ela se parece com objetos – pode ter campos e propriedades. E você pode mesmo passá-las para um método que aceite um parâmetro do tipo `Object`.

```
public struct AlmostSuperhero : IDisposable {
  public int SuperStrength;
  public int SuperSpeed { get; private set; }
  public void RemoveVillain(Villain villain)
  {
    Console.WriteLine("OK, " + villain.Name +
      " surrender and stop all the madness!");
    if (villain.Surrendered)
      villain.GoToJail();
    else
      villain.Kill();
  }
  public void Dispose() { ... }
}
```

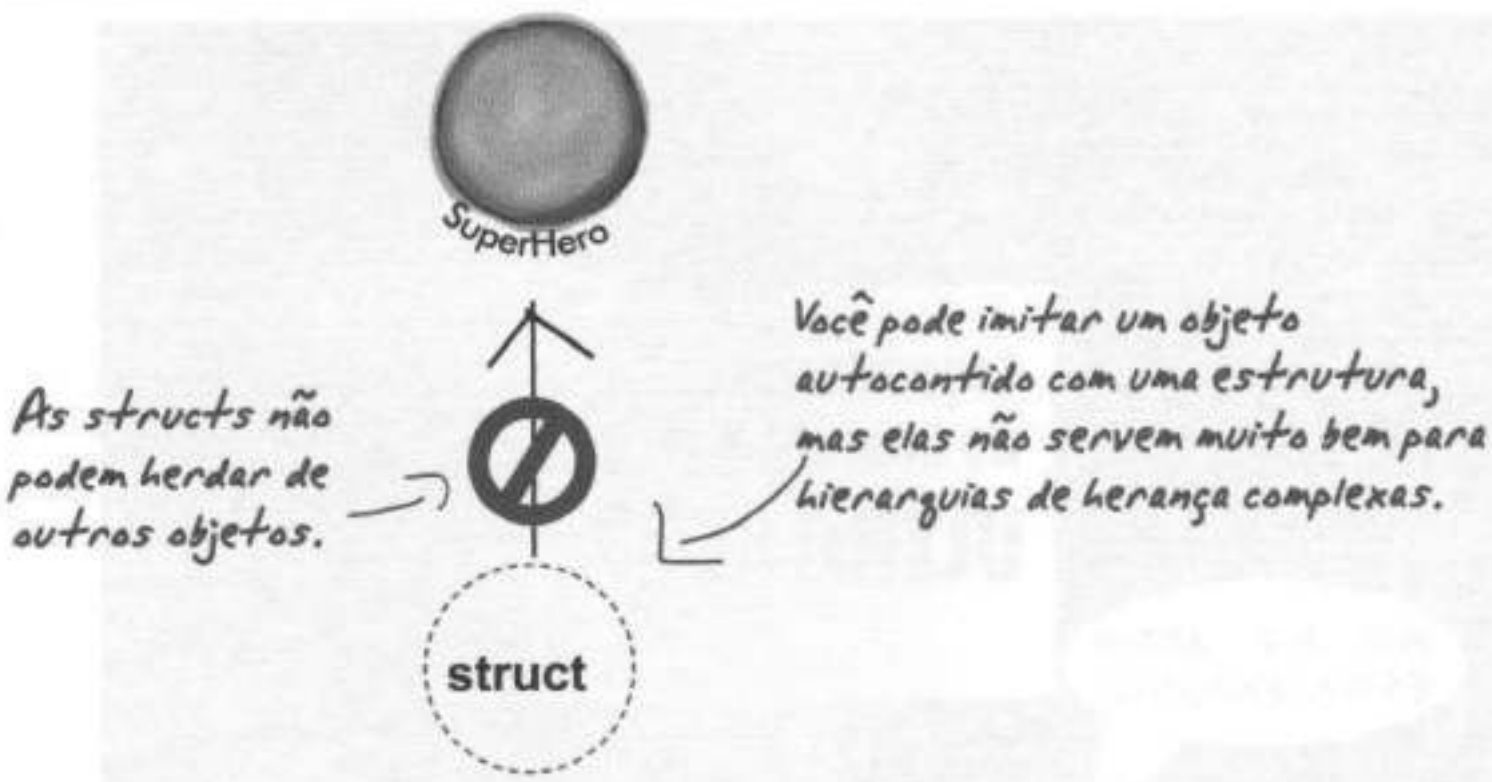
As structs podem implementar interfaces mas não podem derivar de outras classes.

Uma struct pode ter propriedades e campos...

...e definir métodos.

...mas não fica na pilha dinâmica

Mas `structs` não são objetos na pilha dinâmica. Elas podem ter métodos e campos, mas não podem ter finalizadores. Elas também não podem herdar de outras classes ou estruturas, nem nenhuma classe ou estrutura pode derivar delas.



O poder dos objetos reside na sua capacidade de imitar comportamento do mundo real, através de herança e polimorfismo.

As structs servem melhor para armazenar dados e não são tão úteis quanto os objetos quando consideramos representar comportamento efetivo.

Valores são copiados, referências são atribuídas

Você já tem uma noção de como alguns tipos são diferentes de outros. Por um lado você tem **tipos de valor** como `int`, `bool`, e `string`. Por outro, tem **objetos** como `List`, `Stream`, e `Exception`. E eles não funcionam exatamente da mesma forma, não é?

Quando você usa o sinal de igual para atribuir uma variável de tipo de valor a outro, **isso tira uma cópia do valor** e depois os dois não estão associados um ao outro. Por outro lado, quando você usa o sinal de igual com referências, você **aponta as duas para o mesmo objeto**.

Eis aqui uma rápida revisão sobre tipos de valor versus objetos.

- Declaração e atribuição de variáveis funciona exatamente como com tipos de valor ou objetos:

int e string são tipos de valor, List e Exception são objetos.

```
int howMany = 37;
string name = "Fingers";
List<float> temperatures = new List<float>();
Exception ex = new Exception("Does not compute");
```

Estes são todos inicializados da mesma forma.

- No entanto, as diferenças insinuam-se quando você começa a atribuir valores. Tipos de valor providenciam cópias. Eis um exemplo:

Esta linha copia os valores armazenados na variável fifteenMore e adiciona-lhe 15.

Alterar a variável fifteenMore não tem efeito em howMany e vice-versa.

```
int fifteenMore = howMany;
fifteenMore += 15;
Console.WriteLine("howMany has {0}, fifteenMore has {1}",
                  howMany, fifteenMore);
```

O resultado aqui mostra que `fifteenMore` (mais quinze) e `howMany` (quantos) **não** estão associadas:

```
howMany has 25, fifteenMore has 40
```

- Contudo, com atribuição de objetos, você está atribuindo referências, não valores em si:

Esta linha atribui a referência differentList para que ela aponte para o mesmo objeto que a referência temperatures.

```
temperatures.Add(56.5F);
temperatures.Add(27.4F);
List<float> differentList = temperatures;
differentList.Add(62.9F);
```

Ambas as referências apontam para o mesmo objeto.

Assim alterar a `Lista` implica que ambas as referências perceberão a atualização... já que as duas apontam para um único objeto `List`.

```
Console.WriteLine("temperatures has {0}, differentlist has {1}",
                  temperatures.Count(), differentList.Count());
```

O resultado aqui mostra que `differentList` e `temperatures`, na verdade, estão apontando para o **mesmo** objeto:

Quando você chamou differentList.Add(), ele adicionou uma nova temperatura ao objeto para o qual tanto differentList quanto temperatures apontam.

```
temperatures has 3, differentList has 3
```

Estruturas são tipos de valor; objetos são tipos de referência

Quando você cria uma `struct`, está criando um **tipo de valor**. Ou seja, quando você usa o igual numa variável do tipo `struct` para igualá-la a outra, está criando uma nova cópia da estrutura. Então mesmo que uma `struct` se pareça com um objeto, ela não se comporta como um.

1 **Crie uma struct chamada Dog**
Eis aqui uma struct bem simples para registrar o que um cão faz. Ela parece-se com um objeto, mas não é um.

```
public struct Dog {
  public string Name;
  public string Breed;
  public Dog(string name, string breed) {
    this.Name = name;
    this.Breed = breed;
  }
  public void Speak() {
    Console.WriteLine("My name is {0} and I'm a {1}.", Name, Breed);
  }
}
```

Ok, isso não é bom encapsulamento. Permita-nos fazer isso aqui - estamos tentando demonstrar algo.

2 **Crie uma classe chamada Canine**
Faça uma cópia exata da estrutura `Dog` (cão), mas troque a palavra "`struct`" por "`class`" e mude `Dog` para `Canine` (canídeo). Agora você tem uma classe `Canine` com a qual pode brincar, que é quase equivalente à estrutura `Dog`.

3 **Adicione um botão que faça algumas cópias de Dogs e Canines**
Eis o código:

```
Canine spot = new Canine("Spot", "pug");
Canine bob = spot;
bob.Name = "Spike";
bob.Breed = "beagle";
spot.Speak();
Dog jake = new Dog("Jake", "poodle");
Dog betty = jake;
betty.Name = "Betty";
betty.Breed = "pit bull";
jake.Speak();
```

Aponte seu lápis

4 **Antes de apertar esse botão...**
Escreva o que você acha que aparecerá no console quando executar esse código:

.....

.....

Veja o que aconteceu...

As referências spot e bob apontam para o mesmo objeto, então ambas mudam os mesmos campos e acessam o mesmo método Speak() (fale). Mas estruturas não funcionam dessa forma. Quando você cria betty, tira uma cópia dos dados em jake. As duas estruturas são completamente independentes uma da outra.

```
Canine spot = new Canine("Spot", "pug"); (1)
Canine bob = spot; (2)
bob.Name = "Spike";
bob.Breed = "beagle";
spot.Speak(); (3)
```

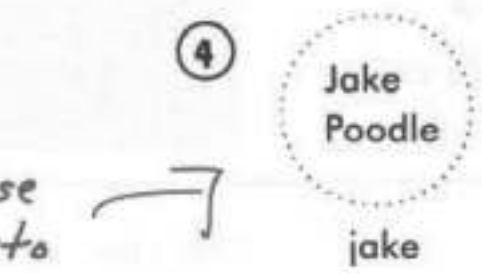
Um novo objeto Canine foi criado e a referência spot apontada para ele.

A nova variável de referência bob foi criada, mas nenhum novo objeto foi adicionado à pilha dinâmica - a variável bob aponta para o mesmo objeto que spot.

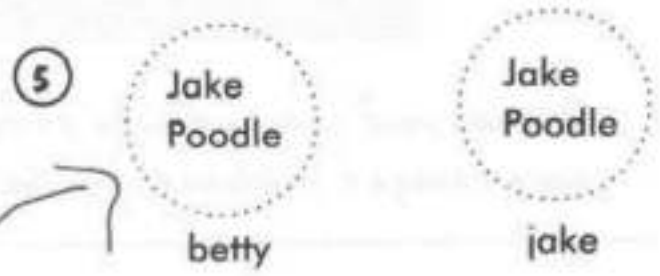
Já que spot e bob apontam para o mesmo objeto, spot.Speak() e bob.Speak() chamam o mesmo método e ambos produzem o mesmo resultado, com Spike e Beagle.

```
Dog jake = new Dog("Jake", "poodle"); (4)
Dog betty = jake; (5)
betty.Name = "Betty";
betty.Breed = "pit bull";
jake.Speak(); (6)
```

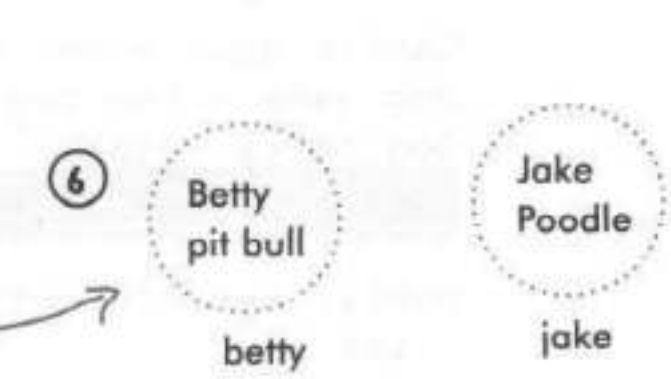
Quando você cria uma nova estrutura, isso se parece realmente semelhante a criar um objeto - você obtém uma variável que pode usar para acessar campos e métodos.



Aqui está a grande diferença. Quando você adicionou a variável betty, criou uma estrutura inteiramente nova, mesmo que não tenha usado a palavra-chave new.



Já que você criou uma cópia nova dos dados, jake não é afetado pelas mudanças feitas nos campos de betty.



Quando iguala uma estrutura a outra, você está criando uma nova CÓPIA dos dados dentro da estrutura. Isso acontece porque ela é um TIPO DE VALOR.

Pilha dinâmica versus estática: mais sobre a memória

Nos Bastidores

É fácil de entender como uma `struct` difere de um objeto - você pode fazer uma nova cópia de uma struct simplesmente usando o sinal de igual e isso não pode ser feito com um objeto. Mas o que está realmente acontecendo nos bastidores? O motivo pelo qual `structs` se comportam como outros valores é que **tipos de valor não residem na pilha dinâmica**. O CLR do .NET divide seus dados em duas partes na memória. Você já sabe que os objetos ficam numa área chamada **heap, ou pilha** dinâmica. Uma outra parte da memória também é mantida e é chamada **stack** (que em português também se traduz para pilha e podemos chamá-la de pilha estática), usada para armazenar todas as variáveis locais declaradas nos métodos e os parâmetros passados para eles. Você pode pensar na pilha estática como um monte de espaços onde se pode encaixar valores. Quando um método é chamado, o CLR adiciona mais espaços no topo dessa pilha. Quando ele retorna, os espaços são removidos.

Lembre-se, quando seu programa executa, o CLR fica gerencia ativamente a memória, lidando com a pilha dinâmica e coletando lixo.

Mesmo que você possa igualar uma variável objeto a uma struct, os dois são coisas diferentes.

O Código

Aqui está código que você pode ver num programa.

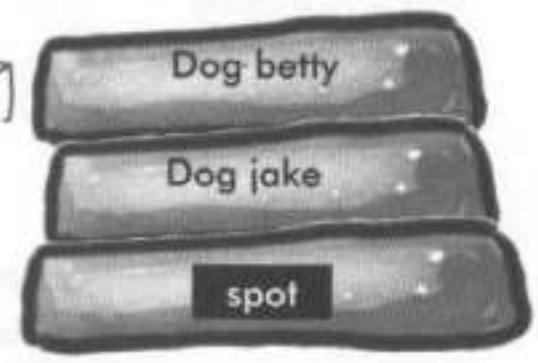
```
Canine spot = new Canine("Spot", "pug");
Dog jake = new Dog("Jake", "poodle");
```

Eis a aparência da pilha estática depois que estas duas linhas de código executam.

A pilha estática

É aqui que structs e variáveis locais ficam.

```
Canine spot = new Canine("Spot", "pug");
Dog jake = new Dog("Jake", "poodle");
Dog betty = jake;
```



Quando você cria uma nova struct - ou qualquer outro tipo de valor - um novo espaço é adicionado à pilha estática. Ele é ocupado pelo valor informado.

```
Canine spot = new Canine("Spot", "pug");
Dog jake = new Dog("Jake", "poodle");
Dog betty = jake;
SpeakThreeTimes(jake);

public SpeakThreeTimes(Dog dog) {
  int i;
  for (i = 0; i < 5; i++)
    dog.Speak();
}
```

Quando você chama um método, o CLR coloca suas variáveis locais no topo da pilha. Ele remove-os quando o método encerra.

Aponte seu lápis Solução

O que você acha que será escrito no console?

My name is Spike and I'm a beagle.
My name is Jake and I'm a poodle.

Espere um pouco. Você não acabou de dizer que podemos igualar uma variável objeto a uma struct? Se os objetos ficam na pilha dinâmica e as structs na estática, o que acontece com elas?

Quando você iguala um objeto a um tipo de valor, ele fica embutido

Algumas vezes você precisa ser capaz de escrever um método que possa ou receber um tipo de valor **ou** um de referência - quem sabe, por exemplo, um que possa trabalhar ou com uma struct Dog ou com um objeto Canine. Se você se encontrar alguma vez nessa situação, pode usar a palavra-chave object:

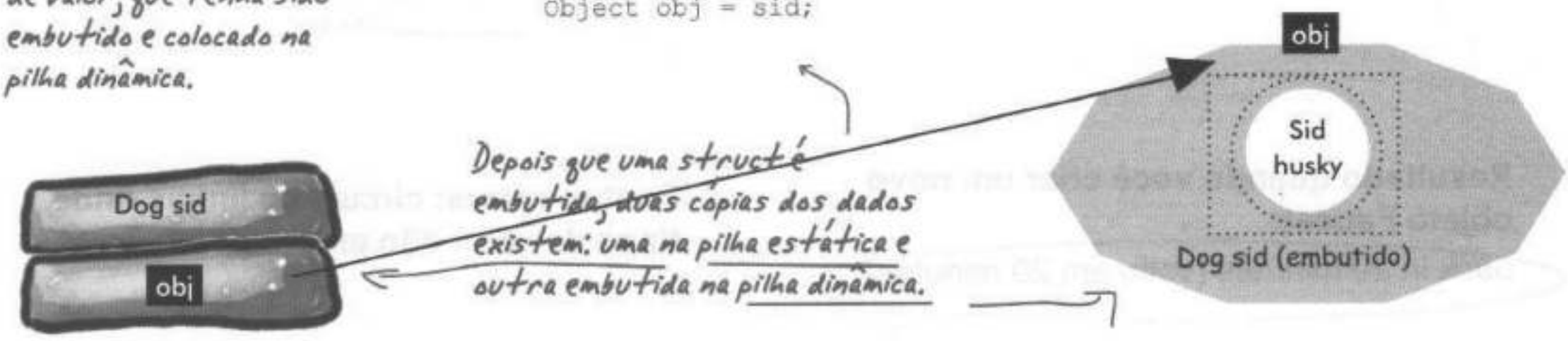
```
public WalkDogOrCanine(object anything) { ... }
```

Se você enviar uma struct a esse método, ela será **embutida** (do inglês "boxed", literalmente "encaixotada") dentro de um "contêiner" especial que permite a existir na pilha dinâmica. Enquanto esse contêiner estiver na pilha dinâmica, você não pode fazer muito com a struct. Você tem que "desencaixar" a struct para poder trabalhar com ela. Por sorte, tudo isso acontece automaticamente quando você iguala um objeto a um tipo de valor, ou passa um tipo de valor a um método que está esperando por um objeto.

Você também pode usar a palavra-chave is para ver se um objeto é uma struct, ou outro tipo de valor, que tenha sido embutido e colocado na pilha dinâmica.

1. Eis com o que as pilhas estática e dinâmica se parecem depois que você cria uma variável de objeto e a iguala a uma struct Dog.

```
Dog sid = new Dog("Sid", "husky");
Object obj = sid;
```

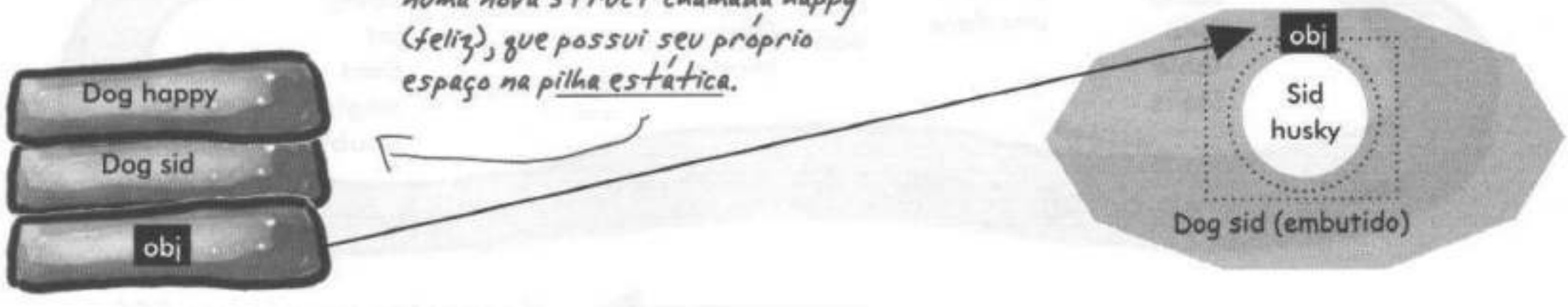


2. Se você quiser desembutir o objeto, tudo a fazer é coerção para o tipo correto e ele será retirado automaticamente do contêiner. É aqui que a palavra-chave `is` é útil.

```
if (obj is Dog)
  Dog happy = (Dog) obj;
```

Estas são structs, então a não ser que elas tenham sido embutidas, não estarão na pilha dinâmica.

Depois que esta linha executa, você terá uma terceira cópia dos dados numa nova struct chamada happy (feliz), que possui seu próprio espaço na pilha estática.



Jogos de Verão

Sua **tarefa** é pegar trechos da piscina e colocá-los nas lacunas no código. Você **pode** usar o mesmo trecho mais de uma vez e não precisa usar todos eles. Seu **objetivo** é fazer com que o código escreva no console o resultado mostrado quando uma **nova instância da classe Faucet (torneira) for criada**.

```
public class Faucet {
  public Faucet() {
    Table wine = new Table();
    Hinge book = new Hinge();
    wine.SetA(book);
    book.SetB(wine);
    wine.Lamp(10);
    book.Garden.Lamp("back in");
    book.Bulb *= 2;
    wine.Lamp("minutes");
    wine.Lamp(book);
  }
}
```

```
public _________ Table {
  public string Stairs;
  public Hinge Floor;
  public void Set(Hinge b) {
    Floor = b;
  }
  public void Lamp(object oil) {
    if (oil ____ int)
      _________.Bulb = (int)oil;
    else if (oil ____ string)
      Stairs = (string)oil;
    else if (oil ____ Hinge) {
      _________ vine = oil ____ _________;
      Console.WriteLine(vine.Table()
    + " " + _________.Bulb + " " + Stairs);
    }
  }
}
public _________ Hinge {
  public int Bulb;
  public Table Garden;
  public void Set(Table a) {
    Garden = a;
  }
  public string Table() {
    return _________.Stairs;
  }
}
```

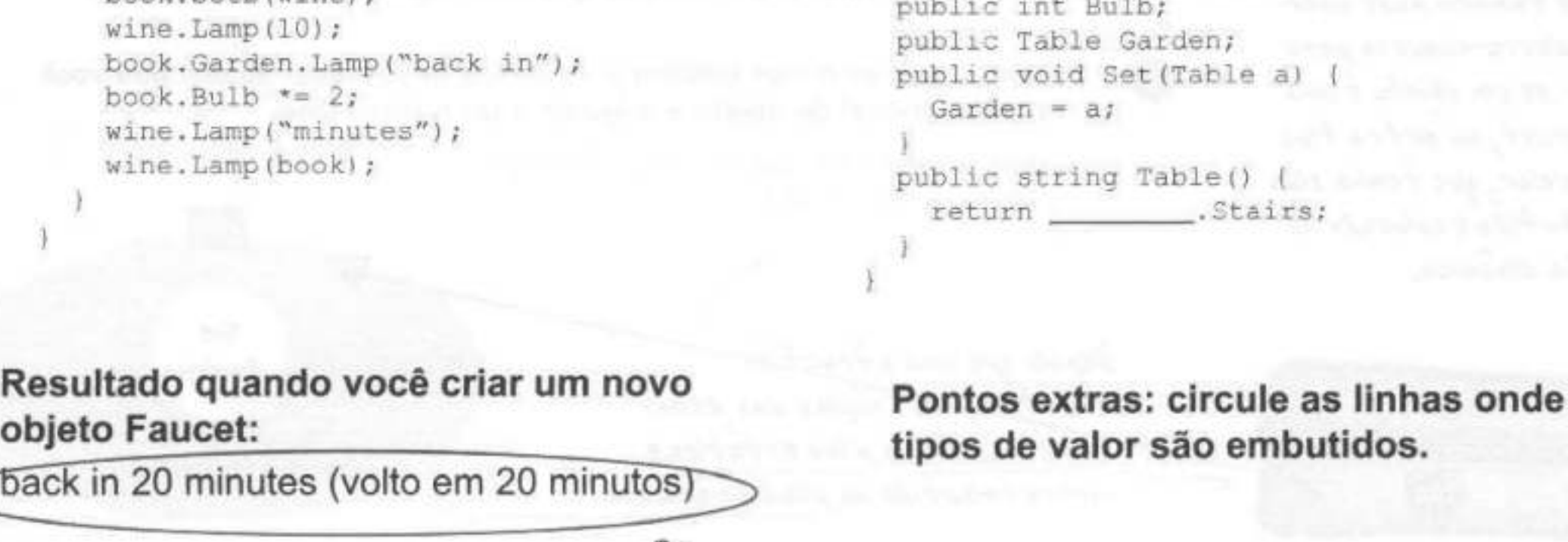
Resultado quando você criar um novo objeto Faucet:

back in 20 minutes (volto em 20 minutos)

Aqui está seu objetivo... conseguir este resultado.

Pontos extras: circule as linhas onde tipos de valor são embutidos.

Nota: cada trecho na piscina pode ser usado mais de uma vez.



Respostas na página 558.

não existem Perguntas Idiotas

P: Certo, volte um pouco. Por que eu devo saber sobre a pilha estática?

R: Porque entender a diferença entre as pilhas estática e dinâmica ajuda a manter seus tipos de referência e de valor corretamente. É fácil de esquecer que as structs e os objetos funcionam de forma bem diferente - quando você usa o sinal de igual em ambas, elas parecem bastante semelhantes. Tendo alguma idéia de como o .NET e o CLR lidam com as coisas nos bastidores ajuda a entender porque os tipos de referência e de valor são diferentes.

P: E quanto a embutir? Por que isso é importante para mim?

R: Porque você precisa saber quando as coisas acabam guardadas na pilha estática e quando os dados são copiados. As duas coisas exigem mais memória e consomem tempo. Quando você faz algo assim algumas vezes (ou até algumas centenas de vezes) no seu programa, não vai notar diferença. Mas digamos que você esteja escrevendo um programa que repita operações como estas até milhões de vezes por segundo. Isso não é tão absurdo de se pensar, já que é exatamente o que o simulador de colméia fazia. Se você perceber que o programa está tomando mais memória, ou ficando cada vez mais lento, é possível torná-lo mais eficiente evitando que tipos de valor sejam embutidos na parte do código repetida.

P: Eu entendi como você conseguiu uma cópia nova quando iguala uma variável struct a outra. Mas em que isso vai ser útil para mim?

R: Um aspecto em que isso é bastante útil é o **encapsulamento**. Observe este campo familiar de uma classe que conhece a própria posição:

```
private Point location;
public Point Location {
  get { return location; }
}
```

Se `Point` fosse uma classe, isso seria um péssimo exemplo de encapsulamento. Não importa que `location` seja privado, como você criou uma propriedade apenas-leitura pública que retorna uma referência para o campo, qualquer outro objeto seria capaz de acessá-lo.
Por sorte, `Point` é na verdade uma struct. E isso quer dizer que a propriedade pública `Location`, na verdade, retorna uma nova cópia do ponto. O objeto que a utilizar pode fazer o que quiser com ela - nenhuma das alterações afetará o campo privado location.

P: Se Point é uma struct, há outras estruturas com as quais eu tenho trabalhado todo esse tempo?

R: Sim! Uma struct realmente útil e muito comum quando se trabalha com gráficos e formulários é **Rectangle**. Ela tem alguns métodos muito úteis bastante práticos quando é preciso determinar limites **e tamanho** e ela automaticamente calcula a posição de seu topo, lado esquerdo e direito, sua largura e altura.
Outra struct útil com a qual você já trabalhou é **Size**. Você já a viu em ação - usou-a quando estava determinando o tamanho de uma string com o método `MeasureString()`.

P: Quando saber se devo criar uma struct ou uma classe?

R: Na maior parte do tempo, os programadores usam classes. Structs têm muitas limitações que podem realmente deixá-las difíceis de lidar para tarefas grandes. Elas não aceitam herança, abstração nem polimorfismo e você já sabe como esses aspectos são importantes para facilitar o desenvolvimento de programas.
As structs são realmente úteis quando você tem dados pequenos e limitados com o qual precisa trabalhar repetidamente. Retângulos e pontos são bons exemplos - não há muitas coisas que possam ser feitas com eles, mas você os utiliza repetidas vezes. As structs tendem a ser relativamente pequenas e limitadas quanto a seu escopo. Se você perceber que tem um pedaço pequeno de alguns tipos diferentes de dados que gostaria de armazenar num campo de uma classe ou passar para um método como parâmetro, provavelmente, terá um bom candidato para criar uma struct.

Uma struct pode ser muito útil quando você quer melhorar o encapsulamento numa classe, porque uma propriedade apenas-leitura que retorna uma struct sempre entrega uma cópia dos dados.

Aponte seu lápis

Esse método supostamente deveria matar um objeto clone, mas ele não funciona. Qual o motivo?

```
private void SetCloneToNull(Clone clone) {
  clone = null;
}
```

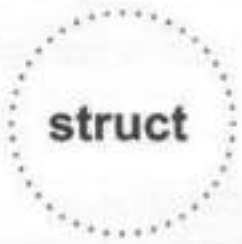
.....

.....

Pergunta surpresa, sabichão!
Resposta na página 551.

Capitão Maravilhoso... mas nem tanto

Com toda essa conversa sobre embutir tipos, você deve ter agora uma boa idéia do que aconteceu com o Capitão Maravilhoso com menos poderes e cansado. De fato, ele não é o Capitão Maravilhoso de forma alguma, mas sim uma estrutura embutida:



VS.

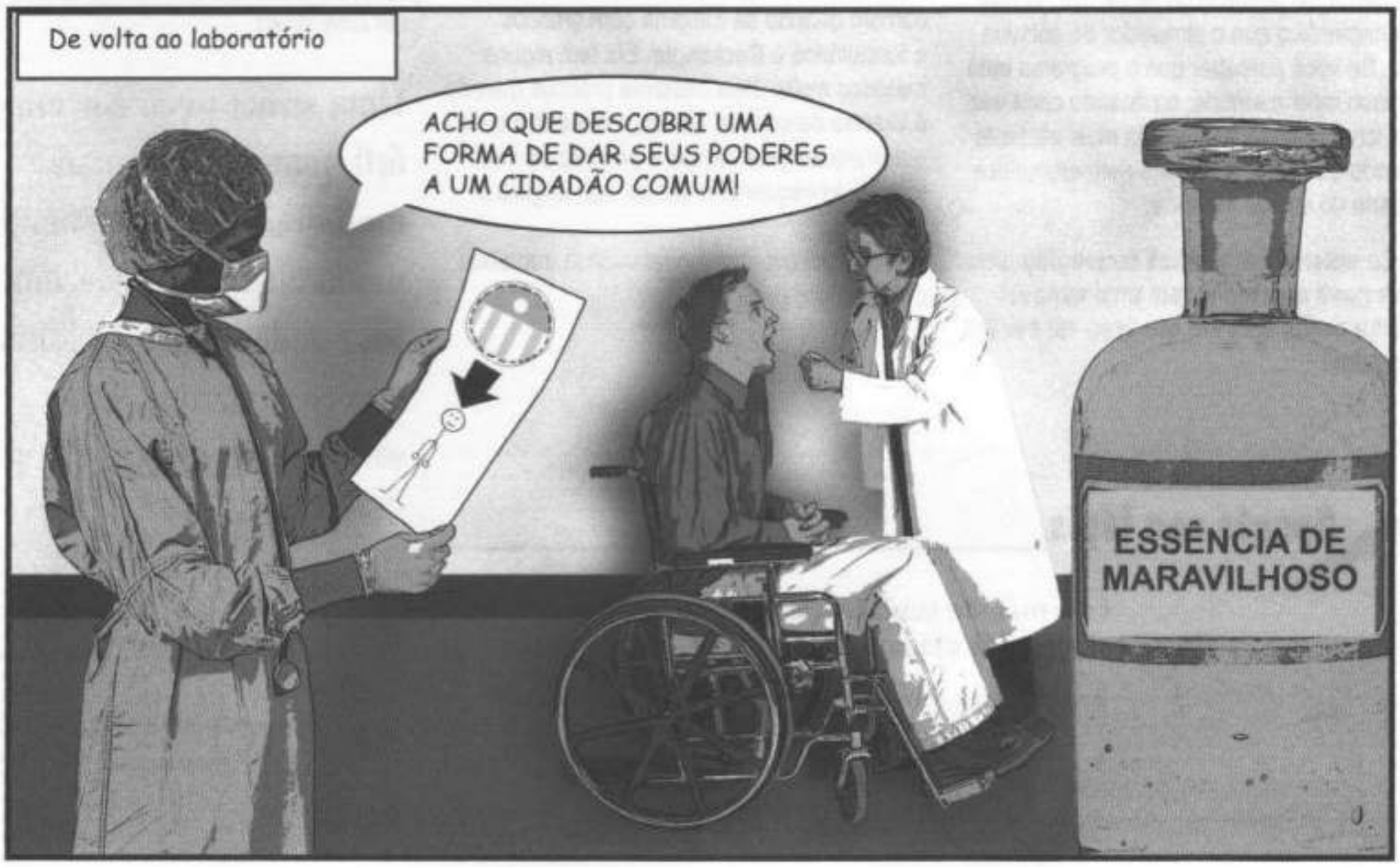
Esta é uma grande vantagem das structs (e outros tipos de valor) - você pode facilmente criar cópias deles.

1. **Structs não podem herdar de classes ou implementar interfaces**
   Não admira que os superpoderes do Capitão pareçam tão fracos! Ele não tem nenhum comportamento herdado.

2. **Structs só podem existir na pilha dinâmica se estiverem embutidas**
   Se não estiverem embutidas, structs nunca podem ser contidas na pilha dinâmica.

1. **Você não pode criar uma nova cópia de um objeto**
   Quando você iguala uma variável de objeto à outra, está copiando uma **referência** para o **mesmo** objeto.

2. **Você pode usar a palavra-chave "as" com um objeto**
   Objetos permitem polimorfismo quando aceitam que um objeto funcione como qualquer um dos quais ele herda.



Métodos de extensão adicionam novos comportamentos a classes EXISTENTES

Algumas vezes você precisa estender uma classe da qual não pode herdar, como, por exemplo, uma classe selada (marcada com a palavra-chave "sealed" - muitas classes do .NET são seladas, então não se pode herdar delas). O C# lhe dá uma ferramenta poderosa para fazer isso: **métodos de extensão**. Quando você adiciona uma classe com métodos deste tipo ao seu projeto, ela **adiciona novos métodos a classes** que já existem. Tudo a fazer é criar uma classe estática e adicionar um método estático que aceite uma instância da classe alvo como primeiro parâmetro usando a palavra-chave `this`:

Então digamos que você tenha uma classe OrdinaryHumam (humano normal):

A classe OrdinaryHuman é selada, então não se pode criar subclasses dela. Mas e se quisermos adicionar um método?

```
public sealed class OrdinaryHuman {
  private int age;
  int weight;
  public OrdinaryHuman(int weight){
    this.weight = weight;
  }
  public void GoToWork() { // codigo para trabalhar
  public void PayBills() { // codigo para pagar contas
}
```

Você usa um método de extensão especificando o primeiro parâmetro usando a palavra-chave this.

Já que queremos estender a classe OrdinaryHuman, fazemos com que o primeiro parâmetro seja this OrdinaryHuman.

```
public static class SuperSoldierSerum {
  public static string BreakWalls(this OrdinaryHuman h, double wallDensity) {
    return ("I broke through a wall of " + wallDensity + " density.");
  }
}
```

Métodos de extensão são sempre estáticos, então ele deve existir dentro de classes estáticas.

Assim que a classe SuperSoldierSerum (soro de supersoldado) é acrescentada ao projeto, OrdinaryHuman ganha um método BreakWalls (quebrar paredes) diretamente - desde que tenha acesso à classe SuperSoldierSerum.

```
private void button1_Click(object sender, EventArgs e) {
  OrdinaryHuman steve = new OrdinaryHuman(185);
  Console.WriteLine(steve.BreakWalls(89.2));
}
```

Aponte seu lápis Solução

Este método deveria matar um objeto Clone, mas ele não funciona. Qual o motivo?

```
private void SetCloneToNull(Clone clone) {
  clone = null;
}
```

Logo o parâmetro clone está apenas na pilha estática e igualá-lo a null não afeta de forma alguma a pilha dinâmica.

Tudo o que este método faz é igualar seu próprio parâmetro a null, mas o parâmetro é apenas uma referência ao Clone. É como colocar um rótulo num objeto e depois arrancá-lo.

não existem Perguntas Idiotas

P: Diga-me de novo porque, em vez de usar extensões, eu não poderia adicionar novos métodos necessários diretamente ao meu código de classes.

R: Você pode fazer isso e, provavelmente, até é recomendado se estamos falando de adicionar um único método numa única classe. Métodos de extensão devem ser usados com moderação e somente quando for impossível alterar a classe com a qual você está trabalhando por algum motivo (como, por exemplo, se ela for parte do Framework .NET ou desenvolvida por terceiros). Os métodos de extensão tornam-se realmente poderosos quando você precisa acrescentar comportamento a alguma coisa a que **normalmente não teria acesso**, como um tipo de dados ou outro objeto nativo do .NET ou outra biblioteca.

P: Por que usar métodos de extensão então? Não é possível apenas estender uma classe usando herança?

R: Claro. Mas quando você precisasse usar seus objetos personalizados, qualquer código que utilizasse os objetos originais, não estendidos, teria de ser alterado. Com métodos de extensão, você pode mudar o comportamento de grupos inteiros de objetos e mesmo adicionar funcionalidades a algumas das classes mais básicas do Framework .NET.
Estender uma classe cria novo comportamento, mas exige que você utilize as novas subclasses se quiser se aproveitar dessas novas características.

P: Os meus métodos de extensão afetarão todas as instâncias de uma classe, ou apenas algumas específicas?

R: Afetará todas as instâncias da classe estendida. De fato, uma vez que você tenha criado um método de extensão, ele aparecerá no IDE ao lado dos métodos normais da classe em questão.

Ah, entendi! Então usamos métodos de extensão para adicionar novos comportamentos a classes nativas do Framework .NET, certo?

Exatamente! Existem algumas classes das quais não se pode herdar.
Abra qualquer projeto e adicione uma classe tentando digitar o seguinte:

```
public class x : string { }
```

Tente compilar o código - o IDE vai levantar um erro porque algumas das classes do .NET são **seladas**, ou seja, você não pode herdar delas. (Você pode fazer isso com suas próprias classes, também! Basta adicionar a palavra-chave sealed na sua classe depois de qualquer modificador de acesso `público` e nenhuma outra classe terá permissão de herdar dela.) Métodos de extensão dão a você uma forma de estender uma classe assim, mesmo que você não possa herdar dela.

Mas isso não é tudo o que você pode fazer com métodos de extensão. Além de estender classes, você pode também estender interfaces. Tudo a fazer é usar um nome de **interface**, em vez de um de classe, depois da palavra-chave this no primeiro parâmetro do método de extensão. Quando fizer isso, o método de extensão é adicionado a **todas as classes que a implementem interface**. Lembra-se daquele código LINQ que adicionou no seu simulador no capítulo 12? A LINQ é toda implementada usando métodos de extensão que estendem a interface `IEnumerable` (você aprenderá muito mais sobre a LINQ no capítulo 15).

Esta é outra coisa que você simplesmente não pode fazer com herança - não existe forma de herdar de uma interface.

Estendendo um tipo básico: string

Freqüentemente, você não tem a chance de alterar o comportamento dos tipos mais fundamentais, como strings. Mas com os métodos de extensão, pode fazer exatamente isso!

Faça isso!

1 Coloque todos os seus métodos de extensão num namespace separado.
É uma boa idéia manter todos os seus métodos de extensão num namespace diferente do restante do seu código. Dessa forma, você não terá problemas para encontrá-los para usar noutro programa. Crie uma classe estática para conter seus métodos, também.

Usar um namespace separado é uma boa ferramenta de organização.

```
namespace Emergency {
  public static class HumanExtensions {
```

A classe necessita ser estática.

2 Crie um método de extensão estático e defina seu primeiro parâmetro como `this` e então o tipo que você está estendendo.
As duas coisas principais que deve saber quando declara um método de extensão é que ele precisa ser estático e ter a classe a qual estende como seu primeiro parâmetro.

O termo this string diz que estamos estendendo a classe string.

O método de extensão precisa ser estático também.

```
public static bool IsDistressCall (this string s){
```

3 Coloque código para avaliar strings dentro do método

```
public static class HumanExtensions {
  public static bool IsDistressCall(this string s){
    if (s.Contains("Help!"))
      return true;
    else
      return false;
  }
}
```

Isto examina a string procurando por um certo valor... algo que, com certeza, não existe na classe string padrão.

4 Crie um formulário e acrescente nele uma string.
Adicione `using Emergency;` ao seu código. Agora, quando você usar uma string, pode usar sem nenhum outro trabalho os métodos de extensão. Você pode ver isso você mesmo digitando o nome da variável string e então um ponto:

```
string message1;
message1 = "An army of clones is wreaking havoc at the factory. Help!";
message1.
```

Assim que você digita o ponto, o IDE mostra uma janela auxiliar com todos os métodos de strings... incluindo o seu de extensão.

Comente a linha using e o método de extensão desaparecerá da janela do IntelliSense.

Esse exemplo de brincadeira mostra a você a sintaxe dos métodos de extensão. Para ter uma idéia real de quão úteis eles são, basta esperar pelo próximo capítulo. Ele é a respeito da LINQ, que é implementanda inteiramente com métodos de extensão.

Ímãs de Geladeira de Extensão

Arrume os ímãs para produzir o seguinte resultado:

a buck begets more bucks
(dinheiro gera mais dinheiro)

```
namespace Upside {

public static class Margin {

public static void SendIt

}

public static string ToPrice

}
```

```
namespace Sideways {
using Upside;

public static class Ticker {
```

```
}
}
}
```

```
s.SendIt()

bool b = true;

Console.Write(s);

public static string Green

if (n == 1)
  return "a buck ";

.SendIt();

i = 3;

if (b == true)
  return "be";

(this string s) {

b.Green();

b.Green();

else
  return "gets";

b = false;

else
  return " more bucks";

(this bool b) {

int i = 1;

(this int n) {

public static void Main {

string s = i.ToPrice()

i.ToPrice()
```

Ímãs de Geladeira de Extensão

Arrume os ímãs para produzir o seguinte resultado:

a buck begets more bucks

(dinheiro gera mais dinheiro)

O namespace Upside (lado de cima) tem as extensões. O Sideways (laterais) contém o ponto de entrada.

A classe Margin estende as strings adicionando um método chamado SendIt() (envia) que simplesmente escreve a string no console e estende os ints acrescentando um método chamado ToPrice() (para preço) que retorna a buck (um dólar) se o inteiro for igual a 1, ou more bucks (mais dinheiro)* se não for o caso.

O método ponto de entrada usa as extensões adicionadas à classe Margin.

```
namespace Upside {

public static class Margin {

  public static void SendIt    (this string s) {

    Console.Write(s);

  }

public static string ToPrice    (this int n) {

    if (n == 1)
      return "a buck ";

    else
      return " more bucks";

  }

public static string Green      (this bool b) {

    if (b == true)
      return "be";

    else
      return "gets";

  }

}
```

```
namespace Sideways {
  using Upside;

  public static class Ticker {

    public static void Main {

      int i = 1;

      string s = i.ToPrice()

      s.SendIt()

      bool b = true;

      b.Green();

      b = false;

      b.Green();

      i = 3;

      i.ToPrice()    .SendIt();

    }

  }

}
```

É aqui que a classe Margin estende os booleanos adicionando um método Green() neles. Se eles forem verdadeiros, Green retorna be, senão ele retorna gets.

O método Green (verde) extense os booleanos - ele retorna a string be (ser) se a variável é verdadeira, e gets (obtém) se ela for falsa.

* NT: buck é um apelido para a nota de um dólar, similar ao nosso uso de, por exemplo, "barão" para mil reais. Assim, as frases são na realidade algo como "um barão" e "mais barões", referindo-se, no entanto, a alguns poucos dólares.

O UNIVERSO

Capitão Maravilhoso Renascido

A morte não foi o fim!

Por Buck Barnes

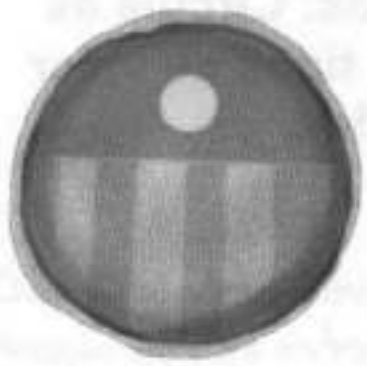
DA REDAÇÃO DO UNIVERSO

VILA OBJETO

Capitão Maravilhoso reverte sua serialização, fazendo um retorno sensasional

Numa incrível reviravolta, o Capitão Maravilhoso retornou para a Vila Objeto. No mês passado, seu caixão foi encontrado vazio, com apenas uma estranha nota deixada onde seu corpo deveria estar. A análise da nota a revelou o DNA do objeto do Capitão - todos os últimos valores de seus campos, capturados fielmente em formato binário.

Hoje, esses dados foram trazidos de volta à vida. O Capitão está de volta, com a serialização revertida através de sua própria brilhante nota. Quando perguntado como ele concebeu tal plano, o Capitão apenas levantou os ombros e murmurou "capítulo 9". Fontes próximas do Capitão recusaram-se a comentar o significado dessa resposta misteriosa, mas admitem que antes de sua tentativa mal sucedida de capturar o Trapaceiro, o Capitão passou muito tempo lendo livros, estudando o método Dispose e persistência. Esperamos que o Capitão Maravilhoso...



O Capitão Maravilhoso está de volta!

...leia mais em MARAVILHOSO na página A-5

Solução dos Jogos de Verão

O método Lamp() (lâmpada) dá valor a diversas strings e ints. Se você chamá-lo com um int, ele atribui o campo Bulb (bulbo) para qualquer objeto apontado por Hinge (dobradiça).

Resultado quando você criar um objeto Faucet:
back in 20 minutes

```
public class Faucet {
  public Faucet() {
    Table wine = new Table();
    Hinge book = new Hinge();
    wine.SetA(book);
    book.SetB(wine);
    wine.Lamp(10);
    book.Garden.Lamp("back in");
    book.Bulb *= 2;
    wine.Lamp("minutes");
    wine.Lamp(book);
  }
}
```

É por isso que Table (mesa, tabela) deve ser uma struct. Se ela fosse uma classe, então wine (vinho) apontaria para o mesmo objeto que book.Garden (livro.jardim), o que faria com que a string back in fosse sobrescrita.

Pontos extras: Circule as linhas onde tipos de valor são embutidos.

Uma vez que o método Lamp() tem um parâmetro de tipo object, quando uma string ou int é passada elas são automaticamente embutidas.

```
public struct Table {
  public string Stairs;
  public Hinge Floor;
  public void Set(Hinge b) {
    Floor = b;
  }
  public void Lamp(object oil) {
    if (oil is int)
      Floor.Bulb = (int)oil;
    else if (oil is string)
      Stairs = (string)oil;
    else if (oil is Hinge) {
      Hinge vine = oil as Hinge;
      Console.WriteLine(vine.Table()
        + " " + Floor.Bulb + " " + Stairs);
    }
  }
}
public class Hinge {
  public int Bulb;
  public Table Garden;
  public void Set(Table a) {
    Garden = a;
  }
  public string Table() {
    return Garden.Stairs;
  }
}
```

Se você passar uma string para Lamp, ela atribui o campo Stairs (escadas) para o que quer que esteja na sequência de caracteres.

Lembre-se, a palavra-chave as funciona apenas com classes, não com estruturas.

Tanto Hinge quanto Table tem um método Set(). O de Hinge atribui para seu campo de tipo Table chamado Garden e o método de Table atribui para seu campo de tipo Hinge chamado Floor (chão, piso).

15 LINQ

Controle seus dados

É um mundo voltado para dados... é melhor você saber como viver nele.

Foram-se os tempos em que você podia programar por dias, mesmo semanas, sem ter de lidar com **vários dados**. Nos dias de hoje, **tudo se relaciona com dados**. De fato, com freqüência, você terá de lidar com dados oriundos de mais de uma fonte... e em **mais de um formato**. Bases de dados, XML, coleções de outros programas... é tudo parte do trabalho de um bom programador C#. E é aí que o **LINQ** entra. Ela não só permite **consultar dados** de uma forma simples e intuitiva, como também fazer **agrupamentos** e unir **dados de muitas fontes**.

Um projeto fácil...

A Empresa de Papel da Vila Objeto quer fazer uma promoção em conjunto com o Café Starbuzz. Eles têm um programa de fidelização de clientes que registra quem compra qual tipo de café e com que freqüência. A Empresa de Papel quer saber quais dos seus **compradores são também clientes regulares do Starbuzz** e enviar-lhes uma caneca gratuita e um cupom que vale uma unidade de seu tipo de café favorito... e cabe a você combinar os dados para gerar a lista de compradores para enviar os brindes.

...mas os dados estão espalhados por todo lado

O Starbuzz mantém seus dados usando classes, agrupadas numa grande `lista`. Mas os dados da Empresa de Papel estão numa base de dados (feita no capítulo 1). Queremos encontrar qualquer comprador do Starbuzz que gastou mais de R$ 90 e esteja na lista de contatos da Empresa de Papel e fazer uma lista deles: **queremos o nome da pessoa, a empresa onde ela trabalha,e seu tipo de café preferido no Starbuzz**.

Os dados do Starbuzz estão numa List<>

O pessoal do Starbuzz forneceu um programa que se conecta ao site deles e puxa todos os dados numa `lista` da classe `StarbuzzData`.

Aqui estão a classe e uma enum no código do Starbuzz.

Você deverá conseguir a lista de dados do Starbuzz e encontrar os compradores que também constem na lista de clientes da Empresa de Papel.

```
public class StarbuzzData
{
  public string Name { get; set; }
  public Drink FavoriteDrink { get; set; }
  public int MoneySpent { get; set; }
  public int Visits { get; set; }
}
public enum Drink {
  BoringCoffee,
  ChocoRockoLatte,
  TripleEspresso,
  ZestyLemonChai,
  DoubleCappuccino,
  HalfCafAmericano,
  ChocoMacchiato,
  BananaSplitInACup,
}
```

Você já tem os dados de compradores

Você criou a lista de contatos da Empresa de Papel da Vila Objeto no capítulo 1 - ela tem parte dos dados de que você precisa.

Base de dados ContactDB

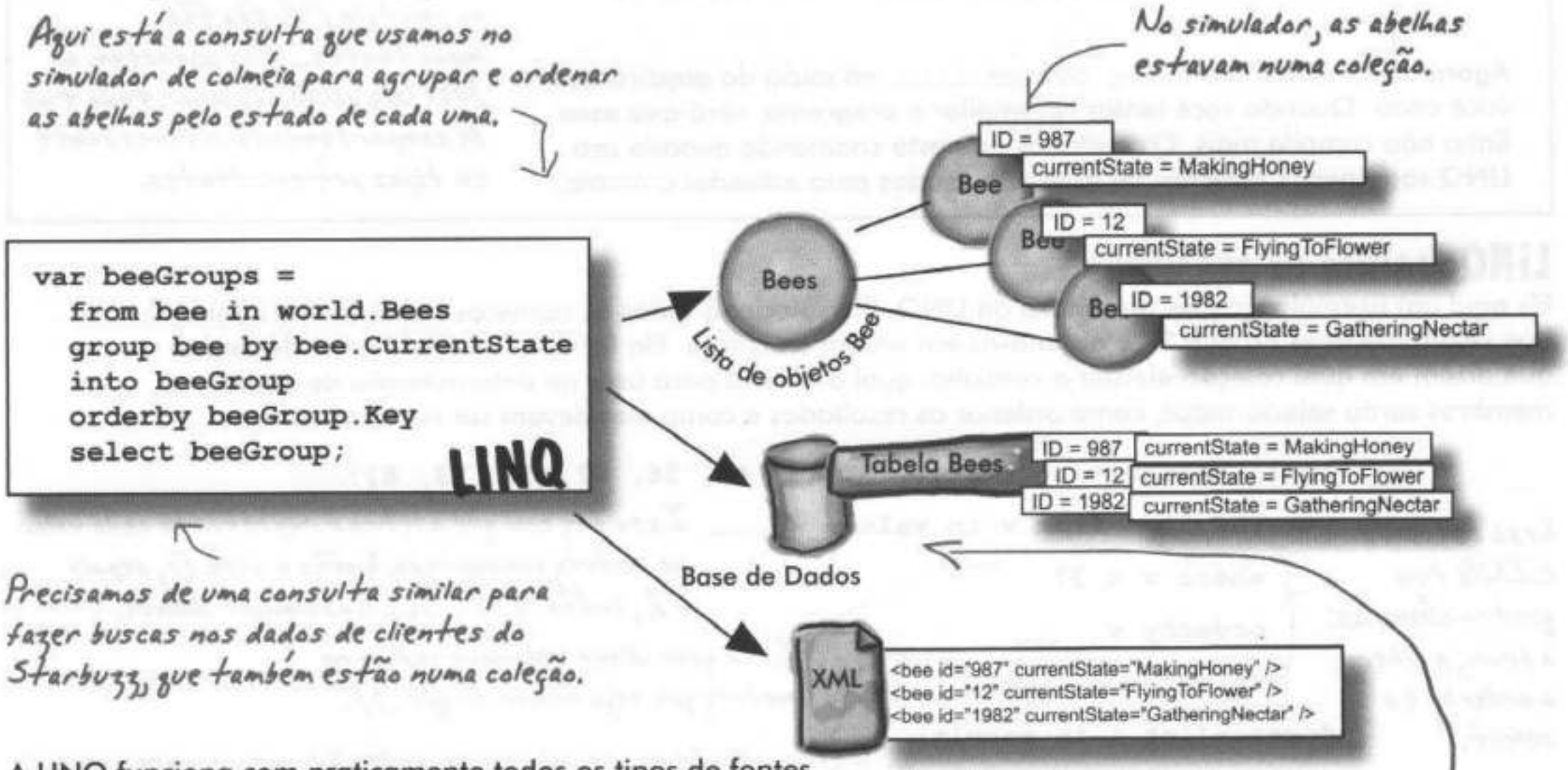
Todos os dados de compradores da Empresa de Papel estão numa base de dados.

Poder do Cérebro

Como você pode combinar os dados do Starbuzz e da Empresa de Papel para conseguir uma lista de dados completa?

LINQ pode buscar dados em múltiplas fontes

Você usou LINQ no simulador de colméia para determinar o que cada grupo de abelhas estava fazendo. Você aproveitou-se do poder da LINQ naquela ocasião para escrever consultas simples para buscar dados numa coleção. A LINQ pode trabalhar com os dados do Starbuzz da mesma forma que fez com as abelhas, ajudando-o a usar consultas para buscar dados de compradores. Desde que uma coleção implemente a interface `IEnumerable`, você pode usar consultas LINQ nela.
Mas a LINQ também permite trabalhar com mais do que apenas coleções. Você pode usar as mesmas consultas para buscar dados numa base, ou mesmo num documento XML. Então uma vez que você tenha coleções sobre controle, poderá usar a LINQ na base de dados da Empresa de Papel.



A LINQ funciona com praticamente todos os tipos de fontes de dados utilizáveis no .NET. Seu código precisa de um `using System.Linq;` no início do arquivo, mas é só isso. Melhor ainda, o IDE automaticamente coloca uma referência à LINQ no início de cada arquivo de código criado no Visual Studio 2008. Então se é essa a versão que você está usando (ou uma mais recente), é só começar a programar e a LINQ estará disponível para você.

Coleções do .NET já são preparadas para usar LINQ

Todos os tipos de coleção do .NET implementam a interface `IEnumerable`. Digite `System.Collections.Generic.IEnumerable` na sua janela do IDE, clique com o botão direito na linha e selecione "Go to Definition". Você verá que a interface define um método `GetEnumerator()`:

Todas as coleções e matrizes implementam IEnumerable. Muitas delas fazem isso implementando IEnumerable<T>, que herda de IEnumerable.

```
namespace System.Collections.Generic {
  public interface IEnumerable<T> : IEnumerable {
    // Summary:
    //     Returns an enumerator that iterates through the collection.
    //
    // Returns:
    //   A System.Collections.Generic.IEnumerator<T> that can be
    //     used to iterate through the collection.
    IEnumerator<T> GetEnumerator();
  }
}
```

Esse T implica que IEnumerable vai funcionar com qualquer objeto ou tipo.

Este é o único método declarado na interface. Todas as coleções implementam esse método. Você pode criar seu próprio tipo de coleção que implemente IEnumerable também... e se você fizer isso, poderá usar LINQ nela.

Nos Bastidores

LINQ usa **métodos de extensão** para permitir-lhe consultar, ordenar e atualizar dados. Veja você mesmo: crie uma array de `inteiros` chamada `linqtest`, coloque alguns números nelas e então digite essas linhas de código (não se preocupe, você vai aprender o que elas fazem num minuto):

```
var result = from i in linqtest where i < 3 select i;
```

Agora comente a linha `using System.Linq;` no início do arquivo que você criou. Quando você tentar recompilar o programa, verá que essa linha não compila mais. Os métodos que está chamando quando usa LINQ são apenas métodos de extensão usados para estender a matriz.

Agora você pode ver porque os métodos de extensão mostrados no capítulo 14 são tão importantes... eles permitem ao .NET (e a você) adicionar todo tipo de comportamento interessante em tipos pré-existentes.

LINQ facilita as consultas

Eis aqui um exemplo simples da sintaxe da LINQ. Ela seleciona todos os números numa matriz inteira que sejam menores do que 37 e ordena-os em ordem crescente. Ela faz isso usando quatro **cláusulas** que dizem em qual coleção efetuar a consulta, qual o critério para usar na determinação de quais membros serão selecionados, como ordenar os resultados e como eles devem ser retornados.

```
int[] values = new int[] {0, 12, 44, 36, 92, 54, 13, 8};
var result = from v in values
             where v < 37
             orderby v
             select v;
foreach(int i in result)
  Console.WriteLine(i);
```

*Essa consulta LINQ tem quatro **cláusulas**: a from, a where, a orderby e a select.*

Isso faz com que a letra v represente cada valor na matriz consultada. Então v será 0, depois 12, então 44, 36... e assim por diante.

Isso quer dizer selecione cada v na matriz que seja menor do que 37.

Então, coloque esses valores em ordem (do menor para o maior).

Se você já usou SQL, pode parecer estranho colocar o select no final, mas é assim que as coisas funcionam na LINQ.

Agora você pode iterar pela matriz de resultados e imprimir cada item de a LINQ retornou.

Resultado:

```
0, 8, 12, 13, 36
```

var

var é uma palavra-chave que diz ao compilador para determinar o tipo de uma variável enquanto o programa é compilado. O .NET deduz o tipo observando a variável local usada para realizar a consulta LINQ. Quando você compilar seu programa, a palavra-chave será substituída pelo tipo correto dos dados com os quais você está trabalhando.

No exemplo acima, quando esta linha é compilada:

```
var result = from v in values
```

O compilador substitui "var" pelo seguinte:

```
IEnumerable<int>
```

Veja bem!

A LINQ é uma característica nova que é parte do Visual C# 3.0 e do Visual Studio 2008. *Se você está usando uma versão anterior do C#, reserve alguns minutos para baixar e instalar o Visual Studio 2008 Express Edition da Microsoft. É gratuito e pode ser instalado lado-a-lado com versões anteriores.*

A LINQ é simples, mas suas consultas não deve ser

Jaime acaba de vender sua companhia recém-criada para um grande investidor e quer pegar uma parte de seus lucros e comprar as edições mais caras do Capitão Maravilhoso que ele puder achar. Mas tudo o que ele tem são dados. Como pode a LINQ ajudá-lo a tirar o pó de suas duas coleções e determinar quais são as revistas mais caras?

1. Jaime baixa uma lista de edições do Capitão Maravilhoso de uma página de fãs. Ele as coloca numa `List<>` de objetos `Comic` (revista em quadrinho) que tem dois campos, `Name` e `Issue` (edição).

```
public class Comic {
  public string Name { get; set; }
  public int Issue { get; set; }
}
```

Jaime usa inicializadores de objeto e de coleção para construir seu catálogo:

```
private static List<Comic> BuildCatalog() {
  List<Comic> comics = new List<Comic>();
  comics.Add(new Comic("Johnny America vs. the Pinko", 6));
  comics.Add(new Comic("Rock and Roll (limited edition)", 19));
  comics.Add(new Comic("Woman's Work", 36));
  comics.Add(new Comic("Hippie Madness (misprinted)", 57));
  comics.Add(new Comic("Revenge of the New Wave Freak (damaged)", 68));
  comics.Add(new Comic("Black Monday", 74));
  comics.Add(new Comic("Tribal Tattoo Madness", 83));
  comics.Add(new Comic("The Death of an Object", 97));
  return comics;
}
```

A edição 74 do Capitão Maravilhoso é chamada de Segunda-feira Negra.

2. Por sorte, existe um ativo mercado para revistas do Capitão Maravilhoso na lista de Gregório. Ele sabe que a edição 57, "Loucura Hippie" teve um erro de impressão, quase todas foram destruídas pela editora e ele achou uma cópia rara recentemente vendida na lista por R$ 13.525. Depois de algumas horas de pesquisa Jaime conseguiu construir um `Dictionary<>` que associa números de edições a preços.

```
private static Dictionary<int, int> GetPrices() {
  Dictionary<int, int> values = new Dictionary<int, int>();
  values.Add(6, 3600);
  values.Add(19, 500);
  values.Add(36, 650);
  values.Add(57, 13525);
  values.Add(68, 250);
  values.Add(74, 75);
  values.Add(83, 25);
  values.Add(97, 35);
  return values;
}
```

A edição 57 vale R$ 13.525,00.

Poder do Cérebro

Olhe com cuidado para a consulta LINQ na página anterior. O que você acha que o Jaime deve colocar na consulta para encontrar as edições mais caras?

Anatomia de uma consulta

É fácil de consultar os dados de Jaime com uma consulta LINQ. Que cláusula `where` (onde) diz ao LINQ quais itens da coleção devem ser incluídos no resultado? E essa cláusula não precisa ser uma comparação simples. Ela pode incluir qualquer comando C# válido – como, por exemplo, usar o dicionário values (valores) para retornar revistas que custem mais de R$ 500. E a cláusula `orderby` (ordenar por) funciona da mesma forma - podemos dizer à LINQ para ordenar os resultados por preço.

```
List<Comic> comics = BuildCatalog();
Dictionary<int, int> values = GetPrices();

var mostExpensive =
    from comic in comics
    where values[comic.Issue] > 500
    orderby values[comic.Issue] descending
    select comic;

    foreach (Comic comic in mostExpensive)
Console.WriteLine("{0} is worth {1:c}",
    comic.Name, values[comic.Issue]);
```

A consulta LINQ puxa objetos Comim da lista de revistas, usando os dados no dicionário values para decidir qual delas selecionar.

A primeira cláusula na consulta é a from (a partir de). Ela diz à LINQ para consultar a coleção comics e o nome comic será usado na consulta para especificar como tratar cada linha de dados individual na coleção.

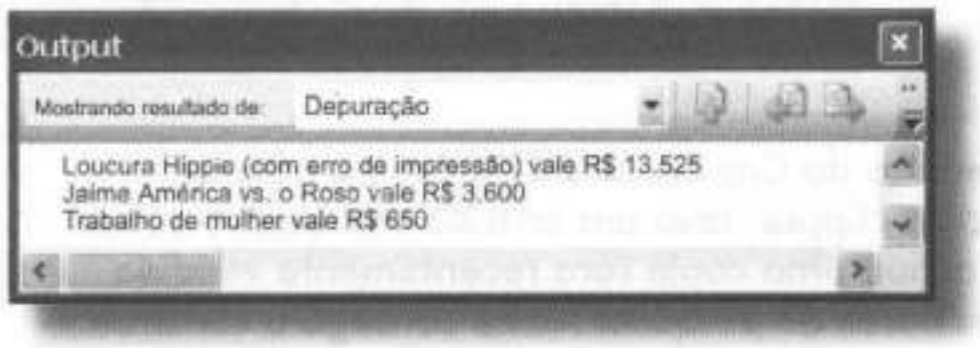
Você pode escolher qualquer nome que queira quando usar uma cláusula fomr. Escolhemos comic.

O nome comic foi definido na cláusula fomr especificamente para poder ser usado na where e na orderby.

As cláusulas where e orderby podem incluir QUALQUER comando C#. Assim, podemos usar o dicionário values para selecionar somente aquelas revistas cujo preço for maior do que R$ 500 e podemos ordenar os resultados de forma que as mais caras apareçam primeiro.

Quando v adiciona {/:c} no resultado de WriteLine, isso diz que o segundo parâmetro deve ser impresso no formato local de moeda.

A consulta retorna seus resultados numa coleção chamada mostExpensive (mais caras). A cláusula select determina o que estará contido no resultado - já que ela seleciona comic, a consulta retorna objetos Comic.



Não se preocupe se você nunca usou SQL - você não precisa conhecer nada sobre ela para trabalhar com LINQ. Mas se você estiver curioso, dê uma olhada em Use a Cabeça SQL.

Eu não engulo essa. Eu já conheço SQL - escrever uma consulta LINQ não é exatamente o mesmo que escrever uma SQL?

LINQ pode parecer com SQL, mas não funciona como ela

Se você já trabalhou muito com SQL, pode ser tentador considerar toda essa conversa sobre LINQ como intuitiva e óbvia - e você não será o único, porque muitos desenvolvedores cometem esse erro. É verdade que a LINQ usa as palavras-chave `select`, `from`, `where`, `ascending` e `join`, que foram emprestadas da SQL. Mas a LINQ é bem diferente dela e se você tentar pensar na LINQ como pensa em SQL vai acabar com código **que não faz o que você espera**.

Uma grande diferença entre as duas é que a SQL opera em **tabelas**, que são bem diferentes de **coleções**. Quando você executa uma `select` SQL numa tabela, pode ter certeza de que a tabela não será atualizada. SQL tem todo o tipo de segurança de dados embutido na qual podemos nos apoiar. E consultas SQL são operações de conjunto, ou seja, elas não examinam as linhas da tabela usando nenhuma ordem previsível. Uma coleção, por outro lado, pode armazenar qualquer coisa - valores, estruturas, objetos, qualquer coisa - e coleções têm ordens específicas (as linhas de uma tabela não estão em nenhuma ordem em particular até que você faça uma consulta SQL que as ordene. Já itens dentro de uma List, por outro lado, estão em ordem). E a LINQ permite efetuar qualquer operação suportada por qualquer coisa que esteja contida na coleção - ela pode mesmo chamar métodos nos objetos contidos. E ela também itera pela coleção, ou seja, as operações são executadas numa ordem específica. Isso pode não parecer muito importante, mas se você está acostumado a lidar com SQL, isso significa que suas consultas LINQ causarão surpresa se se esperar que elas se comportem como SQL.

Existem também muitas outras diferenças entre LINQ e SQL, mas você não precisa entendê-las para conseguir trabalhar com a LINQ com sucesso. Basta manter a mente aberta e não esperar que ela funcione da mesma forma que a SQL.

A LINQ é versátil

Você pode fazer muito mais do que simplesmente retirar uns itens de uma coleção. Você pode modificar itens retornados. E uma vez que você tenha gerado um conjunto de itens de resultado, a LINQ fornece vários métodos para trabalhar com eles. Em resumo, a LINQ dá a você as ferramentas necessárias para gerenciar dados.

Modifique cada item retornar de uma consulta
Esse código adicionará uma string ao final de cada uma das que estiver numa matriz. Ele não modifica a matriz original - ele **cria uma nova coleção** de strings modificadas.

```
string[] sandwiches = { "ham and cheese", "salami with mayo",
                        "turkey and swiss", "chicken cutlet" };
var sandwichesOnRye =
  from sandwich in sandwiches
  select sandwich + " on rye";
foreach (var sandwich in sandwichesOnRye)
  Console.WriteLine(sandwich);
```

Isso adiciona a string on rye (no pão de centeio) para cada item nos resultados da consulta.

As modificações são no resultado da consulta... mas nenhum dos itens na coleção original ou base de dados é afetado.

Note que todos os itens retornados on rye acrescentados ao final.

Resultado:
presunto e queijo no pão de centeio
salame com maionese no pão de centeio
peru e queijo suíço no pão de centeio
frango fatiado no pão de centeio

Efetue cálculos com coleções
Lembre-se, dissemos que a LINQ adiciona novos métodos nas suas coleções (e objetos de acesso a bases de dados)... e alguns deles são muito úteis por si só, mesmo sem obrigar efetivamente o uso de uma consulta.

```
List<int> listOfNumbers = new List<int>();
int length = random.Next(50, 150);
for (int i = 0; i < length; i++)
    listOfNumbers.Add(random.Next(100));
Console.WriteLine("There are {0} numbers",
                  listOfNumbers.Count());
Console.WriteLine("The smallest is {0}",
                  listOfNumbers.Min());
Console.WriteLine("The biggest is {0}",
                  listOfNumbers.Max());
Console.WriteLine("The sum is {0}",
                  listOfNumbers.Sum());
Console.WriteLine("The average is {0:F2}",
                  listOfNumbers.Average());
```

Enquanto você estiver usando o C# 3.0 (parte do Visual Studio 2008), qualquer coleção criada é capaz de usar a LINQ.

Nenhum destes métodos é parte das classes de coleção do .NET... eles foram todos definidos pela LINQ.

Estes são todos métodos de extensão definidos na classe Enumerable no namespace System.Linq.

Armazene todos ou parte dos resultados numa nova coleção
Algumas vezes você quer manter seus resultados de uma consulta LINQ. Você pode usar o comando `ToList()` para fazer isso:

Consultas LINQ não executam até que você acesse os resultados!
Isso é chamado "avaliação tardia" (literalmente "avaliação preguiçosa", do original em inglês "lazy evaluation") - a consulta LINQ não faz efetivamente nenhuma iteração até que um comando que use os resultados da consulta seja executado. É por isso que ToList() é importante aqui: ele diz à LINQ para executar a consulta imediatamente.

```
var under50sorted =
  from number in listOfNumbers
  where number < 50
  orderby number descending
  select number;
List<int> newList = under50sorted.ToList();
```

Dessa vez, estamos ordenando uma lista de números em ordem decrescente, do maior para o menor.

ToList() converte uma variável LINQ num objeto lista, para que você possa manter o resultado de sua consulta. Existem também os métodos ToArray() (para matriz) e ToDictionary(), que fazem exatamente o que o nome indica.

Você pode mesmo recuperar somente um subconjunto dos resultados, usando o método `Take()` (tomar, capturar):

```
var firstFive = under50sorted.Take(6);
List<int> shortList = firstFive.ToList();
foreach (int n in shortList)
  Console.WriteLine(n);
```

Take() recupera o número de itens solicitados, da lista de resultados de uma consulta LINQ. Você pode colocar estes numa nova variável e então convertê-la numa lista.

Dê uma olhada na página oficial "Exemplos para principiantes em LINQ" da Microsoft
Existe muito mais que a LINQ pode fazer. Por sorte, a Microsoft lhe dá uma ótima referência sucinta para ajudar o aprendizado.

http://msdn2.microsoft.com/en-us/vcsharp/aa336746.aspx

não existem Perguntas Idiotas

P: Um monte de novas palavras-chave - from, where, orderby, select... parece uma linguagem totalmente diferente. Por que ela parece tão distinta do resto do C#?

R: Porque ela serve para um propósito diferente. A maior parte da sintaxe do C# é feita para executar uma pequena operação ou cálculo de cada vez. Você pode iniciar um laço, atribuir valor a uma variável, efetuar uma operação matemática, chamar um método... essas são todas operações singulares.
As consultas LINQ parecem diferentes porque uma única linha em LINQ normalmente faz várias coisas ao mesmo tempo. Vamos dar uma olhada mais de perto numa consulta bem simples:

```
var under10 =
  from number in numberArray
  where number < 10
  select number;
```

Ela parece realmente simples - não existem várias coisas nela, certo? Mas é, na realidade, um trecho de código bastante complexo. Pense a respeito do que deve ser feito por um programa para efetivamente selecionar todos os números em `numberArray` (matriz de números) menores que 10. Primeiro, precisa iterar por toda a matriz. Depois, cada número deve ser comparado com 10. Então os resultados precisam ser coletados e reunidos para que seu código possa usá-los.
E é por isso que a LINQ parece um pouco estranha: porque o C# tem de espremer um monte de comportamentos num espaço muito pequeno.

A LINQ ajuda-o a escrever consultas que fazem coisas muito complexas usando muito pouco código.

Pontos de Bala

- **from** é como você especifica a coleção que está sendo consultada. É sempre seguida pelo nome de uma variável, depois por um **in** e o nome da coleção (`from valor in valores`).
- **where** normalmente segue a cláusula from. É nela que você usa condições C# comuns para dizer à LINQ quais itens devem ser recuperados da coleção (`where valor < 10`).
- **orderby** permite ordenar os resultados. É seguida pelo critério escolhido para ordenar e opcionalmente **descending** (descendente) informa que a ordem deve ser revertida (`orderby valor descending`).
- **select** é como você especifica conterá os resultados (`select valor`).
- **Take** permite recuperar somente os primeiros itens de um resultado de uma consulta LINQ (`resultados.Take(10)`). A LINQ providencia outros métodos para cada coleção: `Min()`, `Max()`, `Sum()` (soma) e `Average()` (média).
- Você pode `selecionar` qualquer coisa - não está limitado a selecionar o nome criado na cláusula from. Eis um exemplo: se sua consulta LINQ recupera um conjunto de preços de uma matriz de valores inteiros e nomeia-os como variável na cláusula `from` for `value`, você pode retornar uma coleção de strings de preço da seguinte forma: `select String.Format("{0:c}", value)`.

Isto é exatamente como o {0:x} usado no capítulo 9 quando desenvolveu o dumper hexadecimal. Existem também {0:d} e {0:D} para datas longas e curtas e {0:P} ou {0:Pn} para porcentagem (com n casas decimais).

não existem
Perguntas Idiotas

P: Como a cláusula `from` funciona?

R: É muito parecido com a primeira linha de um loop `foreach`. Algo torna pensar em LINQ complicado é que você não está executando apenas uma única operação. A maior parte dos comandos C# executam apenas uma única tarefa.
Uma consulta LINQ faz algo repetidamente para cada item numa coleção. A cláusula `from` faz duas coisas: ela diz à LINQ qual coleção usar na consulta e atribui um nome para cada membro da coleção consultada.
A forma como a cláusula `from` cria um novo nome para cada item na coleção é realmente parecida com a maneira como um laço `foreach` trabalha. Eis a primeira linha de um laço desse tipo:

```
foreach (int i in values)
```

Esse laço cria uma variável temporária chamada `i`, que recebe em seqüência cada item na coleção `values`. Agora observe uma cláusula `from` numa consulta LINQ na mesma coleção:

```
from i in values
```

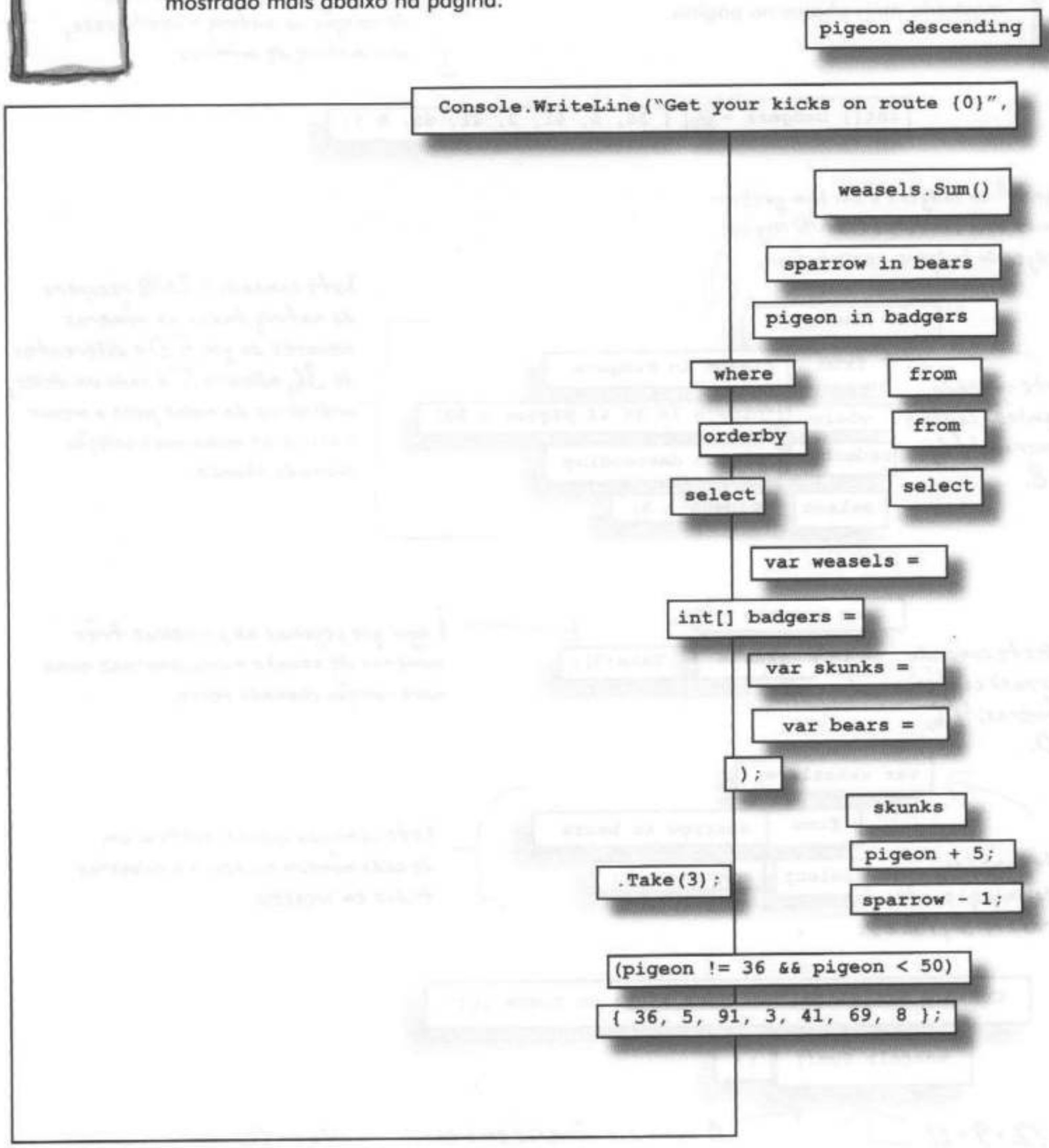
Essa cláusula faz basicamente a mesma coisa. Ela cria uma variável temporária chamada i e atribui-lhe em seqüência cada item na coleção values. O loop `foreach` executa o mesmo bloco de código para cada item da coleção, enquanto que uma consulta LINQ aplica o mesmo critério na cláusula `where` para cada um deles para determinar se eles devem ou não ser incluídos no resultado.

P: Como a LINQ decide o que aparece nos resultados?

R: É para isso que serve a cláusula `select`. Todas as consultas LINQ retornam uma coleção, e cada item na coleção tem o mesmo tipo. Isso diz à LINQ exatamente o que a coleção de resultado deve conter. Quando você está consultando uma matriz ou lista com um único tipo - como, por exemplo, uma matriz de `inteiros` ou uma `List<string>` - é óbvio o que colocar na cláusula `select`. Mas o que dizer se você estiver selecionando de uma lista de objetos `Comic`, por exemplo? Você pode fazer o que fez o Jaime e selecionar a classe inteira. Mas você também pode modificar a última linha da consulta para `selecionar comic.Name` e retornar uma coleção de strings. Ou você pode `selecionar comic.Issue` e ter no final uma coleção de `inteiros`.

Ímãs de geladeira da LINQ

Reorganize os ímãs de forma a produzir o resultado mostrado mais abaixo na página.



Resultado:

Get your kicks on route 66 (nt: "sinta-se estimulado na rodovia 66")

Solução dos Ímãs de geladeira da LINQ

Reorganize os ímãs de forma a produzir o resultado mostrado mais abaixo na página.

A LINQ começa com algum tipo de coleção ou matriz - neste caso, uma matriz de inteiros.

```
int[] badgers = { 36, 5, 91, 3, 41, 69, 8 };
```

from pigeon* in badgers é um bom quebra-cabeça, mas uma consulta LINQ ilegível. from badger in badgers seria melhor.

```
var skunks =
    from pigeon in badgers
    where (pigeon != 36 && pigeon < 50)
    orderby pigeon descending
    select pigeon + 5;
```

Depois deste comando, skunks (gambás) contém quatro números: 46, 13, 10 e 8.

Este comando LINQ recupera da matriz todos os números menores do que 50 e diferentes de 36, adiciona 5 a cada um deles, ordena-os do maior para o menor e coloca-os numa nova coleção chamada skunks.

```
var bears =
    skunks .Take(3);
```

Depois deste comando, bears (ursos) contém três números: 46, 13 e 10.

É aqui que pegamos os primeiros três números de skunks e colocamo-nos numa nova coleção chamada bears.

```
var weasels =
    from sparrow in bears
    select sparrow - 1;
```

Depois deste comando, weaseals (doninhas) contém três números> 45, 12 e 9.

Este comando apenas subtrai um de cada número em bears e coloca-os todos em weasels.

```
Console.WriteLine("Get your kicks on route {0}",
    weasels.Sum() );
```

45 + 12 + 9 = 66

A soma dos números em weasels resulta em 66.

Resultado:

Get your kicks on route 66

*NT: pidgeon: pombo. badger: texugo.

A LINQ pode combinar seus resultados em grupos

Você já sabe que pode usar LINQ para agrupar seus resultados, porque foi o que fizemos com o simulador de colméia. Vamos olhar mais de perto para a consulta e ver como isso funciona.

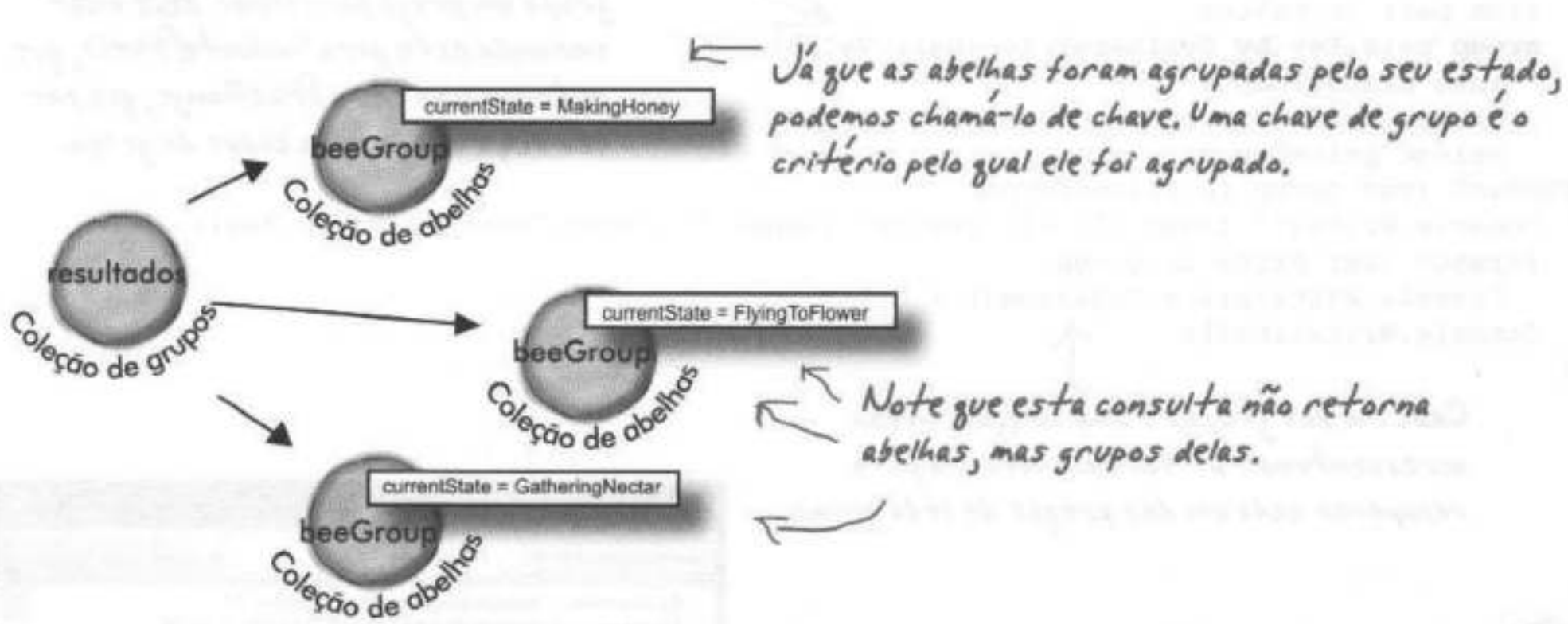
```
var beeGroups =
from bee in world.Bees
group bee by bee.CurrentState
into beeGroup
orderby beeGroup.Key
select beeGroup;
```

1. A consulta começa como qualquer outra que você já tenha visto - recuperando objetos bee individuais da coleção `world.Bees`, um objeto tipo `List<Bee>`.

2. A próxima linha na consulta apresenta uma nova palavra-chave: **group** (grupo). Isso diz à consulta para retornar **grupos** de abelhas. Ou seja, em vez de retornar uma coleção única, uma **coleção de coleções** será retornada. **group bee by bee.CurrentState** diz à LINQ para retornar um grupo para cada valor diferente da propriedade `CurrentState` que ela encontrar nas abelhas que selecionar. Finalmente, precisamos dar à LINQ um nome para os grupos. É para isso que serve a próxima linha: **into beeGroup** indica que o nome "beeGroup" se refere aos novos grupos.

3. Agora que temos grupos, podemos manipulá-los. Já que estamos retornando uma coleção de grupos, podemos usar a propriedade `orderby` para colocar os grupos na ordem dos valores da enum CurrentState (Idle, FlyingToFlower, etc.). **orderby beeGroup.Key** diz à consulta para colocar os grupos na coleção em ordem, usando para isso a chave do grupo. Uma vez que agrupamos as abelhas por seu `CurrentState`, isso é usado como chave.

4. Agora precisamos apenas usar a palavra-chave select para indicar o retornará pela consulta. Já que estamos trabalhando com grupos, selecionamos seu nome: **select beeGroup**.



Combine os valores de Jaime em grupos

Jaime compra muitas revistas em quadrinho baratas, algumas de preço médio e umas poucas bem caras. Ele quer saber quais são suas opções antes de decidir quais outras comprar. Ele conseguiu os preços naquela lista do Gregório e colocou-os num `Dictionary<Int, Int>` usando seu método `GetPrices()` (recuperar preços) - vamos usar LINQ para agrupá-los em três categorias: uma de revistas baratas que custem menos de R$ 100, uma para preço médio entre R$ 100 e R$ 1000 e uma para revistas muito caras que custem mais de R$ 1000. Criaremos uma enum `PriceRange` (faixa de preços) que usaremos como chave para os grupos e um método chamado EvaluatePrice() (avalie preço) para avaliar um preço e retornar uma `PriceRange`.

1 Todos os grupos precisam de uma chave - usaremos uma enum para isso

A chave do grupo é algo que todos os membros têm em comum. Ela pode ser qualquer coisa: uma string, um número, mesmo uma referência de objeto. Pocuraremos nos preços que Jaime conseguiu na lista de Gregório. Cada grupo que a consulta retornar será uma coleção de números de edições e a chave do grupo será uma enum PriceRange. E o método EvaluatePrice() usa um preço como parâmetro e retorna uma PriceRange:

```
public enum PriceRange { cheap, midrange, expensive }
public PriceRange EvaluatePrice(int price) {
  if (price < 100) return PriceRange.cheap;
  else if (price < 1000) return PriceRange.midrange;
  else return PriceRange.expensive;
}
```

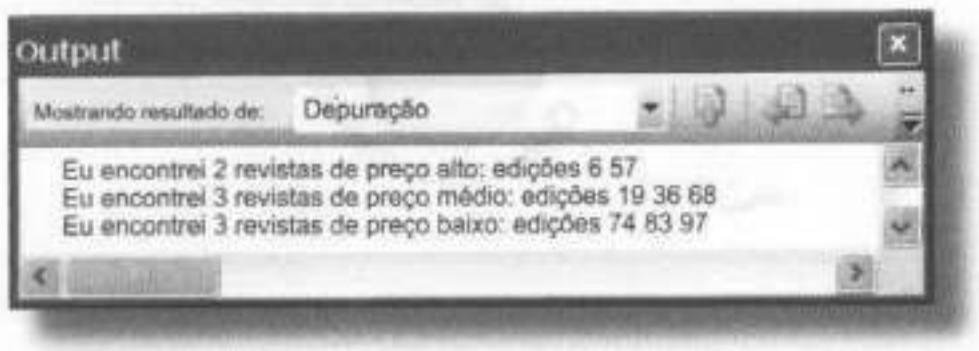
2 Agora podemos agrupar as revistas pela sua categoria de preços

A consulta LINQ retorna uma **coleção de coleções**. Cada uma das coleções dentro dos resultados tem uma propriedade Key (chave), que equivale à `PriceRange` retornada por `EvaluatePrice()`. Observe atentamente a cláusula `group by` - estamos recuperando pares do dicionário, usando o nome `pair` (par) para cada um deles: `pair.Key` é o número da edição e `pair.Value` é o preço conseguido na lista de Gregório. Adicionar `group pair.Key` diz à LINQ para criar grupos de números de edições e então juntá-los baseando-se na categoria de preços retornada por `EvaluatePrice()`:

```
Dictionary<int, int> values = GetPrices();
var priceGroups =
  from pair in values
  group pair.Key by EvaluatePrice(pair.Value)
    into priceGroup
    orderby priceGroup.Key descending
    select priceGroup;
foreach (var group in priceGroups) {
  Console.Write("I found {0} {1} comics: issues ", group.Count(), group.Key);
  foreach (var price in group)
    Console.Write(price.ToString() + " ");
  Console.WriteLine();
}
```

A consulta determina em qual grupo um preço particular deve ficar enviando este para EvaluatePrice(), que retorna uma enum PriceRange, que por sua vez é usada como chave do grupo.

Cada um dos grupos é uma coleção, então acrescentamos um foreach interno para recuperar cada um dos preços de cada grupo.



Jogos de Verão

Sua tarefa é pegar trechos de código da piscina e colocá-los nas lacunas no programa. Você pode usar o mesmo trecho mais de uma vez e não precisa usar todos eles. Seu **objetivo** é fazer o código produzir o seguinte **resultado**:

Horses enjoy eating carrots, but they love eating apples.*

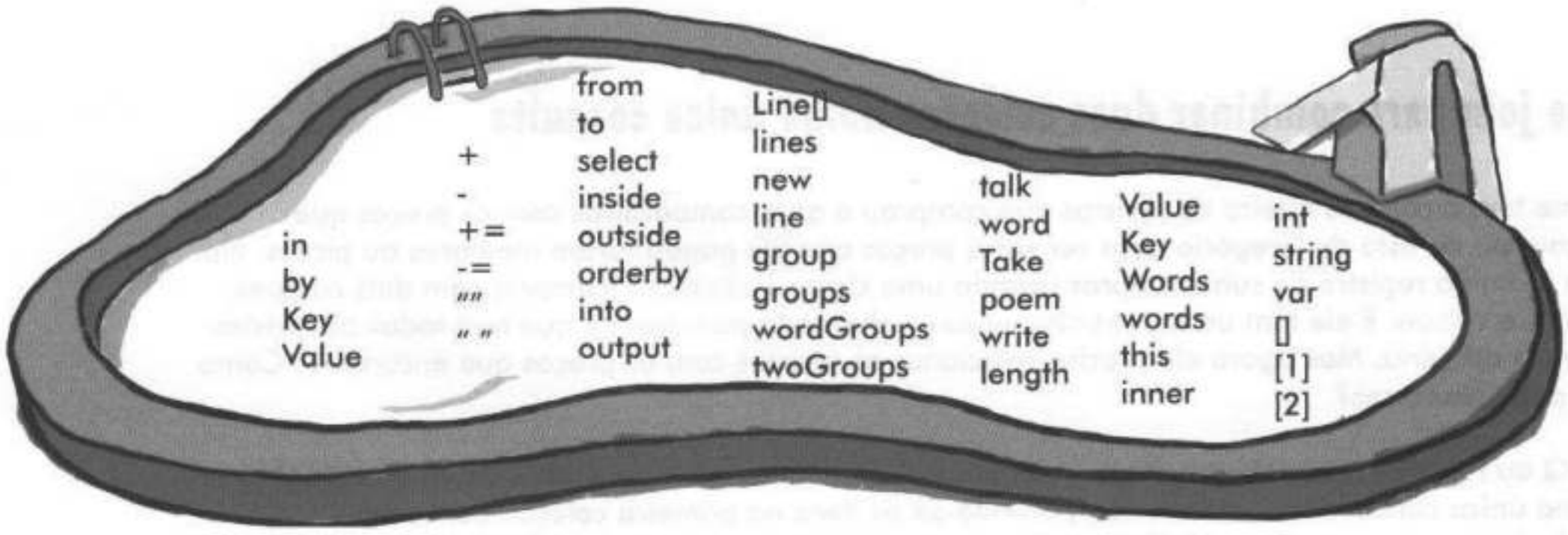
```
public class Line {
  public string[] Words;
  public int Value;
  public Line(string[] Words, int Value) {
      this.Words = Words; this.Value = Value;
  }
}
```

Dica: a LINQ ordena strings em ordem alfabética.

```
Line[] lines = {
  new Line( { "eating", "carrots,",
          "but", "enjoy", "Horses" } , 1),
  new Line( { "zebras?", "hay",
          "Cows", "bridge.", "bolted" } , 2),
  new Line( { "fork", "dogs!",
          "Engine", "and" }, 3 ) ,
  new Line( { "love", "they",
          "apples.", "eating" }, 2 ) ,
  new Line( { "whistled.", "Bump" }, 1 ) };
```

```
var _______ =
  from _______ in _______
  _______ line by line._______
  into wordGroups
  orderby _________._______
  select _________;
____ __________ = words._______(2);
foreach (var group in twoGroups)
{
  int i = 0;
  foreach (_______ inner in _______)
  {
    i++;
    if (i == _______.Key) {
      var poem =
      _______ word in _______._______
      _________ word descending
      _______ word + ____;
      foreach (var word in _______)
        Console.Write(word);
    }
  }
}
```

Nota: cada trecho na piscina pode ser usado mais de uma vez!



*NT: Cavalos gostam de comer cenouras, mas eles adoram mesmo comer maçãs.

Solução dos Jogos de Verão

```
public class Line {
  public string[] Words;
  public int Value;
  public Line(string[] Words, int Value) {
    this.Words = Words; this.Value = Value;
  }
}
Line[] lines = {
  new Line( new { "eating", "carrots,", "but", "enjoy", "Horses" } , 1),
  new Line( new { "zebras?", "hay", "Cows", "bridge.", "bolted" } , 2),
  new Line( new { "fork", "dogs!", "Engine", "and" }, 3 ) ,
  new Line( new { "love", "they", "apples.", "eating" }, 2 ) ,
  new Line( new { "whistled.", "Bump" }, 1 )
};

var words =
  from line in lines
 group line by line.Value
  into wordGroups
  orderby wordGroups.Key
  select wordGroups;

var twoGroups = words.Take(2);

foreach (var group in twoGroups)
{
  int i = 0;
  foreach (var inner in group) {
    i++;
    if (i == group.Key) {
      var poem =
        from word in inner.Words
        orderby word descending
        select word + " ";
      foreach (var word in poem)
        Console.Write(word);
    }
  }
}
```

Esta primeira consulta LINQ divide os objetos Line (linha) na matriz lines[] em dois grupos, organizados por seu Value (valor), em ordem crescente usando Value como chave.

Os dois primeiros grupos são as linhas com Values 1 e 2.

Este laço faz uma consulta LINQ no primeiro objeto Line do primeiro grupo e no segundo do segundo.

Você percebeu que as duas frases Horses enjoy eating carrots, but e they love to eating apples estão em ordem alfabética?

Resultado: **Horses enjoy eating carrots, but they love eating apples.**

Use join para combinar duas coleções numa única consulta

Jaime tem a coleção inteira de revistas que comprou e quer compará-las com os preços que encontrou na lista de Gregório para ver se os preços que ele pagou foram melhores ou piores. Ele tem mantido registro de suas compras usando uma classe `Purchase` (compra) com dois campos, `Issue` e `Price`. E ele tem uma `List<Purchase>` chamada `purchases` que tem todas as revistas que ele adquiriu. Mas agora ele precisa relacionar as revistas com os preços que encontrou. Como ele pode fazer isso?

LINQ ao resgate! Sua palavra-chave `join` (juntar) permite **combinar dados de duas coleções** numa única consulta. Isso é feito comparando-se os itens na primeira coleção com seus equivalentes na segunda (a LINQ é inteligente o suficiente para fazer isso de forma eficiente - na prática, ela não compara cada par possível de itens a não ser que seja obrigada). O resultado final é um que combina cada par correspondente.

1. Comece sua consulta com a cláusula `from` normal. Mas, em vez de segui-la, com o critério usado para determinar o que aparecerá nos resultados, adicione:

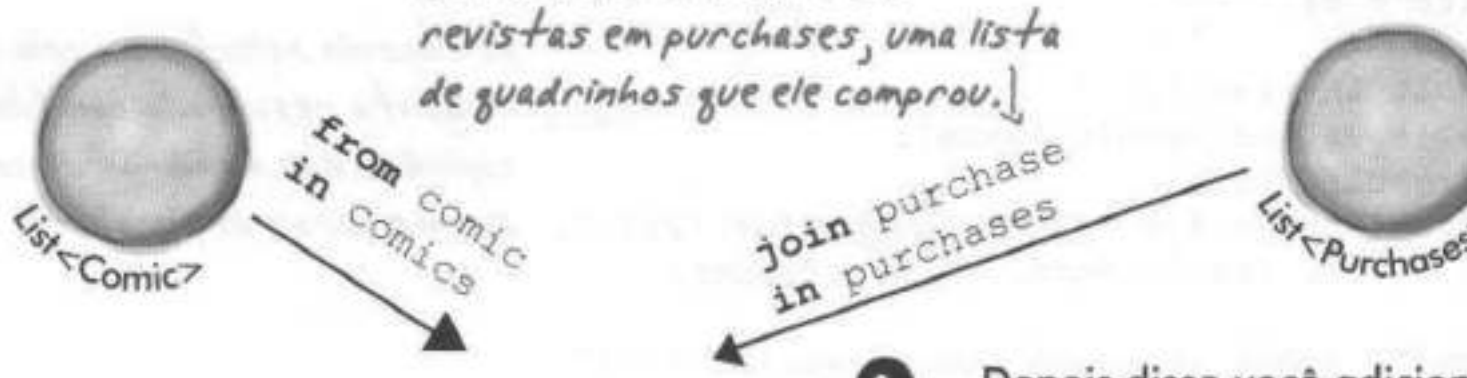
```
join name in collection
```

A cláusula `join` diz à LINQ para iterar pelas duas coleções para encontrar pares correspondentes de um membro em cada coleção. Ela atribui name em cada iteração ao membro que recuperará da coleção associada. Você usará esse nome na cláusula where.

Jaime mantém seus dados numa coleção de objeto Purchase chamada purchases.

```
public class Purchase {
  public int Issue
             { get; set; };
  public int Price;
             { get; set; };
}
```

Jaime está incluindo suas revistas em purchases, uma lista de quadrinhos que ele comprou.



*Você já viu **tipos anônimos**. Quando você usa um inicializador de objeto, cria um desses anônimos ao digitar uma série de nomes e valores dentro das chaves.*

2. Depois disso você adiciona a cláusula **on** (em), que diz à LINQ como encontrar as correspondências entre as duas coleções. Segue-se o nome do membro na primeira coleção que se está usando para procurar correlações, seguido de um **igual** e o nome do membro na segunda coleção que deve corresponder ao da primeira.

```
on comic.Issue
equals purchase.Issue
```

3. Você continuará a consulta LINQ com o `where` e `orderby` como sempre. Você pode encerrá-la com uma cláusula `select` normal, mas, na maior parte das vezes, desejará retornar resultados com dados das duas coleções. É por isso que você usa um **select new** para criar um conjunto personalizado de resultados usando um **tipo anônimo**.

O select new é seguido de chaves com os dados que devem ser retornados.

```
select new { comic.Name,
comic.Issue, purchase.Price }
```

Issue = 6	name = "Johnny America"	Price = 3600
Issue = 19	name = "Rock and Roll"	Price = 375
Issue = 57	name = "Hippie Madness"	Price = 13215

Jaime poupa um monte de grana

Parece que Jaime conseguiu barganhar bem. Ele criou uma lista de objetos Purchase com suas compras e comparou-as com os preços achados na lista de Gregório.

1. **Primeiro Jaime criou sua coleção para usar join**
Jaime já tem sua coleção original - ele só precisou usar esse método `BuildCatalog()` (criar catálogo) existente. Então tudo a fazer foi escrever um outro método `FindPurchases()` (encontrar compras) para criar sua lista de objetos `Purchase`.

```
public sList<Purchase> FindPurchases() {
  List<Purchase> purchases = new List<Purchase>() {
    new Purchase() { Issue = 68, Price = 225 },
    new Purchase() { Issue = 19, Price = 375 },
    new Purchase() { Issue = 6, Price = 3600 },
    new Purchase() { Issue = 57, Price = 13215 },
    new Purchase() { Issue = 36, Price = 660 },
  };
  return purchases;
}
```

Jaime pagou R$ 13.215 pela edição 57.

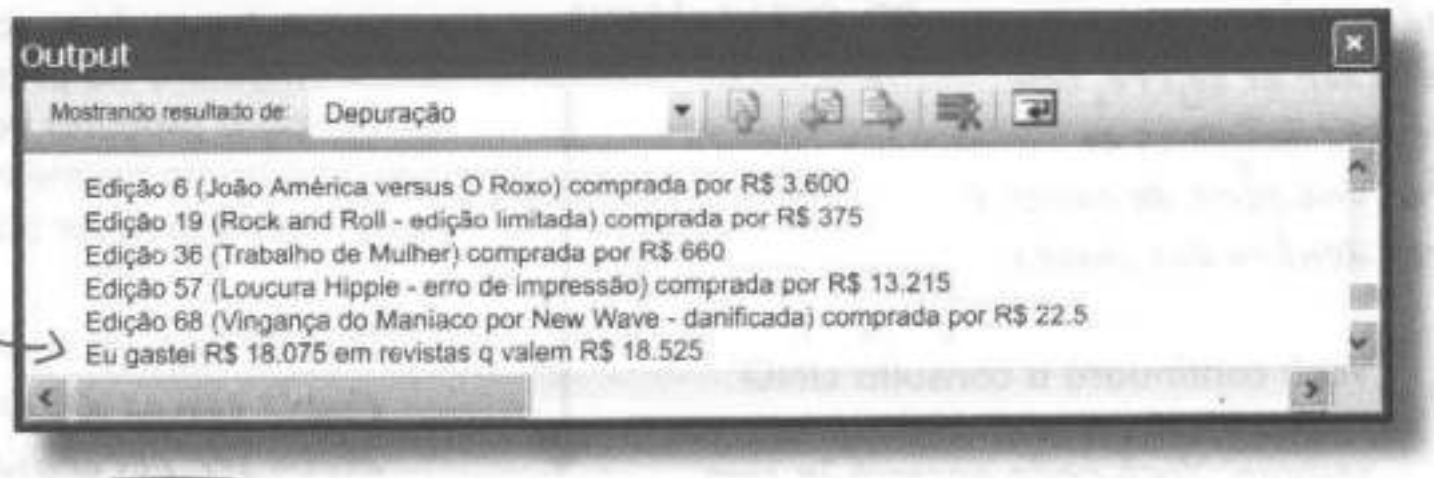
2 **Agora ele pode fazer a junção!**
Você já viu todas as partes desta consulta... agora aqui está ela, completa.

```
List<Comic> comics = BuildCatalog();
Dictionary<int, int> values = GetPrices();
List<Purchase> purchases = FindPurchases();
var results =
  from comic in comics
  join purchase in purchases
  on comic.Issue equals purchase.Issue
  orderby comic.Issue ascending
  select new { comic.Name, comic.Issue, purchase.Price };
int gregsListValue = 0;
int totalSpent = 0;
foreach (var result in results) {
  gregsListValue += values[result.Issue];
  totalSpent += result.Price;
  Console.WriteLine("Issue #{0} ({1}) bought for {2:c}",
        result.Issue, result.Name, result.Price);
}
Console.WriteLine("I spent {0:c} on comics worth {1:c}",
        totalSpent, gregsListValue);
```

Quando Jaime usou a cláusula join, a LINQ comparou cada item na coleção comics com cada um na purchases para ver qual deles tinha comic.Issue igual a purchase.Issue.

A cláusula select new cria um conjunto resultado com Name e Issue copiados dos membros de comic e Price dos membros de purchase.

Jaime está muito feliz de saber LINQ, porque ela permitiu-lhe perceber como barganhou bem!



Ok, já sei que o Jaime brincou com sua lista de revistas em quadrinhos usando consultas LINQ para checar sua coleção... mas e o problema dobre a promoção do Starbuzz? Ainda não sei como a LINQ trabalha com bases de dados.

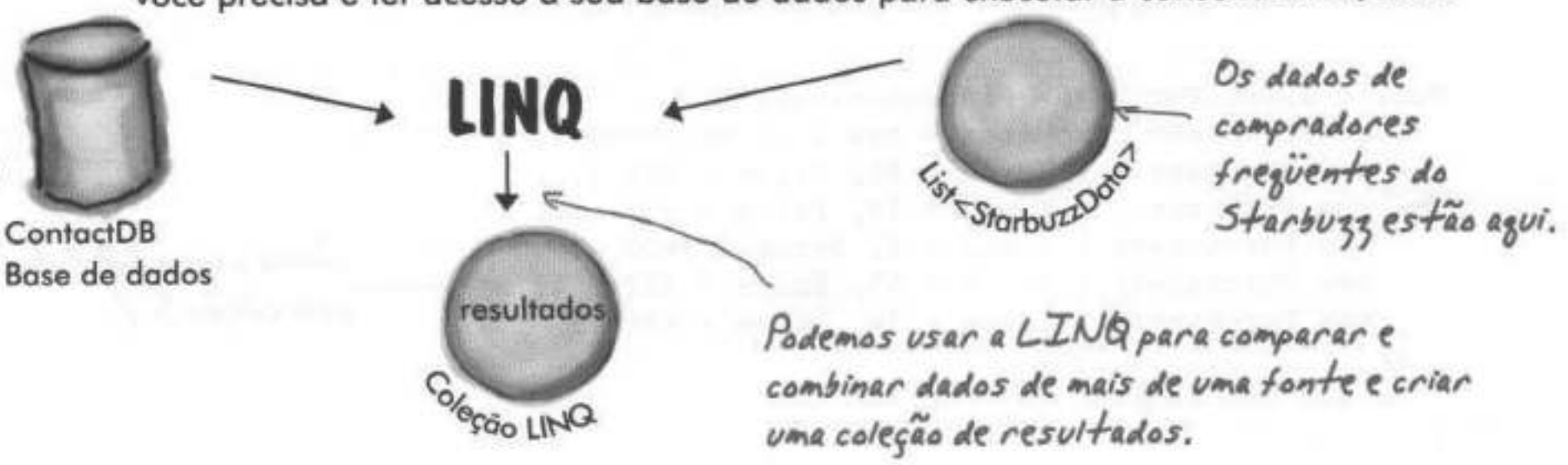
Mesmo que nos bastidores a LINQ e a SQL sejam muito diferentes, quando você escrever seu código ele será muito parecido com outras consultas LINQ.

A LINQ usa a mesma sintaxe com bases de dados e com coleções

Você já viu no capítulo 1 como o .NET facilita trabalhar com uma base de dados. O IDE lhe dá uma forma realmente conveniente de se conectar com bases de dados, adicionar tabelas e mesmo conectar dados dessas tabelas aos seus formulários.

Agora você pode usar aquela mesma base de dados com a qual já se conectou e consultá-la usando LINQ. Não só isso, a LINQ permite combinar seus dados vindos de bases com aqueles de objetos sem esforço extra.

De fato, você pode usar exatamente a mesma sintaxe de consulta... tudo de que você precisa é ter acesso à sua base de dados para executar a consulta LINQ nela.



Conecte a LINQ a uma base de dados SQL

A LINQ opera em coleções que implementem `IEnumerable`, certo? Então faz sentido acessar a sua base de dados SQL usando um objeto que também implemente essa interface. E o C# facilita adicionar um objeto assim ao seu projeto.

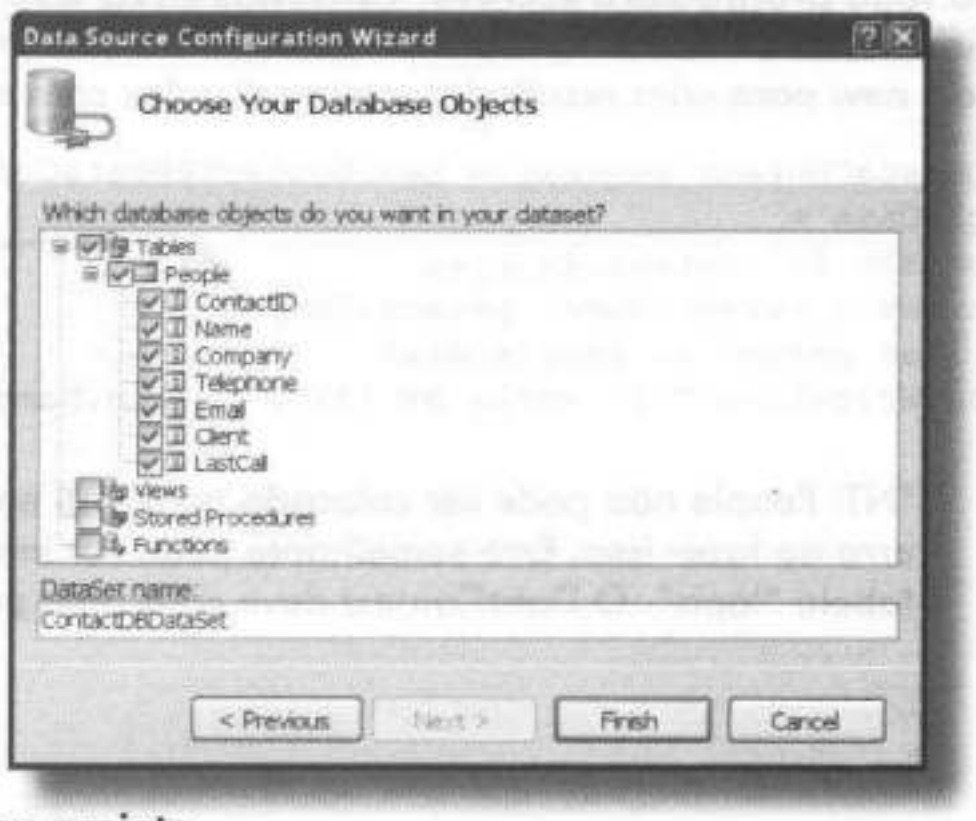
* * Faça isso *

1 **Adicione a base de dados de contatos da Vila Objeto a um novo projeto**
No capítulo 1, você criou uma base de dados de contatos para a Empresa de Papel da Vila Objeto e salvou-a num arquivo chamado `ContactDB.mdf`. Inicie um novo projeto de aplicativo Windows, clique com o botão direito no seu projeto no Navegador de Solução, selecione "Add Existing Item" e adicione essa base de dados. Tenha certeza de selecionar "Data Files" (arquivos de dados) na lista de filtragem "Objects of Type" (objetos do tipo).

2 **O IDE mostrará o Assistente de Configuração de Fontes de Dados**
Escolha a tabela People selecionando sua caixa de checagem. Clique em Finish (finalizar) - o assistente criará um conjunto de dados chamado ContactDBDataSet e o acrescentará ao seu projeto automaticamente.

Este é o mesmo assistente usado no capítulo 1 quando criou pela primeira vez a base de dados. Ele adiciona classes ao seu projeto que permitem acessar a base diretamente.

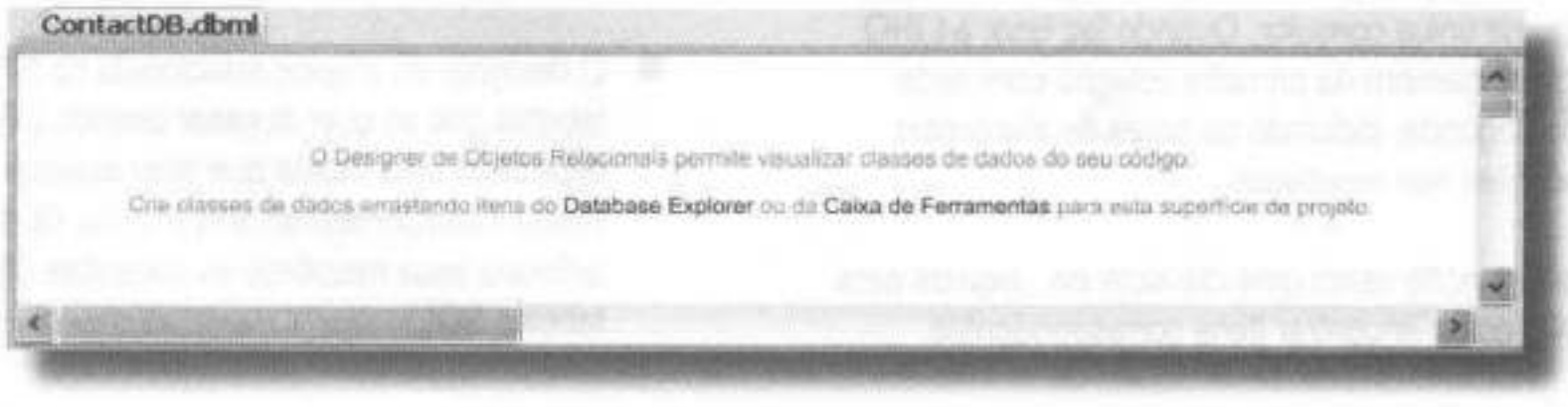
Reserve um minuto e volte até o capítulo 1 para ver como você fez isso.



3 **Adicione as Classes LINQ para SQL no seu projeto**
Clique com o botão direito no projeto no Navegador de Solução e escolha "Add New Item". A lista de ícones familiar será mostrada - escolha **LINQ to SQL** Classes (classes LINQ para SQL) e dê-lhe o nome `ContactDB.dbml`.

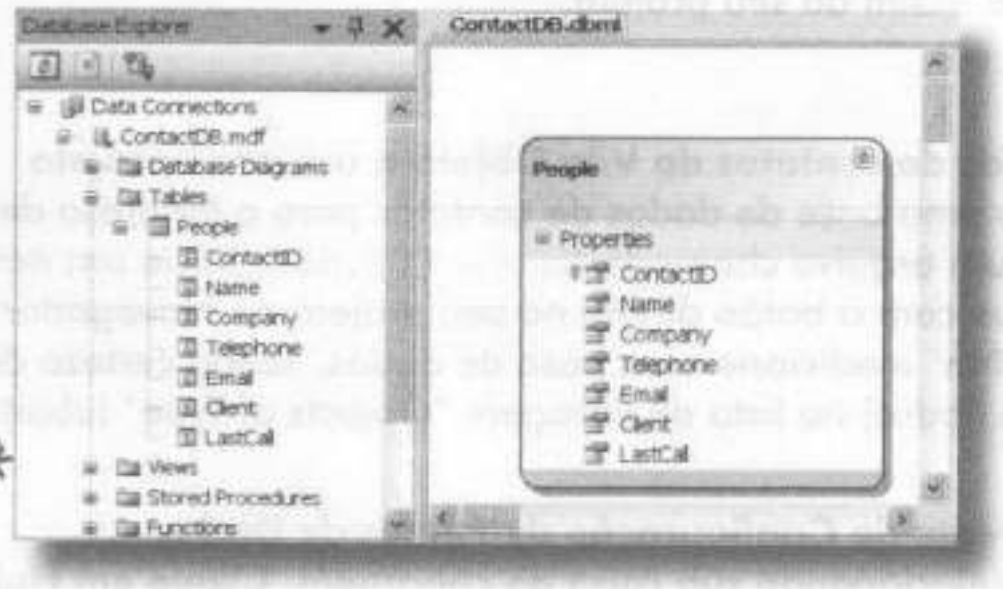
Quando você adiciona as classes LINQ para SQL no seu projeto, o IDE automaticamente acrescenta uma classe de coleção no projeto que você pode usar com a LINQ.

4 **O IDE tem um designer para criar sua coleção SQL**
Assim que você adicionar as classes LINQ para SQL ao seu projeto, o IDE mostra uma janela vazia chamada Object Relational Designer (designer de objetos relacionais). Eis com o que ela se parecerá:



5 **Arraste a tabela People para o designer de objetos relacionais**
Clique no link Database Explorer na janela do designer de objetos relacionais - o IDE vai abrir a janela Database Explorer (navegador de base de dados). Abra o nó Tables, clique no ícone da tabela People, arraste-o até o designer de objetos relacionais e salve o projeto.

O DataContext é um pouco esperto demais para seu próprio bem. Como ele sabe da existência de uma tabela People, ele assume que ela contém várias linhas, uma para cada "People" (pessoas)... Assim, ele tem um membro chamado Peoples* para conter cada People individual.



Assim que você salvar o projeto, o IDE automaticamente vai adicionar uma classe DataContext (contexto de dados) ao seu projeto - uma de suas propriedades é a coleção que você pode consultar usando LINQ, que é conectada à base de dados.

6 **Está tudo pronto para escrever consultas LINQ que recuperam dados da base**
Adicione um botão ao formulário - aqui está o código dele. Note como usamos a palavra-chave select new para criar resultados personalizados com apenas Name e Company (empresa).

```
ContactDBDataContext context = new ContactDBDataContext();
var peopleData =
  from person in context.Peoples
  select new { person.Name, person.Company };
foreach (var person in peopleData)
  Console.WriteLine("{0} works at {1}", person.Name, person.Company);
```

Pratique um pouco o uso de **select new**. Este recuperará apenas os valores das colunas Name e Company da base de dados.

*NT: People não pode ser colocado no plural em inglês, então o DataContext comete um erro ao fazer isso. Erro semelhante pode ser imaginado em português se criarmos uma tabela "lápis". O DataContext deve criar uma propriedade "lapises", nesse caso.

Pontos de Bala

- A cláusula **group** diz à LINQ para agrupar os resultados - quando você a usa, a LINQ cria uma coleção de coleções de cada grupo.
- Cada grupo contém membros com um item em comum, chamado de **chave** do grupo. Use a palavra-chave **by** para especificar a chave de cada grupo. Cada coleção respectiva a cada grupo tem um membro **Key** com a chave daquele grupo.
- Use uma cláusula **join** para fazer a LINQ combinar duas coleções numa única consulta. Quando faz isso, a LINQ compara cada membro da primeira coleção com cada membro da segunda, incluindo os pares de elementos correspondentes nos resultados.
- Consultas de junção usam uma cláusula **on...equals** para dizer à LINQ como encontrar itens correspondentes.
- Quando você executa uma consulta de junção, geralmente quer um conjunto de resultados com membros de ambas as coleções. A cláusula **select new** permite criar resultados personalizados dessa forma.
- A LINQ pode consultar uma base de dados SQL usando as **classes LINQ para SQL**. Já que a LINQ funciona apenas com coleções que implementam `IEnumerable`, essas classes providenciam uma coleção que permite acessar as tabelas e consultá-las como se fossem coleções.
- O designer de objetos relacionais do IDE permite escolher tabelas que se quer acessar usando LINQ. Quando você especifica uma tabela que quer acessar, ele adiciona uma classe DataContext ao seu projeto. Quando ela é instanciada, adiciona seus membros às consultas LINQ para acessar tabelas SQL.

não existem Perguntas Idiotas

P: Você pode voltar atrás um pouco e explicar de novo o que é uma var?

R: Sim, claro. A palavra-chave `var` soluciona um problema chato que ocorre com o uso da LINQ. Normalmente, quando você chama um método ou executa um comando, fica totalmente claro que tipos você está usando. Se você tem um método que retorna uma `string`, por exemplo, então só poderá armazenar seu resultado numa variável ou campo desse tipo. Mas a LINQ não é tão simples. Quando você cria um comando LINQ, ele normalmente retorna um tipo que não está definido em lugar nenhum no programa. Sim, você sabe que será alguma espécie de coleção. Mas de que tipo? Não se sabe - porque os objetos que estarão contidos nela dependem inteiramente do que você colocar na consulta LINQ.

Veja esta consulta, por exemplo:

```
var mostExpensive =
  from comic in comics
  where values[comic.Issue] > 500
  select comic;
```

O que aconteceria se modificasse a última linha para:

```
select comic.Issue;
```

Essas são consultas LINQ perfeitamente válidas. E isso representa um problema para o C# - as duas resultam em tipos diferentes e teríamos de adicionar novos comandos para defini-los. Em vez disso, o C# fornece-nos a palavra-chave `var`, que diz ao compilador "Certo, sabemos que isso resultará num tipo válido, mas não podemos dizer exatamente qual será, nesse ponto. E já que você pode, por favor, descubra isso por si próprio. Assim não precisamos nos preocupar com isso. Agradecemos muito."

P: Eu não entendi muito bem como o join funciona.

R: Funciona com qualquer par de coleções. Digamos que você tenha uma coleção de jogadores de futebol chamada `players` (jogadores) - seus itens são objetos com propriedades `Name`, `Position` e `Number`. Então podemos recuperar jogadores com número maior do que 10 na camisa com a seguinte consulta:

```
var results =
  from player in players
  where player.Number > 10
  select player;
```

Vamos supor que devamos descobrir o tamanho da camisa de cada jogador e que exista uma coleção `jerseys` (malhas) com itens com uma propriedade Number e uma `Size`. Um join funcionaria muito bem para isso:

```
var results =
  from player in players
  where player.Number > 10
  join shirt in jerseys
  on player.Number
  equals shirt.Number
  select shirt;
```

P: Mas espere aí, essa consulta apenas resultará num monte de camisas. E se eu quiser associar cada jogador com o tamanho da camisa e não quiser saber qual número está escrito nelas?

R: É para isso que serve o `select new`. Ele permite construir um **tipo anônimo** com apenas os dados que você quer. E ele também permite escolher os dados das coleções que você está fazendo junção. Então, por exemplo, você pode selecionar somente o nome do jogador, o tamanho da camisa e mais nada:

```
var results =
  from player in players
  where player.Number > 10
  join shirt in jerseys
  on player.Number
  equals shirt.Number
  select new {
    player.Name,
    shirt.Size
  };
```

O IDE é inteligente o suficiente para determinar exatamente quais resultados são criados com a consulta. Se você implementar um laço para enumerar os resultados, assim que você digitar o nome da variável o IDE mostrará uma lista do IntelliSense.

```
foreach (var r in results)
  r.
```

Note como a lista tem Name e Size. Se você adicionar mais itens à cláusula select new, eles também aparecerão nessa lista. Isso porque a consulta estaria criando um tipo anônimo diferente com outros membros.

P: Mas isso só funciona com junções?

R: Não, você pode usar `select new` com qualquer consulta LINQ em que queira criar uma coleção de resultados que inclua apenas alguns itens.

P: Eu sempre deverei adicionar essas classes LINQ para SQL se eu quiser usar LINQ para acessar bases SQL? O que elas são?

R: Sim, você sempre terá. Elas são objetos LINQ que implementam a interface `IEnumerable`. Uma base de dados SQL não implementa essa interface, normalmente... ou qualquer outra, na verdade, porque elas não são objetos. Então se você quiser que a LINQ funcione com uma fonte de dados SQL - ou com qualquer outra fonte que você possa consultar - precisará de um objeto que interaja com a fonte e que implemente `IEnumerable`.

É por isso que o IDE providencia as classes LINQ para SQL. Quando você as adiciona a seu projeto, elas automaticamente fazem tudo o que é preciso para conectar a LINQ à base SQL: permitem arrastar objetos de base de dados para o Designer de Objetos Relacionais, automaticamente lendo suas tabelas e criando classes (como a classe People) que a LINQ pode usar para acessar os dados.

Você pode usar "select new" para criar resultados personalizados de consultas LINQ que incluam apenas os itens desejados na coleção retornada.

Use uma consulta de junção para conectar o Starbuzz e a Vila Objeto

Agora você tem todas as ferramentas de que precisa para combinar os dados do Starbuzz e da Empresa de Papel da Vila Objeto num conjunto de resultados finais.

Faça isso

1 **Adicione os dados SQL ao seu projeto**
Se você ainda não fez isso, crie um novo projeto de aplicativo Windows e adicione a base SQL ContactDB nele. Então adicione as classes LINQ para SQL e escreva uma consulta de teste simples apenas para ter certeza de que está tudo funcionando.

2 **Crie os objetos Starbuzz**
Eis aqui a lista com os dados de clientes do Starbuzz. Adicione-a ao seu projeto:

```
public class StarbuzzData { {
  public string Name { get; set; }
  public Drink FavoriteDrink { get; set; }
  public int MoneySpent { get; set; }
  public int Visits { get; set; }
}
public enum Drink {
  BoringCoffee, ChocoRockoLatte, TripleEspresso,
  ZestyLemonChai, DoubleCappuccino, HalfCafAmericano,
  ChocoMacchiato, BananaSplitInACup,
}
```

Os dados do Starbuzz estão numa coleção de objetos StarbuzzData. Eles contém diversos itens - você não precisará de todos eles para a promoção, então deverá selecionar apenas os necessários na consulta LINQ.

O Starbuzz tem várias bebidas ótimas e cada cliente tem seu preferido.

```
public List<StarbuzzData> GetStarbuzzData() {
  List<StarbuzzData> list = new List<StarbuzzData>() {
    new StarbuzzData() {
      Name = "Janet Venutian", FavoriteDrink = Drink.ChocoMacchiato,
      MoneySpent = 255, Visits = 50 },
    new StarbuzzData() {
      Name = "Liz Nelson", FavoriteDrink = Drink.DoubleCappuccino,
      MoneySpent = 150, Visits = 35 },
    new StarbuzzData() {
      Name = "Matt Franks", FavoriteDrink = Drink.ZestyLemonChai,
      MoneySpent = 75, Visits = 15 },
    new StarbuzzData() {
      Name = "Joe Ng", FavoriteDrink = Drink.BananaSplitInACup,
      MoneySpent = 60, Visits = 10 },
    new StarbuzzData() {
      Name = "Sarah Kalter", FavoriteDrink = Drink.BoringCoffee,
      MoneySpent = 110, Visits = 15 }
  };
  return list;
}
```

GetStarbuzzData() usa inicializadores de coleção e de objetos para configurar os objetos Starbuzz.

Criamos este método de forma de forma a existir alguns nomes que também apareçam na lista de contatos da Vila Objeto. Se você usou nomes diferentes, certifique-se de ter alguns que existam nas duas fontes.

3 **Agora faça a junção entre a base de dados SQL e a coleção Starbuzz**
Adicione um botão ao projeto que executará a consulta e exibirá os resultados no console. Eis o código da consulta:

```
List<StarbuzzData> starbuzzList = GetStarbuzzData();
ContactDBDataContext context = new ContactDBDataContext();
var results =
  from starbuzzCustomer in starbuzzList
  where starbuzzCustomer.MoneySpent > 90
  join person in dataContext.Peoples
  on starbuzzCustomer.Name equals person.Name
  select new { person.Name, person.Company,
               starbuzzCustomer.FavoriteDrink };
```

Aqui é onde a cláusula select new recupera nome e empresa da base de dados e a bebida favorita dos dados do Starbuzz e coloca tudo numa única coleção resultante.

Precisaremos fazer uma junção para combinar os dados do Starbuzz com os de clientes na tabela People.

O membro Peoples no DataContext é uma coleção que lhe dá acesso à tabela People na base de dados.

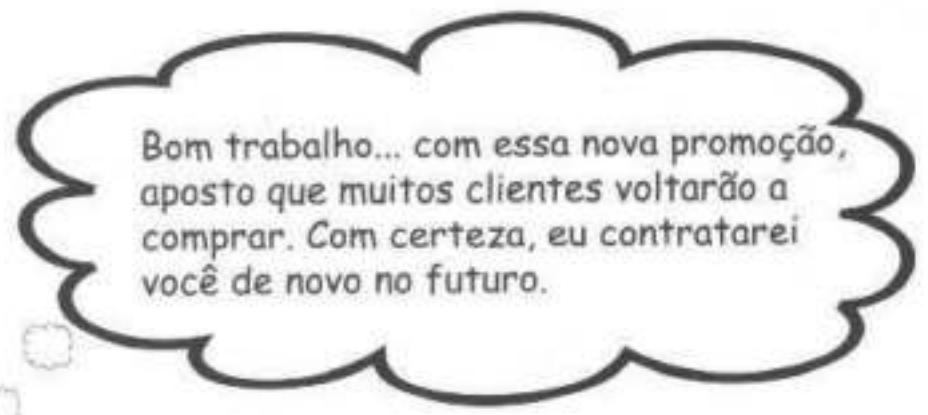
E é bem fácil escrever os resultados no console:

```
foreach (var row in results){
  Console.WriteLine("{0} at {1} likes {2}",
            row.Name, row.Company, row.FavoriteDrink);
}
```

Verifique os resultados - certifique-se de que tudo esteja funcionando como esperado.

Edite consultas com o LINQPad

Há uma ótima ferramenta de aprendizado para explorar e usar a LINQ, o LINQPad. Ele está disponível no site de Joe Albahari (um dos revisores técnicos que ajudaram a nos certificar de que Use a Cabeça C# está razoavelmente livre de erros). Você pode baixá-lo de: http://www.albahari.com/linqpad.html



Nome: Data:

Laboratório C#3

Invasores

Este laboratório contém uma especificação que descreve um programa para você criar, usando o conhecimento adquirido nos últimos capítulos.

Este projeto é maior do que os que você viu até agora. Então leia tudo antes de começar e reserve algum tempo para pensar. E não se preocupe se ficar preso em uma parte – não há nada novo aqui, então você pode continuar lendo o livro e voltar ao laboratório mais tarde.

Preenchemos alguns poucos detalhes de projeto para você e certificamo-nos que você tenha todas as peças de que precisará... e mais nada.

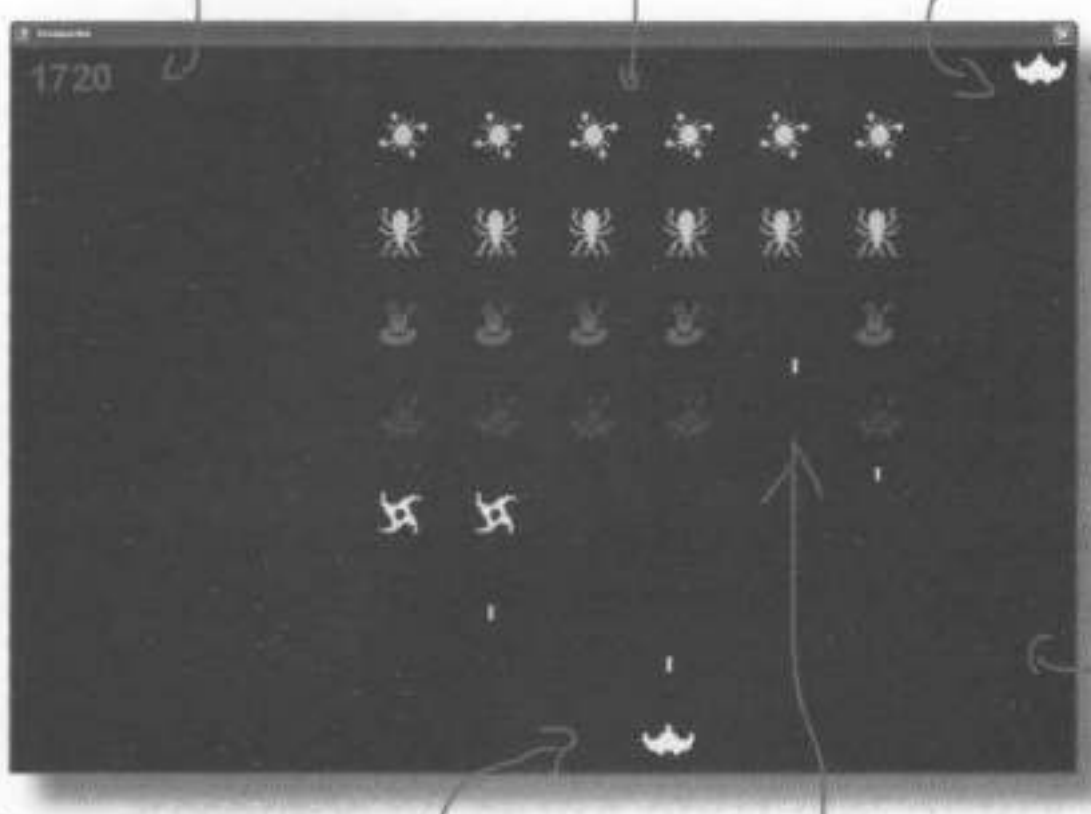
É de sua responsabilidade terminar o trabalho. Você pode baixar o executável deste laboratório na nossa página de Internet... mas lhe não daremos o código da resposta.

O avô de todos os videogames

Neste laboratório homenagear ao mais popular, reverenciado e copiado ícone da história dos videogames, um jogo que não precisa de maiores apresentações. **É hora de você desenvolver Space Invaders.**

Os invasores atacam em hordas de 30. A primeira horda move-se vagarosamente e dispara alguns tiros por vez. A próxima horda se move mais rápido e dispara mais tiros e com mais freqüência. Se todos os 30 invasores de uma horda forem destruídos, a próxima ataca.

Conforme o jogador destrói os invasores, sua pontuação sobe. Ela é mostrada no canto superior esquerdo.



O jogador inicia com três naves. A primeira entra direto em jogo e as outras duas são mantidas na reserva. As naves sobressalentes são mostradas no canto superior direito.

As estrelas coloridas no fundo piscam, mas não afetam o jogo em nada.

O jogador movimenta a nave para a esquerda e para a direita e dispara contra os invasores. Se um tiro atingir um deles, ele é destruído e a pontuação do jogador sobe.

Os invasores disparam de volta. Se um dos tiros atingir a nave, o jogador perde uma vida. Se todas as vidas acabaram, ou se os invasores alcançarem a parte inferior da tela, o jogo termina com um grande FIM DO JOGO no meio da tela.

Sua missão: defender o planeta contra hordas e hordas de invasores

Os invasores atacam em hordas e cada uma é uma formação compacta de 30 invasores individuais. Conforme o jogador os destrói, a pontuação aumenta. Os invasores mais abaixo têm a forma de estrelas e valem 10 pontos. As naves valem 20, os discos 30, os insetos 40 e os satélites 50. O jogador começa com três vidas. Se ele perder todas ou os invasores alcançarem a parte inferior da tela, o jogo acaba.

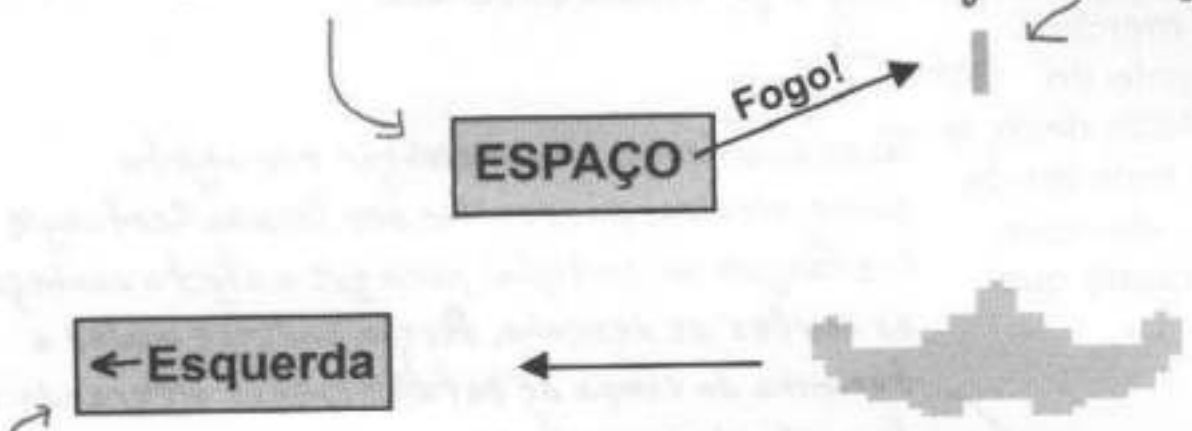
Existem cinco diferentes tipos de invasores, mas todos eles comportam-se da mesma forma. Eles começam na parte de cima da tela e movem-se para a esquerda até atingirem a borda da tela. Então, eles descem e começam a mover-se pra direita. Quando atingem a borda direita, descem de novo e voltam a andar para a esquerda. Se eles atingirem a parte inferior da tela, o jogo termina.

A primeira horda de invasores pode disparar dois tiros de cada vez - eles pararão de atirar se houver mais de dois tiros na tela. A próxima horda dispara três, a próxima quatro, e assim por diante.

A barra de espaço dispara. Mas só pode haver dois tiros da tela de cada vez. Quando um tiro atinge algo ou desaparece, outro pode ser disparado.

Se um tiro atinge um invasor, ambos desaparecem. Se não for o caso, o tiro desaparece quando atinge o topo da tela.

O jogo deve gerenciar todas as teclas pressionadas. Então pressionar a seta para a direita e a barra de espaço deve fazer a nave mover-se para a direita e disparar (se já não houver na tela dois tiros).



A seta para a esquerda move a nave na direção da lateral esquerda da tela.

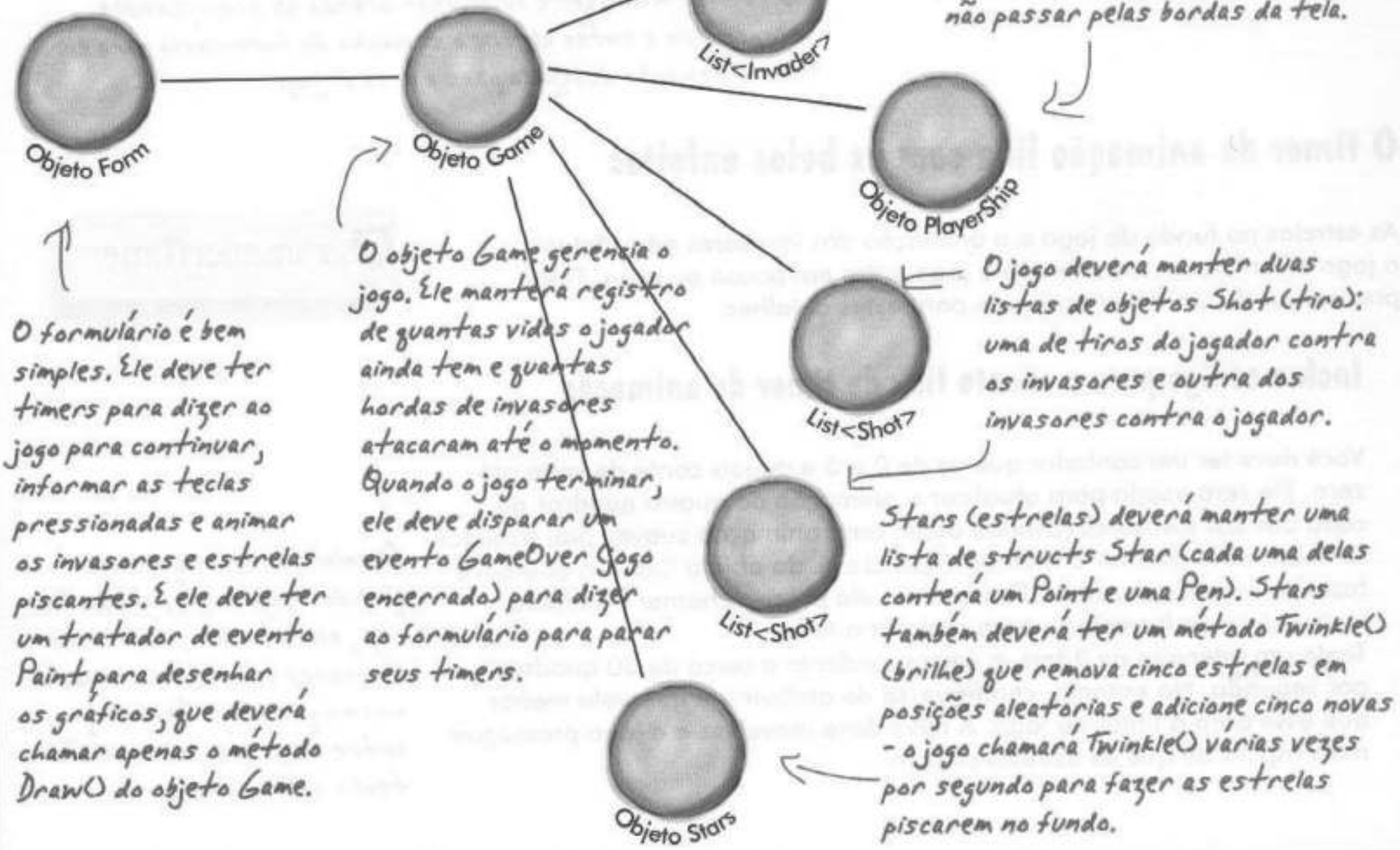
A seta para a direita move a nave para a direita.

A estrutura de Invasores

Invasores precisam manter registro de hordas de 30 invasores (incluindo suas posições, tipo e valor em pontos), da nave do jogador, dos tiros disparados por jogador e invasores e das estrelas no fundo da tela. Assim como no laboratório da Missão, você precisará de um objeto Game para controlar tudo isso e também para coordenar entre o formulário e os objetos do jogo.
Aqui está uma visão geral do que você deverá criar:

Todos os invasores na tela serão armazenados numa lista. Quando um deles for destruído, é removido dela para que o jogo deixe de desenhá-lo.

O objeto que representa a nave deve manter registro de sua posição e se mover para a direita e para a esquerda, certificando-se de não passar pelas bordas da tela.



O formulário é bem simples. Ele deve ter timers para dizer ao jogo para continuar, informar as teclas pressionadas e animar os invasores e estrelas piscantes. E ele deve ter um tratador de evento Paint para desenhar os gráficos, que deverá chamar apenas o método Draw() do objeto Game.

O objeto Game gerencia o jogo. Ele manterá registro de quantas vidas o jogador ainda tem e quantas hordas de invasores atacaram até o momento. Quando o jogo terminar, ele deve disparar um evento GameOver (jogo encerrado) para dizer ao formulário para parar seus timers.

O jogo deverá manter duas listas de objetos Shot (tiro): uma de tiros do jogador contra os invasores e outra dos invasores contra o jogador.

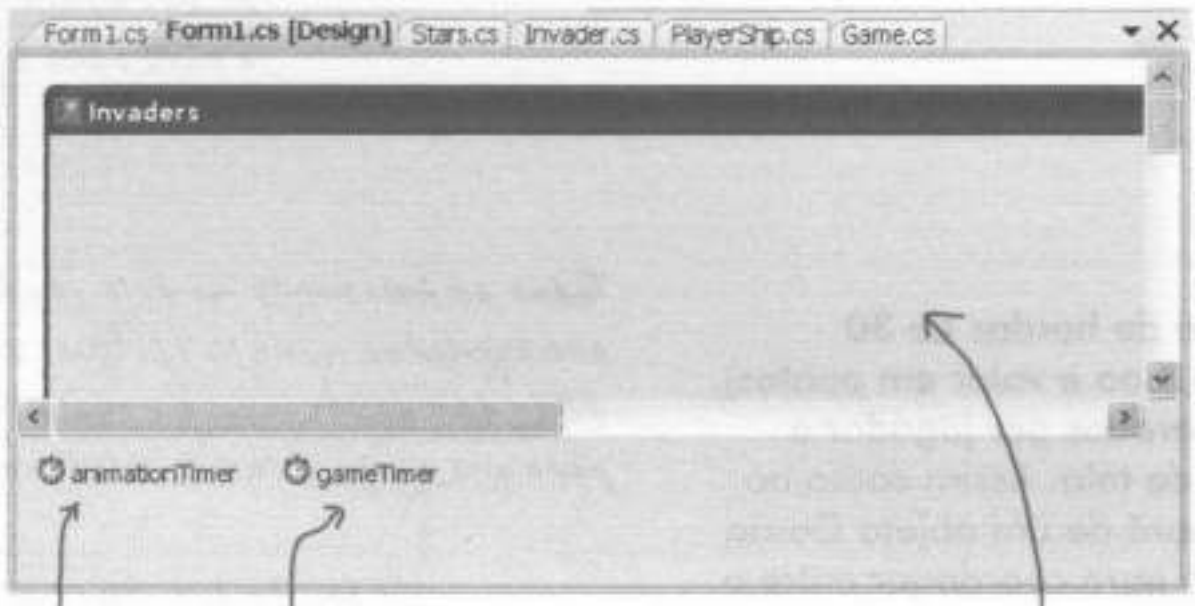
Stars (estrelas) deverá manter uma lista de structs Star (cada uma delas conterá um Point e uma Pen). Stars também deverá ter um método Twinkle() (brilhe) que remova cinco estrelas em posições aleatórias e adicione cinco novas - o jogo chamará Twinkle() várias vezes por segundo para fazer as estrelas piscarem no fundo.

Desenvolva o formulário de Invasores

O formulário de Invasores tem apenas dois controles: um timer para manter a animação (que faz as estrelas piscarem e as imagens dos invasores mudarem de quadro) e outro para lidar com o andamento do jogo (a marcha dos invasores para esquerda e direita, o movimento do jogador e os tiros do jogador e dos invasores). Além disso, o único desenvolvimento nos formulários será um tratador de evento para lidar com GameOver e outros para lidar com KeyUp (tecla liberada) e KeyDown (tecla pressionada) que gerenciarão a entrada vinda do teclado.

O formulário dispara um evento KeyDown sempre que uma tecla é pressionada e um KeyUp quando ela é liberada.

Quando o formulário inicializar seu objeto Game, ele deve passar-lhe seu DisplayRectangle (retângulo de exibição) para que o objeto conheça os limites de desenho. Assim pode-se mudar o tamanho do campo de batalha apenas alterando o tamanho do formulário.

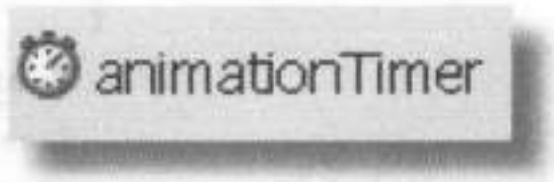


Você deve adicionar dois timers: animationTimer e gameTimer.

Atribua FixedSingle para a propriedade FormBorderStyle, true para a DoubleBuffered (também do formulário), remova os botões minimizar e maximizar usando as propriedades respectivs e então altere o tamanho do formulário para ele ter o tamanho desejado para a área de jogo.

O timer de animação lida com os belos enfeites

As estrelas no fundo do jogo e a animação dos invasores não afetam o jogo e continuam mesmo que o jogo entre em pausa ou pare. Então precisamos de um timer separado para estes detalhes.



Inclua código para o evento tick do timer de animação

Você deve ter um contador que vá de 0 a 3 e depois conte de volta até zero. Ele será usado para atualizar a animação de quatro quadros de cada um dos invasores (criando assim uma animação suave). Seu tratador também deve chamar o método `Twinkle()` do objeto `Game`, o que deve fazer as estrelas piscarem. Finalmente, ele precisa chamar o método `Refresh()` do formulário para repintar a tela.
Tente um intervalo de 33ms, o correspondente a cerca de 30 quadros por segundo. No entanto, certifique-se de atribuir um intervalo menor que esse para o timer do jogo. A nave deve mover-se e o jogo prosseguir mais rápido do que as estrelas piscam.

A animação ocorre mesmo quando o jogo está parado. Ou seja, as estrelas piscam e os invasores continuam animados mesmo que o jogo termine, entre em pausa, ou ainda não tenha sido iniciado.

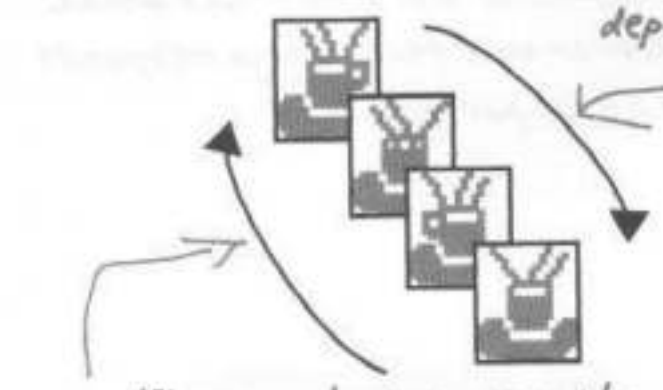
Ajuste o timer até conseguir uma animação suave

Com um intervalo de animação de 33ms, atribua 10ms para o timer do jogo. Dessa forma, a seqüência de jogo deve ocorrer mais rápido do que as animações (que são apenas enfeites de fundo bonitos). Ao mesmo tempo, o método `Go()` no objeto `Game` (disparado pelo timer do jogo, falaremos sobre isso em breve) pode consumir muitos ciclos da CPU. Se a CPU estiver ocupada lidando com o jogo, o timer de animação ficará esperando até que ela esteja mais livre e apenas depois disparará (e animará as estrelas e os invasores).

Uma alternativa é atribuir 5ms para ambos os timers e o jogo vai efetuar as animações quase tão rápido quanto seu sistema pode agüentar (embora em máquinas mais rápidas a animação possa ficar incomodamente rápida).

Se o intervalo do timer de animação for 33ms, mas o método Go() do objeto Game demorar mais do que isso para executar, a animação ficará parada até que o método encerre.

Um invasor começa na célula de animação 0, vai para a 1, depois 2 e então 3...



...e então reverte, indo de volta para a 2, depois 1 e então 0.

Testamos numa máquina mais lenta e descobrimos que deixar o tempo de animação em 100ms e o intervalo do jogo em 50ms resultou numa taxa de quadros de cerca de 10 por segundo, o que é com certeza aceitável. Você pode tentar começar assim e reduzir os intervalos até achar que está bom.

Responda ao uso do teclado

Antes que você possa criar o código do timer do jogo, precisa escrever os tratadores de evento para `KeyDown` e `KeyUp`. O primeiro é disparado quando uma tecla é pressionada e o segundo quando ela é liberada. Na maior parte dos casos, podemos simplesmente agir, por exemplo, disparando um tiro ou abandonando o jogo.

Algumas teclas, como as setas para direita e esquerda, vamos querer armazenar numa lista para que o timer do jogo possa usá-las para movimentar a nave do jogador. Então também precisaremos de uma lista de teclas pressionadas no objeto do formulário:

Então se o jogador estiver segurando a seta para esquerda e a barra de espaço ao mesmo tempo, a lista conterá Keys.Left (teclas, esquerda) e Keys.Space.

Precisaremos de uma lista de teclas para manter controle sobre quais delas foram pressionadas. Veremos em breve como o timer do nosso jogo vai precisar dessa lista para os movimentos.

```
List<Keys> keysPressed = new List<Keys>();
private void Form1_KeyDown(object sender, KeyEventArgs e) {
  if (e.KeyCode == Keys.Q)
    Application.Exit();

  if (gameOver)
    if (e.KeyCode == Keys.S) {
      // code to reset the game and restart the timers
      return;
    }
```

A tecla Q (de quit, sair, desistir) encerra o jogo.

Se o jogo terminou, reinicie tudo e comece de novo.

A enum Keys define todas as teclas que você pode querer vigiar.

Você vai precisar completar este código.

Mas queremos que isso funcione apenas se o jogo tiver terminado. Pressionar S não deveria reiniciar um jogo que está no meio.

```
    if (e.KeyCode == Keys.Space)
      game.FireShot();
    if (keysPressed.Contains(e.KeyCode))
      keysPressed.Remove(e.KeyCode);
    keysPressed.Add(e.KeyCode);
  }
```

A barra de espaço dispara.

Removendo a tecla e adicionando-a novamente, fazemos com que ela seja a última (mais atual) contida na lista.

A tecla que é pressionada é adicionada na lista de teclas, que usaremos em breve.

```
private void Form1_KeyUp(object sender, KeyEventArgs e) {
  if (keysPressed.Contains(e.KeyCode))
    keysPressed.Remove(e.KeyCode);
}
```

Queremos que a tecla mais recentemente pressionada esteja no alto da lista, então se o jogador esbarrar num monte de teclas ao mesmo tempo, o jogo deve responder à última pressionada. Assim, quando ele liberar uma tecla, o jogo responde à próxima na lista e assim por diante.

Quando uma tecla é liberada, ela deve ser removida da lista de teclas pressionadas.

O timer do jogo lida com o movimento e com a mecânica

A principal tarefa do timer do jogo será chamar o método `Go()` da classe `Game`. Mas ele também responderá a qualquer tecla pressionada. Por isto, deve verificar a lista keysPressed para lidar com qualquer tecla que tenha sido informada pelos eventos `KeyDown` e `KeyUp`:

Nosso objeto List<Keys> é keysPressed e ele é gerenciado pelos tratadores de evento KeyUp e KeyDown. A tecla no índice zero será sempre a mais recentemente pressionada.

Certifique-se de que sua nomenclatura corresponda ao nome dos métodos de tratamento.

```
private void gameTimer_Tick(object sender, EventArgs e) {
  if (keysPressed.Count() >= 1) {
    switch (keysPressed[0]) {
      case Keys.Left:
        game.MovePlayer(Direction.Left);
        break;
      case Keys.Right:
        game.MovePlayer(Direction.Right);
        break;
    }
  }
  game.Go();
}
```

Os eventos KeyUp e KeyDown usam a enum Keys para especificar uma tecla. Usaremos Keys.Left e Keys.Right para movimentar a nave.

Precisamos apenas lidar com movimento. Outras teclas, como a barra de espaço, o Q para abandonar o jogo, etc., são tratadas no método KeyDown() descrito antes.

Finalmente, podemos chamar Go() no objeto Game para permitir que o jogo continue.

```
public enum Direction {
  Left,
  Right,
  Up,
  Down,
}
```

Tiros movimentam-se para cima ou para baixo, o jogador para a direita e para a esquerda, os invasores para direita, esquerda e para baixo. Você precisará desta enum para manter controle de todas estas direções.

Mais um detalhe do formulário: o evento GameOver

Adicione um campo privado `bool` chamado `gameOver` no formulário que só será verdadeiro quando o jogo tiver terminado. Então adicione um tratador de evento no objeto `Game` chamado `GameOver` que pare o timer do jogo (mas não o de animação; então as estrelas continuarão piscando e os invasores ainda estarão animados), atribua `true` para `gameOver` e chame o método `Refresh()` do formulário.
Quando você escrever o tratador de evento `Paint` do formulário, faça com que ele verifique o campo `gameOver`. Se este for `verdadeiro`, escreva FIM DO JOGO com grandes letras amarelas no meio da tela. Em seguida escreva "pressione S para iniciar um novo jogo ou Q para sair" no canto inferior direito.

Eis um exemplo de adicionar outro evento a um formulário sem usar o IDE. Isto é codificação manual.

O evento de fim de jogo e seu delegado estão contidos na classe Game, que veremos num minuto.

O timer do jogo do formulário é o que faz o jogo andar

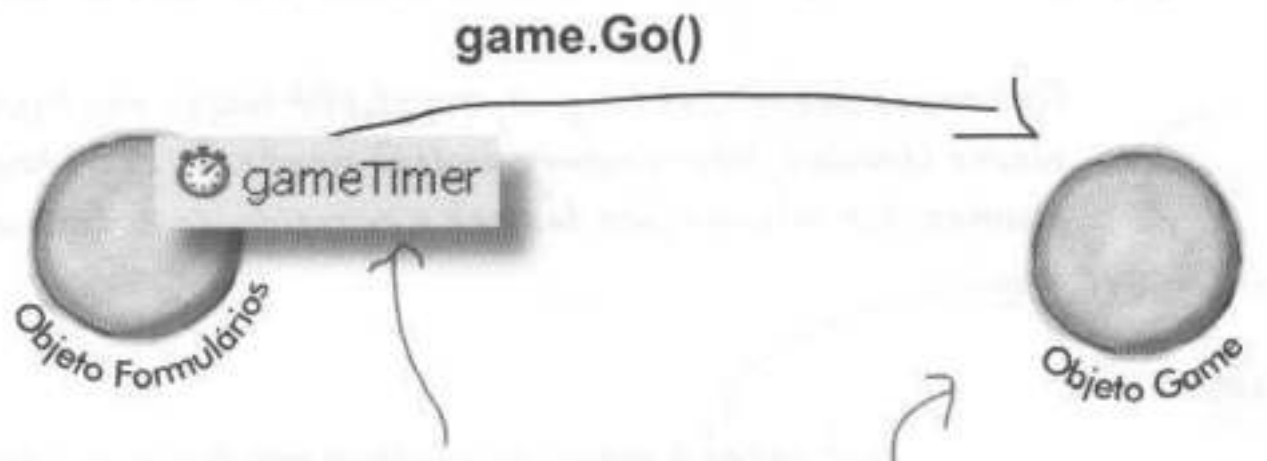
Além de lidar com movimento para a esquerda e para a direita, a tarefa principal do timer do jogo é chamar o método `Go()` do objeto `Game`. É nesse método que toda a mecânica de `jogo` é gerenciada. O objeto mantém controle sobre o estado do jogo e o `jogo` avança o um quadro. Isso implica em:

1. **Verificar se o jogador morreu,** usando a propriedade `Alive`. Quando o jogador morre, o jogo mostrará uma pequena animação da nave implodindo (usando `DrawImage()` para fazer a nave encolher até desaparecer). A animação deve ser feita pela classe `PlayerShip`, logo `Go()` deve checar se o jogador ainda está vivo. Se não está, ela deve `retornar` - dessa forma, os invasores não se movem nem atiram e o jogador consegue uma folguinha (e nesse meio tempo assistirá sua nave ser destruída).

2. **Movimentar cada um dos tiros.** Os disparados pelos `invasores` dirigem-se para baixo e os do jogador, para cima. O `jogo` manterá dois objetos `List<Shot>`, uma para os tiros dos invasores e um para os do jogador. Qualquer tiro que se mover para fora da tela deverá ser removido de sua lista.

3. **Movimentar cada um dos invasores.** `Game` chamará o método `Move()` de cada um dos `invasores` e dizer-lhes para que lado ir. Ele também manterá registro das posições para o caso de precisar movimentá-los para baixo ou mudar de direção. Depois, `Game` deve verificar se é hora dos invasores atirarem e, se for o caso, adicionar um novo objeto `Shot` na `lista`.

4. **Checar colisões.** Se o tiro do jogador atingir um invasor, `Game` deverá retirá-lo da lista apropriada. Em seguida, deverá checar se algum dos tiros dos invasores colidiu com a nave do jogador. Nesse caso, ele mata-o atribuindo `false` à sua propriedade `Alive`. Se o jogador não tiver mais vidas, `Game` deverá levantar o evento `GameOver` para informar o formulário que o jogo acabou. O tratador de evento `GameOver` do formulário interrompe o timer de jogo, então `Go()` não será mais chamada.



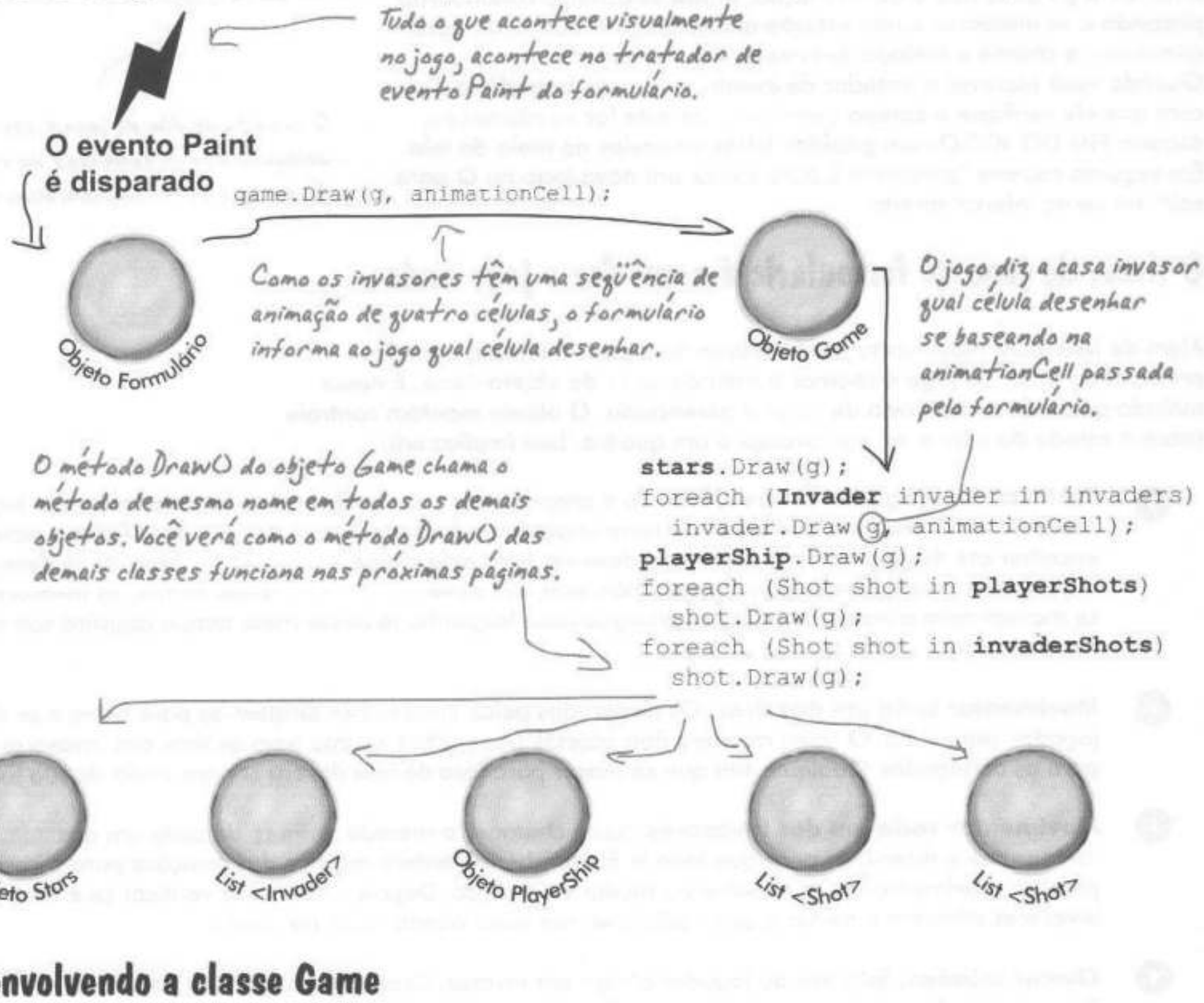
Eis onde o evento GameOver da página anterior entra em jogo.

O timer do jogo dispara mais vezes do que o de animação, fazendo com que a velocidade do jogo seja alta.

Go() no objeto Game() lida com tudo, desde movimentar os tiros, até verificar se naves ou invasores foram atingidos.

Controlando os Gráficos

Nos laboratórios anteriores, os formulários usaram controles para exibir os gráficos. Mas agora que você sabe como usar `Graphics` e buffer duplo, o objeto `Game` deve lidar com grande parte dos gráficos. O formulário deve ter um tratador de evento `Paint` (que é onde você vai colocar o código de buffer duplo). Mas você vai delegar o restante do desenho para o objeto `Game` cada vez que esse evento disparar.



Desenvolvendo a classe Game

A classe Game é quem controla o jogo Invasores. Eis aqui um começo para o que essa classe deve ser, embora ainda exista muito trabalho a ser feito.

```
public class Game {
  private int score = 0;
  private int livesLeft = 2;
  private int wave = 0;
  private int framesSkipped = 0;

  private Rectangle boundaries;
  private Random random;
```

Os campos score (pontuação), livesLeft (vidas restante) e waves (hordas, literalmente ondas) manterão registro de algumas das informações básicas a respeito do estado do jogo.

Você usará o campo de quadros pulados para deixar os invasores mais lentos no começo do jogo - a primeira horda deve **pular seis quadros** antes de se mover, a próxima cinco, depois quatro e assim por diante.

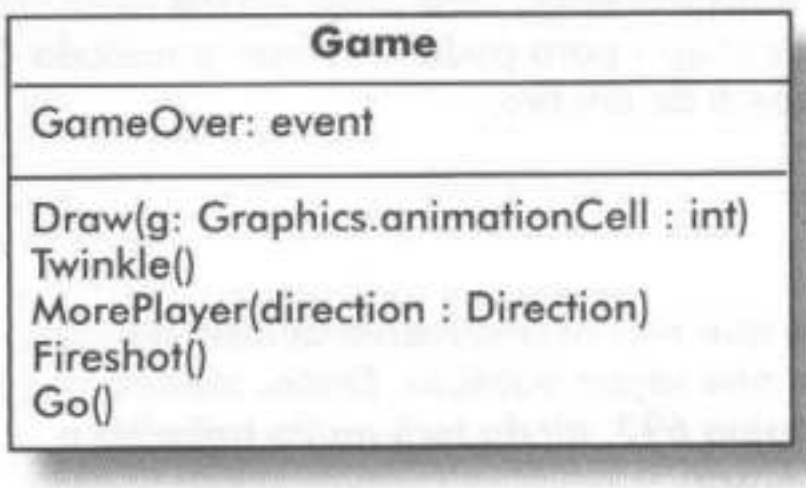
```
    private Direction invaderDirection;
    private List<Invader> invaders;
    private PlayerShip playerShip;
    private List<Shot> playerShots;
    private List<Shot> invaderShots;
    private Stars stars;

    public event EventHandler GameOver;
    // etc...
}
```

A lista de objetos Invader mantém registro de todos os invasores na horda atual. Quando um invasor é destruído, é removido da lista. O jogo verifica periodicamente para ter certeza de que a lista não está vazia - se estiver, ele envia a próxima horda.

Este objeto Starts contém as estrelas coloridas que aparecem no fundo.

O objeto Game dispara o evento GameOver quando o jogador morre e não tem mais nenhuma vida. Você incluirá o método tratador desse evento no formulário e o associará ao evento GameOver do objeto Game.

A maior parte desses métodos combinam outros de outros objetos para fazer com que uma ação específica ocorra.



Lembre-se, estes são métodos públicos. Você pode precisar de muitos outros métodos privados para estruturar seu código de forma que faça sentido para você.

Os métodos da classe Game

A classe Game deverá ter cinco métodos públicos que são disparados por diferentes eventos que acontecerão no formulário.

1. **O método Draw() desenha o jogo num objeto Graphics**
   O método `Draw()` deve receber dois parâmetros: um objeto `Graphics` e um inteiro com a célula de animação (um número entre 0 e 3). Primeiro, ele deve desenhar um retângulo preto que preencha todo o formulário (usando o retângulo de exibição armazenado em `boundaries`, recebido do formulário). Então o método deve desenhar as estrelas, os invasores, a nave do jogador e depois os tiros. Finalmente, ele deve desenhar a pontuação no canto superior esquerdo, as naves do jogador no superior direito e um grande "FIM DO JOGO" em letras amarelas se `gameOver` for `true`.

2. **O método Twinkle() faz as estrelas piscarem**
   O tratador de evento do timer de animação do formulário precisa ser capaz de fazer as estrelas piscarem, então o objeto Game deverá ter um método de uma única linha que chame `stars.Twinkle()`.

Escreveremos código para o objeto Stars daqui a algumas páginas.

3. **O método MovePlayer() movimenta o jogador**
   O tratador de evento de teclado no formulário precisa mover a nave do jogador, então o objeto Game também precisa de um método de duas linhas que receba uma enum `Direction` como parâmetro, verifique se o jogador está morto ou não e chame `playerShip.Move()` para efetuar esse movimento.

4. **O método FireShot() faz com que o jogador dispare um tiro nos invasores**
   O método `FireShot()` verifica se existem menos de dois tiros do jogador na tela. Se for o caso, ele deve adicionar um novo tiro na lista `playerShots` na posição correta.

5 **O método Go() faz com que o jogo avance**

O timer de animação do formulário chama o método `Go()` do objeto Game de 10 a 30 vezes por segundo (dependendo da velocidade da CPU do seu computador). O método `Go()` faz tudo que o jogo precisa fazer para avançar um quadro:

- ★ O jogo verifica se o jogador está morto usando sua propriedade Alive. Se ele ainda está vivo, o jogo não acabou ainda - se está, o formulário deverá parar o timer de animação com seu método `Stop()`. Nessas condições o método `Go()` não fará mais nada até que o jogador esteja vivo de novo - ele deve apenas `retornar`.
- ★ Cada tiro precisa ser atualizado. O jogo precisa iterar pelos dois objetos `List<Shot>`, chamado o método `Move()` de cada tiro encontrado. Se algum desses métodos retornar false, ou seja, esse tiro saiu da tela - então ele deve ser apagado da lista.
- ★ O jogo então avança cada invasor e permite que eles disparem.
- ★ Finalmente, ele checa para ver se colisões ocorreram: primeiro se algum tiro passou por cima de algum invasor (e nesse caso ambos são removidos de suas `listas`) e se o jogador foi atingido. Adicionaremos uma propriedade `Rectangle` chamada `Area` nas classes Invader e `PlayerShip` - para podermos usar o método Contains() e determinar se a área da nave se sobrepõe à de um tiro.

Completando a classe Game

O problema com os diagramas de classe é que eles normalmente deixam de fora qualquer propriedade ou método que não sejam públicos. Então, mesmo depois de ter terminado os métodos da página 693, ainda terá muito trabalho a fazer. Eis aqui algumas coisas para você pensar:

O construtor deve configurar tudo

O objeto `Game` precisa criar todos os demais objetos - os `Invader`, o `PlayerShip`, as `listas` para manter os tiros e o objeto `Stars`. O formulário passa um objeto `Random` inicializado e sua própria estrutura `DisplayRectangle` (para que `Game` possa **determinar os limites do campo de batalha**, o que será usado para checar se os tiros saíram da tela e se os invasores atingiram as bordas para que eles mudem de direção). Logo seu código precisa criar tudo mais que o jogo precise.

Falaremos mais a respeito desses objetos individualmente nas próximas várias páginas deste laboratório.

Desenvolva um método NextWave()

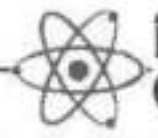
Um método simples para criar a próxima horda de invasores será bem útil. Ele deve adicionar uma nova `lista` de objetos `Invader` no campo de batalha, colocando os 30 invasores em suas posições iniciais em seis colunas, aumentar o campo wave em 1, e configurar o campo `invaderDirection` para que os invasores comecem a se movimentar inicialmente para o lado direito da tela. Ele também deve mudar o campo `framesSkipped`.

Eis um exemplo de como um método privado pode realmente ajudar na organização de sua classe Game

Algumas outras idéias para métodos privados

Aqui vão algumas outras idéias para métodos privados com os quais você pode brincar. Veja se alguma delas ajuda o projeto da sua classe `Game`.

- ✓ Um método para ver se o jogador foi atingido `()CheckForPlayerCollisions)`
- ✓ Um para ver se algum dos invasores foi atingido `(CheckForInvaderCollisions)`
- ✓ Um para mover todos os invasores `(MoveInvaders)`
- ✓ Um para permitir aos invasores dispararem `(ReturnFire)`



Poder do Cérebro

É possível mostrar propriedades e métodos protegidos e privados num diagrama de classe, mas você raramente verá isso sendo feito. Por que você acha que isso é tão raro?

A LINQ facilita a detecção de colisões

Isso parece realmente complicado quando você ler pela primeira vez, mas cada consulta LINQ será apenas um parâmetro de linhas de código. Aqui vai uma dica: não as complique demais!

Você tem coleções de invasores e de tiros e precisa consultá-las para encontrar certos invasores e tiros. Todas as vezes que você ouvir coleções e procurar na mesma frase, deve pensar na LINQ. Eis o que precisa ser feito:

1. **Descubra se a formação de invasores chegou na borda do campo de batalha**
   Os invasores precisam mudar de direção se qualquer um deles estiver dentro de uma área de 100 pixels da borda do campo de batalha. Quando eles estão marchando para a direita, uma vez que alcancem a borda direita do formulário o jogo precisa informá-los que eles precisam descer e começar a marchar para a esquerda. A mesma checagem deve ser feita quando eles estiverem marchando pra esquerda. Para isso acontecer, adicione um método privado `MoveInvaders()` que deve ser chamado por `Go()`. A primeira coisa que ele deve fazer é verificar e atualizar o campo privado `frameSkipped` e `retornar` se o quadro em que estivermos tiver de ser pulado (dependendo do número da horda). Então ele deve checar para qual direção os invasores estão se movendo. Se eles estiverem indo para a direita, `MoveInvaders()` deve usar LINQ para procurar na lista InvaderCollection por qualquer invasor a menos de 100 pixels do limite direito. Se algum for encontrado, os invasores devem ser informados para marchar para baixo e `invaderDirection` deve ser alterado para `Direction.Left`. Se nenhum for encontrado, deve-se fazer os invasores marcharem para a direita. Por outro lado, se eles estiverem indo para a esquerda, o procedimento inverso deve ser efetuado, usando outra consulta LINQ para ver se os invasores estão a mais de 100 pixels da borda esquerda. Da mesma forma faça-os descer e mude a direção se eles estiverem.

2. **Determine quais invasores podem atirar**
   Adicione um método privado chamado `ReturnFire()` que deve ser chamado por `Go()`. Primeiro, ele deve retornar se a lista de tiros dos invasores já possuir `wave mais um` tiro. Ele também deve retornar se `random.Next(10) < 10 - wave` (isso fará os invasores dispararem ao acaso e não a todo momento). Se os dois testes falharem, ele pode usar LINQ para agrupar os invasores por suas `Location.X` e ordenar usando `descending`. Uma vez que se tenham os grupos, pode-se escolher um deles ao acaso e usar seu método `First()` para achar o invasor mais abaixo da coluna. Certo, nesse ponto você terá aquele que vai atirar - você pode então adicionar um disparo na lista de tiros dos invasores exatamente abaixo do meio desse invasor (use a `Area` dos invasores para determinar a posição do tiro).

3. **Verifique se ocorreram colisões com o jogador e invasores**
   Você vai precisar criar um método para verificar a ocorrência de colisões. Existem três tipos delas para verificar e o método `Contains()` da struct `Rectangle` será muito útil - passe a ele qualquer `Point` e ele retornará `true` se o ponto estiver dentro do retângulo.

   - ★ Use LINQ para achar qualquer invasor morto iterando pelos tiros da lista de disparos do jogador e selecionar qualquer `invasor` cuja `Area` contiver a posição do tiro. Remova o invasor e o tiro.
   - ★ Adicione uma consulta para determinar se qualquer invasor atingiu a parte inferior da tela - se for o caso, encerre o jogo.
   - ★ Você não vai precisar da LINQ para procurar por tiros colididos com o jogador, apenas um laço e a propriedade `Area` dele (lembre-se, **você não pode modificar uma coleção de dentro de um laço foreach**. Se tentar, vai conseguir uma InvalidOperationException com uma mensagem dizendo que a coleção foi modificada).

Desenvolvendo a classe Invader

A classe Invader controla um invasor individual. Quando o objeto `Game` criar uma nova horda de invasores, ele deverá ter 30 instâncias de Invader num objeto `List<Invader>`. Cada vez que o método `Go()` for chamado, ele, por sua vez, ativará o método Move() de cada invasor. E cada vez que o método `Draw()` for chamado, o método de mesmo nome de cada invasor deve ser chamado também. Então você precisará criar os métodos `Move()` e `Draw()`. E você talvez queira adicionar um método privado chamado `InvaderImage()` também - ele será realmente útil quando você estiver desenhando o invasor. Certifique-se de chamá-lo dentro do método `Draw()` para manter o campo image atualizado.

```
public class Invader {
  private const int HorizontalInterval = 10;
  private const int VerticalInterval = 40;
  public enum Type {
    Bug,
    Saucer,
    Satellite,
    Spaceship,
    Star,
  }
  private Bitmap image;
  public Point Location { get; private set; }
  public Type InvaderType { get; private set; }
  public Rectangle Area { get {
    return new Rectangle(location, image.Size); }
  }
  public int Score { get; private set; }
  public Invader(Type invaderType, Point location, int score) {
    this.InvaderType = invaderType;
    this.Location = location;
    this.Score = score;
    image = InvaderImage(0);
  }
  // Additional methods will go here
}
```

A constante HorizontalInterval deve determinar quantos pixels um invasor se move a cada vez que ele dá um passo. VerticalInterval é o número de pixels que um invasor desce quanto a formação chega à borda do campo de batalha.

O invasor usa a enum Type para determinar que tipo de nave inimiga ele é.

Veja o que fizemos com a propriedade Area. Uma vez que sabemos a posição do invasor e seu tamanho (a partir de seu campo image), podemos acrescentar um acessador get que calcula um Rectangle da área coberta... ou seja, você pode usar o método **Contains()** do retângulo dentro de uma query LINQ para **detectar se qualquer tiro colidiu** com um invasor.

Invader
Location: Point InvaderType: Type Area: Rectangle Score: int
Draw(g: Graphics, animationCell: int) Move(direction: Direction)

Desenvolva os métodos de Invader

Existem cinco diferentes invasores e cada um deles tem quatro imagens de célula de animação.

Os três principais métodos de `Invader` são `Move()`, `Draw()` e `InvaderImage()`. Vamos examinar um de cada vez:

Movimente as naves invasoras

Primeiro, você precisa de um método para mover as naves invasoras. O objeto `Game` deve informar uma direção, usando a enum `Direction` e só então a nave deve se mover. Lembre-se, o objeto `Game` é o responsável por determinar se os invasores devem se mover para baixo ou mudar de direção, de forma que nossa classe `Invader` não se preocupe com isso.

```
public void Move(Direction direction) {
  // Este metodo precisa mover a nave na
  // direcao especificada
}
```

Desenhe a nave - a célula de animação correta

Cada `invasor` sabe como se desenhar. Dado um objeto `Graphics` e a célula de animação a ser usada, o invasor pode mostrar a si mesmo na área de jogo usando esse objeto, que deve ser passado a ele por `Game`.

```
public void Draw(Graphics g, int animationCell) {
  // Este metodo precisa desenhar a imagem da
  // nave, usando a celula de animacao correta
}
```

Recupere a imagem do invasor correta

Freqüentemente, você vai precisar recuperar a imagem correta se baseando na célula de animação que deve ser usada, então pode querer colocar esse código num método dedicado. Desenvolva um método `InvaderImage()` que retorna um `Bitmap` específico dada uma célula de animação.

```
private Bitmap InvaderImage(int animationCell) {
  // Este sera mais um metodo de conveniencia, e
  // ele retorna o bitmap correto para a celula especificada
}
```

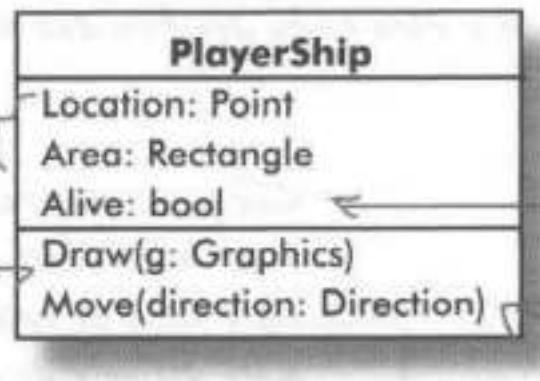
Cada invasor sabe seu tipo. Então se você passar para InvaderImage() um número de célula de animação, ele pode retornar um Bitmap que tenha a imagem certa.

A nave do jogador pode se mover e morrer

A classe `PlayerShip` controla a nave do jogador. Ela é parecida com a classe Invader, mas mais simples.

As propriedades Location e Area são exatamente as mesmas da classe Invader.

O método Draw() desenha a nave do jogador na posição certa - a não ser que ele esteja morto, e nesse caso ela desenha uma animação da nave sendo esmagada pelo disparo.



Quando a nave é atingida por um disparo, o jogo atribui false para sua propriedade Alive. O jogo então impede os invasores de se moverem até que a nave reinicie a propriedade e a altere para true novamente.

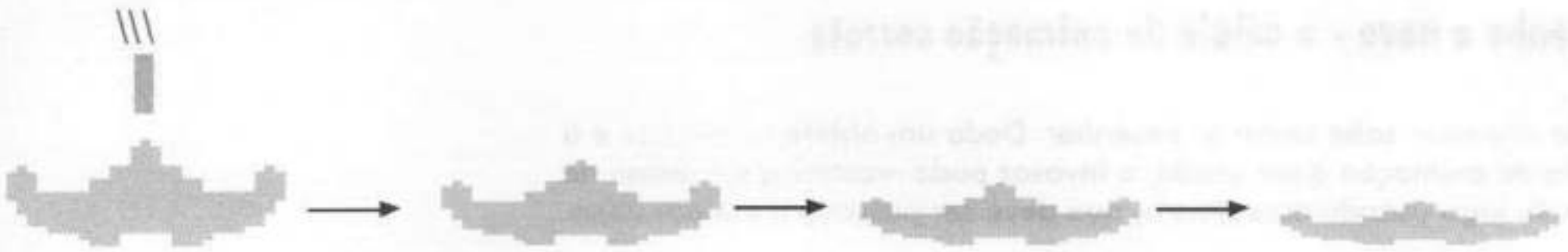
O método Move() tem um parâmetro, uma enum Direction, e deverá mover o jogador nessa direção.

Anime a nave do jogador quando ela for atingida

O método `Draw()` deve receber um objeto `Graphics` como parâmetro. Então ele deve checar a propriedade `Alive`. Se o jogador estiver vivo, ele é desenhado usando a propriedade `Location`. Se estiver morto, em vez de desenhar o bitmap comum, `PlayerShip` usa seu campo privado `deadShipHeight` (altura da nave morta) para animar a nave sendo lentamente esmagada pelo tiro. Depois de três segundos morta, a propriedade `Alive` deve ser alterada de volta para `true`.

PlayerShip precisará receber um Rectangle com o tamanho da área de jogo em seu construtor e certificar-se de que a nave não se mova para fora dessa área em Move().

Esperar três segundos é fácil - use o acessador set da propriedade Alive para dar a um campo privado do tipo DateTime o valor DateTime.Now. A primeira coisa que o método Go() da nave deve fazer é usar um TimeSpan para verificar se três segundos se passaram. Se ainda não for o caso, continue executando a animação de esmagamento. Assim os três segundos transcorrerem, modifique Alive de volta para true para que o jogo saiba que deve continuar (você usou um truque semelhante no simulador de colméia).



```
public void Draw(Graphics g) {
  if (!Alive) {
```

Reinicie o campo deadShipHeight e desenhe a nave

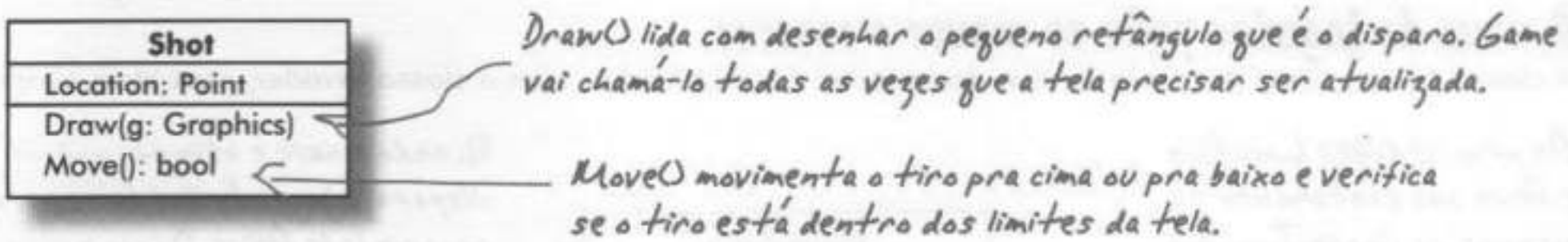
```
  } else {
```

Verifique o campo deadShipHeight. Se ele for maior do que zero, diminua seu valor em 1 e use DrawImage() para desenhar a nave um pouco mais achatada.

```
  }
}
```

"Tiros disparados"

Game deve ter duas listas de objetos Shot: uma para os tiros do jogador, que se movem para cima na tela, e outra para os disparos dos inimigos, que se movem para baixo. Os tiros precisam apenas de alguns detalhes para funcionar: uma posição tipo Point, um método para desenhá-lo e um para se mover. Aqui está o diagrama de classe:



Aqui está o início da classe Shot:

```
public class Shot {
  private const int moveInterval = 20;
  private const int width = 5;
  private const int height = 15;
  public Point Location { get; private set; }
  private Direction direction;
  private Rectangle boundaries;
  public Shot(Point location, Direction direction,
              Rectangle boundaries) {
    this.Location = location;
    this.direction = direction;
    this.boundaries = boundaries;
  }
  // seu codigo aqui
}
```

Você pode ajustar estes valores para fazer o jogo ficar mais fácil ou mais difícil... quanto menor o tiro mais fácil desviar-se dele; tiros mais rápidos são mais difíceis de evitar-se.

Como o disparo atualiza sua própria posição no método Move(), a posição pode ser uma propriedade automática apenas-leitura.

Direction é a enum que define Up e Down.

O jogo passa o retângulo de exibição do formulário como parâmetro boundaries do construtor para que o tiro saiba quando sair da tela.

Sua tarefa é ter certeza de que Draw() receba um objeto Graphics que desenhe o tiro como um retângulo amarelo. Então Move() deve deslocar o disparo para cima ou para baixo e retornar true se o tiro ainda estiver dentro dos limites do jogo.

Brilha, brilha... você é quem diz

A última classe de que você vai precisar é Stars. Existirão 300 estrelas e esta classe armazenará todas elas, fazendo com que cinco sejam exibidas e cinco desaparecem a cada vez que `Twinkle()` for chamado.

Primeiro, no entanto, você vai precisar de uma `struct` para cada estrela:

Aqui vai mais uma dica: inicie o projeto com apenas um formulário, uma classe Game e uma Stars. Veja se você consegue com que um retângulo preto representando o céu seja desenhado cheio de estrelas. Isso deve dar a você uma base sólida para acrescentar as demais classes e métodos.

```
private struct Star {
  public Point point;
  public Pen pen;
  public Star(Point point, Pen pen) {
    this.point = point;
    this.pen = pen;
  }
}
```

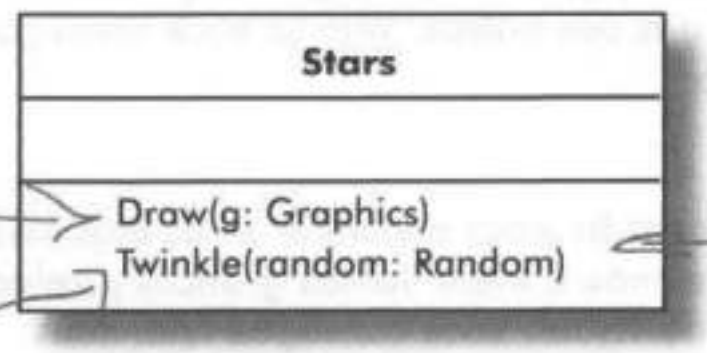
Cada estrela tem um ponto (sua posição) e uma caneta (para determinar sua cor).

Tudo o que Star faz é armazenar dados... ela não tem comportamento algum.

A classe Stars precisará manter uma `List<Star>` parâmetro armazenar 300 destas `structs Star`. Você precisará criar um construtor para Star que povoe essa lista. Ele deve receber um `Rectangle` com os limites de exibição e uma instância de `Random` para usar e criar os Points em posições aleatórias onde as estrelas devem ser colocadas.

Eis o diagrama para a classe `Stars`, com os demais métodos de que você vai precisar.

Você pode definir a estrutura Star dentro de Stars.cs, já que apenas essa classe vai usar essa estrutura.



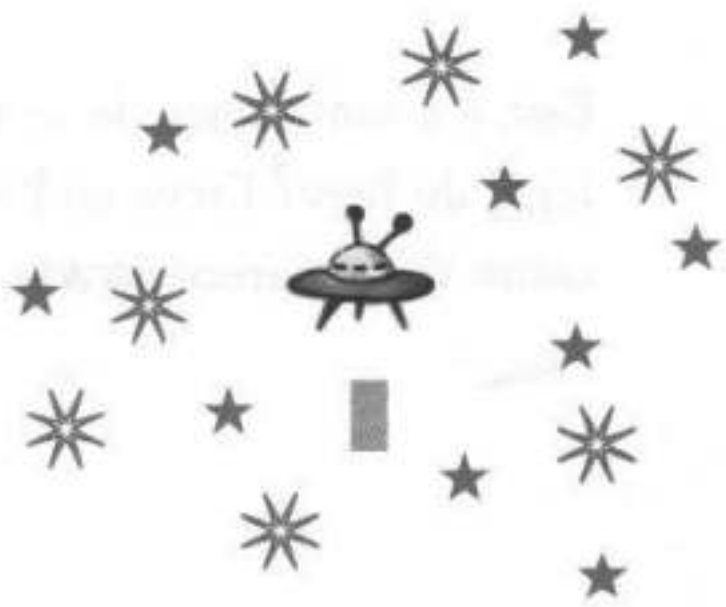
Draw() desenhará todas as 300 estrelas...

...e Twinkle() remove cinco estrelas e adiciona cinco novas.

Game manterá uma instância de Random que todos os objetos podem usar.

`Draw()` deve desenhar todas as estrelas na lista e `Twinkle()` deve remover cinco delas aleatoriamente e adicionar mais cinco em substituição.

Você pode querer criar um método `RandomPen()` para facilmente dar uma cor aleatória às estrelas quando uma nova delas for criada. Ela deve retornar uma das cinco cores possíveis para estrelas, gerando um número entre 0 e 4 e selecionando um objeto `Pen` correspondente.



E ainda pode ser feito mais...

Acha que o jogo está bem legal? Você pode levar as coisas ainda mais adiante com algumas poucas melhorias:

Adicione explosões animadas
Faça com que cada invasor exploda depois de ser atingido e exiba brevemente um número para que o jogador saiba quantos pontos ele ganhou com aquele invasor.

Acrescente uma nave-mãe
De vez em quando, uma nave-mãe valendo 250 pontos atravessa o topo do campo de batalha. Se o jogador conseguir atingi-la, ganha um bônus. Inclua escudos

Adicione escudos
Adicione escudos flutuantes para que o jogador possa se ocultar. Você pode adicionar escudos simples que barre os tiros de inimigos e do jogador. Depois disso, se você realmente quiser que seu jogo seja bom, modifique-os para que o jogador e os invasores possam abrir furos neles depois de um certo número de tiros.

Tente fazer com que os escudos suportem um número menor de tiros conforme o jogo avança.

Crie bombardeiros
Crie um tipo especial de inimigo que mergulhe e bombardeie o jogador. Um bombardeiro deve sair da formação, avançar na direção do jogador, voar rente à parte inferior da tela e voltar à sua posição original.

Desenvolva mais armas
Inicie uma corrida armamentista! Bombas inteligentes, lasers, mísseis teleguiados... Existe todo tipo de armas que tanto o jogador quanto os invasores podem usar para atacar uns aos outros. Veja se você consegue adicionar três novas armas ao jogo.

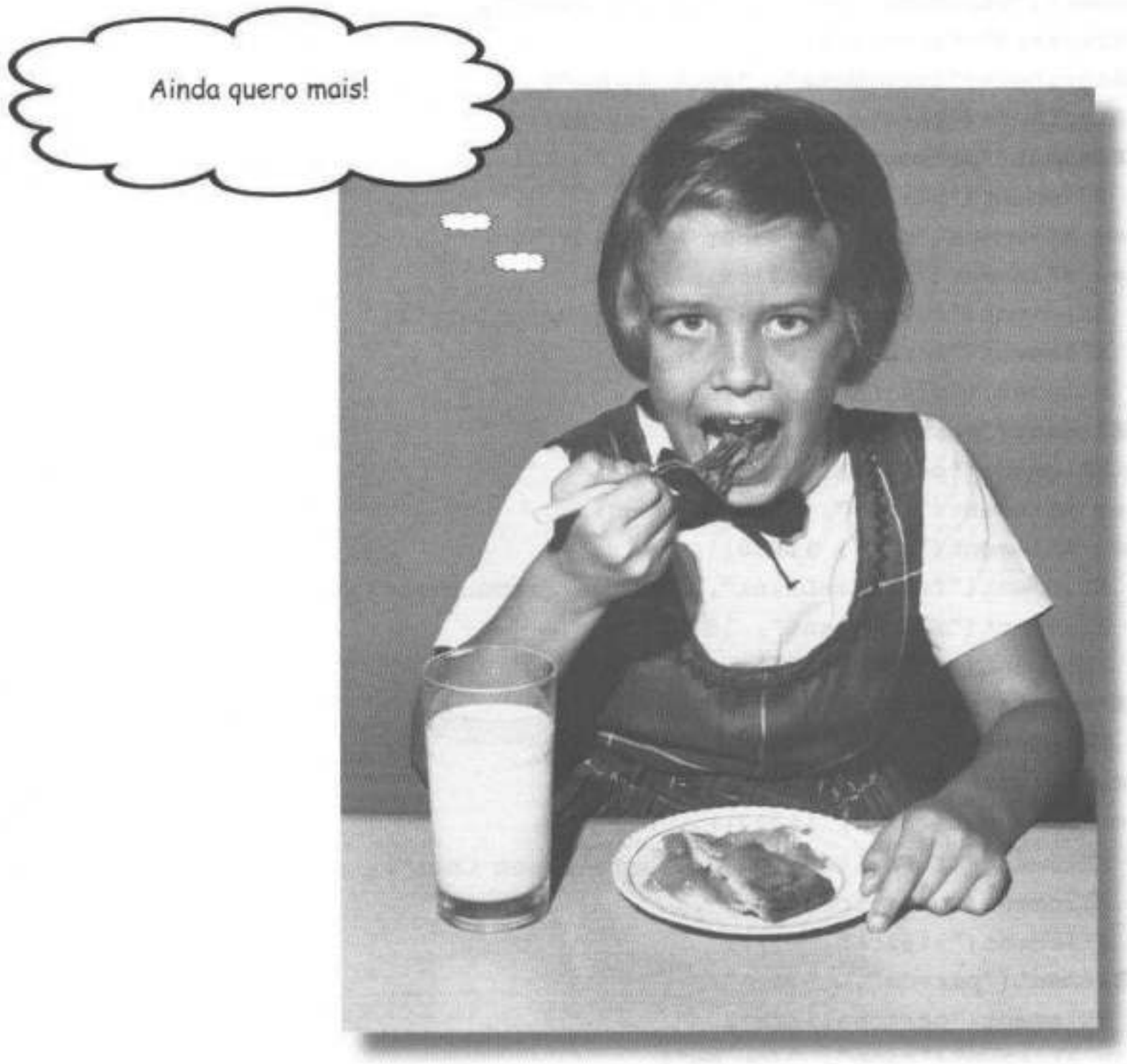
Incremente os gráficos
Você pode ir até **www.altabooks.com.br** para encontrar mais arquivos gráficos para escudos simples, nave-mãe e mais. Temos gráficos pixelados para dar aquele estilo de anos 80 ao visual. Você consegue criar seus próprios gráficos para dar ao jogo um novo estilo?

Um bom projeto de classes deve permitir que você altere os gráficos com mudanças mínimas de código.

Esta é a sua chance de se mostrar! Você criou uma nova versão muito legal do jogo? Entre no Fórum do Use a Cabeça C# e conte vantagem como você merece: **www.altabooks.com.br**

Apendice I: sobras

Os 5 principais tópicos que gostaríamos de ter incluido neste livro



A diversão está apenas começando!

Mostramos a você muitas ferramentas ótimas para desenvolver **programas poderosos** em C#. Mas não seria possível incluir **cada uma das ferramentas, tecnologias e técnicas** neste livro - simplesmente não existem páginas o suficiente. Tivemos que fazer algumas escolhas realmente difíceis a respeito sobre o que incluir e o que deixar de fora. Aqui apresentamos alguns itens que não sobreviveram aos cortes. Ainda que não possamos nos aprofundar neles, acreditamos que eles são **importantes e úteis**, e queremos dar a você uma pequena introdução a respeito deles.

1 LINQ para XML

XML - Extensible Markup Language, linguagem extensível de marcação - é um formato para arquivos e streams de dados que representam dados complexos usando texto. O Framework .NET dá a você algumas ferramentas muito poderosas para criar, carregar e salvar arquivos XML. E uma vez que você tenha em suas mão os dados XML, pode usar LINQ para fazer consultas. Adicione "`using System.Xml.Linq;`" no início do arquivo e digite o seguinte método - ele gera um documento XML para armazenar os dados de fidelidade de clientes do Starbuzz.

```
private static XDocument GetStarbuzzData() {
  XDocument doc = new XDocument(
    new XDeclaration("1.0", "utf-8", "yes"),
    new XComment("Starbuzz Customer Loyalty Data"),
      new XElement("starbuzzData",
      new XAttribute("storeName", "Park Slope"),
      new XAttribute("location", "Brooklyn, NY"),
    new XElement("person",
      new XElement("personalInfo",
        new XElement("name", "Janet Venutian"),
        new XElement("zip", 11215)),
      new XElement("favoriteDrink", "Choco Macchiato"),
      new XElement("moneySpent", 255),
      new XElement("visits", 50)),
    new XElement("person",
      new XElement("personalInfo",
        new XElement("name", "Liz Nelson"),
        new XElement("zip", 11238)),
      new XElement("favoriteDrink", "Double Cappuccino"),
      new XElement("moneySpent", 150),
      new XElement("visits", 35)),
    new XElement("person",
      new XElement("personalInfo",
        new XElement("name", "Matt Franks"),
        new XElement("zip", 11217)),
      new XElement("favoriteDrink", "Zesty Lemon Chai"),
      new XElement("moneySpent", 75),
      new XElement("visits", 15)),
    new XElement("person",
      new XElement("personalInfo",
        new XElement("name", "Joe Ng"),
        new XElement("zip", 11217)),
      new XElement("favoriteDrink", "Banana Split in a Cup"),
      new XElement("moneySpent", 60),
      new XElement("visits", 10)),
    new XElement("person",
      new XElement("personalInfo",
        new XElement("name", "Sarah Kalter"),
        new XElement("zip", 11215)),
      new XElement("favoriteDrink", "Boring Coffee"),
      new XElement("moneySpent", 110),
      new XElement("visits", 15))));
  return doc;
}
```

Salve e Abra arquivos XML

Você pode exibir um objeto XDocument no console ou salvá-lo num arquivo, e pode carregar um arquivo XML nele:

```
XDocument doc = GetStarbuzzData();
Console.WriteLine(doc.ToString());
doc.Save("starbuzzData.xml");
XDocument anotherDoc = XDocument.Load("starbuzzData.xml");
```

O objeto XDocument tem métodos Load() e Save() para ler e escrever arquivos XML. E este método ToString() transforma tudo dentro do arquivo num grande documento XML.

Consulte seus dados

Eis aqui uma consulta LINQ simples que pesquisa nos dados do Starbuzz usando seu XDocument:

```
var data = from item in doc.Descendants("person")
  select new { drink = item.Element("favoriteDrink").Value,
    moneySpent = item.Element("moneySpent").Value,
    zipCode = item.Element("personalInfo").Element("zip").Value };
foreach (var p in data)
  Console.WriteLine(p.ToString());
```

O método Descendants() (descendentes) retorna uma referência para um objeto que você pode incluir diretamente dentro da sua consulta LINQ.

Você já sabe que a LINQ permite que você chame métodos e use isso como parte da consulta, o que funciona muito bem com o método Element().

E você pode fazer consultas mais complexas também:

```
var zipcodeGroups = from item in doc.Descendants("person")
  group item.Element("favoriteDrink").Value
  by item.Element("personalInfo").Element("zip").Value
    into zipcodeGroup
    select zipcodeGroup;
foreach (var group in zipcodeGroups)
  Console.WriteLine("{0} favorite drinks in {1}",
                    group.Distinct().Count(), group.Key);
```

Element() retorna um objeto XElement, e você pode usar suas propriedades para procurar por valores específicos no seu documento XML.

Leia dados de um canal RSS

Você pode fazer algumas coisas muito poderosas com LINQ para XML.
Eis aqui uma consulta simples para **ler artigos do nosso blog**:

```
XDocument ourBlog = XDocument.Load("http://www.stellman-greene.com/feed");
Console.WriteLine(ourBlog.Element("rss").Element("channel").Element("title").Value);
var posts = from post in ourBlog.Descendants("item")
  select new { Title = post.Element("title").Value,
    Date = post.Element("pubDate").Value};
foreach (var post in posts)
  Console.WriteLine(post.ToString());
```

O método Load() de XDocument tem muitos construtores sobrecarregados. Este aqui recupera dados XML de uma URL.

Cologue um botão num formulário, certifique-se de estar usando using System.Xml.Linq; no início do arquivo, digite esta consulta num tratador de evento, e veja o que ela imprime no console.

Usamos a URL do nosso blogo, Building Better Software (desenvolvendo programas melhores): http://www.stellman-greene.com/

2 Refatoração

Refatoração quer dizer alterar a forma como seu código está estruturado sem mudar seu comportamento. Sempre que você escrever um método complexo, deve parar uns minutos e pensar como você poderia modificá-lo para que ficasse mais fácil de entender. Por sorte, o IDE tem algumas ferramentas de refatoração muito úteis. Existe todo o tipo de refatoração que pode ser feita - aqui estão alguns exemplos que usamos com frequência.

Extrair um método

Quando estávamos escrevendo o renderizador baseado em controles do Capítulo 13, originalmente incluímos o seguinte laço foreach:

```
foreach (Bee bee in world.Bees) {
  beeControl = GetBeeControl(bee);
  if (bee.InsideHive) {
    if (fieldForm.Controls.Contains(beeControl)) {
      fieldForm.Controls.Remove(beeControl);
      beeControl.Size = new Size(40, 40);
      hiveForm.Controls.Add(beeControl);
      beeControl.BringToFront();
    } else if (hiveForm.Controls.Contains(beeControl)) {
      hiveForm.Controls.Remove(beeControl);
      beeControl.Size = new Size(20, 20);
      fieldForm.Controls.Add(beeControl);
      beeControl.BringToFront();
    }
    beeControl.Location = bee.Location;
  }
```

Estas quatro linhas movem um BeeControl do formulário do campo para o da colméia.

E estas quatro movem o BeeControl da colméia para o campo.

Um dos nossos revisores técnicos, Joe Albahari, destacou que isso era um pouco difícil de ler. Ele sugeriu que **extraíssemos esses dois blocos de quatro linhas e os transformássemos em métodos**. Então selecionamos o primeiro bloco, clicamos nele com o botão direito, e selecionamos "Refactor>>Extract Method..." - e a seguinte janela foi exibida:

Digitamos o nome do novo método. Decidimos chamá-lo MoveBeeFormFieldToHive() porque isso em suma descreve o que o código faz.

O IDE examina o código que está selecionado e determina que ele use uma variável BeeControl chamada beeControl, então ele a adicionou como parâmetro do método.

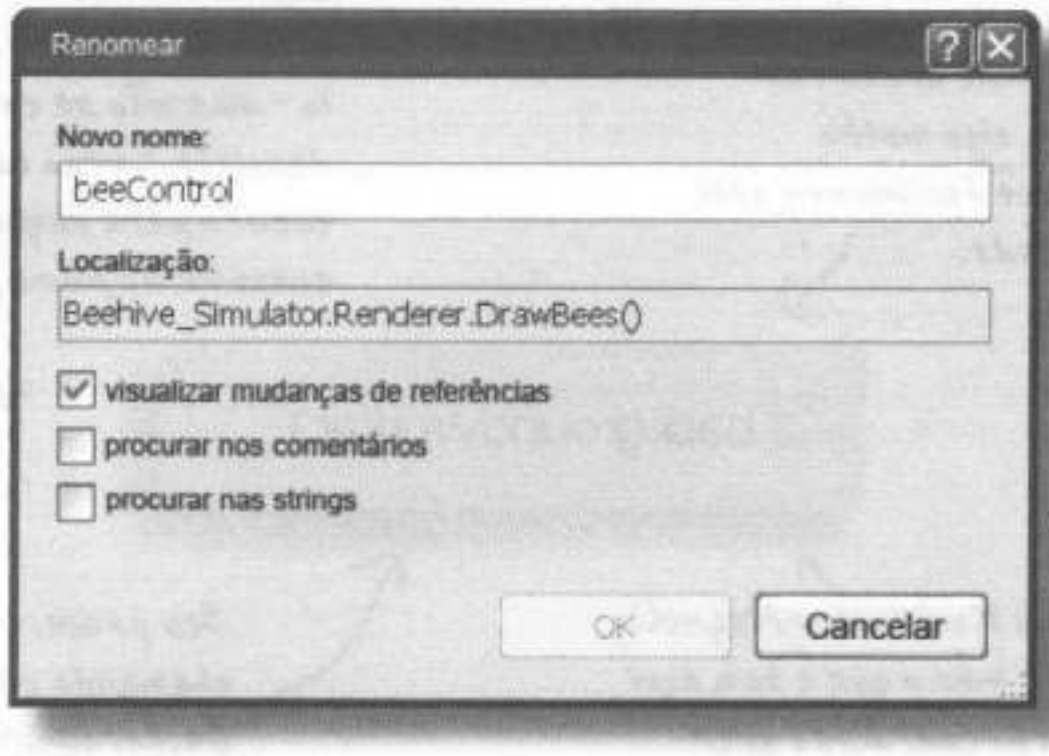
Então fizemos a mesma coisa com o segundo bloco de quatro linhas, extraindo-o para um método que chamamos de MoveBeeFromHiveToField(). Eis como o laço foreach ficou no final - muito mais fácil de ler:

```
foreach (Bee bee in world.Bees) {
  beeControl = GetBeeControl(bee);
  if (bee.InsideHive) {
    if (fieldForm.Controls.Contains(beeControl))
      MoveBeeFromFieldToHive(beeControl);
  } else if (hiveForm.Controls.Contains(beeControl))
    MoveBeeFromHiveToField(beeControl, bee);
  beeControl.Location = bee.Location;
}
```

Renomear uma variável

Lá atrás no Capítulo 3, explicamos como pode-se deixar seu código muito mais fácil de entender ao escolher nomes intuitivos para suas classes, métodos, campos e variáveis. O IDE pode realmente ajudar quando se trata de nomear coisas no código. Basta clicar com o botão direito em qualquer classe, variável, campo, propriedade, namespace, constante - basicamente qualquer coisa que se possa nomear - e escolher "Refactor >> Rename". Fizemos isso com "beeControl" no código do simulador. Eis a janela que apareceu:

Esta janela permite que você escolha um novo nome para o item. Se você tivesse escolhido, digamos, Bobbo, então o IDE iria vasculhar todo o código e mudar **cada ocorrência** para o novo nome.



O IDE faz um trabalho muito minucioso no que se refere a renomear. Se você fizer isso com uma classe, ele mudará cada comando ou instância que a utilize. Você pode clicar em qualquer ocorrência do nome, em qualquer lugar no código, e o IDE vai fazer a alteração no restante do programa.

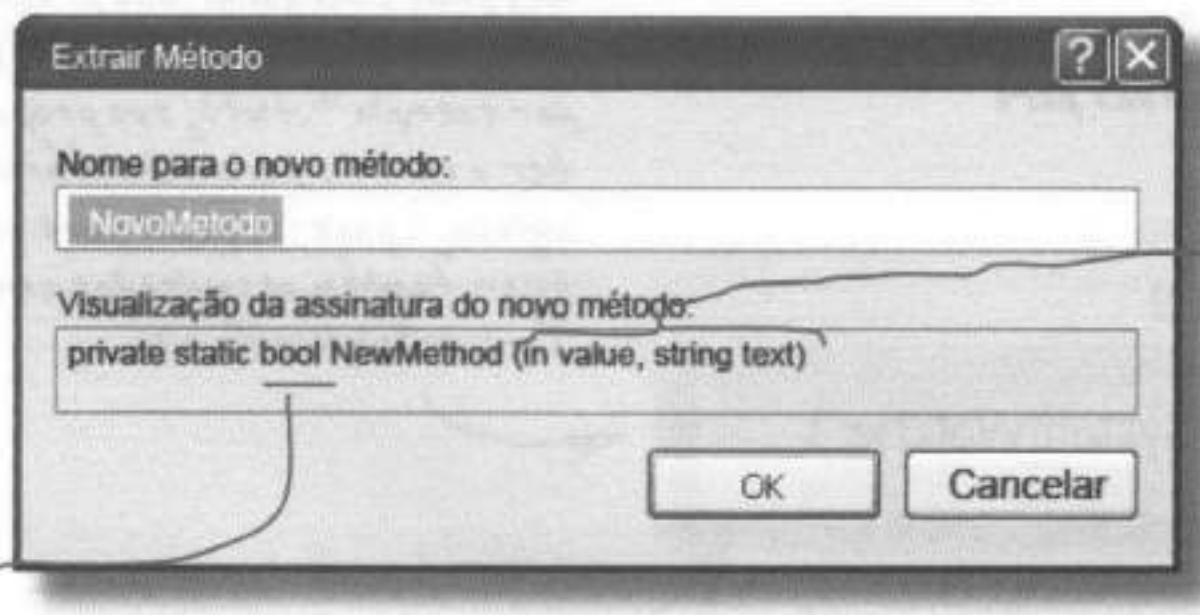
Consolidando uma expressão condicional

Eis aqui uma forma muito interessante de usar a características "Extrair Método". Abra qualquer programa, acrescente um botão, insira o seguinte código no tratador de evento:

```
private void button1_Click(object sender, EventArgs e) {
  int value = 5;
  string text = "Hi there";
  if (value == 36 || text.Contains("there"))
    MessageBox.Show("Pow!");
}
```

Selecione tudo dentro do comando if: `value == 36 || text.Contains("there")`. Então clique com o botão direito e escolha "Refactor >> Extract Method...". Aqui está o que você vai ver:

Todas as expressões condicionais resultam num booleano, então o IDE vai criar um método que retorne um para substituir o teste condicional com uma chamada para esse método.



Não apenas isso fará o código mais fácil de ler, mas agora você tem um novo método que pode reutilizar em outro lugar!

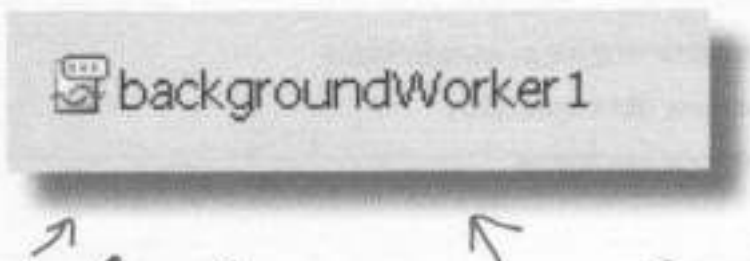
A expressão usa dois variáveis chamadas value e text, então o IDE adiciona parâmetros ao método usando esses nomes.

3 Alguns de nossos componentes favoritos na caixa de ferramentas

Este é um livro a respeito de aprender C#, não sobre vantagens e desvantagens a respeito de componentes que vem com o Framework .NET. Ainda assim, temos nossos favoritos, e gostaríamos de compartilhar um pouco com você.

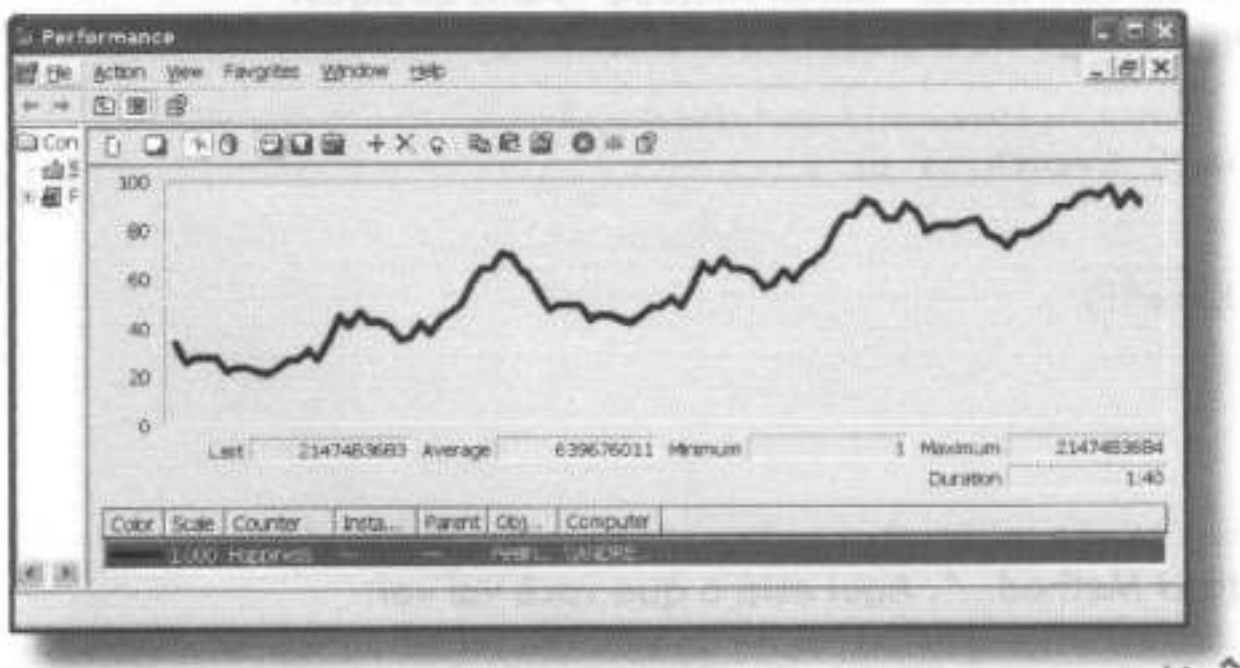
BackgroundWorker é um desse componentes não visuais (como o Timer) que você pode arrastar para seus formulários, e ele faz algo muito interessante - permite que você facilmente crie aplicativos com múltiplas threads.

Basta arrastar um para seu formulário e clicar duas vezes nele (ou, se quiser, instanciá-lo - mas não se esqueça de descartá-lo depois!). Então adicione um tratador de evento para fazer o quer que você queira que execute no plano de fundo.



Quando você chamar seu método RunWorkerAsync(), ele dispara o método DoWork() - mas o que é bom aqui é que ele faz isso numa outra thread. Isso quer dizer que ele vai executar em paralelo com as demais tarefas que seu programa estiver realizando.

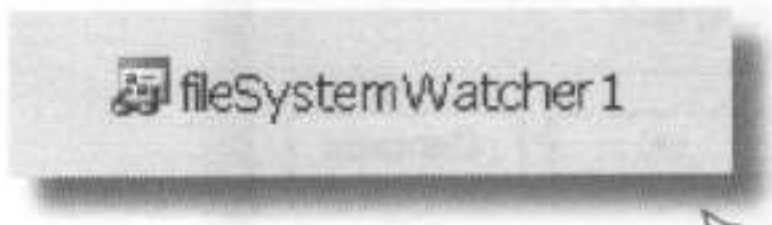
Seu processo pode relatar progresso chamando o método ReportProgress() do BackgroundWorker - isso faz com que ele levante o evento ProgressChanged.



Criamos esse contador Happiness (felicidade) na categoria Feelings (sentimentos), e demos a ele dados usando PerformanceCounter.

O FileSystemWatcher faz o que o nome indica - ele vigia o sistema de arquivos para ver se alguma coisa mudou.

Você pode configurar sua propriedade Filter para o tipo de arquivo que ele deve procurar (como por exemplo *.txt), sua propriedade Path deve ter o caminho para o diretório que deve ser vigiado, e pode-se pedir para que os subdiretórios sejam também pesquisados com a propriedade IncludeSubdirectories.



Uma vez que tenha sido configurado, ele vigia o diretório procurando por qualquer arquivo novo ou alterado. Assim que um arquivo é adicionado, alterado ou apagado, ele dispara seu evento Changed. Ele também tem os eventos Created, Deleted e Renamed para fazer um controle mais específico.

Quando o FileSystemWatcher dispara um evento, ele passa informações sobre o arquivo que foi alterado (ou adicionado, apagado, etc) usando um objeto FileSystemEventArgs.

Quando você estiver escrevendo um programa que fique executando continuamente, é realmente útil monitorá-lo. E o Windows vem com uma ferramenta útil chamada Monitor de Performance (perfmon.exe) que permite que se monitorem processos.

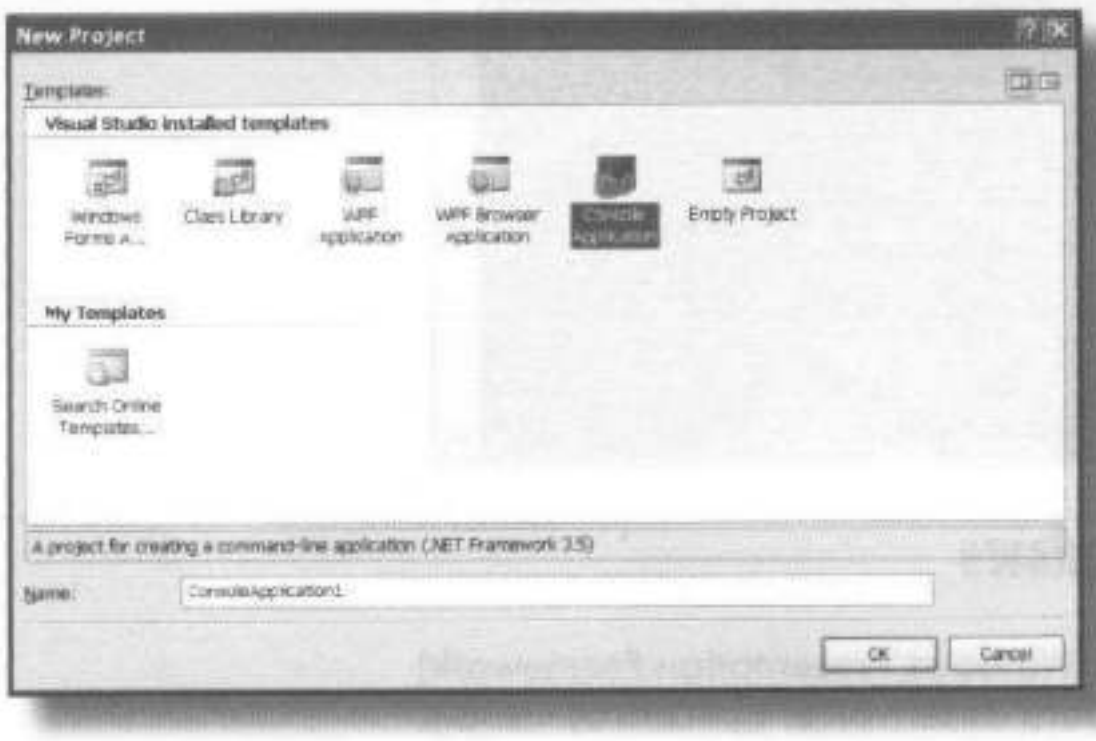
O componente PerformanceCounter (contador de performance) permite que você disponibilize informações sobre seu programa para o sistema de monitoramento de performance do Windows. Use Increment() e Decrement() ou atribua RawValue (valor bruto, literalmente valor crú) diretamente. Assim que você fizer isso, poderá ver os dados no Monitor de Performance.

O Windows divide seus contadores de performance em categorias, então você vai precisar criar uma - existem métodos para isso em SystemDiagnostics. Depois basta associar seu PerformanceCounter com a categoria criada, e você pode começar a enviar informações de diagnóstico, para seu deleite!

4 Aplicativos de Console

Muitos livros sobre C# começam com aplicativos de console. Achamos que isso é chato. É muito mais satisfatório desenvolver programas que parecem, bem... com qualquer coisa que seja. E é exatamente isso que um programa de console não é. Mas algumas vezes você vai precisar escrever um aplicativo de linha de comando. Por sorte, isso é bastante fácil. Veja como:

1 **Crie um projeto de Aplicativo de Console**
Qualquer projeto pode ser um aplicativo de console. Entre em qualquer um, selecione "Properties" no menú Project, e altere "Output type" (tipo de saída) para "Console Aplication". Mas é mais fácil criar uma a partir do zero.



2 **O IDE vai adicionar apenas um arquivo - Program.cs**
E ele terá um ponto de entrada vazio... e é tudo.

```
class Program
{
  static void Main(string[] args)
  {

  }
}
```

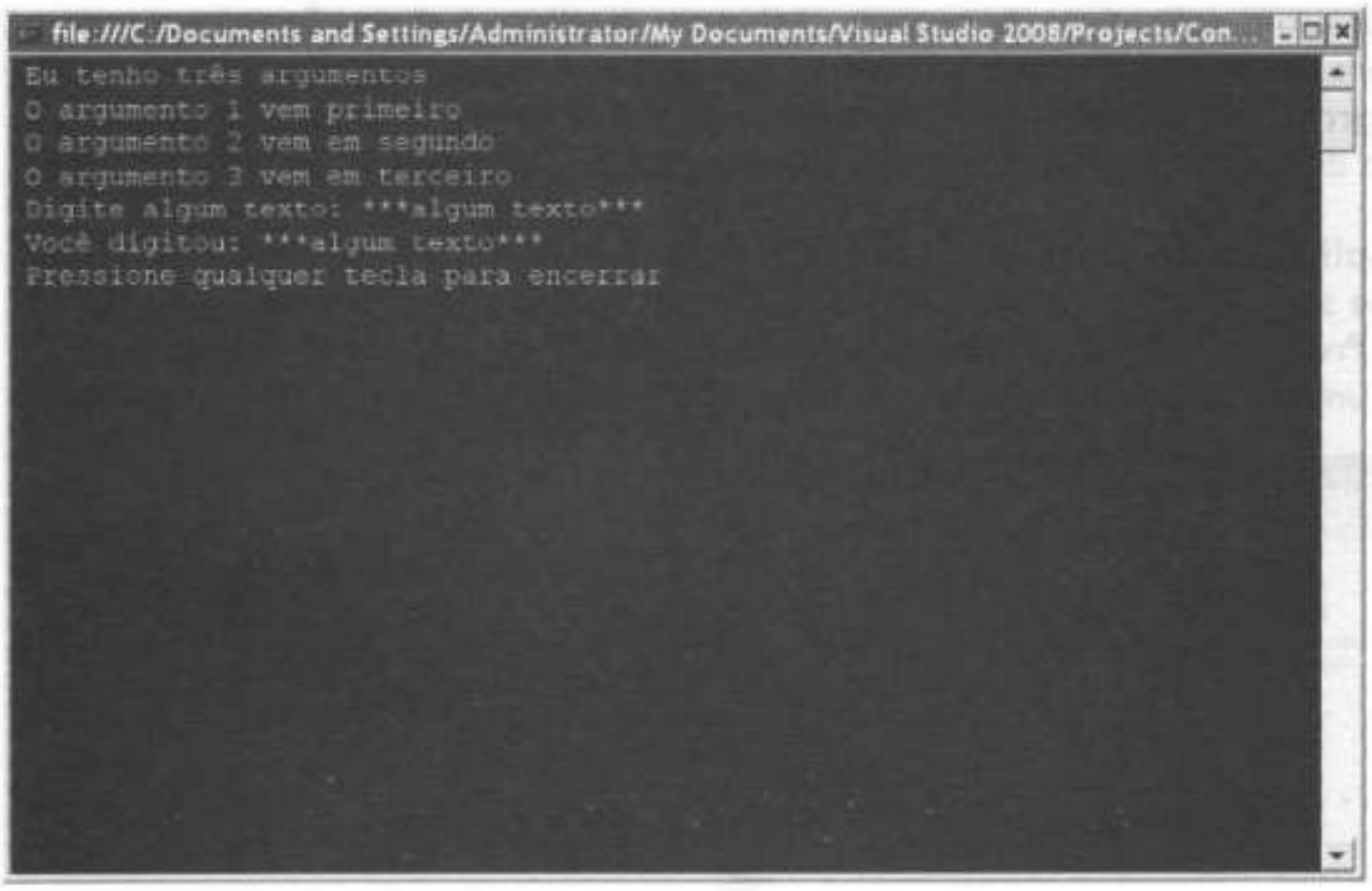
Aqui vai um pequeno projeto para você: pegue o descarregador hexadecimal que você desenvolveu no Capítulo 9 e o transforme num aplicativo de console. Faça com que ele leia um arquivo passado na linha de comando e o imprima como uma descarga hexa. Faça com que ele aceite entradas direto da linha de comando (usando Console.ReadLine) e imprima essa entrada como uma descarga hexa. Então dê uma pesquisada a respeito do comando od do Unix e veja se você pode reproduzi-lo em C#.

então é por isso que essa classe é chamada Console

3 **Use o parâmetro args para obter os argumentos de linha de comando**
Seu ponto de entrada tem um único parâmetro, uma matriz de strings chamada args, que conterá os argumentos de linha de comando. Você já sabe como usar o método Console.WriteLine() - e existem alguns outros métodos úteis do console, incluindo ReadLine() e ReadKey() (ler tecla).

```
class Program {
  static void Main(string[] args) {
    Console.WriteLine("I got {0} arguments", args.Length);
    for (int i = 1; i <= args.Length; i++)
      Console.WriteLine("Argument #{0} is {1}", i, args[i - 1]);
    Console.Write("Enter some text: ");
    string input = Console.ReadLine();
    Console.WriteLine("You entered: {0}", input);
    Console.WriteLine("Press any key to end...");
    Console.ReadKey();
  }
}
```

4 **Depure seu programa numa janela de console**
Quando você depurar seu programa, o IDE mostra uma janela de console. Os métodos ReadLine() e ReadKey() obtém seus resultados dessa mesma janela - basta digitar nela. E ao invés de escrever na janela Output, um aplicativo de console escreve em vez disso também nessa janela. Você pode alterar os argumentos de linha de comando na página "Debug" da janela de propriedades de projeto.



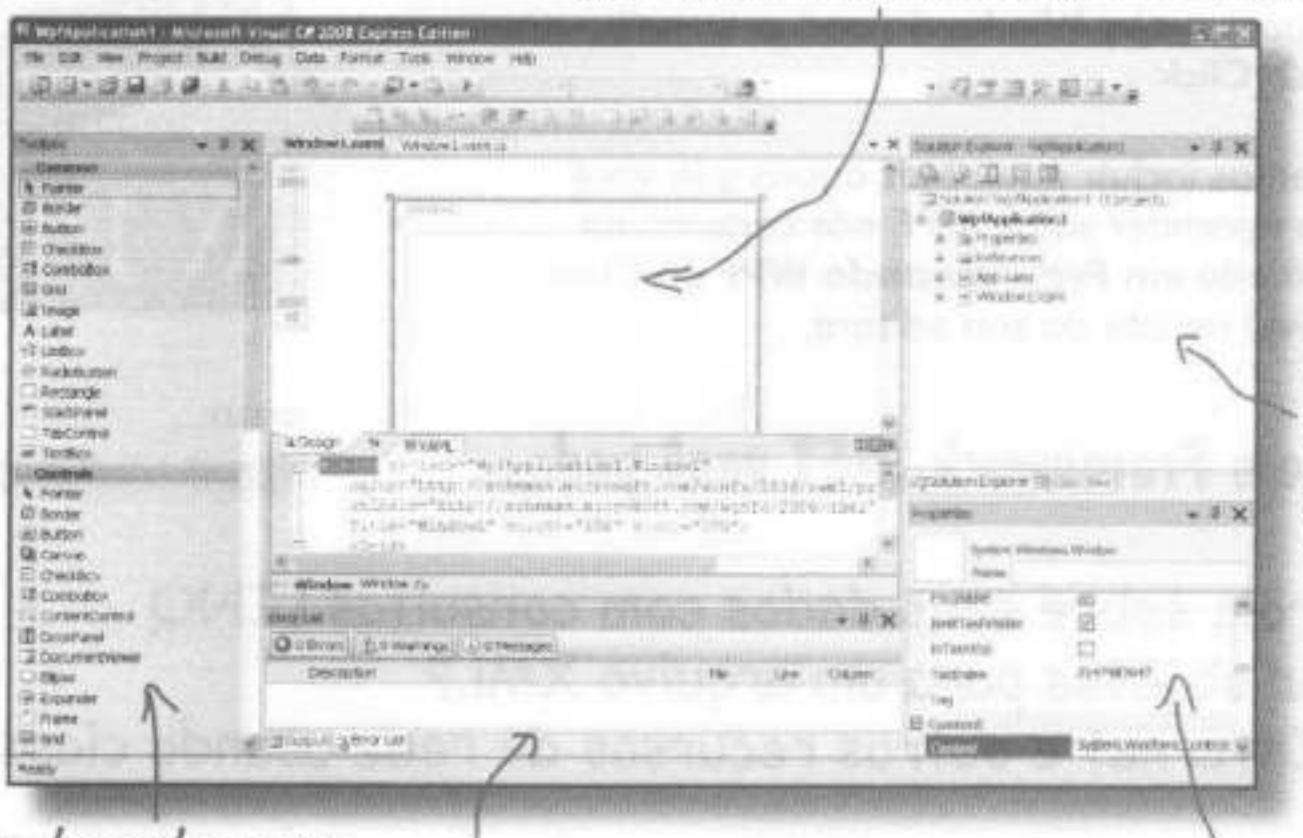
5 Framework de Apresentação do Windows

O Framework de Apresentação do Windows (WPF, Windows Presentation Framework) é a plataforma de próxima geração da Microsoft para desenvolver aplicativos visuais. Ela é extraordinária - tem um layout baseado em XML, controles escaláveis, um sistema totalmente novo para controles, gráficos e animações 2D e 3D, fluxo de texto e formatação de documentos, e mesmo um plugin para navegadores para qualquer plataforma que a utiliza.

Infelizmente, embora o WPF seja realmente legal e uma tecnologia de alta capacidade, não é uma ferramenta particularmente boa para ensinar C#. E esse era nosso objetivo - enfiar conceitos de C# na sua cabeça o mais rápido e fácil possível.

Reserve uns momentos e crie um novo aplicativo WPF. Basta criar um novo projeto usando o IDE, mas não crie um de Windows Forms. Em vez disso, **selecione Aplicativo WPF**. Você imediatamente vai notar uma diferença no IDE.

A maior diferença que você vai perceber é que o designer de formulário não se parece nada com o que você está acostumado. Examinaremos isso melhor num minuto.



Os aplicativos WPF ainda usam classes, como qualquer outro programa C# ou .NET, logo o Navegador de Solução é o mesmo.

Examine atentamente a caixa de ferramentas - ela tem todo um novo conjunto de controles.

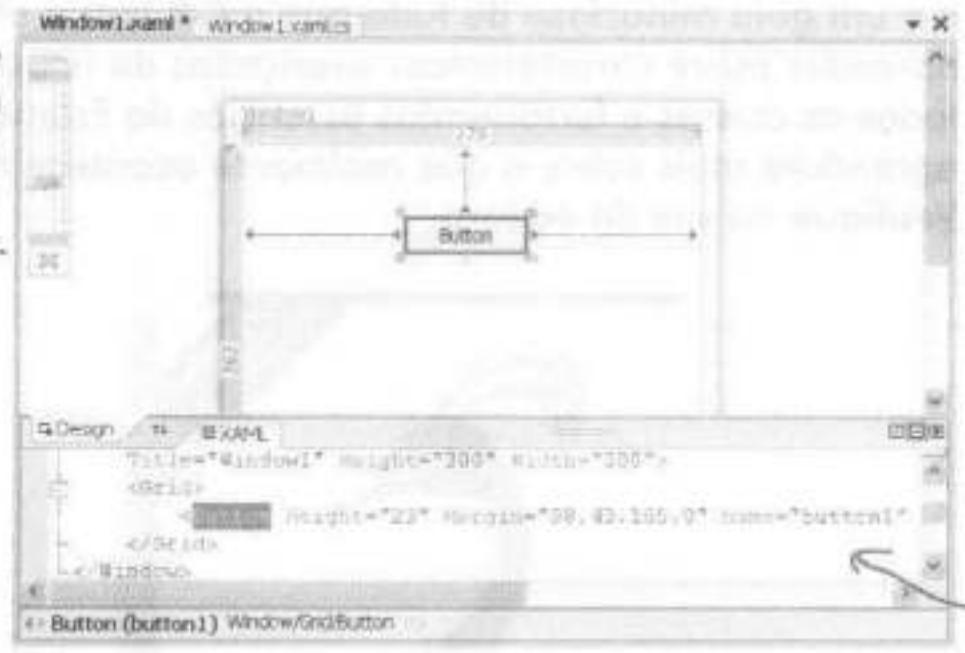
Leia as janelas de lista de erros e resultados já familiares que você tem usado.

As propriedades parecem totalmente diferentes. Isso é porque você usa essa janela para alterar atributos num arquivo XAML, e não propriedades de objetos.

Arraste um botão da caixa de ferramentas para o formulário. Se esse fosse um aplicativo Windows Forms, o IDE adicionaria código em Form1.Designer.cs para adicionar um controle ao objeto Form1. Mas com o WPF é diferente - ele usa uma linguagem baseada em XML chamada XAML para definir como a interface de usuário é apresentada.

XAML é abreviação de Linguagem de Marcação Extensível para Aplicativos (extensible application markup language), e é a linguagem baseada em XML que aplicativos WPF usam para determinar onde todos os controles e outros elementos de UI aparecem.

Arraste o cursor deste controle para cima e para baixo para aumentar ou diminuir o zoom. Mesmo quando você amplia muito, sua interface continua boa - ela não fica pixelada.



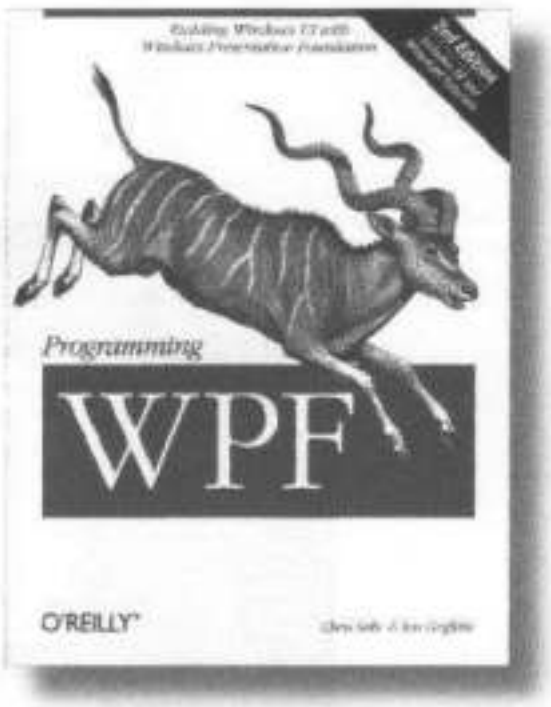
O IDE tem um editor de XML realmente poderoso otimizado para trabalhar com XAML.

Entre no editor XML e adicione um segundo botão digitando a linha em negrito abaixo. Você vai perceber que o IntelliSense do IDE faz um bom trabalho auxiliando você a digitar todas as tags XML.

```
<Grid>
  <Button Height="23" Margin="98,43,105,0" Name="button1"
                  VerticalAlignment="Top" Click="button1_Click">Button</Button>
  <Button Height="23" Margin="5,5,100,20" Name="button2"
                  VerticalAlignment="Top" Click="button2_Click">Another button</Button>
</Grid>
```

Quando você chegar na parte da linha que diz "`Click='button2_Click'`", não digite o nome do tratador de evento. Em vez disso, use a janela do IntelliSense que aparece para dizer ao IDE para adicionar um novo método. Assim que você tiver terminado a linha, verá um novo botão aparecer no designer. Mude para a aba Window1.xaml.cs e você encontrará um novo método button2_Click.

Isso é todo o WPF e XAML que pudemos incluir aqui. Mas agora que você tem as ferramentas para começar a aprender sobre WPF, nós certamente recomendamos que você dê uma olhada em ***Programando WPF*** de Chris Sells e Ian Griffiths. Ele está disponível no site da sua editora.

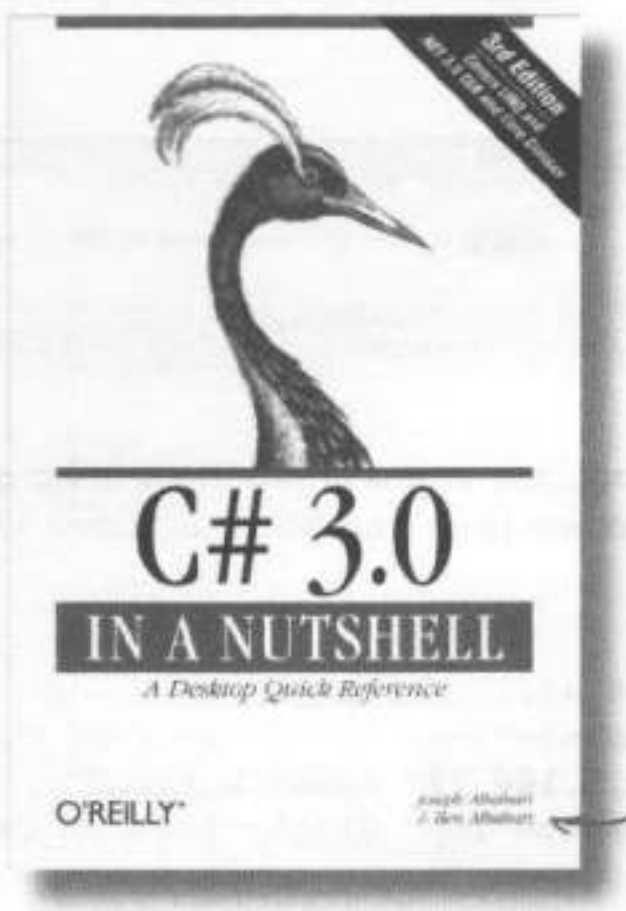


Você sabia que com o C# e o Framework .NET você pode...

- ★ Ter muito mais controle sobre seus dados com consultas LINQ avançadas? Serializar objetos para um arquivo XML?
- ★ Acessas páginas de Internet e outros recursos de rede usando classes nativas?
- ★ Adicionar criptografia avançada e segurança a seus programas?
- ★ Criar aplicativos complexos com múltiplas linhas de execução?
- ★ Distribuir suas classes para que outras pessoas possam usá-las?
- ★ Usar expressões regulares para fazer busca avançada de texto?

Eu não tinha idéia! Onde eu posso aprender mais?

Existe um livro excelente que explica isso tudo!
É chamado C# 3.0 In a Nutshell, de Joseph Albahari e Ben Albahari, e é um guia minucioso de tudo que o C# tem a oferecer. Você vai aprender sobre características avançadas da linguagem, lerá sobre todas as classes e ferramentas essencias do Framework .NET, e aprenderá mais sobre o que realmente acontece nos bastidores do C#. Verifique no site da editora.



Joseph Albahari nos ajudou muito mesmo fazendo neste livro uma revisão técnica realmente minuciosa. Agradecemos muito por toda a sua ajuda, Joe!

Índice Remissivo

Símbolos

! 60
!= 63
.NET 135
" 15
() 106
[] 143

A

A LINQ é simples 563
  é versátil 566
  pode combinar seus resultados em grupos 571
abelha animada 487
About" (Sobre) 15 - 37
abrir um navegador de pasta 350
Abstração 252
abstract 244
abstrata 192
Ace of Spades 273
acessado 165
  acessador get 171 - 176
  acessadores get e set 176
  acesso privado 228
acidentalmente 133
acrescenta herança 195
  classe 195
  adiciona 199
  subclasse 200
Add
  New Table 19
  Resource 484
  Bee() 453
adicionar 17 - 18
  componentes 490
  Item Existente 260 - 298
  um Método 416
Adicione 384
  Adicione tratamento de exceção 406
  Adicione um método Honk() 229
agenda 4 - 6
agenteCIA 165
Aircraft 197
Ajuste 587
AlertLevel 225
  nível de alerta 225
Alguma coisa faz o evento ocorrer 414
Algumas classes nunca deveriam ser instanciadas 243
alguns pássaros não voam 199
Allow Nulls 19
ambigüidade 251
Amount 455
Anatomia de uma consulta 563
Animação 446 - 454 - 477
  buffer duplo 477 - 513
aplicativo 3 - 4 - 12 - 15
  console 289 - 369 - 429
  depurando 391
  executando 263 - 378
  para testar a classe Farmer 171
  para TODOS 33
  Windows 80 - 123
  de Console 289
Apostando 148 - 153
Appliance 234
aprender C# 2 -11
Apresentadora 270
áreas transparentes do gráfico 498
argumentos inválidos 119
ArgumentOutOfRange 407
armadilhas 368
Armas 316
armazenar 224
  dados 224 - 286
arquivo 12
  gráfico 14
arquivos 16
  gráficos 501
array de inteiros 562
árvore genealógica 183
Aspecto 152
Aspen 4
assembly 241
AssignWork 212 - 213
Assistente de Configuração 576
ataque 316
atribuir 85
  a cada objeto 100
  a uma variável 115
  um destino 85
Attack 323
atualizações 526
Atualize Hive para receber uma referência de World 458
autodestruir 532
automaticamente 3 - 13
avô de todos os videogames 584

B

Back 256
  Color 316
  Yard 256

BagsOfFedd 173
bang 249
baralho 275
  de um arquivo 356
  de 52 cartas 299
Barras Doces 92
base de dados 2 - 5 - 17
  SQL 18
baseada em Serviços 17
baseball 432
bastidores 12
bat_HitTheBall 430
BD (base de dados) 7
Bee 217
  BeeControl 481
  BeeGroup 472
Berçário 452
Bet 149
binário 331
BinaryFormatter 359
BinaryWriter 364
bit 23
Bitmap 499 - 500
bloco do programa 201
  finally 393
  de uma linha 189
BLT 200
boas condições 170
bobCash 101
BolaEmJogo 413
boleano 21
Bônus 99
bool 114
botão 1
  de rádio Bin 115
  Events 496
  Lloyd 131
  swap 131
boundaries 320
Breed 129
BringToFront - trazer para a frente 492
Browse for Folder 350
buck 556
Buffer 477
  duplo 513
Build 34
  Solution 228
button1.Text 87
byte 114
bytes 331

C

Cada objeto trata o evento 415
caixa de checagem Table 27
caixa 12
  de imagem 15
  de ferramentas 12 - 83
caixas de diálogo 341
CakeSize 184
Callbacks 432
  usam delegados 432
calculador 123
CalculateCost() 158
cálculo 118
  extra da rainha 219
  com coleções 556
caminho 4 - 78
Campo 20
  Belo 78
  de armazenamento 173
  privado do tipo 465
  privado Queen 212
candidato 201
candy 92
  isolation cooling system 92
  Controller 94
Canines 544
Captura 386
  bloco de comentários 403
caras 100
cardamomo 167
caso especial 133
casting 117
catch 389
Cérebro 20 - 115
chave 18
  primária 19
  são opcionais para blocos de uma linha 189
Chaves e valores 293
checagem 23 - 29
checkBox1.Checked 32
ChedarEnvelhecidaVermont 194
CICS 92 - 93
Cinema 251
classe 244
  abstrata 244 - 246
  base 195 - 196
  Bee 448
  BirthdayParty 184
  de navegação 80
  DinnerParty 159
  Flower 433
  Guy 99
  não pode ter subclasses 241
  Produtora 98
  implementa 231
classes 92
  abstratas 223
  existentes 551
  intuitivas 106
  Navigator 81
  Nunca deveriam ser instanciadas 243
  que implementarem interfaces 227

Clone 335
CLR 535
código 2
  do construtor 215
  para fazer Locations funcionar 453
  para um botão 141
  repetitivo 2
coerção 117
  automaticamente 119
CoffeeMaker 234
coisas diferentes 10
colchetes 135
coleta de lixo do C# 131
coletado 127
coletar o lixo 535
coletor de lixo 397
colméia 210
comando 80
  C# válido 563
  new 83
  if 116
  if/else 354
  switch 354
  using 348
Combinando 118
comentário de uma única linha 427
Common Language Runtime 390
compilador 116
Compilar 33
  a solução" (build Solution) 34
  seu programa 121
componentes favoritos 169
comportamento 8
  comum 320
  de consumo 220
concatenar 119
condições 170
conecte a LINQ 577
conjunto de valores 270
  válidos 270
conseguir uma animação suave 587
construir a tabela 23
construtor 174 - 175
  Deck( ) 356
consulta de junção 578
consumer 236
ContactDB 20
ContactDBDataSet 29
ContactID 18 - 21
Contains() 279
conter 81
controlar 165
  acesso 165
  coisas em seu formulário 149
  ToolStrip 463
  Zoomer 509
controles 10
  de formulário 478
  do Dimmer 184
  e gráficos 477
  vinculados a dados 30
converte 115
cookieRecipe 167
cópia dos dados 545
cor de fundo 497
corridas 148
criando o formulário 305
  tabela 19
  um programa 349
  programas muito legais 3
Crustáceo Dourado 435
custo total para o jantar 162

D

DADOS 3
  do objeto 133
  em binário 364
  em boas condições 170
Data 11
Database Explorer 17
Debug 15
Debugging 15 - 385
decimal 139
decisões 354
Deck 275
Declaração 105
declarações C# válidas 248
Decoration 254
defender o planeta contra hordas e hordas de
  invasores 584
DefendeTheHive() 225
definition 225
delegado 337
delegados em ação 427
delegate 426
dentro da classe 167
depuração 15
depurador 382 - 384
  do IDE 385
  pare seguir o fluxo 391
desaparece 127
descarga hexa 369
descarregador hexa 369
descending 564
descobrir 11
desconto 126
Descrição 21 - 22
descritivas 386
desculpas para passear 378
desencaixar 547
desenhar 94
  um diagrama de classe 90
desenvolva 493

desentocar o erro no gerenciador de desculpas 385
desenvolver 3
  o formulário de Invasores 586
design 217
  de objetos relacionais 577
  cs 12
Destination 88
Destrutores 534
detecção de colisões 593
dia de corridas 147
diagrama 7
  de classe 95 - 96
diálogo 521
  de impressão 521
  padrão 340
Digite "True" ou "False 26
dinâmica versus estática 546
DinnerParty 159
Directory 342 - 343
disparar um evento 412
Dispose 347
Dispose ( ) 535 - 538
distance 149 - 320
distribuição 7
Diversão com Joe e Bob 108
DoMaintenanceTests() 94
double 115 - 116
DoubleBuffered 514
downcasting 236
DrawBees() 495
DrawImage 501
Drawing 2 - 319
duplicação de código 192
duplicação de código em subclasses 192

E

e.HasMorePages 521
efeitos colaterais 129
elementos visuais 2 - 482
Elephant 131
eletrodoméstico 234
else 92 - 100
Emergency 554
Empty Project 203
ENCADEAR 418 - 432
Encapsulamento 155 - 165
  classes 170
encoding 333 - 371
encontrar flores 224
enfeites 586
entendimento do código 92
Entrance 454
Entregador 98
EntregadorPizza 97
enum 271
  Direction 318
enumerar 270
erros no sistema 348
escapePlane 532
escrever 2
  código C# 8 - 16
  dados 331
escrever texto em um arquivo em três passos simples 334
especial 16
especificação 20
  de Identidade 20
estabilidade 537
estado de objeto 357
estáticos 87
estatísticas 316
estilo 12 - 128
estimador 156
estimativa 83
estrutura 585
  de Invasores 585
  de nossa base de dados 17
  natural 93
estruturas são tipos de valor 544
etiquetas 127
Eureka 363
EventArgs 15 - 99
evento 156
  BolaEmJogo 413
  ocorre... 412
  Tick 436
    do timer de animação 586
Evitando exceções 402
Evite erros 348
  desnecessários do sistema de arquivos 405
exceção 103
  herdam 383
  com try e catch 389
Exception 103 - 380
executar 15
  seu aplicativo 15
executável 35
exemplo rápido 170
exercício 99
Existing Item 260
Express 3 - 7
Extensão 78 - 134
Extract Method 121

F

falante 80 - 118
falhas de forma ELEGANTE 405
false 26 - 32
Farmer 173
Faucet 548

Fazendo o código ter sentido 77
FAZER ALGO com um evento 413
fecha-chaves 508
Felino 192
ferramentas 3 - 12 - 15
FIDO 130
FieldForm 491
fila 307
File 342
FileStream 333 - 334
  escreve bytes em um arquivo 333
finalBattleFactory 532
finalidades 10
finalizador de seu objeto 534
finalizar 445 - 462
Finish 29
float 114
flores 4 - 224
Flower 230
fluxo do try/catch 391
fonte de dados 17
foreach 279
Form1 12
  .cs 8
  .Designer.cs 12
formas diferentes 327
Formatando seqüência de caracteres 161
formato binário 362
formulário 2 - 12 - 28
  delega atividade ao objeto Game 318
fórum 330
Framework 2
  .NET 135
Framework de Apresentação 2 - 606
Framework.NET 608
freqüência 175

G

Game 152
GameOver 437 - 438
GatherNectar 232
GC.Collect() 535
GDI+ 502
geladeira da LINQ 569
Gerador de Desculpas 371
Geral 4
GetMenuItem() 135
Go to definition (ir para a definição) 280
Go() 98 - 456
Goleiro 412
GPS 79
gráfico 14 - 497 - 503 - 507
gráficos para desenhar uma figura num formulário 503
Graphical User Interface 84
Graphics 477
GreyHound 129
grupos de abelhas 471
GUI 84
GumballMachine 178
Guy 100

H

habilidades artísticas 247
hall superior (upstairs hallway) 260
Handles 539
healthyOption 161
heap 89 - 546
Height 88
herança 115 - 183 - 207 - 224
  entre interfaces 232
  múltipla 250
herda 196
hexa 363
hierarquia 115
  com as classes Worker 224
  de classe 195
Hipopotamo 193
HIT 320
Hive 398
hiveForm 491
hodômetro 123
HomeTheater 251
Honk 229
hordas 584
Hot for true 256

I

IClown 228
IComparer 288
ícones do inventário 329
ICooksFood 237
ID 19
IDE 1 - 2 - 5
IDisposable 347
  para fazer sua própria limpeza 402
IEnumerable 280
if 91
if/else 62
imagem 12
  em ação 500
Ímas de geladeira 141
  solução 142
impressão 520
índice 134
  de zero 134
ineficiente 190
inicializador de objeto 105
inicializar 103 - 105
  campos privados 174
  Depuração 385
Instalador do Windows 4

instance of the abstract 244
instância 84 - 87
  de GreyHound 130
  de NectarStinger 226
instancias 230
  de uma interface 230
int 19 - 21
IntelliSense do IDE 105
interface de usuário 3
interface IPotion 323
  pública é automaticamente público 226
Interfaces 3 - 12
Internet 147 - 154
intuitivamente 32
Invader 590
Invasores 583
inventário 314
IPalhaço 228
IPotion 323
is 20
Iteração 139

J

janela 316
  de estatísticas 441
  do IntelliSense 105
javaJoe 236
João Podrão 135 - 136
Joe Albahari 581
joe.TakeCash 100
joe.TakeCash(25) 100
jogador 261
jogos de verão 72

K

Key 19
KGB 167
kicks on route 569

L

Label 55
label1 68
  .BackColor 70
Laboratório C#1 147
Laboratório C#2 313
Laboratório C#3 583
laço 64
  foreach 279
Laços 60
lacunas 143
Lamp 558
LavaLouças 96
leia ou escreva seus objetos 359
leitura tipo string 229
Length 61
letra hebraica 362
Ligando os pontos 415
limitação 250
limites 115
linguagem
  baseada em XML chamada XAML 29
  Estruturada de Consulta 18
linhas 134
  new Dog() 134
  using 48
LINQ 472 - 559
  facilita trabalhar com dados 473
  para XML 600
  pode buscar dados 561
lista 238
  de checagem 238
  de Erros 45
lixo 127
Lloyd 131
local da entrevista 78
localização 46
Location 2
LookForEnemies 225
LUCKY 129
lugares mais do que exatamente publico e privado 240

M

Maintenance 225
máquina virtual (virtual machine) 43
mascara o campo 175
masmorra 314
matriz 139
  de objetos Elephant 140
  de referências 134
  para criar um baralho 274
matrizes 133 - 134
Maximizar 32
maximizar e minimizar 158
mbox 45
mecânica 77
membros 216
memória 43
mensagem 103
  de erro 103 - 187
mensagens de erro ÚTEIS 405
menu 15
  Debug 15 - 33
  File (Arquivo) 8
MessageBox.Show 15
MessageBox.Show() 49
metadata 466
método 353
  Close() 353 - 534
  EvaluatePrice() 572
  ResizeImage 499
  tratador de evento 187
  vazio 41

métodos 15
herdados 200
Meu nome é 89
Meus Documentos 8
Microwave 237
milissegundos 69
milliSeconds 464
Missão 231
Mockingbird 191
modelo de classe 192
modificador de acesso privado 228
modificadores de acesso mudam o escopo 241
ModifyRouteToAvoid 79
ModifyRouteToAvoid() 80
morte 251
movimento 154
mudando a linha 101
múltiplas fontes 561
multiplicador de ração 175
myBitmap 501
MyClass 53
myResult 80

N

Name e Company 578
Namespace 42
Não se pode herdar de duas classes 249
nativos 114
nave 584
Navegação 8
de Solução 17
de Solução do IDE 10
Navigator 80 - 81
NectarStringer 226
NET 2 - 6
gera um objeto Esception 381
new 19 - 83
button1 84
Next 29
Nome da Coluna 19
padronizado 417
de classes 91
Verdadeiro 165
notebook 4 - 378
Notícias 270
NotMain 51
Nova Fonte 28
instância para uma referência de interface 231
Tabela 19
novo código LINQ 474
controle de usuário BeeControl 492
novo objeto 90
novo objeto Bet 151
null 26
nulos 19
NumberOfCows 171
NumberOfPeople 159
numberOfTimes 86
numberOfTimes (número de vezes) 81
NumericUpDown 81 - 115
número inteiro 19
números inteiros 20
nvarchar(50) 21

O

object 15
sender 49
Objeto Navigator 85
Exception 396
List 280
Random 135
World 442
objetos 340
nativos 340
são variáveis também 125
tratam o evento 414
Visuais .NET 6 - 16
óbvio 93
O IDE 44
On 566
on rye 566
opcionais 189
OpenFileDialog 343
openToolStripButton 474
operações matemáticas 59
operadores lógicos 63
operárias 210
Ops 117
OR (operação OU lógica) 63
orderby 472 - 473
Oriented Programming 252
Output 171
Output type 171
Outside 254
Oven 234
ovenTemp 167
override 193 - 200

P

P.A.S.T. (Programadores em Prol das Abelhas Sem Teto) 452
Pacote de Distribuição 7
pagamento 152
Painel de Design de Formulários 44
Paint 508
palavra-chave 133 - 204
event 416
reservadas 122
Palhaço 90
palpite 55
Papel da Vila Objeto 440
para conectar o Starbuzz 580

parâmetro inteiro 428
parâmetros 49
parte gráfica do código 468
Pássaro cantor 191
Pausar simulação 467
PayOut 150
People 19
PeopleDiagram 25
PEPS - primeiro a Entrar, Primeiro a Sair 307
UEPS - Último a entrar, Primeiro a Sair 307
persistente 331
PhotoShop 512
PI 61
PictureBox 12 - 14
pilha 90
pilha (heap, em inglês) 89
Pitcher (lançador) 419
Planejador 179
planeta 584
PlanetMission 246
plano diabólico 335
Player 301
PluggedIn 234
Polimorfismo 252 - 253
ponto de entrada 50
  para herdar de uma classe base 196
posição inicial 152
power 236
Preenchendo a classe Hive 455
prejuízos 153
Preliminar 469
Primary Key 19
principal 20
principal (master bedroom) 260
princípio de POO 267
  da programação orientada a objetos 252
PrintDocument 520
Printing 520 - 524
PrintTableRow() 522
printToolStripButton 474
privados 174
private void DisplayDinnerPartyCost() 159
problema de performance 498
procedimentos armazenados 18
Procuradora do Estado 270
Produto Final 154
Program.cs 8 - 10
programa 157
  de Planejamento 185
  Planejador de Festa 179
programação em Windows 2
Programadores 452
programas 3
  C# 49
  poderosos em C#. 3 - 68
projeto de aplicativo Windows 42
promotora de eventos 156
Properties window 490
propriedade BackgroundImageLayout 315
  para o nome da carta 273
  Size 13
  Text 32
  Visible 70
  automáticas 173
  Cash e Name 103
  que funcionem com valor monetário 185
protegido 166
próxima linha do arquivo 337
public 15
  interface 226
público 83

Q

Quando a subcoerção falha 237
  seu programa lança uma exceção 381
quebra-cabeça 72
Queen usa uma matriz de objetos Worker 211
QUENTIN 61
Query 18
Quick Find (busca rápida) 424

R

rádio Bin 115
rainha 219
Random 135
rastrear e prevenir exceções 384
ReadLine() 337
Rebuild 227
Recomeçar simulação 467
Rectangle 319
recursos 3 - 14
refatoração 602
reembolso 98
referências 230
  de interfaces 230 - 243
registro 3 - 20
Remove() 279 - 280
renderizador 480 - 493
Representar artisticamente 480
representar números 271
  usando nomes 271
reset (reiniciar) 467
  Window Layou 9
ResetSimulator() 496
ResizeImage 499
resolver seus problemas REAIS 3
retorna null 237
Retorno 49
reverter um baralho 360
Revisão de Sintaxe 61
RINTINTIN 129 - 130

RoboBee 233
Room 254
rótulo 1 - 32
Route 88
ROVER 55 - 129
Rowboat 202

S

SABE como responder 412
Sailboat 202
salão de apostas 148
sanduíches 135
saveToolStripButton 474
sealed 241
segundo (second bedroom) 260
select beeGroup 472
  new 575
select SQL 565
sender 15 - 61
seqüências de escape 115
Serializable 359
serialização 359
serializado 359
serve para resolver um problema 93
SharpenStinger 225
ShiftsLeft 213 - 217
shiftsWorked 213
Shopper 243
short 117
Show Table Data (Mostrar Dados da Tabela) 26
símbolos matemáticos familiares 59
similaridades entre classes 106
simulador 148
  de colméia 224 - 442
  de pista 148
  de pista de corridas 148
  de zoológico 192
sintaxe básica 472
  da LINQ 473
sistema baseado em turnos 457
  básico 211
sistema de gerenciamento 210
  de gerenciamento de colméia 210
  de navegação 79
sistema de produção 98
Size 13
Size (ou largura e altura) 504
snippets 45
Solução dos Jogos 144
soluções temporárias 404
Solution Explorer 8
Speak() (fale) 545
SPOT 125
SQL 565
stack 306
static 2
StingerLength 225
StingPatrol 225
stream 331
Streams diferentes lêem e escrevem coisas diferentes 332
StreamWriter 334
string 19 - 554
Studio 1
subclasses 194 - 200
subcoerção 236
substantivo 84
substituir 84
  métodos 190
sujo 508
supercoerção 236
swap 131
switch 319
System 2
System.Math 67
System.Windows 42

T

tabela específica 24
TableAdapter 6
Tables 19
Talker() 86
TallGuy 229
tarefa 43
tarefa atual 218
teclado 15
Tecnicamente 87
Televisao 251
TellMe() 137
Terminar 23
Testboats 203
teste 162
  da Catarina 162
  Preliminar 469
this 2
thisMonth 177
Tick 436
  do timer de animação 586
timer 586
  de animação 587
  do jogo 587
  do jogo lida com o movimento e com a mecânica 588
tipo 58
  da variável 114
  de referência 231
  diferentes de inimigos 330
  para armazenar números 115
tipos úteis 59
Title 341
Toolbox 17

ToolStrip 463
Torcedor 412
TotalDistance() 79
toupeira 437
tour 502
transformar o SEU aplicativo 33
tratador de evento Paint 508
tratador de eventos 416
tratamento de evento 412
TreasureHunter (caçador de tesouros) 435
trecho 61
trechos de código 68
Turbine 93
turnos 210

U

uma cafeteira 234
under50sorted.Take(6) 567
Unicode 363
upcasting 234
UpdateCharacters 318
UpdateForm() 102
usando a palavra-chave base 204
  os objetos 317
  um objeto 125
Use a Cabeça 13
  join para combinar duas coleções 574
  um callback 432
UserControl 489
  é uma forma fácil de desenvolver 489
using 2
  providencia um try e um finally para você 401
  System; 2
  System.IO; 333
  System.Windows.Forms 2
uso do teclado 587

V

valor de retorno 49
  descending 568
  do biggestEars 139
valores 23
Value 60
variáveis 134
  Dog 134
variável de referência 126
VÁRIOS valores possíveis 354
VendaEfetuada() 98
verdadeiro/falso 21
Vermont 194
Video 251
videogames 584
Vila Objeto 4
Villain 532
virtual 43
Visible 70
vislumbrar problemas potenciais 96
Visual Studio 1 - 3 - 8
Visual Studio 2008 Express Edition 8
Visualização de Classes 44
Visualizador de Imagens 498
Visualizar 12
Você deve tratar suas exceções 404
void 2
void button1_Click 61
void Main() 2

W

Watch 384
Weapon 317
Weapon herda de Mover 324
while 60
WhoAmI() 131
Width 327
Windows 42
Windows Forms 40
Windows Presentation Framework 2
Worker 224
Worker baba 270
WorkTheNextShift 212
World (mundo) 442
Write 334
WriteLine 171

X

x must be 63
XML 29

Y

Yes 20
yesNo 60

Z

zero 134
zoológico 192
Zoom 13
Zoomer 509